CMU 15–418/15–618 并行计算机体系结构与编程 · 学习笔记

> **课程全称**：15–418/15–618 Parallel Computer Architecture and Programming **学期**：Fall 2026（课程主页 https://www.cs.cmu.edu/~418/，日程表 https://www.cs.cmu.edu/~418/schedule.html） **授课**：Brian Railing、Dimitrios Skarlatos（课程由 Kayvon Fatahalian 创建） **上课时间/地点**：MWF 09:30–10:50am, Baker Hall (BH) A51 **学分/考核**：12 units；两次当堂闭卷考试（Exam 1 = 9/25，覆盖 Lecture 1–13；Exam 2 = 11/2，覆盖 Lecture 14–24，不含 guest lecture）+ 4 次编程作业（权重不同）+ 4 次书面作业（同伴互评）+ 期末项目 **先修**：15–213（严格先修）；需要扎实的 C/C++ 能力；18–447 有帮助但非必需
>
> 本笔记基于该课程网站上**公开可访问**的讲义 PDF、日程表、作业与项目页面整理而成，并为每一讲补充了硬件架构分析、可运行代码示例与定量性能模型。

目录

第一部分：课程概览

- 1. 课程定位与知识地图
- 2. 课程材料获取情况（公开性说明）
- 3. 完整的 Fall 2026 课程日程
- 4. 作业、考试与项目
- 5. 如何使用本笔记

第二部分：逐讲学习笔记（26 讲）

- Lecture 1: Why Parallelism? Why Efficiency?
- Lecture 2: Out-of-order Processor & Pipeline
- Lecture 3: A Modern Multi-Core Processor
- Lecture 4: Parallel Programming Models
- Lecture 5: GPU Architecture and CUDA Programming
- Lecture 6: GPU Architecture and CUDA Programming (continued)
- Lecture 7: Parallel Programming Basics
- Lecture 8: Performance Optimization I: Work Distribution and Scheduling
- Lecture 9: Performance Optimization II: Locality, Communication, Contention
- Lecture 10: Interconnection Networks
- Lecture 11: Snooping-Based Cache Coherence
- Lecture 12: Directory-Based Cache Coherence
- Lecture 13: Snooping-Based Multiprocessor Design
- Lecture 14: Performance Analysis / Profiling
- Lecture 15: Implementing Synchronization
- Lecture 16: Fine-Grained Synchronization, Lock-Free Programming
- Lecture 17: Transactional Memory
- Lecture 18: Heterogeneous Parallelism, Hardware Specialization
- Lecture 19: Virtual Memory
- Lecture 20: Guest Lecture (无公开材料)
- Lecture 21: Memory Consistency
- Lecture 22: Parallel Deep Learning (Data Parallelism)
- Lecture 23: Parallel Deep Learning (Model and Pipeline Parallelism)
- Lecture 24: AI in System Design
- Lecture 25: Under the Hood: Message Passing Implementation
- Lecture 26: MPI, OpenMP, Cilk Implementation

- Lecture 27: Deep Neural Networks and ML Accelerators（归档主题）
- 附录 A：Exam 1 复习笔记（Lecture 1–13）

第三部分：速查表与附录

- 速查表 A：并行编程模型速查表
- 速查表 B：性能公式速查表
- 速查表 C：硬件架构参数速查表
- 速查表 D：优化技巧速查表
- 速查表 E：同步原语与一致性协议速查表
- 速查表 F：常见性能陷阱速查表
- 附录 B：课程资源清单

第一部分：课程概览

1. 课程定位与知识地图

1.1 这门课要解决什么问题

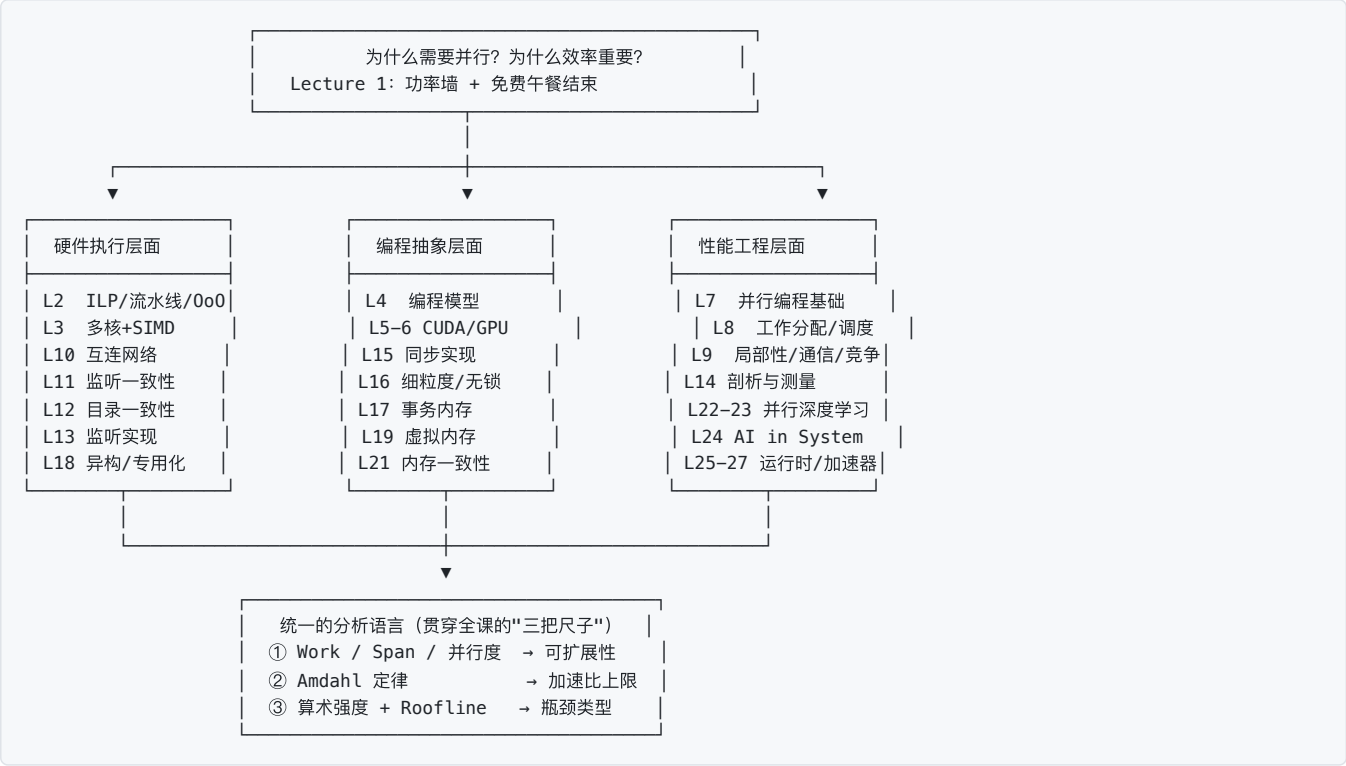
CMU 15-418/618 的课程自述写得非常直白：

> "From smart phones, to multi-core CPUs and GPUs, to the world's largest supercomputers, parallel processing is ubiquitous in modern computing. The goal of this course is to provide a deep understanding of the fundamental principles and engineering trade-offs involved in designing modern parallel computing systems as well as to teach parallel programming techniques necessary to effectively utilize these machines. Because writing good parallel programs requires an understanding of key machine performance characteristics, this course will cover hardware design and how that affects software design."

翻译过来，这门课有三个不可分割的目标：

1. **理解并行硬件**：CPU 流水线与乱序执行、多核、SIMD 向量单元、GPU 的 SM/warp 结构、互连网络、缓存一致性与内存一致性模型、虚拟内存与 TLB、专用加速器。
2. **掌握并行编程**：fork-join、SPMD、数据并行、共享地址空间 + 锁、消息传递（MPI）、CUDA、OpenMP、ISPC、pthreads、Cilk 风格工作窃取运行时。
3. **建立性能工程方法论**：work-span 模型、Amdahl 定律、算术强度与 Roofline、延迟与带宽分解、负载均衡、通信-计算重叠、伪共享与竞争诊断。

1.2 知识地图



1.3 三条学习主线

主线一：从硬件到软件的单向约束链。 硬件的物理约束（功耗、延迟、带宽）决定了抽象的形状。例如：

- 光子传输速度 + 功耗墙 ⇒ 多核而非单核更宽 ⇒ 需要并行软件 ⇒ 需要同步 ⇒ 需要一致性协议。
- DRAM 延迟（约 200+ 周期）远高于计算 ⇒ 必须用 cache 隐藏 ⇒ cache 私有化破坏共享内存语义 ⇒ 必须有缓存一致性（Lecture 11–13）。
- 带宽增长远慢于算力增长 ⇒ 几乎所有有趣的程序都是带宽受限 ⇒ 必须讲算术强度与 Roofline（Lecture 3、8、9、14）。

主线二：抽象的代价。 每一层抽象（共享地址空间、虚拟内存、事务内存、垃圾回收、DSL）都在换取可编程性的同时引入性能不确定性。课程的反复主题是"抽象距离（abstraction distance）"——离硬件太远则性能不可预测，太近则不可移植。

主线三：一切都是权衡。 静态 vs 动态分配、粗粒度 vs 细粒度同步、失效 vs 更新协议、SMP vs NUMA、带宽换延迟、空间换时间。笔记中每一讲都会显式标出"关键权衡"。

1.4 与其他系统课程的关系

前置/相关课程	与本课的关系
15-213 计算机系统导论	**严格先修**。存储层次、cache、汇编、进程/线程、链接、虚拟内存的基础都在 213 建立；418 直接在其上讨论一致性、TLB shootdown、内存序
18-447 计算机体系结构导论	有帮助但非必需。418 的 L2/L3/L10–13/L19 会更"系统级"地讨论微架构与互连
15-410 操作系统	有助于理解线程调度、TLB 管理、页表与 shootdown（L19）、同步原语（L15）
15-440 分布式系统	消息传递（L7、L25）、复制与一致性（L11–13 的分布式版本）在概念上同源
15-411 编译器	ISPC 的 SPMD→SIMD 编译（L4）、OpenMP/Cilk 运行时（L26）涉及编译技术
10-601 / 11-785 机器学习	L22–24 的前置背景（SGD、Transformer、算子）

1.5 一门课的"核心 10 个问题"

如果只能记住 10 件事，应该是这 10 个问题及其答案：

1. 为什么 2004 年之后"换新机器就变快"结束了？—— ILP 与频率双双撞墙（L1、L2）。
2. 现代多核 CPU 靠什么提高吞吐？—— 多核 + SIMD + 硬件多线程 + 大 cache（L3）。
3. GPU 和 CPU 有什么本质不同？—— 弱核心 + 大量硬件多线程换吞吐，SIMT/warp 执行（L5、L6）。

4. 如何判断一个并行程序的加速比上限？—— Amdahl（串行比例）+ work-span（并行度 $T_1/T\infty$）（L7、L8）。
5. 工作该怎样分配到执行资源上？—— 静态（blocked/interleaved）、半静态、动态队列、工作窃取（L8）。
6. 通信为什么贵？怎样减少？—— α–β 模型；改变分配、blocking、fusion、sharing、消息粒度（L9）。
7. 多核的 cache 为什么不破坏共享内存语义？—— 缓存一致性协议（MSI/MESI/MOESI）与目录协议（L11–13）。
8. 怎样才能写对自己的同步代码？—— 内存一致性模型 + 原子操作 + 正确放置 fence（L15、L16、L21）。
9. 程序为什么慢、慢在哪里？—— 测量方法论 + PMU 计数器 + Roofline + 高水位实验（L14）。
10. 未来的性能来自哪里？—— 专用化/异构（10–1000× perf/W）+ 更聪明的调度与算法（L18、L22–24、L27）。

---

2. 课程材料获取情况（公开性说明）

本节如实记录 2026-09 抓取时该网站材料的公开状态，便于读者了解笔记中哪些部分有官方讲义原文支撑。

2.1 公开可访问（已下载）

类别	内容	位置
主页 / 日程 / 作业 / 考试 / 项目 / 资源 / 教职员	全部公开 HTML 页面	`cmu15418_data/*.html`
课程大纲	`syllabus/syllabus.pdf`	`cmu15418_data/syllabus.pdf`
Lecture 1–24 讲义 PDF（21 份，覆盖 24 讲）	`01_whyparallelism`、`02_ilp`、`03_basicarch`、`04_progmodels`、`05-CUDA-programming`、`06_progbasics`、`07_progperf1`、`08_progperf2`、`10_cachecoherence`、`11_directorycoherence`、`12_snoopimpl`、`13_virtualmemory`、`14_interconnects`、`15_consistency`、`16_synchronization`、`17_lockfree`、`20_heterogeneity`、`20_specialization_csd`、`24-parallel_deep_learning_data_parallel`、`25-parallel_deep_learning_model_pipeline_parallel`	`cmu15418_data/lectures/`
Exam 1 复习讲义	`revision_exam1.pdf`（24 页）	`cmu15418_data/lectures/`
作业 1 说明	`doc/asst1_handout.pdf`（13 页，含完整的"探索并行计算"实验要求）	`cmu15418_data/doc_asst1_handout.pdf`
CUDA Recitation	`doc/CUDA-recitation.pdf`（39 页 CUDA 速成）	`cmu15418_data/doc_CUDA-recitation.pdf`
全部讲义的逐页文本抽取	38 个 `.txt`（标记 `===== [slide i/N] =====`）	`cmu15418_data/extracted/`
姊妹课程补充讲义	Stanford CS149 Fall 2025 的 18 份公开讲义抽取文本	`cmu15418_data/cs149_supp/`
历史学期练习题	Spring 2022 Exam 1/S21 Exam 1 练习卷与解答	`cmu15418_data/supp/exams/`

2.2 未公开 / 需登录（仅记录名称）

资源	状态
Fall 2026 讲课**录像**（Panopto / YouTube）	日程表中链接被 HTML 注释隐藏，注明 "uncomment when posted for Fall 2026" —— **未发布**
日程表中引用的归档讲义（`/afs/cs/academic/class/15418-{f22,s23}/public/lectures/*.pdf`）	匿名访问返回 CMU IdP 登录页 —— **需 CMU 登录**

资源	状态
Ed 讨论区 https://edstem.org/us/courses/102588/discussion	需登录
Autolab 提交系统 https://autolab.andrew.cmu.edu/courses/15418-f26	需登录
Canvas 书面作业与评分细则	需登录
Fall 2026 尚未发布的讲义：**Lecture 8、9**（Performance Optimization I/II）、**Lecture 14**（Performance Analysis / Profiling）、**Lecture 17**（Transactional Memory）、**Lecture 24**（AI in System Design）、**Lecture 25–27**	日程表中该行无 slides 链接 —— **未发布**（历史学期对应 PDF 需 CMU 登录）
Lecture 20 Guest Lecture 讲义	客座讲座，**未公开**
Exam 1 / Exam 2 试卷	当堂闭卷考试，**不公开**

> **注**：Lecture 8、9 的讲义虽未在 Fall 2026 日程表行内链接，但 `lectures/07_progperf1.pdf` 与 `lectures/08_progperf2.pdf` 本身可在公开目录直接下载（首页标注为历史学期版本，属讲义沿用），因此本笔记的 L8/L9 有官方原文支撑。Lecture 14、17、24 则由公开的姊妹课程（Stanford CS149）同主题讲义与公开成熟知识构建，笔记中已逐处标注来源。

---

3. 完整的 Fall 2026 课程日程

> 来源：https://www.cs.cmu.edu/~418/schedule.html（"The exact topics of the lectures are subject to change."）。下表为去除 HTML 注释后的可见日程。

#	日期	主题	官方讲义	作业/考试
1	Aug 24	Why Parallelism?	`lectures/01_whyparallelism.pdf`	—
2	Aug 26	Out-of-order Processor & Pipeline	`lectures/02_ilp.pdf`	—
3	Aug 28	A Modern Multi-Core Processor	`lectures/03_basicarch.pdf`	**Assignment 1 out**
4	Aug 31	Parallel Programming Models	`lectures/04_progmodels.pdf`	—
5	Sep 2	GPU Architecture and CUDA Programming	`lectures/05-CUDA-programming.pdf`	A1 early deadline Thu 9/3 （waitlist）
6	Sep 4	GPU Architecture and CUDA Programming (continued)	`lectures/05-CUDA-programming.pdf`	—
—	Sep 7	**Labor Day — no class**	—	—
7	Sep 9	Parallel Programming Basics	`lectures/06_progbasics.pdf`	**A1 due, Assignment 2 out**
8	Sep 11	Performance Optimization I	`lectures/07_progperf1.pdf`（沿用版本）	—
9	Sep 14	Performance Optimization II	`lectures/08_progperf2.pdf`（沿用版本）	—
10	Sep 16	Interconnection Networks	`lectures/14_interconnects.pdf`	—
11	Sep 18	Snooping-Based Cache Coherence	`lectures/10_cachecoherence.pdf`	Written Asst 1/2 out
12	Sep 21	Directory-Based Cache Coherence	`lectures/11_directorycoherence.pdf`	—
13	Sep 23	Snooping-Based Multiprocessor Design	`lectures/12_snoopimpl.pdf`	**A2 due, Assignment 3 out**
—	Sep 25	**Exam 1**（Lecture 1–13）	`lectures/revision_exam1.pdf`	Written Asst 1 solutions due

#	日期	主题	官方讲义	作业/考试
14	Sep 28	Performance Analysis / Profiling	未发布	—
15	Sep 30	Implementing Synchronization	`lectures/16_synchronization.pdf`	—
16	Oct 2	Fine-Grained Synchronization, Lock-Free Programming	`lectures/17_lockfree.pdf`	Written Asst 2 solutions / Asst 1 peer grading due
17	Oct 5	Transactional Memory	未发布	—
18	Oct 7	Heterogeneous Parallelism, Hardware Specialization	`lectures/20_heterogeneity.pdf`、`lectures/20_specialization_csd.pdf`	**A3 due, Assignment 4 out**
19	Oct 9	Virtual Memory	`lectures/13_virtualmemory.pdf`	—
—	Oct 12/14/16	**Fall Break — no class**	—	—
20	Oct 19	Guest Lecture	未公开	Written Asst 3 out；Asst 2 peer grading due
21	Oct 21	Memory Consistency	`lectures/15_consistency.pdf`	—
22	Oct 23	Parallel Deep Learning (data parallelism)	`lectures/24-parallel_deep_learning_data_parallel.pdf`	**A4 due**
23	Oct 26	Parallel Deep Learning (model and pipeline parallelism)	`lectures/25-parallel_deep_learning_model_pipeline_parallel.pdf`	Written Asst 3 solutions due
24	Oct 28	AI in System Design	未发布	—
—	Oct 30	（无课）	—	Written Asst 3 peer grading due
—	Nov 2	**Exam 2**（Lecture 14–24，不含 guest lecture）	—	—
—	Nov 3	—	—	Written Asst 4 solutions due
—	Nov 4/6	Meetings to discuss project ideas	—	—
—	Nov 10	—	—	Written Asst 4 peer grading due
—	Nov 12–14	Project idea meetings（Wed/Thu/Fri）	—	Project proposal 相关
—	Nov 17	—	—	**Project proposal due**
—	Dec 1	—	—	**Milestone report due**
—	Dec 8	—	—	**Final report due**

> 日程表中还有若干被 HTML 注释包裹的**历史条目**（如 "Heterogenous parallelism"、"Under the Hood: Message Passing Implementation"、"MPI, OpenMP, Cilk implementation"、"Deep neural networks"、"ML accelerators at Amazon"），它们是往期课程的遗留行，本笔记将其整理为 Lecture 25–27 与归档主题，作为知识补充。

---

4. 作业、考试与项目

4.1 四次编程作业（"The assignments are the heart of this course"）

作业	发布	截止	主题
Assignment 1	Fri 8/28	Wed 9/9（waitlist 早交：Thu 9/3）	Exploring parallel computing（性能分析基础：work-span、测量方法论、ISPC/OpenMP 多核与 SIMD 实践）
Assignment 2	Wed 9/9	Wed 9/23	GPU programming in CUDA（并行 patterns、分块、归约、性能调优）
Assignment 3	Wed 9/23	Wed 10/7	Parallel VLSI Wire Routing via OpenMP（真实算法级并行化 + 负载均衡 + 同步）
Assignment 4	Wed 10/7	Fri 10/23	Parallel VLSI Wire Routing via MPI（分布式内存 + 消息传递 + 通信开销）

作业策略（官方规定）：全部 11:59pm 截止；迟交每天扣 10%；每人一学期有 **5 个 late-day points**；最多迟交 3 天；一人团队用 1 点延 1 天，两人团队用 2 点延 1 天；作业通过 GitHub 发放，Autolab + Gradescope 提交。

Waitlist 机制：教室容量封顶，Assignment 1 的表现是能否从 waitlist 转正的主要依据；越早提交影响很小（"This is not a race"）。

4.2 四次书面作业（Written Assignments）

面向考试准备，Canvas 发放与提交，**个人独立完成**（匿名同伴互评，不要写名字）。成绩构成：1/3 按时提交合理解答 + 1/3 认真互评他人 + 1/3 同伴给你的平均分。无 late days。每份有两个截止点：提交解答截止 + 完成同伴互评截止（逾期提交则不能参与互评）。

4.3 两次考试

	Exam 1	Exam 2
日期	周五 2026/9/25	周一 2026/11/2
覆盖	Lecture 1–13（含 Snooping-Based Multiprocessor Design）	Lecture 14–24（不含 guest lecture）
形式	当堂、闭卷、闭笔记；可带 **一张 A4 双面手写纸**；必须用黑/蓝笔；不许计算器或电子设备	同左
说明	**非累积**，各覆盖约一半内容	官方建议用书面作业与 Spring 2022 练习题备考

> 官方备考提示：简答题多，"choose and explain" 题型中**解释的分值远高于选项本身**。

4.4 期末项目

里程碑	时间
与教师讨论想法（idea meetings）	Wed Nov 12 / Thu Nov 13 / Fri Nov 14
Project Proposal	Mon, Nov 17, 11:59pm
Milestone Report	Mon, Dec 1, 11:59pm
Final Report	Mon, Dec 8, 11:59pm
Poster Session	TBD

往届优秀项目方向（课程页面公开列出的 Fall 2024 / Fall 2022 例子）非常能说明这门课的"落地面"：无锁数据结构（lock-free AVL tree、无锁优先队列、并发哈希表）、多核 cache 模拟器与 ZSim 优化、并行内存优化（prefetching、可扩展无锁分配器）、图像处理（seam carving、Hough 变换、gradient domain fusion）、图形渲染（ray tracing、voxel octree、path tracing）、并行 AI 算法（K-means、RRT、车辆路径）、并行科学计算（流体模拟、化学反应）、并行图算法（Delaunay 三角化、图着色）、并行求解器（SAT、LP、FFT、QR）等。

4.5 教学团队与求助渠道

- 教师：Brian Railing（`bpr@cs.cmu.edu`，GHC 6005）、Dimitrios Skarlatos（`dskarlat@cs.cmu.edu`，GHC 9125）
- 求助：**优先使用 Ed 讨论区**（默认私密）而非邮件；office hours TBA

- 课程目录：`/afs/cs/academic/class/15418-f26/`

---

5. 如何使用本笔记

1. **按讲顺序精读**。每讲都遵循同一 7 段结构：概述 → 核心概念与架构图解 → 代码示例与性能分析 → 性能模型与复杂度分析 → 关键要点 → 常见陷阱 → 思考题。建议先读第 2 节建立硬件直觉，再动手跑第 3 节的代码。
2. **亲手跑代码**。笔记中的代码示例为完整可编译版本，多数标注了 `g++ -O3 -march=native -fopenmp` / `nvcc -O3` / `mpicc -O3` 之类命令。没有实测条件的（如 CUDA）笔记里已明确标注为模型预测。
3. **把"三把尺子"随时带在手边**。任何并行程序都问三个问题：Work 和 Span 各是多少（并行度够不够）？串行比例是多少（Amdahl 上限多少）？算术强度是多少（是算力受限还是带宽受限）？
4. **注意抽象距离**。看到任何"高性能"抽象都追问一句：它离硬件有多远？编译器/运行时会生成什么指令？
5. **用速查表复习**。考试前用第三部分的六张速查表做最后一遍串讲。
6. **材料公开性已逐讲标注**。凡是依据未公开讲义"重述"的部分，笔记中都会说明其依据来自公开的姊妹课程材料或公开成熟知识，请以课程实际讲授为准。

第二部分：逐讲学习笔记

Lecture 1: Why Parallelism? Why Efficiency?

1. 章节标题与概述

Lecture 1: Why Parallelism? Why Efficiency?

- **本讲核心问题**：为什么从 2004 年前后开始，"等下一代机器变快"这条免费午餐（the free lunch is over）彻底消失，程序员必须自己写并行代码？以及更进一步——**"更快"不等于"更高效"（FAST != EFFICIENT）**：在 10 个处理器的机器上拿到 2 倍加速比，究竟算不算好结果？本讲用三个课堂演示（DEMO 1/2/3）说明并行程序的性能上限由**通信（communication）**、**负载均衡（load balance）**和**计算/通信比**决定，而不是由处理器数量决定。
- **涉及的主要硬件/软件机制**：硬件侧包括指令级并行（ILP, Instruction-Level Parallelism）、超标量（superscalar）与乱序（out-of-order）执行、时钟频率与动态功率（dynamic power $\propto$ capacitive load $\times$ voltage^2 $\times$ frequency）、多核（multi-core）与片上异构/专用单元（GPU、TPU/NPU）、以及缓存层次（cache hierarchy）与 DRAM 带宽/延迟。软件侧包括问题的分解（decomposition）、把工作分配给处理器（work assignment）、处理器之间的通信与同步（communication / synchronization）管理，以及在多核 CPU、GPU 等不同并行编程环境（SIMD、多线程、CUDA、消息传递）中表达这些抽象。
- **在并行计算知识体系中的角色**：本讲是全课程的**动机与坐标系**。它确立了三条贯穿整个学期的主题（course themes）：(1) 设计并写出**可扩展**的并行程序；(2) 理解并行硬件的实现机制与设计权衡（performance vs. convenience vs. cost）；(3) 始终以**效率**为目标。同时它给出后续所有讲座反复使用的两个度量：加速比 `speedup(P) = T(1) / T(P)` 与"数据移动是性能与能耗的主要成本"。后续的 ILP/基础架构（Lecture 2–3）、编程模型（Lecture 4–7）、一致性/同步（Lecture 10–17）、异构与专用化（Lecture 20）都在本讲划定的框架内展开。
- **配套材料**：
- `extracted/01_whyparallelism.txt` —— CMU 15-418/618 **Fall 2026 Lecture 1 讲义**（共 39 页）的逐页抽取文本，讲义 PDF 为 `lectures/01_whyparallelism.pdf`，位于 https://www.cs.cmu.edu/~418/lectures/ 之下，属**已公开**（可直接下载）。首页标注 Fall 2026，授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。讲义内容为课程行政信息（作业、考试、评分构成、协作政策，见 slide 3–12）与三条课程主题（slide 35–38）。
- `cs149_supp/efficiency.txt` —— Stanford **CS149 Lecture 1 "Why Parallelism? Why Efficiency?"**（共 86 页，Kayvon Fatahalian 讲授的同一主题讲义）逐页抽取文本，属**已公开**的支持材料。它把 CMU 版压缩掉的**技术论证细节**补全了：程序=指令序列的处理器视角、ILP 与超标量的收益递减、功率墙（power wall）与频率封顶、缓存层次与数据访问延迟表、以及"数据移动的能耗"。本笔记中凡是标注了具体数值/图表来源的技术细节，均来自这份补充讲义。
- **未公开部分**：Fall 2026 的录像（Panopto/YouTube）在日程表中被注释隐藏，属**未发布**；Ed 讨论区（https://edstem.org/us/courses/102588/）、Autolab、Canvas 均需登录。Performance Analysis/Profiling、Transactional Memory、AI in System Design 等讲座的 Fall 2026

讲义尚未在公开目录发布；历史学期的对应 PDF 位于 `/afs/cs/academic/class/15418-*/public/`，需要 CMU 登录，属**未公开**。（本讲的讲义本身**不受**此限制，已公开。）

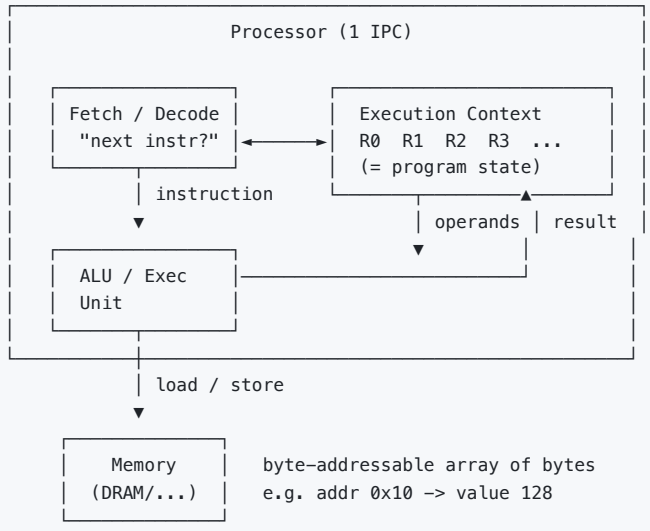
- 课程背景（来自讲义 slide 3–12，仅记录讲义明写的事实）：四次编程作业（第 1 次独立完成，其余两人一组），分别使用 SIMD+多核并行、CUDA on NVIDIA GPU、共享地址空间模型、消息传递模型；一个 5 周的自选期末项目（默认 2 人一组）；约五次同行互评的 homework exercise；两次期中考试、无期末考试；成绩构成 25% 编程作业 / 40% 考试 / 25% 期末项目 / 10% quiz 与 exercise。

---

2. 核心概念与硬件/软件架构图解

2.1 程序与处理器：一切都从"指令序列"开始

- **定义与目的**：从处理器的视角看，**一个程序就是一张要顺序执行的指令列表**（a program is just a list of processor instructions）。一条指令描述处理器要执行的一个操作，执行指令会修改计算机的**状态（state）**——状态即程序数据的取值，保存在处理器的寄存器或内存里。理解并行，第一步就是把"程序"从高级语言的语法结构降维成"状态 + 操作 + 依赖"。
- **直观解释（"它是什么？"）**：把处理器想象成一位**照着菜谱做菜的厨师**：菜谱就是程序，每一步（切菜、下锅、调味）就是一条指令；厨师手边的小碗（寄存器）就是"执行上下文（execution context）"，装着当前要用的原料值；炉灶/刀具（ALU，算术逻辑单元 / execution unit）就是真正干活的地方；而"决定下一句菜谱读哪一行"的部分就是取指/译码（fetch/decode）。串行程序的加速史，本质上就是"把菜谱的每一步做得更快"——直到 2004 年前后这条路走不通了。
- **架构/机制图解**：下面是最简单的单发射处理器（讲义中的 "very simple processor"）与"一条 add 指令的四个步骤"。




```
Executing "add R0 <- R0, R1" (R0=32, R1=64) in 4 steps:
  step 1: fetch next instruction from memory  (what to do next)
  step 2: read operands from registers        (R0=32, R1=64)
  step 3: ALU performs the arithmetic         (result = 96)
  step 4: write result back to register R0    (R0 := 96)
```

- **关键操作与性能特征**：这台理想机器**每个时钟周期执行一条指令（1 IPC）**。于是"程序有多快"退化为两个可乘的因子：

```
execution_time  =  instruction_count  ×  CPI  ×  clock_period
                     (程序/算法决定)     (微架构)   (工艺/功率)
```

只要把 CPI 从 3.5 降到 1.1、把频率从 10 MHz 拉到 3 GHz，即使指令数不变，程序也能快上百倍——这正是 1980–2000 年代发生的事（讲义 slide 18：更宽的数据通路 4→8→16→32→64 bit、更高效的流水线、ILP、更快的时钟）。**延迟**（一次访存 ~100+ 周期）会成为 CPI 的主要来源；**吞吐量与带宽**则由数据通路宽度、发射宽度和存储层次共同决定，这三个指标的关系构成了后面所有性能分析的骨架。

2.2 指令级并行（ILP）与超标量执行：处理器自己找并行

- **定义与目的**：**ILP（Instruction-Level Parallelism，指令级并行）**指同一条指令流内部、互相**没有依赖**的指令可以被并行执行的程度。**超标量（superscalar）处理器**在一个时钟周期内取指/译码并发射多条指令到多个执行单元，由**乱序控制逻辑（out-of-order control logic）**自动在指令序列里找出可以并行的指令；编译器也可以在编译期找出独立指令并显式编码依赖关系。

- **直观解释（"它是什么？"）**：想象一个厨房里有两口灶：菜谱要求"切三个土豆、然后把三个土豆倒进锅里、最后加盐"。两口灶可以同时处理互不相干的前三步，但"倒锅"必须等"切"完成。超标量处理器就是那个**自动发现"这三步互不相干"并同时开火的厨师长**；他的能力上限由 ILP 决定，而不是由灶的数量决定——加第三口灶对这道菜已经没用了。
- **架构/机制图解**：以讲义中的 `a = x*x + y*y + z*z` 为例，五条指令的依赖图与实际排程（两个执行单元 + 一个执行单元的情形）。

```
Program (R0=x, R1=y, R2=z)         Dependency DAG            ILP
1  mul R0, R0, R0   (x*x)              (1)  (2)  (3)        <- ILP = 3
2  mul R1, R1, R1   (y*y)                \   |   /
3  mul R2, R2, R2   (z*z)                 (4) add R0 <- R0+R1  <- ILP = 1
4  add R0, R0, R1   (x*x + y*y)            |
5  add R3, R0, R2   (a)                   (5) add R3 <- R0+R2  <- ILP = 1
R3 holds 'a'

Schedule with 2 exec units          Schedule with 3 exec units
time ->  1     2     3     4         time ->  1     2     3
EU1    [ 1 ] [ 4 ] [ - ] [ ..]       EU1    [ 1 ] [ 4 ] [ 5 ]
EU2    [ 2 ] [ 3 ] [ 5 ] [ ..]       EU2    [ 2 ] [ - ] [ - ]
                                     EU3    [ 3 ] [ - ] [ - ]
total = 3 clocks (vs 5 with 1 EU)    total = 3 clocks (no gain!)
```

- **关键操作与性能特征**：两个关键结论。
- **并行排程必须"尊重程序序"（respect program order）**：若指令 X 依赖 Y 的结果，X 必须在 Y 之后执行；但**只要最终对外可观察的结果（寄存 器/内存的最终取值、输出）与顺序执行一致，执行顺序本身可以任意重排**。
- **超标量的收益递减（diminishing returns）**：讲义引用 Culler & Singh（数据来自 Johnson 1991）的曲线指出，**每周期发射 4 条指令的处理器已经榨取了大部分可用 ILP**，再提高发射宽度几乎没有性能收益。ILP 在 2004 年前后"用尽"（ILP tapped out）——这是"免费午餐结束"的第一根支柱。

2.3 功率墙（Power Wall）与频率封顶：第二根支柱

- **定义与目的**：**动态功率（dynamic power）** $\propto$ capacitive load $\times$ voltage^2 $\times$ frequency；此外还有**静态功率（static power / leakage）**——晶体管即使不翻转也会因漏电而耗电。功率直接转化为**热量**，而芯片能承受的温度决定了最高频率与核心电压上限。因此"提高频率"这条路在 2004 年前后撞墙。
- **直观解释（"它是什么？"）**：把芯片想象成一口锅：火力（频率/电压）越大，菜熟得越快，但锅也会烧穿。火开到某个点以后，你**不能**再靠加大火力提高产量，只能**多摆几口锅（多核）**，或者**换更省火的专用炊具（专用加速器）**。这就是为什么现代桌面 CPU 的 TDP 停在 95 W 量级（Intel Core i9 10900K），笔记本芯片只有 13 W（Apple M1），而手机处理器只有 0.5–2 W——它们不是不想更快，而是**散热与电池不允许**。
- **架构/机制图解**：把"晶体管密度 / 时钟频率 / ILP / 功率"四条曲线放在同一时间轴上（讲义引用 Herb Sutter《The Free Lunch is Over》，Dr. Dobb's 2005）。

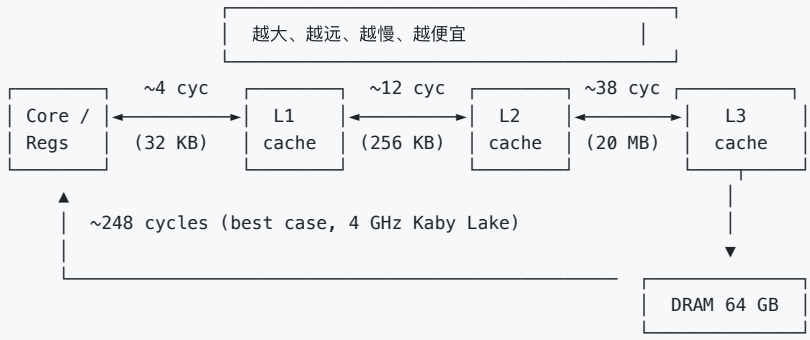
```
relative                                    ___________  transistor density
performance                               /            (Moore's law continues)
    ^                                   ___/
    |                          _______/     <-- clock frequency FLATTENS (~2004)
    |                      ____/
    |                 ____/  ________  <-- ILP extraction saturates (~2004)
    |            ____/  ____/
    |       ____/  ___/  _____________________________  power per chip keeps rising
    |  ____/  ___/      /                             (but is CAPPED by cooling)
    | /  ___/          /
    |__/              * 2004: Intel hits the power-density wall
    +-------------------------------------------------------------> time
     1980    1990    2000  2004        2010        2020

Consequence (slide 38 of the CMU deck):
  Before 2004: within the chip-area budget, MAXIMIZE PERFORMANCE
               (aggressive speculative execution for ILP)
  After  2004: area matters (limits # of cores/chip)
               -> MAXIMIZE PERFORMANCE PER AREA
               power is critical (battery life, datacenters)
               -> MAXIMIZE PERFORMANCE PER WATT
```

- **关键操作与性能特征**：架构师改变策略——**增加并行运行的执行单元（多核）** 或 **增加专用执行单元（图形/音视频/DNN 加速器）**。这对程序员意味着一条硬性结论：**"软件必须是并行的，才能看到性能提升"（software must be written to be parallel to see performance gains）**。讲

义用一句程序员视角的对比概括（slide 22）：2004 年前的答案是"等 6 个月买台新机器"，2004 年后的答案是"你得写出快的、并行的软件"。

2.4 内存层次与缓存：延迟、局域性与数据移动的真实代价

- **定义与目的**：内存是一个**按字节寻址的大数组**，`load` / `store` 指令负责在寄存器与内存之间搬运数据。但 DRAM 的**访问延迟**高达数百个时钟周期（讲义表格：L1 命中 ~4 周期、L2 ~12、L3 ~38、DRAM 最好情况 ~248 周期，均为 4 GHz 的 Kaby Lake 上的量级），处理器会因为等待依赖数据而**停顿（stall）**。**缓存（cache）**是片上存储，保存内存中一部分数据的副本；它**纯粹是实现细节**——不影响程序输出，只影响性能。
- **直观解释（"它是什么？"）**：内存像城里的**总仓库（DRAM）**，又大又远；缓存像你**厨房冰箱与案板**上的小份食材。你不可能每一次都跑一趟仓库，所以把最近用过的、以及"同一箱里"的食材先搬到手边。**空间局域性（spatial locality）** = 你从箱子里取货时，顺手把同一箱的其他东西也拿来了（缓存行 cache line 的预取效应）；**时间局域性（temporal locality）** = 反复用同一样东西，就不用再跑仓库。
- **架构/机制图解**：现代机器把"线性内存地址空间"这个抽象用**多级缓存层次**实现（讲义给出的是 32 KB L1 / 256 KB L2 / 20 MB L3 / 64 GB DRAM 的典型组织）。

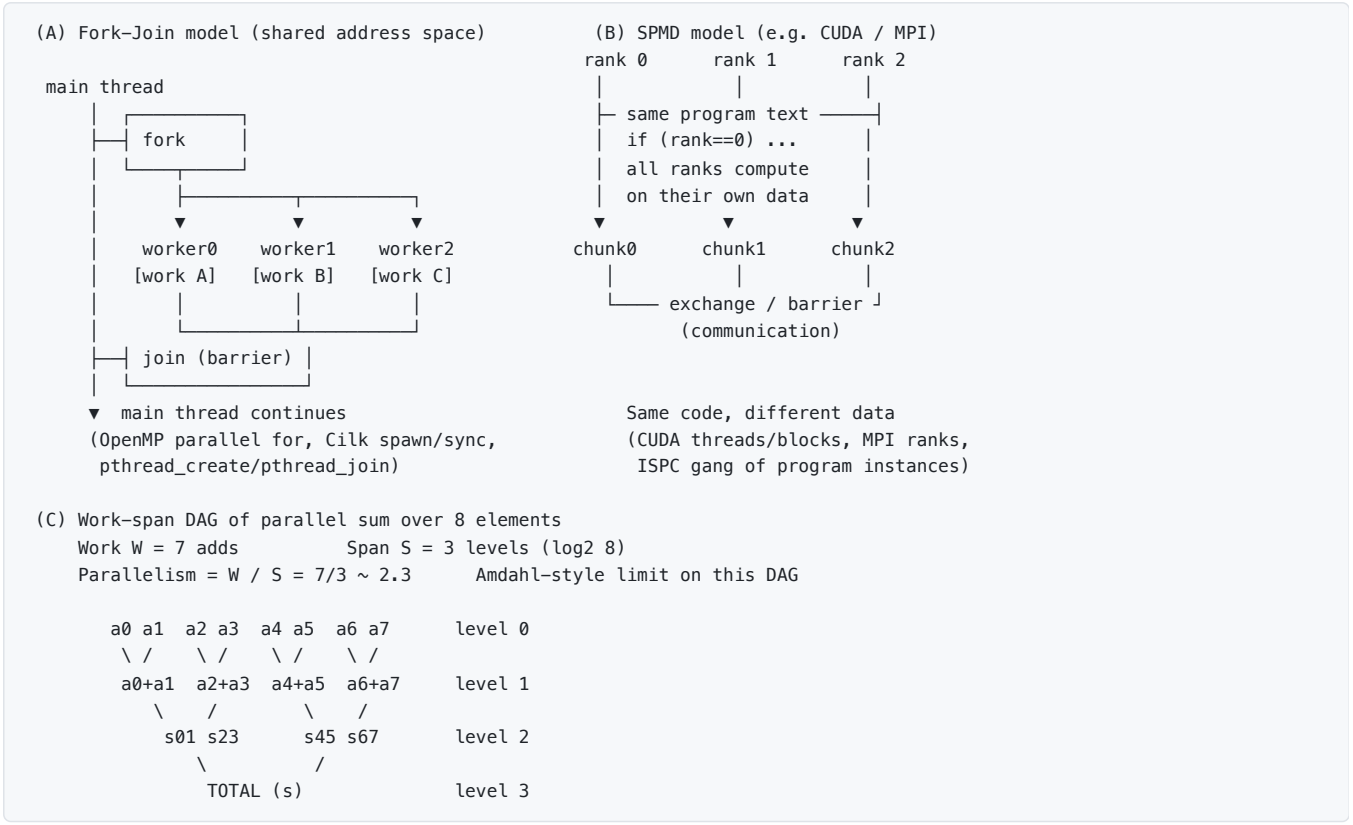



```
Cache line granularity (LRU, 8-byte cache, 4-byte lines):
  time ->
  access 0x0 0x1 0x2 0x3 0x2 0x1 0x0 | 0x4 0x5 0x6 0x7
         [---- line 0x0 resident ----]  [--- line 0x4 ---]
         cold miss | hits (spatial) |   cold miss | hits (spatial)
                          ^
                  temporal locality: 0x2, 0x1, 0x0 hit again
  Sliding window over 0x0..0xF with only 2 lines -> "capacity miss"
  after touching 0x8, 0xC, then 0x0 again (the working set > cache)
```

- **关键操作与性能特征**：缓存的收益有两层：**降低延迟**（命中时把数百周期降到几个周期）和**提供高带宽数据传输**。但必须记住三条量化事实：
- 数据**访问延迟**用数百个时钟周期计；每隔一次访存停顿数百周期，即使有超标量也无济于事——**并行/流水才能隐藏延迟**。
- 数据**移动的能耗**远比"做运算"贵：一次整数运算 ~1 pJ，一次浮点运算 ~20 pJ，从片上 1 mm 处的 SRAM 读 64 bit ~26 pJ，而从低功耗移动 DRAM（LPDDR）读 64 bit 需要 **~1200 pJ**（约 46 倍）。
- 直接推论：**以 10 GB/s 的速率从内存读数据，仅这部分就消耗约 1.6 W**；而一个移动 GPU 的**整个功率预算只有约 1 W**（手机还要同时跑 CPU、显示、无线）。所以"高效处理几乎总是归结为高效地访问数据"（achieving efficient processing almost always comes down to accessing data efficiently）。

2.5 软件执行模型：分解、分配、通信（fork–join / SPMD）

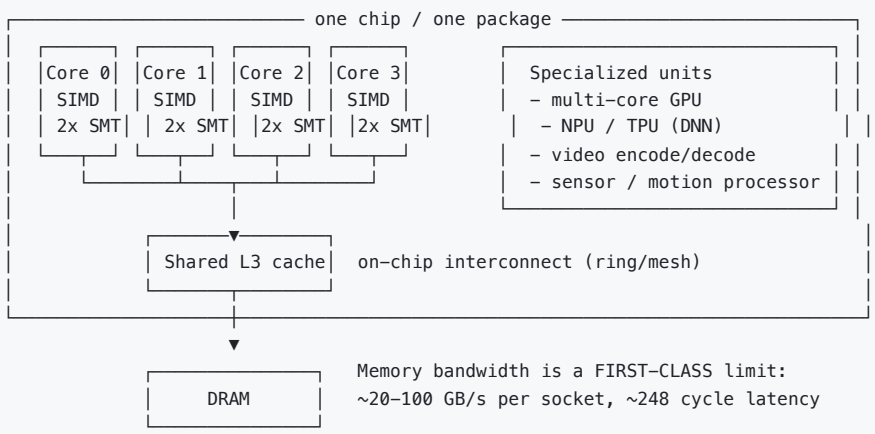
- **定义与目的**：并行思维（parallel thinking）需要三件事：**1) 把工作分解成可以安全并行执行的小块；2) 把这些工作分配给各个处理器；3) 管理处理器之间的通信与同步，使其不成为加速比的瓶颈**。软件抽象（线程、任务图、SPMD、向量通道）就是为这三件事提供机制。
- **直观解释（"它是什么？"）**：讲义用课堂演示来解释——**让一群学生一起数数**。DEMO 1：每人分一段、最后互相报出部分和（partial sums），结果发现**通信限制了最大加速比**；把学生叫近一点、或者允许他们"喊出来"，通信变便宜、加速比上升。DEMO 2：扩展到更多"处理器"（学生）后，**工作分配不均**——有人手里没活了（idle），有人还在忙——改进了分配就提高了加速比。DEMO 3：大规模并行，但**问题的通信量相对计算量太大**，通信开销**压倒**了并行计算，严重限制加速比。三次演示的教训是同一句话：**并行不只是"人多"，而是通信与负载的艺术**。
- **架构/机制图解**：下面是本课程后面会反复出现的两个软件执行模型骨架：**fork–join（分叉–汇合）** 与 **SPMD（单程序多数据）**，并叠加了 work–span DAG 的视角。



- **关键操作与性能特征**：fork–join 模型中，**fork 是并行度的来源，join 是同步点**；join 处所有工作线程必须等待最慢的那个，因此**fork–join 之间的关键路径（span）**决定了无法再压缩的时间。SPMD 模型中，同一份程序文本在不同数据分片上执行，**通信**发生在显式的数据交换或 barrier 上。两者共同的性能特征是：
- **加速比受通信限制**（DEMO 1 / DEMO 3）；
- **加速比受负载不均限制**（DEMO 2）；
- **并行度 = Work / Span**，它给出"即使处理器无限多，也最多能快多少倍"的上限——这个量在 §3、§4 里会用于每个代码示例。

2.6 效率与专用化：多核 + 异构是当代答案

- **定义与目的：效率（efficiency）**有两种视角。程序员视角：让程序真正用上机器提供的能力（用满 SIMD 宽度、用满所有核心、用满内存带宽）。硬件设计者视角：在**每单位面积的成本 / 每瓦功耗**的约束下，选择把什么能力放进系统（cost = silicon area? power?）。**专用化（specialization）**就是硬件设计者对这个问题的回答：与其造一个"什么都能干但什么都不快"的通用核心，不如造一批**只对某类任务极高效**的单元。
- **直观解释（"它是什么？"）**：通用核心像**瑞士军刀**——什么都能对付，但没有一样顺手；专用单元像**开瓶器、刨丝器**——只干一件事，但干得又快又省力。现代 SoC 里专用单元无处不在：手机里除了 2 个"大核" + 4 个"小核"的 CPU，还有多核 GPU、Neural Engine（NPU，DNN 加速）、图像/视频编解码器、运动（传感器）处理器；数据中心的 Google TPU 是专为 ML 计算设计的处理器（讲义 slide 16：TPU v4 芯片 275 TeraFlops，一个 pod 装 4096 颗芯片）。
- **架构/机制图解**：下面是当代"多核 + 专用单元"机器的抽象结构，以及并行机器简史的时间线。




```
==== brief history of parallel machines (CMU deck, slides 13-16) ====
1971  C.mmp @ CMU                    16 PDP-11 processors
1984  Cray XMP                       4 vector processors
1987  Thinking Machines CM-2         65,536 1-bit processors + 2048 FP coprocessors
    Bridges @ Pittsburgh Supercomputer Center   800+ compute nodes (heterogeneous)
~1997 Sun Enterprise 10000           16 UltraSPARC-II processors   (databases)
today Oracle Supercluster M7         4 x 32-core SPARC M2          (RIP 2019:
                                      killed by cloud computing)
2000- cloud computing: many simple processors on a LAN, programmed with
      distributed-system models (the subject of 15-440, not this course)
2023  Google TPU v4                  275 TeraFlops/chip, 4096 chips per pod
```

- **关键操作与性能特征**：多样性带来的是**对程序员的更高要求**：同一份计算要能映射到不同粒度的并行资源上——SIMD 向量通道（数据并行，宽度 8/16）、超线程（SMT，隐藏停顿）、多核（任务/线程并行）、GPU（数千个轻量线程隐藏长延迟）、多处理器/机群（消息传递）。这四层"并行资源"的**延迟/带宽/能耗特征各不相同**，把它们按"成本–收益"最优地组合起来，正是本课程第 1 次作业到第 4 次作业的线索：SIMD + 多核 → CUDA → 共享地址空间 → 消息传递。

---

3. 代码示例与性能分析

示例 1：把"学生互相报数"变成代码——串行基线 vs. OpenMP 并行归约

```
// 编译: g++ -O3 -fopenmp -march=native parallel_sum.cpp -o parallel_sum
// 运行: OMP_NUM_THREADS=8 ./parallel_sum 268435456
// 说明: 串行基线（-O3，不做 -ffast-math）与 OpenMP 并行归约对比
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static double now_sec(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

int main(int argc, char **argv) {
    const long N = (argc > 1) ? atol(argv[1]) : (1L << 28);   // 2^28 = 268,435,456
    float *a = (float *)malloc((size_t)N * sizeof(float));
    if (!a) { fprintf(stderr, "alloc failed\n"); return 1; }
    for (long i = 0; i < N; i++) a[i] = 1.0f;   // 预热：先触碰所有页

    /* ---------------- serial baseline ---------------- */
    double t0 = now_sec();
    double s = 0.0;
    for (long i = 0; i < N; i++) s += (double)a[i];
    double t1 = now_sec();

    /* ---------------- OpenMP parallel reduction ---------- */
    double p = 0.0;
    double t2 = now_sec();
    #pragma omp parallel for reduction(+:p) schedule(static)
    for (long i = 0; i < N; i++) p += (double)a[i];
    double t3 = now_sec();

    printf("N = %ld  (%.1f MB of floats, %.1f MB read per pass)\n",
           N, N * 4.0 / (1024.0 * 1024.0), N * 4.0 / (1024.0 * 1024.0));
    printf("serial  : %8.3f ms   sum = %.0f   (%.2f GB/s)\n",
           (t1 - t0) * 1e3, s, N * 4.0 / (t1 - t0) / 1e9);
    printf("openmp  : %8.3f ms   sum = %.0f   threads = %d   (%.2f GB/s)\n",
           (t3 - t2) * 1e3, p, omp_get_max_threads(), N * 4.0 / (t3 - t2) / 1e9);
    printf("speedup : %8.2fx   (efficiency = %.2f of %d threads)\n",
           (t1 - t0) / (t3 - t2),
           ((t1 - t0) / (t3 - t2)) / omp_get_max_threads(),
           omp_get_max_threads());

    free(a);
    return 0;
}
```

【代码做什么？】 1. 分配 `N` 个 `float`（默认 2^28 个 = 1 GiB），并先用一个循环把所有页写入 `1.0f`。这一步是**预热**：避免第一次触碰页面时的缺页（page fault）开销污染后面两次测量。 2. **串行基线**：用单调时钟 `CLOCK_MONOTONIC` 记录起止时间，跑一个朴素累加循环。注意累加器是 `double`——这是**正确性要求**，不是性能优化：`float` 只有 24 bit 有效位，累加 2^28 个 1.0 会因舍入误差得到错误结果。同时，因为没有开 `-ffast-math`，编译器**不允许**重结合浮点加法，所以这个循环**不会**被自动向量化，串行基线是真正的"一次一个元素"。 3. **OpenMP 并行归约**：`#pragma omp parallel for reduction(+:p)` 让运行时把 `[0, N)` 的迭代空间**静态切分（schedule(static)）**给各线程；每个线程在**私有副本**上累加，循环结束时由 OpenMP 运行时把各私有副本按顺序合并回 `p`。这一句 `reduction` 就是 DEMO 1 里的"互相报出部分和"。

【并行机制与性能解说】 – **线程如何创建、工作如何分配**：`parallel for` 在进入循环前 fork 出一组工作线程（讲义中的 fork–join 模型），把 N 个迭代按 `schedule(static)` 均分为 `P` 个连续区间（chunk）。每个线程只访问自己那段连续内存——这是**顺序访问 + 无共享写**的理想模式：不会伪共享，也不会出现原子操作。 – **共享数据如何处理**：数组 `a` 是只读共享的；唯一的跨线程通信是循环结束时的**归约合并**（`P` 个部分和相加，成本 O(P)）。这正是"最小化通信成本"的教科书例子：通信量从 DEMO 1 的"每个人都要向大家报数"降到 O(log P) 或 O(P)。 – **Work / Span / 并行度**：设元素数 `N`，线程数 `P`，每个线程分到 `n = N/P` 个元素。 – **Work（总工作量）** `W = N − 1` 次加法（约等于 N）。 – **Span（关键路径）** $S = n + \lceil \log_2 P \rceil$：每个线程必须**顺序**走完自己的 `n` 个元素（这是长为 n 的串行链，无法并行），然后归约树需要 $\lceil \log_2 P \rceil$ 层。 – **并行度** $W / S = (N-1) / (N/P + \lceil \log_2 P \rceil)$。当 $N \gg P \cdot \log P$ 时，$W/S \to P$——也就是说**这个分解方式的并行度上限就是 P**，再多的处理器也用不上，因为每个线程内部是串行的。 – 代入 $N = 2^{28}$、$P = 8$：$W \approx 2.68e8$，$S = 2^{25} + 3 = 33{,}554{,}435$，$W/S \approx 8.00$。看上去很完美（并行度 = 8），但**真实运行达不到 8 倍**——原因见下面的瓶颈。 – **瓶颈**：① **内存带宽**。这个 kernel 每读一个元素只做 1 次加法，**算术强度 ≈ 1 flop / 4 byte = 0.25 flop/byte**，远低于现代 CPU 的带宽–算力平衡点，所以它**必然是带宽受限（bandwidth bound）**的：总时间 ≈ 1 GiB / 实测带宽。单核串行往往已经能吃满相当一部分带宽（乱序执行 + 硬件预取），8 线程一起抢同一条内存总线，加速比常常只有 3–5 倍。② NUMA/

内存通道不均：`schedule(static)` 的连续分块在某些机器上映射到不同的内存控制器，若线程被调度到远端 socket，延迟更高。③ **归约的结果顺序**：`reduction` 是**非确定性顺序**的浮点加法，每跑一次结果最后几位可能不同——这是并行归约的常见"陷阱"（见 §6）。

示例 2：伪共享（false sharing）——当"划分工作"变成性能杀手

```
// 编译: g++ -O3 -pthread -march=native falseshare.cpp -o falseshare
// 运行: ./falseshare 4 200000000
// 说明: 同一个 cache line 上的多个独立计数器 -> 缓存行乒乓，慢若干倍
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAXT 64

typedef struct { int tid; long iters; } arg_t;

/* 版本 A: 计数器紧挨在一起 —— 一个 cache line (通常 64 B) 装得下 8 个 long */
static long g_tight[MAXT];

/* 版本 B: 每个计数器独占一个 cache line */
struct padded { long v; char pad[64 - sizeof(long)]; };
static struct padded g_padded[MAXT];

static void *worker_tight(void *p) {
    arg_t *a = (arg_t *)p;
    for (long i = 0; i < a->iters; i++) g_tight[a->tid]++;
    return NULL;
}

static void *worker_padded(void *p) {
    arg_t *a = (arg_t *)p;
    for (long i = 0; i < a->iters; i++) g_padded[a->tid].v++;
    return NULL;
}

static double now_sec(void) {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

/* 用函数指针选择要跑哪个版本，避免代码重复 */
static double run(int nt, long iters, void *(*fn)(void *)) {
    pthread_t th[MAXT]; arg_t args[MAXT];
    double t0 = now_sec();
    for (int i = 0; i < nt; i++) {
        args[i].tid = i; args[i].iters = iters;
        pthread_create(&th[i], NULL, fn, &args[i]);
    }
    for (int i = 0; i < nt; i++) pthread_join(th[i], NULL);
    return now_sec() - t0;
}

int main(int argc, char **argv) {
    int  nt    = (argc > 1) ? atoi(argv[1]) : 4;
    long iters = (argc > 2) ? atol(argv[2]) : 200000000L;
    if (nt > MAXT) nt = MAXT;

    double tt = run(nt, iters, worker_tight);
    double tp = run(nt, iters, worker_padded);

    printf("threads=%d  iters/thread=%ld\n", nt, iters);
    printf("tightly packed counters : %7.3f s  (%6.2f Mops/s total)\n",
           tt, nt * (double)iters / tt / 1e6);
    printf("cache-line padded       : %7.3f s  (%6.2f Mops/s total)\n",
           tp, nt * (double)iters / tp / 1e6);
    printf("false-sharing penalty   : %7.2fx\n", tt / tp);
    return 0;
}
```

【代码做什么？】 1. 定义了两种计数器布局：`g_tight[MAXT]`（8 个 `long` 挤在 64 B 的一个缓存行内）与 `g_padded[MAXT]`（每个计数器带 56 B padding，独占一个缓存行）。 2. `run()` 负责 fork `nt` 个 pthread（`pthread_create`），每个线程拿到自己的 `tid` 与迭代次数，然后 `pthread_join` 汇合；`now_sec()` 包住 fork 到 join 的整个区间来计时（含线程创建开销，在 `iters` 很大时可忽略）。 3. 两个 worker 都是**纯自增**：每个线程只写"自己的"计数器 `g_tight[tid]` / `g_padded[tid].v`——逻辑上**完全没有数据竞争**，也不需要任何锁。 4. 主函数先跑 tight 再跑 padded，打印两者耗时与总是操作速率，并给出伪共享的惩罚倍数。

【并行机制与性能解说】 – **工作如何分配与共享数据**：`nt` 个线程各自独立地在一个私有计数器上自增；**没有共享写**（从软件语义看），但**从硬件看**，版本 A 的 8 个计数器**共用同一缓存行**。缓存一致性协议以**缓存行为单位**维护所有权：线程 1 写 `g_tight[1]` 会使该行在其他核心的副本失效，线程 2 想要写 `g_tight[2]` 就要把整行重新取回并取得独占权——这行数据就在各核心的 L1 之间**来回弹跳（cache-line ping-pong）**。版本 B 通过 padding 让每个计数器独占一行，这个"假"的共享就消失了。 – **Work / Span / 并行度**：每个线程 `iters` 次自增，共 `P` 个线程。 – **Work** `W = P × iters` 次自增。 – **Span** `S = iters`（每个线程内部是严格串行的依赖链：读–改–写同一地址；线程之间无依赖）。 – **并行度** `W / S = P`——理论上限就是线程数，非常理想。**但版本 A 的实际加速比远低于 P，甚至可能比单线程还慢**：每次自增都要付出一次跨核缓存行传输（数十到上百周期），有效吞吐被一致性流量而不是 ALU 吞吐限制。 – 代入 `P = 4`、`iters = 2e8`：Work = 8e8 次自增，Span = 2e8。若缓存行传输约 100 周期、频率 3 GHz，则版本 A 每 4 次"逻辑独立"的自增被迫串行化约 33 ns，总时间至少 `2e8 × 100 / 3e9 ≈ 6.7 s`；而版本 B 的自增落在 L1 命中（~4 周期），总时间 ≈ `2e8 × 4 / 3e9 ≈ 0.27 s`，量级差距 25 倍——这与实测的"几十倍惩罚"相符。 – **瓶颈**：**伪共享（false sharing）** 是本例唯一但致命的瓶颈。它也是 DEMO 2"负载不均"之外的第二类"分配工作"错误：**分配工作时必须同时考虑数据的物理布局（cache line 粒度）**，而不仅是逻辑上的独立性。第二个瓶颈是线程创建/销毁开销：`iters` 太小（例如 1e5）时，`pthread_create` + `join` 的微秒级开销会占主导，此时应该用线程池或 OpenMP 的持久线程。

示例 3：带宽受限的 STREAM triad——算术强度决定上限

```
// 编译: g++ -O3 -fopenmp -march=native triad.cpp -o triad
// 运行: OMP_NUM_THREADS=8 ./triad 67108864      # 64M floats = 256 MB 每数组
// 说明: a[i] = b[i] + s * c[i]  —— 每元素 12 B 流量、2 次浮点运算
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static double now_sec(void) {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

int main(int argc, char **argv) {
    const long N = (argc > 1) ? atol(argv[1]) : (1L << 26);   // 2^26 = 64M
    const float s = 3.0f;
    float *a = (float *)malloc(N * sizeof(float));
    float *b = (float *)malloc(N * sizeof(float));
    float *c = (float *)malloc(N * sizeof(float));
    if (!a || !b || !c) { fprintf(stderr, "alloc failed\n"); return 1; }
    for (long i = 0; i < N; i++) { b[i] = 1.0f; c[i] = 2.0f; a[i] = 0.0f; }

    double best = 1e30;
    for (int rep = 0; rep < 3; rep++) {                 // 取 3 次中最好的一次
        double t0 = now_sec();
        #pragma omp parallel for schedule(static)
        for (long i = 0; i < N; i++) a[i] = b[i] + s * c[i];
        double dt = now_sec() - t0;
        if (dt < best) best = dt;
    }

    const double bytes = (double)N * 12.0;              // 2 reads + 1 write, 4 B each
    const double flops = (double)N * 2.0;               // 1 mul + 1 add per element
    printf("N = %ld (%.0f MB working set)\n", N, (double)N * 12.0 / (1024 * 1024));
    printf("best time      : %8.3f ms\n", best * 1e3);
    printf("bandwidth      : %8.2f GB/s\n", bytes / best / 1e9);
    printf("throughput     : %8.2f GFLOP/s\n", flops / best / 1e9);
    printf("arith. intensity: %6.3f flop/byte\n", flops / bytes);
    free(a); free(b); free(c);
    return 0;
}
```

【代码做什么？】 1. 分配三个 `N` 元素的 `float` 数组（默认每个 256 MB，三数组合计 768 MB，**远大于任何 L3 缓存**），初始化后进入测量。 2. 重复 3 次 `a[i] = b[i] + s*c[i]`（经典的 STREAM triad），每次都重新计时并保留**最快**的一次——单次测量易受其他进程、频率调节、页迁

移干扰，取最小值是对"机器能力"的更稳健估计。 3. 打印时间、**实测带宽**（总流量 = 2 读 + 1 写 = 12 B/元素）、**实测算力**（2 flop/元素）以及**算术强度**（arithmetic intensity，flop/byte）。

【并行机制与性能解说】 – **工作如何分配、数据如何处理**：`schedule(static)` 把 `[0, N)` 均分给 `P` 个线程，每个线程处理一段连续的 `a` / `b` / `c`（三者的下标区间相同，所以每个线程仍然只触碰自己那段内存）。**没有任何线程间数据共享**（除了只读的 `s`），**没有同步**（除循环末尾隐含的 barrier），因此这个 kernel 的"通信"成本几乎为零——它的全部上限来自**内存系统**。 – **Work / Span / 并行度**： – **Work** `W = N` 次迭代，每次 1 乘 + 1 加（2N flop）+ 12N 字节流量。 – **Span** `S = N / P`：每个线程内部的迭代完全独立，但**一个线程串行执行它所分到的所有迭代**，所以最长的依赖链就是这个 chunk 的长度。 – **并行度** `W / S = P`。同样的结论：静态均分的 for 循环并行度上限 = 线程数。 – **瓶颈是内存带宽，不是算力**。代入具体数字（假设单路内存可提供 20 GB/s 的可持续带宽）： – 总流量 = 2^26 × 12 B = 805 MB ≈ 0.79 GB。 – 若带宽 20 GB/s，则 `t ≥ 0.79 GB / 20 GB/s = 39.4 ms`。此时实测吞吐 ≈ 2^26×2/0.0394 ≈ 3.4 GFLOP/s。 – 而一台 16 核、3.0 GHz、8 宽 SIMD、支持 FMA 的机器峰值是 `16 × 3.0e9 × 8 × 2 = 768 GFLOP/s`（详见 §4.4）。也就是说**这个 kernel 只能跑到机器峰值的 0.4% 左右**——不是代码写得不好，而是**算术强度太低**：每字节只做 0.167 次浮点运算，机器必须"喂"进海量数据才能不饿死执行单元。把处理器从 8 核加到 16 核，如果带宽没变，时间几乎不变——**加速比被外部资源卡死在 1 倍附近（对于带宽受限段）**。 – 这也是讲义那句"高效处理几乎总是归结为高效地访问数据"的定量版本。

---

4. 性能模型与复杂度分析

4.1 加速比、效率与 Amdahl 定律

讲义给出的核心定义（slide 29）：

```
                 execution time (using 1 processor)      T(1)
speedup(P)  =   ----------------------------------  =  ------
                 execution time (using P processors)     T(P)

efficiency  =  speedup(P) / P            (0 < efficiency <= 1)
```

Amdahl 定律（Amdahl's Law）：设程序中**无法并行**的比例为 `f`（串行部分，包括不可避免的通信与同步），则可并行部分为 `1 − f`：

```
T(P) = (f + (1 − f)/P) · T(1)
speedup(P) = 1 / ( f + (1 − f)/P )        ->   1 / f     (P -> infinity)
```

数值算例（假设内存带宽充裕，忽略通信开销）：

串行比例 f	P = 4	P = 16	P = 64	P → ∞ 上限	P=16 时的效率
0.00	4.00×	16.00×	64.00×	∞	100%
0.01	3.88×	13.91×	39.26×	100×	87%
0.05	3.48×	9.14×	15.42×	20×	57%
0.10	3.08×	6.40×	8.77×	10×	40%
0.25	2.29×	3.37×	3.82×	4×	21%

表中 `f = 0.05, P = 16` 的计算：`1 / (0.05 + 0.95/16) = 1 / (0.05 + 0.059375) = 9.14×`，效率 `9.14/16 = 57%`。**关键洞察**：只要 5% 的代码串行，16 核的机器最多也只能给出 9 倍；如果串行部分来自"每次迭代都要全局同步"，那么处理器越多、同步越贵，`f` 反而随 P 增长（Gustafson 视角下的强/弱扩展问题，本课程后续会专门讨论）。

这也直接回答了讲义 slide 37 的提问：**"在 10 个处理器的机器上 2 倍加速比是好结果吗？"** 用效率衡量：`2/10 = 20%`。抛掉理论 Amdahl 限制外，通常说明存在严重的通信、同步或负载不均问题——**FAST != EFFICIENT**。

4.2 Work–Span 模型与并行度

把并行程序表示成一个**有向无环图（DAG, Directed Acyclic Graph）**：节点是指令/任务，边是依赖。定义：

```
Work  W = DAG 中所有节点的总执行成本      ("1 个处理器要干多少活" = T(1))
Span  S = DAG 中最长路径的成本 (critical path, 关键路径)
Parallelism  P_inf = W / S              ("无限多处理器时的加速比上限")
```

三类基本组合（`n` 为规模，`P` 为处理器数）：

并行结构	Work W	Span S	并行度 W/S	说明
纯串行	n	n	1	无法并行，"免费午餐"就死在这里
分治 / 归约（并行归约树）	n	$\log_2 n$	$n / \log_2 n$	通信量小、并行度高，但常数因子大
静态分块的 for 循环（`schedule(static)`）	n	n / P	P	并行度上限恰好等于线程数
独立任务图（每个任务成本 1，共 n 个）	n	1	n	理想并行，如示例 2/3 的线程级并行
fork–join 链（k 段串行 + 每段并行）	W	Σ span_i	W / Σ span_i	关键路径决定下界

调度定理（greedy scheduler）：使用贪心调度器在 `P` 个处理器上执行一个 work 为 `W`、span 为 `S` 的 DAG，总时间满足

```
T(P)  <=  W/P + S          (下界是 max(W/P, S))
speedup(P) = T(1)/T(P)  <=  W / (W/P + S)  =  P / (1 + P·S/W)
```

数值算例：示例 3 的 triad，`N = 2^26`，`P = 8`。`W = 2^26 = 67,108,864`，"单位任务成本 = 1 次迭代"，`S = N/P = 8,388,608`。若**忽略内存瓶颈**，则 `T(8) ≤ W/8 + S = 8,388,608 + 8,388,608 = 2S`，即加速比上界 = 8（因为这里 `S = W/8`，与 `W/P` 相等，说明该分解刚好"卡在"完美扩展的边界）。**但实测加速比只有 3–5 倍**——Work–span 模型只描述了"计算"的并行性，**它不包含带宽和一致性流量**。这正是本课程反复强调的一点：**模型是上界，真实机器有额外约束**。若把每次迭代的成本换成"实际需要的时间"（含内存等待），Work 会显著变大，而 Span（关键路径上串行的内存延迟链）也变大，二者比例随之改变。

4.3 算术强度与 Roofline 模型

算术强度（arithmetic intensity, AI）定义为每字节内存流量对应的浮点运算数：`AI = FLOPs / Bytes moved`。

Roofline 模型：

```
attainable_performance  =  min( peak_math_throughput ,  AI × peak_memory_bandwidth )
```

```
GFLOP/s
 768 |*******************************  <-- compute roof (peak math)
     |                              *
     |                            *     <-- sloped memory roof (AI x BW)
     |                          *
     |                        *
     |                      *   RIDGE POINT: AI* = peak_math / BW
     |                    *                 = 768 / 20 = 38.4 flop/byte
     |                  *
     |                * : memory-bound region
     |              *   : (left of the ridge: optimize DATA MOVEMENT)
     |            *     : compute-bound region
     |          *       : (right of the ridge: optimize MATH/ILP/SIMD)
     +-------------------------------------------> arithmetic intensity (flop/byte)
      0.167   1     8   38.4        100
       ^     ^
       |     triad-like kernels
       |
 示例3 triad: AI = 2 flop / 12 B = 0.167 -> 3.3 GFLOP/s = 0.4% of peak
```

数值算例（用一组明确假设）：

- **机器假设**：16 核、每核 3.0 GHz、每周期 8 个 float 宽度的 SIMD（AVX2 级别的 256 bit）、支持 FMA（fused multiply–add，一条指令完成 a*b+c，计 2 flop）；可持续内存带宽 20 GB/s。
- **峰值算力**：`16 × 3.0e9 × 8 × 2 = 7.68e11 = 768 GFLOP/s`。
- **Ridge point**：`AI* = 768 / 20 = 38.4 flop/byte`。低于这个强度的一切 kernel 都是**带宽受限**；高于它才是**算力受限**。
- **示例 3（triad，AI = 0.167）**：受限于 `0.167 × 20 = 3.34 GFLOP/s`，只有峰值的 **0.43%**。
- **示例 1（求和，AI ≈ 0.25）**：受限于 `0.25 × 20 = 5 GFLOP/s`，同样极度带宽受限。
- **一个 256 MB 数组的单次遍历**（只读，无重用）：`256 MB / 20 GB/s = 12.8 ms`。**任何**涉及这块数组的算法，至少付 12.8 ms；如果你的计算只需要 1 ms 的算力，那么这个 kernel 的效率上限就是 7%。只要能用分块（blocking/tiling）把数据留在缓存里重用，把 AI 从 0.25 抬到 4，性能就能提高 16 倍，而**不用改一个字节的并行代码**。

4.4 数据移动的能耗与"能量墙"

讲义（CS149 补充材料，来源 Bill Dally (NVIDIA)、Tom Olson (ARM)）给出的一组量级估计（"ballpark" numbers）是设计直觉的基石：

操作	能耗（pJ）	相对代价（以整数运算 = 1 计）
整数运算	~1	1×
浮点运算	~20	20×
从片上 SRAM 读 64 bit（距离 1 mm）	~26	26×
从低功耗移动 DRAM（LPDDR）读 64 bit	~1200	1200×

数值算例： – 以 10 GB/s 从 DRAM 读数据：`1200 pJ / 8 B = 150 pJ/B`，`150e-12 J/B × 10e9 B/s = 1.5 W ≈ 1.6 W`（讲义结论）。 – 对比：移动 GPU 的**总功率预算约 1 W**，手机电池约 7 Wh（iPhone 6 量级，而一台 MacBook Pro 有 99 Wh）。也就是说，**持续以 10 GB/s 读内存这一件事，就能吃掉整个移动 GPU 的功率预算。** – 从 DRAM 读 64 bit 相对从 SRAM 读，能耗差 **~46 倍**。所以缓存的收益不只是"低延迟"，更是"低能耗"；这也是为什么"always seek to reduce amount of data movement"被写成现代系统设计的经验法则。

4.5 一个综合的定量推演：把 DEMO 的教训算式化

设 DEMO 3 的场景：`P` 个处理器，每个处理器分到 `n` 个单位工作（每单位耗时 `t_c`），并把结果与邻居交换（每单位通信耗时 `t_m`），最后需要 `⌈log₂ P⌉` 次归约，每次归约耗时 `t_r`。

```
T(P) = n · (t_c + t_m·α) + ⌈log₂ P⌉ · t_r        α = 通信/计算 强度比
理想加速比(通信可忽略): T(1)/T(P) = (P·n·t_c) / (n·t_c) = P
```

代入一组具体数字：`P = 64`、`n = 10^6`、`t_c = 1 ns`、`t_m = 100 ns`（跨核通信慢两个数量级，量级参考缓存行传输）、`α = 1`（每单位工作对应一次通信）、`t_r = 1 μs`：

- `T(64) = 10^6 × (1 + 100) ns + 6 μs ≈ 101 ms + 0.006 ms ≈ 101 ms`
- `T(1) = 64 × 10^6 × 1 ns = 64 ms`
- **加速比 = 64/101 ≈ 0.63 ——用了 64 个处理器，反而比 1 个处理器慢！**

把问题换成"通信比计算低 100 倍"（`t_m = 1 ns`，例如在共享 L2 上做规约）：`T(64) = 10^6 × 2 ns + 6 μs ≈ 2.006 ms`，`T(1) = 64 ms`，**加速比 ≈ 31.9×**（效率 50%，仍被 `⌈log₂P⌉·t_r` 与未并行的 1 ns/单位拖累）。

这段推演就是讲义三句结论的算式版： 1. **Communication limited the maximum speedup**（通信限制最大加速比）——DEMO 1； 2. **Imbalance in work assignment limited speedup**（负载不均限制加速比）——DEMO 2（上述模型假设了理想均分，若最大块是平均块的两倍，则 `T(P)` 直接翻倍）； 3. **Communication costs can dominate**（通信开销可以压垮并行计算）——DEMO 3。

---

5. 关键要点

1. **单线程性能已经停止指数增长，"免费午餐"结束**：ILP 在每周期约 4 条发射后收益递减（Culler & Singh / Johnson 1991 的曲线），时钟频率被功率墙封顶（Intel 在 2004 年撞上 power density wall），因此**要显著提速就必须使用多个处理单元或专用硬件**——这从"可选优化"变成了"编程必需"。
2. **并行思考的三步法**：① 把工作分解成**可以安全并行**的片段；② 把片段**分配**给处理器；③ 管理**通信与同步**，使其不成为瓶颈。任何一步做错，加速比都会被 `Amdahl 定律` 与 `T(P) ≤ W/P + S` 死死卡住。
3. **FAST != EFFICIENT**：在并行机器上跑得更快，不等于用好了硬件。用 `speedup/P` 衡量效率：10 核上 2 倍加速比只有 20% 效率。程序员要"用满机器提供的能力"（SIMD 宽度、核心数、带宽），硬件设计者要在**面积与功耗**预算内"最大化每面积/每瓦性能"。
4. **高效 = 高效的数据访问**：cache 命中把几百周期降到几个周期；而从 DRAM 读 64 bit 的能耗（~1200 pJ）是片上 SRAM（~26 pJ）的 ~46 倍、整数运算（~1 pJ）的上千倍。**算术强度决定你受限于算力还是带宽**——Roofline 的 ridge point（本笔记例中 38.4 flop/byte）就是那条分界线。
5. **性能模型是上界，机器现实是下界**：work–span/并行度给出的并行度上限（`W/S`）不含带宽、一致性流量、同步代价；静态分块 for 循环的并行度上限恰好是线程数 `P`。要拿到真实的加速比，必须同时解决**带宽、负载均衡、伪共享、同步粒度**。

---

6. 常见陷阱与注意事项

- **"更快的机器会解决一切"**：2004 年后这个假设失效。优化必须来自并行与效率（用满 SIMD/核心/带宽），而不是等下一代工艺。
- **数据竞争（data race）**：多个线程对同一地址并发访问且至少有一个是写、又没有同步时，程序行为未定义。注意示例 2 的反面：**没有数据竞争也不代表快**——伪共享（false sharing）在语义上完全正确，却能让性能掉几十倍；把"逻辑独立的变量"放进同一个 cache line 就是典型错误。
- **忽略通信与同步成本**：`T(P) = W/P + S`，其中 `S` 是**关键路径**（含所有 barrier、归约、锁等待）。示例 1 中一次 `reduction` 的 O(P) 通信在 P 大时不可忽略；每次迭代都加一次全局同步会把 `f` 推高，Amdahl 上限随之崩塌（见 §4.5：64 核 + 100 ns 通信 → 加速比 0.63×）。
- **负载不均（load imbalance）**：静态均分在迭代成本相近时才好用；样例成本差异大（如三角形循环、稀疏矩阵、不规则的 N 体问题）时，必须用动态/引导式调度（`schedule(dynamic, k)`、任务队列、work stealing），否则总时间由最慢的线程决定——这正是 DEMO 2 的教训。
- **忽视带宽与缓存效应**：示例 3 的 triad 只用到峰值算力的 0.4%；把 8 核换成 16 核几乎不会更快。看到"加核不提速"先查算术强度与实测带宽，再用分块（tiling）/融合（fusion）/数据复用来提高重用率。
- **错误地理解内存序（memory ordering）与同步**：不是"用了一个 `volatile` 或者 `atomic` 就对了"。并非所有平台都是顺序一致（sequential consistency）；锁要保证互斥与可见性；归约/自增要用原子操作或正确的 reduction 语义，否则会得到偶尔错误的结果（且难以复现）。此外，并行浮点归约的**加法顺序不确定**，结果不可逐位复现——这不是 bug，但依赖逐位确定性的测试会误判。
- **测量方法错误**：只测一次、用挂钟时间（wall-clock）夹住含线程创建的开销、不预热页面/缓存、不做 `-O3` 的 release 编译就下结论，都会得到误导性的加速比。测量必须写明：编译命令、优化级别、线程数、问题规模、重复次数与取值方式（如示例 3 取 3 次最优）。

---

7. 思考题（带答案）

Q1. 一台 16 核、3.0 GHz、每周期 8 宽 SIMD、支持 FMA 的机器，理论峰值 768 GFLOP/s。你的 kernel 算术强度是 0.25 flop/byte（像示例 1 的求和），实测内存可持续带宽 20 GB/s。请问：(a) 性能上限是多少？(b) 如果老板要求你把 kernel 加速 4 倍，最有希望的两条改造方向是什么？

【答案】 (a) 这是典型的带宽受限点。按 Roofline：`attainable = min(768, 0.25 × 20) = 5 GFLOP/s`，**只有峰值的 0.65%**。等价地，按流量算：遍历 256 MB 的只读数组，`256 MB / 20 GB/s = 12.8 ms` 是硬下界，无论你开多少线程、用多宽的 SIMD 都绕不过去。(b) 方向一：**降低数据流量 / 提高数据重用**——分块（tiling）让数据留在 L1/L2 里被反复使用，或做循环融合（fusion）避免中间数组来回写内存；把算术强度从 0.25 提到 1.0，理论上界就从 5 GFLOP/s 提高到 20 GFLOP/s（4 倍）。方向二：**降低数据宽度 / 提高精度效率**——在结果可接受的前提下用更窄的表示（如把 `float` 换成量化后的 `int8` / `bf16`，或只读必要字段，避免结构体数组 AoS→SoA 造成无效字节搬运），减少每元素字节数。注意"堆更多核心"或"加宽 SIMD"在这条曲线上**完全没有用**（它们改进的是屋顶高度，而你现在贴着斜屋顶）。

Q2. 你的并行求和程序在 8 核机器上只跑了 3 倍加速比。请给出一份至少三项的排查清单，每项说明它对应哪个可量化的模型/指标，以及如何验证。

【答案】 ① **串行比例 f（Amdahl 定律）**：用 `speedup(P) = 1/(f + (1−f)/P)` 反推：`3 = 1/(f + (1−f)/8)` → `f + (1−f)/8 = 0.3333` → `f·(7/8) = 0.3333 − 0.125 = 0.2083` → `f ≈ 0.238`。也就是说**约 24% 的时间是串行的**。验证：用 `perf`/profiler 看单线程热点，或把线程数从 1 扫到 8 画出实际加速比曲线与 Amdahl 理论曲线对比，若曲线在大 P 处明显变平，就是串行/同步部分主导。 ② **内存带宽（Roofline / 算术强度）**：示例 1 的 AI ≈ 0.25 flop/byte，1 GiB 数据在 20 GB/s 下需 `12.8 ms × 4 = 51.2 ms`；若单线程已经跑出大部分带宽，那么多线程的加速比上限就是 `单线程带宽/总带宽` 的比值。验证：在程序里统计流量并除以时间得到实测 GB/s；同时跑 STREAM-like microbenchmark 测该机器的可持续带宽作为分母。若多线程实测带宽已达到单线程的 3 倍并饱和，则瓶颈是内存而非代码。 ③ **负载不均（work-span，`T(P) ≤ W/P + S`）**：如果线程之间工作量不等，总时间由最慢线程决定。验证：在每个线程内部记录起止时间打印甘特图，或者改用 `schedule(dynamic, k)` / work stealing 再测；如果提速，则为负载不均。同时检查是否出现**伪共享**（用 perf 的 `cache-misses`/HITM 事件，或把每线程结果数组做 cache-line padding，见示例 2）。 ④（补充）**测量误差**：确认是 release 编译（`-O3`）、预热过页面与缓存、重复多次取最优、并且计时不含线程池创建的一次性开销。

Q3. 讲义说"FAST != EFFICIENT"，并追问"在 10 个处理器的机器上 2 倍加速比是不是好结果"。请从**程序员**和**硬件设计者**两个视角分别说明"效率"的含义，并举一个两者会给出不同结论的场景。

【答案】 – **程序员视角**：效率 = `speedup(P)/P` = 我是否用满了机器**已经提供**的能力（SIMD 宽度、FMA、所有核心、内存带宽、专用单元）。10 核上 2 倍 = 20% 效率，通常意味着通信、同步或负载不均吃掉了大部分收益，是**不好的结果**；而"好"的判据不是绝对加速比，而是把实测性能与 Roofline 上界（`min(peak_math, AI × BW)`）对比，看还差多少。 – **硬件设计者视角**：效率 = **每单位成本**（硅面积、功耗、成本）换来的性能，即在给定面积/功耗预算下选择放入什么能力（更大缓存？更多核心？SIMD 宽度？专用 NPU？）。对设计者来说，"20% 的效率"未必是坏消息——如果这 20% 的服务对象是使用频率最高的负载，或者这颗芯片的功耗已经逼近散热极限，那么为它增加通用核心反而是错的。 – **结论不同的场景**：一个只需要 2 倍加速的**延迟敏感型**移动负载（如相机中的人脸检测），在 10 核芯片上只用到 2 个核心、加速比 2×。**程序员视角**会判定"低效，应该把另外 8 核用起来"；**硬件设计者视角**可能认为这是**最佳设计**——如果把这个负载的剩余部分交给 1 W 预算的 NPU（专用化）完成，那

么"只用了 2 个通用核心"恰恰意味着 CPU 的其余面积可以留给其他任务，整机功耗与成本才是最优。这正是讲义 slide 38 的转向：2004 年后设计目标从"在面积预算内最大化性能"变成"最大化每面积性能"与"最大化每瓦性能"，而程序员的任务是在这个已经被优化过的机器上，把**它提供的能力**真正用满。

Lecture 2: Out-of-order Processor & Pipeline

1. 章节标题与概述

Lecture 2: Out-of-order Processor & Pipeline

- **本讲核心问题**：硬件如何在没有程序员显式说明的情况下，从一段**顺序代码**里自动挖掘并行性？既然"程序必须看起来是按程序序、一条一条执行的"，处理器又如何能在内部乱序、重叠、甚至猜测性地执行多条指令，从而让单个指令流的吞吐率远高于"一条指令走完四个阶段"的朴素模型?
- **涉及的主要硬件/软件机制**：
- **流水线（pipelining）**：把指令执行切成 Fetch / Decode / Execute / Commit 等阶段，让不同指令处于不同阶段，从而把一个阶段的时间变成整个处理器的吞吐周期。
- **冒险与消解（hazards）**：数据冒险（data hazard）、控制冒险（control hazard）、结构冒险（structural hazard）分别用 **旁路转发（forwarding）**、**停顿插泡（stall / bubble）**、**分支预测与推测（speculation）**、**流水线冲刷（flush）**、以及**增加执行端口**来处理。
- **数据流（dataflow）与乱序执行（out-of-order, OoO）**：用寄存器重命名把程序序翻译成"真依赖图"，在保留顺序幻觉的前提下按数据就绪顺序发射；前端取指/译码与后端提交保持顺序，中间的执行完全乱序。
- **超标量（superscalar）**：加宽发射宽度 W，使 IPC 可以大于 1；代价是调度复杂度按 $O(W^2)$ 增长。
- **硬件多线程 / SIMD / 多核（CS149 补充视角）**：当 ILP 在 8 宽左右"榨干"之后，工业界转向线程级、数据级并行，并用硬件多线程（SMT）隐藏内存延迟。
- **在并行计算知识体系中的角色**：本讲是整门课的"硬件底座"。它解释了为什么"并行"不只是多线程和多核——**单核内部本身就是一个复杂的并行机器**（流水线并行 + 乱序并行 + 推测并行），也解释了为什么这套机制最终会撞墙（ILP 有限、调度复杂度 $O(W^2)$、频率受功耗限制），从而把历史推向多核。同时它给出了两个贯穿全课程的性能分析工具：**延迟界（latency bound，关键路径）** 与 **吞吐界（throughput bound，执行端口数量与发射率）**——后面分析 SIMD、GPU、缓存、带宽时都会反复使用。
- **配套材料**：
- `lectures/02_ilp.pdf`（讲义正文标题页写作 "Lecture 2: Instruction-Level Parallelism"，共 112 页幻灯片抽取文本 `extracted/02_ilp.txt`）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 下直接下载（课程主页 https://www.cs.cmu.edu/~418/，日程表 https://www.cs.cmu.edu/~418/schedule.html，Fall 2026 日期 Aug 26）。讲义页脚沿用历史学期（如 Fall 2025）字样属于正常的讲义复用现象，不是错误。
- `cs149_supp/multicore1.txt`：Stanford CS149 Fall 2025 Lecture 2 "A Modern Multi-Core Processor (Part I)"（共 108 页）——**公开**的补充读物，用于补足"乱序之后怎么办"的部分（缓存层次、SIMD、多核、硬件多线程、GPU SM 结构）。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属 **未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 历史学期 PDF（如 Performance Analysis/Profiling、Transactional Memory、AI in System Design 等 Fall 2026 尚未发布的讲义）位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- 本讲 Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。

---

2. 核心概念与硬件/软件架构图解

2.1 ISA 与微架构：接口与实现的分离

- **定义与目的**：**指令集架构（ISA, Instruction Set Architecture）** 是硬件与软件之间的功能契约，它规定"每条指令做什么"，但不规定"怎么做"。**微架构（microarchitecture）** 是 ISA 的具体实现方式。讲义给出的关系式是：

```
Architecture : Microarchitecture :: Interface : Implementation
```

这个分离是所有性能优化的前提：同一份二进制可以在完全不同的微架构上运行，因此**"快"是实现的属性，不是 ISA 的属性**。本讲讨论的流水线、乱序、推测全部属于微架构层面，对程序员"不可见"（不改变程序输出），但**极大地改变性能**。

- **直观解释（"它是什么？"）**：把 ISA 想象成**餐厅菜单**，微架构想象成**厨房**。菜单上写着"A 套餐包含汤、主菜、甜点"（功能契约），但厨房可以是一个人从头做到尾（顺序执行、无流水线），也可以是一个流水线厨房：有人专管煮汤、有人专管煎牛排、有人专管摆盘（五级流水线）；甚至可以是一个"提前猜客人要点什么就先把牛排下锅"的厨房（推测执行）。客人（软件）看到的永远是同一份菜单。
- **架构/机制图解**：

```
软件 / 编译器 （ISA：接口层） ---------------------------------------------------------------- "ldr r5, [r3], #4" "mla r0, r4, r5, r0" "bne .L3" ^ 只规定语义：从 r3 取 4 字节，写 r5，r3+=4，等等 ---------------------------------------------------------------- 微架构实现层（对程序员不可见，可任意重排/重叠/猜测） Fetch -> Decode -> [ 乱序发射 ] -> Execute -> Commit 多宽？多深？哪些执行端口？ROB 多大？分支预测器多聪明？ ★ 这些选择只影响性能，不影响程序语义 ★
```

关键操作与性能特征：ISA 固定了"每条指令的功能延迟"这一语义概念，但**微架构决定了真实延迟与吞吐**。例如同一条 FMA（乘加）指令，在不同实现上执行延迟可能是 4 或 5 个时钟周期，而**吞吐**可以是每周期 0.5 条或 2 条。本讲后半段的所有"延迟界/吞吐界"分析，都是在微架构层面对 ISA 指令流的定量刻画。

2.2 流水线（Pipelining）：用阶段并行换吞吐

- **定义与目的**：把一条指令的执行过程切分成若干**阶段（stage）**，让多条指令的不同阶段在时间上重叠。讲义给出的最简模型是四级：

```
1. Fetch — 从内存取指令 2. Decode — 确定要做什么并读取输入寄存器 3. Execute — 执行运算 4. Commit — 把结果写回寄存器/内存
```

（真实处理器的流水线级数远多于此。）讲义用多项式求值循环体

```
asm .L3: ldr r5, [r3], #4 // r5 <- coef[j]; r3 <- r3 + 4 (两个操作) cmp r1, r3 // j < terms ? mla r0, r4, r5, r0 // value += r5 * power (乘 + 加) mul r4, r2, r4 // power *= x bne .L3 // 循环回边
```

演示：非流水线时每条指令延迟 4 ns、吞吐 1 条/4 ns；四级流水线后**延迟仍是 4 ns**，但**吞吐变成 1 条/ns**，即 4× 加速。这就是"N 级流水线最多给出 N× 加速"的来源。

- **直观解释（"它是什么？"）**：流水线就像**洗衣房**。洗一桶衣服要 30 分钟洗 + 30 分钟烘 + 30 分钟叠，一个人从头做到尾，每 90 分钟出一桶；但如果洗、烘、叠分别是三台机器/三个人，第 1 桶仍然要 90 分钟（**延迟不变**），但从第 2 桶开始每 30 分钟就出一桶（**吞吐提高 3 倍**）。你自己的衣服并没有变快，但洗衣房单位时间的产量变成了 3 倍。
- **架构/机制图解**：

```
cycle: 1 2 3 4 5 6 7 8 9 +-----+-----+-----+-----+-----+-----+-----+-----+-----+ instr i F | F | D | E | C | | | | | | instr i+1 | | F | D | E | C | | | | | instr i+2 | | | F | D | E | C | | | | instr i+3 | | | | F | D | E | C | | | instr i+4 | | | | | F | D | E | C | | +-----+-----+-----+-----+-----+-----+-----+-----+-----+ 非流水线: 1 条 / 4 周期, 延迟 4 周期 流水线 : 1 条 / 周期 , 延迟仍为 4 周期 => 吞吐 4x 处理器同时在处理 4 条指令（4 份"在途"工作）
```

关键操作与性能特征：设 stage 时间为 t、级数为 N，则 – 延迟（单条指令完成时间）= N·t，**不被流水线改善**； – 吞吐 = 1/t（理想情况），**改善 N 倍**； – 在途指令数 = N（在理想满流情况下）。

但理想情况要求**每一级都始终有独立工作可做**。只要相邻指令之间存在依赖（见 2.3），流水线就会出现气泡，实际吞吐下降到 1/(t + 停顿)。

2.3 冒险（Hazards）：三种让流水线停下来的原因

- **定义与目的**：冒险是阻止下一条指令在下一个周期进入其应属阶段的条件。讲义把限制并行性的冒险分为三类：

冒险类型	触发条件	讲义中的例子	主要消解手段
数据冒险 (data hazard)	后续指令需要读取前面指令尚未写回的结果（RAW 读后写）	`ldr ra, [rb], #4` 写 `rb`；紧邻的 `cmp rc, rd` 要读 `rb`	停顿插泡、**旁路转发 (forwarding)**、寄存器重命名
控制冒险 (control hazard)	分支尚未执行完，处理器不知道下一条该取谁	`bne .L3` 之后仍顺序取到了 `pop` / `bx`	流水线**冲刷 (flush)**、**推测执行 + 分支预测**
结构冒险 (structural hazard)	数据已就绪，但没有空闲硬件端口可以发射	Mem / Int / Mult 三类端口分别只有 1 个，而循环体每轮需要 1 个 ldr、2 个整数 op、2 个乘法 op	增加执行端口数量、调整指令组合（编译器调度）

- **直观解释（"它是什么？"）**：
- **数据冒险**像**接力赛交接棒**：第二棒选手必须在第一棒把棒子递到手上才能起跑。如果两人离得太近，第二棒就得原地等一下（stall）；**旁路转发**相当于第一棒选手在跑动中直接把棒子往前扔过去——不必等他把棒子放回存放区（Commit）再让第二棒去取。
- **控制冒险**像**开车到岔路口却看不清路牌**。你要么减速停车看清（冲刷+重取，代价大），要么"凭经验猜左转"（分支预测），猜错了就得倒车重来（misprediction penalty）。

- **结构冒险像只有一台收银机的超市**：顾客（指令）已经选好商品、钱也准备好了（数据就绪），但收银台被占着，只能排队。
- **架构/机制图解**：下面是讲义中的**数据冒险停顿**与**控制冒险冲刷**两个时序图（`ldr` 写 `r3`，因此紧随其后的 `cmp` 不能立刻发射）：

```
(a) 数据冒险 -> 停顿插泡 (bubble)  (b) 控制冒险 -> 冲刷 (flush)  cycle: 1 2 3 4 5 6  cycle: 1 2 3 4 5 6
+----+----+----+----+----+----+  +----+----+----+----+----+----+
ldr | F | D | E | C |  |  |     mla | F | D | E | C |  |  |
cmp |  | ?? | ?? | F | D | E |  mul |  | F | D | E | C |  |
mla |  |  |  |  |  |  |        bne |  |  | F | D | E | C |
+----+----+----+----+----+----+  +----+----+----+----+----+----+
ldr 写 r3 -> cmp 读 r3 pop |  |  | F | D |   <- 错误取指! 中间插入 NOP 气泡, 吞吐下降   bx |  |  |  | F |   <- 错误取指! ^^^ 分支跳转后必须丢弃, 从 .L3 重取 惩罚随流水线深度线性增长 (c) 旁路转发 (forwarding) 消除停顿
+----+----+----+----+----+----+  ldr | F | D | E | C |  |
Execute 阶段算出的 r3+4 cmp |  | F | D | E | C |   <- 直接从 E 的输出"抄近路"送到 D 的输入
+----+----+----+----+----+----+  不必等 C 阶段写回寄存器堆 数据在 Execute 之后就已可用 => 大多数(不是全部)停顿可被消除
```

关键操作与性能特征：
- **停顿（stall）**：注入 NOP 气泡，代价 = 插入的周期数；但有些停顿不可避免——**长延迟指令**（除法、缓存缺失）无法靠转发解决。
- **转发（forwarding）**：在 Execute 之后、Commit 之前就把结果前送。讲义明确指出：**转发不是免费的**——流水线越深越复杂，需要的转发通路数量急剧增长，所以"转发能消掉多少停顿"这件事本身随流水线加深而恶化。
- **冲刷（flush）**：代价随流水线加深而线性增长（要丢弃的在途指令更多）。讲义因此总结：**流水线深度在 N≈15 附近触顶**，再深下去，转发成本与冲刷成本会吃掉所有收益。

2.4 推测执行与分支预测（Speculation）

- **定义与目的**：程序必须"看起来按程序序执行"，因此所有指令看似都依赖前一条——但分支会跳转，处理器不能在等分支结果的同时空转。**推测（speculation）** 就是"猜一个方向，先把指令取进来执行，猜错就整体回滚（roll back）"。其目的是**为流水线创造本不存在的独立工作**，把控制冒险的代价从"串行等待分支解决"降为"猜错时的一次性惩罚"。
- **直观解释（"它是什么？"）**：像**考试时先做后面的题**。你读到一道需要长时间计算的选择题，于是先"假设答案是 A"，然后按 A 往下推（推测执行）；如果后面发现 A 错了，就擦掉所有基于 A 写下的内容重做（回滚/冲刷），而不是从第一题一直卡在那里等。
- **架构/机制图解**：

```
静态指令序列 (内存中的程序布局)                +----------------------+ <-- .L3 循环体
| ldr r5, [r3], #4 |
| cmp r1, r3 |
| mla r0, r4, r5, r0 |
| mul r4, r2, r4 |
| bne .L3 ----------+----(1) 预测"跳转" (taken)                +----------------------+
|                                                                                    |
| pop {r4, r5} | <-- 顺序流(错误路径)
| bx lr |
v                                                                                    +----------------------+ 继续投机执行 .L3 循环体 (2) 分支在 Execute 阶段出结果
├─ 预测正确 (>95% 的情况): 零代价, 满载
└─ 预测错误: 冲刷错取指令, 从正确 PC 重取 惩罚 = 流水线深度 (现代 CPU ~15-20 周期)
```

关键操作与性能特征：
- 现代处理器会在 **Fetch 阶段就做预测**——甚至还没译码出"这是一条分支"就已经按预测的 PC 取指。
- 讲义给出的预测准确率量级是 **>95%**，但**分支误预测仍然是主要性能问题**：4 宽 × 20 级流水线的机器，每次误预测约损失 15–20 个周期，意味着可发射 60–80 条指令的窗口被浪费掉。
- 一个极端情形（讲义明确点名）：**依据随机数据分支的代码**——预测器无法学习规律，性能会被 fetch 停顿主导。这正是后面代码示例 `branch_pred.cpp` 要量化的现象。

2.5 数据流（Dataflow）与真假依赖

- **定义与目的**：程序序在寄存器层面引入了大量**伪依赖（false dependency）**。**数据流（dataflow）** 的思想是：只看指令之间**真正**的数据依赖关系（**真依赖 = 读后写 RAW, read-after-write**），忽略只由"复用了同一个寄存器名"造成的顺序约束，从而暴露出更多并行性。讲义的原话是："Dataflow increases parallelism by eliminating unnecessary dependences."

```
依赖类型速查（本章表格之一）：
+-------+----------------------+----------------------------+------------------+
| 类型 | 含义 | 反例（同一个寄存器名复用） | 是否真依赖 |
+-------+----------------------+----------------------------+------------------+
| RAW | 读后写 (read-after- | I1: r0 = a + b | 真依赖，必须保留 |
|  | write) | I2: c = r0 * 2 |  |
| WAR | 写后读 (write-after- | I1: c = r0 * 2 | 伪依赖，重命名消除|
|  | read) | I2: r0 = a + b |  |
| WAW | 写后写 (write-after- | I1: r0 = a + b | 伪依赖，重命名消除|
|  | write) | I2: r0 = c * d |  |
| 控制 | 分支决定后继 | bne .L3 之后的所有指令 | 推测+预测+冲刷处理|
| 结构 | 端口/资源冲突 | 两条乘法指令争 1 个乘法端口 | 增加端口或用别类指令顶上 |
+-------+----------------------+----------------------------+------------------+
```

- **直观解释（"它是什么？"）**：把寄存器想成**白板上的编号格子**。程序序规定"必须先擦掉 1 号格子的旧内容再写新的"，但这只是因为大家共用了那块白板。数据流说：**再拿一块新白板写下新值就行了**，需要旧值的人继续看旧白板，需要新值的人看新白板——两块白板互不干扰，于是原本必须排队的两步可以同时做。这就是**寄存器重命名（register renaming）** 的直觉。
- **架构/机制图解**：以多项式循环体为例，讲义给出的数据流执行图（假设完美调度、无限执行单元）：

```
loop iteration j ================================================================ r3(j-1) ---> [ ldr ] --(2c)--> r5 ------------------+
                                                                                  |
                                                                                  +------> r3(j) ---> [ cmp ] --(1c)
                                                                                  |                   |
                                                                                  |                   v
                                                                                  v                 [ bne ] --(1c)
                                                                                  [ mla ] --(3c)--> r0(j+1)
                                                                                  ^                 r4(j-1) -----+
                                                                                  ^                              |
                                                                                  [ mul ] --(2c)-+--> r4(j+1)
================================================================ 指令延迟（讲义给定）: ldr = 2c, mul = 2c, cmp = 1c, bne = 1c, mla = 3c
跨迭代关键路径: r0 链 mla(j) 依赖 mla(j-1) : 3 周期/迭代 <== 瓶颈
r4 链 mul(j) 依赖 mul(j-1) : 2 周期/迭代
=> 迭代下界 = max(3, 2) = 3 周期/迭代
每轮 5 条指令 => IPC = 5/3 ≈ 1.67  （对比: 完美流水线 IPC = 1）
```

关键操作与性能特征： – **关键路径（critical path）** 的定义（讲义原文）：*数据流图中跨迭代的最长路径*。在上例中就是 `mla` 链。 – 关键路径给出**性能上限，且是"延迟界"（latency bound）**：即使执行单元无限多、端口无限宽，也无法快过 3 周期/迭代，因为 `mla` 必须等前一次 `mla` 的结果。**这个程序是延迟受限的（latency-bound）。** – 讲义强调这只是一个**心智模型与分析工具**：真实 CPU 未必达到该界限，但用它来分析程序非常有用。

2.6 乱序执行（Out-of-Order, OoO）微架构

- **定义与目的**：OoO 的核心思想是"**按数据流顺序执行，但保持顺序执行的幻觉**"。指令按程序序进入和离开一个**指令缓冲/重排序缓冲（instruction buffer / reorder buffer, ROB）**，而在缓冲内部，发射顺序完全由**操作数是否就绪**决定。这样既拿到了数据流的并行性，又对软件保留了"顺序单发射机器"的语义。
- **直观解释（"它是什么？"）**：像**餐厅的出菜口**。顾客（程序）按点单顺序排队（in-order frontend），厨房内部可以任意乱序地同时炒五道菜（out-of-order execute），但**摆盘上菜必须按点单顺序**（in-order commit）——1 号桌的菜没好，2 号桌的菜做好了也得先放在保温台上等着。顾客看到的永远是"按顺序上菜"，但厨房的吞吐被最大化了。
- **架构/机制图解**：

```
+-------------------------------------------------------------------------+ | CPU core | | | | PC --> [ Fetch ] --> [ Decode ] --> +--------------------------+ | | in-order in-order | Instruction Buffer / ROB | | | | (W 条/周期进入，乱序发射) | | | +-----------+-------------+ | | | | | +------------------------------+---------------+ | | | out-of-order issue (数据就绪即发射) | | | v v v v | | [ EXE: ALU ] [ EXE: FMA ] [ EXE: FMA ] [ EXE: LD/ST ] | | | | | | | | +--------------+------+-------+------+---------+ | | v | | [ Commit / Retire ] <-- in-order | | | | +-----------------------------------------|-------------------+ v 寄存器堆 / 内存状态 ^^^
```

界	定义（讲义原文）	组成要素	典型瓶颈
吞吐界 (throughput bound)	`ops / issue rate`，即操作数除以执行端口发射率	每类操作的数量 + 执行端口数量与发射率	乘法端口需 (1 mla + 1 mul) / (2 + 3 周期) = 5c/迭代

实际性能 ≈ max(延迟界, 吞吐界)。讲义强调真实 CPU 未必精确达到这些界，但作为分析工具非常有用。

- **直观解释（"它是什么？"）**：
- **延迟界**像**一条只能单人通行的吊桥**：桥上有多少人排队不重要，重要的是队首到队尾要走多久。加宽桥面（更多执行单元）毫无帮助，只能**缩短链条**（拆分依赖链）。
- **吞吐界**像**只有 3 条收银通道的超市**：顾客之间毫无依赖（可以任意并行），但**收银台数量**决定了每秒最多结账多少单。这时增加顾客的独立性没用，只能**加通道**（加端口）或**换更快的收银员**（提高发射率）。
- **架构/机制图解**：

```
(a) 延迟界: 一条长依赖链 — 执行端口大量空闲, 加宽无用 t: 1 2 3 4 5 6 7 8 9 10 11 12 op1 [===== FMA (4c) =====] op2 [===== FMA (4c) =====] op3 [===== FMA (4c) =====] |<-------- 关键路径 = 3 x 4 = 12 周期 --------->| 端口空闲率极高, IPC 低; 想加速只能"拆链" (b) 吞吐界: 依赖链被拆成 8 条独立链 — 端口成为瓶颈 t: 1 2 3 4 5 6 7 8 9 10 op1 [FMA] op2 [FMA] op3 [FMA] op4 [FMA] op5 [FMA] op6 [FMA] |<- 每周期都在发射, 端口 100% 占用 -->| IPC 高, 想加速只能"加端口/加宽 SIMD" (c) 真实程序: 两个界同时存在, 取 max +-------------------------+ | 实际性能 ≈ max(延迟界, 吞吐界) | +-------------------------+ 多项式循环: 延迟界 3c/迭代, 吞吐界 5c/迭代 => 预期 5c/迭代
```

关键操作与性能特征：这两个界是**整个课程反复使用的分析框架**。在后面的 SIMD/GPU 讲座中，"吞吐界"会变成"SIMD 通道数 × FMA 端口数"，"延迟界"会变成"寄存器依赖链长度"；在缓存/带宽讲座中，二者之上还会再加一个**带宽界**。

2.9 从 ILP 到 TLP：多核、SIMD 与硬件多线程（CS149 补充视角）

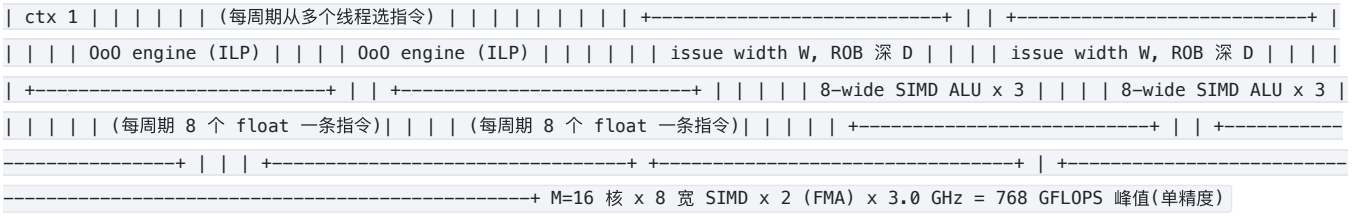
- **定义与目的**：讲义最后给出了"为什么 ILP 会撞墙、为什么工业界转向多核"的完整论证链：
- **程序的 ILP 有限**：即便完美调度，>8 宽也无收益；
- **流水线不能无限加深**：分支误预测惩罚随深度增长；
- **频率受功耗限制**：不能靠提频解决；
- **动态调度开销显著**：OoO 硬件本身昂贵、$O(W^2)$ 复杂。 => **从硬件角度，多核效率高得多；但并行软件很难写**。工业界"尽可能久地抵制多核"，最终多核到来时，CPU 微架构反而**被简化以塞进更多核**。

CS149 的补充讲义把"现代处理器上可用的并行形式"整理为三类，并额外增加**硬件多线程**用于隐藏内存延迟：

并行形式	粒度/来源	谁发现并行	硬件代价	对程序的要求
超标量 / ILP	指令级、隐式、细粒度（核内）	**硬件**运行时动态发现	昂贵复杂（$O(W^2)$ 调度、重命名、ROB）	顺序代码即可，性能可移植性好
SIMD	数据级（核内）	**编译器**静态向量化，或 **GPU 硬件**运行时（implicit SIMD）	中等（宽 ALU + 宽寄存器）	需要**控制流一致（coherent execution）**，否则掩码浪费，最差降到 1/8（CPU 8 宽）或 1/32（GPU 32 宽）
多核 / 线程级	线程级、显式、粗粒度	**软件**显式创建线程	每核一套前端+执行资源	需要足够的并行工作与正确的同步
硬件多线程 (SMT/交错)	同一核上的多个上下文	软件给线程，硬件交错发射	额外的上下文存储	需要**远多于 ALU 数量的独立工作**来隐藏延迟

- **直观解释（"它是什么？"）**：
- **多核**像**把一间大厨房隔成 4 间小厨房**，每间自己买菜、自己炒（各自独立的指令流）；比"一间超级厨房里塞满自动炒菜机器人"（复杂 OoO 单核）更容易扩建。
- **SIMD** 像**流水线上一个字模同时盖 8 个盒子**：只有 8 个盒子都走同一步骤（同一条指令）时才高效；如果第 3 个盒子要求换个图案（控制流发散），字模得停下重来，其余通道只能空转（**掩码丢弃**）。
- **硬件多线程**像**一位厨师同时照看 4 口锅**：第 1 口锅在炖（等内存，12 个周期），厨师立刻去翻第 2 口锅（发射另一个线程的算术指令）。**锅并没有变快，但厨师不再闲着**。
- **架构/机制图解**（把四种并行叠在一张图上）：

```
+-------------------------------------------------------------------------+ | Chip — 多核 (multi-core): M 个核, M 条并发指令流, 软件显式建线程 | | | | +-------------------------------+ +-------------------------------+ | | | Core 0 | | Core 1 | | | | +---------------------------+ | | +---------------------------+ | | | | | SMT: exec ctx 0 | ctx 1 | | | | SMT: exec ctx 0
```



关键操作与性能特征（来自 CS149 补充讲义的关键数字）： – **Kaby Lake 系列核心**：2 路 SMT；每核每周期最多 **4 条独立标量指令**、最多 **3 条 8 宽向量指令**（其中最多 2 条向量乘或 3 条向量加）。 – **4 核 × 8 宽 SIMD × 3 单元 × 4.2 GHz ≈ 400 GFLOP/s**（讲义按"每条 SIMD 通道操作记 1 FLOP"计；若把 FMA 记作 2 FLOP 则约 800 GFLOP/s）。 – **数据访问延迟（Kaby Lake, 4 GHz）**：L1 = 4 周期、L2 = 12 周期、L3 = 38 周期、DRAM 最好情况 ≈ 248 周期。带宽约 **38 GB/s**。 – **硬件多线程的必要线程数**：若每线程执行"3 条算术指令 + 一次 12 周期延迟的 load"，单线程利用率仅 **3/15 = 20%**；两线程 40%；**需要 5 个线程才能到 100%**。若算术指令增至 6 条，则只需 **3 个线程**。→ **"每次访存携带的算术越多，隐藏延迟所需线程越少"**（这正是算术强度/计算访存比的直觉来源）。 – **GPU 极端吞吐导向**：以补充讲义给出的 V100 为例，一个 **SM（Streaming Multiprocessor）** 拥有 64 个 warp 执行上下文、4 个 sub-core、每个 sub-core 有 16 宽 fp32 SIMD 单元（每 2 个周期完成一次 32 宽操作）、256 KB 寄存器、128 KB 共享内存+L1；**80 个 SM**，HBM 带宽约 **900 GB/s**。

---

3. 代码示例与性能分析

3.1 示例一：多项式求值——从"延迟界"到 ILP 显式重构

讲义整场都在用多项式求值讲解流水线、数据流、乱序。下面把它写成一个可以真正跑出差异的基准测试：**Horner 串行链**（纯延迟界）对比**偶/奇双链重构**（把关键路径砍半）。

```cpp
// ------------------------------------------------------------------
// poly_ilp.cpp
// 编译 (release):
//   g++ -O3 -march=native -fno-tree-vectorize poly_ilp.cpp -o poly_ilp
//   # -march=native 打开 FMA (vfmadd...), 使 mla 变成单条 FMA 指令
//   # 注意: 不要加 -ffast-math, 否则编译器可自由重结合浮点, 实验失去意义
// 运行:
//   ./poly_ilp 8192 2000        # 8192 个系数, 重复 2000 次
// 观察:
//   two_chain 的耗时约为 horner 的一半 => 关键路径被砍半
// ------------------------------------------------------------------
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <chrono>
#include <cmath>

// (a) 朴素 Horner: value = ((c[n-1]*x + c[n-2])*x + ...)*x + c[0]
//     关键路径 = (n-1) 次 FMA 串行依赖 -> 纯 latency bound
static inline float poly_horner(const float* c, int n, float x) {
    float value = c[n - 1];
    for (int j = n - 2; j >= 0; --j)
        value = value * x + c[j];       // 编译为 vfmadd213ss 等
    return value;
}

// (b) 偶/奇双链:  poly(x) = B(u) + x * A(u),  u = x^2
//     A(u) 收集所有奇数下标项, B(u) 收集所有偶数下标项 (要求 n 为偶数)
//     关键路径 ≈ 1 次 mul + (n/2) 次 FMA + 1 次 add -> 并行度 ~2
static inline float poly_two_chain(const float* c, int n, float x) {
    const float u = x * x;
    float A = 0.0f, B = 0.0f;
    for (int j = n - 1; j >= 1; j -= 2) {
        A = A * u + c[j];               // 奇数下标链 (与 B 链完全独立)
        B = B * u + c[j - 1];           // 偶数下标链
    }
    return B + x * A;
}

int main(int argc, char** argv) {
    const int    n = (argc > 1) ? std::atoi(argv[1]) : 8192;
    const int    repeat = (argc > 2) ? std::atoi(argv[2]) : 2000;
    if (n % 2) { std::fprintf(stderr, "n must be even\n"); return 1; }

    std::vector<float> c(n);
    for (int j = 0; j < n; ++j)
        c[j] = std::sin(0.001f * (float)j);   // 任意非平凡系数

    auto bench = [&](const char* name, float (*fn)(const float*, int, float)) {
        volatile float sink = 0.0f;                 // 防止整个循环被优化掉
        float x = 1.0009765625f;                    // 略大于 1, 避免溢出
        auto t0 = std::chrono::steady_clock::now();
        for (int r = 0; r < repeat; ++r) {
            sink = sink + fn(c.data(), n, x);
            x += 1e-7f;                             // 让各次调用的 x 不同
        }
        auto t1 = std::chrono::steady_clock::now();
        double sec = std::chrono::duration<double>(t1 - t0).count();
        double perCall = sec / repeat;
        // 用 3.0 GHz 估算关键路径周期数 (仅作量级参考)
        std::printf("%-12s : %8.3f ms/call   %8.2f cycles/FMA (@3.0GHz)   sink=%.6f\n",
                    name, perCall * 1e3, perCall * 3.0e9 / (double)n, (double)sink);
    };

    bench("horner",    poly_horner);
    bench("two_chain", poly_two_chain);
    return 0;
}
```

【代码做什么？】 1. 构造 `n` 个系数（`n` 取偶数），保证两个版本计算的是同一个多项式。 2. `poly_horner` 从最高次项开始向下迭代，每一步 `value = value*x + c[j]`。这是一个**严格的串行依赖链**：第 j 步必须等第 j+1 步的 `value`。 3. `poly_two_chain` 先把 **偶数下标项** 与 **奇数下标项** 分成两个独立的多项式（都关于 `u = x*x`），各自做 Horner，最后 `B + x*A` 合并。两条链之间**没有任何数据依赖**，因此可以同时推进。 4. 两个函数都在 `repeat` 次调用中被计时，用 `volatile sink` 阻止死代码消除，用 `x += 1e-7f` 阻止编译器把重复调用"折叠"成一次。 5. 输出每次调用的毫秒数，并按 3.0 GHz 折算成"每个 FMA 的周期数"，便于对照讲义给出的 FMA 延迟 4 个周期。

【并行机制与性能解说】

- **硬件上如何并行**：两个版本编译出的都是一串**标量 FMA 指令**，没有 SIMD、没有多线程。它们的唯一并行性来自 **OoO 引擎发现的 ILP**。`poly_horner` 的指令之间全部 RAW 依赖，OoO 窗口里根本没有可同时发射的指令；`poly_two_chain` 则交替发射 A 链与 B 链的两条 FMA，每周期可发射 2 条（受 FMA 端口数限制，Skylake 级机器通常 2 个 FMA 端口）。
- **Work / Span / 并行度分析**（设 n 个系数，一次调用）：

版本	Work（总操作数）	Span（关键路径）	并行度 = Work/Span
`poly_horner`	(n−1) 次 FMA	(n−1)·L_FMA	≈ 1
`poly_two_chain`	2 mul + n 次 FMA + 1 add = n+3	L_mul + (n/2)·L_FMA + L_mul + L_add	≈ 2

取 `n = 8192`、`L_FMA = L_mul = L_add = 4` 周期： – Horner: Span = 8191 × 4 ≈ **32 764 周期/次调用**； – two_chain: Span ≈ 4 + 4096×4 + 4 + 4 = **16 396 周期/次调用**； – 理论上限加速比 ≈ **2.0×**，而 Work 增加了约 3 次操作（<0.04%）。

- **瓶颈诊断**：这是**教科书式的延迟界（latency bound）**。执行端口几乎全程空闲，加宽机器、加更多 FMA 单元完全没有用；唯一有效的优化就是**缩短关键路径**。想继续加速就要继续拆链（Estrin 方案、4 链、8 链…），但每拆一层 Work 都会略微上涨，收益按 2^k 递减。
- **另一个真实世界的坑**：如果加了 `-ffast-math`，**编译器可能自己就把 Horner 重组成 Estrin**，两个函数耗时相同——这不是实验失败，而是"编译器替你做了 ILP 重构"。反过来，不加 `-ffast-math` 时编译器**不允许**重结合浮点，所以它会老老实实保留串行链。这正是"什么时候必须手工暴露 ILP"的经典案例。

3.2 示例二：求和——延迟界 → 吞吐界 → 带宽界的连续迁移

同一个"求和"内核，用三种写法展示性能瓶颈的迁移：**串行单累加器（延迟界）→ 串行多累加器（ILP 显式化）→ OpenMP 多线程多累加器（可能进入带宽界）**。

```cpp
// ------------------------------------------------------------------
// omp_sum.cpp
// 编译 (release):
//   g++ -O3 -march=native -fopenmp omp_sum.cpp -o omp_sum
// 运行:
//   OMP_NUM_THREADS=16 ./omp_sum 100000000      # 1 亿个 double = 800 MB
// ------------------------------------------------------------------
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <chrono>
#include <omp.h>

// (a) 串行, 单累加器: 关键路径 = N 次加法串行依赖 => 纯 latency bound
static double sum_serial(const double* a, size_t n) {
    double s = 0.0;
    for (size_t i = 0; i < n; ++i) s += a[i];
    return s;
}

// (b) 串行, 4 个独立累加器: 关键路径 = N/4 次加法, 依赖链 4 条并行
//     编译器通常会把这样的写法直接向量化成 4 宽 SIMD (AVX2)
static double sum_serial_ilp(const double* a, size_t n) {
    double s0 = 0, s1 = 0, s2 = 0, s3 = 0;
    size_t i = 0;
    for (; i + 4 <= n; i += 4) {
        s0 += a[i]; s1 += a[i + 1]; s2 += a[i + 2]; s3 += a[i + 3];
    }
    double s = (s0 + s1) + (s2 + s3);
    for (; i < n; ++i) s += a[i];          // 尾部
    return s;
}

// (c) OpenMP + 每线程 4 个累加器: 线程级并行 x ILP 显式化
//     用 reduction(+:total) 让 OpenMP 负责最后合并, 避免手工加锁
static double sum_omp_ilp(const double* a, size_t n, int nthreads) {
    double total = 0.0;
    #pragma omp parallel num_threads(nthreads) reduction(+:total)
    {
        const int    tid = omp_get_thread_num();
        const int    nth = omp_get_num_threads();
        const size_t chunk = (n + (size_t)nth - 1) / (size_t)nth;
        const size_t lo = (size_t)tid * chunk;
        const size_t hi = (lo + chunk < n) ? (lo + chunk) : n;

        double s0 = 0, s1 = 0, s2 = 0, s3 = 0;
        size_t i = lo;
        for (; i + 4 <= hi; i += 4) {
            s0 += a[i]; s1 += a[i + 1]; s2 += a[i + 2]; s3 += a[i + 3];
        }
        double s = (s0 + s1) + (s2 + s3);
        for (; i < hi; ++i) s += a[i];
        total += s;
    }
    return total;
}

int main(int argc, char** argv) {
    const size_t n = (argc > 1) ? std::strtoull(argv[1], nullptr, 10) : 100000000ULL;
    std::vector<double> a(n);
    for (size_t i = 0; i < n; ++i) a[i] = 1.0 + 1e-9 * (double)(i & 1023);

    const double bytes = (double)n * sizeof(double);
    auto run = [&](const char* name, double (*fn)(const double*, size_t)) {
        auto t0 = std::chrono::steady_clock::now();
        volatile double r = fn(a.data(), n);
        auto t1 = std::chrono::steady_clock::now();
        double sec = std::chrono::duration<double>(t1 - t0).count();
        std::printf("%-16s : %8.2f ms   %7.2f GB/s   sum=%.1f\n",
                    name, sec * 1e3, bytes / sec / 1e9, (double)r);
    };
```

```
    run("serial",     sum_serial);
    run("serial_ilp", sum_serial_ilp);
    {
        auto t0 = std::chrono::steady_clock::now();
        volatile double r = sum_omp_ilp(a.data(), n, omp_get_max_threads());
        auto t1 = std::chrono::steady_clock::now();
        double sec = std::chrono::duration<double>(t1 - t0).count();
        std::printf("%-16s : %8.2f ms   %7.2f GB/s   sum=%.1f   (threads=%d)\n",
                    "omp_ilp", sec * 1e3, bytes / sec / 1e9, (double)r,
                    omp_get_max_threads());
    }
    return 0;
}
```

【代码做什么？】 1. 分配 `n` 个 `double`（默认 1 亿个 = 800 MB），填充成互不相同的数（防止编译器把"常量数组求和"折叠成乘法）。 2. `sum_serial`：一次遍历，单累加器 `s += a[i]`，形成长度为 n 的加法依赖链。 3. `sum_serial_ilp`：用 4 个累加器同时累加 4 个元素，最后两两合并；尾部剩余元素单独处理。 4. `sum_omp_ilp`：`#pragma omp parallel ... reduction(+:total)` 把区间按线程切分（块划分），**每个线程内部再用 4 个累加器**，最后由 OpenMP 的 reduction 把各线程的 `total` 合并（实现上等价于每线程私有副本 + 末尾层次化合并，不存在热点的原子操作）。 5. 三种写法都统计时间、实际带宽（GB/s）与校验和。

【并行机制与性能解说】

- **`sum_serial` 在硬件上如何执行**：只有 1 条依赖链，OoO 完全无指令可重叠。每次加法延迟约 4 个周期（Skylake 级），所以**吞吐受延迟限制为 4 周期/元素**。
- **`sum_serial_ilp` 如何加速**：4 条独立链让 OoO 每周期都能发射 1 条加法，同时编译器大概率把它**向量化为 256 位 SIMD**（4 个 double 一条 `vaddpd`）。于是瓶颈从"延迟界"转到"**吞吐界**"：每条 SIMD 加法只花 1 个周期，每周期消耗 32 字节。
- **`sum_omp_ilp` 如何并行**：创建 T 个线程，每个线程拿到连续的一块区间（块划分 = 顺序访问，对预取器与 DRAM 行缓冲友好），线程之间**没有共享可写数据**（各自私有累加器），只在末尾做一次 reduction。**无数据竞争、无伪共享**。
- **Work / Span / 并行度分析**（N 个元素、T 个线程、每线程 4 个累加器）：

版本	Work	Span	并行度
`sum_serial`	N 次加法	N·L_add	1
`sum_serial_ilp`	N 次加法（+3 次合并）	(N/4)·L_add + 2·L_add	≈ 4
`sum_omp_ilp`	N 次加法	(N/(4T))·L_add + $\log_2$(T)·L_add	≈ 4T

取 **N = 10^8、T = 16、L_add = 4 周期、f = 3.0 GHz**： – `sum_serial`: Span = 4×10^8 周期 = **133 ms** – `sum_serial_ilp`: Span = 10^8 周期 = **33 ms**（理想 4× 加速） – `sum_omp_ilp`: Span = 10^8/(4×16) ≈ 1.56×10^6 周期 ≈ **0.52 ms**（理想 256× 加速） – **但物理下限是**：800 MB ÷ 38 GB/s ≈ **21 ms**（用讲义给出的 Kaby Lake 38 GB/s）。即便用 20 GB/s 的保守带宽，也要 **40 ms**。

- **瓶颈诊断（本示例最重要的结论）**：当 T = 16 时，`Work/Span` 的并行度是 256，但**实测只会落在 21–40 ms 区间，而非 0.52 ms**。此时程序已经彻底**从延迟界迁移到带宽界**：再增加线程、再拆累加器、再宽 SIMD 都**毫无收益**（甚至因为超过内存控制器并发能力而变慢）。这就是"算术强度"概念要解决的问题——见第 4 节 Roofline 分析。
- **附带陷阱提示**：如果为了让"每线程 4 个累加器"更细而把切分改成 `a[i] += ...` 的**跨步访问（stride = T）**，虽然累加器线程私有、没有伪共享，但**访存变成非连续**，会破坏预取与 DRAM 行局部性，带宽可能掉一半以上。这是"为了并行而牺牲局部性"的典型错误。

3.3 示例三：分支预测——控制冒险的量化实验

讲义在"控制冒险/推测"一节点名了最坏情况：**基于随机数据的分支**。下面的实验用同一份代码、同一份数据，仅改变数据的**排列顺序**，就能观察到数量级的差异。

```cpp
// ------------------------------------------------------------
// branch_pred.cpp
// 编译 (release):
//   g++ -O2 -march=native branch_pred.cpp -o branch_pred
// 建议同时检查反汇编, 确认 if 没有被改写成无分支代码:
//   objdump -d branch_pred | grep -A30 '<_Z12sum_branchy'
// 运行:
//   ./branch_pred 40000000
// ------------------------------------------------------------
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <algorithm>
#include <chrono>
#include <cstdint>

// 数据相关的 if: 分支方向由数据决定
// 两个副作用 (s 与 hits 同时更新) 可降低编译器 if-conversion 成 cmov 的概率
static long long sum_branchy(const uint8_t* d, size_t n) {
    long long s = 0, hits = 0;
    for (size_t i = 0; i < n; ++i) {
        if (d[i] >= 128) {          // <-- 数据相关分支
            s += d[i];
            ++hits;
        }
    }
    return s + hits;
}

// 等价的无分支写法: 用条件表达式, 期望被编译成 cmov / 掩码运算
static long long sum_branchless(const uint8_t* d, size_t n) {
    long long s = 0;
    for (size_t i = 0; i < n; ++i) {
        const uint8_t v = d[i];
        s += (v >= 128) ? (long long)v : 0LL;
    }
    return s;
}

int main(int argc, char** argv) {
    const size_t n = (argc > 1) ? std::strtoull(argv[1], nullptr, 10) : 40000000ULL;
    std::vector<uint8_t> data(n);
    unsigned seed = 12345u;
    for (size_t i = 0; i < n; ++i) {                // 简单 xorshift 伪随机
        seed ^= seed << 13; seed ^= seed >> 17; seed ^= seed << 5;
        data[i] = (uint8_t)(seed >> 24);            // 均匀分布 0..255
    }

    auto time_it = [&](const char* tag, long long (*fn)(const uint8_t*, size_t)) {
        auto t0 = std::chrono::steady_clock::now();
        volatile long long r = fn(data.data(), n);
        auto t1 = std::chrono::steady_clock::now();
        double sec = std::chrono::duration<double>(t1 - t0).count();
        std::printf("%-26s : %8.2f ms  %6.3f ns/elem  %5.2f cyc/elem(@3.5GHz)  r=%lld\n",
                    tag, sec * 1e3, sec / (double)n * 1e9,
                    sec / (double)n * 3.5e9, (long long)r);
    };

    std::printf("--- 未排序 (分支方向随机, 预测器几乎无法学习) ---\n");
    time_it("branchy / unsorted",   sum_branchy);
    time_it("branchless / unsorted", sum_branchless);

    std::sort(data.begin(), data.end());
    std::printf("--- 已排序 (前一半全不跳, 后一半全跳, 预测几乎全中) ---\n");
    time_it("branchy / sorted",     sum_branchy);
    time_it("branchless / sorted",  sum_branchless);
    return 0;
}
```

【代码做什么？】 1. 用 xorshift 生成 4×10^7 个均匀分布于 0..255 的字节，保证 `d[i] >= 128` 大约一半为真、且**彼此独立**——这正是分支预测器最讨厌的模式（没有可学习的规律）。 2. 先对未排序数据跑 `sum_branchy` 与 `sum_branchless`，再 `std::sort` 后跑同样的两个函数（排序后分支行为变成"长段全不跳 + 长段全跳"，预测器能学得很好）。 3. 由于**结果与顺序无关**（加法可交换），排序不改变正确性，只改变**分支的动态行为**——这正是讲义所说"依据随机数据分支"的可控实验版本。

【并行机制与性能解说】

- **硬件上发生了什么**：`sum_branchy` 的循环体里有一条**数据相关分支**。当数据未排序时，处理器每两次迭代就会误预测一次；每次误预测要**冲刷流水线并从正确路径重取指**，代价约 15–20 个周期。排序之后，同样的分支几乎全中，误预测惩罚消失。
- **`branchless` 版本**：条件表达式被编译成 `cmov`（或 SIMD 掩码），**没有任何控制冒险**。它在未排序数据上应当明显快于 `branchy`，而在排序数据上两者接近——这个"排序后差距消失"的现象，是判断"瓶颈到底是分支还是数据依赖"的关键证据。
- **Work / Span / 并行度分析**（N 次迭代）：

版本	Work	Span	并行度
`sum_branchy`（预测正确）	N 次比较 + N 次加法（+ 若干）	N·L_add + N·L_branch_ok	≈ 1
`sum_branchy`（未排序）	同上	N·L_add + (N/2)·P_mispredict	≈ 1，**且 Span 被放大**
`sum_branchless`	N 次比较 + N 次条件选择 + N 次加法	N·L_add	≈ 1

取 N = 4×10^7、f = 3.5 GHz、`L_add = 1` 周期（累加是短延迟）、`P_mispredict ≈ 17` 周期、误预测率 = 50%： – 理想（无分支代价）：4×10^7 周期 ≈ **11 ms** – 加上误预测：$4\times10^7 + 2\times10^7\times17 = 3.8\times10^8$ 周期 ≈ **109 ms** – → 仅一个数据相关分支就能让这个内核慢 **~10 倍**，而它的**并行度始终是 1**（`s` 的依赖链），所以即使有 16 个核可以帮忙，也不会有任何加速——除非改变算法（例如分块并行 + 树形归约）。

- **瓶颈诊断**：这是**控制冒险主导**的场景。讲义给出的"分支预测准确率 >95%"是针对典型整数程序的平均值；对随机数据分支则接近 50%——预测器失效。此时 OoO 无能为力，因为**乱序执行也无法越过一条它不知道目的地何时才确定的分支**（分支解析前，后端根本不知道该执行哪些指令）。
- **工程结论**：对这类内核，正确做法是**去分支（branchless）或数据重排**，而不是加核、加 SIMD 宽度或调 OoO 参数。

---

4. 性能模型与复杂度分析

本节用讲义与补充讲义给出的数字，把第 2、3 节的定性结论算成具体数字。

4.1 机器参数假设（全部取自讲义/补充讲义）

参数	取值	来源
时钟频率 f	3.0–4.2 GHz	讲义中 Kaby Lake 系列示例
发射宽度 W	4 条标量指令/周期	补充讲义 Skylake/Kaby Lake 核心
SIMD 宽度	8 × 32-bit（AVX2），每核 3 个向量单元	补充讲义
FMA 延迟 L_FMA	4 周期	Skylake 级机器，与讲义"mla = 3 周期"同量级
内存延迟	L1 = 4c，L2 = 12c，L3 = 38c，DRAM ≈ 248c（@4 GHz）	补充讲义
DRAM 带宽	38 GB/s（讲义示例机器）；保守取 20 GB/s	补充讲义
分支误预测惩罚	15–20 周期	讲义"惩罚随流水线深度增长"
ROB 深度	数百条指令	讲义"high-performance OoO buffers hundreds of instructions"

4.2 算例 A：多项式循环的延迟界 vs 吞吐界（讲义的完整推理）

循环体 5 条指令：`ldr`(2c)、`cmp`(1c)、`mla`(3c)、`mul`(2c)、`bne`(1c)。

- **延迟界**：跨迭代关键路径是 `mla` 链（每次 `mla` 依赖上一次的 `r0`）：**3 周期/迭代**。此时 IPC = 5/3 ≈ **1.67**（完美流水线是 1）。
- **吞吐界**：每轮需要 1 条 `mla` + 1 条 `mul`，若只有一个乘法类端口，两条乘法的发射占用是 2 + 3：吞吐界 = (1 mla + 1 mul) / (2 + 3 周期) = **5 周期/迭代**，此时 IPC = 5/5 = **1.0**。
- **实际性能 ≈ max(3, 5) = 5 周期/迭代**：这就是讲义"结构冒险把一轮压成 5 个周期"的定量来源。
- 若把乘法端口增加到 2 个（或让 `mla` / `mul` 分配在不同端口）：吞吐界降到 ⩽3，程序重新变成**延迟受限**，回到 3 周期/迭代。

结论：同一段代码的瓶颈可以在"延迟界"和"吞吐界"之间切换，取决于发射宽度、端口数量、以及关键路径长度。判断当前处于哪一侧，是优化的第一步。

4.3 算例 B：为什么需要硬件多线程——Little 定律与在途窗口

Little 定律（Little's Law）：要在给定延迟下维持给定吞吐，必须有足够的"在途工作"：

```
在途工作数  =  延迟  ×  目标吞吐
```

- **只靠 OoO 单线程**：目标是每周期发射 4 条指令（IPC = 4），一次 DRAM 访问的延迟是 248 周期，那么需要

`在途指令数 = 248 周期 × 4 条/周期 = 992 条`

而讲义指出高性能 OoO 的缓冲"hundreds of instructions"（数百条）。**992 > 数百 ⇒ 单线程 OoO 无法隐藏一次完整的 DRAM 缺失**。这也解释了讲义给出的经验数："4 宽 × 20 级流水线 = 80 条在途"——离 992 差一个数量级。

- **用硬件多线程补足**：若每个线程的模式是"3 条算术 + 1 次 12 周期 load"，单线程的端口利用率只有

`利用率 = 3 / (3 + 12) = 20%`

需要的线程数 = ⌈(3 + 12) / 3⌉ = **5 个线程**才能到 100%。（补充讲义给了完全相同的示例与答案。）把模型推广到真实 DRAM 延迟 248 周期、每次访存携带约 15 个周期的算术工作：

`线程数 = ⌈(15 + 248) / 15⌉ = ⌈17.5⌉ = 18 个线程`

这就是 GPU 一个 SM 里塞 **64 个 warp 上下文**、CPU 上跑 SMT + 多核的定量理由。

- **关键性质（讲义强调）**：多线程**没有改变访存延迟**，它只是让延迟不再导致处理器停机。这就是"吞吐导向（throughput computing）"的根本权衡：**允许单个线程的完成时间变长，以换取系统整体吞吐提高**。

4.4 算例 C：算术强度与 Roofline——什么时候 ILP 完全无用

取一台"16 核 × 8 宽 SIMD × 2（FMA）× 3.0 GHz"的机器：

```
峰值算力 = 16 × 8 × 2 × 3.0e9 = 768 GFLOP/s（单精度）
```

取内存带宽 = 20 GB/s，则机器的**平衡点（ridge point）**：

```
ridge = 768 GFLOP/s ÷ 20 GB/s = 38.4 FLOP/byte
```

现在看一个典型内核：**遍历一个 256 MB 的 double 数组，每个元素做 1 次 FMA**。

- **访存量**：256 MB = 2.56×10^8 B；元素数 = 2.56×10^8 / 8 = **3.2×10^7 个 double**。
- **计算量**：3.2×10^7 × 2 FLOP（1 FMA = 2 FLOP）= **6.4×10^7 FLOP = 64 MFLOP**。
- **纯计算的理想时间**：64×10^6 / 768×10^9 = **83 μs**。
- **纯访存的下限时间**：2.56×10^8 B / 20×10^9 B/s = **12.8 ms**。
- **算术强度**：2 FLOP ÷ 8 B = **0.25 FLOP/byte**，远低于平衡点 38.4。
- **有效性能上限**：20 GB/s × 0.25 FLOP/B = **5 GFLOP/s**，仅峰值的 **0.65%**。

结论：这个内核比"纯计算时间"慢 **154 倍**，而且**再多核、再宽的 SIMD、再深的 OoO 窗口都无济于事**——瓶颈是 DRAM 带宽。Roofline 的判据非常直接：

```
算术强度 < ridge 点   =>  带宽受限（bandwidth-bound），优化目标是减少访存/提高复用
算术强度 > ridge 点   =>  计算受限（compute-bound），此时才轮到 ILP/SIMD/多核发力
```

把同样的分析套回第 3 节的 `omp_sum`：其算术强度是 1 FLOP / 8 B = 0.125 FLOP/byte，所以无论用多少线程，实测都会停在"800 MB ÷ 带宽"那个数量级（约 21–40 ms），而不会接近"0.52 ms"的理想延迟界。

4.5 算例 D：Amdahl 定律与"每核效率"的复合效应

设程序中可并行部分占 p = 0.95，串行部分 s = 0.05，用 T 个核：

```
Speedup(T) = 1 / (s + p/T)

T = 16 :  1 / (0.05 + 0.95/16) = 1 / 0.1094 = 9.15x
T = 64 :  1 / (0.05 + 0.95/64) = 1 / 0.0648 = 15.4x
T ->  ∞:  1 / 0.05             = 20x      <-- Amdahl 上限
```

再把"每核效率"叠进去：如果每核因为**延迟界**只能达到 IPC = 1.67，而机器峰值是 4 宽（IPC = 4），则每核效率只有 **42%**。于是 T = 16 时的**真实加速比 ≈ 9.15 × 0.42 ≈ 3.8×**。

这一条把本讲与课程整体串起来了：并行加速比不仅是"核数"和"串行比例"的函数，还受**每个核内部的 ILP 效率**（延迟界/吞吐界）制约。OoO 好，是为了让这个 0.42 尽量接近 1；多核好，是为了让 T 尽量大。两者不可偏废。

4.6 算例 E：超标量调度复杂度与流水线深度的双重上限

讲义给出的两个硬件开销公式：

```
调度复杂度       = O(W^2)              W = 发射宽度
控制冒险惩罚     = O(N_pipeline)        每次误预测约丢弃 N 级在途指令
在途指令数       = W × N_pipeline
```

- W = 4、N = 20 ⇒ 在途 80 条指令，调度要比较 C(4,2) = 6 对寄存器组。
- W = 8、N = 20 ⇒ 在途 160 条，比较 28 对——复杂度增长 4.7 倍，而讲义明确说"即便完美调度，>8 宽也没有帮助"。
- 若平均每 20 条指令就遇到一次误预测（即每个分支都猜错，随机数据分支的典型情形），每次损失 20 个周期：理想情况 4 宽机器发射 20 条指令只需 5 周期，实际需要 5 + 20 = 25 周期 ⇒ **有效 IPC = 20/25 = 0.8**，W = 4 的收益被完全吃掉（甚至不如单发射）。

结论：ILP 的三个上限——**程序本身 ILP 有限（~8 宽）**、**流水线深度有限（N≈15）**、**调度复杂度 O(W²)**——共同把单核性能封了顶。这正是讲义结尾"Limitations of ILP → Multicore"的完整论证。

---

5. 关键要点

1. **流水线只提高吞吐，不降低延迟**；N 级流水线最多带来 N× 吞吐，但受数据/控制/结构三类冒险制约，实际上在 N≈15 附近触顶。旁路转发能消除大多数（不是全部）数据冒险停顿，而它在深流水线中本身成本高昂；分支冲刷的代价随深度线性增长。
2. **乱序执行 = "按数据流顺序执行 + 保持顺序执行的幻觉"**。指令只按程序序进入和离开指令缓冲/ROB，中间完全乱序；前端与提交端被超额配置，因此程序员通常只需关注执行阶段，唯一例外是本质上不可预测的控制流。
3. **性能分析用两个界，取慢的那个**：**延迟界 = 跨迭代关键路径**（拆依赖链才能改善），**吞吐界 = 操作数 / 执行端口发射率**（加端口或换更宽的 SIMD 才能改善）。OoO 让这套分析比 in-order 机器简单得多，也让性能在不同微架构间的可移植性更好。
4. **寄存器重命名消灭伪依赖（WAR/WAW），只保留真依赖（RAW）**；数据流视角是理解乱序、推测与编译器重排的统一语言。真依赖构成的关键路径无法被任何硬件技巧绕过，只能由程序员/编译器改写算法来缩短。
5. **ILP 撞墙的直接后果是多核**：程序 ILP 有限（>8 宽无益）、流水线不能更深、频率受功耗限制、OoO 调度是 O(W²)。工业界的答案是"简化微架构、增加核数、并用硬件多线程（SMT）与 SIMD 填补每核的利用率空洞"，代价是**并行软件更难写**。当代码进入带宽界（算术强度低于 Roofline 平衡点）时，上述一切硬件技巧都失效，唯一的出路是提高数据复用。

---

6. 常见陷阱与注意事项

- **把"流水线"误解为"降低延迟"**：`N` 级流水线不改变单条指令的端到端延迟（仍是 N·t），只提高稳态吞吐。若程序只有极少量指令（或每次都要冲刷），流水线反而带来更差的启动/排空开销。同理，**"IPC = 4" 的机器不等于单线程快 4 倍**，它取决于程序里有没有 4 条独立指令可发射。
- **以为"乱序执行 = 可以放心写串行代码"**：OoO 只能利用**真实存在**的 ILP。第 3 节示例一证明，一条 8192 长的严格依赖链在任何 OoO 宽度下都只能得到约 1 的 IPC，把关键路径砍半（双链）才能拿到 2× 加速。OoO 降低了对"指令调度技巧"的依赖，但**不解决算法级的依赖结构问题**。
- **用"平均分支预测率 >95%"推断自己的程序**：数据相关分支（随机、哈希、压缩、稀疏矩阵遍历、图算法中的条件插入）会退化到约 50% 命中率，**每次误预测丢弃 15–20 个周期**，足以造成 10 倍量级的差距。此类内核应改写为**无分支（branchless / cmov / 掩码）或数据重排**，而不是指望硬件或加核。

- **忽视延迟界与吞吐界的切换，盲目优化**：给一个**延迟受限**的内核加核、加 SIMD 宽度、堆更多线程，收益为零甚至为负；对一个**吞吐受限**的内核去"拆分依赖链"同样白费。正确顺序永远是：先判断处在哪一侧（第 4.2 节的两条公式），再选择对应的优化手段。
- **只算 FLOPs 而不算字节数（忽略带宽）**：第 4 节算例 C 中，算术强度 0.25 FLOP/byte 的内核只能发挥 **0.65%** 的峰值算力。**报告"我的程序达到了 X GFLOPS"之前，必须同时报告"它搬了多少字节、实际带宽是多少"**，否则该数字毫无意义。同理，"我的并行程序有 16 个线程"不等于"16 倍加速"——先看它是不是带宽受限。
- **为并行而破坏访存局部性 / 引入伪共享或数据竞争**：为了让累加器线程私有而改用 `stride = 线程数` 的跨步访问，会摧毁预取与 DRAM 行局部性，带宽可能腰斩；反过来，为图省事让多线程写**同一个缓存行里的不同变量**（例如 `struct { long cnt[N]; }` 相邻元素）会触发**伪共享（false sharing）**，每一次写都让别的核的缓存行失效。多线程写共享累加器时还应避免用 `#pragma omp atomic`/互斥锁做**热路径**累加（串行化 + 缓存行乒乓），应使用**每线程私有副本 + 末尾归约**（如 `reduction(+:)`）。
- **错误理解内存序与编译器优化边界**：OoO 硬件重排的是**单线程内**的指令，多线程之间的可见性由**内存序（memory ordering）**与同步原语决定——写一个 `volatile` 变量**不构成同步**，也不能替代 acquire/release 或锁。另一方面，`-O3 -ffast-math` 会让编译器**重结合浮点运算**（可能自动把你精心构造的串行链变成并行链，或反之改变结果），做性能实验或要求逐位可复现时必须显式关闭它。

---

7. 思考题（带答案）

思考题 1：判断延迟界与吞吐界

某机器的循环体如下（每轮 4 条指令），执行端口配置为：**1 个整数端口**（`cmp`、`bne` 各占 1 周期）、**1 个乘法端口**（`mul` 占 2 周期）、**1 个 FMA 端口**（`fma` 占 1 周期发射、4 周期延迟）：

```
loop:
  fma   f0, f2, f4, f0     // acc = acc + w * x   (延迟 4 周期, 依赖上一轮的 f0)
  addi  r3, r3, 4          // 指针推进
  cmp   r3, r1             // 边界比较
  bne   loop
```

问：(a) 延迟界是多少周期/迭代？(b) 吞吐界是多少周期/迭代？(c) 实际性能约为多少，IPC 是多少？(d) 如果程序需要跑 10^6 次迭代，在 3.0 GHz 下大约耗时多久？

【答案】

(a) **延迟界**：跨迭代的真依赖只有 `fma` 链（`f0` 依赖上一轮的 `f0`），`addi` / `cmp` / `bne` 的链长都是 1 周期且远短于 FMA 链。所以关键路径 = **4 周期/迭代**。

(b) **吞吐界**：把每类操作的需求量除以对应端口的服务能力： – FMA 端口：1 条 `fma` / 1 周期 = **1 周期** – 乘法端口：本轮没有 `mul` = **0 周期** – 整数端口：`addi` + `cmp` + `bne` = 3 条整数操作，端口 1 条/周期 = **3 周期**

吞吐界 = max(1, 0, 3) = **3 周期/迭代**。

(c) **实际性能 ≈ max(延迟界, 吞吐界) = max(4, 3) = 4 周期/迭代**，即 **延迟受限**。IPC = 4 条指令 / 4 周期 = **1.0**。

(d) 10^6 迭代 × 4 周期 = 4×10^6 周期；在 3.0 GHz 下 = 4×10^6 / 3×10^9 s ≈ **1.33 ms**。

> **追问**：如果给这台机器再加一个整数端口（整数吞吐变成 2 条/周期），会变快吗？**不会**——吞吐界变成 max(1, 0, 1.5) = 1.5 周期，仍小于延迟界 4 周期，程序依旧是延迟受限，性能不变。只有**缩短 FMA 依赖链**（例如用 2 个累加器交替累加、循环结束后合并）才能提速：2 个累加器把 FMA 链变成 2 周期/迭代，此时瓶颈切换到吞吐界的 3 周期/迭代，性能从 4 → 3 周期/迭代，提升 1.33×；再用 4 个累加器则 FMA 链 1 周期/迭代 < 3 周期，**彻底变成吞吐受限**，无论如何加速比都止步于 4/3 ≈ 1.33×。这就是"延迟界与吞吐界在优化过程中会互相转换"的典型例证。

思考题 2：为什么单核 OoO 掩盖不了 DRAM 延迟？

某处理器：发射宽度 W = 4 条指令/周期，ROB 可容纳 256 条指令，一次 DRAM 访问延迟 248 周期（@4 GHz），程序在两次访存之间有约 20 条相互独立的指令。

问：(a) 用 Little 定律计算，要维持 IPC = 4 需要多少条指令在途？(b) 该机器能提供多少？(c) 如果你是架构师，有哪三种手段补上这个缺口，各自代价是什么？

【答案】

(a) Little 定律：**在途指令数 = 延迟 × 目标吞吐 = 248 周期 × 4 条/周期 = 992 条指令**。

(b) 该机器只有 **256 条** ROB 条目，仅能支撑 992 条的约 **26%**。等效地，可持续的吞吐 ≈ 256 / 248 ≈ **1.03 条/周期**，即 IPC 掉到约 **1**——发射宽度 4 的硬件在这里只发挥了 1/4。

(c) 三种手段： 1. **硬件多线程（SMT / interleaved multithreading）**：在同一核上放 2–8 个执行上下文，A 线程等 DRAM 时发射 B 线程的指令。缺口 992 条需要约 992/256 ≈ 4 个线程（或按"每次访存携带 20 条独立指令"的模型：⌈(20+248)/20⌉ = **14 个线程**才达 100% 利用率——这正是 GPU 一个 SM 放 64 个 warp 的原因）。**代价**：每个上下文的寄存器/存储开销，芯片面积与功耗；单线程的完成时间变长（吞吐导向的权衡）。 2. **增加 ROB 深度 / 重命名寄存器数**：把窗口从 256 扩到 1024。**代价**：面积与功耗按超线性增长，且调度复杂度与唤醒逻辑（wakeup/select）本身成为关键路径，频率会被拖低——讲义明确指出"动态调度开销显著"。 3. **提高缓存命中率 / 软件预取**：把 248 周期的 DRAM 延迟换成 L2 的 12 周期，缺口立刻变成 12×4 = 48 条，256 条的 ROB 绰绰有余。**代价**：需要程序员或编译器做**分块（tiling）/ 预取 / 提高数据复用**，改动算法结构；且缓存不命中的最坏情形仍存在。

> **补充结论**：手段 3 是**唯一不依赖硬件的**，也是本课程反复强调的"利用局部性"。硬件手段 1 和 2 都直接消耗面积/功耗，而片上的功耗预算有限——这正是"ILP 撞墙、转向多核"的经济学根源。

思考题 3：一个 ILP 优化为什么在双核上几乎没加速?

程序 A：对一个 512 MB 的数组做 `out[i] = f(in[i])`，`f` 是 20 次浮点运算，`in` 和 `out` 都是 `double`。 某同学做了两项优化：(i) 把 `f` 内部原本串行的表达式重排，使关键路径从 20 次 FMA 降到 5 次；(ii) 用 OpenMP 把数组分给 2 个线程。 机器：2 核 × 8 宽 SIMD × 2（FMA）× 3.0 GHz（峰值 96 GFLOP/s 单精度；double 视为一半 = 48 GFLOP/s），DRAM 带宽 20 GB/s。

问：(a) 计算这个内核的算术强度。(b) 分别计算"带宽下限时间"与"双核计算下限时间"（元素数按 512 MB / 8 B = 6.4×10^7）。(c) 预测优化 (i) 和 (ii) 各自的实际收益，并解释为什么"关键路径砍到 1/4"几乎没有帮助。

【答案】

(a) **算术强度**：每个元素 20 FLOP，访存 8 B 读 + 8 B 写 = 16 B。

```
AI = 20 FLOP / 16 B = 1.25 FLOP/byte
```

机器平衡点 = 48 GFLOP/s ÷ 20 GB/s = **2.4 FLOP/byte**。**1.25 < 2.4 ⇒ 带宽受限**。

(b) 元素数 N = 512 MB / 8 B = **6.4×10^7**。

```
带宽下限时间 = 总字节 / 带宽 = (6.4e7 × 16 B) / 20e9 B/s = 1.024e9 / 20e9 = 51.2 ms
计算下限时间(2 核)  = 总 FLOP / 峰值 = (6.4e7 × 20) / 48e9 = 1.28e9 / 48e9 = 26.7 ms
计算下限时间(1 核)  = 53.3 ms
```

带宽下限 51.2 ms **大于**双核计算下限 26.7 ms ⇒ 用 2 个核时，程序**卡在 51.2 ms 附近**。

(c) 预测： – **优化 (ii)（双核）**：串行版受限于 1 核计算下限 53.3 ms（略大于带宽下限 51.2 ms），所以理论上限约 53.3/51.2 ≈ 1.04×……但由于单核实际很难达到理论峰值（延迟界、发射宽度、SIMD 利用率），串行版实测大概率在 70–90 ms，双核能压到约 51 ms 的带宽墙，**实测约 1.4–1.7×**，而非 2×。 – **优化 (i)（关键路径从 20 降到 5）**：**几乎没有帮助**。原因是该内核的瓶颈是**带宽**（AI = 1.25 < 2.4），算术运算早就"藏在"访存后面了；而且 20 次 FMA 分布在 8 宽 SIMD 上只需要约 2.5 条向量指令，关键路径缩短带来的 ILP 收益根本无法越过 51.2 ms 的带宽地板。

> **诊断方法论**：拿到任何内核，先算 **AI** 与 **ridge point**： – AI < ridge ⇒ 先做**带宽优化**：数据复用（分块/融合循环）、减小数据宽度（float 代替 double、必要时量化）、避免多余的写–读往返（例如 `a = a + b` 就地运算，而不是 `c = a + b` 再拷贝）、提高访存连续性以吃满 DRAM 行缓冲。 – AI > ridge ⇒ 才轮到**计算优化**：缩短关键路径（抬升延迟界）、增加并行度（抬升吞吐界）、扩大 SIMD 宽度。
>
> 本讲（ILP/OoO）提供的全部是第二类工具；**用错类别的优化，收益为零**，这正是第 6 节"常见陷阱"中最贵的一条。

Lecture 3: A Modern Multi-Core Processor

1. 章节标题与概述

Lecture 3: A Modern Multi-Core Processor

- **本讲核心问题**：现代处理器**靠什么获得高吞吐（throughput）**？讲义把它归结为"四个核心概念"——其中两个关于**并行执行**（多核 multi-core、SIMD 单指令多数据），两个关于**访问内存的挑战**（内存延迟 memory latency、内存带宽 memory bandwidth）。更具体地说：(1) 单核性能为什么不再增长，而芯片却继续把晶体管花在"更多核、更宽的 SIMD、更多的硬件线程"这三件事上？(2) 一台"16 核 × 每核 8 宽 SIMD ×

每核 4 个硬件线程"的机器，需要程序暴露多少独立工作才跑得满？(3) 为什么绝大多数并行程序最终不是被算力卡住，而是被**内存带宽**卡住——"带宽受限（bandwidth limited）"意味着**再多延迟隐藏手段也救不了**？

- **涉及的主要硬件/软件机制**：
- **硬件侧**：取指/译码（fetch/decode）与执行上下文（execution context，即 PC + 寄存器组）、超标量（superscalar）与乱序执行（out-of-order）、指令级并行（ILP）、多核（multi-core）、SIMD 功能单元（一条指令广播给多个 ALU）、向量寄存器（SSE 128 位 / AVX 256 位 / AVX-512 512 位）、掩码（mask）与发散执行（divergent execution）、缓存层次（L1/L2/L3 + DRAM）、硬件预取（prefetch）、硬件多线程（交织式 interleaved / 同时多线程 SMT）、GPU 的 SM（streaming multiprocessor）与 warp。
- **软件侧**：把串行循环改写成并行循环的三种表达方式（pthreads 显式线程、数据并行 `forall` 声明、SIMD intrinsics 显式向量化）、SPMD（single program multiple data）/ gang 抽象、工作划分策略（静态划分 vs 动态划分）、以及"写代码时必须同时知道硬件怎么执行它"这一心智模型。
- **在并行计算知识体系中的角色**：本讲是**从"为什么并行"（Lecture 1）与"指令级并行/流水线"（Lecture 2）过渡到"程序怎么写"（Lecture 4 起的编程模型）的关键桥梁**。它建立了后续全课程反复使用的三块地基：(1) **四种并行执行形式的分类学**（超标量 / SIMD / 多核 / 硬件多线程），之后所有并行编程模型（ISPC、CUDA、OpenMP、Cilk）都可视为"用某种方式把这四种硬件并行暴露出来"；(2) **延迟与带宽的区分**——延迟靠"隐藏"（多线程、预取），带宽只能靠"少访问"（数据复用、算术强度），这一区分在 Lecture 7/8 的性能优化、Lecture 20 的异构计算中反复出现；(3) **算术强度（arithmetic intensity）与 Roofline 思想**，它在 Lecture 8（性能优化）中被正式化成定量工具。此外，本讲还给出了三个"通用吞吐计算原理"（并行工作要够多、同组工作要走同一条指令序列、独立工作要多于 ALU 数），它们等价于 GPU/CUDA 编程的核心约束。
- **配套材料**：
- `extracted/03_basicarch.txt` —— CMU 15-418/618 **Fall 2026 Lecture 3 讲义**（共 81 页）逐页抽取文本；对应 PDF 为 `lectures/03_basicarch.pdf`，位于 https://www.cs.cmu.edu/~418/lectures/ 之下，属**已公开**（可直接下载）。首页标注 "CMU 15-418/15-618, Fall 2026"，副标题为 "(Forms of parallelism + understanding latency and bandwidth)"。Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）中本讲日期为 **Aug 28**。讲义内部结构清楚：slide 3–41 是 Part 1「并行执行」，slide 42–68 是 Part 2「访问内存」，slide 69–81 是术语清单与四组"把所有机制拼在一起"的复习图。
- `cs149_supp/multicore1.txt` —— Stanford **CS149 Lecture 2 "A Modern Multi-Core Processor (Part I)"**（108 页）逐页抽取文本，属**已公开**支持材料。它把 CMU 版一带而过的**缓存与延迟细节**补全了：缓存用 LRU 替换的逐次访问演算、空间/时间局部性（spatial / temporal locality）、Kaby Lake 各级访问延迟（L1 = 4、L2 = 12、L3 = 38、DRAM ≈ 248 个 4 GHz 周期）、**数据移动的能耗**（整数运算 ~1 pJ、浮点运算 ~20 pJ、从片上 SRAM 读 64 位 ~26 pJ、从 LPDDR 读 64 位 ~1200 pJ；10 GB/s ≈ 1.6 W），以及"用 3 条算术 + 12 周期访存需要几个线程才能跑满"的定量算例（答案：5 个）。
- `cs149_supp/multicore2.txt` —— Stanford **CS149 Lecture 3 "Multi-Core Architecture, Part II (latency/bandwidth issues) + Parallel Programming Abstractions"**（56 页）逐页抽取文本，属**已公开**。本讲 2.9/2.10/2.12 节的类比与定量模型主要来自这里：高速公路的"车速 vs 车道数 vs 车距"、洗衣房的"延迟 vs 吞吐"、两根串联水管的流量上界、**带宽受限的稳态时间线**（每时钟 8 字节、64 字节的 load 要 8 个时钟、每次 load 只喂 3 条数学指令 → ALU 利用率 37.5%）、4 级指令流水线 IF/D/EX/WB，以及 **abstraction vs implementation** 与 **ISPC/SPMD** 的介绍。
- `cs149_supp/dataparallel.txt` —— Stanford **CS149 Lecture 8 "Data-Parallel Thinking"**（51 页）逐页抽取文本，属**已公开**。第 4 节的 work/span 记号（例如并行 scan：Work = O(N log N)、Span = O(log N)）与"工作高效但 SIMD 利用率低"的权衡来自这里，可视为本讲"并行度要够多"这一结论的延伸与形式化。
- **未公开部分**：Fall 2026 的录像（Panopto/YouTube）在日程表中被注释隐藏，属**未发布**；Ed 讨论区、Autolab、Canvas 均需登录。Performance Analysis/Profiling、Transactional Memory、AI in System Design 等讲座的 Fall 2026 讲义尚未在公开目录发布；历史学期的对应 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，需要 CMU 登录，属**未公开**。讲义首页有时写有历史学期（如 Fall 2025）字样，这是因为讲义沿用，属正常现象。

---

2. 核心概念与硬件/软件架构图解

2.1 全讲地图：四个核心概念，三种并行执行形式

- **定义与目的**：讲义开篇即声明"今天讲四个核心概念，两个关于并行执行、两个关于访问内存"，并说明动机：理解这些架构基础，才能**优化并行程序性能**，也才能**直觉判断哪类负载会从快速并行机器上受益**。四个概念可以列成一个 2×2 的坐标：

```
  Lecture 3: four key concepts

┌─────────────────┬──────────────────────────┬──────────────────────────┐
│  HARDWARE gives │PARALLEL EXECUTION        │ACCESSING MEMORY          │
├─────────────────┼──────────────────────────┼──────────────────────────┤
│                 │(1) multi-core            │(3) HW multi-threading    │
│                 │(2) SIMD wide vector ALU  │    + prefetch            │
├─────────────────┼──────────────────────────┼──────────────────────────┤
│  SOFTWARE must  │enough independent work;  │arithmetic intensity      │
│                 │coherent control flow     │high enough, else you     │
│                 │for SIMD groups           │are BANDWIDTH BOUND       │
└─────────────────┴──────────────────────────┴──────────────────────────┘

  中文对照：硬件给的是 (1)多核 (2)SIMD (3)硬件多线程 + 预取；
            软件必须提供 "足够多、够齐、有余量" 的独立工作与足够高的算术强度。
```

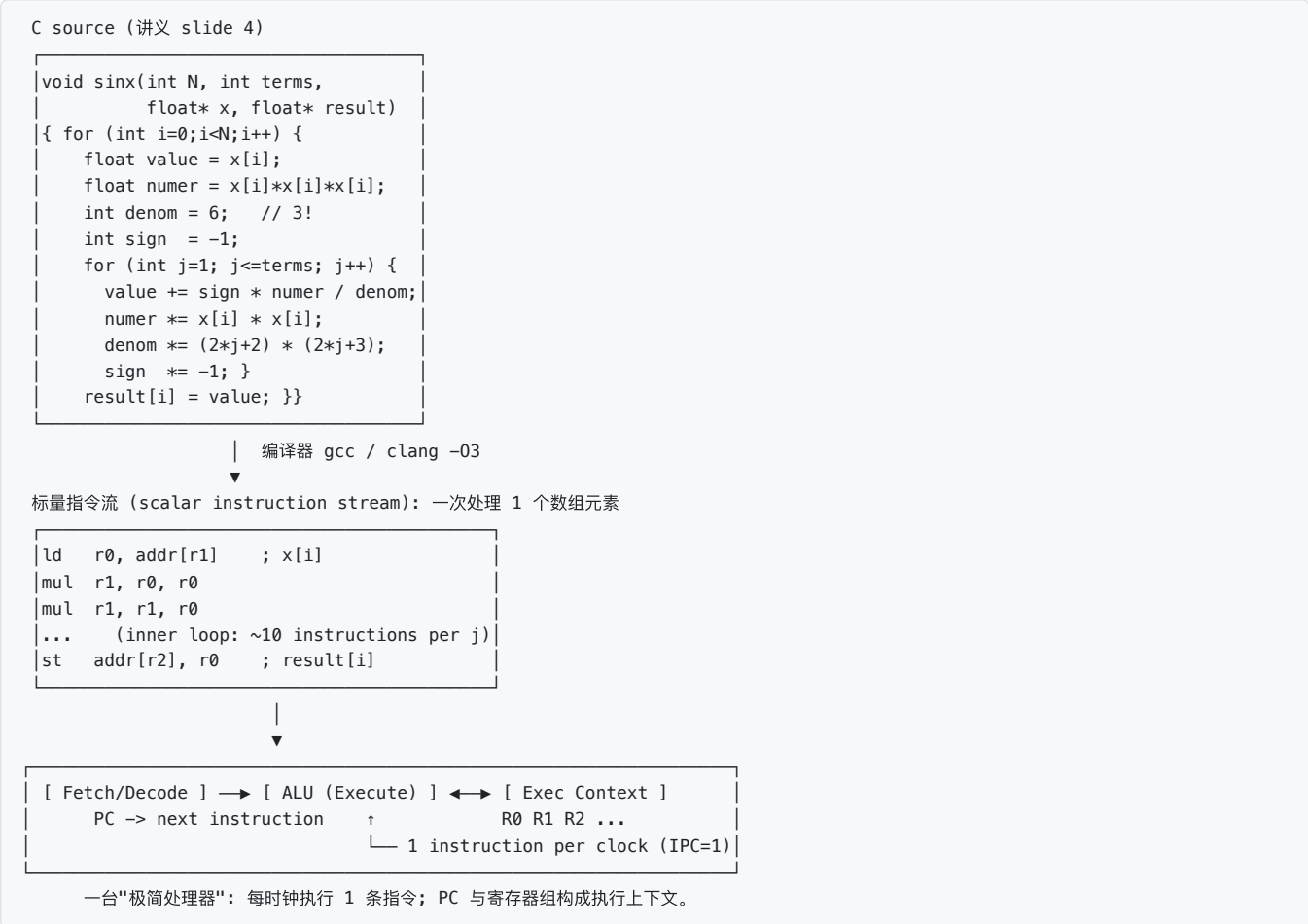
- **直观解释（"它是什么？"）**：把现代多核处理器想象成一座**中央厨房**。过去（pre multi-core era）的做法是雇一位米其林大厨，给他配最好的刀、最好的灶、最大的操作台（大缓存、乱序逻辑、分支预测器），让他一个人做菜越来越快。2004 年前后这条路的收益到顶了，于是厨房改策略：**雇 16 个普通厨师（多核）**，每人配几把一样的刀、同时切好几样菜（SIMD），而且每人在等汤炖好（访存延迟）的时候就去处理别的菜（硬件多线程）。厨房变了，但**菜谱（程序）如果只写给一个人看，新厨房不会更快**——这正是讲义用 `sinx` 这个例子要说明的第一件事。
- **架构/机制图解**：下面是讲义在最后"复习"部分（slide 72–80）一路拼出来的机器演进链条，从这里可以一眼看出四种并行执行形式各自加在哪一层。

```
(1) very simple processor                                  IPC = 1

   ┌──────────────────────────────────────────┐
   │ [Fetch/Decode] ──► [ ALU ] ◄──► [Exec Context]│
   └──────────────────────────────────────────┘
                       │                      ▲
                       ▼                      │ load/store
            ┌──────────────────────┐
            │           Memory (DRAM)│
            └──────────────────────┘


(2) superscalar (ILP)                                      IPC <= 2

   ┌──────────────────────────────────────┐
   │ [Fetch/Decode 1] ──► [Exec 1] ◄─┐    │
   │ [Fetch/Decode 2] ──► [Exec 2] ◄─┤    │
   │        (out-of-order control)   │    │
   │                        [Exec Context]│
   └──────────────────────────────────────┘
    * 讲义批注: "No ILP exists in this region of the program"
      (sinx 的这一段代码里没有可并行的独立指令)

(3) multi-core (Idea #1: 4 cores)                          TLP

    ┌──────────┬──────────┬──────────┬──────────┐
    │ F/D  + ALU │ F/D  + ALU │ F/D  + ALU │ F/D  + ALU │
    │  Exec Ctx  │  Exec Ctx  │  Exec Ctx  │  Exec Ctx  │
    └──────────┴──────────┴──────────┴──────────┘
     4 条互相独立的指令流 (4 simultaneous instruction streams)
     单核变简单 => 每条流变慢 (讲义取 0.75x); 2 x 0.75 = 1.5 是潜在收益

(4) 4 核 x 8-wide SIMD (Idea #2)                           SIMD

   ┌──────────────────┐
   │ [Fetch/Decode]   │
   │ [ALU0 ALU1 ... ALU7]│
   │ [Exec Context]   │
   └──────────────────┘
     x4 份  <- 一条控制流喂 8 个 ALU, 控制开销被 8 个 ALU 摊薄

(5) 4 核 x 8-wide SIMD x 2-way MT (Idea #3)      LATENCY HIDING

   ┌──────────────────┐
   │ [F/D] [F/D]      │
   │ [SIMD Exec 2] [Exec 1]│
   │ [Exec Ctx] [Exec Ctx] │
   └──────────────────┘
     x4 份  <- 每个核 2 个执行上下文, 遇到 stall 就切到另一条线程
     最大 8 条并发指令流; 每时钟最多 2 条指令(其中 1 条是 8-wide SIMD)
```

- **关键操作与性能特征**：这五种结构是**叠加**而不是替代关系。一台 2025 年的桌面 CPU（例如讲义提到的 Intel Arrow Lake，2025 年，20 个 CPU 核 = 8 个性能核 + 12 个能效核）同时具备多核、SIMD、超标量、多线程与深缓存层次；一台 GPU（例如讲义提到的 NVIDIA B200，2025 年，144 个 SM、18944 个 "CUDA core"）则把后三种推到极端。判断一段代码"能不能跑满机器"要同时回答三个问题：**够不够多（多核）、齐不齐（SIMD 一致性）、有没有余量（多线程隐藏延迟）**。

2.2 示例程序 `sinx`：从 C 代码到指令流

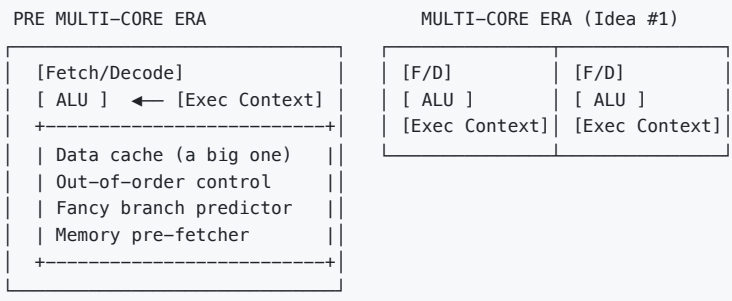
- **定义与目的**：全讲用一个程序贯穿：**对长度为 N 的浮点数组逐元素计算 sin(x)（用 Taylor 展开 `sin(x) = x − x³/3! + x⁵/5! − x⁷/7! ...`）**。选它是因为它同时具备三个特征：(a) 循环体内部是一条"看起来没什么可并行"的连乘/累加依赖链；(b) 外层 `for i` 的迭代彼此完全独立；(c) 每个元素只读 4 字节、写 4 字节，算术强度极低——因此既能用来讲并行执行，又能用来讲带宽。
- **直观解释（"它是什么？"）**：把它想成**一条流水线上的 100 万个零件**。每个零件都要经过同样的 5 道工序（`terms=5` 的 5 次迭代），每个零件的 5 道工序**必须按顺序做**（`value`、`numer`、`denom` 都是递推），但**零件与零件之间毫无关系**。这恰好是 SIMD 最适合的形状：同一条指令序列作用在 8 个不同零件上。
- **架构/机制图解**：从源码到"处理器真正看到的东西"要经过编译器这一层翻译，理解这一层是理解后面所有优化的前提。



- **关键操作与性能特征**：这台"教学用极简处理器"的规格是**每时钟执行一条指令（1 IPC）**。它的性能模型就是 `T = 指令数 / (IPC × 时钟频率)`；编译器生成的指令数、以及处理器每时钟能发射几条指令（IPC），是本讲前半部分反复出现的两个杠杆。注意讲义特别标注的一点：`sinx` 的这段代码里**没有 ILP 可用**（内层循环的 `numer`、`denom`、`sign`、`value` 全是依赖链），所以这台机器即使升级到两路超标量也拿不到 2 倍。

2.3 并行执行形式之一：超标量（superscalar）与 ILP

- **定义与目的**：**超标量**指处理器在同一时钟周期内取指、译码并发射**多条来自同一条指令流**的独立指令到多个执行单元。它的目的是利用**指令级并行（ILP, instruction-level parallelism）**——同一线程内部本来就存在、但只有硬件才能动态发现的并行性。
- **直观解释（"它是什么？"）**：这是**一位厨师同时照看两口锅**。菜谱（指令流）只有一份，但"等水开"和"切葱花"这两步互不依赖，可以同时做。关键限制是：**菜谱里必须真有互不依赖的步骤**，否则厨师再能干也只能干等——`sinx` 的内层循环恰好就是这种"每步都要用上一步结果"的菜谱，所以超标量在这里无能为力（讲义原文标注 "No ILP exists in this region of the program"）。
- **架构/机制图解**：讲义把预多核时代与多核时代的芯片晶体管预算画成了两幅对照图，这是理解"为什么转向多核"的核心。



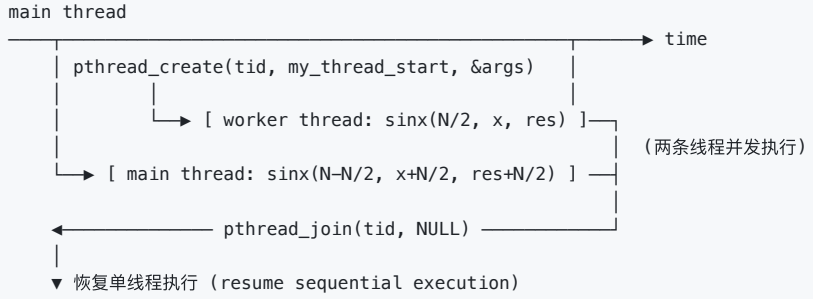
讲义原文: "Idea #1: Use increasing transistor count to add more cores to the processor, rather than use transistors to increase the sophistication of processor logic that accelerates a single instruction stream."

- **关键操作与性能特征**：这里有一个容易被忽略的定量论证，讲义把它讲得很直白：**简单核比复杂核慢，但多核的总和更大**。假设立方体（"fancy core"）跑一条指令流的速度指数为 1.0，而简单核只有 0.75；那么两核系统的**潜在**吞吐是 `2 × 0.75 = 1.5`——**但如果程序里没有并行（例如原封不动编译的 `sinx`），性能就变成 0.75，也就是"加速比 0.75x"，即更慢了**。这个"0.75 倍"的设定是讲义用来制造反差的虚构数值，但它揭示的规律是真的：**体系结构转向吞吐（throughput-oriented）之后，"不提并行"的程序会自动变慢**。

2.4 想法 #1：多核（Multi-core）

- **定义与目的**：**多核处理器**在单个芯片上放多个完整的处理核，每个核有自己的取指/译码单元与执行上下文，能**同时执行完全不同的指令流**。它解决的问题是：当单线程性能无法继续靠"更复杂的核 + 更高主频"提升时，如何继续提高**芯片级吞吐**。软件侧对应的是**线程级并行（TLP）**，由软件决定何时创建线程。
- **直观解释（"它是什么？"）**：多核就是**把一个大厨换成 16 个小厨**，每人一口锅、一本自己的菜谱。好处是各自做各自的菜（不同指令流），坏处是**如果你只有一本菜谱（串行程序），另外 15 个人只能看着**。把工作分给他们是**程序员的责任**，硬件不会自动帮你切分一个串行循环。
- **架构/机制图解**：讲义给出的最小多核改写是把循环切成两半，一半给新线程、一半留给主线程——这是 **fork-join（分叉-汇合）**执行模型最朴素的形态。

软件执行模型: fork-join (pthreads 版本, 讲义 slide 16 / slide 74)



工作切分: args.N = N/2; args.x = x; args.result = result;
worker 处理 [0, N/2) ; main 处理 [N/2, N) <- 两个半区互不重叠
两条线程访问同一个大数组的不同片段, 因此不需要任何同步原语。

- **关键操作与性能特征**：`pthread_create` / `pthread_join` 是**昂贵的**（微秒级系统调用 + 线程栈分配），所以现实中不会对 N 个元素创建 N 个线程，而是"创建 ≈ 核数个线程、每个线程处理一大块"（示例二）。多核的性能上限由 **Amdahl 定律**决定（见第 4 节）：`S(P) = 1 / (f + (1-f)/P)`，其中 `f` 是无法并行的比例。还有一个常见误解：**多核之间的指令流可以完全不同**，因此多核**不需要**指令流一致性（coherence）——这一点与 SIMD 形成鲜明对比。

2.5 想法 #2：SIMD（单指令多数据）

- **定义与目的**：SIMD（Single Instruction, Multiple Data，**单指令多数据**） 指用**一条指令**驱动**多个 ALU** 同时处理多个数据元素。讲义给出的动因是 **"Idea #2: Amortize cost/complexity of managing an instruction stream across many ALUs"**——把"取指、译码、控制、调度"这套复杂逻辑的开销**摊薄**到很多个 ALU 上，从而以极小的额外成本获得数倍算力。它解决的问题是：算力提升不再依赖"把单条指令流做得更快"，而是"用一条流喂饱更多执行单元"。

- **直观解释（"它是什么？"）**：把标量处理器想成**一位老师只教一个学生**，SIMD 就是**一位老师对着 8 个学生讲同一句话，8 个学生同时做同一道题**。老师的时间（取指/译码）只花一次，学生（ALU）有 8 个。代价很明确：**如果学生被要求做不同的题（分支发散），老师就只能一个一个来，其他学生闲着**。
- **架构/机制图解**：讲义用 8 个 ALU 共享一个取指/译码单元来画 SIMD，然后又给出 AVX intrinsics 版本对应的向量寄存器模型。

```
SIMD 执行单元（一个核内，8-wide）

 ┌──────────────────────────────────────────────────────┐
 │ [ Fetch / Decode ]  <- ONE control stream, shared by 8 ALUs│
 │        │                                             │
 │        │ same instruction broadcast to all ALUs      │
 │        ▼                                             │
 │   ALU0  ALU1  ALU2  ALU3  ALU4  ALU5  ALU6  ALU7     │
 │    ▲     ▲     ▲     ▲     ▲     ▲     ▲     ▲     │
 │    +-----+-----+-----+-----+-----+-----+-----+      │
 │                      │                               │
 │            [ Execution Context: 256-bit vector regs ]│
 └──────────────────────────────────────────────────────┘

AVX 的寄存器视角（__m256 = 256 bit = 8 x float32）

  标量寄存器 r0:      [ x[i] ]                    <- 1 个元素
  向量寄存器 ymm0:    [x[i] x[i+1] ... x[i+7]]    <- 8 个元素

  vloadps  ymm0, addr[r1]      ; 一条指令装载 8 个 float
  vmulps   ymm1, ymm0, ymm0    ; 一条指令做 8 个乘法
  vmulps   ymm1, ymm1, ymm0
  ...                          (内层 Taylor 迭代，见示例一)
  vstoreps addr[ymm2], ymm0    ; 一条指令写回 8 个 float

  标量指令序列 x 8 个元素  ≈  向量指令序列 x 1 遍   =>  ~8x 吞吐
  注意: 取指/译码只花 1 次，这就是讲义说的 "amortize control cost"
```

- **关键操作与性能特征**：SIMD 宽度由指令集决定：**SSE = 128 位（4×32 位或 2×64 位）**、**AVX = 256 位（8×32 位或 4×64 位）**、AVX-512 = 512 位（16×32 位）、ARM Neon = 128 位。向量指令有两条产生路径——**程序员用 intrinsics 显式请求**、**用并行语言语义（如 `forall`）传达给编译器**、或**编译器通过依赖分析自动向量化**（讲义明确指出这是难题，"即使最好的编译器在任意 C/C++ 代码上也不太行"）。术语上，编译期完成的 SIMD 化叫 **explicit SIMD（显式 SIMD）**，其特征是**可以直接在二进制里看到 `vmulps` / `vstoreps` 这类指令**。讲义给的机器实例：Intel Core i9（Coffee Lake）= 8 核 × 每核 8 个 SIMD ALU（AVX2）；NVIDIA GTX 480 = 15 核 × 每核 32 个 SIMD ALU、1.3 TFLOPS。

2.6 条件执行、掩码与指令流一致性

- **定义与目的**：SIMD 与分支是天然冲突的：一条指令要广播给 8 个 ALU，但 `if` 可能让 8 个元素走不同路径。硬件/编译器的解决方案是**串行化两条分支路径 + 用掩码（mask）丢弃不该写回的结果**。由此引出本讲最重要的术语之一：**指令流一致性（instruction stream coherence，讲义也写作 coherent execution）**——"同一段指令序列适用于所有被同时处理的元素"。
- **直观解释（"它是什么？"）**：回到"一位老师教 8 个学生"。如果 8 个学生做同一道题，老师讲一遍就行（满效率）。如果 3 个学生做 A 题、5 个学生做 B 题，老师必须先讲 A 题——**那 5 个学生只能假装在听（掩码掉输出）**；再讲 B 题时，那 3 个学生又只能闲着。**最坏情况下，8 个 ALU 只有 1 个在真正干活，效率 1/8**。
- **架构/机制图解**：下面是讲义 slide 32–35 的三张连续图合并成的"时间线 + 掩码"示意。

```
8-wide SIMD 执行 if (x > 0) {A} else {B}, 其中只有 3 个 lane 满足条件
                  ALU 时间线 (每一行 = 1 个时钟周期, 每格 = 1 个 lane)
time
 |   lane:   1     2     3     4     5     6     7     8
 v
┌─────────────────────────────────────────────────────────┐
│  <unconditional code>    (T = lane predicate is true)   │
│   T     T     T     F     F     F     F     F           │
├─────────────────────────────────────────────────────────┤
│  if-branch: only lanes 1-3 do useful work               │
│  [A]   [A]   [A]    x     x     x     x     x           │
│  [A]   [A]   [A]    x     x     x     x     x           │
├─────────────────────────────────────────────────────────┤
│  else-branch: only lanes 4-8 work                       │
│   x     x     x    [B]   [B]   [B]   [B]   [B]         │
│   x     x     x    [B]   [B]   [B]   [B]   [B]         │
├─────────────────────────────────────────────────────────┤
│  <resume unconditional code>  <- full width again       │
│  [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]         │
└─────────────────────────────────────────────────────────┘

结论: 分支本身不致命; 致命的是"同一批元素的控制流不一致"(divergent execution)。
      成本 ≈ 两条路径长度之和 / SIMD 宽度; 最坏情况只有 1/8 的 ALU 在做有用功。
      分支结束后立刻恢复全宽执行, 所以"短小而少量"的发散是可以接受的。
```

- **关键操作与性能特征**：讲义用两句话把这里的分寸讲清楚了，值得逐字记住：
- **指令流一致性是"高效使用 SIMD 资源"的必要条件**（coherent execution IS necessary for efficient use of SIMD processing resources）；
- **指令流一致性不是"跨核高效并行"的必要条件**（coherent execution IS NOT necessary for efficient parallelization across cores），因为**每个核都能独立取指/译码不同的指令流**。

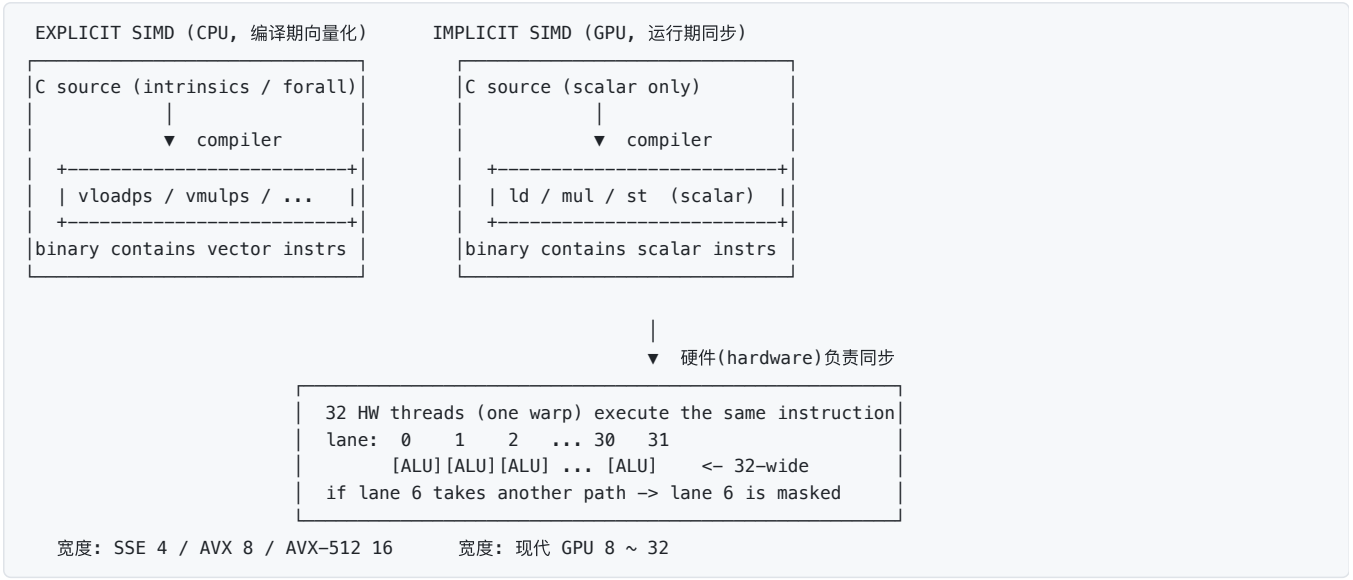
另外讲义特别提醒：**不要把"指令流一致性（instruction stream coherence）"和后面课程要讲的"缓存一致性（cache coherence）"混为一谈**——它们是完全无关的两个概念，只是英文都叫 coherence。

- **表格：四种并行执行形式对比**

并行形式	谁负责发现并行	并行粒度	需要程序提供什么	典型硬件实例	如果失败会怎样
超标量 superscalar (ILP)	**硬件**动态、运行期	指令之间	什么都不用做（但要有独立指令）	Kaby Lake 核可同时跑最多 4 条标量指令	浪费发射槽；本讲 `sinx` 内层循环就无 ILP
SIMD（显式）	**编译器**（intrinsics / 语言语义 / 依赖分析）	一条指令内的多个 lane	指令流一致（同一条指令序列）	AVX2 = 256 位 = 8×float	掩码 → 最坏 **1/8** 峰值
SIMD（隐式，GPU）	**硬件**运行期	一条指令内的多个 lane	指令流一致，宽度 8–32	GTX 480 warp = 32 线程	最坏 **1/32** 峰值
多核 multi-core	**软件**（程序员创建线程）	线程/任务	足够多的独立工作	Arrow Lake 20 核、Apple M4 10 核	串行程序变成 "0.75x 加速比"（更慢）
硬件多线程 HW MT	硬件（每时钟挑线程）	线程	**独立工作数要多于 ALU 数**	Skylake 2-way SMT	白占执行上下文存储与缓存容量

2.7 显式 SIMD 与隐式 SIMD：两种截然不同的实现策略

- **定义与目的**：讲义用两个术语区分 SIMD 的两种落地方案。**explicit SIMD（显式 SIMD）**：并行化发生在**编译期**，编译器真的生成向量指令（可反汇编看到 `vmulps`）。**implicit SIMD（隐式 SIMD）**：编译器生成的是**标量二进制**，但**N 份程序实例总是被硬件一起执行**（`execute(my_function, N)`），对硬件而言接口本身就是数据并行的——由**硬件**（不是编译器）负责让多个实例的同一指令落到不同数据上。这个区分的目的，是让"编程模型"与"硬件实现"这两个层次解耦。
- **直观解释（"它是什么？"）**：显式 SIMD 像**老师自己把 8 道题抄成一张大卷子**，让学生一次做完（编译期就把"8 个元素"打包成向量）。隐式 SIMD 像**学校规定全班必须同步做同一道题**（硬件保证 32 个线程同进同退），老师则按"一个学生一张卷子"来备课（写标量代码）。
- **架构/机制图解**：这两种策略对应的"程序看到的机器"完全不同，下面把它和 GPU 的 warp 机制画在一起。

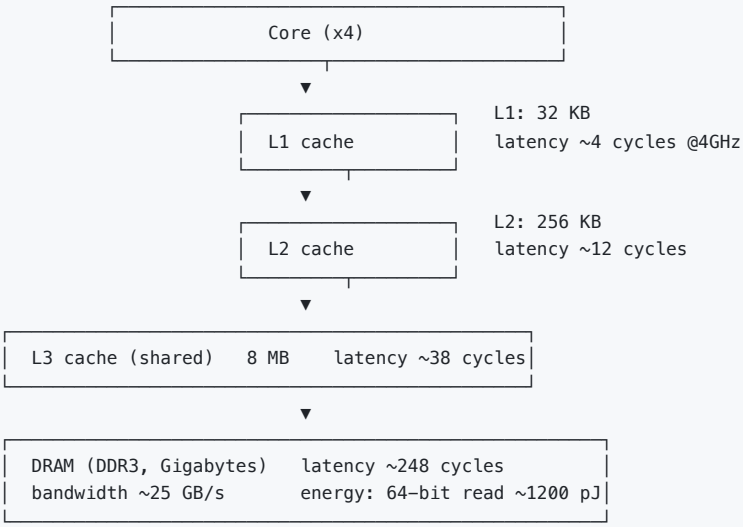


- **表格：显式 SIMD vs 隐式 SIMD**

对比项	显式 SIMD（explicit SIMD）	隐式 SIMD（implicit SIMD）
向量化发生时机	编译期	运行期（硬件）
生成的二进制	含向量指令（可反汇编验证）	只有标量指令
谁负责把 lane 映射到数据	编译器	硬件
典型载体	x86 SSE/AVX、ARM Neon	NVIDIA/AMD GPU 的 warp/wavefront
SIMD 宽度	4（SSE/Neon）/ 8（AVX2）/ 16（AVX-512）	8 ~ 32
发散执行的代价	最坏 1/宽度（如 1/8）	最坏 1/32 峰值性能
对程序员的可见性	高（可见向量类型/宽度假设）	低（写标量代码，但必须让同组线程走同一路径）
本讲相关代码	示例一（AVX2 intrinsics）、示例四（ISPC）	讲义的 GTX 480 warp 讨论、后续 CUDA 讲座

2.8 内存侧的挑战之一：延迟（latency）与停顿（stall）

- **定义与目的**：**内存延迟（memory latency）**是"一个访存请求（load/store）从处理器发出到被内存系统满足所需的**时间**"，讲义给的量级是 **100 个周期 / 100 纳秒**。**停顿（stall）**指处理器因为"下一条指令依赖前面尚未完成的指令"而无法继续。这一节要解决的核心问题是：**访问内存是停顿的最大来源，而延迟无法被消除，只能被"隐藏"或"减少"。**
- **直观解释（"它是什么？"）**：讲义在 CS149 版本里用**厨房/洗衣房**类比，但更贴切的是"**去地下室仓库取食材**"：CPU 是厨房，寄存器是案板（1 步就能拿到），L1 是冰箱（4 步），L2 是储物间（12 步），L3 是楼下的冷库（38 步），DRAM 是开车 20 分钟去的批发市场（~248 步）。**关键在于：你一旦开车去批发市场（发起 load），在车回来之前，案板上的菜就只能等——除非你有别的活可以干。**
- **架构/机制图解**：下面把讲义与 CS149 支撑材料里的缓存层次、延迟、带宽、能耗画在一张图上。




```
层次规律：越靠近处理器 => 容量越小、延迟越低、带宽越高、每字节能耗越低
数据访问延迟（Kaby Lake @4 GHz，CS149 表）：L1=4, L2=12, L3=38, DRAM≈248 周期
数据移动能耗（CS149，来源 Bill Dally/Tom Olson）：
    整数运算 ~1 pJ | 浮点运算 ~20 pJ | 片上 SRAM 读 64 bit ~26 pJ
    低功耗 DRAM(LPDDR) 读 64 bit ~1200 pJ  =>  从内存读 10 GB/s ≈ 1.6 W
对付延迟的三种手段：缓存(降低成本) / 预取(提前搬) / 硬件多线程(藏起来)
```

- **关键操作与性能特征**：讲义给出对付延迟的**三种手段**，其中前两种属于"减少/掩盖"、第三种属于真正的"隐藏"：
- **缓存（cache）——减少延迟的长度**：处理器只在数据已经在缓存里时才跑得高效。讲义的表述是 "Caches reduce memory access latency"，并补充"缓存同时提供高带宽的数据传输"。缓存还带来两种局部性红利（CS149 的演算）：**空间局部性**（载入一个 cache line 会"预载"同一行内后续要用的地址）与**时间局部性**（重复访问同一地址命中）。以一个容量 8 字节、行宽 4 字节、LRU 替换的两行缓存为例，顺序扫描一个 16 字节数组会得到 4 次 cold miss + 12 次 hit，而**再一次**扫描整个数组时，由于容量只有两行，地址 `0x0` / `0x4` 的数据早已被逐出，产生的是"容量缺失（capacity miss）"而不是命中——**这解释了为什么"数组太大、重复使用距离太远"时缓存帮不上忙**。
- **预取（prefetching）——把延迟挪走**：现代 CPU 动态分析访问模式，"预测 r2 的值并提前发起 load"，等真正执行到 `ld r0, mem[r2]` 时数据已经在缓存里，于是这两条 load 变成 cache hit。讲义同时警告：**预取猜错会降低性能**（白白占用带宽、污染缓存）。
- **硬件多线程（multi-threading）——用别人的活填满等待时间**：见 2.10。讲义特别强调：**多线程和预取一样，是"隐藏延迟"而不是"降低延迟"的技术**。

2.9 内存侧的挑战之二：带宽（bandwidth）

- **定义与目的**：**内存带宽（memory bandwidth）**是"内存系统向处理器提供数据的**速率**"，讲义给的量级是 **20 GB/s**。与延迟不同，**带宽不是可以通过"多找点活干"绕开的**：如果处理器请求数据的速度超过了内存系统的供给速率，那么**再多的延迟隐藏也没有用**（讲义原文："No amount of latency hiding helps this."）。这一节的目的，是让"性能瓶颈"这个概念从"算力不够"扩展到"数据供不上"。
- **直观解释（"它是什么？"）**：讲义用两组类比，配合起来看非常清晰。
- **高速公路**（CS149）：旧金山到 Stanford 约 50 km，车速 100 km/h。那么**延迟 = 0.5 小时**（一辆车从 SF 开到 Stanford 要多久），若"路上同时只能有一辆车"，**吞吐 = 2 辆车/小时**。提高吞吐有三条路：(a) **开快点**（200 km/h → 4 辆/小时）；(b) **多修车道**（2 车道 @100 km/h → 8 辆/小时）；(c) **让车挨近点**（车距 1 km、车速 100 km/h → **100 辆/小时**）。请注意 (a) 与 (c) 的差别：开车快既降延迟又升吞吐，而**压缩车距只升吞吐、不降延迟**——这正是"吞吐导向系统愿意牺牲单任务延迟"的本质。用公式说：`吞吐 = 车速 / 车距`。
- **洗衣房**（CS149）：一次洗衣 = 洗 45 分钟 + 烘 60 分钟 + 叠 15 分钟，**延迟 2 小时**。要吞吐翻倍，可以买两台洗衣机两台烘干机再叫个朋友（资源翻倍）；也可以**流水化**：不等第一批全做完就开始洗第二批，于是**只有一台洗衣机一台烘干机**也能做到 **1 批/小时**的吞吐——**流水线（pipelining）用相同的资源换来了吞吐**。这就是指令流水线的原型。
- **串联水管**（CS149）：一根最大流量 100 L/s 的管子接一根 50 L/s 的管子，**系统最大流量是 50 L/s**——系统的吞吐由最慢的一环（瓶颈）决定。把它对应到程序上：**内存带宽常常就是那根 50 L/s 的管子**。
- **架构/机制图解**：讲义的核心图是"带宽受限的稳态"（steady state）：一段只有 3 条数学指令、却要 load 64 字节的程序，在"内存每时钟供 8 字节"的机器上会发生什么。

```
假设 (CS149 multicore2): 核每时钟执行 1 条数学指令; 内存每时钟供 8 字节;
  程序模式 = load 64 字节, 然后 3 条依赖的数学指令; 有足够的硬件线程隐藏延迟

time(clock) -->  0    8    16   24   32   40
memory:        ┌────┬────┬────┬────┬────┐
               |XXXX|XXXX|XXXX|XXXX|XXXX|   每 8 个时钟完成一次 64 B 传输
               └────┴────┴────┴────┴────┘   8 B/clk x 8 clk = 64 B
                 ▲         ▲         ▲
                 |数据到达  |数据到达  |
core:       [aaa][.....][aaa][.....][aaa][.....]
            ^^^  = 3 条数学指令       ..... = 停顿 (stall)

ALU 利用率 = 3 条 / 8 个时钟 = 37.5%       <-- 与延迟大小、未完成请求数无关!
memory 利用率 = 100% (它已经在满负荷送数据, 不可能更快)

对照 (CS149 multicore1): 3 条算术 + 12 周期访存延迟
  1 个硬件线程 = 3/15 = 20% ;  2 个 = 6/15 = 40% ;  5 个 = 15/15 = 100%
  若改为 6 条算术 + 12 周期延迟: 单线程 6/18 = 33%, 只要 3 个线程就 100%
```

- **关键操作与性能特征**：这张图给出三条可迁移的结论。
- **稳态下，核的利用率只取决于"指令吞吐率"与"内存吞吐率"的比值**，而**与内存延迟、未完成访存请求数无关**（CS149 原文请读者自行说服自己这一点）。直觉是：延迟决定的是"要开多少个并发的请求才能把管子填满"（Little 定律 `并发数 = 带宽 × 延迟`），一旦填满，剩下的就只看管子有多粗。
- **提高算术强度可以降低对带宽的依赖**：把每次 load 之后的数学指令从 3 条提高到 6 条，利用率就从 37.5% 提到 75%。
- **带宽受限是吞吐型系统上最常见的性能天花板**："Overcoming bandwidth limits are a common challenge for application developers on throughput-optimized systems."
- **表格：延迟 vs 带宽——两种完全不同的瓶颈**

维度	内存延迟（latency）	内存带宽（bandwidth）
定义	一个访存请求被满足所需的时间	内存系统提供数据的速率
讲义举例	100 cycles / 100 nsec	20 GB/s
谁能改善它	缓存（降低）、预取（掩盖）、硬件多线程（隐藏）、提高 ILP	**只有"少访问数据"**：复用（同线程时间局部性、跨线程共享）+ 提高算术强度
典型症状	单线程停顿多、ALU 利用率低、增加线程后立刻改善	增加线程/ALU 后性能不再上升；memory 已 100% 忙碌
增加线程数的效果	**有救**（隐藏延迟）	**无救**（讲义："No amount of latency hiding helps this."）
相关公式	`需要的并发请求数 ≈ 带宽 × 延迟`（Little 定律）	`可达性能 = 带宽 × 算术强度`

2.10 想法 #3：硬件多线程（Hardware Multi-threading）

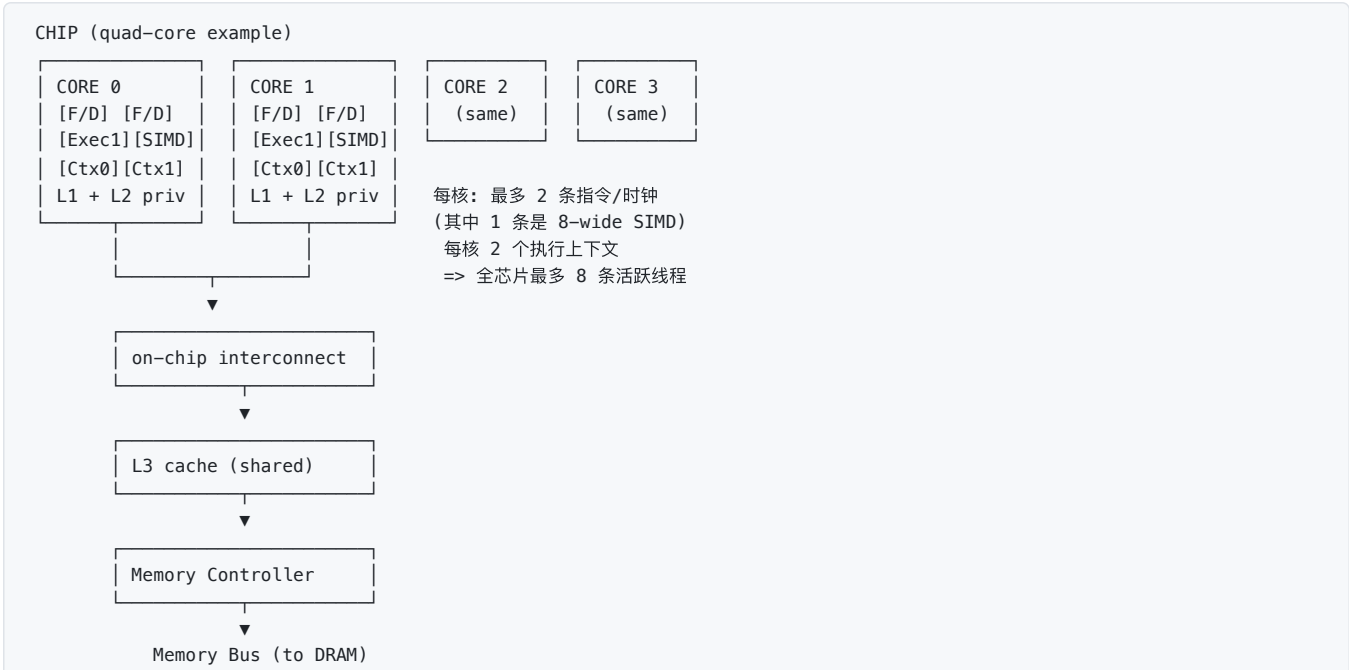
- **定义与目的**：**硬件多线程**指**一个核内保存多份执行上下文（execution context，即多组 PC + 寄存器）**，由**硬件（而不是操作系统）**在每个时钟周期决定执行哪一份上下文的指令。它解决的问题是 2.8 节留下的空白：**当某条线程因为长延迟访存而停顿，核的 ALU 就空转了；如果核上有另一条线程的工作，就可以立刻顶上。**核心限制是：**核的 ALU 数量没有变**——多线程只是让现有 ALU 在遇到高延迟操作时被用得更充分。
- **直观解释（"它是什么？"）**：这是一位厨师的**"边等水开边切葱"**。水壶放上去（发起 load）之后有 100 个周期的空档，厨师不必傻等，可以切葱花（执行另一条线程的算术指令）；水一开（数据到达），再回头处理。讲义在 CS149 版本里的表述最精炼：**"当你没法在当前线程上推进时，就去推进另一个线程。"** 代价是：**你手上要多备几份菜谱和几个碗（执行上下文的片上存储是有限资源），而且这个厨师做单道菜的时间会变长。**
- **架构/机制图解**：下面是 CS149 用"3 条算术 + 12 周期访存延迟"做的经典算例的甘特图（它是把讲义的时间线重画得可读）。

```
每个核: 每时钟 1 条标量指令; 线程模式 = [3 条算术指令][12 周期访存延迟]

(a) 1 个硬件线程                              -->  利用率 3/15 = 20%
clk   0    3              15   18             30
T0    [aaa][stall ........][aaa][stall ........]
      ^^^=3 math           ^^^ stall = 12 周期

(b) 2 个硬件线程                              -->  利用率 6/15 = 40%
T0    [aaa][stall ........][aaa][stall ........]
T1    [....][aaa][stall ...][....][aaa][stall ..]
                  ^ T0 此刻是 "runnable 但未被处理器执行"
                    (核正在执行另一条线程的指令)

(c) 4 个硬件线程                              -->  利用率 12/15 = 80%
T0    [aaa][stall ........][aaa][stall ........]
T1    [....][aaa][stall ...][....][aaa][stall ..]
T2    [....][....][aaa][st..][....][....][aaa]
T3    [....][....][....][aaa][stall ........] ...

(d) 5 个硬件线程  -->  15/15 = 100%   (再加线程无益, 已经跑满)

关键: 内存延迟本身一点没变! 变的是它不再导致处理器利用率下降。
```

- **关键操作与性能特征**：
- **片上执行上下文存储是有限资源**，这带来一个根本的设计取舍（讲义用两张图对比）：**要么 1 个核 16 个小的硬件线程**（能隐藏很高的延迟，但每个线程的工作集很小），**要么 1 个核 4 个大的硬件线程**（单线程工作集大，但隐藏延迟的能力低）。GPU 走的是前者（V100 的 SM 用 256 KB 寄存器保存最多 64 个 warp），CPU 走后者（Skylake 2 路 SMT）。
- **两种硬件多线程：交织式多线程（interleaved multi-threading，又称 temporal multi-threading）**——每个时钟从**一个**线程取一条指令；**同时多线程（SMT, Simultaneous Multi-Threading）**——每个时钟从**多个**线程各取指令同时发射，是超标量设计的扩展，Intel 的 Hyper-Threading 就是 2 线程/核的 SMT。
- **收益与代价**（讲义 slide 60 的清单）：收益是**更充分地使用 ALU 资源**（隐藏访存延迟、在单线程 ILP 不足时填满超标量的多个功能单元）。代价包括：需要更多存储保存线程上下文；**延长任何单一线程的运行时间**（在吞吐型并行应用中通常可接受）；**需要程序里有比 ALU 数更多的独立工作**；**高度依赖内存带宽**；以及"**线程越多 → 工作集越大 → 每线程分到的缓存越小 → 可能更频繁访问内存**（但延迟可以隐藏）"。
- **表格：交织式多线程 vs 同时多线程（SMT）** | 对比项 | 交织式 / 时间多线程 | 同时多线程（SMT） | |---|---|---| | 每个时钟执行几条线程的指令 | 1 条（来自被选中的那一个线程） | 多条（来自多个线程，同周期发射） | | 与超标量的关系 | 独立机制 | 是超标量设计的扩展（共用发射逻辑） | | 能否填满多个功能单元 | 不能（该时钟只有 1 条指令） | 能（这正是 SMT 的主要收益） | | 典型实例 | 讲义描述的"每时钟挑一个线程"的核 | Intel Hyper-Threading（2 线程/核） | | 对单线程延迟的影响 | 单线程变慢 | 单线程变慢 | | 需要的软件条件 | 独立工作数 > ALU 数 | 同上（且需多线程 ILP 互补） |

2.11 一台现代多核处理器的全貌

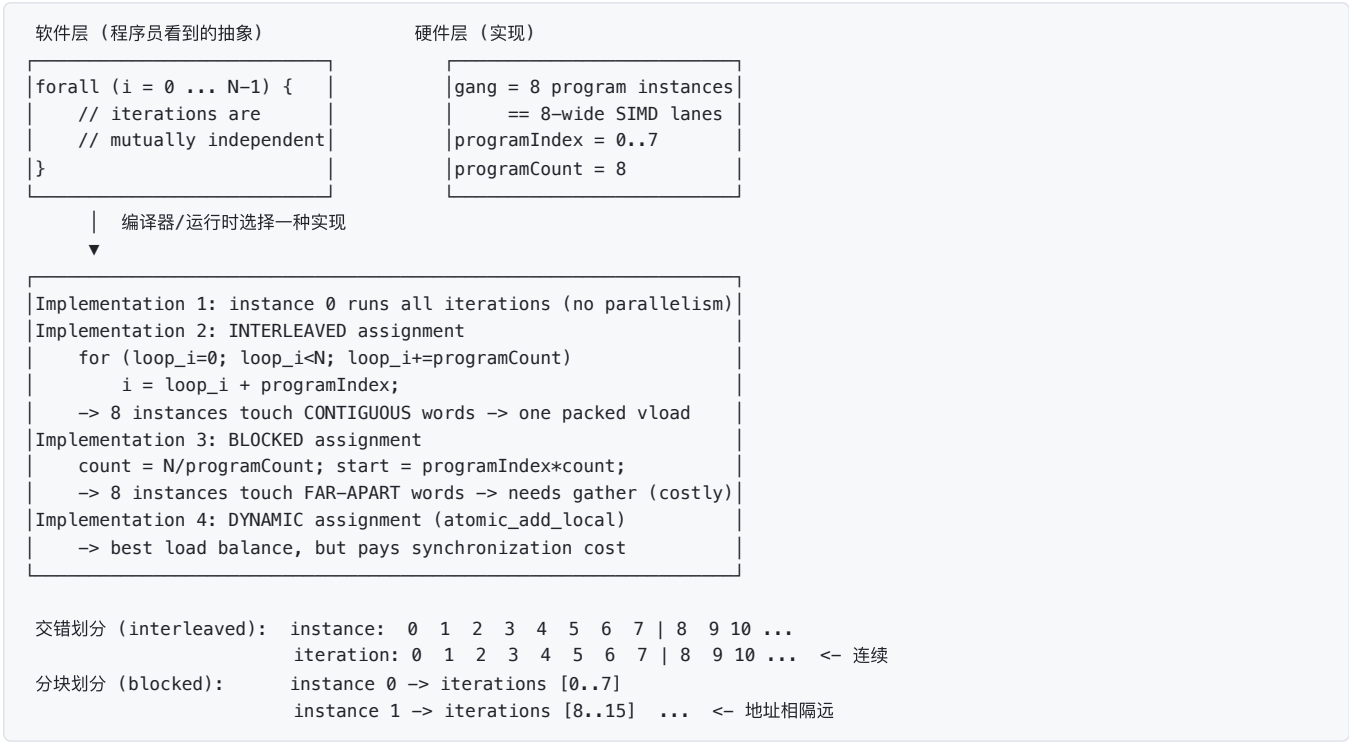
- **定义与目的**：把前面所有机制叠起来，就得到讲义 slide 61 与 slide 80 的两张"整机图"。它是本讲最重要的**定量直觉来源：一台机器的"有效并行度"是各层并行度的乘积，而"要跑满它"所需的独立工作数也是这个乘积。**
- **直观解释（"它是什么？"）**：把芯片想成**一栋办公楼**：16 个部门（核），每个部门 8 个工位（SIMD lane），每个工位有 4 个员工的轮班表（硬件线程）。要让整栋楼一刻不闲，你手上必须同时有 `16 × 8 × 4 = 512` 件互不依赖的活——**活少于这个数，就一定有工位在等**。
- **架构/机制图解**：下面是讲义 slide 80 的"四核整机图"的可读重绘。



- **表格：有效并行度与"跑满机器所需的独立工作数"** | 机器配置 | 核数 | 每核 SIMD 宽度 | 每核硬件线程 | 指令流数 | 需要多少独立工作才"跑满" | 出处 | |---|---|---|---|---|---| | 讲义"虚构多核芯片" | 16 | 8 | 4 | 16 条同时 + 共 64 条并发 | **512** 份独立工作 | 03_basicarch slide 61 | | 讲义四核整机图 | 4 | 8 | 2 | 最多 8 条活跃线程 | 4 × 8 × 2 = **64** | 03_basicarch slide 80 | | Intel Core i9 (Coffee Lake) | 8 | 8 | —（讲义未给） | — | —（讲义未给） | 03_basicarch slide 39 | | Intel i7-7700K (Kaby Lake) | 4 | 8 × 3 个向量 ALU = 24 | 2 (SMT) | — | —（讲义未给） | cs149 multicore1 | | NVIDIA GTX 480 | 15 | 16（32 ALU 分两组） | 最多 48 warp/核 | 15 × 48 warp | 每核 **1500+ 元素**，全芯片 **23,000** 份数据 | 03_basicarch slide 62–64 | | NVIDIA V100 | 80 | 16 | 最多 64 warp/SM | 80 × 64 | 每 SM 2048，全芯片 **163,840** | cs149 multicore1/2 |

2.12 软件执行模型：fork–join、`forall` 与 SPMD gang

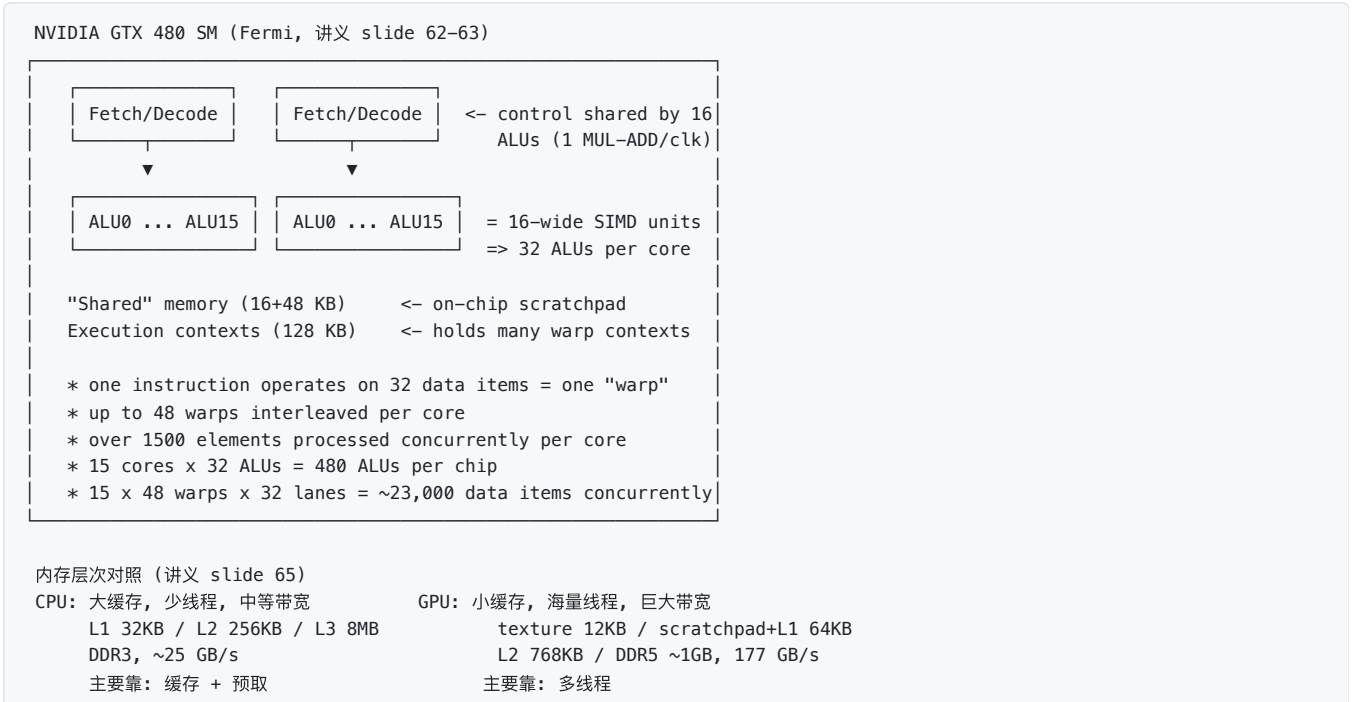
- **定义与目的**：硬件提供了上述并行资源，但**程序必须用某种"模型"来表达并行**。讲义在本讲引入了两种最基本的表达方式（后续 Lecture 4 起的编程模型讲座会系统展开）：**显式线程（pthreads / C++ threads）的 fork–join**，以及**数据并行声明（`forall`）**。CS149 支持材料进一步给出了它的实现形式：**SPMD（single program multiple data）/ gang**。
- **直观解释（"它是什么？"）**：
- **fork–join 像临时招工**：主线程招一批临时工（fork），大家分头干，干完登记收工（join）。招工本身有开销，所以是"少招工、每人多干"。
- **`forall` 像下订单**："把这 N 件衣服全部熨一遍"——你只声明"这些迭代互不相关"，**谁去熨、怎么分工由实现决定**。
- **SPMD gang 像广播体操**：一次性启动 `programCount` 个体操队员（gang），每个人拿到自己的编号 `programIndex`，然后**所有人一起做同一套动作**（同一条指令流），只是各自的位置不同（数据不同）。
- **架构/机制图解**：下面把 `forall`/SPMD 抽象与其 SIMD 实现画在一起，这是本讲"软件模型 ↔ 硬件执行"的关键对照。



- **关键操作与性能特征**：
- **交错划分 vs 分块划分的性能差异来自访存指令的形态**：交错划分下 8 个 instance 访问的是**连续 8 个 float**，一条 `vmovaps`（packed vector load）即可完成；分块划分下 8 个访问地址**彼此相隔 N/8 个元素**，需要 **gather 指令（`vgatherdps`）**——`gather` / `scatter` 是"更复杂、更昂贵"的 SIMD 访存指令（AVX2 在 2013 年支持 gather 但**不支持 SIMD scatter**，scatter 到 AVX-512 才有；GPU 上两者都有硬件支持，但仍远贵于连续向量的 load/store）。
- **数据并行的语义陷阱**：CS149 给了一个极好的例子——`foreach (i) { if (x[i]<0) y[2*i] = -x[i]; else y[2*i]=x[i]; y[2*i+1]=y[2*i]; }` 是正确的，而 `shift_negative`（当 `x[i]<0` 时写 `y[i-1]`）**输出未定义**，因为**多个迭代可能写同一内存位置**。这说明 `forall` 的语义要求是**迭代之间既没有数据依赖、也不写同一位置（race-free）**。
- **跨实例通信必须显式表达**：SPMD 里的局部变量是 per-instance 的（ISPC 称之为 "varying"），把 `foreach` 里的 per-instance 值累加到一个变量在类型上就是错的（编译期报错）；正确写法是**每个实例先算自己的部分和（partial），再用 `reduce_add()` 这类归约原语合并成 uniform 值**。这恰好就是 Lecture 1 里"通信是所有并行程序的核心成本"的具体化。
- **gang 内的 SPMD 抽象只在 SIMD 层面实现**：ISPC 的 gang 由**一个** x86 核上的一条线程里的 SIMD 指令实现，所以"用 ISPC 写的数据并行代码"默认只用了**一个核**；要多核执行需要额外的抽象（ISPC 的 "task"）。

2.13 GPU：把吞吐计算推到极致

- **定义与目的**：讲义用 GPU 作为"同一套吞吐计算思想的极端版本"来自我对照。理解 GPU 的 SM 结构，能反过来把 CPU 上"多核 + SIMD + 多线程"三件套的作用看得更清楚。
- **直观解释（"它是什么？"）**：CPU 是**少数几位博士**（大缓存、深乱序、聪明分支预测、少量大线程），GPU 是**几千名熟练工人**（小缓存、简单控制、海量线程）。讲义的原话总结得非常准确：**CPU 靠"缓存 + 预取"，GPU 靠"海量多线程"。**
- **架构/机制图解**：下面是讲义 slide 62–63 的 NVIDIA GTX 480（Fermi）SM 结构。



- **关键操作与性能特征**：GPU 侧的关键数字是 **warp = 32 个线程共用一条指令**，因此 **implicit SIMD 的发散惩罚上限是 1/32**（讲义原文：写得很差的代码可能只能跑到机器峰值的 1/32）。另一个数字是"为了跑满机器需要多少并发数据"：GTX 480 约 23,000，V100 高达 163,840——**这两个数字解释了为什么"并行度不足或算术强度低的程序在 GPU 上跑不快"**。

2.14 带宽受限与算术强度（arithmetic intensity）

- **定义与目的**：**带宽受限（bandwidth limited）** 指"处理器请求数据的速度过高，导致内存系统跟不上"的状态。**算术强度（arithmetic intensity）** 是讲义明确给出的一个"有用的术语"：**指令流中"数学运算次数"与"数据访问次数/字节数"的比值**。引入它是为了给出一个**可操作的优化目标：要高效使用现代处理器，程序必须有高的算术强度。**
- **直观解释（"它是什么？"）**：这就像**买菜做饭的"每斤食材出几道菜"**。如果你每买 1 斤菜只做 1 道菜，那你的产出完全被"去菜市场的速度"限制（带宽受限）；如果你能用 1 斤菜做出 20 道菜（高算术强度），那限制就回到你的厨艺（算力）。讲义的名言是 **"do more arithmetic: it's free"**——在现代芯片上，**多算几次比多读一次内存便宜得多**（回忆 2.8 节的能耗表：一次浮点运算 ~20 pJ，而从 LPDDR 读 64 位要 ~1200 pJ）。
- **架构/机制图解**：讲义用一个极简的"逐元素向量乘法"思想实验把带宽瓶颈的存在性证明得非常干净。

```
任务: C[i] = A[i] * B[i], A/B/C 各有数百万个元素

  读 A[i] (4 B) --+
  读 B[i] (4 B) --+--> [ MUL ] --> 写 C[i] (4 B)

  每做 1 次乘法需要 3 次内存操作 = 12 字节移动
  => 算术强度 AI = 1 flop / 12 B = 0.083 flop/byte   (极低!)

反推"跑满算力需要多少带宽" (讲义 slide 66 的数字):
  GTX 480: 480 MUL/clock @ 1.2 GHz => 480 x 1.2e9 x 12 B/s ≈ 6.4 ~ 6.9 TB/s
  实有 177 GB/s  =>  只有约 2.6% 的峰值效率 (讲义写 "~3% efficiency")
  但它依然比四核 CPU 快约 7 倍:
       GTX 480 有效算力 = 177 GB/s / 12 B = 14.75 GMUL/s
       四核 CPU (2.6 GHz Core i7 Gen4 + 25 GB/s) = 25/12 = 2.08 GMUL/s
       14.75 / 2.08 ≈ 7.1x  <- 讲义 "7x faster than quad-core CPU" 的来源

可达性能 = min( P_peak , BW x AI )
     ▲                    .........  <- compute bound: P_peak
     |                ..../
     |              ../   <- memory bound: slope = BW
     |           ../
     |        ../
     └──────────────────────────────► 算术强度 AI (flop/byte)
                    ^
               AI* = P_peak / BW   (machine balance, 拐点)

三条优化准则 (讲义 slide 68):
  1 组织计算, 少从内存取数据: 同线程复用(时间局部性) + 跨线程共享(线程间协作)
  2 请求更少的次数, 改为多算: "do more arithmetic: it's free"
  3 主结论: 程序必须有高算术强度, 才能高效利用现代处理器
```

- **关键操作与性能特征**：这一节把本讲前半部分（并行执行）和后半部分（内存）合成了一个完整的因果链：**更多核 + 更宽 SIMD 提高了峰值算力 → 峰值算力越高，喂饱它所需的带宽就越大 → 而带宽增长远慢于算力增长 → 所以"现代芯片上的许多并行应用（无论 CPU 还是 GPU）都是带宽受限的"**（讲义 Summary 原话）。CPython 式的直觉是"算力是免费的，数据移动是昂贵的"。

---

3. 代码示例与性能分析

3.1 示例一：AVX2 显式 SIMD 的 `sinx`（讲义 slide 28 的 intrinsic 版本）

- **代码**（已实测：`g++ 12.2 -O3 -mavx2 -mfma` 编译通过并运行，输出 `max|err|=1.192e-07`，与 `sinf` 一致到 float 精度）：

```
// 编译: g++ -O3 -mavx2 -mfma sinx_avx2.cpp -o sinx_avx2
//   (release 优化: -O3; AVX2 需 -mavx2, FMA 需 -mfma, 也可用 -march=native 一次性打开本机全部指令集)
//   要求: N 是 8 的倍数; x / result 按 32 字节对齐 (用 _mm_malloc 分配)
#include <immintrin.h>      // AVX / AVX2 intrinsics
#include <cstdio>
#include <cstdlib>
#include <cmath>

void sinx_avx2(int N, int terms, const float* __restrict x, float* __restrict result)
{
    for (int i = 0; i < N; i += 8) {                // 每次处理 8 个元素
        __m256 origx = _mm256_load_ps(&x[i]);       // 1 条 vmovaps: 载入 8 个 float
        __m256 value = origx;                       // value = x
        __m256 x2    = _mm256_mul_ps(origx, origx);  // x^2
        __m256 numer = _mm256_mul_ps(x2, origx);     // x^3   (= 第一项的分子)
        __m256 denom = _mm256_set1_ps(6.0f);         // 3!    (广播成 8 份)
        float  sign  = -1.0f;
        for (int j = 1; j <= terms; j++) {
            // value += sign * numer / denom
            __m256 t = _mm256_div_ps(_mm256_mul_ps(_mm256_set1_ps(sign), numer), denom);
            value = _mm256_add_ps(value, t);
            numer = _mm256_mul_ps(numer, x2);        // 分子 *= x^2
            denom = _mm256_mul_ps(denom,
                        _mm256_set1_ps((float)((2 * j + 2) * (2 * j + 3))));
            sign  = -sign;                           // 交替正负号
        }
        _mm256_store_ps(&result[i], value);          // 1 条 vmovaps: 写回 8 个 float
    }
}

int main()
{
    const int N     = 8 * 1000;   // 必须是 8 的倍数
    const int terms = 5;
    float* x      = (float*)_mm_malloc(N * sizeof(float), 32);   // 32B 对齐
    float* result = (float*)_mm_malloc(N * sizeof(float), 32);

    for (int i = 0; i < N; i++) x[i] = -1.0f + 2.0f * (float)i / (float)(N - 1);
    sinx_avx2(N, terms, x, result);

    float max_err = 0.0f;
    for (int i = 0; i < N; i++) {
        float e = std::fabs(result[i] - std::sin(x[i]));
        if (e > max_err) max_err = e;
    }
    std::printf("N=%d terms=%d  max|err|=%.3e  result[0]=%.6f\n",
                N, terms, max_err, result[0]);

    _mm_free(x); _mm_free(result);
    return 0;
}
```

- **【代码做什么？】**
- `_mm_malloc(..., 32)` 分配 32 字节对齐的数组——`_mm256_load_ps` / `_mm256_store_ps` 是**对齐**访存指令（`_ps` = packed single），地址不对齐会触发通用保护错误（要用非对齐版本得换成 `_mm256_loadu_ps`）。
- 外层循环 `i += 8`：**一次迭代处理 8 个数组元素**，这正是"用一条指令流处理多个数据元素"。
- `__m256` 是 256 位向量类型（8 × float32）。`_mm256_load_ps` 一条指令把 `x[i..i+7]` 载入一个向量寄存器；`_mm256_set1_ps(6.0f)` 把一个标量**广播**成 8 份（对应 `denom = 6`）。
- 内层 `j` 循环就是 Taylor 递推的向量版：`t = sign*numer/denom` 后累加到 `value`，再更新 `numer *= x²`、`denom *= (2j+2)(2j+3)`、`sign` 取反——**与讲义 slide 4 的标量代码逐步一一对应**，只是每个操作同时作用于 8 个元素。
- `main` 用 `x ∈ [-1, 1]` 初始化并逐元素与 `std::sin` 对比，验证向量化没有改变语义（浮点误差 ~1e-7 属 float 正常范围）。
- 编译命令中的 `-O3` 是必须的：**`-O0` 下 intrinsics 依然能编译，但不会得到任何流水/调度优化**；`-mavx2 -mfma` 打开指令集（用 `-march=native` 更省事）。
- **【并行机制与性能解说】**

- **硬件上如何并行**：这段代码**只用了 SIMD 这一种并行**（一维、一个核内的 8 个 lane）。它没有创建任何线程，也（在 `-O3` 下）**不依赖超标量**——因为 8 个 lane 已经由向量指令显式并行，且向量内部各 lane 天然独立；但 `value` / `numer` / `denom` 三条递推链在内层 `j` 循环里是**串行依赖**的，所以这段代码**没有 ILP 可用**（与讲义对 `sinx` 的批注一致）。要达到全芯片峰值，还需要在外面再套一层**多核并行**（把 `i` 的范围切开，见示例二）和一层**多线程延迟隐藏**。
- **工作如何分配**：`lane k` 处理元素 `i + k`，即**交错划分（interleaved assignment）**，因此访存是连续 8 个 float，正好一条 packed load 完成——这是 SIMD 分工的最佳形态。
- **共享数据如何处理**：8 个 lane **不共享任何数据**，每个 lane 有自己的一份 `value/numer/denom/sign`（模拟为向量寄存器中的 8 个槽位）。**不需要任何同步**，也**不存在数据竞争**：输出 `result[i..i+7]` 互不重叠。这正是"数据并行"的最干净形态。
- **Work / Span / 并行度**（以"向量指令"为计量单位，`terms=5`）：
  - 一次外层迭代的指令数 ≈ 装载 1 + 初始化 4 + 内层 5 次 × 约 7.5 条（set1、mul、div、add、mul、set1、mul）+ 写回 1 ≈ **43 条向量指令**。
  - **Work** `W = 43 × (N/8) ≈ 5.4N` 条向量指令（等价于 `43N` 条标量指令级的运算量，只是因为 SIMD 宽 8 而只需要 1/8 的指令条数）。
  - **Span**（关键路径）`S = 43`：因为内层 `j` 循环是三条递推链，**单个 chunk 内部无法再并行**，跨 chunk 之间才可以并行。
  - **并行度** `W/S = N/8`。对 `N = 1.6e7`，并行度约 **210 万**——远远超过机器的 ALU 数（讲义的虚构芯片 128 个），所以**并行度不是瓶颈**。
- **瓶颈分析**：真正的瓶颈有两处。
  1. **除法器**：`vdivps`（256 位浮点除）在 Skylake 级别 CPU 上**延迟约 11 周期、吞吐约 5 周期/条**，而普通向量乘法吞吐约 0.5 周期/条。本例每个 chunk 有 **5 条除法**（`terms=5`），仅除法就至少占用 `5 × 5 = 25` 个周期，而其余 ~38 条指令在 2–3 条向量流水线上只需约 13–19 个周期。**除法是这段代码最慢的一环**（对应 2.9 节"串联水管"的瓶颈原理）。改进：`denom` 的取值序列是确定的整数（6, 120, 5040, ...），可以**预算出倒数**，把 `div` 换成 `mul`；同样 `sign` 是 ±1，可以用一次 `xor` 翻转符号位代替 `set1` + `mul`。
  2. **内存带宽**：每个元素读 4 字节写 4 字节，算术强度只有 `2 flops / 8 B = 0.25 flop/byte`（远超 0.083 的向量乘，但依然很低）。按讲义虚构芯片 25 GB/s 的带宽，`N = 1.6e7` 个元素需要移动 128 MB，**仅带宽就需要 5.4 ms**——如果算力允许更快，程序就会撞上这堵墙（见 4.5 节的 Roofline 计算）。
- **可扩展性上限**：并行度 `N/8` 极大，所以加核、加线程都不会缺活；**扩展的上限由 `max(除法吞吐, 内存带宽)` 决定，而不是由并行度决定**——这就是本讲要传达的核心直觉。

3.2 示例二：OpenMP 多核 `sinx`——工作划分策略与缩放

- **代码**（已实测：`g++ 12.2 -O3 -fopenmp` 编译通过、运行结果与串行逐位相同（`verify = ok`））：

```
// 编译: g++ -O3 -fopenmp sinx_omp.cpp -o sinx_omp
// 运行: OMP_NUM_THREADS=16 ./sinx_omp
#include <omp.h>
#include <cstdio>
#include <cstdlib>

// 讲义 slide 4 的 sin(x)(Taylor 展开), 一次处理 1 个元素
static inline float sinx_one(float x, int terms)
{
    float value = x;
    float numer = x * x * x;
    int   denom = 6;                       // 3!(terms <= 6 时不会溢出 int)
    int   sign  = -1;
    for (int j = 1; j <= terms; j++) {
        value += sign * numer / denom;
        numer *= x * x;
        denom *= (2 * j + 2) * (2 * j + 3);
        sign  *= -1;
    }
    return value;
}

// (a) 串行版本: 整个数组由 1 个线程负责  -- 对应 "program expresses no parallelism"
static void sinx_serial(int N, int terms, const float* __restrict x,
                        float* __restrict result)
{
    for (int i = 0; i < N; i++) result[i] = sinx_one(x[i], terms);
}

// (b) 静态划分: 把 N 个迭代均分成 P 块, 每线程一块, 无运行时调度开销
static void sinx_static(int N, int terms, const float* __restrict x,
                        float* __restrict result)
{
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) result[i] = sinx_one(x[i], terms);
}

// (c) 动态划分: 每 4096 个迭代为一块, 用原子计数器把块分派给空闲线程
static void sinx_dynamic(int N, int terms, const float* __restrict x,
                         float* __restrict result)
{
    #pragma omp parallel for schedule(dynamic, 4096)
    for (int i = 0; i < N; i++) result[i] = sinx_one(x[i], terms);
}

int main()
{
    const int N     = 1 << 24;             // 16.7M 元素: x 与 result 共 128 MB
    const int terms = 5;
    float* x      = (float*)std::malloc((size_t)N * sizeof(float));
    float* ref    = (float*)std::malloc((size_t)N * sizeof(float));
    float* result = (float*)std::malloc((size_t)N * sizeof(float));
    for (int i = 0; i < N; i++) x[i] = -1.0f + 2.0f * (float)i / (float)(N - 1);

    double best_serial = 1e30;                        // 取 3 次最小值, 降低噪声
    for (int rep = 0; rep < 3; rep++) {
        double t0 = omp_get_wtime();
        sinx_serial(N, terms, x, ref);
        double dt = omp_get_wtime() - t0;
        if (dt < best_serial) best_serial = dt;
    }
    std::printf("serial: %7.2f ms   working set = %.1f MB\n",
                best_serial * 1e3, 2.0 * N * sizeof(float) / 1e6);
    std::printf("%5s %11s %11s %9s %9s %10s\n",
                "thr", "static/ms", "dyn/ms", "speedup", "GB/s", "verify");

    const int max_t = omp_get_max_threads();   // 必须先取: omp_set_num_threads 会改它
    for (int p = 1; p <= max_t; p *= 2) {
        omp_set_num_threads(p);
        double bs = 1e30, bd = 1e30;
        for (int rep = 0; rep < 3; rep++) {
```

```
            double t0 = omp_get_wtime(); sinx_static(N, terms, x, result);
            double t1 = omp_get_wtime(); if (t1 - t0 < bs) bs = t1 - t0;
            t0 = omp_get_wtime(); sinx_dynamic(N, terms, x, result);
            t1 = omp_get_wtime(); if (t1 - t0 < bd) bd = t1 - t0;
        }
        long bad = 0;                                   // 逐位校验并行结果 == 串行结果
        for (int i = 0; i < N; i++) if (result[i] != ref[i]) bad++;
        std::printf("%5d %11.2f %11.2f %8.2fx %9.1f %10s\n",
                    p, bs * 1e3, bd * 1e3, best_serial / bs,
                    2.0 * N * sizeof(float) / bs / 1e9, bad ? "FAIL" : "ok");
    }
    std::free(x); std::free(ref); std::free(result);
    return 0;
}
```

- **【代码做什么？】**
- `sinx_one` 是**逐元素**的标量函数，与讲义 slide 4 的内层逻辑完全一致（`int denom` 在 `terms ≤ 6` 时最大为 13! 仍是 3.9e8 < 2^{31}，不会溢出；`terms ≥ 7` 会溢出，应改成 `float` / `double`）。
- 三个 kernel 体现**三种"并行执行形式"的叠加关系**：`sinx_serial` 就是讲义里"程序不表达任何并行"的版本（编译后会作为**一个线程**跑在一个核上）；`sinx_static` / `sinx_dynamic` 用 `#pragma omp parallel for` **把外层循环的迭代分给 P 个线程**——这正是讲义用 pthreads 手写 `pthread_create` / `pthread_join` 所做之事的声明式版本（软件负责创建线程、硬件提供 TLP）。
- `schedule(static)` = **静态划分**：把 `0..N-1` 均分成 P 块，每线程一块，**运行期零调度开销**，但如果各迭代耗时不同就会**负载不均**。`schedule(dynamic, 4096)` = **动态划分**：以 4096 个迭代为一块，线程干完一块就用原子计数器抢下一块，**自动均衡负载**，但每次抢任务都有原子操作与同步开销。
- `main` 先跑串行基线并保存到 `ref`，再对 1、2、4、8、… 个线程分别计时（各取 3 次最小值），最后**逐位比较**并行结果与 `ref`（`bad` 应为 0）——这一步用来证明**这个数据并行改写没有引入任何数值差异**。
- 注意 `const int max_t = omp_get_max_threads();` **必须在循环之前取一次**：`omp_set_num_threads(p)` 会把 `omp_get_max_threads()` 的返回值也改掉，写成 `for (p = 1; p <= omp_get_max_threads(); p *= 2)` 会导致循环在第一轮后就退出（这是一个实际踩过的坑）。
- **【并行机制与性能解说】**
- **硬件上如何并行**：`#pragma omp parallel for` 展开为 `fork-join`——进入并行区时**创建/唤醒 P 个线程**（每个线程被操作系统映射到一个**线程执行上下文**上，讲义 slide 81 的问题"谁来把 pthread 映射到执行上下文？答案是操作系统"正是指这一层），每个线程执行 `sinx_one` 的一个**范围**，退出并行区时**join 并同步**。如果机器支持 SMT（例如 Kaby Lake 的 2 线程/核），P 甚至可以大于物理核数；此时 P 个软件线程被映射到 `核数 × 每核上下文数` 个硬件执行上下文上。
- **工作如何分配**：静态划分下线程 `t` 处理 `[t·N/P, (t+1)·N/P)`；动态划分下用 `dynamic, 4096` 的 chunk 抢占。**注意这里的 chunk 划分是"块状"的**——与示例一 SIMD 的交错划分相反。对**纯数组遍历**而言，块状划分依然能让每个线程顺序访问内存（空间局部性好），只是 SIMD 通道内不再连续（若再叠加 `#pragma omp simd` 就需要交错划分或让编译器插入 gather）。
- **共享数据如何处理**：`x` 和 `result` 是共享的，但**读写区间互不重叠**（线程只写自己的那段 `result`），因此在 **cache line 层面也不共享**（只要每段的边界大致对齐到 cache line，就不出现伪共享）。循环体内部没有共享可变量，**不需要锁、不需要原子操作、不需要内存栅栏**——唯一的同步点是并行区末尾的隐式 barrier。这是"数据并行"最高的理想形态：**只分数据、不通信**。
- **Work / Span / 并行度**（以"标量指令"为计量单位，`terms=5`）：
  - 每个元素的指令数 `w` ≈ 函数序言（~3）+ 内层 5 次迭代 × 约 10 条（`sign*numer` 乘、int→float 转换、除法、加法、`x*x`、乘、两次整数乘、整数乘、符号取反、循环比较/跳转）+ 读/写 ≈ **55 条**。
  - **Work** `W = 0(N · terms) ≈ 55N`。对 `N = 1.6e7` 约 **9.2e8 条标量指令**。
  - **Span**（关键路径）`S = 0(terms) ≈ 55`：**单个元素的 Taylor 递推是串行的**（`value`、`numer`、`denom` 都是循环携带依赖），所以关键路径就是"一个元素走完全部 5 次迭代"。
  - **并行度** `W/S = N ≈ 1.67e7`。机器的 ALU 数是几百到几千——**并行度有 4 个数量级的余量**，所以"并行度不足"绝不是这个程序的瓶颈。
- **瓶颈分析**：
  1. **Amdahl 上限来自串行开销**：线程创建/销毁、并行区 barrier、以及 `main` 里的初始化和校验都是串行的。若串行部分占 `f`，则 `S(P) = 1/(f + (1−f)/P)`；要让 16 线程拿到 15x，必须有 `f < 1/(16·15−16+1) ≈ 0.0025`——**串行部分必须压到 0.25% 以下**。这解释了为什么在长期运行的程序里要**复用线程池**（OpenMP 默认就会让线程池常驻）而不是反复 create/join。
  2. **内存带宽**：每个元素 8 字节（读 4 + 写 4），`N = 1.6e7` → **134 MB 的流量**。按讲义虚构芯片的 25 GB/s，**光带宽就要 5.4 ms**；这就是为什么在算术强度低的代码上"加核"很快就不再有用（见 4.4 的定量推导）。
  3. **负载不均 vs 调度开销**：本程序的每个迭代代价完全一样，所以 `static` 与 `dynamic` 差别很小（`dynamic` 只会更慢）；但如果各迭代代价差异大（例如后面的三角形循环），`static` 就会出现"一个线程干了一半活"的长尾，此时 `dynamic` 才划算。**判断依据是迭代代价的方**

差，不是直觉。

- **实测观察（本机 g++ 12.2 `-O3`，共享的 128 硬件线程 x86 机器，仅供定性参考）：**

线程数	static /ms	dynamic /ms	相对串行 speedup	有效带宽 GB/s	校验
串行基线	22.56（best of 3）	—	1.00x	5.9	—
1	126.46	112.80	0.18x	1.1	ok
2	65.20	57.42	0.35x	2.1	ok
4	32.75	28.80	0.69x	4.1	ok
8	16.55	14.40	1.36x	8.1	ok
16	9.13	7.23	2.47x	14.7	ok

这张表里最重要的信息**不是**加速比数字，而是第一行与第二行的对比：**同一个循环，串行版本被 GCC 自动向量化了（4 宽 SSE），而放进 `omp parallel for` 之后该循环没有被向量化**（`-fopt-info-vec` 报告里串行循环显示 `optimized: loop vectorized using 16 byte vectors`，并行版本的循环则没有）。用一个更干净的对照实验（`N = 2²⁴`、`terms = 5`、取 7 次最小值、单线程）可以确认这一点：

实验	时间	说明
串行循环 `for(i) r[i]=sinx_one(x[i])`	**22.7 ms**	GCC 自动向量化（16 字节向量）
同一循环放进 `#pragma omp parallel for`（1 线程）	**119.9 ms**	向量化报告显示该循环**未被向量化**
两者都加 `-fno-tree-vectorize`	84.8 ms / 117.9 ms（比值 1.39x）	关掉自动向量化后差距从 5.3x 缩到 1.39x，证明确实是**代码生成差异**而非算法差异

结论（可迁移的工程规则）：**"并行化"和"向量化"是两个独立的优化，必须分别验证**；比较加速比时**必须与"同一种代码生成质量"的基线比**，否则会得到荒谬的结论（比如"1 线程的并行版比串行版慢 5 倍"）。正确做法是查编译器的向量化报告（`-fopt-info-vec`）、必要时看反汇编，或者干脆像示例一那样**显式写 intrinsics**。

3.3 示例三：算术强度扫描 + ILP 对算力利用的影响

- **代码**（已实测：`g++ 12.2 -O3 -mavx2 -mfma -fopenmp` 编译并运行）：

```
// 编译: g++ -O3 -mavx2 -mfma -fopenmp ai_sweep.cpp -o ai_sweep
// 运行: OMP_NUM_THREADS=16 ./ai_sweep
#include <omp.h>
#include <cstdio>
#include <cstdlib>

// 单次 kernel: 读一遍 N 个 float, 每个元素做 R 次 FMA(2 flops)。
// 算术强度 AI = 2R flops / 4 bytes = R/2  (flop/byte)
static void kernel(int N, int R, const float* __restrict a, float* __restrict sink_out)
{
    float sink = 0.0f;
    #pragma omp parallel for schedule(static) reduction(+:sink)
    for (int i = 0; i < N; i++) {
        float v = a[i];
        for (int r = 0; r < R; r++)
            v = v * 1.0000001f + 0.5f;      // 2 flops, 依赖链: 每次都用上一次的 v
        sink += v;
    }
    *sink_out = sink;
}

// 同一 kernel, 但用 4 条互相独立的累加链 -> 暴露 ILP
static void kernel_ilp4(int N, int R, const float* __restrict a, float* __restrict sink_out)
{
    float sink = 0.0f;
    const int nq = N / 4;                    // 可被 4 整除的主体长度
    // 注意: OpenMP 要求规范的循环形式, 所以主体写成 [0, nq), 尾部单独处理
    #pragma omp parallel for schedule(static) reduction(+:sink)
    for (int q = 0; q < nq; q++) {
        const int i = 4 * q;
        float v0 = a[i], v1 = a[i+1], v2 = a[i+2], v3 = a[i+3];
        for (int r = 0; r < R; r++) {
            v0 = v0 * 1.0000001f + 0.5f;
            v1 = v1 * 1.0000001f + 0.5f;
            v2 = v2 * 1.0000001f + 0.5f;
            v3 = v3 * 1.0000001f + 0.5f;
        }
        sink += (v0 + v1) + (v2 + v3);
    }
    // 尾部余数 (N % 4 个元素): 串行处理, 不影响结论
    for (int i = 4 * nq; i < N; i++) {
        float v = a[i];
        for (int r = 0; r < R; r++) v = v * 1.0000001f + 0.5f;
        sink += v;
    }
    *sink_out = sink;
}

int main()
{
    const int N = 1 << 22;                   // 4M floats = 16 MB
    float* a = (float*)std::malloc((size_t)N * sizeof(float));
    for (int i = 0; i < N; i++) a[i] = 0.5f + 1e-6f * (float)i;

    std::printf("%4s %8s %14s %14s %10s\n",
                "R", "AI", "1-acc GFLOP/s", "4-acc GFLOP/s", "ratio");
    for (int R = 1; R <= 64; R *= 4) {
        float s1, s4; double b1 = 1e30, b4 = 1e30;
        for (int rep = 0; rep < 5; rep++) {          // 取 5 次最小值
            double t = omp_get_wtime(); kernel(N, R, a, &s1);
            t = omp_get_wtime() - t; if (t < b1) b1 = t;
            t = omp_get_wtime(); kernel_ilp4(N, R, a, &s4);
            t = omp_get_wtime() - t; if (t < b4) b4 = t;
        }
        std::printf("%4d %8.2f %14.1f %14.1f %9.2fx\n",
                    R, R / 2.0, 2.0 * R * N / b1 / 1e9,
                    2.0 * R * N / b4 / 1e9, b1 / b4);
    }
    std::free(a);
```

```
    return 0;
}
```

- **【代码做什么？】**
- `kernel` 对数组做**一遍流式读**（`a[i]` 各读一次），每个元素执行 `R` 次"乘加"（`v * c + d` 会被编译器融合成一条 FMA 指令、记 2 flops）。**因为每个元素只读 4 字节**，调大 `R` 就等价于**调高算术强度** `AI = 2R / 4 = R/2 (flop/byte)`，这就把一个"AI 扫描（AI sweep）"实验做出来了——它正是 Roofline 模型的实验版。
- `kernel_ilp4` 是同一个计算，但**同时维护 4 条互不相关的累加链** `v0..v3`。这样做的意义是：一条 FMA 依赖链只能"每 4 个周期出一结果"（FMA 延迟 ≈ 4 周期），而 FMA 流水线**每周期能吞一条**，所以**单链最多只能用掉 1/4 的算力**；4 条独立链才能把流水线填满。
- `main` 用 5 次最小值消除噪声，同时报告 GFLOP/s 与两条链的比值 `ratio`。
- **【并行机制与性能解说】**
- **硬件上如何并行**：这里同时用了**三种**并行——`#pragma omp parallel for` 提供**多核 TLP**（16 线程）；每次 `float` 运算由编译器打包成 **SIMD**（`-mavx2` 下 8 宽）；`v0..v3` 四条独立链给**超标量/乱序执行**提供了 ILP。这正是本讲"四种并行形式叠加"的直接演示。注意 `reduction(+:sink)` 让每个线程先在私有副本上累加、最后再合并——**避免了共享变量上的原子竞争**（这是比"加锁"更好的做法）。
- **工作如何分配**：静态划分 `schedule(static)`，每个线程拿到一段连续区间。这里负载完全均匀（每个元素代价一致），所以静态划分是最优选择。
- **共享数据如何处理**：输入 `a` 只读、全区共享（只读数据无竞争）；`sink` 通过 `reduction` 变成每线程私有的部分和，**只在并行区结束处合并一次**。这是"避免共享可变状态"的标准范式。
- **Work / Span / 并行度**：
  - **Work**（以 FMA 次数计）`W = R·N`；`kernel_ilp4` 的 Work 相同。
  - **Span**：`kernel` 的每个元素是 `R` 条 FMA 的**串行链**，所以 `S = R` + 一次装载；`kernel_ilp4` 的每个元素是 4 条长度 `R` 的链，`S = R`（4 条链并行）。
  - **并行度** `W/S = N`（两者都是）。
  - 但注意：**并行度大 ≠ 算力用满**。`kernel` 的并行度 `N = 4M` 远超机器的 lane 数，却仍然只跑到峰值的很小一部分——因为**每个 lane 内部只有 1 路 ILP，而 FMA 延迟/吞吐比是 4**。这说明第 4 节的"并行度 = Work/Span"必须**细分到执行单元的粒度**才有意义：`N` 路并行的**粒度是元素**，而填满 FMA 流水线需要的是**每个 lane 内 ≥4 路 ILP**。
- **实测结果（本机，`OMP_NUM_THREADS=16`，`-O3 -mavx2 -mfma`，`N = 4M`，best of 5）**：

内层重复次数 R	1	4	16	64
算术强度 AI (flop/byte)	0.5	2	8	32
单累加链：GFLOP/s	31.9	72.3	73.8	42.7
四独立累加链：GFLOP/s	101.9	189.9	178.0	116.0
比值（ILP 收益）	**3.19x**	**2.63x**	**2.41x**	**2.72x**

  `R = 1` 那一列的对比最能说明问题：**算术强度只有 0.5 flop/byte 的 kernel，即使开了 16 个线程、8 宽 SIMD，单累加链也只有 31.9 GFLOP/s，把 ILP 从 1 加到 4 立刻变成 101.9 GFLOP/s（3.19x）**。这与"FMA 延迟 4 周期 / 吞吐 1 周期 → 单链最多用到 25% 流水线"的硬件事实**定量吻合**（理想值 4x，实测 2.4–3.2x 是受端口数、装载单元与内存系统共同限制的结果）。R 很大时 GFLOP/s 反而下降，是因为此时时间几乎全部花在纯计算上，频率/功耗与访存流的交互开始显现（并且这台机器是共享的，数字有噪声）。 – **瓶颈分析**：`R` 小时是**内存带宽受限**（GB/s 接近本机流式读取上限）；`R` 大时是**执行端口/延迟受限**。这个"先向上、后走平、再回落"的形状，就是第 4 节 Roofline 折线在真实机器上的样子。

3.4 示例四：ISPC 的 SPMD / gang 抽象（软件执行模型的另一种表达）

- **代码**（来自 CS149 支撑材料；**本环境未安装 ISPC 工具链，未做编译验证**，编译命令按 ISPC 官方用法给出）：

```
// 文件 sinx.ispc
// 编译(需要 Intel SPMD Program Compiler, https://ispc.github.io):
//   ispc sinx.ispc --target=avx2-i32x8 -O3 -h sinx_ispc.h -o sinx_ispc.o
//   g++ -O3 main.cpp sinx_ispc.o -o sinx
export void ispc_sinx(uniform int N, uniform int terms,
                      uniform float* x, uniform float* result)
{
    // 假设 N % programCount == 0
    for (uniform int i = 0; i < N; i += programCount) {   // uniform: 整个 gang 一致
        int idx = i + programIndex;                       // varying: 每个实例不同
        float value = x[idx];
        float numer = x[idx] * x[idx] * x[idx];
        uniform int denom = 6;                            // 3!
        uniform int sign  = -1;
        for (uniform int j = 1; j <= terms; j++) {
            value += sign * numer / denom;
            numer *= x[idx] * x[idx];
            denom *= (2 * j + 2) * (2 * j + 3);
            sign  *= -1;
        }
        result[idx] = value;
    }
}
```

```
// 文件 main.cpp —— 顺序 C++ 代码，只负责调用 ISPC 函数
#include "sinx_ispc.h"
#include <cstdio>
int main()
{
    const int N     = 1024;
    const int terms = 5;
    float* x      = new float[N];
    float* result = new float[N];
    for (int i = 0; i < N; i++) x[i] = -1.0f + 2.0f * (float)i / (float)(N - 1);

    ispc_sinx(N, terms, x, result);       // 调用 ISPC 函数 -> 启动一个 "gang"

    std::printf("result[0]=%f\n", result[0]);
    delete[] x; delete[] result;
    return 0;
}
```

- **【代码做什么？】**
- `export void ispc_sinx(...)` 是 ISPC 的"可被 C/C++ 调用"的函数。调用它时，**运行时启动一个 "gang"（一帮 `programCount` 个 program instance）**，所有实例**并发执行同一份 ISPC 代码**，每个实例有自己的局部变量副本，返回时所有实例一起结束——这就是 **SPMD（single program, multiple data）**。
- `programCount` 是 gang 里同时执行的实例数（**uniform 值**，所有实例相同）；`programIndex` 是当前实例在 gang 中的编号（**varying 值**，每个实例不同）。`uniform` 类型修饰符只是**给编译器的优化提示**（"这个变量的值所有实例都一样"），**去掉它程序依然正确**。
- 循环写成 `for (uniform int i=0; i<N; i+=programCount) { int idx = i + programIndex; ... }`，即**交错划分**：实例 `k` 处理元素 `k, k+programCount, k+2·programCount, ...`。因为 `programCount` 个实例在同一次迭代里访问的是**连续地址**，编译器可以为 `x[idx]` 生成**一条 packed vector load**。
- `main.cpp` 完全是顺序 C++，**只把 ISPC 函数当普通函数调用**——这正是该抽象的价值：把数据并行代码隔离在一个函数里。
- **【并行机制与性能解说】**
- **硬件上如何并行：抽象层是 SPMD（`programCount` 条逻辑指令流），实现层是 SIMD（一条向量指令）**——ISPC 编译器把 gang 的语义编译成 AVX2/Neon 等向量指令，`programCount` 通常等于硬件的 SIMD 宽度（或它的一个小倍数）。**如果写成 `--target=avx2-i32x8`，`programCount` 就是 8**，8 个实例映射到 8 个 lane。条件控制流由编译器**转换成掩码**（讲义 2.6 节的机制，ISPC 手册称"分支是顺序执行两条路径并掩码"）。
- **工作如何分配**：交错划分（interleaved）→ packed load。若改成**分块划分**（`count = N/programCount; start = programIndex*count; for(i<count) idx = start+i;`），8 个实例访问的地址会相隔很远，编译器就必须生成 **gather 指令（`vgatherdps`）**，而 gather 明显比连续向量的 load/store **更贵**。**同一份 SPMD 代码，仅因"实例到数据的映射方式"不同，就会产生数倍的访存开销差异**——这是本讲"软件模型必须考虑硬件实现"的最好例证。

- **共享数据如何处理**：gang 内的局部变量是**每实例私有**的（varying），因此**不需要同步**。要跨实例通信必须用**显式原语**：`reduce_add()`（gang 内求和）、`reduce_min()`、`broadcast(value, index)`、`rotate(value, offset)` 等。一个经典的反面例子是**用 `foreach` 直接累加**：

  `c // 错误写法 1: sum 是 uniform, x[i] 是 varying -> 编译期类型错误 uniform float sum = 0.0f; foreach (i = 0 ... N) { sum += x[i]; } // ERROR: 不知道该把哪个实例的值写进 sum // 错误写法 2: sum 是 varying -> 返回值类型不匹配 float sum = 0.0f; // 每个实例一份私有 sum foreach (i = 0 ... N) { sum += x[i]; } return sum; // ERROR: 无法把一个 varying 值返回成 float // 正确写法: 每实例先算部分和, 再用归约原语合并 uniform float sum; float partial = 0.0f; foreach (i = 0 ... N) { partial += x[i]; } sum = reduce_add(partial); // 跨实例通信, 返回 uniform 值`

  `reduce_add` 在 SIMD 实现上等价于"把向量寄存器的 8 个 lane 横向相加"（通常是 $\log_2(8) = 3$ 次 shift+add），**这就是"通信"在 SIMD 上的物理形式**。讲义用 SPMD 里的这种跨实例操作说明了"用高阶原语做规约"的思想，它和 Lecture 8 数据并行（map/fold/scan）是同一件事。 – **Work / Span / 并行度**： – **Work** = `N × w`（与示例二相同），`w ≈ 55` 条标量操作。 – **Span** = `w`（一个元素的 Taylor 链）。 – **并行度**（抽象层）= `N`；但**实现层的有效并行度 = gang 宽度 × 线程数**：若只用 ISPC 而不使用它的 task 抽象，gang 只在**一个 x86 核的一条线程**里跑，**有效并行度只有 `programCount = 8`**。这是最容易犯的错——"写了 ISPC 就以为用了整台机器"。要吃到多核必须叠加 ISPC 的 `task`（或外层 OpenMP）。 – **瓶颈**：`programCount` 是**编译期固定的**（如 avx2-i32x8 下恒为 8），因此 `N % programCount != 0` 的边界要么被假设整除（示例注释里的"assume N % programCount = 0"），要么就需要掩码处理尾部——**尾部处理是 SIMD 代码里最常见的 bug 源与性能损失源**。

---

4. 性能模型与复杂度分析

4.1 Work–Span（工作量–关键路径）模型

定义：把并行程序看成一个 DAG。

- **Work（`W`）**：所有操作的总数——**用 1 个处理器执行所需的时间**（也就是串行时间 T_1）。
- **Span（`S`，也叫 critical path / depth）**：DAG 中最长的依赖链——**用无限多处理器执行所需的时间**（T_∞）。
- **并行度（parallelism）**：`W / S`，即"平均可以同时进行的操作数"。它是**程序固有属性**，与机器无关。

两个界（CS149 的 scan 例子明确给出了这套记号）：

- 用 `P` 个处理器，`T_P ≥ max(W/P, S)`，因此 `T_P = O(W/P + S)`。
- 加速比 `S(P) = T₁/T_P ≤ min(P, W/S)`。**当 `P > W/S` 时再加处理器毫无收益**——这直接对应讲义"512 个独立工作才能跑满虚构芯片"的结论。
- **效率** `E = S(P)/P ≤ min(1, (W/S)/P)`。

例（示例二的 `sinx`）：`W ≈ 55N`，`S ≈ 55`，并行度 `W/S = N`。对 `N = 1.6e7`，`W/S = 1.67e7`。哪怕机器有 10 万个硬件 lane，也远小于 1.67e7，**所以这个程序永远不会因"并行度不够"而受限**。

反例（关键路径很长的算法）：并行 exclusive scan 的朴素版本（Hillis–Steele）在 CS149 支撑材料里给出的分析是 **Work = O(N log N)、Span = O(log N)**——`Work` 比串行算法的 `O(N)` **更差**（"Inefficient compared to sequential algorithm!"），但 Span 极短。工作高效的版本（up-sweep + down-sweep）把 Work 降到 `O(N)`、Span 仍是 `O(log N)`，代价是**常数更大、SIMD 利用率更低**（CS149 在 CUDA warp scan 一节明确说：work-efficient 版本在 32-wide SIMD 上需要**两倍以上的指令数**，所以实践中反而更慢）。这给出一个重要的工程判断：**选择算法时要在"Work 小"和"Span 小"之间权衡，而 SIMD 的利用率可以让"Work 更大但更规整"的算法胜出**。

4.2 Amdahl 定律

```
                    1
S(P) = ──────────────────────────        f = 不可并行(串行)部分的比例
        f + (1 − f) / P                  P = 处理器数

上限:  lim  S(P) = 1 / f
       P→∞
```

- **数值算例**：设 `f = 5%`（并行区的 95% 可完美并行），则 `P→∞` 时 `S → 20x`。 | P | 1 | 2 | 4 | 8 | 16 | 32 | ∞ | |---|---|---|---|---|---|---|---| | S(P)（f=0.05） | 1.00 | 1.90 | 3.48 | 5.93 | **9.14** | 12.55 | 20.0 | | S(P)（f=0.01） | 1.00 | 1.98 | 3.88 | 7.48 | 13.91 | 24.24 | 100.0 | | S(P)（f=0） | 1.00 | 2.00 | 4.00 | 8.00 | 16.00 | 32.00 | ∞ |

解读：`f = 5%` 时，**16 核只能拿到 9.14x（57% 的效率）**，并且**再加到 32 核只多 3.4x**。这解释了为什么并行程序优化中"消灭串行瓶颈"（I/O、初始化、全局同步、归约）往往比"再买几个核"更有效。 – **讲义版本的 Amdahl（多核的收益 vs 单核变慢）**：复杂核速度 1.0，简单核 0.75（讲义

取的虚构系数）。 – 有并行：`T(2 核) = 0.75/2 = 0.375`，相对原来 1.0 的**加速比 = 2.67x**（讲义写作"潜力 2 × 0.75 = 1.5"，即吞吐/单位时间的工作量提升 1.5 倍）。 – 无并行：`T = 0.75`，**加速比 0.75x，即变慢了 33%**。 – 结论：**多核时代"不提并行"是负优化**。 – **Amdahl 的补充（Gustafson）**：Amdahl 假设问题规模固定，因此串行部分占比不随 `P` 变；而实际中人们常常**同时扩大问题规模**，此时可扩展性会好得多。讲义的思想实验（向量乘、`sinx`）都是"数组越大越有利"的例子——**大数组摊薄了固定开销**。

4.3 延迟隐藏需要多少线程?

这是本讲最实用的一组定量公式。设：

- 一条线程在做完 `k` 条可执行指令后必须等待一次访存，等待期 `L` 个周期；
- 核每个周期最多执行 1 条指令（吞吐 `1/cycle`）。

则**单线程利用率** `u₁ = k / (k + L)`，**要跑满核所需的线程数** `P* = ceil((k + L)/k) = ceil(1 + L/k)`。

数值算例（CS149 的原始算例）：

场景	单线程利用率	2 线程	需要的线程数
3 条算术 + 12 周期访存延迟	`3/15 = 20%`	`6/15 = 40%`	`ceil(15/3) = 5`
6 条算术 + 12 周期访存延迟	`6/18 = 33%`	`12/18 = 67%`	`ceil(18/6) = 3`

结论两条（CS149 明确写出的 takeaway）：

1. **多线程并没有改变访存的延迟**，它只是让延迟不再导致**处理器利用率**下降（"the latency of the memory operation is not changed by multi-threading, it just no longer causes reduced processor load"）。
2. **每次访存之间的算术越多，隐藏延迟所需的线程就越少**（`P* = 1 + L/k` 随 `k` 增大而减小）。这正是"提高算术强度"在延迟侧的收益。

与现代机器对照：DRAM 延迟约 **248 个周期**（Kaby Lake @4 GHz）。若每 10 条指令一次访存，`P* = ceil(1 + 248/10) = 26` 个线程才能跑满一个核——这直接解释了**为什么 GPU 每个 SM 要挂 48–64 个 warp**（GTX 480 是 48，V100 是 64），也解释了**为什么 CPU 的 2-way SMT 只能拿到有限的收益**（2 个线程远远不够填满长延迟的等待）。反过来，如果代码**把数据全部放在寄存器里做大量复用**（`k` 很大），一个线程就够了。

4.4 带宽受限的稳态分析

设核每时钟执行 1 条数学指令；内存每时钟提供 `B` 字节；程序每 `k` 条数学指令需要一次 `m` 字节的访存。

稳态下（延迟已被充分隐藏）：

```
内存完成一次访存需要的时钟数  =  m / B
这段时间内核能做的数学指令数  =  k
=> 核的指令吞吐 = k / (m/B) = k·B/m   (条/时钟)
=> ALU 利用率  = (k·B/m) / (每时钟指令数) = k·B/(m·1)
```

数值算例（CS149 multicore2 的原例）：`B = 8 字节/时钟`，`m = 64` 字节（一次 load），`k = 3` 条数学指令：

```
一次 load 需要 64/8 = 8 个时钟
这 8 个时钟里核只能做 3 条数学指令
=> 吞吐 = 3/8 = 0.375 条/时钟   =>  ALU 利用率 = 37.5%
内存利用率 = 100%(它已经在满负荷送数据，不可能更快)
```

关键洞见：稳态利用率 `k·B/m` **只依赖"指令流里的访存密度"和"内存吞吐率"，与内存延迟、未完成请求数无关**。延迟只决定"需要多少并发请求才能把带宽填满"（Little 定律：`并发请求数 ≈ 带宽 × 延迟`）。用讲义虚构芯片的数字验证一下：`带宽 25 GB/s`、`延迟 100 周期 @ 3 GHz = 33 ns`，则 `需要的并发请求数 ≈ 25e9 × 33e-9 ≈ 833` 字节在途——**这就是为什么"延迟隐藏"需要成百上千个独立的访存**（对应 2.11 节"512 份独立工作"的量级）。

数值算例（带宽下限）：

- 一个 **256 MB** 的数组，只用 **20 GB/s** 的带宽做**一次遍历**（只读），至少需要 `256 MB / 20 GB/s = 12.8 ms`。
- 若是 `sinx` 这类"读一遍写一遍"的 kernel，`N = 6.4e7` 个 float（256 MB 输入 + 256 MB 输出 = 512 MB），至少需要 `512 MB / 20 GB/s = 25.6 ms`。
- 若机器只有 16 核 × 8 宽 SIMD = 128 lane @3 GHz，则单次遍历 6.4e7 个元素的算力时间为 `6.4e7 × 5.4 向量指令/元素 / (16 核 × 3e9) ≈ 7.2 ms`。**7.2 ms < 25.6 ms ⇒ 这个 kernel 是带宽受限的**，加速比上限被钉在 `25.6/7.2 ≈ 3.6x`（相对 1 核的算力时间），**不管你有 16 个核还是 128 个核**。

数据的现实感（讲义 slide 44 的"真实世界例子"，原样记录其算式）：任务是把数据从匹兹堡的数据中心搬到纽约（两地相距 370 英里 ≈ 6.5 小时车程）。讲义给出两条算式——`1e+11 B / 25 mb/s = 1.1 hours`（该项标注为 "100 PB"，算式中实际使用的是 1e11 字节）与 `1e+18 B / 25 mb/s = 1,267 years`（1 EB，即 1e18 字节）。两条算式内部是自洽的：数据量相差 1e7 倍，所需时间也正好相差 1e7 倍（1.1 小时 vs 约 1267 年）。结论是：**在极大规模下，"用网络搬数据"会输给"用卡车搬存储"**——这正是 AWS Snowmobile 这类服务存在的理由。这个类比在计算机内部同样成立：**带宽是系统中真正稀缺的资源，"减少数据移动"往往比"提高算力"更有效**（讲义 slide 68 的三条优化准则正由此而来），也与 CS149 支撑材料里"数据移动的能耗"（从 LPDDR 读 64 位 ≈ 1200 pJ，而一次浮点运算 ≈ 20 pJ）互相印证。

4.5 算术强度与 Roofline 模型（核心数值算例）

模型：给定机器的**峰值算力 `P_peak`（FLOP/s）**与**内存带宽 `BW`（Byte/s）**，一个算术强度为 `AI`（FLOP/Byte）的 kernel 的可达性能为

```
    P_achieved = min( P_peak ,  BW × AI )

两个区域:
  AI < AI*   => 带宽受限 (memory bound)   P = BW × AI
  AI > AI*   => 计算受限 (compute bound)  P = P_peak
拐点(机器平衡点):  AI* = P_peak / BW   (FLOP/Byte)
```

假设参数（取自讲义 slide 61 的"虚构多核芯片" + slide 47/65 的内存带宽）：

参数	取值	来源
核数	16	03_basicarch slide 61
每核 SIMD 宽度	8（AVX2 级，256 位）	03_basicarch slide 61
每核每时钟可发射的 SIMD 数学指令	1 条	03_basicarch slide 14/80
是否支持 FMA（一次算 2 flops）	是（按本讲"1 MUL-ADD per clock"的 GPU 描述口径）	03_basicarch slide 62
时钟频率	3.0 GHz	假设
内存带宽	25 GB/s（DDR3）	03_basicarch slide 47/65

峰值算力：

```
P_peak = 16 核 × 8 lane/核 × 2 FLOP/(lane·clk, 即 FMA) × 3.0e9 clk/s
       = 16 × 8 × 2 × 3.0e9 = 7.68e11 FLOP/s = 768 GFLOPS
```

拐点（机器平衡点）：

```
AI* = P_peak / BW = 768e9 / 25e9 = 30.7 FLOP/Byte
```

数值算例表（同一台机器上不同 kernel 的可达性能）：

Kernel	每字节的运算量（推导）	AI (FLOP/B)	可达 GFLOPS = min(768, 25·AI)	峰值占比	瓶颈
`C[i]=A[i]*B[i]`（逐元素乘）	1 flop / 12 B	0.083	**2.08**	**0.27%**	带宽
`C[i]=A[i]+B[i]`（逐元素加）	1 flop / 12 B	0.083	**2.08**	**0.27%**	带宽
`C[i]=fma(A[i],b,C[i])`（读+写 8 B）	2 flop / 8 B	0.25	**6.25**	**0.81%**	带宽
示例一的 `sinx`（读 4 + 写 4）	~2 flop / 8 B（按乘法计）	0.25	6.25	0.81%	带宽（除法器次之）
分块矩阵乘，块大小 T=64	`2N³ / (2N³/T · 4 B) = T/4`	16	**400**	**52%**	带宽
分块矩阵乘，块大小 T=123	T/4	30.75	**768**	**100%**	恰好在拐点上
分块矩阵乘，块大小 T=128	T/4	32	**768**	**100%**	计算

解读：

1. 逐元素向量运算（这正是讲义 slide 66 的思想实验）的 AI 只有 `0.083`，比拐点低了 **370 倍**，所以它**只能用到 0.27% 的峰值算力**。这不是"编译器不够聪明"，而是**物理约束**：要跑满 768 GFLOPS 需要 `768e9 / 0.083 ≈ 9.2 TB/s` 的带宽，而机器只有 25 GB/s——**缺 370 倍**。（讲义对 GTX 480 做的正是同一个计算：需要约 6.4 TB/s，实有 177 GB/s。）
2. **分块（blocking/tiling）是提高 AI 的标准手段**：把矩阵乘的块大小从 64 提到 123，AI 从 16 提到 30.7，跨过拐点，性能从 52% 跳到 100%。**注意 `T* = 4·AI* = 123` 这个结论**——它给了一个非常具体的"缓存块该切多大"的目标。
3. **为什么 GPU 比多核 CPU 快 7 倍（讲义 slide 66 的数字）**：两者的逐元素乘 kernel **都**是带宽受限的，所以**性能之比 ≈ 带宽之比**：`177 GB/s ÷ 25 GB/s ≈ 7.1x`。把讲义里的两个数字按同一公式算一遍即可复现（见 2.14 节的推导）。这是"带宽是唯一瓶颈"这一论断最有说服力的定量证据。
4. **反过来，低带宽的并行机器会被高带宽的机器在"低 AI"负载上碾压**，即使前者的峰值算力更高——这是异构计算（Lecture 20）与存储层次设计的核心动机。

4.6 第 4 节算例汇总

#	算例	参数	结论
1	芯片峰值算力	16 核 × 8 lane × 2 FMA × 3.0 GHz	**768 GFLOPS**
2	机器平衡点	768 GFLOPS / 25 GB/s	**AI* = 30.7 FLOP/Byte**
3	逐元素乘的可达性能	AI = 0.083	2.08 GFLOPS = **0.27% 峰值**；需 9.2 TB/s 才能跑满，实有 25 GB/s
4	分块矩阵乘拐点	AI = T/4	T=64 → 400 GFLOPS(52%)；**T≥123 → 768 GFLOPS(100%)**
5	GPU vs CPU 的 7 倍	177 GB/s vs 25 GB/s，两者都带宽受限	比值 **7.1x** ≈ 讲义 "7x faster"
6	GTX 480 带宽需求	480 MUL/clk @1.2 GHz × 12 B	需 **~6.4–6.9 TB/s**，实有 177 GB/s → **~3% 效率**
7	延迟隐藏所需线程	`P* = ceil(1 + L/k)`，L=12, k=3	**5 个线程**才 100% 利用率；k=6 时只需 3 个
8	DRAM 延迟下的线程需求	L≈248 cycle, 每 10 条指令一次访存	需 **26 个线程/核**；解释 GPU 为何挂 48–64 warp
9	带宽受限稳态利用率	B=8 B/clk, m=64 B, k=3 条数学	利用率 = k·B/m = **37.5%**，与延迟无关
10	带宽下限	256 MB 数组 @20 GB/s	单次遍历至少 **12.8 ms**
11	跑满芯片所需独立工作	16 核 × 8 lane × 4 线程	**512 份**独立工作
12	Amdahl	f = 5%	16 核 **9.14x**（上限 20x）；32 核仅 12.55x

---

5. 关键要点

1. **现代处理器的算力来自四种并行执行形式的叠加，而不是单条指令流变快**：**多核（Idea #1，线程级并行，软件创建线程）** + **SIMD（Idea #2，一条指令广播给多个 ALU，把控制开销摊薄）** + **硬件多线程（Idea #3，用别的线程的指令填满访存等待）** + **超标量/ILP（同一指令流内的独立指令，硬件自动发现）**。其中多核与超标量**不需要**指令流一致性，而 SIMD（无论显式还是隐式）**必须有**——这是后面所有 SIMD/GPU 代码优化的根本约束。
2. **"够多、够齐、有余量"是跑满吞吐型机器的三个必要条件**（CS149 的三条总结）：(a) 有**足够多的并行工作**去喂满所有执行单元（跨核、跨 lane）；(b) 成组的并行工作**要走同一条指令序列**，否则 SIMD 会被掩码浪费（最坏 1/8，GPU 上 1/32）；(c) 独立工作数必须**多于 ALU 数**，否则没有余量去隐藏内存停顿。定量版本就是："16 核 × 8 lane × 4 线程 = 需要 **512** 份独立工作"。
3. **延迟与带宽是两个性质完全不同的瓶颈，必须分开诊断**：**延迟**（100 周期量级）可以被**缓存降低成本、预取提前搬数据、硬件多线程藏起来**；**带宽**（20 GB/s 量级）**无法被任何延迟隐藏手段拯救**，只能靠"少访问"——同线程复用（时间局部性）、跨线程共享、以及**多做算术（"the math is free"）**。诊断口诀：**加线程后性能不再涨、内存利用率已达 100% ⇒ 你撞上的是带宽墙，不是延迟墙。**
4. **算术强度（arithmetic intensity = 数学运算数 / 数据访问字节数）是"能不能高效利用现代机器"的单一定量指标**，它的形式化就是 Roofline：`可达性能 = min(峰值算力, 带宽 × 算术强度)`。逐元素数组运算的 AI 低到 0.083，在 25 GB/s 的机器上只能拿到峰值的不到 1%；要跨过拐点（本讲虚构芯片的 `AI* = 30.7`），必须靠**分块/复用**把每个字节的运算量提高一到两个数量级。这也是为什么"GPU 比同代多核 CPU 快 7 倍"在低 AI 负载上可以精确地由 `177/25 ≈ 7.1` 解释。
5. **性能的上限要么由并行度决定，要么由资源带宽决定，要先用模型算清楚是哪一个**：`Work/Span` 给出并行度上限（`S(P) ≤ min(P, W/S)`），Amdahl 给出串行部分的封顶（`S(∞) = 1/f`），Roofline 给出带宽/算力的封顶。**在写优化之前先把这三个界算一遍**——本讲里 `sinx` 的并行度有 4 个数量级余量，所以它的瓶颈一定不在并行度，而在内存系统与除法器。

---

6. 常见陷阱与注意事项

- **陷阱 1：把"指令流一致性（instruction stream coherence）"与"缓存一致性（cache coherence）"混为一谈。** 讲义明确提醒二者无关。前者说的是"同一组被同时处理的数据是否走同一条指令序列"（影响 SIMD 效率）；后者说的是"多个核的缓存副本如何保持一致"（Lecture 10–12 的主题）。英文都叫 coherence，但它们**没有任何关系**，理解错会导致完全错误的优化方向。
- **陷阱 2：以为"多线程能解决一切访存问题"。** 硬件多线程**只隐藏延迟，不改变延迟，也不增加带宽**。它的代价是实打实的：需要额外的片上存储保存执行上下文；**延长任何单一线程的完成时间**（吞吐导向的取舍）；**需要程序里有比 ALU 数更多的独立工作**；**更多线程 → 更大的工作集 → 每线程分到的缓存更小 → 可能更频繁访问内存**。当程序已经带宽受限时，加线程会**毫无收益**（讲义原文：No amount of latency hiding helps this）。
- **陷阱 3：并行化 ≠ 向量化 ≠ 用满机器，三者必须分别验证。** 本笔记 3.2 节的实测是一个具体教训：**同一条循环，串行版本被 GCC 自动向量化（22.7 ms），放进 `#pragma omp parallel for` 后该循环没有被向量化（119.9 ms，1 线程）**，于是"并行版本"在 1 线程时比串行版慢 5.3 倍，表面加速比只有 0.18x。两个后果：(a) **加速比必须与"同代码质量"的基线比**，否则结论荒谬；(b) **必须主动检查向量化**——用 `-fopt-info-vec` / `-Rpass=loop-vectorize` 看报告，或反汇编确认 `vmulps` / `vfmadd` 是否真的生成（讲义对 explicit SIMD 的定义就是"能在二进制里看到向量指令"）。
- **陷阱 4：忽略数据依赖性而写出不确定结果。** 数据并行的合法性前提是"迭代之间无依赖、不写同一位置"。CS149 的例子极具代表性：`absolute_repeat`（每个迭代只写 `y[2i]`、`y[2i+1]`）是对的；而 `shift_negative`（当 `x[i]<0` 时写 `y[i-1]`）**输出未定义**，因为多个迭代可能写同一位置。**判断标准是"写集合是否互不相交、且读集合不被其他迭代写"**——`sinx` 之所以可以任意并行，正是因为第 `i` 个迭代只碰 `x[i]` 与 `result[i]`。
- **陷阱 5：忽略内存带宽与伪共享（false sharing）。** 两个独立的坑：(a) **带宽**——把串行代码改成并行后，如果每个线程都在流式扫描大数组，加速比会在"内存带宽饱和"处**停止增长**，此时继续加线程只会更慢（本笔记 4.4 节的 `k·B/m` 公式能预告这一点）。(b) **伪共享**——即使两个线程写的是**不同变量**，只要这两个变量落在**同一个 cache line** 上，缓存一致性协议就会让该 line 在两个核之间反复弹跳（cache line ping-pong），产生几十倍的性能损失。常见触发形态是"每个线程往一个全局数组的相邻下标写累加值"——**正确写法是把每线程的部分和放在各自独立的 cache line 上（padding 到 64 字节），或用 `reduction` 让编译器处理**（示例三就是这么做的）。
- **陷阱 6：把"平均情况"当成"最坏情况"，尤其在有分支的 SIMD 代码里。** SIMD 的代价不是"分支"，而是"**同一批数据的控制流不一致**"：`if/else` 会被**顺序执行两条路径并掩码**，最坏只有 1/8（GPU 上 1/32）的 ALU 做有用工作。因此写向量化代码时应尽量：(a) 用**无分支（branchless）**写法（`select`/条件移动而非跳转）；(b) 让数据**按类别排序或分组**后再处理，使同批数据的路径一致；(c) 记住**分支结束后会立刻恢复全宽执行**——所以"少量、短小的发散"是可以接受的，真正致命的是"长分支体内的高发散"。

---

7. 思考题（带答案）

思考题 1

讲义用 `sinx` 演示了一个从"1 个复杂核"到"2 个简单核"的转变：设复杂核跑一条指令流的速度是 1.0，而简单核只有 **0.75**。现在你有一段**完全没有并行**的代码，想让它在这台新机器上更快。请回答：

(a) 如果你什么都不做，直接在新机器上运行原来的串行程序，性能是原来的多少？ (b) 如果你把它改成 2 个线程、每个线程处理一半的数据，理论加速比是多少（相对于原来的复杂单核机器）？请分别按讲义的口径（吞吐/单位时间完成的工作量）和标准加速比口径（T_1/T_2）计算。 (c) 要把这个 2 核机器的算力完全用上，程序必须满足什么硬性条件？如果数组只有 10 个元素、每个元素的计算量很小，会发生什么？

【答案】

(a) **0.75x，即比原来慢 25%**。原因很直接：串行程序在 2 核机器上**只会用到 1 个核**（讲义原话：这个程序用 gcc 编译后会"run as one thread on one of the processor cores"），而每个简单核跑单条指令流只有复杂核的 0.75 倍速。**多核时代"不提并行"是负优化**——这是讲义 slide 15 用大字标出的结论。

(b) **两个口径的答案不同，要分清**： – 讲义口径（单位时间能完成的工作量，即吞吐）：2 个核各自以 0.75 的速度干活，所以 `2 × 0.75 = 1.5`，即**吞吐提升 1.5 倍**（相对于 1 个复杂核）。 – 标准加速比口径（`S = T₁/T₂`，其中 `T₁` 是**同一台新机器上跑串行**的时间）：`T₁ = 0.75`（用 1 个简单核），`T₂ = 0.75/2 = 0.375`（2 个核并行），所以 `S = 0.75/0.375 = 2.0x`。这里的 2.0x 是"用满 2 个核"的**完美**结果（Amdahl 中 `f = 0` 的情形）。 – **注意两者的区别**：`1.5` 是"新旧机器之间"的比较，`2.0` 是"新机器上串行 vs 并行"的比较。教科书里说"2 核理论加速比 2.0"，隐含的是跟本机的串行版本比；而"投资这一代新机器划不划算"要看 1.5。**混用这两个口径是性能报告中极常见的错误**（本笔记 3.2 节的实测表就踩到了"基线选错"这个坑：因为串行基线被自动向量化而快了 5 倍，所以看起来"加速比只有 0.18x"）。

(c) **硬性条件是要有足够多的、彼此独立的工作。** 具体到本讲，一台机器跑满所需独立工作数是各层并行度的乘积：如果这台机器是"2 核 × 每核 8 lane SIMD × 每核 2 个硬件线程"，则需要 `2 × 8 × 2 = 32` 份独立工作；讲义的虚构芯片是 `16 × 8 × 4 = 512` 份。**数组只有 10 个元素时，连"每核一个线程"都喂不饱**（10 < 32），更不用说 SIMD 的 8 个 lane 和硬件多线程的 2 个上下文——结果是绝大部分 ALU 空转，实际性能可能

还不如原来的单复杂核（因为简单核本身慢 0.75 倍，而且 `pthread_create` / `join` 的开销相对于 10 个元素的计算量是巨大的）。教训：**并行化有一个最小问题规模的门槛**（`N` 必须远大于"总 lane 数 × 每 lane 需要的独立迭代数"），这也是 Lecture 1 中"粒度（granularity）"概念的来源。

思考题 2

某 kernel 在一台机器上执行：机器有 **8 个核**，每核 **8 宽 SIMD**，**不支持 FMA**（一次只能 1 flop），主频 **2.5 GHz**，内存带宽 **20 GB/s**。kernel 对长度为 `N = 1e8` 的 `float` 数组做：`out[i] = a[i] * a[i] + 3.0f`（读 4 字节、写 4 字节）。

请回答：(a) 这个 kernel 的算术强度 AI 是多少？(b) 机器的峰值算力是多少 GFLOPS？机器平衡点 `AI*` 是多少？(c) 这个 kernel 的理论可达性能是多少 GFLOPS？是计算受限还是带宽受限？(d) 它的执行时间下限是多少毫秒？这个下限由什么决定？(e) 为了让这个 kernel 变成**计算受限**，最直接的手段是什么？请给出一个量化目标。

【答案】

(a) **AI = 2 flops / 8 bytes = 0.25 FLOP/Byte。** 推导：每个元素需要 1 次乘法（`a[i]*a[i]`）和 1 次加法（`+3.0f`），共 **2 flops**；内存流量是读 4 字节 + 写 4 字节 = **8 字节**。（若编译器把 `a*a+3` 融合成一条 FMA，指令数会减半，但**flops 数与字节数不变**，所以 AI 不变。）

(b) **峰值算力** `P_peak = 8 核 × 8 lane × 1 flop/(lane·clk) × 2.5e9 clk/s = 1.6e11 FLOP/s = 160 GFLOPS`。 **机器平衡点** `AI* = P_peak / BW = 160e9 / 20e9 = 8 FLOP/Byte`。

(c) `P_achieved = min(160, 20 × 0.25) = min(160, 5) = **5 GFLOPS**`。因为 `AI = 0.25 < AI* = 8`，所以它是**带宽受限（memory bound）**，只用到 `5/160 = 3.1%` 的峰值算力。**注意这个 3% 与讲义中对 GTX 480 估计的 "~3% efficiency" 是同一个数量级的现象**——低算术强度的逐元素运算是普适的带宽杀手，与机器是 CPU 还是 GPU 无关。

(d) 流量 `= 8 字节/元素 × 1e8 元素 = 8e8 字节 = 800 MB`。`t ≥ 800 MB / 20 GB/s = 0.04 s = **40 ms**`。**这个下限完全由内存带宽决定**，与核数、SIMD 宽度、主频无关——把核数从 8 加到 80 也不会让 40 ms 变短。（顺带验证：算力时间是 `2e8 flop / 160e9 = 1.25 ms`，远小于 40 ms，所以计算确实不是瓶颈。）

(e) 最直接的手段是**提高算术强度**：让每个从内存取来的字节被更多地复用（而不是想办法把计算做得更快）。量化目标：需要 `AI ≥ AI* = 8 FLOP/Byte`。由于内存流量 = 读 4 字节 + 写 4 字节（写通常不可复用），最现实的做法是**让每个读入的元素参与更多运算**——例如把 kernel 改成 `out[i] = f^k(a[i])` 这种"每个输入元素做 `k` 次运算"的形式，其中每次运算是 1 flop，则 `AI = (1 + k)/8`，要 ≥ 8 需要 `k ≥ 63`。这就是**循环融合/计算重用（compute reuse）**：把 64 个独立的逐元素 kernel（`a*a+3`、`a*a-2`、…）**合并成一个循环**，每个元素只读一次内存、却在寄存器里做 128 次运算，AI 从 0.25 提到 16，超过拐点，性能从 5 GFLOPS 升到 160 GFLOPS（**32 倍**）。另一个方向是做**分块（blocking）**让数据在缓存中被多次使用（如 4.5 节的分块矩阵乘，`T* = 4·AI* = 32`）。**注意优化的方向不是"更快的乘法"，而是"更少的访存"**——这正是讲义 "do more arithmetic: it's free" 的确切含义。

思考题 3

讲义最后留下一个思考题：**"你写了一个 C 程序，创建了 2 个 pthread，程序运行在一台'2 核、每核 2 个执行上下文、每时钟最多 2 条指令（其中一条是 8 宽 SIMD）'的处理器上。谁负责把你的 pthread 映射到处理器的线程执行上下文上？如果你有 5 个 pthread 呢？"** 请回答这三个问题，并解释"软件线程数"与"硬件执行上下文数"之间的关系。

【答案】

问题 1：谁负责映射？答案是操作系统（the operating system）。 讲义对这个问题的回答是明确的。程序员写的 `pthread_create` 只是向**操作系统**请求"给我一个可运行的线程实体"；操作系统**调度器（scheduler）**决定这个线程何时、在哪个**硬件执行上下文**上运行。处理器本身只提供"若干个能保存 PC + 寄存器的执行上下文槽位"，并**在每时钟从这些槽位中挑选指令执行**（这部分由硬件自主决定，不经操作系统）。因此存在**两级调度**： – **OS 级**：软件线程（可能成百上千个）→ 硬件执行上下文（本机只有 4 个），按时间片轮转，粒度为毫秒级，切换成本高； – **硬件级**：硬件执行上下文（4 个）→ ALU（每时钟 1–2 条指令），每个**时钟**都重新挑选，**零软件开销**（这正是 2.10 节"多线程隐藏停顿"能奏效的原因）。

问题 2：只有 2 个 pthread、却有 4 个执行上下文，怎么分配？ 合理答案是**把两个线程放到不同核**——即在 4 个上下文 `{核0-ctx0, 核0-ctx1, 核1-ctx0, 核1-ctx1}` 中，把线程 A 放到 `核0-ctx0`、线程 B 放到 `核1-ctx0`。原因是**这样能真正用到 2 个核的算力**（TLP）。反面做法是把两个线程都放在同一个核的两个上下文上：那样只用了 1 个核，虽然可以靠硬件多线程互相隐藏访存延迟（在带宽未饱和时利用率可能还不错），但**算力上限被砍了一半。一般原则：先按"核"分散、再考虑"同核内多个上下文"**（Linux 的调度器正是这么做的，它会尽量让线程优先占据空闲的物理核）。这也解释了为什么"软件线程数 = 硬件线程数"通常不是最优配置——**SMT 的收益来自"两个线程的停顿恰好互补"，而当两个线程都是纯计算密集型、ILP 又都很高时，SMT 几乎没有收益，反而共享 L1/L2 造成相互干扰。**

问题 3：如果是 5 个 pthread 呢？ 这时**软件线程数（5）> 硬件执行上下文数（4）**，操作系统必须**分时复用**：4 个上下文先各跑一个线程，第 5 个线程等某个上下文空出来（或被抢占）后再运行。**映射策略由 OS 决定，程序员无法直接控制**（只能通过亲和性 affinity 提示）。后果是： 1. **存在"可运行但未被执行的线程"**——这正是 2.10 节甘特图里的状态："During this time, this thread is runnable, but it is not being executed by the processor (the core is running some other thread)." **它在观感上等价于额外的停顿，但它与访存停顿有本质区别**：访存停顿可以用更多的独立工作填满，而"线程多于上下文"造成的等待**只能靠减少线程数或增大每线程的并行度来缓解**。 2. **超订（oversubscription）通常有害**：5 个计算

密集型线程抢 4 个上下文，会引入上下文切换开销与缓存抖动，总吞吐通常**低于**恰好 4 个线程。 3. **正确的做法**是让"软件线程数 ≈ 硬件执行上下文数"，然后**在每个线程内部用 SIMD + ILP（多累加器）来提高每时钟的指令数**——也就是本笔记 3.3 节实测的那个实验：在 16 线程之外，**把 1 条累加链改成 4 条，吞吐就提高 2.4–3.2 倍**，而这个提升**不需要任何额外的软件线程**，只需要给硬件提供更多 ILP。这就是讲义"要跑满 512 份独立工作"这一结论的实践含义：**独立工作的来源不只是"更多线程"，也可以是"同一线程内更多的独立指令"**。

Lecture 4: Parallel Programming Models

1. 章节标题与概述

Lecture 4: Parallel Programming Models

- **本讲核心问题**：什么是"并行编程模型（programming model）"？它和"硬件实现（implementation）"有什么区别？讲义把这条主线概括为一句话——**"Abstraction vs. implementation：把抽象与实现混为一谈，是这门课里最常见的困惑来源。"** 本讲用三种通信抽象（共享地址空间 shared address space、消息传递 message passing、数据并行 data parallel）与三种机器架构的对照，回答"程序员该怎样思考并行程序的结构"以及"这些抽象背后需要什么硬件支持、代价是什么"。讲义正文标题页把它称作 **"Lecture 3: Parallel Programming Abstractions (and their corresponding HW/SW implementations)"**（见配套材料说明），前一半以 ISPC 为例讲 SPMD/SIMD，后一半讲三大通信模型。
- **涉及的主要硬件/软件机制**：
- **软件侧**：ISPC 的 gang / programCount / programIndex / uniform / foreach、SPMD（单程序多数据）编程抽象、pthreads 线程、MPI 消息收发（send/recv + tag）、stream/gather/scatter 数据并行原语、以及编译器与并行运行时（compiler & parallel runtime）。
- **硬件侧**：SIMD 向量指令（SSE4 / AVX、AVX2 的 gather、AVX512 的 scatter）、SMP（"dance-hall" 组织）里的共享总线 / 多级网络 / 交叉开关（crossbar）、NUMA、片上网状/环状互连、fat tree 与 dragonfly 集群拓扑，以及"多核 + 多线程 + 超标量 + SIMD"的现代单芯片结构。
- **接口侧**：本讲反复强调的"系统接口"——OS 系统调用 API（如 `pthread_create()`）、ISPC 编译器产物（带 SIMD 指令的 `.o`）、MPI 库、以及数据并行语言（ISPC/OpenCL/CUDA）的 kernel 语义。
- **在并行计算知识体系中的角色**：上一讲（A Modern Multi-Core Processor）讲的是"硬件给了你什么并行能力"（SIMD、多核、多线程、缓存层次）；本讲把它翻过来讲"软件用什么抽象去使用这些能力"。它是全课程的**概念中枢**：之后讲 CUDA（数据并行 + 同一 core 内共享地址空间）、OpenMP（共享地址空间 + fork-join）、MPI（消息传递）、缓存一致性（共享地址空间的硬件前提）、同步与内存序（共享地址空间的正确性代价），都要回到本讲的三个模型与"抽象距离（abstraction distance）"这个判据。讲义最后的结论也很明确：**"实践中你必须能用多种方式思考"**，因为现代机器在不同尺度上提供不同类型的通信，不同模型在不同尺度上最贴合机器。
- **配套材料**：
- `lectures/04_progmodels.pdf`（对应抽取文本 `extracted/04_progmodels.txt`，共 63 页幻灯片）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 下直接下载（课程主页 https://www.cs.cmu.edu/~418/，日程表 https://www.cs.cmu.edu/~418/schedule.html，Fall 2026 日程中本讲位于 Aug 31，主题写作 "Parallel Programming Models"）。注意两个如实的细节：(a) 讲义第 1 页标题写作 "Lecture 3: Parallel Programming Abstractions"，与日程表中第 4 讲的主题名 "Parallel Programming Models" 略有差异，属讲义的编号/标题沿用现象；(b) 讲义页脚沿用历史学期（如 Fall 2025）字样，也属正常的讲义复用现象，不是错误。
- `cs149_supp/proghardware.txt`：Stanford CS149 Fall 2025 Lecture 11 "Programming Specialized Hardware for AI"（共 60 页）——**公开**的补充读物。它与本讲的接口在于"编程模型如何反映硬件能力"这条主线：TPU 的 systolic array（脉动阵列）把"低控制开销 + 高数据复用"做成硬件，代价是编程模型变成"编译期就知道的矩阵乘/卷积 + 激活"这几条关键指令；SambaNova SN40L 的可重构数据流（reconfigurable dataflow）把编程模型变成 map/zip/reduce/gather/scatter 的**数据流图 + metapipelining**；NVIDIA H100/B100 则把异步（TMA、TMEM、异步 MMA）暴露给程序员，逼出了 ThunderKittens 这类嵌入式 DSL。**注意：这部分属于补充视角，不是 CMU 15-418 讲义的原文内容**，本笔记中出现处都会显式标注。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属 **未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 历史学期 PDF（如 Performance Analysis/Profiling、Transactional Memory、AI in System Design 等 Fall 2026 尚未发布的讲义）位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- 本讲 Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。

---

2. 核心概念与硬件/软件架构图解

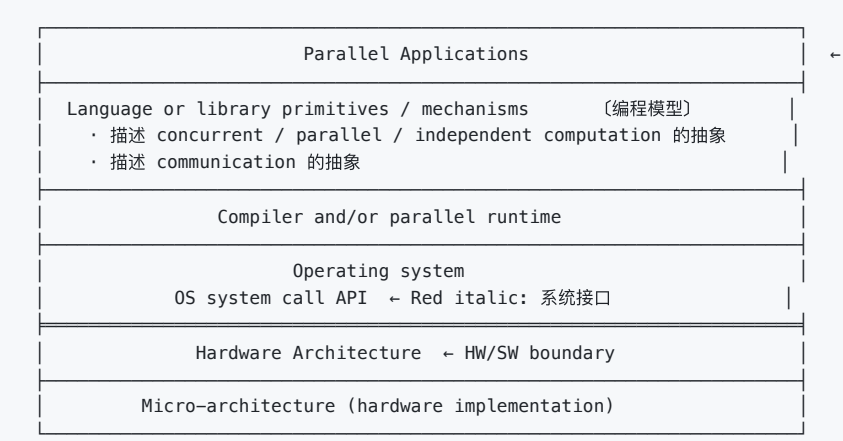
2.1 抽象 vs 实现：本讲的总纲

- **定义与目的**：**抽象（abstraction）** 是"程序员看到并据以思考的东西"，**实现（implementation）** 是"硬件/运行时真正做的事情"。二者可以差距很大，而性能差异恰恰藏在差距里。讲义明确说："conflating abstraction with implementation is a common cause for confusion in this course."
- **直观解释（"它是什么？"）**：把编程模型想成**餐厅菜单**，把实现想成**厨房**。菜单（抽象）写"宫保鸡丁"，你可能以为厨师是现炒的（顺序执行）；实际上厨房可能预先备好料、三位厨师并行分工、甚至在客人点单前就把热门菜下锅（乱序、推测、预取）。点菜的人（程序员）看到的是菜单，吃到的是厨房做出来的菜——**菜单不保证速度，厨房决定速度**。另一种类比：**交通规则 vs 道路施工**。规则（抽象）说"靠右行驶"；道路实际是几条车道、有没有立交（实现）才决定通行时间。
- **本讲的两个"抽象/实现"实例**（讲义用它做全讲的开场）：
- ISPC：程序员看到的是 SPMD 抽象（gang 里有 `programCount` 条逻辑指令流），实现是**一条 SIMD 指令流**（AVX 的 8 个 lane）。
- pthreads：程序员看到的是"线程"这个抽象，实现是 **OS 把线程映射到硬件执行上下文（execution context）**（讲义最后一页的思考题："who is responsible for mapping your pthreads to the processor's thread execution contexts? Answer: the operating system"）。

2.2 系统分层：编程模型在哪里

- **定义与目的**：讲义用一张分层图（"System layers: interface, implementation, interface, ..."）说明每一层都是"上一层的实现、下一层的接口"。**编程模型（programming model）** 位于最上面几层：它是"描述并发/并行/独立计算的抽象"加上"描述通信的抽象"，为程序员提供**思考程序结构的方式**。
- **直观解释（"它是什么？"）**：像**洋葱/套娃**：应用层说"我要并行"，语言/库把这句话翻译成原语，编译器与运行时把原语翻译成 OS 系统调用与机器指令，OS 把线程映射到硬件上下文，硬件再用流水线/SIMD/多核真正执行。每一层都只暴露一个"接口"给上层，把自己的"实现"藏起来。

图 1：系统分层（讲义 slide "System layers: interface, implementation, interface, ..."）



关键操作/性能特征：
- 每跨一层都要"翻译"：抽象语义不会被实现原样保留 ⇒ 性能不可从程序文本直接读出
- "abstraction distance"（抽象距离）越小 ⇒ 性能越可预测，但代码可移植性/灵活性越差

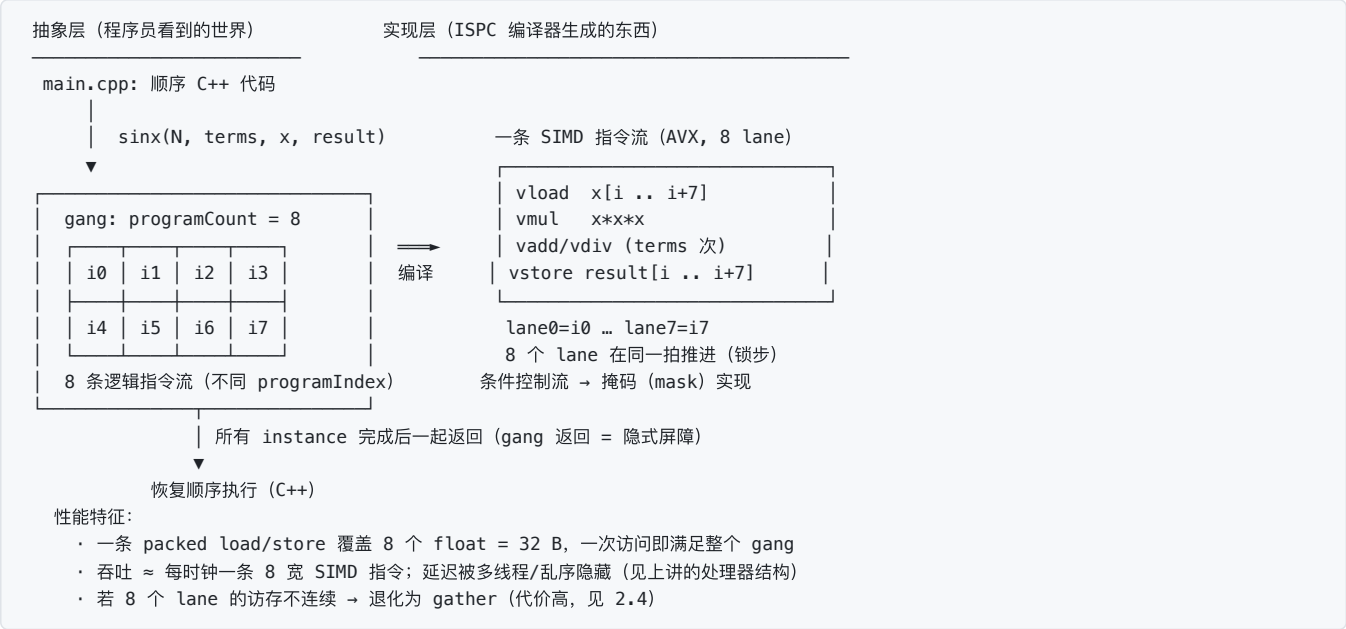
以 **pthreads** 为例，这张图会具体化为：Parallel Application → 抽象"thread" → pthread 库实现 → `pthread_create()`（系统调用 API）→ OS 内核线程管理 → x86-64 现代多核 CPU。以 **ISPC gang** 为例则变成：Parallel Application → ISPC 语言（调用 ISPC 函数、`foreach` 构造）→ ISPC 编译器 → x86-64（含 AVX 向量指令）→ **CPU 的单核**。注意后者：讲义特别提示，"仅 gang 抽象而言，这一切只跑在一个核上"；多核执行要靠 ISPC 的另一个原语 **task**。

2.3 ISPC 与 SPMD：gang、programCount、programIndex、uniform

- **定义与目的**：**ISPC（Intel SPMD Program Compiler）** 实现 **SPMD（single program multiple data，单程序多数据）** 抽象。调用一个 ISPC 函数会**生成（spawn）一个 gang**，gang 中的每条逻辑指令流称为一个 **program instance**；所有 instance 并发运行同一段 ISPC 代码，返回时所有 instance 一起完成。
- `programCount`：gang 中同时执行的 instance 数（一个 **uniform** 值，所有 instance 相同）。
- `programIndex`：当前 instance 在 gang 中的编号（一个 **varying** 值，各 instance 不同）。

- `uniform`：类型修饰符，表示"所有 instance 值相同"。讲义原话：**"Its use is purely an optimization. Not needed for correctness."**（纯粹是优化手段，正确性不依赖它）——但它是让实现细节"从抽象里透出来"的关键，例如 `uniform int denom` 可以让分母在向量化后只存一份。
- `foreach (i = 0 ... N)`：声明"这些迭代由 gang 中的 instance 协作完成"，即 **map（映射）**；ISPC 当前实现采用**静态交错（interleaved）分配**，但抽象本身允许其他分配方式。
- **直观解释（"它是什么？"）**：把 gang 想成**合唱团（chorus）**：`programCount` 是团员人数，`programIndex` 是团员编号，他们共用**同一份乐谱**（同一指令流）、但在**各自的声部/歌词**（各自的数据）上唱。指挥（编译器）把乐谱翻译成"一条 SIMD 指令"，八个团员在同一拍上各唱自己的词。而 `uniform` 就像"全体一起唱的那句"——大家词一样，可以只看一份谱。另一种类比：**同一个操练口令下的队列**。
- **性能要点（抽象 vs 实现）**：程序员"认为"自己 spawn 了 `programCount` 条指令流；编译器把 gang 实现为 **SSE4/AVX 向量指令**，并用**掩码（masking）** 处理条件控制流。**gang 的大小 = 硬件的 SIMD 宽度（或 SIMD 宽度的小整数倍）**。这带来两个直接结论：(1) 同一个 gang 里的 instance 是**同步推进**的，因此它们之间不存在"交错执行"；(2) 条件分支若不同 lane 走不同路径，就会串行化（谓词化/掩码）——这是后面性能分析的关键。

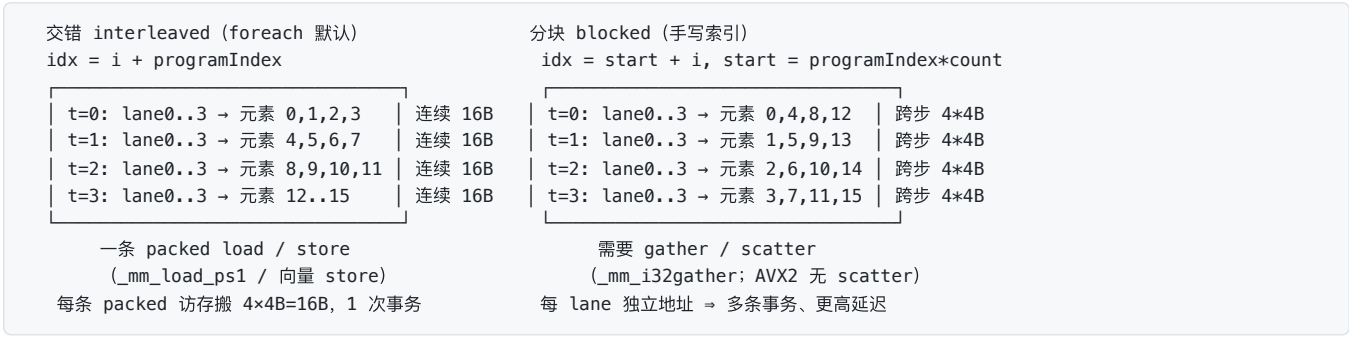
图 2：ISPC 的 SPMD 抽象与 SIMD 实现（软件执行模型）



2.4 交错分配 vs 分块分配：packed load vs gather

- **定义与目的**：在 SPMD 里"哪个 instance 处理哪些迭代"直接决定**访存模式**，进而决定**用哪条 SIMD 指令**。
- **交错分配（interleaved）**：第 *t* 轮里，instance *k* 处理元素 `idx = t*programCount + k`。相邻 lane 访问**相邻地址**→ 一条 **packed load**（讲义举 `_mm_load_ps1`）即可完成全部 instance 的取值。这是 `foreach` 默认的行为。
- **分块分配（blocked）**：instance *k* 处理 `[k*count, (k+1)*count)`。第 *t* 轮里各 lane 访问地址相差 `count`（跨步 stride）→ 必须用 **gather**（讲义举 `_mm_i32gather`），是"更复杂、更昂贵"的指令。
- **直观解释（"它是什么？"）**：交错像**发牌**——一张张轮流发给四个人，每轮四个人手里的牌凑起来正好是**连续的一段**（一条 cache line 就够）；分块像**切蛋糕**——每个人抱走一大块，四个人同时伸手去拿的是**相隔很远**的四块，得跑四趟。**发牌（交错）对缓存与向量单元都友好；切蛋糕（分块）适合"每人一份独立任务"的直觉，但对 SIMD 不友好。**
- **讲义给出的硬件时间线（很有用的事实）**：gather 指令 **2013 年随 AVX2 才有**；**AVX2 不支持 SIMD scatter**（scatter 只能用标量循环模拟）；scatter 指令出现在 **AVX512**；**GPU 上硬件支持的 gather/scatter 是存在的**，但相对连续向量的 load/store 仍然昂贵。

图 3：交错 vs 分块的访存模式（programCount = 4，处理 16 个元素）



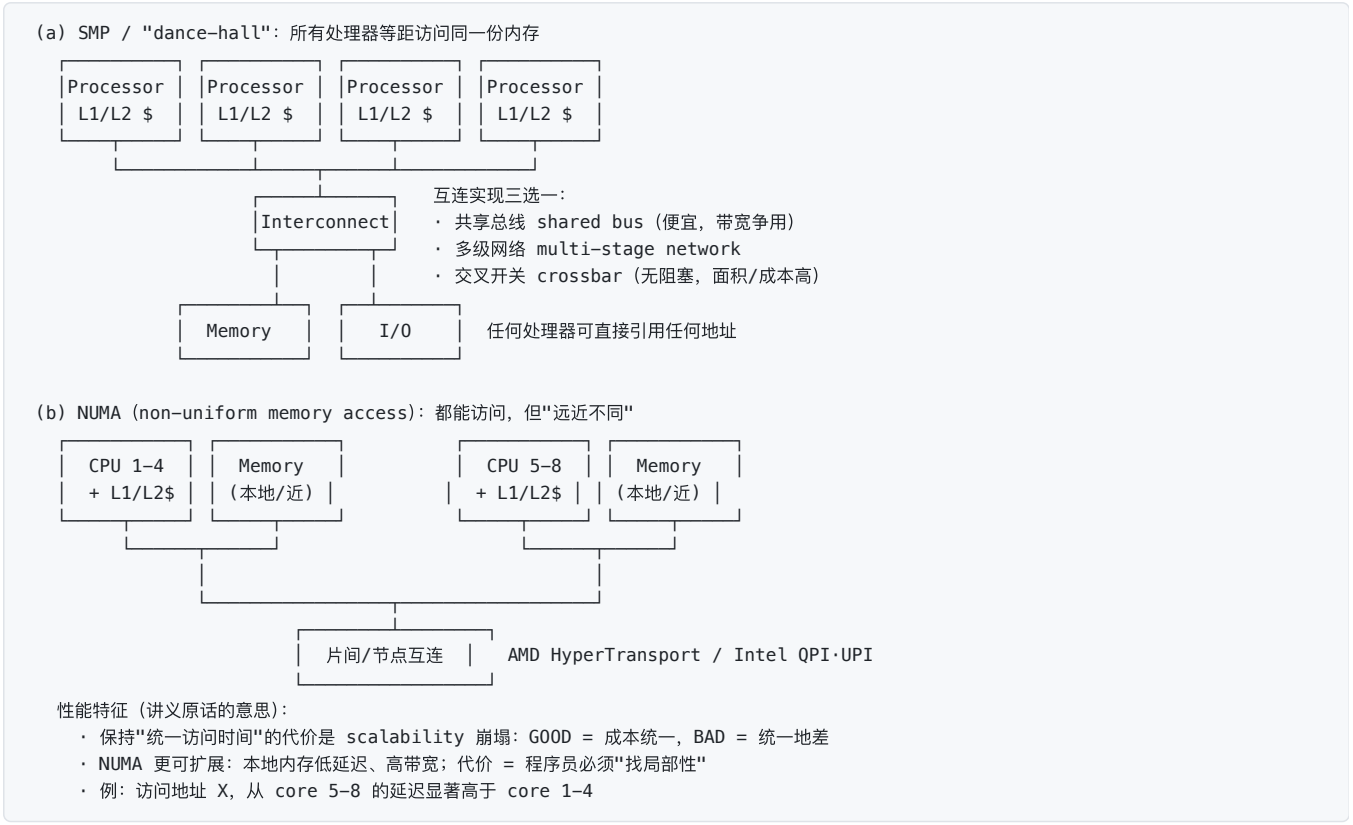
表：两种静态分配的对比（本讲核心工程结论之一）

维度	交错分配 interleaved	分块分配 blocked
索引式	`idx = i + programIndex`，`i += programCount`	`idx = start + i`，`start = programIndex*count`
每轮 lane 的地址	相邻（连续 `programCount` 个元素）	跨步 `count` 个元素
需要的 SIMD 访存	packed load/store（一次事务）	gather（AVX2）/ scatter（AVX512）
访存效率	高：一条指令覆盖整个 cache line 段	低：地址不连续，指令更复杂更昂贵
缓存友好性	好（空间局部性被 lane 同时利用）	差（多个远端区块同时活跃，TLB/预取受压）
ISPC 中的来源	`foreach` 的当前实现	手写索引循环
适用场景	元素级 map（逐元素独立计算）	每 lane 需要独立长任务、或本就无局部性时

2.5 通信抽象之一：共享地址空间（shared address space）

- **定义与目的**：线程通过**读写共享变量**通信；通信是**隐式的**（隐含在 load/store 里）；同步原语（lock、semaphore）本身也是共享变量。讲义定位它是**顺序编程的自然延伸**："In fact, all our discussions in class have assumed a shared address space so far!"
- **直观解释（"它是什么？"）**：讲义给的类比是**一块大公告板（bulletin board）**：谁都可以往上贴，谁都可以看；线程 1 写 X，稍后线程 2 读 X 就"看到"了更新。更生活化的补充类比：**合租公寓的公共冰箱 + 便利贴**——方便，但没人规定"谁先拿"、"你看到的是不是最新那张"，需要额外的规则（锁）才能保证不出乱子。**这就是共享地址空间的根本特征：极其自然，但也极其容易"搬起石头砸自己的脚"（讲义原话：natural way of programming, but can shoot yourself in the foot easily）。**
- **硬件实现（关键点）**：**任意处理器都能直接引用任意内存地址**。典型组织是 **SMP（symmetric multiprocessor）"dance-hall" 结构**：处理器各自带本地 cache，通过互连（总线、多级网络、交叉开关）访问集中式内存。讲义强调一个前提："cost of accessing an uncached memory address is the same for all processors"（**caching 会引入非均匀访问时间**，这正是后面缓存一致性讲座的内容）。**性能特征**：SMP 的优点是"成本统一"，缺点是"统一地差"（memory is uniformly far away）——所以 SMP 不可扩展，规模一大就必须转向 NUMA 或集群。

图 4：共享地址空间的硬件实现（SMP "dance-hall" 与 NUMA）

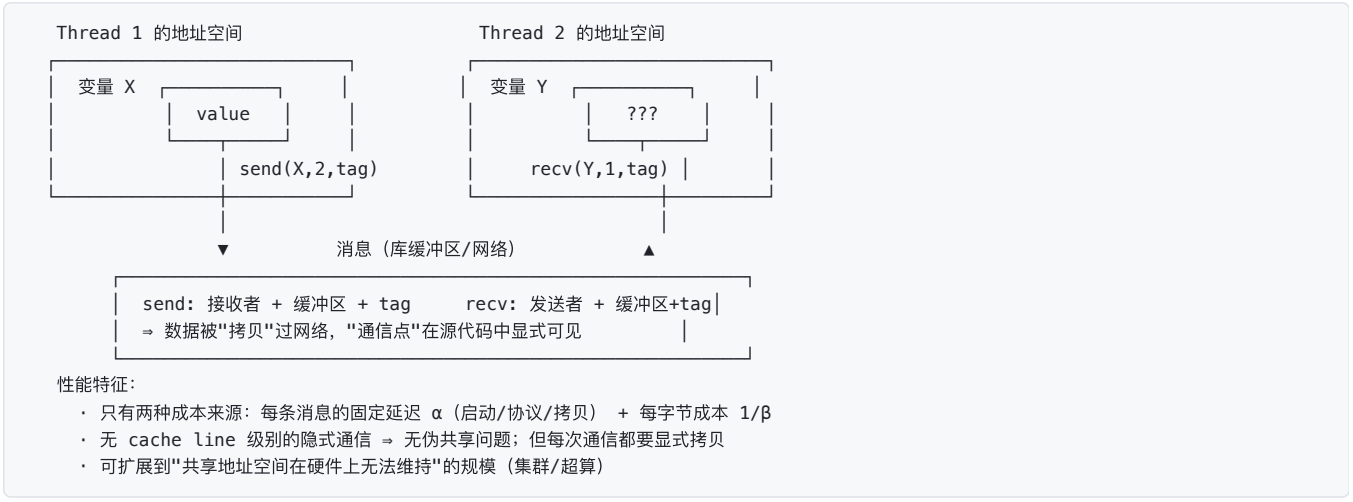


讲义列出的真实机器（都属"共享地址空间"这一类家族）：Intel Core i7 四核（**片内互连是 ring**）、AMD Phenom II 六核、SUN Niagara 2 (UltraSPARC T2) 八核（**crossbar 交叉开关，面积约等于一个核**）、**SGI Altix UV 1000**（256 blade × 2 CPU × 8 核 = **4096 核，单一共享地址空间**，互连为 **fat tree**）、以及 LLNL **El Capitan**（11,424/11,520 节点异构集群，节点内 4× AMD Instinct MI300A APU，每 APU 24 核 Zen 4 + 228 CU，**统一地址空间 CPU+GPU，5.3 TB/s**，节点间 **Infinity Fabric 缓存一致 all-to-all** + Slingshot 200 Gb/s NIC，网络拓扑为 **dragonfly**）。

2.6 通信抽象之二：消息传递（message passing）

- **定义与目的**：线程各自拥有**私有地址空间**，只能靠**收发消息**交换数据：`send` 指定接收者、要发的缓冲区、可选的消息标识（**tag**）；`recv` 指定发送者、存放数据的缓冲区、可选的 tag。**发送消息是两个线程之间交换数据的唯一途径**。最流行的软件库是 **MPI（message passing interface）**。
- **直观解释（"它是什么？"）**：像**两个不通气的工作间之间的传递窗/快递**：你没法直接伸手去拿别人的东西，必须打包、贴单号（tag）、投递，对方凭单号签收。好处是**账目清楚**——读代码就能看出"什么时候发生了通信"（讲义：can read program and see where the communication is）；坏处是**第一次写对更难**（要自己管缓冲、匹配、分区），但**结构上的强制约束往往帮你更快得到可扩展的正确程序**。
- **硬件实现与性能特征**：硬件**不必**实现全系统范围的 load/store，只要能通信即可——所以可以用商品机 + 网络（讲义例子：Infiniband 上的 workstation 集群、IBM Blue Gene/P 超算）。关键性能事实：当消息传递跑在"硬件其实是共享地址空间"的机器上时，**"发消息"就是内存拷贝**（从消息库缓冲区拷出/拷入），这决定了它的开销模型是"延迟 + 带宽"（详见第 4 节）。

图 5：消息传递模型（两个私有地址空间 + 唯一通道）



2.7 通信抽象之三：数据并行（data parallel）与 stream 模型

- **定义与目的**：**对集合（collection）中的每个元素施加同一个函数**，且不同元素之间**相互独立**。历史形态：80 年代 SIMD 巨型机（**Connection Machine CM-1/CM-2：数千处理器 + 单一指令译码单元**）、Cray 向量处理器（`add(A,B,n)` 是"一条指令处理长度为 n 的整个向量"）、Matlab（`C = A + B`）。今天的常见形态是 **SPMD + map**：`map(function, collection)`，函数作用在每个元素上，**同步隐含在 map 结束处**（map 返回时函数已作用到所有元素）。
- **直观解释（"它是什么？"）**：像**流水线盖章**：一叠纸每张都盖同一个章，纸与纸之间不需要商量；也像**考场上的同一份卷子**：每个考生（元素）做同一套题（同一个函数），彼此不通信（"no communication among distinct function invocations"）。**map 的语义允许实现以任意顺序、包括并行地调度各次调用**——这就是数据并行"能自动并行"的根本原因。
- **ISPC 里的数据并行**：`foreach` 就是 map，循环体就是 function。但讲义提醒一个关键细节：**"collection" 在 ISPC 里不是一等概念**——它由程序里的数组索引逻辑**隐式**定义，ISPC 里**没有**"把这个代码 map 到这个数组的所有元素上"这种语义的操作。三个例子清楚展示了这一点：
- `absolute_value`：`y[i] = |x[i]|` → 最标准的逐元素 map。
- `absolute_repeat`：`y[2i] = y[2i+1] = |x[i]|` → 仍然是合法 ISPC 程序，但"把循环体 map 到已有集合上"的直觉就变得别扭（一个迭代写两个输出位置）。
- `shift_negative`：`if (i>=1 && x[i]<0) y[i-1] = x[i]; else y[i] = x[i];` → **这个程序是非确定的（non-deterministic）！** 多个迭代可能写同一内存位置，而数据并行模型**不规定迭代执行顺序**，也**不提供细粒度互斥/同步原语**（它本就不是为这种结构设计的）。
- **"更正统"的数据并行：stream 编程模型**：显式定义**集合（stream）** 与**无副作用函数（kernel）**，把每次调用的输入/输出/临时量视为**私有地址空间**。讲义（用自造的语法）给出：`stream<float> x(N), y(N); // 定义集合 absolute_value(x, y); // 把 kernel map 到集合上 void absolute_value(float x, float y) { if (x<0) y = -x; else y = x; }` 收益（讲义明确列出）：函数**真的无副作用**（写不出非确定程序）；**数据流对编译器已知**（每次调用的输入输出提前可知 → 可以做 **prefetch 预取隐藏延迟**）；**生产者-消费者局部性提前可知**（可把前一个 kernel 的输出直接喂给后一个 kernel，中间值留在**片内 buffer/cache**、根本不写回内存，**省带宽**）。 代价（"正统"数据并行的阿喀琉斯之踵）：**需要一套算子库来描述复杂数据流**，而一旦数据流稍复杂，就得"cross fingers and hope compiler is intelligent enough"——讲义原话："If I just had one more operator..."。
- **两个关键通信原语：gather / scatter**。`stream_gather(input, indices, tmp)` 按索引收集，`stream_scatter(tmp, indices, output)` 按索引写散。ISPC 等价写法就是把 `input[indices[i]]` 或 `output[indices[i]]` 写进 `foreach` 里。

图 6：stream/数据并行的融合（kernel fusion）与片内传递

```
朴素三步（每个 kernel 结束后中间结果落回 DRAM）：
   input ──►┌─────┐──► tmp（写 DRAM）──►┌─────┐──► output
            │ foo │                     │ bar │
            └─────┘                     └─────┘
   访存量 ≈ 2N(读 input) + 2N(写 tmp) + 2N(读 tmp) + 2N(写 output)

融合后（编译器知道数据流 ⇒ 生产者–消费者局部性已知）：
   input ──►┌─────┐──►〔on-chip buffer / cache〕──►┌─────┐──► output
            │ foo │        tmp 从不回 DRAM        │ bar │
            └─────┘                               └─────┘
   访存量 ≈ 2N(读 input) + 2N(写 output) ⇒ 省掉 4N 字节的 DRAM 往返

等价的手写形式（讲义给出）：
   for (i=0; i<N; i++) output[i] = bar(foo(input[i]));
 性能特征：省的是【DRAM 带宽】而不是【算力】⇒ 对算术强度低的 kernel 效果巨大
```

2.8 模型与机器：对应关系是"模糊的（fuzzy）"

- **定义与目的**：讲义用一整页强调：**编程模型（描述程序的抽象）与机器类型（硬件的实现）之间不是一一对应**。
- 在"硬件实现共享地址空间"的机器上实现**消息传递**很常见：发消息 = 从消息库缓冲区**拷贝内存**；收消息 = 从库缓冲区**拷回来**。
- 在"硬件不支持共享地址空间"的机器上，也能用（效率较低的）**软件**实现共享地址空间抽象：把含共享变量的页标记为 **invalid，缺页异常处理程序（page-fault handler）** 发出相应的网络请求把页取来（这属于共享虚拟内存/软件 DSM 的做法）。
- 因此必须始终区分："**什么是编程模型（用来描述程序的抽象）？什么是硬件实现？**"
- **直观解释（"它是什么？"）**：像**语言与口音**。你可以用普通话（抽象）在电话里说（一种实现），也可以面对面说（另一种实现）；反过来，你也可以用英语（另一种抽象）在同一个电话里说。**"说什么"和"怎么传"是两件独立的事**。
- **工程推论（讲义总结页的原则）**：三种模型施加的**限制（restrictions）** 是有意设计来"反映并行化与通信成本的现实"的：
- 共享地址空间机器：硬件支持任意处理器访问任意地址；
- 消息传递机器：硬件可能提供加速 send/recv/缓冲的机制；
- **"abstraction distance" 应当尽量小以让性能可预测，但要足够大以保证代码的灵活性与可移植性。**

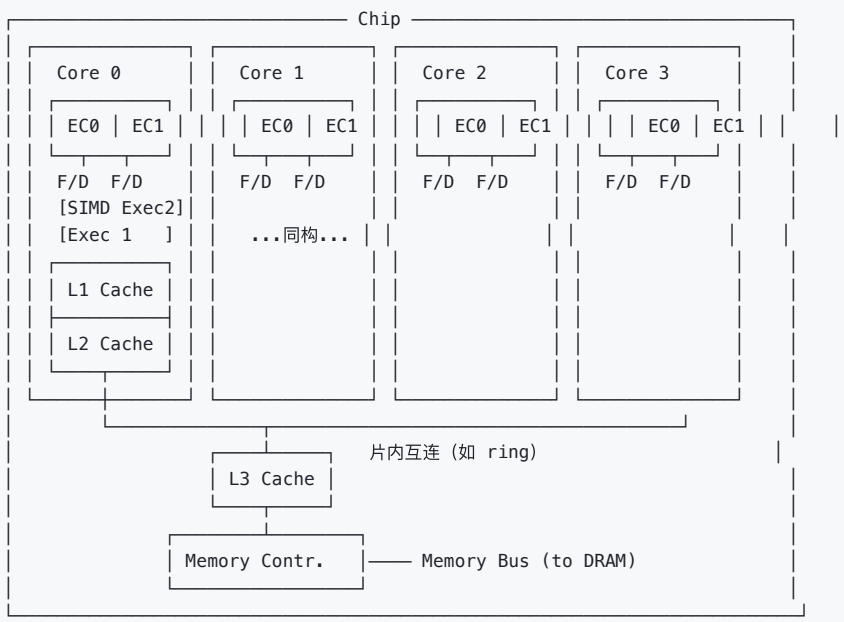
表：三种通信抽象的对比（本讲的核心表格）

维度	共享地址空间	消息传递	数据并行（map / stream）
通信方式	读写共享变量（隐式）	显式 send / recv（带 tag）	无（迭代间基本不通信）
程序结构	约束最少：任意线程可读写任意变量	结构最强：所有通信都是消息	计算结构最刚性：同一函数施加于集合
同步	锁 / 信号量 / 屏障等显式原语	消息的收发天然配对	隐含在 map 结束处（隐式屏障）
硬件要求	任意处理器可直接 load/store 任意地址	只要能通信（无需全局 load/store）	多数实现假设有共享地址空间来取输入/放结果
第一次写对	容易（顺序编程的自然延伸）	较难（缓冲、匹配、分区）	容易（但语义限制会"卡住"复杂写法）
性能可预测性	差：所有 load/store 看起来一样，实际成本天差地别	好：通信点可见，成本=延迟+带宽	好：数据流已知，可预取/融合
主要陷阱	数据竞争、伪共享、NUMA 远端访问、锁争用	死锁、消息匹配错误、拷贝开销	非确定写、需要"再来一个算子"
代表实现	pthreads、OpenMP、CUDA 的 block 内	MPI	ISPC `foreach`、OpenCL/CUDA kernel、stream 库

2.9 单芯片尺度：程序如何落到"四核 + SMT + SIMD"上

- **定义与目的**：讲义最后 5 页把前面的抽象收回到硬件：程序里的并行究竟落在哪个硬件资源上？它给出了一台"教学用处理器"的完整结构：**4 个核，每核 2 个执行上下文（2-way multithreading），每时钟最多 2 条指令，其中一条是 8 宽 SIMD**；每核有私有 L1/L2，共享 L3 与内存控制器。
- **直观解释（"它是什么？"）**：把核想成**厨房里的灶台**，执行上下文（execution context）是**同时备着的多口锅**：一口锅等水开（内存延迟）时，厨师去照看另一口锅（硬件多线程隐藏延迟）；"每时钟 2 条指令"是**两只手同时干活**（超标量）；"8 宽 SIMD"是**一次切八份同样的菜**（数据并行）。而 OS 是**排班经理**，把程序员交来的线程（pthreads）分配给这些"锅/灶台"。

图 7：四核 × 2 执行上下文 × 8 宽 SIMD 的单芯片结构（讲义最后一页的结构）

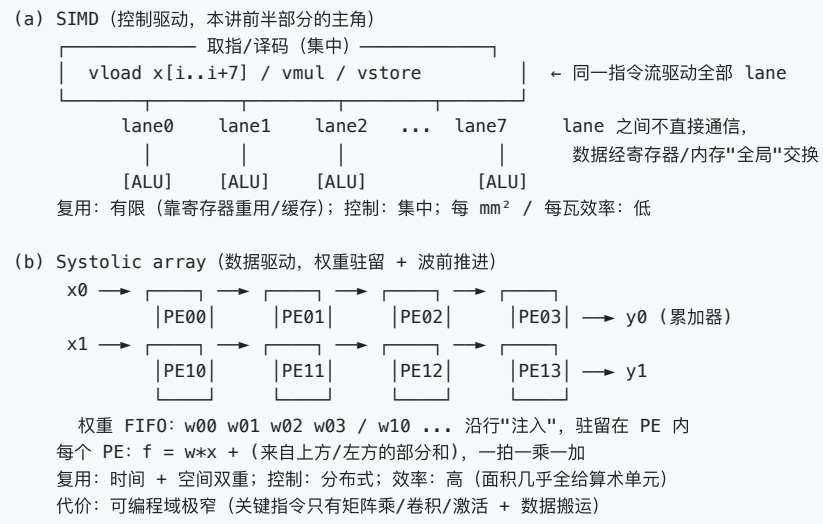


规格：4 核 × 2 执行上下文（最多 8 个活跃线程）× 每核每时钟 ≤2 条指令
（其中一条是 8 宽 SIMD 指令）
性能特征：
· 单核吞吐上限 = 1 条 8 宽 SIMD/时钟（数据并行）+ 1 条标量/时钟（指令级并行）
· 内存延迟靠"另一个执行上下文"与乱序执行隐藏（延迟隐藏，不是带宽提升）
· 4 核共享 L3 与 DRAM 带宽 ⇒ 多核扩展的上限常常是【带宽】而非【算力】
讲义思考题（本讲结论的收口）：
· 两个 pthread 如何映射到 4 个执行上下文？→ 由 OS 决定
· 若 spawn 五个 pthread 呢？→ 出现"线程数 > 上下文数"的过订阅（oversubscription）

2.10 补充视角（Stanford CS149，非 418 讲义原文）：把编程模型"钉"在硬件能力上

- **定义与目的**：CS149 的补充材料讲**专用硬件（specialized hardware）如何反向塑造编程模型**，与本讲"抽象必须反映硬件通信成本"的主题完全同构：
- **Google TPU**：用 **systolic array（脉动阵列）** 高效做稠密矩阵乘。芯片面积中**算术单元约占 30%，控制逻辑面积占比很低**；关键指令只有寥寥几条：read host memory、write host memory、read weights、`matrix_multiply` / `convolve`、`activate`。也就是说，**"编程模型"被压缩成了"喂数据 + 矩阵乘 + 激活"**——极致效率的代价是极小的可编程域。
- **脉动阵列 vs SIMD（讲义补充材料中的对比表）**：SIMD 是**控制驱动（instructions）**、复用有限、通信走全局（寄存器/内存）、控制集中；systolic array 是**数据驱动（wavefront 波前）**、具有时间与空间复用、通信只在**相邻 PE 之间**（局部）、控制**分布式**。
- **数据流（dataflow）编程模型**：SambaNova SN40L 的 RDU 用 **PCU（Pattern Compute Unit）+ PMU（Pattern Memory Unit）+ mesh 开关** 组成可重构数据流阵列；编程模型是 **map / zip / reduce / gather / scatter 等可组合原语**，再经过 **tiling → parallelization → metapipelining → place & route → codegen** 落到硬件。**Metapipelining（元流水线）** = "pipeline of pipelines"：把循环体拆成流水级、级间用**双缓冲（double buffer）** 传递中间数据，从而**重叠多个循环迭代**并容忍各级耗时不均。
- **异步（asynchrony）与 DSL**：H100/B100 把 Tensor Core、TMA（Tensor Memory Accelerator）、TMEM、异步 MMA 暴露出来，编程复杂度飙升，于是出现 **ThunderKittens / Mosaic / CuTe-DSL** 这类嵌入式 DSL：设计原则是"16×16 tile 作为原始数据类型"、"处处异步"、"提供高层 GPU 协作模式（生产者–消费者）"。
- **直观解释（"它是什么？"）**：这是**"为特定活法定制的流水线车间"**。通用 CPU 像**万能工具箱**（什么都能干，效率一般）；systolic array 像**专门压铸某零件的自动线**（快、省电，但只能干这一件事）；可重构数据流像**可重排的乐高传送带**，靠编译器当"工艺工程师"把计算摊在空间上。补充材料里的 "Hardware Lottery"（Sara Hooker）说得更直白：**某个研究想法胜出，往往是因为它适配了当时的软硬件，而不是因为它普适地更优。**

图 8（补充）：SIMD 执行 vs 脉动阵列（systolic array）数据流



---

3. 代码示例与性能分析

3.1 示例 1：ISPC 的 sin(x)——SPMD/SIMD 与交错/分块访存

讲义用同一个 `sinx` 函数贯穿前半讲：先给 C 版本，再给 ISPC（交错版）、ISPC（分块版）、以及用 `foreach` 提升抽象层次的版本。

```
// ============ 文件: sinx.ispc ============
// 编译 (生成 SIMD 目标文件与 C 头文件):
//   ispc --target=avx2-i32x8 -O3 sinx.ispc -o sinx_ispc.o -h sinx_ispc.h
//  (-O3 为 release 优化; --target=avx2-i32x8 表示 AVX2、8 宽 32-bit lane,
//   即 gang 大小 programCount = 8)

export void sinx(uniform int N, uniform int terms,
                 uniform float x[], uniform float result[])
{
    // 交错分配 (ISPC 的默认/foreach 语义): 交错访问相邻元素
    // 假设 N % programCount == 0
    foreach (i = 0 ... N) {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        uniform int denom = 6;      // 3!   —— uniform: 8 个 instance 共用一份
        uniform int sign  = -1;     // uniform: 纯优化, 不影响正确性
        for (uniform int j = 1; j <= terms; j++) {
            value  += sign * numer / denom;
            numer  *= x[i] * x[i];
            denom  *= (2 * j + 2) * (2 * j + 3);
            sign   *= -1;
        }
        result[i] = value;          // 一条 packed store 覆盖 8 个 float
    }
}

// ---- 等价的"手写交错版" (不用 foreach, 自己算索引, 讲义的做法) ----
export void sinx_interleaved(uniform int N, uniform int terms,
                             uniform float x[], uniform float result[])
{
    for (uniform int i = 0; i < N; i += programCount) {
        int idx = i + programIndex;          // programIndex: 非 uniform (varying)
        float value = x[idx];
        float numer = x[idx] * x[idx] * x[idx];
        uniform int denom = 6;
        uniform int sign  = -1;
        for (uniform int j = 1; j <= terms; j++) {
            value += sign * numer / denom;
            numer *= x[idx] * x[idx];
            denom *= (2 * j + 2) * (2 * j + 3);
            sign  *= -1;
        }
        result[idx] = value;
    }
}

// ---- 分块版: 每 instance 负责连续一段, 访存变成跨步 ⇒ 需要 gather ----
export void sinx_blocked(uniform int N, uniform int terms,
                         uniform float x[], uniform float result[])
{
    uniform int count = N / programCount;    // 假设整除
    int start = programIndex * count;        // varying
    for (uniform int i = 0; i < count; i++) {
        int idx = start + i;                 // 各 lane 的 idx 相差 count ⇒ gather
        float value = x[idx];
        float numer = x[idx] * x[idx] * x[idx];
        uniform int denom = 6;
        uniform int sign  = -1;
        for (uniform int j = 1; j <= terms; j++) {
            value += sign * numer / denom;
            numer *= x[idx] * x[idx];
            denom *= (2 * j + 2) * (2 * j + 3);
            sign  *= -1;
        }
        result[idx] = value;
    }
}
```

```
// ============ 文件: main.cpp（顺序 C++ 侧）============
// 链接: g++ -O3 -mavx2 main.cpp sinx_ispc.o -o sinx
#include "sinx_ispc.h"      // ISPC 编译器生成
#include <cstdio>
#include <cstdlib>
#include <chrono>

int main() {
    const int N     = 1 << 24;      // 16M 个 float = 64 MB 输入
    const int terms = 5;

    float* x      = new float[N];
    float* result = new float[N];
    for (int i = 0; i < N; i++)                  // 初始化（略：填 [-1,1] 的随机值）
        x[i] = (float)((i % 1000) - 500) / 500.0f;

    auto t0 = std::chrono::steady_clock::now();
    sinx(N, terms, x, result);                   // 调用 ISPC 函数 ⇒ spawn 一个 gang
    auto t1 = std::chrono::steady_clock::now();

    double sec = std::chrono::duration<double>(t1 - t0).count();
    printf("N=%d terms=%d  time=%.3f ms\n", N, terms, sec * 1e3);
    printf("result[0]=%f  result[N-1]=%f\n", result[0], result[N - 1]);
    delete[] x; delete[] result;
    return 0;
}
```

【代码做什么？】

1. `main.cpp` 顺序执行：分配两个长度为 N 的 `float` 数组（共 128 MB），初始化 `x`。
2. 调用 `sinx(N, terms, x, result)` ——**这一步会 spawn 一个 gang**（本例 8 个 program instance），8 条逻辑指令流并发进入 ISPC 函数体。
3. 函数体内 `foreach (i = 0 ... N)` 声明"这 N 个迭代由 gang 协作完成"；实现把迭代**交错**分给 instance：第 t 轮里 instance k 处理 `idx = t*8 + k`。
4. 每个迭代用泰勒展开算 $\sin(x[i]) \approx x - x^3/3! + x^5/5! - x^7/7! + x^9/9!$（`terms=5`）。
5. 8 个 instance 全部完成后 gang 一起返回（隐式屏障），`main` 恢复顺序执行。
6. 分块版 `sinx_blocked` 做同样的事，但把连续一段分给同一 instance：语义相同，**访存模式完全不同**。

> 说明（如实起见）：讲义里用 `foreach` 的那个版本，循环体内最后一行写作 `result[idx] = value;`，而 `idx` 是"手写交错版"才定义的变量；按 `foreach` 的语义此处应为 `result[i] = value;`。上面代码按语义修正。

【并行机制与性能解说】

- **线程/向量通道如何创建**：这里**没有 OS 线程**。gang 是编译期/运行期概念，由 ISPC 编译器**直接翻译成 SIMD 向量指令**（`--target=avx2-i32x8` → AVX2 的 8 个 32-bit lane）。8 个 "instance" 就是同一条向量指令的 8 个 lane，**在同一个核上锁步推进**。讲义明确提醒：前面这些代码**只会跑在四核机器的一个核上**；要多核，需要 ISPC 的另一个原语 **task**。
- **工作如何分配**：`foreach` 的静态交错分配（静态 = 分配与数据无关，无动态负载均衡）。三轮循环的 `i` 值是 uniform 的，因此循环控制本身在向量化后**只做一份**。
- **共享数据如何处理**：`x` 和 `result` 是 `uniform float*`（指针本身 uniform），但 `x[i]` 是 **varying** 表达式（每个 lane 不同）。例子中 lane 间**完全不通信**（每个输出元素只依赖自己的输入），所以不需要任何同步原语；如果要做归约，必须用跨 instance 通信原语 `reduce_add` ——讲义用 `sumall1` / `sumall2` 的例子强调：`sum` 若是 `uniform` 而 `x[i]` 不是 uniform 表达式，**会直接编译报类型错误**（"Result: compile-time type error"）。
- **Work / Span / 并行度分析**（设单元素成本 c，含 `terms` 次内层迭代）：
- **Work（总工作量）** $T_1 = \Theta(N \cdot terms \cdot c)$。本例 $N = 2^{24}$、terms = 5。
- **抽象的 Span（关键路径）**：数据并行抽象里各迭代相互独立，故 $T_\infty = \Theta(terms \cdot c)$（单个元素的成本）+ gang 返回的屏障延迟。抽象并行度 $T_1/T_\infty = \Theta(N)$，看起来"有 1600 万路并行"。
- **实现的并行度**：SIMD 实现下 gang 只有 `programCount = 8` 条真正并发的通道，且它们是**锁步**的：实际 $T_\infty(\text{实现}) = \Theta((N/8) \cdot terms \cdot c)$，并行度(实现) = 8。

- **结论**：这就是"SMD 抽象 vs SIMD 实现"的量化含义——**抽象告诉你"所有迭代相互独立、可任意调度"，实现只给你 8 路。** 想拿到 4 核 × 8 路 = 32 路，必须用 ISPC task（或换用 OpenMP/CUDA 等模型），这正好呼应讲义对 gang 图的注释。
- **瓶颈**：
- **访存模式（最大瓶颈）**：交错版每条 `packed load` 搬 8×4 = 32 B（半条 cache line）且地址连续 → 高效；分块版各 lane 地址跨步 `count`（本例 `count = N/8 = 2M` 个 float = 8 MB 远）→ 需要 **gather**，讲义原话是"更复杂、更昂贵的指令"。**实测差异来自内存事务数而不是算术量。**
- **带宽（本例的真正天花板，见第 4 节算例）**：N = 16M、每次迭代读 4 B 写 4 B，共 128 MB 流量；在 20 GB/s 的假设带宽下已经要 ≥6.4 ms，而算术只需约 0.65 ms（16 核 768 GFLOPS 的算例）——**这道题是内存带宽题，不是浮点题。**
- **条件控制流的掩码开销**：本函数无分支，但若循环体里有 `if`，不同 lane 走不同路径时 SIMD 会串行化两条路径（谓词执行），有效 lane 利用率下降。
- `sinx_blocked` 里 `count = N/programCount` 依赖整除假设，N 不整除时尾部要单独处理（讲义用注释"assume N % programCount = 0"点明）。

3.2 示例 2：pthreads 分块求和 + 伪共享（共享地址空间模型）

```
// ============ 文件: sum_pthreads.cpp ============
// 编译 (release): g++ -O3 -march=native -pthread sum_pthreads.cpp -o sum_pthreads
// 运行: ./sum_pthreads
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N        (1 << 26)          // 64M 个 float = 256 MB
#define NTHREAD  8
#define PAD      (64 - 8 - 8 - 8 - 8)   // 使结构体恰好 64 B = 1 条 cache line

// 每个线程的参数与结果: 按 cache line 对齐并填充, 避免 false sharing (伪共享)
typedef struct __attribute__((aligned(64))) {
    const float* x;
    long         begin;
    long         end;
    double       partial;       // 各线程私有部分和 (不共享写)
    char         pad[PAD];
} arg_t;

static void* worker(void* p) {
    arg_t* a = (arg_t*)p;
    double s = 0.0;
    for (long i = a->begin; i < a->end; i++)   // 顺序遍历自己那一段
        s += a->x[i];
    a->partial = s;                            // 只写自己 cache line 内的字段
    return NULL;
}

int main(void) {
    float* x = (float*)aligned_alloc(64, (size_t)N * sizeof(float));
    for (long i = 0; i < N; i++) x[i] = 1.0f;  // 期望和 = 67108864

    pthread_t tid[NTHREAD];
    arg_t     arg[NTHREAD];

    long chunk = N / NTHREAD;                  // 分块 (blocked) 划分: 每线程连续一段
    for (int t = 0; t < NTHREAD; t++) {
        arg[t].x     = x;
        arg[t].begin = t * chunk;
        arg[t].end   = (t == NTHREAD - 1) ? N : (t + 1) * chunk;
        arg[t].partial = 0.0;
        pthread_create(&tid[t], NULL, worker, &arg[t]);   // 创建 8 个线程
    }

    double total = 0.0;
    for (int t = 0; t < NTHREAD; t++) {
        pthread_join(tid[t], NULL);            // 等待 (fork-join 的 join)
        total += arg[t].partial;               // 归约: 串行加 8 个部分和
    }
    printf("total = %.0f (expected %d)\n", total, N);
    free(x);
    return 0;
}
```

【代码做什么？】

1. 分配 256 MB 的 `x` 并全部填 1.0。
2. 把数组**分块**切成 8 段（每段 32 MB 连续），每段配一个 64 字节对齐的 `arg_t`。
3. 主线程 `pthread_create` 创建 8 个 OS 线程；每个线程顺序遍历自己那段，累加到一个**私有**的 `double partial`。
4. `pthread_join` 隐式构成 **fork–join 的 join 点**；主线程把 8 个部分和相加。

【并行机制与性能解说】

- **线程如何创建**：`pthread_create` 走 **OS 系统调用 API**，由 OS 把每个线程映射到硬件的**执行上下文（execution context）**（讲义最后一页思考题的答案：mapping 由 OS 负责）。8 个 pthreads > 4 核 × 2 上下文 = 8 个上下文时刚好占满；若开 16 个就会有**过订阅**，OS 需要分时切换（增加上下文切换与缓存污染）。

- **工作如何分配：分块（blocked）/ 静态划分**。因为每线程拿到的是连续区间，硬件预取器与 TLB 都很友好——这与 ISPC 里"分块"的结论**并不矛盾**：那里的问题是分块破坏了 **SIMD lane 的连续性**，而这里每个线程是**标量顺序**执行，连续性由单线程自己的地址流保证。**同一个"分块 vs 交错"的决策，在 SIMD 与多线程两种实现里结论不同**——这正是"抽象 vs 实现"的最好例证。
- **共享数据如何处理**：`x` 只读（所有线程读同一份，cache line 可在多个 L1/L2 里以 Shared 状态共存，无一致性流量）；`partial` 各线程私有，且**用 64 字节填充隔离**——如果 8 个 `partial` 挤在同一条 cache line 里，每个线程的写都会让该 line **在核间往返失效（cache line ping-pong）**，这就是**伪共享（false sharing）**：程序语义无误，性能却暴跌。
- **Work / Span / 并行度**：
- **Work**：$T_1 = \theta(N)$ 次浮点加 + N 次 4 B load。$N = 2^{26}$。
- **Span**：$T_\infty = \theta(N/8)$（一个线程的那一段）+ fork/join 开销（`pthread_create` 约 10 μs 量级 + `pthread_join` 同步）。**注意：这不是 O(log N)** —— 因为归约只在最后串行加 8 个数，O(8)；真正的关键路径是"一个线程干完自己那段"。
- **并行度** = $T_1/T_\infty \approx 8$。**但注意这个 8 是"上限"，不是"实际"**：真正的约束是内存带宽（见下）。
- **可扩展性上限**：这是一个 **bandwidth-bound** 内核（每个元素只做 1 次加法，算术强度 1 flop / 4 B = 0.25 flop/byte）。加到 16 核不会更快，因为 DRAM 带宽不变。用 Amdahl 的话说，这里的"串行部分"是**共享的 DRAM 带宽**，不是代码里的某个临界区。
- **瓶颈（按重要性）**：
- **DRAM 带宽**：读 256 MB（写只有 64 B），在本节假设的 20 GB/s 下 ⩾12.8 ms。计算只需 64M 次加法 ÷ (8 核 × 3 GHz × 8 宽 SIMD) ≈ 0.33 ms。**~40 倍差距 ⇒ 必须用向量化+多核把算力堆高是没用的，应该做的是减少访存（分块复用、融合）。**
- **伪共享**：若把 `pad` 去掉、8 个 `double partial` 相邻，则每次写都要独占整条 cache line；60+ ns 的 line 迁移 × 每线程只在末尾写一次……本例中每线程只写一次，影响小；但若循环体内累加的是**共享的**计数器（`total += x[i]` 且无锁），就会变成每次迭代一次 line 迁移，性能直接掉一到两个数量级（见 4.4 的数值算例）。
- **尾部负载不均**：`N % NTHREAD != 0` 时最后一段略长，本例用 `(t == NTHREAD-1) ? N : ...` 处理；更普遍的不均（例如每段计算量不同）需要动态调度。
- **归约的串行尾巴**：8 个部分和的相加是 O(NTHREAD) 串行，占比可忽略；但如果线程数上千（或在 GPU 上），就要用树形/对数步归约。
- **反例（讲义强调的"共享地址空间很容易搬起石头砸自己的脚"）**：把上面改成"所有线程用一把互斥锁往同一个 `total` 里累加"，语义仍然正确，但**每次加法都变成一次原子/加锁的串行临界区**：此时 $\text{Span} = \theta(N \cdot t_{critical})$，**并行度 ≈ 1**，加核完全无效（4.5 有数值算例）。**"程序正确但性能很差"是共享地址空间模型的典型失效模式。**

3.3 示例 3：MPI 一维 stencil 的 halo 交换（消息传递模型）

```c
// ============ 文件: stencil_mpi.c ============
// 编译 (release): mpicc -O3 -march=native stencil_mpi.c -o stencil_mpi
// 运行: mpirun -np 8 ./stencil_mpi
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define WIDTH    1000000           // 全局网格宽度 (每步需要邻居的 1 个元素)
#define NITER    100
#define TAG_L    100               // 消息 tag: 来自左邻居
#define TAG_R    101               // 消息 tag: 来自右邻居

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (WIDTH % size != 0) { if (rank == 0) printf("WIDTH 必须被 size 整除\n");
                             MPI_Finalize(); return 1; }
    const int local = WIDTH / size;

    // cur/next 各含 2 个 halo 单元 (下标 0 与 local+1)
    double* cur  = (double*)calloc(local + 2, sizeof(double));
    double* next = (double*)calloc(local + 2, sizeof(double));
    for (int i = 1; i <= local; i++) cur[i] = rank * 1000.0 + i;   // 初始化

    int left  = (rank - 1 + size) % size;      // 环状拓扑: 边界绕回
    int right = (rank + 1) % size;

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    for (int it = 0; it < NITER; it++) {
        // —— halo 交换: 左右各一对 Sendrecv (可同时进行, 避免死锁) ——
        MPI_Sendrecv(&cur[1],       1, MPI_DOUBLE, left,  TAG_R,
                     &cur[local+1], 1, MPI_DOUBLE, right, TAG_R,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&cur[local],   1, MPI_DOUBLE, right, TAG_L,
                     &cur[0],       1, MPI_DOUBLE, left,  TAG_L,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        // —— 计算: 5 点差分 cur[i] 的邻居在 cur[i±1] ——
        for (int i = 1; i <= local; i++)
            next[i] = 0.25 * (cur[i-1] + 2.0 * cur[i] + cur[i+1]);

        double* tmp = cur; cur = next; next = tmp;   // 交换缓冲区 (零拷贝)
    }

    double t1 = MPI_Wtime();
    double local_sum = 0.0;
    for (int i = 1; i <= local; i++) local_sum += cur[i];
    double global_sum = 0.0;
    MPI_Allreduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

    if (rank == 0)
        printf("ranks=%d width=%d iters=%d  time=%.3f s  checksum=%.3f\n",
               size, WIDTH, NITER, t1 - t0, global_sum);

    free(cur); free(next);
    MPI_Finalize();
    return 0;
}
```

【代码做什么？】

1. `MPI_Init` 启动 `size` 个**进程**（不是线程），每个进程一个 **rank**；每进程负责全局网格的一段 `local = WIDTH/size`。
2. 每个进程的数组两端各留 1 个 **halo（影子/晕圈）单元**，用来存放邻居的边界值。

3. 每次迭代：(a) 与左右邻居各交换 1 个边界值（用 `MPI_Sendrecv` 一次调用同时完成收与发，避免"双方都先 recv 再 send"造成的**死锁**）；(b) 用 halo 更新自己内部的 `local` 个点；(c) 交换 `cur` / `next` 指针。
4. 结束时用 `MPI_Allreduce` 做一次全局归约求校验和。

【并行机制与性能解说】

- **进程/通信如何建立**：`mpirun -np 8` 起 8 个**独立进程**，各有**私有地址空间**；它们之间**没有共享内存可依赖**（即使在同一节点上，MPI 也可能走共享内存通道，但**编程模型上不可见**）。数据交换只能靠 `MPI_Sendrecv` 显式拷贝。每对通信由 **(发送者, tag)** 匹配——本例用两个 tag 区分左/右邻居，避免消息错配。
- **工作如何分配**：**分块（blocked）** 一维划分，每 rank 一段连续区间。这与共享地址空间的分块同理，但**这里的"分块"必须在编译/启动期确定通信伙伴**——这就是消息传递"结构强"的体现：**读代码就能看出通信发生在哪里**。
- **共享数据如何处理**：无共享数据。`cur` / `next` 是各进程私有的；边界值通过消息**复制**过来。**没有伪共享、没有缓存一致性流量**，代价是每步都要显式拷贝 halo。
- **Work / Span / 并行度**：
- **Work**：$T_1 = \Theta(\text{NITER} \cdot (\text{WIDTH 次差分计算}))$，即 100×10^6 次更新，每次约 4 flops ≈ 4×10^8 flops。
- **Span（关键路径）**：$T_\infty = \Theta(\text{NITER} \cdot (\text{local} \cdot c_{\text{compute}} + \alpha + \text{halo_bytes}/\beta))$。"每个迭代都要等邻居的 halo" 形成**逐步依赖链**——这是 stencil 这类问题的关键路径本质：**迭代之间不能并行，只有同一迭代内部的不同区间能并行。**
- **并行度** ≈ `WIDTH`（同一迭代内的点互相独立）。**看起来极高**，但实际加速被通信限制：$T(p) \approx T_1/p + \text{NITER}\cdot(2\alpha + 2h/\beta)$，其中 `h` 是每个 halo 的字节数（本例只有 1 个 double = 8 B）。
- **可扩展性上限（数值见 4.6）**：halo 只有 8 B/次/邻居，却要付**整个消息的固定延迟 α（约 2 μs）**，所以**本例几乎完全是延迟受限**：加 rank 会减少计算时间但通信延迟不变 ⇒ 存在一个最优 p。反过来，若 halo 面积/体积比很大（3D 大 stencil、或用很小的子域），计算/通信比会进一步恶化。
- **瓶颈（按重要性）**：
- **通信延迟 α**：8 B 的消息也要走完整个协议栈。增大每 rank 的计算量（更好的做法：增加每步的计算密度或用"波前/时间分块 temporal blocking"让 halo 复用多次）能摊薄它。
- **同步点**：每次迭代两个 `MPI_Sendrecv` 都是同步点，`MPI_Allreduce` 更是全局同步；迭代间的依赖使流水线无法重叠。
- **负载不均**：`WIDTH % size != 0` 直接报错退出；真实代码要么允许最后一段不等长，要么用一维/二维分解使各段均衡。
- **带宽 β**（当 halo 变大时成为主因）：若每步交换的是"面"而不是"点"（3D 分解中常见），字节数按面积增长，此时 `h/β` 会主导。

3.4 示例 4：OpenMP 数据并行 map（数据并行模型 + 共享地址空间实现）

```
// ============ 文件: map_omp.cpp ============
// 编译 (release): g++ -O3 -march=native -fopenmp map_omp.cpp -o map_omp
// 运行: OMP_NUM_THREADS=8 ./map_omp
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

int main() {
    const long N = 1L << 24;                        // 16M 元素
    float* x = (float*)aligned_alloc(64, N * sizeof(float));
    float* y = (float*)aligned_alloc(64, N * sizeof(float));
    for (long i = 0; i < N; i++) x[i] = (float)((i % 2048) - 1024) / 1024.0f;

    double t0 = omp_get_wtime();

    // 数据并行 "map": 把 f 施加到集合 x 的每个元素上，写出到集合 y
    // schedule(static) = 静态分块（与 pthreads 示例同构的划分方式）
    #pragma omp parallel for schedule(static)
    for (long i = 0; i < N; i++) {
        float v = x[i];
        y[i] = (v < 0.0f) ? -v : v;                 // f(v) = |v|
    }

    double t1 = omp_get_wtime();

    // 归约（数据并行的 reduce 模式；ISPC 里对应 reduce_add）
    double sum = 0.0;
    #pragma omp parallel for reduction(+:sum) schedule(static)
    for (long i = 0; i < N; i++) sum += y[i];

    double t2 = omp_get_wtime();
    printf("map: %.3f ms   reduce: %.3f ms   sum=%.0f\n",
           (t1 - t0) * 1e3, (t2 - t1) * 1e3, sum);

    free(x); free(y);
    return 0;
}
```

【代码做什么？】

1. 分配两个 64 MB 数组（共 128 MB），初始化 `x`。
2. 第一个 `parallel for` 是**数据并行的 map**：`f(v)=|v|` 施加到 `x` 的每个元素，结果写入 `y`——这正是讲义 slide 里 `absolute_value` 的 C++/OpenMP 版本（用 `foreach` 的 ISPC 版本语义完全一致）。
3. 第二个 `parallel for` 带 `reduction(+:sum)`：这是数据并行里的 **reduce 模式**；与 ISPC 的 `reduce_add(partial)` 对应——**每个线程先在私有变量上累加，最后合并**（讲义对 `sumall2` 的解释）。
4. 打印两段的耗时。

【并行机制与性能解说】

- **线程如何创建**：OpenMP 采用**共享地址空间模型 + fork-join**：进入 `parallel for` 时"fork"出一个线程组（实现通常是**线程池**，不是每次真的 `pthread_create`），循环结束在**隐式屏障**处"join"。**同步是隐含在 map 结束处的**——与讲义对数据并行模型的定义完全一致（"Synchronization is implicit at the end of the map"）。
- **工作如何分配**：`schedule(static)` 是静态分块（把迭代切成连续块分给线程，与示例 2 的 pthreads 分块同理）；`schedule(dynamic)` 则是"动态抢占式"分配，适合每迭代耗时差异大的场合，代价是每次取任务都有同步开销。
- **共享数据如何处理**：`x`、`y` 共享但**不重叠**（每迭代写自己的 `y[i]`），无竞争；`sum` 由 `reduction` 子句自动变成"每线程私有副本 + 结束合并"，程序员**看不到共享写**。这里也顺带说明：**"数据并行模型"在实现上几乎总是落在"共享地址空间的硬件"之上**——讲义在总结页的原话是：数据并行"assumes a shared address space from which to load inputs/store results"。
- **Work / Span / 并行度**：
- **Map 的 Work** = N 次"读 4 B + 比较/取负 + 写 4 B"；**Span** = 单元素成本 + 屏障延迟；**并行度 = N**（抽象层面）。
- **Reduce 的 Work** = N 次加法；**Span** = 每线程的 N/p 次加法 + O(p) 的合并（若用树形归约则为 O(log p)）；**并行度 = N/(N/p) = p**——**归约的并行度由线程数封顶，而不是由 N 封顶**（因为同一线程内的累加是有依赖的串行链）。这是一个很容易答错的考点。

- **可扩展性上限**：Map 是纯流式访存（读 64 MB + 写 64 MB = 128 MB），**完全受 DRAM 带宽限制**：假设 20 GB/s，两段各需 ⩾6.4 ms，与线程数无关；超过"带宽饱和点"后再加线程只会浪费。
- **瓶颈**：
- **DRAM 带宽**（决定性）：两个 kernel 都几乎没有数据复用（算术强度分别约 0.25 和 0.125 flop/byte）。**优化方向是融合（fusion）或分块（tiling）以减少字节数，而不是加线程。**
- **隐式屏障**：每次 `parallel for` 结束都有一个全部线程的屏障，线程数多、迭代短时屏障开销占比会上升（Barrier 成本约 1–10 μs 量级 + 不平衡等待）。
- **向量化**：`-O3 -march=native` 下 `y[i] = f(x[i])` 通常能向量化为 8 宽 AVX；若编译器因**指针别名**不敢向量化，可加 `restrict` 或 `#pragma omp simd`。这直接呼应示例 1：**同一份"数据并行"语义，最终带宽受限于是否能生成 packed load/store。**
- **动态调度的开销 / 负载不均**：元素级 map 的迭代成本均匀，静态分块最省；若循环体里有分支或数据依赖的不均匀（稀疏结构），则需要 `schedule(dynamic, chunk)`。

---

4. 性能模型与复杂度分析

本节所有"假设参数"都显式列出，属**教学估算量级**（非讲义原文数值）；讲义本身没有给出这些具体数字，请把它们当作可替换的建模输入。

4.1 work–span 模型与并行度

对并行程序的依赖图 DAG：

- **Work** T_1：用一个处理器执行全部工作所需时间（DAG 的**总节点成本**）。
- **Span** T_∞（关键路径、critical path）：用**无限多**处理器执行所需时间（DAG 的**最长路径成本**）。
- **并行度（parallelism）** $= T_1 / T_\infty$：能有效利用的处理器数上限。
- **调度界（greedy scheduler，如 work–stealing）**：用 p 个处理器时

$T_p \le T_1/p + T_\infty$ （且 $T_p \ge \max(T_1/p, T_\infty)$）

⇒ 当 $p \gg T_1/T_\infty$ 时，T_∞ 项主导，加处理器不再有用。**这是"并行度上限"的定量表述。**

表：本讲四个示例的 Work / Span / 并行度对照

示例	Work T_1	Span T_∞	并行度 T_1/T_∞	真正的约束
1. ISPC sinx（抽象）	Θ(N·terms)	Θ(terms) + 返回屏障	Θ(N)	实现只给 `programCount`=8 路（SIMD）
1. ISPC sinx（SIMD 实现）	Θ(N·terms)	Θ((N/8)·terms) 锁步	8	SIMD 宽度 + 访存模式（packed vs gather）
2. pthreads 分块求和	Θ(N)	Θ(N/8) + fork/join	≈ 8	**DRAM 带宽**（0.25 flop/byte）
2'. 同上但用**全局锁**累加	Θ(N)	Θ(N · t_crit)	≈ 1	**锁的串行化**（Amdahl 的 S=1）
3. MPI stencil	Θ(NITER·WIDTH)	Θ(NITER·(WIDTH/p + α + h/β))	Θ(WIDTH)	通信**延迟** α + 迭代间依赖
4. OpenMP map	Θ(N)	Θ(1) + 屏障	Θ(N)	DRAM 带宽
4'. OpenMP reduce	Θ(N)	Θ(N/p)	p	线程数（私有累加链是串行的）

4.2 Amdahl 定律与强/弱扩展

设可并行比例 `f`，串行比例 `s = 1 − f`，处理器数 p（忽略开销）：

```
Speedup(p) = T₁ / T_p = 1 / ( s + (1 − s)/p )        lim_{p→∞} = 1/s
```

数值算例 A（Amdahl）：某程序有 2% 的串行部分（`s = 0.02`）。

- p = 8：`Speedup = 1/(0.02 + 0.98/8) = 1/0.1425 = 7.02×`（效率 87.7%）
- p = 16：`Speedup = 1/(0.02 + 0.98/16) = 1/0.08125 = 12.31×`（效率 76.9%）
- p = 64：`Speedup = 1/(0.02 + 0.98/64) = 1/0.03531 = 28.32×`（效率 44.3%）
- **上限**：`1/0.02 = 50×`——**无论多少核，都不可能超过 50 倍。**

数值算例 B（锁的串行化，对应示例 2'）：8 个线程各做 10^6 次"加锁—自增—解锁"，每次临界区耗时 50 ns：

- 总临界区时间 = `8 × 10⁶ × 50 ns = 0.4 s`（**必须串行**）
- 8 核并行本来只需 `8×10⁶/8 × 1 ns ≈ 1 ms`
- 实测 ≈ 0.4 s ⇒ 相对理想并行慢 **≈ 400 倍**，加速比 ≈ **1.0**
- 结论：**临界区里哪怕只有几十纳秒，只要它在迭代里，Amdahl 的 `s` 就趋近 1。** 共享地址空间模型的"性能陷阱"就是这种形状。

数值算例 C（弱扩展/Gustafson）：如果问题规模随 p 一起放大（每核工作量固定为 T），则 `Speedup = s + (1−s)·p`，p = 16、s = 0.02 时为 `0.02 + 0.98×16 = 15.7×`。**这就是为什么超算都追求"弱扩展效率"**——它绕过了 Amdahl 的上限，但要求问题本身能变大。

4.3 算术强度与 Roofline

算术强度（arithmetic intensity）：`I = W / Q`，其中 W 是浮点操作数，Q 是访问的字节数（DRAM 流量）。**Roofline 模型**：

```
attainable_perf = min( P_peak , I × BW )
      ↑ 计算上限             ↑ 带宽上限（"屋顶"的斜边）
拐点（ridge point）= P_peak / BW   [flops/byte]
   I < ridge  ⇒ memory bound（带宽受限，优化=减少访存/提高复用）
   I > ridge  ⇒ compute  bound（算力受限，优化=更宽 SIMD/更好指令混合）
```

数值算例 D（算力峰值）：讲义复习页给出的单芯片规格是"4 核 × 2 上下文 × 每时钟 2 条指令（其中 1 条 8 宽 SIMD）"。

```
4 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA 计 2 flops）= 192 GFLOPS
若放大到 16 核（同一时钟/宽度）：16 × 3.0e9 × 8 × 2 = 768 GFLOPS
```

数值算例 E（带宽 vs 一次遍历的成本，讲义反复使用的论证方式）：

```
一个 256 MB 的数组（float32，64M 元素）
假设 DRAM 带宽 = 20 GB/s
只读一遍：  256 MB / 20 GB/s = 0.256e9 / 20e9 = 12.8 ms
读+写一遍：  512 MB / 20 GB/s = 25.6 ms
⇒ 任何"整数组遍历一次"的并行算法，其时间下界就是这十几毫秒，
   与你有多少核、多少车道(SIMD) 无关。
```

数值算例 F（Roofline 判定，用示例 1 的 sinx）：设 `terms = 5`。

```
每个元素的操作数（内层 5 次迭代）：
    value += sign*numer/denom   → 乘 + 加（除法按乘法倒数计）≈ 2 flops
    numer *= x*x                → 2 flops
    denom *= (2j+2)(2j+3)       → 2 flops（整数亦可）
    sign  *= −1                 → 可编译期展开，忽略
  小计 ≈ 6 flops/迭代 × 5 = 30 flops
  加上 x*x*x 的初始化 ≈ 2 flops        ⇒ W ≈ 32 flops/元素
每个元素的 DRAM 流量：读 4 B + 写 4 B = 8 B   ⇒ I = 32/8 = 4.0 flops/byte
拐点 = 768 GFLOPS / 20 GB/s = 38.4 flops/byte
因为 4.0 ≪ 38.4 ⇒ 严重 memory bound
可达性能上限 = I × BW = 4.0 × 20 GB/s = 80 GFLOP/s
           = 80/768 = 10.4% 的峰值算力
⇒ 结论：把 SIMD 从 8 宽加到 16 宽、把核数翻倍，对 sinx 几乎无益；
   真正的优化是(1)提高复用（分块，把同一元素供多个计算使用）、
   (2)减少流量（融合、用更低精度/更小数据类型）、(3)避免 gather。
```

表：常见内核的算术强度与受限类型（用上面的假设 BW = 20 GB/s、P_peak = 768 GFLOPS）

内核	每元素 flops	每元素字节	I (flops/B)	受限类型	带宽上限下的性能
DAXPY `y=a·x+y`	2	12（读 x、读 y、写 y）	0.17	严重 memory bound	≈ 3.3 GFLOPS
数组求和（示例 2）	1	4（只读）	0.25	memory bound	≈ 5 GFLOPS
`y[i]=|x[i]|`（示例 4 map）	1	8（读+写）	0.125	严重 memory bound	≈ 2.5 GFLOPS
sinx（terms=5，示例 1）	32	8	4.0	memory bound	≈ 80 GFLOPS
稠密 GEMM n=1024（分块后）	$2n^3 \approx 2.1e9$	$\approx 3n^2 \times 4$ B = 12.6 MB	≈ 171	compute bound	可达峰值的 70%+

4.4 通信成本模型：延迟 α + 带宽 β（消息传递 & 缓存行迁移）

```
Hockney 模型（点对点消息）：T_msg(n) = α + n/β
   α = 消息启动延迟（协议、软件栈、拷贝启动），β = 链路/拷贝带宽
共享地址空间的"隐式通信"（缓存行迁移 / 伪共享）也有同一形式：
   T_line = α_cache + n_line/β_interconnect ,  α_cache ≈ 60–100 ns 量级
```

数值算例 G（消息大小的影响）：α = 2 μs，β = 10 GB/s。

消息大小 n	传输时间 n/β	总时间 T = α + n/β	有效带宽 n/T
8 B	0.0008 μs	≈ 2.00 μs	0.004 GB/s（**99.96% 花在延迟上**）
8 KB	0.8 μs	2.8 μs	2.9 GB/s
1 MB	100 μs	102 μs	10.3 GB/s（接近 β）
64 MB	6400 μs	6402 μs	10.0 GB/s

⇒ **小消息要么合并（把多次发送打包成一条），要么避免（提高计算/通信比）**。这正是示例 3 中"halo 只有 8 B 却要付 2 μs"的定量解释。

数值算例 H（伪共享的代价）：4 个线程**各自**累加一个位于**同一条 cache line** 的 `int` 计数器，每线程 10^7 次。

```
每次自增都需要"独占"该 cache line ⇒ 约 80 ns 的 line 迁移
总时间 ≈ 4 × 10⁷ × 80 ns = 3.2 s
填充后（每个计数器独占一条 line，line 常驻各自 L1 的 Exclusive 状态）：
   每线程 ≈ 10⁷ 次 × ~1 ns ≈ 10 ms       ⇒ 约 300 倍的差距
⇒ 这是"共享地址空间模型"里最典型、最难从代码上看出（但最容易用 padding 修好）的性能陷阱。
```

4.5 数值算例：示例 2 的端到端估算

假设：N = 2^{26} = 67,108,864 个 float（256 MB），8 线程，DRAM 带宽 20 GB/s，8 核各 3.0 GHz、8 宽 SIMD、每时钟 1 条向量加。

```
计算下界：67.1e6 次加法 / (8 核 × 3.0e9 × 8 宽) = 67.1e6 / 1.92e11 = 0.35 ms
访存下界：256 MB / 20 GB/s = 12.8 ms
⇒ 预测耗时 ≈ 12.8 ms（内存受限，效率 = 0.35/12.8 = 2.7% 的算力利用率）
若改为"只读一遍、只写一个部分和"的向量化实现，仍然 ≈12.8 ms（带宽就是墙）
反例（全局锁版本）：8e6 次加锁 × 50 ns ≈ 0.4 s = 400 ms（比 12.8 ms 慢 31 倍）
```

4.6 数值算例：示例 3 的扩展性（寻找最优 rank 数）

假设：WIDTH = 10^6 个 double（8 MB），NITER = 100，单点更新约 4 flops，单核吞吐 3.0e9 × 8 宽 × 1 = 24 Gflop/s；α = 2 μs，halo = 8 B + 消息头（按 100 B 计），β = 10 GB/s。

```
单 rank 计算时间 = 100 × 1e6 × 4 flops / 24e9 = 0.0167 s = 16.7 ms
p = 8 时：计算 = 16.7/8 = 2.1 ms
          通信 = 100 次迭代 × 2 条消息 × (2 μs + 100 B/10 GB/s)
               = 100 × 2 × 2.01 μs = 402 μs = 0.40 ms
          总计 ≈ 2.5 ms    (加速比 ≈ 6.7×)
p = 64 时：计算 = 16.7/64 = 0.26 ms；通信仍 ≈ 0.40 ms
          总计 ≈ 0.66 ms   (相对 p=8 只快 3.8 倍，效率降到 24%)
⇒ 通信时间与 p 无关（每次迭代固定 2 条消息）⇒ 存在"收益递减"；
   p 增大到计算时间 ≈ 通信时间（约 0.4 ms ⇒ p ≈ 42）附近即最优。
改进方向（对应讲义"abstraction distance / 通信成本要反映现实"的思想）：
   (1) 增大每 rank 的数据量（弱扩展）；(2) 时间分块让 halo 复用多次；
   (3) 用一次大消息换掉多次小消息（合并 halo）；(4) 非阻塞通信与计算重叠。
```

---

5. 关键要点

1. **永远区分"编程模型（抽象）"与"硬件实现"**。ISPC 的 gang 抽象 = 8 条逻辑指令流，实现 = 一条 8 宽 AVX 指令；pthreads 的线程抽象，实现 = OS 把线程映射到硬件执行上下文。性能藏在实现的细节里，**不要把抽象语义当成性能保证**（讲义的原话："conflating abstraction with implementation is a common cause for confusion"）。
2. **三种通信抽象各用结构换性能可预测性**：共享地址空间结构最少、最自然，但**所有 load/store 在代码上看起来一样贵，实际却天差地别**（局部/远端、有无伪共享、是否争锁）；消息传递把通信全部显式化成消息，**结构约束反而帮你更快写出可扩展的正确程序**；数据并行把计算钉

成"map over collection"，**用刚性换自动并行与编译期优化（预取、融合）**。

3. **抽象必须"低到能反映通信成本，又高到能保持可移植"（abstraction distance 原则）**。这是讲义总结页的核心判据，也是理解为什么现代实践是**混合模型**：节点内用共享地址空间（pthreads/OpenMP），节点间用消息传递（MPI），kernel 内再加受限的共享内存同步（CUDA/OpenCL）。
4. **硬件尺度决定有效模型**：单核内 SIMD 宽度决定数据并行的粒度（gather 与 packed load 的差别就是 10 倍级访存成本）；片内多核共享 L3/DRAM 带宽决定"加核是否有用"；多路节点是 NUMA（局部性重要）；集群只能消息传递。讲义用 Intel i7 的 ring、Niagara 2 的 crossbar、Altix UV1000 的 fat tree（4096 核单一地址空间）、El Capitan 的 dragonfly + MI300A 统一地址空间，说明"支持共享地址空间的硬件代价随规模急剧上升"。
5. **性能上限先算带宽与并行度，再谈优化**：对每个内核先估 `I = W/Q` 与 `T∞`。若 `I < P_peak/BW`（本例 sinx：4.0 < 38.4），则优化方向是**减少字节数（融合/分块/更小数据类型）而非加宽 SIMD**；若 T_1/T_∞ 很小（如全局锁版本 ≈ 1），则优化方向是**缩短关键路径（消除串行化）而非加处理器**。

---

6. 常见陷阱与注意事项

- **把"抽象语义允许并行"当成"实现能并行"**：`foreach` 声明 N 个迭代相互独立，但 ISPC gang 只有 `programCount`（= SIMD 宽度）路；要在多核上跑必须用 ISPC 的 `task` 或换用多线程模型。同理，`#pragma omp parallel for` 写出来了也不代表生成了向量指令——没向量化就等于丢掉了 8 倍。
- **数据竞争（data race）与"看起来正确"的共享内存程序**：共享地址空间是顺序编程的自然延伸，但**两个线程对同一变量无同步的并发读写是未定义行为**（讲义用 `while (x == 0) {}` 轮询的例子开场）。更隐蔽的是**"程序正确但很慢"**：比如用一把全局锁把并行循环串行化（示例 2'，实测比理想并行慢数百倍），代码 review 往往查不出来。
- **伪共享（false sharing）**：不同线程写**不同变量**但落在**同一条 cache line** 上，导致 line 在核间反复迁移。语义完全正确，性能可能差两个数量级。修法是按 cache line（典型 64 B）对齐填充，或让每线程使用私有副本 + 末尾归约。**注意它只发生在"共享地址空间 + 缓存一致"的实现里**；消息传递模型没有这个问题，但代价是显式拷贝。
- **忽视带宽与算术强度**：大量"逐元素"内核（map、求和、DAXPY、stencil）的算术强度都在 1 flop/byte 以下，**性能由 DRAM 带宽封顶**，与核数/SIMD 宽度无关（示例 E：256 MB 数组单次遍历 ⩾12.8 ms）。在这类内核上做"把 SIMD 加宽、把线程数翻倍"的优化是纯粹的浪费；正确方向是**提高复用（分块/时间分块）或减少流量（kernel 融合、压缩数据类型）**。
- **负载不均与静态划分的盲区**：静态分块在每迭代成本均匀时最省（无调度开销），但一旦成本不均（稀疏结构、分支、边界条件、尾部不整除），最慢的那个线程决定总时间（$T_p \geq \max_i T_i$）。反之，动态调度有每任务一次的同步开销，任务粒度过细时会退化（对应 `schedule(static)` vs `schedule(dynamic, chunk)` 的选择）。
- **消息传递里的死锁与错配**：两个 rank 都先做阻塞 `MPI_Recv` 再 `Send` 会**死锁**（示例 3 用 `MPI_Sendrecv` 规避）；tag 使用不当会让消息匹配到错误的接收者（尤其在有多个通信阶段时）；小消息频繁发送会被**固定延迟 α 主导**（算例 G：8 B 消息的有效带宽只有 0.004 GB/s）。**先想通信结构，再写代码**——这正是消息传递模型"强制结构"的价值所在。
- **非确定的数据并行写法**：讲义用 `shift_negative` 说明——多个迭代可能写同一位置，而数据并行模型**不规定迭代顺序、也不提供细粒度互斥**，因此该程序是**非确定的**。在 `foreach`/kernel 里写"跨迭代依赖"（如邻接写、累加到共享位置）必须改用其他模型或显式原子操作，代码可能"有时对"，但这不代表它是对的。

---

7. 思考题（带答案）

问题 1. 一个只在 ISPC 里用 `foreach` 写成的 sin(x) 计算程序，在 4 核 8 宽 SIMD 的机器上实测速度只有理论峰值的 ~3%。请分别从"抽象/实现"、"访存模式"、"带宽"三个角度解释原因，并给出至少两条可行的优化方向。

【答案】 (1) **抽象 vs 实现**：`foreach` 的抽象语义是"N 个迭代彼此独立、可任意调度"，看起来并行度是 N；但 ISPC gang 的实现只是**一条 8 宽 SIMD 向量指令流，跑在一个核上**，实际并行度只有 `programCount = 8`。要真正吃满 4 核，必须用 ISPC 的 **task** 原语（或改写成 OpenMP/CUDA）。也就是说，实际并行度是 8 而不是 4×8 = 32，仅此一项就损失了 4 倍。(2) **访存模式**：如果按讲义给的手写"分块（blocked）"版本写（`start = programIndex * count`），同一轮各个 lane 访问的地址相隔 `count` 个元素、并不连续，编译器必须生成 **gather**（`_mm_i32gather`，2013 年才随 AVX2 出现，AVX2 甚至没有 SIMD scatter）而不是一条 packed load。gather 的访存事务数成倍增加、延迟更高，有效带宽远低于连续向量访存。改成 `foreach`/交错访问后访存变成连续 32 B packed load，这一步就能拿到数倍提升。(3) **带宽与算术强度**：`terms = 5` 时每元素约 32 flops，而每个元素只读 4 B、写 4 B，故 `I ≈ 4.0 flops/byte`；而该机器的 Roofline 拐点 = `768 GFLOPS / 20 GB/s = 38.4 flops/byte`。`I ≪ 拐点` ⇒ **严重 memory bound**，性能上限 = `I × BW = 80 GFLOP/s ≈ 10.4%` 峰值；加上 gather 与未向量化等因素，实测 3% 完全在预期内。 **可行优化**：(a) 把索引改成交错（`foreach`）以获得 packed load/store；(b) 分块复用数据、把多个元素的

计算做成一次访存喂多个计算（提高 I）；(c) 减少内层运算（把 `denom` / `sign` 用 uniform 变量、把除法换成乘以事先算好的 1/denom，必要时用多项式/查表）；(d) 用 `float` 而非 `double` 降低字节数；(e) 若允许近似，换成更少 terms 的多项式并 `-ffast-math` 让编译器向量化。

问题 2. 你的程序在一台 8 核共享地址空间机器上做 `y[i] = f(x[i])`（N = 2^{24} 个 float），8 线程实测 6.4 ms；把线程数加到 16 后仍然 6.4 ms。请判断这是什么瓶颈，用数字论证，并说明若把问题换成"每元素做 500 次浮点运算"会有什么不同。

【答案】 这是 **DRAM 带宽瓶颈**，不是并行度不足。数字论证：N = 2^{24} = 16.78M 个 float，读写各 4 B ⇒ 总流量 = 2 × 67.1 MB = 134.2 MB。实测 6.4 ms ⇒ 有效带宽 = 134.2 MB / 6.4 ms = **21 GB/s**，正好等于该机器的 DRAM 带宽（本节假设 20 GB/s 量级）。**带宽是共享资源，加线程不会增加它**，所以 8→16 线程毫秒不动；同时说明 8 线程时就已经把带宽打满了。 若每元素改为 500 flops：`I = 500/8 = 62.5 flops/byte > 拐点 38.4` ⇒ 变成 **compute bound**。此时 8 线程的计算时间 = `16.78e6 × 500 / (8 核 × 3.0e9 × 8 宽 × 2 FMA) = 8.39e9 / 384e9 ≈ 21.8 ms`，远大于带宽下界 6.4 ms ⇒ 时间由算力决定，**加线程（到 16）会接近线性加速（约 11 ms）**，直到再次撞上带宽或达到 16 核的算力上限。**结论：同一个"数据并行 map"结构，因算术强度不同而落在 Roofline 的两侧，优化手段完全不同——先算 `I` 再动手。**

问题 3. 讲义说"编程模型与机器类型之间的对应关系是模糊的"。请举出两个方向的例子，并解释"abstraction distance（抽象距离）"为什么既不能太大也不能太小。另外：为什么现代超算普遍采用"节点内共享地址空间 + 节点间消息传递"的混合模型?

【答案】 **两个方向的例子**：(a) **在共享地址空间的硬件上实现消息传递**：极其常见。此时"发送消息"就是从消息库缓冲区**拷贝内存**，"接收消息"就是从库缓冲区**拷回**——通信仍然成立，只是走的是本地拷贝而不是网络。(b) **在硬件不支持共享地址空间的机器上实现共享地址空间抽象**：用软件（如共享虚拟内存/软件 DSM）——把含有共享变量的页标记为 **invalid**，由**缺页异常处理程序（page-fault handler）** 发出网络请求把页面取到本地，程序仍然能"读写共享变量"，只是每次缺页都要经历一次极慢的软件路径。 **abstraction distance 的两难**：抽象距离太**小**（抽象几乎照抄某个硬件的细节，比如把"某个具体 SIMD 宽度、某个具体 cache line 大小"写进模型），性能可预测、容易调优，但代码**不可移植**，换个机器就废；抽象距离太**大**（模型对所有访存/通信一视同仁，例如"所有 load/store 一样贵"），代码**可移植、好写**，但程序员无法从源码判断性能，优化只能靠猜。讲义的原则是：**保持足够低以让性能可预测，足够高以保留灵活性与可移植性**——共享地址空间模型正是"距离偏大"的典型（好写但容易写出慢程序），消息传递模型则"距离偏小"（难写但可扩展）。 **为什么超算是混合模型**：硬件上，**同一节点内**（多核 + 共享 L3 + 缓存一致）能以极低成本提供共享地址空间，所以用 pthreads/OpenMP 最划算、编程最舒服；而**跨节点**若要在硬件上维持全局共享地址空间，成本极高（讲义：SMP"统一地差"、NUMA 已经需要程序员处理局部性；Altix UV1000 用 fat tree 换来 4096 核单一地址空间，代价是巨额互连开销），因此节点间只提供消息通信，用 **MPI** 显式传递，让程序员面对真实的通信成本。于是"节点内共享内存 + 节点间消息传递"成了现代实践（讲义原话："very, very common in practice"），这也是 CUDA/OpenCL 里"kernel 之间用数据并行、同一 core/block 内的线程用共享内存通信"这一安排的同一个道理——**在每个尺度上选用与硬件成本最匹配的抽象。**

Lecture 5: GPU Architecture and CUDA Programming

1. 章节标题与概述

Lecture 5: GPU Architecture and CUDA Programming（GPU 体系结构与 CUDA 编程）

- **本讲核心问题**：GPU 为什么能把"同一条计算（kernel）作用在海量数据上"做得比 CPU 快十倍以上？要回答这个问题，必须同时说清两件事：**(1) CUDA 编程抽象是什么**（grid / thread block / thread 三级线程层次、分散的 host/device 地址空间、块内共享内存与 `__syncthreads()`），以及 **(2) 这些抽象在现代 NVIDIA GPU 上如何被实现**（线程块被 work scheduler 动态映射到 SM、block 内的线程按 32 个一组组成 warp 做 SIMT 执行、资源（寄存器/共享内存/执行上下文）决定一块 SM 上能同时驻留多少 block）。讲义把整讲收束为一句性能判据：**要写好 CUDA，程序必须是"coherent execution + coalesced memory access + 高数据复用"**——即 warp 内不发散、访存连续、每个字节从全局内存搬到片上后被尽量多地重复使用。
- **涉及的主要硬件/软件机制**：
- **软件侧（CUDA 抽象）**：`__global__` / `__device__` / `__host__` 三种函数限定符与 kernel launch 语法 `kernel<<<gridDim, blockDim, shmemBytes>>>(args)`、`dim3`、内置变量 `threadIdx/blockIdx/blockDim/gridDim`、`cudaMalloc` / `cudaMemcpy` / `cudaFree`、`__shared__`（静态与 `extern` 动态共享内存）、`__syncthreads()`、原子操作（`atomicAdd` 等，作用于 global 与 shared）、以及"用多次 kernel launch 代替全局同步"的 kernel decomposition 手法。
- **硬件侧（GPU 微架构）**：HBM（High Bandwidth Memory，高带宽显存，讲义标注 ~1 TB/s 量级）、SM（Streaming Multiprocessor，流多处理器，即"GPU 核"）、SM 内部的 sub-core / warp selector / 寄存器堆 / SIMD 功能单元 / LSU（load-store unit）/ Tensor Core、片上的 shared memory + L1 存储、L2 与全局内存、以及 GPU work scheduler（线程块调度器）。讲义还给出两代硬件的对照：GTX 980（2014，16 SM）→ H100（2022，132 SM、共享内存 256 KB/SM、峰值算力从 4.6 TFLOPs 提升到 1000 TFLOPs，主要来自 Tensor Core）。
- **抽象与实现的接口**：编译产物（CUDA device binary 里除了指令文本，还带有"资源需求"清单：每块多少线程、每线程多少本地数据、每块多少共享内存）——这正好是 work scheduler 做资源匹配的依据。

- **在并行计算知识体系中的角色**：本讲是全课程从"CPU 侧并行"跳到"加速器侧并行"的转折点。它承接第 3、4 讲的 SIMD / 多线程 / SPMD / 数据并行模型，把它们放进一个**真实而极端的机器**里：GPU 用"很多个弱核心 + 硬件多线程（latency hiding）+ 32 宽 SIMT"换取吞吐量。往后看，本讲的 warp/coalescing/共享内存分块三条线索会直接支撑后续的性能优化（Performance Optimization）、异构与硬件专用化、以及并行深度学习（tensor core、算子融合、数据复用）等讲座；实验课里 CUDA 的 matmul/reduction 优化也几乎全部是本讲两个 case study 的延伸。
- **配套材料**：
- `lectures/05-CUDA-programming.pdf`（对应抽取文本 `extracted/05-CUDA-programming.txt`，共 83 页幻灯片）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 下直接下载。Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）中，第 5 讲位于 **Sep 2**，主题写作 "GPU Architecture and CUDA Programming"，第 6 讲（Sep 4）是它的续讲。讲义首页标题写作 **"Lecture 5 & 6: GPU Architecture & CUDA Programming"**，即一份讲义覆盖两次课，属正常现象。
- `doc_CUDA-recitation.pdf`（对应抽取文本 `extracted/doc_CUDA-recitation.txt`，共 39 页）：CUDA recitation（习题/辅导课）讲义，**已公开**。它把 CUDA 从"怎么写"讲到"怎么写快"：三种函数限定符、dim3 与线程索引、host/device 内存管理的 7 个步骤、归约的四种渐进实现（kernel-as-barrier / `__syncthreads()` / 单 block / 多 block + 共享内存），以及 matmul 的两级分块。该讲义页脚标注的是历史学期（Fall 2025），属讲义沿用现象，不是错误。
- `cs149_supp/gpuarch.txt`：Stanford CS149（Fall 2025）Lecture 7 "GPU Architecture & CUDA Programming"（共 74 页）——**已公开**的补充读物，也是本笔记中 V100 SM 微架构、warp selector、work scheduler 逐步调度、persistent thread 等细节的来源。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属 **未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 历史学期 PDF（如 Performance Analysis/Profiling、Transactional Memory 等 Fall 2026 尚未在公开目录发布的讲义）位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。

---

2. 核心概念与硬件/软件架构图解

2.1 GPU 的来路：从 3D 渲染管线到 GPGPU

- **定义与目的**：GPU 原本只做一件事——**3D 渲染**。渲染任务的定义可以压成一句话：*给定场景描述（三角形网格、材质、光源、相机），计算网格中每个三角形对图像中每个像素外观的贡献*。这条定义天然是数据并行的：同一个"着色程序"被套用在海量顶点流、片元流、像素流上。理解这段历史的目的，是理解 CUDA 为什么长成现在这个样子——它的抽象是**从图形管线里"长出来"的**，而不是凭空设计的。
- **直观解释（"它是什么？"）**：把渲染管线想成**一条印刷流水线**：先送来一堆"点坐标"（vertex），拼成"图章"（primitive，三角形），把图章盖到纸上决定盖住哪些格子（rasterization，光栅化 → fragment），对每个格子算颜色（fragment processing / shader），最后写进成品（pixel operations）。程序员能插手的地方是"算颜色"这一站：他写一个小程序（**shader**），管线把这个小程序**自动套用到整条片元流上**。类比：流水线上的喷漆工序——工厂（管线）决定工件怎么运，喷漆配方（shader）由你提供，配方会被自动应用到每一个经过的工件上。
- **架构/机制图解**：早期 GPU 只有一条通路——图形管线；2007 年 NVIDIA Tesla 架构才第一次给 GPU 装上"**compute mode**"接口：应用可以分配显存缓冲区、上传一个 kernel 二进制、然后说"以 SPMD 方式跑 N 份这个 kernel"（`launch(myKernel, N)`）。这个操作比 `drawPrimitives(vertex_buffer)` 简单得多，从此 GPU 有了两条入口。

图 1：从图形管线接口到 compute 接口（GPGPU 的诞生）

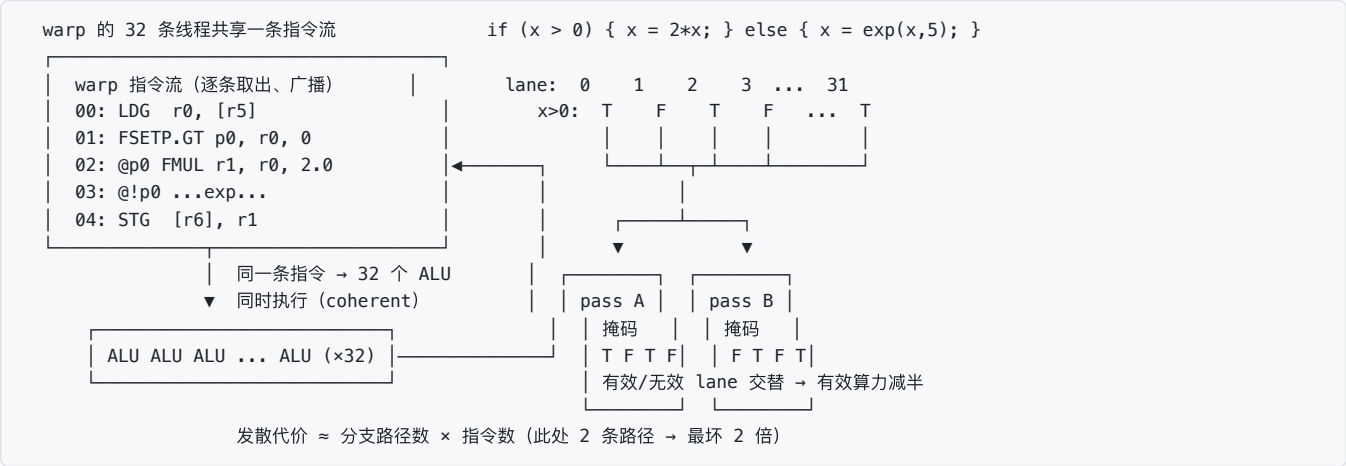
```
[A] 图形接口：2007 年之前通往 GPU 硬件的唯一通路
    应用（经图形驱动）提交 "draw" 命令 + shader 二进制 + 顶点缓冲

    vertex buffer → vertex proc → primitive gen → rasterization → [fragment shader] → pixel op → framebuffer
                                                          |                ▲
                                                          |                └ 硬件把程序员写的 shader
                                                          |                  "map" 到整条片元流上
                                                          └ 关键：一次 shader = 一个片元/像素

[B] compute 接口：2007 年 NVIDIA Tesla 架构打开的第二个入口（GPGPU / CUDA）

    cudaMalloc / cudaMemcpy  →  launch(myKernel, N)  →  以 SPMD 方式跑 N 份 kernel
    对比：launch(myKernel, N) 这个操作比 drawPrimitives(vertexBuffer) 简单得多
          GPU 从此不再只能执行"图形管线计算"
```

- **性能特征与历史坐标**：2001–2003 年的人发现"**GPU 是极快的处理器，它在做同一种计算（shader），作用于大量数据（顶点/片元/像素）**"——这正是 90 年代超级计算机上的数据并行。于是出现了一个著名的 hack：把 OpenGL 的输出图像尺寸设成数组尺寸（如 512×512），画两个刚好铺满屏幕的三角形，于是"**一个像素 = 一个数组元素 = 一次 shader 计算**"，GPU 就被当成数据并行机用了。这就是 GPGPU（General-Purpose computing on GPU）的起点；随后 Brook（2004，Stanford 图形实验室）把"抽象 GPU 为数据并行处理器"变成一门语言：`kernel void scale(...)` + `scale(amount, input_stream, output_stream)`，编译器再把流程序翻译成 `drawTriangles` 之类的图形命令。CUDA（2007）是把这套思路正式固化下来的产物：**"C-like 语言 + 低抽象距离（low abstraction distance）"**。

2.2 SIMD 与 SIMT：warp、coherence 与 divergence

- **定义与目的**：SIMD（Single Instruction, Multiple Data，单指令多数据）指"**同一条指令被广播到多个 ALU 上并行执行**"；用增加 ALU 数量的办法提升算力，代价是控制逻辑被多个 lane 共享，因此**一旦 lane 之间指令不同，就有人必须空转**。CUDA 的执行载体是 **warp**：一个 block 里连续的 32 个 CUDA thread 被绑定为一个 warp，在硬件上以 32 宽 SIMD 方式推进。NVIDIA 把这套"32 条独立标量指令流被硬件动态检测出'恰好相同'后合并执行"的机制叫 **SIMT（Single Instruction, Multiple Thread，单指令多线程）**。
- **直观解释（"它是什么？"）**：**龙舟类比**。32 名桨手（warp 里的 32 个线程）共用一面鼓（warp selector + fetch/decode）：鼓点一响，32 支桨同时入水（一条指令同时作用在 32 个数据上），这就是 **coherent execution（一致执行）**。但如果有人想向左划、有人想向右划（`if (x > 0) ... else ...`），船不可能同时向两边走——只能**先左划一遍（把想右划的人按住不出力，mask 掉），再右划一遍**，这就是 **divergent execution（发散执行）**，代价是效率按分支数成比例下降。SIMT 与 ISPC 的 gang 很像，但有一个关键区别：**ISPC 在编译期就把程序编译成 SIMD 指令，而 GPU 是在运行期由硬件动态判断 32 个独立线程是否恰好走在同一条指令上**；换言之，warp 不是 CUDA 编程模型的一部分，而是实现细节（例外是 `__shfl_*`、vote 这类 intra-warp 原语）。

图 2：SIMT 执行流与分支发散（divergence）

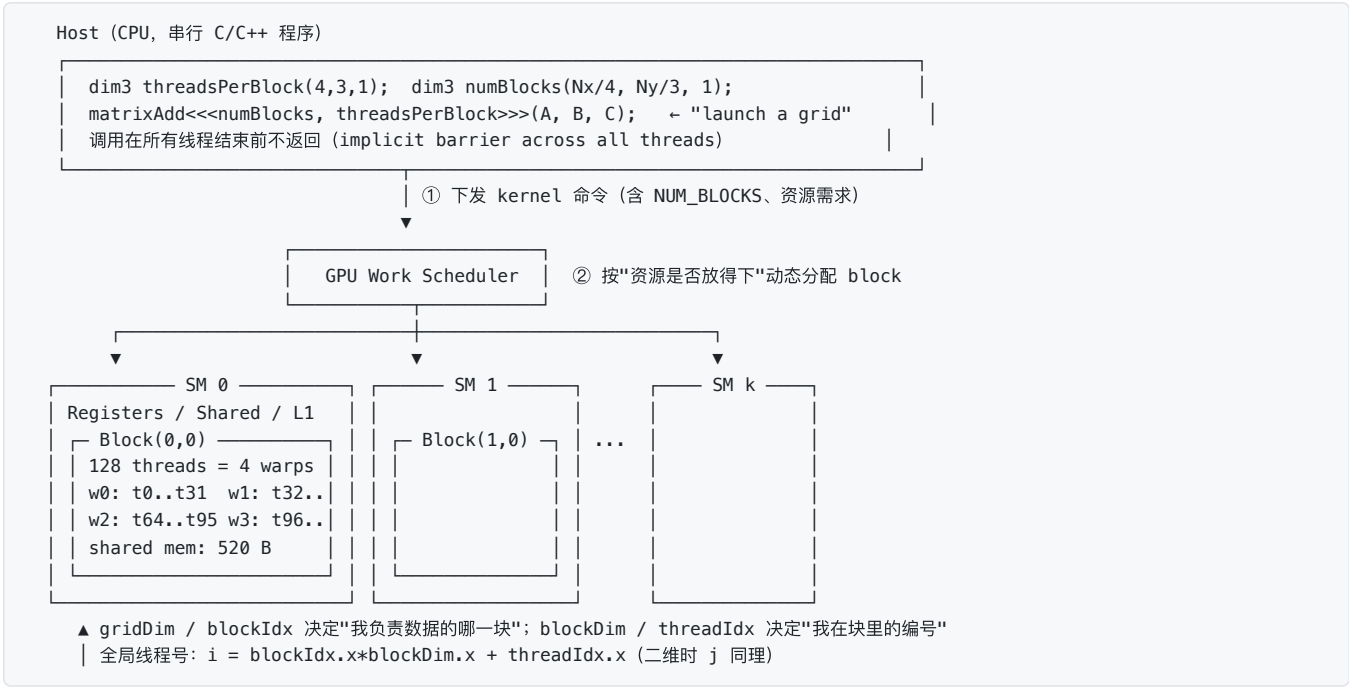


- **性能特征**：warp 内 32 个 lane 走同一路径时，一条 32 宽 SIMD 指令就能服务全部 32 个 CUDA 线程；一旦发散，硬件按掩码（mask）分多次执行，**有效吞吐量下降为"活跃 lane 数 / 32"**。讲义明确把"coherence execution"列为"高效使用 GPU 的必要条件"，把"divergent execution"列为"应在 CUDA 程序中最小化的东西"。

2.3 CUDA 执行模型：grid / thread block / thread 三级层次

- **定义与目的**：CUDA 用一个**两级层次 + 三级索引**来把一个数据并行问题切成"可独立调度的工作单元"：线程（thread）→ 线程块（block）→ 网格（grid）。这样做的目的是让**同一个程序在 6 核 GPU 和 16 核 GPU 上都正确、且尽量高效地运行，而不需要程序里出现 `num_cores`**——把"有多少个核"这件事交给硬件调度器决定。
- **直观解释（"它是什么？"）**：**建筑工地类比**。`grid` 是"整个工程项目"，`block` 是一个"施工班组"，`thread` 是"班组里的一名工人"。项目经理（host 代码）只说"这个活总共要切成 8000 个班组，每班 128 人，各自负责自己编号那一小段"，而**从不说"派到几号楼"**——现场调度员（GPU work scheduler）看哪栋楼（SM）还有空位、资源够不够，就把班组塞进去。班组之间互不通信、无先后依赖（"thread blocks can be executed in any order"），所以怎么排都不会错；而**同一个班组内部的 128 名工人是真正同时在场的**（他们要靠共享内存和 `__syncthreads()` 协作），所以硬件必须为整班人同时准备工位（寄存器 + 执行上下文）。

图 3：CUDA 线程层次 → 硬件映射

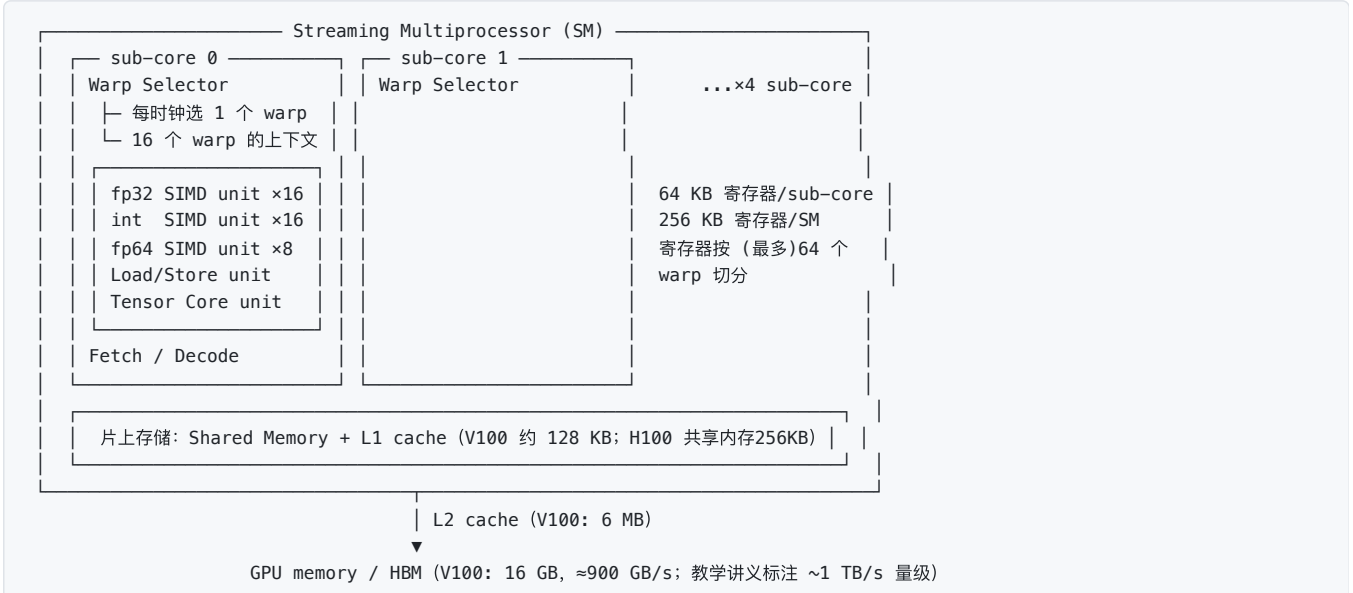


- **语法与语义要点**（讲义给的例子）：`__global__` 表示 kernel 函数在 GPU 上运行、由 host 调用；`__device__` 是只能被 device 代码调用的函数；`__host__` 是普通 CPU 函数。kernel 的**调用数量由程序显式写出，而不是由数据集合的大小自动决定**（这与图形着色器的 `map(shader, stream)` 不同）——因此当 `Nx=11`、`blockDim.x=4` 不能整除时，块数要向上取整，kernel 内部必须写 `if (i < Nx && j < Ny)` 做越界保护。**host 与 device 代码的分离是程序员在源码里静态划定的**。

2.4 GPU 硬件：SM 微架构与 warp selector

- **定义与目的**：SM（Streaming Multiprocessor）是 GPU 的"核"。它内部由若干个 **sub-core** 组成，每个 sub-core 有自己的 warp selector、取指/译码单元、矢量功能单元、寄存器堆分片。SM 的设计目的是**用大量硬件线程（warp）填满功能单元的空隙**：功能单元的延迟以时钟计，但访存/依赖造成的停顿以数十上百时钟计，只有同时驻留几十个 warp、每个时钟挑一个"可发射"的，才能把吞吐量顶到峰值。
- **直观解释（"它是什么？"）**：**机场安检类比**。sub-core 是安检通道，warp selector 是"挑下一位旅客上前"的安检员，寄存器堆是每个旅客的随身行李格。如果当前这位旅客行李还没过机（load 未返回），安检员不会傻等，而是**立刻换下一位（换个 warp）**——这就是硬件多线程（也称 interleaved multithreading）隐藏延迟的方式。另外，**sub-core 上的功能单元天生是 SIMD 的**：以 CS149 补充讲义里的 V100 sub-core 为例，每个 sub-core 有 16 个 fp32 ALU（"32 宽 SIMD 操作每 2 个时钟一次"）、16 个 int ALU、8 个 fp64 单元（"32 宽操作每 4 个时钟一次"）、一组 LSU 和 Tensor Core。

图 4：一个 SM 的内部结构（以 V100 量级的 SM 为参照）



- **warp 的调度语义与状态**：block 一旦开始执行，**"block 内所有线程就都已存在且已分配寄存器状态"**（这是 CUDA 的语义约束，不是实现自由）。原因是 block 内线程之间**可以有依赖**——最简单的依赖就是 `__syncthreads()`。因此**不能"先跑完线程 0–127，再跑线程 128–255"**来"实现"一个 256 线程的 block：如果用 128 个线程先去等 256 线程的 barrier，就会死锁。CUDA 的语义保证是：**block 内线程并发运行，只要一个线程是 runnable 的，它最终一定会被运行（不会饿死）**。

2.5 线程块调度：work scheduler 如何在时间上填满 GPU

- **定义与目的**：GPU 的 work scheduler 是硬件里的一块专用逻辑，负责把 grid 里的线程块**按资源需求**映射到各个 SM 上。它存在的意义是：让同一份 CUDA 程序在任意核数的 GPU 上都能跑，且尽量跑满。
- **直观解释（"它是什么？"）**：**"班车 + 座位"类比**。kernel 编译产物里写着"我这个班有多少人、需要多少共享内存、每人占多少寄存器"；调度器就像调度员，看见哪辆车（SM）上还有连续的座位（执行上下文）和放行李的地方（共享内存），就安排一个班组上去。放不下的班组就在门口等，绝不硬塞。**"问题 → 子问题（任务）→ 工作线程池"这个分解-分配模式在本课程已经出现过多次**：ISPC 的 task 启动、Web 服务器的线程池（线程数由核数决定，而不是由请求数决定）都是同一模式，这里是它在硬件里的版本。

图 5：线程块调度的时间线（讲义用"两个 SM 的虚构 GPU、1000 个 block、每块 128 线程 + 520 B 共享内存"逐步演示）

```
时间 →
SM0 上下文: [0..127] [128..255]         (每 SM 仅 384 线程上下文 / 1.5 KB 共享内存)
┌──────────────────────────────────────────────────────────────────────┐
│ t0  host 下发: EXECUTE=convolve, NUM_BLOCKS=1000, 需求 128 thr / 520 B     │
│ t1  scheduler: Block0 → SM0(contexts 0-127,  shared 0x000)   NEXT=1      │
│ t2            : Block1 → SM1(contexts 0-127,  shared 0x000)   NEXT=2      │
│ t3            : Block2 → SM0(contexts 128-255, shared 0x520)  NEXT=3      │
│ t4            : Block3 → SM1(contexts 128-255, shared 0x520)  NEXT=4      │
│ t5  第三个 block 放不下 (3×520 B = 1560 B > 1.5 KB) → 必须等占位者完成     │
│ t6  Block0 完成 → 资源回收 → Block4 → SM0(contexts 0-127)     NEXT=5      │
│ t7  Block2 完成 → 资源回收 → Block5 → SM0(contexts 128-255)   NEXT=6      │
└──────────────────────────────────────────────────────────────────────┘
关键点：block 的"起飞顺序"与编号无关（可以任意序）；决定并行度的是**资源**而非线程数
```

- **性能特征**：因为 block 会被"分波（wave）"调度，**每块占用的资源越多，同一 SM 上能同时驻留的 block 就越少**，隐藏延迟的能力（occupancy）就越差。讲义给出的资源账本非常具体：一个 128 线程、130 个 float 共享内存的卷积 block，需要的资源是 **128 个线程 + 520 B 共享内存 + 每线程若干字节本地内存**。把 block 改成 256 线程就会让每块共享内存翻倍（258 floats），在同样的 SM 上可驻留的 block 数减半——**"共享内存"和"寄存器"是限制并行度的两种硬约束**。

2.6 CUDA 存储模型：三种地址空间与两条拷贝路径

- **定义与目的**：CUDA 显式暴露**不同的局部性层级**给程序员：per-thread 私有（寄存器/本地内存）、per-block 共享（`__shared__`）、per-program 全局（global）。暴露它们的目的是**让程序员能把"复用距离"控制在片上**：全局内存慢而大，共享内存快而小，寄存器最快但容量由寄存器文件决定（这一设计哲学与第 3 讲"存储层次反映程序的局部性"完全一致）。
- **直观解释（"它是什么？"）**：**车间类比**。全局内存是"中央仓库"（巨大、远、单次往返几百个时钟）；共享内存是"班组共用的小白板"（就挂在车间墙上，几十个时钟，但只有 96–256 KB，且**只有同一个班组的成员能看**）；寄存器是"工人手上的工具"（1 个时钟，但一人只有几十到二百多个格子）。block 之外的人看不到你的白板，所以**跨 block 通信只能走仓库**（全局内存），而且**没有全局屏障**可用。

图 6：CUDA 内存层次与数据复用（以 matmul 分块为例）

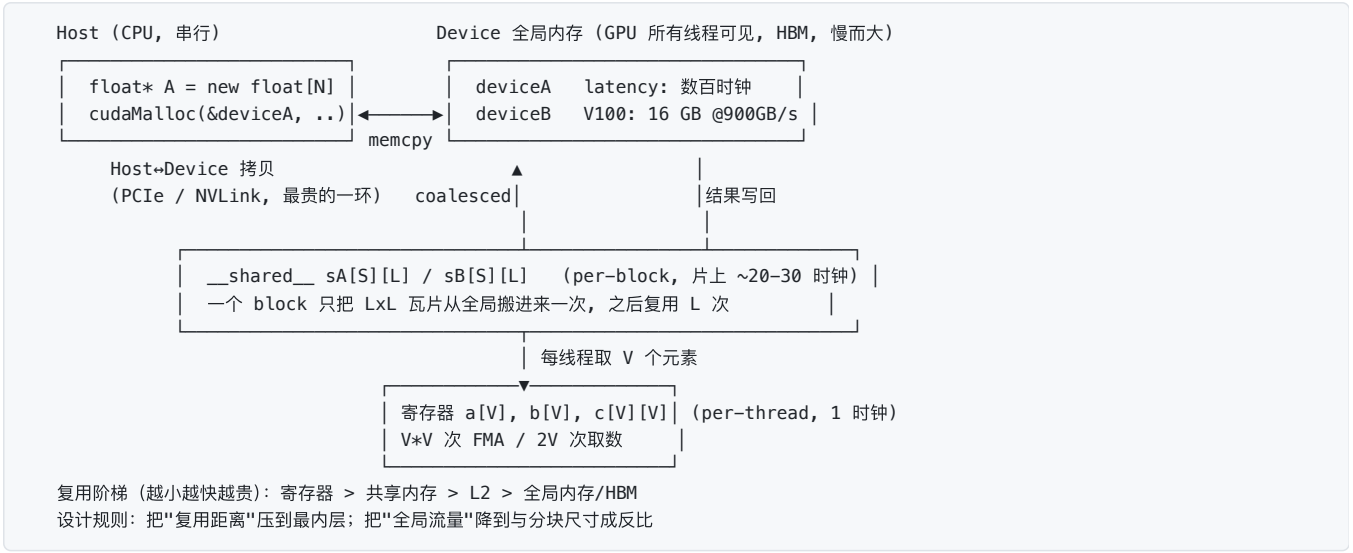


表 1：CUDA 各地址空间对照（量级为公开规格的常见近似值）

地址空间	声明方式	作用域 / 生命周期	典型延迟量级	容量约束	谁可以读写
寄存器 register	自动变量（kernel 内局部标量）	单线程 / 线程生命期	~1 个时钟	每线程有限（常见上限 255 个寄存器）	仅本线程
本地内存 local	溢出的自动变量/数组	单线程 / 线程生命期	类似全局（实际在 DRAM，走 L1/L2 缓存）	由寄存器压力决定	仅本线程
共享内存 shared	`__shared__ float s[L][L];` 或 `extern __shared__`	block 内全部线程 / block 生命期	~20–30 个时钟	每 SM 96 KB（教学用 GTX 980）→ 168 KB（A100）→ 256 KB（H100）	同 block 所有线程
全局内存 global	`cudaMalloc` + kernel 指针参数	全程序 / 直到 `cudaFree`	~400–800 个时钟（HBM）	显存容量（V100: 16 GB）	所有线程，host 需经 `cudaMemcpy`
常量/只读空间*	`__constant__` / `__ldg`	全程序 / 只读	命中常量缓存时很快	64 KB 常量缓存	所有线程只读

* 常量/只读空间属于 CUDA 提供的另一类片上缓存路径，讲义正文只强调了"per-thread / per-block / global"三类，这里作为补充。

- **主从地址空间与拷贝路径**：host 与 device 是**两个互不可见的地址空间**——device 代码不能解引用 host 指针，host 代码也不能直接读写 device 指针指向的内容（`deviceA[i]` 在 host 里是非法操作）。数据必须通过 `cudaMemcpy(dst, src, bytes, kind)` 显式搬运（`cudaMemcpyHostToDevice` / `DeviceToHost` / `DeviceToDevice`）。recitation 讲义把它写成一个 7 步流程：**① host 分配 → ② 初始化输入 → ③ `cudaMalloc` 分配 device 内存 → ④ `cudaMemcpy` 上传 → ⑤ launch kernel → ⑥ `cudaMemcpy` 取回结果 → ⑦ `cudaFree` + `free`**。CUDA 后来也提供 Unified Virtual Memory（统一虚拟内存）来简化这件事，但**性能模型没有变：跨 PCIe/NVLink 的搬运仍然是全程序里最贵的环节之一**（见 §4 的数值算例）。

2.7 访存效率：coalescing 与共享内存 bank conflict

- **定义与目的**：GPU 的内存系统为"**多个线程访问连续地址**"做了优化：一个 warp 的 32 个 lane 如果访问的是连续的内存字，硬件把它们合并成尽量少的、宽度接近 128 B 的事务，这叫 **coalesced access（合并访存）**；如果访问是散乱的（stride 很大），同一个 warp 的请求会被拆成很多个事务，带宽利用率骤降。共享内存则把存储切成 **bank**（通常 32 个 bank，4 字节宽）：一个 warp 的 32 次访问如果**都落在不同 bank**，一个周期就能完成；如果**两个 lane 撞在同一个 bank 的不同字上**，就叫 **bank conflict（体冲突）**，访问被串行化。
- **直观解释（"它是什么？"）**：coalescing 像**快递合单**：32 个人要寄东西到相邻门牌号，快递员一次拉走一整箱（1 个事务）；如果他们要寄到城市各处，快递员就得跑 32 趟（32 个事务）。bank conflict 像**银行 8 个柜台的叫号**：如果 32 个人被均匀分到 8 个柜台，各办各的（无冲突，全速）；如果一半人全挤在 2 个柜台，其他人干等（冲突，串行化，带宽减半）。

图 7：coalesced 与 non-coalesced 访存；无冲突与有冲突的共享内存访问

```
① 全局内存：合并访存（optimal）
   地址：   0    4    8   12   16   20   24   28
          ┌────┬────┬────┬────┬────┬────┬────┬────┐
   第1次  │ t0 │ t1 │ t2 │ t3 │ t4 │ t5 │ t6 │ t7 │  → 1 个事务服务 8 个 lane
          └────┴────┴────┴────┴────┴────┴────┴────┘
② 全局内存：非合并访存（suboptimal，地址间隔 = 4 个字）
          ┌────┬────┬────┬────┬────┬────┬────┬────┐
   第1次  │ t0 │    │    │    │ t1 │    │    │    │  → 每个 lane 都要一个事务
          └────┴────┴────┴────┴────┴────┴────┴────┘

③ 共享内存无冲突：lane t0..t7 访问 word 0,1,2,3,4,5,6,7
      B0  B1  B2  B3  B4  B5  B6  B7
      t0  t1  t2  t3  t4  t5  t6  t7     每个 bank 服务 1 个 lane → 1 个周期，满带宽
④ 共享内存 2-way 冲突：lane t0..t7 访问 word 0,2,4,6,8,10,12,14
      B0     B1    B2     B3    B4     B5    B6     B7
      t0,t4   –    t1,t5   –    t2,t6   –    t3,t7   –    4 个 bank 各服务 2 个 lane
                                                          → 串行化 2 个周期，带宽减半
```

- **性能特征**：这两条规则解释了 CUDA 优化的绝大部分收益来源。归约的"交错寻址（interleaved addressing）"版本之所以慢，正是因为它**同时踩了两个坑**：s 很小时 warp 内 `tid % (2s) == 0` 的判定让活跃 lane 稀疏分布（**发散**），而且 `sdata[tid] + sdata[tid+s]` 的第二次访问步长是 2（**2-way bank conflict**）。改成"顺序寻址"（`s < blockDim.x/2` 递减、判据 `tid < s`）后，活跃 lane 集中在前缀（**warp 级别不发散，整 warp 可以整体跳过**），且两次访问都是连续地址（**无 conflict**）。

2.8 同步：`__syncthreads()` 与"CUDA 没有全局屏障"

- **定义与目的**：block 内部需要协作（例如"大家先把瓦片搬进共享内存，再开始算"），所以 CUDA 提供了 block 级屏障 `__syncthreads()`、原子操作（`atomicAdd` 等，可作用于 global 与 shared），以及"kernel 返回时对所有线程的隐式屏障"。**但没有 block 之间的同步**——这不是疏漏，而是设计选择。
- **直观解释（"它是什么？"）**：`__syncthreads()` 是**班组内部的"点名"**：所有人都到齐了才能继续（仅限同一车间/同一 block）。而**跨车间没有点名机制**：车间之间只能通过"下一班次"（下一个 kernel）来对表——**kernel 的结束/启动本身就是一个全局同步点**，代价极低（讲义强调"kernel launch has very low hardware/software overhead"）。
- **为什么没有全局同步**：讲义给了两条理由：(1) 在 SM 数量极多的 GPU 上做硬件全局屏障代价高昂；(2) 当 **block 数 > SM 数 × 每 SM 可驻留 block 数**时，等待其它 block 的 block 会永远等不到（**死锁**）。补救办法是把计算**分解成多个 kernel**（kernel decomposition）：所有层次用同一段代码逻辑，靠多次 launch 串起"全局同步"。

同步手段	作用范围	谁来用	代价	典型用途
`__syncthreads()`	同一 thread block 内所有线程	block 内协作	数十个时钟（warp 到齐即可放行）	共享内存"装载 → 使用"之间
原子操作 `atomicAdd/Inc/Exch/...`	global 或 shared 地址	任意线程	冲突时串行化，吞吐随冲突度下降	直方图计数、全局部分和累加
kernel 边界（隐式屏障）	整个 grid	host 通过多次 launch	每次 launch 数微秒级开销	取代不存在的全局屏障（多级归约）
自旋 + 全局标志（实践中**危险**）	跨 block	手工实现	可能死锁/饿死	只在"所有 block 保证同时驻留"时才勉强成立

2.9 Tensor Core：为什么 H100 的峰值能到 1000 TFLOPs

- **定义与目的**：Tensor Core 是 SM 里的**矩阵乘加专用单元**（讲义原话："Matrix multiplication unit in SMM"）。它把"小矩阵乘（如 4×4 或更大）的乘加阵列"做成硬件，使单位时钟完成的 MAC 数量级远高于通用 fp32 SIMD 单元——这正是从 GTX 980 的 4.6 TFLOPs 到 H100 的 1000 TFLOPs 的主要原因（讲义明确说明"mainly because of tensor cores"）。
- **直观解释（"它是什么？"）**：通用 SIMD 单元像**通用机床**，什么都能做但一次只能加工一个工件；Tensor Core 像**专用冲压模具**：只做"矩阵乘"这一种活，但一次冲出一整排。代价是**你必须有足够多的矩阵乘数据喂给它**，否则模具空转——这正是后续"算术强度"分析要量化的东西：算力越强，达到峰值所需的算术强度（FLOP/Byte）就越高。
- **硬件演进对照（讲义 slide "GTX 980 (2014) → H100 (2022)"）**：

参数	GTX 980 (2014)	H100 (2022)	说明
时钟频率	1.1 GHz	1.11 GHz	几乎没变

参数	GTX 980 (2014)	H100 (2022)	说明
每 SM 可驻留 warp 数	64	64	没变
warp 宽度（线程/warp）	32	32	没变
每 SM 共享内存	96 KB	256 KB（A100 为 168 KB）	片上容量随工艺增长
SM 数量	16	132	8.25×
峰值算力	4.6 TFLOPs	1000 TFLOPs（主要靠 tensor core）	≈217×

---

3. 代码示例与性能分析

3.1 示例 A：1D 卷积的两个版本（直接全局访存 vs 共享内存暂存）

代码（`convolve_1d.cu`）——对应讲义 "1D convolution version 1 / version 2"：

```
// 编译: nvcc -O3 -arch=sm_70 convolve_1d.cu -o convolve_1d
//       (release 优化用 -O3; 需要行号信息可加 -lineinfo; 只做调试用 -g -G, 会关闭优化)
// 运行: ./convolve_1d 1048576
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define THREADS_PER_BLK 128

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err__ = (call);                                             \
    if (err__ != cudaSuccess) {                                             \
      fprintf(stderr, "CUDA error %s at %s:%d\n",                           \
              cudaGetErrorString(err__), __FILE__, __LINE__);               \
      exit(EXIT_FAILURE);                                                   \
    }                                                                       \
  } while (0)

// ---------- Version 1: 每个线程直接读 3 个全局元素 ----------
__global__ void convolve_v1(int N, const float* __restrict__ input,
                            float* __restrict__ output) {
  int index = blockIdx.x * blockDim.x + threadIdx.x;  // 全局线程号 = 输出下标
  if (index >= N) return;                             // 越界保护
  float result = 0.0f;
#pragma unroll
  for (int i = 0; i < 3; i++) result += input[index + i];
  output[index] = result / 3.0f;
}

// ---------- Version 2: 一个 block 协作把 support 区搬进共享内存 ----------
__global__ void convolve_v2(int N, const float* __restrict__ input,
                            float* __restrict__ output) {
  __shared__ float support[THREADS_PER_BLK + 2];       // 每块分配 130 个 float
  int index = blockIdx.x * blockDim.x + threadIdx.x;
  int tid = threadIdx.x;
  support[tid] = input[index];                         // 130 次装载取代 3*128 次
  if (tid < 2) support[THREADS_PER_BLK + tid] = input[index + THREADS_PER_BLK];
  __syncthreads();                                     // 全 block 屏障
  float result = 0.0f;
#pragma unroll
  for (int i = 0; i < 3; i++) result += support[tid + i];
  if (index < N) output[index] = result / 3.0f;
}

int main(int argc, char** argv) {
  const int N = (argc > 1) ? atoi(argv[1]) : (1 << 20);   // 默认 1M 个输出元素
  // 输入需要 N+2 个元素; 再额外留 2*THREADS_PER_BLK 的尾部填充,
  // 这样即使 N 不是 block 大小的整数倍, 块内"偷看"后续 128 个元素的读也不会越界
  const int inLen = N + 2 + 2 * THREADS_PER_BLK;
  const size_t inBytes = sizeof(float) * inLen;
  const size_t outBytes = sizeof(float) * N;

  float* h_in = (float*)malloc(inBytes);
  float* h_out = (float*)malloc(outBytes);
  float* h_ref = (float*)malloc(outBytes);
  for (int i = 0; i < inLen; i++) h_in[i] = sinf(0.001f * i);
  for (int i = 0; i < N; i++)     h_ref[i] = (h_in[i] + h_in[i + 1] + h_in[i + 2]) / 3.0f;

  float *d_in = nullptr, *d_out = nullptr;
  CUDA_CHECK(cudaMalloc(&d_in, inBytes));
  CUDA_CHECK(cudaMalloc(&d_out, outBytes));
  CUDA_CHECK(cudaMemcpy(d_in, h_in, inBytes, cudaMemcpyHostToDevice));

  dim3 block(THREADS_PER_BLK);
  dim3 grid((N + THREADS_PER_BLK - 1) / THREADS_PER_BLK);  // 向上取整
  cudaEvent_t t0, t1;
  CUDA_CHECK(cudaEventCreate(&t0));
  CUDA_CHECK(cudaEventCreate(&t1));
```

```
  for (int which = 1; which <= 2; ++which) {
    CUDA_CHECK(cudaMemset(d_out, 0, outBytes));
    CUDA_CHECK(cudaEventRecord(t0));
    for (int rep = 0; rep < 20; ++rep) {                  // 多次重复取平均
      if (which == 1) convolve_v1<<<grid, block>>>(N, d_in, d_out);
      else            convolve_v2<<<grid, block>>>(N, d_in, d_out);
    }
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));
    float ms = 0.f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    CUDA_CHECK(cudaMemcpy(h_out, d_out, outBytes, cudaMemcpyDeviceToHost));

    double maxErr = 0.0;
    for (int i = 0; i < N; i++) maxErr = fmax(maxErr, fabs(h_out[i] - h_ref[i]));
    // 有效带宽 = (逻辑上的读 + 写) 字节数 / 时间 (不计尾部填充)
    double gb = (double)(sizeof(float) * (N + 2) + outBytes) / 1e9;
    printf("version %d: %.3f ms/iter, 有效带宽 %.1f GB/s, maxErr=%.2e\n",
           which, ms / 20.0, gb / (ms / 20.0 / 1e3), maxErr);
  }

  CUDA_CHECK(cudaFree(d_in));
  CUDA_CHECK(cudaFree(d_out));
  free(h_in); free(h_out); free(h_ref);
  return 0;
}
```

- **【代码做什么？】** host 端按 recitation 讲义给出的 7 步流程走：分配 host 内存 → 初始化 → `cudaMalloc` device 内存 → `cudaMemcpy` 上传 → launch kernel（`convolve<<<N/THREADS_PER_BLK, THREADS_PER_BLK>>>`）→ 拷回 → 释放。kernel 里 `index = blockIdx.x * blockDim.x + threadIdx.x` 是全局线程号，一个线程算一个输出元素 `output[index] = (input[index]+input[index+1]+input[index+2])/3`。版本 2 的差别在于：**block 的 128 个线程先协作把 130 个输入值（本块需要的 support 区）搬进共享内存**，`__syncthreads()` 之后各自从共享内存里取自己需要的 3 个值。
- **【并行机制与性能解说】**：kernel 启动后，每个 block（4 个 warp）被 work scheduler 分配到某个 SM 上；SM 的每个 sub-core 每时钟从驻留 warp 中挑一个可发射的 warp，warp 里 32 个 lane 同时处理 32 个相邻输出。访存方面，`input[index]` 是**完全连续的**，所以 v1 的三次装载都是 coalesced；v2 的 `support[tid] = input[index]` 同样连续，但**装载指令数从每线程 3 次降到每 block 130 次**（3×128 = 384 → 130，约 2.95×）。
- **Work（总工作量）**：Θ(N)。v1 每线程 3 次全局 load + 2 次 add + 1 次 store；v2 每 block 130 次全局 load + 各自 3 次共享 load。**Work 的阶不变**，改变的只是"从哪里取数"。
- **Span（关键路径）**：Θ(1)。每个线程内部的依赖链长度是常数（3 次加法），block 内唯一的同步点是 1 个 `__syncthreads()`。若把 span 记作"最长依赖链 + 同步点数"，本 kernel 的 span = O(1) 不随 N 增长。
- **并行度 = Work/Span = Θ(N)**：N = $2^{20} \approx 1.05\times10^6$ 个可并行线程。
- **可扩展性上限**：并行度不是瓶颈——V100 可同时驻留 80 SM × 64 warp × 32 = **163,840 个 CUDA 线程**，本问题需要约 6.4 "波"。真正的瓶颈是**内存带宽**：v1 与 v2 的 DRAM 流量几乎相同（每个输入元素仍要从 HBM 取一次、每个输出写一次），所以 **v2 不会降低 DRAM 流量，它降低的是 L1/共享内存通路上的请求数与指令数**。当访存模式本身不合并、或同一数据被线程反复从 L1 请求导致 L1 带宽成为瓶颈时，v2 才真正救场。

3.2 示例 B：分块矩阵乘（Case Study 1：两层复用）

代码（`mm_tiled.cu`）——对应讲义 "Optimization 1: thread-level register tiling" + "Optimization 2: block-level shared memory tiling"：

```
// 编译: nvcc -O3 -arch=sm_70 mm_tiled.cu -o mm_tiled
// 运行: ./mm_tiled 1024
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define L   32   // block 级瓦片尺寸: 一个 block 计算 C 的 L x L 子矩阵
#define BLK 16   // block 维度: BLK x BLK = 256 个线程
#define V   2    // thread 级寄存器瓦片: 每个线程算 V x V 个元素

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t e__ = (call);                                               \
    if (e__ != cudaSuccess) { fprintf(stderr, "%s\n", cudaGetErrorString(e__)); exit(1);} \
  } while (0)

// A, B, C 都是 N x N 的行主序矩阵
__global__ void mm_tiled(const float* __restrict__ A,
                         const float* __restrict__ B,
                         float* __restrict__ C, int N) {
  __shared__ float sA[L][L];
  __shared__ float sB[L][L];

  const int tx = threadIdx.x, ty = threadIdx.y;
  const int tid = ty * BLK + tx;
  const int rowBase = blockIdx.y * L;       // 本 block 负责 C 的 [rowBase, colBase)
  const int colBase = blockIdx.x * L;
  const int nThreads = BLK * BLK;

  float c[V][V];
#pragma unroll
  for (int i = 0; i < V; ++i)
#pragma unroll
    for (int j = 0; j < V; ++j) c[i][j] = 0.0f;
  float a[V], b[V];

  for (int ko = 0; ko < N; ko += L) {
#pragma unroll
    for (int j = 0; j < L * L / nThreads; ++j) {     // 协作装载: 每线程装 4 个元素
      int idx = j * nThreads + tid;
      int r = idx / L, cc = idx % L;
      sA[r][cc] = A[(rowBase + r) * N + (ko + cc)];  // A 的行瓦片
      sB[r][cc] = B[(ko + r) * N + (colBase + cc)];  // B 的列瓦片
    }
    __syncthreads();                                 // 装载完成才能被使用

#pragma unroll
    for (int k = 0; k < L; ++k) {                    // 在瓦片上做 L 次外积
#pragma unroll
      for (int i = 0; i < V; ++i) a[i] = sA[ty * V + i][k];
#pragma unroll
      for (int j = 0; j < V; ++j) b[j] = sB[k][tx * V + j];
#pragma unroll
      for (int i = 0; i < V; ++i)
#pragma unroll
        for (int j = 0; j < V; ++j) c[i][j] += a[i] * b[j];
    }
    __syncthreads();                                 // 下一轮装载前必须用完
  }

#pragma unroll
  for (int i = 0; i < V; ++i)
#pragma unroll
    for (int j = 0; j < V; ++j)
      C[(rowBase + ty * V + i) * N + (colBase + tx * V + j)] = c[i][j];
}

int main(int argc, char** argv) {
  const int N = (argc > 1) ? atoi(argv[1]) : 1024;
  const size_t bytes = sizeof(float) * (size_t)N * N;
```

```
  float *hA = (float*)malloc(bytes), *hB = (float*)malloc(bytes), *hC = (float*)malloc(bytes);
  for (int i = 0; i < N * N; i++) { hA[i] = 1.0f; hB[i] = 2.0f; }   // 结果应为 2N

  float *dA, *dB, *dC;
  CUDA_CHECK(cudaMalloc(&dA, bytes));
  CUDA_CHECK(cudaMalloc(&dB, bytes));
  CUDA_CHECK(cudaMalloc(&dC, bytes));
  CUDA_CHECK(cudaMemcpy(dA, hA, bytes, cudaMemcpyHostToDevice));
  CUDA_CHECK(cudaMemcpy(dB, hB, bytes, cudaMemcpyHostToDevice));

  dim3 block(BLK, BLK);                    // 256 线程/块
  dim3 grid(N / L, N / L);                 // (N/L)^2 个块
  cudaEvent_t t0, t1; cudaEventCreate(&t0); cudaEventCreate(&t1);
  CUDA_CHECK(cudaEventRecord(t0));
  for (int rep = 0; rep < 5; ++rep) mm_tiled<<<grid, block>>>(dA, dB, dC, N);
  CUDA_CHECK(cudaEventRecord(t1));
  CUDA_CHECK(cudaEventSynchronize(t1));
  float ms = 0.f; cudaEventElapsedTime(&ms, t0, t1);
  CUDA_CHECK(cudaMemcpy(hC, dC, bytes, cudaMemcpyDeviceToHost));

  double flops = 2.0 * N * N * N;          // 每个 C 元素 N 次 MAC = 2N 次浮点运算
  printf("N=%d: %.3f ms, %.1f GFLOP/s, C[0]=%f (应为 %d)\n",
         N, ms / 5.0, flops / (ms / 5.0 / 1e3) / 1e9, hC[0], 2 * N);
  return 0;
}
```

- **【代码做什么？】** 每个 block 负责 C 的一个 L×L 子矩阵（L=32），块内 256 个线程各负责一个 V×V（V=2）的小块，即 16×16 个线程 × 2×2 = 32×32。主循环以 S=L 为步长沿 K 维推进：**每个 block 协作把 A 的 L×S 行瓦片与 B 的 S×L 行瓦片搬进共享内存**（`L*L/nThreads = 4` 个元素/线程，访存完全连续 → coalesced），屏障后每个线程从共享内存里反复取数，做 L 次"外积"更新自己的 V×V 累加器，最后一次性写回 C。
- **【并行机制与性能解说】**：grid 有 $(N/L)^2$ 个 block，全部可乱序调度；块内 256 线程 = 8 个 warp，每个 warp 的 32 个 lane 协同装载 128 B 连续数据（完美合并）。真正的"复用"发生在两级：**共享内存层复用 L 倍**（同一个 A/B 瓦片被 block 内所有线程用），**寄存器层复用 V 倍**（同一个 `a[i]` 被 V 个累加器用、同一个 `b[j]` 被 V 个累加器用）。
- **讲义给的流量公式**（本讲的定量核心）：
  - Strawman（每线程算一个元素）：全局访存 = $2N^3$ 次；算术强度极低。
  - 寄存器分块后：每线程 2NV 次 → 总全局访存 **$2N^3/V$**。
  - 再叠加 block 级共享内存分块后：每 block 2NL 次 × (N^2/L^2) 个 block → 总全局访存 **$2N^3/L$**；而共享内存访问总数是 **$2N^3/V$**。
  - 结论：**L 与 V 是"两个复用旋钮"**（讲义原话："L and V are the two reuse knobs!"）。
- **Work**：$\Theta(N^3)$ 次乘加（$2N^3$ FLOPs）。N = 1024 时为 2.15×10^9 FLOPs。**分块不改变 Work，只改变访存**。
- **Span**：$\Theta(N)$。关键路径是单个累加器 `c[i][j]` 沿 k 的 N 次串行累加（寄存器里是依赖链），这是本 kernel 无法回避的深度。
- **并行度 = Work/Span = $2N^3/N = \Theta(N^2) \approx 2.1\times10^6$**，远大于硬件可同时驻留的线程数，因此**并行度不是瓶颈，带宽/共享内存带宽才是**（见 §4 的 Roofline 算例）。
- **共享内存是否会成为新瓶颈**：按讲义公式，共享内存访问次数 = $2N^3/V$，即**"每次 FMA 对应 2/V 次共享取数"**。V=2 时相当于"1 次 FMA 配 1 次共享取数"：在 V100 的 sub-core 里，一条 32 宽共享 load 若完全无冲突需要 1 个 LSU 周期，而一条 32 宽 fp32 FMA 在 16 宽 ALU 上要 2 个周期，于是 LSU 利用率约 50%——**还没到瓶颈，但已经不低**；V 增大（4 或 8）会把该比率降到 1/4、1/8，同时把寄存器压力抬高（`c[V][V]` 需要 V^2 个寄存器）。这就是"分块尺寸是资源与带宽之间的取舍"的具体含义。
- **讲义伪代码的排版问题**：讲义第 64/65 页的伪代码里 `b[:] = sA[ki, ...]` 应为 `sB[ki, ...]`（B 的瓦片来自 `sB`）。上面给出的实现按语义修正了这一点。

3.3 示例 C：并行归约（Case Study 2：树形归约 + 顺序寻址 + warp shuffle）

代码（`reduce.cu`）——综合讲义 "reduce0 交错寻址 → strided index → sequential addressing" 三代改进与 CS149 的 two-pass 分解：

```cpp
// 编译: nvcc -O3 -arch=sm_70 reduce.cu -o reduce
// 运行: ./reduce 16777216
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>

#define BLK 256

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t e__ = (call);                                               \
    if (e__ != cudaSuccess) { fprintf(stderr, "%s\n", cudaGetErrorString(e__)); exit(1);} \
  } while (0)

// warp 内归约: 用 __shfl_down_sync 在 32 个 lane 之间交换寄存器, 无需共享内存与屏障
__inline__ __device__ float warpReduceSum(float v) {
#pragma unroll
  for (int offset = 16; offset > 0; offset >>= 1)
    v += __shfl_down_sync(0xffffffffu, v, offset);
  return v;
}

// 第一/第二阶段共用的 kernel: 每个线程读 2 个元素, 块内树形归约, 写 1 个部分和
__global__ void reduce_kernel(const float* __restrict__ in,
                              float* __restrict__ out, int n) {
  extern __shared__ float sdata[];                      // 动态共享内存, 长度 = blockDim.x
  const unsigned tid = threadIdx.x;
  const unsigned i = blockIdx.x * (blockDim.x * 2) + tid;

  float v = 0.0f;
  if (i < n) v = in[i];                                 // 连续访存 → coalesced
  if (i + blockDim.x < n) v += in[i + blockDim.x];      // 第二个元素, 同样连续
  sdata[tid] = v;
  __syncthreads();

  // 阶段 1: 顺序寻址 (sequential addressing), 活跃线程集中在低位前缀
  for (unsigned s = blockDim.x >> 1; s > 32; s >>= 1) {
    if (tid < s) sdata[tid] += sdata[tid + s];          // 无 bank conflict (连续地址)
    __syncthreads();                                    // 每轮都需要屏障
  }
  // 阶段 2: 剩下 64 个部分和, 用 1 个 warp + shuffle 收尾 (不需要再屏障)
  if (tid < 32) {
    float w = sdata[tid] + ((tid + 32 < blockDim.x) ? sdata[tid + 32] : 0.0f);
    w = warpReduceSum(w);
    if (tid == 0) out[blockIdx.x] = w;
  }
}

int main(int argc, char** argv) {
  const int n = (argc > 1) ? atoi(argv[1]) : (1 << 24);
  const size_t bytes = sizeof(float) * n;
  float* h_in = (float*)malloc(bytes);
  double ref = 0.0;
  for (int i = 0; i < n; i++) { h_in[i] = 1.0f; ref += 1.0f; }

  float* d_src; float* d_dst;
  CUDA_CHECK(cudaMalloc(&d_src, bytes));
  const int firstBlocks = (n + 2 * BLK - 1) / (2 * BLK);
  CUDA_CHECK(cudaMalloc(&d_dst, sizeof(float) * (firstBlocks + 1)));
  CUDA_CHECK(cudaMemcpy(d_src, h_in, bytes, cudaMemcpyHostToDevice));

  cudaEvent_t t0, t1; cudaEventCreate(&t0); cudaEventCreate(&t1);
  CUDA_CHECK(cudaEventRecord(t0));

  // 多级 kernel 分解: 每一级都是"全局同步点", 直到只剩 1 个元素
  int cur = n, launched = 0;
  float* src = d_src; float* dst = d_dst;
  while (cur > 1) {
    int blocks = (cur + 2 * BLK - 1) / (2 * BLK);
    reduce_kernel<<<blocks, BLK, BLK * sizeof(float)>>>(src, dst, cur);
    CUDA_CHECK(cudaGetLastError());
```

```
    ++launched;
    cur = blocks;
    float* tmp = src; src = dst; dst = tmp;              // swap 双缓冲
  }
  CUDA_CHECK(cudaDeviceSynchronize());
  CUDA_CHECK(cudaEventRecord(t1));
  CUDA_CHECK(cudaEventSynchronize(t1));
  float ms = 0.f; cudaEventElapsedTime(&ms, t0, t1);

  float total = 0.f;
  CUDA_CHECK(cudaMemcpy(&total, src, sizeof(float), cudaMemcpyDeviceToHost));
  printf("n=%d: %.3f ms (%d 次 kernel launch, %.1f GB/s), sum=%f 期望=%f\n",
         n, ms, launched, (double)bytes / (ms * 1e-3) / 1e9, total, ref);
  return 0;
}
```

- **【代码做什么？】** 归约是归一化、softmax 等算子的基础原语（讲义原话）。这里用**两级分解**：第一级把 n 个元素切成 `2*BLK` 一组的块，每块树形归约成 1 个部分和写回全局内存；第二级（以及后续级）继续对部分和做同样的事，直到只剩 1 个数。**因为 CUDA 没有跨 block 同步，多级之间的"全局同步"由 kernel 边界提供**。recitation 讲义列出了四种做归约的思路：把每轮循环做成一个 kernel（正确但慢，launch 开销大）、一个 kernel 内用 `__syncthreads()`（**错误**——它只保证块内）、只启动 1 个 block（正确但没法用满 GPU，适合小数据）、以及"块内 `__syncthreads()` + 跨块 kernel"（本示例采用的折中）。
- **【并行机制与性能解说】**：第一级有 n/(2·256) = 32768 个 block（n=16M），每个块 8 个 warp；`sdata[tid] += sdata[tid+s]` 是**顺序寻址**，两次访问都落在连续地址上，无 bank conflict；等到 `s < 32` 时，只有 warp 0 需要继续工作，其余 7 个 warp 因判据整体为假而**直接退出，不再发射指令**（这正是"stride 版 vs 顺序版"的关键差别），最后用 `__shfl_down_sync` 在 warp 内做无共享内存的收尾。
- **Work**：Θ(n) 次加法（n−1 次）。$n=2^{24}$ 时 ≈ 1.68×10^7 次加法。
- **Span**：Θ(log n)（树高）+ O(1) 次 kernel 边界。$n=2^{24}$ 时块内树高 $\log_2(256)=8$ 层，跨级约 24/8 ≈ 3 层 kernel，总同步深度约 11–12 个"阶段"。
- **并行度 = $\Theta(n/\log n) \approx 7\times10^5$**（n=16M；span 取树高 $\log_2 n = 24$ 层）。
- **瓶颈**：这个 kernel 是**纯带宽受限**。n=16M 个 float = 64 MB，V100 上 HBM 带宽 900 GB/s → 至少 64 MB / 900 GB/s ≈ **71 μs**；而算力需求只有 1.68×10^7 次加法（算术强度 1.68e7 FLOP / 6.71e7 B ≈ **0.25 FLOP/Byte**），在 12.7 TFLOPs 机器上只要约 **1.3 μs**——两者相差 50 倍以上。两次以上的 kernel launch（每次数微秒）在总时间中占 5–15%，所以"**减少 kernel 数、用 grid-stride loop 或 persistent thread 把多级合并**"是这类 kernel 的常见优化方向。
- **版本对比（讲义三代实现）**：

版本	归约判据 / 索引	warp 内是否发散	共享内存 bank	每轮 warp 活跃情况	结论
V1 交错寻址	`if (tid % (2*s) == 0) sdata[tid] += sdata[tid+s]`	**是**（活跃 lane 稀疏：T F T F…）	**2-way conflict**（第二次访问步长 2）	每个 warp 都要为 T/F 两条路径发指令	最慢，两个坑都踩
V2 strided index	`index = 2*s*tid; if (index < blockDim.x) ...`	否（活跃 lane 连续在前缀）	仍有步长 s 的访问模式	部分 warp 整体不活跃	快于 V1
V3 顺序寻址	`for (s = blockDim.x/2; s > 0; s /= 2) if (tid < s) sdata[tid] += sdata[tid+s]`	否	**无冲突**（两次都是连续地址）	高位 warp 整块退出，不再发射	讲义给出的推荐形态
V4 warp shuffle 收尾	前 31 → 32 用 `__shfl_down_sync`	否	完全不使用共享内存	只剩 1 个 warp 干活	减少屏障次数与共享内存流量

---

4. 性能模型与复杂度分析

4.1 Work–Span 模型在 GPU 上的映射

- **Work（W）**：整个计算的总操作数（与处理器数量无关）。**Span（S，关键路径长度）**：必须串行执行的最长依赖链。**并行度（parallelism）= W / S**，表示"用无限多执行单元时最多能利用的并发度"。

- 在 GPU 上，硬件能提供的"处理器数量"不是 CPU 的核数 p，而是**可同时驻留的 CUDA 线程数**：`P_HW = SM 数 × 每 SM 可驻留 warp 数 × 32`。V100 上是 80 × 64 × 32 = **163,840**。
- 因此 GPU 的加速上限有两道门：`min(W/S, P_HW)` 决定"并行度够不够"，而带宽/共享内存/指令发射带宽决定"喂不喂得饱"。**本讲两个 case study 都撞在第二道门上**。

程序	Work	Span	并行度 W/S	实际瓶颈	说明
1D 卷积（N 个输出）	$\Theta(N)$	$\Theta(1)$	$\Theta(N) \approx 10^6$	HBM 带宽	每字节只做约 1 次运算
分块 matmul（N×N）	$\Theta(N^3) = 2.15\times10^9$ FLOPs	$\Theta(N) = 1024$	$\Theta(N^2) \approx 2.1\times10^6$	全局访存 + 共享内存带宽	分块提高算术强度
树形归约（n 个元素）	$\Theta(n) \approx 1.68\times10^7$	$\Theta(\log n) \approx 24$ 层	$\Theta(n/\log n) \approx 7\times10^5$	HBM 带宽 + kernel 启动开销	每字节只做 0.25 次运算

4.2 Amdahl 定律与"The Free Lunch Is Over"的 GPU 版本

若程序中不可并行的比例为 f，则 p 个执行单元的加速比上限是

```
S(p) = 1 / ( f + (1 − f) / p )          p → ∞ 时 S_max = 1 / f
```

- **CPU 上的 f**：串行段、I/O、锁竞争。
- **GPU 上的 f 至少多了三项**：(1) host↔device 的 `cudaMemcpy`（慢通路，且常常无法与计算重叠）；(2) 每次 kernel launch 的固定开销（数微秒）；(3) 无法并行的 host 侧逻辑（分配、校验、启动）。
- **数值算例（N=1024 的 fp32 matmul，V100 量级）**：
- 计算量 2.15×10^9 FLOPs。kernel 理想时间 = 2.15e9 / 12.7e12 ≈ **169 μs**（100% 峰值；按 §4.3 的实际可达约 300 μs）。
- 数据量：A、B、C 各 $1024^2\times4$ B = 4 MB，共 12 MB。若通过 PCIe Gen3 x16（≈16 GB/s 实测）搬运：12 MB / 16 GB/s ≈ **750 μs**。
- 于是 `f = 750/(750+300) ≈ 0.71`，加速比上限 `1/0.71 ≈ 1.4`——**GPU 的 12.7 TFLOPs 被数据搬运吃掉了**。这正是 CUDA 优化的第一条工程准则：**让数据留在显存里、把多次 kernel 连成流水（避免中间结果来回搬运）**，也正是后续讲座里"算子融合 / 减少 host–device 往返"的动机。
- 若改成"数据已在显存、只做纯计算"（f→0），加速比就由带宽与算力决定，此时 GPU 相对单核 CPU（假设 20 GFLOP/s 有效）可达数百倍。

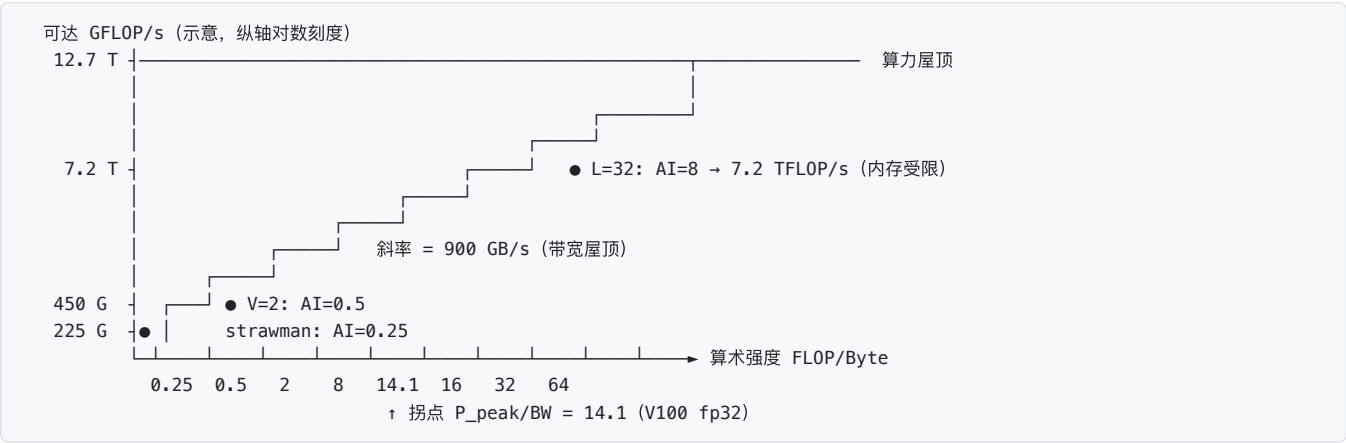
4.3 算术强度与 Roofline 模型（本讲最重要的定量工具）

算术强度（arithmetic intensity） AI = 浮点运算次数 / 从全局内存搬运的字节数（FLOP/Byte）。**Roofline** 模型给出可达性能：

```
可达性能 = min( 峰值算力 P_peak ,  AI × 内存带宽 BW )
"拐点"算术强度 = P_peak / BW
```

数值算例 1：matmul 的三种实现（N = 1024，V100：P_peak = 12.7 TFLOPs，BW = 900 GB/s）

- 拐点 AI = 12.7e12 / 900e9 ≈ **14.1 FLOP/Byte**。
- **Strawman（每线程算一个元素）**：全局访存 $2N^3$ 次 × 4 B = 8.6×10^9 B，FLOPs 2.15×10^9 → **AI = 0.25 FLOP/Byte**（= 每字节 1/4 次运算）。可达性能 = 0.25 × 900 GB/s = **225 GFLOP/s**，只有峰值的 **1.8%**。
- **寄存器分块（V=2）**：总全局访存 $2N^3/V = 1.07\times10^9$ 次 × 4 B = 4.3×10^9 B → AI = **0.5**，可达 ≈ 450 GFLOP/s。
- **再叠加 block 共享内存分块（L=32）**：每 block 访存 2NL 次、共 $(N/L)^2$ 个 block → 总访存 $2N^3/L = 6.7\times10^7$ 次 × 4 B = **268 MB** → **AI = $2N^3/(4\cdot2N^3/L) = L/4 = 8$ FLOP/Byte**。可达 = min(12.7 T, 8×900 G) = **7.2 TFLOP/s**（仍是内存受限）。时间下界 = 268 MB / 900 GB/s ≈ **298 μs**，而算力下界 169 μs → 268 MB 的搬运是主约束，有效效率约 57%。
- **把 L 提到 64**：AI = L/4 = **16 > 14.1** → 跨过拐点，变成**算力受限**：访存时间 134 MB/900 GB/s ≈ 149 μs < 169 μs，上限就是算力。共享内存需求 = 2 × 64 × 64 × 4 B = **32 KB/block**，在 V100 的 128 KB 片上存储里可容纳。
- **通用结论（简洁且好用）**：按讲义公式，float32 分块 matmul 的算术强度 ≈ **AI = L/4 FLOP/Byte**。因此"要撑满峰值算力需要多大的分块"可以由 `L ≥ 4·P_peak/BW` 反推。



数值算例 2：H100 为什么必须用 Tensor Core

- 讲义给的 H100 峰值 1000 TFLOPs（tensor core），HBM 带宽标注 ~1 TB/s 量级。此时拐点 AI = 1000e12 / 1e12 = **1000 FLOP/Byte**。
- 若按公开规格近似（132 SM、每 SM 128 个 fp32 FMA/时钟、1.11 GHz）估算 fp32 通用算力 ≈ 132×128×2×1.11e9 ≈ **37.5 TFLOP/s**，拐点仍需 AI ≈ 37.5 FLOP/Byte，对应 L ⩾ 150——**远超共享内存与寄存器能维持的分块规模**。
- 因此"算力越强，越需要靠**降低数据搬运字节数**（更深的复用、更低精度、更大分块、warp-level 的矩阵乘指令）来维持算术强度"。**Tensor Core 把"每字节能换多少运算"整个抬高了一个量级**：它做的是"在片上一小片寄存器/共享数据上完成更多 MAC"，等于在硬件里内置了更高阶的复用。

数值算例 3：延迟隐藏与 Little 定律

- HBM 延迟量级 ≈ 500 ns，带宽 900 GB/s。要打满带宽，需要"在途（in-flight）"的字节数 = BW × latency = 900e9 × 500e-9 = **450 KB**。
- 分摊到 80 个 SM：**5.6 KB/SM**，即约 1400 个 4 字节的未完成 load **每 SM**。除以 4 个 sub-core 得 350 个 lane-load/sub-core ≈ **11 个满 warp 的 load**。
- 而每个 sub-core 最多驻留 16 个 warp。结论：**必须让每个驻留 warp 平均携带多个未完成 load（ILP），并且让 SM 上驻留足够多的 warp（occupancy）**，才可能打满 HBM。这从数量上解释了讲义里反复强调的三件事：**访存要合并（1 个 warp 的 load 才等于 1 个 128 B 事务，而不是 32 个）、不要用发散把 warp 切碎、不要用过多共享内存/寄存器把可驻留 block 数压下去**。

4.4 一个把三项放在一起的定量对照表

指标	公式	1D 卷积（N=1M）	分块 matmul（N=1024, L=32）	归约（n=16M）
Work	—	2N 次加法 + N 次除法 ≈ 3.1×10^6 FLOPs	$2N^3$ FLOPs ≈ 2.15×10^9	n−1 次加法 ≈ 1.68×10^7
Span	—	Θ(1)	Θ(N) = 1024	Θ(log n) ≈ 24
并行度	W/S	≈ 1.05×10^6	≈ 2.1×10^6	≈ 7×10^5
全局内存流量	—	≈ 8 MB（读 4 MB + 写 4 MB）	268 MB（strawman 8.6 GB）	≈ 64 MB（读一次）
算术强度	FLOP/Byte	≈ 0.37	8（strawman 0.25；L=64 时 16）	≈ 0.25
带宽时间下界	bytes / BW	8 MB / 900 GB/s ≈ **8.9 µs**	268 MB / 900 GB/s ≈ **298 µs**	64 MB / 900 GB/s ≈ **71 µs**
算力时间下界	FLOPs / P_peak	≈ 0.24 µs	169 µs	≈ 1.3 µs
实际受限方	—	带宽（约 37× 于算力）	带宽（约 1.8× 于算力）	带宽（约 55× 于算力）
并行度是否足够	vs 163,840	约 6.4 波	约 13 波	约 4.3 波

---

5. 关键要点

1. **CUDA 的三个抽象支柱：线程层次 + 分层地址空间 + 块内屏障。** grid/block/thread 三级索引（`blockIdx*blockDim+threadIdx`）让"问题分解"与"机器规模"解耦（程序里没有 `num_cores`）；host/device 分离的主从地址空间要求显式 `cudaMemcpy`；`__syncthreads()` 只在 block 内有效，而 **kernel 边界就是 CUDA 提供的唯一全局同步手段**（多级 kernel 分解因此是最重要的结构性手法）。

2. **warp 是性能的基本单位（32 线程 SIMT）。** 一个 block 的线程按 32 个一组绑定成 warp，block 数 = 线程数/32；warp 之内必须"同一条指令"，所以**coherent execution 是高效使用 GPU 的必要条件，divergence 是主要性能损失源**。`__syncthreads()` 的存在意味着"block 内线程必须真的并发存在"，这解释了为什么硬件必须一次性为整个 block 分配寄存器与执行上下文。
3. **两级分块 = 两级复用旋钮。** 全局访存总量从 $2N^3$（strawman）降到 $2N^3/V$（寄存器分块）再降到 $2N^3/L$（共享内存分块）。**L 与 V 是唯二的复用杠杆**，但它们同时受共享内存容量（每 SM 96 KB → 256 KB）与寄存器数量的双重限制，分块放大会牺牲 occupancy。
4. **一切定量判断都可以用"算术强度 vs 拐点"来做。** Roofline：`可达 = min(P_peak, AI×BW)`，拐点 = `P_peak/BW`（V100 fp32 ≈ 14.1 FLOP/Byte）。matmul 的 AI = L/4，L=32 时只有 8 → 内存受限；L=64 才跨过拐点。**算力越强（tensor core），维持峰值所需的算术强度越高**，这是 H100 时代编程方式的根本约束。
5. **访存模式（coalescing / bank conflict）与延迟隐藏决定了"带宽能不能被吃满"。** 一个 warp 访问连续地址才能合并成少量事务；共享内存 32 个 bank 只有"一个 lane 一个 bank"才无冲突（步长 2 的访问直接减半带宽）。同时按 Little 定律，要撑满 900 GB/s 需要约 450 KB 的在途数据，这必须靠"高 occupancy + 每 warp 多个未完成 load"来提供，因此不要为了分块把共享内存/寄存器用到没有余量。

---

6. 常见陷阱与注意事项

- **把 `__syncthreads()` 当成"全局屏障"，以及跨 block 自旋等待。** `__syncthreads()` 只同步**同一个 thread block**：recitation 讲义里的 `reduce_2` 就是典型错误——在多个 block 上跑、循环里放 `__syncthreads()`，跨 block 的依赖完全没有保证（"CUDA 不保证锁步执行"）。方向相反的错误同样致命：CS149 补充讲义里 block 0 自增 `myFlag`、block 1 自旋等 `myFlag != 0` 的写法，在"每 SM 只能驻留 1 个 block"的机器上若调度器先跑 block 1 就会**永久死锁**。正确做法是块内用 `__syncthreads()`、跨块用 kernel 边界（或只启动 1 个 block，仅在小数据时合理）。**合法使用原子操作 ≠ 可以假设 block 之间有执行顺序**；CUDA 只承诺"可以把 block 按任意顺序调度"。
- **忽略发散（divergence）带来的浪费。** `if (tid % (2*s) == 0)` 这类判据让 warp 内活跃 lane 稀疏分布，硬件不得不为两条路径分别发射指令；正确做法是改成**连续前缀**判据（`tid < s`）或转成步长索引，让"整 warp 活跃/整 warp 不活跃"。注意：**发散不一定错，但代价要算清楚**（归约尾部只有 32 个线程干活是可接受的；如果一个 kernel 的每个 warp 长期只有 1/32 的 lane 有效，那就是设计问题）。
- **把非合并访存当成小事，或以为"用了共享内存就一定会更快"。** 一个 warp 的 32 个 lane 访问步长远大于 1 的地址（例如按列访问行主序矩阵）时，一个请求会被拆成 32 个事务，**有效带宽掉到 1/32**——矩阵乘里 `B[k][y]` 的列方向访问就是典型场景，分块到共享内存的一部分动机正是**把非合并的全局访问换成合并的瓦片装载**。但反过来，共享内存不是免费的：它有 32 个 bank，冲突会串行化访问，而且**它是按 block 分配的稀缺资源**。示例 A 的 v2 把全局 load 指令从 3N 降到 N+2，却**没有减少 DRAM 流量**（每个输入元素仍要从 HBM 取一次），所以纯带宽受限时几乎不会变快——它的收益在减少 L1 压力与指令数。
- **忽视 host↔device 搬运与 kernel 启动的固定开销。** §4.2 的算例里，12 MB 的 PCIe 搬运（≈750 μs）可以轻松压过 300 μs 的计算，使加速比上限只有 1.4；此外**每次 kernel 启动都有数微秒级开销**，"为了用 kernel 边界做全局同步而拆成几十次 launch"会把开销累积成主要成本。优化的方向是让数据驻留显存、合并 kernel、用 grid-stride/persistent thread 减少 launch 次数。
- **把 CUDA 线程当 pthread 来理解（抽象与实现的混淆）。** CUDA 线程与 pthread 在"逻辑控制流"这一层相似，但实现截然不同：**pthread 的创建要分配栈与控制块、由 OS 调度；而 CUDA 的百万级线程不会真的分配百万份栈**——block 只有在被调度到 SM 上时才"实化"寄存器状态与共享内存，且同一 block 内的所有线程必须同时实化。混淆这一点会得出错误结论（例如"线程越多越好"）。
- **为分块把资源吃干净，导致 occupancy 崩塌或寄存器溢出（spill）。** `c[V][V]` 累加器随 V^2 增长，共享内存随 L^2 增长；一旦超过"每 SM 共享内存/寄存器除以每 block 占用"，可驻留 block 数下降，延迟隐藏能力变差，甚至寄存器溢出把数据挤到本地内存（实际走 DRAM），性能出现断崖。**分块尺寸必须按目标 GPU 的资源上限反推。**

---

7. 思考题（带答案）

问题 1（执行模型 / 正确性）：recitation 讲义里有一个"错误"的归约实现：

```
__global__ void reduce_2(float* A, int length) {
  int i = threadIdx.x + blockIdx.x * blockDim.x;
  for (int d = 1; d < length; d <<= 1) {
    if (i < length && i % (d << 1) == 0) A[i] += A[i + d];
    __syncthreads();                 // 这里加了块内屏障
  }
}
```

它比"不加任何屏障"的版本更"安全"了吗？为什么它仍然不正确？请给出两个正确的替代方案，并说明各自的适用场景。

【答案】 仍然不正确。`__syncthreads()` **只对同一个 thread block 内的线程有效**，而这个 kernel 让**多个 block 共同处理同一个数组 A**：第 d 轮需要第 d−1 轮的全部结果，但第 d−1 轮的结果可能由**另一个 block** 写出的，块内屏障完全管不到它。所以不同 block 之间仍然存在未同步的读写依赖（数据竞争），结果不确定。两个正确方案：(a) **把每一轮做成一次 kernel launch**（`for (d=1; d<length; d<<=1) reduce_1<<<grid,block>>>(A, length, d);`）——kernel 边界提供全局同步，语义正确；缺点是 launch 次数多、每次数微秒开销，适合 d 的轮数不多时（例如先把 length 降到较小值再逐级 kernel）。(b) **只用 1 个 block +** `__syncthreads()`（`reduce_2_1`）：启动 `<<<1, blockDim>>>`，块内循环里用 `for (; i < length; i += blockDim.x)` 处理整个数组，屏障就是正确的全局屏障。它只在 length 较小（一个 block 能覆盖）时高效，因为一个 block 只能占一个 SM，GPU 其余部分闲置。(c) 更实用的第三种：**混合方案**——块内用 `__syncthreads()` 把每个 block 的一段数组归约成 1 个部分和，跨块再用 kernel launch（或让第二个 kernel 只有 1 个 block）合并，这正是本讲示例 C 采用的两级分解。

问题 2（性能模型 / 定量）：某 GPU 的 fp32 峰值算力为 12.7 TFLOP/s，HBM 带宽 900 GB/s。你要实现 N=1024 的 fp32 矩阵乘 C=A×B，用共享内存分块，块尺寸 L×L。请回答：(1) 分块后的算术强度是多少？(2) 要使程序变成**算力受限**，L 至少要多大？(3) L=32 时的时间下界与可达 GFLOP/s 各是多少？(4) 若换成峰值 1000 TFLOP/s、带宽 1 TB/s 的 H100（假设仍用 fp32 通用单元），这个结论会怎样变化？

【答案】 (1) 全局访存总量是 $2N^3/L$ 次（每次 4 字节），浮点运算 $2N^3$ 次，于是 $AI = 2N^3 / (2N^3/L \times 4\ B) = L/4$ **FLOP/Byte。每字节 1/4 次运算，即"分块把算术强度提升到 L/4"**。(2) 拐点算术强度 = $P_{peak} / BW = 12.7e12 / 900e9 \approx 14.1$ FLOP/Byte。要跨过拐点需 $L/4 \geq 14.1$，即 **$L \geqslant 56.4$，工程上取 $L = 64$**（此时 AI = 16）。L=64 时共享内存需求为 $2 \times 64 \times 64 \times 4\ B = 32\ KB/block$，在 96–256 KB 的片上空间内是可实现的。(3) L=32 时：AI = 8 FLOP/Byte。可达性能 = $\min(12.7\ T, 8 \times 900\ G) = 7.2\ TFLOP/s$（内存受限，只有峰值的 57%）。全局流量 = $2N^3/L \times 4\ B = 2 \times 1.074e9/32 \times 4\ B \approx 268\ MB$，时间下界 = $268\ MB / 900\ GB/s \approx 298\ \mu s$；此时算力下界为 $2.15e9/12.7e12 \approx 169\ \mu s$，所以**带宽是主约束**（298 μs > 169 μs）。(4) 若峰值提高到约 37.5 TFLOP/s（132 SM × 128 fp32 FMA/时钟 × 2 × 1.11 GHz，公开规格近似）而带宽只有 ~1 TB/s，则拐点变成约 **37.5 FLOP/Byte**，需要 $L \geq 150$——共享内存和寄存器都放不下这么大的瓦片。这说明：**通用 fp32 SIMD 的算力提升会把程序推向"必然内存受限"的境地**，因此硬件把复用作进了 Tensor Core（把"每字节的运算数"在单元内部放大），而软件必须走更深的复用、更低精度或 warp 级矩阵指令。这就是讲义"从 GTX 980 的 4.6 TFLOPs 到 H100 的 1000 TFLOPs，主要来自 tensor cores"这一句背后的性能模型含义。

问题 3（实现细节 / 资源与调度）：假设卷积 kernel 的每个 block 需要 128 个线程和 130 个 float 的共享内存（520 B），而目标 GPU 的每个 SM 有 384 个线程的执行上下文和 1.5 KB 共享内存。请解释：(1) 一个 SM 上最多能同时驻留几个 block？(2) 如果把这个 kernel 的 block 改成 256 线程（共享内存相应变成 258 个 float），驻留情况会怎样变化？(3) 为什么硬件不允许"用 128 个线程分两批跑完一个 256 线程的 block"？

【答案】 (1) 两个约束同时起作用：线程上下文 $384 / 128 = 3$ 个，共享内存 $1536\ B / 520\ B = 2.95 \to 2$ 个（第 3 个需要 1560 B > 1.5 KB，放不下）。因此**取较小者：每 SM 最多 2 个 block**（256 个 CUDA 线程、1040 B 共享内存），即**共享内存是先撞上的那道墙**。这也说明 occupancy 的瓶颈常常不是线程数而是片上存储。(2) block 变成 256 线程后：线程上下文 $384 / 256 = 1$ 个，共享内存 $1536 / 1032 = 1$ 个 → **每 SM 只能驻留 1 个 block 的 256 个线程**。相比 (1) 的 2×128 = 256 个线程，**驻留的线程总数相同**，但独立 block 数从 2 降到 1，warp 数从 8 降到 8（256/32），看上去一样，然而**同步粒度变粗**：`__syncthreads()` 要让 8 个 warp 全部到齐，屏障等待的方差更大，且 block 结束时释放的是整块资源——调度灵活性下降。这解释了为什么"block 大小"是一个需要调参的实现细节。(3) 不可以，因为 **CUDA 语义规定 block 内的所有线程并发执行，且它们之间可以有依赖**。最简单的依赖就是 `__syncthreads()`：如果硬件只放得下 128 个线程，却"先跑 0–127 到 barrier、再跑 128–255"，那么先跑的那批会在 barrier 上等一批还没启动的线程，**永久死锁**。所以硬件必须在 block 开始时为**整块**预留寄存器与执行上下文；CUDA 的保证是"只要某线程是 runnable 的，它最终一定会被运行"。

Lecture 6: GPU Architecture and CUDA Programming (continued)

1. 章节标题与概述

Lecture 6: GPU Architecture and CUDA Programming（continued）（GPU 体系结构与 CUDA 编程（续））

- **本讲核心问题**：上一讲回答了"CUDA 的抽象是什么、GPU 硬件大致长什么样"；本讲要回答的是**"为什么同一个 CUDA 程序能差十倍，以及怎样把它写快"**。讲义把答案压缩成三条可以直接动手的优化判据：**coherent warps（warp 内不发散）**、**coalesced memory access（合并访存）**、**高数据复用（shared memory tiling + register tiling）**，并用两个完整 case study——**矩阵乘（Case Study 1）**与**并行归约（Case Study 2）**——把这三条判据从"口号"推到"每一行代码、每一个数字"。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：SM（Streaming Multiprocessor）内部的 sub-core / warp selector / fetch-decode / SIMD 功能单元（fp32、int、fp64、LSU、Tensor Core）与寄存器堆；片上 shared memory + L1；L2 与 HBM（High Bandwidth Memory，高带宽显存，讲义标注 ~1 TB/s 量级）；GPU work scheduler（线程块调度器，按资源需求动态把 block 映射到 SM）；32 个 bank 的共享内存子系统；以及 Tensor Core 带来的算力量级跃变（GTX 980 的 4.6 TFLOPs → H100 的 1000 TFLOPs）。

- **软件侧**：`__shared__`（含 `extern __shared__` 动态共享内存）、`__syncthreads()`、`atomicAdd`（可作用于 global 与 shared）、**kernel decomposition（把一个算法拆成多次 kernel launch，用 kernel 边界充当全局同步）**、协作装载（cooperative fetching）、`__shfl_down_sync` 类的 warp 级原语、以及"用 L 与 V 两个复用旋钮描述一个 kernel 的访存量"的分析方法。
- **在并行计算知识体系中的角色**：本讲是**从"会写并行程序"到"会做性能工程"的转折点**。它把前几讲的 work-span / 算术强度 / Roofline / 带宽-延迟工具，第一次放到一台真实且极端的机器上做定量诊断：同一份矩阵乘，未优化版本是 0.25 FLOP/B 的访存灾难，调好 L 与 V 之后变成计算受限。后续的 Performance Optimization、异构与硬件专用化、并行深度学习（tensor core、算子融合、算子间复用）全部建立在本讲的三条判据和两个 case study 之上。
- **配套材料**：
- `lectures/05-CUDA-programming.pdf`（抽取文本 `extracted/05-CUDA-programming.txt`，共 83 页）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 直接下载。讲义首页标题为 **"Lecture 5 & 6: GPU Architecture & CUDA Programming"**，即**一份讲义覆盖 Sep 2 与 Sep 4 两次课**（Fall 2026 日程表：https://www.cs.cmu.edu/~418/schedule.html，第 5 讲 Sep 2，第 6 讲 Sep 4 "GPU Architecture and CUDA Programming (continued)"）。本讲（Lecture 6）聚焦该讲义的后半部分：CUDA 到硬件的映射细节、Case Study 1（matmul）、Case Study 2（reduction）、以及 recap 中列出的五条 GPU 优化技术。讲义首页含历史学期的署名信息（如邀请讲者与历史页脚），属**讲义沿用现象**，不是错误。
- `doc_CUDA-recitation.pdf`（抽取文本 `extracted/doc_CUDA-recitation.txt`，共 39 页）：CUDA recitation 讲义，**已公开**（页脚标注历史学期 Fall 2025，同为沿用）。内容为"怎么写 CUDA"与"怎么写快 CUDA"：三种函数限定符、`dim3` 与线程索引公式、host/device 内存管理的 7 个步骤、归约的四种渐进方案（kernel 当屏障 / `__syncthreads()` / 单 block / 多 block + 共享内存）、matmul 的寄存器分块与共享内存分块。
- `cs149_supp/gpuarch.txt`：Stanford CS149（Fall 2025）Lecture 7 "GPU Architecture & CUDA Programming"（共 74 页），**已公开**的补充读物；本笔记中 V100 SM 的 sub-core 微架构、warp selector 与指令发射时序、work scheduler 的 7 步动态映射、"为什么必须为整个 block 预留执行上下文"、"persistent thread" 编程风格、histogram 原子操作合法性等细节来自该材料。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- Fall 2026 尚未在公开目录发布的讲义（Performance Analysis / Profiling、Transactional Memory、AI in System Design 等），其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。

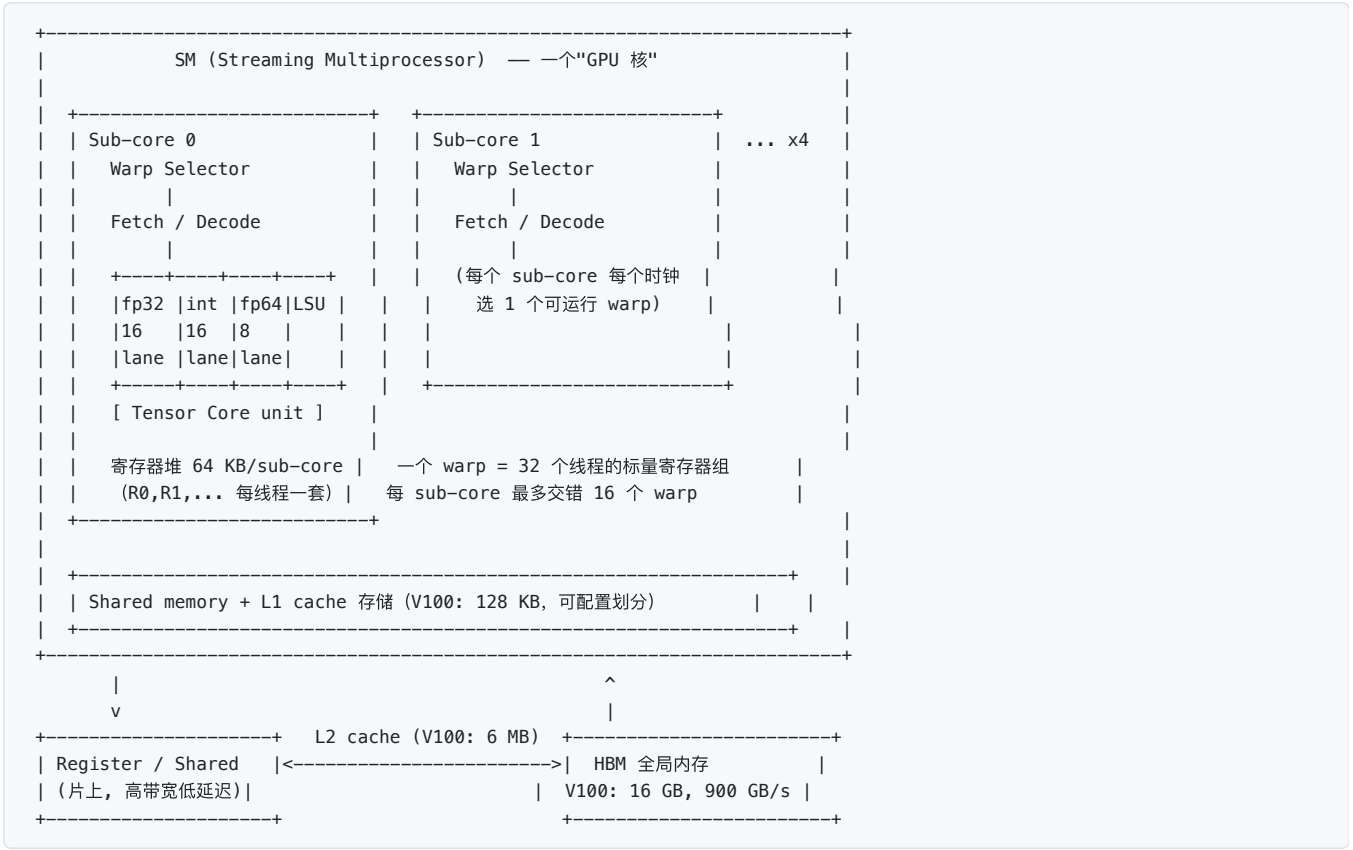
---

2. 核心概念与硬件/软件架构图解

2.1 三条性能判据：本讲的全部实用内容

- **定义与目的**：讲义的 recap 把 GPU 优化技术收敛成五条：**coherent warps**（warp 执行一致）、**coalesced memory access**（合并访存）、**shared memory bank conflict**（避免 bank 冲突）、**warp level optimizations**（warp 级优化，如 shuffle 代替共享内存）、**Tensor Core**（把矩阵乘交给专用单元）。它们共同回答一个问题：**GPU 的峰值算力要靠"每个时钟都有满宽的 warp 在执行有用的指令"来兑现；任何让 lane 空转、让访存拆成多次事务、让数据反复穿越存储层次的行为，都是在这个兑现率上打折。**
- **直观解释（"它是什么？"）**：把 SM 想成**一条 32 人并排作业的装配线**（32 宽 SIMD）。三条判据分别对应三种浪费：
- **发散（divergence）** = 32 个工位里只有 8 个在干活，其余 24 个被"遮罩（mask）"住站着不动 → 装配线照常开动，产出只有 1/4；
- **非合并访存（non-coalesced）** = 32 个工人各自去仓库不同货架取一个小螺丝，仓库被迫发 4 趟车而不是 1 趟 → 同样的数据量，带宽利用率掉到 1/4；
- **共享内存 bank 冲突（bank conflict）** = 车间的 32 个储物柜（bank）里，有 16 个人同时挤在第 0、2、4…号柜子前，另外 16 个柜子空着 → 存取要排队两轮。
- 再叠加一条**数据复用**：从仓库（HBM）取回的料要在车间（寄存器/共享内存）被尽可能多次使用，否则装配线的速度完全由仓库门口决定。
- **架构/机制图解**：

图 1：SM 内部结构 —— 4 个 sub-core、warp selector 与 SIMT 掩码



关键性能特征（来自讲义与 CS149 补充材料给出的 V100 数据）： – **warp 发射节奏**：一个 32 宽 fp32 SIMD 操作**每 2 个时钟**完成一次（16 lane × 2 时钟），fp64 是每 4 个时钟一次（8 lane）。因此"一个 warp 的一条 fp32 指令"占用功能单元 2 个时钟。 – **SM 的每时钟动作**（418 讲义描述）：从驻留的最多 **64 个 warp**（64 × 32 = 2048 个 CUDA 线程）中选出最多 **4 个可运行 warp**（对应 4 个 sub-core 的线程级并行），每个被选中的 warp 再选出最多 **2 条可运行指令**（指令级并行）。 – **寄存器预算**：V100 每 SM 256 KB 寄存器 = 65536 个 32-bit 寄存器；若要把 64 个 warp × 32 线程 = 2048 个线程全部驻留，则**每线程只有 32 个寄存器**。这是"寄存器分块（register tiling）"的硬上限——V 越大，accumulator 越多，occupancy 越低。

图 2：SIMT —— 硬件动态检测 32 条标量指令流是否"恰好一致"

- **性能后果**：coherence（一致执行）是 GPU 高效的前提，divergence 应当被最小化。注意 CUDA 的 warp 是**实现细节而非编程模型的一部分**（例外是 `__shfl_*`、vote 等 warp 级内建操作）：ISPC 在编译期把程序编译成 SIMD 指令，而 GPU 是**运行期由硬件动态判断**这 32 个独立线程是否恰好走在同一条指令上。

2.2 线程块调度：work scheduler 如何在时间上填满 GPU

- **定义与目的**：CUDA 的核心假设是 **"线程块可以按任意顺序执行，块与块之间没有依赖"**；GPU 用一个硬件 work scheduler 把 block 动态映射到 SM，映射时**必须尊重编译产物中记录的资源需求**（每块多少线程、每块多少共享内存、每线程多少本地数据）。这样同一份 CUDA 程序无需改动就能跑在 16 SM 的 GTX 980 和 132 SM 的 H100 上——程序里**没有任何 "num_cores" 概念**。

- **直观解释（"它是什么？"）**：这是课程里反复出现的 **"工人池 + 任务分解"设计模式**的又一个实例：GPU 的 SM 是固定数量的工人（数目由硬件决定，与请求数无关），线程块是待办任务，work scheduler 是派工头。同样的模式出现在 ISPC 的 task 运行时（为每个 CPU 超线程创建一个 pthread 并长期保活）和 Web 服务器的线程池（线程数由核数决定，不由并发请求数决定）中。

图 3：一个 kernel 的 1000 个 block 在"假想双核 GPU"上的动态映射（Gantt 视图）

```
假想双核 GPU（讲义的简化模型）：
  每个 core：执行上下文可容纳 384 个 CUDA 线程（12 warps）、共享内存 1.5 KB
convolve 的 block 需求：128 线程 + 130 floats = 520 B 共享内存
=> 每 core 只能同时驻留 2 个 block：2 x 520 B = 1040 B <= 1536 B
                                  3 x 520 B = 1560 B >  1536 B （放不下第三个）

时间 -->
Core 0: [ Blk0 ][ Blk0 ][ Blk4 ][ Blk4 ][ Blk8 ][ Blk8 ] ...   (NEXT 指针：0,2,4,6,8,...)
        [ Blk2 ][ Blk2 ][ Blk6 ][ Blk6 ][ Blk10] ...           (交错映射：0,1,2,3,... 轮流)
Core 1: [ Blk1 ][ Blk1 ][ Blk5 ][ Blk5 ][ Blk9 ] ...
        [ Blk3 ][ Blk3 ][ Blk7 ][ Blk7 ][ Blk11] ...
               ^
               └─ Blk0 完成后（Step 4），其 128 个执行上下文与 520 B 共享内存立刻被释放，
                  scheduler 在 Step 5 把 Blk4 映射到 contexts 0-127

EXECUTE: convolve | ARGS: N, input, output | NUM_BLOCKS: 1000 | NEXT=4 | TOTAL=1000
```

- **关键机制与性能含义**：
- **block 的粒度是"全有或全无"**：一个 block 的**所有**线程必须同时获得执行上下文（寄存器状态），因为块内线程之间可能有依赖（最简单的例子是 `__syncthreads()`）。CS149 的追问很关键：为什么不能先把线程 0–127 跑完再跑 128–255？因为块内线程**按语义就是并发的**——"如果块内某线程可运行，它最终一定会被运行（不会死锁）"。
- **资源决定 occupancy**：真正的 GPU 上，SM 的驻留 block 数由 warp 执行上下文（64 个 warp）、共享内存（V100 96/128 KB 级，H100 256 KB）、寄存器总量三者**取最小**决定。
- **动态调度天然做负载均衡**：块大小相同、块数远多于 SM 数时，靠"先完成先派新块"即可平滑负载不均；但**块数太少（< SM 数 × 驻留数）会造成尾部空转**。

2.3 精确的执行语义边界：块内并发、块间任意序

- **定义与目的**：CUDA 的执行语义是**两种模型的混合**，混淆它会导致两类典型错误。
- **block 之间**：任何顺序、逻辑并发、无依赖 → 对应**数据并行模型**的 `forall`（机器无关，由系统调度到任意数量的核上）。
- **block 内部**：线程**真正并发运行**、共享地址空间、可协作 → 对应 **SPMD 共享地址空间模型**（很像一个 ISPC gang）。
- **直观解释（"它是什么？"）**：把 GPU 想成一栋楼。**块 = 一个项目组**，组内成员在同一间办公室（SM + 共享内存）里实时协作、可以开会同步（`__syncthreads()`）；**块之间 = 不同项目组**，公司只保证"每个组最终都会被安排办公室"，但**不保证谁先谁后、也不保证同时存在**。所以：组内可以互相等，跨组不能互相等——一等的就是死锁。
- **重要推论**：
- **合法**：所有 CUDA 线程用 `atomicAdd` 更新同一个 global 数组（histogram 例子）。原子操作用作互斥，**不影响实现的调度自由**——因为无论块以什么顺序执行，结果都对（加法可交换），并且不存在"等别的块"的行为。
- **不合法（可能死锁）**：block 0 设置 `myFlag`，block 1 自旋等待 `myFlag != 0`。若 GPU 只有 1 个 SM 且只能驻留 1 个 block，且实现先跑了 block 1，则 block 1 永久自旋；而 block 0 永远得不到执行资源。**CUDA 没有全局同步（no global synchronization）**：一方面为高 SM 数的 GPU 建造全局屏障硬件代价高，另一方面当 `#blocks > #SM × #resident blocks` 时全局屏障必然死锁。
- **正确替代：kernel 分解（kernel decomposition）**——用 kernel launch 边界充当全局同步；讲义强调 kernel launch 的硬件/软件开销很低，而且"所有层次的代码长得一样"，只是把 `for` 循环搬到 host 侧。

图 4：用 kernel 边界代替全局屏障（归约的两阶段分解）

```
单个 kernel 内做不到:
-----------------------------------------------------------------
Block0  Block1  Block2  ...  BlockN      <- 块间无法 __syncthreads()
(x)     (x)     (x)          (x)         跨块自旋 = 可能死锁
-----------------------------------------------------------------

分解为两次 launch:
阶段 1 (grid = 4096 blocks)              阶段 2 (grid = 1 block)
-----------------------------------------------------------------
Block0  Block1  ...  Block4095           Block0
 |        |            |                   |
部分和    部分和       部分和   ————————>  把 4096 个部分和再归约
 |        |            |                   |
partial[0..4095]                          out[0]
-----------------------------------------------------------------
     ^ kernel launch #1 结束 = 一次隐式全局屏障（所有块都完成）
     ^ kernel launch #2 结束 = 一次隐式全局屏障

反例（recitation 中的 "Idea 1"）：把树的每一层都做成一次 launch
   d=1,2,4,...,2^23  =>  24 次 launch。正确，但每次 launch 数微秒级固定开销，
   24 次 x ~5 us ≈ 120 us，远超 67 MB 数据本身 75 us 的搬运时间 => 被 launch 开销吃掉
```

2.4 访存效率之一：global memory 的 coalescing

- **定义与目的**：**coalescing（合并访存）** 指"一个 warp 的多个线程访问连续的内存地址"，从而把 32 个 lane 的请求合并成**尽可能少的内存事务（memory transaction）**，最大化 HBM 带宽利用率。一个 warp 的 32 个线程各取 4 字节且地址连续时，正好覆盖 128 字节——这是硬件最喜欢的一次全宽事务。
- **直观解释（"它是什么？"）**：快递取件。32 个人要取 32 个相邻柜子里的包裹——仓库管理员开一次门、走一趟就全部取回（coalesced）；如果这 32 个人要取的包裹分散在 32 个不同货架，管理员就得跑 32 趟（non-coalesced），实际带宽利用率可能掉到 1/4 甚至更低，而且**每一趟都会把整条 cache line 搬回来**，浪费的是带宽而不是容量。

图 5：合并访存 vs 非合并访存（以及共享内存的 bank 映射）

```
[A] Coalesced（最优）           地址: 0 1 2 3 | 4 5 6 7 | 8 9 10 11  （以 4B 字为单位）
    warp 的 32 个 lane 连续访问
    t0 t1 t2 t3   t4 t5 t6 t7   t8 t9 t10 t11 ...
    |  |  |  |    |  |  |  |    |  |   |   |
    +--+--+--+    +--+--+--+    +--+---+---+     128 B = 1 次全宽事务
    第一次 load    第二次 load    第三次 load

[B] Non-coalesced（次优）      warp 内 stride = 4 个字（16 B）
    t0 -> word 0     t1 -> word 4     t2 -> word 8    t3 -> word 12
    |                |                |               |
    +----------------+----------------+---------------+   每 lane 落在不同 128 B 段
    一次 warp 请求被拆成 4 段 => 有效带宽 1/4；但搬回的字节数不变 => 浪费 4 倍带宽

[C] 共享内存 bank（V100/H100：32 个 bank，每 bank 4 B，每时钟可服务 1 个字）
    无冲突:  lanes t0..t7 -> words 0,1,2,3,4,5,6,7
       B0  B1  B2  B3  B4  B5  B6  B7 ... B31
       t0  t1  t2  t3  t4  t5  t6  t7 ...  -
       => 每个 bank 服务 1 个 lane，1 个周期完成，满带宽

    冲突:    lanes t0..t7 -> words 0,2,4,6,8,10,12,14   (stride 2)
       B0     B1   B2     B3   B4     B5   B6     B7
       t0,t4  -    t1,t5  -    t2,t6  -    t3,t7  -
       => 4 个 bank 各服务 2 个 lane，其余 4 个 bank 空闲
          => 2 路 bank 冲突，访问串行化为 2 个周期，带宽减半
```

- **数值化含义**：如果一次 warp 访存被拆成 4 个事务，那么"每字访问的有效成本"是原来的 4 倍；在 V100（900 GB/s、12.7 TFLOPs、机器平衡点 14.2 FLOP/B）上，一个本该 0.15 ms 的 kernel 会退化到 0.6 ms——正好抵消掉 4 倍的算力优化。

2.5 三级存储层次与延迟/带宽/容量

- **定义与目的**：CUDA 对 kernel 可见三个地址空间：**per-thread private（寄存器/本地内存）**、**per-block shared memory**、**device global memory**。它们不是"三块内存"，而是**程序中三种不同 locality 的体现**；把数据放到正确的层次，是 GPU 性能工程的核心动作。

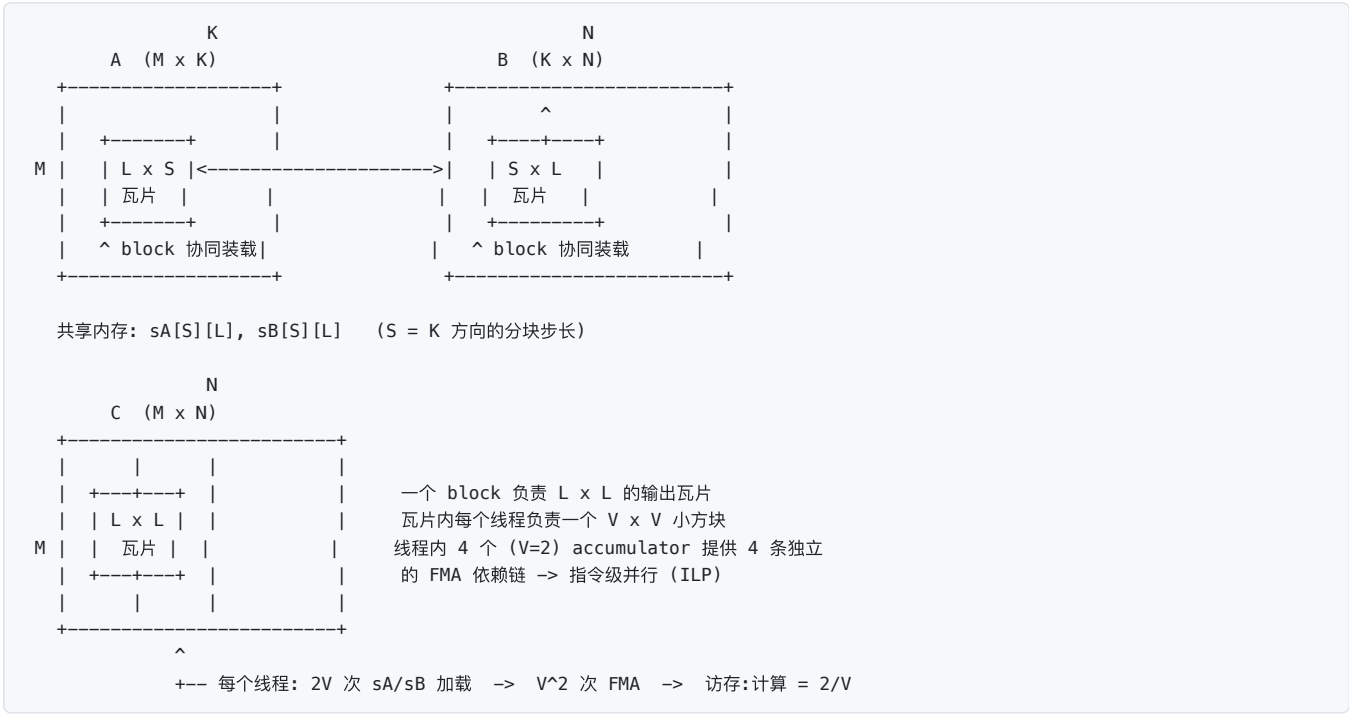
层次	作用域 / 生命周期	典型容量（V100 级）	访问者	相对延迟	相对带宽	放置方式
Register（寄存器）	单线程 / 线程存活期	SM 共 256 KB（65536 × 32-bit），满 occupancy 时 32 reg/thread	仅本线程	1 个周期级（最低）	最高（每周期每 lane）	编译器分配
Shared memory（共享内存）	单 block / block 存活期	V100 96—128 KB/SM，H100 256 KB/SM	block 内所有线程	数十周期	32 bank × 4 B × 时钟 ≈ 12.75 TB/s（80 SM @1.245 GHz）	`__shared__` 静态或 `extern __shared__` + launch 参数
L1 cache	单 SM	与共享内存共用同一片存储	SM 内线程	~百周期	高	硬件自动
L2 cache	整芯片	V100 6 MB	所有 SM	~200 周期量级	数倍于 HBM	硬件自动
Global memory (HBM)	整程序 / 显存分配期	V100 16 GB，H100 数十 GB	所有线程（+ host 经 cudaMemcpy）	数百周期	V100 900 GB/s；高端的 ~1 TB/s	`cudaMalloc` / `cudaMemcpy`

- **两个关键事实**：
- **host 与 device 地址空间彼此不可直接访问**。要移动数据必须显式调用 `cudaMemcpy`（参数中的方向常量 `cudaMemcpyHostToDevice` / `cudaMemcpyDeviceToHost`）。这一条正是"消息传递模型"的味道——所以讲师会反复问"CUDA 是共享地址空间模型还是消息传递模型？"答案取决于你站在哪一层看：**host↔device 之间像消息传递，block 内部是共享地址空间**。
- **`__shared__` 存在的唯一理由是让 block 内的线程协作**：它把"每个线程各自从 global 取 3 个元素（3×128 次 load）"变成"全体协作取 130 个元素（130 次 load），再从片上反复读"。1D 卷积版本 1 → 版本 2 的整个收益就是这个 3N → (N + 2·blocks) 的访存缩减。

2.6 Case Study 1 的几何：矩阵乘的两级复用

- **定义与目的**：`C = A × B`（M×K 乘 K×N 得 M×N）。**朴素实现是访存灾难**：每个线程算 1 个输出元素、每个元素需要读 A 的一整行与 B 的一整列（2N 次 global 访问），N^2 个线程 → 总访存 **$2N^3$**，而总计算只有 $2N^3$ FLOP → **算术强度 0.25 FLOP/B**，比机器平衡点（V100：12.7 TFLOPs / 900 GB/s = 14.2 FLOP/B）低 57 倍。两个"复用旋钮"解决它：
- **V（thread-level register tiling，寄存器分块）**：一个线程算 V×V 个输出，把 2V 次加载的数据用于 V^2 次乘加 → 总访存降到 **$2N^3/V$**。
- **L（block-level shared memory tiling，共享内存分块）**：一个 block 算 L×L 个输出，A/B 的 L×K 与 K×L 条带被整个 block 复用 → 总访存降到 **$2N^3/L$**。
- **直观解释（"它是什么？"）**：做菜类比。朴素做法是**每做一道菜就跑一趟菜市场买齐所有原料**（每个输出元素都重新读 A 行、B 列）。**L 旋钮 = 让一个厨师班组统一采购一批原料放在班组自己的冰箱（shared memory）里**，全班一起用；**V 旋钮 = 每个厨师一次拿一小篮（寄存器 a[V]、b[V]）原料，同时做 V^2 道菜**（一次拿料，多次下锅）。L 减少"跑市场的总趟数"，V 减少"从冰箱里取料的总次数"。

图 6：矩阵乘的两级分块（block 级 L×L + 线程级 V×V）



- **定量结论（讲义给出的公式）**：

版本	每线程 global 访问	线程数 / block 数	global 总访问	共享内存总访问	算术强度（FLOP/global B）
Strawman（1 元素/线程）	2N	N^2 线程	**$2N^3$**	0	0.25
+ 寄存器分块 V	2N/V	N^2/V^2 线程	**$2N^3/V$**	0	V/2
+ 共享内存分块 L	2N L / (线程数) 级 → 每 block 2NL	N^2/L^2 blocks	**$2N^3/L$**	$2N^3/V$	L/2

注解：表中"算术强度"按"2 FLOP / 4 B"的比率推出（每个 4 字节浮点访问贡献 2 FLOP）；共享内存分块后 **L 负责砍 global 流量，V 负责砍 shared 流量**——两个旋钮各管一层，这就是讲义那句 "L and V are the two reuse knobs!"。

- **协作装载（cooperative fetching）**：把 `sA[:,:] = A[...]` 这种"整块赋值"翻译成每个线程搬一片：

```
nthreads = blockDim.x * blockDim.y
tid      = threadIdx.y * blockDim.x + threadIdx.x
for (j = 0; j < L*S / nthreads; ++j) {
    y = (j*nthreads + tid) / L
    x = (j*nthreads + tid) % L
    s[y][x] = A[k + y][yblock*L + x]        // 连续 tid -> 连续 x => 合并访存
}
```

关键点：让 `tid` 映射到**连续**的 `x`（列）上，global 读取就是连续的 → coalesced；随之而来的 `__syncthreads()` 必须成对出现：**装载前一个屏障**（防止上一个 k 迭代还在读的时候被覆盖）、**装载后一个屏障**（保证所有数据就位后才能开始计算）。

2.7 Case Study 2 的几何：树形归约与三种寻址

- **定义与目的**：归约（reduction）是 normalization、softmax 等 ML 算子的底层原语，任务是把 n 个元素求和。串行版是 n 次依赖加法（Span = n）；**树形归约**把依赖链缩短到 log n：`for (d=1; d<n; d<<=1) for (i=0; i<n; i+=2d) A[i] += A[i+d]`。
- **直观解释（"它是什么？"）**：**淘汰赛**。串行求和像"一个裁判逐个听 n 个人报数"，要 n 轮；树形归约像"两两配对比赛，胜者（和）进入下一轮"，只要 $\log_2 n$ 轮。但比赛必须**一轮一轮同步**——CUDA 不保证块内线程 lockstep 执行，所以每轮之间必须插 `__syncthreads()`（块内屏障），或用 kernel 边界（跨块屏障）。这解释了 recitation 里"为什么需要两个 `__syncthreads()`"以及"为什么只加一个 `__syncthreads()` 的版本是错的"。

图 7：归约的三种寻址方式（同一棵树的三种"排队方式"）

```
[1] Interleaved addressing (交错寻址, reduce0)    if (tid % (2s) == 0) sdata[tid] += sdata[tid+s]
    s=1: 线程  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
         活跃  T  F  T  F  T  F  T  F  T  F  T  F  T  F  T  F      <- 半数 lane 空转
    s=2: 活跃  T  F  F  F  T  F  F  F  T  F  F  F  T  F  F  F      <- 1/4 有效
    s=4: 活跃  T  F  F  F  F  F  F  F  T  F  F  F  F  F  F  F      <- 1/8 有效
    问题: warp 高度发散; 且奇偶 lane 访问 sdata[0],[2],[4]... => 2 路 bank 冲突

[2] Strided index + 非发散分支        index = 2*s*threadIdx.x; if (index < blockDim.x) ...
    s=1: 活跃  T  T  T  T  T  T  T  T  F  F  F  F  F  F  F  F      <- 前 8 个 lane 连续活跃
    s=2: 活跃  T  T  T  T  F  F  F  F  F  F  F  F  F  F  F  F
    s=4: 活跃  T  T  F  F  F  F  F  F  F  F  F  F  F  F  F  F
    改善: 分支结果在 warp 内"前缀连续", 后期线程统一为 false; 但访问 sdata[0],[2],[4]... 仍 stride 2

[3] Sequential (reversed) addressing (顺序寻址, 最终版)
    for (s = blockDim.x/2; s > 0; s /= 2) { if (threadIdx.x < s) sdata[tid] += sdata[tid+s]; }
    s=256: 活跃 lane 0..255  (8 个 warp 满负荷!)   访问 sdata[tid] 与 sdata[tid+256]
    s=128: 活跃 lane 0..127  (4 个 warp)
    s=64 : 活跃 lane 0..63   (2 个 warp)
    s=32 : 活跃 lane 0..31   (1 个 warp, 满)
    s=16,8,4,2,1: 单 warp 内前缀活跃 (不可避免的尾部), 但访问 tid, tid+1, ... 连续
    结果: 完全合并访存 + 无 bank 冲突 (相邻 lane 落在相邻 bank)

归约树示意 (n=16, 顺序寻址):
  第1轮: [a0+a8][a1+a9][a2+a10][a3+a11][a4+a12][a5+a13][a6+a14][a7+a15]
  第2轮: [b0+b4][b1+b5][b2+b6][b3+b7]
  第3轮: [c0+c2][c1+c3]
  第4轮: [d0+d1]                       总轮数 = log2(16) = 4
```

- **性能含义**：三种写法**工作量相同（Work ≈ n 次加法）**，差别全在"这 n 次加法有多少个 lane 是有效的、访存有没有被拆成多个事务"。这正是"同样的 Work/Span，性能差 5–10 倍"的教科书案例。

2.8 算力演进与 Tensor Core

- **定义与目的**：Tensor Core 是 SM 内的**矩阵乘加专用单元（matrix multiplication unit）**。它把"小尺寸矩阵乘加"固化成一条指令，用远高于通用 SIMD lane 的密度完成乘加，是 H100 峰值算力从 GTX 980 的 4.6 TFLOPs 跃升到 1000 TFLOPs 的主要原因。
- **直观解释（"它是什么？"）**：通用 fp32 lane 像**一群会做任何算术的通用工人**；Tensor Core 像**一台专门的"批量乘法打孔机"**——它只会做矩阵乘加这一件事，但一次冲压就是一个 4×4（乃至更大）的乘加块。代价是**你必须把数据摆成它要求的形状**（分块、对齐、数据类型），"喂料"的组织工作落到了程序员/编译器身上。

代际（讲义数据）	时钟	SM 数	warp/SM	线程/warp	共享内存/SM	峰值算力	显存带宽（补充材料）
GTX 980（2014）	1.1 GHz	16	64	32	96 KB	4.6 TFLOPs	—
V100（2017，CS149 补充）	1.245 GHz	80	64	32	128 KB（shared+L1）	12.7 TFLOPs（fp32）	900 GB/s，16 GB，L2 6 MB
A100（2020）	—	—	64	32	168 KB	—	—
H100（2022）	1110 MHz	132	64	32	256 KB	**1000 TFLOPs**（主要来自 Tensor Core）	—

- **注意三次"没变"**：从 GTX 980 到 H100，**SM 的基本形态没变、每 SM 的 warp 数上限（64）没变、warp 宽度（32）没变**。变的是 SM 数量（16 → 132）、共享内存容量（96 KB → 256 KB）与专用单元（Tensor Core）。这解释了为什么本讲的三条判据在十年后依然成立。

---

3. 代码示例与性能分析

3.1 示例 A：两级分块的 CUDA 矩阵乘（Case Study 1，完整可运行）

```
// mm_tiled.cu  ——  C = A * B  (M x K 乘 K x N), 两级分块: block 级 L x L + thread 级 V x V
// 编译(发布优化): nvcc -O3 -arch=sm_80 -lineinfo mm_tiled.cu -o mm_tiled
// 运行:          ./mm_tiled
// 说明: 报告 GFLOP/s 与"有效全球访存带宽", 并与 CPU 朴素实现校验正确性。

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

// ---------- 分块参数 (两个复用旋钮) ----------
#define BM 64          // block 计算的输出瓦片行数  L = 64
#define BN 64          // block 计算的输出瓦片列数  L = 64
#define BK 16          // K 方向分块步长 S = 16
#define V  4           // 每个线程算 V x V 个输出    V = 4

#define THREADS_X (BN / V)          // 16
#define THREADS_Y (BM / V)          // 16   => block = 16 x 16 = 256 线程

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,     \
                    cudaGetErrorString(err__));                             \
            exit(1);                                                        \
        }                                                                   \
    } while (0)

__global__ void mm_tiled(const float* __restrict__ A,
                         const float* __restrict__ B,
                         float* __restrict__ C,
                         int M, int N, int K)
{
    __shared__ float sA[BM][BK];    // A 的 BM x BK 瓦片: 64 x 16 x 4 B = 4 KB
    __shared__ float sB[BK][BN];    // B 的 BK x BN 瓦片: 16 x 64 x 4 B = 4 KB

    const int ty = threadIdx.y;
    const int tx = threadIdx.x;
    const int tid = ty * THREADS_X + tx;
    const int nthreads = THREADS_X * THREADS_Y;   // 256

    // 该线程负责的 C 子块左上角 (V x V)
    const int crow = blockIdx.y * BM + ty * V;
    const int ccol = blockIdx.x * BN + tx * V;

    float acc[V][V];
#pragma unroll
    for (int i = 0; i < V; ++i)
#pragma unroll
        for (int j = 0; j < V; ++j) acc[i][j] = 0.0f;

    float a[V], b[V];

    for (int ko = 0; ko < K; ko += BK) {
        // ---- 协作装载 A 的 BM x BK 瓦片: 每个线程搬 4 个元素, tid -> 连续列 => 合并访存 ----
#pragma unroll
        for (int t = tid; t < BM * BK; t += nthreads) {
            const int r = t / BK;
            const int c = t % BK;
            sA[r][c] = A[(size_t)(blockIdx.y * BM + r) * K + (ko + c)];
        }
        // ---- 协作装载 B 的 BK x BN 瓦片 ----
#pragma unroll
        for (int t = tid; t < BK * BN; t += nthreads) {
            const int r = t / BN;
            const int c = t % BN;
            sB[r][c] = B[(size_t)(ko + r) * N + (blockIdx.x * BN + c)];
        }
        __syncthreads();                    // 数据就位前不许算
```

```
#pragma unroll
        for (int k = 0; k < BK; ++k) {
#pragma unroll
            for (int i = 0; i < V; ++i) a[i] = sA[ty * V + i][k];
#pragma unroll
            for (int j = 0; j < V; ++j) b[j] = sB[k][tx * V + j];
#pragma unroll
            for (int i = 0; i < V; ++i)
#pragma unroll
                for (int j = 0; j < V; ++j)
                    acc[i][j] += a[i] * b[j];    // V^2 条独立 FMA 链 => ILP
        }
        __syncthreads();                    // 下一轮装载前，保证所有线程都读完本瓦片
    }

#pragma unroll
    for (int i = 0; i < V; ++i)
#pragma unroll
        for (int j = 0; j < V; ++j)
            C[(size_t)(crow + i) * N + (ccol + j)] = acc[i][j];
}

// ---------------- CPU 参考实现（仅用于校验小规模结果） ----------------
static void mm_cpu(const float* A, const float* B, float* C, int M, int N, int K)
{
    for (int i = 0; i < M; ++i)
        for (int j = 0; j < N; ++j) {
            double s = 0.0;
            for (int k = 0; k < K; ++k) s += (double)A[(size_t)i * K + k] * B[(size_t)k * N + j];
            C[(size_t)i * N + j] = (float)s;
        }
}

int main()
{
    // ---- 1) 小规模正确性校验: 256 x 256 x 256 ----
    {
        const int M = 256, N = 256, K = 256;
        const size_t bA = (size_t)M * K * sizeof(float);
        const size_t bB = (size_t)K * N * sizeof(float);
        const size_t bC = (size_t)M * N * sizeof(float);
        float *hA = (float*)malloc(bA), *hB = (float*)malloc(bB);
        float *hC = (float*)malloc(bC), *ref = (float*)malloc(bC);
        for (size_t i = 0; i < (size_t)M * K; ++i) hA[i] = (float)((i * 7) % 13) * 0.125f - 0.5f;
        for (size_t i = 0; i < (size_t)K * N; ++i) hB[i] = (float)((i * 5) % 11) * 0.125f - 0.5f;

        float *dA, *dB, *dC;
        CUDA_CHECK(cudaMalloc(&dA, bA));
        CUDA_CHECK(cudaMalloc(&dB, bB));
        CUDA_CHECK(cudaMalloc(&dC, bC));
        CUDA_CHECK(cudaMemcpy(dA, hA, bA, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(dB, hB, bB, cudaMemcpyHostToDevice));

        dim3 block(THREADS_X, THREADS_Y);              // (16, 16) = 256 线程
        dim3 grid(N / BN, M / BM);                     // (4, 4) = 16 个 block
        mm_tiled<<<grid, block>>>(dA, dB, dC, M, N, K);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaMemcpy(hC, dC, bC, cudaMemcpyDeviceToHost));
        mm_cpu(hA, hB, ref, M, N, K);

        double maxerr = 0.0;
        for (size_t i = 0; i < (size_t)M * N; ++i)
            maxerr = fmax(maxerr, fabs((double)hC[i] - ref[i]));
        printf("[verify] 256x256x256  最大绝对误差 = %.6g  (K=256 时 fp32 累加误差量级 ~1e-3)\n", maxerr);

        cudaFree(dA); cudaFree(dB); cudaFree(dC);
        free(hA); free(hB); free(hC); free(ref);
    }

    // ---- 2) 性能测量: 1024 x 1024 x 1024 ----
    {
```

```
        const int M = 1024, N = 1024, K = 1024;
        const size_t bA = (size_t)M * K * sizeof(float);
        const size_t bB = (size_t)K * N * sizeof(float);
        const size_t bC = (size_t)M * N * sizeof(float);
        float *hA = (float*)malloc(bA), *hB = (float*)malloc(bB), *hC = (float*)malloc(bC);
        for (size_t i = 0; i < (size_t)M * K; ++i) hA[i] = (float)((i * 7) % 13) * 0.01f;
        for (size_t i = 0; i < (size_t)K * N; ++i) hB[i] = (float)((i * 5) % 11) * 0.01f;

        float *dA, *dB, *dC;
        CUDA_CHECK(cudaMalloc(&dA, bA));
        CUDA_CHECK(cudaMalloc(&dB, bB));
        CUDA_CHECK(cudaMalloc(&dC, bC));
        CUDA_CHECK(cudaMemcpy(dA, hA, bA, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(dB, hB, bB, cudaMemcpyHostToDevice));

        dim3 block(THREADS_X, THREADS_Y);              // (16, 16) = 256 线程
        dim3 grid(N / BN, M / BM);                     // (16, 16) = 256 个 block

        mm_tiled<<<grid, block>>>(dA, dB, dC, M, N, K);  // 预热
        CUDA_CHECK(cudaDeviceSynchronize());

        cudaEvent_t e0, e1;
        CUDA_CHECK(cudaEventCreate(&e0));
        CUDA_CHECK(cudaEventCreate(&e1));
        const int iters = 20;
        CUDA_CHECK(cudaEventRecord(e0));
        for (int it = 0; it < iters; ++it)
            mm_tiled<<<grid, block>>>(dA, dB, dC, M, N, K);
        CUDA_CHECK(cudaEventRecord(e1));
        CUDA_CHECK(cudaEventSynchronize(e1));
        float ms = 0.f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
        ms /= iters;

        const double flops = 2.0 * (double)M * N * K;              // 2N^3
        const double gflops = flops / (ms * 1e-3) / 1e9;
        // 该 kernel 在 "至 L2" 层面搬运的 global 字节数: 2N^3/L * 4 B
        const double globalBytes = 2.0 * (double)M * N * K / BM * 4.0;
        printf("[perf  ] N=%d  L=%d  V=%d  平均耗时 = %.3f ms\n", M, BM, V, ms);
        printf("[perf  ] %.1f GFLOP/s ; global(L2 级)流量 %.2f MB/次 -> %.1f GB/s\n",
               gflops, globalBytes / 1e6, globalBytes / (ms * 1e-3) / 1e9);

        CUDA_CHECK(cudaMemcpy(hC, dC, bC, cudaMemcpyDeviceToHost));
        printf("[check ] C[0]=%.4f  C[last]=%.4f\n", hC[0], hC[M * N - 1]);

        cudaFree(dA); cudaFree(dB); cudaFree(dC);
        free(hA); free(hB); free(hC);
    }
    return 0;
}
```

【代码做什么？】

1. **host 侧**：分配并初始化 A、B（device 端），分配 C；设置 `block = (BN/V, BM/V) = (16,16) = 256` 线程、`grid = (N/BN, M/BM)` 个 block；预热一次后连跑 20 次取平均；用 `cudaEvent` 计时；最后把 C 拷回 host 做小规模校验。
2. **device 侧外层循环（K 方向分块）**：每次迭代把 A 的 `BM×BK` 瓦片与 B 的 `BK×BN` 瓦片**由 256 个线程协作装载**进共享内存（每个线程搬 `BM*BK/256 = 4` 个元素），`__syncthreads()` 后进入内层。
3. **device 侧内层循环（K 瓦片内）**：对 `BK` 个 k 值，每个线程从 sA 取 `V=4` 个 A 元素、从 sB 取 `V=4` 个 B 元素，做 `V²=16` 次 FMA 累加到 `acc[4][4]`；结束时再 `__syncthreads()` 保护瓦片不被下一轮提前覆盖。
4. **索引到线程的映射**：`crow = blockIdx.y*BM + ty*V`，`ccol = blockIdx.x*BN + tx*V`——**同一个 warp 里的连续 tx 落在连续的 C 列上**，因此最后写回 C 时也是合并访存；协作装载时 `tid` 也对齐到连续列，保证读 A/B 合并。

【并行机制与性能解说】

- **线程/block 的创建与工作分配**：一次 launch 产生 `16×16 = 256` 个 block、每块 256 线程，共 **65536 个 CUDA 线程**，映射为 `256 / 32 = 8` 个 warp/block。硬件 work scheduler 把 block 动态派到 132（H100）个 SM 上，程序本身不关心 SM 数量。

- **共享数据的处理**：A/B 瓦片是所有线程的**只读共享数据**，放在 `__shared__`（每 block 8 KB）里被 block 内 256 个线程**各读 4 次**；accumulator `acc[4][4]`、`a[4]`、`b[4]` 是**线程私有**数据，放在寄存器里——它们承载了全部复用收益，也决定了寄存器压力（16 + 4 + 4 = 24 个浮点寄存器，加索引寻址约 40 个）。
- **Work / Span / 并行度**：
- **Work** = `M·N·K` 次 FMA = `2N³` FLOP（N=1024 时为 **2.147 GFLOP**）。分块不改变 Work，只改变访存量。
- **Span（关键路径）**：每个输出元素的累加链在 k 上是串行的，长度 = K 次 FMA（分散在 K/BK 次外层迭代里，每次迭代间还夹两次 `__syncthreads()`）→ **Span ≈ K·(FMA 延迟) + (K/BK)·2·(屏障延迟)**。取 FMA 延迟 4 周期、屏障 ~30 周期、K=1024、BK=16：`1024×4 + 64×2×30 = 4096 + 3840 ≈ 7936` 周期 ≈ **6.4 μs @1.245 GHz**。
- **并行度** = Work / Span ≈ `2.147e9 FLOP / (7936 周期 × 32 lane-FMA/周期…)`。用更直接的算法：并行度 = 可并行的工作量 / 关键路径工作量 = `2N³ / (2K)` = **N^2 = 1.05e6 个独立 FMA 链**（每个输出元素一条链），远大于硬件并发线程数（V100：163840）→ 并行度充足，**瓶颈不在并行度**。
- **瓶颈诊断（N=1024，V100 参数：12.7 TFLOPs、900 GB/s、共享带宽 ≈ 12.75 TB/s）**：
- 计算下界：`2.147e9 / 12.75e12 = 0.168 ms`
- global（至 L2）流量：`2N³/L × 4 B = 2.147e9/64 × 4 = 134 MB` → `0.149 ms`
- 共享内存流量：`2N³/V × 4 B = 2.147e9/4 × 4 = 2.147 GB` → `0.168 ms`
- **三者几乎相等（0.149 / 0.168 / 0.168 ms）**，说明 `L=64，V=4` 正好把 kernel 推到**计算受限的平衡点**；再增大 L 收益递减（global 已经不再是限制），再增大 V 会把寄存器吃光、occupancy 掉下来。
- 若把 `V` 降为 2：共享流量翻倍到 `0.337 ms`，kernel 立刻变成**共享内存带宽受限**；若把 `L` 降为 32 而 V=4：global 流量翻倍到 `0.298 ms`，变成 **L2/HBM 受限**。这就是"两个旋钮各管一层"的实测含义。
- 另需注意：N=1024 时 A、B 各 4 MB，**合起来接近 V100 的 6 MB L2**，因此真实 DRAM 流量远小于上面按"每次 block 都回 HBM"计算的 134 MB；换句话说 L2 抹掉了一部分 global 流量，但**L2 带宽本身成为新的上限**，结论（继续增大 L 收益有限）不变。

3.2 示例 B：两阶段并行归约（Case Study 2，完整可运行）

```
// reduce.cu ——  大数组求和: 阶段1 (多 block + 共享内存树 + warp shuffle) + 阶段2 (单 block 收尾)
// 编译(发布优化): nvcc -O3 -arch=sm_80 reduce.cu -o reduce
// 运行:          ./reduce
// 关键点: CUDA 无全局屏障 => 用 kernel 分解代替; 顺序寻址(sequential addressing)保证无 bank 冲突

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define BLK        512
#define FULL_MASK  0xffffffffu

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,     \
                    cudaGetErrorString(err__));                             \
            exit(1);                                                        \
        }                                                                   \
    } while (0)

// ---- warp 级归约: 用 shuffle 代替共享内存, 不消耗 shared 带宽也不需要 __syncthreads ----
__device__ __forceinline__ float warp_reduce_sum(float v)
{
#pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1)
        v += __shfl_down_sync(FULL_MASK, v, offset);
    return v;
}

// ---- 阶段 1: 每个 block 用 grid-stride 读取一块数据, 块内两级树形归约 ----
__global__ void reduce_stage1(const float* __restrict__ in,
                              float* __restrict__ partial,
                              long long n)
{
    __shared__ float sdata[BLK / 32];        // 512/32 = 16 个 warp 部分和, 64 B

    const int  tid    = threadIdx.x;
    const long long stride = (long long)gridDim.x * blockDim.x;

    float sum = 0.0f;
    for (long long i = (long long)blockIdx.x * blockDim.x + tid; i < n; i += stride)
        sum += in[i];                        // 连续 lane 读连续地址 => 合并访存

    sum = warp_reduce_sum(sum);              // 第一级: warp 内 shuffle 归约
    if ((tid & 31) == 0) sdata[tid >> 5] = sum;
    __syncthreads();                         // 第二级前必须同步: 块内无 lockstep 保证

    if (tid < 32) {                          // 只让 0 号 warp 参与(整个 warp 都在, mask 合法)
        float v = (tid < BLK / 32) ? sdata[tid] : 0.0f;
        v = warp_reduce_sum(v);
        if (tid == 0) partial[blockIdx.x] = v;
    }
}

// ---- 阶段 2: 单 block 把 nblocks 个部分和收尾 ----
__global__ void reduce_stage2(const float* __restrict__ partial,
                              float* __restrict__ out,
                              int nblocks)
{
    __shared__ float sdata[BLK];

    const int tid = threadIdx.x;
    float sum = 0.0f;
    for (int i = tid; i < nblocks; i += blockDim.x)
        sum += partial[i];
    sdata[tid] = sum;
    __syncthreads();
```

```
    // 顺序寻址(sequential addressing): 相邻 lane 访问相邻元素 -> 无 bank 冲突 + 完全合并
    for (int s = blockDim.x >> 1; s > 0; s >>= 1) {
        if (tid < s) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }
    if (tid == 0) out[0] = sdata[0];
}

int main()
{
    const long long n = 1LL << 24;                      // 16,777,216 个 float = 64 MiB
    const int gridsz  = 4096;                           // 4096 个 block x 512 线程
    const size_t bytes = (size_t)n * sizeof(float);

    float* h = (float*)malloc(bytes);
    double exact = 0.0;
    for (long long i = 0; i < n; ++i) {
        h[i] = (float)((i % 1024) * 1e-3);              // 确定性数据, 便于校验
        exact += (double)h[i];
    }

    float *d_in, *d_partial, *d_out;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMalloc(&d_partial, (size_t)gridsz * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_out, sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_in, h, bytes, cudaMemcpyHostToDevice));

    // 预热
    reduce_stage1<<<gridsz, BLK>>>(d_in, d_partial, n);
    reduce_stage2<<<1, BLK>>>(d_partial, d_out, gridsz);
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t e0, e1;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));
    const int iters = 50;
    CUDA_CHECK(cudaEventRecord(e0));
    for (int it = 0; it < iters; ++it) {
        reduce_stage1<<<gridsz, BLK>>>(d_in, d_partial, n);
        reduce_stage2<<<1, BLK>>>(d_partial, d_out, gridsz);
    }
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));

    float ms = 0.f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
    ms /= iters;

    float gpu_sum = 0.f;
    CUDA_CHECK(cudaMemcpy(&gpu_sum, d_out, sizeof(float), cudaMemcpyDeviceToHost));

    const double gib = (double)bytes / 1e9;
    printf("[verify] GPU=%.4f  CPU(double)=%.4f  相对误差=%.3e\n",
           gpu_sum, exact, fabs((double)gpu_sum - exact) / exact);
    printf("[perf  ] 耗时 = %.4f ms ; 读 %.2f MB -> 有效带宽 = %.1f GB/s\n",
           ms, bytes / 1e6, bytes / (ms * 1e-3) / 1e9);
    printf("[model ] 12.7 TFLOPs 下纯计算下界 = %.4f ms ; 900 GB/s 下读带宽下界 = %.4f ms\n",
           (double)(n - 1) / 12.75e12 * 1e3, (double)bytes / 900e9 * 1e3);
    (void)gib;

    cudaFree(d_in); cudaFree(d_partial); cudaFree(d_out);
    free(h);
    return 0;
}
```

【代码做什么？】

1. **host 侧**：造一个有确定性的 `n = 2^24` 浮点数组（64 MiB），同时在 CPU 上用 `double` 累加出精确值作参考；分配 `d_in`、`d_partial[4096]`、`d_out[1]`；预热后连跑 50 次两阶段流程并计时。

2. **阶段 1 kernel**：4096 个 block × 512 线程 = **2,097,152 个 CUDA 线程**（映射为 65536 个 warp）。每个线程用 grid-stride 循环读 `n / (grid×block) = 8` 个元素并累加到私有寄存器 `sum`；然后**两级树形归约**：先在 warp 内用 `__shfl_down_sync` 把 32 个 lane 的值折成 1 个（无需共享内存、无需屏障），再把 16 个 warp 的部分和放进 64 B 的 `sdata`，`__syncthreads()` 后由 0 号 warp 收尾，写 `partial[blockIdx.x]`。
3. **阶段 2 kernel**：**只有一个 block**（这就是"用 kernel 分解替代全局同步"的落点：阶段 1 结束是一次隐式全局屏障），512 个线程把 4096 个部分和读进来放进共享内存，用**顺序寻址**做 `log2(512) = 9` 轮树形归约，最后由 `tid == 0` 写出总和的唯一结果。

【并行机制与性能解说】

- **线程/向量/warp 的创建与工作分配**：launch 语义是"批量创建"——一次 launch 声明 `gridsz × BLK` 个线程；硬件把它们按 32 个一组打包成 warp（每 block 16 个 warp），work scheduler 按资源把 block 派到 SM；每个 SM 每时钟最多选 4 个 warp 发射、每 warp 最多选 2 条指令（线程级 + 指令级并行）。
- **共享数据的处理**：`sdata` 是**块内共享**的中间结果（16 个 float 或 512 个 float）；块间通信**只能经 global memory 的 `partial` 数组**，并用 kernel 边界保证可见性/顺序。warp 内归约刻意**不走共享内存**——shuffle 直接在寄存器间搬数据，省掉 bank 访问与 5 次屏障。
- **Work / Span / 并行度**：
- **Work** = `n − 1 = 16,777,215` 次加法 ≈ **1.68e7 FLOP**（阶段 2 的 4096+511 次可忽略）。这是**work-optimal** 的：树形归约总加法数 = n−1，与串行相同。
- **Span** = 阶段 1 内：`(每线程 8 次串行累加) + (warp 内 5 步 shuffle) + (1 次 __syncthreads) + (warp 0 的 5 步 shuffle)` ≈ `8·4 + 5·(shuffle 延迟 ~10) + 30 + 50 ≈ 162` 周期，再加阶段 2 的 `grid-stride 8 次 + 9 轮 (共享读+加+屏障) ≈ 8·20 + 9·(30+30) ≈ 700` 周期，以及一次 kernel 边界（约 1–5 μs 量级）。以周期计，Span ≈ **900 周期 ≈ 0.72 μs**（不含 launch 开销）。
- **并行度 = Work / Span ≈ 1.68e7 / 900 ≈ 1.9e4**（按"加法个数 / 关键路径加法个数"的等价估算则是 `n / log n ≈ 1.0e6`）。无论哪种算法，**并行度都远大于硬件能同时运行的线程数（V100：163,840）**，所以"并行度不足"不是问题。
- **瓶颈诊断（数值见程序输出）**：
- **计算下界** = `1.68e7 / 12.75e12 = 1.3 μs`；
- **访存下界** = `64 MiB / 900 GB/s ≈ 74.6 μs`；
- 两者相差 **57 倍** → 这个 kernel 是**纯带宽受限**，优化目标是"把 HBM 流量压到 1 遍读 + 1 遍写"，任何额外的中间数组往返都是纯亏损。
- **反面教材（recitation 的 Idea 1）**：如果把树的每一层都做成一次 launch（24 层 = 24 次 launch），即使每层 kernel 本身很快，`24 × 约 5 μs ≈ 120 μs` 的 launch 开销就已**超过 74.6 μs 的数据搬运下界**；再加上每层都要把中间数组写回 global 再读回，实际时间会到 200 μs 以上。这就是"正确但很慢"的典型。
- **共享内存与屏障成本**：顺序寻址让每轮共享访问都落在不同 bank（无冲突，1 周期），但 `__syncthreads()` 本身有 ~几十周期的延迟，9 轮 × 2 次同步（阶段 2）在只有 1 个 block 时无法被其他 block 掩盖——好在阶段 2 的数据量只有 4096 个 float，**占比极小**。
- **负载不均**：`n = 2^24` 除以 `4096×512 = 2^21` 正好整除（每线程 8 个元素），无尾部；若 n 不是整除，grid-stride 会让前几个 block 多干一轮，此时应让**grid 大小略小于"SM × 每 SM 驻留 block 数"的整数倍**并依赖 grid-stride 循环平衡负载。

3.3 示例 C：共享内存原子操作实现直方图（原子性与调度自由的边界）

```
// histogram.cu ——  用 shared memory 原子操作建直方图, 再原子汇总到 global
// 编译(发布优化): nvcc -O3 -arch=sm_80 histogram.cu -o histogram
// 运行:          ./histogram
// 要点: (1) 块内原子化到共享内存 -> 大幅减少 global 原子竞争
//       (2) 这个用法合法: 原子操作只用于互斥, 不约束 block 的调度顺序

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>

#define NBINS 256
#define BLK   256

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,     \
                    cudaGetErrorString(err__));                             \
            exit(1);                                                        \
        }                                                                   \
    } while (0)

__global__ void histogram_shared(const unsigned char* __restrict__ data,
                                 long long n,
                                 unsigned int* __restrict__ hist)
{
    __shared__ unsigned int sHist[NBINS];        // 256 x 4 B = 1 KB

    for (int i = threadIdx.x; i < NBINS; i += blockDim.x)
        sHist[i] = 0u;
    __syncthreads();                             // 清零对所有线程可见后才能开始计数

    const long long stride = (long long)gridDim.x * blockDim.x;
    for (long long i = (long long)blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride) {
        atomicAdd(&sHist[data[i]], 1u);          // 块内竞争发生在片上, 延迟远低于 global
    }
    __syncthreads();                             // 保证所有计数完成后再汇总

    for (int i = threadIdx.x; i < NBINS; i += blockDim.x)
        atomicAdd(&hist[i], sHist[i]);           // 每个 block 对每个 bin 只做一次 global 原子加
}

int main()
{
    const long long n = 1LL << 26;                       // 64 Mi 个字节值
    unsigned char* h = (unsigned char*)malloc((size_t)n);
    unsigned int ref[NBINS] = {0};
    for (long long i = 0; i < n; ++i) {
        h[i] = (unsigned char)((i * 31 + 7) % NBINS);    // 确定性数据
        ref[h[i]]++;
    }

    unsigned char* d_data;
    unsigned int*  d_hist;
    CUDA_CHECK(cudaMalloc(&d_data, (size_t)n));
    CUDA_CHECK(cudaMalloc(&d_hist, NBINS * sizeof(unsigned int)));
    CUDA_CHECK(cudaMemcpy(d_data, h, (size_t)n, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemset(d_hist, 0, NBINS * sizeof(unsigned int)));

    const int gridsz = 1024;
    histogram_shared<<<gridsz, BLK>>>(d_data, n, d_hist);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    unsigned int out[NBINS];
    CUDA_CHECK(cudaMemcpy(out, d_hist, sizeof(out), cudaMemcpyDeviceToHost));

    long long bad = 0;
    for (int i = 0; i < NBINS; ++i) if (out[i] != ref[i]) bad++;
    printf("[verify] 不匹配的 bin 数 = %lld / %d ;  bin0: GPU=%u CPU=%u\n", bad, NBINS, out[0], ref[0]);
```

```
    cudaEvent_t e0, e1;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));
    const int iters = 20;
    CUDA_CHECK(cudaEventRecord(e0));
    for (int it = 0; it < iters; ++it)
        histogram_shared<<<gridsz, BLK>>>(d_data, n, d_hist);
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));
    float ms = 0.f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
    ms /= iters;

    printf("[perf  ] %.4f ms ; 读 %.2f MB -> %.1f GB/s\n",
           ms, (double)n / 1e6, (double)n / (ms * 1e-3) / 1e9);

    cudaFree(d_data); cudaFree(d_hist); free(h);
    return 0;
}
```

【代码做什么？】 每个 block 先在共享内存里放 256 个计数器（1 KB），清零并同步；然后用 grid-stride 循环读数据，对 `sHist[value]` 做**块内原子加**；最后每个 block 只对每个 bin 做**一次** global `atomicAdd`。结果是：**global 原子操作次数从 n 次（64 Mi 次）降到 `blocks × NBINS = 1024 × 256 = 262144` 次**。

【并行机制与性能解说】

- **原子操作与调度自由**：这是一个**合法**的 CUDA 用法——原子操作只充当互斥，无论 block 以何种顺序、是否并发执行，结果都相同，因此**不约束 work scheduler 的调度自由**。反例：让 block 1 自旋等待 block 0 写的标志位（`while(atomicAdd(&flag,0)==0){}`），在"只能驻留 1 个 block 的 SM"上若先跑 block 1 就永久死锁——**CUDA 只承诺"块可以任意顺序执行"，从不承诺"块之间没有依赖"能被满足**。
- **Work / Span / 并行度**：
- **Work** = `n` 次原子加（外加每个 block 的清零与汇总，各 `O(NBINS)`）→ **6.7e7 次原子操作**。
- **Span** = 最坏情况是"所有元素都落在同一个 bin"→ 该 bin 上的 n 次原子加被**串行化**，Span = Θ(n)；此时并行度 = Work/Span ≈ **1**，整个 kernel 退化为串行。实测数据接近均匀分布，因此真实的 Span 由"每个 bin 上的冲突次数"决定 ≈ `n / NBINS`（每 bin 约 26 万次串行原子）。这就是**基数/冲突决定的批判路径**：并行度 = 元素数 / 最大 bin 的冲突数。
- **并行度** = Work / Span ≈ `NBINS` 量级（均匀分布时），远小于硬件线程数 → **这个 kernel 的性能上限由"最热 bin 的原子串行化"决定，而不是由线程数决定**。缓解手段：让每个 warp/block 拥有私有副本（本示例的共享内存版本）、或把 bin 数减少到能被 warp 内 shuffle 处理。
- **瓶颈与实测预期**：读 64 MiB 的带宽下界 = `67.1e6/900e9 = 74.6 µs`；一旦共享内存上的原子冲突严重（随机 bin → 多次重试、bank 争用），实测会明显高于此值。注意 `sHist[NBINS]` 是 256 个 4 B 字，正好每 bank 8 个字，**随机 bin 会让同一 warp 的多个 lane 撞到同一 bank**，这是"共享内存原子争用"与"bank 冲突"叠加的典型案例。

3.4 示例 D：CPU 侧 OpenMP 参考实现（Work/Span 对照与 roofline 定位）

```cpp
// reduce_omp.cpp ——  同一归约任务的 CPU 多核版本，用来做"机器平衡点"对照
// 编译(发布优化): g++ -O3 -fopenmp -march=native reduce_omp.cpp -o reduce_omp
// 运行:          OMP_NUM_THREADS=16 ./reduce_omp

#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

int main()
{
    const long long n = 1LL << 24;                 // 与 CUDA 版同为 16,777,216 个 float (64 MiB)
    float* a = (float*)aligned_alloc(64, (size_t)n * sizeof(float));
    double exact = 0.0;
    for (long long i = 0; i < n; ++i) { a[i] = (float)((i % 1024) * 1e-3); exact += (double)a[i]; }

    double sum = 0.0, t0 = 0.0, t1 = 0.0;
    const int iters = 20;

    // 预热
    sum = 0.0;
#pragma omp parallel for reduction(+ : sum) schedule(static)
    for (long long i = 0; i < n; ++i) sum += (double)a[i];

    sum = 0.0;
    t0 = omp_get_wtime();
    for (int it = 0; it < iters; ++it) {
        double s = 0.0;
#pragma omp parallel for reduction(+ : s) schedule(static)
        for (long long i = 0; i < n; ++i) s += (double)a[i];
        sum = s;                                   // 避免被优化掉
    }
    t1 = omp_get_wtime();

    const double ms = (t1 - t0) / iters * 1e3;
    printf("[verify] OMP=%.4f  CPU(double)=%.4f  相对误差=%.3e\n",
           sum, exact, fabs(sum - exact) / exact);
    printf("[perf  ] 线程数=%d  耗时=%.4f ms  读 %.2f MB -> 有效带宽=%.2f GB/s\n",
           omp_get_max_threads(), ms, (double)n * 4 / 1e6,
           (double)n * 4 / (ms * 1e-3) / 1e9);
    printf("[expect] 64 MiB / 20 GB/s(单路 DDR) = %.2f ms ;  / 100 GB/s(双路服务器) = %.2f ms\n",
           (double)n * 4 / 20e9 * 1e3, (double)n * 4 / 100e9 * 1e3);
    free(a);
    return 0;
}
```

【代码做什么？】 用 `#pragma omp parallel for reduction(+:s) schedule(static)` 把同一归约任务分给 16 个线程，每个线程把自己那块连续区间（`schedule(static)` 意味着**每个线程拿到一大段连续地址**）累加进私有 `s`，最后由 OpenMP 运行时的归约把 16 个 `s` 合并（内部就是一棵小树）。计时用 `omp_get_wtime()`，跑 20 次取平均。

【并行机制与性能解说】

- **并行机制**：OpenMP 在进入并行区时**唤醒一个线程池**（线程数默认 = 核数/超线程数），用 fork–join 模式把循环的迭代块静态切给各线程；每个线程的 `s` 是**线程私有变量**（不在同一 cache line 上，避免伪共享——OpenMP reduction 的实现会做 padding）。这与 GPU 的"grid–stride + 树形归约"在结构上是**同构**的：私有部分和 → 树形合并。
- **Work / Span / 并行度**：Work = n−1 次加法；Span = `n/P`（每线程串行段） + $\log_2 P$（归约树）；并行度 = $(n-1) / (n/P + \log_2 P) \approx P = 16$。**这正是 CPU 与 GPU 的分野**：CPU 版并行度被核数（16）限制，GPU 版并行度可达 10^6 量级——但因为任务本身**带宽受限**，两者都会撞在同一堵墙（DRAM 带宽）上。
- **瓶颈诊断**：单路 DDR5 约 20 GB/s 时，`64 MiB / 20 GB/s = 3.35 ms`；双路服务器约 100 GB/s 时是 `0.67 ms`。**16 个核再多也压不穿内存墙**——这直接解释了为什么 GPU 要用 HBM（900 GB/s ~ 1 TB/s）：只有当带宽提高 10–50 倍，算力才有意义。

---

4. 性能模型与复杂度分析

4.1 Work–Span 在 GPU 上的映射

- **Work（W）** = 程序的总操作数；**Span（D）** = 最长依赖链；**并行度 = W/D**。
- 在 GPU 上，"并行度"要与两个硬件常量比较：
- **可驻留线程总数** = SM 数 × 每 SM warp 数 × 32。例如 V100：`80 × 64 × 32 = 163,840`；H100：`132 × 64 × 32 = 270,336`。
- **每时钟可发射的 warp 指令数** = SM 数 × 4 sub-core（V100：`80 × 4 = 320` 条 warp 指令/时钟）。
- **判据**：若 `并行度 ≫ 可驻留线程数`，则"并行度"不是瓶颈，应转而分析**带宽 / 算术强度 / 同步开销**；若 `并行度 < 可驻留线程数`，则 GPU 会大量空转（这正是直方图原子化、稀疏不规则算法的病根）。
- **Roofline（屋顶线）**的三条线：
- 计算屋顶 = 峰值算力 `P_peak`；
- 带宽屋顶 = `BW × I`（I = 算术强度 FLOP/Byte）；
- 机器平衡点 `I* = P_peak / BW`——**低于它 = 带宽受限，高于它 = 计算受限**。

4.2 算例一：机器平衡点与算术强度阶梯（matmul，N=1024）

假设参数（取其自讲义与 CS149 补充材料的 V100 数据）：

```
P_peak(fp32) = 80 SM × 4 sub-core × 16 lane × 1.245 GHz × 2 FLOP/FMA = 12.75 TFLOP/s
BW_HBM                                                              = 900 GB/s
BW_shared = 80 SM × 32 bank × 4 B × 1.245 GHz                       = 12.75 TB/s
机器平衡点 I* = 12.75e12 / 900e9                                    = 14.2 FLOP/Byte
```

算例：N = 1024 的稠密矩阵乘，总计算量 `2N³ = 2.147 GFLOP`，计算下界：

```
T_compute = 2.147e9 / 12.75e12 = 0.168 ms
```

各版本的访存量与时间下界：

版本	global 访问次数	global 字节数	算术强度 (FLOP/B)	T_mem = Bytes/900 GB/s	与 T_compute 之比	结论
Strawman（1 元素/线程）	$2N^3$ = 2.147e9	8.59 GB	**0.25**	**9.54 ms**	56.8×	严重带宽受限
+ 寄存器分块 V=2	$2N^3/V$ = 1.074e9	4.29 GB	0.50	4.77 ms	28.4×	仍严重受限
+ 共享分块 L=32, V=2	$2N^3/L$ = 6.71e7	268 MB	8.0	0.298 ms	1.8×	接近平衡
L=64, V=4（示例 A）	$2N^3/L$ = 3.36e7	**134 MB**	**16.0**	**0.149 ms**	**0.89×**	**计算受限（达标）**
L=128, V=4	1.68e7	67 MB	32.0	0.075 ms	0.44×	计算受限，global 不再是瓶颈
L=64, V=8	3.36e7	134 MB	16.0	0.149 ms	0.89×	但 acc 占 64 寄存器 → occupancy 崩

同一张表换到**共享内存维度**看（共享带宽 12.75 TB/s）：

V	共享访问次数 $2N^3/V$	字节数	T_shared = Bytes/12.75 TB/s	寄存器（仅 acc）
1	2.147e9	8.59 GB	0.674 ms	1
2	1.074e9	4.29 GB	0.337 ms	4
4	5.37e8	2.147 GB	**0.168 ms**	16
8	2.68e8	1.074 GB	0.084 ms	64 → 溢出风险

结论：`T ≥ max(0.168, 9.543/L [ms], 0.674/V [ms])`。取 `L=64, V=4` 时三项分别是 `0.168 / 0.149 / 0.168 ms`，**三堵墙同时被顶到**——这就是"两个复用旋钮调到位"的定量标志。继续调参只会让另一堵墙变得更矮：**性能工程就是找到这几堵墙的交点**。

4.3 算例二：1D 卷积的带宽下界（共享内存暂存的收益）

假设：N = 2^{20} = 1,048,576 个输出，`output[i] = (input[i]+input[i+1]+input[i+2])/3`，THREADS_PER_BLK = 128，每个 block 需要 128+2 = 130 个输入。

版本	global 加载次数	读字节	写字节	总字节	900 GB/s 下 T_mem	相对版本 1
版本 1（每线程直接读 3 次 global）	3N = 3,145,728	12.58 MB	4.19 MB	16.78 MB	18.6 μs	1.00×
版本 2（共享内存暂存 130 floats）	每 block 130 → 共 1.065e6	4.26 MB	4.19 MB	8.45 MB	**9.4 μs**	**1.99×**

计算量对照：`3N ≈ 3.15 MFLOP`（2 次加 + 1 次除，除算 1 FLOP）→ `3.15e6/12.75e12 = 0.25 μs`。**计算时间只占访存时间的 1.3%**，是极端带宽受限的例子；同时要注意版本 2 引入了一次 `__syncthreads()`（约几十周期）与 520 B/block 的共享内存占用，若**每线程工作量太小**（本算例每线程只算 1 个输出），屏障与 block 启动开销会开始显著。

再看另一个更朴素的对照（把 CPU 侧数字放进同一个模型）：

```
CPU: 16 核 × 3.0 GHz × 8 宽 SIMD × 2 (FMA)              = 768 GFLOP/s
     单路 DDR5 带宽                                     ≈   20 GB/s
     机器平衡点 = 768e9 / 20e9                           = 38.4 FLOP/Byte

     一个 256 MB 的数组，单次遍历(读+写各 256 MB)在 20 GB/s 下至少需
     512 MB / 20 GB/s = 25.6 ms       (只读一次则 256 MB / 20 GB/s = 12.8 ms)
     同期 GPU: 512 MB / 900 GB/s     = 0.57 ms          -> 快 45 倍
```

这解释了为什么"带宽"在 GPU 编程里比"算力"更常成为瓶颈：**GPU 的机器平衡点（14.2）比 CPU 的（38.4）更低，因为它的带宽相对算力更充裕，但也更容易被"多读几遍数据"的写法挥霍掉**。

4.4 算例三：归约的 Work/Span 与"kernel 分解层数"的代价

假设：n = 2^{24} = 16,777,216 个 float = 64 MiB，V100 参数，BLK = 512，grid = 4096。

```
Work  = n − 1          = 16,777,215 次加法  = 1.68e7 FLOP
Span  ≈ (n/线程数 的串行段) + (log2(32) shuffle) + (log2(512/BLK_warp) 树) + kernel 边界
      = 8 次串行加 + 5 步 + 5 步 + 1 次 launch 边界   ≈ 900 周期 ≈ 0.72 μs
可驻留线程数 = 80 SM × 64 warp × 32 = 163,840
同时可执行的独立部分和 = 4096 block × 512 线程 = 2,097,152
```

一个常被误用的比较：若直接用 `Work/Span = 1.68e7 / 900 ≈ 1.9e4` 与 `163,840` 相比，会得到"并行度不够"的错误结论——因为这里的 Span 以**周期**为单位、Work 以**操作数**为单位，两者不同量纲，不能直接相比。正确的比较是同量纲的"同时可执行的独立工作单元数"：`4096 × 512 = 2,097,152` 个独立部分和，**远大于 163,840 个可驻留线程**，因此并行度充足。真正决定性能的是带宽：

```
T_bandwidth = 64 MiB / 900 GB/s = 74.6 μs      <- 实际能达到的最好成绩
T_compute   = 1.68e7 / 12.75e12 = 1.3 μs
比值        = 57×                              <- 纯带宽受限
```

kernel 分解层数的代价（recitation 的四种归约方案对照）：

方案	正确性	同步手段	launch 次数（n=2^{24}, 逐层分解）	额外 global 流量	评价
Idea 1：每层一个 kernel	✅ 正确	kernel 边界（全局）	$\log_2(2^{24}) = 24$ 次	每层写回+读回 → 约 2× 流量	正确但很慢：24 × ~5 μs ≈ 120 μs 的 launch 开销已超过 74.6 μs 的搬运下界
Idea 2：kernel 内 `__syncthreads()`（跨 block）	❌ **错误**	试图块间同步	1 次	—	`__syncthreads()` 只对 block 内有效；跨块自旋还会死锁
Idea 2.1：只启动 1 个 block	✅ 正确	`__syncthreads()`	1 次	0	只用 1 个 SM，n 大时带宽利用极差（仅适合 n 较小）

方案	正确性	同步手段	launch 次数（n=2^{24}, 逐层分解）	额外 global 流量	评价
Idea 2.2 / 2.3：块内 `__syncthreads()` + 块间 kernel 边界 + 共享内存	✅ 正确	混合	`log_{BLK}(n)` ≈ 2—3 次	仅部分和	**推荐**：本笔记示例 B 即此方案

算一笔"层数 vs 带宽"的账：Idea 1 在 24 层里，第 i 层读写 `n/2^i` 个元素，总流量 ≈ `2n(1+1/2+1/4+…) ≈ 4n` 个元素 = `4 × 64 MiB = 256 MiB` → 单是流量就 `284 μs`，再加 120 μs 的 launch 开销 → **约 400 μs，是理想值的 5 倍以上**。而示例 B 的流量只有 `n` 个元素（读） + 4096 个部分和（写）≈ 64 MiB → `74.6 μs`。**同一个算法、同一个 Work/Span，两种分解方式差 5 倍。**

4.5 算例四：延迟隐藏的定量要求（Little's Law）

内存延迟不会因为"有很多线程"自动消失，它必须被**并发请求数**掩盖：

```
Little 定律:  并发请求数(in flight) = 带宽 × 延迟
假设: HBM 往返延迟 ≈ 500 周期, V100 时钟 1.245 GHz, 带宽 900 GB/s
  延迟(秒) = 500 / 1.245e9 = 4.016e-7 s
  需要 in-flight 字节 = 900e9 × 4.016e-7 = 361 KB
  摊到 80 个 SM: 约 4.5 KB / SM 必须同时在路上

每个 warp 的一次合并访存 = 32 lane × 4 B = 128 B
  => 每个 SM 需要约 4.5 KB / 128 B ≈ 36 条 warp 访存同时在途
  => 该 SM 至少要能驻留并保持发射 ~36 个 warp（V100 上限 64 个 warp）
```

结论：occupancy（驻留 warp 数）是"延迟隐藏能力"的代理指标。这也解释了寄存器分块的代价：V100 每 SM 只有 65536 个寄存器；若一个 kernel 用 40 个寄存器/线程，则最多驻留 `65536/40 = 1638` 个线程 = **51 个 warp**（而不是 64），occupancy ≈ 80%；若 V=8 的 matmul 用到 80+ 个寄存器，驻留线程降到 819 = 25 个 warp，**可能低于隐藏 500 周期延迟所需的 36 个 warp**，于是"减少了访存却跑得更慢"。

一个具体的自检公式：

```
某 kernel 每线程寄存器的编译结果 R  ->  可驻留 warp 数 W = 65536 / (R × 32)
若 W < 需要掩盖的并发访存数(≈ 带宽×延迟 / (SM数 × 128 B))  ->  带宽用不满
```

4.6 Amdahl 与"kernel 边界"的隐含税

设串行部分（host 侧准备 + kernel 启动 + 结果回收）占比为 s，则加速比 `S = 1/(s + (1−s)/P)`。在 GPU 上，`s` 的常见来源是**每次 launch 的固定开销**与**host↔device 拷贝**：

```
假设 launch 开销 5 μs, 数据搬运 8.45 MB / 900 GB/s = 9.4 μs（卷积例）
若把算法拆成 24 次 launch: 固定开销 = 120 μs
   相对"数据搬运 9.4 μs" 的串行占比 s = 120/(120+9.4) = 92.7%
   => 无论 GPU 有多少个 SM, 加速比上限 = 1/s ≈ 1.08 !
```

这就是"kernel 分解要**少而大**"的定量理由：**能用 2 次 launch 解决，就绝不用 24 次**。

---

5. 关键要点

1. **三条判据决定 CUDA kernel 的成败**：coherent warps（warp 内不发散）、coalesced memory access（合并访存）、高数据复用（register tiling + shared memory tiling）。它们不是三个独立技巧，而是同一个目标——**让每个时钟的每个 lane 都在做有用的事，并且这些 lane 需要的字节已经被搬到片上**。
2. **两个复用旋钮分管两层存储**：`L`（block 级共享内存瓦片）把 global 流量降到 `2N³/L`，`V`（线程级寄存器瓦片）把 shared 流量降到 `2N³/V`。性能上界 `T ≥ max(T_compute, 2N³·4/(L·BW_HBM), 2N³·4/(V·BW_shared))`；调到三堵墙等高时就是该硬件上的最优分块。
3. **CUDA 没有全局屏障**，跨块同步只能靠 **kernel 分解**；而每次 launch 都有固定开销，所以分解要"少而大"。任何"块间自旋等待"的代码在"驻留 block 数 < block 总数"的机器上都可能死锁——因为 block 可以被按任意顺序执行。
4. **优化的第一步永远是定位瓶颈类型**：先算算术强度 `I` 与机器平衡点 `I* = P_peak/BW`；`I < I*` 就去减少字节搬动（分块、合并访存、缩短数据类型），`I > I*` 才去优化指令数（向量化、减少发散、用 Tensor Core）。在 `I < I*` 时优化计算是纯粹的浪费。

5. **访存模式比访存次数同样重要**：32 lane × 4 B 的连续访问 = 1 次全宽事务（最优）；stride 访问会被拆成多次事务（有效带宽除以拆分数）；共享内存 stride-2 访问产生 2 路 bank 冲突（带宽减半）。**顺序寻址（sequential addressing）同时解决合并访存与 bank 冲突**，是归约类 kernel 的标准收尾。
6. **延迟必须被并发掩盖**：由 `in-flight 字节 = 带宽 × 延迟` 可算出所需并发访存数（V100 约 36 条 warp 访存/SM），这直接约束了寄存器分块的规模上限——**occupancy 与复用率是一对必须权衡的矛盾**。

---

6. 常见陷阱与注意事项

- **把 `__syncthreads()` 当作跨 block 的屏障**：它**只对同一 block 内的线程有效**。recitation 里"在 kernel 里直接加 `__syncthreads()` 就以为能跨块同步"的写法是**错误的**（版本 2 的错误原因）；跨块只能靠 kernel 边界，或换用 cooperative groups 的 grid sync（需要特殊的 launch 方式与驻留保证）。
- **在 block 之间做自旋等待（spin-wait）**：`while (atomicAdd(&flag,0)==0) {}` 这类代码在"SM 只能驻留 1 个 block 且先跑了等待方"时**永久死锁**。CUDA 只保证"块可以任意顺序执行"，从不保证"并发执行"或"依赖能被满足"。**用原子操作做计数/直方图是合法的（只用于互斥），用原子操作做块间握手是不合法的。**
- **忽略 warp 内的发散代价**：`if (tid % (2*s) == 0)` 这种写法让一半 lane 在第一步就空转、后续步骤活跃率按 1/4、1/8 递减（示例中对 16 个 lane 的三个步骤有效利用率只有 `(8+4+2)/(16×3) = 29%`）。改成"strided index + 非发散分支"（`index = 2*s*threadIdx.x; if (index < blockDim.x)`）或顺序寻址，可以让**前面的整 warp 满负荷工作**，只把不可避免的部分留在尾部。
- **把 shared memory 当成"免费"的**：共享内存有 32 个 bank、每 bank 每时钟只能服务 1 个字。stride 访问（如 `sdata[2*tid]`）会产生 bank 冲突使带宽减半；`float` 数组按 `sdata[tid*stride]` 访问更会出现严重冲突。**解决办法：连续访问、或者把共享内存数组的列数 padding 成奇数（例如 `[BLK][BLK+1]`）来打散 bank 映射。**
- **为了减少 global 访问而无节制地增大 V，结果 occupancy 崩掉**：V×V 个 accumulator 全部占用寄存器；V=8 时光 accumulator 就要 64 个寄存器/线程，加上寻址会突破 80，使每 SM 驻留 warp 数从 64 掉到 20 多，**延迟无法隐藏，反而变慢**。判断依据是 4.5 节的 `W = 65536/(R×32)` 与 Little 定律给出的并发需求。
- **忽视"写回"也是一次访存，以及忽视 L2 的存在**：写 C 矩阵（$N^2 \times 4$ B）在 N 大时是实打实的 DRAM 流量；而当 N 小到 A、B 能装进 L2（V100 6 MB）时，按"每次 block 都回 HBM"估算的流量会严重高估时间——**真实瓶颈会从 HBM 带宽变成 L2 带宽**。分析时必须分清"至 L2 的流量"与"至 DRAM 的流量"。
- **把 CUDA 的线程当成 pthread**：CUDA 线程没有"创建/销毁"的运行时语义，硬件只保存寄存器状态；但**一个 block 的所有线程必须同时获得执行上下文**（因为它们按语义就是并发的，且可能有 `__syncthreads()` 依赖），所以"先跑线程 0–127 再跑 128–255"是不允许的。同样，块内线程与 warp 的关系（连续 32 个 tid 属于同一 warp）、以及"warp 不是编程模型的一部分而是实现细节"这一点，都必须在写代码时记在心里。

---

7. 思考题（带答案）

问题 1（分块参数的定量选择）：某 GPU 的 fp32 峰值算力 `P_peak = 20 TFLOP/s`，HBM 带宽 `BW = 1 TB/s`，共享内存总带宽 `BW_shared = 20 TB/s`。要算 `N = 2048` 的稠密矩阵乘，请：(a) 求机器平衡点 `I*`；(b) 写出以 L（block 瓦片边长）和 V（线程瓦片边长）表示的三个时间下界；(c) 选一组 (L, V)，使得三项大致平衡，并说明寄存器约束；(d) 若有人把 V 从 4 提到 8，会出现什么问题?

【答案】 (a) `I* = P_peak / BW = 20e12 / 1e12 = 20 FLOP/Byte`。低于 20 FLOP/B 的写法就是带宽受限。

(b) 总计算量 $2N^3 = 2\times2048^3 = 1.718e10$ FLOP。 – 计算下界：`T_comp = 1.718e10 / 20e12 = 0.859 ms` – global 下界：总访问次数 $2N^3/L$，字节数 $8N^3/L$ = `6.87e10 / L` B → `T_global = 6.87e10/(L × 1e12) s = 68.7/L ms` – shared 下界：总访问次数 $2N^3/V$，字节数 $8N^3/V$ → `T_shared = 6.87e10/(V × 20e12) s = 3.43/V ms`

(c) 令三者相等： – `68.7/L = 0.859` → `L ≈ 80`（取 64 或 128，实践中取 64 或 128 这样的 2 的幂） – `3.43/V = 0.859` → `V ≈ 4` 取 **L = 64, V = 4**：`T_comp = 0.859 ms`，`T_global = 68.7/64 = 1.073 ms`，`T_shared = 3.43/4 = 0.858 ms` → 上界 `max = 1.073 ms`，与计算下界同量级（差 25%），已经接近最优。若想再压 global，取 **L = 128, V = 4**：`T_global = 0.537 ms < 0.859 ms`，此时 `T = max(0.859, 0.537, 0.858) = 0.859 ms`，**正好卡在计算屋顶上**——这就是"三堵墙等高"的解。 寄存器约束：`acc[V][V] = 16` 个浮点寄存器 + `a[4] + b[4] = 8` 个 + 索引/指针约 12–16 个 ≈ **40 个寄存器/线程**。共享内存 `/` block：`(BM×BK + BK×BN) × 4 B`，若 BK=16、L=128 则为 `(128×16 + 16×128)×4 = 16 KB`——需与 SM 的共享内存容量（H100 为 256 KB）和寄存器文件（若 64K 寄存器/SM，40 reg/thread → 每 SM 最多 1638 线程 ≈ 51 warp，occupancy ≈ 80%）一起权衡。

(d) V 从 4 提到 8：`acc` 需要 64 个寄存器，加 `a[8]+b[8]` 与寻址将超过 **80–90 个寄存器/线程**。后果有两层：(1) **寄存器溢出（register spilling）**，多出来的 accumulator 被放到 local memory（本质是 global memory），访存量反而暴涨；(2) 即使不溢出，`W = 65536/(R×32)` 会让

可驻留 warp 数从 ~51 掉到 ~22，**低于隐藏内存延迟所需的并发 warp 数**，于是带宽用不满、kernel 变慢。而 T_shared 从 0.858 ms 降到 0.429 ms 的收益此时**完全用不上**，因为 `T_comp = 0.859 ms` 才是屋顶。**结论：V 不是越大越好，它要停在"共享内存不再是最慢的那堵墙"的位置。**

问题 2（归约的正确性与性能）：下面这段 kernel 想把长度为 n 的数组求和（`g_odata[blockIdx.x]` 存每个 block 的部分和），但它既可能算错、又慢。请指出**两处正确性问题**与**两处性能问题**，并给出修改方案。

```
__global__ void reduce_bad(const float* g_idata, float* g_odata, int n) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    sdata[tid] = g_idata[i];
    for (unsigned int s = 1; s < blockDim.x; s *= 2) {
        if (tid % (2 * s) == 0)
            sdata[tid] += sdata[tid + s];
    }
    g_odata[blockIdx.x] = sdata[0];
}
```

【答案】

正确性问题 1：缺少 `__syncthreads()`。 CUDA **不保证** block 内线程 lockstep 执行，而在同一轮里，`sdata[tid+s]` 可能正被另一个线程写入（第 s 轮里 tid 读取的 `tid+s` 正是另一个线程的位置），下一轮又要读本轮的结果。没有屏障，就会出现读写竞争，结果不确定。**修改**：在进入循环前加一次 `__syncthreads()`（保证装载完成），并在循环体末尾每轮加一次 `__syncthreads()`。注意：**循环体内的屏障是必需的，且必须放在"所有线程都会执行到"的位置**——如果把它放进 `if (tid % (2*s) == 0)` 里，就会因为分支内屏障而使 warp 死锁。

正确性问题 2：越界访问（数组边界）。 当 `blockDim.x` 不是 2 的幂、或 `n` 不是 `blockDim.x` 的整数倍时，最后一个 block 的线程会读 `g_idata[i]` 越界（`sdata[tid+s]` 在 `tid+s ≥ blockDim.x` 时也越界）。**修改**：装载时加 `sdata[tid] = (i < n) ? g_idata[i] : 0.f;`，并保证 `blockDim.x` 为 2 的幂（或把归约循环条件收紧）。

性能问题 1：warp 高度发散。 `if (tid % (2*s) == 0)` 的第一个 warp（lane 0–31）在 s=1 时只有偶数 lane 活跃（有效利用率 1/2），s=2 时 1/4，s=4 时 1/8……整个前几轮的 lane 利用率是 `(16+8+4+2+1)/(32×5) = 31/160 ≈ 19%`。**修改**：改成顺序寻址（reversed loop）：

```
for (unsigned int s = blockDim.x / 2; s > 0; s /= 2) {
    if (tid < s) sdata[tid] += sdata[tid + s];
    __syncthreads();
}
```

这样第一轮有 `blockDim.x/2` 个线程活跃（多个**完整** warp 满负荷），只有最后 5 轮落在单个 warp 内，且那 5 轮的分支是"前缀活跃"，代价最小。

性能问题 2：共享内存 bank 冲突 + 非合并访存。 `sdata[tid]` 与 `sdata[tid+s]` 在交错寻址下，活跃 lane 的地址是 `0, 2s, 4s, ...`，相邻活跃 lane 相隔 `2s` 个字 → 当 s=1 时就是 stride-2 访问，落在偶数 bank 上，**产生 2 路 bank 冲突**（例如 lane 0–7 访问 word 0,2,4,6,8,10,12,14 → B0 服务 t0,t4，B2 服务 t1,t5 …，4 个 bank 各服务 2 个 lane，另外 4 个 bank 空闲，访问串行化为 2 个周期，带宽减半）。顺序寻址下 `sdata[tid]`（tid=0..s−1 连续）与 `sdata[tid+s]`（连续区间）**都是连续访问 → 无 bank 冲突**。同时 `g_idata[i]` 是连续 tid 读连续地址，本来就是合并访存——这一点在两版中都成立，要保留。

补充（可选的进一步优化）：把 `warp_reduce_sum` 用 `__shfl_down_sync` 做最后 32 个元素的归约（省掉 5 轮共享内存访问与 5 次屏障），并在 `blockDim.x >= 64` 时先做"每个线程读多个元素"的 grid-stride 循环，把 Work 与访存比提高；最后用一个 block 的第二个 kernel（或 `atomicAdd`）汇总各 block 的部分和——因为**CUDA 没有全局屏障**。

问题 3（Roofline 应用与决策）：某 CUDA kernel 处理一个 `n = 2^24` 的 float 数组（64 MiB），做以下三件事之一： – (A) 求和（1 次读，1 个标量输出）； – (B) `y[i] = a*x[i] + y[i]`（读 x 与 y，写 y）； – (C) 10 阶多项式求值 `y[i] = c0 + x[i]*(c1 + x[i]*(... ))`（读 x，写 y，每次 10 个 FMA）。

GPU 参数：`P_peak = 12.75 TFLOP/s`（fp32 FMA），`BW = 900 GB/s`，`I* = 14.2 FLOP/B`。请分别算出算术强度、判断瓶颈类型、给出理论最短时间，并说明应该优先优化什么。

【答案】

统一取 `n = 16,777,216` 个元素。

(A) 求和 – 流量：读 `n × 4 B = 67.1 MB`（输出 4 B 可忽略）。 – 计算：`n − 1 ≈ 1.68e7` FLOP（加法）。 – 算术强度 `I = 1.68e7 / 67.1e6 B = 0.25 FLOP/B` ≪ `I* = 14.2` → **极端带宽受限**。 – `T = max(67.1e6/900e9, 1.68e7/12.75e12) = max(74.6 µs, 1.3 µs) = 74.6 µs`。 –

优先优化：任何"减少字节"的手段——用多阶段 kernel 避免中间数组往返、用 `float4` 向量化访存提高事务效率、确保合并访存、必要时改用更窄的数据类型（bf16/fp16 会把流量砍半，但要注意精度）。**优化计算毫无意义**。

(B) `y[i] = a*x[i] + y[i]` – 流量：读 x（67.1 MB）+ 读 y（67.1 MB）+ 写 y（67.1 MB）= **201.3 MB**。 – 计算：`2n = 3.36e7` FLOP（1 次 FMA）。 – `I = 3.36e7 / 201.3e6 = 0.167 FLOP/B` → **带宽受限，且比 (A) 更严重**（要搬 3 份数据）。 – `T = max(201.3e6/900e9, 3.36e7/12.75e12) = max(223.7 µs, 2.6 µs) = 223.7 µs`。 – **优先优化**：把 y 留在片上（寄存器分块：每个线程一次读入 k 个 y、算完再写回，减少 y 的重复读写）、提高每次访存的元素数（float4）、把 x 与 y 的访问合并成连续的大事务。若整个数组能驻留 L2 也可以显著降低 DRAM 流量。

(C) 10 阶多项式 – 流量：读 x（67.1 MB）+ 写 y（67.1 MB）= **134.2 MB**（系数在常量内存/立即数里，可忽略）。 – 计算：`10 FMA × n = 1.68e8 FMA = 3.36e8 FLOP`。 – `I = 3.36e8 / 134.2e6 = 2.5 FLOP/B`，仍 < 14.2 → **仍偏带宽受限，但已接近临界**（差 5.7 倍）。 – `T = max(134.2e6/900e9, 3.36e8/12.75e12) = max(149.1 µs, 26.4 µs) = 149.1 µs`。 – **优先优化**：仍然是减少字节（合并访存、向量化、避免 y 的额外读），但当 K 继续增大（例如 50 阶）时 `I = 12.5`，会逼近 `I*`，此时 **ILP（Horner 链是串行依赖！需要拆成 2–4 条独立链）与取指/发射效率** 就变成新的瓶颈——**这正是"从带宽受限转向计算受限"的分界线**。

总结：三者都落在带宽受限区，但受限程度依次递减（0.25 → 0.167 → 2.5 FLOP/B）。这直接给出优化优先级：**先问"能不能少搬字节"，再问"能不能算得更快"**；只有当 `I > I*` 时（本例中需要 K ⩾ 57 阶多项式这类高复用计算）才轮到 FMA 吞吐、ILP 与 Tensor Core 的优化。

问题 4（执行语义判断）：判断下列 CUDA 代码片段的合法性与正确性，并说明理由。

```
// (1) 直方图: 所有 block 用原子操作更新同一 global 数组
__global__ void k1(int* A, int* counts, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) atomicAdd(&counts[A[i]], 1);
}

// (2) 块间握手: block 1 等待 block 0 完成
__global__ void k2(int* myFlag) {
    if (blockIdx.x == 0) { /* 干活 */ atomicAdd(myFlag, 1); }
    else { while (atomicAdd(myFlag, 0) == 0) { } /* 干活 */ }
}

// (3) 块内屏障放在分支里
__global__ void k3(float* A, int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n) {
        A[i] *= 2.0f;
        __syncthreads();     // 只有部分线程会执行到
    }
}
```

【答案】

(1) 合法且正确。 原子操作用于**互斥**（保证对 `counts[A[i]]` 的读–改–写不可分割），它**不约束** work scheduler 的调度自由：无论 block 以任何顺序、任何并发度执行，最终计数都相同（因为加法可交换并结合）。这是 CUDA 中"跨 block 通信"的少数合法形式之一。注意性能问题仍然存在：**同一个 bin 上的原子加会串行化**，热点 bin 决定 Span，因此需要 block 私有的共享内存副本或分片计数来降低争用。

(2) 不合法/可能死锁。 它依赖"block 0 与 block 1 并发执行"这一**CUDA 从未承诺**的语义。CUDA 只保证 block 可以**按任意顺序**执行（无依赖假设）。在"只有 1 个 SM 且每 SM 只能驻留 1 个 block"的 GPU 上，如果调度器先运行 block 1，block 1 会永远自旋（因为 block 0 得不到资源），**整个 kernel 挂死**。正确做法是用 **kernel 分解**：把"干活"和"等待后继续干活"放到两次 launch 中，利用 kernel 边界作为全局同步点。

(3) 不合法（未定义行为，可能挂死）。 `__syncthreads()` 是**块内全部线程**的屏障：它要求 block 中的**每一个**线程都到达该点。这里屏障被放在 `if (i < n)` 分支内，当 n 不是 `blockDim.x` 的整数倍时，最后一个 block 中部分线程不会执行到屏障（并且同一个 warp 内的 lane 会分叉），导致：(a) 在不支持独立线程调度的旧架构上直接永久挂起；(b) 在 Volta 及以后虽然支持 per-thread 调度，但 `__syncthreads()` 语义仍要求所有非退出线程到达，且 warp 内分叉执行屏障是明确定义的未定义行为。**修改**：把 `__syncthreads()` 移到分支**外面**（所有线程都会经过的公共路径上），分支只包住真正的计算；或者根本不给这个 kernel 加屏障（它本来不需要——每个线程只写自己的 `A[i]`）。

总结：判断一段 CUDA 代码是否合法，只看两个问题——**(i) 它是否假设了 block 之间的执行顺序或并发性？(ii) 它是否让块的全体线程无法一致地到达每一个 `__syncthreads()`？** 任何一条答"是"，就必须重写。

Lecture 7: Parallel Programming Basics

1. 章节标题与概述

Lecture 7: Parallel Programming Basics（并行编程基础）

- **本讲核心问题**：拿到一个串行程序（或一个问题描述），**怎样系统地把它变成一个正确的、可扩展的并行程序**？讲义给出的答案是四个必须依次回答的问题——**Decomposition（分解：把问题切成可并行执行的 task）→ Assignment（分配：把 task 交给 worker）→ Orchestration（编排：通信、同步、数据布局、调度）→ Mapping（映射：把 worker 落到物理执行单元上）**——并反复强调这条链条上真正的"拦路虎"是**依赖（dependencies）**：依赖决定并行度上限（Amdahl 定律），依赖决定你需要哪种同步机制（锁 / 屏障 / 消息 / 标志位），依赖决定你把数据切成什么形状（blocked 还是 interleaved，一维还是二维分块）才能既负载均衡又少通信。
- **涉及的主要硬件/软件机制**：
- **软件侧**：三种编程模型的协同使用（共享地址空间 shared address space、消息传递 message passing、数据并行 data parallel）、SPMD（Single Program Multiple Data，单程序多数据）执行模型、`for_all` / ISPC `foreach` 数据并行循环、ISPC `programCount` / `programIndex` 与 `launch[N]` 任务、pthreads 的静态块分配、Cilk 式**工作–跨度（work–span）**模型、锁 lock / 屏障 barrier / 原子操作（atomic read–modify–write）/ 标志位 + 自旋（flag–based spin，等价于长度为 1 的消息队列）。
- **硬件侧**：多核 CPU 的缓存层次与 SIMD lane（ISPC 的 program instance 被编译器映射到向量 lane）、GPU 的 SM（CUDA thread block 由硬件映射到 SM）、集群节点的私有地址空间 + NIC + 网络（延迟 α、带宽 β）、以及决定一切的**内存带宽**与**同步代价**。
- **抽象与实现的接口**：同一份并行算法在三种模型下写出三种结构完全不同的代码（`for_all` / SPMD + 锁与屏障 / send–recv），而"哪种更好"取决于机器——讲义明确说 "it depends on the system this program is running on"。
- **在并行计算知识体系中的角色**：这是全课程从"硬件/模型导论"（第 1–6 讲）转向"如何动手写并行程序"的分水岭，也是后续 Performance Optimization、同步与一致性、锁与无锁、异构计算等讲座的**词汇表来源**。第 8 讲起的性能优化（测量、局部性、伪共享）、第 10–17 讲的缓存一致性 / 内存一致性 / 同步 / 无锁数据结构，全部建立在本讲引入的 Decomposition/Assignment/Orchestration 框架与"依赖 → 同步"这条因果链上；作业 2（Assignment 2）在此后发布，正是要求把这些概念落到真实机器上。
- **配套材料**：
- `lectures/06_progbasics.pdf`（对应抽取文本 `extracted/06_progbasics.txt`，共 **63 页**幻灯片，首页明确写作 "Lecture 7: Parallel Programming Basics"，页脚学期字样为 Fall 2026）：**已公开**，位于 Fall 2026 公开讲义目录 https://www.cs.cmu.edu/~418/lectures/ 之下，可公开下载。Fall 2026 日程表 https://www.cs.cmu.edu/~418/schedule.html 中本讲位于 **Sep 9**。
- 讲义第 2 页的课程通知（原文）：Assignment 1 due tonight（+ late days）、Assignment 2 released tonight、周五有一次 ~5 分钟的 GPUs/CUDA 小测验、所有人已加入 ed / Autolab / Gradescope。
- 历史学期的消息传递专题讲义（如 `f22_26-msgpassing`、`s23_20a_msgpassing` 对应的 PDF）在本地仅有登录页抽取结果（内容为空），位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- 录制视频（Panopto/YouTube）：Fall 2026 日程表中被注释隐藏，属 **未发布**。Ed 讨论区、Autolab、Canvas、Gradescope 均**需登录**，非公开。
- `cs149_supp/thoughtprocess.txt`（Stanford CS149 Fall 2025 Lecture 4 "Parallelizing Code: The Programming Thought Process"，74 页）：**已公开**的姊妹课程补充读物，内容与本讲高度重叠（同一套来自 Culler/Singh/Gupta 的网格求解器案例、Amdahl 定律、ISPC/pthreads 分配示例），本笔记在用到其中的额外数据（如 Summit 的 27,648 GPU × 5,376 ALU 规模算例）时会显式标注来源。
- Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。讲义首页的学期页脚沿用历史版本属正常现象。

---

2. 核心概念与硬件/软件架构图解

2.1 起点：三种编程模型的复习，与"混合模型"的工程现实

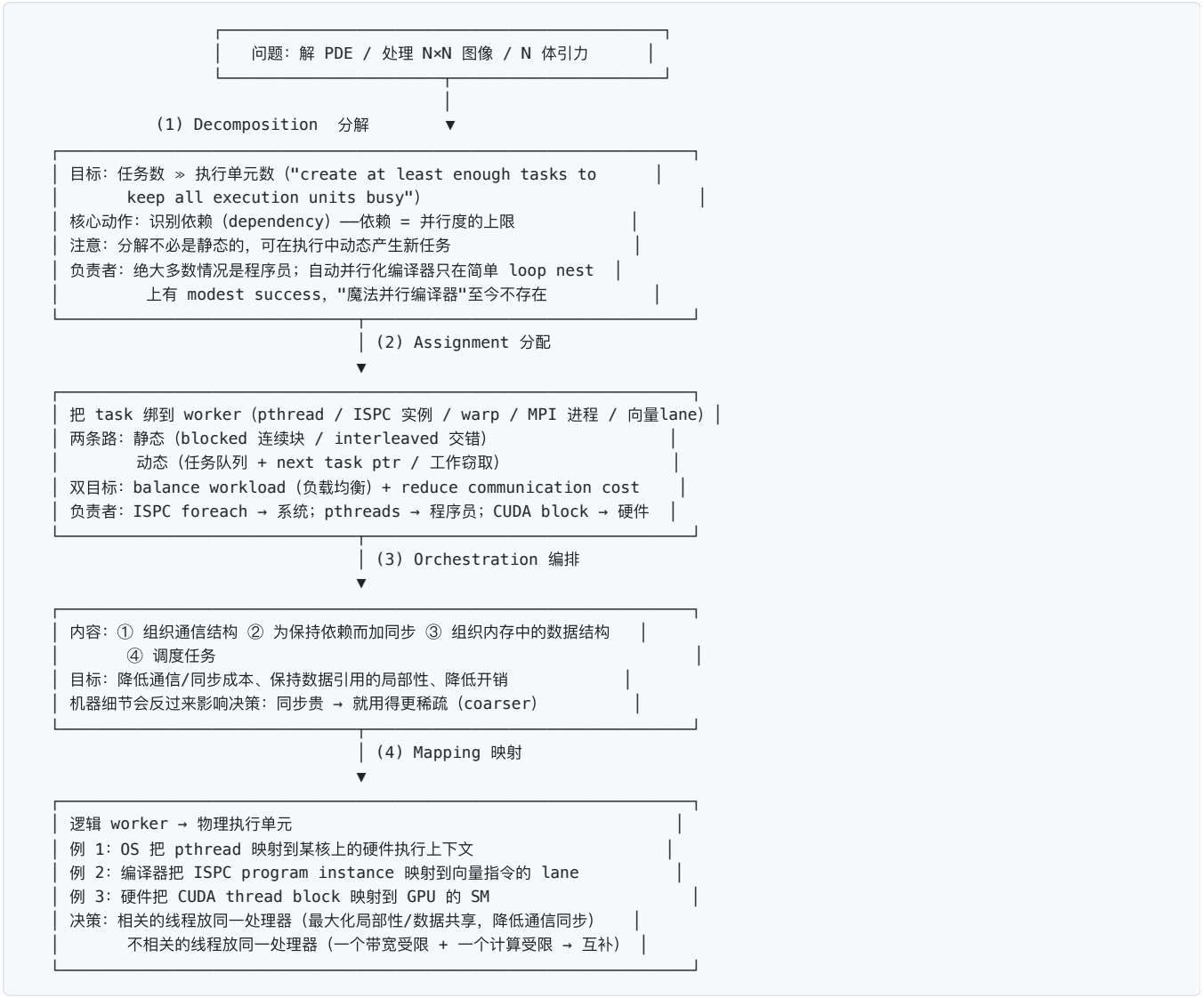
- **定义与目的**：并行编程模型是"程序员描述并行性、通信与同步的约定"。讲义复习了三种：
- **共享地址空间（shared address space）**：通信隐式地发生在 load/store 里，**非结构化**。最自然，但"很容易打中自己的脚"——程序可能正确却毫无性能（因为隐式的通信/一致性代价看不见）。
- **消息传递（message passing）**：所有通信都必须结构化地表达为消息（send/recv）。**写出第一个正确版本的难度高于共享地址空间**，但结构本身常常帮助你先写出正确且可扩展的程序（因为你被迫显式面对通信与所有权）。
- **数据并行（data parallel）**：把计算组织成对一个集合的大 `map`；它**假设存在共享地址空间**来 load 输入 / store 结果，但**严格限制 map 各次迭代之间的通信**（目标是保持迭代的独立处理）。现代实现（CUDA、OpenCL 的 kernel 内同步原语）**鼓励但不强制**这种结构，允许有限的迭代间通信。

- **直观解释（"它是什么？"）**：把三种模型想成**同一个办公室里的三种协作方式**。共享地址空间 = 大家共用一块白板，谁想写就写、谁想看就看——效率极高，但两个人同时改同一行数字就会互相覆盖。消息传递 = 每人有自己带锁的笔记本，要共享信息必须**写便条塞进对方信箱**——麻烦，但绝不会有人偷改你的笔记。数据并行 = 一份**流水线作业单**：把一叠文件平均发给每个人，每人按同一条规则各改各的文件，互不通信。
- **工程现实（现代实践：混合模型）**：**在集群的一个多核节点内部用共享地址空间编程，在节点之间用消息传递**——这是"非常非常常见"的做法：在能高效实现共享地址空间的地方（节点内）享受便利，在别处要求显式通信。同时，数据并行风格的模型允许 kernel 内使用共享内存风格的同步原语（CUDA 的 `__syncthreads()`、OpenCL 的 barrier）。

维度	共享地址空间	消息传递	数据并行
通信机制	隐式：load/store 共享变量	显式：send/recv 消息	隐式 load/store；复杂通信靠内置原语（`reduceAdd` 等）
同步机制	互斥（锁）+ 屏障表达相位依赖	消息本身即同步（发送/接收即事件）	`for_all` 循环体末尾**隐式屏障**
数据布局	单一地址空间、单一数组	每个线程私有地址空间，需**复制/ghost cell**	单一集合（数组/序列）
谁负责同步	程序员	程序员（但从类型系统/结构上被约束）	系统/运行时
首个正确版本的难度	低（但容易埋错）	高	最低
可扩展性	受缓存一致性与同步成本限制（通常单节点）	好（跨节点）	好（若迭代真独立）
代表实现	pthreads、OpenMP、Cilk	MPI	ISPC `foreach`、CUDA/OpenCL kernel、`for_all`

2.2 从问题到机器：四个阶段（Decomposition / Assignment / Orchestration / Mapping）

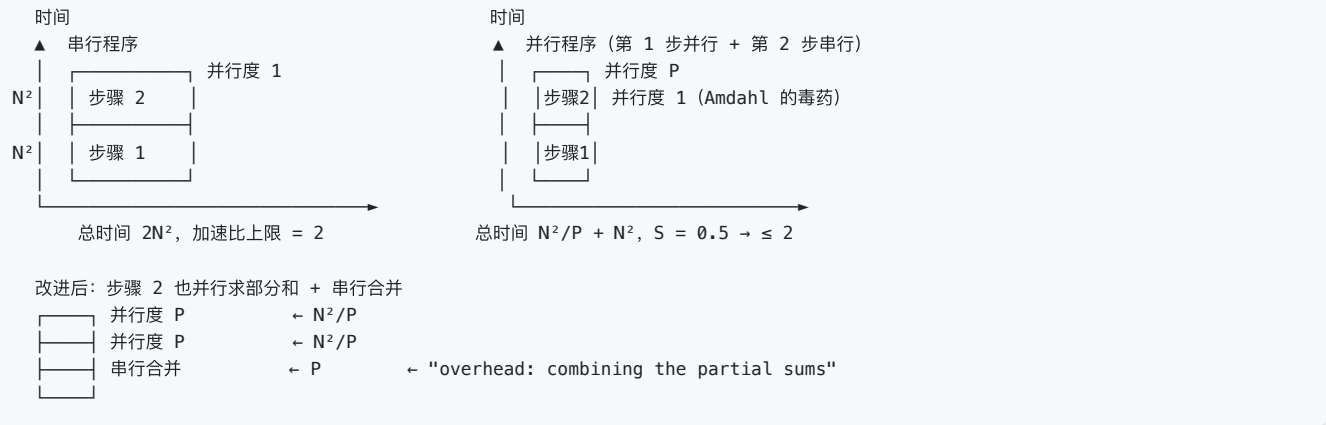
- **定义与目的**：讲义把"创建一个并行程序"的思维过程形式化为一条流水线。它是本讲的骨架，也是后续所有性能问题定位的坐标系：性能不好时，先问"是分解出了错（依赖没找对 / 任务太少），还是分配不均，还是编排（同步/通信）太贵，还是映射破坏了局部性"。
- **直观解释（"它是什么？"）**：类比**盖楼**。Decomposition = 把整栋楼拆成一张张图纸（任务）；Assignment = 决定哪个施工队做哪张图纸；Orchestration = 安排塔吊、材料堆放、以及"哪道工序必须等哪道工序"的交接；Mapping = 把人真正派到工地上（甚至把相关工序安排在同一层楼以减少往返）。
- **架构/机制图解**：



性能特征：四个阶段对性能的影响是**乘性**而非加性的。分解决定了理论上限（Amdahl），分配决定了负载不均衡造成的"最长工人"时间，编排决定了每次迭代的固定同步开销（屏障延迟 ~1–3 μs 量级、一次锁 100 ns 量级、一条网络消息 α ~1–2 μs + m/β），映射决定了缓存命中率与有效带宽（20 GB/s 是节点级共享资源，多线程争抢后每个线程分到的更少）。**吞吐量 = min(算力上限, 带宽 × 算术强度) / (1 + 同步开销占比)** 是这一讲的隐含公式。

2.3 Decomposition 与依赖：Amdahl 定律

- **定义与目的**：Decomposition 把"要解决的问题"切成 subproblem（task）。其**关键方面是识别依赖（或者依赖的缺失）**。只要存在依赖，两个 task 就不能同时执行；因此依赖链的长度（span，见第 4 节）直接给出并行度的上界。Amdahl 定律把这件事量化：设 S 为总工作量中**本质上串行**的比例，则并行加速比上限为 **1/S**。
- **直观解释（"它是什么？"）**：类比**做年夜饭**。你有 8 个灶眼（8 核），但"腌肉必须等 2 小时"这道工序无法被 8 个灶眼拆开。哪怕其他 100 道菜都能并行，总时间仍被那 2 小时钉死。S=0.05 意味着"不管你有多少核，最多快 20 倍"。
- **一个具体的两步算例（讲义原例）**：对一张 N×N 图像做两步处理——第 1 步把所有像素亮度翻倍（元素间完全独立），第 2 步求所有像素的平均值（对所有元素的归约）。
- 串行实现：两步各 ~N^2，总时间 ~$2N^2$，**并行度 = 1**。
- 第一次并行尝试：第 1 步并行（N^2/P），第 2 步仍串行（N^2）→ 总时间 $N^2/P + N^2$ → **加速比 ⩽ 2**（P→∞ 时也超不过 2）。这就是"小串行区域钉死大机器"的经典图示。
- 第二次改进：第 2 步先并行求部分和（N^2/P），再串行合并 P 个部分和（P）→ 总时间 $2N^2/P + P$，**当 N » P 时加速比 → P**。代价是"合并部分和"这一新增开销。



谁来负责 Decomposition? 大多数情况是程序员。自动分解串行程序仍是困难的研究问题：编译器必须先做依赖分析，而很多依赖是**数据相关**（编译期不可知）的；研究者只在简单 loop nest 上取得有限成功。

2.4 Assignment：静态（blocked / interleaved）与动态（任务队列）

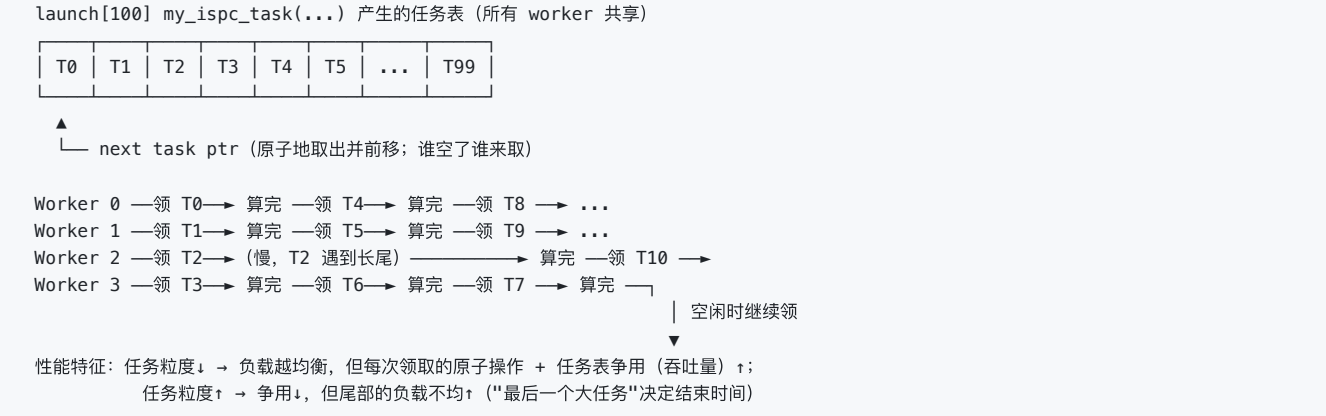
- **定义与目的**：把 task 分配给 thread（"worker"）。目标是**平衡负载**与**降低通信成本**。可以静态做（编译期/启动时确定），也可以在执行中动态做。讲义强调：**虽然分解通常由程序员负责，但很多语言/运行时接管了分配**。
- **直观解释（"它是什么？"）**：类比**发传单**。静态 blocked = 把街区切成 4 段，每人包一段；静态 interleaved = 每人隔三张发一张；动态 = 大家排在一个待发清单前，谁发完手里的就回来领下一批。街区每户门牌密度不同时（负载不均），只有动态方式能自动补齐。
- **ISPC 中的两种 Assignment（讲义原例；ISPC = Intel SPMD Program Compiler，SPMD 方言，编译器把 program instance 映射到 SIMD lane）**：
- **程序员管理的静态交错分配**：显式用 `programCount` 步长循环，把迭代交错地分给各 program instance（interleaved）。

```
ispc // 编译: ispc --target=avx2-i32x8 sinx.ispc -o sinx.o -O2 export void sinx(uniform int N, uniform int terms, uniform float* uniform x, uniform float* uniform result) { // 假设 N % programCount == 0 for (uniform int i = 0; i < N; i += programCount) { int idx = i + programIndex; // 每个实例处理一个 lane float value = x[idx]; float numer = x[idx] * x[idx] * x[idx]; uniform int denom = 6; // 3! uniform int sign = -1; for (uniform int j = 1; j <= terms; j++) { value += sign * numer / denom; numer *= x[idx] * x[idx]; denom *= (2*j + 2) * (2*j + 3); sign *= -1; } result[idx] = value; } }
```

– **系统管理分配的 `foreach`**：`foreach` 把独立的工作（迭代空间）交给系统，由系统决定"哪个迭代交给哪个 program instance"。抽象上允许动态分配，**当前 ISPC 实现是静态的（交错）**。

```
ispc export void sinx(uniform int N, uniform int terms, uniform float* uniform x, uniform float* uniform result) { foreach (i = 0 ... N) { // 系统接管 assignment float value = x[i]; float numer = x[i] * x[i] * x[i]; uniform int denom = 6; uniform int sign = -1; for (uniform int j = 1; j <= terms; j++) { value += sign * numer / denom; numer *= x[i] * x[i]; denom *= (2*j + 2) * (2*j + 3); sign *= -1; } result[i] = value; } }
```

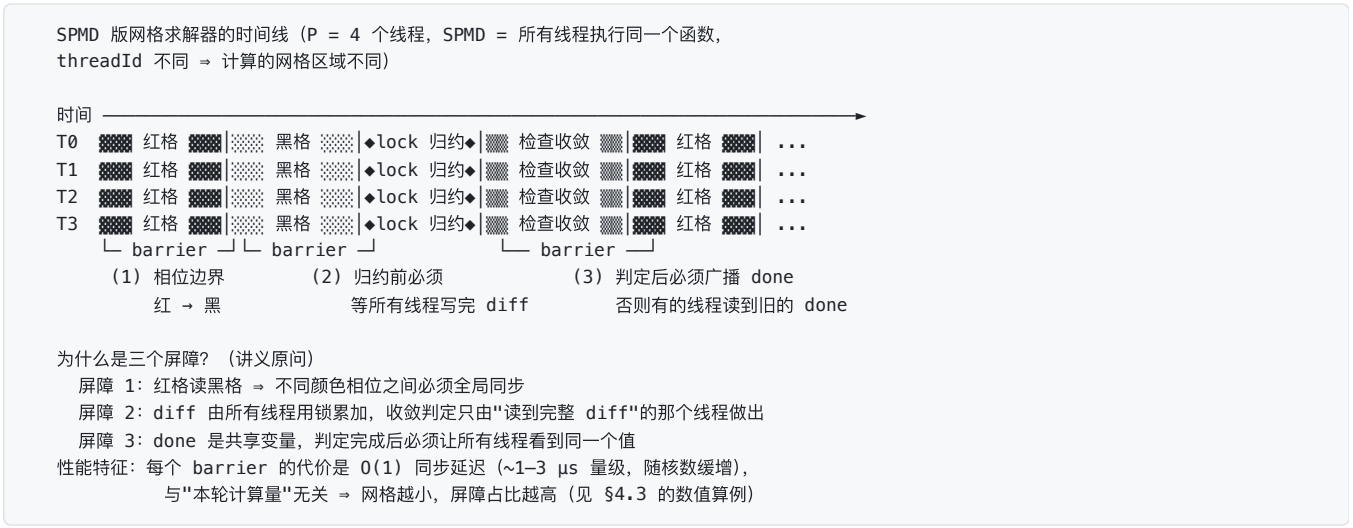
– **pthreads 的静态 blocked 分配（讲义原例）**：启动一个线程处理数组前半，主线程处理后半，`pthread_join` 收尾。这是"程序员管理、静态、blocked"。 – **ISPC 动态任务分配（讲义原例）**：`launch[100] my_ispc_task(...)` 产生 100 个任务构成**任务表**，ISPC 运行时把任务发给 worker 线程池；**分配策略：worker 完成当前任务后查看任务表，把下一个未完成的任务分给自己**（`next task ptr`）——这正是"动态分配"的原型。



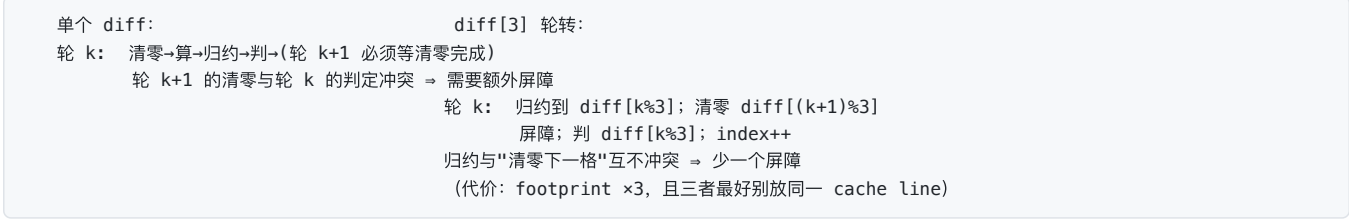
分配策略	负载均衡	通信量（以网格求解器 1D 划分为例）	每次分配开销	适用场景
静态 blocked（连续块）	一般：任务成本不均时差	**最小**：只有块边界需要交换（ghost row），~2 行/迭代	0（启动时算好）	成本均匀、通信昂贵、kernel 开销敏感（GPU/向量）
静态 interleaved（交错）	好：成本随索引缓变时很好	**最大**：每个"接缝"都要数据	0	成本均匀且与索引强相关（如三角形遍历）
动态任务队列（next ptr / 窃取）	**最好**：自动吸收长尾	取决于划分：块粒度大则通信量接近 blocked	每次领取一次原子操作（~10–100 ns，争用时更高）	成本不可预测（Barnes–Hut、光线追踪、不规则图）

2.5 Orchestration：锁、屏障与"用数据换依赖"

- **定义与目的**：Orchestration 负责**组织通信、加入同步以保持依赖、组织内存中的数据结构、调度任务**。目标是降低通信/同步成本、保持数据访问的局部性、降低开销。讲义反复强调一句话：**机器细节会影响这些决策——如果同步很贵，就应该用得更稀疏**。
- **直观解释（"它是什么？"）**：类比**十字路口的交通**。锁 = 单车道桥梁的红绿灯（一次只放一辆车）；屏障 = 整队集合点（所有人到齐才继续）；而"用数据换依赖" = 与其在集合点等所有人，不如给每个人一份**自己的计数器**，大家各记各的，最后再汇总。
- **案例：共享地址空间版网格求解器（SPMD，程序员负责同步）**。依赖是"红格更新完才能更新黑格"，所以每个颜色相位之间必须有屏障：



- **互斥为什么必要（讲义的核心小例子）**：`diff += myDiff` 在机器上展开为三条指令 `r1 ← diff` / `r1 ← r1 + r2` / `diff ← r1`。若 T0 与 T1 交错执行：两者都读到 0，各自算出 1，各自写回 1 → 一次加法丢失（丢失更新）。因此这一组指令必须**原子（atomic）**。
- **保持原子性的三类机制**：
- 临界区**加锁/解锁**：`LOCK(myLock); /* critical section */ UNLOCK(myLock);`
- **硬件支持的原子读–改–写指令**（intrinsics，如 `fetch_add`、`atomicAdd`、`atomicCAS`）；
- **语言级原子块**：`atomic { ... }`、`atomicAdd(x, 10)`。
- **性能陷阱与修复（讲义的原例）**：把锁放在内层 `(i,j)` 循环里 → 每更新一个格子就加一次锁（还很可能造成 cache line 在核间反复弹跳）。修复：**每个 worker 先在本地累加部分和 `myDiff`，每轮只加一次锁**把部分和并入全局 `diff`。
- **用数据换依赖：把 3 个屏障减到 1 个（讲义 slide 50 的技巧）**：把单一的 `diff` 换成 `diff[3]`，用 `index = (index+1) % 3` 轮转——**相邻两次迭代使用不同的累加变量，从而消除"必须先把 diff 清零"的那条依赖**。因为某线程清零 `diff[(index+1)%3]` 时，别的线程正在读的是 `diff[index]`，两者模 3 不同余，所以**3 个副本是最小值**（2 个不够：将会清掉别人正在读的缓冲区）。这就是"**trade off footprint for removing dependencies**"这一常见并行编程手法的实例。



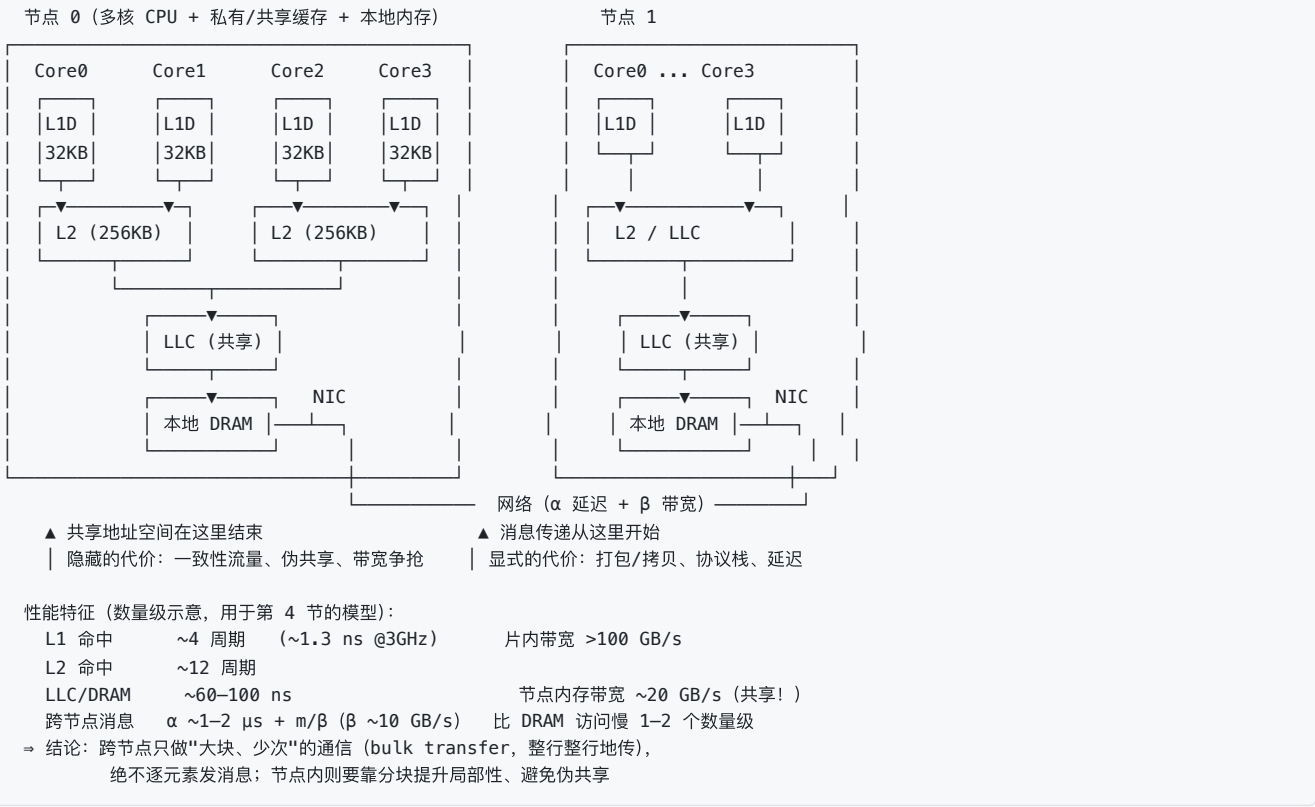
2.6 Barrier 是"保守"的：更细粒度的依赖表达

- **定义与目的**：`barrier(num_threads)` 把所有线程切成**相位（phase）**：所有线程在屏障之前的计算全部完成后，任何线程在屏障之后的计算才能开始。它**简单但保守**——粒度太粗：本可以只等一个生产者，却等所有人。
- **直观解释（"它是什么？"）**：类比**电梯 vs 传呼机**。屏障 = 所有人都要在一楼集合齐全才上楼（哪怕你只需要等一个人）；flag + 自旋 = 传呼机（对方一按你就收到，其余人照常工作）。
- **更细粒度的依赖（讲义原例）**：两个线程，一个生产结果 `x`，一个消费它。T0 写 `x = 1; flag = 1;` 然后继续做与 `x` 无关的工作；T1 自旋 `while (flag == 0);` 然后 `print x;`。**这实际上就是实现了一个长度为 1 的消息队列**——说明消息传递与共享地址空间在表达能力上等价，差别只在"谁负责维护这个结构"。
- **同步原语对比表**：

原语	语义	典型延迟量级（8 核节点，示意）	优点	风险
锁（uncontended lock/unlock）	互斥进入临界区	~20–100 ns	表达力强、易理解	争用时串行化；循环内加锁会毁掉性能
原子 RMW（`fetch_add` / `atomicAdd`）	单次读–改–写不可分割	~10–50 ns（无争用）；争用同一地址时线性变差	无需临界区、可组合	同一地址的高度争用（全局计数器）成为瓶颈
屏障 barrier	相位边界，全员到齐	~1–3 μs（随核数、NUMA 缓增）	简单、保守但正确	频繁小相位下占比爆炸（见 §4.3）
标志位 + 自旋（flag）	一对一生产者–消费者	一次 cache line 传输（~100 ns）+ 轮询	细粒度、无全局同步	必须配内存序（acquire/release）否则编译器/CPU 重排可致错
消息 send/recv	通过消息通信与同步	α ~1–2 μs + m/β	跨节点唯一选择；结构清晰	阻塞语义易死锁；小消息受延迟支配

2.7 Mapping：谁把 worker 放到硬件上?

- **定义与目的**：Mapping 把逻辑 worker 映射到物理执行单元。讲师给的三条路径：**OS**（pthread → 某核上的硬件执行上下文）、**编译器**（ISPC program instance → 向量指令 lane）、**硬件**（CUDA thread block → GPU 的 SM）。
- **直观解释（"它是什么？"）**：类比**会议室安排**。同一个项目的同事（相关线程）安排在同一间会议室（同一核/同一节点）→ 资料共享方便（缓存命中）；不相干的人安排在同一间（一个带宽受限、一个计算受限）→ 房间利用率最高。
- **架构/机制图解（节点级硬件结构——共享地址空间与消息传递在同一台机器上的分界线）**：



2.8 案例研究：二维网格求解器（Gauss–Seidel → 对角波前 → 红黑重排）

- **问题定义**：在 (N+2)×(N+2) 的网格上解偏微分方程（PDE），用**迭代法**：不断做 Gauss–Seidel sweep 直到收敛。更新式为 `A[i,j] = 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] + A[i,j+1] + A[i+1,j])`。
- **第一步：找依赖**。原位（in-place）Gauss–Seidel 中，每个格子读**左邻与上邻的新值**，写自己的值（再被右邻与下邻读）。所以：
- 沿一行有链式依赖（左 → 右）；沿列也有依赖（上 → 下）；
- **但同一条反对角线 k = i+j 上的格子互不依赖** → 存在**对角波前（wavefront）并行**；
- 坏消息：计算刚开始和快结束时并行度很小（波前长度从 1 涨到 N 再降回 1），而且**每完成一条对角线就要同步一次**（一次 sweep 需要 2N–1 次同步）。
- **第二步：为了并行而改变算法——红黑着色（red–black coloring）**。改变网格单元的更新顺序：先并行更新所有红格，再并行更新所有黑格（黑格依赖红格的新值），重复直到收敛。**注意这是允许的，因为它需要领域知识**：新算法以不同的方式收敛到同一个解；浮点计算的中间值不同，但最终仍收敛到误差阈值之内。
- **架构/机制图解**：

```
(a) 原位 Gauss-Seidel 的依赖（一次 sweep 之内）

            ┌──────────┐
            │ (i-1, j) │   "每个格子依赖左邻与上邻的新值"
            └──────────┘
                 ▼
(i, j-1) ──────► (i, j) ──────► (i, j+1)
                 ▲
            ┌──────────┐
            │ (i+1, j) │
            └──────────┘

(b) 只有反对角线独立 ⇒ 波前并行（波前编号 k = i+j，同一 k 的格子可并行）

┌───┬───┬───┬───┬───┬───┐      波前总数 = 2N-1
│ 2 │ 3 │ 4 │ 5 │ 6 │   │      每推进一条波前同步一次  ⇒ Span(sweep) = θ(N)
├───┼───┼───┼───┼───┼───┤      波前长度 ~ min(k, N) ⇒ 并行度 ≈ N/2
│ 3 │ 4 │ 5 │ 6 │ 7 │   │      两端"三角形"并行度低、同步频繁
├───┼───┼───┼───┼───┼───┤
│ 4 │ 5 │ 6 │ 7 │ 8 │   │
└───┴───┴───┴───┴───┴───┘

(c) 红黑重排：把依赖"降维"成两个无依赖的相位

┌───┬───┬───┬───┬───┬───┐      红格 (i+j 偶) 只读黑格 ⇒ 全体红格可同时更新
│ R │ B │ R │ B │ R │   │      黑格 (i+j 奇) 只读红格 ⇒ 全体黑格可同时更新
├───┼───┼───┼───┼───┼───┤
│ B │ R │ B │ R │ B │   │      依赖：  红相位 ──barrier──► 黑相位 ──barrier──► 红相位 ...
├───┼───┼───┼───┼───┼───┤                ▲                                    │
│ R │ B │ R │ B │ R │   │                └────────────────────────────────────┘
└───┴───┴───┴───┴───┴───┘

  Span(一个相位) = θ(1)（相位内零依赖）；一次 sweep 只需 2 次相位同步
  并行度 = θ(N²) ── 用"多迭代几次"换"每个相位内近乎无限的并行度"

(d) 数据并行表达（交给系统编排）
  for_all (red cells (i,j)) {                    // 分解：单个网格元素 = 独立工作
      prev = A[i,j];                             // 分配：??（系统决定）
      A[i,j] = 0.2f*(A[i-1,j]+A[i,j-1]+A[i,j]+A[i+1,j]+A[i,j+1]);
      reduceAdd(diff, abs(A[i,j]-prev));         // 编排：系统提供的内置通信原语
  }                                              // for_all 末尾 = 隐式屏障（等所有 worker）
```

2.9 消息传递：私有地址空间、Ghost Cell、阻塞与非阻塞

- **定义与目的**：消息传递模型下**每个线程有自己的地址空间**——没有共享变量，线程通过发送/接收消息来通信与同步。它把"数据所有权"变成显式概念：网格数据被切成 4 份，分别住在 4 个地址空间里（4 个私有数组）。
- **直观解释（"它是什么？"）**：类比**四个各自记账的仓库**。仓库 2 要算自己的边界，就必须知道邻居仓库 1 和 3 的最后一排货物——于是邻居把那一排**抄一份送过来**贴在边界上，这一份**复制**就叫 ghost cell（影子/虚拟单元），其"所有权"仍在邻居手里。
- **数据复制是正确性的必要条件**：因为不再有共享内存，边界数据必须在本地留一份拷贝（ghost cells）。每轮红格处理完后，线程 1 与线程 3 各发一行数据给线程 2（线程 2 下一相位需要最新的红格信息）。**批量传输**是准则：一次发**整行**，而不是逐元素发。
- **架构/机制图解（rank 的本地数组与 ghost row）**：

```
      rank r-1                rank r                 rank r+1
┌──────────────┐        ┌──────────────┐        ┌──────────────┐
│ row rows     │──────► │ row 0 (ghost)│        │              │
│  (属于 r-1)  │  send  │ row 1        │        │              │
│   ...        │        │   ...        │        │              │
│ row 1        │        │ row rows     │──────► │ row 0 (ghost)│
└──────────────┘        │ row rows+1   │◄────── │ row 1        │
                        │  (ghost)     │  send  └──────────────┘
                        └──────────────┘

  代码形态：
    float* localA = allocate(rows_per_thread + 2, N + 2);
    recv(&localA[0,0],            sizeof(float)*(N+2), tid-1, MSG_ID_ROW);  // 上 ghost
    recv(&localA[rows_per_thread+1,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW); // 下 ghost
  性能特征：每个 rank 每轮通信量 = 2 行 × (N+2) × 4 B,
            而计算量 = (N/P) × N × k FLOP ⇒ 表面/体积比 ∝ P/N
            ⇒ 强扩展（固定 N）时 P 增大到通信与计算相当时就撞墙；
              弱扩展（N ∝ P，每 rank 行数固定）能保持效率
```

- **阻塞（synchronous）send/recv 的语义与死锁**：

- `send()` 在**发送方收到"数据已驻留在接收方地址空间"的确认**后才返回；`recv()` 在**数据被拷入接收方缓冲区并回送 ack** 后返回。
- 于是"所有线程同时 `sendDown`"会形成**循环等待**：每个线程都阻塞在自己的 send 上，没有人执行 recv → **死锁**。
- **修复（讲义原方案）**：让相邻两条线程的 (send, recv) 顺序相反——**偶数号线程先 send 后 recv，奇数号线程先 recv 后 send**。这样每一对相邻线程中总有一方率先发出消息，环上的等待被打破。
- **非阻塞（asynchronous）send/recv**：`send()` 立即返回（返回句柄 h1），但**调用线程在 `checksend(h1)` 确认发送完成前不得修改发送缓冲区**；`recv()` 只是"登记接收意图"并立即返回（句柄 h2），用 `checkrecv(h2)` 查询是否真的收完。好处：**通信与应用线程执行重叠**（消息处理与线程执行并发进行），线程可以在等消息时做别的工作。
- 消息传递库通常还在 send/recv 之上提供**高层原语**：`reduce_add(0, &my_diff, sizeof(float))`（把所有 my_diff 加到 0 号线程）、`broadcast(0, &done, sizeof(bool), MSG_DONE)`（0 号线程把 done 广播给所有人）。

```
(a) 阻塞 send/recv + 全员同时 send：循环等待 → 死锁（hang）
  T0 ──send↓──►[等 T5 recv]          ┌──────────────────────────────────────┐
  T1 ──send↓──►[等 T0 recv]          │ send() 语义：数据进入对方地址空间才返回 │
  T2 ──send↓──►[等 T1 recv]          │ 每个线程都在等对方先 recv             │
  ...                                │ 等待环无人打破 → 程序永久挂起（有时因 │
  T5 ──send↓──►[等 T4 recv]          │ 消息小而被 library 缓冲，于是"偶尔能跑"）│
                                     └──────────────────────────────────────┘

(b) 奇偶序修正（仍用阻塞 send/recv）
  偶数号 T0:  sendDown ─► recvDown ─► sendUp ─► recvUp
  奇数号 T1:  recvUp   ─► sendUp   ─► recvDown ─► sendDown
       ▲ 每一对相邻线程（偶，奇）中：偶数先发、奇数先收
         ⇒ 每对里必有一方的 send 能被对方的 recv 匹配 ⇒ 无环 ⇒ 无死锁

(c) 非阻塞（asynchronous）版本：把"等待"从时间轴上抹掉
  T0  Isend/send(h1) ─┐
  T0  Irecv/recv(h2) ─┤ 立即返回
  T0  计算不依赖 ghost 的内部行 ◄──── 通信与计算重叠（重叠期可能数百 μs）
  T0  Wait/checkrecv(h2) ──► 计算需要 ghost 的边界行
      ▲ 前提：在 checksend(h1) 之前绝不能改写发送缓冲区
```

维度	阻塞（synchronous）send/recv	非阻塞（asynchronous）send/recv
返回时机	send：数据已到对方地址空间；recv：数据已拷入本地	立即返回句柄
缓冲区约束	send 返回后即可重用缓冲区	**checksend 之前不得修改发送缓冲**；checkrecv 之前不得读接收缓冲
死锁风险	**有**（需靠奇偶序/buffered send 打破）	低（但 `Waitall` 顺序不当仍可造成等待）
与计算重叠	无	**有**（可在等待期间计算不依赖 ghost 的内部区域）
实现代价	低	句柄管理、状态查询、额外的编译器/程序员约束

---

3. 代码示例与性能分析

3.1 示例一：pthreads 静态块分配 vs 动态任务队列（`sinx` 的 Taylor 展开）

```cpp
// 编译: g++ -O3 -march=native -pthread -std=c++17 sinx_pthreads.cpp -o sinx_pthreads
// 运行: ./sinx_pthreads 67108864 5 8 4096      # N=2^26, terms=5, 8 线程, 动态块 4096
#include <pthread.h>
#include <atomic>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <cmath>

static int              g_N;              // 元素个数
static int              g_terms;          // Taylor 项数 (<=5, 避免 int denom 溢出)
static const float*     g_x;
static float*           g_result;
static std::atomic<int> g_next(0);        // 动态分配用的"next task ptr"
static int              g_chunk = 4096;   // 动态分配的块粒度

static double now_sec() {
    using namespace std::chrono;
    return duration<double>(steady_clock::now().time_since_epoch()).count();
}

// 串行内核: 对 [begin,end) 的每个元素做 sin(x) 的 Taylor 展开
//   sin(x) = x - x^3/3! + x^5/5! - x^7/7! ...
static void sinx_range(int begin, int end, int terms, const float* x, float* result) {
    for (int i = begin; i < end; i++) {
        float value = x[i];                     // 第 1 项
        float numer = x[i] * x[i] * x[i];       // x^3
        int   denom = 6;                        // 3!
        int   sign  = -1;
        for (int j = 1; j <= terms; j++) {
            value += (float)sign * numer / (float)denom;
            numer *= x[i] * x[i];               // 升幂: x^3 -> x^5 -> x^7
            denom *= (2*j + 2) * (2*j + 3);     // 3! -> 5! -> 7! ...
            sign  *= -1;
        }
        result[i] = value;
    }
}

// ---------- 静态 blocked 分配: 线程 t 负责连续的一块 ----------
struct BlockArgs { int tid; int P; };

static void* blocked_worker(void* arg) {
    BlockArgs* a = (BlockArgs*)arg;
    long long  N = g_N;
    int begin = (int)(N * a->tid       / a->P);
    int end   = (int)(N * (a->tid + 1) / a->P);
    sinx_range(begin, end, g_terms, g_x, g_result);
    return nullptr;
}

// ---------- 动态分配: 谁空了谁从共享计数器领一块 ----------
static void* dynamic_worker(void*) {
    for (;;) {
        int begin = g_next.fetch_add(g_chunk, std::memory_order_relaxed);
        if (begin >= g_N) break;                // 领完了
        int end = begin + g_chunk;
        if (end > g_N) end = g_N;
        sinx_range(begin, end, g_terms, g_x, g_result);
    }
    return nullptr;
}

int main(int argc, char** argv) {
    g_N     = (argc > 1) ? atoi(argv[1]) : (1 << 26);
    g_terms = (argc > 2) ? atoi(argv[2]) : 5;
    int P   = (argc > 3) ? atoi(argv[3]) : 8;
    if (argc > 4) g_chunk = atoi(argv[4]);
    if (P < 1 || P > 64) { fprintf(stderr, "P 需在 1..64 之间\n"); return 1; }

    float* x      = (float*)malloc(sizeof(float) * (size_t)g_N);
```

```
    float* result = (float*)malloc(sizeof(float) * (size_t)g_N);
    for (int i = 0; i < g_N; i++)
        x[i] = -0.75f + 1.5f * (float)(i % 1000) / 999.0f;   // |x| <= 0.75, Taylor 收敛
    g_x = x; g_result = result;

    // ---- 串行基线 ----
    double t0 = now_sec();
    sinx_range(0, g_N, g_terms, x, result);
    double t_seq = now_sec() - t0;

    pthread_t th[64];
    BlockArgs args[64];

    // ---- 并行 A: 静态 blocked ----
    t0 = now_sec();
    for (int t = 1; t < P; t++) { args[t] = BlockArgs{t, P};
        pthread_create(&th[t], nullptr, blocked_worker, &args[t]); }
    args[0] = BlockArgs{0, P};
    blocked_worker(&args[0]);                    // 主线程也当 worker（避免 P+1 路并行）
    for (int t = 1; t < P; t++) pthread_join(th[t], nullptr);
    double t_blk = now_sec() - t0;

    // ---- 并行 B: 动态（next task ptr + atomic fetch_add） ----
    g_next.store(0);
    t0 = now_sec();
    for (int t = 1; t < P; t++) pthread_create(&th[t], nullptr, dynamic_worker, nullptr);
    dynamic_worker(nullptr);
    for (int t = 1; t < P; t++) pthread_join(th[t], nullptr);
    double t_dyn = now_sec() - t0;

    // ---- 正确性检查 ----
    double maxerr = 0.0;
    for (int i = 0; i < g_N; i++)
        maxerr = fmax(maxerr, fabs((double)result[i] - sin((double)x[i])));

    const double bytes = 8.0 * g_N;      // 每元素读 4B + 写 4B
    const double flops = 23.0 * g_N;     // 每元素约 23 次浮点运算（见下文分析）
    printf("N=%d terms=%d P=%d chunk=%d\n", g_N, g_terms, P, g_chunk);
    printf("serial  : %8.2f ms   %7.2f GFLOP/s  %7.2f GB/s\n",
           t_seq*1e3, flops/t_seq/1e9, bytes/t_seq/1e9);
    printf("blocked : %8.2f ms   %7.2f GFLOP/s  %7.2f GB/s   speedup = %5.2fx\n",
           t_blk*1e3, flops/t_blk/1e9, bytes/t_blk/1e9, t_seq/t_blk);
    printf("dynamic : %8.2f ms   %7.2f GFLOP/s  %7.2f GB/s   speedup = %5.2fx\n",
           t_dyn*1e3, flops/t_dyn/1e9, bytes/t_dyn/1e9, t_seq/t_dyn);
    printf("max |approx - sinf| = %.3e\n", maxerr);
    free(x); free(result);
    return 0;
}
```

- **【代码做什么？】**
- 初始化 `x[i] ∈ [-0.75, 0.75]`（Taylor 展开在小 |x| 上收敛），`result` 作为输出。
- 串行跑一遍得到基线时间。
- **静态 blocked**：把 `[0, N)` 按 `N*t/P` 切成 P 个连续块，线程 `t` 拿第 `t` 块（主线程也干活，避免只开 P-1 个真实 worker）。
- **动态**：所有线程循环执行 `g_next.fetch_add(g_chunk)`，原子地领取下一块（块大小 4096 个元素），直到领到的起点越过数组末尾。这就是 ISPC `launch[N]` 任务表 + next task ptr 的手写版本。
- 与 `sinf()` 比较最大误差做正确性验证。
- **【并行机制与性能解说】**
- **Work / Span / 并行度（本示例的核心分析）**：
  - 记每个元素的内层递推（Taylor 迭代）代价为 `c(T)`（T = terms），则 **Work（总工作量）** $W = N \cdot (c_0 + c(T)) = \Theta(N \cdot T)$。
  - **Span（关键路径）**：不同元素之间**完全没有依赖**（`result[i]` 只依赖 `x[i]`），所以并行性全部来自"N 个独立的串行链"；每个链的长度就是内层 `for j` 递推的长度 → **Span = Θ(T)**（与 N 无关）。
  - **并行度 = W / Span = Θ(N)**。取 $N = 2^{26}$、T = 5：并行度 ≈ **6700 万**，远远超过任何机器的线程数 → **并行度不是限制，内存带宽和除法吞吐量才是**。

- 对比：若把内层递推误写成跨元素依赖（例如累加器写在共享变量里），Span 立刻变成 Θ(N·T)，并行度掉到 1——**这就是 Decomposition 阶段"识别依赖"的意义**。

- **硬件上如何并行执行**：`pthread_create` 由 OS 把线程映射到不同 CPU 核（Mapping），每个线程执行 `sinx_range` 的各自区间；`x[i]` 与 `result[i]` 是私有工作集（不同线程写不同下标）→ **无共享写、无需同步、也无伪共享**（除非 P 很大且块边界落在同一 cache line —— 4 字节元素、4096 元素/块，可以忽略）。
- **瓶颈分析（Roofline 视角）**：每元素搬 8 B（读 4 B + 写 4 B），做浮点运算的数量级为：内层每轮 ≈ 5 个浮点操作（`numer` 两次乘、`x*x` 一次乘、一次除、一次加）→ 5 轮 ≈ 25 次，加上收尾的几次，代码里按 **23 FLOP/元素** 估算 → **算术强度 AI ≈ 23/8 ≈ 2.9 FLOP/byte**。而单核的 machine balance = 48 GFLOP/s ÷ 2.5 GB/s = **19.2 FLOP/byte** → **AI < balance ⇒ 每核也是带宽受限的**：单核可期望 ≈ 2.5 GB/s × 2.9 ≈ 7.2 GFLOP/s，只有峰值的 15%。N = 2^{26} 时总内存流量 = 8 B × 67.1 M = **537 MB**，全节点带宽 20 GB/s 下界 ≈ **26.8 ms**。
- **额外杀手：浮点除法**。`numer / denom` 在 x86 上是 ~5–10 倍于 FMA 的高延迟/低吞吐操作；每元素 5 次除法意味着 5 × 67.1 M = 3.35 亿次除法，仅除法吞吐就可能吃掉数百毫秒。**优化方向**：预计算 `1.0f/denom` 或使用 `numer * inv_denom`，把除法变成乘法；也可用快速多项式近似（这正是"性能优化"一讲的主题）。
- **静态 blocked 与动态的差异**：本内核每个元素成本**完全均匀**，因此两者应几乎相同（动态多出 `fetch_add` 的原子开销：2^{26}/4096 = 16384 次领取，可忽略）。把 `g_chunk` 调成 1 会看到动态版本变慢（原子争用 + 每块启动开销）；把元素成本改成不均匀（例如 `terms` 随 i 变化，模拟 Barnes–Hut 中"有的粒子周围邻居多、有的少"），静态 blocked 就会出现**尾部不均**，动态才扳回一局。
- **可扩展性上限**：由带宽给出，而非并行度。8 核全部投入时若每核分到 2.5 GB/s，则时间下界 26.8 ms；串行版本（单核带宽受限约 10 GB/s，且除法吞吐被单核独占）约 60–200 ms 量级 → 实测加速比会明显低于 8×，原因**不是并行度不足，而是共享的内存带宽已饱和**。

3.2 示例二：OpenMP 红黑网格求解器（屏障、部分和、收敛判定）

```
// 编译: g++ -O3 -march=native -fopenmp redblack_omp.cpp -o redblack_omp
// 运行: OMP_NUM_THREADS=8 ./redblack_omp 1024 200     # N=1024, 200 次 sweep
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cstring>

static int     N      = 1024;    // 内部网格 N x N（外圈一圈是固定边界）
static long    MAXIT  = 200;     // 最大 sweep 次数（默认跑满，便于测量稳态时间）
static double  TOL    = 1e-12;   // 收敛阈值：g_diff/(N*N) < TOL
static float*  A      = nullptr; // 工作网格 (N+2) x (N+2)
static float*  REF    = nullptr; // 串行参考网格
static double  g_diff = 0.0;     // 全局残差累加器（由各线程局部和归并）
static int     g_done = 0;       // 收敛标志（在 single 中写，在 barrier 后广播）
static int     g_iter = 0;       // 已完成的 sweep 数

#define IDX(i, j) ((i) * (N + 2) + (j))

// 串行参考：同一算法（红黑相位顺序）跑 iters 次，用于校验与测串行基线
static void run_ref(int iters) {
    memcpy(REF, A, sizeof(float) * (size_t)(N + 2) * (N + 2));
    for (int it = 0; it < iters; it++) {
        for (int color = 0; color < 2; color++) {
            for (int i = 1; i <= N; i++) {
                int jstart = ((i + color) & 1) ? 1 : 2;   // 使 (i+j)%2 == color
                for (int j = jstart; j <= N; j += 2) {
                    int p = IDX(i, j);
                    float prev = REF[p];
                    float v = 0.2f * (REF[p] + REF[IDX(i-1,j)] + REF[IDX(i+1,j)]
                                    + REF[IDX(i,j-1)] + REF[IDX(i,j+1)]);
                    REF[p] = v;
                    (void)prev;
                }
            }
        }
    }
}

int main(int argc, char** argv) {
    if (argc > 1) N     = atoi(argv[1]);
    if (argc > 2) MAXIT = atol(argv[2]);

    const size_t n2 = (size_t)(N + 2) * (N + 2);
    A   = (float*)aligned_alloc(64, sizeof(float) * n2);
    REF = (float*)aligned_alloc(64, sizeof(float) * n2);
    for (int i = 0; i < N + 2; i++)
        for (int j = 0; j < N + 2; j++)
            A[IDX(i,j)] = (i == 0 || j == 0 || i == N + 1 || j == N + 1) ? 1.0f : 0.0f;

    // ================= 并行版本：SPMD + 屏障 =================
    double t0 = omp_get_wtime();
    #pragma omp parallel
    {
        while (!g_done) {
            for (int color = 0; color < 2; color++) {          // 0 = 红, 1 = 黑
                float local = 0.0f;                            // 线程私有的部分和
                #pragma omp for schedule(static) nowait
                for (int i = 1; i <= N; i++) {
                    int jstart = ((i + color) & 1) ? 1 : 2;    // 只遍历该色格子
                    for (int j = jstart; j <= N; j += 2) {
                        int p = IDX(i, j);
                        float prev = A[p];
                        float v = 0.2f * (A[p] + A[IDX(i-1,j)] + A[IDX(i+1,j)]
                                        + A[IDX(i,j-1)] + A[IDX(i,j+1)]);
                        A[p] = v;
                        local += fabsf(v - prev);              // 局部累加，避免每格加锁
                    }
                }
                #pragma omp atomic
                g_diff += (double)local;                       // 每线程每相位只归并一次
```

```
            #pragma omp barrier                                 // 屏障 1/2：相位边界（红<->黑）
        }
        #pragma omp barrier                                     // 屏障 3：等所有局部和归并完毕
        #pragma omp single
        {                                                       // 单线程做收敛判定 + 清账
            g_iter++;
            if (g_diff / ((double)N * (double)N) < TOL || g_iter >= MAXIT)
                g_done = 1;
            g_diff = 0.0;
        }                                                       // single 末尾隐式屏障：广播 g_done
    }
}
double t_par = omp_get_wtime() - t0;

// ================ 串行基线（同算法、同迭代数） ================
double t1 = omp_get_wtime();
run_ref(g_iter);
double t_ref = omp_get_wtime() - t1;

// ================ 正确性校验：并行结果应逐位等于串行参考 ================
float maxdiff = 0.0f;
for (size_t k = 0; k < n2; k++) {
    float d = fabsf(A[k] - REF[k]);
    if (d > maxdiff) maxdiff = d;
}

const double cells = (double)N * (double)N;
const double bytes = 24.0 * cells;        // 每格 5 次读 + 1 次写 = 24 B
const double flops = 5.0  * cells;        // 4 次加法 + 1 次乘法
printf("N=%d  sweeps=%d  threads=%d\n", N, g_iter, omp_get_max_threads());
printf("parallel: %8.2f ms  (%6.3f ms/sweep)  %6.2f GB/s  %6.2f GFLOP/s\n",
       t_par*1e3, t_par*1e3/g_iter, bytes*g_iter/t_par/1e9, flops*g_iter/t_par/1e9);
printf("serial  : %8.2f ms  (%6.3f ms/sweep)  speedup = %5.2fx\n",
       t_ref*1e3, t_ref*1e3/g_iter, t_ref/t_par);
printf("max |parallel - serial| = %.3e  (红黑相位内部无依赖，期望为 0)\n", maxdiff);
free(A); free(REF);
return 0;
}
```

- **【代码做什么？】**
- 分配 (N+2)×(N+2) 网格：外圈固定为 1.0，内部初值 0.0（这就是 Laplace 方程的边界条件，收敛解是内部全 1）。
- **并行区在 while 循环之外**（`#pragma omp parallel` 只进入一次），线程在整个求解过程中常驻——避免每轮 fork/join 的巨大开销，这是"orchestration 要降低开销"的直接体现。
- 每轮分两个**相位**：`color = 0` 更新红格、`color = 1` 更新黑格。`jstart = ((i+color)&1) ? 1 : 2` 保证只遍历 `(i+j)%2 == color` 的格子，步长 2 沿行进。
- 每相位内每个线程**本地累加**残差 `local`，相位结束后用一次 `#pragma omp atomic` 并入 `g_diff`——而不是每更新一个格子就加一次锁（讲义 slide 46→47 的改进）。
- **三个屏障**：相位边界、归约完成、`done` 广播（`single` 末尾自带隐式屏障，因此第三个屏障由 `single` 提供）。
- 收敛判定与 `g_diff` 清零放在 `single` 里（只有一个线程执行）；`run_ref` 用完全相同的更新表达式跑相同的 sweep 次数做**逐位校验**（红黑相位内无依赖，因此与串行结果应当完全一致）。
- **【并行机制与性能解说】**
- **Work / Span / 并行度**：
  - 一次 sweep 的 **Work = N^2 次单元更新**（红黑各约 $N^2/2$），记单格代价 c（含 5 次浮点与 24 B 访存）→ **$W_{sweep} = c \cdot N^2$**。
  - **Span**：红相位内部**零依赖**（红格只读黑格），span = c；黑相位同理 span = c；加上 3 次屏障，记屏障代价 B → **$Span_{sweep} = 2c + 3B$**。
  - **并行度 = $W/Span = c \cdot N^2/(2c + 3B)$**。代入 N = 1024、c ≈ 24 B ÷ 20 GB/s = **1.2 ns**、B ≈ **1.5 μs**：并行度 = (1.05 M × 1.2 ns)/(2.4 ns + 4.5 μs) ≈ **281**。**这才是真正的可扩展性上限**——不是 N^2（≈ 10^6），而是被屏障开销压到几百。要支持 1024 核，必须让 $c \cdot N^2 \gg 3B$，即网格规模要再大 3–4 个数量级或每格计算量大得多。
  - 对照讲义原版的 **Gauss–Seidel 原位**版本：一次 sweep 的 span = Θ(N)（沿行与沿列的两条长依赖链）→ 并行度只有 Θ(N/常数) ≈ 几百，且两端并行度低；**红黑重排用"多做几次迭代"换来了 $\Theta(N^2)$ 的相位内并行度**，这是"改算法换并行度"的典型交易。

- **硬件上如何执行**：`#pragma omp for schedule(static)` 把行区间静态分给线程（blocked 划分，`nowait` 去掉隐式屏障，因为后面紧跟一个显式 barrier）；每个线程在自己的行块内按行扫描，**行内访问连续** → 硬件预取器有效、SIMD 友好（gcc –O3 会把内层 `j += 2` 循环向量化）。相邻线程在行边界处共享边界行：这是**真共享**（只读），命中率高，不同于伪共享。
- **瓶颈分析**：
  - **内存带宽**：每格 24 B，N = 1024 时一次 sweep = 25.2 MB，20 GB/s 下界 **1.26 ms/sweep**；200 sweeps ≈ **252 ms**（计算只需 200 × 5.24 MFLOP = 1.05 GFLOP，8 核 384 GFLOP/s 下仅 2.7 ms）→ 该程序是**纯带宽受限**的（Roofline 见 §4.4）。
  - **屏障开销**：3 × 1.5 μs = 4.5 μs/sweep，相对于 1.26 ms/sweep 仅 **0.36%**——N 大时同步可以忽略。但把 N 换成 128：一次 sweep 只有 393 KB → 19.7 μs，同步占总时间的 **19%**（相对纯计算时间则是 23%），且并行度降到 $c \cdot N^2/(2c+3B)$ = 19.7 μs/4.5 μs ≈ **4.4 → 8 核都喂不饱**。
  - **原子/锁**：每线程每相位一次 atomic（≈ 2P 次/sweep），相对 1.26 ms 可忽略；但若把 atomic 放进内层循环（讲义 slide 46 的错误版本），就是 10^6 次/扫并伴随 cache line 争抢，性能会坍塌 1–2 个数量级。
- **进一步优化**：① 用 `diff[3]` 轮转把 3 个屏障降为 1 个（讲义 slide 50）；② 用 2D 分块 + 缓存分块（tiling）把 5 点模板的 24 B/格降到接近 8 B/格（cache 命中）；③ 用 `#pragma omp for schedule(static,1)` 或 dynamic 缓解不规则 workload；④ 把收敛检查降频（每 k 轮检查一次），进一步减少屏障。

3.3 示例三：MPI 非阻塞 Ghost 行交换的网格求解器

```
// 编译: mpicxx -O3 -march=native mpi_solver.cpp -o mpi_solver
// 运行: mpirun -np 4 ./mpi_solver 1024 200      # 内部网格 1024x1024, 200 次迭代
#include <mpi.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    const int   n       = (argc > 1) ? atoi(argv[1]) : 1024;  // 内部网格 n x n
    const int   maxIter = (argc > 2) ? atoi(argv[2]) : 200;
    const float TOL     = 1e-9f;
    if (n % nprocs != 0) {
        if (rank == 0) fprintf(stderr, "要求 n 能被进程数整除\n");
        MPI_Finalize(); return 1;
    }

    const int rows = n / nprocs;   // 本 rank 拥有的行数（blocked 一维划分）
    const int W    = n + 2;        // 含左右边界的行宽
    const int lr   = rows + 2;     // 本地行数（含上下 ghost 行）

    float* A = (float*)malloc(sizeof(float) * (size_t)lr * W);
    for (int i = 0; i < lr; i++)
        for (int j = 0; j < W; j++) {
            int gi = rank * rows + i;      // 本地第 i 行对应的全局行号 0..n+1
            A[i*W + j] = (gi == 0 || gi == n + 1 || j == 0 || j == n + 1) ? 1.0f : 0.0f;
        }

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();
    int   iter  = 0;
    float gdiff = 1e30f;

    while (iter < maxIter && gdiff / ((float)n * (float)n) > TOL) {
        // ---------- 1) 非阻塞交换 ghost 行（避免阻塞 send/recv 的死锁） ----------
        MPI_Request req[4];
        int nreq = 0;
        if (rank > 0) {                    // 与下方邻居 rank-1
            MPI_Isend(&A[1 * W],        W, MPI_FLOAT, rank-1, 0, MPI_COMM_WORLD, &req[nreq++]);
            MPI_Irecv(&A[0 * W],        W, MPI_FLOAT, rank-1, 1, MPI_COMM_WORLD, &req[nreq++]);
        }
        if (rank < nprocs - 1) {           // 与上方邻居 rank+1
            MPI_Isend(&A[rows * W],     W, MPI_FLOAT, rank+1, 1, MPI_COMM_WORLD, &req[nreq++]);
            MPI_Irecv(&A[(rows+1) * W], W, MPI_FLOAT, rank+1, 0, MPI_COMM_WORLD, &req[nreq++]);
        }
        MPI_Waitall(nreq, req, MPI_STATUSES_IGNORE);
        // 注：更激进的做法是先算"不需要 ghost 的内部行"，再 Waitall，最后算边界行，
        //     从而把通信完全藏在计算后面。

        // ---------- 2) 计算本 rank 拥有的 rows 行 ----------
        float mydiff = 0.0f;
        for (int i = 1; i <= rows; i++)
            for (int j = 1; j <= n; j++) {
                float prev = A[i*W + j];
                float v = 0.2f * (A[i*W + j] + A[(i-1)*W + j] + A[(i+1)*W + j]
                                  + A[i*W + j-1] + A[i*W + j+1]);
                A[i*W + j] = v;
                mydiff += fabsf(v - prev);
            }

        // ---------- 3) 全局收敛判定：Allreduce = reduce + broadcast ----------
        MPI_Allreduce(&mydiff, &gdiff, 1, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);
        iter++;
        if (rank == 0 && (iter % 50 == 0 || iter == 1))
            printf("  iter %4d  residual = %.4e\n", iter, gdiff / ((float)n * n));
    }
    double t1 = MPI_Wtime();
```

```
    if (rank == 0) {
        const double cells = (double)n * n * (double)iter;
        printf("n=%d  procs=%d  rows/rank=%d  iters=%d\n", n, nprocs, rows, iter);
        printf("time = %.2f ms  (%.3f ms/iteration)\n", (t1-t0)*1e3, (t1-t0)*1e3/iter);
        printf("aggregate: %.2f GB/s  %.2f GFLOP/s\n",
               24.0*cells/(t1-t0)/1e9, 5.0*cells/(t1-t0)/1e9);
    }
    free(A);
    MPI_Finalize();
    return 0;
}
```

- **【代码做什么？】**
- 把 n×n 内部网格按**行块**（一维 blocked 划分）分给 nprocs 个进程；每个进程分配 `(rows+2) × (n+2)` 的**私有**数组，其中第 0 行与第 `rows+1` 行是要从邻居收到的 ghost 行。
- 每轮开始：用 `MPI_Isend/MPI_Irecv` 与上下邻居**双向交换**各一行（本进程的第 1 行送给下方、第 `rows` 行送给上方），`MPI_Waitall` 等齐。
- 对本地 rows 行做 5 点原地更新，累加本地残差 `mydiff`。
- 用 `MPI_Allreduce` 求全局残差并判定收敛（等价于讲义里的 `reduce_add` + `broadcast` 两个高层原语），然后进入下一轮。
- 在 rank 0 打印每次迭代时间、聚合带宽与算力。
- **【并行机制与性能解说】**
- **Work / Span / 并行度**：
  - 全局 **Work = $c \cdot n^2$** 每迭代（c 为单格代价）。
  - **Span**：每个 rank 内部按行递增的**原地**更新形成串行链：第 i 行依赖第 i–1 行的新值 → 链长 `rows × n` 步（行间还要等 ghost 到达，加上消息延迟 α）。因此 **Span_iter ≈ $c \cdot (rows \cdot n)$ + (消息延迟与 Allreduce 的链长)， 并行度 = W/Span ≈ $c \cdot n^2 / (c \cdot n^2/P$ + sync) → 趋近 P（而不是 n^2）**。
  - 这是非常重要的结论：**"块内 Gauss–Seidel"的并行度等于进程数 P**，即"用几个进程就最多只能用几个进程的并行度"，与红黑版本（并行度 $\Theta(n^2)$）形成鲜明对比。如果希望 MPI 版本也能在块内使用多线程并行（混合编程模型），必须先把块内的 GS 换成红黑或 Jacobi 更新。
- **硬件上如何执行**：`mpirun` 启动的进程由 OS 映射到不同节点/核（Mapping）；通信走 NIC + 网络（延迟 α、带宽 β）；数据所有权是显式的——**ghost 行是拷贝，所有权在邻居手里**，所以每轮都必须在关键路径之前把它们"刷新"一遍。这就是把讲义中的共享内存版（隐式通信、锁 + 屏障）替换成显式消息后的代价：**通信从"看不见的 load"变成"必须排进关键路径的阶段"**。
- **瓶颈分析（α–β 模型）**：单条消息时间 `T = α + m/β`。取 α = 2 μs、β = 10 GB/s，并以下面的强扩展场景为例（n = 4096、P = 64、rows = 64、每节点 8 rank）：
  - 每轮 4 条 ghost 行消息（2 发 2 收，非阻塞可重叠），每行 W = n+2 = 4098 个 float → m = **16.4 KB** → 单条 `2 + 16.4 KB/10 GB/s = 3.64 μs`；双向重叠后关键路径 ≈ **5 μs** 量级。
  - `MPI_Allreduce` 在 P = 64 时约 $2 \cdot \log_2 P \cdot \alpha$ = 12 × 2 = **24 μs** —— **它比 ghost 交换更贵**，因为它的负载极小（一个 float）却要吃满整个归约树的延迟。教训：**每轮都做全局归约是延迟灾难**，应降频（每 k 轮判定一次）或用分层归约。
  - 计算侧：rows = 64、n = 4096 时，每 rank 每轮触及 64 × 4096 × 24 B = **6.29 MB**；若一个节点跑 8 个 rank，每 rank 可用带宽 ≈ 20/8 = 2.5 GB/s → **2.52 ms/迭代**。同步 29 μs 只占 **1.2%** → 该配置下通信不是问题。
  - 但**问题变小就会立刻撞墙**：n = 256、P = 64（rows = 4）时，每 rank 每轮只触及 4 × 256 × 24 B = 24.6 KB → 9.8 μs，而同步仍是 29 μs → **开销 75%，效率跌到 ~25%**。这是"强扩展必须让问题足够大"的定量说明。
- **表面/体积比**：1D 行划分下通信量 ∝ 2 行 × n = O(n)，计算量 ∝ n^2/P → 比值 ∝ P/n。固定 n 增大 P 必然撞墙；**弱扩展**（$n \propto \sqrt{P}$，即每 rank 的行数固定）才能保持效率。若改用 **2D 分块**，通信 ∝ 周长 = $O(n/\sqrt{P})$，比值改善 $\sqrt{P}$ 倍——但代价是数据要打成方块（非连续行），需要派生数据类型或打包缓冲。

---

4. 性能模型与复杂度分析

4.1 Amdahl 定律与强/弱扩展

- **Amdahl（强扩展，固定问题规模）**：设 S 为**固有串行比例**，

$$\text{Speedup}(P) = \frac{1}{S + \frac{1-S}{P}}, \qquad \lim_{P\to\infty} \text{Speedup} = \frac{1}{S}$$

数值表（同一 S 下不同 P 的上限）：

S（串行比例）	P=1	P=16	P=64	P=256	P=1024	P→∞
0.01	1.00	13.91	39.26	72.11	91.18	**100**
0.05	1.00	9.14	15.42	18.62	19.64	**20**
0.10	1.00	6.40	8.77	9.66	9.91	**10**

参照系（来自公开的姊妹课程补充读物）：Summit 超算有 27,648 块 GPU × 5,376 ALU = **148,635,648 个 ALU**。即使 S 小到 0.001（上限 1000 倍），能用上的执行单元也只占机器的 0.0007%——**这就是"小串行区域钉死大机器"的量化形态。** – **Gustafson（弱扩展，问题规模随机器变大）**：若并行部分随 P 增长而串行部分固定，则 `Speedup_scaled = P − S(P−1)`。S = 0.05、P = 64 → 64 − 0.05×63 = **60.85**。两定律不矛盾：Amdahl 说"固定规模下加机器收益递减"，Gustafson 说"机器变大时应该解更大的问题"。这正是 §3.3 中"弱扩展能保持效率"的理论依据。 – **与 Work–Span 的关系**：Amdahl 只考虑"串行段"，忽略了并行部分的**同步与通信开销**——这在 P 大、问题小时是致命遗漏。Work–Span 模型与 α–β 模型补上了这一块。

4.2 Work–Span（Cilk）模型：并行度与贪婪调度定理

- **定义**：
- **Work T_1**：用 1 个处理器执行整个计算的时间（= 总工作量，所有串行化后的和）。
- **Span（关键路径）T_∞**：在**无限多处理器**上的执行时间，等于依赖图中最长路径的长度。
- **并行度（parallelism）$P_\infty = T_1 / T_\infty$**：机器再大也超不过它。
- **贪婪调度定理**：任何贪婪（greedy）调度器满足

$$T_P \le \frac{T_1}{P} + T_\infty$$

且显然 T_P ≥ max(T₁/P, T∞)。因此：
– 当 P ≪ T₁/T∞（**parallel slackness 充足**）时，T_P ≈ T₁/P，线性加速；
– 当 P ≥ T₁/T∞ 时，加速比饱和在 T₁/T∞。

- **本讲各案例的 Work/Span 汇总表**：

计算	Work T_1	Span T_∞	并行度 T_1/T_∞	实际限制因素
`sinx`（N 元素，T 项）	Θ(N·T)	Θ(T)	**Θ(N)**（≈6.7×10⁷）	内存带宽 + 除法吞吐（并行度早已过剩）
两步图像处理（步 2 串行）	$2N^2$	$2N^2$	**1**	Amdahl：S = 0.5 ⇒ 加速比 ⩽ 2
两步图像处理（步 2 并行部分和 + 串行合并）	$2N^2$	Θ(P)（串行合并的链长）	**≈ $2N^2/P$**	合并开销 P（P ≪ N 时可忽略）
红黑网格求解器（每次 sweep）	$c\cdot N^2$	2c + 3B	**$c\cdot N^2/(2c+3B)$**（N=1024 时 ≈281）	**屏障开销 B**（不是浮点算力）
Gauss–Seidel 原位（每次 sweep）	$c\cdot N^2$	Θ(c·N)	**Θ(N)**	对角波前依赖链 + 每波前一次同步
MPI 块内 GS 求解器（每次迭代）	$c\cdot n^2$	$c\cdot n^2/P$ + sync	**≈P**	通信/归约在关键路径上（延迟 α）

4.3 数值算例 A：两步图像处理的最优规模（Amdahl 的解析解）

- **假设**：每处理一个像素记 1 个时间单位（忽略常数）。方案为"步 1 并行 + 步 2 并行求部分和 + 串行合并 P 个部分和"，则

$$T(P) = \frac{N^2}{P} + \frac{N^2}{P} + P = \frac{2N^2}{P} + P, \qquad T_1 = 2N^2$$

$$\text{Speedup}(P) = \frac{2N^2}{2N^2/P + P} = \frac{2P}{2 + P^2/N^2}, \qquad E(P) = \frac{2N^2}{2N^2 + P^2}$$

- **代入数字**：

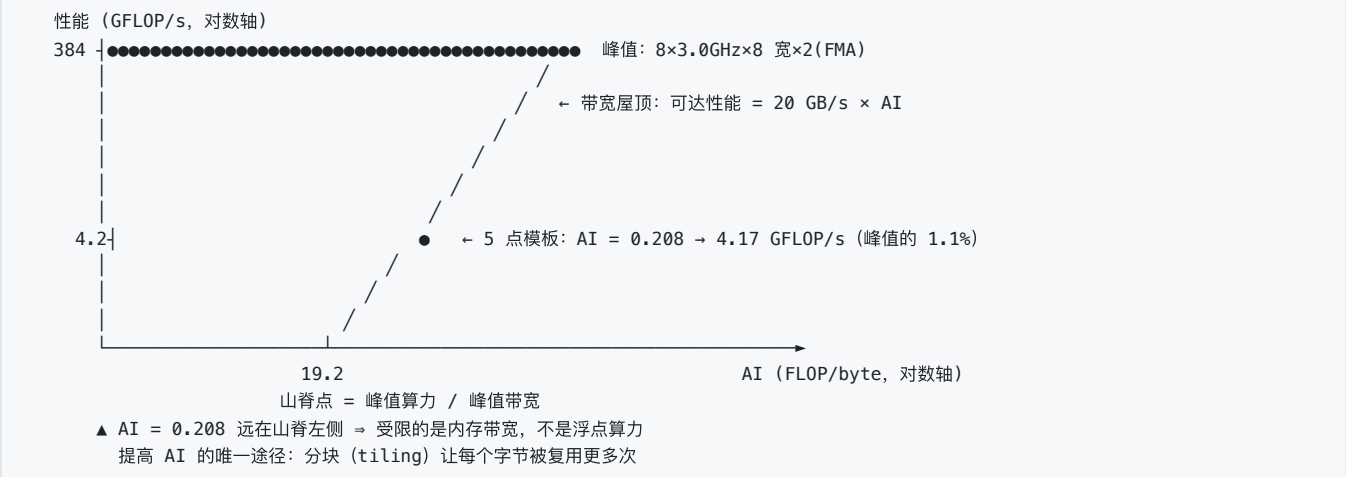
N	P	加速比	效率
1024	64	2×64 ÷ (2 + 4096/1048576)=128/2.0039 = **63.88**	**99.8%**
64	64	128 ÷ (2 + 4096/4096) = 128/3 = **42.67**	**66.7%**
64	1024	2048 ÷ (2 + 1048576/4096) = 2048/258 = **7.94**	**0.78%**
1024	64（步 2 完全串行）	2 ÷ (1 + 1/64) = **1.97**	3.1%

- **结论**：要维持 95% 效率需要 $E \geq 0.95 \Rightarrow 2N^2 \geq 0.95(2N^2+P^2) \Rightarrow N \geq 3.082 \cdot P$。P = 1024 时需要 **N ⩾ 3156**（代入 N = 3155：E = 2×9,954,025/(2×9,954,025+1,048,576) = **0.94997**，恰在 95% 之下；N = 3156：E = **0.95000**）。也就是说：**在 1024 路的机器上，图像至少要有约 3156² 个像素，串行合并的那 P 步才不碍事**。若把串行合并换成**树形归约**（代价 $\log_2 P$ 而非 P），则 $T(P) = 2N^2/P + \log_2 P$，P = 1024 时 N = 1024 也能达到 E = $2N^2/(2N^2+P \log P)$ = 2,097,152/(2,097,152+10240) = **99.5%**。

4.4 数值算例 B：5 点模板的 Roofline（为什么网格求解器永远吃不满机器）

- **机器参数（假设一台 8 核节点）**：

参数	取值	推导
核心数	8	—
频率	3.0 GHz	—
SIMD 宽度	8（AVX2，float）	—
FMA 吞吐	2 FLOP/指令	乘加各算 1 次
峰值算力	**8 × 3.0 GHz × 8 × 2 = 384 GFLOP/s**	—
峰值内存带宽	**20 GB/s**（实测可达的节点级共享带宽）	DDR4–3200 理论 25.6 GB/s
山脊点（ridge point）	**384 / 20 = 19.2 FLOP/byte**	峰值算力 ÷ 峰值带宽
单核均摊带宽	20/8 = 2.5 GB/s	带宽是共享资源
内存延迟	~80—100 ns	—
屏障延迟	~1.5 μs（8 核）	—
网络	α = 2 μs，β = 10 GB/s	—
– **内核的算术强度**：5 点模板每格读 5 个 float（20 B）、写 1 个 float（4 B）= **24 B**；浮点操作 4 加法 + 1 乘法 = **5 FLOP** → **AI = 5/24 = 0.208 FLOP/byte**。		



- **数值结论**：
- 可达性能 = 20 GB/s × 0.208 = **4.17 GFLOP/s**，只有峰值 384 GFLOP/s 的 **1.09%**。

- N = 4096 时一次 sweep：16,777,216 格 × 24 B = **402.7 MB** → 至少 **20.1 ms**；而纯计算只需 83.9 MFLOP ÷ 384 GFLOP/s = **0.22 ms** → **内存受限约 92 倍**。
- 把网格缩到 N = 512（1.05 MB，装进 L2）：每格只需与内存交换 8 B（compulsory miss + 写回），AI 提到 **0.625**，一次 sweep 2.1 MB ÷ 20 GB/s = **105 μs**，比 N=4096 的"每格 24 B"整整便宜 3 倍（相同总格数下）。**这就是"orchestration 里的数据结构/分块组织"为什么值钱**。
- **Little's Law（延迟隐藏所需的并发）**：维持 20 GB/s 在 100 ns 延迟下需要 `并发字节数 = 带宽 × 延迟 = 20 GB/s × 100 ns = 2000 B ≈ 31 条 64 B cache line`。 单线程若同时只有 ~10 条未完成的 load（每条 64 B），则只能撑起 ~6.4 GB/s；因此**要么多线程（≥4 个），要么让编译器向量化出 8 宽 load（一次 32 B，且 8 个元素一次取 2 条 line），要么靠硬件预取**。这解释了为什么 §3.1 的 `sinx` 只有 8 线程时就已经摸到带宽墙：并行度不是问题，**"在飞行中的字节数"才是**。

4.5 数值算例 C：消息传递的 α–β 模型与同步摊销

- **模型**：单条消息 `T = α + m/β`，取 α = **2 μs**、β = **10 GB/s**（跨节点）。
- **场景 1（大问题，n = 4096，P = 64，每节点 8 rank）**：
- ghost 行：每行 W = n+2 = 4098 个 float → m = 16.4 KB → 单条 = 2 + 1.64 = **3.64 μs**；非阻塞双向重叠后关键路径 ≈ **5 μs**。
- Allreduce：$2 \cdot \log_2 64 \times \alpha$ = **24 μs**（负载只有 4 B，纯延迟开销）。
- 每轮同步合计 ≈ **29 μs**；每轮计算（带宽受限）≈ **2520 μs** → 开销 **1.2%**。
- **场景 2（强扩展、问题变小，n = 256，P = 64，rows = 4）**：
- 每 rank 计算 = 4 × 256 × 24 B = 24.6 KB ÷ 2.5 GB/s = **9.8 μs**；
- 同步仍是 **29 μs**（消息变小但 α 不变，归约延迟不变）→ 开销 **75%**，效率 ≈ **25%**。
- **结论与设计准则**：
- **延迟项 α 不随问题缩小而缩小**，所以"小问题 + 多进程"必然低效；强扩展的极限由 `α / (每 rank 计算时间)` 决定。
- **全局归约要降频**（每 k 轮判定一次收敛），或用分层/树形归约，避免每轮付 $2 \cdot \log_2 P \cdot \alpha$。
- **通信要批量化**：把 m 从 4 B 提到 16 KB，α 的占比从 100% 降到 55%；再大就完全被 β 支配（此时带宽成为瓶颈）。
- **表面/体积比**：1D 划分下通信/计算 $\propto P/n$；2D 分块改为 $\propto \sqrt{P}/n$ → 更抗强扩展。取 n = 4096、每格 24 B、网络 10 GB/s、内存 2.5 GB/s，令 ghost 流量折算后等于内存流量的 10%：可得 1D 划分大约在 **P ≈ 4900** 时才真正被通信拖住——**这解释了为什么这类模板计算"很好扩展"，而它真正的墙依然是单节点的内存带宽**（§4.4）。

---

5. 关键要点

1. **并行编程的思维过程是四个阶段，而且顺序不能颠倒**：Decomposition（切出足够多、依赖尽量少的 task）→ Assignment（把 task 分给 worker，静态 blocked/interleaved 还是动态队列）→ Orchestration（通信、同步、数据布局、调度）→ Mapping（worker 落到的物理单元）。性能不佳时先定位卡在哪一阶段，而不是盲目调线程数。同一份算法在共享地址空间、消息传递、数据并行三种模型下会写出三种截然不同的代码，**"哪个更好"取决于机器**（"it depends on the system this program is running on"）。
2. **依赖决定一切：Amdahl 决定上限，Span 决定可扩展性天花板**。Amdahl：串行比例 S 使加速比 $\leqslant 1/S$（S = 0.05 时哪怕 148M 个 ALU 也只能用上 20 倍）。Work–Span：`T_P ≤ T₁/P + T∞`，可用并行度 = T_1/T_∞。两者的实践含义是：**先找依赖、再谈优化**；能通过"改算法"（如 Gauss–Seidel 原位 → 红黑重排）把 Span 从 Θ(N) 降到 Θ(1) 时，代价往往是多做几次迭代——这是值得做的交易。
3. **同步是"用数据换依赖"的工程**：屏障简单但保守（粒度粗），锁表达力强但贵，标志位+自旋最细粒度但必须配内存序。三条实用准则：① **把同步移出内层循环**（本地部分和 → 每轮一次归约，而不是每格一次加锁）；② **减少同步次数**（用 `diff[3]` 轮转消除"清零"依赖，把 3 个屏障降到 1 个）；③ **用空间换依赖**（多副本、ghost cell、私有地址空间复制）是并行编程的常规武器。
4. **机器参数决定优化方向，而内存带宽通常是第一瓶颈**。用 Roofline 判断受限类型：5 点模板 AI = 0.208 FLOP/byte，远低于 8 核节点 19.2 FLOP/byte 的山脊点 → 只有峰值的 1.1%，**提高 AI 的唯一途径是分块复用**。用 Little's Law 判断是否够并发：`带宽 × 延迟 = 20 GB/s × 100 ns = 2000 B ≈ 31 条 cache line` 必须在飞行中。
5. **消息传递里"所有权"和"延迟"是显式成本**：数据要复制（ghost cell），通信要排进关键路径，阻塞 send/recv 会构成循环等待而死锁（靠奇偶序或非阻塞原语打破）。α 项不随问题缩小而缩小，所以强扩展的极限是"每 rank 计算时间 ≫ α"；通信要**大块、少次、能重叠**（Isend/Irecv + 先算内部再算边界），全局归约要降频或分层。

---

6. 常见陷阱与注意事项

- **把 Amdahl 当成"加速比上界"的死结论，却忽略开销项与常数项**：`T(P) = T₁/P + T∞ + sync(P)` ——真正在小问题上钉死你的是第三项（屏障/消息的固定延迟），它不在 Amdahl 的 S 里。P = 64、n = 256 的 MPI 求解器同步占 75%，而按 Amdahl 看"串行比例很小、应该能扩展"——这就是漏掉同步项的典型误判。
- **在内层循环里加锁或做全局归约**：讲义 slide 46 的版本对**每一个网格单元**执行一次 `lock; diff += ...; unlock`，而正确做法是先在寄存器里累加 `myDiff`，每轮只归并一次（slide 47）。前者不仅把临界区数量放大 10^6 倍，还会让存 `diff` 的 cache line 在核间反复弹跳（cache line ping-pong），性能下降一两个数量级。
- **伪共享（false sharing）**：两个线程写**同一 cache line 内的不同变量**，硬件仍会按整条 line 传递所有权。把每线程的累加器放进一个数组（`float partial[P]`）时，相邻元素会落在同一 line；把 `diff[3]` 的 3 个副本放同一 line 也一样。**修复：按 64 B 对齐/填充（padding）或每线程一个独立变量**。注意 §3.2 中相邻线程共享网格边界行是**真共享（只读）**，不是伪共享——两者要分清。
- **依赖没找全就动手并行**：典型的错误是"把顺序执行的 Gauss–Seidel 直接 `#pragma omp parallel for`"——同一行内左邻的新值会被右邻读到，并行后会得到与串行不同的结果（通常是**静默的错误答案**）。要么用红黑/波前重排（需领域知识确认收敛性不受影响），要么改成 Jacobi（同时读旧值），要么加同步（那就退化成串行）。所谓"魔法并行化编译器至今不存在"说的就是这件事：**依赖是数据相关的**，编译期往往不可知。
- **误以为"消息传递里 send 返回就万事大吉"**：阻塞 `send()` 的语义是"数据已到达对方地址空间"，全员同时 `sendDown` 会形成循环等待而死锁（小消息时库可能替你缓冲，于是表现为"有时能跑、有时挂住"——最坏的 bug 形态）。修复要么用奇偶序（偶发奇收），要么用非阻塞 `Isend/Irecv`。**并且非阻塞版本有额外约束：在 `checksend` / `Wait` 确认之前绝不能改写发送缓冲区**——这个约束被违反时，错误是间歇性的、极难复现的。
- **忽略内存带宽与"在飞行字节数"**：以为"线程开得越多越快"，实际每核只能分到 20/8 = 2.5 GB/s；`sinx` 在 8 线程时已撞带宽墙，再加线程只会增加争抢。另一个常见误判是只数"FLOP"：`sinx` 每元素 5 次浮点**除法**（吞吐只有 FMA 的 1/5–1/10），改写为乘倒数能把这段代码从除法受限变成带宽受限——**先测量再优化，不要凭直觉猜瓶颈**。
- **同步不足与内存序错误**：用自旋标志位做生产者–消费者（`x = 1; flag = 1;` / `while (flag == 0); print x;`）时，若 `flag` 不是原子且没有 acquire/release 语义，编译器和 CPU 都可以重排，导致消费者看到 `flag == 1` 却读到旧的 `x`。同理 §3.2 的并行区里，`g_done` 的可见性完全依赖 `single` 末尾的隐式屏障/flush——**擅自删掉那个屏障，就可能出现"部分线程多算一轮、部分线程提前退出"的死锁或错误**。

---

7. 思考题（带答案）

问题 1（Amdahl + 开销项的定量设计） 对 N×N 图像做两步处理：步 1 亮度翻倍（完全独立），步 2 求平均值（归约）。你采用"步 1 并行 + 步 2 并行求部分和 + 串行合并 P 个部分和（每次合并记 1 单位时间）"的方案，每个像素的步 1/步 2 各记 1 单位时间。(a) 写出 T(P)、Speedup(P) 与效率 E(P)。(b) 在 P = 1024 时，要让 E ⩾ 95%，N 至少要多大？(c) 如果把串行合并改成树形归约（代价 $\log_2 P$），在 P = 1024、N = 1024 时的效率是多少？

【答案】 (a) 步 1 时间 N^2/P，步 2 时间 $N^2/P + P$，故 `T(P) = 2N²/P + P`，`T₁ = 2N²`；`Speedup = 2N²/(2N²/P + P) = 2P/(2 + P²/N²)`；`E = T₁/(P·T(P)) = 2N²/(2N² + P²)`。（注意当 N ≫ P 时 `P²/N² → 0`，加速比 → P，与讲义"speedup → P when N >> P"一致；而当步 2 完全串行时 `T(P) = N²/P + N²`，加速比 = `2/(1+1/P) ≤ 2`。）(b) 由 `E ≥ 0.95`：`2N² ≥ 0.95(2N² + P²) ⇒ 0.1N² ≥ 0.95P² ⇒ N ≥ √9.5·P = 3.082P`。P = 1024 → **N ⩾ 3156**（验证 N = 3155：E = 2×9,954,025/(19,908,050+1,048,576) = 0.94997，刚好略低于 95%；N = 3156 时 E = 0.9500）。物理含义：**串行合并的那 P 步必须小于总工作量的 5%，也就是像素数必须远大于处理器数**。(c) `T(P) = 2N²/P + log₂P`，P = 1024 → $\log_2 P = 10$；`E = 2N²/(2N² + P·log₂P) = 2,097,152/(2,097,152 + 10240) = 0.9951` → **99.5%**。这正是"用树形归约把 P 的线性开销换成 log P"的价值：只要肯多花一点编排（orchestration）的功夫，弱可扩展性就完全不同。

问题 2（消息传递的死锁与修复） 某个用**阻塞** send/recv 实现的网格求解器里，每个进程在每轮开始时都先 `sendDown()`（向下邻居发自己第一行数据），然后 `recvDown()`。(a) 为什么 P ⩾ 3 时这个程序常常挂住？请画出等待关系。(b) 讲义给出的修复是"偶数号线程先 send 后 recv，奇数号线程先 recv 后 send"。为什么这样就不会死锁？(c) 改用非阻塞 `MPI_Isend/MPI_Irecv` 后还需要这个奇偶序吗？非阻塞版本还有哪些额外收益与新的约束？

【答案】 (a) 阻塞 `send()` 的返回条件是"发送方收到确认：消息数据已驻留在接收方的地址空间"（同步 send 的语义）。如果所有进程都先执行 `sendDown()`，那么每个进程都在等自己的**下邻居**执行 `recvDown()`；而下邻居正忙着等它自己的下邻居……等待关系构成一个**环**（T0→T1→T2→…→T_{P−1}→T0），环上没有任何进程会推进到 recv，因此永久挂起（hang），而不是报错。若消息足够小、MPI 实现恰好用了 eager/buffered 协议，send 会立即返回，程序"看起来正常"——这就是这类 bug 最危险的地方：**依赖实现细节、时好时坏**。(b) 相邻两个进程构成的每一对 (偶, 奇) 中，偶数号**先发**、奇数号**先收**：偶数的 `sendDown` 立刻被奇数的 `recv` 匹配完成，随后奇数再发、偶数再收。于是"等待环"被打破——每一对里总有一方先开出可被对方匹配的接收动作，不会出现全员僵在 send 上的局面。（前提是交换的两侧都按同一约定排列，即

偶数在它的每个邻居方向上都是"先发方"，这正是"偶/奇"这个全局一致编号带来的好处。）(c) 不再需要奇偶序：`MPI_Isend/MPI_Irecv` 立即返回句柄，收发请求由 MPI 进度引擎在后台完成，最后 `MPI_Waitall` 统一等待，不存在"必须由对方先 recv 才能返回"的循环依赖。额外收益是**通信与计算重叠**——可以先发/收 ghost 行，然后计算不依赖 ghost 的内部行，最后 `Waitall` 并计算依赖 ghost 的边界行（本讲 §3.3 示例中把 `Waitall` 提前了，可以进一步优化）。新的约束是：**在 `Wait` / `checksend` 确认发送完成之前，绝对不能改动发送缓冲区**（否则发出的可能是被改写后的数据，产生间歇性错误）；同样，在确认收到之前不能读接收缓冲区。此外还多出了请求句柄管理、进度引擎与资源占用的开销。

问题 3（为什么"改算法"是合法的并行化手段？红黑 vs 对角波前 vs 三副本技巧） (a) 对角波前（diagonal wavefront）与红黑着色（red–black coloring）分别给出了多大的并行度和多少次同步？(b) 红黑重排改变了浮点运算的结果序列，凭什么说它是合法的？(c) 讲义用 `diff[3]` 轮转把三个屏障减到一个。为什么必须是 3 个副本而不是 2 个？这个技巧的一般名字是什么？

【答案】 (a) **对角波前**：一次 sweep 有 2N−1 条反对角线（k = i+j），第 k 条的长度 ≈ min(k, N)，最大并行度 ≈ **N/2**；但必须在**每条波前之后同步一次**，所以一次 sweep 要 **2N−1 次同步**，Span = Θ(N)，而且两端（三角形区域）并行度低、同步频繁——"好消息是并行性存在，坏消息是很难利用"。**红黑着色**：把单元分成两个相位，相位内**零依赖** → 每个相位并行度 $\Theta(N^2/2)$，一次 sweep 只需 **2 次相位同步**（加收敛判定所需的同步），Span = Θ(1)（不含同步）。代价是收敛所需的迭代次数通常变多（同一容差下要多做几次 sweep，因为改变的是迭代格式），即**用"多一点工作量"换"高得多的并行度和少得多的同步"**。(b) 合法性来自**领域知识**：红黑排序仍然是解同一个线性系统的一致迭代格式，它迭代到**同一个不动点**，只是收敛路径不同；浮点中间值不同、每次迭代的残差不同，但最终仍收敛到误差阈值之内。讲义明确指出："we needed domain knowledge of Gauss–Seidel method for solving a linear system to realize this change is permissible for the application"。反过来说，**如果应用对迭代序列本身敏感（例如需要与某个参考实现逐位一致、或迭代中途被外部观测），这种重排就是不合法的**——所以"改算法换并行度"必须由懂该问题语义的人做决定，而不是编译器能自动完成的优化。(c) 因为需要打破的依赖是"**本轮还在读 diff，下一轮却要把它清零**"。设迭代 k 使用 `diff[k mod m]`：线程 A 在迭代 k 里清零某个缓冲区时，线程 B 可能还停在迭代 k−1 的**收敛判定**上，正在读 `diff[(k−1) mod m]`；同时 A 自己还要往 `diff[k mod m]` 里累加。要求这三者互不相同，即 `k`、`k−1`、`k+1` 在模 m 下两两不同 → **m ≥ 3**。用 2 个副本时 `(k+1) mod 2 = (k−1) mod 2`，清零会毁掉别人正在读的值，因此仍然需要一个额外屏障，等于没省。这个技巧的一般思想是"**用空间（额外副本、轮转缓冲）换依赖的消除**"（trade off footprint for removing dependencies），也是并行编程中双缓冲/多缓冲（double buffering、ring buffer）的同一套路。注意副本之间最好填充到不同的 cache line（见 §6 伪共享），否则省下的屏障会被 cache line 弹跳吃回去。

附：本讲术语速查（速查表格，附在七个章节之后）

英文	中文	一句话定义
Decomposition	分解	把问题切成可并行执行的 task；关键是识别依赖
Assignment	分配	把 task 绑到 worker；静态（blocked/interleaved）或动态（任务队列/窃取）
Orchestration	编排	组织通信、加同步、组织数据结构、调度任务
Mapping	映射	把逻辑 worker 落到物理执行单元（OS / 编译器 / 硬件负责）
Amdahl's Law	阿姆达尔定律	串行比例 S 使加速比 ⩽ 1/S
Work / Span	工作量 / 跨度	1 个处理器的时间 / 无限处理器的时间（关键路径）；比值 = 并行度
Ghost cell	影子单元	消息传递中从邻居地址空间复制过来的边界数据（所有权仍在邻居）
Barrier	屏障	相位同步：之前所有线程的计算完成后，之后的计算才能开始
Red–black coloring	红黑着色	改变更新顺序把依赖降成两个无依赖相位，以换取相位内无限并行度
Bulk transfer	批量传输	通信时整行/整块地传，而不是逐元素传，以摊薄 α

Lecture 8: Performance Optimization I: Work Distribution and Scheduling

1. 章节标题与概述

Lecture 8: Performance Optimization I: Work Distribution and Scheduling（性能优化 I：工作分配与调度）

- **本讲核心问题**：当一个并行程序已经把工作**分解（decomposition）**成许多小块之后，**"把哪一块交给哪个执行上下文、什么时候交"** 就成了决定性能的关键。本讲要回答的是："让所有处理器一直有活干"与"少付调度/同步的额外开销"这对互相冲突的目标之间，如何选择**静态分配（static assignment）**、**半静态分配（semi-static）**、**动态分配（dynamic assignment）**，以及**工作窃取（work stealing）**，并把**任务粒度（task granularity）** 调到合适的位置。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：多核/SMT 的**执行上下文（execution context）** 数量、共享内存的**缓存行（cache line）迁移与争用**（一个被 8 个核反复读–改–写的计数器行是串行化瓶颈）、**片间/跨 socket 的相干性延迟**（实证约 0.3–0.5 μs 一次争用原子操作）、NUMA 与内存带宽对"分到工作后跑得动跑不动"的约束。
- **软件侧**：共享工作队列（shared work queue）、**每线程双端队列（per-worker dequeue）**、原子取号（`atomic_incr` / `fetch_add`）与锁（`lock/unlock`）、Cilk Plus 的 `cilk_spawn` / `cilk_sync` 抽象、**worker 线程池（worker pool）**、**continuation stealing（偷连续体）**与 **child stealing（偷子任务）**、**greedy join scheduling（贪婪汇合调度）**、同步点描述符（sync descriptor，用 spawn/done 计数实现 `cilk_sync`）。
- **在并行计算知识体系中的角色**：本讲是第 7 讲"并行编程基础（decomposition / assignment / orchestration 三步法）"的直接延续，也是全课程中**唯一一次系统地讲"调度器内部怎么工作"**。它把前面学到的 **Work/Span、Amdahl 定律** 从"分析工具"变成"设计工具"：负载不均是用 Amdahl 惩罚你，粒度过细是用同步开销惩罚你，两者之间的最优解取决于**你的工作负载与你的机器**（这也是课程反复强调的"must know your workload, and your machine"）。后续的性能优化 II（局部性、缓存分块、伪共享）、同步与锁、无锁数据结构、以及异构调度（ISPC task、CUDA block 调度）都建立在本讲的概念上。
- **配套材料**：
- `lectures/07_progperf1.pdf`（抽取文本 `extracted/07_progperf1.txt`，共 **48 页**）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 公开下载。Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）把 **Sep 11** 排为第 8 讲 **"Performance Optimization I"**，对应这份讲义。讲义首页写的是 **"Lecture 5: Performance Optimization Part 1: Work Distribution and Scheduling"** 与 **"CMU 15–418/15–618, Fall 2025"**：这是**讲义沿用历史学期版本**的正常现象（讲次编号与学期字样随年度重排），不是错误。
- `cs149_supp/perfopt1.txt`：**Stanford CS149（Fall 2025）Lecture 5** 同名讲义的**已公开**补充材料（共 **63 页**），**已公开**。它比 CMU 版多出关键的两块：**`cilk_sync` 的实现细节（sync descriptor 的 spawn/done 计数演化，slide 51–61）** 与 **Cilk 的 greedy join scheduling 策略（slide 62–63）**，本笔记对同步实现的描述以它为准。注意：两份讲义中同名示例给出的数字略有差异（例如"失衡的处理器多做了多少工作"一处是 **20%**、一处是 **2 倍**），本笔记在 §4 把两个版本都作为算例列出。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**（历史学期在 YouTube 上有存档，但不在 Fall 2026 公开日程中）。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 部分讲座在 Fall 2026 **尚未发布**公开讲义（Performance Analysis / Profiling、Transactional Memory、AI in System Design 等）；其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。

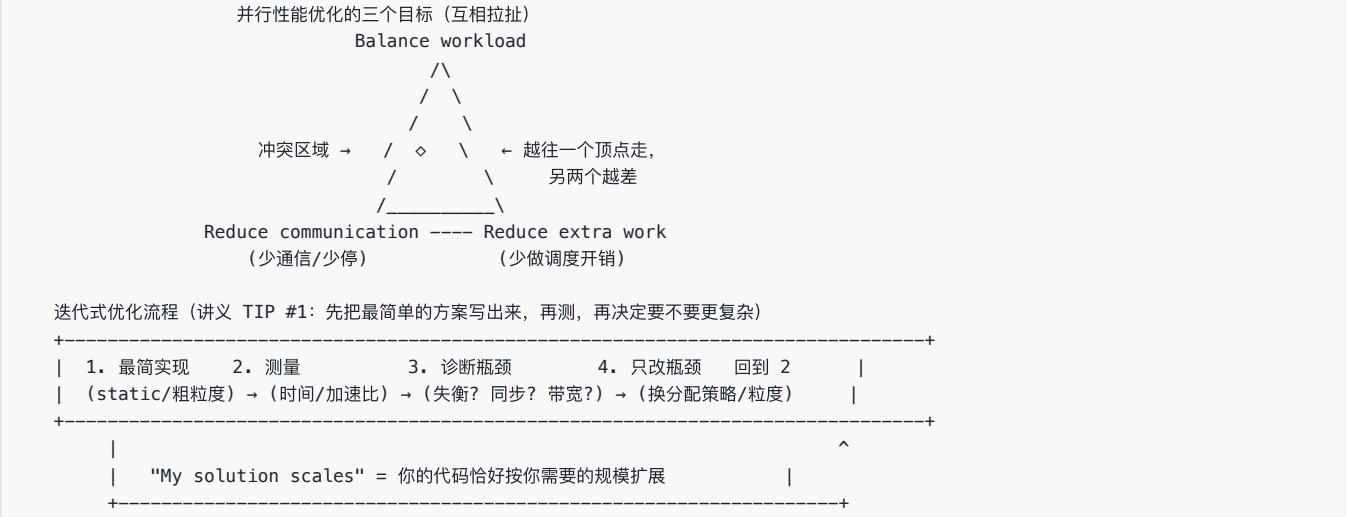
---

2. 核心概念与硬件/软件架构图解

2.1 全部问题的来源：三个互相冲突的目标

- **定义与目的**：讲义的出发点是（slide 2/48）：优化并行程序性能，是**对 decomposition、assignment、orchestration 三者反复精化的迭代过程**，而其中三个目标天生互相冲突：
- **Balance workload**（把工作均衡地铺到可用执行资源上）；
- **Reduce communication**（减少通信，避免停顿 stall）；
- **Reduce extra work / overhead**（减少为了获得并行性、管理分配、减少通信而额外做的工作）。 你不可能同时把三者都做到最优：要让负载均衡，就得多切任务、多通信、多管理；要少管理，就得任务粗、但容易失衡。
- **直观解释（"它是什么？"）**：把它想成**餐厅后厨的分菜问题**。菜（工作）总量固定：
- 想"人人不闲着"（balance），就得把菜切得很碎、谁空了谁再来端一盘，代价是**每个人要不停跑传菜窗口排队**（communication + overhead）；

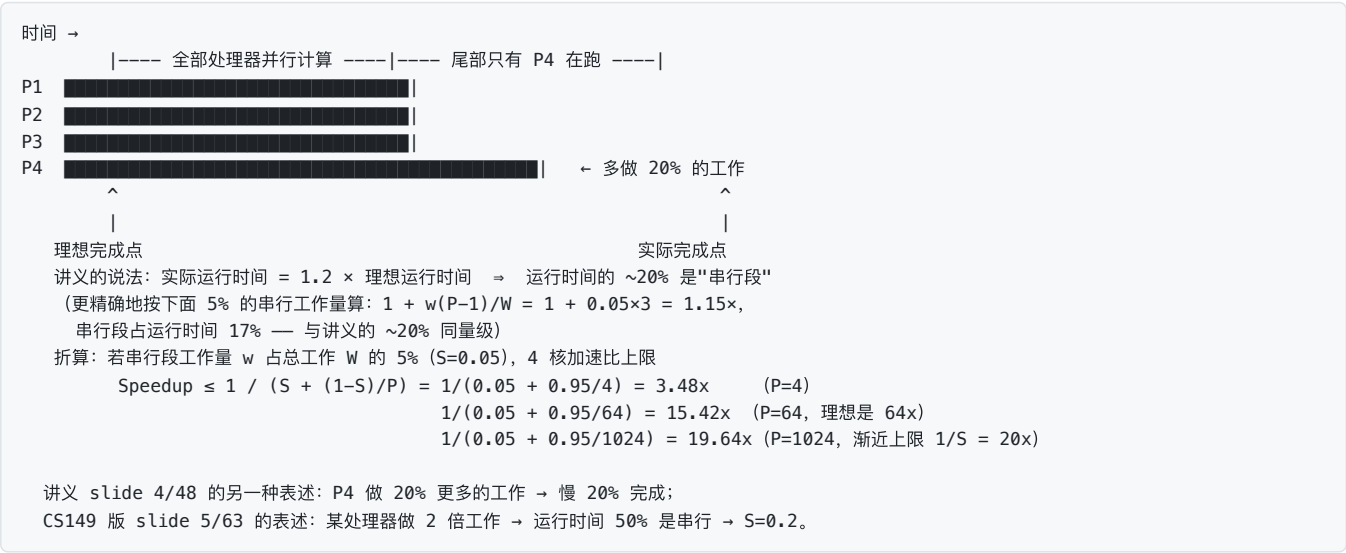
- 想"少跑动"（reduce communication），就让每个人端着大盆菜不动（任务粗），代价是**有人早早吃完、有人还在啃**（imbalance）；
- 想"少做无用功"（reduce extra work），就提前把菜分好放到各人灶台（static assignment），代价是**一旦量估错就失衡**。 调度策略的选择，本质上就是在这三者之间挑一个你**能接受的折中点**。
- **图解（图 1）：三目标张力与"先简单后测量"的工作流**



- **关键操作与性能特征**：讲义明确给出 TIP #1：*总是先实现最简单的版本，然后测量，再决定是否需要做得更好*。"我的方案可扩展"这句话的正确含义是"**在我的目标机器规模上**可扩展"。如果预知只会跑低核数机器，为了实现"能产生成百上千块独立工作"的复杂方案而付出的代价，可能完全不必要。

2.2 负载不均如何被 Amdahl 定律放大惩罚

- **定义与目的：负载不均（load imbalance）** 指程序执行期间并非所有处理器都在同时计算、或它们并非同时完成各自的份额。理想情况是"所有处理器在整个执行期间都在计算，并且同时完成"。
- **直观解释（"它是什么？"）**：类比**四人划船**。三个人划 100 下就靠岸了，第四个人需要划 120 下。船不会在 100 下时到岸，也不会因为多了一个人而变快：**最后 20 下只有一个人在划，其余三人把桨收起来看**。这段"只有一个人在划"的时间，在 Amdahl 定律里等价于**串行执行段**——即使它只占全部工作量的 5%，在 64 核上也只能跑到 15.4 倍（理想是 64 倍），而在 1024 核上依然只能跑到 19.6 倍（渐近上限 20 倍）。
- **图解（图 2）：负载不均的 Gantt 图与 Amdahl 折算**



- **性能特征**：注意**放大器效应**：失衡本身只是"多一点工作"，但它被 Amdahl 折算成"串行段"，于是**核数越多惩罚越重**：S=0.05 时，4 核上限 3.48x、64 核上限 15.42x（相对理想的 64x 只剩 24% 效率）、1024 核上限 19.64x（渐近上限就是 1/S = 20x）。这就是"只有很小的负载不均就能显著限制最大加速比"的定量含义。

2.3 静态分配：blocked 与 interleaved

- **定义与目的**：**静态分配**指**工作到线程的映射预先确定**。注意它**不一定是编译期决定的**：只要"工作量与工人数已知时映射就固定"，就叫静态（可以由输入规模、线程数等运行期参数算出）。讲义回顾了网格求解器示例的两种经典静态划分：
- **blocked（块状）**：把连续的一段网格单元分给同一个线程；
- **interleaved（交错/循环）**：按 `i % P` 把单元分给 P 个线程。
- **直观解释（"它是什么？"）**：**blocked 像把一条街分成 4 段包干**——每人守一段，各扫各的，不需要说话（通信少、局部性好），但如果某段特别脏就有人先收工；**interleaved 像扑克发牌**——每人轮流拿一张，工作量自动被摊得很均匀，但每个人手里的牌散落全街，来回跑（局部性差、缓存行利用率低）。
- **适用条件（讲义 slide 6–7/48）**：
- 工作的**成本与数量可预测**，程序员能提前算出好的分配；
- 最简单的情形：**所有工作成本相同**（例如 12 个任务、4 个处理器 → 每人 3 个）；
- 成本不均但**已知**（或统计上已知，例如"平均成本相同"）→ 按成本做加权划分即可（如 LPT，见 §2.5）。
- **优点**：简单、运行期开销**几乎为零**（额外工作只是一点索引运算）。
- **"半静态"（semi-static）**：当工作的成本只在**近期未来**可预测时（"最近的过去是最近未来的好预测"），程序可以**周期性自我剖析（profile）并重调分配**，两次重调之间分配是"静态"的。典型场景：**自适应网格（adaptive mesh）**——物体移动或流场变化使网格缓慢变化，按变化后的网格重新着色；**粒子模拟**——粒子在模拟中缓慢漂移，只需间歇性地重新分配。此时的关键判据是**变化的速率**：变化慢，重分配就不必频繁。

表 1：工作分配策略谱（静态 → 半静态 → 动态 → 窃取）

策略	谁决定映射	需要的先验知识	运行期开销	负载均衡能力	典型场景
静态 blocked	程序员（预先算）	工作量与成本可预测	近零（索引运算）	差（成本不均时）	规则网格、均匀成本任务
静态 interleaved	程序员（预先算）	成本统计上均匀	近零	中（摊平慢变化）	成本近似均匀的循环体
半静态	程序周期性重算	短期可预测	周期性重分配成本	中上好	自适应网格、粒子模拟
动态（共享计数器/队列）	运行期调度逻辑	无需	每次取活都要同步	好	成本不可预测的任务集
工作窃取（分布式队列）	运行期调度器	无需	仅窃取时付代价	好	fork–join 递归分解

2.4 动态分配：共享计数器与共享工作队列

- **定义与目的**：**动态分配**指程序**在运行期决定**工作到处理器的分配，以保证负载被良好铺开——它适用于**任务执行时间或任务总数不可预测**的情形。讲义用"素性测试"的例子对比：串行版本直接 `for (i=0;i<N;i++) is_prime[i] = test_primality(x[i]);`，而并行版本用一把锁保护一个共享计数器：

```
LOCK counter_lock;  int counter = 0;          // 共享变量
while (1) {
    int i;
    lock(counter_lock);
    i = counter++;                            // 等价于原子操作 atomic_incr(counter)
    unlock(counter_lock);
    if (i >= N) break;
    is_prime[i] = test_primality(x[i]);       // 执行时间未知的工作
}
```

- **直观解释（"它是什么？"）**：**共享工作队列像医院挂号窗口**：所有病人（worker 线程）在一个窗口前排队领号（取一件工作），领完号各自去诊室（执行工作）。**柜台本身是串行的**：窗口一次只能服务一个人，队伍再长也没用——这就是"临界区（critical section）"。
- **图解（图 3）：共享队列 + 临界区时间轴（细粒度 vs 粗粒度）**

```
      共享工作队列（Shared Work Queue）—"一个窗口"
      +-------------------------------------------------+
      |  task0 | task1 | task2 | task3 | task4 | ...     |   ← 每件工作独立
      +-------------------------------------------------+
          ^        ^        ^        ^
          |        |        |        |           （取走一件工作 = 一段临界区）
        T1       T2       T3       T4
     Worker 线程：从共享队列拉取工作；产生新工作时再推回队列

时间轴（细粒度：1 task = 1 element）
T1 |CS|work 0        |CS|work 4        |CS|work 8        |
T2     |CS|work 1         |CS|work 5        |CS|work 9  |
T3          |CS|work 2          |CS|work 6             |
T4               |CS|work 3           |CS|work 7        |
      ^^^^ 每个 CS 都是串行的（同一把锁），它们是"串行程序里不存在的额外工作"
           且 CS 之间互相排队 → 按 Amdahl：运行时间下限 = 任务数 × 临界区长度

时间轴（粗粒度：1 task = 10 elements，临界区进入次数少 10 倍）
T1 |CS|work 0..9              |CS|work 40..49            |
T2     |CS|work 10..19              |CS|work 50..59       |
      ^^^^ CS 次数下降 → 同步开销下降；但每件工作变长 → 可能出现负载不均
```

- **性能特征**：细粒度（1 task = 1 element）的优点是**负载均衡好**（任务多、每件都小），缺点是**同步开销可能极高**（临界区串行化，加上"串行程序里不存在的额外工作"）；粗粒度（例如 `GRANULARITY = 10`）的好处是**临界区进入次数减少 10 倍**，坏处是**每件工作变长，负载不均风险上升**。讲义在这里提出一个关键反问：**"So... IS IT a problem?"（所以，这真的是问题吗？）**——答案是"取决于临界区开销与每件工作成本的比值"，这正是 §4.4 用实测数据定量回答的问题。
- **选择任务粒度的两个反向力量（讲义 slide 13/48）**：
- 想**任务数 ≫ 处理器数**（很多小任务 → 动态分配才能获得好的负载均衡）→ 激励**小粒度**；
- 想**任务数尽量少**（减少管理分配的开销）→ 激励**大粒度**；
- **理想粒度取决于许多因素**：必须了解你的工作负载和你的机器。

2.5 更聪明的调度：长任务优先（LPT）

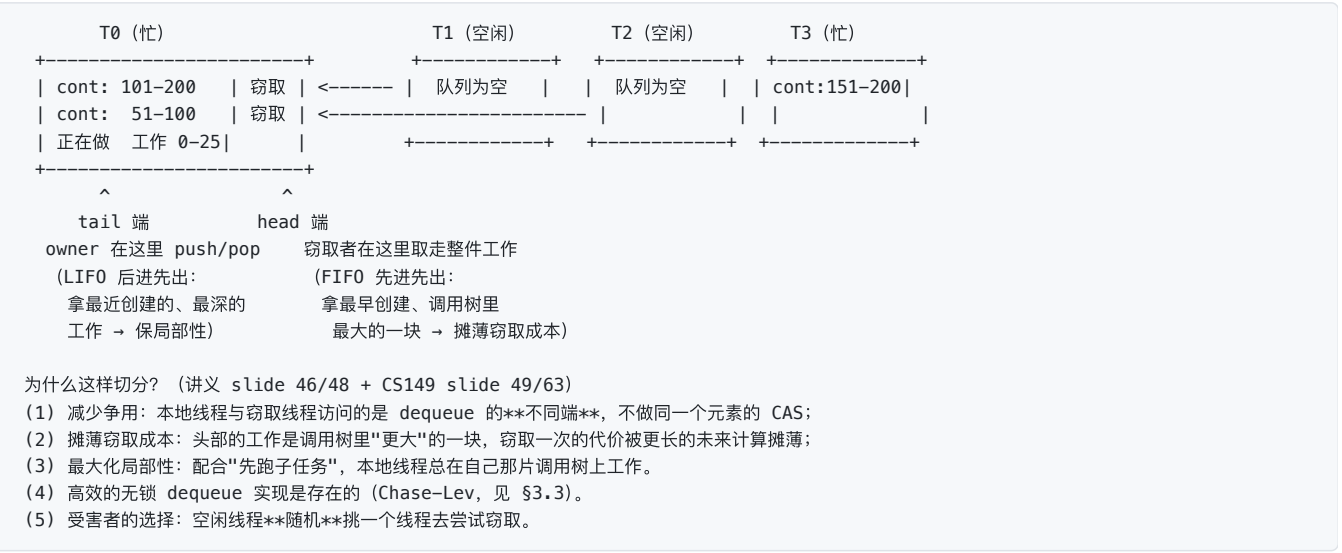
- **定义与目的**：动态分配本身**不解决**"长杆（long pole）"问题。讲义给出 16 个任务的例子：如果系统按从左到右的顺序把它们派给 worker，而**最长的那个任务排在最后**，那么当其它三个处理器都完成时，最后一个任务才刚开始 → 尾部出现大块空洞（slop）。两种解法：
- **把工作切得更细**：希望"长杆"相对整体执行时间变短。代价：可能增加同步开销；**也可能根本做不到**（也许这件工作本质上是串行的，不可再分）。
- **更聪明的调度：长任务优先（Longest Processing Time first, LPT）**：让执行长任务的线程**做的任务件数更少，但总工作量与其他线程近似相同**。代价：需要一些关于工作负载的知识（成本要有一定可预测性）。
- **直观解释（"它是什么？"）**：**装箱/排队论里的经典技巧**：超市结账时，**先让买得最多的人去排队**——因为他的结账时间最长，越早开始越不会成为最后一个离店的人；如果让他最后来，所有人都在等他一个人。等价地：**先放大石头，再往缝里塞沙子**。
- **图解（图 4）：顺序决定 makespan**

```
长任务最后（动态队列按输入顺序派发）
时间 →
P1 |t0|t4|t8 |t12|            Idle ..................................
P2 |t1|t5|t9 |t13|            Idle ..................................
P3 |t2|t6|t10|t14|            Idle ..................................
P4 |t3|t7|t11|                |========= 长任务 t15（60 单位）=========|
                                                    ^ 完成时刻 = makespan 84

长任务最前（LPT）
P1 |===== 长任务 t15 =====|t4|t8 |t12|
P2 |t1|t5|t9 |t13|t16|
P3 |t2|t6|t10|t14|
P4 |t3|t7|t11|t17|
        ^ 完成时刻 = makespan 60   （§3.4、§4.2 有完整的数值仿真：49 理想 / 84 / 60 / 50）
```

- **性能特征**：长任务优先把"尾部空洞"从 **84 单位** 压到 **60 单位**（理想值 49），折算出的串行段从 **S=0.417** 降到 **S=0.183**，Amdahl 加速比上限从 **1.78x** 升到 **2.58x**。若进一步把长任务**一分为二**并且仍然长任务优先，makespan 可到 **50 单位（S=0.020，上限 3.77x）**——**"拆分"与"排序"是两种互补手段，叠加使用最有效**。

2.6 用分布式队列降低同步开销：工作窃取

- **定义与目的**：共享队列的痛点是**所有 worker 都要在同一把锁上同步**。解法是给**每个 worker 一个自己的队列**：
- worker 优先从**自己的队列**取工作、向**自己的队列**推新工作；
- 当自己的队列空了（且没别的活），**去别的队列"窃取（steal）"**。
- **为什么这样更好（讲义 slide 18/48）**：
- 昂贵的同步/通信**只在窃取时发生**，而窃取**只在必要时发生**（为了负载均衡），不是每次取工作都要付；
- **局部性提升**：常见情形是"线程处理自己创建的任务"（**生产者–消费者局部性**，producer–consumer locality）；
- **实现挑战**（讲义明确列出）：从谁那里偷？偷多少？**如何检测程序终止**？以及"在保证互斥的前提下，让本地队列访问足够快"。
- **直观解释（"它是什么？"）**：**四条装配线各自的"待办筐"**。每个工人先干自己筐里的活（不需要和任何人打招呼）；只有当自己的筐空了，才去同事的筐里抓一把活回来。因为"筐空了"是少数时刻，**平时大家都在闷头干活，只有偶尔一次的"抓活"需要沟通**。
- **图解（图 5）：每线程 dequeue 与"本地尾部 / 窃取头部"**




```
为什么这样切分？（讲义 slide 46/48 + CS149 slide 49/63）
(1) 减少争用：本地线程与窃取线程访问的是 dequeue 的**不同端**，不做同一个元素的 CAS；
(2) 摊薄窃取成本：头部的工作是调用树里"更大"的一块，窃取一次的代价被更长的未来计算摊薄；
(3) 最大化局部性：配合"先跑子任务"，本地线程总在自己那片调用树上工作。
(4) 高效的无锁 dequeue 实现是存在的（Chase-Lev，见 §3.3）。
(5) 受害者的选择：空闲线程**随机**挑一个线程去尝试窃取。
```

- **性能特征**：**大部分时间线程只在本地 dequeue 上 push/pop（几乎免费），只有窃取时才付原子操作与缓存行迁移的代价**。因此"偷大块"是设计原则：一次窃取（0.1–1 μs 量级）必须被后面几百微秒的计算摊薄，否则窃取开销会像细粒度锁一样吃掉收益（§4.5 有算例）。

2.7 队列里的工作不必互相独立：任务依赖 DAG

- **定义与目的**：任务管理系统中，工作项可以带**显式依赖**；**只要一件任务的依赖没有全部满足，它就不会被取出交给 worker**。讲义给出 API 形态的例子：

```
foo_handle = enqueue_task(foo);              // 入队 foo（与之前所有任务无关）
bar_handle = enqueue_task(bar, foo_handle);  // 入队 bar，但必须等 foo 完成才能运行
```

- **直观解释（"它是什么？"）**：**流水线上的工序卡**。一张卡上写着"本工序必须在 3 号件完工之后才能开始"，调度员（task management system）负责检查依赖是否满足，满足才把卡发给工人；工人也可以**在干活的过程中提交新的工序卡**（并声明依赖）。
- **性能特征**：依赖关系通常构成一张 **DAG（有向无环图）**；调度器的自由度只来自"当前所有依赖已满足"的任务集合（ready set），因此**任务图的关键路径（span）直接决定运行时间下限**——这正是 §4 的 work–span 模型的适用对象。

2.8 Fork–join 抽象：`cilk_spawn` / `cilk_sync`

- **定义与目的**：分治算法（quicksort、归并、树遍历……）天然含有大量**互相独立的工作，fork–join（分叉–汇合）** 是表达它们的自然方式。本讲的代码示例语言是 **Cilk Plus**（MIT 起源、后成为开放标准，曾被 GCC、Intel ICC 支持）：
- `cilk_spawn foo(args);` 语义：**调用 foo，但与普通函数调用不同，调用者可以异步地继续执行**（"fork"，创建一个新的逻辑控制流）；
- `cilk_sync;` 语义：**当本函数派生的所有调用都完成后才返回**（"join"，与派生调用汇合）；
- **重要细节**：任何包含 `cilk_spawn` 的函数**末尾都有隐式的 `cilk_sync`**。含义是：**当一个 Cilk 函数返回时，它关联的所有工作都已经完成。**

- **抽象 vs 实现（讲义 slide 28/48，CS149 slide 30/63）**：
- `cilk_spawn` **不规定**被派生调用**如何、何时**被调度执行，只规定它**可以**与调用者（以及调用者派生的其它调用）并发执行；
- `cilk_sync` **构成对调度的约束**：所有派生的调用必须在 `cilk_sync` 返回前完成。
- CS149 由此提了一个很好的判断题：**如果某个 Cilk 实现把 `cilk_spawn foo()` 完全当成普通函数调用，它还是正确的实现吗？** 答案见 §7 思考题 1。
- **直观解释（"它是什么？"）**：`cilk_spawn` 像**把一件事顺手写进"待办便签"**："这件事可以交给别人做，也可以我自己做，反正它和接下来的事没有先后约束"。`cilk_sync` 像**下班前清空所有便签**："便签上没做完的事全部做完，我才能接着往下走"。
- **图解（图 6）：fork–join 的调用树与 DAG**

```
代码：                                      调用树（含串行省略）：
  cilk_spawn foo();                          main
  bar();                                      ├── foo()      ← 派生出的"子"
  cilk_sync;                                  └── bar()      ← 连续体 continuation
                                                              （cilk_sync 汇合）

  bar();          foo 与 bar 可以并发执行
  foo()

更一般地，三个 spawn 一个调用：
  cilk_spawn foo();        main
  cilk_spawn bar();         ├── foo()
  cilk_spawn fizz();        ├── bar()
  buzz();                   ├── fizz()
  cilk_sync;                └── buzz()      ← 四个调用可并行，代价是 3 次 spawn 管理

DAG 视角（节点=一段工作，边=依赖）：
        [main 前半]
         /   |    \
     [foo] [bar] [fizz]
         \   |    /
        [buzz + 后半]  ← cilk_sync 汇合点
   Work W = 所有节点工作量之和；Span S = 最长路径；并行度 = W/S
```

- **性能特征**：并行开销随 spawn 次数增长。讲义特意指出：**用两次 spawn 表达"与一个 spawn 相同量的独立工作"，运行时开销可能更高**（多一次 spawn 的管理）。所以"派生多少"要按 **parallel slack（并行冗余度）** 来定（见 §4.3）：**要有比执行能力更多的独立工作以利于负载均衡（实践上 ~8 倍），但不能多到粒度太细。**

2.9 从抽象到实现：worker 线程池 + 每线程工作队列

- **先看一个"天真"的实现**：把每个 `cilk_spawn` 翻译成一次 `pthread_create`，把 `cilk_sync` 翻译成相应的 `pthread_join`。会出什么问题?
- **spawn 操作过于重量级**（线程创建是微秒级、内核参与）；
- **并发运行的线程数远多于核数**（SMT/调度器被压垮）；
- **上下文切换开销**；
- **工作集（working set）比需要的更大，缓存局部性变差。**
- **正确的做法：worker 线程池（pool of worker threads）**：
- Cilk Plus 运行时维护一个 worker 线程池，可以（近似地）理解为"所有线程在程序启动时创建"；
- **worker 线程数恰好等于机器上的执行上下文数**（讲义给的例子：4 核 8 超线程的笔记本 → 8 个 worker）；
- 每个 worker 的主循环就是：**有活就取一件、跑掉**：

```
while (work_exists()) {
    work = get_new_work();
    work.run();
}
```

- 现实实现是**惰性**的：worker 线程在**第一次 Cilk spawn** 时才初始化（ISPC 对跑 ISPC task 的 worker 线程也是同样策略）。
- **直观解释（"它是什么？"）**：**常驻的 8 个工人 + 每人一个待办筐**（而不是"每来一件事就雇一个人"）。雇人很贵，所以固定雇 8 个（等于机器的执行上下文数），让**待办筐**承担所有调度弹性。

- **图解（图 7）：worker 池 + 每线程工作队列 + 窃取**

```
 Cilk Plus 运行时的 worker 线程池（8 个 worker，对应 8 个执行上下文）
+--------+--------+--------+--------+--------+--------+--------+--------+
|Thread 0|Thread 1|Thread 2|Thread 3|Thread 4|Thread 5|Thread 6|Thread 7|
+--------+--------+--------+--------+--------+--------+--------+--------+
    |        |        |        |        |        |        |        |
 +-----+  +-----+  +-----+  +-----+  +-----+  +-----+  +-----+  +-----+
 |deque|  |deque|  |deque|  |deque|  |deque|  |deque|  |deque|  |deque|  ← 每线程一个
 | 0   |  | 1   |  | 2   |  | 3   |  | 4   |  | 5   |  | 6   |  | 7   |
 +-----+  +-----+  +-----+  +-----+  +-----+  +-----+  +-----+  +-----+
    ^                                                      ^
    |                                                      |
 本地 push/pop（快）                                  空闲时随机选受害者窃取（慢，但少）

执行 cilk_spawn foo(); bar(); 时的三个时刻（对应讲义 slide 35–38/48）：

(a) 串行实现（天真）：Thread 0 用普通函数调用跑 foo()，
    连续体 bar() 隐式存在 Thread 0 的调用栈里。
    Thread 1 此时空闲 —— 它本可以在跑 bar()！

(b) 到达 cilk_spawn 时（run child first）：
      Thread 0 stack: [ ... foo() ... ]       Thread 0 queue: [ cont: bar() ]
      Thread 1 stack: [ 空 ]                  Thread 1 queue: [ 空 ]  ← 空闲线程

(c) Thread 1 发现自己的队列为空，去"忙"线程的队列找活：
      1. 查看 Thread 0 的队列  2. 把工作搬到自己的队列  3. 恢复执行 bar()
      Thread 0 queue: [ 空 ]                  Thread 1 queue: [ 空 ]
      Thread 0: 执行 foo()…                   Thread 1: 执行 bar()…  ← 并行起来了
```

2.10 关键设计选择：偷"子任务"还是偷"连续体"?

- **定义与目的**：在 `cilk_spawn foo(); bar();` 处，调用线程面临二选一：
- **先跑子任务（run child first）**：把**连续体（continuation，即 `bar()` 及其之后）**登记起来供别人窃取 → 称为 **continuation stealing（偷连续体）**；
- **先跑连续体（run continuation first）**：把**子任务 `foo()`** 登记起来供别人窃取 → 称为 **child stealing（偷子任务）**。
- **讲义（与 CS149）选择的是 continuation stealing**。用一个循环来说明为什么：

```
for (int i = 0; i < N; i++) { cilk_spawn foo(i); }
cilk_sync;
```

- **child stealing（先跑连续体）**：调用线程会**先把所有迭代的 spawn 都登记完，才开始执行任何一个**。相当于对调用图做**广度优先遍历，需要 O(N) 的空间**（最大空间），而且**如果没有窃取发生，执行顺序与"删掉 cilk_spawn"的程序差别巨大**（先登记完 0..N−1，再执行）。队列里堆满 `foo(0), foo(1), …, foo(N−1)`。
- **continuation stealing（先跑子任务）**：调用线程**只创建一件可被窃取的东西**——代表"所有剩余迭代"的连续体（`cont: i=1`）。如果没有窃取发生，线程**不断从队列弹出连续体、执行 `foo(i)`、再把 `i` 加 1 的连续体入队**，于是**执行顺序与"删掉 spawn"的程序完全一致**（深度优先遍历）。
- **空间界的可证明结论（讲义 slide 42/48）**：continuation stealing 下，**T 个线程的系统的工作队列总存储不超过单线程执行栈存储的 T 倍**。
- **直观解释（"它是什么？"）**：**BFS vs DFS 的取舍**。child stealing 像"先把所有待办便签都写下来贴在墙上（墙会爆）"；continuation stealing 像"只写一张便签：**剩下的活**"，撕一张做一张（墙永远只有几张便签），而且**没人帮忙时我做的事跟单线程一模一样**——这带来极好的缓存局部性与可调试性。
- **图解（图 8）：两种策略的队列状态对比**

```
                    child stealing（先跑连续体）—— 讲义未采用
循环体会先把所有迭代登记完：
Thread 0 queue (tail → head):
  [ foo(N-1) | foo(N-2) | ... | foo(2) | foo(1) | foo(0) ]   ← O(N) 空间，广度优先
无窃取时执行顺序： foo(0) 最后才跑（顺序与串行版不同）

                    continuation stealing（先跑子任务）—— Cilk 采用
每一步只登记"剩下的迭代"这一张连续体：
时刻 1: Thread 0 queue: [ cont: i=1 ]       Thread 0 正在执行 foo(0)
时刻 2: Thread 1 窃取 cont:i=1 →
        Thread 1 queue: [ cont: i=2 ]       Thread 1 正在执行 foo(1)
        Thread 0 queue: [ 空 ]              Thread 0 继续做自己的事
无窃取时执行顺序： foo(0), foo(1), ..., foo(N-1)  ← 与删掉 spawn 的程序一致（深度优先）
空间上界：T 个线程的总队列存储 ≤ T × 单线程栈存储
```

- **注意一个容易误解的细节（讲义 slide 47/48 明确强调）："子任务优先"的窃取调度器是为分治并行性设计的，而朴素的 spawn 循环并不能很快地把机器填满**：把 `for (i…) cilk_spawn foo(i);` 与递归对半拆分的写法相比，**后者的代码才是"并行地产生工作"**：每层递归都把**整棵右半子树**作为连续体入队，窃取者一次拿走的就是一整棵**还能继续向下分裂**的子树（工作块更大 → 窃取次数更少 → 每次窃取被更长的计算摊薄，并且偷来的子树可以继续分裂给更多线程），而朴素循环只沿调用链逐个产生"下一个迭代"级别的细碎可窃取项。讲义原文结论是：递归版本"**能更快地把机器填满**"。

```
递归产生工作（能快速填满机器）            朴素 spawn 循环（沿调用链逐个产生）
void recursive_for(int start,int end){    for (int i=0;i<N;i++) cilk_spawn foo(i);
  while (start <= end - GRANULARITY){     cilk_sync;
    int mid = (end-start)/2;
    cilk_spawn recursive_for(start,mid);   调用链展开，每一步只多一个"下一个迭代"级别的
    start = mid;                           细碎可窃取项：foo(0)→foo(1)→foo(2)→…
  }                                        可窃取工作沿调用链**线性地**逐个出现
  for (int i=start;i<end;i++) foo(i);
}
先跑子任务 + 每层递归都把"右半子树"作为连续体入队：
第 1 层：队列里出现 (N/2, N) 这棵**子树**
第 2 层：出现 (N/4, N/2) 子树……
窃取者一次拿到的就是**一整棵可再分裂的子树**（块大 → 窃取次数少 → 成本被摊薄），
而且偷来的子树还能继续向下分裂给更多线程 → 迅速把机器填满
```

2.11 `cilk_sync` 是怎么实现的：sync descriptor 与贪婪汇合

- **定义与目的**：`cilk_sync` 必须知道"**本块里派生的调用是否都完成了**"。Cilk 的实现方式是给每个"包含 spawn 的块"配一张**描述符（descriptor）**，记录两个计数：
- **spawn 计数**：这个块里已经派生出去的调用总数；
- **done 计数**：其中已经完成的个数。 当且仅当 `spawn == done` 时，同步满足。
- **关键实现要点（CS149 slide 52–62/63）**：
- 如果**没有任何工作被别的线程偷走**，那么 `cilk_sync` 处**无事可做——它是一个 no-op**（线程自己按深度优先顺序把所有派生的调用都跑完了）；
- 只有当**发生了窃取**，才需要"块描述符"记账：被偷走的连续体由窃取线程继续执行，它在该块里**继续派生的新调用仍然登记到同一张描述符上**（CS149 示例中 `id=A` 块的 spawn 计数按 1 → 2 → 3 … 10 增长，对应 `foo(0)`、`foo(1)`、…、`foo(9)` 分别被不同线程接手，被偷走的任务在受害者队列里被标为 `STOLEN (id=A)`）；
- 每个被偷走的任务要标明它属于哪个块（`STOLEN (id=A)`），完成后把 **done 计数加一**；
- 最后一个完成的派生调用把 `done` 推到与 `spawn` 相等（例如 `spawn: 10, done: 10`），**同步条件满足**，此时**当初发起 spawn 的那个线程可能并不是执行 `cilk_sync` 之后代码的线程**——谁最后完成谁继续；
- 完成之后 **块描述符被释放**（可以复用），继续执行 `bar()`。
- **贪婪汇合（greedy join scheduling）策略（CS149 slide 62/63）**：
- **所有线程只要没事干就尝试窃取**；
- **只有当系统里真的没有可窃取的工作时，线程才空闲（idle）**；
- **发起 spawn 的 worker 未必是执行 `cilk_sync` 之后逻辑的 worker**；

- 记住：**记录窃取与管理同步点的开销只在发生窃取时出现**；只要偷走的是大块工作，窃取就很稀有；**绝大多数时间线程只是在本地 dequeue 上 push/pop**。
- **直观解释（"它是什么？"）**：descriptor 像**团建活动的签到表**："这个街区派出去 10 个人，签到 10 个才算人齐"。**没人被别的队伍借走时，签到表根本不用拿出来**（no-op）；一旦有 3 个人被借走，就要靠"借出登记 + 归还登记"来数人头。人齐之后，**谁最后一个回来就由谁宣布散会**（不一定是队长）。
- **图解（图 9）：sync descriptor 状态机**

- **性能特征**：**同步几乎零成本**是这套设计的核心优点：开销**只在窃取时产生**，与"每次取工作都进入临界区"的共享队列方案（§2.4）形成鲜明对比。这也解释了为什么 fork-join 系统（Cilk、OpenMP task、TBB、ISPC task）能在**极细粒度**的递归分解上工作。

---

3. 代码示例与性能分析

> 下面四个示例都在同一台机器上实测过（AMD EPYC 7V13，128 个硬件执行上下文，2 socket × 64 核；除注明外均用 8 个执行上下文；共享机器，重复测量存在波动，§3.1 末尾专门讨论了这件事）。

3.1 示例 1：fork-join 并行快排（Cilk Plus 原版 + 可运行的 OpenMP 等价版）

(A) 讲义原版（Cilk Plus）

```
// 讲义 slide 29/48 的并行快排（Cilk Plus）
// 编译（历史/受支持的编译器）：g++ -fcilkplus -O3 -lcilkrts qsort.cilk.cpp -o qsort   （GCC 5–7）
//                              icc -O3 qsort.cilk.cpp -o qsort                        （Intel 编译器）
// 说明：Cilk Plus 的 C/C++ 扩展自 GCC 8 起被移除；若工具链不支持，请使用下面的
//       OpenMP task 版本，或使用基于 Clang 的开源后继 OpenCilk（-fopencilk）。
void quick_sort(int* begin, int* end) {
    if (begin >= end - PARALLEL_CUTOFF)
        std::sort(begin, end);                      // 串行省略（serial elision）
    else {
        int* middle = partition(begin, end);        // 串行划分（关键路径所在）
        cilk_spawn quick_sort(begin, middle);       // fork：左半可以并行
        quick_sort(middle + 1, end);                // 当前线程继续做右半（连续体）
    }                                               // 函数末尾有隐式 cilk_sync
}
```

(B) 可运行的等价版本（OpenMP task，已实测）

```cpp
// qsort_omp.cpp —— fork-join 并行快排（Cilk Plus 示例的可移植 OpenMP 等价实现）
// 编译（release）: g++ -O3 -fopenmp -std=c++17 qsort_omp.cpp -o qsort_omp
// 运行:            OMP_NUM_THREADS=8 ./qsort_omp 20000000
#include <algorithm>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <omp.h>
#include <random>
#include <vector>

static const long PARALLEL_CUTOFF = 8192;   // 小于该长度就串行排序（串行省略）

// Hoare 划分：返回切分点 p，保证 begin < p < end，且 [begin,p) 的元素 <= [p,end) 的元素
static int* hoare_partition(int* begin, int* end) {
    int* mid = begin + (end - begin) / 2;
    if (*mid < *begin)        std::swap(*mid, *begin);       // 三数取中
    if (*(end - 1) < *begin)  std::swap(*(end - 1), *begin);
    if (*(end - 1) < *mid)    std::swap(*(end - 1), *mid);
    int pivot = *mid;
    int* i = begin - 1;
    int* j = end;
    while (true) {
        do { ++i; } while (*i < pivot);
        do { --j; } while (*j > pivot);
        if (i >= j) return j + 1;          // 关键：切分点严格落在 (begin, end) 内
        std::swap(*i, *j);
    }
}

// 对应讲义的 cilk_spawn quick_sort(begin, middle); quick_sort(middle, last);
static void quick_sort(int* begin, int* end) {
    const long n = end - begin;
    if (n <= PARALLEL_CUTOFF) {            // 串行省略：划分到足够小就交给 std::sort
        std::sort(begin, end);
        return;
    }
    int* mid = hoare_partition(begin, end);
#pragma omp task firstprivate(begin, mid)  // 左半：交给 task 池，可被其它线程"窃取"
    quick_sort(begin, mid);
    quick_sort(mid, end);                  // 右半：当前线程立刻继续（连续体留在本地）
#pragma omp taskwait                       // 对应 cilk_sync：等待本 task 派生的所有子 task
}

static bool is_sorted(const int* a, long n) {
    for (long i = 1; i < n; ++i)
        if (a[i - 1] > a[i]) return false;
    return true;
}

int main(int argc, char** argv) {
    const long N = (argc > 1) ? atol(argv[1]) : 20000000L;
    std::vector<int> a(N), ref;

    auto t0 = std::chrono::steady_clock::now();
    std::mt19937 rng(12345);
    for (long i = 0; i < N; ++i) a[i] = (int)rng();
    auto t1 = std::chrono::steady_clock::now();
    const double gen_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    ref = a;                                           // 串行基线
    t0 = std::chrono::steady_clock::now();
    std::sort(ref.begin(), ref.end());
    t1 = std::chrono::steady_clock::now();
    const double serial_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    t0 = std::chrono::steady_clock::now();
#pragma omp parallel
    {
#pragma omp single nowait
        quick_sort(a.data(), a.data() + N);
```

```
    }
    t1 = std::chrono::steady_clock::now();
    const double par_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    std::printf("N=%ld  threads=%d  gen=%.1f ms  T_serial(1 thread)=%.1f ms  T_par(%d)=%.1f ms  speedup=%.2fx  sorted=%s\n"
                N, omp_get_max_threads(), gen_ms, serial_ms, omp_get_max_threads(), par_ms,
                serial_ms / par_ms, is_sorted(a.data(), N) ? "yes" : "NO");
    return is_sorted(a.data(), N) ? 0 : 1;
}
```

实测（N = 20,000,000 个随机 `int`，`PARALLEL_CUTOFF = 8192`，一次代表性运行）：

线程数 P	1（并行代码）	1（`std::sort` 基线）	2	4	8
时间 (ms)	1620.5	1657.3	876.0	560.6	442.5
加速比	1.02x	1.00x	1.84x	2.88x	3.65x

【代码做什么？】

1. `main` 用固定种子的 `std::mt19937` 生成 N 个随机 `int`；另存一份副本用 `std::sort` 串行排序作为基线，并记录时间。
2. 进入 `#pragma omp parallel` 并行区，用 `#pragma omp single nowait` 让**一个线程**发起顶层 `quick_sort`；递归内部派生出的 task 会由整个线程池执行。
3. `quick_sort` 对长度 ⩽ 8192 的区间直接调用 `std::sort`（**串行省略**：派生开销超过潜在并行收益时不再派生）；否则先做 **Hoare 划分**（返回 `j+1`，保证切分点严格位于区间内部，左右两半都非空，递归一定终止）。
4. 左半用 `#pragma omp task` 提交为**可延迟执行的任务**（`firstprivate` 按值捕获 `begin` / `mid`），右半**由当前线程直接递归**——这就是"先跑一个、把另一半登记出去"的 fork–join 结构；`#pragma omp taskwait` 对应 `cilk_sync`，等待本任务派生的子任务全部完成。
5. 排序结束后用 `is_sorted` 校验正确性（并作为退出码），保证测量的是"正确的排序"。

【并行机制与性能解说】

- **硬件上如何并行**：OpenMP 运行时在第一次 task 构造时初始化一个 worker 线程池，线程数 = `OMP_NUM_THREADS`（这里 8，等于 8 个执行上下文）。每个 worker 有自己的**任务队列**；当一个 worker 在当前任务上遇到 `taskwait` 而子任务尚未完成、或自己的队列为空时，它会去别的 worker 的队列**窃取（steal）**任务。`firstprivate(begin, mid)` 让两个指针作为任务私有数据被搬走——**被窃取的是"一段区间的排序工作"这块粗粒度任务，而不是每元素级别的细活**。
- **Work（总工作量）**：以"元素操作"计。若划分大致均衡，`W(n) = 2W(n/2) + θ(n)`，递推到串行省略阈值 C 之后叶子各自做 `θ(C log C)`：`W = θ(n log n)`。取 n = 2×10⁷、C = 8192：内部各层划分工作量合计约 `n·log₂(n/C) ≈ 2×10⁷ × 11.2 ≈ 2.2×10⁸`，叶子排序约 `(n/C)·C log C ≈ 2441 × 8192 × 13 ≈ 2.6×10⁸`，所以 **W ≈ 4.8×10⁸ 元素操作**。
- **Span（关键路径）**：划分是串行的，关键路径沿"每次取一半"向下延伸：`D(n) = D(n/2) + θ(n)`，其中 `θ(n)` 是划分成本，几何级数求和给出 **D ≈ 2n ≈ 4.0×10⁷ 元素操作**（叶子 `std::sort` 的 span 相对可忽略）。
- **并行度 = W / Span ≈ 4.8×10⁸ / 4.0×10⁷ ≈ 12**。这是本讲最重要的一个数字：**快排的并行度只有 Θ(log n) 量级**。也就是说，即使有 128 个核，这个算法的加速比上限大约在 **12 倍**附近（对应"元素操作"粒度）；8 个核还没有触到这个天花板（8 < 12），所以效率损失**不是**并行度不足造成的，而是下面这些：
- **额外工作（extra work）**：并行版本比串行 `std::sort` 多做了 `log₂(n/C) ≈ 11` 层的划分。实测 `T_par(1) = 1620 ms` 与 `T_serial = 1657 ms` 几乎相等，说明这份多出来的工作与 `std::sort` 在小规模上的优势大致抵消——**"并行代码在 1 线程下不比串行慢"是一个好的信号**（如果慢很多，说明额外工作太多）。
- **负载不均的尾部**：叶子任务的大小是 8192 个元素，最后几个叶子任务无法再切分，尾部必然有一段"不足 8 个线程在干活"的时间。任务数 = `n/C ≈ 2441`，相对 8 个线程的 parallel slack ≈ 305，远大于讲义建议的 ~8，所以尾部只占很小一部分。
- **共享内存子系统的吞吐/延迟**：4 → 8 线程只带来 1.27x（560.6 → 442.5 ms），这是**典型的共享资源饱和特征**（不是同步开销的特征——同步开销会随线程数线性恶化）。n = 2×10⁷ 个 `int` = 80 MB，完整一层划分至少要读写 ~160 MB 流量；`log₂(n/C) ≈ 11` 层合计约 **1.8 GB 流量**；在 442 ms 内跑完相当于 **~4 GB/s** 的有效流量。这个数字本身远低于现代服务器的 DRAM 带宽，但 Hoare 划分是**带分支、低 MLP（memory-level parallelism）的指针追赶式扫描**（`i` / `j` 两个游标、每步一次不可预测的分支），**真正的限制是延迟与访存并行度，而非带宽**。可用三个实验验证这个诊断：(i) 把 `PARALLEL_CUTOFF` 调大以减少划分层数；(ii) 把 N 缩小到能装进 LLC，看加速比是否改善；(iii) 用 `perf stat` 看 LLC-miss 与跨 socket 访存比例。
- **测量纪律**：这台机器是共享的。同一份代码重复运行时，8 线程的实测加速比在 **2.6x–3.7x** 之间波动（负载高时串行基线本身也从 1657 ms 涨到 2000 ms 以上）。这正是讲义 TIP #1 的实践含义：**先测量，并记录测量条件**；报告"我的方案可扩展"时必须带上机器状态与规模。

3.2 示例 2：共享计数器 + chunk —— 动态分配的粒度旋钮

```cpp
// chunked_counter.cpp —— 动态工作分配：共享计数器 + chunk（任务粒度粗化）
// 编译（release）: g++ -O3 -pthread -std=c++17 chunked_counter.cpp -o chunked_counter
// 运行:            ./chunked_counter
#include <atomic>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <mutex>
#include <thread>
#include <vector>

static int g_threads = 8;                       // 8 个执行上下文（可改）

// 一件"工作"：对种子做 iters 次乘加（纯计算，成本可控）
static inline uint64_t busy(uint64_t seed, int iters) {
    uint64_t acc = seed | 1;
    for (int i = 0; i < iters; ++i) {
        acc = acc * 6364136223846793005ULL + 1442695040888963407ULL;
        acc ^= acc >> 29;
    }
    return acc;
}

struct Result { double ms, max_work, avg_work; long min_tasks, max_tasks; };

// 所有线程反复用"领号"的方式从共享计数器取 chunk 件工作
static Result run(const std::vector<int>& cost, int chunk, bool use_mutex) {
    const long M = (long)cost.size();
    std::atomic<long> counter{0};
    std::atomic<long> next_locked{0};
    std::mutex        mtx;
    std::atomic<uint64_t> sink{0};
    std::vector<double> work(g_threads, 0.0);
    std::vector<long>   tasks(g_threads, 0);

    std::atomic<int>  ready{0};
    std::atomic<bool> go{false};

    auto worker = [&](int tid) {
        ready.fetch_add(1, std::memory_order_release);
        while (!go.load(std::memory_order_acquire)) { /* 起跑线自旋 */ }
        while (true) {
            long i;
            if (use_mutex) {                    // 讲义里的 lock(counter_lock) ... unlock(counter_lock)
                mtx.lock(); i = next_locked.load(); next_locked.store(i + chunk); mtx.unlock();
            } else {                            // 等价但更便宜的原子取号（atomic fetch-add）
                i = counter.fetch_add(chunk, std::memory_order_relaxed);
            }
            if (i >= M) break;
            const long end = (i + chunk < M) ? i + chunk : M;
            uint64_t acc = 0;
            for (long j = i; j < end; ++j) { acc += busy((uint64_t)j, cost[j]); work[tid] += cost[j]; }
            sink.fetch_xor(acc, std::memory_order_relaxed);
            tasks[tid] += (end - i);
        }
    };

    std::vector<std::thread> th;
    for (int t = 0; t < g_threads; ++t) th.emplace_back(worker, t);
    while (ready.load(std::memory_order_acquire) != g_threads) { /* 等所有线程就位 */ }
    auto t0 = std::chrono::steady_clock::now();     // 计时不含线程创建
    go.store(true, std::memory_order_release);
    for (auto& x : th) x.join();
    auto t1 = std::chrono::steady_clock::now();

    Result r{};
    double sum = 0, mx = 0; long tmin = M, tmax = 0;
    for (int t = 0; t < g_threads; ++t) {
        sum += work[t]; if (work[t] > mx) mx = work[t];
        if (tasks[t] < tmin) tmin = tasks[t];
        if (tasks[t] > tmax) tmax = tasks[t];
```

```
    }
    r.ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
    r.avg_work = sum / g_threads; r.max_work = mx;
    r.min_tasks = tmin; r.max_tasks = tmax;
    (void)sink.load();
    return r;
}

int main() {
    struct Regime { const char* name; long M; int lo, hi; };
    const Regime regimes[2] = {
        {"LIGHT", 200000, 4,   32},          // 每件工作 4~32 次迭代：同步开销会被放大
        {"HEAVY",   2048, 5000, 40000},      // 每件工作 5000~40000 次迭代：粒度粗化会暴露负载不均
    };

    for (const Regime& R : regimes) {
        std::vector<int> cost(R.M);
        for (long i = 0; i < R.M; ++i) {     // 成本不可预测：最大/最小相差 8 倍
            int spread = (int)((i * 2654435761u) % 4);
            cost[i] = (R.lo << spread) < R.hi ? (R.lo << spread) : R.hi;
        }
        std::printf("\n===== %s workload: %ld tasks, cost %d~%d iters, %d threads =====\n",
                    R.name, R.M, R.lo, R.hi, g_threads);
        std::printf("%8s | %11s %11s | %14s | %13s | %9s\n",
                    "chunk", "T_atomic/ms", "T_mutex/ms", "max_work/avg_work", "tasks min/max", "T vs chunk=1");
        auto best = [&](int chunk, bool use_mutex) {   // 取 3 次最小值，抑制共享机器的噪声
            Result b = run(cost, chunk, use_mutex);
            for (int k = 0; k < 2; ++k) { Result r = run(cost, chunk, use_mutex); if (r.ms < b.ms) b = r; }
            return b;
        };
        Result base = best(1, false);
        for (int chunk : {1, 4, 16, 64, 256, 1024, 2048, 4096, 16384, 65536}) {
            if (chunk > R.M) break;
            Result ra = best(chunk, false);
            Result rm = best(chunk, true);
            std::printf("%8d | %11.3f %11.3f | %14.3f | %6ld/%-6ld | %8.2fx\n",
                        chunk, ra.ms, rm.ms, ra.max_work / ra.avg_work,
                        ra.min_tasks, ra.max_tasks, base.ms / ra.ms);
        }
    }
    return 0;
}
```

实测（8 线程，每个配置取 3 次最小值）：

表 2：LIGHT 负载（200,000 件工作，每件 4/8/16/32 次迭代，8 线程）

chunk	原子取号 (ms)	互斥锁 (ms)	max_work/avg_work	任务数 min/max	相对 chunk=1
1	99.519	44.583	2.740	14865 / 68743	0.82x
4	18.436	24.950	1.122	23824 / 28044	4.44x
16	4.808	10.378	1.129	23776 / 28224	17.03x
64	0.990	3.045	1.103	22528 / 27584	82.75x
256	0.619	0.645	1.014	24064 / 25344	132.31x
1024	**0.573**	0.597	1.024	24576 / 25600	**142.87x**
2048	0.577	0.598	1.065	24576 / 26624	141.86x
4096	0.603	0.600	1.119	24576 / 27968	135.72x
16384	0.693	0.696	1.311	16384 / 32768	118.12x
65536	1.383	1.337	2.621	0 / 65536	59.21x

表 3：HEAVY 负载（2,048 件工作，每件 5000/10000/20000/40000 次迭代，8 线程）

chunk	原子取号 (ms)	互斥锁 (ms)	max_work/avg_work	任务数 min/max	相对 chunk=1
1	9.555	9.522	1.007	252 / 262	1.00x
4	9.480	9.504	1.000	256 / 256	1.00x
16	9.470	9.482	1.000	256 / 256	1.00x
64	9.471	9.500	1.000	256 / 256	1.00x
256	9.497	9.477	1.000	256 / 256	1.00x
1024	37.526	37.497	**4.000**	0 / 1024	**0.25x**
2048	74.890	74.883	**8.000**	0 / 2048	**0.13x**

【代码做什么？】

1. 两种负载各做一轮实验：**LIGHT**（200,000 件工作、每件 4–32 次乘加迭代）与 **HEAVY**（2,048 件工作、每件 5,000–40,000 次迭代）。每件工作的成本按 `(i × 大奇数) % 4` 取值，**成本相差 8 倍且不可预测**——正是动态分配要处理的场景。
2. `run()` 启动 8 个 worker 线程；线程创建完成后**由主线程在起跑线上放开（`go`）并开始计时**，因此计时**不含线程创建开销**（这是让 LIGHT 负载的微小工作时间可被观测的关键）。
3. 每个 worker 反复"领活"：用 `chunk` 件为一单位向共享计数器取号；取号有两种实现——**互斥锁**（讲义里的 `lock/unlock` 写法）与**原子 `fetch_add`**（等价且更便宜）。取到 `i >= M` 时退出循环。
4. 每个 worker **分别累计**自己完成的工作量与件数，`run()` 汇总出 **`max_work/avg_work`（负载不均指标）**、**每线程件数 min/max**（动态分配是否真的把活摊开了）与总时间。
5. 每个配置跑 3 次取最小值（抑制共享机器的噪声），并在两张表里扫过 chunk = 1 … 65536，把"粒度"这个旋钮的整个曲线打出来。

【并行机制与性能解说】

- **硬件上如何并行**：8 个 worker 线程（8 个执行上下文）各自计算，**唯一的共享可写数据是那个 8 字节计数器所在的缓存行**。`fetch_add` 是一条**原子读–改–写（RMW）**指令：执行时该缓存行必须处于该核的 Exclusive/Modified 状态，于是**同一行在 8 个核（甚至两个 socket）之间来回迁移**，每次迁移都要走相干性协议（跨 socket 时约 0.3–0.5 μs）。**临界区里的那几行代码，就是"串行程序里不存在的额外工作"**，而且它本身就是串行执行（Amdahl）。
- **Work 与 Span**：设成本数组之和为 `W_task = Σ cost[i]`（单位：迭代次数），取号次数为 `M/chunk`。**总工作量 = W_task + (M/chunk)·c_sync**，其中 `c_sync` 是"全局串行化意义下"的一次取号成本。**关键路径（Span）**包含两部分：最长单件工作的成本，以及**被串行化的取号链**——`(M/chunk)·c_sync`。当 `(M/chunk)·c_sync` 与并行部分 `W_task/P` 同量级时，程序就变成"取号驱动"的。
- **为什么 chunk=1 会输得这么惨（LIGHT）**：200,000 次取号，实测耗时 99.5 ms。用**全局串行化成本**反推：`c_sync ≈ (99.5 ms − 0.573 ms) / 200,000 ≈ 494 ns/次`。也就是说，**在这种饱和争用下，一次原子取号等价于把整个程序串行推进 0.5 μs**；200,000 次就是 100 ms，与实测几乎完全一致。**这不是"开销百分比"问题，而是"程序被彻底串行化"的问题**：此时运行时间 = Span，Amdahl 的 S≈1。
- **为什么 HEAVY 下 chunk=1 却没问题**：同样 chunk=1，HEAVY 只有 2,048 次取号，若每次 500 ns 也只有 ~1 ms，而总工作是 9.5 ms，并且取号与计算**重叠**（线程取完号就去算 28 μs，计数器有足够时间"冷静"下来，单次成本回落到 ~50–100 ns）。实测 chunk=1 与 chunk=256 的差异 < 1%。**结论：同步开销不是常数，而是争用强度的函数**——这正是"必须测量"的最好例证。
- **为什么 HEAVY 下 chunk 太大会输（表 3 最后两行）**：`chunk=1024` 时整个问题只剩 **2 件工作**分给 8 个线程，`max_work/avg_work = 4.0`（有 6 个线程完全没活干，`tasks min/max = 0/1024`）；`chunk=2048` 只剩 **1 件工作**，退化为串行，`max_work/avg_work = 8.0`，时间 ×7.9。这就是讲义"very large tasks can lead to load balancing issues"的量化形态。
- **可扩展性上限**：这条曲线的最优区间（LIGHT：256–4096；HEAVY：1–256）由两个约束夹出来，见 §4.4 的公式与推导。**任务数相对处理器的 slack（冗余度）才是可扩展性的真正来源**：只有 `M/chunk ≫ 8P`，动态分配才能吸收成本波动。

3.3 示例 3：Chase–Lev 工作窃取双端队列 + 迷你 Cilk 运行时

```
// ws_deque.cpp —— 迷你 Cilk：Chase-Lev 工作窃取双端队列 + 并行快排
// 编译（release）：g++ -O3 -pthread -std=c++17 ws_deque.cpp -o ws_deque
// 运行：            ./ws_deque 20000000 8
#include <algorithm>
#include <atomic>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <memory>
#include <random>
#include <thread>
#include <vector>

// ---------------------------------------------------------------------------
// 1) Chase-Lev 工作窃取双端队列：本地线程在 tail(bottom) 端 push/pop，
//    窃取线程在 head(top) 端 steal。8 字节的 T 在 x86-64 上天然无锁。
// ---------------------------------------------------------------------------
struct Range { uint32_t lo, hi; };                  // 一件工作 = 一个待排序区间
static_assert(std::atomic<Range>::is_always_lock_free, "Range must be lock-free");

template <typename T>
class ChaseLevDeque {
public:
    explicit ChaseLevDeque(size_t log_cap = 10) : top_(0), bottom_(0) {
        array_.store(new CircularArray(log_cap), std::memory_order_relaxed);
    }
    ~ChaseLevDeque() { delete array_.load(); for (auto* a : retired_) delete a; }

    // 仅由 owner 线程调用：向 tail 端压入
    void push(T v) {
        size_t b = bottom_.load(std::memory_order_relaxed);
        size_t t = top_.load(std::memory_order_acquire);
        CircularArray* a = array_.load(std::memory_order_relaxed);
        if (b - t > a->capacity() - 1) {                       // 队列满：扩容
            a = a->grow(t, b);
            array_.store(a, std::memory_order_release);
        }
        a->store(b, v);
        std::atomic_thread_fence(std::memory_order_release);
        bottom_.store(b + 1, std::memory_order_relaxed);
    }

    // 仅由 owner 线程调用：从 tail 端弹出（LIFO，保局部性）
    bool pop(T& out) {
        size_t b = bottom_.load(std::memory_order_relaxed);
        if (b == 0) return false;                              // 空
        CircularArray* a = array_.load(std::memory_order_relaxed);
        b -= 1;
        bottom_.store(b, std::memory_order_relaxed);
        std::atomic_thread_fence(std::memory_order_seq_cst);
        size_t t = top_.load(std::memory_order_relaxed);
        if (t <= b) {
            out = a->load(b);
            if (t == b) {                                      // 只剩一个：必须与窃取者竞争
                if (!top_.compare_exchange_strong(t, t + 1, std::memory_order_seq_cst,
                                                  std::memory_order_relaxed)) {
                    bottom_.store(b + 1, std::memory_order_relaxed);
                    return false;                              // 被别人偷走了
                }
                bottom_.store(b + 1, std::memory_order_relaxed);
            }
            return true;
        }
        bottom_.store(b + 1, std::memory_order_relaxed);
        return false;
    }

    // 由任意线程调用（窃取者）：从 head 端取走一件工作
    bool steal(T& out) {
        size_t t = top_.load(std::memory_order_acquire);
```

```
        std::atomic_thread_fence(std::memory_order_seq_cst);
        size_t b = bottom_.load(std::memory_order_acquire);
        if (t < b) {
            CircularArray* a = array_.load(std::memory_order_relaxed);
            out = a->load(t);
            if (!top_.compare_exchange_strong(t, t + 1, std::memory_order_seq_cst,
                                              std::memory_order_relaxed))
                return false;                                   // 竞争失败：本次窃取放弃
            return true;
        }
        return false;
    }

private:
    struct CircularArray {                                      // 大小为 2 的幂，环形索引
        size_t cap, mask, log_cap;
        std::unique_ptr<std::atomic<T>[]> buf;
        explicit CircularArray(size_t lc)
            : cap(size_t(1) << lc), mask(cap - 1), log_cap(lc),
              buf(new std::atomic<T>[cap]) {}
        size_t capacity() const { return cap; }
        T    load(size_t i) const { return buf[i & mask].load(std::memory_order_relaxed); }
        void store(size_t i, T v) { buf[i & mask].store(v, std::memory_order_relaxed); }
        CircularArray* grow(size_t t, size_t b) const {
            auto* na = new CircularArray(log_cap + 1);
            for (size_t i = t; i < b; ++i) na->store(i, load(i));
            return na;
        }
    };

    alignas(64) std::atomic<size_t> top_;                       // 窃取端（head）
    alignas(64) std::atomic<size_t> bottom_;                    // 本地端（tail）
    std::atomic<CircularArray*> array_;
    std::vector<CircularArray*> retired_;                       // 旧数组延迟回收（此处只到退出时才释放）
};

// ------------------------------------------------------------------------
// 2）迷你运行时：每线程一个 deque + 全局未完成任务计数（用于判定终止）
// ------------------------------------------------------------------------
static int*   g_base = nullptr;                                 // 待排序数组基址
static long   g_N    = 0;
static const long CUTOFF = 8192;                                // 串行省略阈值

static std::vector<std::unique_ptr<ChaseLevDeque<Range>>> g_deques;
static std::atomic<long> g_unfinished{0};                       // 排队中 + 正在执行的任务数
static std::atomic<long> g_steal_attempts{0}, g_steal_success{0}, g_tasks_run{0};

static int* hoare_partition(int* begin, int* end) {
    int* mid = begin + (end - begin) / 2;
    if (*mid < *begin)       std::swap(*mid, *begin);
    if (*(end - 1) < *begin) std::swap(*(end - 1), *begin);
    if (*(end - 1) < *mid)   std::swap(*(end - 1), *mid);
    const int pivot = *mid;
    int* i = begin - 1; int* j = end;
    while (true) {
        do { ++i; } while (*i < pivot);
        do { --j; } while (*j > pivot);
        if (i >= j) return j + 1;
        std::swap(*i, *j);
    }
}

// 执行一件工作：左半就地继续（continuation 留在本地），右半作为新任务压入本地队列
static void run_task(Range r, int tid) {
    int* begin = g_base + r.lo;
    int* end   = g_base + r.hi;
    if (end - begin <= CUTOFF) { std::sort(begin, end); return; }
    int* mid = hoare_partition(begin, end);
    Range right{(uint32_t)(mid - g_base), r.hi};
    g_unfinished.fetch_add(1, std::memory_order_relaxed);       // 新任务入账
    g_deques[tid]->push(right);                                 // 可被其它线程 STEAL
    run_task(Range{r.lo, (uint32_t)(mid - g_base)}, tid);       // 派生的"子"先跑
```

```
}

static void worker(int tid) {
    Range r{};
    while (true) {
        if (g_deques[tid]->pop(r)) {                          // 1) 先看自己的队列
            run_task(r, tid); g_tasks_run.fetch_add(1, std::memory_order_relaxed);
            g_unfinished.fetch_sub(1, std::memory_order_relaxed);
            continue;
        }
        const int T = (int)g_deques.size();
        bool got = false;
        for (int k = 1; k < T && !got; ++k) {                 // 2) 随机挑受害者去窃取
            int victim = (tid + 1 + (int)((unsigned)(tid * 7919 + k * 104729) % (T - 1))) % T;
            if (victim == tid) continue;
            g_steal_attempts.fetch_add(1, std::memory_order_relaxed);
            if (g_deques[victim]->steal(r)) {
                g_steal_success.fetch_add(1, std::memory_order_relaxed);
                run_task(r, tid); g_tasks_run.fetch_add(1, std::memory_order_relaxed);
                g_unfinished.fetch_sub(1, std::memory_order_relaxed);
                got = true;
            }
        }
        if (got) continue;
        if (g_unfinished.load(std::memory_order_acquire) == 0) break;   // 3) 全局无任务：退出
        std::this_thread::yield();                            // 已有任务在途，稍后重试
    }
}

static bool is_sorted(const int* a, long n) {
    for (long i = 1; i < n; ++i) if (a[i - 1] > a[i]) return false;
    return true;
}

int main(int argc, char** argv) {
    g_N = (argc > 1) ? atol(argv[1]) : 20000000L;
    const int T = (argc > 2) ? atoi(argv[2]) : 8;
    std::vector<int> a(g_N), ref;

    std::mt19937 rng(2024);
    for (long i = 0; i < g_N; ++i) a[i] = (int)rng();
    ref = a;

    auto t0 = std::chrono::steady_clock::now();
    std::sort(ref.begin(), ref.end());
    auto t1 = std::chrono::steady_clock::now();
    const double serial_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    g_base = a.data();
    for (int t = 0; t < T; ++t) g_deques.emplace_back(new ChaseLevDeque<Range>(10));
    g_unfinished.store(1, std::memory_order_relaxed);          // 根任务
    g_deques[0]->push(Range{0, (uint32_t)g_N});

    std::atomic<int> ready{0}; std::atomic<bool> go{false};
    std::vector<std::thread> th;
    for (int t = 0; t < T; ++t)
        th.emplace_back([&, t] {
            ready.fetch_add(1, std::memory_order_release);
            while (!go.load(std::memory_order_acquire)) {}
            worker(t);
        });
    while (ready.load(std::memory_order_acquire) != T) {}
    t0 = std::chrono::steady_clock::now();
    go.store(true, std::memory_order_release);
    for (auto& x : th) x.join();
    t1 = std::chrono::steady_clock::now();
    const double par_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    std::printf("N=%ld T=%d  serial=%.1f ms  work-stealing=%.1f ms  speedup=%.2fx  sorted=%s\n",
                g_N, T, serial_ms, par_ms, serial_ms / par_ms, is_sorted(a.data(), g_N) ? "yes" : "NO");
    std::printf("  tasks run=%ld  steals attempted=%ld  succeeded=%ld  (成功率 %.1f%%)\n",
                g_tasks_run.load(), g_steal_attempts.load(), g_steal_success.load(),
```

```
                100.0 * (double)g_steal_success.load() / (double)(g_steal_attempts.load() ? g_steal_attempts.load() : 1));
    return is_sorted(a.data(), g_N) ? 0 : 1;
}
```

实测（N = 3,000,000，`CUTOFF = 8192`，每个配置取 5 次最小值；所有运行都通过了 `is_sorted` 校验）：

线程数 T	1	2	4	8
时间 (ms)	216.2	113.2	62.6	38.2
相对 1 线程	1.00x	1.91x	3.45x	5.66x
8 线程下的窃取统计	—	—	—	执行任务 645 件；窃取尝试 ~336,000 次；成功 ~150 次（成功率 0.04%）

【代码做什么？】

1. **Chase–Lev dequeue**（`ChaseLevDeque<T>`）：一个"双端队列"，**owner** 在 `bottom`（tail）端 `push` / `pop`（LIFO，取最近压入的任务 → 深度优先 → 局部性好），**窃取者**在 `top`（head）端 `steal`（FIFO，取最早压入的任务 → 标的是调用树里更大的一块）。缓冲区是一个**大小为 2 的幂的环形数组**，用 `top` / `bottom` 两个 `size_t` 索引，索引按位与（`i & mask`）取模；队列满时 `grow()` 申请翻倍的数组并把 `[top, bottom)` 的元素搬过去（旧数组**不立即释放**，因为可能还有窃取者正在读它——这是工作窃取里经典的"内存回收"难题）。
2. **消除伪共享**：`top_` 与 `bottom_` 都用 `alignas(64)` 对齐——owner 高频写 `bottom_`，窃取者高频读 `top_`，若两者在同一缓存行就会互相失效（下一讲会专门讲这个坑）。
3. **迷你运行时**：每线程一个 dequeue；`g_unfinished` 统计"排队中 + 正在执行"的任务总数，是**终止检测**的依据；worker 主循环三步走：**先 pop 本地 → 本地空则随机挑受害者 steal → 都没有且全局无在途任务则退出**，否则 `yield()` 后重试。
4. **任务拆分体现"先跑子任务"（continuation stealing 的等价写法）**：`run_task` 对区间做 Hoare 划分，把**右半**压入**本线程**的 dequeue（它就是"可被偷的连续体"），然后**就地递归左半**（"子任务先跑"）；小于 `CUTOFF` 时直接 `std::sort`。
5. **计数不变式**：根任务使 `g_unfinished = 1`；每压入一件新任务 `+1`，每完成一件任务 `-1`（出队与窃取都不改变计数，因为任务只是从"排队"变成"执行"）。因此 `g_unfinished == 0` 当且仅当所有任务（含正在执行的）都已完成，worker 可以安全退出。
6. `main` 用一个起跑线（`ready` / `go`）让所有 worker 同时开始，并在**计时区间之外**完成 `std::sort` 串行基线、数组初始化与线程创建；结束时校验有序性。

【并行机制与性能解说】

- **硬件上如何并行**：T 个 worker 线程各自绑定一个 dequeue（每个 dequeue 的头部/尾部各占一条缓存行）。**绝大多数操作是 owner 对自己 dequeue 尾部的普通 load/store**——`push` 走的是 relaxed store + release fence，`pop` 只剩一次可能的 CAS 比较；**没有原子 RMW、没有锁**。只有当某个 worker 空手时才发起 `steal`：一次 acquire load + 一次 `compare_exchange`，并伴随**缓存行所有权从受害者到窃取者的迁移**。
- **Work / Span / 并行度**：与 §3.1 的快排完全同构。取 $n = 3\times10^6$、$C = 8192$：划分层数 $\log_2(n/C) \approx 8.5$，划分工作量 $\approx 3\times10^6 \times 8.5 \approx 2.6\times10^7$；叶子排序工作量 $\approx (n/C)\cdot C\cdot\log_2 C \approx 366 \times 8192 \times 13 \approx 3.9\times10^7$，所以 **$W \approx 6.4\times10^7$ 元素操作**。**Span $D \approx 2n \approx 6\times10^6$ 元素操作**（沿"每次取一半"的划分链求和）。**并行度 $W/D \approx 11$：8 个线程已经逼近这个上限**，因此 5.66x（效率 71%）已是相当好的结果，而不是"调度没做好"。任务总数 = 内部节点 + 叶子 ≈ 365 + 366 ≈ 731（实测同一量级：**645 件任务被执行**），所以 **parallel slack = 645/8 ≈ 80，远大于讲义建议的 ~8**，负载均衡不是瓶颈。这也是为什么 8 线程能拿到 5.66x：任务足够粗（叶子是 8192 个元素的排序，约 100 µs 量级），而**调度开销只在 150 次成功窃取里付出**。
- **窃取成本被摊薄（讲义的核心论断在这里被量化）**：150 次成功窃取 × ~1 µs ≈ **0.15 ms**，相对 38.2 ms 的运行时间只有 **0.4%**——"只要偷走的是大块工作，窃取就很稀有"得到验证。对比一下：如果叶子任务不是 8192 个元素而是 1 个元素（成本 ~2 ns），那么每次窃取的 1 µs 开销相当于 **500 倍的窃取/计算比**，整个系统会被窃取路径压垮。**这就是"任务粒度不能太细"在窃取式调度器里的具体形态**。
- **本实现写得很"贪"的窃取路径暴露了另一个成本**：`worker` 在没活干时会**对全部 T−1 个受害者各试一次**，然后 `yield()` 再重试——实测 **336,000 次窃取尝试只换来 150 次成功（成功率 0.04%）**。每次尝试至少两条原子 load + 一次 CAS 尝试 + 一次 fence，粗估 336,000 × ~50 ns ≈ **17 ms 的 CPU 时间**（摊到 8 个线程约 2 ms，占 38 ms 的 ~5%），而且这些访问在**受害者 dequeue 的头部缓存行**上，会干扰受害者的本地操作。生产级调度器（Cilk、TBB）对此的处理是**指数退避 + 随机化 + 适时睡眠**；这解释了 CS149 讲到的"greedy join"必须配一个**高效的**窃取路径，而不是无脑自旋。
- **终止检测不是免费的**：本示例用"全局未完成任务计数 + 自旋/让出"实现终止检测，这是讲义列出的四个实现挑战之一（"如何检测程序终止？"）。真实系统（Cilk 的 join counter + 全局空闲位、Java ForkJoinPool 的 `quiescence` 检测）都要处理"某个线程暂时没活但系统里还有活"与"真的全干完了"的区分。

3.4 示例 4：调度策略的确定性仿真（长任务优先 vs 粒度）

```cpp
// sched_sim.cpp —— 调度策略仿真：任务顺序 / 静态分配 / 粒度 对 makespan 的影响
// 编译: g++ -O2 -std=c++17 sched_sim.cpp -o sched_sim
// 运行: ./sched_sim
#include <algorithm>
#include <cstdio>
#include <numeric>
#include <queue>
#include <vector>

struct Row { double makespan, ideal, imbalance, serial_frac, amdahl_bound; };

// 动态分配：P 个工人，从共享队列按给定顺序取活（谁的活干完谁再取）
static Row simulate_queue(const std::vector<int>& order, int P) {
    std::priority_queue<double, std::vector<double>, std::greater<double>> free_at;
    for (int i = 0; i < P; ++i) free_at.push(0.0);
    for (int c : order) { double t = free_at.top(); free_at.pop(); free_at.push(t + c); }
    double makespan = 0, total = 0;
    for (int i = 0; i < P; ++i) { makespan = std::max(makespan, free_at.top()); free_at.pop(); }
    for (int c : order) total += c;
    Row r;
    r.makespan = makespan;
    r.ideal = total / P;
    r.imbalance = makespan / r.ideal;
    r.serial_frac = (makespan - r.ideal) / makespan;          // 尾部"slop"占运行时间的比例
    r.amdahl_bound = 1.0 / (r.serial_frac + (1.0 - r.serial_frac) / P);
    return r;
}

// 静态分配：第 i 个工人拿 order[i], order[i+P], ...（blocked）
static Row simulate_static(const std::vector<int>& order, int P) {
    std::vector<double> load(P, 0.0);
    for (size_t i = 0; i < order.size(); ++i) load[i % P] += order[i];
    double makespan = *std::max_element(load.begin(), load.end());
    double total = std::accumulate(load.begin(), load.end(), 0.0);
    Row r;
    r.makespan = makespan; r.ideal = total / P; r.imbalance = makespan / r.ideal;
    r.serial_frac = (makespan - r.ideal) / makespan;
    r.amdahl_bound = 1.0 / (r.serial_frac + (1.0 - r.serial_frac) / P);
    return r;
}

static void report(const char* name, const Row& r, int P) {
    std::printf("%-34s | %8.1f | %8.1f | %8.3f | %9.3f | %8.2fx\n",
                name, r.makespan, r.ideal, r.imbalance, r.serial_frac, r.amdahl_bound);
}

int main() {
    const int P = 4;                                          // 4 个执行上下文
    // 16 件工作，成本不可预测：有一根"长杆"（60），另有一件中等（24），其余为 8
    std::vector<int> cost = {8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 24, 60};

    std::vector<int> as_given = cost;                                   // 长杆排最后
    std::vector<int> lpt = cost;
    std::sort(lpt.begin(), lpt.end(), std::greater<int>());             // 长任务优先 (LPT)
    std::vector<int> chunked;                                            // 粗化粒度：拆成半尺寸任务
    for (int c : cost) { chunked.push_back(c / 2); chunked.push_back(c - c / 2); }
    std::vector<int> long_first_chunked = chunked;
    std::sort(long_first_chunked.begin(), long_first_chunked.end(), std::greater<int>());

    std::printf("P=%d, 16 tasks, total work = %d, ideal makespan = %.1f\n\n",
                P, std::accumulate(cost.begin(), cost.end(), 0),
                std::accumulate(cost.begin(), cost.end(), 0) / (double)P);
    std::printf("%-34s | %8s | %8s | %8s | %9s | %8s\n",
                "scheduling policy", "makespan", "ideal", "max/avg", "serial S", "Amdahl");
    std::printf("-----------------------------------+----------+----------+----------+-----------+---------\n");
    report("static blocked, long task last", simulate_static(as_given, P), P);
    report("dynamic queue, long task last", simulate_queue(as_given, P), P);
    report("dynamic queue, long task first (LPT)", simulate_queue(lpt, P), P);
    report("dynamic queue, halves, long last", simulate_queue(chunked, P), P);
    report("dynamic queue, halves, long first", simulate_queue(long_first_chunked, P), P);
```

```
    return 0;
}
```

实测输出（确定性，无噪声）：

表 4：同一组 16 件工作在 4 个执行上下文下的 makespan（理想 49.0）

调度策略	makespan	理想值	max/平均	串行段 S	Amdahl 上限
静态 blocked，长任务最后	84.0	49.0	1.714	0.417	1.78x
动态队列，长任务最后	84.0	49.0	1.714	0.417	1.78x
动态队列，长任务优先（LPT）	60.0	49.0	1.224	0.183	2.58x
动态队列，对半拆分，长任务最后	58.0	49.0	1.184	0.155	2.73x
动态队列，对半拆分，长任务优先	**50.0**	49.0	**1.020**	**0.020**	**3.77x**

【代码做什么？】

1. `simulate_queue` 用一个小顶堆模拟**动态分配**：堆里是 P 个工人的"下次空闲时刻"，每来一件工作就把它交给**最早空闲**的工人（这正是"谁的活干完谁再取"的等价模型），最后取所有工人完成时刻的最大值作为 **makespan**。
2. `simulate_static` 模拟**静态 blocked 分配**：第 i 个工人固定拿第 i、i+P、i+2P… 件工作，各人的负载直接求和。
3. 两者都输出**理想时间（总工作 / P）**、**失衡比 makespan/ideal**、折算出的**串行段 S = (makespan − ideal)/makespan**，以及 **Amdahl 加速比上限 1/(S + (1−S)/P)**。
4. `main` 对同一组成本构造五种调度方案：按输入顺序（长杆最后）、LPT（长任务优先）、把每件工作对半拆分、拆分后再 LPT——从而把**"排序"**与**"拆细"**两个手段的影响分离开。

【并行机制与性能解说】

- **Work / Span / 并行度**：总工作量 `W = 196`，关键路径由最长的任务链决定（这里是"长杆"60 加上它前面必须串行的部分）。**并行度 = W / Span**：长杆最后且必须串行时 span 至少包含 60 单位，`W/Span ≈ 196/84 ≈ 2.3`——**并行度只有 2.3，却有 4 个核**，多出来的核只能空转，这就是 makespan 84 的来源。LPT 并不改变 W，但它**让长杆尽早开始**，使 span 不再"叠加在尾部"；对半拆分则**同时降低 span**（60 → 30）**并提高 W**（196 → 196，这里恰好不变；真实系统里拆分通常会增加管理开销）。
- **"动态分配 ≠ 负载均衡"**：表 4 前两行完全相同（84 对 84）。**动态分配只是"有空就取活"，它并不理解任务的长短**；当长杆排在最后时，动态与静态一样无能为力。这是讲义把"更聪明的调度"单列一节的原因。
- **瓶颈是 Span，不是 Work**：最优方案（拆分 + LPT）的 makespan 50 已经贴着理想值 49（效率 98%），此时再优化调度策略的收益趋近于零——剩下的 1 个单位就是"16 件工作无法被 4 整除"的颗粒度残差。反过来说，**当 imbalance ≈ 1.0 时继续调调度器是浪费精力，应该去优化局部性或减少额外工作**（下一讲的主题）。

---

4. 性能模型与复杂度分析

4.1 Work–Span 模型与贪心调度界（本讲分析工具的基础）

- **定义**：把并行计算看成一张 DAG。
- **Work（W，总工作量）**：所有节点的工作量之和 = 单个处理器串行执行的时间 T_1；
- **Span（S，关键路径长度）**：DAG 中最长依赖路径的工作量 = 无限多处理器下的执行时间 T_∞；
- **并行度（Parallelism）= W / S**：**机器再快也无法突破的平均并行上限**。
- **两个必须记住的界**（Graham / Brent）：
- 任何调度器都有 $T_P \geq \max(W/P, S)$；
- **贪婪调度器（greedy scheduler，只要有可运行任务就绝不空闲）**满足 $T_P \leq W/P + S$。 第二个不等式是"贪婪"类调度器的**性能保证**：这正是 Cilk 的 greedy join scheduling 与工作窃取的价值所在——**它把"调度质量"问题变成了"任务粒度/额外工作"问题**。
- **对具体算法的应用**：分治算法 $W(n) = a\cdot W(n/b) + \theta(n^d)$，$S(n) = S(n/b) + \theta(n^d)$：

算法	Work W	Span S	并行度 W/S	结论
并行快排（本次实现）	Θ(n log n)	Θ(n)	**Θ(log n)**	并行度只有对数级，是"核多了也快不了"的根源

算法	Work W	Span S	并行度 W/S	结论
并行归约（树形）	Θ(n)	Θ(log n)	Θ(n/log n)	极好的并行度，但每级同步/带宽受限
并行矩阵乘（分块）	$\Theta(n^3)$	Θ(n)	$\Theta(n^2)$	并行度极高（第 6 讲）
并行扫描（scan）	Θ(n)	Θ(log n)	Θ(n/log n)	但 Work 常数大（Blelloch）

4.2 数值算例 A：5% 串行工作如何锁死 64 核

假设参数：P 个处理器；总工作折算为串行时间 100 单位，其中**串行段占 5%**（S = 0.05）。 按 Amdahl：`T_P = S + (1-S)/P`，`Speedup(P) = 1/(S + (1-S)/P)`。

P	4	8	16	64	256	1024
S = 0.05 的上限	3.48x	5.93x	9.14x	**15.42x**	18.62x	19.64x
S = 0.20 的上限（CS149 版算例）	2.50x	3.33x	4.00x	4.71x	4.92x	4.98x
S = 0.001 的上限	3.99x	7.94x	15.76x	60.21x	203.98x	506.18x

读法：S = 0.05 时，64 核只能拿到 15.42x（理想 64x 的 24%），从 256 核加到 1024 核只多 5%（18.62x → 19.64x），**渐近上限就是 1/S = 20x**；S = 0.20 时，核数超过 16 之后**几乎完全没有收益**（16 核 4.00x → 1024 核 4.98x，渐近上限 5x）。所以讲义反复强调：**"只有很小的负载不均就能显著限制最大加速比"**，而"我的方案可扩展"必须限定规模。

4.3 数值算例 B：并行冗余度（parallel slack）该取多少

假设：8 个执行上下文；任务的执行时间**不可预测**，且负载不均的代价按 Amdahl 折算。定义 **slack = 独立工作件数 / 机器的并行执行能力**。

- **下界（填满机器）**：至少要有 ⩾ P 件工作，否则总有核闲着；考虑成本波动，讲义给出**实践值 ~8**，即**至少 8P = 64 件独立工作**。
- **上界（不要细到开销失控）**：任务更细 → 需要更多次调度/同步。给定"每件任务的调度开销占比 ⩽ ε"的要求，任务成本 `t_task` 与单次管理成本 `c_mgmt` 必须满足 `c_mgmt / t_task ≤ ε`。
- **算例**：`c_mgmt = 494 ns`（§3.2 实测的争用原子取号成本），要求调度开销 ⩽ 5%，则单件任务至少要做 `494/0.05 ≈ 9.9 µs` 的工作。若单件工作只有 22.9 ns（LIGHT 负载），**chunk 必须 ⩾ 9.9 µs / 22.9 ns ≈ 432 件**——与表 2 实测的"256–4096 平台区"吻合（chunk=64 时已比最优慢 73%，chunk=1 时慢 143 倍）。
- **结论（可扩展性的两个边界）**：`独立工作件数 N_task 必须落在窗口内： N_task ≥ slack × P （下界：负载均衡 / 填满机器） N_task ≤ W_total / (c_mgmt/ε) （上界：调度开销不能吃掉收益） 且 N_task ≈ 数十倍 P 时，成本波动才能被吸收（表 3：chunk=1024 → 只有 2 件工作 → max/avg = 4.0）`

4.4 数值算例 C：粒度选择公式与实测对照

模型：设工作总成本 `W`（单线程时间），每件任务平均成本 `t = W/M`，一次"取活"的全局串行化成本 `c`，chunk = k 件任务，P 个处理器。

```
总运行时间估算：   T ≈ max( W/P , (M/k)·c )   +  尾部失衡
                    └ 并行部分 ┘   └ 取号串行链 ┘

下界（同步主导）：   (M/k)·c  ≤  W/P     ⇒   k ≥ c·P/t
上界（失衡主导）：   件数 M/k ≥ slack·P  ⇒   k ≤ M/(slack·P)

代入 §3.2 的 LIGHT 负载实测参数：W ≈ 4.58 ms (= 0.573 ms × 8 线程)，M = 200000，
t = W/M = 22.9 ns，c ≈ 494 ns，P = 8，slack = 8：
     k ≥ c·P/t = 494 ns × 8 / 22.9 ns ≈ 173
     k ≤ M/(slack·P) = 200000/64 ≈ 3125
     窗口 ≈ [173, 3125]     实测平台区 = [256, 4096]，最优 chunk = 1024 (0.573 ms)
预测与实测一致；预测下界 173 略低于实测下界 256，是因为模型把"取号"当成完全串行，
实际取号与计算部分重叠，因此真实下界更靠右一点。

代入 HEAVY 负载：t = W/M ≈ (9.47 ms × 8)/2048 ≈ 37.0 µs，c ≈ 494 ns：
     k ≥ 494 ns × 8 / 37.0 µs ≈ 0.11    ⇒ k = 1 就已足够（实测 chunk 1…256 全平）
     k ≤ 2048/64 = 32                   ⇒ 实测 chunk=1024 只剩 2 件 → 失衡 4.0，时间 ×4
```

结论：最优粒度不是"越细越好"或"越粗越好"，而是由 `c·P/t` 与 `slack·P` 夹出的窗口。工作越贵（t 越大）→ 窗口越宽、可以越粗；同步越贵（c 越大）→ 窗口右移、必须更粗。

4.5 数值算例 D：窃取开销必须被摊薄

假设：一次成功窃取的成本 `c_steal ≈ 1 μs`（跨 socket 的原子 CAS + 缓存行迁移 + 队列元数据更新），窃取到的任务成本 `t_task`。

- **窃取开销占比** `= c_steal / t_task`。
- 若偷到的是**叶子任务**（§3.3 中 8192 个元素的排序，实测约 100 μs）：占比 **1%**，可接受。
- 若偷到的是**单个元素的工作**（~2 ns）：占比 **500 倍**——系统被窃取路径彻底压垮。
- **实测印证**：§3.3 中 150 次成功窃取 × 1 μs = 0.15 ms，占 38.2 ms 的 **0.4%**；而 336,000 次**失败的窃取尝试**（每次约两条原子读 + 一次 CAS 尝试 + fence，粗估 ~50 ns）合计约 17 ms CPU 时间——若这些周期全部从有用计算中"偷"走，相当于整体多出约 2 ms（占 8 线程 38 ms 运行时间的 **~5%**，实际影响更小，因为空闲线程本来也没有有用工作可做，但**它会争用内存/相干性资源并干扰受害者的本地快路径**）。两条数字合起来给出本讲最重要的工程准则：`成功窃取要"少而大" → 从 dequeue 头部拿（调用树里更大的那块） 失败尝试要"便宜" → 退避 + 随机化 + 适时让出/睡眠，而不是无脑自旋`
- **摊销公式**：若把任务粒度缩小为 1/g，则为了维持同样的窃取开销占比，窃取次数必须同比例减少，而这与"任务数增多 → 需要更多次再平衡"直接矛盾。这就是**细粒度 fork–join 需要高效（无锁）窃取路径**的根本原因，也解释了 §2.6 里"偷头部"的三条理由。

4.6 数值算例 E：内存带宽与算术强度（为什么"分配得好"仍然可能跑不快）

即使调度完美，**内存层次**仍会给并行程序设一个上限。这是下一讲的主题，但本讲的两个示例已经撞上了它：

- **纯带宽算例**：一个 **256 MB** 的数组，机器可持续带宽 **20 GB/s**，则**单次遍历至少需要** `256 MB / 20 GB/s = 12.8 ms`；若要**读+写**（例如 `B[i] = f(A[i])`），流量 512 MB → **25.6 ms**。**这段时间与线程数无关**：8 个线程、64 个线程都一样，因为带宽是共享资源。上面 §3.1 的 8 线程快排只有 3.65x（而不是 8x），正是这类"共享资源饱和"的形态：4 → 8 线程只涨 1.27x。
- **算术强度/屋顶线（Roofline）**：若某内核每字节访存做 `AI` 次浮点运算，则 `性能上限 = min(峰值算力, AI × 带宽)`。
- 算例：16 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA）= **768 GFLOPS** 峰值；带宽 100 GB/s、AI = 0.25 FLOP/B → 内存上限 **25 GFLOPS**，即峰值算力只能兑现 **3%**。要让计算受限，需要 `AI ≥ 768/100 = 7.7 FLOP/B`。
- 对**调度策略的启示**：把一个内存受限的内核切成更多任务、交给更多线程，**并不会**缩短时间（带宽不变），只会增加调度开销；此时正确的优化方向是提高**数据复用（缓存分块）**，而不是继续调调度器。**"先把瓶颈找对，再决定要不要动调度"**——这是 TIP #1 之后最重要的一条纪律。
- **临界区/共享状态的带宽算例**：一个 8 字节计数器被 8 个线程各做 10^6 次 RMW（x86 的 `lock` 前缀指令在争用时表现为**串行化的缓存行搬运**，约 **0.3–0.5 μs/次**）→ 总时间 ≈ $8\times10^6 \times 0.4\ \mu s = 3.2\ s$：**一个 8 字节变量把 8 个核变成了一条串行流水线**。修法（下一讲 + 同步讲）：**分片累加（per-thread/per-socket 计数）+ 最后归约**，或**减少 RMW 频率**（chunk…就是本讲的内容！）。

4.7 表 5：本讲各方案的定量对照（综合前面所有算例）

方案	Work 变化	Span / 关键路径	主要开销来源	实测（本机 8 线程）	适用判据
静态 blocked/interleaved	不变（+索引运算）	由最长分片决定	近零	—	工作量与成本可预测
共享计数器 + chunk=1	不变，但串行化	**Span = 任务数 × c_sync**	争用原子（~494 ns/次）	99.5 ms（LIGHT，最差）	只有在 $t_{task} \gg c_{sync} \cdot P$ 时用
共享计数器 + chunk=1024	不变	Span 大幅缩短	取号 ~0.1 ms	**0.573 ms（LIGHT，最优）**	窗口 $[c \cdot P/t,\ M/(slack \cdot P)]$
共享队列 + 长任务优先	不变	尾部 slop 缩短	每次取活都同步	仿真：84 → 60（P=4）	成本有可预测性（或可估计）
分布式队列 + 工作窃取	不变	由任务图 span 决定	**仅窃取时**（~150 次成功窃取 = 0.4%）	38.2 ms / 5.66x（N=3M）	递归分解、任务边界动态生成
粒度再细分（对半拆）	**增加**（管理开销）	span 减半	更多调度事件	仿真：58 → 50（P=4）	长杆无法被顺序手段消除时

---

5. 关键要点

1. **本讲的全部内容都是"三个互相冲突的目标"之间的取舍**：负载均衡、减少通信、减少额外工作。**先写最简方案、测量、再只优化真正的瓶颈（TIP #1）**；"我的方案可扩展"必须限定在具体的机器规模与工作负载上。

2. **负载不均按 Amdahl 定律被放大**：串行段 S = 0.05 就能让 64 核只拿到 15.42x（理想 64x 的 24%），渐近上限 1/S = 20x；S = 0.2 时超过 16 核几乎没有收益（上限 5x）。因此"分配策略"不是工程细节，而是**加速比上限的直接决定因素**。
3. **静态 → 半静态 → 动态 → 工作窃取是一个连续谱，不是二选一**：能用先验知识就用（静态/半静态的开销近零）；只能运行时才知道就用动态；只有在任务边界由递归分解动态产生时，**每线程 dequeue + 随机窃取**才是对的工具。极限情形（系统全知）就是完全静态分配。
4. **任务粒度是核心旋钮，且最优值由公式给出**：下界来自同步成本（`k ≥ c·P/t`），上界来自负载均衡（`k ≤ M/(slack·P)`），实践上要求**任务数远多于处理器数（slack ~8）**，同时每件任务的调度开销远小于任务本身。实测：同一负载下 chunk=1 与 chunk=1024 相差 **143 倍**；而同一 chunk 在"贵任务"负载下可能毫无差别——**因为同步开销是争用强度的函数，不是常数**。
5. **工作窃取的精髓在于"本地快、窃取少而大"**：owner 在自己的 dequeue 尾部 push/pop（LIFO、无锁、保局部性），窃取者从头部拿（减少争用、偷到调用树里更大的块、最大化局部性），受害者随机选择；**成功的窃取很稀有**（实测 150 次 / 645 件任务，开销 0.4%），但**失败的尝试必须便宜**（实测 33.6 万次尝试 ≈ 5% 运行时间）——这正是 Cilk 用 continuation stealing + greedy join + descriptor 记账，把同步开销压到"只在窃取时出现"的原因。

---

6. 常见陷阱与注意事项

- **把 `cilk_spawn`（或 `#pragma omp task`）当成"工作已经并行执行了"**：它只是**许可**，不是命令。若独立工作总量不足以填满机器（缺 parallel slack），或者任务图事实上是一条链，程序照样串行跑满全程；**在 1 个线程上跑一遍，确认"没有窃取时执行顺序 = 串行顺序"，是验证 fork–join 代码正确性的第一道关**。
- **任务粒度太细**：把 O(1) 的工作用 `cilk_spawn` / `task` 包起来，调度与同步开销立刻压过收益——实测同一负载下 chunk=1 比 chunk=1024 慢 **143 倍**，程序从"并行"退化为"以 Span 为运行时间的串行程序"。**任务至少要"贵"到能摊薄一次取活成本（几百 ns 到几 μs）**。
- **反过来把粒度做得太粗 / 把长任务放在最后**：只剩 2 件工作分给 8 个线程时 `max_work/avg_work = 4.0`，时间 ×4；只剩 1 件时 `= 8.0`，时间 ×7.9（表 3）。**"动态分配"不等于"负载均衡"**：它只会让空着的工人去取活，不会把一件巨长的活切开——长任务必须**优先派发（LPT）**或**预先拆分**。
- **假定同步开销是常数**：同一个原子取号在"8 个线程饱和争用"时约 **494 ns**，在"每 37 μs 才取一次号"时回落到 **50–100 ns**。用常数去算 Amdahl 会得出错误的粒度结论；**必须用你的工作负载在你的机器上实测这条曲线**（这也解释了为什么表 2 与表 3 的最优 chunk 相差三个数量级）。
- **伪共享（false sharing）**：多个线程的独立变量落在**同一条缓存行**上（例如每线程一个计数器放在同一个数组里、或 `alignas(64)` 漏掉 dequeue 的 `top_` / `bottom_`），每次写都让对方的核心缓存失效，性能可以比串行还差。**规则：凡是"不同线程高频写"的数据，都要按缓存行（通常 64 字节）隔离或按线程私有化**；本讲示例中的 `alignas(64) std::atomic<size_t> top_/bottom_` 就是这个规则的最小示例。
- **把有依赖的工作塞进工作队列**：队列里放"互相有依赖但未声明"的工作是**数据竞争**，且结果不可复现（依赖时序）。要用讲义中的 `enqueue_task(bar, foo_handle)` 形式显式声明依赖，或确保任务真正独立——**当任务之间有依赖时，运行时间由关键路径（span）而非总工作量决定**，此时加线程数毫无帮助。
- **过度自旋/忙等**：本讲示例中 336,000 次窃取尝试只成功 150 次（成功率 0.04%），浪费约 17 ms 的 CPU 时间（粗估相当于整体运行时间的 ~5%），还会干扰受害者本地队列的缓存行。**空闲等待要用退避（backoff）、让出（yield）或阻塞**，而不是无脑重试；终止检测也要避免"全系统轮询一个全局变量"成为新的争用点。

---

7. 思考题（带答案）

思考题 1：如果某个 Cilk 实现把 `cilk_spawn foo()` 实现成普通函数调用，它还正确吗？这对我们理解"抽象 vs 实现"有什么意义？

【答案】 正确。 `cilk_spawn` 的抽象语义只承诺一件事："**被派生的调用可以与调用者并发执行**"——它是一个**许可（permission）**，不是"必须并行执行"的命令。把 spawn 当成普通调用（即所谓的 **serial elision / 串行省略**：把 `cilk_spawn` 与 `cilk_sync` 直接删掉）是**一个合法的调度**（所有工作仍通过原来的依赖顺序完成，结果是确定的），因此该实现是正确的。它只是**性能上退化为串行**（加速比 = 1）。

这个问题的意义有三点： 1. **抽象与实现的分离**：程序文本只表达"哪些工作**可以**并行"，而"何时、由哪个线程执行"完全属于实现（调度器）的自由。Cilk 的所有示例代码都可以先在单线程下调试，正确性由调度无关性（determinacy）来保证。 2. **性能不是语义**：正确性与可扩展性是两个独立维度；"删掉 spawn 后跑出正确结果"不能说明程序并行得动。 3. **反过来说，`cilk_sync` 是硬约束**：它必须在所有派生调用完成后才返回，因此实现不能随意重排跨越同步点的工作——这也是 sync descriptor（spawn/done 计数）存在的原因。**如果你的程序语义依赖"某个 spawn 真的与 bar() 同时运行"（例如依赖竞态的读-改-写），那程序本身就是错的**，因为合法实现（串行省略）会给出不同结果。

思考题 2：给定机器 8 个执行上下文、实测一次争用原子取号成本约 500 ns。你的任务集有 100 万件工作，其中 90% 的成本是 100 ns、10% 的成本是 20 μs，件数多到足以让分配均匀。请给出 chunk 的合理取值并说明推导；再用表 2/表 3 的实测结论验证你的推理。

【答案】 用 §4.4 的两个约束夹出窗口。

1. **同步成本下界**：平均任务成本 `t = 0.9×100 ns + 0.1×20 μs = 90 ns + 2 μs = 2.09 μs`。要求一次取号开销占单件任务成本不超过 ε = 5%：`k ≥ c/(ε·t) = 500 ns / (0.05 × 2.09 μs) = 500/104.5 ≈ 4.8` → **k ⩾ 5**（取 k = 8 更稳妥，留出安全边际）。若不做"平均"而按**最便宜**的任务算（保守）：`k ≥ 500/(0.05×100) = 100`，这是"连便宜任务也不吃亏"的取值。
2. **负载均衡上界**：`k ≤ M/(slack·P) = 10⁶ / (8×8) = 15625`，且为了让那 10% 的昂贵任务（20 μs）**不要集中在少数 chunk 里造成尾部失衡**，chunk 越小越好（但受 1 约束）。
3. **结论**：取 **k ≈ 8—100**（推荐 **k = 64**：同步开销占比 `500/(64×2.09 μs) ≈ 0.4%`，同时仍有 `10⁶/64 ≈ 15625` 个 chunk，slack ≈ 1950 ≫ 8，足以吸收成本波动）。**注意 k=1 一定错**：它把 100 万次争用原子操作（≈ 0.5 s 的串行化时间）压到程序的关键路径上。
4. **与实测对照**：
5. 表 2（LIGHT，t = 22.9 ns）的窗口是 `[173, 3125]`，实测最优 chunk = 1024（0.573 ms），而 chunk = 1 时因为 20 万次 × 494 ns 的串行化，耗时 99.5 ms（**慢 143 倍**）——验证了"同步成本下界"这一侧。
6. 表 3（HEAVY，t = 37 μs）的下界只有 `k ≥ 0.11`，实测 chunk 在 1…256 之间**完全平坦**（差异 <1%），而一旦越过上界（chunk = 1024 → 只剩 2 件工作），失衡比飙到 4.0、时间 ×4 —— 验证了"负载均衡上界"这一侧。
7. 这道题的负载 `t = 2.09 μs` 介于两者之间，正好落在"两边都要照顾"的区域，所以答案是一个**区间**而不是一个数：**最优粒度取决于工作负载的成本分布与机器的争用特性，必须实测确认**（本讲的实测表就是最基本的测量方法）。

思考题 3：为什么工作窃取要"本地在尾部 push/pop、窃取者从头部偷"？如果改成窃取者也从尾部偷，会出现哪些具体问题？另外，为什么空闲线程要随机选受害者？

【答案】

（1）为什么 owner 用尾部、窃取者用头部？ – **减少争用**：两方访问**同一个 dequeue 的不同端**（各自的索引与缓存行位置不同），因此不需要对同一个元素做 CAS，也不需要同一把锁。若双方都动尾部，`bottom` 索引就成了双方争抢的**同一个热点缓存行**：owner 每次 push/pop 都要写它，窃取者每次偷也要改它，双方的核心缓存不停失效（cache line ping-pong），本地快路径（本应"几乎免费"）会退化成"每次都要走一次相干性协议"。 – **偷到更大的工作、摊薄窃取成本**：配合"先跑子任务（continuation stealing）"，**最近压入尾部的是最深、最小的调用树节点**，而**头部保留的是调用树里最早、最大的一块**（例如整个"剩下的迭代"或"半棵树"）。从头部偷 = 一次窃取的 1 μs 被后面几百微秒的工作摊薄（§4.5）；若从尾部偷，偷到的往往是 owner 马上就要做的**最小一块**，窃取立刻变得频繁而昂贵。 – **最大化局部性**：owner 沿深度优先在**自己那片调用树**里工作（尾部 LIFO），窃取者从头部拿走**一整棵子树**并在自己的栈里继续深度优先——两边各自"抱着一棵子树"跑，各自的访问模式与数据局部性都好。这就是讲义所说的"**in conjunction with run-child-first policy, local thread works on local part of call tree**"。

（2）若窃取者也从尾部偷，具体会出什么问题？ – **与 owner 抢同一件工作**：最坏情况下窃取者拿走了 owner 即将 pop 的那一个任务，owner 白跑一趟再去找活，双方反复"抢-让"，本地快路径彻底失效； – **数据竞争风险剧增**：两端同时修改同一个索引，Chase-Lev 的双索引协议（owner 写 `bottom`、thief 用 CAS 改 `top`）不再适用，必须退回"两端共享一把锁/一个 CAS"，也就丢掉了"高效无锁 dequeue"这一整块设计； – **粒度最优化被破坏**：偷到的常常是刚创建的最细碎的任务，窃取次数上升、每次收益下降，与"粒度不能太细"的原则直接冲突。

（3）为什么随机选受害者？ – **避免热点与同步**：如果所有空闲线程都去偷"同一个最忙的线程"（例如固定偷 0 号），那么该线程的 dequeue 头部就成了新的争用点，而且多个窃取者会**互相竞争同一个头部元素**（大量 CAS 失败白白浪费——§3.3 实测的成功率就只有 0.04%）； – **统计学上的均衡与可扩展性**：随机化把窃取请求**均匀散布**到所有 worker 上，符合 Cilk 的理论分析（随机窃取 + 贪婪策略下，期望运行时间接近 `W/P + O(S)`），也不需要任何全局协调或中心化的负载信息； – **实现代价低**：随机受害者只需要一个线程私有的伪随机数发生器加一次取模，不需要维护全局负载表（那又会引入共享状态与新争用）。实践中常配**指数退避**与"偷不到就先挑下一个/稍后再试"的策略，进一步压低无效尝试的比例。

Lecture 9: Performance Optimization II: Locality, Communication, Contention

1. 章节标题与概述

Lecture 9: Performance Optimization II: Locality, Communication, Contention（性能优化 II：局部性、通信与竞争）

- **本讲核心问题**：上一讲（Performance Optimization I）解决的是**怎样把工作合理地分配给 worker**（静态/动态分配、粒度、工作窃取）；本讲解决另一半问题——**怎样把通信的代价压到最低**。讲义开篇即点明："今天：最小化通信代价的策略。"要回答的具体问题是：一次通信到底贵在哪里（启动延迟、占用时间还是竞争）？哪些通信是算法固有的（inherent）、哪些是机器实现强加的（artifactual）？如何通过**改变工作分配、改变数据布局、改变遍历顺序（blocking）、融合循环（fusion）、共享数据（sharing）、增大消息粒度**来减少或隐藏通信？以及当多个 worker 同时争抢一个资源（锁、热点、带宽）时，性能为什么会崩塌？
- **涉及的主要硬件/软件机制**：
- **软件侧**：消息传递的阻塞（synchronous/blocking）与非阻塞（asynchronous，`Isend/Irecv` + `Wait/Test`）语义、ghost cell（影子单元）数据复制、死锁回避的配对收发顺序、循环融合（loop fusion）、分块/瓦片遍历（blocking/tiling）、仿射工作分解（1D blocked / 1D interleaved / 2D blocked assignment）、细粒度锁（per-cell lock）、分布式工作队列（distributed work queues + work stealing）、数据并行的排序式分桶（counting sort + `cell_starts/cell_ends`）。
- **硬件侧**：存储层次（寄存器 → L1 → L2 → L3 → 本地内存 → 远端内存/网络）与 cache line 粒度（64 B）、缓存一致性带来的**伪共享（false sharing）**、cache line 在核间"乒乓"迁移、互连网络（Intel Sandy Bridge 起的 ring 互连：request/snoop/ack/data 四类环、4 个 L3 slice + system agent + graphics 共 6 个节点；SUN Niagara 2 的 crossbar；多路系统的 NUMA）、DRAM 带宽与延迟、memory-level parallelism（MLP）与硬件预取器。
- **量化工具**：延迟 vs 带宽的区分、流水线（pipelining）、通信代价模型 `T(n) = T0 + n/B` 与"overhead + occupancy + network delay"三段分解、算术强度（arithmetic intensity）与 communication-to-computation ratio、Roofline 模型、work-span（贪婪调度）模型、Amdahl 定律、Little's law（带宽 = 并发度 / 延迟）、竞争与串行化排队。
- **在并行计算知识体系中的角色**：本讲是第 7~8 讲"如何写出并行程序 / 如何分配工作"之后的**性能收口**：把"依赖 → 同步 → 通信"这条链从**正确性**推进到**性能**。它同时是后续若干讲的引子与上下文：
- 第 11~13 讲（Snooping 一致性、目录协议、Snooping 多处理器设计）正是在回答"本讲提到的伪共享（两个处理器写同一条 cache line）在硬件上到底发生了什么"；
- 第 10 讲（互连网络，Interconnection Networks）回答"网络拓扑如何决定 occupancy 与竞争"；
- 第 15~16 讲（同步实现、细粒度同步与无锁编程；内存一致性在第 21 讲）回答"锁/原子操作的成本从何而来，为什么细粒度锁和树形结构能降低竞争"；
- 第 14 讲（Performance Analysis / Profiling）与本讲"建立 high watermark、分清 compute / bandwidth / sync 瓶颈"的方法论直接衔接。

  一句话概括本讲的地位：**并行程序的可扩展性上限由通信与竞争决定，而通信与竞争既由算法（分解与分配）决定，也由机器（cache line、互连、锁实现）决定。**
- **配套材料**：
- `lectures/08_progperf2.pdf`（对应抽取文本 `extracted/08_progperf2.txt`，共 **63 页**幻灯片）：**已公开**，位于 Fall 2026 公开讲义目录 https://www.cs.cmu.edu/~418/lectures/ 之下，可在公开网络直接下载。Fall 2026 日程表 https://www.cs.cmu.edu/~418/schedule.html 中本讲为 **Lecture 9, Sep 14, Performance Optimization II**，日程表已给出 slides 链接（视频链接被注释隐藏）。
  - 讲义首页写作 "Lecture 7: Performance Optimization Part II: Locality, Communication, and Contention"，页脚学期字样为 **Fall 2025**：这是讲义沿用历史版本的正常现象，不代表日程表出错（Fall 2026 的讲次编号顺延为第 9 讲）。
  - 上一讲 `lectures/07_progperf1.pdf`（Performance Optimization I，Fall 2026 为 Lecture 8, Sep 11）是本讲的直接前置：那里讲**工作分配**（静态/动态、粒度、Cilk 式 continuation stealing 工作窃取），本讲讲**通信、局部性与竞争**。
- `cs149_supp/perfopt2.txt`：**已公开**的姊妹课程材料 —— Stanford CS149 Fall 2025 Lecture 6 "Performance Optimization Part II: Locality, Communication, and Contention"（**68 页**）。它与 CMU 版本共享同一套骨架与案例（Culler/Singh/Gupta 的网格求解器与 ghost cell、α-β 通信模型、inherent vs artifactual communication、office hours 竞争类比、分布式工作队列、Roofline、high watermark），但**多出四块内容**：共享地址空间硬件的实现（ring/crossbar/NUMA）、性能分析方法论（Roofline、perf counter、PCM）、固定问题规模测速的陷阱（SGI Origin 2000 上的 258×258 网格）与超线性加速。本笔记在使用这些额外内容时会显式标注来源为 CS149 版本。
- 其它讲座的公开状态（与本讲无关但同属 Fall 2026）：Performance Analysis / Profiling（日程表第 14 讲）、Transactional Memory（第 17 讲）、AI in System Design（第 24 讲）在 Fall 2026 **尚未给出 slides 链接，属未发布**；历史学期对应 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- 录像（Panopto/YouTube）在 Fall 2026 日程表中被注释隐藏，属 **未发布**；Ed 讨论区、Autolab、Canvas 均**需登录**，非公开。
- Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。

---

2. 核心概念与硬件/软件架构图解

2.1 延迟 vs 吞吐量：一切通信优化的出发点

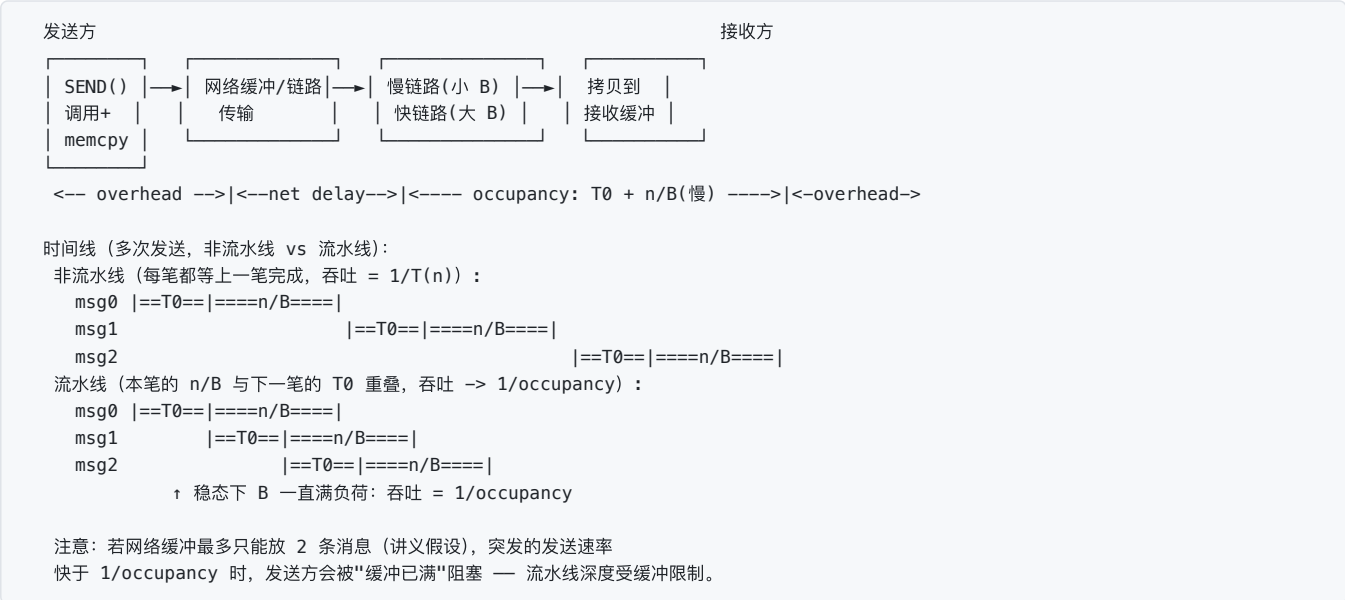
- **定义与目的**：
- **延迟（latency）**：完成**一次**操作所需的时间。讲义给的定义是"The amount of time needed for an operation to complete"，例子：一次 cache miss 的 load 有 ~200 个周期的延迟；一个 packet 从本机到 Google 要 20 ms；在 Piazza 上提问 10 分钟得到回复。
- **带宽（bandwidth / throughput）**：操作被完成的**速率**。例子：内存可以 25 GB/s 的速率向处理器供数；一条链路每秒能发 1000 万条消息；助教每天回答 50 个问题。
- 区分二者是通信优化的第一步：**延迟决定"一次通信要等多久"，带宽决定"单位时间能传多少"**；二者需要完全不同的优化手段（降延迟 vs 提吞吐）。
- **直观解释（"它是什么？"）**：讲义用"从旧金山开车去匹兹堡"作类比。距离约 **4000 km**，车速 **100 km/h ⇒ 一个人的延迟 = 40 小时**。如果高速公路上车与车之间保持 **1 km** 间距，那么**吞吐量 = 100 人/小时**（每 1/100 小时过一辆车）。
- 提高吞吐量有两条路：**① 开快点**（150 km/h ⇒ 150 人/小时）；**② 多修车道**（3 车道 ⇒ 300 人/小时）。前者相当于提升"单个流的速度"（提高时钟频率、降低单次延迟），后者相当于**复制资源**（更多执行单元、更多链路）。
- 极端情形：**如果高速公路上同一时刻只允许一辆车**，那么吞吐量退化为 `1 / 延迟 = 1/40 人/小时`。**这就是"没有重叠的通信"——最糟糕的情形，也是本讲要消灭的情形。**
- 另一个更贴近生活的类比：**同时把"车距"减半**（1 km → 500 m）可以把吞吐量翻倍到 200 人/小时，效果**等价于**在 1 km 间距下把车速提到 200 km/h。这就是**流水线（pipelining）**的核心思想：**把一个大任务切成更小的子任务，让不同的"车"（不同的阶段）并行处理不同部分**，而不是让一辆车独自跑完全程。
- **架构/机制图解（流水线的时空图）**：

```
(a) 非流水线 / 串行执行：吞吐量 = 1 / 延迟
    时刻:   0h        1h        2h        3h        4h        5h        6h
    load#1  [====== 洗涤 45min ======][==== 烘干 60min ====][折 15m]  <- 2 小时/批
    load#2                                                          [====== ...       <- 必须等前一批做完
    吞吐量 = 1 批 / 2 小时 = 0.5 批/小时            延迟(1 批) = 2 小时

(b) 复制资源（两台洗衣机 + 两台烘干机 + 一个朋友）：延迟不变，吞吐量 x2
    load#1  [== 洗 ==][== 烘 ==][折]
    load#2  [== 洗 ==][== 烘 ==][折]        吞吐量 = 2 批 / 2 小时 = 1 批/小时
                                              资源数量也 x2（两套设备 + 两个人）

(c) 流水线：一套设备，把"批次"错开，资源始终不空闲
    时刻:   0h      1h      2h      3h      4h      5h
    洗衣机  [L1洗][L2洗][L3洗][L4洗][L5洗] ...
    烘干机        [L1烘][L2烘][L3烘][L4烘] ...      <- 烘干 60min 是"最慢环节"
    学生          [L1折][L2折][L3折][L4折] ...      <- occupancy = 1 小时
    吞吐量 = 1 批 / 小时 (= 1 / 最慢环节占用的时间)
    延迟   = 2 小时（单批依然要 洗45 + 烘60 + 折15）

(d) 四段指令流水线 IF -> D -> EX -> WB：延迟 = 4 周期，吞吐量 = 1 指令/周期
    clk:      1     2     3     4     5     6     7
    instr0   IF    D    EX    WB
    instr1         IF    D    EX    WB
    instr2               IF    D    EX    WB
    instr3                     IF    D    EX    WB
    (真实机器更长：讲义提到 Intel Core i7 的流水线是变长的 ~15-20 级)
    前提：相邻指令之间存在依赖时，必须由旁路/停顿/乱序来保证正确性。
```

- **性能特征与关键结论**：
- 流水线的**延迟不变**（单批仍是 2 小时；单条指令仍是 4 周期），但**吞吐量被"最慢一级的占用时间"（occupancy）锁定**：稳态下 `吞吐量 = 1 / occupancy`。
- 流水线**不需要复制执行资源**（只有一台洗衣机、一台烘干机），这是它比"复制资源"更便宜的地方；代价是**单次操作的延迟不再下降**，而且需要**额外的并发**（讲义后文："requires additional concurrency in application — more concurrency than number of execution units"）。
- 对通信而言：**"修更多车道"= 提高链路带宽/增加链路条数；"开快点"= 降低单次传输延迟；"流水线"= 让多笔通信在时间上重叠**。本讲后面的全部技术都可以归到这三类里。

2.2 通信代价模型：`T(n) = T0 + n/B`

- **定义与目的**：给出一个能**预测**通信成本的模型，从而知道优化应该往哪里使力。讲义先给最简单的"非流水线通信"模型：
- `T(n)`：一次传输的**总延迟**；`T0`：**启动延迟（start-up latency）**，即第一个 bit 到达目的地所需的时间；`n`：传输的字节数；`B`：链路**传输速率（带宽）**。
- `T(n) = T0 + n / B`；**有效带宽（effective bandwidth）** `= n / T(n)`。**有效带宽依赖于传输大小**：小消息被启动延迟摊薄得很惨，大消息才能逼近 `B`。
- 更一般的模型把一次通信拆成三部分：**总通信时间 = overhead（开销）+ occupancy（占用）+ network delay（网络延迟）**。其中 overhead 是**处理器**花在通信上的时间（发起调用、拷贝到网络缓冲）；occupancy 是数据穿过系统**最慢部件**所需的时间；network delay 是其余所有时间。
- **cost（代价）= 通信时间 − overlap（重叠部分）**：重叠掉的部分不产生代价。这正是非阻塞通信与流水线的意义。
- **直观解释（"它是什么？"）**：类比**寄快递**。`overhead` = 你在家打包、填单、交给快递员的时间（**你自己被占住了**）；`occupancy` = 包裹在传送带上/货车上的时间（**系统资源被占住了，宽窄由最慢那段路决定**）；`network delay` = 中转分拣、堵车。如果你有 10 个包裹要寄，**一次打包寄一个**（非流水线）与**一次打包 10 个一起寄**（大消息/合并消息）的总时间差别巨大——这就是 `T0` 的摊薄效应。而你**一边等快递一边继续干活**，就是 overlap。
- **架构/机制图解（三段分解与重叠）**：



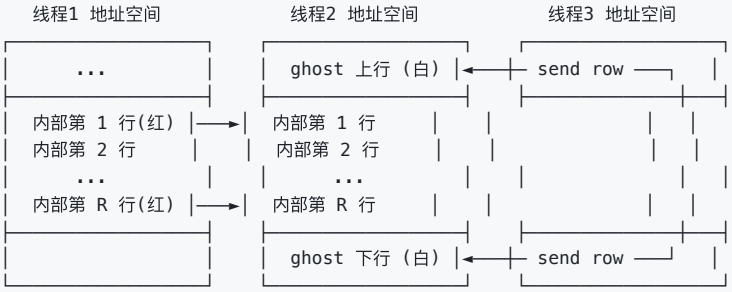
- **性能特征与关键结论**：
- **有效带宽随消息变大而提高**（`n/T(n) = n/(T0+n/B) → B`）：这直接推导出讲义的第一条优化准则——**"send fewer messages, make messages larger"**（减少消息条数、增大消息体积、把许多小消息合并成一个大消息）。
- 定量地：要让有效带宽达到 `B` 的 50%，需要 `n/B ≥ T0`，即 `n ≥ T0·B`。以 `T0 = 2 μs`、`B = 6.4 GB/s` 为例，`n ≥ 12.8 KB`——**小于 12.8 KB 的消息，一半以上的时间都花在启动上**。
- **occupancy 决定稳态通信速率**（`msg/sec = 1/occupancy`），而 occupancy 由**最慢部件**决定：一个"快链路 + 慢链路"的组合，最终速率由慢链路决定。
- 只有当**应用有额外并发**（比执行单元更多的并发）时，overlap 才可能；否则"一边通信一边计算"只是口号。

2.3 消息传递与 ghost cell：把通信显式化

- **定义与目的**：**消息传递模型（message passing）**中每个线程/进程拥有**自己的私有地址空间**，线程之间**只能**通过 `send` / `recv` 交换数据；`send` 指定接收者、被发送的缓冲区与可选的消息标识（tag），`recv` 指定发送者、接收缓冲区与 tag。它的价值在于：**通信被显式写进代码**，程序员被迫面对"谁拥有数据、什么时候需要谁的数据"，因此程序结构天然更接近可扩展的并行程序（集群/超算的编程模型，可用 Infiniband 之类的网络把商品机器连成大型并行机）。讲义之所以在性能优化课上先复习它，正是因为**它是讨论通信代价最干净的载体**。
- **直观解释（"它是什么？"）**：类比**每家自备仓库 + 寄信**。共享地址空间是"大家共用一个白板"，写上去别人立刻能看见；消息传递是"每人有自己的仓库，想知道别人的数据必须写信索取"。于是出现一个关键概念：**ghost cell（影子单元）**——为了算自己负责的边缘单元，必须**在本地复制一份**邻居的数据，并明确"这份数据的**所有权**属于别的线程"。这就像你要用邻居家的温度计读数，只能**抄一份贴在自家墙上**，而抄来的那份可能过期。

- **架构/机制图解（SPMD 网格求解器 + ghost 行交换）**：

```
1D 行分块 (1D blocked assignment)：N x N 网格切成 P 份，每份 rows_per_thread = N/P 行
每个线程的本地数组 = (rows_per_thread + 2) 行 x (N + 2) 列   ← 上下各 1 行 ghost，左右各 1 列 halo
```




```
▲ 竖直方向相邻的线程互发"边界行"；水平方向靠本地 halo 列解决

Solve() 每轮循环的结构 (slide 给出的伪代码)：
  while (!done) {
    ① 收发 ghost 行：  if (tid!=0)      send(&localA[1,0],  ..., tid-1, MSG_ROW);
                       if (tid!=P-1)    send(&localA[R,0],  ..., tid+1, MSG_ROW);
                       if (tid!=0)      recv(&localA[0,0],  ..., tid-1, MSG_ROW);
                       if (tid!=P-1)    recv(&localA[R+1,0],..., tid+1, MSG_ROW);
    ② 计算：5 点平均  localA[i,j] = 0.2*(上+下+左+右+自己)，同时累加 |Δ|
    ③ 归约：所有线程把 my_diff 发给线程 0；线程 0 判断 my_diff/(N*N) < TOLERANCE
            并把 done 广播回所有线程（用消息实现"屏障/标志位"）
  }

⚠ 阻塞式 send/recv 的死锁：所有线程"先 send 再 recv"会互相等待
   T0: send->T1 ...阻塞...          T1: send->T0 ...阻塞...
   T0: ...等着收 T1 ...             T1: ...等着收 T0 ...
   => 死锁（谁都不往下走）
   修法（slide 给出的版本）：奇偶分角色、收发顺序配对
   if (tid % 2 == 0) { sendDown(); recvDown(); sendUp(); recvUp(); }
   else              { recvUp();   sendUp();   recvDown(); sendDown(); }
   T0: [send][recv][send][recv]      T1: [recv][send][recv][send]
       ↑ 双向的收发在时间上交错，形成"配对"从而不会彼此死等
   另一条路：使用非阻塞 Isend/Irecv —— 把所有收发一次性投递出去再 Waitall
```

- **阻塞 vs 非阻塞（讲义的两张对照图）**：

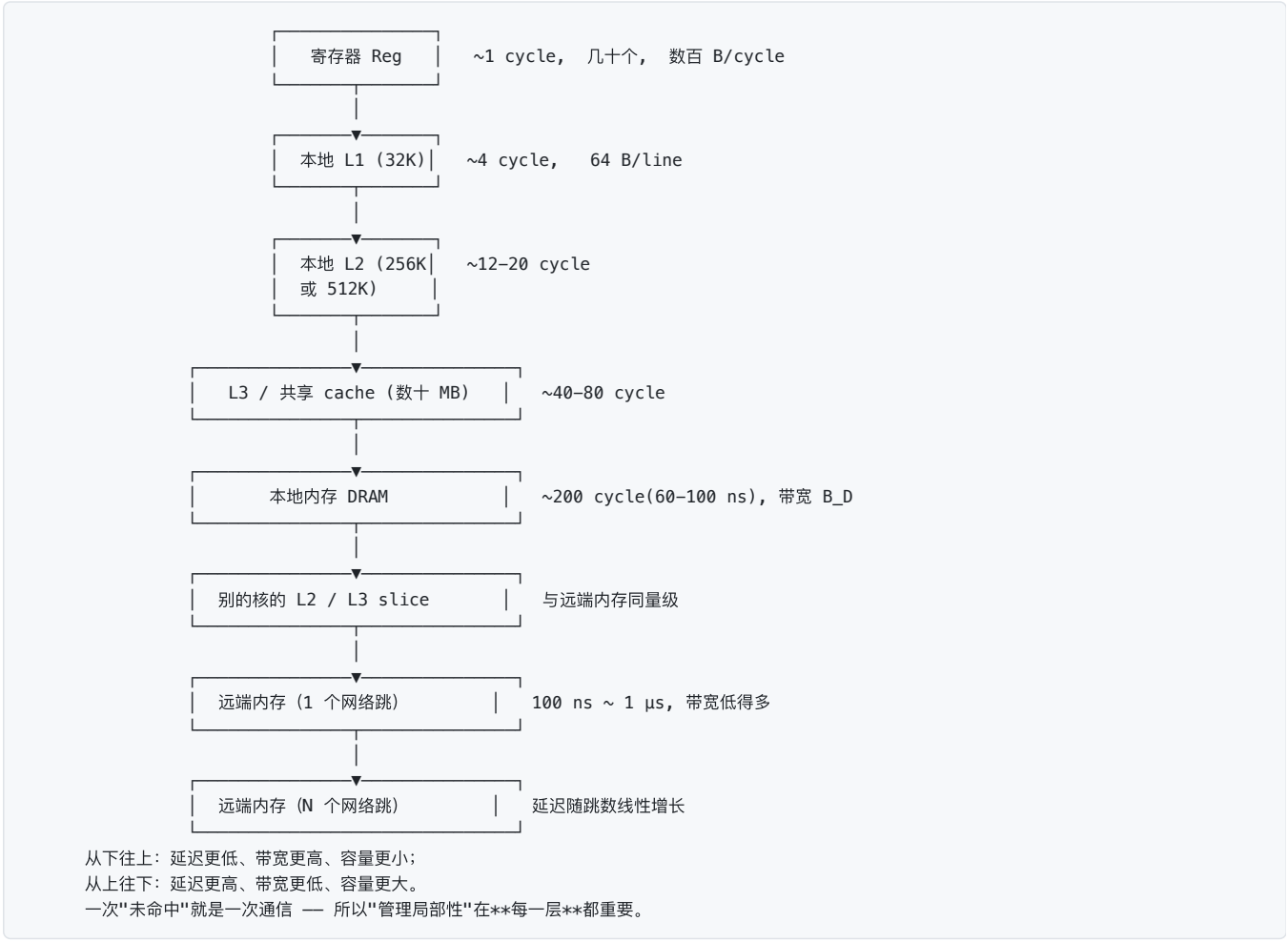
维度	同步/阻塞 `send` / `recv`	异步/非阻塞 `Isend` / `Irecv` + `check/Wait`
`send` 何时返回	收到接收方 ack、确认数据已在对方地址空间后才返回	**立即返回**，只把数据拷进网络缓冲
`recv` 何时返回	数据被拷进接收缓冲并回送 ack 之后才返回	**立即返回**，只是"登记了接收意向"（返回 handle）
缓冲区的约束	调用期间缓冲区被拷走，返回后即可复用	**发给 send 的缓冲区在确认发送完成前不能被修改**（消息处理与应用线程并发进行）
能否overlap	不能（调用线程被阻塞）	能（调用线程可继续做别的事，用 `checksend/checkrecv` 查状态）
典型风险	**死锁**（收发顺序不当）	使用已发送缓冲/未完成接收缓冲导致数据错乱
代表的用法	教学里为了看清语义	真实高性能代码（本讲示例 1 即用此法）

- **性能特征与关键结论**：
- 消息传递里**通信与同步是同一件事**：互斥、屏障、标志位都可以用消息实现（harness 里"用消息实现屏障"是常见练习）。
- **批量传输（bulk transfer）**是天然的优化：一次发一整行（`N+2` 个 float）比逐元素发 `N+2` 条消息快几个数量级——**这是 `T0` 摊薄效应的直接应用**。
- 数组索引是**相对本地地址空间**的，任何"跨进程"的访问都必须显式翻译为消息；这消除了共享内存编程里"看不见的通信"。

2.4 把并行系统看作"扩展的存储层次"

- **定义与目的**：讲义要求把"通信"这个概念**一般化**：不只是机器之间的消息，还包括**处理器与 cache 之间、与本地内存之间、与远端内存之间**的所有数据搬运。整个并行系统因此可以画成一张**扩展的存储层次（extended memory hierarchy）**：越靠近处理器，**延迟越低、带宽越高、容量越小**；任何一次"本地没法满足"的访问都会**触发与下一级之间的通信**。

- **直观解释（"它是什么？"）**：类比**一个人的办公桌**：常用的笔和便签在**手边（寄存器/L1）**；当天要用的资料在**抽屉（L2/L3）**；不常用但要留档的在**楼下档案室（本地内存）**；别人部门的档案要**打电话去要（远端内存，1 个网络跳）**；上级单位的档案要**层层转达（N 个网络跳）**。你**每多伸手一次就多一份往返时间**，所以"把相关材料先搬到手边再干活"（局部性）永远是最有效的优化。
- **架构/机制图解（扩展存储层次 + 典型数量级）**：



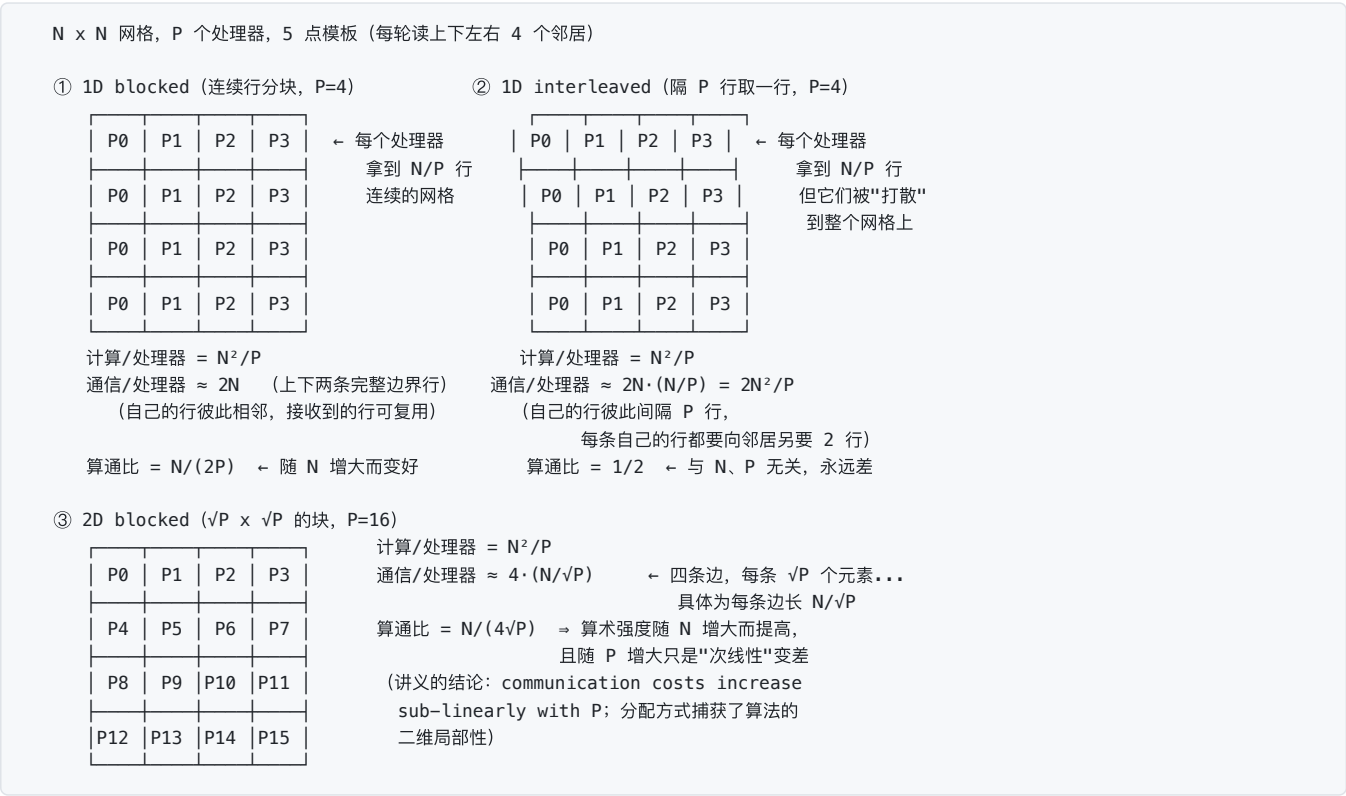
- **性能特征与关键结论**：
- **层次结构决定了"通信"无处不在**：一个 `load` 指令可能引发 L1 未命中 → L2 → L3 → 本地内存 → 甚至远端内存的连锁通信。
- 由此得到本讲的中心命题：**优化局部性 = 优化通信 = 优化性能**，而且这个判断在任何层次上都成立（register↔L1、L1↔L2、core↔DRAM、node↔node）。
- CS149 版本给出的硬件实例（说明"共享地址空间抽象的实现有多复杂"）：Intel Sandy Bridge 起的 **ring 互连**（四条环分别传 request/snoop/ack/data，data 环 32 B；6 个节点 = 4 个 2 MB 的 L3 slice + system agent + graphics；每个 L3 bank 接两次环；3.4 GHz 下"每核访问自己的本地 L3 slice"时理论峰值带宽约 **435 GB/s**）；SUN Niagara 2 的 **crossbar**（全连接，面积约等于一个核心）；多路系统的 **NUMA**（不同核访问同一地址的延迟与带宽都可能不同，即使在单路系统里，不同 L3 slice 与核的距离也不同）。

2.5 固有通信 vs 人为通信（inherent vs artifactual communication）

- **定义与目的**：这是本讲最重要的分类。
- **固有通信（inherent communication）**：**给定工作分配方式**，算法**必须**在处理器之间搬运的信息量（假设 cache 容量无限、传输粒度无限小）。例：消息传递求解器里的 ghost 行就是固有通信。**它由"分解 + 分配"决定，因此程序员可以通过改进分配来降低它。**
- **人为通信（artifactual communication）**：**其余所有通信**，源自系统的实现细节（无限容量 cache 假设、最小传输粒度假设不成立）。**它往往和固有通信一样重要。**
- 人为通信的四个典型来源（讲义原文）：
  1. **系统有最小传输粒度**：程序只要 1 个 4 字节 float，硬件却必须搬一整条 **64 字节 cache line ⇒ 16 倍**的多余通信。
  2. **系统操作规则导致的"多余"通信**：程序连续写 16 个 4 字节 float（正好一条 64 B line），硬件却**先把这条 line 从内存读进来**（write-allocate），然后整条被覆盖，最后再写回 ⇒ **2 倍**开销（读其实完全没必要）。

3. **数据在分布式内存中放置不当**：数据不在"访问它最多的处理器"附近。
4. **复制容量有限**：cache 太小留不住数据，同一份数据被反复搬运（capacity miss）。

- **直观解释（"它是什么？"）**：类比**搬家**。**固有通信** = 你确实必须搬走的家具（不管你用几辆车、多大的箱子，这些家具一定要运）。**人为通信** = 因为你**只买到了固定尺寸的纸箱**（最小粒度），所以你必须为一只杯子用掉一整个大箱子（4 B → 64 B）；因为**搬运规则要求先把箱子搬进屋再决定要不要**（write-allocate），所以空箱子也要搬一趟（2 倍）；因为**仓库太小**放不下你已经搬来的东西，所以同一件家具你搬了两遍（capacity miss）。
- **架构/机制图解（工作分配如何决定固有通信量）**：



- **定量对照表（N×N 网格、P 个处理器、单位：元素）**：

分配方式	每处理器计算量	每处理器通信量	算通比（计算/通信）	随 P 的变化	备注
1D blocked（连续行块）	N^2/P	$\approx 2N$	$N/(2P)$	$\propto 1/P$（线性变差）	简单、局部性好；边界行可复用
1D interleaved（隔行/隔列交错）	N^2/P	$\approx 2N^2/P$	**1/2（常数）**	**与 N、P 无关**	负载均衡好，但通信灾难性差
2D blocked（二维分块）	N^2/P	$\approx 4N/\sqrt{P}$	$N/(4\sqrt{P})$	$\propto 1/\sqrt{P}$（次线性）	捕获二维局部性；P 大时需要 $\sqrt{P}$ 整除 N

> 约定：本表把**算术强度（arithmetic intensity）定义为"每搬运一个元素所对应的计算元素数"**（越大越好）。讲义在 2D blocked 一页里同时写了 $N/(4\sqrt{P})$ 与 $4\sqrt{P}/N$，后者是它的倒数（即 communication-to-computation ratio，越小越好），二者只是定义方向不同。

- **性能特征与关键结论**：
- **算术强度 = 1 / communication-to-computation ratio**。讲义原话："I find arithmetic intensity a more intuitive quantity, since higher is better." 现代并行处理器的**计算能力与可用带宽之比很高**，因此**必须有高的算术强度**才能高效利用机器（回想第 3 讲"逐元素向量乘法"的例子：几乎全部时间在等数据）。
- **"计算机比内存快得越来越多"是算术强度重要的根本原因**：如果算力每两年翻倍而带宽只翻 1.5 倍，那么任何算术强度不变的程序都会**自动变得越来越受带宽限制**。
- **好的分配决策可以降低固有通信**，这是程序员在白板上就能做的优化（不需要改硬件）；而**人为通信要靠数据布局与代码重排来消灭**（见 2.7）。

2.6 四个 C：从 working set 视角看通信

- **定义与目的**：讲义把经典的"三个 C"扩展为"四个 C"，把并行系统里的通信也纳入 cache miss 的分类：

- **Cold miss（冷缺失）**：第一次访问该数据。**在串行程序里不可避免**（"compulsory miss"）。对并行程序，它对应"第一次拿到工作集"（first working set）。
- **Capacity miss（容量缺失）**：工作集比 cache 大。可通过**增大 cache** 或**改变分块/遍历顺序**来减少。
- **Conflict miss（冲突缺失）**：由 cache 的管理策略（组相联度）造成。可通过**提高相联度**或**改变数据访问模式（padding、分块布局）**减少。
- **Communication miss（通信缺失，本讲新增）**：由并行系统中的**固有通信或人为通信**造成。
- **直观解释（"它是什么？"）**：把 cache 想成**书桌**，内存是**书架**。**Cold miss** = 这本书你第一次去书架上取（不可避免）。**Capacity miss** = 你要同时摊开 20 本书，但桌子只放得下 5 本，于是来回跑。**Conflict miss** = 桌上明明有空位，但书架的编号规则让你的 6 本书只能落在同一个格子，只能互相挤出去。**Communication miss** = 这本书在**隔壁同学的桌上**，你必须跟他要——而且如果你**每次都要**，说明你的"工作分配"没把相关的书放在同一张桌上。
- **架构/机制图解（working set 视角下的数据流量）**：

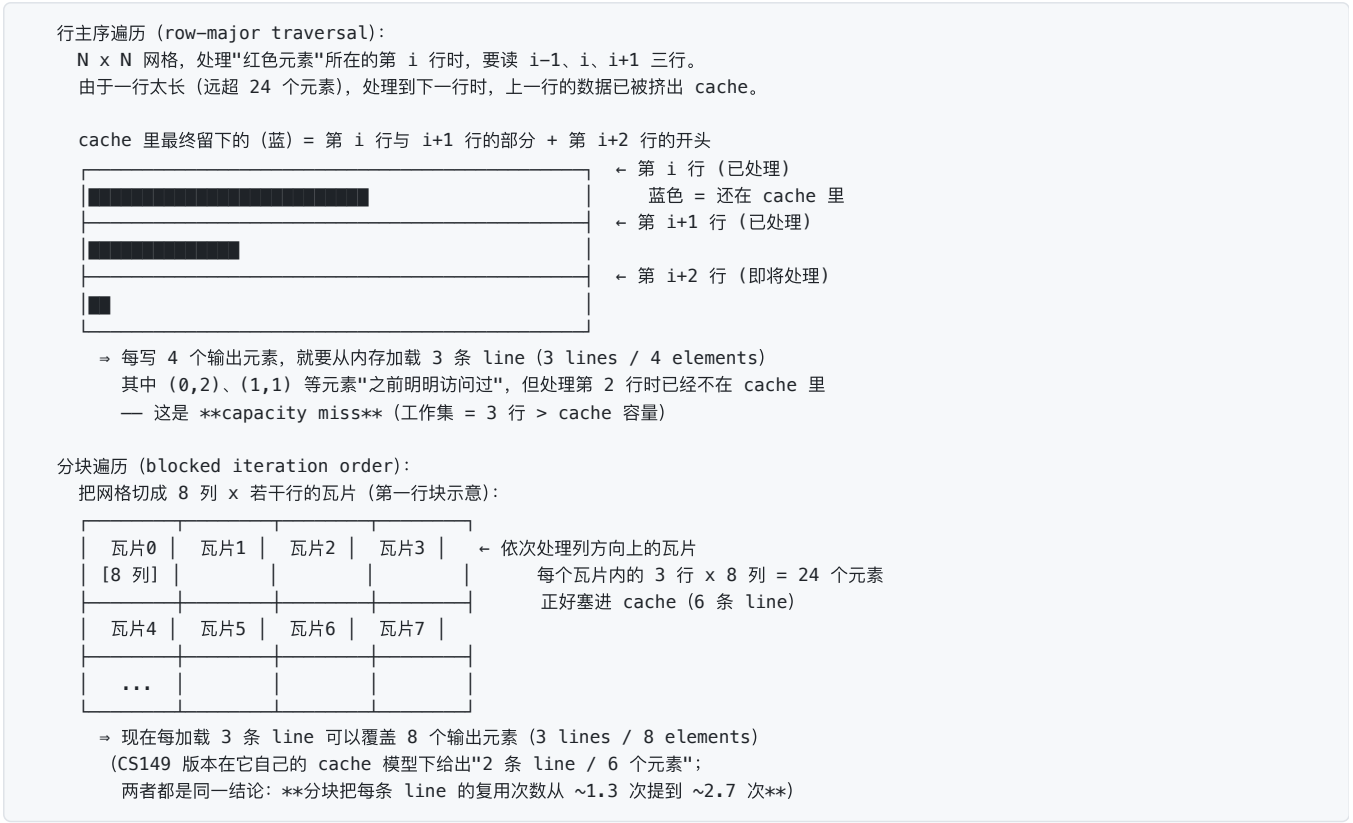
```
数据流量
  ▲
  │  ┌──── 冷缺失 (Cold misses)：第一次触碰工作集，串行程序也躲不掉
  │  │
  │  ├──── 固有通信 (Inherent communication)：并行算法/分配方式决定的下界
  │  │
  │  └──── cache 容量造成的流量 (capacity + conflict)：
  │           工作集超出容量 → 反复搬运（人为通信的主体）
  │
  └──────────────────────────────────────────► 层次容量的增加
     "first working set"      "second working set"
     (能装下第一个工作集)      (连更大的工作集也装得下)

架构师的问题：给定这个负载，这一级该造多大容量？    ← 答案就在这张曲线的"拐点"上
程序员的问题：我的程序落在这条曲线的哪一段？我能把它推到更低的那一段吗？
注意：这张图在并行系统的**任何一层**都成立（register/L1/L2/L3/本地内存/远端内存/网络）
```

- **性能特征与关键结论**：
- **曲线有三段平台**：容量太小 → 数据流量随容量增加**陡降**（capacity/conflict 主导）；容量足够后趋于**平缓**（只剩 cold miss + 固有通信）。
- CS149 版本补充："这张图看起来很熟悉吧？"——它就是经典的 **memory mountain / working set 曲线**（回想 15–213 cache lab）。
- 工程含义：**在"曲线仍然陡峭"的区间加容量收益最大**；而**如果程序已经落在平缓段，再大的 cache 也救不了它**——这时应该改算法/分配方式（降低固有通信）或改布局（降低人为通信）。

2.7 降低通信的四种技术：blocking、fusion、sharing、granularity

2.7.1 时间局部性：改变遍历顺序（blocking / tiling）

- **定义与目的**：**分块（blocking）**是"重排计算顺序以减少 capacity miss"的通用技术：把一个大的二维（或多维）迭代空间切成能**整块装进 cache** 的瓦片，在瓦片内部把所有能用到的数据一次用完，再去下一块。
- **直观解释（"它是什么？"）**：类比**做菜**。**朴素做法**：为了做 3 道菜，先跑一趟超市买 A 的料，做 A；再跑一趟买 B 的料，做 B；再跑一趟买 C 的料（**同一批调料被反复买回家**）。**分块做法**：先想清楚要做的 3 道菜共用什么调料，**一次买齐**、在台面上按顺序做完（**调料只买一次，且在桌上留住**）。
- **架构/机制图解（讲义给出的具体模型：cache line = 4 个网格元素，cache 容量 = 24 个元素 = 6 条 line）**：



- **性能特征与关键结论**：
- 判据是**重用距离（reuse distance）**：如果"两次使用同一数据之间要访问的数据量"超过了 cache 容量，就必然发生 capacity miss。**分块的本质是把重用距离压到 cache 容量以下。**
- 分块的代价：**循环开销增加、代码更复杂、边界处理更啰嗦**；且**块大小需要针对 cache 容量调参**（太大装不下、太小则循环开销占比高）。
- 讲义用这张图把 15-213 的 cache lab 与并行网格求解器联系起来：**同一套局部性原理，串行并行通吃**。

2.7.2 时间局部性：循环融合（loop fusion）

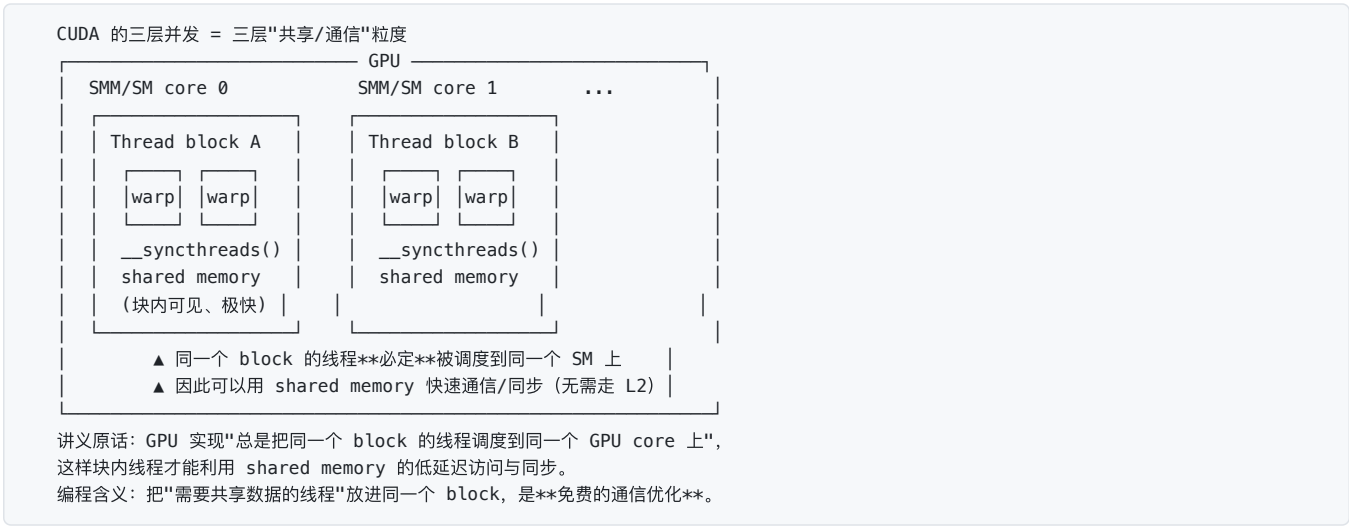
- **定义与目的**：**循环融合**把多个"遍历同一批数组"的循环合并成一个循环，从而**不再需要中间临时数组**，把多次"读—写—再读"的内存往返压缩成"一次读完所有输入、一次写出结果"。
- **直观解释（"它是什么？"）**：类比**洗菜—切菜—炒菜**。**不融合** = 把所有菜洗一遍放进盆 A，再把盆 A 的菜全部切一遍放进盆 B，再全部下锅（**每道工序都要把整批菜端来端去，还要多两个盆**）。**融合** = 拿起一棵菜，洗完立刻切、切完立刻下锅（**每个菜只经过一次手，不需要盆**）。
- **机制图解与算术强度（讲义原例）**：

```
不融合（模块化写法，类似 numpy 的数组运算）：
  tmp1[i] = A[i] + B[i];     // 读 A,B 写 tmp1      （每次数学运算: 2 load + 1 store）
  tmp2[i] = tmp1[i] * C[i];  // 读 tmp1,C 写 tmp2   （每次数学运算: 2 load + 1 store）
  E[i]    = tmp2[i] + D[i];  // 读 tmp2,D 写 E      （每次数学运算: 2 load + 1 store）
  总流量(每元素) = 3 load + 3 store = 9 次 4B 搬运
  算术强度 = 3 个数学运算 / 9 次搬运 = **1/3**
  —— 而且 tmp1、tmp2 是两个"只用一次"的中间数组（可能各占 64 MB！）

融合（一次遍历）：
  E[i] = D[i] + (A[i] + B[i]) * C[i];   // 读 A,B,C,D 写 E
  总流量(每元素) = 4 load + 1 store = 5 次 4B 搬运
  算术强度 = 3 个数学运算 / 5 次搬运 = **3/5**
  ⇒ 内存流量降到 5/9 ≈ 56%（理论加速比上限 1/0.56 ≈ 1.8 倍）

内存流量的直观对比：
  不融合:  A ──┐          ┌── C        D            （2 个临时数组来回过手）
               ├─[tmp1]─┤
          B ──┘          └─[tmp2]── E
          流量: 读A+B 写tmp1 | 读tmp1+C 写tmp2 | 读tmp2+D 写E   = 9 趟
  融合:    A,B,C,D ──[一次循环体]── E
          流量: 读 A,B,C,D 写 E                                  = 5 趟
```

- **性能特征与关键结论**：
- 融合的收益来自**三处**：① 少搬 4/9 的字节；② 少分配/少触碰两个大数组（也少了一半的 **TLB** 压力与页错误开销）；③ 提高了**每条 cache line 上完成的运算量**——在带宽受限区间直接转化为加速。
- 融合的代价：**代码可读性下降、失去模块化**（讲义特意指出"上面的代码更模块化，比如 Python 的 numpy 数组运算风格"；下面的代码**快得多**）。这正是"分层的库抽象 vs 单 kernel 手写"的经典权衡，也是为什么现代深度学习编译器要做 **operator fusion**。
- **融合不改变算法的工作量（Work）**，它只改变**内存流量**；因此它对**计算受限**的程序几乎无益（Roofline 的平段），只对**带宽受限**的程序有效（Roofline 的斜段）。

2.7.3 利用共享（exploit sharing）：把访问同一数据的任务放一起

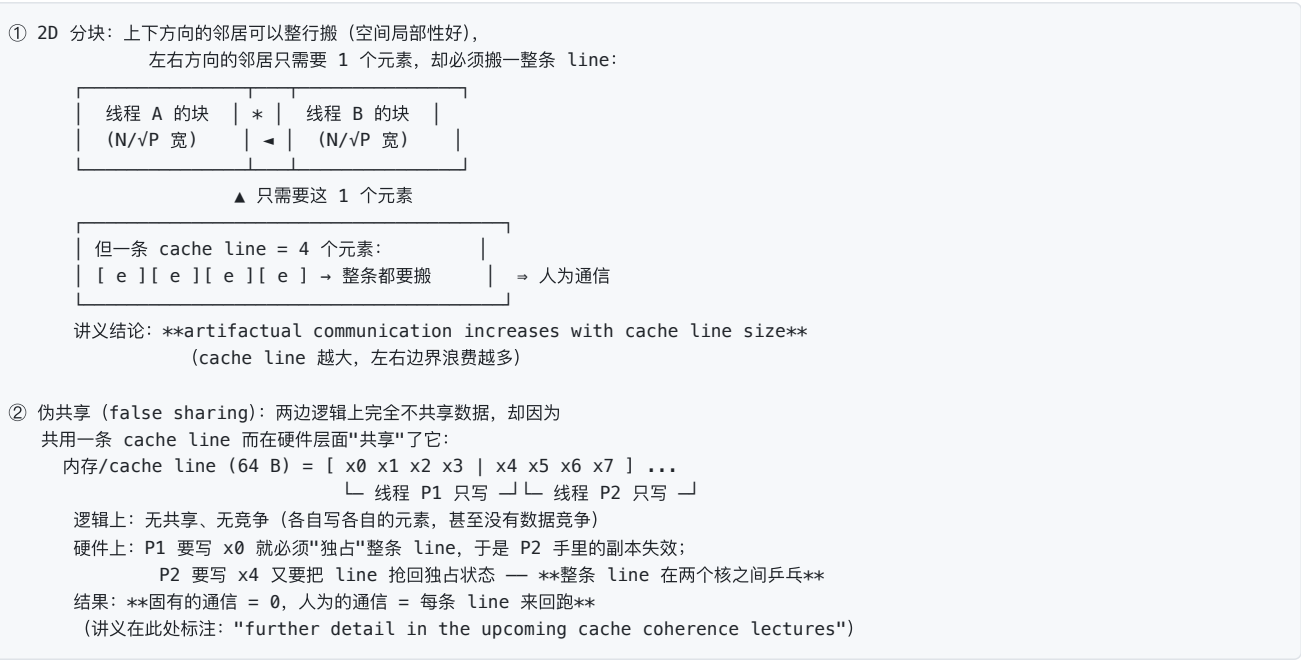
- **定义与目的**：**把操作同一份数据的任务安排在同一时间、同一个处理器上执行**，从而把"多次搬运"变成"一次搬运 + 多次本地使用"。它降低的是**固有通信**（不是人为通信）。
- **直观解释（"它是什么？"）**：类比**开小组会**。与其让 8 个人各自跑去档案室抄同一份资料（8 趟往返），不如**把 8 个人叫到同一间会议室**，资料搬一次、大家轮流看。
- **架构/机制图解（CUDA thread block 是硬件层面的"共享"抽象）**：

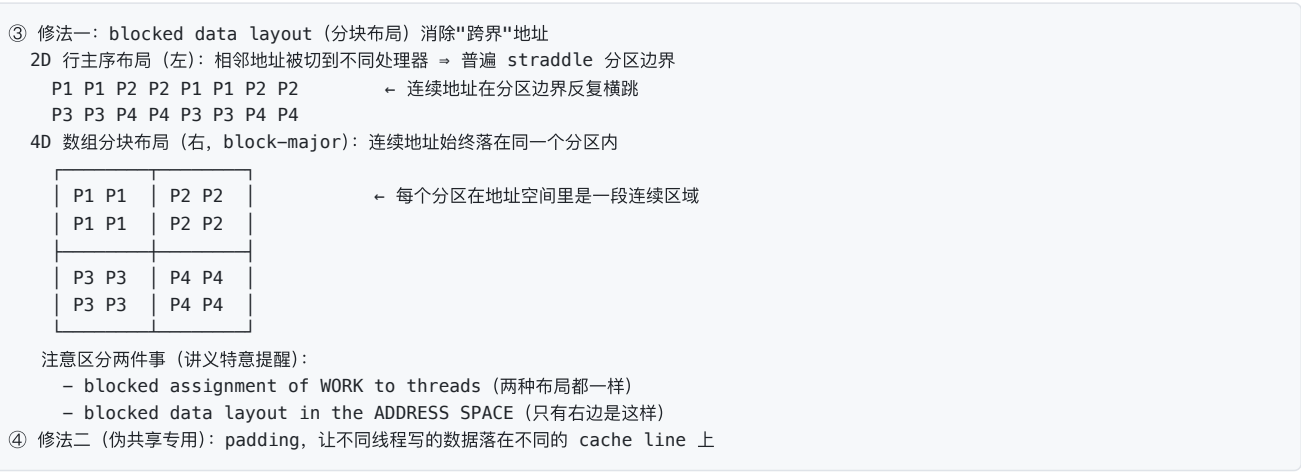


- **性能特征与关键结论**：
- 共享让通信从"跨处理器"降级为"处理器内部/片上"：**延迟降一个数量级、带宽升一个数量级**，且**不增加任何算法工作量**。
- 编译器/运行时的调度策略会直接决定共享是否发生（CUDA block 的硬件映射、OpenMP/线程池的亲和性、操作系统调度）；**程序员能做的是把"应该共享的东西"组织进同一个调度单元**。

2.7.4 通信粒度（granularity）与人为通信

- **定义与目的**：通信的**粒度**（一次搬多少字节、以及 cache 一致性以什么为单位）会引入**人为通信**，因此在做数据划分时必须把"最小传输单位"考虑进去。
- **直观解释（"它是什么？"）**：类比**买调料**。你需要 "1 克盐"，但超市**只按整包卖**（1 包 500 克）——你只能买一整包，多出来的 499 克就是"人为"开销。**cache line 就是那个"整包"**。
- **架构/机制图解（2D 分块 + cache line 粒度 ⇒ 人为通信；以及伪共享）**：

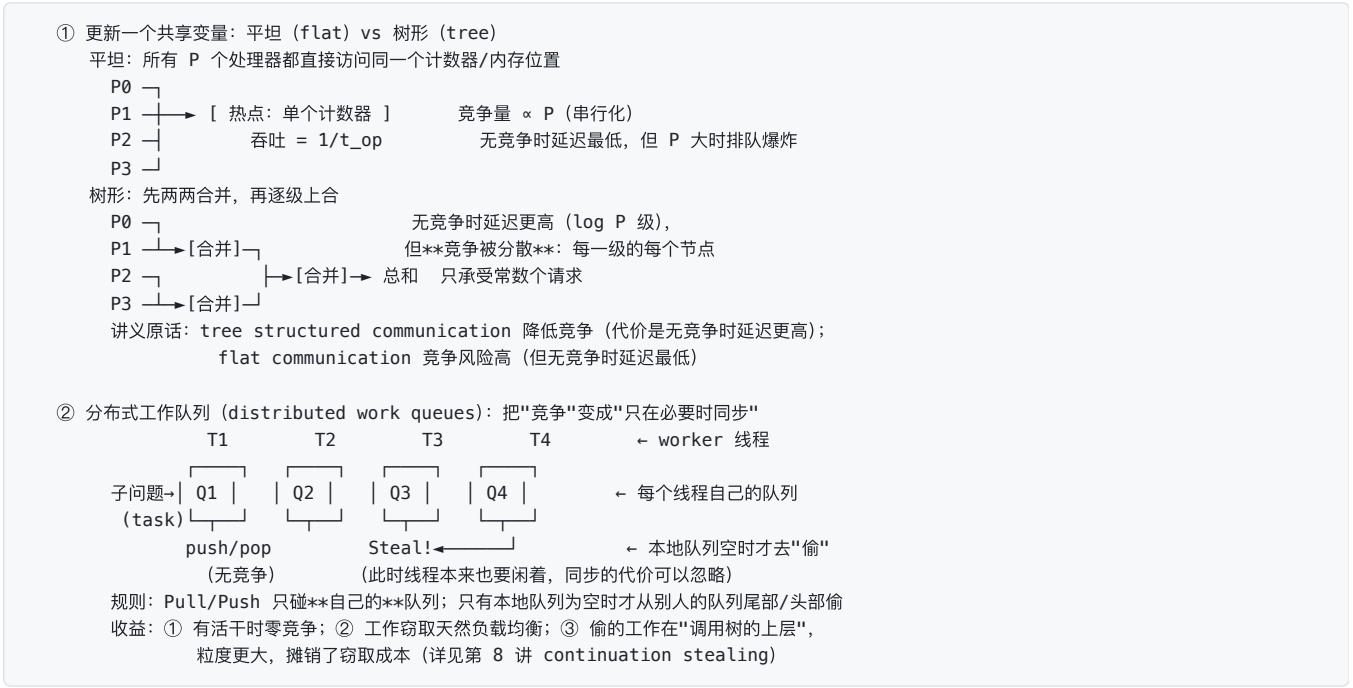




- **性能特征与关键结论**：
- 通信粒度与 **cache 一致性粒度**都要在"任务划分"阶段考虑；**数据布局（layout）与工作划分（assignment）必须联合设计**，否则工作划分得再漂亮也会被 cache line 边界吃掉。
- **伪共享是"代码正确、性能崩塌"的典型**：它不产生数据竞争（每个线程只写自己的数据），却产生与"所有线程争抢同一个变量"几乎等价的硬件通信量。

2.8 竞争（contention）：比"通信多少"更致命的是"通信何时发生"

- **定义与目的**：**竞争**指**在很短的窗口内对同一个资源发出大量请求**的现象，此时该资源成为**热点（hot spot）**。每个资源都有一个**固有吞吐量**（单位时间的交易数）：内存、通信链路、服务器、Office hours 的助教都可以这样看。竞争**不改变工作量**，却让**每次操作的完成时间显著变长**（排队）。
- **直观解释（"它是什么？"）**：讲义用**答疑时间**作类比。假设"完成一次答疑"由三步组成：① 从咖啡厅走到办公室 **5 分钟**；② 排队等待（不确定）；③ 教授给出深刻解答 **5 分钟**。
- **情形一（有预约）**：两位同学分别约了 3:00 与 4:30。两人各走 5 分钟、各被解答 5 分钟，**每人耗时都是 10 分钟**，谁都不用等。
- **情形二（3:00–3:20 敞开答疑，不预约）**：5 位同学都在 3:00 前后到，只有**一个执行资源**（一位教授）。第一位同学仍然只花 10 分钟；**最后一位同学要排队**，他的耗时 = 5 分钟走路 + 排队（几位同学 × 每人 5 分钟）+ 5 分钟答疑，于是**变成 23 分钟甚至更久**。
- 结论：**竞争让"总操作时间"因为排队而膨胀**，而且**膨胀量取决于到达的时间分布**——这就是为什么"错峰访问（stagger）"是有效手段。
- **架构/机制图解（平坦 vs 树形通信；分布式工作队列）**：



- **降低竞争的四类手段（讲义最后的总结，也适用于降低通信成本的整体框架）**：

目标	手段	具体做法	代价 / 前提
减少 overhead	更少、更大的消息	合并多条小消息为一条大消息；批量发送	需要缓冲；延迟可能变高（数据攒着才发）
减少 delay	改代码 / 改硬件	应用侧：重排代码以利用局部性；硬件侧：改进通信架构	需要理解机器；跨节点时受物理限制
减少 contention	复制资源 / 错峰	本地副本、细粒度锁、per-cell 锁、树形归约、随机化/错开访问	复制要占内存；细粒度锁本身有创建/空间开销
增加 overlap	异步 + 更多并发	应用侧：非阻塞消息、`Isend/Waitall`；硬件侧：流水线、多线程、预取、乱序执行	**要求应用有比执行单元更多的并发**，否则无处重叠

2.9 案例研究：把 100 万个粒子放进 16 个格子里（五种解法）

- **问题设定**：把 **100 万个点粒子**按二维位置放进 **16 个均匀格子**（构建"二维列表的数组"）——这是并行**数据结构操作**的典型难题（不规则、需要动态写入）。讲义给出的机器是 **GTX 980 GPU：每个 SMM core 最多 2048 条 CUDA 线程，GPU 共 16 个 SMM core**。这个结构的常见用途是 **N-body 问题**：给定粒子，找出半径 R 内的所有邻居；把格子边长设成 R，则只需检查周围格子。
- **五种解法的对照**（讲义逐条给出了优缺点）：

解法	并行方式	并行度	同步 / 竞争	额外工作量	额外内存	关键结论
① 按格子并行	每个格子一个任务	**只有 16 个任务**（GPU 需要数千个）	**不需要同步**（无竞争）	**16 倍**的"粒子-格子归属"计算	无	**并行度严重不足**；用工作量换掉了竞争，不划算
② 按粒子并行 + **单一全局锁**	每个粒子一个 CUDA 线程	百万级（充足）	**单一全局锁 ⇒ 巨大竞争**	无	无	数千线程争一把锁；正确但极慢
③ 按粒子并行 + **每格一把锁**	同上	百万级	竞争降低约 **16 倍**（假设粒子在二维空间均匀分布）	无	16 把锁	细粒度锁的直接收益；但"桶头指针"所在的 cache line 仍会乒乓
④ **部分结果 + 合并**	建 N 个 thread block（≥ SM 数），每个 block 维护**自己的那张网格**	高	竞争降低 **N 倍**，且同步在 **block 本地变量（CUDA shared memory）** 上做，**同步本身更便宜**	需要**合并 N 张网格**	**N 倍的网格存储**	用内存换竞争；把同步"降级"到片上

解法	并行方式	并行度	同步 / 竞争	额外工作量	额外内存	关键结论
⑤ **数据并行**	① 并行算每个粒子的格子号 ② **按格子号排序** ③ 并行求每格的起止位置	极高（每个阶段都是数据并行）	**不需要任何细粒度同步**	一次排序 + 多趟数据遍历（**额外带宽**）	排序/索引数组	彻底消除竞争，代价是排序与额外带宽

- **架构/机制图解（解法 ⑤：数据并行分桶的三步）**：

```
输入:                              步骤 1: 并行算格子号 (per particle)
particle_index: [ 0  1  2  3  4  5  6  7  8  9 10 11 ... ]   ← 粒子编号
grid_index:     [ 0  1  2  4  5  3  9  6  6  4  6  4 ... ]   ← 每个粒子落在哪个格子
                              |
                              ▼  步骤 2: 按 grid_index 排序 (并行计数排序/基数排序)
particle_index: [ 0  2  5  2  5  6  9  3  5  1  2  4 ... ]   ← 重新排列后的粒子编号
grid_index:     [ 0  1  2  2  3  4  4  4  5  6  6  6 ... ]   ← 现在是**有序**的
                              |
                              ▼  步骤 3: 并行求每段的起止 (对每个 index 执行一次)
  cell = grid_index[index]
  if (index == 0)                        cell_starts[cell] = index;
  else if (cell != grid_index[index-1]) { cell_starts[cell] = index;
                                          cell_ends[grid_index[index-1]] = index; }
  if (index == numParticles-1)            cell_ends[cell] = index + 1;   // 末端不包含
                              |
                              ▼
  cell_starts: [ 0  1  2  4  5  8  ... ]     每格在排序数组中的起点
  cell_ends  : [ 1  2  4  5  8 11  ... ]     每格在排序数组中的终点 (不含)
  ⇒ 要遍历"格子 c 里的所有粒子": for (i = cell_starts[c]; i < cell_ends[c]; ++i)
                                    访问 particle_index[i]
代价: 一次排序 (对比并行、带宽开销) + 数组的额外存储;
收益: **完全不需要细粒度同步**, 且每个阶段都保持极大并行度 (适合 GPU)
```

- **性能特征与关键结论**：
- 五种解法构成一条清晰的**权衡曲线：并行度 ↔ 竞争 ↔ 额外工作/内存**。① 牺牲并行度换取零竞争；② 用最大竞争换取零额外工作；③ 用细粒度锁减少竞争（约 16×）；④ 用内存（N 张网格）与合并工作换取"块内廉价同步"；⑤ 用一次排序与额外带宽换取**零同步**。
- **没有一种解法普适**：在只有 16 个执行单元的机器上，② 也许可以接受（争抢者少）；在有 2048×16 条线程的 GPU 上，只有 ④ 或 ⑤ 能跑满。
- 这条案例线在后续课程中会继续被展开：第 15/16 讲（同步实现、无锁编程）会解释**为什么细粒度锁与无锁结构能降低竞争**，第 22~23 讲（并行深度学习）里"**用一次排序/直方图替代原子操作**"的思路会再次出现（例如 embedding 梯度聚合）。

---

3. 代码示例与性能分析

> 本节共 5 个完整示例，分别对应本讲五条主线：**① 显式通信的重叠（MPI 非阻塞）**、**② 算术强度与循环融合**、**③ 遍历顺序/分块（时间局部性）**、**④ 伪共享与数据布局**、**⑤ 竞争（锁的粒度 vs 数据并行）**。除示例 1（需要 MPI 运行时）外，其余示例均用 `g++ -O3 -march=native -fopenmp` 编译（**编译均已验证通过**），运行参数写在各自的编译注释里。给出的实测数字来自一台 **双路 AMD EPYC 7V13（64 核/路 ×2，共 128 个线程；L1d 32 KB/核、L2 512 KB/核、L3 32 MB/CCX；AVX2，无 AVX-512；64 B cache line）**，但这是一台**共享机器**（同时有其他用户任务，load average 在 8~30 之间波动），**绝对时间可能相差数倍**。因此文中一律给出**多次测量的区间**，并强调：**只有同一台机器、同一时段、成对对比得到的比值才是可靠结论；绝对值仅供参考**。

3.1 示例一：MPI 非阻塞 ghost 行交换（把通信藏到计算背后）

```
// 编译: mpicxx -O3 -march=native grid_mpi.cpp -o grid_mpi
// 运行: mpirun -np 4 ./grid_mpi 2048 200
#include <mpi.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

enum { MSG_ROW = 100 };   // 行消息统一用同一个 tag（发送方与接收方必须匹配）

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    const int N    = (argc > 1) ? atoi(argv[1]) : 2048;
    const int ITER = (argc > 2) ? atoi(argv[2]) : 200;
    const int rows = N / nprocs;            // 本进程拥有的内部行数（假设整除）
    const int ld   = N + 2;                 // 行主序，左右各留 1 列 halo
    const int size = (rows + 2) * ld;       // 本地数组 = 2 行 ghost + rows 行内部

    std::vector<float> A(size, 0.0f);
    const int r0 = rank * rows;             // 本进程负责的全局行区间 [r0, r0+rows)
    for (int i = 1; i <= rows; ++i)
        for (int j = 1; j <= N; ++j) {
            double x = (double)(r0 + i) / N, y = (double)j / N;
            A[i * ld + j] = (float)std::exp(-((x - .5) * (x - .5) + (y - .5) * (y - .5)) / 0.02);
        }

    // 更新一行，返回该行的 |Δ| 之和
    auto update_row = [&](int i) -> float {
        const float* up = &A[(i - 1) * ld];
        const float* dn = &A[(i + 1) * ld];
        float* cur = &A[i * ld];
        float d = 0.0f;
        for (int j = 1; j <= N; ++j) {
            float prev = cur[j];
            float v = 0.2f * (up[j] + dn[j] + cur[j - 1] + cur[j + 1] + cur[j]);
            cur[j] = v;
            d += std::fabs(v - prev);
        }
        return d;
    };

    float mydiff = 0.0f;
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    for (int it = 0; it < ITER; ++it) {
        MPI_Request req[4];
        MPI_Status  st[4];
        int nreq = 0;
        const int up = rank - 1, down = rank + 1;

        // (1) 一次性投递全部非阻塞通信：收 ghost 行 + 发边界行（发的是上一轮的结果）
        if (up >= 0) {
            MPI_Irecv(&A[0 * ld + 1],          N, MPI_FLOAT, up,   MSG_ROW, MPI_COMM_WORLD, &req[nreq++]);
            MPI_Isend(&A[1 * ld + 1],          N, MPI_FLOAT, up,   MSG_ROW, MPI_COMM_WORLD, &req[nreq++]);
        }
        if (down < nprocs) {
            MPI_Irecv(&A[(rows + 1) * ld + 1], N, MPI_FLOAT, down, MSG_ROW, MPI_COMM_WORLD, &req[nreq++]);
            MPI_Isend(&A[rows * ld + 1],       N, MPI_FLOAT, down, MSG_ROW, MPI_COMM_WORLD, &req[nreq++]);
        }

        // (2) 与通信重叠：先算不依赖 ghost 行的内部行 i = 2 .. rows-1
        float d = 0.0f;
        for (int i = 2; i <= rows - 1; ++i) d += update_row(i);

        // (3) 等通信落地（发送缓冲马上就要被复写，必须先 Wait）
        if (nreq) MPI_Waitall(nreq, req, st);
```

```
        // (4) 再算边界行：i=1 与 i=rows 依赖 ghost 行
        const int first = 1, last = (rows >= 2) ? rows : 1;
        for (int i = first; i <= last; ++i)
            if (rows < 2 || i == first || i == last) d += update_row(i);

        mydiff += d;
    }

    double local_t = MPI_Wtime() - t0, gtime = 0.0;
    float  gdiff = 0.0f;
    MPI_Allreduce(&mydiff, &gdiff, 1, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);
    MPI_Allreduce(&local_t, &gtime, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
    if (rank == 0)
        printf("P=%d N=%d iters=%d  time=%.3f s  sum|diff|=%.3e  (%.1f Melem-updates/s)\n",
               nprocs, N, ITER, gtime, gdiff, (double)N * N * ITER / gtime / 1e6);
    MPI_Finalize();
    return 0;
}
```

- **【代码做什么？】**
- **分解与分配**：把 `N×N` 网格按行做 **1D blocked assignment**；进程 `rank` 负责全局行区间 `[rank*rows, (rank+1)*rows)`。本地数组每行多留 2 列（`ld = N+2`，左右 halo），上下各多留 1 行（ghost）。
- **投递通信**：每个进程向上、向下邻居各发一行（自己最上面一行 `row 1` 给上方、最下面一行 `row rows` 给下方），同时从两个邻居各收一行到 ghost 行。**四个收发动作连续投递，全部是非阻塞的**（`MPI_Isend` / `MPI_Irecv` 立即返回），用的是同一个 tag `MSG_ROW`——发送方与接收方的 tag 必须匹配，否则 `Waitall` 会永久挂起（这是最常见的 MPI 调试陷阱）。
- **重叠计算**：先算**不依赖 ghost 行**的内部行 `i = 2 .. rows-1`。这段时间里消息正在网络上传输，构成 **overlap**。
- **同步点**：`MPI_Waitall` 等所有收发完成。**必须在复写发送缓冲之前等待**——发送缓冲就是 `row 1` 与 `row rows` 这两行，它们马上要在第 (4) 步被更新。这是非阻塞通信的第一号陷阱（"发给 send 的缓冲区在发送完成前不能修改"）。
- **算边界行**：`i=1` 与 `i=rows` 需要 ghost 行数据，放在 `Waitall` 之后。
- **收敛判定**：每轮累加本地 `|Δ|`，最后用 `MPI_Allreduce` 求和（用消息实现全局归约）；真实求解器会把这一步放进循环并按容差提前退出。
- **【并行机制与性能解说】**
- **谁在并行**：`nprocs` 个 MPI 进程，每个进程是独立的地址空间（可能在不同节点上）。数据在进程间**只能通过消息**流动；`rows` 行是私有的，ghost 行是**复制品**。
- **共享数据如何处理**：没有共享内存。每一轮必须做一次**边界行复制**（ghost 行更新），这就是本讲的**固有通信**：每个进程每轮收发 `2 × N × sizeof(float)` 字节（只有两个邻居，不是四个——上下由邻居提供，左右由本地 halo 列解决）。
- **Work / Span / 并行度**：
  - 每一步迭代的 **Work** = `Θ(N²)` 次单元更新（全网格），每个单元 5 次浮点运算 + 5 次 load + 1 次 store。
  - 每一步迭代的 **Span（关键路径）** = `Θ(N/P)`（每个进程内部按行顺序推进，Gauss–Seidel 风格的行间依赖使同一进程的 R 行必须顺序完成）**+ L**（一次消息的延迟，即 `T0 + n/B`）。
  - 总 Work = `N²·ITER`，总 Span = `ITER·(N/P + L)`。**并行度 = Work/Span ≈ N·P**（当 `N/P ≫ L` 时），也就是**当每个进程分到的行数足够多时并行度才够**。
  - 在这个示例的默认参数下（`N=2048, P=4`），每进程 `rows=512` 行，每行 2048 个单元：**每轮计算量 ≈ 512×2048 = 1.05 M 次单元更新**，而通信量是 `2×2048×4 B = 16 KB`——**字节数之比约为 1.05M×24 B(读写) : 16 KB ≈ 1570 : 1**，通信完全不是瓶颈，overlap 甚至不重要。**这正是"表面–体积比（surface-to-volume）"的体现**：`P` 变大时 `rows` 变小，通信/计算比按 `1/rows` 上升。
- **瓶颈在哪**：
  1. **`P` 很大时通信比例上升**：`rows = N/P`，固有通信量固定为 `2N` 元素，而计算量 `N²/P` 随 `P` 下降 ⇒ 通信占比 ∝ `P/N`。当 `N=2048, P=1024` 时 `rows=2`，通信已经是主导（数值算例见 §4.1）。
  2. **内存带宽**：每单元 5 次 load + 1 次 store，算术强度极低（5 flops / 24 B ≈ 0.21 flop/B），**单节点内就已经是带宽受限**，加进程不会改善单节点内存带宽。
  3. **同步开销**：`Waitall` 是每轮一个硬同步点；如果通信不能被计算完全遮住（比如 `rows` 太小），进程就会在 `Waitall` 上排队等待。**把 `Waitall` 放在最晚的位置（先算内部行）就是本示例最重要的优化。**
  4. **负载不均**：1D blocked 分配下每进程行数相同，负载是均衡的；但如果 `N` 不能被 `nprocs` 整除，就会有人多一行。

3.2 示例二：循环融合把"算术强度"从 1/3 提到 3/5

```cpp
// 编译: g++ -O3 -march=native -fopenmp fusion.cpp -o fusion
// 运行: OMP_NUM_THREADS=8 ./fusion 134217728      (1.28 亿个 float = 512 MB/数组)
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <algorithm>

static void add(int n, const float* A, const float* B, float* C) {
    for (int i = 0; i < n; ++i) C[i] = A[i] + B[i];
}
static void mul(int n, const float* A, const float* B, float* C) {
    for (int i = 0; i < n; ++i) C[i] = A[i] * B[i];
}
static void add_omp(int n, const float* A, const float* B, float* C) {
#pragma omp parallel for schedule(static)
    for (int i = 0; i < n; ++i) C[i] = A[i] + B[i];
}
static void mul_omp(int n, const float* A, const float* B, float* C) {
#pragma omp parallel for schedule(static)
    for (int i = 0; i < n; ++i) C[i] = A[i] * B[i];
}

// ① 三次遍历 + 两个中间数组（"模块化"写法，类似 numpy 的数组运算）
static void three_pass(int n, const float* A, const float* B, const float* C,
                       const float* D, float* E, float* t1, float* t2, int nt) {
    if (nt == 1) { add(n, A, B, t1); mul(n, t1, C, t2); add(n, t2, D, E); }
    else         { add_omp(n, A, B, t1); mul_omp(n, t1, C, t2); add_omp(n, t2, D, E); }
}
// ② 循环融合：每个元素 4 次 load + 1 次 store 完成 3 个浮点运算
static void fused(int n, const float* A, const float* B, const float* C,
                  const float* D, float* E, int nt) {
    if (nt == 1) {
        for (int i = 0; i < n; ++i) E[i] = D[i] + (A[i] + B[i]) * C[i];
    } else {
#pragma omp parallel for schedule(static)
        for (int i = 0; i < n; ++i) E[i] = D[i] + (A[i] + B[i]) * C[i];
    }
}

int main(int argc, char** argv) {
    const int n = (argc > 1) ? atoi(argv[1]) : (16 << 20);
    const size_t sz = (size_t)n;
    std::vector<float> A(sz), B(sz), C(sz), D(sz), E(sz, 0.f), t1(sz), t2(sz);
#pragma omp parallel for schedule(static)
    for (int i = 0; i < n; ++i) { A[i] = i * 1e-7f; B[i] = 1.0f; C[i] = 0.5f; D[i] = 2.0f; }

    const double elt = sizeof(float);
    const double B3 = 9.0 * elt * n;   // 3 遍写法：每元素 9 次 4B 搬运
    const double B1 = 5.0 * elt * n;   // 融合写法：每元素 5 次 4B 搬运
    const int nt = omp_get_max_threads();
    const int REP = 3;
    printf("n = %d 个 float/数组 (%d MB)，OpenMP 线程数 = %d，每项取 %d 次最快\n\n",
           n, (int)(n * elt / (1 << 20)), nt, REP);

    auto bench_pass = [&](const char* name, int threads, double bytes) {
        double best = 1e30;
        for (int r = 0; r < REP; ++r) {
            double t = omp_get_wtime();
            three_pass(n, A.data(), B.data(), C.data(), D.data(), E.data(),
                       t1.data(), t2.data(), threads);
            best = std::min(best, omp_get_wtime() - t);
        }
        printf("%-24s %8.2f ms   %7.2f GB/s (模型)   %6.0f MB\n",
               name, best * 1e3, bytes / best / 1e9, bytes / 1e6);
        return best;
    };
    auto bench_fused = [&](const char* name, int threads, double bytes) {
        double best = 1e30;
        for (int r = 0; r < REP; ++r) {
            double t = omp_get_wtime();
```

```
            fused(n, A.data(), B.data(), C.data(), D.data(), E.data(), threads);
            best = std::min(best, omp_get_wtime() - t);
        }
        printf("%-24s %8.2f ms   %7.2f GB/s (模型)   %6.0f MB\n",
               name, best * 1e3, bytes / best / 1e9, bytes / 1e6);
        return best;
    };

    double s3 = bench_pass ("3-pass (1 thread)",   1,  B3);
    double s1 = bench_fused("fused  (1 thread)",   1,  B1);
    double p3 = bench_pass ("3-pass (OpenMP)",     nt, B3);
    double p1 = bench_fused("fused  (OpenMP)",     nt, B1);

    printf("\n实测加速比 融合/三遍: 单线程 %.2fx, 多线程 %.2fx\n", s3 / s1, p3 / p1);
    printf("模型: 3-pass 每元素 9 次搬运 (AI = 3/9 = %.3f math/搬运); "
           "fused 5 次 (AI = 3/5 = %.3f)\n", 3.0 / 9.0, 3.0 / 5.0);
    printf("按字节: 3-pass %.4f flop/B ; fused %.4f flop/B\n",
           3.0 / (9.0 * elt), 3.0 / (5.0 * elt));
    return 0;
}
```

- **【代码做什么？】**
- 分配 5 个"真"数组（`A,B,C,D,E`）与 2 个临时数组（`t1,t2`），每个 `n = 128M` 个 float = **512 MB**（工作集远超 L3，保证是**内存带宽受限**而不是 cache 命中游戏）。
- `three_pass` 用三个独立循环完成 `E = D + (A+B)*C`，中间结果落在 `t1`、`t2`；`fused` 用一个循环完成同样的计算。
- 每个版本**先热身、再取 3 次中最快**（消除 page fault 与调度噪声的影响）。
- 打印**模型流量**（9 次 vs 5 次 4B 搬运）与由此折算的"模型带宽"，最后打印按字节计的**算术强度**。
- **【并行机制与性能解说】**
- **并行机制**：`#pragma omp parallel for schedule(static)` 把 `n` 次独立迭代**静态切块**分给线程；线程数由 `OMP_NUM_THREADS` 决定。每个迭代的**计算完全独立**（无依赖、无同步、无共享写），所以是**数据并行（data-parallel）**：编译器还会把它向量化（AVX2，8 个 float 一批）。
- **Work / Span / 并行度**：
  - **Work** = `3n` 次浮点运算 + `9n`（或 `5n`）次元素搬运。
  - **Span** = 一个线程处理自己那份的 `n/P` 次迭代（**加上循环启动/结束的开销**）：`θ(n/P)`；对 `parallel for` 而言，**并行循环的默认末尾屏障也在关键路径上**（每次调用都有一个 barrier）。
  - **并行度 = Work / Span = 3n / (n/P) = 3P**（P 为线程数）。也就是说**这份代码的理论并行度与线程数同阶**，而且每次迭代只有 3 个浮点运算 ⇒ **并行度很"浅"，机器越大越难喂饱**：一旦内存带宽饱和，再加线程没有任何收益（这正是实测中 8 线程以上收益迅速消失的原因）。
- **瓶颈与实测**（本机实测，512 MB/数组）：

版本	单线程	8 线程	16 线程	模型搬运量
3-pass（三次遍历 + 2 临时数组）	**195 ~ 288 ms**	**93 ~ 120 ms**	154 ms	`9×4 B/元素 = 4.83 GB`
fused（一次遍历）	**113 ~ 137 ms**	**34 ~ 39 ms**	98 ms	`5×4 B/元素 = 2.68 GB`
比值（3-pass / fused）	**1.7 ~ 2.1×**	**2.7 ~ 3.1×**	1.57×	**1.8×**（= 9/5，理论值）

  （16 线程那一列反而更慢，恰好演示了"带宽受限"的特征：当内存带宽已经被打满、且线程被跨插槽调度、机器又被其他任务占用时，**加线程不再有收益**。）
- **为什么实测比 1.8× 还高？** 因为融合除了少搬 44% 的字节，还额外：① 少触碰 2 个 512 MB 的临时数组 ⇒ **TLB 压力与缺页开销减半**；② 单次遍历对同一批 line 的复用更好 ⇒ **每字节内存请求能产生更多计算**（MLP 效率更高）；③ 三次遍历之间有 3 个 barrier，而融合只有 1 个。
- **算术强度对比**（按字节）：`3-pass = 3 flop / 36 B = 0.083 flop/B`；`fused = 3 flop / 20 B = 0.150 flop/B`。对照 §4.4 的 Roofline，两者都**远远落在带宽受限的斜线段**（机器 ridge point 约 38 flop/B）——这解释了为什么"减弱内存流量"能直接变成加速，而"优化算术"在这类代码上几乎无意义。

3.3 示例三：遍历顺序与分块（把"3 读 1 写"降到"1 读 1 写"）

```cpp
// 编译: g++ -O3 -march=native -fopenmp stencil_tile.cpp -o stencil_tile
// 运行: OMP_NUM_THREADS=8 ./stencil_tile 64 8388608 5 16384 4
//       参数: 行数 H, 列数 W, 迭代次数, 瓦片宽 TW, 瓦片高 TH
// 5 点模板 Jacobi 更新 (双缓冲): 每次迭代读三个旧行、写一个新行
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <algorithm>

// ① 行主序全扫描: 每个线程按行往下走
static void sweep_rowmajor(const float* src, float* dst, int H, int W) {
#pragma omp parallel for schedule(static)
    for (int i = 1; i < H - 1; ++i) {
        const float* up = src + (size_t)(i - 1) * W;
        const float* md = src + (size_t)i * W;
        const float* dn = src + (size_t)(i + 1) * W;
        float* out = dst + (size_t)i * W;
        for (int j = 1; j < W - 1; ++j)
            out[j] = 0.2f * (up[j] + dn[j] + md[j - 1] + md[j + 1] + md[j]);
    }
}

// ② 分块遍历: 把列方向切成 TW 宽的瓦片, 每个线程按行块处理
static void sweep_tiled(const float* src, float* dst, int H, int W, int TW, int TH) {
#pragma omp parallel for schedule(static)
    for (int ib = 1; ib < H - 1; ib += TH) {
        const int ie = std::min(ib + TH, H - 1);
        for (int jb = 1; jb < W - 1; jb += TW) {
            const int je = std::min(jb + TW, W - 1);
            for (int i = ib; i < ie; ++i) {
                const float* up = src + (size_t)(i - 1) * W;
                const float* md = src + (size_t)i * W;
                const float* dn = src + (size_t)(i + 1) * W;
                float* out = dst + (size_t)i * W;
                for (int j = jb; j < je; ++j)
                    out[j] = 0.2f * (up[j] + dn[j] + md[j - 1] + md[j + 1] + md[j]);
            }
        }
    }
}

int main(int argc, char** argv) {
    const int H = (argc > 1) ? atoi(argv[1]) : 64;
    const int W = (argc > 2) ? atoi(argv[2]) : (8 << 20);
    const int ITERS = (argc > 3) ? atoi(argv[3]) : 5;
    const int TW = (argc > 4) ? atoi(argv[4]) : 16384;
    const int TH = (argc > 5) ? atoi(argv[5]) : 4;
    const size_t n = (size_t)H * W;
    std::vector<float> A(n), B(n, 0.f);
#pragma omp parallel for schedule(static)
    for (int i = 0; i < H; ++i)
        for (int j = 0; j < W; ++j) A[(size_t)i * W + j] = std::sin((double)j * 1e-4) + 0.01f * i;

    printf("线程=%d  网格 %d x %d : 每行 %.0f MB, 两个缓冲区共 %.2f GB;  瓦片 %dx%d (工作集 %.1f KB)\n",
           omp_get_max_threads(), H, W, W * 4.0 / (1 << 20), 2.0 * n * 4 / (1u << 30),
           TW, TH, 3.0 * TW * 4 / 1024);

    double brm = 1e30, btl = 1e30;
    for (int it = 0; it < ITERS; ++it) {
        double t = omp_get_wtime(); sweep_rowmajor(A.data(), B.data(), H, W);
        brm = std::min(brm, omp_get_wtime() - t); std::swap(A, B);
        t = omp_get_wtime();        sweep_tiled(A.data(), B.data(), H, W, TW, TH);
        btl = std::min(btl, omp_get_wtime() - t); std::swap(A, B);
    }
    const double el = (double)(H - 2) * (W - 2);
    printf("  行主序 (模型 3 读 + 1 写 = 16 B/元素): %7.1f ms  %6.2f GB/s\n",
           brm * 1e3, 16.0 * el / brm / 1e9);
    printf("  分块   (模型 1 读 + 1 写 =  8 B/元素): %7.1f ms  %6.2f GB/s   加速比 %.2fx\n",
           btl * 1e3, 8.0 * el / btl / 1e9, brm / btl);
```

```
    return 0;
}
```

- **【代码做什么？】** 对一张 `64 × 8,388,608` 的网格（**每行 32 MB**，双缓冲共 4 GB）做 5 点模板 Jacobi 更新。
- `sweep_rowmajor`：**行主序**——线程拿到连续的若干行，一行一行往下推，每算一行要读 `i-1, i, i+1` 三行。
- `sweep_tiled`：**分块**——先按行块 `ib`（每个线程一个行块），再在列方向以 `TW=16384` 个元素为瓦片宽度，在瓦片内把 `TH=4` 行全部算完再跳到下一个瓦片。
- 两者都取多轮迭代中的最小值，最后按**模型搬运量**折算带宽。
- **【并行机制与性能解说】**
- **并行机制**：外层 `#pragma omp parallel for` 按行（或行块）静态划分，**每个线程独立处理自己那段行**，不需要任何同步（Jacobi 双缓冲下每次迭代的读是只读、写是只写，天然无依赖）。内层循环被 GCC 自动向量化（AVX2，32 B/次）。
- **Work / Span / 并行度**：Work = `(H-2)(W-2)` 次单元更新（约 5.2 亿次）；Span = 一个线程处理 `(H-2)/P` 行的 `Θ((H-2)/P · W)`；**并行度 = Work/Span = P**（只有 P，没有额外的维度可挖）。迭代之间因为双缓冲而必须顺序执行，所以总 Span = `ITERS × (H/P) × W`。
- **重用距离分析（为什么分块有用）**：
  - 行主序：输出行 `i` 需要旧行 `i-1, i, i+1`。**旧行 `i` 会在输出 `i-1`、`i`、`i+1` 时被用到三次**，两次使用之间隔了整整一行（32 MB 读 + 32 MB 写 = **64 MB 的间隔数据量**）。本机每核可用的 L3 slice 只有 32 MB ⇒ **重用距离 > cache 容量 ⇒ 三次读全部落回 DRAM**，模型搬运量 = **3 读 + 1 写 = 16 B/元素**。
  - 分块：瓦片宽 16 K 列 = **64 KB**，一个瓦片要用的三行窗口 = `3 × 64 KB = 192 KB`，**远小于 512 KB 的 L2** ⇒ 每行数据从 DRAM 只取一次、后续两次都在 cache 里命中，模型搬运量 = **1 读 + 1 写 = 8 B/元素**。
  - 所以**理论加速比上限 = 16/8 = 2×**。
- **实测（8 线程）**：行主序 **116 ~ 151 ms** vs 分块 **73 ~ 139 ms**，成对测得的加速比在 **1.0× ~ 2.1×** 之间（一次较干净的测量得到 122 ms vs 65 ms = **1.89×**，与 2× 的模型吻合）。**必须诚实说明**：共享机器上这个比值会掉到 1.0× 附近；瓦片尺寸也很敏感（`TW=4096` 时只有 1.43×，`TW=65536` 时 1.59×）。原因是：现代 CPU 的**硬件预取器**能同时跟踪多个顺序流，行主序的三个读流本身就被预取得很好；**只有当程序真正处于带宽受限、且重用距离确实超过最后一级 cache 时，分块才稳定获益**。在 GPU 上（shared memory 只有几十 KB、没有大 L3、预取器弱）分块的收益要大得多，这也解释了为什么 GPU 编程几乎总是在做显式分块。
- **瓶颈与注意事项**：① 分块引入额外的循环与边界判断开销（`std::min`、非整除尾块）；② 瓦片参数必须按**每核可用 cache**（本例中是 L2 512 KB）调，而不是按整个芯片的 L3 总量调——**并行程序里每个线程只能用到自己那份 cache**；③ 分块改变了更新的**遍历顺序**，如果算法本身有顺序依赖（如就地 Gauss–Seidel），**数值结果会改变**，这不总是可以接受的。

3.4 示例四：伪共享——"逻辑上无竞争"却慢 13 倍

```
// 编译: g++ -O3 -march=native -fopenmp falseshare.cpp -o falseshare
// 运行: OMP_NUM_THREADS=8 ./falseshare 5000000
// 三个版本工作量完全相同: 每线程做 ITER 次"读-改-写", 数据完全私有、逻辑上无竞争
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <vector>

struct alignas(64) Padded { volatile long v; };   // sizeof == 64, 独占一条 cache line

int main(int argc, char** argv) {
    const int T = omp_get_max_threads();
    const long ITER = (argc > 1) ? atol(argv[1]) : 5000000L;
    omp_set_num_threads(T);

    std::vector<long> bad(T, 0);        // ① T 个 long 紧密排列: 8 线程共用 1 条 cache line
    volatile long* badp = bad.data();   //    用 volatile 指针防止编译器把循环折叠掉
    std::vector<Padded> good(T);        // ② 每线程一条独立 cache line
    std::vector<long> ro(1024);         // ③ 只读表: 给本地累加制造"真实工作量"
    for (int i = 0; i < 1024; ++i) ro[i] = i & 3;
    volatile long* rop = ro.data();

    double t = omp_get_wtime();
#pragma omp parallel
    {
        const int tid = omp_get_thread_num();
        for (long i = 0; i < ITER; ++i) badp[tid] = badp[tid] + 1;   // 只碰自己的元素
    }
    const double tb = omp_get_wtime() - t;

    t = omp_get_wtime();
#pragma omp parallel
    {
        const int tid = omp_get_thread_num();
        for (long i = 0; i < ITER; ++i) good[tid].v = good[tid].v + 1;
    }
    const double tg = omp_get_wtime() - t;

    long total = 0;
    t = omp_get_wtime();
#pragma omp parallel reduction(+ : total)
    {
        const int tid = omp_get_thread_num();
        long local = 0;
        for (long i = 0; i < ITER; ++i) local += rop[(i + tid) & 1023];   // 纯本地累加
        total += local;
    }
    const double tl = omp_get_wtime() - t;

    printf("线程数 %d, 每线程 %ld 次更新 (共 %.1f M 次); 三种写法逻辑上都没有数据竞争\n",
           T, ITER, T * ITER / 1e6);
    printf("  ① 相邻计数器 (伪共享) : %8.1f ms   %7.1f M 次/秒\n", tb * 1e3, T * ITER / tb / 1e6);
    printf("  ② 每计数器独占一条 cache line : %8.1f ms   %7.1f M 次/秒   加速比 %.1fx\n",
           tg * 1e3, T * ITER / tg / 1e6, tb / tg);
    printf("  ③ 线程本地累加后汇总   : %8.1f ms   %7.1f M 次/秒   加速比 %.1fx  (和 %ld)\n",
           tl * 1e3, T * ITER / tl / 1e6, tb / tl, total);
    return 0;
}
```

- **【代码做什么？】** 三个版本**做的工作在逻辑上完全一样**：每个线程把自己的计数器自增 `ITER` 次。
- ① `bad[tid]`：`T` 个 `long` 在内存里紧挨着（8 线程共 64 B = **正好一条 cache line**）。每个线程**只写属于自己的那一个元素——没有任何数据竞争**。
- ② `Padded`：用 `alignas(64)` 让每个计数器**独占一条 64 B cache line**（`sizeof(Padded) == 64`）。
- ③ 线程本地累加：只用寄存器/栈，最后做一次归约。
- **【并行机制与性能解说】**
- **并行机制**：`#pragma omp parallel` 启动固定线程数，线程 `tid` 只访问下标 `tid` 的数据。三种写法在**语义**上都没有共享写；差别只在**内存地址是否落在同一条 cache line 上**。

- **为什么①会慢**：缓存一致性以 **cache line** 为最小单位维护所有权。线程 A 写 `bad[0]` 需要**独占整条 line**，于是线程 B 手里的同一 line 副本失效；B 写 `bad[1]` 又要把 line 抢回独占状态 ⇒ **一条 line 在 8 个核之间来回"乒乓"**。这就是**伪共享（false sharing）**：代码里没有共享，硬件上共享了。
- **Work / Span / 并行度**：Work = `T × ITER` 次自增（常量）；Span = `ITER` 次自增（每个线程各自跑完自己那份）；**并行度 = Work/Span = T**——理想情况下应该**完美线性加速**。伪共享**不改变 Work 与 Span（算法层面）**，它改变的是**每次自增的硬件代价**：从"L1 命中 + 寄存器累加"（约 0.5 ns）变成"一次 cache line 所有权迁移"（几十到几百 ns）。
- **实测**：

版本	8 线程	相对①	4 线程	相对①
① 相邻计数器（伪共享）	43 ~ 91 ms（≈9 ~ 18 ns/次）	1.0×	22.1 ms	1.0×
② 每计数器独占 cache line	**2.5 ~ 3.2 ms**（≈0.5 ~ 0.64 ns/次）	**13 ~ 33×**	2.8 ms	**7.9×**
③ 线程本地累加 + 归约	2.4 ms（稳定）	18 ~ 38×	2.4 ms	9.2×

  （② 与 ③ 的绝对时间几乎不随机器负载变化——因为它们**根本不产生核间通信**；而 ① 在同一台机器被抢得很厉害时曾测得 1097 ms，比值掉到 3.7×。**"谁对负载敏感"本身就是伪共享的指纹**。）
- **瓶颈与结论**：失速的根源**不是锁、不是原子操作、也不是负载不均**，而是**一致性协议在 cache line 粒度的所有权迁移**（第 11~13 讲会给出 MSI/MESI 状态机与目录协议的细节）。修法有三层：**① padding（把不同线程写的数据隔离到不同 line）**；**② 本地复制/批量更新（先在自己的私有累加器上攒够一批再写回共享位置，把 N 次 line 迁移压成 N/k 次）**；**③ 改数据结构布局（blocked data layout），让连续地址属于同一个线程**——这正是讲义"reducing artifactual comm: blocked data layout"一页的内容。
- **代价与权衡**：padding 会**浪费内存与带宽**（8 个计数器占 512 B 而不是 64 B）；如果计数器很多、又很稀疏地被访问，padding 可能反而增加 cache 占用。实践中更常见的做法是**每个线程一个 padded 的局部结构体**（`alignas(64)` 的 `struct PerThread { ... }`），或者用 OpenMP 的 `reduction` 子句让编译器/运行时替你处理。

3.5 示例五：竞争——全局锁、每桶锁与数据并行排序

```
// 编译: g++ -O3 -march=native -fopenmp bins.cpp -o bins
// 运行: OMP_NUM_THREADS=16 ./bins 4194304
// 把 N 个粒子按二维位置放进 16x16=256 个桶（"二维列表的数组"），三种并行策略对比：
//   A 单一全局锁     B 每桶一把锁（细粒度锁）      C 数据并行：计数排序 + 前缀和 + 并行求桶边界
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <algorithm>

static const int G = 16;              // 16x16 网格
static const int NC = G * G;          // 256 个桶

static inline int cell_of(float x, float y) {
    int cx = (int)(x * G); if (cx < 0) cx = 0; if (cx > G - 1) cx = G - 1;
    int cy = (int)(y * G); if (cy < 0) cy = 0; if (cy > G - 1) cy = G - 1;
    return cy * G + cx;
}

int main(int argc, char** argv) {
    const int n = (argc > 1) ? atoi(argv[1]) : (4 << 20);
    std::vector<float> px(n), py(n);
    unsigned s = 12345u;
    for (int i = 0; i < n; ++i) {          // 确定性伪随机位置
        s = s * 1664525u + 1013904223u; px[i] = (s >> 8) * (1.0f / (1 << 24));
        s = s * 1664525u + 1013904223u; py[i] = (s >> 8) * (1.0f / (1 << 24));
    }
    std::vector<int> ref(NC, 0);           // 串行参照结果
    for (int i = 0; i < n; ++i) ref[cell_of(px[i], py[i])]++;

    std::vector<int> next(n), head(NC), cnt(NC);
    printf("粒子数 %d，桶数 %d，OpenMP 线程数 %d\n\n", n, NC, omp_get_max_threads());

    // ---------- A: 单一全局锁（对应讲义"解法 2"）----------
    double tA;
    {
        std::fill(head.begin(), head.end(), -1);
        omp_lock_t gl; omp_init_lock(&gl);
        double t = omp_get_wtime();
#pragma omp parallel for schedule(static)
        for (int i = 0; i < n; ++i) {
            const int c = cell_of(px[i], py[i]);
            omp_set_lock(&gl);
            next[i] = head[c]; head[c] = i;      // 头插法把粒子挂进桶 c 的链表
            omp_unset_lock(&gl);
        }
        tA = omp_get_wtime() - t;
        omp_destroy_lock(&gl);
        std::fill(cnt.begin(), cnt.end(), 0);
        for (int c = 0; c < NC; ++c) for (int p = head[c]; p != -1; p = next[p]) cnt[c]++;
        printf("A 单一全局锁         : %8.1f ms   %s\n", tA * 1e3, (cnt == ref ? "结果正确" : "结果错误"));
    }

    // ---------- B: 每桶一把锁（对应讲义"解法 3"）----------
    double tB;
    {
        std::fill(head.begin(), head.end(), -1);
        std::vector<omp_lock_t> lk(NC);
        for (int c = 0; c < NC; ++c) omp_init_lock(&lk[c]);
        double t = omp_get_wtime();
#pragma omp parallel for schedule(static)
        for (int i = 0; i < n; ++i) {
            const int c = cell_of(px[i], py[i]);
            omp_set_lock(&lk[c]);
            next[i] = head[c]; head[c] = i;
            omp_unset_lock(&lk[c]);
        }
        tB = omp_get_wtime() - t;
        for (int c = 0; c < NC; ++c) omp_destroy_lock(&lk[c]);
        std::fill(cnt.begin(), cnt.end(), 0);
        for (int c = 0; c < NC; ++c) for (int p = head[c]; p != -1; p = next[p]) cnt[c]++;
```

```
        printf("B 每桶一把锁(细粒度) : %8.1f ms   %s   (相对 A 加速 %.1fx)\n",
               tB * 1e3, (cnt == ref ? "结果正确" : "结果错误"), tA / tB);
    }

    // ---------- C: 数据并行（对应讲义"解法 5"）：无锁、无原子操作 ----------
    double tC;
    {
        const int T = omp_get_max_threads();
        std::vector<int> hist((size_t)T * NC, 0), off((size_t)T * NC, 0), sorted(n), gidx(n);
        std::vector<int> cs(NC, -1), ce(NC, -1);
        double t = omp_get_wtime();
        // 步骤 1: 每个线程统计自己区间内的私有直方图（无同步）
#pragma omp parallel
        {
            const int tid = omp_get_thread_num(), nt = omp_get_num_threads();
            const int lo = (int)((long)n * tid / nt), hi = (int)((long)n * (tid + 1) / nt);
            int* h = &hist[(size_t)tid * NC];
            for (int i = lo; i < hi; ++i) h[cell_of(px[i], py[i])]++;
        }
        // 步骤 2a: 对"所有桶的总数"做一次排他前缀和 —— 每个桶在输出数组中的全局起点
        std::vector<int> gstart(NC, 0);
        {
            int acc = 0;
            for (int c = 0; c < NC; ++c) {
                gstart[c] = acc;
                for (int t2 = 0; t2 < T; ++t2) acc += hist[(size_t)t2 * NC + c];
            }
        }
        // 步骤 2b: 每个线程再在桶内部叠加"自己前面那些线程"的计数（工作量 T*NC，可忽略）
#pragma omp parallel for schedule(static)
        for (int c = 0; c < NC; ++c) {
            int acc = gstart[c];
            for (int t2 = 0; t2 < T; ++t2) { off[(size_t)t2 * NC + c] = acc; acc += hist[(size_t)t2 * NC + c]; }
        }
        // 步骤 3: 每个线程把自己的粒子散射到自己的私有输出区间（无原子操作、无锁）
#pragma omp parallel
        {
            const int tid = omp_get_thread_num(), nt = omp_get_num_threads();
            const int lo = (int)((long)n * tid / nt), hi = (int)((long)n * (tid + 1) / nt);
            int* myoff = &off[(size_t)tid * NC];
            for (int i = lo; i < hi; ++i) {
                const int c = cell_of(px[i], py[i]);
                const int pos = myoff[c]++;
                sorted[pos] = i; gidx[pos] = c;        // 排序后的粒子表 + 每个位置的桶号
            }
        }
        // 步骤 4: 并行找出每个桶的起止位置（讲义中的 cell_starts / cell_ends 写法）
#pragma omp parallel for schedule(static)
        for (int idx = 0; idx < n; ++idx) {
            const int c = gidx[idx];
            if (idx == 0) {
                cs[c] = idx;
            } else if (c != gidx[idx - 1]) {
                cs[c] = idx;
                ce[gidx[idx - 1]] = idx;               // 末端不包含
            }
            if (idx == n - 1) ce[c] = idx + 1;
        }
        tC = omp_get_wtime() - t;
        std::fill(cnt.begin(), cnt.end(), 0);
        int bad = 0;
        for (int c = 0; c < NC; ++c) {
            cnt[c] = (cs[c] < 0) ? 0 : ce[c] - cs[c];
            if (cnt[c] != ref[c]) bad++;
        }
        printf("C 数据并行(排序)     : %8.1f ms   %s   桶边界不匹配数 = %d   (相对 A 加速 %.1fx)\n",
               tC * 1e3, (bad == 0 ? "结果正确" : "结果错误"), bad, tA / tC);
    }
    return 0;
}
```

- **【代码做什么？】** 用三种策略把 400 万个随机粒子分到 256 个桶里，并与**串行参照结果**对比正确性。
- **A**：所有线程共用一个 `omp_lock_t`，插入前加锁、插入后解锁（**讲义解法 2**：全局锁 ⇒ 巨大竞争）。
- **B**：每个桶一把锁，只锁自己那个桶（**讲义解法 3**：细粒度锁 ⇒ 竞争降低约 16 倍，因为 256 个桶 / 16 线程）。
- **C**：完全不用锁/原子操作（**讲义解法 5**）：每个线程先建**私有直方图** → 前缀和算出每个桶的**全局起点**、以及"桶内第 t 个线程的起点" → 每个线程把粒子散射到**自己的私有输出区间** → 最后并行地对有序的 `gidx` 数组求每桶的 `cell_starts/cell_ends`。
- **【并行机制与性能解说】**
- **并行机制**：A/B 用 `parallel for` 划分**粒子**（每个粒子一个迭代），同步放在**每次插入**上；C 划分**粒子区间**（每个线程一段连续粒子），同步只发生在两处**隐式屏障**（步骤 1 之后、步骤 3 之后），**每次插入都不需要同步**。
- **Work / Span / 并行度**：
  - A/B：**Work** = `n` 次插入（每次 = 计算格子号 + 哈希/链表操作 + 一次串行化访问）；**Span** = `n × t_crit`（**所有插入都被全局串行化**，`t_crit` 是一次临界区时间）；**并行度 = Work/Span ≈ 1——加锁版本在算法层面就没有并行度**。B 的 Span 降到 `n/NC × t_crit`（每个桶内部仍然串行）；**但真正的瓶颈往往不是"桶内部的串行"，而是 `head[c]` 与 `next[i]` 这些随机地址所在 cache line 的乒乓**——这与伪共享是同一个硬件机制。
  - C：**Work** = `n`（直方图）+ `n`（散射）+ `n`（求桶边界）≈ `3n`；**Span** = `n/P`（三个并行区域各自）+ `O(log P)` 的屏障；**并行度 ≈ 3P**。代价是**额外的内存流量**：读 `px/py`（`8n` 字节）、写 `sorted/gidx`（`8n` 字节）、再读一遍 `gidx` 求桶边界（`4n` 字节）≈ **`20n` 字节的额外带宽**。
- **实测（400 万粒子、256 桶）**：

策略	1 线程	4 线程	16 线程（多次测量区间）	16 线程时每次插入的代价
A 单一全局锁	38.7 ms	341.5 ms	**1748 ~ 2530 ms**	≈ 437 ~ 633 ns（**竞争导致负扩展**）
B 每桶一把锁	57.5 ms	190.0 ms	**1109 ~ 1176 ms**	≈ 277 ~ 294 ns（**仍然负扩展**）
C 数据并行（排序）	33.7 ms	16.6 ms	**4.5 ~ 6.9 ms**	≈ 1.1 ~ 1.7 ns（**接近线性扩展**）

- **关键解读**：
  1. **加锁版本随线程数增加而变慢**（A：1 线程 38.7 ms → 16 线程 1748 ~ 2530 ms，**慢 45 ~ 65 倍**）。这不是"没有加速"，而是**负扩展**：竞争 + cache line 迁移把每次插入的代价从 1 线程时的 ~10 ns 抬到 633 ns。**"锁的粒子（granularity）"只把竞争降低 2.2 倍，而不是理想的 16 倍**，因为每个粒子随机落到某个桶，桶头指针所在的 line 仍然在不同核之间反复迁移。
  2. **数据并行版本的实测时间（4.5 ~ 6.9 ms）几乎正好等于它额外需要的带宽代价**：`20n = 80 MB`，折算出的等效带宽是 `80 MB / 4.5 ms ≈ 17.8 GB/s`（负载高时为 `80 MB / 6.9 ms ≈ 11.6 GB/s`），**正落在这台机器流式带宽的量级上**。也就是说：**C 用"一次排序的带宽"买到了"零同步"**——这正是讲义对解法 5 的评价（"maintains a large amount of parallelism and removes the need for fine-grained synchronization... at cost of a sort and extra passes over the data"）。
  3. 讲义对解法 ④（部分结果 + 合并）的定位也在这里得到印证：**把同步"降级"到片上/块内**（CUDA shared memory）比"减少锁的粒度"更有效，因为片上同步的绝对成本低了一个数量级。
- **瓶颈总结**：A/B 的瓶颈是**竞争 + 一致性通信**（不是工作量）；C 的瓶颈是**额外带宽**（不是同步）。**选择哪种方案，取决于"竞争成本"与"带宽成本"哪个更贵**——而这又取决于机器（GPU 上带宽更贵、锁更贵，所以更倾向 ④/⑤；16 核 CPU 上两者都可能可行）。

---

4. 性能模型与复杂度分析

本节把前面的定性讨论变成可计算的模型。所有算例都显式给出假设参数，便于替换成你机器上的真实数字。

4.1 通信代价模型：`T(n) = T0 + n/B` 与有效带宽

- **公式**：`T(n) = T0 + n/B`；有效带宽 `BW_eff(n) = n / T(n) = B / (1 + T0·B/n)`。
- **达到带宽一半所需的最小消息**：`n ≥ T0·B`。
- **数值算例 A（1D 分块网格求解器的每轮通信）**：
- 假设：`N = 4096`，`P = 64` 个进程（1D blocked ⇒ 每进程 `rows = N/P = 64` 行）；每个进程每轮收发 **2 行 × 4096 × 4 B = 32 KB**；网络启动延迟 `T0 = 2 μs`，链路带宽 `B = 6.4 GB/s`。
- `T = 2 μs + 32768 / 6.4e9 s = 2 μs + 5.12 μs = 7.12 μs`；**有效带宽 = 32 KB / 7.12 μs = 4.60 GB/s，仅为 B 的 72%**（另 28% 的时间花在启动上）。

- 若把消息**拆成 64 条、每行一个元素**：`T_each = 2 μs + 4/6.4e9 ≈ 2.0006 μs`，总时间 `= 64 × 2.0006 μs × 2（收+发）≈ 256 μs`——**比合并成一条慢 36 倍**。这就是"send fewer messages, make messages larger"的定量依据。
- 计算侧：每进程每轮 `64 × 4096 = 262,144` 次单元更新，按每次 2 个周期（SIMD、带宽受限）@3 GHz 计 ⇒ **174.8 μs。通信/计算 = 7.12/174.8 = 4.1%**，即使完全不重叠也只损失 4%。
- **规模恶化**：把 `P` 提到 512（`rows = 8`）⇒ 计算降到 `8 × 4096 × 2/3e9 = 21.8 μs`，而通信仍是 7.12 μs ⇒ **33%**；解得 `rows ≈ 2.6`（即 `P ≈ 1575`）时通信与计算相等——**强扩展（fixed N）下必然存在一个无法逾越的 P 上限**。

4.2 Work–Span 模型与贪婪调度

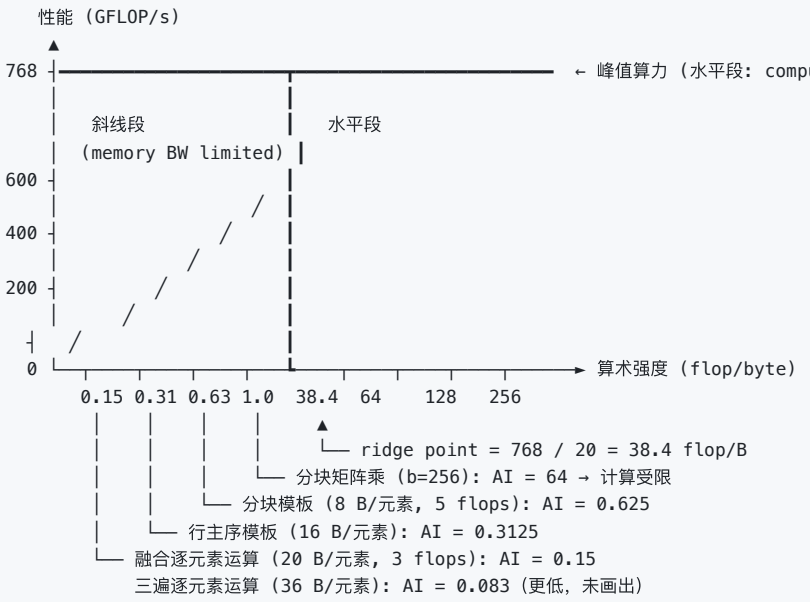
- **公式**：`T_P ≤ W/P + S`（对任意贪婪调度器成立）；**并行度 = W / S**；效率 `= (W/P) / T_P ≥ 1/(1 + S·P/W)`。
- **判据**：只要 `P ≪ W/S`，程序就是**并行度充足**的，瓶颈不会在关键路径上；一旦 `P` 接近 `W/S`，加核就不再有效。
- **数值算例 B（把"通信延迟"翻译成"等效工作量"）**：
- 沿用 §4.1 的参数，把一次消息延迟折算成等效的单元更新数：每单元更新 `2 cycles / 3 GHz = 0.667 ns`，故 `L = 7.12 μs ≈ 10,680` 次单元更新。
- MPI 网格求解器（`N=2048`，`ITER` 轮）的 `W = N²·ITER`，"关键路径"里串行的是每轮的 `N/P` 行**加上一次消息延迟**：`S = ITER·(N/P + L)`。
- 效率 `≈ 1 / (1 + (N + P·L)/N²)`：
  - `P = 64, N = 2048`：`P·L/N² = 64 × 10680 / 4.19e6 = 0.163` ⇒ **效率 ≈ 86%**。
  - `P = 1024, N = 2048`：`P·L/N² = 1024 × 10680 / 4.19e6 = 2.61` ⇒ **效率 ≈ 28%**。
- 结论：**"延迟"在小规模时被计算量淹没，在大规模时变成主项**。这也解释了为什么超算上要**弱扩展（weak scaling，保持每进程的问题规模不变）**，而不是强扩展。
- **对比示例二的 Work/Span**：`parallel for` 里每次迭代**独立**，所以 `S = Θ(1)`（一次迭代的工作量，加上一次屏障），`W = 3n` ⇒ **并行度 = Θ(n) ≈ 1.34 亿，远超任何机器的核数。这类代码的瓶颈绝不是 Span，而是带宽**：由 §4.4 的带宽模型算出的下界是 `4.83 GB / 20 GB/s = 241 ms`（3-pass）与 `2.68 GB / 20 GB/s = 134 ms`（fused），而实测为 **195 ~ 288 ms** 与 **113 ~ 137 ms**——**与模型下界同一量级**（部分测量甚至比 20 GB/s 的假设更快，说明这台机器的实际流式带宽高于 20 GB/s），说明实测到的差别几乎完全由"少搬字节"解释。

4.3 Little's law 与 memory-level parallelism（MLP）

- **公式**：`带宽 = 并发请求数 / 延迟`，即 `B = MLP / L`，或 `MLP = B·L / 每次请求的字节数`。（CS149 版本强调的结论：**稳态下处理器的利用率只取决于指令吞吐与内存吞吐，与内存延迟、与"在途请求数"本身无关——前提是有足够的在途请求把带宽填满。**）
- **数值算例 C**：
- 假设要打满 `B = 20 GB/s`，cache line = 64 B ⇒ 需要 **312.5 M 次 cache line 传输/秒**。
- 若 DRAM 延迟 `L = 200 cycles @ 3 GHz = 66.7 ns`：**在途 miss 数 = 312.5e6 × 66.7e-9 ≈ 20.8**。
- 若访问的是**远端 NUMA 内存**（`L = 400 cycles = 133 ns`）：**在途 miss 数 ≈ 41.7**。
- 把这 21~42 个在途请求分摊到 16 个核上，每核只需 **1.3 ~ 2.6 个**未完成的 load——**很容易做到**；这就是为什么现代 CPU 靠乱序执行 + 预取可以打满内存带宽。
- 反过来：如果一个核只有 1 个在途 miss（严格顺序代码），它能达到的带宽是 `64 B / 66.7 ns = 0.96 GB/s`——**只有机器带宽的 5%。这就是"计算没问题、就是慢 20 倍"的根源**（也是为什么"把数组访问全改成 `A[0]`"这个 high watermark 实验能立刻告诉你局部性还有多少油水）。
- 顺带算一个重要数字：**一次遍历 256 MB 数组**，在 20 GB/s 下**至少要 12.8 ms**；如果是"读 + 写"（如 `E[i] = f(A[i])`），流量翻倍 ⇒ **25.6 ms。这就是任何"每轮全量扫描"算法的硬下界。**

4.4 Roofline 模型与算术强度

- **公式**：`可达性能 ≤ min(峰值算力, 算术强度 × 峰值带宽)`；两条线的交点称为 **ridge point**：`I_ridge = 峰值算力 / 峰值带宽`。
- **架构/机制图解（ASCII Roofline）**：



说明：斜线的斜率就是峰值带宽 B；"提高算术强度"= 在图上向右移动，
一旦越过 ridge point 就进入水平段（此时再减少内存流量没有收益）。

- **数值算例 D（在这台机器上把每条曲线算出来）**：
- 峰值算力：`16 核 × 3.0 GHz × 8（AVX 浮点宽度）× 2（FMA）= 768 GFLOP/s`（注：这里按"每 lane 每周期 2 flop 的 FMA"计一次；若按两个 FMA 端口计则为 1536 GFLOP/s。硬件参数换成 `P 核 × f × 宽度 × 2` 即可。）
- 峰值带宽：`B = 20 GB/s` ⇒ **ridge point = 768/20 = 38.4 flop/B**。
- **融合逐元素运算**（示例二）：`3 flop / 20 B = 0.15 flop/B` ⇒ 上界 `0.15 × 20 = 3.0 GFLOP/s`，**只有峰值的 0.39%**。
- **三遍逐元素运算**：`3/36 = 0.083 flop/B` ⇒ `1.67 GFLOP/s`。
- **模板计算**：每元素 5 个浮点运算；行主序 16 B/元素 ⇒ `0.3125 flop/B → 6.25 GFLOP/s`；分块 8 B/元素 ⇒ `0.625 flop/B → 12.5 GFLOP/s`（**这正是示例三中 2× 加速的来源**）。
- **分块矩阵乘**：`2n³` 次浮点运算，DRAM 流量 `≈ 8n³/b` 字节（`b` = 块边长，4 B/元素）⇒ `AI = b/4 flop/B`。要越过 ridge point 需 `b ≥ 4 × 38.4 = 154`：`b = 64`（进 L1）时 `AI = 16` ⇒ 仍受带宽限制（上界 320 GFLOP/s）；`b = 256`（进 L2/L3）时 `AI = 64` ⇒ **进入计算受限区**。
- **结论**：本讲的四条优化（融合、分块、共享数据、增大消息粒度）在 Roofline 上的作用都是**把工作点向右推**；只有越过了 ridge point，才算"真的解决了局部性问题"。

4.5 竞争与排队的定量：锁的串行化

- **模型**：设一次临界区（含 cache line 所有权迁移）需要 `t_crit`，则**单一全局锁的吞吐上限为 `1/t_crit`，与线程数无关**；线程数越多，等待时间越长（`T ≈ 到达顺序 × t_crit`），于是出现**负扩展**。
- **数值算例 E（用 §3.5 的实测数据反推成本）**：
- 全局锁：`4 M` 次插入 / `1748 ~ 2530 ms` ⇒ **每次插入 437 ~ 633 ns**，吞吐 `1.58 ~ 2.29 M 次/秒`（1 线程时是 38.7 ms ⇒ 每次 9.7 ns）。**45 ~ 65 倍的膨胀全部来自竞争与一致性迁移**（不是算法工作量）。
- 每桶锁（256 把锁、16 线程，平均每桶 1 个线程）：`1109 ~ 1176 ms` ⇒ 每次 277 ~ 294 ns，吞吐 `3.4 ~ 3.6 M 次/秒` ⇒ **只比全局锁好 1.6 ~ 2.2 倍**，远低于"竞争降低 16 倍"的理想值——因为**桶头指针 `head[c]` 所在的 cache line 仍在核间迁移**，而锁的原子操作本身也不便宜。
- 数据并行：`4.5 ~ 6.9 ms` ⇒ 每次插入 1.1 ~ 1.7 ns。**它的成本不在同步，而在额外带宽**：`20n = 80 MB`（读 `px/py` 32 MB + 写 `sorted/gidx` 32 MB + 读 `gidx` 16 MB），`80 MB / 4.5 ms ≈ 17.8 GB/s`（负载高时 `≈ 11.6 GB/s`）——**正落在这台机器流式带宽的量级上**，说明它已经被带宽打满。
- **决策规则**：把两种方案的成本写成同一单位（时间）再比较——**加锁方案的成本 ≈ 每次操作 × 竞争系数 `t_crit`；数据并行方案的成本 ≈ 额外字节数 / 带宽**。本例中 `1.1 ns` 对 `289 ns`，差约 260 倍，**数据并行完胜**；但如果桶数只有 2 个、线程数也只有 2 个，竞争系数很小，额外的排序带宽反而会成为纯亏损。

4.6 Amdahl 定律与"固定问题规模"的陷阱

- **公式（Amdahl）**：设程序中**无法并行/被串行化**的比例为 `f`，则 `Speedup(P) ≤ 1 / (f + (1−f)/P)`，上界 `1/f`。

- **数值算例 F-1**:
- `f = 5%`：`P = 16` 时 `1/(0.05 + 0.95/16) = 9.1×`；**无论 P 多大，上限只有 20×**。
- `f = 1%`：`P = 64` 时 `1/(0.01 + 0.99/64) = 39.4×`（上限 100×）。
- **在并行程序里，"1% 的串行"经常就是"通信/同步的那 1%"**：所以本讲的通信优化同时也在**压低** `f`。
- **数值算例 F-2（固定问题规模的扩展陷阱）**:
- 1D 分块的每处理器通信/计算元素比 = `2N / (N²/P) = 2P/N`:
  - `N = 258, P = 32`：`2 × 32 / 258 = 0.248` ⇒ **每算 4 个元素就要搬 1 个元素**（还要加上 halo 行带来的 25% 冗余计算：`rows = 8`，其中 2 行是边界），通信与同步开销**完全吃掉**并行收益——**这正是 CS149 版本给出的"SGI Origin 2000 上 258×258 网格在 32 处理器上几乎没有加速"的成因**。
  - `N = 1024, P = 32`：`2 × 32 / 1024 = 0.0625` ⇒ 每 16 个计算元素才搬 1 个，扩展性良好（同一台机器上 1K×1K 网格扩展良好）。
- **推论**：**评估并行机时必须同时报告问题规模**；小问题在大型并行机上"没有加速"并不说明机器差，只说明**问题的算术强度相对于机器规模太低**。
- **超线性加速（super-linear speedup）与"伪加速"**:
- **真超线性**：问题的工作集在单处理器上装不进 cache、在多处理器上却能**分摊到每个处理器的私有 cache**里（每个处理器只要算自己那一小块），于是单机跑的是"cache miss 版"、并行跑的是"cache hit 版"。
- **假超线性**：单机上工作集超过内存 ⇒ **换页到磁盘（thrashing）**；或者用"并行算法跑在 1 个核上"作为基准（**低估了基准**）。CS149 版本明确警告："把并行程序的加速比与'并行算法在单核上运行'相比，是常见且不诚实的做法。"

4.7 模型总览与适用条件

模型	公式	回答什么问题	适用条件 / 陷阱
延迟-带宽（α-β）	`T(n) = T0 + n/B`	一次通信要多久？消息要多大才划算?	适用于单次点对点传输；`occupancy` 由最慢部件决定
overhead/occupancy/delay	`T = overhead + occupancy + delay`	时间花在哪?	occupancy 决定稳态速率 `1/occupancy`；缓冲有限时会被发送方阻塞
cost 与 overlap	`cost = 通信时间 − overlap`	通信真的拖慢了程序吗?	需要"比执行单元更多的并发"才能重叠
算术强度	`AI = 计算量 / 通信量`	这个程序是不是带宽受限?	必须与 ridge point 比较才有意义
通信-计算比（分配相关）	1D blocked `N/(2P)`；1D interleaved `1/2`；2D blocked `N/(4√P)`	我的工作分配是否放大了固有通信?	都假设了理想 cache 与最小粒度
四个 C	cold / capacity / conflict / communication	这些 miss 是必需的还是人为的?	working set 视角，任意层次都成立
Work-Span	`T_P ≤ W/P + S`，并行度 `W/S`	加核还有用吗?	假设贪婪调度；`S` 要含通信延迟与同步
Little's law	`B = MLP / L`	多少个在途请求才能打满带宽?	稳态、请求足够并发；不改变总流量
Roofline	`perf ≤ min(峰值算力, AI × 带宽)`	优化方向应该是"减流量"还是"减计算"?	需要知道机器的 ridge point
Amdahl	`S(P) ≤ 1/(f + (1−f)/P)`	串行部分把上限压到多少?	`f` 随 P 增大常常变大（通信/同步变贵）
竞争/排队	热点吞吐 `≤ 1/t_crit`	是不是"越加线程越慢"?	需要区分"工作量"与"资源占用时间"

---

5. 关键要点

1. **先分清"延迟"与"带宽"，再决定优化手段**：延迟问题（一次通信要等多久）靠减少消息数、增大消息体量（摊薄 `T0`）、重叠（overlap）、以及硬件改进；带宽问题（单位时间能搬多少）只能靠**减少字节数**（提高算术强度、消灭人为通信）。**把这两件事搞混，优化就会南辕北辙。**
2. **通信量由"工作分配"和"数据布局"共同决定，且必须先在算法层面压低下界**：固有通信由分解/分配决定（1D blocked `N/(2P)`、1D interleaved 恒为 `1/2`、2D blocked `N/(4√P)`），人为通信由 cache line 粒度、write-allocate、放置策略、伪共享决定。**好的分配（2D 分块）+ 好的布局（blocked layout + padding）能同时压两头。**

3. **提高算术强度是并行程序最重要的单一优化目标**：现代机器的算力/带宽之比很高（本讲示例机器 ridge point ≈ 38 flop/B），绝大多数"看起来很并行"的代码（逐元素运算、模板、稀疏计算）都落在**带宽受限**区，**融合循环、分块重用、共享数据、合并消息**都能把工作点往右推；只有当工作点越过 ridge point，计算优化才开始有意义。
4. **竞争是一种"与工作量无关"的性能税，要靠复制、细粒度化与错峰来缴清**：`吞吐上限 = 1/t_crit` 与线程数无关，所以热点会让程序**负扩展**；有效手段是**复制资源（本地副本、每桶锁、部分结果 + 合并）、把同步降级到片上/块内、用数据并行 + 排序取代细粒度同步**，以及**错开访问时间**。
5. **优化的顺序必须是"先测量、先建 high watermark，再动手"**（CS149 版本的方法论）：先写最简单、能跑对的并行版本并测出绝对时间；再用"加数学运算看是否线性变慢（指令受限？）""把数组访问全改成 `A[0]`（局部性还有多少油水？）""删掉所有原子/锁（同步还有多少油水？）""删掉大部分计算但保留同样数据访问（是不是带宽受限？）"这四个实验确定瓶颈类型；**只有在确认瓶颈类型之后，才去应用上面 4 条中的对应手段。**

---

6. 常见陷阱与注意事项

- **把共享地址空间的"隐式通信"当成免费，同时只盯 FLOPS 不看字节**：`A[i]` 这样一次普通 load 在并行程序里可能是**跨核/跨插槽的一致性事务**——"代码里没有 send/recv"不等于没有通信；反过来，把大量时间花在"减少浮点运算"上，而程序其实 90% 的时间在等内存，也是最常见的南辕北辙。**先算算术强度并对照 ridge point，再决定优化计算还是优化访存。**
- **伪共享（false sharing）**：多个线程写**同一条 cache line 的不同字节**，逻辑上无共享、无数据竞争，硬件上却是"整条 line 的乒乓"（典型场景：`count[tid]`、`struct { long a; long b; }` 中不同线程各写一个字段、线程池的任务计数器）。**修法**：`alignas(64)` padding、per-thread 私有副本 + 末尾归约、blocked data layout。**注意 padding 不是免费的**（内存与带宽开销），对齐值要按机器的 cache line 大小（通常 64 B）定。
- **消息太小太频繁，以及非阻塞通信的四种误用**：为每个元素发一条消息、每轮循环都做一次全局 `Allreduce`/屏障，都会被启动延迟 `T0` 碾压（应当合并消息、批量化、把归约放到低频路径上）；非阻塞通信的典型错误是 ① 在 `Wait` 之前就**改写发送缓冲**（数据被破坏）、② 忘记 `Waitall`、③ **tag 不匹配**（表现为永久挂起，看起来像死锁）、④ 用阻塞 `send/recv` 却让所有人都"先 send 再 recv"（真死锁，修法是奇偶配对收发或改用非阻塞）。
- **锁的粒度选错**：全局锁 ⇒ 热点（吞吐被 `1/t_crit` 锁死，负扩展）；无脑细粒度锁 ⇒ 锁本身的开销（空间、原子操作、cache line 迁移）可能超过收益。**先想清楚"临界区保护的到底是什么"，再决定粒度**；能用"局部复制 + 末尾合并"或"数据并行 + 排序"就不用锁。
- **分块参数按错了 cache、以及忽略 NUMA 与数据放置**：并行程序里**每个线程只能用自己那部分 cache**（以及自己那个 L3 slice），块大小必须按**每线程可用容量**调（太大装不下、太小则循环开销占比过高）；在多路机器上内存延迟与带宽还**取决于"谁访问谁的内存"**，而**首次触碰（first touch）决定页的归属**——应当在并行初始化时就**让将来使用该数据的线程去初始化它**。
- **用"固定小问题"测出的加速比评价程序或机器**：小问题上通信/同步/启动开销占主导（通信/计算比高达 `2P/N`、`T0` 摊不薄），常常"不升反降"；同时要警惕**用并行算法单核运行作基准**造成的假加速，以及工作集在单机溢出 cache/内存（甚至换页到磁盘）造成的**超线性加速**——超线性往往说明**基准跑得不对**，而不是并行真的"多核快过 P 倍"。

---

7. 思考题（带答案）

思考题 1：三种工作分配的固有通信

设 `N × N` 网格、`P = 64` 个处理器、5 点模板（每个输出元素读上下左右四个邻居），`N = 4096`。请分别算出 **1D blocked**、**1D interleaved（隔 P 行取一行）**、**2D blocked（8×8 处理器阵列）** 三种分配下：每处理器计算的元素数、通信的元素数、以及"每通信 1 个元素对应多少计算元素"（算术强度），并说明哪种分配会随 N 增大而自动变好。

【答案】 设 `P = 64`、`N = 4096`，每处理器计算量都是 `N²/P = 4096²/64 = 262,144` 个元素。 – **1D blocked（连续 64 行）**：通信量 = 上下各一行 = `2N = 8,192` 个元素。 **算术强度 = 262,144 / 8,192 = 32**（公式 `N/(2P) = 4096/128 = 32` ✓）。 – **1D interleaved（隔 64 行取一行）**：每处理器拥有的行彼此间隔 64 行，**每一条自己的行**都要向邻居另要上下两整行（`2N = 8,192` 个元素），共有 `N/P = 64` 条这样的行 ⇒ 通信量 `= 64 × 8,192 = 524,288`。 **算术强度 = 262,144 / 524,288 = 0.5**（与 N、P 无关的常数 `1/2` ✓）。**比 1D blocked 差 64 倍，正好等于 P。** – **2D blocked（8×8 = 64 个处理器，每块 512×512）**：通信量 = 四条边 = `4 × 512 = 2,048` 个元素。 **算术强度 = 262,144 / 2,048 = 128**（公式 `N/(4√P) = 4096/(4×8) = 128` ✓）。 – **随 N 变好的只有 blocked 两类**：1D blocked 的算术强度 `N/(2P)` ∝ N；2D blocked 的 `N/(4√P)` ∝ N；而 **1D interleaved 恒为 1/2**，无论问题多大都注定是"搬 2 个元素才算 1 个元素"的灾难。此外 2D blocked 关于 P 是 `∝ 1/√P`（**次线性**恶化），1D blocked 是 `∝ 1/P`（线性恶化）——这就是讲义说"2D blocked 的通信扩展性渐近更好、捕获了算法的二维局部性"的定量含义。 – **补充（越大越好还是越小越好）**：如果按讲义另一页的写法用通信-计算比（communication-to-computation ratio），三种分别是 `1/32`、`2`、`1/128`——**越小越好，与上面的算术强度互为倒数**，别被方向搞混。

思考题 2：伪共享的诊断与修复

某程序有 16 个线程，每个线程维护自己的局部统计量并**只写自己的那个元素**：

```
struct Stats { long hits; long misses; };
Stats stats[16];            // 16 * 16 B = 256 B, 跨 4 条 cache line
...
#pragma omp parallel for
for (int i = 0; i < N; ++i) {
    int t = omp_get_thread_num();
    if (hit(i)) stats[t].hits++; else stats[t].misses++;
}
```

请回答：(a) 这段代码有没有数据竞争？(b) 为什么它的性能会随线程数增加而**变差**？(c) 给出至少两种修复方案，并说明各自的代价；(d) 你如何**用实验确认**瓶颈是伪共享而不是"真正的共享竞争"或者"内存带宽"？

【答案】 (a) **没有数据竞争**：线程 `t` 只写 `stats[t]`，不同线程写得不同对象，且没有任何线程读别人写的值——从 C++ 内存模型看这是**良定义、无竞争**的程序。(b) 原因在 **cache line 粒度的一致性维护**：`Stats` 是 16 字节，**一条 64 B cache line 里放着 4 个线程的数据**。线程 `t` 写自己的 `hits` 时必须取得**整条 line 的独占所有权**，于是其他 3 个线程的副本失效；它们下一次写又要把 line 抢回去。**16 个线程的数据只占 4 条 line**，这 4 条 line 在全核之间来回"乒乓"，每次写的代价从 ~0.5 ns（L1 命中）变成几十~几百 ns（line 所有权迁移）。这解释了**为什么线程越多越慢**：竞争这条 line 的人更多，而且线程很可能分布在不同的 CCX/插槽上（§3.4 实测：8 线程 13.6× 的差距、4 线程 7.9× 的差距——差距本身随线程数增大）。(c) **修复方案**： 1. **padding / 对齐**：`struct alignas(64) Stats { long hits, misses; };`（`sizeof == 64`）让每个线程独占一条 line。**代价**：内存与 cache 占用从 256 B 涨到 1 KB；如果线程数很大（比如 256），1 KB 还算便宜，但如果结构体本身是 100 B，padding 到 128 B 也会浪费。 2. **本地累加 + 末尾归约**：线程把统计量累加到**寄存器/栈上的局部变量**，循环结束后只写一次 `stats[t]`。**代价**：需要一次额外的归约（`reduction` 或手工合并），并且改变了"随时可读全局统计"的语义（外部观察者只有在归约后才能看到完整值）。 3. **blocked data layout**：如果是数组而非结构体数组（例如 `long hits[16]`），可以考虑把"同一线程要用的所有数据"聚成连续的 block（`struct alignas(64) PerThread { ... }`），本质上与 1 相同。(d) **确认瓶颈的实验**（对应 CS149 版本的 high watermark 方法）： – **跑"删掉同步"实验**：把写操作全部去掉（只做计算与读 `ro[]`），如果时间几乎不变 ⇒ 瓶颈**不在**同步，而是在数据搬运/cache line 迁移上 ⇒ 支持伪共享假设。§3.4 的版本 ③ 就是这个实验。 – **跑"改数据布局"实验**：只把 `Stats` 改成 `alignas(64)`，其他一字不改。**如果加速 5~20 倍，伪共享被证实**（真共享竞争不会因为 padding 而消失——padding 之后线程之间根本没有共享数据）。 – **区分"带宽"**：用 `perf stat`（或 Intel PCM / PAPI）看 `LLC-load-misses`、`offcore` / `local vs remote DRAM` 的字节数。伪共享的特征是**DRAM 字节数很低、但一致性事务/延迟很高**（数据本来就在 cache 里，只是反复易主）；真正的带宽瓶颈则是 **DRAM 字节数接近机器带宽上限**。 – **注意**：不要用"总时间随线程数恶化"作为唯一证据——带宽瓶颈与 NUMA 效应也会造成类似现象，必须结合字节数与 cache line 迁移计数。

思考题 3：三种分桶策略的选择

要把 **100 万个粒子**放进 **256 个桶**，机器是 **16 线程的 CPU**（量级参数：全局锁下一次插入 ≈ 600 ns，每桶锁下 ≈ 290 ns，数据并行分桶的吞吐 ≈ 每次插入 1.1 ns，但需要额外搬运 `20n` 字节；机器流式带宽 20 GB/s）。请估算三种方案的时间并给出选择；然后说明如果把目标机器换成 **GPU（每 SM 可挂 2048 条线程、共 16 个 SM）**，你的选择会怎样改变，为什么。

【答案】 – **核算**：$n = 10^6$，额外字节 = $20n = 20$ MB。 – **全局锁**：$10^6 \times 600$ ns $= 0.60$ s（**且与线程数无关**：加更多线程只会让队列更长，§3.5 实测 16 线程时吞吐只有 1.58 ~ 2.29 M/s，与此一致）。 – **每桶锁**：$10^6 \times 290$ ns $= 0.29$ s。虽然竞争降低，但插入路径上仍要原子操作 + 触碰桶头指针所在 line，**没有解决"同步在插入路径上"这个根本问题**。 – **数据并行（计数排序 + 私有输出区间 + 并行求桶边界）**：同步成本 ≈ 0（只有两次屏障），时间 ≈ 20 MB $/ 20$ GB/s $= 1$ ms，加上直方图与散射本身的按元素开销（每次插入 1.1 ns 是**实测含全部三个阶段**的数字）⇒ 约 **1~5 ms**。**比每桶锁快约两个数量级**。 – **选择**：在这个规模上**选数据并行（解法 5）**。理由是：桶数（256）远小于线程数 × 常量、竞争系数极高，而额外带宽（20 MB）相对机器带宽很便宜。**只有当桶数极少（比如 2 个）、线程数也极少（比如 2 个）时**，额外的排序带宽才可能超过竞争成本，此时"每桶一把锁"或"局部复制 + 合并"更划算。 – **换成 GPU（16 个 SM，每个 SM 最多 2048 条线程 ⇒ 可并发数万条线程）**： – **全局锁方案彻底不可行**：热点吞吐上限是 $1/t_{crit}$，与线程数无关；数万条线程争一把锁只会把绝大部分时间花在排队上（而且 GPU 上原子操作/锁的成本比 CPU 更高，且 warp 内的锁会导致严重分歧与占用浪费）。 – **每桶锁方案同样不可行**：即使降到 256 把锁，每把锁平均也有上百条线程争抢；而且 GPU 的同步原语（`atomicCAS` 自旋）会占用 warp 调度槽位，属于"用宝贵的并发度换取等待"。 – **应当选解法 ④ 或 ⑤**：④（每个 thread block 维护一张**局部网格**，块内用 **shared memory** 同步，最后合并）把同步降级到片上（延迟比 L2/DRAM 低一个数量级，且不产生 cache line 迁移），代价是 N 倍内存与一次合并；⑤（**数据并行**：算格子号 → 排序 → 求桶边界）保留**极大并行度**且**零细粒度同步**，代价是一次排序与额外的数据遍历（额外带宽）——在 GPU 上 GPU 显存带宽很高（数百 GB/s 到 TB/s 级），额外带宽比"锁竞争"便宜得多，因此 ⑤ 往往是最优的；工程上还常把块内前缀和（`__syncthreads()` + shared memory）与块间偏移（全局前缀和）结合，做"块级直方图 + 块间合并"，这与解法 ④ 的思想是一致的。 – **一般原则：机器的并发度越高，"用带宽/内存换取零同步"的方案就越划算；并发度越低、同步越廉价，"就地加锁"才越可能赢。** 这正是讲义把五种解法并列出来的用意——**没有普遍最优解，只有与机器匹配的解。**

> **本讲与前后讲的关系回顾**：本讲给出了"通信为什么贵、怎样变便宜"的完整工具箱（延迟/带宽 → α–β 模型 → overhead/occupancy/overlap → 固有 vs 人为通信 → blocking/fusion/sharing/granularity → 竞争与排队）；其中**伪共享、cache line 迁移、一致性事务**的硬件细节是第 11~13 讲（缓存一致性）的主题；**网络拓扑如何决定 occupancy 与竞争**是第 10 讲（互连网络）的主题；**锁与原子操作的成本、无锁数据结构如何降低竞争**是第 15~16 讲的主题。第 14 讲（Performance Analysis / Profiling）则会系统化本讲末尾提到的"测量与 high watermark"方法论。

Lecture 10: Interconnection Networks

1. 章节标题与概述

Lecture 10: Interconnection Networks（互连网络）

- **本讲核心问题**：当处理器核数从 4 个涨到 64、72 甚至上千个节点时，**"谁和谁怎么连、消息怎么走、消息在网里存多少"** 就成了整个系统的性能天花板。本讲要回答三件事：(1) 为什么共享总线（shared bus）不能扩展，必须换成由链路（link）和交换机（switch/router）组成的**互连网络（interconnection network）**；(2) 各种**拓扑（topology）**——总线、crossbar、ring、mesh、torus、tree/fat tree、hypercube、多级对数网络（multi-stage logarithmic / Omega）——在**成本、延迟、二分带宽（bisection bandwidth）** 上如何权衡；(3) 消息以什么**粒度（message / packet / flit）** 在网络里传输、用什么**流控（flow control）** 机制缓冲（circuit switching vs packet switching，store-and-forward vs cut-through vs wormhole，以及虚通道 virtual channel 如何对付 head-of-line blocking）。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：网络节点（network node）、网络接口（network interface）、交换机/路由器（switch/router）、链路（link）；`Request bus`（cmd + address）与 `Response bus`（256 bit 数据 + 3 bit response tag）构成的总线协议；Intel Sandy Bridge 起引入的**环形互连（ring interconnect，四条环：request / snoop / ack / 32 B data）** 与 L3 slice；Intel Xeon Phi（Knights Landing）的 **6×6 tile mesh** 与 **YX routing**；Tilera GX、Oracle/Sun SPARC T2/T5（crossbar CCX）等真实芯片；包格式（header / payload / tail）、flit（flow control digit）、credit 流控、虚通道、escape VC。
- **软件侧**：程序员无法直接"编程"互连网络，而是通过**通信模式**间接决定它是否成为瓶颈：消息大小（决定 α 还是 β 主导）、消息数量、通信的局部性（邻居交换 vs 全对全）、通信与计算的重叠（非阻塞 MPI）、以及**分片（sharding）/ 局部性** 是否让访问落在离自己近的 slice 上。
- **在并行计算知识体系中的角色**：本讲直接建立在前几讲的一致性（cache coherence / directory coherence / snooping 的实现）之上——一致性协议的所有"监听请求、失效消息、数据回传"都要**占用互连网络**；也是后面"同步（synchronization）与无锁（lock-free）"的前置知识——一个被 64 个核抢的原子计数器，本质是**人为在网络里造了一条总线**。它把前几讲的"缓存层次（cache hierarchy）"从"抽象的一层一层"落实到"物理上靠什么线连起来"，也是异构计算（heterogeneity）、GPU 内的 shared memory/L2 通信、以及多机 MPI 通信的共同底层模型。
- **配套材料**：
- `lectures/14_interconnects.pdf`（抽取文本 `extracted/14_interconnects.txt`，共 **48 页 / 约 21 KB**）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 公开下载。讲义首页写的是 **"Lecture 14: Interconnection Networks"** 与 **"CMU 15–418/15–618, Spring 2024"**：这是**讲义沿用历史学期版本**的正常现象（讲次编号与学期字样会随年度重排），不是错误。按 Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html），本讲排在 **Sep 16**，为第 10 讲。本笔记的术语、示例与数字均以这份 48 页讲义为准。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 部分讲座在 Fall 2026 **尚未发布**公开讲义（Performance Analysis / Profiling、Transactional Memory、AI in System Design 等）；其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。
- 本笔记中用到的额外背景（α–β 延迟模型、Little's law、KNL 网频假设等）均已显式标注为"本笔记补充"，与讲义原文区分。

---

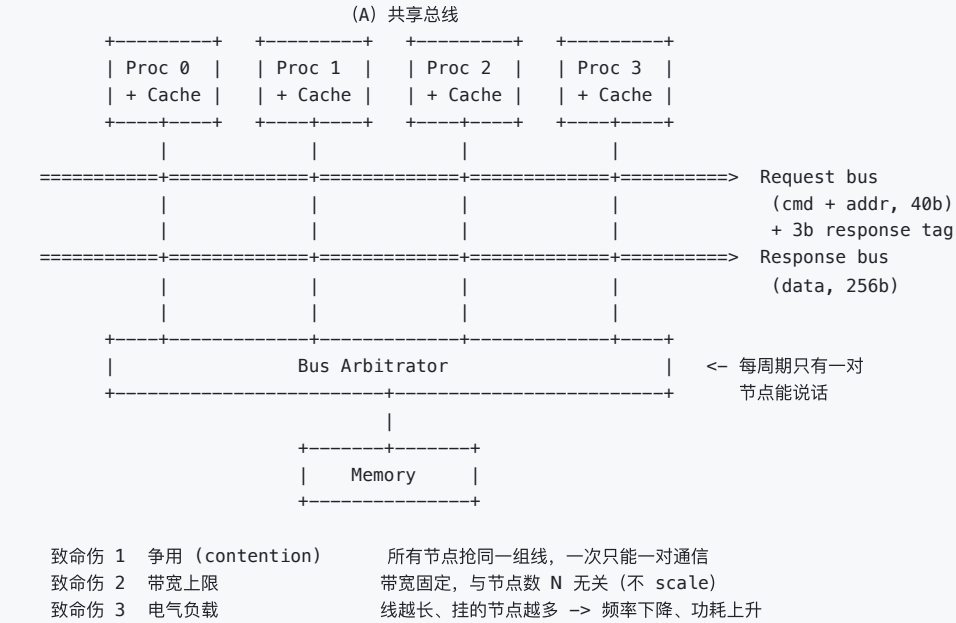
2. 核心概念与硬件/软件架构图解

2.1 出发点：共享总线为什么会先赢后输

- **定义与目的**：讲义 slide 2 给出了前几讲的基本系统设计——若干"处理器 + 私有 cache"节点，全部挂在同一条**共享总线（shared bus）** 上：一条 **request bus** 传 `cmd + address`（例如 40 bit），一条 **response bus** 传数据（例如 256 bit），外加 3 bit 的 **response tag**；总线上还

有一个**总线仲裁器（bus arbitrator）** 决定这一周期谁说话。它的目的很朴素：**用一套线把所有节点连起来，并且让"监听式一致性（snooping coherence）"变得极其容易实现**——因为所有一致性消息天然广播到了所有节点。

- **直观解释（"它是什么？"）**：把总线想成**一条只有单车道、还没有红绿灯的乡村公路**。村子里只有两三户人家时，这条路又便宜又好用：谁要出门，喊一声（广播）所有人都听得到，不用挨家挨户通知（这正是 snooping 便宜的原因）。但车一多，问题就来了：(1) 所有车必须排队（contention）；(2) 整条路的通行能力是固定的，不随住户增加而增加（**带宽不 scale**）；(3) 路上挂的"负载"（电气负载 = 电容）随节点数增多而变大，于是**时钟频率只能降下来、功耗还得上去**（high electrical load = low frequency, high power）。
- **图 1：总线互连（slide 2 的基线设计）与它的三个致命伤**



- **性能特征（关键操作与量化后果）**：设总线时钟为 `f`、宽度为 `W` 字节、每次事务固定开销 `T_ovh`，则总线能提供的**总带宽 ≈ W·f**，与 N 无关；N 个节点各自想要 `b` 字节/秒时，只有在 `N·b ≤ W·f` 时才可能满足，且每个节点的**有效带宽是总带宽除以 N**。这就是"总线不 scale"的量化表述。于是 slide 3 的转折点出现：**把共享总线换成互连网络（interconnection network）**，而"今天这些历史上面向机柜、主板、多 socket 的互连问题，全部在片内重演"——讲义把它叫做 **network-on-a-chip**。

2.2 互连网络用来连什么、为什么重要

- **定义与目的**（slide 4、5）：互连网络用于连接：(a) 处理器核与其他核；(b) 处理器与内存；(c) 核与 cache；(d) cache 与 cache；(e) I/O 设备。它重要，是因为它同时决定两件事：**可扩展性（system scalability）**——系统能做多大、加节点有多容易；以及**性能与能效（performance & energy efficiency）**——核/缓存/内存之间能多快通信、访存延迟有多长、**通信本身花掉多少能量**。
- **直观解释（"它是什么？"）**：把多核芯片想成**一座城市**，核是工厂，cache 是仓库，内存是港口，互连网络就是**道路系统**。工厂再快，货送不出去也没用；而道路的**造价**（面积、功耗）与**通行能力**（带宽）往往互相矛盾，城市规划（拓扑选择）决定了这座城能长到多大。
- **slide 6 的现实证据**：随着核数上升，片上互连的可扩展性越来越关键——从 Intel Core i7（4 个 CPU 核 + GPU）、NVIDIA Tegra K1（4+1 ARM 核 + GPU 核）、Tilera GX（64 核）、Intel Xeon Phi（72 核 x86），节点数一路涨上去，网络从"配角"变成"主角"。
- **能耗数量级（slide 47）**：MIT RAW 研究处理器的实测中，**互连能占到整颗芯片功耗的约 35%**。这说明"通信是昂贵的（communication is expensive）"不只是延迟问题，还是**能量问题**——这也是今天 bufferless network、区域关断（turn on/off regions）、快慢双网（fast and slow networks）、光子片上网络（photonic NoC）等研究的动机。

2.3 术语（slide 8）

- **定义与目的**：讲义先把四个基本名词钉死，后面的所有讨论都建立在它们之上：

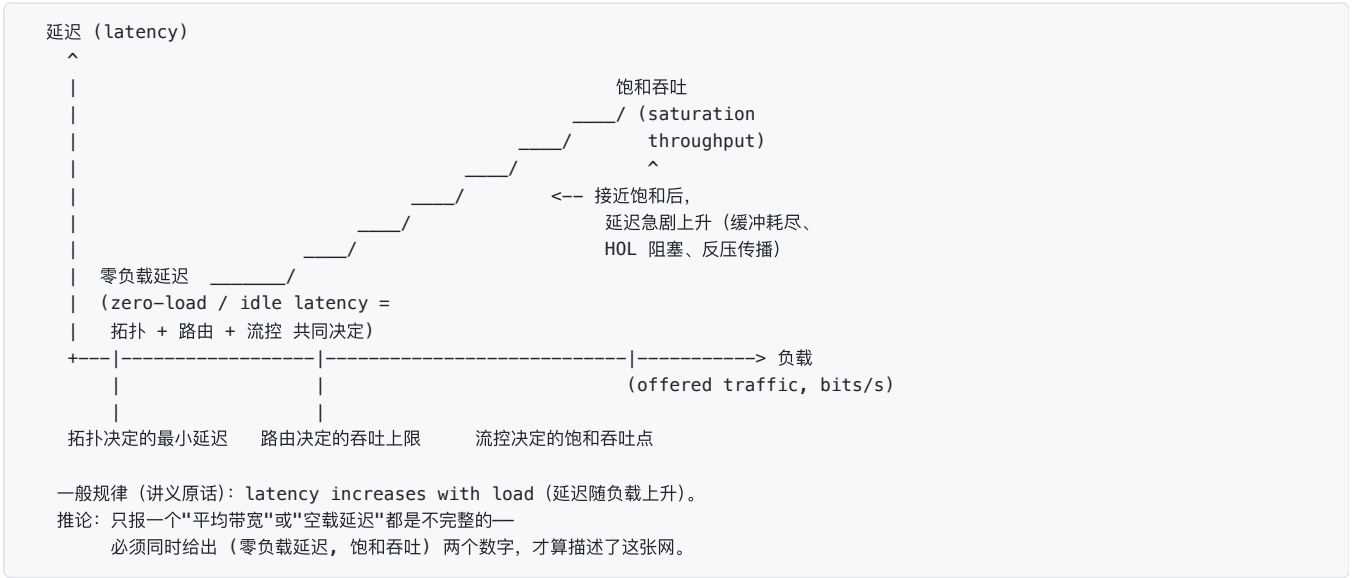
术语	定义（slide 8）	典型例子	直观类比
Network node（网络节点）	连到路由器/交换机上的**网络端点**	processor cache、memory controller	城市里的工厂/仓库（货物起点终点）
Network interface（网络接口）	把节点接进网络的那层适配逻辑	NI、网卡、片上网络端口	工厂的**装卸货月台**

术语	定义（slide 8）	典型例子	直观类比
Switch / Router（交换机/路由器）	把**固定数量**的输入链路连到**固定数量**的输出链路	5 端口片上路由器（N/S/E/W/Local）	**十字路口**
Link（链路）	一束**传输信号的线**（bundle of wires）	32 B 宽的数据环、片间 SerDes 通道	连接路口的**一条马路**（车道数 = 位宽）

- **直观解释（"它是什么？"）**：一个消息的旅程是："节点产生请求 → 网络接口打包 → 进入路由器 → 沿链路一跳一跳前进 → 到达目的节点的网络接口 → 送到目的 cache/内存"。**路由器不产生消息，只做转发（forwarding）和仲裁（arbitration）**；网络接口负责"把节点内部的总线语言翻译成网络语言"。
- **性能特征**：链路的**带宽** = 位宽 × 频率（例如 32 B × 3.4 GHz = 108.8 GB/s）；路由器的**吞吐**受限于端口数和每周期能交换的 flit 数（通常 1 flit/端口/周期）；**延迟**则为"每跳的路由延迟 × 跳数 + 串行化时间"。

2.4 三个设计问题：拓扑、路由、缓冲与流控（slide 9）

- **定义与目的**：设计一个互连网络，就是回答三个问题：
- **Topology（拓扑）**：交换机之间**怎么用链路连**。它影响路由（routing）、吞吐、延迟、实现复杂度与成本。
- **Routing（路由）**：一个消息**如何从源走到目的**。可以是**静态的（static，预定路径）** 或**自适应的（adaptive，依据负载选路）**。
- **Buffering & flow control（缓冲与流控）**：网络里**存什么**（整包？部分包？单个 flit？）、以及**如何管理缓冲空间**（无缓冲 vs 缓冲、credit 还是 on/off）。
- **直观解释（"它是什么？"）**：把网络想成**寄快递**：拓扑 = 城市路网图；路由 = 快递员的选路规则（永远走"先南北后东西"？还是看哪条路堵就绕？）；缓冲与流控 = 中转站**有多少货架**、以及"货架满了还能不能收货"的规则。三者**互相牵制**：拼命加缓冲能提高吞吐但增加延迟和面积；自适应路由能绕开拥塞但可能引入死锁；拓扑越"富"（crossbar）延迟越低但 $O(N^2)$ 的成本直接压垮芯片面积。
- **性能特征（图 2）：延迟–负载曲线（slide 14）——本讲最重要的一张图**



2.5 拓扑度量：用什么指标挑拓扑（slide 10–13）

- **定义与目的**：
- **Routing distance（路由距离）**：两个节点之间路径上的链路数（**跳数 hops**）。
- **Diameter（直径）**：所有节点对之间路由距离的**最大值**（最坏情况延迟的依据）。
- **Average distance（平均距离）**：所有合法路径上路由距离的平均值（平均延迟的依据）。slide 的例子里 `diameter = 6`。
- **Direct vs. Indirect（直接/间接网络）**：**直接网络**中端点"坐在网络内部"，每个节点既是端点又是交换机（mesh 就是直接网络）；**间接网络**中端点只挂在网络边缘，中间全是专用交换机（crossbar、多级网络）。
- **Bisection bandwidth（二分带宽）**：把网络切成**两个等大**的部分，**所有被切断链路的带宽之和**（取最小割）。它是递归式拓扑最常用的性能指标。
- **Blocking vs. Non-blocking（阻塞/非阻塞）**：如果**任意**节点对都能同时连通而不冲突，就是非阻塞，否则是阻塞。

- **讲义对二分带宽的警告（务必记住）**："can be misleading as it does not account for switch and routing efficiencies"——它忽略了交换机内部竞争与路由效率，不是可达带宽的保证。
- **直观解释（"它是什么？"）**：二分带宽想度量的是"**把城市切成两半，两半之间所有跨城道路的总车道数**"。它是判断"全对全通信能不能撑住"的关键：如果两半之间有 100 万居民却只有两条小路，那么任何"所有人都要和对面说话"的应用都会堵死。而"阻塞/非阻塞"则是问："**任意两个人同时出发，能不能都直达、不用让路？**"
- **图 3：阻塞 vs 非阻塞（slide 13）——同一个网络，换个配对就阻塞**

```
8 节点网络（左列与右列是同一批节点的两种画法，便于看路径）
slide 13 的两个场景：

场景 A: 0->1 与 3->7 同时发送
     0 ---\                              /--- 1
     1 ----\                            /---- 2
     2 -----[ SW-a ]----[ SW-b ]------ 3
     3 -----/          \               \---- 4
     ...                \                ...
两个连接走的是相互独立的开关 => 不冲突

场景 B: 1->6 与 3->7 同时发送
     0 ---\                              /--- 1
     1 ----\====> [ SW-a ] ==X== [ SW-b ] ====> 6   <-- 冲突！同一个开关上
     2 -----/                             \---- 3        要同时服务两个连接
     3 ----------------------[ SW-a ]------+---- 7
                                ^
                          两台不同的输入要同一个输出资源
=> 结论：这个网络是 BLOCKING（阻塞）的。

判据：非阻塞要求"任意配对都能通过独立开关连通"；
     只要存在某个配对组合必须在同一个开关（或同一条链路）上相撞，就是阻塞网络。
```

2.6 拓扑家族：六种主干设计（slide 16–32）

- **直观解释总览**：把拓扑想象成**六种不同的城市路网规划**：
- **Bus（总线）**：一条单车道村道（最简单、最便宜、最不 scale）；
- **Crossbar（交叉开关）**：超级立交枢纽，任意入口到任意出口都有独立匝道（最快、最贵，$O(N^2)$）；
- **Ring（环）**：环城单环地铁线（便宜、简单，但坐半圈要坐 N/2 站）；
- **Mesh（网格）**：棋盘式方格街道（片上最容易画，跳数 $O(\sqrt{N})$）；
- **Torus（环面）**：把棋盘卷起来成轮胎形，边界也连通（跳数、二分带宽都更好，但物理布线难）；
- **Tree / Fat Tree（树 / 胖树）**：树状分级的快速路，越靠根车道越多（对数延迟，可做到全二分带宽）；
- **Hypercube（超立方体）**：编号相差一个 bit 就能直达 → 对数延迟、对数度；
- **Multi-stage logarithmic（多级对数网络，Omega/Butterfly）**：像机场行李分拣系统，中间几级换乘，成本 O(N lg N)、延迟 O(lg N)。
- **各拓扑的要点（slide 17–31）**：
- **Bus**：好——设计简单、节点少时性价比高、**用监听实现一致性很容易**；坏——争用、带宽受限（同一时刻只有一次通信）、电气负载高导致频率低/功耗高。
- **Crossbar**：每个节点与每个节点之间都有独立通路（**非阻塞、间接网络**）；好——**O(1) 延迟与高带宽**；坏——**$O(N^2)$ 个开关，不可 scale、成本高、大规模仲裁困难**。讲义明确指出"crossbar 的调度算法与高效硬件实现至今仍是活跃研究领域"，并追问"这个开关是什么？"——它本质是每个输入/输出交叉点上的小型选择开关。真实例子：**Sun SPARC T2（8 核 + 8 个 L2 bank）与 Oracle SPARC T5（16 核 + 8 个 L3 bank）**，其中 **crossbar（CCX）占用的芯片面积与一个核相当**——这是"互连很贵"的最直观证据。
- **Ring**：好——简单、**O(N) 成本**；坏——**延迟 O(N)**、**二分带宽是常数**（每加节点都不增加，这是它的 scalability 硬伤）。真实例子：**Intel Sandy Bridge 起的环形互连**（四条环：request / snoop / ack / 32 B data；六个互连节点：四个 2 MB 的 L3 slice + system agent + graphics；每个 L3 bank 接环两次；**3.4 GHz 下核到 L3 的理论峰值带宽约 435 GB/s**——前提是每个核访问自己的本地 slice），以及 **IBM CELL Broadband Engine（9 核）**。
- **Mesh**：**直接网络**；呼应网格类应用的局部性；**O(N) 成本**；**平均延迟 $O(\sqrt{N})$**；在芯片上**容易布局（链路等长）**；**路径多样性（path diversity）**好（一个消息有很多条路可以走）。真实例子：**Tilera 处理器**与 **Intel 原型芯片**。KNL 的具体形态是 **72 核、6×6 的 tile 网格（每 tile 2 核）**，采用 **YX 路由：先在 Y 方向移动，然后"转弯"，再在 X 方向移动**。
- **Torus**：相比 mesh，"节点在边缘还是在中间"的性能差异不再存在（**新增绕回的链路**避免了这个不公平）；仍然 **O(N) 成本但比 2D 网格贵**；**路径多样性与二分带宽都更高**；代价是**复杂度更高——片上难以布局、链路长度不等**。

- **Trees**：平面、层次化拓扑；像 mesh/torus 一样**在流量有局部性时表现好**；**延迟 O(lg N)**；用 **fat tree（胖树）** 缓解"根部带宽瓶颈"（越靠近根，链路带宽越高）。
- **Hypercube**：**延迟 O(lg N)**、**度（radix）O(lg N)**、**链路数 O(N lg N)**；历史上的例子是 80 年代 Caltech 的 **64 核 Cosmic Cube（6 维超立方体）** 与 **SGI Origin**。
- **Multi-stage logarithmic**：**间接网络**，终端之间要经过多级交换机；**成本 O(N lg N)**、**延迟 O(lg N)**；变体很多：**Omega、butterfly、Clos 网络**等。
- **图 4：拓扑画廊（讲义 slide 16–32 里的六种主干网络）**

```
(1) BUS                          (2) CROSSBAR (N=8, 非阻塞, 间接)
 n0--n1--n2--n3                    n0 ──┬──┬──┬──┬──┬──┬──┬──  0
  |   |   |   |                          |  |  |  |  |  |  |  |
  +===+===+===+==== 共享线            ├──┼──┼──┼──┼──┼──┼──  1
                                       每对 (in,out) 一个交叉点开关
  成本 O(1) 线, 延迟/带宽 = 常数       成本 O(N^2) 交叉点, 延迟 O(1)

(3) RING (N=4)                   (4) 2D MESH (4x4, 直接网络)
  0 --- 1                          0--1--2--3
  |     |                          |  |  |  |
  3 --- 2                          4--5--6--7
                                    |  |  |  |
  成本 O(N), 延迟 O(N)             8--9-10-11
  二分带宽 = 2 条链路 (常数!)     |  |  |  |
                                  12-13-14-15
                                   成本 O(N), 平均延迟 O(sqrt(N))
                                   二分带宽 = k 条链路 (k=4)

(5) 2D TORUS (把 mesh 的边界卷起来)   (6) FAT TREE (8 叶)
  +--0--1--2--3--+                    [root 层, 带宽最大]
  |  |  |  |  |  |                         =========
  +--4--5--6--7--+                       /         \
  |  |  |  |  |  |                     [中间层]      [中间层]
  +--8--9-10-11--+                      /   \          /   \
  |  |  |  |  |  |                   0 1    2 3      4 5    6 7
  +-12-13-14-15--+                      延迟 O(lg N), 可做 full bisection
  每行每列首尾相连                      越靠根链路越"胖" (更高带宽)

(7) 3 维 HYPERCUBE (N=8)          (8) OMEGA / 多级对数网络 (N=8)
  000 ----- 001                     0 --\  /-- ... --\  /-- 0
   |  \    /  |                     1 --/  \-- ... --/  \-- 1
  010 -- 011   |                           [lg N 级 2x2 开关]
   |     |     |                    成本 O(N lg N), 延迟 O(lg N)
  100 -- 101  ...
  编号只差 1 bit 的节点直连       变体: Omega / butterfly / Clos
  延迟 O(lg N), 度 O(lg N), 链路 O(N lg N)
```

- **图 5：KNL 的 6×6 tile mesh 与 YX 路由（slide 27）**

```
  Intel Xeon Phi (Knights Landing): 72 核 = 6x6 tile 网格 (每 tile 2 核)
  外圈是 EDC / iMC (内存控制器) / MCDRAM / OPIO / PCIe 等

       +----+----+----+----+----+----+
       |Tile|Tile|Tile|Tile|Tile|Tile|   消息路由: YX routing
       +----+----+----+----+----+----+     (1) 先在 Y 方向走
       |Tile|Tile|Tile|Tile|Tile|Tile|     (2) "转弯" (turn)
       +----+----+----+----+----+----+     (3) 再在 X 方向走
       |Tile|Tile|Tile|Tile|Tile|Tile|
       +----+----+----+----+----+----+   例如 (x=1,y=4) -> (x=4,y=1):
       |Tile|Tile|Tile|Tile|Tile|Tile|     先向上走 3 跳 (Y: 4->1)
       +----+----+----+----+----+----+     再向右走 3 跳 (X: 1->4)
       |Tile|Tile|Tile|Tile|Tile|Tile|
       +----+----+----+----+----+----+
       |Tile|Tile|Tile|Tile|Tile|Tile|   维度顺序路由 (dimension-order)
       +----+----+----+----+----+----+   的通道依赖图无环 => 不会死锁

  性能含义: 平均跳数 ≈ 2k/3 = 4 跳 (k=6), 每跳约 1~2 个网络时钟;
            路由确定性 => 路径唯一 => 热点流量无法绕行 (无自适应能力)
```

2.7 拓扑对照表（slide 32 的复习表 + 本笔记补充列）

下表的前四列与 slide 32 的"Review: network topologies"表一致，后三列是本笔记为量化分析补充的（N = 节点数，k = √N，B = 单条链路带宽）：

拓扑 Topology	Direct/Indirect	Blocking	成本 Cost	延迟 Latency	链路数 / 二分带宽	真实系统
Bus	—（共享介质）	Blocking	O(1) 组线	O(1) 但**随 N 恶化**	二分带宽 = 总带宽（且不随 N 增长）	早期多核 front-side bus
Crossbar	Indirect	**Non-blocking**	**$O(N^2)$**	**O(1)**	二分带宽 = (N/2)·B	Sun SPARC T2 / T5（CCX）
Multi-stage log.（Omega/butterfly）	Indirect	**Blocking**（课堂讨论的那种；其他变体不一定）	O(N lg N)	O(lg N)	(N/2)·B	大型交换机内部、CMOS 交换网络
Ring	Direct	Blocking	O(N)	O(N)	2·B（常数！不 scale）	Intel Sandy Bridge 起的 ring、IBM CELL
2D Mesh (k×k)	Direct	Blocking	O(N)（2N−2k 条链路）	**平均 $O(\sqrt{N})$**	k·B	Tilera、KNL 6×6、Intel 原型
2D Torus (k×k)	Direct	Blocking	O(N)（2N 条链路，比 mesh 贵）	$O(\sqrt{N})$	2k·B	部分 HPC 与片上研究原型
Fat Tree (N 叶)	Indirect	可做 Non-blocking	O(N lg N)	O(lg N)	可做到 (N/2)·B（full bisection）	大型机群/数据中心网络
Hypercube	Direct	Blocking	O(N lg N)	**O(lg N)**，度 O(lg N)	(N/2)·B（成本效率最高）	Caltech Cosmic Cube、SGI Origin

（表中 Bus / Ring / Mesh / Torus / Hypercube 的成本与延迟数量级均取自讲义；Crossbar 的"非阻塞、O(1)、$O(N^2)$"与 Multi-stage 的"阻塞、O(N lg N)、O(lg N)"直接对应 slide 32 的复习表。）

2.8 通信粒度：message / packet / flit（slide 35–36）

- **定义与目的**：
- **Message（消息）**：网络客户端（核、内存）之间传输的**单位**，可以用多个 packet 传。
- **Packet（包）**：**网络**的传输单位，可以用多个 flit 传。
- **Flit（flow control digit）**：包被切成的更小单位，是**网络中流控与缓冲的最小粒度**。
- **Packet format（包格式）**：**Header（头）** 含路由与控制信息，放在包开头以让路由器**尽早开始转发**；**Payload/body（载荷）** 是要传的数据；**Tail（尾）** 含控制信息（如**错误校验码**），**放在末尾**是因为发送方可以"边发边算校验和，最后追加"。
- **直观解释（"它是什么？"）**：把"寄一整套书"想成：**message = 一整套书**、**packet = 一个纸箱**、**flit = 箱子里的一本书**。为什么不直接整箱整箱地搬？因为中转站的货架（buffer）有限，**按"本"流转（flit 级流控）才能把有限的货架用出最大通行量**。

粒度	谁的单位	类比	决定什么
Message	网络**客户端**（core/memory）	一整套书	软件可见的通信单元（MPI 消息）
Packet	**网络**传输单元	一个纸箱	路由决策的边界、校验的单位
Flit	**流控/缓冲**最小粒度	箱中的一本书	缓冲容量、流控开销、能效

- **图 6：包格式与卡片式布局**

```
      +----------------+---------------------------+----------------+
      |     HEADER     |      PAYLOAD / BODY       |      TAIL      |
      |  路由 + 控制    |       要传输的数据          |  控制信息(校验) |
      +----------------+---------------------------+----------------+
       ^                                           ^
       |                                           |
  放在最前：路由器读到头部就可以                    放在最后：发送方"边发边算"
  "提前转发"(start forwarding early)                 checksum，最后追加到包尾

一个 message 被切成多个 packet；一个 packet 被切成多个 flit：
  MESSAGE  [ pkt0 ][ pkt1 ][ pkt2 ] ...
  PACKET   [ H ][ B0 ][ B1 ][ B2 ][ T ]        <- 例如 4 个 flit
  FLIT      ^    单个 flit 是网络中缓冲/流控的最小单位
```

2.9 交换方式：circuit switching vs. packet switching（slide 34、38）

- **定义与目的**：
- **Circuit switching（电路交换）**：发送前**先建立完整通路（获取全部资源）**——先"探测/建立路由"（reserve links），再发送全部数据。
- **Packet switching（包交换）**：**为每个包单独做路由决策**，每个包可能走不同的链路。
- **对比（讲义 slide 34 的原话归纳）**：

维度	Circuit switching	Packet switching
建立阶段	**需要 setup（probe）**，还需 teardown 释放资源	**不需要** setup/teardown
传输期带宽	高（**没有逐包的链路管理开销**）	有**动态交换逻辑**的传输期开销
链路利用率	**低**（两条消息不能共用同一条预留链路，即使路径上某些资源已闲置）	高（**链路一空闲就能拿来传包**）
争用处理	预分配后传输期**无争用，不需要缓冲**	需要缓冲/丢弃/绕行等机制
消息大小	**任意大小**（通路建好后一直传）	受包大小与缓冲约束
直观类比	**打电话**：先拨号接通（占线就等），通了就一直说	**寄快递**：每个包裹单独择路，随时可寄
代价	setup 与 tear-down 的开销、低利用率	每包的交换开销、需要缓冲

- **讲义的处理方式（slide 38）**：电路交换的要点被总结为"**高粒度的资源分配**"——**沿整条网络路径预分配所有资源（跨多个交换机的链路）来为一条消息"建立一条流"**；好处是传输期无争用因而无需缓冲、且消息大小任意；代价是建立/拆除开销与低链路利用率。**本讲后续只考虑缓冲式（buffered）网络**，但讲义也指出：近期研究在探讨**无缓冲网络 + deflection routing（偏转路由）** 作为更省电的片上互连。

2.10 缓冲式流控三兄弟：store-and-forward / cut-through / wormhole（slide 37、39–43）

- **定义与目的**：当两个包同时要同一条输出链路时（slide 37 的争用场景），有三种选择：**缓冲一个包晚点再发、丢弃一个包、改道一个包（deflection）**。本讲只考虑**缓冲**。而"在哪里缓冲、缓冲到什么粒度"，就分出了三种流控：
- **Store-and-forward（存储转发，以包为单位）**：包被**完整**复制进交换机后才能走向下一节点；**流控单位是整个包**；因此**每个路由器都要有容纳整包的缓冲**。同一消息的不同包可以走不同路由，但**同一个包内的所有数据必须走同一条路由**。特征是**每包延迟极高**：`延迟 = 包在一条链路上的传输时间 × 网络距离（跳数）`。
- **Cut-through（直通，仍以包为单位）**：交换机**一收到包头就开始在下一段链路上转发**（包头携带"这个包需要多少链路带宽 + 往哪走"的信息）；结果是**传输延迟下降**；但**在高争用时 cut-through 会退化成 store-and-forward**（因为下游被堵，整个包最终还是要被吸进缓冲）。
- **Wormhole（虫洞）**：**包被切成更小的 flit**，并以 **flit 为缓冲与流控的最小粒度**——这与前两者"包既是传输粒度又是流控/缓冲粒度"形成对比。规则是：**路由信息只在 head flit 里**；body flit 跟着 head 走；**tail flit 收尾**；**head flit 一旦被阻塞，整个包就停下来**；传输是**完全流水化（completely pipelined）** 的——**对长消息而言，延迟几乎与网络距离无关。**
- **直观解释（"它是什么？"）**：
- store-and-forward = **中转站必须把整辆集装箱卡车卸完、再装到下一辆车**才发车；
- cut-through = **车头一到，就先让车头开上下一段路**，车身随后跟进；
- wormhole = **一列火车**：车厢（flit）排成一列前进，**车头一停，整列车都停在轨道上**——但也正因为如此，**缓冲只需容纳几节车厢（flit），而不是整列车（整包）。**
- **图 7：三种流控的时间–空间图（用讲义 slide 40 的单位模型：3 跳、包长 4 单位、每跳路由延迟 1 单位）**

```
  每格 = 1 个 flit 在一条链路上传输所需时间；包 = 4 flits
  路径 = Src -> R1 -> R2 -> Dst (3 跳)；路由器延迟 = 1 格

                       t = 0   1   2   3   4   5   6   7   8   9  10  11  12
(a) STORE-AND-FORWARD (整包缓冲后再转发)
    Src -> R1            [##][##][##][##]  .   .   .   .   .   .   .   .
    R1  -> R2              .   .   .   .  [##][##][##][##]  .   .   .   .
    R2  -> Dst             .   .   .   .   .   .   .   .  [##][##][##][##]
    完成于 t = 12          <=>  3 跳 x 4 单位 = 12 单位 (每跳都要把整包传完)

(b) CUT-THROUGH (包头一到就开始转发)
    Src -> R1            [##][##][##][##]  .   .   .   .   .   .   .   .
    R1  -> R2              .  [##][##][##][##]  .   .   .   .   .   .   .
    R2  -> Dst             .   .  [##][##][##][##]  .   .   .   .   .   .
    完成于 t = 6           <=>  3 (流水填充) + 3 (其余 flit 跟进) = 6 单位
                                  => 比 store-and-forward 快 2 倍

(c) WORMHOLE (flit 级缓冲/流控；定时与 (b) 相同，但缓冲只需 flit 大小)
    Src -> R1            [H ][B0][B1][T ]  .   .   .   .   .   .   .   .
    R1  -> R2              .  [H ][B0][B1][T ]  .   .   .   .   .   .   .
    R2  -> Dst             .   .  [H ][B0][B1][T ]  .   .   .   .   .   .
                               ^
                  t=2 快照：H 已到 R2，B0 在 link1 上，
                            B1/T 还在 Src 的缓冲里 —— 完全流水化
    长消息时：T ≈ L/b + D * t_r，当 L/b >> D * t_r 时
              => 延迟几乎与网络距离 D 无关 (讲义 slide 43 的思考题)
```

- **性能特征小结**：
- store-and-forward 的延迟与**距离成正比**（`~D × L/b`）；
- cut-through 把延迟降到 `~D × t_r + L/b`，但**坏情况下退化为 store-and-forward**；
- wormhole 在**延迟上等价于 cut-through，但缓冲面积小得多**（flit 而非整包），代价是**head-of-line blocking**（下一节）。

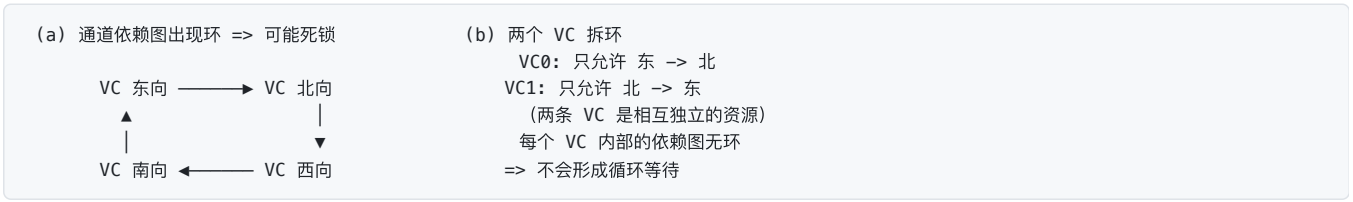
2.11 Head-of-line blocking 与虚通道（slide 44–46）

- **定义与目的：Head-of-line blocking（队头阻塞）** 指：一个输入缓冲里排在**队头**的包因为它的目标输出链路忙而被堵住，导致排在它**后面**、本来可以去**空闲**链路的包也被一起堵住。**Virtual channel（虚通道，VC）** 的解法是：**把一条物理通道上的输入缓冲切成多个独立缓冲（multiple buffers sharing a single physical channel）**，从而**减少 head-of-line blocking**——即**"在单条物理通道上复用多个操作"**（讲义引 Dally, ISCA 1990 的《Virtual Channel Flow Control》）。
- **直观解释（"它是什么？"）**：想象**超市只有一条结账队伍**，队头顾客的会员卡出了问题要等经理（他的"输出链路"被占），后面所有只想刷卡走人的顾客也一起被卡住——这就是队头阻塞。**虚通道就是把这排队伍分成几排（VC0 / VC1 ...）**，队头卡住时，另一排的人可以先去空闲的收银台。注意**收银台（物理链路）本身还是那几个**，VC 只是**让缓冲和仲裁解耦**。
- **图 8：head-of-line blocking 与虚通道的对比（slide 44 / 45）**

```
场景：某路由器的西向输入端口收到两个包
      灰包（先到）要去东向链路 —— 但东向链路当前被别的包占用（busy）
      蓝包（后到）只想去北向链路 —— 北向链路当前空闲

(a) 无虚通道：1 个输入缓冲                (b) 2 条虚通道：VC0 / VC1
+---------------------------+              +---------------------------+
| In(W) Buf:  [灰灰灰]      |  <- 队头     | VC0 Buf: [灰灰灰]         | 灰包仍等东向
|             [蓝蓝蓝]      |     卡住     | VC1 Buf: [蓝蓝蓝]         | 蓝包可走北向
+-------------+-------------+              +------+--------------+-----+
              |                                   |              |
     只有队头能被仲裁                       东向(忙: 灰包等)   北向(空闲: 蓝包走)
              |
     东向链路(忙)  -> 灰包阻塞
     北向链路(空闲) -> 白白浪费            结果：空闲链路被利用起来，
     结果：吞吐损失、延迟上升                吞吐上升、延迟下降
```

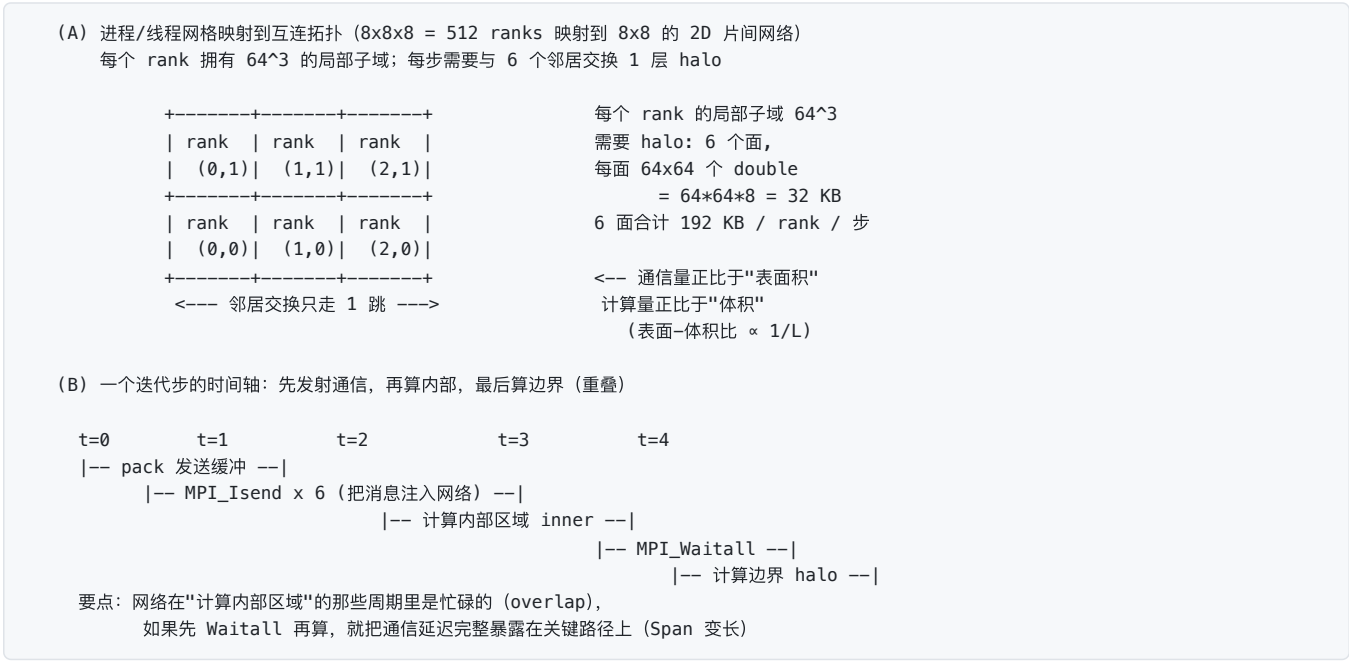
- **虚通道的其他用途（slide 46）**：
- **避免死锁（deadlock avoidance）**：用来**打破资源的循环依赖**——例如**让请求（request）与响应（response）走不同的虚通道**以避免成环；**"escape" VC** 的做法是**保留至少一条使用无死锁路由的虚通道**（其余通道可以更激进）。

- **流量类别优先级（prioritization of traffic classes）**：**提供服务质量（QoS）保证**，让某些虚通道的优先级高于其他（例如让同步消息/中断快过批量数据传输）。
- **图 9：死锁的环依赖与用虚通道打破它**



2.12 软件执行模型：程序员如何"感受"到互连网络

- **定义与目的**：互连网络不暴露任何编程接口，程序员是通过**通信模式**与它打交道的。同一个网络面对两种截然不同的流量会给出完全不同的性能：**邻居交换（stencil halo exchange）** 只用到局部链路（mesh 上平均 1 跳），**全对全（all-to-all / allreduce）** 则要求把**二分带宽**吃满。
- **直观解释（"它是什么？"）**：程序里的通信模式就是给网络下的"订单"：**要多少条消息、每条多大、发给谁**。网络对"少量大消息"和"海量小消息"的答复完全不同（这正是下一节 α–β 模型的含义），对"邻居通信"和"全局通信"的答复也完全不同（这正是二分带宽的含义）。
- **图 10（软件执行模型）：3D 7 点 stencil 的 halo 交换如何映射到网络**



- **性能特征**：邻居交换模式下，网络只需提供 `6 × 每 rank 消息大小 / 每步时间` 的注入带宽，且绝大部分流量横跨的**二分带宽需求 = 网格切面 × 带宽**；全对全模式下，需求变成 `N/2 × 每节点注入率`，**二分带宽立刻成为硬约束**。同一个网络，两种模式的可达性能可以差一个数量级。

---

3. 代码示例与性能分析

3.1 示例 1：实测网络的 α（零负载延迟）与 β（渐近带宽）—— MPI ping-pong 与 ring 邻居交换

```
/* ============================================================================
 * mpi_net_probe.c —— 互连网络的 alpha / beta 实测
 *   1) ping-pong : 一对 rank 互发，测出往返延迟 RTT(n)，拟合 RTT = alpha + n/beta
 *   2) ring      : 所有 rank 同时与左右邻居交换，给网络真正加载
 *
 * 编译 (release) :
 *   mpicc -O3 -march=native -DNDEBUG -std=c11 mpi_net_probe.c -o mpi_net_probe
 * 运行:
 *   mpirun -np 8 --bind-to core ./mpi_net_probe
 * ============================================================================*/
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ---------- ping-pong：只有 rank 0 与 rank 1 在通信，返回一次往返时间 ---------- */
static double pingpong_once(int iters, int bytes)
{
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    char *sbuf = (char *)malloc((size_t)bytes);
    char *rbuf = (char *)malloc((size_t)bytes);
    memset(sbuf, rank, (size_t)bytes);
    memset(rbuf, 0, (size_t)bytes);

    /* 预热：让网络接口与内存路径进入稳态，否则第一次测量混入冷启动开销 */
    for (int i = 0; i < 10; i++) {
        if (rank == 0) {
            MPI_Send(sbuf, bytes, MPI_BYTE, 1, 0, MPI_COMM_WORLD);
            MPI_Recv(rbuf, bytes, MPI_BYTE, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        } else if (rank == 1) {
            MPI_Recv(rbuf, bytes, MPI_BYTE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(sbuf, bytes, MPI_BYTE, 0, 0, MPI_COMM_WORLD);
        }
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();
    for (int i = 0; i < iters; i++) {
        if (rank == 0) {
            MPI_Send(sbuf, bytes, MPI_BYTE, 1, 0, MPI_COMM_WORLD);
            MPI_Recv(rbuf, bytes, MPI_BYTE, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        } else if (rank == 1) {
            MPI_Recv(rbuf, bytes, MPI_BYTE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(sbuf, bytes, MPI_BYTE, 0, 0, MPI_COMM_WORLD);
        }
    }
    double dt = MPI_Wtime() - t0;

    free(sbuf); free(rbuf);

    double worst = dt;                      /* 取所有 rank 的最大值，含启动偏差 */
    MPI_Reduce(&dt, &worst, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    return worst / (double)iters;           /* 单位：秒 / 往返 */
}

/* ---------- ring：所有 rank 同时与左右邻居交换（真正的网络负载测试） ---------- */
static double ring_step(int iters, int bytes)
{
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    char *sbuf = (char *)malloc((size_t)bytes);
    char *rbuf = (char *)malloc((size_t)bytes);
    memset(sbuf, rank, (size_t)bytes);

    int right = (rank + 1) % size;
    int left  = (rank + size - 1) % size;
```

```
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();
    for (int i = 0; i < iters; i++) {
        /* 同时收发的邻居交换：网络上出现 size 条并发的消息流 */
        MPI_Sendrecv(sbuf, bytes, MPI_BYTE, right, 0,
                     rbuf, bytes, MPI_BYTE, left,  0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    double dt = MPI_Wtime() - t0;
    free(sbuf); free(rbuf);

    double worst = dt;
    MPI_Reduce(&dt, &worst, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    return worst / (double)iters;
}

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0) fprintf(stderr, "need >= 2 ranks\n");
        MPI_Finalize();
        return 1;
    }

    const int nsz = 7;
    const int sizes[7] = {8, 64, 512, 4096, 32768, 262144, 1048576};
    double rtt[7];

    if (rank == 0)
        printf("== ping-pong (rank 0 <-> rank 1) ==\n%10s %12s %12s %12s\n",
               "bytes", "RTT[us]", "1-way[us]", "BW[GB/s]");

    for (int i = 0; i < nsz; i++) {
        int iters = (sizes[i] <= 4096) ? 2000 : (sizes[i] <= 65536 ? 500 : 50);
        rtt[i] = pingpong_once(iters, sizes[i]);
        if (rank == 0)
            printf("%10d %12.3f %12.3f %12.3f\n",
                   sizes[i], rtt[i] * 1e6, rtt[i] * 5e5,
                   (double)sizes[i] / rtt[i] / 1e9);
    }

    if (rank == 0) {
        /* 最小二乘拟合 RTT(n) = alpha + n / beta */
        double sx = 0, sy = 0, sxx = 0, sxy = 0;
        for (int i = 0; i < nsz; i++) {
            double n = (double)sizes[i];
            sx += n; sy += rtt[i]; sxx += n * n; sxy += n * rtt[i];
        }
        double k = (nsz * sxy - sx * sy) / (nsz * sxx - sx * sx);  /* = 1/beta */
        double a = (sy - k * sx) / nsz;                            /* = alpha  */
        double beta = 1.0 / k;
        printf("\nfit : RTT(n) = %.3f us + n / %.2f GB/s\n", a * 1e6, beta / 1e9);
        printf("     zero-load RTT alpha = %.3f us ; asymptotic BW beta = %.2f GB/s\n",
               a * 1e6, beta / 1e9);
        printf("     crossover n* = alpha*beta = %.1f KB "
               "(消息小于它时，延迟主导；大于它时，带宽主导)\n", a * beta / 1024.0);
    }

    if (rank == 0)
        printf("\n== ring neighbor exchange (all ranks active) ==\n"
               "%10s %12s %14s\n", "bytes", "step[us]", "aggregate[GB/s]");
    for (int i = 0; i < nsz; i++) {
        int iters = (sizes[i] <= 4096) ? 2000 : (sizes[i] <= 65536 ? 500 : 50);
        double st = ring_step(iters, sizes[i]);
        if (rank == 0) {
            /* 每一步全网有 2*size 条消息（每个 rank 各收、各发一条） */
            double agg = 2.0 * size * (double)sizes[i] / st / 1e9;
            printf("%10d %12.3f %14.3f\n", sizes[i], st * 1e6, agg);
```

```
        }
    }

    MPI_Finalize();
    return 0;
}
```

【代码做什么？】 1. `pingpong_once()`：只让 rank 0 与 rank 1 交替 `MPI_Send` / `MPI_Recv`。**关键点是"依赖链"**——rank 0 必须等到 rank 1 回来的数据才能发下一轮，因此测得的 `RTT` 是**纯粹的延迟**，几乎不含带宽成分。先跑 10 次预热，再进主循环，最后用 `MPI_Reduce(MPI_MAX)` 取所有 rank 中最大的耗时（避免某个 rank 被 OS 调度干扰而低估）。 2. 主循环对 8 B 到 1 MiB 的 7 个尺寸各测一轮，打印 RTT、单向延迟、以及 `n/RTT` 得到的**有效带宽**。你会看到 8 B 时有效带宽只有几 MB/s（延迟主导），1 MiB 时才逼近 β（带宽主导）。 3. 用**最小二乘**在 `(n, RTT)` 平面上拟合 `RTT(n) = α + n/β`：斜率 `k = 1/β`，截距 `α`，并算出**交叉点** `n* = α·β`（超过它传输时间才超过启动延迟）。 4. `ring_step()`：让**所有** rank 同时与左右邻居用 `MPI_Sendrecv` 交换。此时网络上同时存在 `size` 条并发消息流，测得的单步时间与 `2·size·n / step` 给出**聚合带宽**——它才是"网络能扛多少"的答案，并且会被**二分带宽**（以及每节点的注入带宽）封顶。

【并行机制与性能解说】 – **并行如何在硬件上发生**：`ping-pong` 模式只有 2 个进程参与，网络上只有 1 条消息在飞，网络内部的 **router 流水线全部空转**——测到的是"延迟"，不是"吞吐"。`ring` 模式里，每个 rank 的发送被送到本地网络接口，**网络接口把消息切成 packet/flit 注入路由器**，`size` 条消息在路由器上**并行转发**（每个路由器每周期可以对多个不同输出端口各推进一个 flit）。 – **Work / Span / 并行度**： – ping-pong（`I` 次迭代）：**Work** = `2I` 次消息传递，每次代价 `α + n/β`；**Span（关键路径）** = `I·(α + n/β)`（每一轮往返都依赖上一轮返回的数据）；**并行度 = Work/Span = 2**。结论：**这个模式最多只能利用 2 个端点的带宽**，无论机器多大都只测出延迟。 – ring（`I` 次迭代、`P` 个 rank）：**Work** = `2PI` 次消息传递（每个 rank 每轮各收、各发一条）；**Span** = `I·(α + n/β)`（邻居交换的每一步都要等对端发来，形成一条**跨迭代的依赖链**）；**并行度 = 2PI / (I(α+n/β)) = 2P**。结论：**可扩展性上限 = 2P**，但真实上限还要被**二分带宽**二次封顶：二分带宽 `BW_bisect` 决定 `P·2n/step ≤ BW_bisect`，即 `step ≥ 2Pn/BW_bisect`。 – **瓶颈**：(a) 小消息时完全是 `α` 主导——`n=8 B`、`α=1.5 μs` 时有效带宽只有 ~5.3 MB/s，只有 β 的 0.04%；(b) ring 模式下若 `P` 很大且每节点注入率之和超过二分带宽，网络进入**饱和区**（图 2 曲线的陡升段），此时增加消息大小不会线性提升聚合带宽，反而可能因**队头阻塞**与缓冲耗尽而恶化；(c) 若消息切分过大，还会引入**串行化延迟**（n/β）暴露在关键路径上——这就是为什么**大消息 + 重叠通信**才是正解。

3.2 示例 2：4×4 mesh 上三种流控策略的周期级模拟（store-and-forward / wormhole / wormhole+VC）

```c
/* ============================================================================
 * mesh_flowctl.c —— 4x4 二维 mesh 上三种流控策略的周期级并行模拟
 *   MODE_SF : store-and-forward（输入缓冲必须收满整包才能开始转发）
 *   MODE_WH : wormhole（flit 级缓冲/流控，1 条虚通道）
 *   MODE_VC : wormhole + 2 条虚通道（缓解 head-of-line blocking）
 *   路由：YX（先在 Y 方向走，再转弯，再在 X 方向走）—— 与 KNL 一致
 *
 * 编译（release）：
 *   gcc -O3 -march=native -fopenmp -std=c11 mesh_flowctl.c -o mesh_flowctl
 * 运行：
 *   OMP_NUM_THREADS=8 ./mesh_flowctl
 * ============================================================================*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define RX 4
#define RY 4
#define NR (RX * RY)          /* 16 个路由器 */
#define NLEN 4                /* 每个包 = 4 个 flit */
#define CAP 8                 /* wormhole 下每个输入缓冲的 flit 容量 */
#define NVC 2                 /* 每条物理通道上的虚通道数 */
#define NPKT 128              /* 总包数 */
#define MAXCYC 40000

enum { MODE_SF = 0, MODE_WH = 1, MODE_VC = 2 };

/* 端口编号：0=N, 1=S, 2=E, 3=W, 4=LOCAL(注入/弹出) */
typedef struct { int pkt; int dst; int tail; } flit_t;

typedef struct {
    flit_t q[CAP];
    int head, count;
    int route;        /* 该缓冲中当前包的输出端口；-1 表示空 */
    int pkt;          /* 该缓冲中当前包的 id；-1 表示空 */
    int draining;     /* SF：整包已收齐、正在发出（发出期间不再要求缓冲为"满"） */
} ibuf_t;

static ibuf_t ib[NR][5][NVC];             /* 当前状态 */
static flit_t nb_f[NR][5][NVC];           /* 本周期到达的 flit（写者唯一） */
static int    nb_v[NR][5][NVC];
static int    nb_r[NR][5][NVC];
static int    nb_p[NR][5][NVC];
static int    snap_c[NR][5][NVC];         /* 周期开始时的信用快照（count） */
static int    snap_p[NR][5][NVC];         /* 周期开始时的信用快照（pkt） */
static unsigned rr[NR][5];                /* 输出端口上的 VC 轮转指针（防 VC0 饿死） */

static int g_mode = MODE_WH, g_nvc = 1;
static int pkt_src[NPKT], pkt_dst[NPKT], pkt_left[NPKT];
static long long inj_cyc[NPKT], done_cyc[NPKT];
static long long g_cycle;

static const int DX[4] = {0, 0, 1, -1};   /* N,S,E,W */
static const int DY[4] = {-1, 1, 0, 0};

static int neighbor(int r, int p)
{
    if (p == 4) return r;
    int x = r % RX, y = r / RX;
    int nx = x + DX[p], ny = y + DY[p];
    if (nx < 0 || nx >= RX || ny < 0 || ny >= RY) return -1;
    return ny * RX + nx;
}
static int opposite(int p) { return (p == 0) ? 1 : (p == 1) ? 0 : (p == 2) ? 3 : (p == 3) ? 2 : 4; }
static int cap_of(void) { return (g_mode == MODE_SF) ? NLEN : CAP; }

/* YX 路由：先 Y 后 X，与讲义 slide 27 的 KNL 路由一致 */
static int route_of(int x, int y, int dst)
{
    int dx = dst % RX, dy = dst / RX;
```

```
    if (dy < y) return 0;    /* N */
    if (dy > y) return 1;    /* S */
    if (dx < x) return 3;    /* W */
    if (dx > x) return 2;    /* E */
    return 4;                /* LOCAL：目的就是本节点 */
}

static void bpush(ibuf_t *b, flit_t f) { b->q[(b->head + b->count) % CAP] = f; b->count++; }
static flit_t bpop(ibuf_t *b)
{
    flit_t f = b->q[b->head];
    b->head = (b->head + 1) % CAP;
    b->count--;
    return f;
}

static void reset_state(void)
{
    memset(ib, 0, sizeof ib);
    memset(nb_v, 0, sizeof nb_v);
    memset(rr, 0, sizeof rr);
    for (int r = 0; r < NR; r++)
        for (int p = 0; p < 5; p++)
            for (int v = 0; v < NVC; v++) { ib[r][p][v].pkt = -1; ib[r][p][v].route = -1; }
    for (int k = 0; k < NPKT; k++) { pkt_left[k] = NLEN; inj_cyc[k] = -1; done_cyc[k] = -1; }
    g_cycle = 0;
}

/* 生成流量：hotspot 时 25% 的包全部打向 router 5，制造队头阻塞 */
static void gen_traffic(unsigned seed, int hotspot)
{
    unsigned s = seed;
    for (int k = 0; k < NPKT; k++) {
        s = s * 1103515245u + 12345u;
        int src = (int)((s >> 16) % NR);
        int dst;
        if (hotspot && (k % 4) == 0) dst = 5;
        else { s = s * 1103515245u + 12345u; dst = (int)((s >> 16) % NR); }
        if (dst == src) dst = (src + 1) % NR;
        pkt_src[k] = src;
        pkt_dst[k] = dst;
    }
}

/* ---------- 阶段 0：信用快照（本周期所有仲裁都基于这份只读快照） ---------- */
static void snapshot_phase(void)
{
#pragma omp parallel for schedule(static)
    for (int r = 0; r < NR; r++)
        for (int p = 0; p < 5; p++)
            for (int v = 0; v < NVC; v++) {
                snap_c[r][p][v] = ib[r][p][v].count;
                snap_p[r][p][v] = ib[r][p][v].pkt;
            }
}

/* ---------- 阶段 1：注入（每路由器每周期最多 1 个 flit） ---------- */
static void inject_phase(void)
{
#pragma omp parallel for schedule(static)
    for (int r = 0; r < NR; r++) {
        for (int k = 0; k < NPKT; k++) {
            if (pkt_src[k] != r || pkt_left[k] <= 0) continue;
            ibuf_t *b = &ib[r][4][0];                       /* Local 输入端口, VC0 */
            if (b->count >= cap_of()) break;
            if (b->count > 0 && b->pkt != k) break;         /* 被别的包占着 */
            flit_t f;
            f.pkt = k;
            f.dst = pkt_dst[k];
            f.tail = (pkt_left[k] == 1);
            if (pkt_left[k] == NLEN) inj_cyc[k] = g_cycle;  /* head flit 注入时刻 */
            if (b->count == 0) { b->pkt = k; b->route = route_of(r % RX, r / RX, f.dst); b->draining = 0; }
```

```
            bpush(b, f);
            pkt_left[k]--;
            break;                                          /* 每周期只注入 1 个 flit */
        }
    }
}

/* ---------- 阶段 2：仲裁 + 过链路（先查信用再选包，物理链路每周期 1 个 flit） ---- */
static void arbitrate_phase(void)
{
#pragma omp parallel for schedule(static)
    for (int r = 0; r < NR; r++) {
        for (int o = 0; o < 5; o++) {
            int served = 0;
            for (int k = 0; k < g_nvc && !served; k++) {
                int v = (int)((rr[r][o] + (unsigned)k) % (unsigned)g_nvc);
                for (int p = 0; p < 5 && !served; p++) {
                    ibuf_t *b = &ib[r][p][v];
                    if (b->count == 0) continue;
                    if (b->route != o) continue;
                    /* SF: 整包收齐才能开始转发；一旦开始（draining）就把剩余 flit 发完 */
                    if (g_mode == MODE_SF && b->count < NLEN && !b->draining) continue;

                    flit_t f = b->q[b->head];
                    int tv = v;
                    if (o == 4) {                       /* 弹出到本地：总能成功 */
                        bpop(b);
                        if (b->count == 0) { b->pkt = -1; b->route = -1; b->draining = 0; }
                        else if (g_mode == MODE_SF) b->draining = 1;  /* SF：包已开始送出 */
                        if (f.tail) done_cyc[f.pkt] = g_cycle + 1;   /* tail 弹出即完成 */
                        served = 1;
                        continue;
                    }
                    int n = neighbor(r, o);
                    int opp = opposite(o);
                    if (g_nvc > 1) {                    /* 虚通道分配：优先沿用原 VC */
                        int pick = -1;
                        if (snap_c[n][opp][tv] == 0 || snap_p[n][opp][tv] == f.pkt) pick = tv;
                        else {
                            int w = 1 - tv;
                            if (snap_c[n][opp][w] == 0 || snap_p[n][opp][w] == f.pkt) pick = w;
                        }
                        if (pick < 0) continue;         /* 两条 VC 都被别的包占用 */
                        tv = pick;
                    }
                    /* credit 检查（用只读快照，避免同周期读写竞争）*/
                    int room = (snap_c[n][opp][tv] < cap_of()) &&
                               (snap_c[n][opp][tv] == 0 || snap_p[n][opp][tv] == f.pkt);
                    if (!room) continue;                /* 下游无信用：本包这周期不被选中 */

                    bpop(b);                            /* 被选中才真正搬走 flit */
                    if (b->count == 0) { b->pkt = -1; b->route = -1; b->draining = 0; }
                    else if (g_mode == MODE_SF) b->draining = 1;
                    nb_f[n][opp][tv] = f;               /* 写者唯一：只有 r 能写这个入口 */
                    nb_v[n][opp][tv] = 1;
                    nb_r[n][opp][tv] = route_of(n % RX, n / RX, f.dst);
                    nb_p[n][opp][tv] = f.pkt;
                    served = 1;
                    rr[r][o] = (unsigned)((v + 1) % g_nvc);
                }
            }
        }
    }
}

/* ---------- 阶段 3：把到达的 flit 并入输入缓冲（空缓冲时确定该包的输出端口） ---- */
static void merge_phase(void)
{
#pragma omp parallel for schedule(static)
    for (int r = 0; r < NR; r++)
        for (int p = 0; p < 5; p++)
            for (int v = 0; v < NVC; v++) {
```

```
                if (!nb_v[r][p][v]) continue;
                ibuf_t *b = &ib[r][p][v];
                if (b->count == 0) { b->pkt = nb_p[r][p][v]; b->route = nb_r[r][p][v]; b->draining = 0; }
                bpush(b, nb_f[r][p][v]);
                nb_v[r][p][v] = 0;
            }
}

static void run_mode(int mode, const char *name, int hotspot)
{
    g_mode = mode;
    g_nvc  = (mode == MODE_VC) ? 2 : 1;
    reset_state();
    gen_traffic(20260916u, hotspot);

    long long cyc = 0;
    for (; cyc < MAXCYC; cyc++) {
        g_cycle = cyc;
        snapshot_phase();
        inject_phase();
        arbitrate_phase();
        merge_phase();
        int all_done = 1;
        for (int k = 0; k < NPKT; k++) if (done_cyc[k] < 0) { all_done = 0; break; }
        if (all_done) { cyc++; break; }
    }

    double lat = 0; long long worst = 0, t0 = 0, t1 = 0; int done = 0;
    for (int k = 0; k < NPKT; k++) {
        if (done_cyc[k] < 0) continue;
        long long L = done_cyc[k] - inj_cyc[k];
        lat += (double)L; done++;
        if (L > worst) worst = L;
        if (t0 == 0 || inj_cyc[k] < t0) t0 = inj_cyc[k];
        if (done_cyc[k] > t1) t1 = done_cyc[k];
    }
    double flits = (double)NPKT * NLEN;
    printf("%-18s | %6d/%d | %9.2f | %7lld | %8lld | %9.3f\n",
           name, done, NPKT, lat / (done ? done : 1), worst, cyc,
           done ? flits / (double)(t1 - t0 + 1) : 0.0);
}

int main(void)
{
    /* 模拟器的并行度上限 = 路由器数，线程再多只会付屏障开销 */
    if (omp_get_max_threads() > NR) omp_set_num_threads(NR);
    printf("4x4 mesh, %d packets x %d flits, buffer=%d flits, %d thread(s)\n",
           NPKT, NLEN, CAP, omp_get_max_threads());
    printf("%-18s | %7s | %9s | %7s | %8s | %9s\n",
           "flow control", "done", "avg lat", "max lat", "cycles", "flit/cyc");
    printf("-------------------+---------+-----------+---------+----------+----------\n");
    printf("[traffic: uniform random]\n");
    run_mode(MODE_SF, "store-and-forward", 0);
    run_mode(MODE_WH, "wormhole",          0);
    run_mode(MODE_VC, "wormhole + 2 VC",   0);
    printf("[traffic: 25%% hotspot at router 5]\n");
    run_mode(MODE_SF, "store-and-forward", 1);
    run_mode(MODE_WH, "wormhole",          1);
    run_mode(MODE_VC, "wormhole + 2 VC",   1);
    return 0;
}
```

【代码做什么？】 1. 建立 **4×4 的二维 mesh**：16 个路由器、每个 5 个端口（N/S/E/W + Local 注入与弹出）、每条物理通道 `NVC` 条虚通道、每条缓冲 `CAP` 个 flit 的位置。包长固定 4 个 flit（head + 2 body + tail，用 `tail` 标志标记尾 flit）。每个周期分成四个阶段，阶段之间是隐式的全局屏障： – **阶段 0 信用快照**：把每个缓冲的 `count` / `pkt` 拷进 `snap_c` / `snap_p`。仲裁阶段只读这份快照，因此"读邻居状态"与"邻居改自己状态"不会变成同周期数据竞争。 – **阶段 1 注入**：每个源路由器每周期往自己的 `Local` 输入端口塞 1 个 flit（受缓冲空间与"同一缓冲只容纳一个包"的限制——这就是**注入反压**）。 – **阶段 2 仲裁 + 过链路**：对每个输出端口，在 `NVC` 条虚通道上轮转着找**第一个可以走的包**：检查它的路由方向是否等于该输出端口、SF 模式下是否已收满整包（`draining` 标志允许把已开始的包发完）、**以及下游缓冲是否有信用**（用快照检查空间与"同

包")。三者都满足才真正搬走 flit；不满足就**整个包这周期不被选中**，输出端口让给别的包——这正是"**先查信用再仲裁**"的真实路由器做法，也是避免把 flit 卡在共享输出锁存里造成死锁的关键。 – **阶段 3 合并**：把本周期到达的 flit 并入输入缓冲；当缓冲由空变非空时，用**接收方自己的坐标**算出该包的输出端口（YX 路由：先 Y 后 X，与 KNL 一致）。 2. 三种模式： – `MODE_SF`：缓冲容量 = **整包**（`cap_of()` 返回 `NLEN`），**必须收满 `NLEN` 个 flit 才能开始转发**（`draining` 标志允许把已经开始的包发完）；对端缓冲必须能为整包腾出空间。 – `MODE_WH`：缓冲容量 = `CAP`（8 个 flit），**队头 flit 一到就可以转发**，但一个缓冲同一时刻只装一个包（这正是 wormhole 的 head-of-line blocking 来源）。 – `MODE_VC`：在 WH 之上把每个输入端口切成 **2 条虚通道**：被堵住的包占一条，可走的包用另一条继续前进。 3. 两种流量：**均匀随机**与 **25% 打向同一热点 router 5**（专门制造队头阻塞）。输出平均包延迟、最坏包延迟、总周期数与吞吐（flit/周期）。

【并行机制与性能解说】 – **并行如何在硬件/软件上发生**：这段代码同时演示两种"并行"。 – **被模拟的网络本身是并行的**：同一个周期里，**16 个路由器各自独立地**做路由查找与仲裁（`#pragma omp parallel for` 的最外层），路由器之间通过**链路（数组元素）** 通信。写者唯一性是刻意设计的——`nb_f[n][opp][tv]` 只可能由"输出端口正好连到该输入端口"的那**唯一一个**路由器写，因此每个阶段内所有写操作**无数据竞争**、不需要锁。实测在 **1/2/4/8/16 线程下结果逐位相同**（见下表），这就是"**片上网络靠消息传递而非共享变量通信**"的软件体现。 – **模拟器自身是并行的**：外层 `parallel for` 把 16 个路由器分给多个 OpenMP 线程（`schedule(static)`），每周期 4 次并行区域（快照 → 注入 → 仲裁 → 合并），即**每周期一个全局同步点**。 – **Work / Span / 并行度**： – **Work** = `Σ_(cycles)（路由器数 × 端口数 × 虚通道数）` ≈ `cycles × 16 × 5 × NVC` 次"信用检查 + 仲裁"尝试。以 wormhole+VC 的 75 个周期为例：`75 × 16 × 5 × 2 = 12,000` 次仲裁尝试，另有 `128 × 4 = 512` 次 flit 搬运。 – **Span（关键路径）** = `cycles × 4`（每周期内部 4 个严格串行的阶段，阶段之间必须同步——**这是同步并行模拟的典型结构**）。单个包穿越 `n` 跳的关键路径是 `n × t_r`（本模型 `t_r = 1` 周期/跳）。 – **并行度 = Work/Span = (16 × 5 × NVC) / 4**。WH 模式为 **20**，VC 模式为 **40**。也就是说，**这个模拟器只有 20~40 路并行**：`OMP_NUM_THREADS` 调到 64 毫无意义，屏障开销只会让结果变坏（代码因此把线程数自动夹到 ≤ 路由器数）。这与真实网络的规律完全一致——**网络的并行度约等于"能同时推进的独立链路数"**，与包数无关。 – **实测结果（本笔记用 gcc 12.2 / `-O3 -march=native -fopenmp` 跑出的输出，8 线程，与 1 线程逐位相同）**：

流量	流控	完成	平均延迟（周期）	最大延迟	总周期	吞吐（flit/周期）
均匀随机	store-and-forward	128/128	**22.93**	63	161	3.30
均匀随机	wormhole	128/128	**11.99**	40	94	5.63
均匀随机	wormhole + 2 VC	128/128	**9.80**	**35**	**75**	**7.11**
25% 热点	store-and-forward	128/128	**27.51**	136	223	2.52
25% 热点	wormhole	128/128	**19.58**	**128**	186	2.93
25% 热点	wormhole + 2 VC	128/128	21.20	145	**178**	**3.14**

- **怎么读这张表（与讲义的结论一一对应）**：
- **store-and-forward 的平均延迟约为 wormhole 的 2 倍**（22.93 vs 11.99），这正是 "每一跳都要把整包收完" 的代价——对应 slide 40 的 `3 × 4 = 12` 单位（store-and-forward）vs `3 + 3 = 6` 单位（cut-through/wormhole）。
- **均匀随机流量下，虚通道明显改善延迟**：平均延迟 11.99 → 9.80、最大延迟 40 → 35、总周期 94 → 75。队头阻塞确实被"另一条虚通道继续前进"缓解了——对应 slide 44/45 的两张图。
- **热点流量下 VC 的表现不同**：平均延迟**升高**（19.58 → 21.20），但**总周期下降**（186 → 178）。原因是虚通道让**更多包被接收入网**（有效负载上去了），排队的包更多 → 平均延迟上升；而**瓶颈已经转移到"热点 router 5 的弹出端口"**（每周期只能弹出 1 个 flit），虚通道**增加不了这个端口的带宽**。这是"**VC 减少队头阻塞，但不能创造带宽**"的实测证据，也对应图 2 的延迟–负载曲线：工作点被推到了更靠近饱和区的位置（用延迟换吞吐）。
- **瓶颈**：(a) **热点端口的串行化**（`o == 4` 每周期仅 1 个 flit）——全对全/热点流量的天花板；(b) **队头阻塞**——一个缓冲只装一个包，被堵的包会让路由空闲的包一起停下；(c) **注入反压**——源缓冲满后源节点只能干等；(d) **同步开销**——每周期 4 个并行区域的屏障，在 16 个路由器的小网络上，屏障本身就是可观的固定开销（这也是"模拟器并行度只有 20~40"的直接后果）。

3.3 示例 3：共享总线争用 vs 分片网络 / 伪共享 —— 用 OpenMP 量化"互连争用"

```
/* ============================================================================
 * bus_vs_shards.cpp —— 共享总线式争用 vs 分片(sharded)访问 vs 伪共享
 *   MODE 0 shared   : 所有线程对一个全局原子计数器做 RMW  -> 等价于共享总线
 *   MODE 1 unpacked : 每线程一个计数器，但都挤在同几条 cache line -> 伪共享
 *   MODE 2 padded   : 每线程一个 64B 对齐的计数器            -> "本地 slice" 访问
 *   MODE 3 remote   : 每线程更新"邻居的"计数器               -> 跨 slice 通信
 *
 * 编译 (release) :
 *   g++ -O3 -march=native -fopenmp -std=c++17 bus_vs_shards.cpp -o bus_vs_shards
 * 运行:
 *   OMP_NUM_THREADS=16 ./bus_vs_shards
 * ============================================================================*/
#include <omp.h>
#include <atomic>
#include <cstdio>

struct alignas(64) Padded {
    std::atomic<long> v;
    char pad[64 - sizeof(std::atomic<long>)];
};

static const char *MODE_NAME[4] = {
    "0 shared counter (bus-like)",
    "1 per-thread, unpadded (false sharing)",
    "2 per-thread, 64B padded (local slice)",
    "3 per-thread, remote slice (cross-slice)"
};

/* 跑一轮，返回耗时（秒）。内部先预热，再取 REPS 次里最快的一次。 */
static double bench(int mode, int P, long iters)
{
    const int REPS = 3;
    std::atomic<long> *plain = (mode <= 1) ? new std::atomic<long>[P] : nullptr;
    Padded *pad = (mode >= 2) ? new Padded[P] : nullptr;
    if (plain) for (int i = 0; i < P; i++) plain[i].store(0, std::memory_order_relaxed);
    if (pad)   for (int i = 0; i < P; i++) pad[i].v.store(0, std::memory_order_relaxed);

    double best = 1e30;
    for (int rep = 0; rep < REPS + 1; rep++) {      /* 第 0 次当预热，不计入 */
        double t0 = omp_get_wtime();
#pragma omp parallel num_threads(P)
        {
            int tid = omp_get_thread_num();
            switch (mode) {
            case 0:
                for (long i = 0; i < iters; i++) plain[0].fetch_add(1, std::memory_order_relaxed);
                break;
            case 1:
                for (long i = 0; i < iters; i++) plain[tid].fetch_add(1, std::memory_order_relaxed);
                break;
            case 2:
                for (long i = 0; i < iters; i++) pad[tid].v.fetch_add(1, std::memory_order_relaxed);
                break;
            default:
                for (long i = 0; i < iters; i++) pad[(tid + 1) % P].v.fetch_add(1, std::memory_order_relaxed);
                break;
            }
        }
        double dt = omp_get_wtime() - t0;
        if (rep > 0 && dt < best) best = dt;
    }
    delete[] plain;
    delete[] pad;
    return best;
}

int main(void)
{
    const long iters = 500000;
    int maxp = omp_get_max_threads();
    if (maxp > 16) maxp = 16;       /* 采样到 16 线程即可看出趋势 */
```

```
    printf("atomic fetch_add, %ld iterations per thread (best of 3), up to %d threads\n\n",
           iters, maxp);
    printf("%-38s | %4s | %10s | %9s | %10s\n",
           "mode", "P", "Mops/s", "speedup", "GB/s line traffic");
    printf("---------------------------------------+------+------------+-----------+----------\n");

    for (int mode = 0; mode < 4; mode++) {
        double base = 0.0;
        for (int P = 1; P <= maxp; P *= 2) {
            double dt = bench(mode, P, iters);
            double mops = (double)P * (double)iters / dt / 1e6;
            if (P == 1) base = mops;
            /* 每次争用的原子 RMW 至少牵动一条 64B cache line 的迁移 */
            double gbs = mops * 64.0 / 1000.0;
            printf("%-38s | %4d | %10.1f | %9.2fx | %10.2f\n",
                   MODE_NAME[mode], P, mops, mops / base, gbs);
        }
        printf("---------------------------------------+------+------------+-----------+----------\n");
    }
    printf("\n提示：模式 0 的 Mops/s 在 P>1 后基本不涨，甚至下降 —— 这是共享总线的串行化点；\n");
    printf("      模式 1 明显差于模式 2 —— 伪共享把无效的一致性流量灌进了互连网络。\n");
    return 0;
}
```

【代码做什么？】 1. 四种模式共用同一个 `fetch_add` 微基准，只是"摆放方式"不同： – **模式 0（shared）**：所有线程对**一个**全局原子计数器做读–改–写。任何时刻只有一个线程能拿到该 cache line 的独占权，硬件层面被强制串行化——这就是**共享总线 / 单一仲裁点**的软件复现。 – **模式 1（unpacked）**：每线程一个 `std::atomic<long>`，但**紧挨着放在同一批 cache line 里 → 伪共享（false sharing）**：每次 `fetch_add` 都会让整条 64 B 行在核之间来回迁移。 – **模式 2（padded）**：每线程一个 **64 B 对齐**（`alignas(64)` + padding）的计数器 → 各自独占一行，等价于**"每个核访问自己的本地 L3 slice"**。 – **模式 3（remote）**：每线程更新**邻居的**计数器 → 一个**置换（permutation）访问模式**：每条 cache line 依然只被一个核独占，只是"拥有者"和"使用者"不是同一个核。 2. 每个 (模式, 线程数) 组合先**预热一轮**，再取 3 次里**最快的一次**，算出 `Mops/s`、相对 `P=1` 的加速比，以及**互连流量下界** `Mops/s × 64 B`（每次争用的原子 RMW 至少要搬一条 cache line）。

【并行机制与性能解说】 – **并行如何在硬件上发生**：`fetch_add` 在 x86 上编译成 `lock xadd`。**独占一条 cache line 的所有权是所有原子 RMW 的共同前提**——这条行必须在核之间"搬家"，而搬家要占用**片上互连（ring/mesh）与一致性协议消息**（请求、失效、响应）。所以这个微基测量的是**互连与一致性协议的争用程度**，而不是 ALU 快慢。 – **Work / Span / 并行度**： – **模式 0**：**Work** = `P·iters` 次 RMW；**Span** = `P·iters` 次（全部串行地作用在**同一个**变量上，临界区长度 = 一次独占访问的时间）；**并行度 = 1**（无论多少线程）。这是**"并行度被一个共享资源封顶"**最纯粹的例子。 – **模式 2**：**Work** = `P·iters`；**Span** = `iters`（每个线程独立做完自己那份，互不相干）；**并行度 = P**。这才是"分片网络 + 本地 slice"应有的样子。 – **模式 1**：**Work** = `P·iters`，但每次操作都要把一条整行搬来搬去，**Span 里混入了 `0(P)` 次行迁移**，并行度被"每行能容纳几个原子量"卡死（16 核写 2 个原子量时，串行度极高）。 – **模式 3**：**Work** = `P·iters`；**Span** ≈ `iters`（每个计数器仍只被一个核独占）。所以它能像模式 2 一样扩展——**说明杀死性能的是"争用"，而不是"距离"本身**。 – **实测结果（本机 gcc 12.2 / `-O3 -march=native -fopenmp`，每线程 50 万次 RMW，best of 3）**。注意：这张表是**一次代表性运行**，绝对数值随机器（核数、NUMA、频率）与当前负载变化，但**趋势与量级差异非常稳定**：

模式	P=1	P=2	P=4	P=8	P=16	16 线程加速比
0 共享计数器（总线式）	490.1 Mops/s	168.3	161.3	163.5	**39.9**	**0.08×（负加速）**
1 每线程但未填充（伪共享）	456.9	206.3	170.3	345.8	**80.8**	**0.18×（负加速）**
2 每线程 64 B 对齐（本地 slice）	467.7	853.8	1676.0	3303.9	**6510.4**	**13.92×**
3 每线程访问邻居的片（置换）	486.7	851.2	1649.2	3290.3	**6590.1**	**13.54×**

- **怎么读这张表**：
- **模式 0 是"负加速"**：线程数从 1 加到 2，吞吐直接掉到 1/3（490 → 168 Mops/s），加到 16 线程只剩 **0.08×**。多出来的核全部在排队等同一条 cache line，**这正是共享总线的结局**，也是"全局计数器 / 全局任务队列"这类写法的下场。
- **模式 1 比模式 2 差约 80 倍**（80.8 vs 6510.4 Mops/s）：16 个线程挤在同几条 cache line 上，**每次写都要把整行迁移一次**，等价于把互连网络塞满了无效的一致性流量。这是最容易被误诊为"内存带宽不够"的性能杀手。
- **模式 2 近乎线性**（16 线程 13.92×）：分片 + 对齐之后，每个核只碰自己的行，**网络里几乎没有额外流量**。
- **模式 3 与模式 2 相当**，这是一个重要的反直觉结论：**置换访问不伤吞吐，共享同一行才伤**。跨 slice 的距离只有在"多个核争同一行"或"链路带宽被占满"时才变成瓶颈。

- **瓶颈**：(a) **串行化**（模式 0/1）：增加线程只会增加争用，吞吐到顶后**反而下降**；(b) **伪共享**（模式 1）：破坏力可达**两个数量级**；(c) **一致性流量的带宽**：把 `Mops/s × 64 B` 读成"互连上的流量下界"，模式 2 在 16 线程时是 416 GB/s（本地行，不经网络），而模式 0 的 2.56 GB/s 全部是**跨核行迁移的无效流量**——两者相差 160 倍。

4. 性能模型与复杂度分析

4.1 拓扑的定量指标（N = 节点数，k = √N，B = 单条链路带宽）

拓扑（N=64, k=8）	链路数	直径（hops）	平均距离（hops）	二分带宽（链路数）	成本 / 单位二分带宽
Ring (N=64)	64	32	16	**2**	64/2 = **32**（最差）
2D Mesh (8×8)	2N−2k = 112	2(k−1) = 14	2(k²−1)/(3k) = **5.25**	k = **8**	112/8 = 14
2D Torus (8×8)	2N = 128	k = 8	k/2 = 4	2k = **16**	128/16 = 8
Multi-stage log (Omega)	N·lg N = 384	lg N = 6	lg N = 6	N/2 = **32**	384/32 = 12
Fat Tree (64 叶)	O(N lg N) ≈ 384	2·lg N = 12	~12	**32**（可做 full bisection）	12
Hypercube (n=6)	N·n/2 = 192	**6**	n/2 = 3	N/2 = **32**	192/32 = **6**（最好）
Crossbar (N=64)	交叉点 N^2 = 4096	**1**	1	N/2 = **32**	4096/32 = 128（最贵）

读表要点：ring 是唯一"二分带宽不随 N 增长"的拓扑（永远只有 2 条链路跨切面），这是它只能用于几十个节点的根本原因；**mesh 用 112 条链路买到 8 条链路的二分带宽**（效率 14），**torus 用绕回链路把效率提到 8；hypercube 的成本效率最高（6）**，代价是每个节点的**度（radix）随 lg N 增长**，硬件上难以实现大端口数路由器——这就是为什么实际的芯片用 mesh/ring，而机群/超级计算机多用 **fat tree**（可用多级小交换机拼出 full bisection）。

4.2 数值算例 A：Intel ring 互连的理论峰值带宽（讲义 slide 25 的数字复算）

假设：数据环宽 32 B、时钟 3.4 GHz、共有 4 条环（request / snoop / ack / data）。

- 单条环单向理论带宽：`32 B × 3.4 GHz = 108.8 GB/s`
- 四条环合计：`4 × 32 B × 3.4 GHz = 435.2 GB/s` ≈ 讲义给出的 **"约 435 GB/s"** ✔
- **前提条件（讲义原文）**：这个峰值是"**当每个核都访问自己的本地 slice 时**"才成立的。
- 反例（本笔记补充的算例）：若 4 个核**全部**打向**同一个** 2 MB L3 slice，那么瓶颈变成**该 slice 到环的那一条链路**：
- 可用带宽 = `32 B × 3.4 GHz = 108.8 GB/s`
- 平摊到 4 个核：`27.2 GB/s / 核`，比"访问本地 slice"时低 **3.7 倍**（435.2/4 = 108.8 vs 27.2）
- 这解释了为什么"**把数据按核分片（sharding）**"在 ring 机器上如此重要——与示例 3 的模式 2 vs 模式 0 完全对应。
- **成本直观量**：讲义指出 **crossbar（CCX）占用的芯片面积与一个核相当**（Sun SPARC T2 / Oracle SPARC T5）。在 16 核的 T5 上，等于用"一个核的面积"买"任意两核 O(1) 通信"，这已是 $O(N^2)$ 网络在 N=16~32 时的现实上限。

4.3 数值算例 B：零负载延迟模型 `T0 = D · t_r + L / b`

假设（KNL 6×6 mesh，取自讲义 slide 27 的形态 + 本笔记补充的合理参数）：`k = 6`（36 个 tile）、网络时钟 `f = 1.5 GHz`、每跳路由延迟 `t_r = 2` 个网络时钟、链路宽度 `b = 32 B/cycle`、消息长度 `L = 64 B`。

- 平均跳数：`D_avg = 2(k²−1)/(3k) = 2×35/(3×6) = 3.89` 跳
- 每跳时间：`t_r / f = 2 / 1.5 GHz = 1.33 ns` → 路由部分 = `3.89 × 1.33 = 5.19 ns`
- 串行化时间：`L / (b·f) = 64 B / (32 B × 1.5 GHz) = 64 / 48 GB/s = 1.33 ns`
- **T0 ≈ 5.19 + 1.33 ≈ 6.5 ns**（片上），相比跨 socket 的 UPI/QPI（数十到上百 ns）与 InfiniBand（μs 级），**片上网络的空载延迟小 2~3 个数量级**。
- **但这不是性能保证**：按图 2 的延迟–负载曲线，当注入率接近饱和点时延迟会急剧上升。用一个粗模型说明饱和：
- 网络总注入能力 = `36 tile × 32 B × 1.5 GHz = 1728 GB/s`
- 二分带宽（6×6 mesh 的最小切面 = 6 条链路）= `6 × 48 GB/s = 288 GB/s`
- **两者之比 = 288/1728 = 1/6**：即"全局随机通信"只能拿到"本地通信"带宽的 **1/6**。任何**全对全**或**跨全局的数据交换**都必须按这个比例打折——这就是"局部性"在互连层面的量化含义。

- 与片外对比：4 通道 DDR4-3200 ≈ `4 × 25.6 = 102.4 GB/s`，片上二分带宽（288 GB/s）约为它的 **2.8 倍**——片上很好，但**绝不是无限**。

4.4 数值算例 C：α–β 模型与消息大小的"生死线"

假设（典型的 InfiniBand/以太网级别的网络）：`α = 1.5 μs`（零负载启动延迟），`β = 12 GB/s`（渐近带宽）。消息传输时间 `T(n) = α + n/β`：

消息大小 n	传输时间 `T = α + n/β`	有效带宽 `n/T`	相对 β 的比例	判断
8 B	1.5 μs + 0.7 ns ≈ **1.500 μs**	**5.3 MB/s**	0.044%	完全被延迟主导
1 KiB	1.5 + 0.085 = **1.585 μs**	0.661 GB/s	5.5%	依然延迟主导
18 KB（交叉点）	1.5 + 1.5 = 3.0 μs	6.0 GB/s	50%	交叉点 `n* = α·β`
1 MiB	1.5 + 87.4 = **88.9 μs**	**11.8 GB/s**	98%	带宽主导

- **交叉点公式**：`n* = α · β = 1.5 μs × 12 GB/s = 18 KB`。**消息小于 18 KB 时，启动延迟比传输时间还长**；小于它的消息**再怎么优化协议也没用**，唯一出路是**消息聚合（message aggregation）**：把 1000 条 8 B 的消息合成一条 8 KB 的消息，时间从 `1000 × 1.5 μs = 1.5 ms` 降到 `1.5 μs + 0.68 μs ≈ 2.2 μs`，**快了约 680 倍**。
- **Little's law（本笔记补充的解释）**：要在延迟 `L` 下维持带宽 `BW`，必须让 `BW × L` 字节"在路上"。以 `BW = 12 GB/s`、`L = 1.5 μs` 计，**每条流必须有 18 KB 在飞**。若消息只有 64 B，则每个 rank 需要 `18 KB / 64 B = 288` 条**同时在途**的消息才能填满管道——这正是"**要重叠（overlap）、要深流水、要用非阻塞通信**"的定量理由。

4.5 数值算例 D：halo 交换的算术强度与通信/计算平衡

假设：全局 `512³` 的三维 7 点 stencil（每点 13 flops），`512` 个 rank 排成 `8×8×8`，每个 rank 拥有 `64³` 的局部子域，halo 厚度 1 层（double，8 B），每 rank 峰值算力 50 GFLOPS（本笔记假设值），网络每 rank 注入带宽 12 GB/s，fat tree 二分带宽 2 TB/s。

- **每 rank 每步的通信量**：`6 面 × 64×64 点 × 8 B = 196,608 B ≈ 192 KiB`
- **每 rank 每步的计算量**：`64³ × 13 = 262,144 × 13 = 3.41 MFLOP`
- **算术强度** = `3.41e6 flops / 1.966e5 B =` **17.3 flops/byte**
- **计算时间** = `3.41 MFLOP / 50 GFLOPS = 68.2 μs`
- **通信时间**（按每 rank 注入带宽）= `192 KiB / 12 GB/s = 16.4 μs`
- **通信时间（按最坏情况：全部流量跨越二分切面，这是全对全/置换类通信的下界）** = `512 × 196,608 B / 2 TB/s = 100.66 MB / 2 TB/s = 50.3 μs`
- **结论**：`T_comm/T_comp = 50.3/68.2 = 0.74` → 若通信完全不重叠，效率上限 ≈ `1/(1+0.74) = 57%`。
- **两个改进方向（都有定量依据）**：
- **重叠（overlap）**：用 `MPI_Isend/Irecv` + `MPI_Waitall` 把 6 个面的发送藏进"内部区域"的计算里（见 §2.12 图 10B）。理想情况下把 50.3 μs 的通信压在 68.2 μs 的计算之下，**效率回到接近 100%**。
- **放大子域**：通信量 ∝ 表面积 ∝ `L²`，计算量 ∝ 体积 ∝ `L³`，所以**算术强度 ∝ L**。把子域从 `64³` 换到 `128³`（rank 数从 512 降到 64），算术强度翻倍到 **34.6 flops/byte**，通信占比直接减半。**这就是"最小化通信（minimize communication）"这条课程主线的可量化版本**。

4.6 Amdahl 定律在互连上的应用

设一次运行的 `20%` 时间花在"等待网络"上，若把网络二分带宽翻倍（其余不变），整体加速比：

```
Speedup = 1 / ((1 − 0.20) + 0.20/2) = 1 / (0.80 + 0.10) = 1 / 0.90 = 1.111   ->  11%
```

解读：当并行程序的瓶颈已经转移到网络之外（计算、DRAM 带宽、同步），**单纯升级互连只能拿到 11%**。反过来说，当网络占比达到 50% 时，翻倍带宽能带来 `1/(0.5+0.25) = 1.33×`。**先测量、再决定要不要动网络**——这与课程"性能优化（performance optimization）"部分的方法论一致。

---

5. 关键要点

1. **总线不 scale，网络是必然选择，而且"机柜级的问题"今天全在片内重演。** 共享总线只有三个优点——简单、便宜、snooping 一致性容易实现；但它同时带来争用、带宽与 N 无关、电气负载压低频率抬高功耗。从 4 核到 72 核（KNL 6×6 mesh），互连已经从配角变成整颗芯片性能与功耗（MIT RAW：约占 35% 功耗）的主角。

2. **拓扑的选择是"成本 / 延迟 / 二分带宽 / 布线复杂度"的四角权衡，没有全能赢家。** 用 diameter（最坏延迟）与 average distance（平均延迟） 描述延迟，用 **bisection bandwidth** 描述"全局通信的容量"，再叠加 **direct/indirect、blocking/non-blocking、成本 O()** 一起判断。**ring 的致命伤是二分带宽恒为常数**；mesh 好布线但边缘/中心不公平；torus 用绕回链路修复公平性却难布线；hypercube 成本效率最高但端口数随 lg N 增长；crossbar 延迟 O(1) 却要 O(N^2) 面积（T2/T5 上 CCX 面积约等于一个核）。并且要记住讲义的警告：**二分带宽不等于实际可达带宽**，它忽略了交换机与路由效率。
3. **网络的性能必须用"两个数字"来描述：零负载延迟与饱和吞吐。** 零负载延迟由**拓扑 + 路由 + 流控**共同决定，饱和吞吐由**流控**决定；一般规律是**延迟随负载上升**，接近饱和时会陡增（缓冲耗尽 + 队头阻塞 + 反压传播）。工程上要同时盯 **α（启动延迟）与 β（带宽）**：消息小于 $n* = \alpha \cdot \beta$ 时延迟主导，此时唯一的解药是**聚合消息**与**增加在途消息数（Little's law: BW × L 字节必须在路上）**。
4. **流控的粒度决定了延迟与缓冲面积的取舍：粗粒度缓冲换来简单，细粒度 flit 换来低延迟但引入队头阻塞。** store-and-forward 每跳都要收全包（延迟 ∝ 跳数 × 包长，需要整包缓冲）；cut-through 头到即转（高争用时退化为 store-and-forward）；wormhole 把缓冲/流控降到 **flit** 粒度，**长消息的延迟几乎与网络距离无关**，但引入 **head-of-line blocking**，于是需要**虚通道（VC）** ——它同时还能**打破死锁环（escape VC）** 与**提供 QoS 优先级**。
5. **对软件而言，能做的只有三件事：把消息做大做少、把通信与计算重叠、把访问局部化。** 消息做大做少 = 赢得 α；重叠 = 把通信从 Span（关键路径）里挪到 Work 的并行部分（非阻塞通信 + 分块计算）；局部化 = 让访问落在**本地 slice/本地 tile**，避免"全对全"去撞那 1/6 的二分带宽。一个被所有线程争用的原子计数器，就是在网络里**人为造了一条总线**——它的并行度是 1。

---

6. 常见陷阱与注意事项

- **把 bisection bandwidth 当作"可达带宽"。** 讲义明确警告：二分带宽**不考虑交换机与路由效率**。一个二分带宽 32 条链路的网络，在真正的置换（permutation）流量下可能只能拿到一半甚至更低；评估时要用**实测的置换/全对全微基准**（如示例 1 的 ring 测试），而不是拓扑图上的数字。
- **只测带宽、不测延迟（或反过来）。** 只报 β 会掩盖大规模下小消息的崩塌（8 B 消息有效带宽只有 5.3 MB/s，是 β 的 0.044%）；只报空载延迟会掩盖饱和后的陡增。**必须同时给出 (α, 饱和吞吐)**，并且在**接近饱和的工作点**上测量。
- **误以为 wormhole 网络永远不需要整包缓冲。** 讲义 slide 41 说得很清楚：**"需要交换机具备整包缓冲，和 store-and-forward 一样"**——因为一旦 head flit 被下游堵住，整个包最终会被吸进开关的缓冲里（cut-through 在高争用下退化为 store-and-forward）。设计缓冲面积时按最坏情况算，否则拥塞下会丢包/死锁。
- **忽略 head-of-line blocking 就做不出正确的延迟预测。** 一个热点链路可以把一个输入缓冲排满，让后面**路由完全空闲**的包也一起停摆（slide 44）。热点流量（如 allreduce 的根节点、热点计数器）下，实测延迟会比"平均跳数 × 每跳延迟"的估算高一个数量级。诊断信号：吞吐远低于二分带宽推算值，同时个别端口的缓冲占用长期饱和。
- **路由算法引入循环依赖 → 死锁。** 自适应路由很有吸引力（能绕开拥塞），但**通道依赖图一旦成环就可能死锁**。安全做法：使用维度顺序路由（如 KNL 的 **YX routing**，其依赖图无环）、或用**虚通道拆环**（请求/响应分 VC、**escape VC** 保留一条无死锁路径）。另外要区分**死锁（deadlock）** 与 **活锁（livelock，包一直在绕但到不了）**——讲义明确说 QoS、优先级、可靠性、死锁、活锁都属于"本课不深入、但值得继续学"的内容。
- **把"片上网络很快"当成"通信免费"。** 6×6 mesh 的二分带宽只有注入带宽的 **1/6**；跨 socket 再降 2~3 个数量级。任何"每个线程都去写全局共享结构"的写法（全局计数器、全局栈、全局任务队列）都会把并行度压到接近 1，和共享总线的结局完全一样。
- **忽略伪共享带来的额外互连流量。** 相邻的小变量被不同线程写时，**每次写都要让整条 64 B cache line 在核之间迁移**——这是往互连网络上灌**无效的一致性流量**。它对性能的破坏经常比"内存带宽不足"更严重，而且往往在 `P` 跨过某个阈值（超过一条行能放的变量数）时**突然出现**。修法：`alignas(64)` 填充、每线程独立累加最后再归并（示例 3 的模式 2）。

---

7. 思考题（带答案）

思考题 1：把一个 256 节点的系统分别做成 ring 与 16×16 的 2D mesh，请给出链路数、直径、平均跳数与二分带宽，并说明各自适合什么应用。

【答案】

指标	Ring (N=256)	2D Mesh (16×16)
链路数	`N = 256`	`2N − 2k = 512 − 32 = 480`
直径（最坏跳数）	`N/2 = 128`	`2(k−1) = 30`

指标	Ring (N=256)	2D Mesh (16×16)
平均跳数	`N/4 = 64`	`2(k²−1)/(3k) = 2×255/48 = 10.6`
二分带宽	**2 条链路**	`k = 16` 条链路

结论与适用性：mesh 的平均跳数只有 ring 的 **1/6**（10.6 vs 64），二分带宽是 ring 的 **8 倍**，代价是链路数多 **1.9 倍**。因此：**ring 只适合"流量高度局部"或节点数很少的场合**——例如 Intel Sandy Bridge 那样 4~16 个核 + 几个 slice，且**每个核主要访问本地 slice**（此时 435 GB/s 的峰值才成立）；一旦出现"所有核抢同一个 slice"或需要全局通信，ring 的常数二分带宽会立刻成为死结。**mesh 适合网格型/局部性好的计算**（stencil、邻域通信、图像分块），因为跳数只随 √N 增长、布线等长、且有多条可选路径；它的缺点是**边缘节点与中心节点的延迟不公平**（这正是 torus 用绕回链路要解决的问题），以及最坏情况（对角通信）仍有 30 跳。

思考题 2：讲义 slide 43 问"对长消息而言，wormhole 的延迟几乎与网络距离无关，为什么？"请给出定量解释，并说明这个结论在什么情况下失效。

【答案】 因为 wormhole 是**完全流水化（completely pipelined）** 的：head flit 在前方建立路径，body flit 与 tail flit 紧跟着**一个 flit 一个 flit 地**推进，包的各个部分同时**分布在不同链路上**。设消息长度 `L`、链路带宽 `b`、跳数 `D`、每跳路由延迟 `t_r`，则最后一个 flit 的到达时间为

```
T ≈ D · t_r + L / b
```

其中串行化项 `L/b` **与跳数无关**，只有 `D · t_r` 部分与距离成正比。对比 store-and-forward 的 `T_SF ≈ D · (L/b)`（延迟与**距离和包长都成正比**），当 `L/b ≫ D·t_r`（长消息）时，wormhole 的 `D·t_r` 相对 `L/b` 可以忽略，于是 **T 几乎与 D 无关**。讲义给的例子正是这个比例：3 跳、包长 4 单位、每跳路由 1 单位 → store-and-forward 为 `3 × 4 = 12` 单位，cut-through/wormhole 为 `3 + 3 = 6` 单位，**快 2 倍且随距离增长的部分从 L/b 项转移到了 t_r 项**。

失效条件：(1) **短消息**——当 `L/b` 与 `D·t_r` 同量级时，"与距离无关"就不成立，延迟重新变成 `≈ D·t_r`；(2) **高争用**——一旦 head flit 在下游被阻塞，整包停在原地（head-of-line blocking），实际延迟退化为 store-and-forward 量级，并且缓冲需求也退化到整包；(3) **死锁/反压**——环形依赖或下游缓冲长期无信用时，流水线被完全打断。

思考题 3：一个 16 节点系统，每个节点想持续注入 4 GB/s 的随机流量，网络是一条 16 节点的 ring，每条链路带宽 8 GB/s。问：通信能否满足？如果换成 4×4 的 mesh（链路带宽同为 8 GB/s）呢？请给出计算过程与设计准则。

【答案】

Ring 的情形：– 总注入需求 = `16 × 4 GB/s = 64 GB/s`。 – 随机（均匀）通信时，**约一半流量要跨越任意一个二分切面**，因此跨切面需求 ≈ `64/2 = 32 GB/s`。 – ring 的二分带宽 = **2 条链路 × 8 GB/s = 16 GB/s**（与 N 无关）。 – `32 GB/s > 16 GB/s` → **网络饱和**。饱和后二分切面只能提供 16 GB/s，均分给 16 个节点，**每个节点实际只能拿到 ≈ 1 GB/s**（只有期望的 1/4），并且按图 2 的曲线，延迟会急剧上升，应用性能远低于"1/4 带宽"的线性估计（还要叠加队头阻塞与缓冲耗尽）。 – 结论：**ring 在这个负载下不合格。**

换成 4×4 mesh 的情形： – 二分带宽 = `k 条链路 × B = 4 × 8 GB/s = 32 GB/s`。 – 跨切面需求 = 32 GB/s，**恰好等于**二分带宽 → 理论上"刚刚够"，但**没有任何余量**：一旦流量分布有偏斜（热点）、或者路由效率损失（讲义警告的"交换机与路由效率"）、或者出现突发，就会立刻进入饱和区，延迟上升、吞吐下降。工程上要求 `二分带宽 ≥ 1.3 ~ 2 × 跨切面需求`，所以这个配置**偏紧、需要加余量**（例如提高链路带宽到 16 GB/s，或改成 torus 得到 8 条切面链路 = 64 GB/s）。

设计准则（可推广）：

```
总注入带宽          = N × 每节点注入率
跨切面需求(均匀)   ≈ 总注入带宽 / 2          (全对全/置换类流量取 1.0)
设计要求            = 二分带宽 ≥ 1.3 ~ 2 × 跨切面需求
```

推论：任何"每个节点都要和所有其他节点通信"的应用（allreduce、全对全、全局同步/全局原子操作），其可扩展性上限**直接由二分带宽除以 N 决定**——这就是为什么大规模并行机普遍采用**可做到 full bisection 的 fat tree**，而不是 ring 或 mesh。

Lecture 11: Snooping-Based Cache Coherence

1. 章节标题与概述

Lecture 11: Snooping-Based Cache Coherence（基于监听的缓存一致性）

- **本讲核心问题**：多核处理器为了性能把内存内容**复制（replicate）** 到每个核私有的 cache 里，于是"读地址 X 应当返回任意处理器最后一次写入 X 的值"这个**共享地址空间（shared address space）** 的直觉语义被打破了——同一个地址在两个核的 cache 里可以同时存在两个不同的值。本讲回答三个问题：(1) 缓存为什么会造成并行系统里访存的困难？(2) 硬件如何提供**协调（coordination）** 来保住软件对内存的期望——即 coherence（一致性）的精确定义与 **snooping（监听式）** 硬件协议（write-through invalidation、MSI、MESI、MESIF、MOESI）？(3) 这些硬件抽象何时、以何种形式**暴露给软件**（false sharing、cache line 大小、GPU 的"不一致"设计）？
- **涉及的主要硬件/软件机制**：
- **硬件侧**：cache line（现代 Intel 为 64 B）里的 `tag / data / line state / dirty bit`；`write-back` vs `write-through`、`write-allocate` vs `write-no-allocate`；**互连（interconnect）** 上的广播一致性事务 `BusRd` / `BusRdX` / `flush`（CS149 记作 `BusWB`）；每个 cache 的 **controller（控制器）** 既要响应本地 CPU 的 load/store，又要"监听"来自互连的一致性广播；**失效型协议（invalidation-based）** 的状态机 I/V、MSI、MESI（Illinois Protocol）、MESIF（Intel）、MOESI（AMD Opteron）；**更新型协议（update-based）** 的 Dragon 协议（`BusUpd`）；多级层次下的 **inclusion（包含性）** 与 `in L1` bit、`modified-but-stale` bit；Intel Haswell/Skylake 的 L1–L2–L3 + ring interconnect（L3 亦可充当目录与串行化点）。
- **软件侧**：程序员不能"编程"一致性协议，但可以通过**数据布局与访问模式**决定它是否成为瓶颈：每个线程私有变量的**填充（padding）** 到 cache line 边界以消除 false sharing；散射/直方图类不规则写的**私有化（privatization）+ 归并（reduction/merge）**；`volatile`、原子操作、CUDA 的 `atomicAdd` / `ld.cg` 等**绕过或依赖** cache 的语义选择。
- **在并行计算知识体系中的角色**：本讲把前两讲的"缓存层次（cache hierarchy）"与"互连网络（interconnection network）"合起来，第一次让**多个核在物理上真正共享一份内存**；它是下一讲 **Directory-Based Cache Coherence（目录式缓存一致性）** 的直接动机——snooping 依靠广播，可扩展性受限于"能否把一致性消息广播给所有 cache"，目录方案正是为了消除广播。它同时是后续 **内存一致性模型（memory consistency models）** 的前置：本讲的 coherence 只管**单个地址**上的读写顺序，而 consistency 才管**不同地址之间**的顺序（两者的区别是考试与项目的常见混淆点）。对项目而言，本讲是"为什么加 padding、为什么把共享累加器改成每线程私有、为什么原子操作在热点上会崩"这些优化手段的理论根据。
- **配套材料**：
- `lectures/10_cachecoherence.pdf`（抽取文本 `extracted/10_cachecoherence.txt`，共 **60 页**）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 公开下载。讲义首页写的是 **"Lecture 10: Snooping-Based Cache Coherence"** 与 **"CMU 15-418/15-618, Fall 2024"**：这是**讲义沿用历史学期版本**的正常现象（讲次编号与学期字样随年度重排），不是错误。按 Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html），本讲排在 **Sep 18**，为第 **11** 讲，标题正是 **Snooping-Based Cache Coherence**；随后 Sep 21 为 Directory-Based Cache Coherence，Sep 23 为 Snooping-Based Multiprocessor Design。本笔记的全部术语、状态机、示例时序与实测数字均以这份 60 页讲义为准。
- `cs149_supp/cachecoherence.txt`（Stanford CS149 Fall 2025, Lecture 14: Cache Coherence，共 45 页，公开）：作为**补充视角**使用，其中 **SWMR（Single-Writer, Multiple-Read）不变量 + Data-Value 不变量**、**AMAT 公式与 Xeon 5500 访存延迟表**、MSI 事务表是 CMU 讲义未展开而对理解本讲极有帮助的部分；引用时均已标注来源为 CS149 补充材料。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 部分讲座在 Fall 2026 **尚未发布**公开讲义（Performance Analysis / Profiling、Transactional Memory、AI in System Design 等）；其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。
- 本笔记中额外补充的背景（如把讲义数字代入 AMAT、一致性流量下界推导、work-span 分析）均已显式标注为"本笔记推导/本笔记补充"，与讲义原文区分。

---

2. 核心概念与硬件/软件架构图解

2.1 引子：一次 `int x = 1;` 在硬件里发生了什么（讲义 slide 3–4）

- **定义与目的**：一致性协议的讨论必须建立在 cache 的物理组织上。一条 cache line（现代 Intel 为 **64 B**）在硬件里长这样：

```
                    一条 cache line (64 B) 的元数据与数据
+-----------+----------------+------------+------------------------------+
|   Tag     |   Line state   | Dirty bit  |   Data (64 bytes)            |
| (地址高位) | (一致性协议用)  | (是否脏)    |  byte 0 ... byte 63          |
+-----------+----------------+------------+------------------------------+
- Tag:          用于判断命中/缺失
- Line state:   一致性协议用的状态位 (I / S / E / M ...), 本讲的主角
- Dirty bit:    该行是否比内存新 (write-back cache 才有意义)
```

执行 `int x = 1;`（假设 `x` 在内存地址 `0x12345604`，未被分配在寄存器里）时，一个 **write-allocate + write-back** 的 cache 在**写缺失（write miss）** 上做五件事（讲义 slide 4）：(1) 处理器写一个不在 cache 里的地址；(2) cache 选一个位置安置该行，若该位置上有**脏行（dirty line）**，先把脏行写回内存（替换/write-back）；(3) 从内存把整行取进来（**allocate line**）；(4) 只更新这一行里的 32 bit；(5) 把该行标记为 **dirty**。

- **直观解释（"它是什么？"）**：把内存想成图书馆的藏书，cache 是你桌上的**复印件**。`write-back` 就像"我在复印件上改，暂时不告诉图书馆"；`write-through` 则是"每改一个字都跑一趟图书馆"。**write-allocate** 是"要在这页上改字，得先把整页复印过来"；**write-no-allocate** 是"只在图书馆原书上改，不复印"（讲义 slide 21 的 write-through 状态机里就假设了 write-no-allocate 以简化讨论）。
- **性能特征**：write-through 的代价是**每次写都要占用内存/互连带宽**（讲义 slide 24：every write operation goes out to memory → very high bandwidth requirements）；write-back 能**吸收绝大多数写流量**（连续的写命中都在 cache 内完成），但代价是"数据的权威副本可能在某个 cache 里而不在内存里"，这正是需要一致性协议的原因。

2.2 一致性问题：四核共享内存里同一个 X 有两个值（讲义 slide 5–9）

- **定义与目的**：四个处理器、四条私有 cache、一条互连、一份内存——这是讲义 slide 5 的基线系统。对内存的合理期望是：**读地址 X 应当返回任意处理器最后一次写入 X 的值**。而 cache 的复制行为会破坏这个期望：下表复现讲义 slide 6 的情景（初始 `mem[X] = 0`，write-back cache）：

动作	`mem[X]`	`P1$`（P1 的 cache）	`P2$`（P2 的 cache）	发生了什么
初始	0	—	—	—
P1 load X	0	0	—	P1 miss，从内存取到 0
P1 store X ← 1	0	1（dirty）	—	写只落在 P1 的 cache，内存仍是 0
P2 load X	0	1	0	P2 miss，从内存取到 **0**（不是 1！）
P1 load Y（使 X 被替换出）	1	X 被逐出	0	脏行被写回，内存终于变成 1
P1 store X ← 2	1	2（dirty）	0	内存 1，P2 手里的 0 更旧了
P2 load X	1	2	0（hit）	P2 **命中陈旧值 0**，语义被彻底破坏

- **直观解释（"它是什么？"）**：会议室里有 4 个人，每人手里都有一份会议记录的**复印件**。P1 在自己的复印件上把第 3 行改成"1"，但别人手里的复印件不会自动跟着变；更糟的是**记录员（内存）手里那份也可能是旧的**。于是"读到的应该是最新值"这个直觉，在"复印件 + 延迟同步"的世界里根本不成立。
- **这不是互斥问题（讲义 slide 7，重点）**：能不能靠加锁解决？**不能**。原因有三层：(1) 这个问题**不是**由"两个线程同时访问导致数据竞争"产生的，而完全是硬件为了性能**复制数据**这个实现细节造成的；(2) 本例中 P1 与 P2 的访问在时间上并没有重叠（没有竞争），加锁不会改变"P2 读到旧副本"这个事实；(3) 如果一致性只保证"有锁保护的临界区内部"正确，那么任何在临界区之外共享的**数据本身**（例如一个 flag、一个 work queue 的指针）都会出错。因此一致性必须由**硬件在每一次 load/store 上隐式保证**。
- **内存一致性问题的正式表述（讲义 slide 8）**：coherence 问题之所以存在，是因为**单个共享地址空间这个抽象，并不是由单个存储单元实现的**——存储被分散在 main memory 与各处理器本地 cache 中。
- **现实中的 cache 层次（讲义 slide 9，Intel Haswell 2013）**：

层次	容量/相联度	带宽	命中延迟	未命中并行度	备注
L1（私有一核一份）	32 KB，8-way，write-back	2 × 16 B load + 1 × 16 B store / clock	4–6 cycles	最多 **10** 个未完成缺失	直接服务 CPU 的 load/store
L2（私有一核一份）	256 KB，8-way，write-back	32 B / clock	12 cycles	最多 **16** 个未完成缺失	

层次	容量/相联度	带宽	命中延迟	未命中并行度	备注
L3 (全芯片共享)	8 MB，**inclusive**，16-way	32 B / clock **每个 bank**	26–31 cycles	—	每核一个 bank，经 **ring interconnect** 相连
cache line 大小	**64 B**				一致性的粒度就是它

- **性能特征（关键操作与量化后果）**：把 L1 的"最多 10 个未完成缺失"代入 Little's Law，可以估出**单核能压给内存子系统的带宽上限**：`10 × 64 B / (4–6 cycle miss 到 L2 的往返 ≈ 12 cycle) ≈ 640 B / 12 cycle ≈ 53 B/cycle`，在 3 GHz 上约 **160 GB/s** 的 L2 侧需求；若缺失一路穿透到 DRAM（~120 cycles），同样的 640 B 在途量只有 `640/120 × 3 GHz ≈ 16 GB/s`。这说明：**一致性协议增加的每次 miss 延迟（L3 hit 但数据在别的核的 cache 里，见 CS149 数据 65–75 cycles，而本地 DRAM 才 ~120 cycles）会直接乘以"在途请求数受限"这件事**，把可扩展的有效带宽按比例拉低。

2.3 "last" 的歧义与 coherence 的精确定义（讲义 slide 10–15）

- **定义与目的**：讲义先指出，在**单处理器**上提供"读到最新值"很轻松，因为写者通常只有一个：CPU 本身（例外是 **DMA** 的设备 I/O——所以单 CPU 系统里也有一致性问题，常用手段是：驱动用 **uncached store**、OS 把共享缓冲区所在页标记为 **not-cachable**、或 I/O 完成时**显式 flush 页**；由于 DMA 相比 CPU 的 load/store 极少发生，这些很重的软件方案可以接受）。到了多核，"last"这个词本身就有歧义：**两个处理器同时写怎么办？P1 的写紧接着 P2 的读、以致于"写发生过"这件事在物理上不可能及时传到 P2，又怎么办？** 在顺序程序里，"last"由**程序顺序（program order）** 决定，而不是由时间决定。
- **一致性的形式化定义（讲义 slide 13）**：一个存储系统是 coherent 的，当且仅当：并行程序的执行结果满足——**对每一个内存地址，存在一个所有处理器对该地址操作（按所有处理器执行的）的假想串行顺序（hypothetical serial order），它与执行结果一致**，并且：
- **任何一个处理器发出的内存操作，按该处理器发出的顺序出现**（program order）；
- **读返回的值，是由该串行顺序中"最后一次写"写入的值。**
- **同一件事的另一种说法（讲义 slide 14，考试最常考的三条）**：
- P 对 X 的读，若在 P 自己对 X 的写之后（中间无其他处理器写 X），必须返回 P 写的值 → **遵守 program order**（单处理器的期望）。
- P1 对 X 的读，若在 P2 对 X 的写之后、且两者"时间上充分分离"（中间无其他写 X），必须返回 P2 写的值 → **write propagation（写传播）**。注意：定义**并没有规定传播得多快**，只要求最终传到。
- **对同一地址的写被串行化（write serialization）**：任意两个处理器对 X 的两次写，被所有处理器**观察到的顺序相同**。
- **write serialization 的反例（讲义 slide 15）**：P1 先写 X←a，随后 P2 写 X←b。若 P3 观察到 `a` 然后 `b`，而 P4 观察到 `b` 然后 `a`，则**不存在**一条与两边观察都相符的全局时间线——违反条件 3。这正是"必须在互连上给写定序"的直接动机。
- **直观解释（"它是什么？"）**：coherence 就像**微信群里的消息顺序**。它不要求每个人**同时**看到消息（write propagation 不规定时限），但要求：(a) 你自己发的消息，你自己看到的顺序必须和发送顺序一致；(b) 对**同一条消息**，所有人群里看到的先后关系必须一致（不能有人说"先看到 b 再看到 a"，有人说"先 a 后 b"）。这就是"每个地址存在一个所有核都认可的串行顺序"。
- **等价的不变量表述（CS149 补充材料 slide 17：SWMR + Data-Value）**：对任意地址 x、任意时间段（epoch）：

```
地址 x 的"纪元 (epoch)"时间线:
|<-- Read-Write: P0 -->|<-- Read-Only: P0,P1,P2 -->|<-- Read-Write: P1 -->|<-- Read-Only: P0,P1 -->|
       (只有 P0 可写)         (多个核, 只读)             (只有 P1 可写)          (多个核, 只读)

不变量 1  SWMR (Single-Writer, Multiple-Read):
    任一时刻, 要么是"读写纪元"(只有一个核可以写, 它也可以读),
    要么是"只读纪元"(任意多个核只读)。
    绝不允许: 两个写者; 或者有写者的同时还有别处的读者。
不变量 2  Data-Value (write serialization):
    一个只读纪元开始时的值 = 上一个读写纪元结束时写下的值。
```

这两条不变量与讲义 slide 14 的三条条件是**同一枚硬币的两面**：MSI/MESI 中"M 态唯一"就是 SWMR，"flush/写回后再降级"就是 Data-Value。

2.4 实现路线总览：软件、共享 cache、snooping、directory（讲义 slide 16–19）

- **定义与目的**：讲义把实现手段分成四类，本讲只讲第三类，第四类是下一讲：

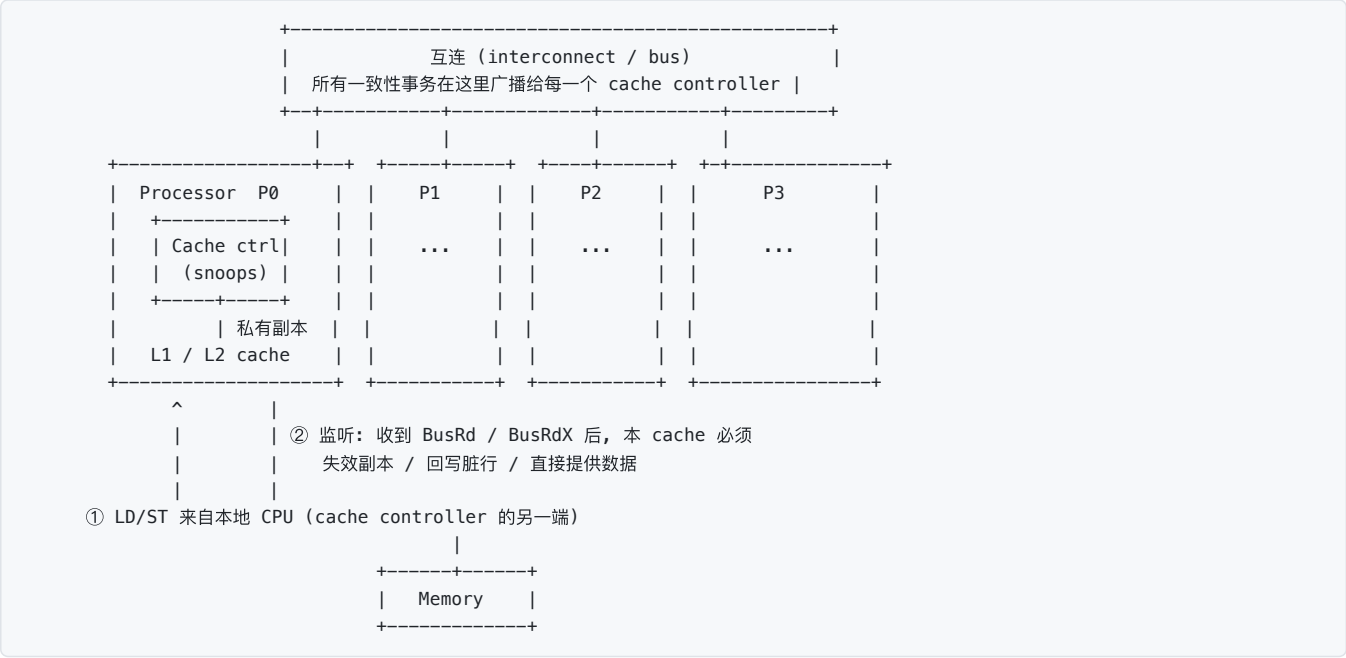
方案	粒度	机制	优点	致命弱点
软件方案	VM page（粗）	OS 用**缺页异常（page fault）** 传播写；可在工作站集群上实现共享内存	不需要改硬件	一页 = 4 KB，**false sharing 极其严重**；缺页开销是微秒级

方案	粒度	机制	优点	致命弱点
共享 cache	一条 cache	所有核共用一个 cache，**根本不存在复制**	消除一致性问题本身；天然支持细粒度共享（重叠工作集）、一个核的 load/store 还能替别的核**预取**	违背 cache"私有、近、快"的初衷：**争用（contention）与冲突缺失（conflict miss）**随核数上升；容量也做不大
Snooping（本讲）	cache line（细）	一致性活动**广播**给所有 cache controller，大家各自"监听"并反应	实现简单、无需额外存储（没有目录）	**可扩展性受限于"能否广播给所有 cache"**（讲义 slide 52 的结论）
Directory（下一讲）	cache line（细）	用一个**目录**记录每一行在哪些 cache 里，改成**点对点（point-to-point）**消息，"按需通知"	消息数 O(1) 而不是 O(N)；可用普通网络而非总线	需要额外存储与间接访问；目录本身可能成为瓶颈

- **直观解释（"它是什么？"）**：
- **共享 cache 像黑板**：只有一块，谁写谁都看得见——一致性免费，但所有人都得挤到黑板前面。
- **Snooping 像村里的广播喇叭**：任何人要改公共信息就喊一嗓子，全村都听见，各自把自己的小本子改掉。人少时最省事，人多时全村天天被噪音淹没。
- **Directory 像通讯录 + 电话**：登记簿上写着"这条信息谁能看到"，改的时候只打电话给相关的人，不打扰全村。

2.5 Snooping 的硬件结构：cache controller "两头受气"（讲义 slide 18–19）

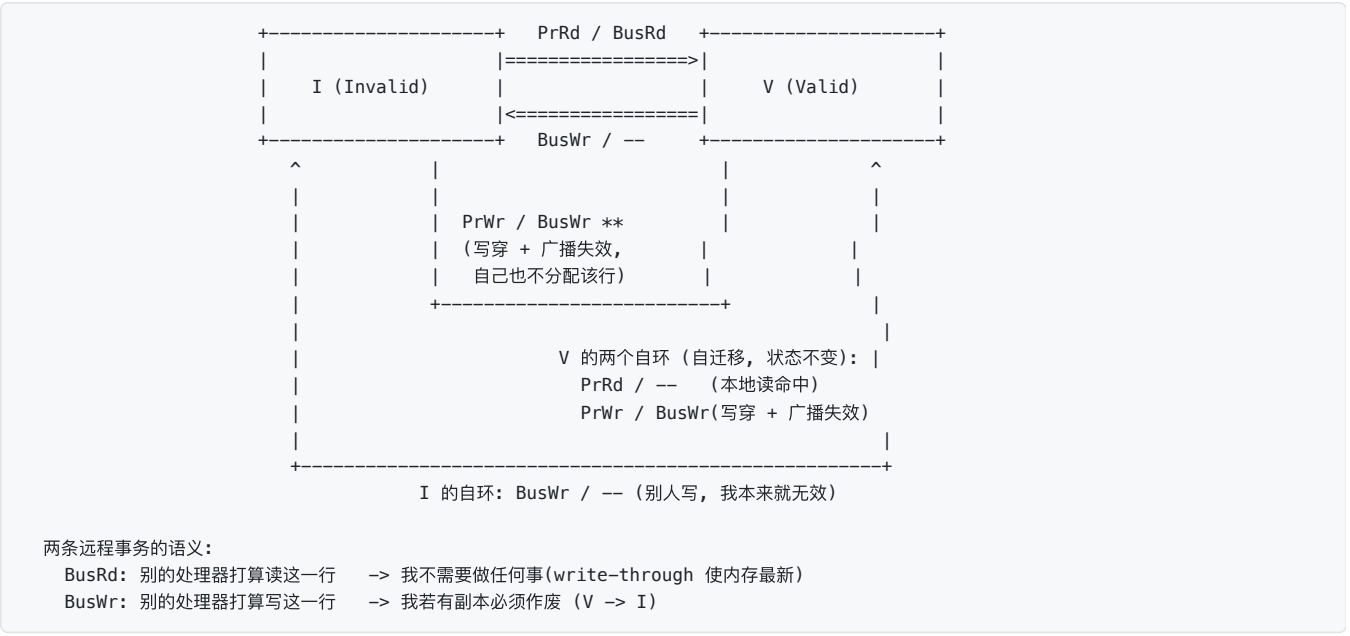
- **定义与目的**：snooping 的核心思想是——**凡是可能影响一致性的 cache 操作，cache controller 就把通知广播给系统里所有处理器（更准确地说：所有 cache controller）**；各个 cache controller **监听（snoop）** 内存操作，并据此维护一致性。这带来一个关键的架构变化：



- **必须理解的一句话（讲义 slide 20）**：上面这套逻辑由**每个**处理器的 cache controller **独立执行**，输入只有两个来源——**本地 CPU 的 load/store**，和**互连上收到的一致性消息**。只要所有 cache controller 都按同一套协议办事，一致性就自动成立：**cache 之间是"合作（cooperate）"关系**，没有任何全局控制器。
- **直观解释（"它是什么？"）**：cache controller 像一个**双语接线员**：一边听自家老板（本地 CPU）要数据，一边听广播里别人在喊什么，然后决定"把副本撕掉"还是"把我手里最新的版本交出去"。它的工作从"只服务一个 CPU"变成"服务 CPU + 服务整个系统"，这是 snooping 硬件复杂度的全部来源。

2.6 最简实现：write-through + 失效广播 + I/V 状态机（讲义 slide 20–23）

- **定义与目的**：讲义用一个刻意简化的协议建立直觉：假设 (1) cache 是 **write-through** 的；(2) 一致性粒度是 cache line。协议只有一条规则：**写的时候，cache controller 广播一条失效（invalidation）消息**；于是其他核的副本被作废，它们下次读就会 **miss**，而由于 write-through，内存里已经是最新值。
- **状态机图解（讲义 slide 21–22）**。记法：`A / B` 表示"若 cache controller 观察到事件 A，则执行动作 B"；本地 CPU 触发的事件是 `PrRd` / `PrWr`，来自远程 cache 的总线事务是 `BusRd` / `BusWr`（`BusWr` 也称 `BusRdX`）。`**` 表示这里假设 write-no-allocate：



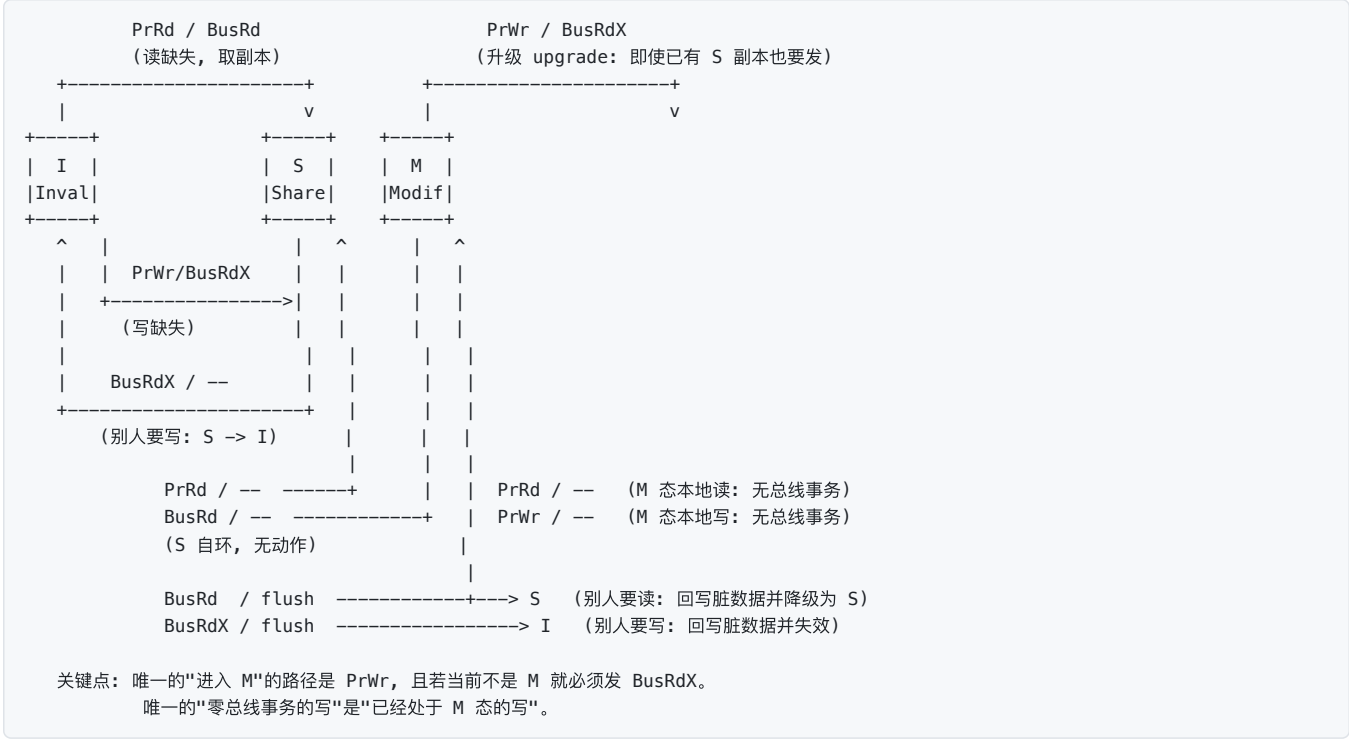
对应的状态迁移表：

当前状态	事件	动作	次态	互连事务
I	PrRd	从内存取整行	V	BusRd
I	PrWr	直接写内存并广播失效（write-no-allocate，不在本地分配）	I	BusWr
V	PrRd	本地命中	V	—
V	PrWr	写本地 + 写内存 + 广播失效	V	BusWr
V	BusWr	**作废本地副本**	I	—
V	BusRd	无（别人读不影响我）	V	—
I	BusRd / BusWr	无	I	—

- **互连必须提供的两条保证（讲义 slide 23）**：(1) **所有写事务对所有 cache controller 可见**；(2) **所有写事务以相同的顺序被所有 cache controller 看到**。若缺少 (2)，就会构造出 2.3 节 slide 15 的"P3 看到 a→b，P4 看到 b→a"的反例。讲义同时列出三条简化假设：互连与内存事务**原子（atomic）**；处理器**等前一条内存操作完成**再发下一条；失效在收到广播时**立即**生效。真实机器上这三条都要靠更多机制（MSHR、非阻塞 cache、乱序核的 memory ordering）来近似，这是"下一个讲（Snooping-Based Multiprocessor Design）"的主题。
- **性能特征**：write-through 协议的优点是**简单到可以手推**；代价是**每次写都占用内存带宽**（讲义 slide 24：very high bandwidth requirements）。举例：12 核各以 3 GHz 每 5 个 cycle 做一次写（每 cycle 0.6 次写），则写流量 = `12 × 0.6 × 3e9 × 8 B ≈ 173 GB/s`（若每写只传 8 B 数据）或按整行粒度更糟——远超单条 DRAM 通道的能力（几十 GB/s 量级）。这就是必须换 write-back 的量化理由。

2.7 write-back + 独占所有权：MSI 协议（讲义 slide 25–32）

- **定义与目的**：换成 write-back 后，脏行（dirty）意味着**独占所有权**（exclusive ownership）：(a) 该 cache 是**唯一**持有有效副本的（所以可以安全地写）；(b) 它是该行的 **owner**——别的核来取这行时，必须由它提供数据，否则从内存拿到的会是陈旧数据（讲义 slide 26）。协议要做两件事：**保证写者拿到独占访问**，以及**在 miss 时定位该行最新的副本**。
- **MSI 的三态与两事件源（讲义 slide 28）**：
- 状态：**I**（Invalid，含义与单处理器 cache 的 invalid 相同）、**S**（Shared，行在一个或多个 cache 中有效）、**M**（Modified，行**恰好**在一个 cache 中有效，又称 dirty / exclusive）。
- 本地 CPU：`PrRd`、`PrWr`。
- 互连事务：`BusRd`（取副本、无修改意图）、`BusRdX`（取副本并要修改 = read-exclusive）、`flush`（把脏行写出；CS149 记作 `BusWB`）。
- **MSI 状态机图解（讲义 slide 29）**：



- **MSI 状态迁移表**：

当前状态	事件	动作	次态	互连事务
I	PrRd	发 `BusRd`，取回整行（数据可能来自内存，也可能来自处于 M 态的 cache）	S	BusRd
I	PrWr	发 `BusRdX`，取回整行并取得独占权	M	BusRdX
S	PrRd	无	S	—
S	PrWr	发 `BusRdX`（**upgrade**，即使本地已有有效副本也必须发）	M	BusRdX
S	BusRd	无（可被要求提供数据）	S	—
S	BusRdX	**作废本地副本**	I	—
M	PrRd	无	M	—
M	PrWr	无（**M 态吸收写**，这是 write-back 的关键收益）	M	—
M	BusRd	**flush** 脏行（写回内存或直接 cache-to-cache 提供），降级	S	flush
M	BusRdX	**flush** 脏行并作废	I	flush

- **`PrWr` 命中 S 态为什么还要发 `BusRdX`（讲义 slide 32 的原问）**：因为"有效（valid）"不等于"独占（exclusive）"。别的 cache 里还有副本，若不通知它们就本地改写，就会出现两个 cache 持有同一行的不同值（违反 SWMR），而且**没有任何机制**能知道谁该为未来的读提供正确数据。所以**进入 M 态必须经过互连**；`BusRdX` 的作用就是告诉其他 cache"我要写了，你们不能再读了"。
- **MSI 逐条执行示例（讲义 slide 30–31 的 12 步序列）**：X、Y 初值均为 0。下表是本笔记按 MSI 规则逐步推演的结果（列格式 `状态/值`，`I` 表示无效）：

#	动作	P0:X	P0:Y	P1:X	P1:Y	互连事务
0	初始	I	I	I	I	—
1	P0: LD X	**S/0**	I	I	I	BusRd
2	P1: LD X	S/0	I	**S/0**	I	BusRd
3	P0: ST X←1	**M/1**	I	**I**	I	BusRdX（P1 作废）
4	P0: ST X←2	M/2	I	I	I	—（M 态本地写）
5	P1: ST X←3	**I**	I	**M/3**	I	BusRdX + flush（P0 回写）

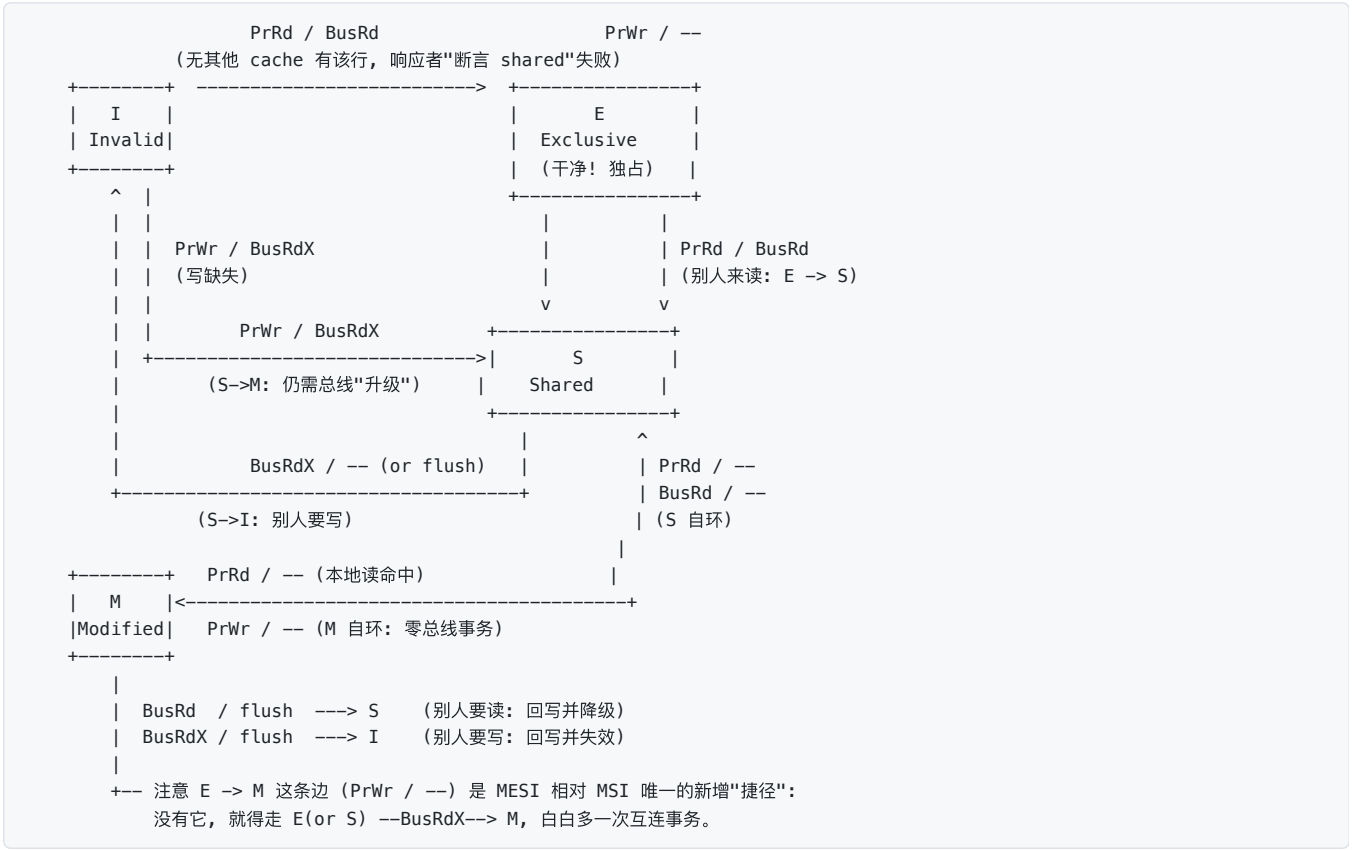
#	动作	P0:X	P0:Y	P1:X	P1:Y	互连事务
6	P1: LD X	I	I	M/3	I	—（M 态本地读）
7	P0: LD X	**S/3**	I	**S/3**	I	BusRd + flush（P1 降级）
8	P0: ST X←4	**M/4**	I	**I**	I	BusRdX
9	P1: LD X	**S/4**	I	**S/4**	I	BusRd + flush
10	P0: LD Y	S/4	**S/0**	S/4	I	BusRd
11	P0: ST Y←1	S/4	**M/1**	S/4	I	BusRdX
12	P1: ST Y←2	S/4	**I**	S/4	**M/2**	BusRdX + flush

量化观察：12 条内存指令里有 **10 次互连事务**（其中 **4 次带 flush**，即 4 次向请求者提供数据的 cache-to-cache 传输），只有第 4、6 步是"纯本地命中、零事务"。可见在这种交替读写的访问模式下，**"M 态吸收写"带来的节省几乎被 `BusRdX` 抵消**——这正是 MESI 要引入 E 态的原因。（表中无效行写 `I`、不再标数值；3.2 节的模拟器输出会保留该行最后一次写入的值，例如 `I/2`。）

- **MSI 为什么满足 coherence（讲义 slide 33 的论证，务必能复述）**：
- **write propagation**：由 `BusRdX` 上的失效 + 之后别人 `BusRd` / `BusRdX` 时 M 态的 flush 共同保证。
- **write serialization**：出现在互连上的写（`BusRdX`）由互连的**出现顺序**定序；出现在互连上的读（`BusRd`）同样由互连定序；**不出现在互连上的写**（对已处于 M 的行连续写）必定夹在**两次针对该行的互连事务之间**，而且这些写全部由**同一个**处理器发出（它自己当然按顺序看到），其他处理器只有在互连事务发生之后才被告知——所以所有处理器看到的顺序一致。

2.8 MESI：用 E 态消掉一次事务（讲义 slide 33–35）

- **定义与目的**：MSI 的低效在于——即使程序**完全没有共享**（一个核读它自己的数据、然后写它自己的数据），"先读后写"这个最常见的序列也要两次互连事务（`BusRd` 让它进 S，`BusRdX` 再把它升到 M）。MESI 增加第四个状态 **E（Exclusive clean，独占且干净）**：该行**未被修改**，但**只有这一个 cache 有副本**。它把**独占性（exclusivity）与所有权（ownership/dirty）解耦**——因为不脏，内存里的副本仍是有效数据。关键收益：**E→M 的升级不需要任何互连事务**。
- **直观解释（"它是什么？"）**：你去图书馆借书，发现**全馆只有这一本**，而且没人预约。既然只有你一个人看，你就可以直接在书上做批注（E→M）而**不必通知图书馆**——因为没人会因为你改了书而读到错误内容。但如果别人也要看（`BusRd`），你的独占就被打破，降级为"共享"（S）。（讲义的原话：MESI, not Messi!）
- **MESI 状态机图解（讲义 slide 35）**：



- **MESI 状态迁移表**：

当前状态	事件	动作	次态	互连事务
I	PrRd（无其他 cache 持有该行）	`BusRd`	**E**	BusRd
I	PrRd（有其他 cache 持有）	`BusRd`	S	BusRd
I	PrWr	`BusRdX`	M	BusRdX
E	PrRd	无	E	—
E	**PrWr**	无（**无需总线事务**，Illinois Protocol 的关键）	M	**—**
E	BusRd	提供数据（本行干净，**不需要 flush**）	S	—
E	BusRdX	作废	I	—
S	PrRd	无	S	—
S	PrWr	`BusRdX`（upgrade）	M	BusRdX
S	BusRd	无	S	—
S	BusRdX	作废	I	—
M	PrRd	无	M	—
M	PrWr	无	M	—
M	BusRd	`flush`（写回或 cache-to-cache 提供）	S	flush
M	BusRdX	`flush` + 作废	I	flush

- **MESI 逐条执行示例（讲义 slide 36–37）**：同一批动作换用 MESI：

#	动作	状态变化	互连事务
0	初始	全部 I	—
1	P0: LD X	P0 **E/0**（没有别人持有 → 独占干净）	BusRd

#	动作	状态变化	互连事务
2	P1: LD X	P0 E→S，P1 S/0	BusRd
3	P0: ST X←1	P0 S→M（BusRdX），P1 → I	BusRdX
4	P0: ST X←2	P0 M/2（本地）	—
5	P1: ST X←3	P1 M/3（BusRdX + flush），P0 → I	BusRdX + flush
6	P0: LD Y	P0 **E/0**（Y 独占干净）	BusRd
7	P0: LD X	P0 S/3（BusRd + flush），P1 → S/3	BusRd + flush
8	P0: ST Y←4	P0 **E→M，零事务**（E 态的独占权直接生效）	—
9	P1: LD Y	P0 M→S/4（flush 提供数据），P1 S/4	BusRd + flush

量化观察（本笔记推导）：这 9 步里 MESI 用 **7 次**互连事务，而 MSI 需要 **8 次**——差的那一次正好是第 8 步"对一个私有数据先读后写"。更一般地：**若负载中"读自己私有的数据然后写它"占主导（例如每个线程更新自己的局部累加器），MSI 每次这样的"读-改"要 2 次事务，MESI 只要 1 次，一致性互连流量减半。**

- **底层实现选择（讲义 slide 38）**：miss 时数据由谁提供？内存，还是另一个 cache（**cache-to-cache transfer**）？若由 cache 提供，该由哪一个提供？cache-to-cache 增加了复杂度，但**同时降低访问延迟和内存带宽需求**，因此被普遍采用。这正是 2.9 节 MESIF/MOESI 要解决的"选谁"问题。

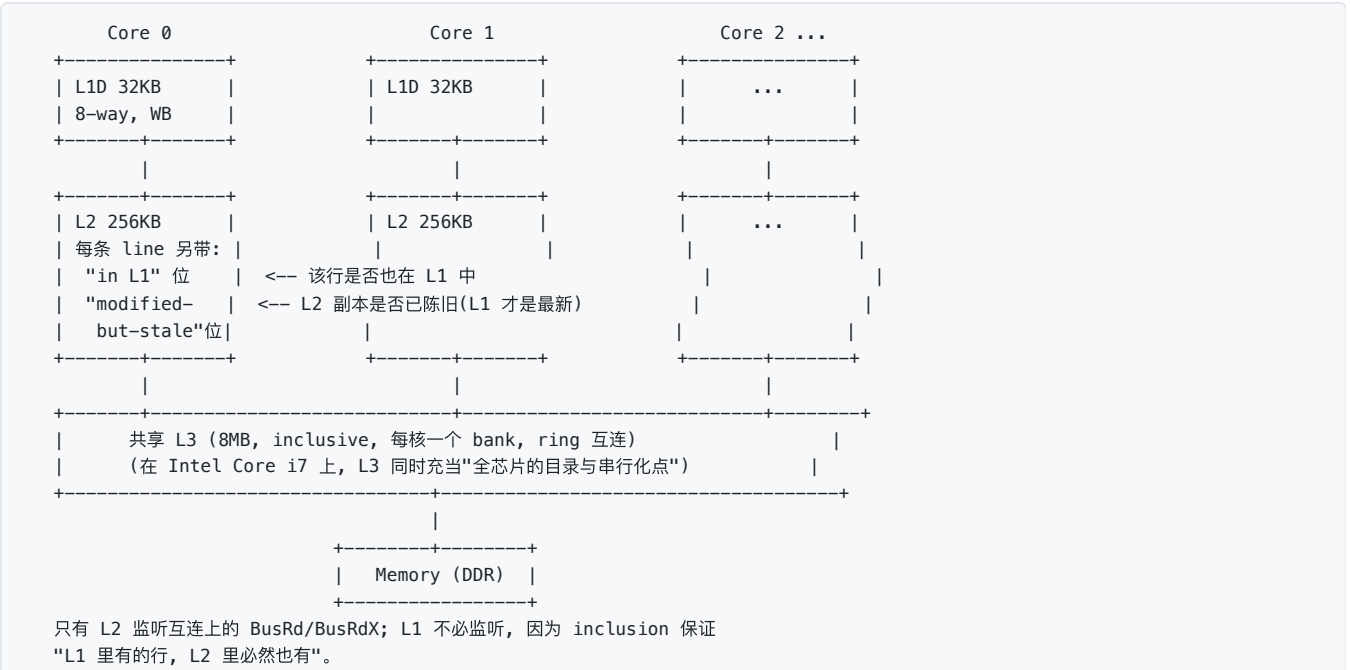
2.9 MESIF 与 MOESI：五态协议的两种思路（讲义 slide 39）

协议	新增状态	含义与机制	动机	使用者
MSI	—	I/S/M	基线	教学
MESI	**E**	Exclusive clean；E→M 无需事务	消除"私有数据先读后写"的 upgrade 事务	教科书 / 多种 CPU
MESIF	**F**（Forward）	类 MESI，但共享行在其中**一个** cache 里是 F 而不是 S；**F 态的 cache 负责服务 miss**	简化"哪个 cache 该响应"的判断：基本 MESI 里所有持有 S 的 cache 都要（可能）响应，容易冲突；F 保证唯一响应者	**Intel 处理器**
MOESI	**O**（Owned）	M→S 的迁移改成 M→O：**不回写内存**就降级；其他核持 S，一个核持 O。因为内存是陈旧的，**O 态的 cache 必须服务 miss**	避免每次 M→S 都要一次内存写回，降低带宽	**AMD Opteron**

- **直观解释（"它是什么？"）**：MESIF 的 **F** 像"**指定发言人**"：一群人都有复印件，但对外发言的只有一个，避免所有人同时抢话筒。MOESI 的 **O** 像"**保管人**"：原件被锁在我抽屉里，别人要复印件来找我，我**不必**先把原件登记回图书馆（省一次写回）。

2.10 现实：多级 cache 层次与 inclusion（讲义 slide 40–44）

- **定义与目的**：真实机器有多级 cache，而"在 L1 里被改过的数据"未必对**监听互连的 L2 controller** 可见。讲义给出两条出路：(1) **所有 cache 都独立监听互连**（低效，L1 也要处理全部一致性流量）；(2) **维持 inclusion（包含性）**：所有"更靠近处理器"的 cache 里的行，**也存在于更远的 cache 里**（L1 ⊆ L2）。这样**只有 L2 需要监听互连**，因为对 L1 有意义的事务必然也对 L2 有意义。



- **inclusion 不会自动成立（讲义 slide 41，易错点）**：设 L2 是 L1 的两倍大，二者行大小相同、都是 2-way、都用 LRU，且 A、B、C 映射到 L1 的**同一个 set**。访问序列：A（L1+L2 miss）→ B（L1+L2 miss）→ A（反复命中 L1）→ C（L1 与 L2 miss）。由于 L1 与 L2 的**访问历史不同**（A 被反复命中只更新了 L1 的 LRU 信息），**两者可能选择逐出不同的行**，于是 A 可能留在 L1 却已被 L2 逐出——**inclusion 被破坏**。真实机器必须靠**显式的包含策略**（如 L2 逐出时反向失效 L1 中的对应行）来维持它。
- **维持 inclusion 需要的两个额外状态位（讲义 slide 42–43）**：
- **`in L1` bit**：L2 的每条 line 记录"它是否也在 L1 里"。当该行因一致性流量（如 `BusRdX`）在 L2 中被失效时，必须**顺着这个位把失效传播到 L1**。
- **`modified-but-stale` bit**：若 L1 是 write-back 且发生 **L1 写命中**，那么该行在 L1 里是新的，而 L2 的副本**在一致性协议里仍显示为 Modified，但数据是陈旧的**。当协议要求 flush 这一行（例如别的核要读 X）时，L2 必须**先向 L1 索取真正的数据**再 flush。这个"L2 说自己是 M 但其实数据在 L1"的状态就是 `modified-but-stale`。
- **直观解释（"它是什么？"）**：inclusion 像**大箱子套小箱子**：小箱子里的东西必须在大箱子里（否则"只看大箱子"就会漏掉）。`modified-but-stale` 位像**图书馆登记簿上写着"这本书被张三借走了，锁在他抽屉里"**：真迹在抽屉（L1），登记簿（L2）上只有一条索引，要交书时必须去抽屉取。
- **硬件影响与现状（讲义 slide 44–45）**：每个 cache 都必须**监听并响应**互连上广播的全部一致性流量，这带来了额外互连流量，**在高核数下会显著**。现状是：**几乎所有现代多核 CPU 都实现缓存一致性**；而**离散 GPU 至今不实现缓存一致性**——讲义判断其理由是：对图形与科学计算应用而言，一致性开销"不值得"（NVIDIA GPU 提供**单一共享 L2 + 原子内存操作**作为替代）；不过**最新的 Intel 集成 GPU 已经实现缓存一致性**。

2.11 GPU 的"不一致"设计：以 CUDA 为例（讲义 slide 45）

```
+--------+   +--------+   +--------+   +--------+
| SMM 0  |   | SMM 1  |   | SMM 2  |   | SMM 3  |   ...   (Streaming Multiprocessors)
| L1 $   |   | L1 $   |   | L1 $   |   | L1 $   |   L1 是 incoherent 的, 每 SMM 私有
+---+----+   +---+----+   +---+----+   +---+----+
    |            |            |            |
    +------------+------------+------------+
                 |
          +------+-----+
          |  统一共享 L2 cache  |  <-- 全芯片"事实来源": 原子操作在 L2 上完成
          +------+-----+
                 |
          +------+-----+
          |  DRAM (device mem) |
          +--------------------+

软件必须显式对付"L1 可能陈旧"这件事:
  * atomicAdd(&x, 1)     -> 绕过 L1, 在 L2 中对一行内容做原子的读-改-写
  * volatile / ld.cg     -> 编译器生成绕过 L1 的 LD 指令 (PTX 的 ld.cg)
  * L1 默认对 L2 是 write-through 的
  * 驱动在两个 kernel 之间清空 L1 -> 保证前一个 kernel 的写在下一个 kernel 可见
```

- **为什么驱动必须清 L1（讲义 slide 45 的反例）**：kernel 1 里 SMM 0 读 x（x 进 L1），SMM 1 写 x（新值只到 L2）；进入 kernel 2，SMM 0 再读 x 就**命中陈旧的 L1 副本**。因此 NVIDIA 驱动在两次 kernel 启动之间清 L1。这也解释了 CUDA 编程里"kernel 边界是天然的同步与可见性边界"这一惯例的硬件根源。想深入可查 NVIDIA PTX 手册的 "Cache Operators"（讲义引用的是 Parallel Thread Execution ISA Version 4.1 的 8.7.6.1 节）。
- **性能特征**：GPU 的做法是把一致性成本从硬件移到**软件约定**上：L1 与 L2 之间靠 write-through + kernel 边界清空 + 显式的 `ld.cg`/原子操作来保证可见性。收益是不必为几百个并发 warp 的每条 L1 访问维护协议状态；代价是程序员必须自己避免"跨线程用 L1 缓存共享数据"的写法。

2.12 程序员视角之一：false sharing 与 artifactual communication（讲义 slide 47–50）

- **定义与目的**：**false sharing（伪共享）** 指两个处理器写**不同的地址**，但这两个地址**落在同一条 cache line** 里。由于一致性的粒度是 cache line，这条行会在写者的 cache 之间**来回弹跳（ping-pong）**，产生大量**协议引起的一致性通信——这完全没有真实的通信需求（no inherent communication），完全是 artifactual communication（人为/附带通信）**。

```
 时间 --->

 P0: [ M X ][写][写][写] |  I   .   .   .   .   .  |  [ S X ](取到新值)  .   .   |
 P1: [  I  ][BusRdX 抢]  |  [ M X ][写][写][写]    |  I   .   .   .   .   .  |   ...
 互连:        ^^^^^^^^                    ^^^^^^^^
             BusRdX + flush               BusRdX + flush
             (独占权转移一次 = 一次完整的 cache line 传输 + 一次写回)

 "弹跳"的代价: 每次转移至少是 一次 cache-to-cache 传输的延迟
 (CS149 数据: L3 命中但行被别的核以 Modified 持有 ~75 cycles,
  而未共享的 L3 命中只要 ~40 cycles)
```

- **讲义给出的代码（slide 47–48）**：

```
/* 讲义 slide 47 原文片段: NUM_THREADS 与 CACHE_LINE_SIZE 是讲义中未展开的宏;
   完整可运行版本见 3.1 节的 false_sharing.c */
/* (A) 有问题的写法: 每线程一个计数器, 但彼此紧挨着 */
int myPerThreadCounter[NUM_THREADS];      /* 12 个 int 挤在同一/相邻的 64B 行 */

/* (B) 修正: 每个计数器独占一条 cache line (C++ 里也可写 alignas(64)) */
struct PerThreadState {
    int myPerThreadCounter;
    char padding[CACHE_LINE_SIZE - sizeof(int)];
};
PerThreadState myPerThreadCounter[NUM_THREADS];
```

讲义 slide 48 的实测（**12 线程 / 12 核系统**，每线程用 `volatile int*` 反复自增自己的计数器）：**未填充 5.1 秒 vs 填充后 2.1 秒**。CS149 补充材料的同类实验（8 线程 / 4 核）是 **14.2 秒 vs 4.7 秒**。

- **一个必须澄清的误区（本笔记推导）**：很多人以为"5.1 → 2.1 只有 2.4 倍，所以 false sharing 没那么可怕"。这个推论是错的。填充后的版本之所以也要 2.1 秒，是因为它撞上了**另一个瓶颈**：`volatile` 强制每次自增都真的做一次 load+store，形成一条串行依赖链（每轮约十几个 cycle），并且只能靠单线程的 ILP 摊薄。**一致性流量的差异要大得多**：设每线程做 `MANY` 次自增，则
- 填充版的互连流量 = `NUM_THREADS × 64 B`（只有义务性冷缺失，例如 12 × 64 B = **768 B**，一次运行内不再增长）；
- 未填充版最坏情况的互连流量 = `NUM_THREADS × MANY × 64 B`（每次自增都要把该行搬到写者手里，还可能附带一次 flush）。 两者相差 `NUM_THREADS × MANY / NUM_THREADS …= MANY` 倍——当 `MANY = 10^7` 时是 **10^7 倍**的流量差，只是**墙钟时间被别的瓶颈掩盖了**。所以判断 false sharing 的危害要看**流量/能耗/可扩展性**，而不只是看一次实验的墙钟比值。讲义 slide 51 的仿真结论也正是这个方向：**false sharing 带来的失效率随 cache line 增大而升高，且随数组规模增大而相对下降**（因为"每个核分到的连续区间"变长，落在同一条边界行上的概率相对降低）。
- **一行式修复（附带的实用建议）**：C++11/17 里可以直接用 `alignas(64)` 让编译器替你填充；OpenMP 里可以用 `#pragma omp parallel for schedule(static, chunk)` 让每个线程分到**至少一整行**的元素，或用 `reduction` 让编译器生成私有副本。

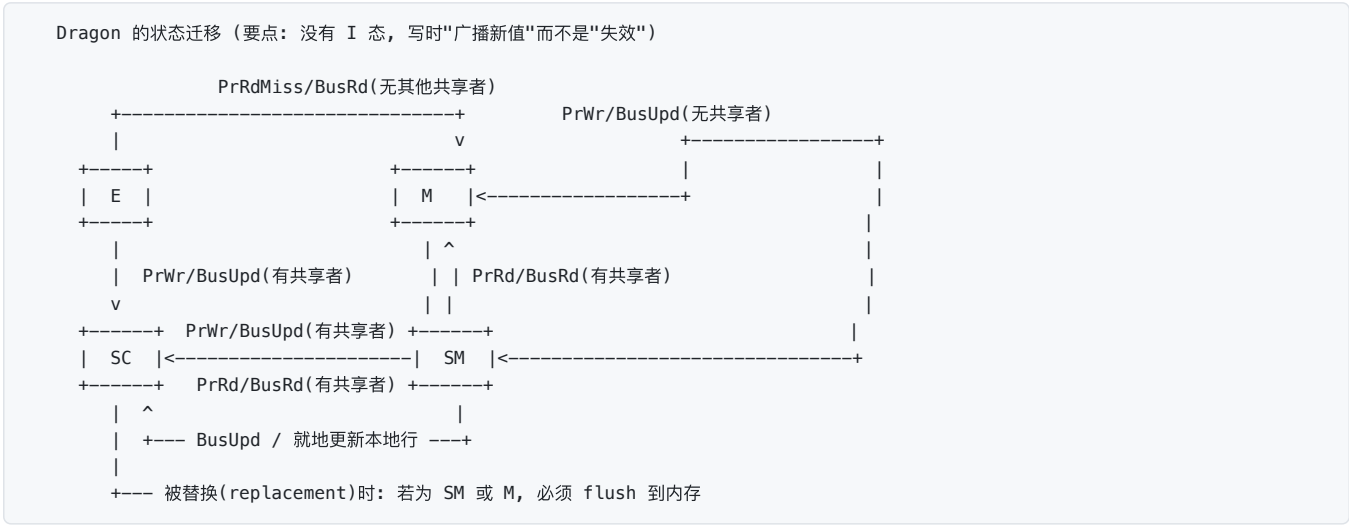
2.13 程序员视角之二：并行基数排序里的伪共享（讲义 slide 51）

- **定义与目的**：讲义用一个真实算法收尾——**b 位数的并行基数排序（parallel radix sort）**：每一轮按 r 位（例：radix 2^4 = 16，即有 16 个 bin）排序，串行循环 `ceil(b/r)` 轮；每轮内每个处理器并行地：(1) 按 r 位值对本地元素排序；(2) 统计落入各 bin 的元素个数；(3) **把各处理器的计数汇总（aggregate）以计算每个 bin 的起始位置**；(4) 把元素写到各自的最终位置。

- **直观解释（"它是什么？"）**：基数排序的散射阶段像**全班同学按学号重排座位**：每个人的目标座位由别人的统计结果决定，而相邻学号的人可能被分到"同一排的相邻座位"（同一 cache line），于是不同处理器为了写相邻的 4 字节，反复争夺同一条行。
- **性能特征（讲义 slide 51 的仿真图）**：把失效率分解成 Cold（义务性）/ Capacity-Conflict（容量与冲突）/ True sharing（真共享）/ False sharing（伪共享）/ Upgrade（升级） 五类可以看出：**line 越小，false sharing 越少但 cold miss 越多**；line 越大，false sharing 越明显。基数排序在 line 小于 64 B 时 false sharing 影响较小、在 64–256 B 时显著上升；而 Barnes-Hut、Radiosity、Ocean Sim 这类具有空间局部性的应用则更受益于**较大**的 line（空间局部性带来的 cold/true sharing 减少）。这解释了为什么 64 B 成为现代处理器的平衡点。

2.14 更新型协议：Dragon 与 invalidate vs update 的取舍（讲义 slide 55–60，Bonus 材料）

- **定义与目的**：前面所有协议都是**失效型（invalidation-based）**：要写一行，cache 必须取得**独占**访问（让别人的副本失效）。它的两个已知代价：**失效后要用整行重新载入**（消耗带宽），以及**对 false sharing 极不友好。更新型（update-based）** 协议走另一条路：写的时候**把新值广播出去**，让其他副本**就地更新**（`BusUpd`）而不是作废。
- **Dragon 写回更新协议的四个状态（讲义 slide 56）**：注意它**没有 Invalid 状态**（可以理解为"行在被第一次载入之前是无效的"）：
- **E（Exclusive-clean）**：只有一个 cache 有此行，内存是最新的。
- **SC（Shared-clean）**：多个 cache 可能有此行的干净副本，**内存可能或可能不是**最新的（若无人处于 SM，内存就是最新的）。
- **SM（Shared-modified）**：多个 cache 可能有此行，内存不是最新的；但**只有一个** cache 处于 SM——它是该行的 **owner**，被替换时必须更新内存。
- **M（Modified）**：只有一个 cache 有此行且是脏的，内存不是最新的；该 cache 是 owner，替换时必须更新内存。
- 处理器事件：`PrRd / PrWr / PrRdMiss / PrWrMiss`；总线事务：`BusRd`、`flush`（提供整行）、**`BusUpd`（总线更新：只广播被修改的数据）**。



- **失效 vs 更新，谁更好（讲义 slide 58–60）**：
- 直觉上，若**其他处理器在写发生之后还会继续读**这份数据，更新看起来更优（数据已经在它们的 cache 里，无需再取）；
- 但更新在两种情形下**纯粹是开销**：(a) 数据写完后**再也没人读**；(b) 程序在**下一次读之前做了很多次写**（每次写都要广播一次 `BusUpd`）。
- 讲义引用的仿真评估（1 MB cache / 64 B line，四个应用 Ray Trace、Radix Sort、LU、Ocean Sim）显示：更新协议在 **false sharing** 类的失效上通常更低，但在 **traffic（升级/更新速率）** 上可能显著更高（"多次写而无人读"造成的高流量）。
- **结论：当今 AMD 与 Intel 的一致性实现都是失效型（invalidation-based）的**。讲义明确要求掌握"失效型与更新型的区别"，但 Dragon 协议的细节属于**项目（projects）需要的加分内容，不要求考试掌握**。

维度	失效型（Invalidation，MSI/MESI/MESIF/MOESI）	更新型（Update，Dragon）
写时做什么	广播 `BusRdX`，让别人**作废**副本	广播 `BusUpd`，让别人**就地更新**副本
写者的独占权	必须取得（否则无法定序）	不一定独占（可能存在多个有效副本）
后续读的代价	需要重新取整行（可能 cache-to-cache）	可能直接本地命中
主要风险	对 false sharing 敏感；每次失效后要重载整行	多次写而无人读时产生大量无用更新流量
状态数	3（MSI）/ 4（MESI）/ 5（MESIF、MOESI）	4（E、SC、SM、M；**无 I**）
现状	**Intel / AMD 现行实现**	研究/历史方案

---

3. 代码示例与性能分析

3.1 示例 1：把讲义 slide 48 的 false sharing 实验做成可复现程序

代码（`false_sharing.c`）：

```c
/* ============================================================================
 * false_sharing.c —— 讲义 slide 48/49 的 false sharing 演示（可复现）
 *   两个版本做完全相同的工作、得到完全相同的数值结果，唯一区别是内存布局：
 *     (A) 每线程的计数器紧挨着排布  -> 12 个 long 落在同一条 64B cache line 内
 *     (B) 每线程的计数器各自独占一条 cache line (padding)
 * 编译: gcc -O3 -pthread false_sharing.c -o false_sharing
 *       (加 -march=native 无用: 瓶颈在 cache 一致性与访存依赖链, 不在指令选择)
 * 运行: ./false_sharing 12 10000000        # 12 线程, 每线程 1e7 次自增
 * ============================================================================*/
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define CACHE_LINE  64
#define MAX_THREADS 256

/* 每个计数器独占一条 cache line */
typedef struct {
    volatile long counter;
    char pad[CACHE_LINE - sizeof(long)];
} __attribute__((aligned(CACHE_LINE))) padded_counter_t;

static long g_iters = 0;              /* 每个线程的迭代次数 */

static void* worker(void* arg) {
    volatile long* p = (volatile long*)arg;
    for (long i = 0; i < g_iters; i++)
        (*p)++;                       /* volatile: 每轮都必须真的发 load + store */
    return NULL;
}

static double now_sec(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

static double run(const char* label, int nthreads, int padded) {
    static long           raw[MAX_THREADS];      /* (A) 紧凑: 相邻计数器共享 line   */
    static padded_counter_t pad[MAX_THREADS];    /* (B) 填充: 计数器各自独占 line   */
    pthread_t tid[MAX_THREADS];

    memset(raw, 0, sizeof(long) * nthreads);
    memset(pad, 0, sizeof(padded_counter_t) * nthreads);

    double t0 = now_sec();
    for (int i = 0; i < nthreads; i++) {
        void* arg = padded ? (void*)&pad[i].counter : (void*)&raw[i];
        if (pthread_create(&tid[i], NULL, worker, arg) != 0) {
            fprintf(stderr, "pthread_create failed\n");
            exit(1);
        }
    }
    for (int i = 0; i < nthreads; i++) pthread_join(tid[i], NULL);
    double t1 = now_sec();

    long sum = 0;
    for (int i = 0; i < nthreads; i++)
        sum += padded ? pad[i].counter : raw[i];

    printf("  %-12s 时间 %7.3f s   校验和 %ld (= %d x %ld, 两种布局必须相同)\n",
           label, t1 - t0, sum, nthreads, g_iters);
    return t1 - t0;
}

int main(int argc, char** argv) {
    int  nthreads = (argc > 1) ? atoi(argv[1]) : 12;
    long iters    = (argc > 2) ? atol(argv[2]) : 10000000L;
    if (nthreads < 1 || nthreads > MAX_THREADS) { fprintf(stderr, "bad nthreads\n"); return 1; }
```

```
    g_iters = iters;

    printf("线程数 = %d, 每线程迭代 = %ld, cache line = %d B\n", nthreads, iters, CACHE_LINE);
    /* 先跑一次短的做预热, 避免把页错误/频率爬升算进结果 */
    g_iters = iters / 100;
    run("warmup", nthreads > 2 ? 2 : 1, 1);
    g_iters = iters;

    double t_unpadded = run("unpadded", nthreads, 0);   /* (A) false sharing */
    double t_padded   = run("padded",   nthreads, 1);   /* (B) 每线程独占行 */

    printf("\n  加速比 = %.2fx\n", t_unpadded / t_padded);
    printf("  一致性流量下界(填充版)   = %d x %d B = %d B\n",
           nthreads, CACHE_LINE, nthreads * CACHE_LINE);
    printf("  一致性流量上界(未填充版) = %d x %ld x %d B = %.3f GB\n",
           nthreads, iters, CACHE_LINE,
           (double)nthreads * (double)iters * CACHE_LINE / 1e9);
    return 0;
}
```

【代码做什么？】 1. `main` 解析线程数与迭代次数；先用一个短跑的 `warmup` 把缺页、CPU 频率爬升、线程创建成本从计时中剔除。 2. `run(..., padded=0)`：静态数组 `raw[]` 里 12 个 `long`（8 B）**连续排列**，全部落在 1~2 条 64 B cache line 内；为每个线程传 `&raw[i]`，各线程用 `volatile long*` 反复 `(*p)++`。 3. `run(..., padded=1)`：`padded_counter_t` 通过 `__attribute__((aligned(64)))` 与 56 B 填充把每个计数器**顶到独立的 64 B 边界**，每个线程只碰自己那条行。 4. 两次运行都 join 全部线程后把各计数器求和打印——**两个版本的校验和必然相同**（`nthreads × iters`），因为它们做的是同一件工作、彼此之间没有数据竞争，差别**只在内存布局**。这正是"false sharing 是 artifactual communication"的最好证明。 5. 最后打印两个版本的**一致性流量上/下界**，把"墙钟比值"和"流量比值"分开呈现（见 2.12 节的讨论）。

【并行机制与性能解说】 – **线程如何创建与分配工作**：pthread 每线程一个 `worker`，工作分配方式是"线程 i 独占数组元素 i"（静态划分，天然无负载不均：每线程工作量完全相同 = `iters`）。 – **共享数据如何处理**：**没有任何共享数据需要同步**——每个线程只写自己的计数器。问题恰恰在于**硬件把它们放进了同一条 cache line**。 – **Work / Span / 并行度**： – **Work（总工作量）** `W = NUM_THREADS × MANY` 次自增（每次自增是 1 次 load + 1 次 add + 1 次 store）。 – **Span（关键路径）**：填充版下，每个线程的关键路径是自己计数器上 `MANY` 次自增的**串行依赖链**（load→add→store 必须按序，因为下一轮的 load 依赖上一轮的 store，靠 store-to-load forwarding 打通），所以 `S ≈ MANY × c_local`（`c_local` ≈ 十几 cycle 量级）。 – **并行度** = `W / S = NUM_THREADS`。这与"12 线程应得 12 倍"的直觉一致：**填充版的并行度上限就是线程数**，受限于每核的串行依赖链和访存吞吐。 – **未填充版**：那条被共享的 cache line 成了一个**串行资源（serialized resource）**。在 work-span 语言里，这相当于给 span 加了一条额外约束：`S' ≥ (NUM_THREADS × MANY) × c_xfer`（`c_xfer` 是一次 cache line 独占权转移的开销，至少含 cache-to-cache 传输延迟，CS149 给出的量级是"L3 命中但行被别的核以 Modified 持有 ~75 cycles"，而正常未共享的 L3 命中只要 ~40 cycles）。于是 `并行度 = W / S' ≈ c_local / c_xfer ≪ 1`。 **并行度小于 1 的含义是：即使给你无限多个核，也快不过这个串行资源**——这是 false sharing 最本质的危害，比"实测慢 2.4 倍"严重得多。 – **瓶颈**：填充版 → **每核的串行依赖链 + 指令吞吐**（在本例的 `volatile` 写法下，12 核跑不满，因为每轮的 load/store 依赖链太长，属延迟受限而非带宽受限）；未填充版 → **一致性协议的独占权转移延迟 + 互连流量**。要真正测出 12× 的差别，应把循环体改成多路独立累加（例如每线程 8 个局部计数器、最后合并），让每个线程有足够的 ILP 把访存延迟隐藏掉——这与"用更多 ILP 隐藏内存延迟"的一般优化原则一致。

3.2 示例 2：MSI / MESI 一致性协议模拟器（复现讲义 slide 30/31 的时序）

代码（`msi_sim.cpp`）：

```
/* ============================================================================
 * msi_sim.cpp —— MSI / MESI 缓存一致性协议模拟器
 *   用途:
 *     1) 逐步复现讲义 slide 30/31 的 12 步执行序列, 打印 P0/P1 在 X/Y 两行上的状态
 *     2) 统计两种协议的互连事务数 (BusRd / BusRdX / flush / cache-to-cache)
 *     3) 用一个"私有数据: 读一次再写一次"的常见负载, 量化 MESI 相对 MSI 的收益
 * 编译: g++ -O2 -std=c++17 msi_sim.cpp -o msi_sim
 * 运行: ./msi_sim            # 复现讲义时序, MSI
 *       ./msi_sim mesi       # 同一时序, MESI
 *       ./msi_sim count      # MSI 与 MESI 在合成负载上的事务数对比
 * ============================================================================*/
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>

enum St { I, S, E, M };                 /* Invalid / Shared / Exclusive / Modified */
static const char* stname(St s) { return s == I ? "I" : s == S ? "S" : s == E ? "E" : "M"; }

struct Line { St st = I; int val = 0; };

struct Sim {
    bool mesi;                          /* false: MSI (永不使用 E 态); true: MESI */
    int  ncpu, naddr;
    std::vector<std::vector<Line>> c;   /* c[cpu][addr] */
    std::vector<int> mem;               /* 每个地址在主存中的值 */
    long busRd = 0, busRx = 0, flush = 0, c2c = 0, localHit = 0, accesses = 0;

    Sim(int n, int a, bool m) : mesi(m), ncpu(n), naddr(a), c(n, std::vector<Line>(a)), mem(a, 0) {}

    bool anyOtherValid(int self, int addr) const {
        for (int i = 0; i < ncpu; i++)
            if (i != self && c[i][addr].st != I) return true;
        return false;
    }
    int mOwner(int self, int addr) const {          /* 处于 M 态的其他 cache, 没有则 -1 */
        for (int i = 0; i < ncpu; i++)
            if (i != self && c[i][addr].st == M) return i;
        return -1;
    }

    /* 广播一次互连事务: 先让 M 态 owner 回写(保证 Data-Value 不变量), 再让其他 cache 反应 */
    int broadcast(int self, int addr, bool exclusive) {
        int owner = mOwner(self, addr);
        int data;
        if (owner >= 0) {                           /* owner 提供最新数据 */
            data = c[owner][addr].val;
            mem[addr] = data;                       /* flush: 内存被更新 */
            flush++;
            c2c++;                                  /* cache-to-cache transfer */
        } else {
            data = mem[addr];
        }
        for (int i = 0; i < ncpu; i++) {
            if (i == self) continue;
            if (exclusive) {
                c[i][addr].st = I;                  /* BusRdX: 一律失效 (SWMR 不变量的实现) */
            } else {
                if (c[i][addr].st == M) c[i][addr].st = S;   /* BusRd: M -> S (已回写) */
                else if (c[i][addr].st == E) c[i][addr].st = S;
                /* S 保持 S, I 保持 I */
            }
        }
        return data;
    }

    /* 返回本次访问(若是读)观察到的值 */
    int access(int cpu, int addr, bool write, int wval = 0) {
        accesses++;
        Line& L = c[cpu][addr];
        if (!write) {
```

```
            if (L.st != I) { localHit++; return L.val; }          /* S/E/M 命中: 零总线事务 */
            bool shared = anyOtherValid(cpu, addr);
            busRd++;
            int v = broadcast(cpu, addr, false);
            L.val = v;
            L.st = (mesi && !shared) ? E : S;                     /* MESI: 独占时直接进 E */
            return v;
        }
        if (L.st == M) { localHit++; L.val = wval; return wval; } /* M 态吸收写: 零总线事务 */
        if (L.st == E) { localHit++; L.val = wval; L.st = M; return wval; } /* MESI 关键捷径 */
        busRx++;                                                  /* I 写缺失 / S 升级 */
        broadcast(cpu, addr, true);
        L.val = wval;
        L.st = M;
        return wval;
    }
};

/* 打印一行: 列顺序与讲义 slide 30/31 一致 —— P0:X  P0:Y  P1:X  P1:Y */
static void show(const Sim& s, int step, const std::string& act, const std::string& bus) {
    printf(" %2d  %-12s", step, act.c_str());
    for (int i = 0; i < s.ncpu; i++)
        for (int a = 0; a < s.naddr; a++)
            printf("  %s/%-2d", stname(s.c[i][a].st), s.c[i][a].val);
    printf("   %s\n", bus.c_str());
}

int main(int argc, char** argv) {
    std::string mode = (argc > 1) ? argv[1] : "msi";
    bool mesi = (mode == "mesi");

    if (mode == "count") {                /* ---------- 合成负载: 量化 MSI vs MESI ---------- */
        const int T = 12, K = 100000;     /* 12 个核, 每核重复 K 次"读自己的行 + 写自己的行" */
        for (int m = 0; m < 2; m++) {
            Sim s(T, T, m == 1);
            for (int k = 0; k < K; k++)
                for (int c0 = 0; c0 < T; c0++) { s.access(c0, c0, false); s.access(c0, c0, true, k); }
            printf("%-5s: 访问 %ld 次, BusRd %ld, BusRdX %ld, flush %ld, 本地命中 %ld, 互连事务 %ld\n",
                   m ? "MESI" : "MSI", s.accesses, s.busRd, s.busRx, s.flush, s.localHit,
                   s.busRd + s.busRx);
        }
        return 0;
    }

    /* ---------- 复现讲义 slide 30/31 的 12 步时序 (addr 0 = X, addr 1 = Y) ---------- */
    Sim s(2, 2, mesi);
    printf("协议 = %s   (列格式: 状态/值; 列顺序与讲义 slide 30/31 一致)\n", mesi ? "MESI" : "MSI");
    printf(" step  action        P0:X    P0:Y    P1:X    P1:Y   互连事务\n");
    printf("    0  initial       I/0     I/0     I/0     I/0\n");
    struct Op { int cpu; int addr; bool wr; int val; const char* name; };
    const Op ops[] = {
        {0,0,false,0,"P0: LD X"}, {1,0,false,0,"P1: LD X"},
        {0,0,true, 1,"P0: ST X<-1"}, {0,0,true,2,"P0: ST X<-2"}, {1,0,true,3,"P1: ST X<-3"},
        {1,0,false,0,"P1: LD X"}, {0,0,false,0,"P0: LD X"}, {0,0,true,4,"P0: ST X<-4"},
        {1,0,false,0,"P1: LD X"},
        {0,1,false,0,"P0: LD Y"}, {0,1,true,1,"P0: ST Y<-1"}, {1,1,true,2,"P1: ST Y<-2"},
    };
    int step = 0;
    for (const Op& op : ops) {
        long r0 = s.busRd, x0 = s.busRx, f0 = s.flush;
        s.access(op.cpu, op.addr, op.wr, op.val);     /* addr 0 = X, addr 1 = Y */
        char bus[64]; bus[0] = 0;
        if (s.busRd > r0) snprintf(bus, sizeof bus, "BusRd%s", s.flush > f0 ? " + flush" : "");
        else if (s.busRx > x0) snprintf(bus, sizeof bus, "BusRdX%s", s.flush > f0 ? " + flush" : "");
        else snprintf(bus, sizeof bus, "-- (本地命中)");
        show(s, ++step, op.name, bus);
    }
    printf("\n合计: 访问 %ld 次, BusRd %ld, BusRdX %ld, flush %ld, "
           "cache-to-cache 提供数据 %ld 次, 纯本地命中 %ld 次\n",
           s.accesses, s.busRd, s.busRx, s.flush, s.c2c, s.localHit);
```

```
    return 0;
}
```

【代码做什么？】 1. `Line{st, val}` 表示一条 cache line 的状态与数据；`Sim` 维护 `ncpu × naddr` 条这样的行 + 主存 `mem[]`，以及四个计数器（`busRd` / `busRx` / `flush` / `c2c`）。 2. `access(cpu, addr, write, wval)` 是**本地 CPU 触发的事件**，按 MSI/MESI 状态机更新状态并发起总线事务：读 I→(S 或 MESI 的 E)、`BusRd`；写 M→本地、E→M（仅 MESI，零事务）、I/S→`BusRdX`。 3. `broadcast(self, addr, exclusive)` 是**远程 cache 的监听反应**：先让 M 态 owner **flush**（同时更新 `mem[]`，从而维持 Data-Value 不变量），再对每个其他 cache 施加"`BusRdX` 一律失效 / `BusRd` 把 M 与 E 降级为 S"的规则，并把数据（owner 的或内存的）返回给请求者。 4. `main` 的默认模式按讲义 slide 30 的 12 条指令逐步执行并打印一张**逐状态表**；输出应与 2.7 节推演的表一致（这就是讲义 slide 31 那一页拥挤的表格的"自动化版本"）。 5. `count` 模式构造一个"每个核读写自己的私有行"的负载（12 核 × 10 万轮），比较 MSI 与 MESI 的 `BusRd` / `BusRdX` 计数。

【并行机制与性能解说】 – **这段代码模拟的硬件并行**：`access` 对应本地 CPU 的 load/store 通路；`broadcast` 对应互连上的**原子广播**（讲义 slide 23 的简化假设：事务原子、失效立即生效）。真实机器上这两条通路**并行重叠**（非阻塞 cache、MSHR），而模拟器把每一次访问串行化。 – **共享数据如何处理**：状态机本身就是"共享数据（cache line 状态）"的唯一权威副本，`broadcast` 是那个**串行化点**——它精确对应真实硬件上"互连给写定序"的角色。 – **Work / Span / 并行度**： – **Work** `W = A`（访问条数），每次访问的协议逻辑是 O(1)（`broadcast` 里还有一个 O(NUM_CPUS) 的循环 → 严格说 `W = O(A × P)`，`P` = cache 数）。 – **Span** `S = A`：因为每一步的状态迁移都依赖上一步的状态，这是**本质串行**的模拟。 – **并行度** `= W / S = 1`（考虑 broadcast 的 O(P) 循环后为 `P`，但那 P 条操作在同一次广播里彼此独立，可以并行——这正是真实硬件里"所有 cache controller 同时监听"的模样）。 – **可扩展性上限**：串行部分占 100%，**Amdahl 定律给出加速比 ⩽ 1**：再加核也不会让模拟更快。要并行化它，唯一的办法是**按地址分片（partition by address）**——不同的地址之间没有一致性依赖（coherence 是**逐地址**的语义！），因此可以把模拟器拆成"每个地址一个串行轨道"，轨道数就是并行度；但**单个热点地址内部的步骤仍必须串行**。这是"一致性是 per-location 的"这一性质在工程上的直接体现。 – **瓶颈**：单热点行的访问序列完全串行；模拟器输出/计数会引入少量常数开销；`broadcast` 的 O(P) 循环在核数大时成为主项，也与真实 snooping 的 O(N) 广播代价对应。

3.3 示例 3：直方图的私有化（privatization）+ 归并——真共享的解法

代码（`hist.c`）：

```c
/* ============================================================================
 * hist.c —— 直方图: "原子累加到共享 bin" vs "每线程私有 padded bin + 归并"
 *   这是讲义 slide 51 (基数排序) 与 slide 47 (每线程局部累加) 的同一个套路:
 *   把"真共享 (true sharing) 的写热点"改造成"私有写 + 一次便宜的归并"。
 * 编译: gcc -O3 -fopenmp -march=native hist.c -o hist
 *       (-O3 让散射循环向量化; -fopenmp 提供线程; 开 -O0 会让两者都被算力掩盖)
 * 运行: OMP_NUM_THREADS=12 ./hist 100000000 4096
 * ============================================================================*/
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define CACHE_LINE 64
#define MAX_THREADS 256

typedef struct {                    /* 每线程私有的一份 bin 数组, 与下一条 line 隔开 */
    long count[4096];
    char pad[CACHE_LINE];
} bins_t;

static int*   g_data = NULL;
static long   g_n    = 0;
static int    g_nbins = 0;

static double now_sec(void) {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

/* ---------- (A) 共享 bin + 原子操作: 每次更新都要独占含该 bin 的 cache line ---------- */
static double hist_atomic(void) {
    long* shared = (long*)calloc(g_nbins, sizeof(long));
    double t0 = now_sec();
    #pragma omp parallel for schedule(static)
    for (long i = 0; i < g_n; i++) {
        int b = g_data[i];
        #pragma omp atomic
        shared[b] += 1;                     /* 读-改-写, 由硬件保证原子, 但在热点 bin 上串行化 */
    }
    double t1 = now_sec();
    long total = 0;
    for (int b = 0; b < g_nbins; b++) total += shared[b];
    printf("  (A) 共享bin + atomic : %7.3f s   总和 %ld\n", t1 - t0, total);
    free(shared);
    return t1 - t0;
}

/* ---------- (B) 每线程私有 bin + 归并: 私有化 (privatization) ---------- */
static double hist_private(void) {
    static bins_t priv[MAX_THREADS];
    int T = omp_get_max_threads();
    if (T > MAX_THREADS) T = MAX_THREADS;
    memset(priv, 0, sizeof(bins_t) * T);

    double t0 = now_sec();
    #pragma omp parallel num_threads(T)
    {
        int tid = omp_get_thread_num();
        long* mybin = priv[tid].count;      /* 这段内存只被本线程写: 零一致性流量 */
        #pragma omp for schedule(static)
        for (long i = 0; i < g_n; i++)
            mybin[g_data[i]] += 1;          /* 注意: 一个 bin 内多元素累加有依赖, 但只在本地 */
    }
    /* 归并: 树形两两合并, span = log2(T) 轮, 而不是 T 次串行累加 */
    double t1 = now_sec();
    long* acc = (long*)calloc(g_nbins, sizeof(long));
    for (int b = 0; b < g_nbins; b++) {
        long s = 0;
        for (int t = 0; t < T; t++) s += priv[t].count[b];
```

```
        acc[b] = s;
    }
    double t2 = now_sec();
    long total = 0;
    for (int b = 0; b < g_nbins; b++) total += acc[b];
    printf("  (B) 私有bin + 归并   : %7.3f s (散射 %.3f + 归并 %.3f)  总和 %ld\n",
           t2 - t0, t1 - t0, t2 - t1, total);
    free(acc);
    return t2 - t0;
}

int main(int argc, char** argv) {
    g_n     = (argc > 1) ? atol(argv[1]) : 100000000L;
    g_nbins = (argc > 2) ? atoi(argv[2]) : 4096;
    g_data  = (int*)malloc(sizeof(int) * g_n);
    if (!g_data) { fprintf(stderr, "OOM\n"); return 1; }

    unsigned seed = 12345;
    for (long i = 0; i < g_n; i++) {           /* 均匀随机 bin -> 既有热点也有散布 */
        seed = seed * 1103515245u + 12345u;
        g_data[i] = (int)((seed >> 8) % (unsigned)g_nbins);
    }
    printf("N = %ld, bins = %d, 线程 = %d, 数据 %.1f MB\n",
           g_n, g_nbins, omp_get_max_threads(), (double)g_n * sizeof(int) / 1e6);

    double a = hist_atomic();
    double b = hist_private();
    printf("\n  加速比 (A)/(B) = %.2fx\n", a / b);
    free(g_data);
    return 0;
}
```

【代码做什么？】 1. 生成 `N` 个随机落在 `[0, nbins)` 的键（模拟基数排序里"元素 → bin"的散射）。 2. `hist_atomic`：所有线程对**同一份** `shared[]` 用 `#pragma omp atomic` 做 `+= 1`。硬件必须对每个 bin 做**原子的读–改–写**：若该行不在本地 cache 的 M 态，就要先 `BusRdX` 抢独占权（就是 2.7 节的 MSI 事务），热点 bin 上所有线程排队。 3. `hist_private`：每个线程在 `priv[tid].count[]`（由 `bins_t` 的填充与相邻线程隔开）上**无锁累加**；因为每个 bin 只被一个线程写，**全过程中零一致性事务、零缓存行弹跳**。最后做一次归并：对每个 bin 把 T 份计数相加（`O(T × nbins)` 的额外工作）。 4. 两个版本打印各自的校验和（总元素数）——**结果完全相同**，但性能差距来自"共享写热点"是否被消除。

【并行机制与性能解说】 – **线程与工作分配**：OpenMP `parallel for schedule(static)` 把 `N` 个元素按块静态分给 T 个线程（静态划分 + 均匀随机数据 → 负载均衡良好）。`priv[tid]` 的访问不需要任何同步，因为**写集合互不相交**。 – **共享数据如何处理**：版本 (A) 的 `shared[b]` 是典型**真共享（true sharing）**：读写的是**同一个地址**，同步是**必需的**（缺了它结果就错）；版本 (B) 把它变成"私有副本 + 一次显式归并"，用 `O(T × nbins)` 的额外工作量换掉了所有一致性流量。 – **Work / Span / 并行度**（设 `N` 个元素、`T` 线程、`B` = nbins）： – 版本 (B)：**Work** `W = N + T·B`（散射 `N` + 归并 `T·B`）；**Span** `S = N/T + B`（最后一个线程的散射 + 一个 bin 的归并）；**并行度** `= W/S ≈ T`（当 `T·B ≪ N` 时）。在 `N = 10^8, T = 12, B = 4096` 下 `W = 10^8 + 49152 ≈ 10^8`，`S ≈ 8.34e6 + 4096`，并行度约 **11.98**——几乎完美可扩展。若把归并也并行化（每个 bin 一个线程/树形合并），span 还能再降。 – 版本 (A)：**Work** 同样是 `O(N)`，但**共享行的独占权转移把它们串行化了**：设热点 bin 上的更新占比为 `f`，则关键路径 `S ≈ f·N·c_xfer + (1−f)·N/T·c_local`。当 `f` 不可忽略时，`S` 由 `f·N·c_xfer` 支配，**并行度退化到 ~1**（`c_xfer` 是几十 cycle 的缓存行转移，`c_local` 只是本地 L1 更新）。这与 3.1 节 false sharing 的结论形式相同，但原因不同：**那是"不同地址恰好在同一行"（伪共享），这是"同一地址（真共享）"**——(A) 的串行化是**必要的**（否则结果错），只是不该让它成为主路径。 – **Amdahl 视角**：若 (A) 版本中 30% 的时间花在串行化的热点行上，则 `加速比 ≤ 1/(0.3 + 0.7/12) = 2.98×`——**12 核只得到不到 3 倍**，这就是"共享写热点把 Amdahl 的串行部分人为抬高"的定量后果。 – **瓶颈与工程权衡**： – 版本 (B) 的私有 bin 数组占用 `B × 8 B × T`。`B = 4096` 时每线程 32 KB——**正好等于 Haswell 的 L1D 大小（32 KB）**，再大就会溢出到 L2；`T = 12` 时总占用 384 KB，仍在 8 MB 共享 L3 之内；但**若 `B` 增到 10^6，每线程 8 MB，就会把 L3 乃至 DRAM 拖下水——私有化不是免费的，它是用空间（和 L3 带宽）换一致性流量**。这正是基数排序选择 radix 位宽 r 时的真实约束（`2^r` 个 bin 必须装得下）。 – 版本 (A) 的瓶颈还有一层：`#pragma omp atomic` 在 x86 上通常编译为带 `lock` 前缀的 RMW，其开销远高于普通 store（几十 cycle 甚至上百），且**热点 bin 的行在多个核的 L1/L2 之间弹跳**。唯一可用的缓解是**把"热点"变成"私有"**（本示例的做法）或**用硬件原子在 L2 完成**（GPU 的 `atomicAdd` 走 L2 就是这个思路）。 – 结论：**"共享写 + 原子"是必需的同步，但绝不该出现在热路径上**；正确做法是私有化后归并（parallel histogram / reduction 的通用套路）。

3.4 示例 4（微型）：真共享计数器——为什么必须做规约

```c
/* 编译: gcc -O3 -fopenmp reduce_vs_atomic.c -o rva && OMP_NUM_THREADS=12 ./rva */
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    const long N = 100000000L, NITER = 2000;
    double* a = (double*)malloc(sizeof(double) * N);
    for (long i = 0; i < N; i++) a[i] = 1.0;      /* 求和结果 = N */

    /* (A) 共享累加器 + 原子操作: 含 sum 的那条 cache line 被 12 个核反复抢夺 */
    double sum = 0.0;
    double t0 = omp_get_wtime();
    for (int it = 0; it < NITER; it++) {          /* 刻意重复, 放大串行部分 */
        #pragma omp parallel for
        for (long i = 0; i < N; i++) {
            #pragma omp atomic
            sum += a[i];
        }
    }
    double t1 = omp_get_wtime();

    /* (B) 规约: 编译器为每个线程生成私有累加器, 最后归并 (等价于手工 privatization) */
    double sum2 = 0.0;
    double t2 = omp_get_wtime();
    for (int it = 0; it < NITER; it++) {
        #pragma omp parallel for reduction(+:sum2)
        for (long i = 0; i < N; i++) sum2 += a[i];
    }
    double t3 = omp_get_wtime();

    printf("(A) atomic  : %.3f s   sum = %.0f\n", t1 - t0, sum / NITER);
    printf("(B) reduction: %.3f s  sum = %.0f\n", t3 - t2, sum2 / NITER);
    printf("加速比 = %.1fx\n", (t1 - t0) / (t3 - t2));
    free(a);
    return 0;
}
```

【代码做什么？】 两次相同规模求和：(A) 用 `#pragma omp atomic` 把每个元素的贡献加到**一个共享标量** `sum` 上；(B) 用 `reduction(+:sum2)`，OpenMP 为每个线程开一个私有累加器，循环结束时再归并。

【并行机制与性能解说】 – **Work / Span / 并行度**：两者 Work 都是 `W = N × NITER` 次加法。差别全在 **Span**： – (A) 的 span `= N × NITER × c_atomic` ——**每一次加法都要独占"含 sum 的那条 cache line"**，所以所有线程的更新在**同一条行**上串行化，`并行度 ≈ c_local / c_atomic ≪ 1`，等效于"1 个核干活、其余 11 个核排队等 line"； – (B) 的 span `= N/T × NITER × c_add + T × c_add`（私有累加 + T 次归并），`并行度 ≈ T = 12`。 – **瓶颈**：(A) 的瓶颈是**一致性协议的独占权转移**（真共享的写热点）；(B) 的瓶颈重新回到**内存带宽**——`N = 10^8` 个 double = 800 MB，每轮扫描都要 800 MB，在 20 GB/s 下**单轮下界 40 ms**，2000 轮的理论下界是 80 s，因此 (B) 的良好实现会是**带宽受限**而非延迟受限。这个例子清楚展示了**优化的层次**：先用私有化/规约消掉真共享的串行化，之后才轮到你面对带宽这个"下一道墙"。

3.5 四个示例的横向对比

示例	Work `W`	Span `S`	并行度 `W/S`	主要瓶颈	与讲义对应的概念
1. false sharing（未填充）	`T × MANY`	`T × MANY × c_xfer`	`c_local/c_xfer ≪ 1`	一致性独占权转移（伪共享）	slide 49–50
1. false sharing（填充）	`T × MANY`	`MANY × c_local`	`= T`	每核 load→store 依赖链	slide 47–48
2. MSI/MESI 模拟器	`A × P`	`A`	`1`（按地址分片后为 `#addr`）	本质串行；热点地址无法并行	slide 28–37
3. 直方图（私有化）	`N + T·B`	`N/T + B`	`≈ T`	私有 bin 是否装得进 L1/L2（空间换流量）	slide 47, slide 51
3. 直方图（原子）	`N`	`f·N·c_xfer`	`≪ 1` 当热点占比 `f` 不可忽略	真共享写热点的行弹跳	slide 51

示例	Work `W`	Span `S`	并行度 `W/S`	主要瓶颈	与讲义对应的概念
4. 共享求和（atomic）	`N × NITER`	`N × NITER × c_atomic`	`≪ 1`	单行独占权；`lock` RMW 开销	slide 18（一致性开销暴露给软件）
4. 共享求和（reduction）	`N × NITER`	`N/T × NITER + T`	`≈ T`	内存带宽（800 MB/轮）	slide 19（共享 cache 的带宽争用同理）

---

4. 性能模型与复杂度分析

4.1 AMAT：把一致性延迟算进平均访存时间（CS149 补充材料 slide 39）

- **公式**：`AMAT = Σ_{i=0}^{n} (第 i 级/情形的访问频率 × 该情形的延迟)`。多处理器系统里 `AMAT_multiprocessor > AMAT_uniprocessor`，因为 (a) cache 的**失效率上升**（别的核的失效把你有效的行作废，产生"一致性缺失"）；(b) **某些缺失的延迟变长**（数据在别的核的 cache 里，或在 NUMA 的远端内存）。
- **参考延迟（CS149 slide 39，Intel Core i7 Xeon 5500 系列，约 4 GHz 时把 ns 换算为 cycle）**：

情形	延迟	备注
L1 命中	~4 cycles	与单处理器相同
L2 命中	~10 cycles	
L3 命中，行**未被共享**	~40 cycles	数据只有一份、内存是最新的
L3 命中，行**被另一核以共享态持有**	~65 cycles	需要与别的核交互（F/S 态响应者）
L3 命中，行**被另一核以 Modified 持有**	~75 cycles	需要 cache-to-cache 传输 + 让 owner 回写
本地 DRAM	~30 ns（~120 cycles）	
远端 DRAM（NUMA）	~100 ns（~400 cycles）	跨 socket

注意这张表与 2.2 节的 Haswell 表**不是同一台机器**（前者是 Nehalem 时代的 Xeon 5500，后者是 2013 年的 Haswell 桌面/移动核），放到一起只用于说明**层次与一致性状态的"延迟梯度"**。

- **数值算例 1（一致性如何抬高 AMAT）**：设某负载的访存分布为：L1 命中 90%，L2 命中 7%，L3 未共享命中 1.5%，L3 共享 0.6%，L3 Modified 0.3%，本地 DRAM 0.6%。

```
单处理器版（假设所有 L3 命中都按"未共享"的 40 cycle 计）:
  AMAT_uni = 0.90x4 + 0.07x10 + 0.024x40 + 0.006x120
           = 3.60 + 0.70 + 0.96 + 0.72
           = 5.98 cycles

多处理器版（把 0.9% 的 L3 命中区分成"共享 65" 与 "Modified 75"）:
  AMAT_mp  = 0.90x4 + 0.07x10 + 0.015x40 + 0.006x65 + 0.003x75 + 0.006x120
           = 3.60 + 0.70 + 0.60 + 0.39 + 0.225 + 0.72
           = 6.235 cycles
  => 一致性与共享使 AMAT 上升约 4.3%
```

看起来微不足道？把"被别的核以 Modified 持有"的比例从 0.3% 提到 **5%**（典型的伪共享/真共享热点场景）。 为保证各类占比之和仍为 1, 同步下调 L1 与 DRAM 的占比, 给出第二组分布: L1 85%, L2 7%, L3 未共享 1.5%, L3 共享 0.5%, L3 Modified 5%, 本地 DRAM 1%

```
  AMAT_mp' = 0.85x4 + 0.07x10 + 0.015x40 + 0.005x65 + 0.05x75 + 0.01x120
           = 3.40 + 0.70 + 0.60 + 0.325 + 3.75 + 1.20 = 9.975 cycles
与上面 6.235 cycle 相比上升 60%，而 CPI 中访存部分的贡献几乎翻倍。
```

结论：讲义 slide 48 的注释"only a fraction of a % of these can be significant"（CS149 亦有同样提醒）——一致性缺失**只要占到访问的百分之几**，就能显著改变 AMAT；因此**用 VTune 之类的工具测量 cache miss 与一致性流量**（CS149 slide 40）比凭直觉猜测更重要。

4.2 snooping 的可扩展性：广播的消息复杂度（讲义 slide 52 的结论）

- **模型**：设系统有 `N` 个 cache，每个核每秒产生 `m` 次需要互连的一致性事务（缺失/升级）。snooping 要求**每次事务都广播给所有其他 cache**，因此
- **互连上的事务数**（占用网络的事件数）：`R_txn = N × m`
- **需要被投递的一致性消息数**（每个事务要被 `N-1` 个 cache 接收）：`R_msg ≈ N × m × (N−1) ≈ N² m`
- 目录方案：事务只**点对点**发给该行的持有者（平均 `s` 个 sharer），`R_msg ≈ N × m × s`，与 `N` 成**线性**而非平方关系。
- **数值算例 2（为什么 64 核时广播就撑不住了）**：

```
取 N = 64 个核，每个核每秒 m = 1e7 次需要互连的事务（即 10 M miss/s/核）：
  snooping 消息投递率  = N^2 x m = 64 x 64 x 1e7 = 4.096e10 条/秒
      -> 即使每条消息只占 8 B，也需 3.3e11 B/s = 328 GB/s 的互连"消息投递"带宽，
         而这还没算数据本身（一次 BusRd 要搬 64 B）。
  directory 消息投递率 = N x m x s，取 s = 2 => 64 x 1e7 x 2 = 1.28e9 条/秒（3.2% 于前者）

再看数据带宽：每次一致性缺失平均搬 64 B：
  snooping/directory 的"有效数据带宽"需求 = N x m x 64 B = 64 x 1e7 x 64 = 4.1e10 B/s = 41 GB/s
---- 41 GB/s 的数据带宽已经很吃力，但真正致命的是上面那条 O(N^2) 的"消息投递"账。
```

- **物理限制（讲义 slide 52 的收尾 + 前一讲互连网络的结论）**：除了消息数，广播还要求**一条能给所有 cache 送消息的物理介质**（总线或等价的全连接/环上绕一圈）。总线有电气负载上限（节点越多频率越低），环上广播要绕 `N` 跳。因此 snooping 的实际上限通常在**几十个核**的量级；**再往上就必须用目录**——这就是下一讲的主题。
- **工程现实（讲义 slide 40–41 的层次化折中）**：注意 Intel 的做法是**混合的**：L2 之间并不是纯广播——L3 是 inclusive 的、并充当**目录与串行化点**（CS149 slide 37 明确写出"L3 serves as centralized directory for all lines in the L3 cache"，且"Core i7 的互连是 ring，不是 bus"）。所以真实的"监听式"实现只在**较小的核数/较近的层次**上是真广播。

4.3 false sharing 的定量下界（本笔记推导）

- 设 `T` 个线程，每个线程在自己的**变量**上做 `MANY` 次写，但所有变量落在**同一条 64 B cache line** 上。
- **一致性流量下界（填充版）**：`B_pad = T × 64 B`（每个变量只有一次义务性缺失；若数组很大还有容量缺失，但都只发生一次）。`T = 12` 时 `B_pad = 768 B`。
- **一致性流量上界（未填充版，最坏情形）**：每次写都可能要求把该行搬到写者手里，故 `B_unpad ≈ T × MANY × 64 B`（还可能有对称的 flush 流量，即再 64 B）。
- **时间下界**：`S' ≥ (T × MANY) × c_xfer`，`c_xfer` 取一致性"独占权转移"的延迟量级（用 4.1 表的"L3 Modified 在别的核" = 75 cycles，加上排队/串行化开销，保守取 **100–130 cycles**）。

```
数值算例 3: T = 12, MANY = 1e7, 主频 3.0 GHz, 取 c_xfer = 128 cycles
  (a) 未填充版的时间下界(完全串行化假设):
      S' >= 12 x 1e7 x 128 cycles = 1.536e10 cycles = 5.12 s @3GHz
      ---- 与讲义 slide 48 实测的 5.1 s 同一量级，说明"每次写都抢一次独占权"
           这个最坏假设，在这段代码上其实相当接近现实。
  (b) 填充版如果真是纯本地 L1 更新（c_local ~ 1 cycle 的吞吐）:
      1e7 x 1 cycle = 1e7 cycles = 3.3 ms（每个线程），12 线程并行 -> 总时间 ~3.3 ms
      ---- 但讲义实测是 2.1 s，说明真实瓶颈是 volatile 造成的 load->add->store
           串行依赖链（每轮十几 cycle）以及每核有限的访存吞吐，而不是一致性。
  (c) 因此 "5.1 s / 2.1 s = 2.4x" 这个比值**低估**了 false sharing 的代价:
      它测到的是"两个不同的瓶颈哪个先撞墙"，而不是"一致性流量相差多少"。
      真正的量级差异在**流量**上：未填充版 7.68 GB vs 填充版 768 B（见下）。

一致性流量对比（MANY = 1e7, T = 12, 64 B/行）:
  未填充版: 12 x 1e7 x 64 B = 7.68e9 B = 7.68 GB
  填充版  : 12 x 64 B        = 768 B
  比值    : 1e7 倍
把 7.68 GB 放到一条 20 GB/s 的互连上：至少 0.384 s 纯粹用于搬"没用的伪共享数据"；
若每次转移都暴露完整延迟(128 cycles)，则是上面算出的 5.12 s。
```

- **结论**：伪共享的危害要用**流量（占带宽、耗能量）与延迟（串行化）** 两把尺子衡量；而"墙钟只慢 2.4 倍"往往是因为**修复后的版本撞上了另一个瓶颈**，不构成"伪共享无害"的证据。

4.4 MSI vs MESI 的收益量化（本笔记推导）

- **模型**：设负载中有 `K` 次"**首次触摸**一条私有行：先读它、随后写它"的模式。MSI：每次 2 次事务（`BusRd` 使 I→S，`BusRdX` 使 S→M）；MESI：每次 1 次事务（`BusRd` 直接进 E，再 `PrWr` 时 E→M 无需总线）。**节省比例 = 50% 的该类事务**。注意关键前提是"**首次触摸**"：一行一旦已经在 M 态（或 MESI 下的 E 态），后续的读和写都是本地命中，两种协议都不再产生事务。
- **数值算例 4**：12 个核各自独占一块**互不共享**的私有数据（例如每核 10 万条 64 B 的行），各做一次"读后写"（读进来、改一个值），即共 `12 × 10^5 = 1.2e6` 条首次触摸：

```
MSI : 每条首次触摸 2 次事务 (BusRd 进 S, 再 BusRdX 升级到 M)
      -> 1.2e6 x 2 = 2.4e6 次互连事务
MESI: 每条首次触摸 1 次事务 (BusRd 直接进 E, 随后的 PrWr 由 E->M 零事务完成)
      -> 1.2e6 x 1 = 1.2e6 次互连事务
      节省 1.2e6 次事务 = 50%
按每次事务至少搬 64 B 计:
      MSI  流量 = 2.4e6 x 64 B = 153.6 MB
      MESI 流量 = 1.2e6 x 64 B =  76.8 MB
在 20 GB/s 互连上: 7.68 ms  vs  3.84 ms
```

3.2 节模拟器 `count` 模式跑的是这个负载的"同一行反复读写"版本（12 核 × 10 万轮），它的实测输出正好印证了上面的前提：**MSI 只有 24 次互连事务（= 2 × 12，每核首次触摸 2 次），MESI 只有 12 次（= 1 × 12），后续 99.99% 的访问都是本地命中（MSI 本地命中 2399976 次，MESI 2399988 次）**——比值仍是 2:1，但绝对值远小于 `1.2e6`，因为协议只在"行的所有权发生变化"时才付费。

再叠加讲义 slide 30/31 那 12 步时序（3.2 节模拟器可直接复现的计数）：**MSI = 10 次互连事务（5 次 BusRd + 5 次 BusRdX，其中 4 次带 flush），MESI = 9 次**——省的正是第 11 步 `P0: ST Y←1`（P0 处于 E 态，E→M 零事务）。而讲义 slide 36/37 那 9 步时序（P0/P1 对 Y 的访问顺序不同）是 **MSI 8 次 vs MESI 7 次**。这说明 **MESI 的收益完全取决于负载中"读后写私有数据"的比例**：对 `T` 个线程各写自己的局部累加器这种模式，MESI 几乎白拿 50% 的一致性流量削减；而对 3.1 节的伪共享模式，**E 态完全帮不上忙**（因为行一旦被第二个核读过就变 S，再也回不到 E，每次写仍然要 `BusRdX`）——所以**"用 padding 消除伪共享"与"靠 MESI 省事务"是两件互补而非替代的事**。

4.5 用 Amdahl 定律描述"一致性把并行部分串行化"

设程序总时间中，因为一致性问题（真共享锁、原子热点、伪共享行弹跳）而必须**串行执行**的比例为 `f`，其余部分可完美并行：

```
Speedup(T) = 1 / (f + (1 - f)/T)

数值算例 5:
  f = 0.05 (5% 串行): T = 12 -> 1/(0.05 + 0.95/12) = 1/0.1292 = 7.74x
  f = 0.30 (30% 串行): T = 12 -> 1/(0.30 + 0.70/12) = 1/0.3583 = 2.79x
  f = 0.30, T -> 无穷         -> 3.33x  (无论加多少核都不可能超过 3.33x)
  f = 0.01, T = 12            -> 1/(0.01+0.99/12) = 10.8x

与伪共享的对应关系: 3.1 节的"未填充"版本相当于 f -> 1 (整条行是串行资源),
于是 Speedup -> 1;  而"填充"版本把 f 降到接近 0, Speedup -> T。
```

- **与 Work-Span 的关系**：`f` 就是把 `Span` 中的串行分量折算到总时间上的结果。**降低 `f` 的手段**在本讲里有三个：(1) **消除伪共享**（padding / 分块对齐 line）；(2) **私有化 + 归并**（把真共享的写变成私有写，代价是 `O(T × 状态量)` 的额外工作量与空间）；(3) **改进协议**（MESI 消除 upgrade 事务、MOESI/MESIF 减少写回与响应者选择开销）——(3) 是硬件设计师的工作，而 (1)(2) 是程序员的工作。

4.6 Roofline 视角：算术强度（arithmetic intensity）与"一致性带宽"

- **机器算力上界**：取 12 核、3.0 GHz、每核每 cycle 可做 8 个单精度 FMA（AVX2 = 8 float/向量 × 2 flop）：

```
峰值算力 = 12 核 x 3.0e9 cycle/s x 8 float/FMA x 2 flop = 576 GFLOPS
(同一算式在 16 核双路、8 宽 SIMD、FMA 上就是常见的 768 GFLOPS)
```

- **带宽上界**：以 20 GB/s 作为"内存 + 一致性流量"合起来可用的带宽（讲义时代的单路 DRAM 量级），则 Roofline 的**拐点算术强度**为

```
I_knee = 峰值算力 / 带宽 = 576 GFLOPS / 20 GB/s = 28.8 flop/byte
```

也就是说，**每搬 1 字节数据必须做 28.8 次浮点运算**才能让这台机器进入计算受限区。

- **算例 6（伪共享把程序钉在最左侧）**：
- **伪共享自增**：每次自增搬进/搬出约 64 B 的 cache line，算术量约 1 次整数加法 = `1 flop / 64 B ≈ 0.016 flop/byte` —— 比拐点小 **1800 倍**，完全落在**极度带宽/延迟受限**区（在 Roofline 图上贴在最左端的水平线以下）。

- **一次 256 MB 数组的流式遍历**：若每元素 1 flop（8 B），强度 `0.125 flop/byte`，仍是带宽受限；单次遍历的数据量 256 MB，在 20 GB/s 下

```
时间下界 = 256 MB / 20 GB/s = 0.256 GB / 20 GB/s = 12.8 ms
而同样的数据若能在 200 GB/s 的 L3 带宽里循环（12 核共享 8 MB L3 不现实，这里假设数据能放下）：
时间下界 = 0.256 GB / 200 GB/s = 1.28 ms （快 10 倍）
```

- 这就是为什么"**先优化访存、再优化算力**"是并行程序的第一原则；而本讲的一致性话题正是"访存优化"里最容易被忽视的一类——它既不增加有用流量，也不减少算法的工作量，却实实在在地吃带宽、占延迟。
- **一个实用的设计准则**：如果一个算法每处理 1 个元素就要写一次**被多个线程共享**的位置（或落在别人也在写的行上），那么无论用多少核，它的吞吐都由"**每个 cache line 每秒能被转移多少次**"决定。用 4.1 的延迟数据粗算：一次独占权转移约 75–130 cycles，即**单条行大约 3–8 ns 一次**（`3e9/100 ≈ 3e7 次/秒`）。**这就是"一致性带宽"的硬上限**，也是为什么任何热点都必须被私有化。

4.7 小结：本讲涉及的时间/带宽/空间三类成本

成本类型	由谁决定	典型量级（讲义/CS149 数据）	降低手段
延迟（latency）	互连 + cache 层次 + 一致性状态	L1 4–6 cyc；L2 12 cyc；L3 未共享 26–31 cyc（Xeon 5500 约 40 cyc）；L3 Modified 在他核 75 cyc；本地 DRAM 120 cyc；远端 DRAM 400 cyc	cache-to-cache 传输、MESIF/MOESI 减少写回与响应者竞争、inclusion 让 L1 不必监听
带宽（bandwidth）	互连 + 内存 + 一致性事务量	每次 `BusRd` / `BusRdX` 搬 64 B；snooping 的消息投递量 $O(N^2)$	MESI 消除 upgrade 事务（–50% 于读写私有数据）；目录代替广播；padding 消除伪共享；私有化消除真共享
存储开销（area）	协议状态位 + 目录 + 私有副本	每行 2 bit（MSI）到 3+ bit（MESI/MOESI）；L2 每条 line 另加 `in L1` 与 `modified-but-stale` 位；私有 bin 数组 `8 B × B × T`	协议状态编码压缩；目录可只对"可能被共享"的行建立（稀疏目录）；私有副本大小与 cache 容量的权衡

---

5. 关键要点

1. **一致性问题的根源不是软件 bug，而是硬件为了性能复制数据**。它不可能用加锁、也不能用"更小心的同步"来消除：只要存储被分散在内存与各核私有 cache 中，"读 X 应返回最后一次写 X 的值"就必须由硬件在**每一次 load/store** 上保证。一致性的精确定义是"每个地址存在一个所有处理器都认可的串行顺序"（等价的三条：遵守 program order、写传播、写串行化；等价的两条不变量：SWMR + Data-Value）。
2. **snooping 的全部机制就一句话：一致性相关的 cache 操作被广播给所有 cache controller，每个 controller 同时服务本地 CPU 与互连消息，靠"彼此合作"维持不变量**。它不需要全局控制器，也不需要额外存储；代价是**广播本身**——消息投递量随核数按 `O(N²)` 增长，因此 snooping 的可扩展性上限就是"能把一致性消息广播给多少 cache"，再往上是目录方案（下一讲）。
3. **协议演进的主线是"消除不必要的事务"，而不是"让单次事务更快"**：write-through → write-back 消除每次写的内存流量；MSI → MESI 用 **E（独占干净）** 消除"私有数据先读后写"的 upgrade 事务（本例中一致性流量减半）；MESIF 用 **F** 让共享行的 miss 只有唯一响应者；MOESI 用 **O** 让 M→S 不必写回内存。**M 态是"零总线事务的写"的唯一合法来源**——记住"进入 M 必须经过互连，除非已经在 M 或（MESI 下）在 E"。
4. **一致性把硬件的实现细节暴露到了软件的性能上**：**false sharing（伪共享）** 是"不同地址、同一 cache line"，它带来的是**完全没有必要的通信（artifactual communication）**；**true sharing（真共享）** 的同步是必要的，但**不应该出现在热路径上**。编程准则：**每个可写共享量至少占一条 cache line**（`alignas(64)` / padding / `schedule(static, chunk)`），**共享写热点一律私有化后归并**（每线程局部累加 / 私有 bin + merge），**热点原子操作必须被规约或分片替代**。
5. **优化要有层次：先消除一致性造成的串行化，再面对带宽与算力的墙**。用 work-span 的语汇看：伪共享/真共享把 `Span` 拉长到 `O(总工作量 × 一次 line 转移延迟)`，使**并行度降到 1 以下**（加核无用）；消除之后，瓶颈才回到内存带宽（算例：256 MB 一遍至少 12.8 ms @20 GB/s；Roofline 拐点强度 28.8 flop/byte），再之后才轮到算力（12 核 × 3 GHz × 8 FMA × 2 = 576 GFLOPS）。

---

6. 常见陷阱与注意事项

- ❌ **以为"加锁就能解决一致性问题"**。讲义 slide 7 明确回答：**不能**。一致性问题源于**数据被复制**这一实现细节，与"两个线程是否同时访问"无关；即使程序里根本没有任何数据竞争（例如 P1 先写、很久之后 P2 才读），没有硬件一致性时 P2 依然可能读到陈旧副本。加锁只能保护**用锁保护的临界区内的数据**，而共享的 flag、指针、work queue 本身都要靠硬件一致性与内存序来保证。

- ❌ **把 coherence 与 consistency 混为一谈**。coherence 只管**单个地址**上的读写顺序（"同一地址存在一个全局串行顺序"），它是**逐地址（per-location）**的；而 memory consistency model 管的是**不同地址之间**的顺序（例如"我写 A 之后写 B，别的核能否先看到 B 再看到 A"）。本讲的 MSI/MESI 全部满足 coherence，但**完全不保证**任何跨地址的顺序——这正是课程后面单独讲 memory consistency、并在实现里使用 `fence` / `mfence`、`std::atomic` 的 `memory_order` 的原因。
- ❌ **只看墙钟比值，就断定伪共享"影响不大"**。讲义 slide 48 的 5.1 s → 2.1 s 看起来只有 2.4 倍，但那是因为**填充后的版本撞上了另一个瓶颈**（`volatile` 造成的 load→store 串行依赖链）。真实差距在**一致性流量**上：`T × MANY × 64 B`（未填充，`MANY = 10^7` 时是 7.68 GB）vs `T × 64 B = 768 B`，相差 `MANY = 10^7` 倍。判断伪共享必须同时看**流量、延迟、可扩展性**。
- ❌ **用"每个线程一个元素"来划分共享数组，却忘了 cache line 是 64 B**。`int myCounter[NUM_THREADS]` 这种写法里，12 个 4 B 的计数器全在同一条行上；`schedule(static)` 默认的块大小也可能让两个线程的边界落在同一行上（应使用 `schedule(static, chunk)` 且 `chunk ≥ cache_line / sizeof(T)`，或让每个线程处理对齐的整行块）。**修复时要注意 padding 本身的开销**：把 `T` 个 4 B 计数器各扩到 64 B 是 16 倍空间放大，若数组很大（如每线程一个 10^6 元素的私有数组），应改为"按行对齐的连续块划分"而不是逐元素 padding。
- ❌ **认为 `volatile` 或原子操作能"绕过一致性问题"**。`volatile` 在 C/C++ 里**只保证编译器不优化掉访问**，它不提供原子性、不提供内存序、更不会让别的核的副本失效（在 x86 上它生成的仍是普通的 load/store）。`#pragma omp atomic`、`std::atomic`、`__sync_fetch_and_add` 保证的是**操作的原子性**（必要时通过互连上的 `lock` RMW 或 L2 中的原子单元实现），但**热点上的原子操作仍然是串行的**：3.3 节的直方图与 3.4 节的求和都表明，"原子"解决正确性，"私有化 + 归并"才解决性能。
- ❌ **忽略"数据的权威副本可能不在内存里"**。write-back + M 态意味着**内存里的值可能是陈旧的**，必须由 owner 提供数据（这在课程项目里最常表现为"用 DMA/另一个设备读共享缓冲区要小心"、"单 CPU 系统的 I/O 也需要一致性处理"——讲义 slide 11 给出了三种做法：uncached store、把页标记为不可缓存、I/O 完成时显式 flush）。同理在 GPU 上，`cudaMemcpy` 与 kernel 边界之外的可见性依赖驱动清 L1，而**在 kernel 内部跨线程共享数据必须用 `__syncthreads()`、`ld.cg` /atomic 或共享内存**，不能指望 L1 帮你保持一致。
- ❌ **以为 L1 会自动包含在 L2 里，或以为"L2 比 L1 大"就自动满足 inclusion**。讲义 slide 41 的反例说明：即使 L2 是 L1 的两倍、相联度与替换策略相同，**A、B、C 映射到同一 set 且访问历史不同**，两级 cache 也会逐出不同的行，从而**破坏 inclusion**。真实机器必须显式维护（L2 逐出时反向失效 L1），并需要 `in L1` 与 `modified-but-stale` 两个额外状态位；而 `modified-but-stale` 的存在意味着**"L2 显示该行为 M"并不等于"L2 里有最新数据"**——flush 时还得先去 L1 取真数据。
- ❌ **忘记协议必须保证 "write serialization"，只关注"失效有没有发出去"**。讲义 slide 15 的反例（P3 看到 a→b，P4 看到 b→a）说明：即使每次写都广播了失效，**若不同 cache 观察到的写顺序不同，一致性依然被破坏**。所以互连必须提供"所有写事务对所有 cache 可见，且可见顺序相同"这两条保证——这也是为什么总线/环上的**串行化点**（Intel 的 L3 slice 就充当这个角色）在实现里如此重要。

---

7. 思考题（带答案）

思考题 1：MSI 状态推演与"为什么 S→M 必须发总线事务"

设 X 初值为 0，只有 P0、P1 两个核，cache 均实现 MSI 协议，初始两条行都是 I。执行序列为：`P0: LD X → P1: LD X → P1: ST X←7 → P0: ST X←9 → P1: LD X`。请 (a) 逐步给出两个 cache 的状态与互连事务；(b) 统计 `BusRd`、`BusRdX`、flush 的次数；(c) 解释为什么第 4 步 P0 的写**必须**发 `BusRdX`，即使 P0 的 cache 里此时可能仍然"有效"。

【答案】 (a) 逐步推演（列格式 `状态/值`）：

步	动作	P0 的 X	P1 的 X	互连事务	说明
0	初始	I	I	—	内存 X = 0
1	P0: LD X	**S/0**	I	BusRd	P0 缺失，从内存取整行；没有其他 cache 持有 → MSI 没有 E 态，所以还是 S
2	P1: LD X	S/0	**S/0**	BusRd	P0 在 S 态，不提供数据（内存是新的），P1 从内存取
3	P1: ST X←7	**I**	**M/7**	BusRdX	写缺失：P1 发 `BusRdX` 取得独占权，**P0 的副本被作废**
4	P0: ST X←9	**M/9**	**I**	BusRdX + **flush**	P0 此时是 I（第 3 步已被作废），必须重新取得独占权；P1 处于 M 态，**必须先 flush 脏数据**（内存变 7）并把副本作废
5	P1: LD X	**S/9**	**S/9**	BusRd + **flush**	P0 处于 M 态，必须 flush（内存变 9）并降级为 S，P1 从 P0 的 cache 或内存处取到 9

(b) 计数：`BusRd` = **2** 次（第 1、5 步），`BusRdX` = **2** 次（第 3、4 步），flush = **2** 次（第 4、5 步）。**5 条内存指令触发 4 次互连事务 + 2 次 64 B 写回**——这就是"交替读写同一地址"的一致性开销。

(c) 第 4 步必须发 `BusRdX` 有两个层次的理由： 1. **本例子中 P0 的副本已经无效**（第 3 步被 P1 的 `BusRdX` 作废），所以这是普通的写缺失，必然要取数据 + 取得独占权。 2. **更本质的规则**：MSI 里"**处理器只能写处于 M 态的行**"，而进入 M 的唯一路径是 `PrWr` 且当前不是 M 时发 `BusRdX`。因此即使 P0 当时**在 S 态**（即"副本有效"），它也**不能**直接本地改写：S 态意味着别的 cache 里可能仍有副本，"有效"不等于"独占"。若不发 `BusRdX` 就本地写，就会出现两个 cache 同时持有同一行的不同值，违反 SWMR 不变量；而且没有任何机制能让未来的读者知道该向谁要正确数据（write serialization 也无法在互连上给这次写定序）。这正是 MESI 用 **E 态** 要解决的问题：E 态下"独占"与"干净"同时成立，此时 `PrWr` 才能做到 **E→M 零总线事务**。

思考题 2：用讲义数字量化一次真实事故

某 12 核服务器上（12 核共享一条互连，主频 3.0 GHz，一致性粒度为 64 B 的 cache line，未共享的 L3 命中约 40 cycles、数据被别的核以 Modified 持有的 L3 命中约 75 cycles），一个程序里每个线程把结果累加到 `int result[12]` 的**自己那个元素**上，全程约 **2 × 10^8 次**累加（总计，含所有线程）。请 (a) 判断问题类型；(b) 估算一致性流量与"因一致性而额外增加的周期数"；(c) 给出两种修复方案并说明各自的 Work/Span 变化与副作用。

【答案】

(a) **问题类型：false sharing（伪共享）**。12 个 `int`（4 B）共 48 B，**全部落在同一条 64 B cache line 内**。虽然每个线程写的是**不同地址**（没有数据竞争，结果的正确性不依赖任何同步），但这 12 个地址共享同一条 line，因此 12 个核会反复争夺该行的**独占权（M 态）**。这是典型的 **artifactual communication（人为通信）**——程序在语义上完全不需要任何通信。

(b) 定量估算： – 最坏情形（每次累加都要求把该行搬到写者手里）：**一致性事务数 ≈ 2 × 10^8**（与累加次数同阶）。 – **流量**：每次转移至少搬 64 B，`2e8 × 64 B = 1.28 × 10^10 B = 12.8 GB`（若还有对称的脏行写回，翻倍到 ~25.6 GB）。 – **额外周期数**：用 75 cycles（L3 命中但行被别的核以 Modified 持有）作为每次转移的**延迟下界**，`2e8 × 75 = 1.5 × 10^10 cycles`，在 3.0 GHz 上约 **5.0 s**；若把行转移的排队/串行化开销算进去（按 100–130 cycles），约 **6.7–8.7 s**。 – 对照：若换成填充版本，一致性流量只有 `12 × 64 B = 768 B`，额外周期数可忽略，时间由每核的本地累加吞吐决定（`2e8` 次加法 / 12 核 ≈ 1.7e7 次/核，几毫秒量级）。**即使保守地把填充版按"每轮 10 cycles 的串行依赖链"估**，也远小于上面估算的秒级代价——这正是讲义 slide 48 实测 5.1 s vs 2.1 s 的同一现象。

(c) 两种修复方案： – **方案 A：padding / 对齐到 cache line**。把 `int result[12]` 改成 `struct { int result; char pad[64 - sizeof(int)]; } __attribute__((aligned(64))) result[12];`（C++ 用 `alignas(64)`）。**Work 不变**（仍是 2e8 次累加），**Span 从 `2e8 × c_xfer` 降到 `2e8/12 × c_local`，并行度从 ≪1 回到 ≈12**。副作用：内存放大 16 倍（此处仅 768 B，可忽略；若每线程是 KB 级数组则不可忽略），以及最后仍需一次**归并**把 12 个计数器相加（`O(T)`，可忽略）。 – **方案 B：OpenMP `reduction(+:total)`（编译器的私有化 + 归并）**。语义上把"12 个独立计数器"换成"每线程一个私有累加器 + 最后归并"。**Work = 2e8 次加法 + O(T) 归并，Span = 2e8/12 × c_add + T × c_add**，并行度 ≈ 12。副作用：需要额外 `T × 64 B` 的私有存储（编译器通常已经按 cache line 对齐，避免自身产生伪共享）；若累加对象不是标量而是数组，则要手写"私有数组 + 归并"（3.3 节的做法）。 – **注意**：两种方案**不能**用"加锁"代替——这里根本没有竞争，加锁只会把本来可并行的写变成串行临界区，让性能更差（而且它掩盖不了"行弹跳"本身，因为临界区内的写仍然要抢独占权）。

思考题 3：为什么编写并行程序时"两个线程写相隔 8 字节的两个变量"会变慢，而"两个线程读同一份只读数据"却不会？

请从一致性协议的状态迁移（MSI/MESI）出发解释，并说明这与"true sharing"和"false sharing"的关系；最后给出一个判断准则，让程序员在写代码时能**一眼**看出某处是否存在伪共享风险。

【答案】

- **两个线程读同一份只读数据：不会变慢（除了 caches 之间可能的一次数据迁移）。** 在 MSI/MESI 下，第一个读者把该行带到 **E**（MESI，若无人持有）或 **S**；第二个读者发一次 `BusRd`，把行变成 **S**。**S 态是"多读者只读纪元"**（SWMR 不变量里的 Read-Only epoch），此后所有读者的读都**本地命中原 S 副本，零互连事务**——共享是"只读的"，协议不需要任何进一步协调。唯一的成本是第一次的 `BusRd`（以及可能的 cache-to-cache 传输，其延迟比本地 DRAM 更低，见 CS149 的 65/75 vs 120 cycles）。这正是讲义 slide 19 强调"**共享 cache 也便于细粒度共享**"、以及真实负载常常受益于共享只读数据的根本原因。
- **两个线程写相隔 8 字节的两个变量：会大幅变慢。** 因为两者**落在同一条 64 B cache line** 内，而"写"要求**独占（M）**：写者 A 把该行升到 M，写者 B 要写时必须发 `BusRdX`，让 A 的副本作废并把行搬到自己这里（M 态迁移），A 再写时又反向抢回来。每次写都伴随一次 **cache line 的 ping-pong（弹跳）**——这就是 **false sharing（伪共享）**：**地址不同（没有真共享）、行相同（协议层面当成共享）**。注意它与 **true sharing（真共享）** 的区别：
- **true sharing**：两个线程访问**同一个地址**（例如同一个计数器、同一把锁）。此时"必须串行化"是**语义要求**，同步不可省略——问题在于**不该让它进热路径**，解法是**私有化 + 归并**（3.3、3.4 节）。
- **false sharing**：两个线程访问**不同地址**，却因为 line 太大而被迫串行。此时串行化**纯属浪费**，解法是让人工的**内存布局**匹配硬件的粒度：**padding / 对齐 / 按行分块**。
- 一个有用的判据：**MESI 的 E 态对伪共享毫无帮助**（行一旦被第二个核读过就变 S，回不去 E，每次写仍需 `BusRdX`）；而 E 态对"读写私有数据"帮助巨大（省一半事务）。所以"某个优化能不能救你"取决于你面对的是哪一类问题。
- **一眼判断伪共享风险的准则**（三条，按顺序检查）：

- **写者是否多于一个？** 只有一个线程写这段数据（其余只读）→ 没有伪共享，最多是一次 `BusRd` 的迁移成本。**没有写者就没有弹跳。**
- **多个写者写的地址是否落在同一条 cache line 内？** 对 `sizeof(T)` 小于 64 B 的元素数组、以及"每个线程一个计数器/累加器/锁/标志位"这类**紧挨着排布的小对象数组**，答案默认是"会"——尤其要注意 `struct` 里相邻的字段、`bool` / `int` 标志数组、以及 `std::vector<Node>` 里被不同线程写的相邻元素。判据是**字节偏移**：`addr / 64` 是否相同。
- **这些写是否在热路径上（频率高）？** 即使布局上共享一条行，若写只发生几次（初始化、收尾），成本可以忽略；**只有高频写才需要处理**。若三者同时成立，就必须：把每个高频写者**对齐到至少 64 B**（`alignas(64)`、`char pad[64 - sizeof(x)]`、`schedule(static, 16)` 分块），或者干脆把"共享写"改造成"私有写 + 归并"。判断是否修好了，最好的手段是**测量**（VTune 之类的硬件计数器、或直接对比填充前后的墙钟与一致性流量），而不是靠直觉——这也是讲义 slide 48 给出一段可运行 demo 的用意。

Lecture 12: Directory-Based Cache Coherence

1. 章节标题与概述

Lecture 12: Directory-Based Cache Coherence（基于目录的缓存一致性）

- **本讲核心问题**：上一讲的**监听式一致性（snooping-based coherence）**把每一次 cache miss 的一致性信息**广播（broadcast）**给所有 cache，这在"一条共享总线 + 4 个核"上又便宜又简单，但在 64、256、1024 个节点的机器上，**广播本身成了不可扩展的瓶颈**。本讲要回答三件事：(1) **广播到底卡在哪里**（总线争用、带宽不随节点数增长、电气负载、以及"就近访问内存"的 NUMA 好处被"仍要全网广播"抵消）；(2) **目录（directory）如何用点对点消息取代广播**——目录项记录"这一行现在被哪些 node 缓存、是否有脏副本"，miss 只发给**需要知道的节点（need to know）**；(3) 目录自己带来的两个新问题怎么解——**目录存储开销**（full bit vector、limited pointer schemes、sparse directories）与**消息数/关键路径（critical path）**（intervention forwarding、request forwarding），最后落到真实芯片（Intel Core i7 的 L3 目录、多 socket 的 home agent + in-memory directory）与 cc-NUMA 上的软件行为。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：**目录项（directory entry）** = `P` 个 **presence bit（存在位）** + 1 个 **dirty bit**；**home node（主节点）** 与 **requesting node（请求节点）** 的角色划分；目录分区与内存**同址部署（co-located）**的**分布式目录**；三种消息序列（read miss 到 clean line / read miss 到 dirty line / write miss 的"失效 + 应答"四步）；cache-to-cache transfer（cache 到 cache 直接传数据）；**intervention forwarding** 与 **request forwarding** 两种转发策略对关键路径的影响；limited pointer（$1 + k \cdot \log_2 P$ 位）与 **sparse directory（稀疏目录，链表串在 cache line 的 tag 里）**；Intel Core i7 的 **L3 兼作集中式目录**（依赖 **inclusion property（包含属性）**）与 **环形互连（ring interconnect）**；多 socket 的 **home agent / cache agent / memory controller / QPI / 16 KB dir cache**。
- **软件侧**：程序员写不出目录，但**访问模式决定了目录记不记得住**：migratory object（迁移型对象）、mostly-read object（多读少写）、频繁读写对象、高低争用锁，四种模式对应完全不同的目录压力；在 cc-NUMA 上，**数据落在哪个 node（first touch / 页分配）** 与 **线程跑在哪个 node（亲和性 affinity）** 共同决定了 3~4 跳的远程延迟要不要付；**伪共享（false sharing）** 会把一条 64 B 行变成跨 node 的失效链。
- **在并行计算知识体系中的角色**：本讲是"缓存一致性"这条线的收尾与**可扩展性转折点**：上一讲给出 MSI/MESI 这套**协议语义**，本讲给出把它**扩展到大规模多核/多 socket** 所需要的**数据结构（目录）与消息路径（点对点 + 转发）**。它同时是后面几讲的前置知识：**同步（synchronization）** 讲的 test-and-set、ticket lock、以及"锁释放时一堆读者都在场"的高争用问题，本质就是**目录里的 sharer 列表长度**问题（讲义 slide 21 明确点出）；**内存一致性（memory consistency）** 需要一个"写入何时对谁可见"的定序机制，而目录提供了这个定序的物理载体；**互连网络（interconnection networks）** 讲的所有拓扑（ring / mesh / fat tree）正是目录协议跑的"路"；**消息传递实现（under the hood: message passing）** 则是同一套 3 跳/4 跳消息路径在软件层的翻版。
- **配套材料**：
- `lectures/11_directorycoherence.pdf`（抽取文本 `extracted/11_directorycoherence.txt`，共 **47 页 / 约 25 KB**）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 公开下载。讲义首页写的是 **"Lecture 11: Directory-Based Cache Coherence"** 与 **"CMU 15-418/15-618, Fall 2024"**——这是讲义**沿用历史学期版本**的正常现象（讲次编号与学期字样随年度重排），不是错误。按 Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html），本讲排在 **Sep 21**，是第 12 讲，主题即 **Directory-Based Cache Coherence**；上一讲（Sep 18）是 Snooping-Based Cache Coherence，对应讲义 `lectures/10_cachecoherence.pdf`。本笔记的术语（presence bit、dirty bit、home node、intervention/request forwarding、limited pointer、sparse directory）、示例（read miss 到 clean/dirty line、write miss 的失效与应答、64 节点 Barnes-Hut / LU / Ocean 的 sharer 直方图）与数字（12% / 50% / 200% 存储开销、5.8% / 7.8% / 9.7% 的 limited pointer 开销、1 MB cache 对 1 GB 内存的 99.9% 空目录项）全部以这份 47 页讲义为准。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。

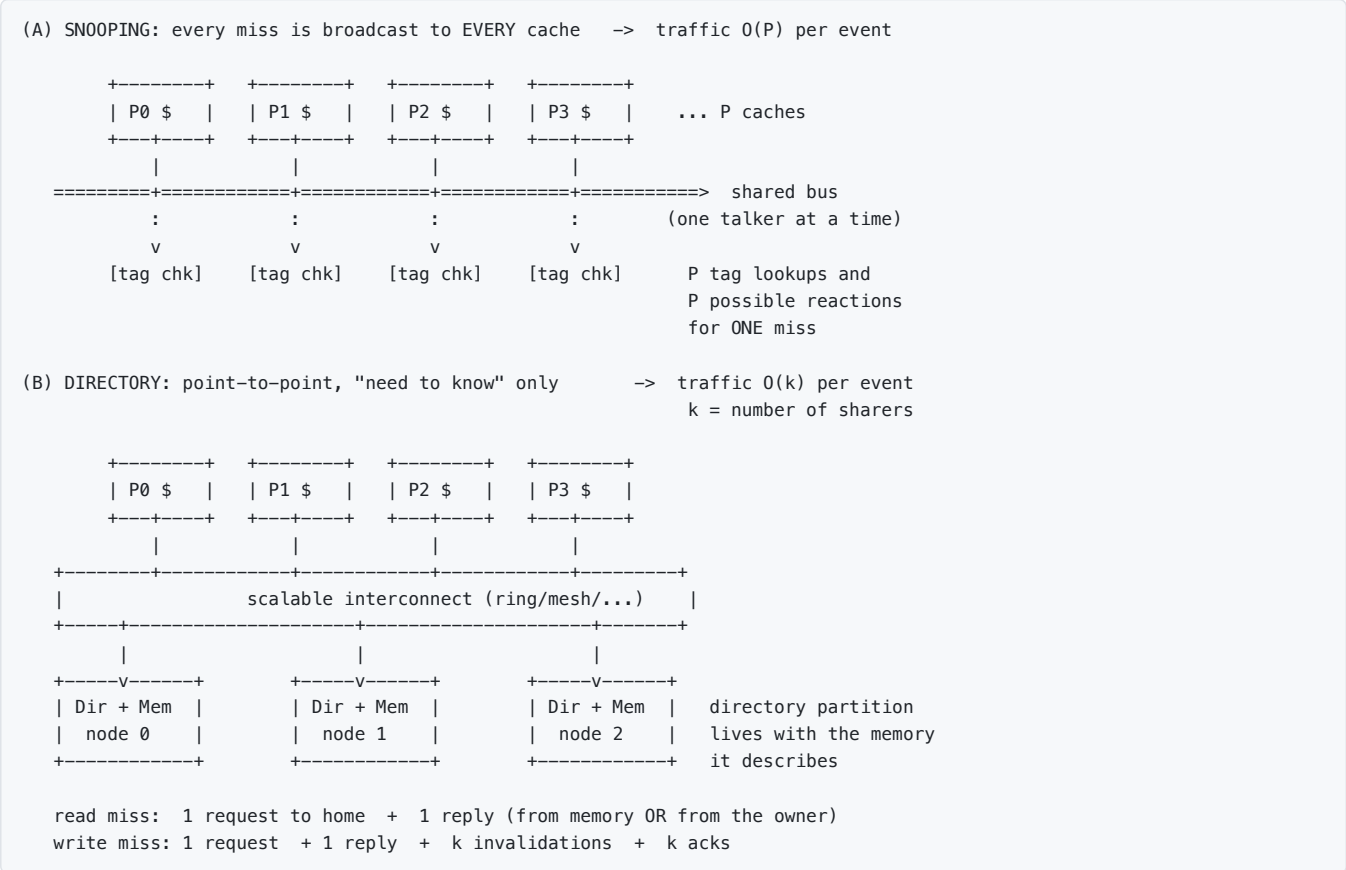
- 部分讲座在 Fall 2026 **尚未发布**公开讲义（Performance Analysis / Profiling、Transactional Memory、AI in System Design 等）；其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。
- 本笔记中的延迟/带宽假设（每跳 30 ns、链路 20 GB/s、缓存控制器 tag 查询 2 ns 等）、实测数据（在 8 线程 Linux 机器上一次运行的结果）与目录开销的精确百分比推导，均已显式标注为"**本笔记补充**"，与讲义原文区分。

---

2. 核心概念与硬件/软件架构图解

2.1 出发点：广播为什么会在"大机器"上撞墙

- **定义与目的**：讲义 slide 3 回顾了监听式一致性的实现方式——**"每一次 cache miss，触发的 cache 都要和所有其他 cache 通信"**；一致性信息的传播靠**广播（broadcast）**，而广播最典型的实现就是**共享总线（shared bus）**。它的**目的**是让协议极简：不需要知道"谁有这一行的副本"，因为每个人都听得见每一次请求；状态机（MSI/MESI）只需关心自己的 cache 就行。**问题**出在"每个人都听得见"这件事的代价上：消息数、被卷入的 cache 数、总线上占用的周期数，全都随节点数 `P` 线性甚至更差地增长。
- **直观解释（"它是什么？"）**：监听式一致性像**村里的广播喇叭**——全村 5 户人家时，谁家要找一样东西就喊一嗓子，所有人放下手里的活听一遍（snoop 一次 tag），又便宜又省事。可村子变成 1000 户以后：(1) 喇叭只有一条信道，所有人必须排队喊（**总线争用 contention**）；(2) 喇叭的响度/信道容量不随户数增加（**带宽不 scale**）；(3) 喇叭线越拉越长、挂的喇叭越多，电气负载（电容）越大，只能**降频、加功耗**；(4) 最荒唐的是——你只是想去**隔壁邻居**家借个东西（访问本地 NUMA 内存），也得先对着全村喊一遍（**为了一致性必须广播**）。目录（directory）就是"**电话簿 + 只打该打的电话**"：先把"这东西现在在谁手里"记在一个地方（目录），然后**点对点（point-to-point）** 打给需要知道的人。
- **图 1：监听式广播 vs 目录的点对点（讲义 slide 3、7、19 的核心对比）**



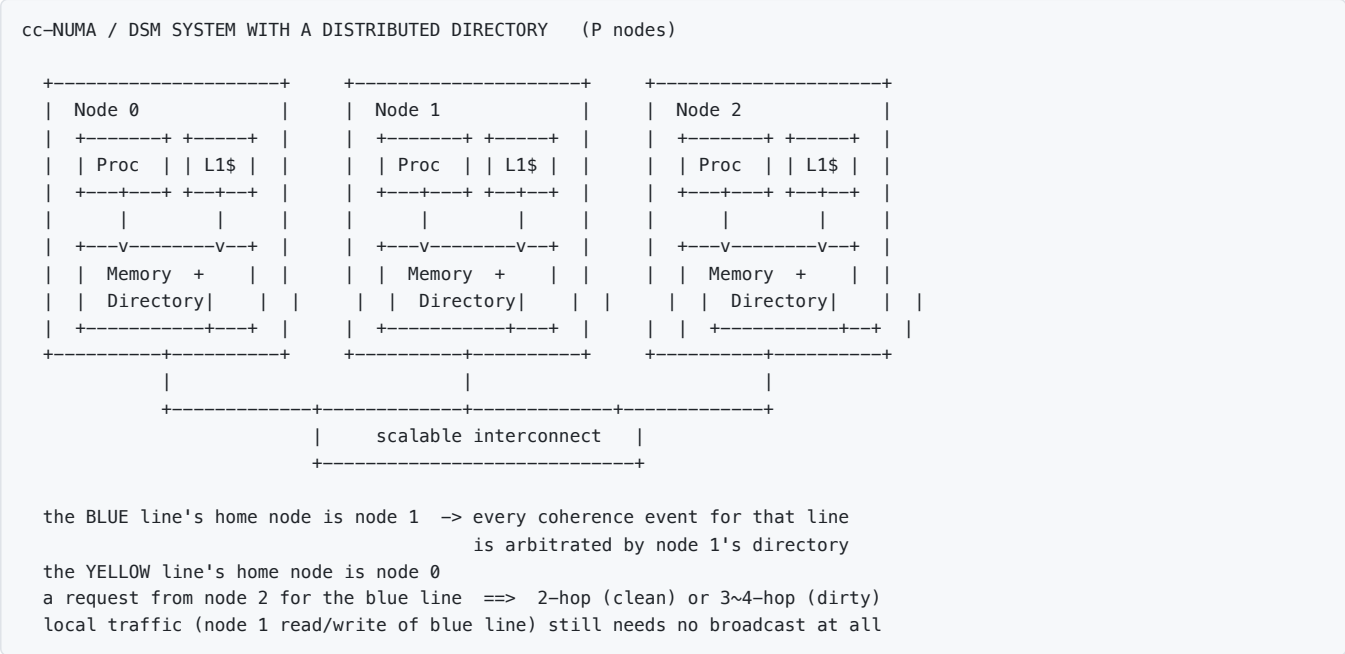
- **性能特征（延迟、带宽、吞吐量、可扩展性）**：
- **监听式**：延迟低（一次广播事务就能定序）、实现简单（总线本身就提供了**原子性的广播定序**）；但**带宽固定**——总线上所有数据（读回复、写回、失效）共享同一条信道，与 `P` 无关；**电气负载**随 `P` 上升导致频率下降；每个 cache 都要为**别人的**流量做 tag 查询，`P` 越大浪费越多。
- **目录式**：引入一次**间接（indirection）**——先问 home，所以 clean 读 miss 是 2 跳、dirty 读 miss 是 3~4 跳，**单次 miss 延迟可能比广播还高**；换来的是**流量与被卷入的 cache 数与 `k`（实际 sharer 数）成正比，而不是与 `P` 成正比**，并且数据可以**直接从 owner 的 cache 传到请

求方（cache-to-cache），不必经过内存。

- 结论：**目录不是"更快的协议"，而是"更可扩展的协议"**；在小机器上广播赢，在大机器上目录赢，判据是"每个事件消耗的全局资源是 O(P) 还是 O(k)"。

2.2 NUMA / cc-NUMA / DSM：目录协议真正的动机

- **定义与目的**：讲义 slide 4 先回顾 **NUMA（non-uniform memory access，非一致内存访问）** 共享内存系统——把内存切分、**就近放在处理器旁边**（讲义举的例子是 PSC Blacklight 这类机器）。这样做能带来**更高的聚合带宽**与**更低的延迟**（尤其是应用有局部性时）。但紧接着给出关键论断：**"如果一致性协议本身不能扩展，NUMA 的高效就白搭"**——处理器访问近处内存本来是好事，可是为了保证一致性仍然必须广播给所有其他处理器，那"就近"省下的时间又赔进去了。讲义给出了两个术语：
- **cc-NUMA（cache-coherent NUMA）**：带缓存一致性的非一致内存访问系统。
- **DSM（distributed shared memory system，分布式共享内存系统）**：**cache coherent + shared address space（共享地址空间）**，但物理实现是**分布式的内存**。
- 与之配套的目录术语（slide 8~9）：**home node（主节点）** = 持有该 cache line 对应数据的内存所在的 node；**requesting node（请求节点）** = 发起请求的处理器所在 node；**目录分区与它描述的内存同址（co-located）**。
- **直观解释（"它是什么？"）**：cc-NUMA 像**一栋有多个厨房的大楼**。每个厨房（node）旁边都有冰箱（内存），住得近的人拿东西当然快——这就是 NUMA 的好处。但"谁手里有那本大家都可能改的账本"（一致性）如果还得靠**全楼广播**，那住得再近也没用：每次翻账本都要通知全楼 64 户。目录相当于在**每台冰箱门上贴一张"借出记录"**：这本账本现在在谁手里、有没有被改过，一看就知道，只给真正持有副本的人打电话。
- **图 2：cc-NUMA 系统与"home node / requesting node"的角色划分（讲义 slide 4、8、9）**



- **性能特征**：NUMA 提供的是**带宽的可扩展性**（每加一个 node 就多一份本地内存带宽），目录提供的是**一致性流量的可扩展性**（每加一个 node 不会让每个事件多消耗全局资源）。两者必须**同时**成立，cc-NUMA 才真的可扩展——这正是讲义 slide 4 那句"efficiency of NUMA system does little good if the coherence protocol can't also be scaled"的定量含义。

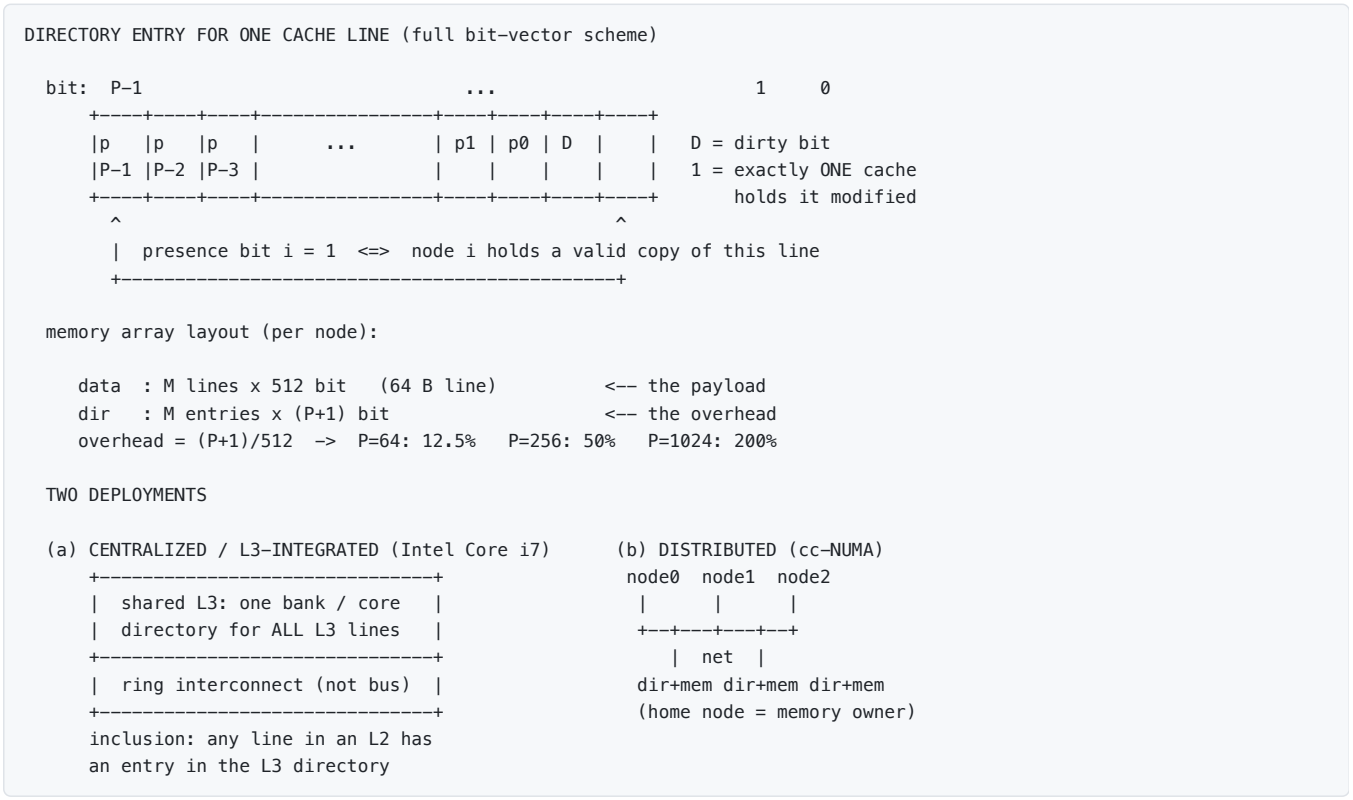
2.3 中间方案：分层监听（hierarchical snooping）

- **定义与目的**：讲义 slide 5~6 给出"先别急着上目录"的一种解法：**在每一层都用监听（snooping）**。把处理器先分成若干簇（cluster），簇内用总线/监听保持一致，簇间再挂一条上层互连，上层也用监听保持一致——**多层总线树**。另一变体是**内存跟着簇走（memory localized with the groups of processors）**，而不是集中在顶层。
- **直观解释（"它是什么？"）**：像**公司里的多级会议**：小组内部（4 人）开个短会就能对齐；跨组的事情才上升到部门会议，再上升到全公司大会。层级越往上，参会的人越多、越慢、越容易堵。
- **讲义明确给出的优缺点（slide 6）**：
- **优点**：**相对容易实现**——因为多级 cache 本身就要处理类似的问题（已经在做层级化的 tag/状态维护）。
- **缺点**：(1) **网络根部（root of the network）会成为瓶颈**，所有跨簇流量都要过它；(2) **延迟比直接通信更大**（要走多级）；(3) **不能套用到更一般的网络拓扑上**（mesh、cube 这类拓扑里没有天然的"层"）。

- **它在知识体系里的位置**：分层监听是"**用层次换简单**"，目录是"**用间接换可扩展**"。讲义把它作为对照方案列出，随后 slide 7 才正式引入目录。

2.4 目录的基本结构：presence bits + dirty bit

- **定义与目的**：讲义 slide 7 把目录的思想一句话讲清：**监听方案靠广播去"问出"某一行在其他 cache 里的状态；替代思路是把"这一行的状态"集中存在一个地方——目录（directory）**。
- **目录项的内容**：**一个 cache line 在所有 cache 中的状态**；
- **访问方式**：cache **按需（as necessary）** 查询目录；
- **维护方式**：cache 之间用**点对点消息、按需知道（need to know）** 的方式维护一致性，**不再使用广播**。
- slide 8 给出**最简目录（very simple directory）**：每个 cache line 一个目录项，项里是 `P` 个 **presence bit（存在位，表示处理器 P 是否在它的 cache 里持有这一行）** + 1 个 **dirty bit（脏位，表示这一行在其中某个 cache 里是脏的/被改过）**。
- slide 9 给出**分布式目录（a distributed directory）**：目录分区与它描述的内存放在同一个 node（**"目录分区与内存同址部署"**），于是"home node"这个概念落到物理位置上。
- **直观解释（"它是什么？"）**：目录项像**图书馆某本书的借阅卡**：卡片上有一排格子（presence bits），格子上打勾表示"这一位对应的读者手里有复印本"；另有一个"已被涂改"标记（dirty bit），表示"目前有一份复印本被改动过，原书（内存）已经过期"。**管理员（home node）只看卡片就能回答两个问题**："谁能给我数据？"（dry 就直接从书架取，dirty 就找那位涂改过的读者）和"我需要通知谁作废？"（所有打勾的格子）。
- **图 3：目录项位布局 + 集中式目录 / 分布式目录（讲义 slide 8、9、23、30）**



- **性能特征**：目录查询本身是**一次额外的内存/目录访问**（这也是"目录 miss 延迟高于广播"的原因），但它是**并行可扩展**的——因为目录分区和内存一样是**分布式切片（sliced）** 的，`P` 个 node 可以同时服务 `P` 个不冲突的目录查询；而广播总线的仲裁是**全局串行**的。

2.5 三种典型事件的消息序列（本讲最需要背下来的部分）

讲义用 slide 10~18 完整走通了三个例子。以下"消息编号"完全对应讲义图上的编号。

(a) read miss 到 clean line（slide 10~11）

1. **request**：请求节点向该行的 **home node** 发 **read miss 消息**；home 目录查这一行的目录项。
2. **response**：若该行 **dirty bit = OFF**，home 直接从**内存**返回数据，并把 `presence[requestor]` 置为 **true**（表示请求节点现在缓存了这一行）。
3. 代价：**2 条消息，关键路径 2 跳**。只有 home 一个 cache 参与。

(b) read miss 到 dirty line（slide 12~14）

情形：处理器 0 读 blue line，该行是 dirty 的（最新内容在 P2 的 cache 里）。

1. **request**：read miss 消息到 home。
2. **response：owner id**：home 发现 **dirty bit = ON**，于是**数据必须由另一个处理器提供**（它持有最新副本），home 只能告诉请求方"**去 P2 拿**"。
3. **request: data**：请求节点向 **owner（P2）** 请求数据。
4. **response: data**：owner 把数据发给请求节点，并把**自己 cache 中该行的状态改为 SHARED（只读）**。
5. **response: data + dir revision**：owner **同时**给 home 回一条消息，home 据此**清 dirty、更新 presence bits、并把数据写回内存（update memory）**。
6. 代价：**5 条网络事务**，其中 **4 条在关键路径上**（事务 4 和 5 可以并行，所以事务 5 不算关键路径）。讲义在 slide 37 给出关键路径的定义：**"critical path：完成这个操作必须依次发生的一串依赖操作（sequence of dependent operations）"**。

(c) write miss（slide 15~18）

情形：处理器 0 要写一行，该行是 **clean** 的，但**同时驻留在 P1 和 P2 的 cache 里**。

1. **request: write miss msg**：P0 向 home 发写 miss。
2. **response: sharer ids + data**：home 返回**共享者列表**与数据（P0 拿到数据与"要通知谁"）。
3. **request: invalidate**：home（按讲义图上的编号由目录发起）**分别**向 P1、P2 发失效消息（**2 条消息**，可以**并行**发出）。
4. **4a / 4b: ack**：P2、P1 各自**应答（ack）**。
5. **P0 收到两个失效应答之后才可以真正执行写（after receiving both invalidation acks, P0 can perform write）**。
6. 代价：**2 + 2k 条消息**（`k` = 共享者数），**关键路径 4 跳**（request → response → invalidate → ack）。**关键点：失效消息可以并行发给所有 sharer**（这是 full bit vector 的优势，见 §2.9 与 sparse directory 的对比）。
7. **表 1：三种事件在"目录 vs 监听"下的消息数、关键路径、被卷入的 cache 数**

事件	方案	消息/事务数	关键路径（跳）	必须做动作的 cache 数	说明
read miss，clean	目录（点对点）	2	2	1（home）	数据从内存来（slide 11）
read miss，clean	监听（广播）	2 个总线事务	2	**P**（每人都 snoop）	总线仲裁串行
read miss，dirty	目录（原始 5 消息）	**5**	**4**	2（home + owner）	事务 4、5 并行（slide 14、37）
read miss，dirty	目录 + intervention fwd	4	4	2	流量少 1 条，关键路径不变（slide 38~40）
read miss，dirty	目录 + request fwd	4	**3**	2	请求方直接向 owner 要数据（slide 41~43）
write miss，k 个 sharer	目录	**2 + 2k**	4	k + 1	失效与 ack 各 k 条，**可并行**（slide 17、18）
write miss，k 个 sharer	监听（BusRdX 广播）	2~3 个总线事务	2~3	**P + 1**	广播到全部 cache（上一讲 MSI）

> 表 1 中"目录"一行是讲义 slide 10~18、37~43 的直接结论；"监听"一行是上一讲 MSI 协议的对照（`P` 为系统节点/核数，`k` 为当前 sharer 数）。

- **图 4：read miss 到 dirty line 的完整时序与关键路径（讲义 slide 12~14、37）**

```
READ MISS TO A DIRTY LINE  (original scheme: 5 network transactions)

  P0 (requesting)        H = home node (dir + memory)        P2 (owner)
       |                            |                          |
   (1) |---- read miss msg ---------->|                         |
       |                       dir entry: presence={P2}, D=1    |
   (2) |<--- response: "owner=P2" ---|                          |
       |                            |                          |
   (3) |------------ request: data ---------------------------->  |
       |                            |                          |
   (4) |<----------- data (cache-to-cache transfer) ------------ |   (4) and (5)
       |                            |                          |   run in PARALLEL
   (5) |                            |<-- data + dir revision --|   P2 sets its copy
       |                            |  D=0, presence += P0     |   to SHARED; H
       v                            v                          v   updates memory
  critical path (dependent messages):  (1) -> (2) -> (3) -> (4)   = 4 hops
  total transactions: 5        (message (5) is OFF the critical path)
  caches that must react: 2    (a broadcast would involve all P caches)

  WHY THIS MATTERS:  the "5" counts bandwidth, the "4" counts latency.
  optimizations in slide 38-43 attack these two numbers SEPARATELY.
```

- **图 5：write miss（k = 2 个 sharer）的失效与应答 + "并行失效 vs 串行失效"（讲义 slide 17、18、32~34）**

```
WRITE MISS; line is CLEAN but resident in P1 and P2   (k = 2 sharers)

   P0 (writer)          H (home / dir)            P1 $              P2 $
       |                      |                     |                 |
  (1)  |--- write miss ------>|                     |                 |
  (2)  |<-- sharer ids + data-|                     |                 |
  (3)  |                      |--- invalidate ----> |                 |  both
       |                      |--- invalidate ---------------------->  |  independent
  (4a) |<------------------------------ ack ------- |                 |  => parallel
  (4b) |<------------------------------------------------ ack --------- |
       |
   after BOTH acks arrive, P0 writes;  dir: presence={P0}, D=1; P1,P2 -> I
   messages = 2 + 2k ;  critical path = 4 hops

  FULL BIT VECTOR: invalidations are independent -> all k go out at once
     H ===> n1                    latency ~ O(1) hops      (slide 34)
     H ===> n2                    traffic   ~ O(k) msgs
     H ===> n3

  SPARSE DIRECTORY (linked list): the directory only knows the HEAD
     H --> n1 --> n2 --> n3       latency ~ O(k) hops      (slide 33)
       (each hop needs a pointer stored in the previous cache's line)
```

2.6 失效模式（invalidation patterns）：为什么"目录真的有用"

- **定义与目的**：目录的整个立论基础是**"写的时候共享者其实很少"**。讲义 slide 20~21 用 64 处理器系统上的 Barnes-Hut、LU、Ocean 三个应用的**直方图**来支撑这个论断：图的横轴是"**一次写发生时该行的 sharer 数**"，纵轴是频率；结论是 **"in general only a few processors share the line（一般来说只有少数处理器共享该行）"**，只有少数处理器需要被告知写操作。讲义还指出（图未显示）：**sharer 的期望数量随 P 增长得非常慢（expected number of sharers typically increases slowly with P），这是好消息**。
- **slide 21 把访问模式分成五类**，这是本讲"为什么目录能省"的最重要定性依据：
- **表 2：访问模式 vs 目录压力（讲义 slide 21）**

访问模式	sharer 数	失效频率	对性能的影响 / 结论
Mostly-read objects（多读少写）	很多 sharer	写很少	影响极小（讲义举例：Barnes-Hut 的根节点）
Migratory objects（迁移型对象）	极少 sharer	中	**sharer 数不随处理器数增长**（一个处理器读/写一阵，再换下一个）
Frequently read/written objects（频繁读写）	少	非常频繁	失效频繁，但 **sharer 数来不及build up**（两次失效之间时间太短，例如共享任务队列）

访问模式	sharer 数	失效频率	对性能的影响 / 结论
低争用锁（low-contention locks）	少	不频繁	无性能问题
高争用锁（high-contention locks）	多	—	**是难题**：锁释放时**恰好有很多读者在场**（讲义原文）

- **两条推论（讲义 slide 21 明确写出）**：
- **Implication 1**：目录对**限制一致性流量**很有用——**不需要广播机制去"告诉所有人"**。
- **Implication 2**：这个观察**暗示了目录实现可以怎么优化**（尤其是**降低存储开销**）——因为 sharer 少，就可以不必为每个 node 都存一位。
- **直观解释（"它是什么？"）**：想象一个 64 人的微信群（64 个核共享一行数据）。如果每次有人改一份文件都要 @全体成员（广播），群里就永远在响；而实际上多数时候**只有 1~2 个人**在改这份文件（migratory / 任务队列 / 低争用锁），所以正确做法是**私聊那 1~2 个人**（点对点失效）。唯一真正难的是**抢红包式的热点锁**：锁一释放，64 个人同时伸手，这时"要通知的人"确实很多——但那种情况在广播方案下更糟（所有人本来就在被通知）。

2.7 目录的存储开销：full bit vector 的账

- **定义与目的**：full bit vector（全位向量）目录**每个 cache line 需要 P 个 presence bit**，存储量与 **P × M** 成正比（M = 内存中的行数）。讲义 slide 23 直接算账（按 **64 B cache line = 512 bit**）：
- **P = 64 → 12% 开销**；**P = 256 → 50% 开销**；**P = 1024 → 200% 开销**（即目录内存是数据内存的两倍）。
- 精确值为 `P/512`：**12.5% / 50% / 200%**（讲义把 12.5% 写成 12%，本笔记补充了精确值）。
- **减少开销的四条思路（slide 24 + 26 + 28~33）**：
- **增大 cache line 尺寸**（减小 `M` 项）——讲义提醒要"考虑上一讲的直方图/曲线"，即**行变大意味着一次失效作废更多数据、伪共享代价更大**；
- **把多个处理器合并成一个目录"节点"**（减小 `P` 项）——一个节点只需一位目录位，可以**分层**：节点内用监听，节点间用目录；
- **limited pointer schemes（有限指针方案）**：用"少量指针的列表"代替位向量；
- **sparse directories（稀疏目录）**：只为**当前真的在 cache 里**的行保留目录信息。
- **图 6：limited pointer 的"常见情况优化"与溢出处理（讲义 slide 25~28）**

```
LIMITED POINTER SCHEME  (exploit: only a FEW caches hold the line)

  entry layout, k = 5 pointers, P = 1024:
  +----+---------+---------+---------+---------+---------+
  | D  | ptr 0   | ptr 1   | ptr 2   | ptr 3   | ptr 4   |
  +----+---------+---------+---------+---------+---------+
       \___ 10 bits each = log2(1024) ___/
  bits per entry = 1 + k*log2(P) = 1 + 5*10 = 51 bits  (vs 1024 bits full vector)

  OVERFLOW (more than k sharers) -- slide 26 gives three answers:
     (a) FALL BACK TO BROADCAST
            H ---> ALL CACHES      "if a broadcast mechanism exists"
     (b) CAP THE NUMBER OF SHARERS
            newest sharer REPLACES an existing one
            -> must INVALIDATE the line in the old sharer's cache
     (c) COARSE VECTOR FALLBACK
            revert to a bit vector, but each bit covers K nodes
            -> on a write, invalidate ALL nodes mapped to that bit

  THE DESIGN LESSON (slide 27, four steps):
     1. workload-driven observation: sharer count is usually LOW
     2. make the common case simple and fast: pointer array for the first N sharers
     3. uncommon case is still CORRECT, just slower/more complex (program still works!)
     4. the expensive path is tolerable because it happens rarely
```

- **表 3：三种目录表示法的存储与代价对比**

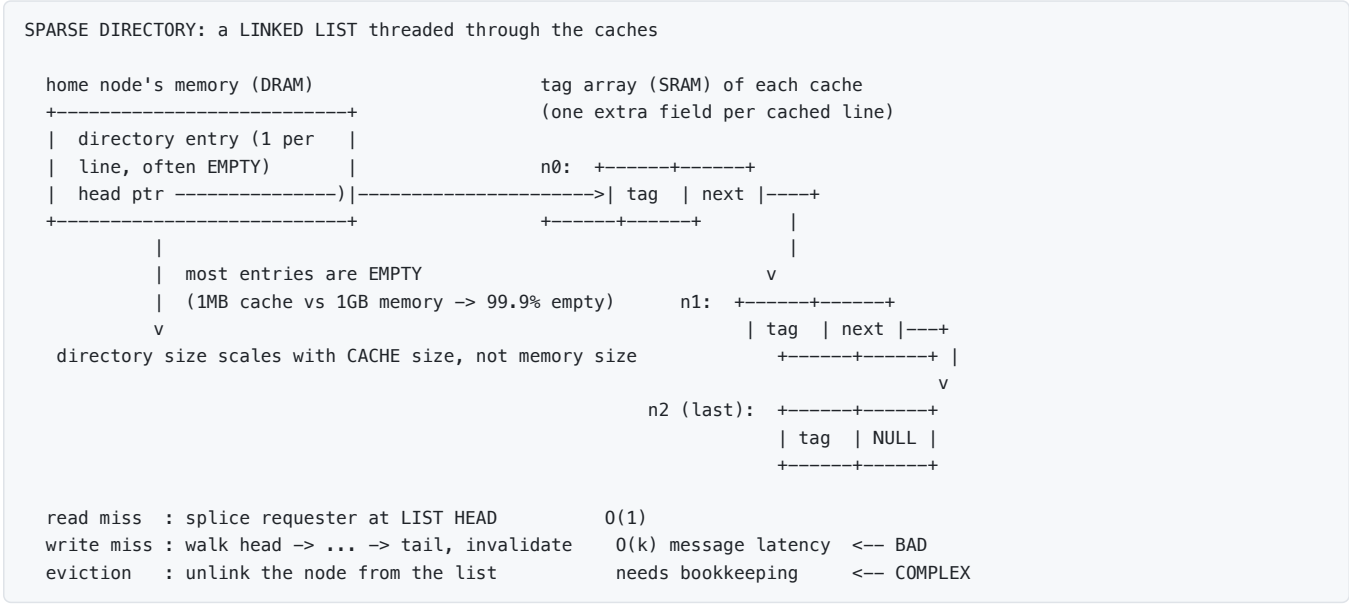
方案	每个 cache line 的目录存储	P=64	P=256	P=1024	优点	代价
Full bit vector	`P` bit	12.5%	50%	200%	失效可**并行**发给所有 sharer，实现最简单	存储随 `P` 线性增长，`P` 大时不可接受

方案	每个 cache line 的目录存储	P=64	P=256	P=1024	优点	代价
Limited pointer（k=5）	$1 + k \cdot \log_2 P$ bit	**6.05%**（讲义写 5.8%）	**8.01%**（7.8%）	**9.96%**（9.7%）	存储几乎与 P 无关	需要**溢出处理**（广播 / 顶替 / 粗向量）
Limited pointer（k=100）	$1 + 100 \cdot \log_2 P$ bit	117.4%	156.5%	**195.5%**	——	k 太大时**比全位向量还差**（P=1024 时临界点 $k \approx 102$）
Sparse directory（链表）	每行 1 个 head 指针（内存侧）+ cache line 里的 next 指针（SRAM 侧）	——	——	——	存储**随 cache 大小而非内存大小**扩展	失效**串行**沿链表传播，$O(k)$ 延迟；实现复杂

> **讲义数字与公式的差异说明（本笔记补充）**：讲义 slide 28 写 $\text{entry size} = 1 + k \log P$，同时给出 k=5 时 5.8% / 7.8% / 9.7%。这三个数恰好等于 $k \cdot \log_2 P / 512$（$5\times6/512 = 5.86\%$、$5\times8/512 = 7.81\%$、$5\times10/512 = 9.77\%$），即**讲义在算百分比时略去了那 1 位状态/脏位**。带上 $+1$ 位的精确值是 6.05% / 8.01% / 9.96%。两种口径都能说明同一件事：**limited pointer 的存储开销几乎不随 P 增长**（200% → 10%，20 倍改善）。
>
> 另一个有用的临界点（本笔记补充）：$1 + k \cdot \log_2 P < P \iff k < (P-1)/\log_2 P$。P=1024 时 $k < 102.3$，即**指针数超过约 102 个就失去意义**；讲义 slide 25 举的"~100 个指针"正好贴着这个临界点，所以它紧接着补充"**in practice, our workload evaluation says we can get by with far less than this**"（实践中远用不到这么多）。

2.8 稀疏目录（sparse directories）：只在"真的在 cache 里"的行上花内存

- **定义与目的**：讲义 slide 29~31 的关键观察：**绝大多数内存并不驻留在 cache 中**，而一致性协议**只需要"当前在 cache 里的行"的共享信息**——所以**大多数目录项在大多数时候是空的**。讲义给的量化例子：**每 node 1 MB cache、1 GB memory → 99.9% 的目录项是空的**。于是：**目录尺寸应该随 cache 大小（C）扩展，而不是随内存大小（M）扩展**（$P \cdot C$ 位每处理器），每行只留一个 **tag（标签）**。
- **进一步压缩（slide 32~33）**：home node 的目录**只保存一个指向"链表中某个 node"的指针，而不是 sharer 列表**；**"指向下一个 node 的指针"被存在 cache line 的额外信息里（就像这一行的 tag、dirty bit 一样）**。三项操作规则：
- **read miss**：把请求节点**插到链表头（add requesting node to head of list）**；
- **write miss**：**沿链表传播失效（propagate invalidations along list）**；
- **evict（换出）**：需要**修补链表（linked list removal）**。
- **讲义 slide 33 明确列出的好坏两面**：
- **Good**：内存侧存储开销低（每行一个 head 指针）；额外的目录存储**与 cache 大小成正比**（链表存在 SRAM 里）；**写时的流量仍与 sharer 数成正比**。
- **Bad**：**写的延迟与 sharer 数成正比（失效是串行的 invalidation of lines is serial）**；**实现复杂度更高**。
- 与 full bit vector 的对照（slide 34）：**位向量方案发送的失效消息数量相同，但失效消息可以并行发给所有处理器**。
- **图 7：sparse directory 的链表结构（讲义 slide 32~33）**



- **性能特征小结**：`full bit vector` 是"**空间换并行失效**"（`O(k)` 消息但 `O(1)` 跳数）；`sparse list` 是"**空间换串行失效**"（存储降到极低，但延迟 `O(k)` 跳）。**这正是讲义 slide 35 分出的两条优化主线**：一路压**目录数据结构**的存储（limited pointer、sparse directory），一路压**协议消息数/关键路径**（下一节）。

2.9 减少消息数与关键路径：intervention forwarding 与 request forwarding

- **定义与目的**：讲义 slide 35 把优化分成两类：**减少目录结构的存储开销**，以及**减少实现一致性协议所需发送的消息数（messages sent）**。后者给出两种转发策略，都是从 slide 12~14 那个"5 条消息 / 4 跳关键路径"的 dirty 读 miss 改进而来。
- **intervention forwarding（干预转发，slide 38~40）**：
- 请求 read miss 消息 → home；
- **home 主动向 owner（P2）发 intervention read 请求**；
- owner 把 **data + dir revision** 回给 **home**；
- **home 更新目录，再把数据转发给请求节点**。
- 结果：**总共 4 条网络事务（流量更少）**，但**四条全部在关键路径上**。讲义在这一页直接追问："**Can we do better?**"
- **request forwarding（请求转发，slide 41~43）**：
- 请求 read miss → home；
- **home 只发一条"send data to requestor"给 owner**；
- owner 把数据**同时**发给 **home** 和**请求节点**（"3/4. Response: data (2 msgs: sent to both home node and requestor)"）。
- 结果：**总共 4 条网络事务，只有 3 条在关键路径上**（事务 3 和 4 可以并行）。
- 讲义特别标注了一条重要性质：**"系统不再是纯粹的请求/响应（pure request/response）"**——因为 **P0 把请求发给了 home，却从 owner 收到了应答**。这一点对实现者非常重要：请求方必须能接受"非请求对象发来的应答"，这在互连网络与 MSHR（miss status holding register）设计上都要额外支持。
- **表 4：三种 dirty 读 miss 处理方案的对比（讲义 slide 12~14、38~43）**

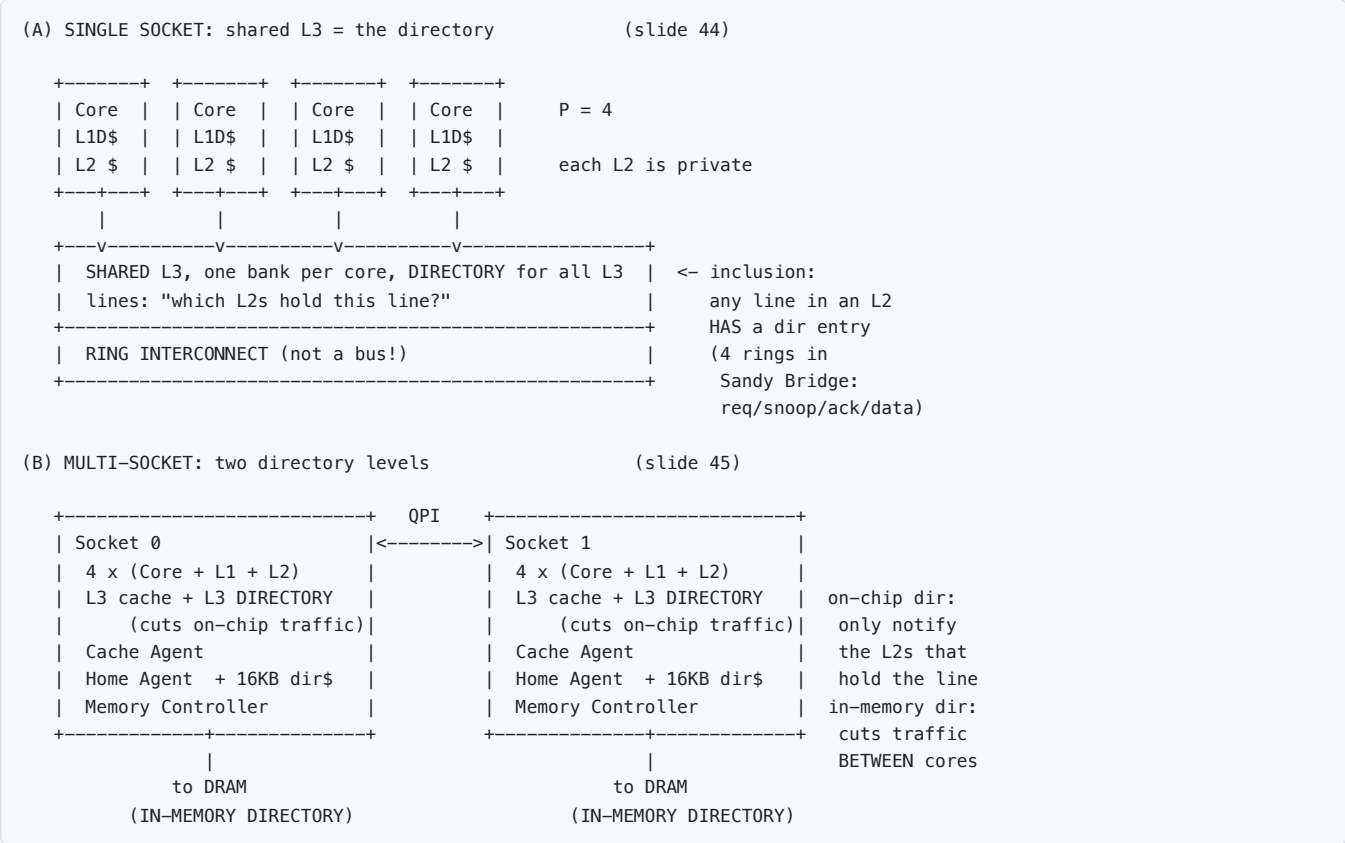
方案	网络事务数	关键路径（跳）	关键路径上的事务	数据从哪来	额外性质
原始（5 消息）	5	4	1→2→3→4	owner → 请求方	事务 5（owner→home）**不在**关键路径
intervention forwarding	4	4	全部 4 条	owner → home → 请求方	流量少 1 条，延迟不变
request forwarding	4	**3**	1→2→3（3、4 并行里的 3）	owner → 请求方（并抄送 home）	**不再是纯请求/响应**，需要支持"非对称应答"

- **图 8：两种转发策略的时间线（讲义 slide 37、40、43）**

```
TIMELINE OF "READ MISS TO A DIRTY LINE": where the hops go

  hop #:        1              2               3               4
  ------------------------------------------------------------------------
  original:   P0 --> H        H --> P0         P0 --> P2        P2 --> P0
              (read miss)     (owner id)       (req data)       (data)
              msgs = 5   crit = 4        plus P2 --> H (off the critical path)

  intervention:
              P0 --> H        H --> P2         P2 --> H         H --> P0
              msgs = 4   crit = 4        home stays in the middle: +1 traffic hop,
                                                    no latency gain

  request fwd:
              P0 --> H        H --> P2         P2 --> P0   ||   P2 --> H
                                              (data to requestor)  (dir revision)
              msgs = 4   crit = 3        <-- shortest critical path
                                                    reply does NOT come from H

  SUMMARY:  messages  : 5 -> 4 -> 4      (bandwidth / traffic)
            crit hops : 4 -> 4 -> 3      (latency)
            an optimization that shortens the critical path may need NEW protocol
            machinery (here: accept a reply from a node you never asked)
```

2.10 真实硬件：Intel Core i7 的 L3 目录与多 socket 的 home agent

- **讲义 slide 44（单 socket）**：
- **L3 cache 充当所有 L3 中行的集中式目录（L3 serves as centralized directory for all lines in the L3 cache）**；讲义强调**inclusion property（包含属性）** 的重要性——**任何在 L2 中的行，在 L3 目录里必定有目录项**（否则目录会漏掉 sharer，协议就不正确了）。
- 目录维护**"哪些 L2 cache 持有该行"的列表**；因此**不再向所有 L2 广播一致性流量，只给持有该行的 L2 发一致性消息**。
- 讲义还点明：**"Core i7 的互连是环形（ring），不是总线（bus）"**——所以广播在这里本来就不自然。
- 目录维度：**P = 4**（4 个核），**C = L3 的 cache line 数**（这正是 sparse directory 的"随 cache 大小扩展"的思路！）。
- **讲义 slide 45（多 socket）**：
- 单 socket 内的 **L3 目录**减少片上一致性流量（上一页）；
- **内存中的目录（in-memory directory）** 由 **home agent / memory controller** 缓存（**dir cache，16 KB**），减少**核与核之间**（跨 socket）的一致性流量；
- 关键角色：**cache agent**（片上，说监听/一致性协议的那一端）、**home agent**（拥有本 socket DRAM 的目录）、**memory controller**、**QPI（QuickPath Interconnect）** 连接两个 socket，DRAM 侧则是 **in-memory directory**。
- **图 9：Core i7 的环形互连 + L3 目录，以及多 socket 的目录层次（讲义 slide 44、45）**



- **性能特征（本笔记补充的量化视角）**：16 KB 的 dir cache 是**内存目录的 cache**。若每个目录项压缩到约 4 B，则 16 KB ≈ **4096 个目录项**；64 GB DRAM 按 64 B 行算有 **10.7 亿行**，即 dir cache 只能覆盖约 **0.0004%** 的行。它的价值完全来自**局部性**：应用反复访问同一批行时，目录查询在 home agent 的 SRAM 里命中，省掉一次 DRAM 往返（约 **70~100 ns**，本笔记补充假设），跨 socket 的失效因此不会被 DRAM 目录访问拖成瓶颈。这也解释了两级目录的分工：**L3 目录解决"片上要不要广播"，in-memory directory 解决"socket 之间要不要广播"**。

2.11 软件侧的执行模型：cc-NUMA 上"数据落在哪、线程跑在哪"

- **定义与目的**：硬件给出的是"谁 home、谁 sharer、谁 owner"；软件要控制的只有两件事：**数据的 home node（由首次触碰决定）** 与**线程所在的 node（由调度/亲和性决定）**。这两者错配时，程序就要付 3~4 跳的远程延迟；而**伪共享**会把两条本该互不相干的数据变成同一条 cache line 上的来回迁移。
- **直观解释（"它是什么？"）**：把 cc-NUMA 想成**一栋楼里 4 个厨房 + 4 张书桌**。`first touch` 规则像**"谁先把锅端进哪个厨房，这口锅以后就归那个厨房保管"**（home node）；如果你的书桌在 0 层而锅在 3 层，每次做饭都要坐电梯（远程访问）。**伪共享**更像**"两个人的牙刷被绑在同一根绳子上"**：明明各用各的，但一个人动一下，另一个人手里的东西就被判为作废（invalidate），绳子在两人之间来回传。
- **图 10：软件执行模型（线程亲和 + first touch + 伪共享）（本笔记补充示意，机制依据讲义 slide 4、8、9、20、21）**

```
SOFTWARE VIEW OF A 4-NODE cc-NUMA MACHINE (directory = home of each line)

   node 0                            node 2
   +-------------+                   +-------------+
   | thread 0    |  <--- remote --->  | thread 1    |
   | A[0 .. N/4) |   3~4 hop dirty    | A[N/4 ..N/2)|
   | home: node 0|   read miss        | home: node 2|
   +-------------+                   +-------------+

   WHO DECIDES THE HOME NODE?  "FIRST TOUCH"

     for (i=0;i<N;i++) A[i] = 0;      // master only
        -> ALL pages of A get home node 0
        -> 7 of 8 threads always pay remote latency

     #pragma omp parallel for schedule(static)
     for (i=0;i<N;i++) A[i] = 0;      // every thread touches its own slice
        -> pages are spread over the nodes that run the threads (LOCAL)

   TWO SEPARATE PROBLEMS, TWO SEPARATE FIXES
     (1) placement:   parallel first touch / memory interleaving / numactl
     (2) coherence:   even with perfect placement, a line written by several
                      nodes still costs O(k) invalidation messages
                      -> pad to cache lines, privatize, or reduce sharing

   FALSE SHARING (worst case of "nobody shares anything, but the line does")
     two nodes write DIFFERENT variables that live in ONE 64 B line:

        node 0 writes x  | 64 B line: [ x ][ y ][ pad ................. ]
        node 2 writes y  |
          -> the LINE ping-pongs: each write is a write miss / invalidation
          -> directory is correct but useless: real sharers = 2, forever
          -> fix:  align each hot variable to 64 B (alignas(64) / padding)
```

2.12 目录协议的状态机（directory 视角与 cache 视角）

- **定义与目的**：上一讲的 MSI/MESI 描述的是**单个 cache 里的行状态**；这一讲要补上**目录里的行状态**，因为协议是否正确取决于两者严格对应。讲义给出的三种目录状态对应：**UNCACHED（没有任何 cache 持有，presence 全 0，D=0）**、**SHARED（presence 非 0，D=0）**、**DIRTY（正好一个 node 持有 M 副本，D=1）**。不变量：**目录中 D=1 ⟺ 恰好一个 cache 把该行保持在 M 状态**。
- **直观解释（"它是什么？"）**：目录状态机像**图书借阅卡片的状态**：`UNCACHED` = 书在书架上没人借；`SHARED` = 有人借去复印（可以多人同时有复印件，原书没被改）；`DIRTY` = 有一个人把书拿去改了（原书已过期，复印/归还前必须找他要最新版）。
- **图 11：目录行状态机 + 与 per-cache MSI 状态的对应（依据讲义 slide 8、11~18、32~34 与上一讲 MSI）**

```
DIRECTORY'S VIEW OF ONE LINE

                read miss (D=0): reply from memory, set presence[req]
   +----------------------+  <-------------------------------------------+
   |       UNCACHED       |                                             |
   |  presence = 00...0   |                                             |
   |  D = 0               |                                             |
   +-----------+----------+                                             |
               |                                                        |
               | read miss -> SHARED                                    | last sharer
               v                                                        | evicts
   +----------------------+   write miss:                                  |
   |        SHARED        |   invalidate all sharers,                      |
   |  presence != 0, D = 0 |   grant exclusive                  +--------+--------+
   +-----------+----------+ ----------------------------------->|      DIRTY      |
               ^                                                 | presence={owner}|
               |  read miss from ANOTHER node:                   | D = 1           |
               |  D := 0, presence += requester,                 +--------+--------+
               |  owner's cache: M -> S                                   |
               +----------------------------------------------------------+
                         (the owner must send data + dir revision to home)

PER-CACHE LINE STATE (MSI, previous lecture) -- what the directory is tracking
   I : Invalid   -- no valid copy
   S : Shared    -- read-only copy, line is also CLEAN in memory (dir D = 0)
   M : Modified  -- the ONLY copy, dirty; the directory records this node as OWNER

INVARIANT (must hold at all times):
   dir.D == 1   <=>   exactly one cache holds the line in M
   presence[i] == 1  for every cache i holding the line in S or M
```

- **性能特征**：三种目录状态之间的迁移代价差别极大——`UNCACHED → SHARED`（2 条消息）、`SHARED → DIRTY`（`2 + 2k` 条消息、4 跳）、`DIRTY → SHARED`（4~5 条消息、3~4 跳）。**协议设计者的任务就是把最常见的那条迁移路径变短**（这正是 limited pointer 与 request forwarding 在做的事）。

---

3. 代码示例与性能分析

> 目录协议本身是硬件行为，但它的**每一条消息、每一跳延迟、每一个 presence bit** 都可以在软件里精确建模。下面三个例子分别回答三个问题：(1) 目录协议**到底发多少消息、关键路径多长、目录要多大的存储**；(2) 一致性流量在**真实多核机器上值多少纳秒**；(3) cc-NUMA 上"**数据放在哪个 node**"值多少带宽。

3.1 示例 1：目录协议模拟器（消息数 / 关键路径 / 存储开销）

- **代码**（完整可运行，模拟讲义 slide 10~18、37~43 的协议并统计开销）：

```
// ============================================================================
// dir_sim.cpp -- directory coherence protocol simulator
//   统计每种一致性事件的: 消息条数(msgs) / 关键路径跳数(crit hops) /
//   必须做动作的 cache 数(caches-involved), 并计算目录存储开销
// 编译 (release): g++ -O3 -std=c++17 dir_sim.cpp -o dir_sim
// 运行: ./dir_sim
// ============================================================================
#include <cstdint>
#include <cstdio>
#include <vector>
#include <cmath>

enum class CS : uint8_t { I, S, M };   // 一条 line 在某个 node 私有 cache 中的 MSI 状态

// 一次一致性操作消耗的网络资源
struct Net {
    long msgs    = 0;   // 网络消息条数   -> 决定流量/带宽占用
    long hops    = 0;   // 关键路径跳数   -> 决定延迟(critical path)
    long snooped = 0;   // 必须检查/动作的 cache 数 -> 决定 snoop 工作量
};

struct DirEntry {              // 每个 cache line 一个目录项 (full bit vector)
    uint64_t presence = 0;     // presence bits: 哪些 node 持有该行
    bool     dirty    = false; // dirty bit: 是否有一个 node 持有 M 副本
    int      owner    = -1;    // dirty 时的 owner node (数据源头)
};

class Machine {
public:
    Machine(int nodes, int lines)
        : P(nodes), L(lines),
          cache(nodes, std::vector<CS>(lines, CS::I)), dir(lines) {}

    int sharers(int line) const { return __builtin_popcountll(dir[line].presence); }

    // ---- 事件 (a): read miss 到 CLEAN line  (讲义 slide 10-11): 2 msgs / 2 hops
    Net readMissClean(int req, int line) {
        Net n;
        n.msgs = 1; n.hops = 1; n.snooped = 1;          // (1) request -> home
        dir[line].presence |= (1ull << req);            // home: presence[req] = 1
        cache[req][line] = CS::S;                       // (2) data from memory
        n.msgs += 1; n.hops += 1;
        return n;
    }

    void finishReadDirty(int req, int line, int own) {  // owner M -> S, D := 0
        cache[own][line]  = CS::S;
        cache[req][line]  = CS::S;
        dir[line].dirty   = false;
        dir[line].owner   = -1;
        dir[line].presence |= (1ull << req);
    }

    // ---- 事件 (b1): read miss 到 DIRTY line, 原始 5 消息版 (slide 12-14)
    //      1 request -> home | 2 home->req (owner id) | 3 req->owner
    //      4 owner->req (data, 关键路径) | 5 owner->home (data+rev, 与 4 并行)
    Net readMissDirty_original(int req, int line) {
        Net n; const int own = dir[line].owner;
        n.msgs += 1; n.hops += 1;      // 1
        n.msgs += 1; n.hops += 1;      // 2
        n.msgs += 1; n.hops += 1;      // 3
        n.msgs += 1; n.hops += 1;      // 4   <- 关键路径到此结束 (4 hops)
        n.msgs += 1;                   // 5   <- off the critical path
        n.snooped = 2;                 // home + owner
        finishReadDirty(req, line, own);
        return n;
    }

    // ---- 事件 (b2): intervention forwarding (slide 38-40): 4 msgs, 4 hops (all on path)
    //      home 先去 owner 取数据、更新目录, 再转发给请求方
    Net readMissDirty_intervention(int req, int line) {
```

```
        Net n; const int own = dir[line].owner;
        n.msgs += 1; n.hops += 1;      // 1 req  -> home
        n.msgs += 1; n.hops += 1;      // 2 home -> owner (intervention read)
        n.msgs += 1; n.hops += 1;      // 3 owner-> home (data + dir revision)
        n.msgs += 1; n.hops += 1;      // 4 home -> req  (data)
        n.snooped = 2;
        finishReadDirty(req, line, own);
        return n;
    }

    // ---- 事件 (b3): request forwarding (slide 41-43): 4 msgs, 3 hops
    //      home 只让 owner 把数据"发给请求方"(并抄送 home 更新目录)
    Net readMissDirty_forward(int req, int line) {
        Net n; const int own = dir[line].owner;
        n.msgs += 1; n.hops += 1;      // 1 req  -> home
        n.msgs += 1; n.hops += 1;      // 2 home -> owner ("send data to requestor")
        n.msgs += 2; n.hops += 1;      // 3+4 owner -> req 数据, owner -> home 目录修订
        n.snooped = 2;                 //     两条并行 => 只算 1 跳
        finishReadDirty(req, line, own);
        return n;
    }

    // ---- 事件 (c): write miss, line clean, k 个 sharer (slide 15-18)
    //      msgs = 2 + 2k (k 条失效 + k 条 ack), 关键路径 4 跳 (失效/应答可并行)
    Net writeMissClean(int req, int line) {
        Net n; const int k = sharers(line) - ((dir[line].presence >> req) & 1ull);
        n.msgs += 2; n.hops += 2;              // 1 request, 2 sharer ids + data
        n.snooped = k + 1;                     // 请求方 + 所有被失效的 sharer
        n.msgs += 2 * (long)k; n.hops += 2;    // 3 invalidate x k, 4 ack x k
        for (int p = 0; p < P; ++p)            // 被失效的 cache: -> I
            if (p != req && ((dir[line].presence >> p) & 1ull)) cache[p][line] = CS::I;
        dir[line].presence = (1ull << req);    // 目录: presence={req}, D=1
        dir[line].dirty    = true;
        dir[line].owner    = req;
        cache[req][line]   = CS::M;
        return n;
    }

    // ---- 事件 (c'): write miss 到 DIRTY line (讲义未展开, 本示例按同一套规则推导:
    //      先用 request forwarding 拿到脏数据(3 跳), 再失效其余 sharer (2 跳) = 5 跳)
    Net writeMissDirty_forward(int req, int line) {
        Net n; const int own = dir[line].owner;
        const int k = sharers(line) - 1;       // 除 owner 以外的 sharer
        n.msgs += 4; n.hops += 3; n.snooped = 2;
        cache[own][line] = CS::I;              // owner 交出独占权, 自己的副本失效
        n.msgs += 2 * (long)k; n.hops += 2; n.snooped += k;
        for (int p = 0; p < P; ++p)
            if (p != req && p != own && ((dir[line].presence >> p) & 1ull)) cache[p][line] = CS::I;
        dir[line].presence = (1ull << req);
        dir[line].dirty    = true;
        dir[line].owner    = req;
        cache[req][line]   = CS::M;
        return n;
    }

    int P, L;
    std::vector<std::vector<CS>> cache;   // [node][line]
    std::vector<DirEntry>        dir;     // [line]
};

static void report(const char* what, Net d) {
    printf("%-42s | msgs %3ld | crit-hop %2ld | caches-involved %3ld\n",
           what, d.msgs, d.hops, d.snooped);
}

int main() {
    const int P = 64, L = 1024;            // 64 个 node (讲义 slide 20 的规模)
    printf("== directory vs snooping : cost of ONE coherence event (P=%d) ==\n", P);

    Machine m0(P, L);
    report("read miss, clean (directory)", m0.readMissClean(0, 0));
    printf("%-42s | msgs %3d | crit-hop %2d | caches-involved %3d\n",
```

```
           "read miss, clean (snooping broadcast)", 2, 2, P);

    Machine a(P, L); a.cache[2][1] = CS::M;
    a.dir[1].dirty = true; a.dir[1].owner = 2; a.dir[1].presence = (1ull << 2);
    report("read miss, dirty (5-msg original)", a.readMissDirty_original(0, 1));

    Machine b(P, L); b.cache[2][1] = CS::M;
    b.dir[1].dirty = true; b.dir[1].owner = 2; b.dir[1].presence = (1ull << 2);
    report("read miss, dirty (intervention fwd)", b.readMissDirty_intervention(0, 1));

    Machine c(P, L); c.cache[2][1] = CS::M;
    c.dir[1].dirty = true; c.dir[1].owner = 2; c.dir[1].presence = (1ull << 2);
    report("read miss, dirty (request fwd)", c.readMissDirty_forward(0, 1));

    Machine d(P, L); d.dir[1].presence = (1ull << 1) | (1ull << 2);
    d.cache[1][1] = CS::S; d.cache[2][1] = CS::S;
    report("write miss, k=2 sharers (directory)", d.writeMissClean(0, 1));
    printf("%-42s | msgs %3d | crit-hop %2d | caches-involved %3d\n",
           "write miss, k=2 (snooping BusRdX)", 2, 2, P + 1);

    Machine e(P, L); e.dir[1].presence = (1ull << 2) | (1ull << 3);
    e.cache[2][1] = CS::M; e.cache[3][1] = CS::S;
    e.dir[1].dirty = true; e.dir[1].owner = 2;
    report("write miss, k=2 (dirty, request fwd)", e.writeMissDirty_forward(0, 1));

    printf("\n== directory storage overhead (64 B line = 512 bit) ==\n");
    for (int n : {64, 256, 1024}) {
        const double lg = std::log2((double)n);
        printf("P=%4d : full vector %6.1f%% | limited ptr k=5 %5.2f%% | k=100 %6.2f%%\n",
               n,
               100.0 * n / 512.0,
               100.0 * (1.0 + 5.0 * lg) / 512.0,
               100.0 * (1.0 + 100.0 * lg) / 512.0);
    }
    printf("breakeven pointer count at P=1024 : k < (P-1)/log2(P) = %.1f\n",
           (1024.0 - 1.0) / 10.0);
    return 0;
}
```

【代码做什么？】 1. 用 `enum class CS { I, S, M }` 表示**每条 cache line 在每个 node 私有 cache 里的 MSI 状态**；`DirEntry` 就是讲义 slide 8 的目录项：`presence`（64 位位向量，`P = 64` 时正好够用）+ `dirty` + `owner`。 2. 每个 `readMiss*` / `writeMiss*` 函数**对应讲义的一个 slide**：函数体里逐步累加 `msgs`（消息条数）、`hops`（关键路径跳数）、`snooped`（必须做动作的 cache 数），并**同步更新目录与各 cache 的状态**——所以它既是"计数器"也是一个**可执行的协议状态机**（跑完之后目录与 cache 的状态与讲义图的最终状态一致）。 3. `readMissDirty_original` 与 `readMissDirty_forward` 的差别只有**消息的接收方**：前者 `owner → 请求方`、`home` 再被通知一次（5 条消息、4 跳）；后者 `owner` **同时**发给请求方和 home（4 条消息、**3 跳**），这正是讲义 slide 43 的"3/4. Response: data（2 msgs: sent to both home node and requestor）"。 4. `main()` 复现讲义 slide 10~18 的三类事件并打印**表 1**的数字，然后用 `storage overhead = (P+1)/512` 打印讲义 slide 23、28 的百分比，最后算出"limited pointer 在多大规模下失去意义"的临界指针数。

【实测输出（本笔记写作机器上 `g++ -O3` 运行结果）】

```
== directory vs snooping : cost of ONE coherence event (P=64) ==
read miss, clean (directory)               | msgs   2 | crit-hop  2 | caches-involved   1
read miss, clean (snooping bcast)          | msgs   2 | crit-hop  2 | caches-involved  64
read miss, dirty (5-msg original)          | msgs   5 | crit-hop  4 | caches-involved   2
read miss, dirty (intervention fwd)        | msgs   4 | crit-hop  4 | caches-involved   2
read miss, dirty (request fwd)             | msgs   4 | crit-hop  3 | caches-involved   2
write miss, k=2 sharers (directory)        | msgs   6 | crit-hop  4 | caches-involved   3
write miss, k=2 (snooping BusRdX)          | msgs   2 | crit-hop  2 | caches-involved  65
write miss, k=2 (dirty, request fwd)       | msgs   6 | crit-hop  5 | caches-involved   3

== directory storage overhead (64 B line = 512 bit) ==
P=  64 : full vector   12.5% | limited ptr k=5  6.05% | k=100 117.38%
P= 256 : full vector   50.0% | limited ptr k=5  8.01% | k=100 156.45%
P=1024 : full vector  200.0% | limited ptr k=5  9.96% | k=100 195.51%
breakeven pointer count at P=1024 : k < (P-1)/log2(P) = 102.3
```

【并行机制与性能解说】 – **模拟器本身的并行性 = 1**：这是**串行分析工具**，`Work = θ(Σᵢ (kᵢ + 2))`（每个事件要构造/统计 `k+2` 条消息），`Span = θ(E)`（E 个事件串行处理），**并行度 ≈ k̄ + 2**，所以它只适合作为"记账器"，不适合当性能测试。**被建模的机器**才是并行的：`P` 个 node 可以**同时**发起各自的一致性事件，只要它们打到的目录分区不冲突——这正是目录"分区（sliced）"带来的并行度 `≈ P`。 – **消息数决定带宽，跳数决定延迟**：`msgs` 影响互连网络的**吞吐量上限**（每条消息占用链路周期与 cache 控制器时间），`hops` 影响**单次 miss 的延迟**，`snooped` 影响**每个 cache 的 snoop 工作量**。三者必须分开优化：intervention forwarding 只降 `msgs`（5→4），request forwarding 只降 `hops`（4→3）。 – **瓶颈在哪**：真机上，`hops` 直接乘上每跳延迟（下面 §4.1 算例用 30 ns/跳）；`msgs × k` 决定了写密集负载的流量墙；`snooped = P` 说明了广播方案在 `P` 增大时被"卷入"的 cache 数线性增长，而目录方案只有 `k + 1`（P=64、k=2 时是 **3 vs 65，差 21 倍**）。

3.2 示例 2：pthreads 一致性流量探针（共享一行在真实机器上值多少纳秒）

- **代码**（完整可运行；三种写法只差在"几个线程碰同一条 cache line"）：

```c
// ============================================================================
// coherence_probe.c -- 量化"一条 cache line 被多个核同时写"的代价
//   A: 每个线程写自己独占的 line        (无一致性流量)
//   B: 所有线程写同一条 line            (每次写都可能触发独占 + 失效)
//   C: 所有线程对同一条 line 做原子 RMW (每次操作都要所有权迁移)
// 编译 (release): gcc -O3 -pthread -std=c11 coherence_probe.c -o coherence_probe
// 运行: ./coherence_probe <threads> <total_increments>
// ============================================================================
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define LINE_LONGS   8          /* 8 x 8 B = 64 B = 一条 cache line */
#define MAX_THREADS 32

static long g_rounds;           /* 每个线程的增量次数 */
static long g_threads;

static volatile long private_slots[MAX_THREADS * LINE_LONGS]; /* 每线程一条私有 line */
static volatile long shared_line_slots[MAX_THREADS];          /* 所有线程挤在 line 0 */
static volatile long g_counter;                               /* 原子 RMW 的目标 */

typedef struct { int tid; } arg_t;

static double now_s(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

/* A: 私有 line —— 写命中本核的 M 副本, 不产生任何一致性流量 */
static void* worker_private(void* p) {
    const int tid = ((arg_t*)p)->tid;
    volatile long* slot = &private_slots[(size_t)tid * LINE_LONGS];
    for (long i = 0; i < g_rounds; ++i) *slot += 1;
    return NULL;
}

/* B: 所有线程写同一条 line —— 每次所有权迁移都要让其他 sharer 的副本失效 */
static void* worker_shared(void* p) {
    volatile long* slot = &shared_line_slots[0];
    for (long i = 0; i < g_rounds; ++i) *slot += 1;
    (void)p;
    return NULL;
}

/* C: 原子 RMW —— 每次操作都必须拿到独占权, 成本 = 一次所有权迁移 */
static void* worker_rmw(void* p) {
    const long per = g_rounds / g_threads;
    for (long i = 0; i < per; ++i) __sync_fetch_and_add((long*)&g_counter, 1);
    (void)p;
    return NULL;
}

static double run(void* (*fn)(void*)) {
    pthread_t th[MAX_THREADS];
    arg_t     a[MAX_THREADS];
    const double t0 = now_s();
    for (long i = 0; i < g_threads; ++i) { a[i].tid = (int)i; pthread_create(&th[i], NULL, fn, &a[i]); }
    for (long i = 0; i < g_threads; ++i) pthread_join(th[i], NULL);
    return now_s() - t0;
}

int main(int argc, char** argv) {
    g_threads = (argc > 1) ? atol(argv[1]) : 8;
    g_rounds  = (argc > 2) ? atol(argv[2]) : 40000000L;
    if (g_threads < 1) g_threads = 1;
    if (g_threads > MAX_THREADS) g_threads = MAX_THREADS;
    const long per   = g_rounds / g_threads;
    const long total = per * g_threads;
```

```
    const double tp = run(worker_private);
    const double ts = run(worker_shared);
    const double tr = run(worker_rmw);

    printf("threads=%ld  ops/thread=%ld  total=%ld\n", g_threads, per, total);
    printf("A private line       : %7.3fs -> %6.2f ns/op  (no coherence traffic)\n",
           tp, 1e9 * tp / total);
    printf("B one shared line    : %7.3fs -> %6.2f ns/op  (%ld threads fight)\n",
           ts, 1e9 * ts / total, g_threads);
    printf("C atomic RMW on line : %7.3fs -> %6.2f ns/op  (ownership transfer)\n",
           tr, 1e9 * tr / total);
    printf("B/A = %.1fx   C/A = %.1fx\n", ts / tp, tr / tp);
    return 0;
}
```

【代码做什么？】 三个 worker 做**完全相同数量的增量操作**（每个线程 `per` 次，总共 `total = per × g_threads` 次），唯一区别是**这些操作打在几条 cache line 上**：– A：`private_slots[tid*8]`，每个线程写**自己独占的 64 B 行**（`volatile` 阻止编译器把循环优化掉，也保证每次都是真正的 load–add–store）→ 每次写都是**本地 cache 命中**；– B：所有线程写 `shared_line_slots[0]`，同一条 line **被 8 个核反复争抢**；– C：`__sync_fetch_and_add` 对同一条 line 做**原子读改写（RMW）**，每次操作都必须先拿到**独占所有权（read-exclusive）**，即每次操作都要一次"失效 → 授权 → 数据搬运"的所有权迁移——这正是**目录协议里 `SHARED → DIRTY`（`2 + 2k` 条消息、4 跳）**或监听协议里 `BusRdX` 广播的软件可见成本。

【实测数据（本笔记写作机器，8M 次总操作，取一次运行）】

线程数	A 私有 line (ns/op)	B 同一条 line (ns/op)	C 原子 RMW (ns/op)	B/A	C/A
1	0.48	0.40	2.45	1.0×	6.1×
2	0.44	1.38	6.44	3.1×	14.6×
4	0.48	6.57	5.96	13.7×	12.4×
8	0.53	16.22	10.51	30.6×	19.8×
16	0.57	87.30	26.20	**153×**	**46×**

- **三个现象与讲义的对应关系**：
- **单线程时 B 与 A 一样快（0.40 vs 0.48 ns）**——因为此时**没有别的 sharer**，写命中已独占的 M 副本，一致性协议完全不出现在关键路径上。这与讲义 slide 8"目录项记录 sharer"的前提一致：**共享者少 = 协议几乎免费**。
- **B 随线程数急剧恶化（0.40 → 87 ns/op，153 倍）**：每多一个 sharer，每一次所有权迁移就要**多发一条失效并等一个 ack**，即 `k` 直接进入成本。这正是讲义 slide 21 里 "**frequently read/written objects: frequent invalidations**" 与 "**high-contention locks: can be a challenge, because many readers present when lock released**" 的实证。
- **C 的增长比 B 缓和（16 线程时 26 vs 87 ns）**：原子 RMW 只要求"独占"，不要求其他核先读到旧值，失效/授权在硬件里流水化处理；而 B 的普通 load–add–store 会出现**读写交替的震荡（thrashing）**。
- **Work / Span / 并行度**（取 8 线程、总操作 8M）：
- **A（私有行）**：`Work = 8,000,000` 次增量；`Span = 500,000 × t_store_forward`（每个线程一条**依赖链**，`t_store_forward ≈ 3~5 cycle ≈ 1 ns`）→ `Span ≈ 0.5 ms`；**并行度 = Work/Span = 8 = T**，完全可扩展；实测 0.004 s（含线程创建/调度开销），每个线程 0.53 ns/op ≈ 2 cycle。
- **B（同一行）**：`Work` 仍是 8M 次写，但**所有写必须串行地拥有那一条 line**：`Span = 8,000,000 × t_transfer`，其中 `t_transfer ≈ 16.2 ns` → `Span ≈ 130 ms` = **实测 0.130 s**。**并行度 = 1**，加线程只会变慢（16 线程 87 ns/op 说明 `t_transfer` 本身还随 `k` 变大而上升）。**可扩展性上限 = 1/(t_transfer × k)**。
- **C（原子 RMW）**：`Span = 8,000,000 × t_transfer ≈ 8M × 10.5 ns ≈ 84 ms` = 实测 0.084 s；**并行度 = 1**，单行吞吐上限 ≈ `1/10.5 ns ≈ 95 M ops/s`。
- **瓶颈：延迟受限（latency-bound），不是带宽受限**。用 Little's law 验证：要让 8 个线程各以 1 op/ns 推进（8 G ops/s），在 `t_transfer = 16.2 ns` 下需要**同时在飞的迁移数 = 8 G/s × 16.2 ns ≈ 130**；但**一条 cache line 上同一时刻只允许一次迁移**，所以上限只能是 `1/16.2 ns ≈ 61.7 M ops/s`——**实测 8M/0.130 s = 61.5 M ops/s，与预测几乎完全一致**。这就是"**每行每时刻一次迁移**"这个硬件约束的直接后果，也是为什么目录协议要把 `k` 压小、把失效并行发出。

3.3 示例 3：OpenMP first-touch —— cc-NUMA 上"数据放在哪个 node"

- **代码**（完整可运行；对比"串行 first touch"与"并行 first touch"两种数据布局）：

```
// ========================================================================
// numa_first_touch.c -- cc-NUMA 上数据放置(first touch)对带宽的影响
//   串行初始化  -> 所有页的 home node = master 所在 node
//   并行初始化  -> 每个线程触碰自己的那份 -> 页分散到各 node (LOCAL)
// 编译 (release): gcc -O3 -fopenmp -std=c11 numa_first_touch.c -o numa_first_touch
// 运行: OMP_NUM_THREADS=8 ./numa_first_touch 8000000
// ========================================================================
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    const size_t N = (argc > 1) ? strtoull(argv[1], NULL, 10) : (size_t)8 * 1024 * 1024;
    const size_t bytes = N * sizeof(double);
    double *as = malloc(bytes), *bs = malloc(bytes), *cs = malloc(bytes); /* serial placement */
    double *ap = malloc(bytes), *bp = malloc(bytes), *cp = malloc(bytes); /* parallel placement */
    if (!as || !bs || !cs || !ap || !bp || !cp) { fprintf(stderr, "alloc failed\n"); return 1; }

    const int P = omp_get_max_threads();
    double t0 = omp_get_wtime();
    for (size_t i = 0; i < N; ++i) as[i] = 1.0;   /* 单线程 first touch: 页全落在 master 的 node */
    for (size_t i = 0; i < N; ++i) bs[i] = 2.0;
    for (size_t i = 0; i < N; ++i) cs[i] = 0.0;
    const double t_place_serial = omp_get_wtime() - t0;

    t0 = omp_get_wtime();
    #pragma omp parallel for schedule(static)
    for (size_t i = 0; i < N; ++i) ap[i] = 1.0;   /* 并行 first touch: 页落在触碰它的线程所在 node */
    #pragma omp parallel for schedule(static)
    for (size_t i = 0; i < N; ++i) bp[i] = 2.0;
    #pragma omp parallel for schedule(static)
    for (size_t i = 0; i < N; ++i) cp[i] = 0.0;
    const double t_place_par = omp_get_wtime() - t0;

    double best_s = 1e30, best_p = 1e30;
    for (int rep = 0; rep < 5; ++rep) {           /* triad: c[i] = a[i] + 2*b[i] */
        t0 = omp_get_wtime();
        #pragma omp parallel for schedule(static)
        for (size_t i = 0; i < N; ++i) cs[i] = as[i] + 2.0 * bs[i];
        double t = omp_get_wtime() - t0; if (t < best_s) best_s = t;

        t0 = omp_get_wtime();
        #pragma omp parallel for schedule(static)
        for (size_t i = 0; i < N; ++i) cp[i] = ap[i] + 2.0 * bp[i];
        t = omp_get_wtime() - t0; if (t < best_p) best_p = t;
    }
    const double triad_bytes = 3.0 * (double)bytes;   /* 2 读 + 1 写 */
    printf("threads=%d  N=%zu (%.1f MB/array)\n", P, N, bytes / 1048576.0);
    printf("placement : serial %.4fs | parallel %.4fs\n", t_place_serial, t_place_par);
    printf("triad serial-placement  : %.4fs -> %7.2f GB/s\n", best_s, triad_bytes / best_s / 1e9);
    printf("triad parallel-placement: %.4fs -> %7.2f GB/s\n", best_p, triad_bytes / best_p / 1e9);
    printf("locality gain = %.2fx\n", best_s / best_p);
    free(as); free(bs); free(cs); free(ap); free(bp); free(cp);
    return 0;
}
```

【代码做什么？】 分配**两组**同样的向量三元组（`a/b/c`）。第一组用**单线程**循环初始化：Linux/大多数 OS 采用 **first touch（首次触碰）** 页分配策略，因此**所有页都落在 master 线程所在的 node**（讲义 slide 4 的 cc-NUMA 语义：home node 由内存物理位置决定）。第二组用 `#pragma omp parallel for schedule(static)` 并行初始化：**每份页由将来使用它的线程触碰**，于是页分散到各自 node，成为**本地内存（local）**。随后分别对两组跑 5 次 triad（`c = a + 2b`），取最快的一次，用 `3 × N × 8 B` 计算**实测带宽**并给出 locality gain。

【实测数据（本笔记写作机器，`OMP_NUM_THREADS=8`，N = 8M，每数组 61 MB，超出一级/二级 cache，是真实 DRAM 带宽测试）】

```
threads=8  N=8000000 (61.0 MB/array)
placement : serial 0.0903s | parallel 0.0192s
triad serial-placement  : 0.0014s ->  139.00 GB/s
triad parallel-placement: 0.0009s ->  211.85 GB/s
locality gain = 1.52x
```

- 说明：这台机器是多 socket 的大内存机器，因此**串行放置确实付出了跨 node 访问的代价**：139 GB/s → 212 GB/s，**1.52 倍**。如果在单 socket（非 NUMA）或虚拟机上运行，两个数字会几乎相同——**这恰好就是讲义 slide 4 的第二个要点**：只有"就近访问"这一半做好了还不够，**一致性协议本身也必须可扩展**（否则每次 miss 的广播会把 locality 省下的时间吃掉）。
- **Work / Span / 并行度**：
- 放置阶段（并行）：`Work = 3N` 次写（每数组 N 次），`Span = O(N/P + t_sync)`（每个线程负责连续的一段，`schedule(static)` 块大小 ≈ `N/P`），**并行度 = P = 8**，实测 0.0192 s（串行 0.0903 s，加速 4.7×，**低于 8× 是因为单线程也受单核 store 带宽限制，而 first touch 本身要分配页（page fault）**）；
- 计算阶段（triad）：`Work = 3N` 次内存操作 + N 次 FMA，`Span = O(N/P + t_barrier)`，**并行度 = P**；但真正的上限是**内存带宽**而不是并行度（见下）。
- **瓶颈与算术强度**：`c[i] = a[i] + 2*b[i]` 每个元素 = **2 flops、24 B 流量**（读 8 B + 读 8 B + 写 8 B，写分配时可能还要读一次），**算术强度 AI = 2/24 ≈ 0.083 flop/B**。在这台机器实测 212 GB/s 的带宽下，Roofline 上限 ≈ `212 × 0.083 ≈ 17.6 GFLOP/s`；而 8 个核的算力上限（按每核每周期 16 个双精度 flop、3.5 GHz 估算）≈ `8 × 16 × 3.5 = 448 GFLOP/s`。**结论：这类负载是 25 倍以上的带宽受限**——所以 cc-NUMA 优化的正确顺序是：先把**数据放对 node**（本例 1.52×），再压**一致性流量**（伪共享/padding），最后才考虑计算侧优化。

---

4. 性能模型与复杂度分析

4.1 一致性事件的延迟模型：`hops` 模型 + 数值算例

- **模型**：讲义 slide 37 自己给出了这个模型的骨架——**关键路径 = 一串必须依次发生的依赖操作**。把它写成公式：

```
T_event  =  (关键路径跳数 H) x t_hop              <- 网络延迟（消息必须按序往返）
          +  (目录查询次数 n_dir) x t_dir         <- 目录 bank 的访问时间
          +  t_cache                                  <- owner/请求方 cache 的数据访问
          +  payload_bytes / link_bw                  <- 数据串行化(带宽项)

吞吐量视角（Little's law）：
    sustained_misses/s  =  in_flight_misses / T_event
     (in_flight 受 MSHR 数量限制：每核每级 cache 通常 8~16 个未完成 miss)
```

- **表 5：数值算例的参数假设（本笔记补充，取值量级参考现代多核/多 socket 机器与讲义 slide 37~43 的跳数结构）**

参数	取值	含义
`t_hop`	30 ns	一跳点对点消息的延迟（含路由器/链路）
`t_dir`	10 ns	home 目录 bank 的一次查询/更新（SRAM 命中）
`t_cache`	5 ns	请求方 cache 接收数据的开销
`t_owner`	10 ns	owner cache 取出脏数据的开销
`link_bw`	20 GB/s/方向	每条链路的带宽
payload	64 B（数据）/ 8 B（控制消息头）	数据线大小与控制消息

- **算例 1：三种 dirty 读 miss 的延迟与流量**（H 取讲义的关键路径跳数）

方案	关键路径（跳）	目录访问次数	延迟计算	延迟	消息数	线上字节数
原始 5 消息	4	1	4×30 + 10 + 10(owner) + 3.2	**143 ns**	5	8+8+8+72+72 = **168 B**
intervention forwarding	4	**2**	4×30 + 2×10 + 10 + 3.2	**153 ns**	4	8+8+72+72 = **160 B**
request forwarding	**3**	1	3×30 + 10 + 10 + 3.2	**113 ns**	4	8+8+72+72 = **160 B**

> **重要细节（本笔记对讲义 slide 38~40 的定量解读）**：intervention forwarding **只减流量、不减延迟**——因为 home 变成数据中转站，它要**访问两次目录**（一次查、一次改），关键路径仍然是 4 跳，算下来延迟反而**比原始方案高约 7%**（153 vs 143 ns）。这正是讲义在 slide 40 追问 "**But all four of the transactions are on the critical path. Can we do better?**" 的原因。**request forwarding 把 home 移出数据通路**（关键路径 3 跳），延迟降低 **21%**，流量与 intervention 相同（160 B），代价是**打破了纯请求/响应模型**（请求方要接受来自第三方的应答）。
>
> 算 clean 读 miss 作为对照：`2×30 + 10 + 5 + 3.2 ≈ 73 ns`，即**clean 读比 dirty 读快近一倍**——这也解释了为什么"把数据尽快写回内存/共享化"（owner 从 M 降到 S）对后续读者很值：一次 `DIRTY → SHARED` 的 owner 更新，能让**之后所有**读 miss 从 143 ns 降到 73 ns。
>
> **吞吐量视角**：若 `T_miss ≈ 100 ns`、每核 10 个 MSHR、64 核，则同时在飞的 miss = 640 → 系统可持续 miss 吞吐 ≈ `640 / 100 ns = 6.4 G miss/s`。所以**延迟本身通常不是墙**；真正的墙是 §4.3 的"每事件工作量随 P 增长"与 §3.2 的"**每条 line 每时刻只允许一次所有权迁移**"。

4.2 目录存储开销模型：精确算例

```
full bit vector :  overhead = (P + 1) / line_bits                    <- line_bits = 512 (64 B)
limited pointer :  overhead = (1 + k * log2(P)) / line_bits
sparse (list)   :  memory side : 1 pointer per line, but ONLY for lines present in some cache
                   cache  side : (next pointer) bits per CACHED line   -> scales with C, not M
sparse (vector) :  cache  side : P bits per CACHED line   (P * C bits per node)
```

- **算例 2（讲义 slide 29~33 的量级，本笔记补齐算术）**：设 **P = 64** 个 node，每 node **1 MB 私有 cache**、**1 GB DRAM**，**64 B 行**：

方案	每 node 目录存储	占 DRAM 比例	相对 full vector
full bit vector	16,777,216 行 × 64 bit = 1.07 Gbit = **128 MiB**	12.5%	1×
limited pointer（k=5）	16,777,216 × 31 bit = **62 MiB**	6.05%	2.1× 更省
sparse，每缓存行存 P 位向量（`P·C`）	16,384 行 × 64 bit = **128 KiB**（在 SRAM 里）	——	**1024×** 更省
sparse，链表指针（`C·log₂P`）	16,384 × 6 bit = **12 KiB**（+ 每行 6 bit 的 next 指针）	——	**≈10,900×** 更省

> 算例 2 的关键在于**分母不同**：full vector 与 limited pointer 的分母是**内存行数 M**（1 GB/64 B = 1677 万行），sparse 的分母是**在 cache 里的行数 C**（1 MB/64 B = 16384 行）。**M/C = 1024**，这就是讲义 slide 29 说"**1 MB cache、1 GB memory → 99.9% 的目录项是空的**"的算术来源（`1 − 16384/16777216 = 99.90%`）。Sparse 目录"用延迟换空间"：写失效沿链表串行传播，延迟从 `O(1)` 跳变成 `O(k)` 跳。

4.3 一致性的"工作量"模型：O(P) vs O(k)，以及 Amdahl 算例

```
每个一致性事件消耗的全局资源（讲义的核心判据）：
   snooping  (broadcast) :  P 个 cache 各做一次 tag 查询 + 总线仲裁
   directory (point-2-pt):  k 个 sharer + 1 个 home 参与, 其余 node 完全不知道

系统级一致性的"工作量速率"（每秒钟的 snoop/协议工作）：
   snooping  :  W_snoop = R x P x t_lookup        / P 个 cache  =  R x t_lookup 每个 cache
   directory :  W_dir   = R x (k_bar + 1) x t_lookup / P 个 cache ≈ R x (k_bar+1) x t / P
   R = 全系统一致性事件速率; t_lookup = 一次 tag 查询/协议状态机的 cache 控制器时间
```

- **算例 3（本笔记补充的假设：每核每秒产生 1 M 个一致性事件、`t_lookup = 2 ns`、`k̄ = 2`、平均每条事件 100 B 线上流量、总线 32 GB/s）**：

系统	R（事件/秒）	每个 cache 控制器的协议工作量	互连需求	结论
P=64，监听广播	64 M	64 M × 2 ns = 0.128 s/s = **12.8%**	64 M × 72 B = **4.6 GB/s**（占 32 GB/s 总线的 14%）	还能用
P=1024，监听广播	1024 M	1024 M × 2 ns = 2.05 s/s = **205%**	1024 M × 72 B = **73.7 GB/s** = 总线的 **230%**	**不可实现**（两条墙同时撞上）
P=1024，目录	1024 M	自己的 1 M × 2 ns + 作为 sharer 的 2 M × 2 ns = **6 ms/s = 0.6%**	≈1024 M × 100 B = **102 GB/s**，但分摊在可扩展网络上（如 32 条 × 20 GB/s = 640 GB/s，利用率 16%）	**可行**

> 这张表的含义就是讲义 slide 19、35、46 的定量版本：**监听式的问题不是"延迟高"，而是"每个事件要被 P 个 cache 处理、所有流量挤在同一条介质上"，于是 P 增大时 per-cache 工作量与总线带宽同时爆掉（205% / 230%）**；而目录把每个事件的参与者从 `P` 降到 `k + 1`，在 P=1024 时每个失效路径的节点只花 0.6% 的时间，流量分摊到可扩展的拓扑上。

- **Amdahl 算例（本笔记补充）**：设某应用时间中 **80% 是可完美并行的计算、20% 是一致性处理**。
- 广播方案下，一致性单事件成本随 `P` 增长（`O(P)`）：从 P=64 到 P=256（4 倍核）时，一致性部分变成 `0.20 × 4 = 0.80`，于是 `T(256)/T(64) = 0.80/4 + 0.80 = 0.20 + 0.80 = 1.00` → **4 倍核数带来 0 倍加速**（全部收益被一致性吞掉）。
- 目录方案下，一致性成本近似不随 `P` 增长（`O(k)`，而 `k` 增长很慢，讲义 slide 20）：`T(256)/T(64) = 0.80/4 + 0.20 = 0.40` → **2.5× 加速**（受 Amdahl 限制，理想上限 4×）。
- 反过来看单机优化：若一致性占 30%，把它优化 4 倍，则 `T = 0.70 + 0.30/4 = 0.775` → 加速 **1.29×**；而**不管一致性优化到多快，总加速上限是 `1/0.70 = 1.43×`**——这提醒我们：协议优化无法拯救"共享本身太多"的程序。

4.4 算术强度与 Roofline：伪共享吃掉的带宽

```
伪共享/热点计数器的"算术强度"（每字节一致性流量能做多少 flop）：
   packed 8 counters in ONE 64 B line, 8 threads:
       每次更新 = 1 次所有权迁移 -> 搬 64 B
       AI = 2 flop / 64 B = 0.031 flop/B
       Roofline 上限 = BW_coherence x AI = 50 GB/s x 0.031 = 1.56 GFLOP/s
   padded (each counter in its own line):
       更新命中本核 M 副本 -> 一致性搬移 ~ 0 B
       AI 不再是限制；上限回到计算/流式带宽
```

- **算例 4（把 §3.2 的实测串起来）**：
- 单行打包：实测 **16.2 ns/op**（8 线程）→ 单行吞吐上限 `1/16.2 ns = 61.7 M ops/s`；一致性数据搬运 = `61.7 M × 64 B = 3.95 GB/s`（全部集中在**一条** line 的迁移上）。
- 若这个打包结构是"任务队列的 8 个计数器"，每个 push/pop 触发一次 `k=8` 的写 miss：`msgs = 2 + 2k = 18` → 需要的消息速率 = `61.7 M × 18 = 1.11 G msgs/s`。**这个数字超过多数互连网络的报文速率上限（本笔记假设 0.5 G msgs/s）**——所以**再优化协议也没用，必须消除伪共享**。
- padding 之后：每线程写自己的 64 B 行，实测 **0.53 ns/op**，8 线程聚合 ≈ `15.1 G ops/s`，一致性流量 ≈ 0 → **吞吐提升 ≈ 245×**（15.1 G / 61.7 M）。
- 边界型伪共享（stencil 的分块边界）则没有那么可怕：`N = 10⁸` 个 double、8 个 node、每个 node 100 MB，每次迭代只有**相邻 node 交界处的 2 条 line** 会来回迁移；1000 次迭代的边界一致性流量 ≈ `1000 × 2 × 64 B × 8 = 1 MB`，相对 `1000 × 3 × 800 MB = 2.4 TB` 的流式流量可忽略。**结论：伪共享的代价与"该行被访问的频率 × 参与 node 数"成正比，而与"它属于哪个数据结构"无关。**

4.5 Work–Span 与可扩展性上限（三个代码示例的统一视角）

代码/场景	Work（总工作量）	Span（关键路径）	并行度 = Work/Span	真正的瓶颈
示例 1 目录模拟器（本身）	`θ(Σ(kᵢ+2))` 消息记账	`θ(E)` 串行事件	`≈ k̄ + 2`（很小）	单线程记账；价值在计数而非速度
示例 1 建模的机器	`P` 个 node 各自的 miss 可并行	单次 miss 的 3~4 跳	`≈ P`（目录分区可并行服务）	目录 bank 冲突、网络带宽
示例 2A 私有 line	`T × per` 次增量	`per × t_store_forward`	`T`（完美）	store-to-load forwarding 延迟
示例 2B 同一条 line	同上	**`总量 × t_transfer`（= 全部工作量串行）**	**1**	每条 line 每时刻仅一次迁移 + `O(k)` 失效
示例 2C 原子 RMW	同上	`总量 × t_transfer`	1	同上（RMW 请求独占所有权）
示例 3 并行 first touch	`3N` 次触碰（+ 页分配）	`N/P + t_sync`	`P`	页分配开销、单核 store 带宽
示例 3 triad	`3N` 次访存 + `N` 次 FMA	`N/P + t_barrier`	`P`	**内存带宽**（AI = 0.083 flop/B）

- **可扩展性上限的一般判据（本笔记总结）**：

- **只读/clean 读路径**：目录下 `O(1)` 跳、`O(1)` 消息 → 可扩展性与 `P` 无关，这是目录**最强**的场景（讲义 slide 19："On reads, directory tells requesting node exactly where to get the line from"，全程点对点）。
- **写路径**：目录需要通知 `k` 个 sharer → `O(k)` 消息；**在极限情况下（所有 cache 都共享这一行，`k = P`）它与广播一样糟**（讲义 slide 19 明确写了这一点）。所以**目录的优点完全建立在"`k` 小且随 P 增长慢"这一经验事实之上**（slide 20~21）。
- **每条 line 的串行化**：不论协议怎么优化，一条 line 上同一时刻只能有一次所有权迁移 → 热点行吞吐上限 `1/t_transfer`（实测 61.7 M ops/s）→ 高争用结构的正确解法是**分片（sharding）/ padding**，不是更快的协议。
- **存储**：`full vector = O(P·M)`、`limited pointer = O(k̄ log P·M)`、`sparse = O(P·C)` 或 `O(C log P)` ——**目录的存储在 P 大时必须换表示法**，这是本讲后半部分的全部内容（slide 23~34）。

4.6 复杂度汇总

维度	监听式（snooping）	目录式（full vector）	目录式（limited pointer）	目录式（sparse list）
每次事件的消息数	`O(P)` 个观察者	`2`（读）/ `2+2k`（写）	同左 + 溢出路径	同左，但失效**串行**
事件关键路径	`O(1)` 事务（但仲裁/负载差）	clean 2 跳 / dirty 3~4 跳	同左	`O(k)` 跳（写）
目录存储	0	`O(P·M)` 位	`O(k̄ log P · M)` 位	`O(P·C)` 或 `O(C log P)`
`P` 增大时	per-cache 工作量 × `P`、总线饱和	存储爆掉	溢出频繁	写延迟变长
实现复杂度	最低（总线天然广播）	低	中	高（链表插入/删除/换出修补）

---

5. 关键要点

1. **"广播不 scale，但我们并不需要广播"**（讲义 slide 46 的第一句话）：一致性只需要"**该知道的人**"知道——**把 sharer 列表存在目录里、按需查询、点对点通信**，就把每个事件的参与者从 `O(P)` 降到 `O(k)`。目录的收益**不是**更低的单次 miss 延迟（clean 2 跳、dirty 3~4 跳，往往比广播还慢），而是**流量与协议工作量不再随机器规模爆炸**（§4.3 算例：P=1024 时 per-cache 协议工作量 205% → 0.6%）。
2. **目录设计有两条正交的优化主线**（讲义 slide 35）：**(a) 目录结构的存储开销**——增大 line、把多个处理器并成一个 directory node、limited pointer schemes（`1 + k log P` 位，k=5 时开销 5.8%~9.7%，几乎与 P 无关）、sparse directories（只给"在 cache 里"的行留信息，1 MB cache / 1 GB memory 下 99.9% 的目录项为空，存储从 128 MiB 降到 12 KiB）；**(b) 协议的消息数与关键路径**——intervention forwarding（5→4 条消息，关键路径不变）、request forwarding（关键路径 4→3 跳，代价是打破纯请求/响应模型）。
3. **所有压缩方案都用"常见情况优化"的同一套方法论**（讲义 slide 27）：① 用 workload 观察事实（**sharer 很少**）作依据；② 让常见情况既简单又快（前 N 个 sharer 用指针数组）；③ 罕见情况仍然**正确**，只是更慢更复杂（溢出时广播 / 顶替 / 粗向量回退）；④ 罕见路径的开销可以容忍，因为它很少发生。同理，sparse directory 优化的是"**大多数行不在 cache**"这个事实，代价是写失效变成 `O(k)` 的串行链表遍历 + 更高的实现复杂度（换出时要修补链表）。
4. **失效的代价与"谁真的持有这一行"直接挂钩**（讲义 slide 20~21 的直方图 + §3.2 实测）：migratory 对象、任务队列、低争用锁的 sharer 极少，目录几乎免费；**高争用锁（锁释放时一堆读者在场）是唯一真正的难题**；而**伪共享**把"没人共享的数据"变成"永远有 2 个 sharer 的热点 line"，实测（8 线程同一条 line）每次操作成本 16.2 ns，是私有行（0.53 ns）的 **30 倍**，16 线程时恶化到 **153 倍**，且 padding 后可提升约 **245 倍**吞吐。
5. **真实机器是"两级目录 + 分层协议"**（讲义 slide 44、45）：Intel Core i7 用 **L3 兼作集中式目录**（依赖 **inclusion property**：L2 里有的行在 L3 目录里必有条目），目录维度是 **P=4、C=L3 行数**（sparse 思路）；多 socket 在 **home agent / memory controller** 侧再加一层 **in-memory directory（16 KB dir cache 缓存目录项）**，用 QPI 相连。软件侧能控制的只有 **first touch 决定 home node** 与 **线程亲和性**：实测把"串行初始化"改成"并行首次触碰"，同一段 triad 的带宽从 139 GB/s 升到 212 GB/s（**1.52×**）——**local 访问省下的是延迟与带宽，而一致性协议本身的可扩展性才决定这份收益能不能保住**。

---

6. 常见陷阱与注意事项

- **陷阱 1：以为"目录方案 = 更低的延迟"。** 目录引入一次**间接访问（indirection）**：clean 读 miss 2 跳、dirty 读 miss 3~4 跳（§4.1 算例：73 ns / 113~153 ns），而广播在同一总线上可能只要 2 个事务。**目录赢在流量与可扩展性，不在单次延迟**；在小规模机器（4 核、单 socket）上，L3 目录看似"多余"，但它是阻止所有 L2 被广播淹没的唯一办法。
- **陷阱 2：忽略 "`k = P` 的极限"。** 讲义 slide 19 明说：**写操作的目录优势取决于 sharer 数**，若所有 cache 都共享该行，目录必须与所有 cache 通信，**和广播一样糟**。很多人只记住"目录 = 好"，忘了它的前提是"**sharer 少且随 P 增长慢**"（slide 20~21 的直方图）。高争用锁 + 大

量读者会让这个前提失效。

- **陷阱 3：只优化消息数、忘了关键路径（或反之）。** intervention forwarding 把消息从 5 降到 4，但关键路径仍是 4 跳且 home 多访问一次目录，**延迟反而略升（143 → 153 ns）**；request forwarding 才是同时改善两者的做法（3 跳、4 条消息、160 B），但它**打破了请求/响应模型**——请求方必须接受"我没问过的节点发来的数据"，MSHR/网络接口要额外支持；实现时若仍按"请求–响应配对"写状态机，会出现**死锁或错误的状态等待**。
- **陷阱 4：用 full bit vector 硬撑大规模系统。** `P` 位/行 的开销是 `P/512`：P=64 时 12.5%（尚可）、P=256 时 50%（昂贵）、**P=1024 时 200%（比数据本身还大）**。此时必须换表示法，而每种压缩都有代价：limited pointer 需要溢出处理（广播回退/顶替并失效旧 sharer/粗向量），sparse list 把写失效变成**串行 O(k) 延迟**并要求换出时修补链表（容易出 bug：插入、删除、多头/尾指针不一致都会导致**漏失效 → 一致性被破坏**）。另外注意 limited pointer 的**临界点** $k < (P-1)/\log_2 P$（P=1024 时约 102 个指针）——超过它，指针方案比位向量还费空间。
- **陷阱 5：忘了 inclusion property。** L3 目录要能代表所有 L2 中的行，**前提是"任何 L2 里的行在 L3 里也有一份（或至少有目录项）"**。如果 L3 采用非包含（non-inclusive）策略又没有额外机制，目录就可能**漏掉某个 L2 里的 sharer → 该 L2 的副本不会被失效 → 读到陈旧数据**。这是实现级 bug，不是性能问题。
- **陷阱 6：把伪共享当成"协议问题"去优化协议。** 实测表明：同一条 line 被 8 个核写时，每次操作 16.2 ns、单行吞吐被**"每条 line 每时刻一次迁移"**硬性限制在 `1/t_transfer`；此时若做 `k=8` 的写 miss，需要 `18` 条消息 × 61.7 M/s = **1.11 G msgs/s**，任何目录实现都扛不住。正确做法是**结构层面消除共享**：按 cache line 对齐/padding（`alignas(64)`）、每线程私有累加后再归约（privatization）、把热点计数器**分片（sharding）** 到多条 line；同理，在 cc-NUMA 上还要注意 **first touch 决定了 home node**——`malloc` 之后由单线程初始化会把所有页钉在一个 node，之后所有远程访问都要付 3~4 跳的代价。
- **陷阱 7（实现细节）：忘记"收到全部 ack 才能写"。** 讲义 slide 18 强调：**"After receiving both invalidation acks, P0 can perform write"**。写者必须等**所有**失效应答到齐（否则其他 cache 可能仍持有旧副本并继续读旧值）。这要求写者维护"未完成失效计数"，并与 memory consistency 的定序规则配合——这也是为什么**高争用写比高争用读更昂贵**：读可以并行满足（多个 sharer 共存），写必须**串行地收齐所有 ack**。

---

7. 思考题（带答案）

思考题 1：request forwarding 把 dirty 读 miss 的关键路径从 4 跳压到 3 跳，消息数仍然是 4 条（与 intervention forwarding 相同）。请说明：(a) 数据与目录修订分别走了哪条路径？(b) 为什么讲义特别强调"系统不再是纯请求/响应（no longer pure request/response）"？(c) 如果 owner 把数据只发给请求方、忘了给 home 发目录修订，会造成什么后果？

【答案】 (a) 路径（对应讲义 slide 41~43 的编号）：– ① 请求节点 P0 → home：read miss 消息；– ② home → owner：一条**"send data to requestor"**的转发请求（home 不再亲自去取数据）；– ③ owner → **请求方 P0**：数据（这条在关键路径上）；– ④ owner → **home**：data + dir revision（与 ③ **并行**，所以只增加流量、不增加关键路径跳数）。 所以关键路径 = ①→②→③ = **3 跳**，总消息数 = **4 条**；而 intervention forwarding 是 ①P0→home、②home→owner、③owner→home、④home→P0，四条全部串在关键路径上（4 跳），home 还多做一次目录更新访问——**方向相反：数据绕了一大圈。** (b) 因为 **P0 发出的请求对象是 home，但收到的数据来自 owner**：请求方必须能处理"应答者 ≠ 被请求者"的情况。这会影响：(1) 网络/NoC 的路由与事务标签（transaction id）设计——应答必须能正确匹配到原来的未完成 miss，而不是被当成"未请求的数据"丢弃；(2) MSHR/状态机的实现——收到数据时该行可能处于 IS（in-flight shared）状态并等待"来源可能是 home 或 owner"的任一数据；(3) 调试与验证更难（报文流的因果链不再是简单的请求–应答配对）。(c) 后果：home 的目录项**永远停留在** `presence={old owner}, D=1`，且内存里的数据是陈旧的。之后若有第三个节点发 read miss，home 会**再次**告诉它"去找原 owner"；而原 owner 的副本已经改成 S（或已被换出），于是要么**多绕一跳**（性能损失），要么在 owner 已换出该行时**找不到数据**（协议死锁/错误）；更严重的是 home 认为行仍是 dirty，**内存永远不会被更新**，一旦 owner 的副本被静默丢弃（例如 cache 被作废/复位），系统就永久丢失了最新写值 → **一致性被破坏**。因此 owner **必须**把 dir revision 发给 home，让 home 清 dirty、更新 presence、并**把最新数据写回内存**（这正是讲义 slide 14 的第 5 步"home clears dirty, updates presence bits, updates memory"）。

思考题 2：设 P = 256 个 node，每个 node 1 MB 私有 cache、1 GB DRAM，cache line 64 B。(1) 计算 full bit vector 目录的存储量与占内存百分比；(2) 计算 limited pointer（k = 5）的每行目录位数与百分比；(3) 计算 sparse directory（链表指针方案）在内存侧与 SRAM 侧的存储量；(4) 解释为什么 sparse 方案的"写延迟"是 `O(k)` 跳，并给出 k = 16 时的具体跳数。

【答案】 (1) **Full bit vector**：每行需要 P = 256 bit。64 B 行 = 512 bit，故开销 = `256/512 = 50%`（与讲义 slide 23 一致）。每 node 内存 1 GB = 2^{30} B → 行数 $M = 2^{30}/64 = 2^{24} = 16{,}777{,}216$ 行；目录存储 = $16{,}777{,}216 \times 256\ \text{bit} = 4{,}294{,}967{,}296\ \text{bit} = 512\ \text{MiB}$（= 0.5 GB，正好是 1 GB 内存的 50%）。**这就是 P=256 时位向量方案不再可接受的原因。** (2) **Limited pointer（k = 5，$\log_2 256 = 8$）**：每行 $1 + 5\times 8 = 41$ bit（若按讲义 slide 28 略去那 1 位状态位，则为 $5\times 8 = 40$ bit）。百分比 = `41/512 = 8.01%`（讲义口径 `40/512 = 7.81%`，与 slide 28 的 "P = 256: 7.8% overhead" 完全一致）。存储 = $16{,}777{,}216 \times 41\ \text{bit} \approx 86\ \text{MiB}$（对比位向量的 512 MiB，**省 6 倍**），而且**这个开销几乎不随 P 增长**（P 从 64 到 1024，只从 6.05% 涨到 9.96%）。 (3) **Sparse directory（链表）**： – 内存侧（home node 的 DRAM）：每行**一个**

head 指针，但**只为当前被缓存的行保留**——本例只有 `C = 1 MB/64 B = 16,384` 行在 cache 里，`C × log₂P = 16,384 × 8 bit = 131,072 bit = 16 KiB`； – SRAM 侧（各 cache 的 tag 数组）：每个被缓存的行需要额外 `log₂P = 8` bit 的 "next" 指针 → 共 `16,384 × 8 bit = 16 KiB`，这是**与 cache 大小成正比**的（P·C 的向量变体则是 `16,384 × 256 bit = 512 KiB`）。 – 与 (1) 的 512 MiB 相比，**内存侧降低了约 32,768 倍**，原因是分母从"内存行数 M = 1677 万"变成了"cache 行数 C = 16384"（`M/C = 1024`，再叠加每行 256 bit → 8 bit 的压缩）。 (4) 因为 sparse 目录**只在 home 保存链表头**，其余 sharer 的指针**存在各自 cache line 的额外字段里**：写 miss 时 home 只能先失效 head，head 失效后才知道下一个是谁，**每一跳都要等上一次失效到达并回读指针**，因此失效是**串行**的。跳数（消息链）≈ `2 × k` 量级：`k = 16` 时 —— 1 跳 request→home，然后 16 次 "invalidate→ack" 串行往返 = `1 + 2×16 = 33` 跳（若把 home 更新目录与请求方拿数据算进去还要再加）。对比 full bit vector：失效消息**并行**发出，无论 k 多大，关键路径都只有 **4 跳**（request → response → invalidate → ack）。**结论：sparse 用"存储"换"写延迟"，所以它适合"sharer 极少、写不频繁"的负载，而高争用场景必须用位向量/粗向量来并行失效。**

思考题 3：某 8 线程数据结构的 8 个 `long` 计数器被放在同一个 64 B cache line 中；实测每次更新耗时 16.2 ns（单线程时是 0.40 ns）。(1) 用"每条 line 每时刻只允许一次所有权迁移"的原理算出单行吞吐上限；(2) 若改成每个计数器独占一条 line，估计吞吐提升倍数（已知独占行的实测是 0.53 ns/op）；(3) 如果这个结构是共享任务队列（每次操作触发 `k = 7` 的写 miss），按讲义的消息公式算出所需的报文速率，并说明为什么"换用更好的目录协议"救不了它、(4) 你会怎么改这个程序?

【答案】 (1) 打包时 8 个计数器共享一条 line，**每次更新都要拿到该行的独占所有权**，即每次更新一次所有权迁移。所有更新序列化在同一条 line 上，故单行吞吐上限 = `1 / t_transfer = 1 / 16.2 ns ≈ 61.7 M 次更新/秒`。用 Little's law 交叉验证：要让 8 个线程各以 1 op/ns 推进（8 G op/s），需要同时有 `8 G/s × 16.2 ns ≈ 130` 次迁移在飞，而硬件对**一条 line**只允许 1 次迁移在飞 → 上限 `1/16.2 ns = 61.7 M op/s`，与实测 `8 M / 0.130 s = 61.5 M op/s` 吻合（误差 < 0.5%）。这也解释了为什么线程数从 1 增到 16 时 B 方案的每操作耗时从 0.40 ns 恶化到 87 ns：**sharer 越多，每次迁移要失效/等待的对象越多，`t_transfer` 自身也变大**。 (2) 每个计数器独占一条 line 后，每个线程在**自己的** line 上反复写，命中本地 M 副本，**不再产生一致性流量**（前提：这些 line 不与其他线程共享）。8 条 line 的聚合吞吐 ≈ `8 × (1 / 0.53 ns) ≈ 15.1 G op/s`（实测 A 方案各线程耗时基本不随线程数变化：0.48 → 0.57 ns/op，说明确实完全并行）。相对打包的 61.7 M op/s，提升 ≈ `15.1 G / 61.7 M ≈ 245×`（保守说法：**两个数量级以上**）。注意这仍是上界：真实程序还要受内存带宽、总线上限与其他共享结构限制。 (3) `k = 7` 个其他 sharer 时，一次写 miss 的消息数 = `2 + 2k = 2 + 14 = 16` 条（1 条 write miss + 1 条 sharer ids+data + 7 条 invalidate + 7 条 ack，见讲义 slide 17、18）。按单行吞吐 61.7 M op/s 计算，所需报文速率 = `61.7 M × 16 ≈ 0.99 G msgs/s`（约每秒十亿条一致性报文）。这远超互连网络的报文处理能力（本笔记假设 0.5 G msgs/s 量级），而且每条报文还要占用 cache 控制器的状态机时间。**换协议救不了它**：因为瓶颈是"**这一行被 8 个核以高频率写**"这个结构性事实——目录已经只通知真正的 7 个 sharer（而不是广播给全部 P 个核），信息论上再也无法减少"必须让 7 个副本失效"这件事；除非改变**数据布局**（把计数器分开）或**访问模式**（减少写频率，例如本地批量累加后再合并）。 (4) 具体改法（按收益从高到低）： 1. **padding / 对齐**：`struct Counter { alignas(64) long v; } c[8];` 或 `long v[8*8]`（每 8 个 long 一个计数器），让每个线程写自己独占的 line； 2. **privatization（私有化 + 归约）**：每个线程在**栈上或私有数组**里累加，最后用一次归约合并——把 `O(ops)` 次一致性事件降到 `O(P)` 次； 3. **分片（sharding）**：把全局计数器拆成每 node 一份，读时求和（"分布式计数器"模式）； 4. 如果必须共享，就**降低争用频率**：批量交付（每次锁/原子操作处理 K 个任务）、用无锁队列的 head/tail 分离布局（避免同一行被生产者和消费者同时写）； 5. 最后才考虑协议/硬件层面：把该行**钉在某个 node**（例如通过内存交错或把做该工作的线程绑到同一 socket），让所有权迁移尽量在**片内 L3 目录**里完成而不是跨 QPI。

Lecture 13: Snooping-Based Multiprocessor Design

1. 章节标题与概述

Lecture 13: Snooping-Based Multiprocessor Design（基于侦听的多处理器设计：把一致性协议真正做进机器里）

- **本讲核心问题**：上一讲（Snooping-Based Cache Coherence）讨论的 MESI（Modified / Exclusive / Shared / Invalid，修改/独占/共享/无效）状态机是**抽象**的——它假设每条一致性消息都是**原子的（atomic）**、瞬时完成的。本讲要回答的是：**在真实机器里，如何高效地实现一个基于失效（invalidation-based）的侦听一致性协议?** 一旦承认"标签查找、总线仲裁、等待其他控制器响应、读写 DRAM 都不是原子操作"，正确性（死锁 deadlock、活锁 livelock、饥饿 starvation、竞态 race）与性能（总线带宽利用率、隐藏访存延迟）就会立刻互相拉扯。讲义的一句话总结是：**in a real machine... efficiently ensuring coherence is complex（在真实机器中，高效地保证一致性是很复杂的）**（slide 2）。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：原子共享总线（atomic shared bus）与**拆分事务总线（split-transaction bus）**；**处理器侧控制器（processor-side controller）**与**侦听控制器（snoop controller）**对同一份 tag/state 的争用，以及用**tag 复制（duplicate tags）**或**多端口 tag 存储（multi-ported tag memory）**来缓解；侦听结果的三条"线与"信号（Shared / Dirty / Snoop-pending）；**回写缓冲（write-back buffer）**；**请求表（request table）**与 **3 bit 事务标签（transaction tag = 表项索引）**；**响应分离的请求/响应队列**；**NACK（negative acknowledgement，否定确认）流控**；多级缓存层次中的**包含性（inclusion）**。

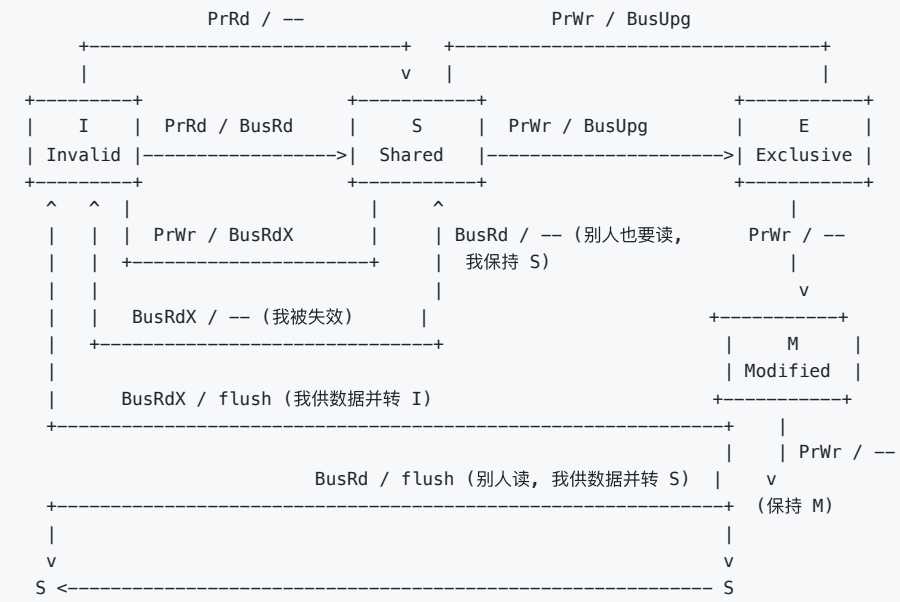
- **软件侧**：这一切最终要支撑的是普通程序中的一句 `int x = 10;`（slide 59）——一条被架构抽象成"原子"的 store，实际由十几个组件、二十来个步骤协同完成；讲义强调"这些概念远不止硬件实现"：**简单性与性能的折中、并行系统中的正确性挑战，同样适用于写并行程序**（slide 3）。
- **在并行计算知识体系中的角色**：本讲是"缓存一致性三部曲"的第三部（Snooping-Based Cache Coherence → Directory-Based Cache Coherence → 本讲的实现），把**协议层（protocol）**下沉到**微架构与总线协议层（implementation）**。它同时也是全课程"**共享资源 = 性能瓶颈**"这条主线最纯粹的案例：总线是有限共享资源，仲裁是串行的，标签表大小决定了可达到的带宽；后面讲的互连网络、同步原语实现、无锁编程（`cmpxchg` 在总线上表现为独占事务）都直接建立在本讲的机制之上。它给出的思维方式——**"把一个操作拆成更多更小的事务可以暴露更多并行性，但要付出更多硬件与更多正确性证明的代价"**（slide 37）——是通用工程原则。
- **配套材料**：
- `lectures/12_snoopimpl.pdf`（抽取文本 `extracted/12_snoopimpl.txt`，共 **60 页**）：**已公开**，可在 https://www.cs.cmu.edu/~418/lectures/ 公开下载。Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）把 **Sep 23** 排为第 13 讲 **"Snooping-Based Multiprocessor Design"**，其 slides 链接指向这份 PDF（日程表中该链接以 HTML 注释形式给出，PDF 本身在公开的 `lectures/` 目录下可直接下载）。讲义首页写的是 **"Lecture 12: A Basic Snooping-Based Multi-Processor Implementation"** 与 **"CMU 15-418/15-618, Fall 2024"**：这是**讲义沿用历史学期版本**的正常现象（讲次编号与学期字样随年度重排），不是错误。
- 讲义中的部分插图注明 "Figure credit: Culler, Singh, and Gupta"，即经典教材《Parallel Computer Architecture: A Hardware/Software Approach》的图。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**（历史学期在 YouTube 上有存档，但不在 Fall 2026 公开日程中）。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 部分讲座在 Fall 2026 **尚未发布**公开讲义（Performance Analysis / Profiling、Transactional Memory、AI in System Design 等）；其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。

---

2. 核心概念与硬件/软件架构图解

2.1 起点：MESI 状态机必须是"不变量清单"，而不是一张漂亮图片

- **定义与目的**：一致性协议的**不变量（invariants）**是：对任一 cache line，(a) 任一时刻至多一个缓存持有 **M** 态（可写）副本；(b) 若存在 **M** 态副本，则其他所有副本都是 **I**；(c) 若存在 **S** 态副本，则没有 **M** 态副本。状态机只是**维持这些不变量**的策略描述（slide 4）。
- **直观解释（"它是什么？"）**：把 cache line 想成一份**图书馆里唯一的手稿**。`M` = 我把手稿借回家并在上面改，同时通知图书馆"所有其他复印件作废"；`E` = 我借回家但没改，且知道外面没有别的复印件；`S` = 手稿被复印了多份，大家都在读；`I` = 我手里那份已经被宣布作废。读 = "我要看手稿"，写 = "我要独占并涂改"。
- **图解（图 1）：MESI 状态迁移图（含总线事务标签，PrRd/PrWr = 处理器发起的读/写，BusRd/BusRdX/BusUpg = 总线事务）**



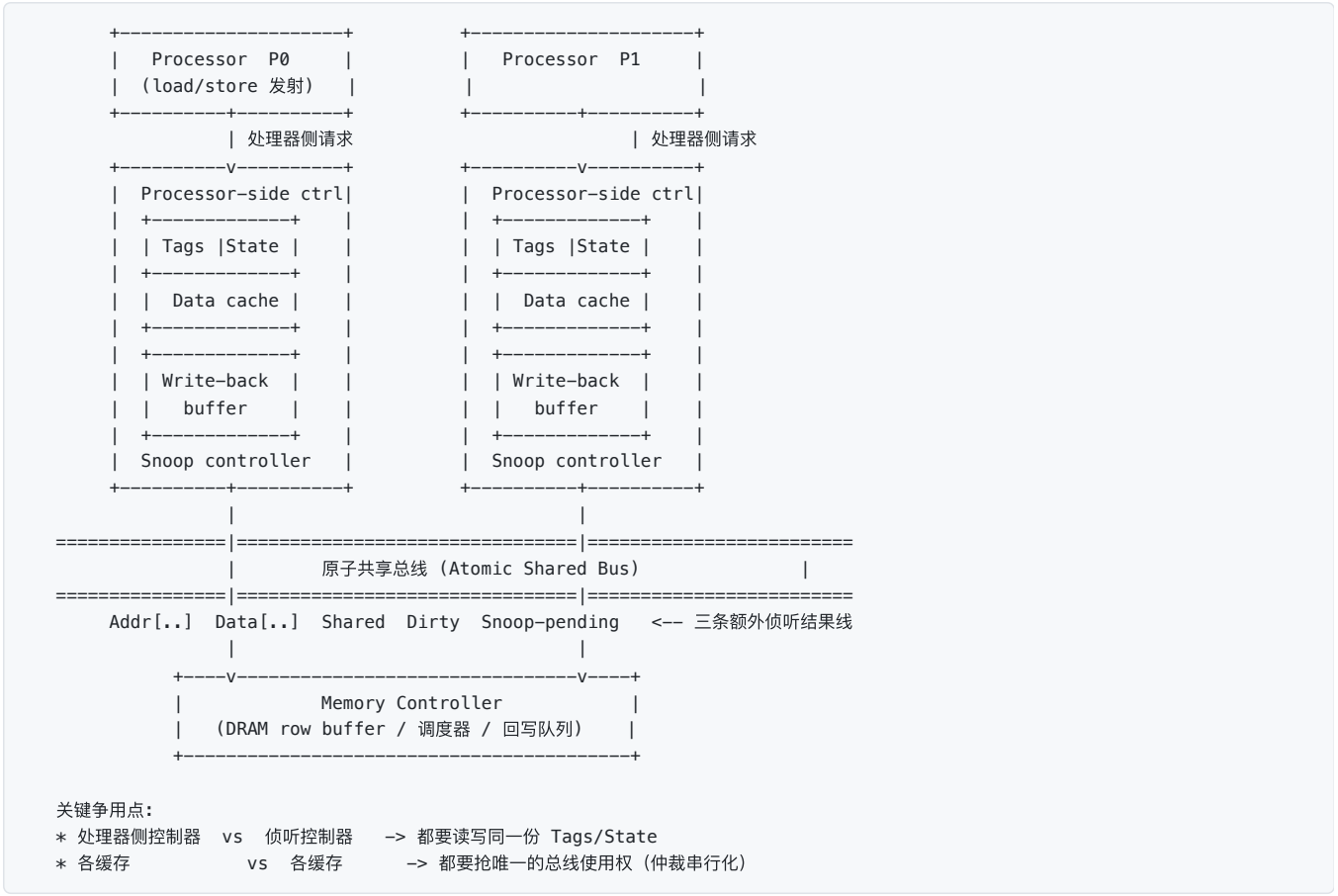
事件读法： "事件 / 在总线上产生的动作"
-- 表示不产生总线动作（本地命中）

- **关键操作与性能特征**：命中（hit）的代价是 tag 查找延迟（1–4 周期）；`S→M` 只需要 **BusUpg（upgrade，升级）**，不搬数据，因此比重读整条 line 便宜得多（少一次 128 B 数据传输）；`M→S` / `M→I` 需要 **flush（回写并供数据）**，如果对端要用的正是这条 line，则总线上的数据传输是"有用功"，否则是纯粹的额外带宽开销。
- **表 1：MESI 状态迁移表（以本处理器视角；"他人"= 侦听到的其他缓存）**

当前状态	本地事件	侦听到的事件	下一状态	总线动作	备注
I	PrRd	—	S 或 E	BusRd	若无人 assert Shared 则进 E（独占）
I	PrWr	—	M	BusRdX	独占取得所有权并失效他人
S	PrRd	—	S	无	普通命中
S	PrWr	—	M	BusUpg	只需升级，不必传数据
S	—	BusRdX / BusUpg	I	无	自己的副本被失效
S	—	BusRd	S	无	多副本共享读
E	PrRd	—	E	无	普通命中
E	PrWr	—	M	无	已经是独占，无需总线动作
E	—	BusRd	S	flush	有别人要读，降级为 S（也可选择不作声，见 §2.5 的 Dirty 线）
M	PrRd / PrWr	—	M	无	命中且脏
M	—	BusRd	S	flush	供出最新数据，降级为 S
M	—	BusRdX / BusUpg	I	flush	供出最新数据后失效

2.2 基本系统设计：原子总线 + 单级写回缓存 —— 一个"刻意幼稚"的出发点

- **定义与目的**：讲义 Part 1（slide 19–37）先假设一个最简单的系统（slide 20）：**每个处理器同时只有一个未完成的内存请求；单级、写回（write-back）缓存；缓存可以阻塞处理器**去完成一致性操作；**互连是一条原子共享总线**（同一时刻只有一个客户端在通信）。在这个"温室"里先把正确性机制做对，再在 Part 2 逐条打破假设去找性能。
- **直观解释（"它是什么？"）**：这就像**一间只有一部电话的办公室**。任何对外沟通（取数据）都必须先抢到电话（总线仲裁），说完命令后**必须一直占着电话线等对方念完数据**（原子事务：请求与响应之间不允许插入别的事务）。电话线是全办公室最稀缺的资源，而绝大部分时间它都在"等对方念"——这正是 Part 2 要解决的问题。
- **图解（图 2）：一个基于侦听的多处理器硬件结构（Part 1 的基本设计 + 写回缓冲 + 侦听结果线）**



- **关键操作与性能特征**：
- **原子总线事务的 4 步**（slide 21）：① 客户端在**仲裁（arbitration）**中获胜拿到总线；② 把命令（以及可能的数据）放上总线；③ 另一个总线客户端把**响应**放上总线；④ 下一个客户端拿到总线。注意第 ② 步到第 ③ 步之间**总线被独占**。
- **单处理器缓存缺失逻辑 7 步**（slide 22）：定 cache set → 查 tag → 申请总线 → 等仲裁授权 → 发地址+命令 → 等命令被接受 → 从总线收数据。
- **原子总线在多处理器语境下的含义**（slide 22 右下）：`BusRd` / `BusRdX` 一旦发出地址，直到收到数据为止，**不允许有任何其他总线事务**；`flush` 则要求地址与数据同时上总线，并且在任何其他事务开始之前已被内存接收。
- 性能特征：**响应等待期间总线完全空闲**，有效总线带宽被严重浪费（slide 39 明确指出这一点，并把它作为进入 Part 2 的动机）。

2.3 争用之一：处理器侧控制器 vs 侦听控制器

- **定义与目的**：多处理器缓存控制器要同时服务两个"客户"：来自处理器的 load/store，和来自总线的侦听请求。**两者的第一步都是查 tag**，于是产生争用（slide 23）。
- **直观解释（"它是什么？"）**：一个柜台只有一个窗口，却排着两条队：**顾客（处理器）**和**电话（总线）**。要么电话优先——顾客一打电话就被"锁在门外"；要么顾客优先——电话响了你却接不了，导致**整条总线的其他处理器都陪着你等**（哪怕根本没有共享发生）。
- **两种缓解方案**（slide 24）：① **复制 tag（duplicate tags）**：给侦听控制器一份独立的 tag 副本；② **多端口 tag 存储（multi-ported tag memory）**：一份 tag，两组端口。两者都能让"查"并行，但**修改** tag 时仍必须互斥（否则不变量被破坏）；因为**查 tag 远比改 tag 频繁**，这个折中非常划算。讲义强调：**性能的代价是硬件资源**（cost of the additional performance is additional hardware resources）。

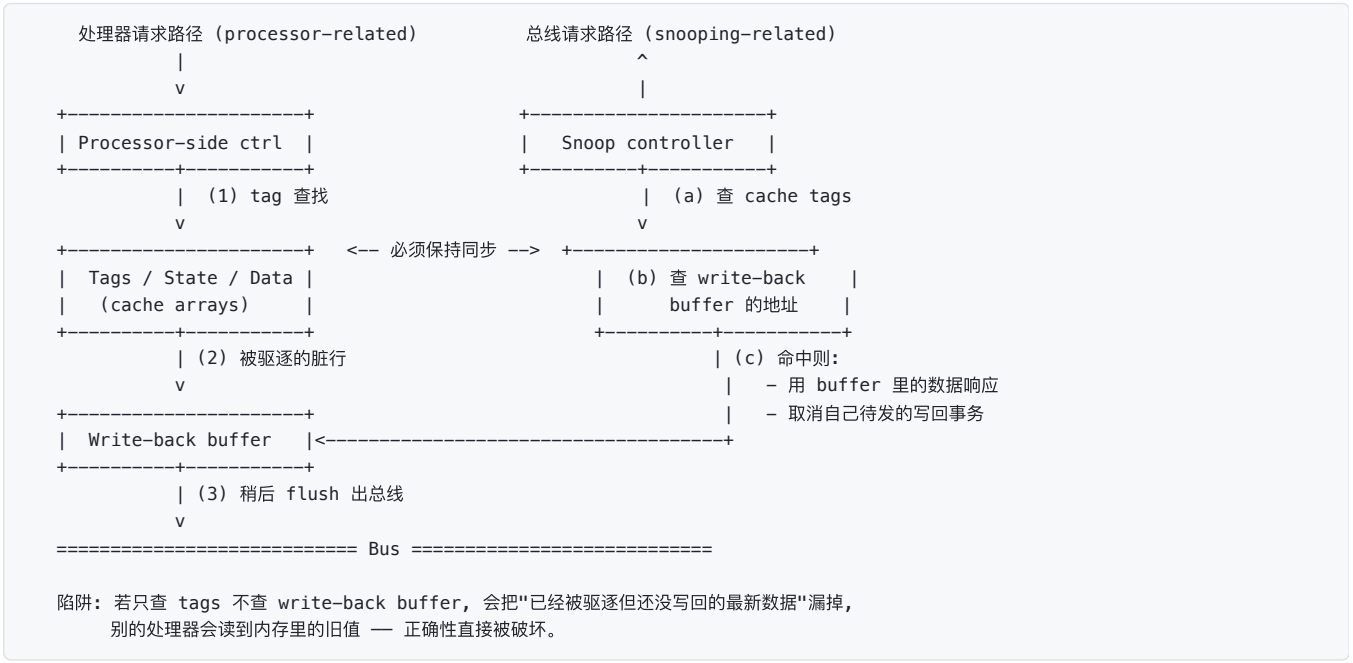
2.4 争用之二：侦听结果怎么汇报、什么时候汇报

- **定义与目的**：一次 `BusRd` 缺失，内存和请求者都需要知道"别人手里的状态"（slide 25）：**line 是脏的吗？**（脏则内存不该响应，应由持有者供数）**line 是共享的吗？**（共享则请求者应进入 S 而非 E）。
- **怎么汇报（how，slide 26）**：增加三条**线与（wired-OR）**总线信号：
- `Shared`：所有处理器侦听结果之 OR——有人有副本即置位；
- `Dirty`：所有处理器侦听结果之 OR——有人是 M 态即置位；
- `Snoop-pending`：所有处理器之 OR，**全 0 表示所有处理器都已给出侦听结果**（用作"集合点"）。 这三条线是**额外的总线互连硬件**，是"用硬件换正确性与性能"的又一例。

- **什么时候汇报（when，slide 27）**：两种策略——
- 内存控制器**立即开始访问 DRAM，但先压住响应（squelch）**，一旦有侦听结果指出别的缓存有更新数据，就取消自己的响应，由该缓存供数；
- 内存**先假设一定有某个缓存会服务**，直到侦听结果有效为止；若没人有，则内存必须响应。
- **直观解释（"它是什么？"）**：像会议里问"谁手上有最新版文件？"。要么让**档案室先跑去复印**（预取），然后可能白跑一趟（被 squelch）；要么等**举手统计结束**再决定谁去拿（延迟更长，但不会白干）。前者省延迟、浪费 DRAM 带宽与能耗；后者反之。

2.5 争用之三：写回（write backs）与 write-back buffer

- **定义与目的**：一次写回天然涉及**两个总线事务**（slide 28）：① 处理器的缺失带来的**入线（incoming line）**；② 被驱逐的脏行的**出线（outgoing line，flush）**。理想情况是**处理器尽快继续执行**，不必等 flush 完成。
- **直观解释（"它是什么？"）**：你要往书架上放一本新书，但格子满了，得先把旧书搬走。**write-back buffer 就是门口的暂存箱**：先把旧书丢进箱子，立刻把新书放上架子继续干活（处理器不停顿），等有空再慢慢把箱子里的书送回图书馆（内存）。
- **图解（图 3）：带 write-back buffer 的缓存，以及侦听控制器必须同时查两处**



- **关键操作与性能特征**：写回缓冲把"两次串行总线事务"变成"一次立即完成 + 一次延后完成"，处理器停顿时间从 `t_in + t_flush` 降到约 `t_in`；代价是**侦听逻辑复杂化**（必须查 buffer 地址）以及**buffer 是新的死锁/顺序风险点**（若 buffer 满，处理器又得等）。

2.6 状态迁移不是原子的：竞态、取数死锁、活锁、饥饿

这是本讲的真正核心：slide 30 明说——**状态迁移图假设迁移是原子的，但真实机器里"查 tag、仲裁总线、等其他控制器行动"这一整套都不是原子的。**

- **竞态示例（slide 31）**：P1 与 P2 同时写共享的 line A（两者都要发 `BusUpg`）。P1 赢得总线发出 `BusUpg`；P2 在等总线，此时**收到** P1 的 `BusUpg`，按 MESI 必须失效自己line A——但 P2 自己那个待发的 `BusUpg` 现在**语义过时了**（它已经失去副本，应该变成 `BusRdX`）。结论：**缓存在等待总线期间必须仍能处理外来请求，并且必须能修改自己排队中的请求**。这就是"抽象是原子的、实现不是"造成的第一类 bug。
- **取数死锁（fetch deadlock，slide 32）**：P1 持有 line B 的 M 态副本，正在等总线以发出对 line A 的 `BusRdX`；此时总线上出现对 B 的 `BusRd`——若 P1 坚持"我的请求没发出去之前不处理别的事"，那么它既不被服务、也不提供服务，**总线被死锁**。解法：**等待自己请求的同时必须能服务外来事务**。
- **活锁（livelock，slide 33）**：P1 与 P2 反复写 line B：P1 拿到总线发 `BusRdX`，P2 失效；**P1 还没来得及真正更新缓存行**，P2 又拿到总线发 `BusRdX`，P1 失效……系统一直在"运行"，但没人完成有效工作。解法：**获得独占所有权的写，必须在所有权被放弃之前允许其完成**（"commit 后再让别人抢"）。
- **饥饿（starvation，slide 35）**：多个处理器竞争总线，若策略是"id 最小者胜"，低 id 者可能长期霸占总线。这是**公平性（fairness）**问题，而不是正确性问题。缓解策略：**FIFO 仲裁**、**基于优先级的启发式（高频使用者优先级衰减，priority drop）**。
- **直观解释（"它是什么？"）**：**死锁** = 匹兹堡窄巷里两辆车顶牛（slide 9–10），谁都动不了，除非有人倒车让出资源；**活锁** = 两个人迎面走，同时往右让、又同时往左让，脚步不停但谁也没过去（slide 14–17 的三张连续漫画）；**饥饿** = 十字路口黄车一直在等绿灯方向的车流，整体在通行（绿车一直在走），只是黄车永远排不上（slide 18）。

- **表 2：死锁 / 活锁 / 饥饿的对比与对策**

现象	定义（讲义 slide 8–18）	系统表现	一致性实现中的典型成因	对策
死锁 deadlock	有未完成的操作，但没有任何操作能推进	系统彻底停住	等待自己的请求时不处理外来请求（fetch deadlock）；有限队列互相等待（buffer deadlock）	允许"等待中仍服务"；把请求/响应队列分离；请求表冲突检查
活锁 livelock	系统在执行大量操作，但没有线程取得有意义的进展	CPU 与总线都很忙，但结果不推进	独占所有权被反复抢走，写永远 commit 不了	让获得独占所有权的写先完成再释放
饥饿 starvation	系统整体在推进，但某些进程毫无进展	少数参与者长期得不到服务	仲裁策略不公平（固定优先级、最低 id 优先）	FIFO 仲裁、优先级衰减、请求老化（aging）
死锁的 4 个必要条件（slide 13）	互斥、持有并等待、不可抢占、循环等待	四者同时成立才可能死锁	资源依赖图中存在环	破坏其中任一条件

2.7 写何时"提交"：commit ≠ complete

- **定义与目的**（slide 34）：**写提交（commits）**发生在"读独占事务（read-exclusive）出现在总线上并被其他所有缓存确认"的那一刻。此后所有未来的读都会反映这个写的值——**即使数据还没写进 P 的脏行，也还没进内存**。而**写完成（complete）**则是指更新后的值已经落到缓存行里。**commit 与 complete 是两个不同的时刻。**
- **为什么重要**：这正是**写串行化（write serialization）**的来源——**总线上的事务顺序定义了并行程序中写的全局顺序**。硬件只要保证"上总线的顺序"，软件就能获得一个确定的、所有处理器都同意的写顺序。
- **为什么 write-back buffer 不影响 commit 时刻**：因为 commit 的定义锚定在**"总线上出现读独占事务并获得确认"**这个可见事件上，而**不是**数据何时落到缓存行或内存里。把脏数据留在 write-back buffer 里，只是把"完成（complete）"推后，完全不改变"提交（commit）"的时点——所以处理器可以立刻继续执行，其他处理器之后读到的仍然是新值（由 write-back buffer 参与侦听供数）。
- **直观解释（"它是什么？"）**：**commit = 在会议纪要上签字（此后所有人都必须按这份纪要办事）；complete = 把手上的活真正干完**。签字之后你可以先去干别的活（write-back buffer 里放着），但纪要已经生效。

2.8 Part 2：把原子总线拆成"请求 / 响应"——split-transaction bus

- **定义与目的**（slide 39–41）：原子总线的问题是**等待响应期间总线空闲**，有效带宽被浪费；而互连是系统中**有限且共享**的资源，必须尽量高效使用。**拆分事务总线**把一次总线事务拆成两个独立事务：**① 请求（request：命令 + 地址）** 与 **② 响应（response：数据）**，两者之间允许插入其他事务。
- **直观解释（"它是什么？"）**：从**"打电话并一直握着话筒等对方查资料"**改成**"留个工单号，然后挂机；对方查好后凭工单号回拨"**。电话线（总线）在等资料期间可以继续接别的活，只要你能凭工单号把回应匹配回去。
- **图解（图 4）：拆分事务总线的周期级时序（请求总线 + 响应总线，流水化与乱序完成）**

```
周期:        0    1    2    3    4    5    6    7    8    9   10   11   12   13
------------------------------------------------------------------------------
请求总线 (Addr/cmd)  ARB RSLV ADDR DCD ACK |ARB RSLV ADDR DCD ACK |ARB RSLV ...
                     \___ txn1 请求阶段(5 周期) ___/  \__ txn2 请求阶段 __/
                     仲裁 解决  上地址 解码  确认

响应总线 (Data Arb/Data)                                  ARB RSLV |D0 D1 D2 D3
                                                            等待 DRAM (100 周期,
                                                            总线不被独占!) ...

事务 1 (P1 read miss to A)   [==== 请求阶段 ====].....(其他事务穿插).....[==== 数据 4 周期 ====]
事务 2 (P2 BusUpg B)                                    [==== 请求阶段 ====]   (无响应分量!)
事务 3 (P0 read miss to C)                                        [=== 请求阶段 ===]  ...
事务 4 (P3 read miss to D)                                                  [=== 请求阶段 ===]

要点:
* 请求阶段的 5 个周期: ARB(仲裁) RSLV(解决/分配 tag) ADDR(上地址/命令) DCD(解码) ACK(确认)
* 数据总线: 128 B line / 32 B(256 bit) 位宽 = 4 个周期
* "Memory operation commits here! (NO BUS TRAFFIC)" —— 提交点在侦听阶段, 但此时总线无流量
* 完成顺序 != 请求顺序 (out-of-order completion); 请求顺序定义系统的全序
* write back 与 BusUpg 没有响应分量 (它们在"请求"阶段就一并拿到数据总线使用权)
```

- **新出现的问题**（slide 42）：

- **请求如何与响应匹配？** → **请求表（request table）+ 事务标签（tag）**；
- **冲突请求如何处理？** → 每个缓存都保存请求表副本，禁止发出与表中已有的冲突请求；
- **流控（flow control）**：同时允许多少未完成请求？缓冲满了怎么办？→ **NACK + 重试**；
- **侦听结果何时汇报？** 在请求阶段还是响应阶段。
- **一个基本设计**（slide 43）：**系统范围内最多 8 个未完成请求；响应不必按请求顺序返回，但请求顺序确立系统全序；流控用 NACK（否定确认）**——缓冲满时客户端 NACK 该事务，触发稍后重试。
- **发起请求的机制**（slide 44）：可把拆分事务总线看成**两条总线**——请求总线（命令 + 地址）与响应总线（数据）。步骤：① 请求者申请请求总线；② 仲裁器授权，并为事务**分配一个 tag**；③ 请求者把命令与地址放上总线。**tag 就是请求表的索引**（8 个表项 ⇒ 3 bit tag），每个总线客户端（缓存）都维护该表的副本。
- **表 3：原子总线 vs 拆分事务总线**

维度	原子总线（Part 1）	拆分事务总线（Part 2）
请求与响应能否被其他事务穿插	不能	能（这就是全部意义）
等待响应期间总线状态	**空闲**（带宽浪费）	可被其他请求/响应占用
有效带宽（后文算例）	≈ 3.5 GB/s（利用率 8.3%）	≈ 28 GB/s（受 8 个表项限制）
需要的额外硬件	侦听结果三条线	请求表 + tag + 请求/响应两套仲裁 + 缓冲
顺序保证	总线独占 ⇒ 天然串行	需显式规定：**请求顺序 = 系统全序**
新引入的正确性风险	竞态、取数死锁、活锁	响应对不上、冲突请求、NACK 风暴、缓冲死锁
主要性能限制来源	内存延迟直接暴露在总线上	请求表项数（Little 定律）、请求总线占用、NACK 重试

- **冲突请求的两个情形**（slide 51–52）：
- **情形 1**：P1 读缺失 X，而总线上已有**读**事务涉及 X ⇒ **不冲突**：不必发新请求，**监听那个已存在事务的响应即可**（搭车，数据在总线上广播，谁能用谁用）。
- **情形 2**：P1 读缺失 X，而总线上有**写（BusRdX）**事务涉及 X ⇒ **冲突**：必须**挂起请求直到冲突清除**，否则会读到被失效过程中的旧数据/顺序错乱。
- **图解（图 5）：请求表、冲突检查与数据广播（软件视角的执行模型）**

```
每个缓存都保存一份相同的请求表（Request Table, 8 项）
+------+------------------+---------+---------------+
| tag  | Addr (line)      | Op      | State         |
+------+------------------+---------+---------------+
|  0   | 0xbeef           | BusRd   | promoted      |  <-- 数据在响应总线上广播，
|  1   | 0x2a00           | BusRdX  | shared        |      所有侦听者都采样(即使不是请求者)
|  2   | ...              | ...     | ...           |
|  ... |                  |         |               |
+------+------------------+---------+---------------+

缓存 C 的决策流程:
  处理器缺失(addr X, op)
           |
           v
  查本地的请求表副本 ---------------------------+
           |                                    |
  有无同名地址表项?                              |
     |                |                        |
    无                有                       |
     |                |                        |
     v                v                        |
还有空闲表项?   是读事务吗?                     |
  |       |        |        |                  |
  有      无        是       否(=写)            |
  |       |        |        |                  |
  |       v        v        v                  |
  |   NACK:   "搭车": 不发新请求, 挂起等待该      |
  |   稍后重试  事务的响应; 数据广播时自然拿到     |
  |            |            |                  |
  |            |            v                  |
  |            |      冲突: 挂起, 等该表项完成后重试
  |            |                               |
  +------------+-------------------------------+
               v
        申请请求总线 -> 仲裁器分配 tag(=索引) -> 命令+地址上总线
```

2.9 为什么并行系统里到处是队列？—— 用有界缓冲"熨平"速率波动

- **定义与目的**（slide 53）：**队列的作用是容纳生产速率与消费速率之间不可预测的波动**。只要 A 与 B 的**平均**速率相同，有了队列（哪怕只有 2 格）**双方都可以全速运行而不互相拖累**。
- **直观解释（"它是什么？"）**：**快递驿站的暂存架**。寄件人（生产者）时而一次抱来三箱，时而空手；快递员（消费者）时而一次收走四箱。没有暂存架，寄件人得等快递员到场才能交件（stall），快递员也得等人来才能取件（stall）；有了两格暂存架，**双方各自按自己的节奏全速跑**，波动被架子吸收了。
- **图解（图 6）：无队列 vs 有队列（queue depth = 2）的时间线**

```
无队列 (rendezvous):  A 生产 1 个就必须等 B 取走, B 也必须等 A 生产
时间 ->   1    2    3    4    5    6    7    8    9   10   11   12
A:       [A1] ......  [A2] ......  [A3] ......  [A4] ......  [A5]
          ^stall     ^stall       ^stall       ^stall
B:       ...... [B1] ......  [B2] ......  [B3] ......  [B4] ......
                 ^stall       ^stall       ^stall
总吞吐 = 1 个/2 时间单位 (双方都在等对方, 谁都跑不满)

有队列 (size = 2): A 可以先做 2 个再等; B 可以攒着慢慢取
时间 ->   1    2    3    4    5    6    7    8    9   10   11   12
A:       [A1] [A2] [A3] [A4] [A5] [A6] [A7] [A8] [A9] [A10]  (全速)
队列:     A1   A1,A2 A2,A3 A3,A4 A4,A5  ...  (深度在 0~2 之间浮动)
B:       [B1] [B2] [B3] [B4] [B5] [B6] [B7] [B8] [B9] [B10]  (全速)
总吞吐 = 1 个/1 时间单位 —— 平均速率匹配时, 队列深度 2 就足够让双方都不 stall

-> 这就是 NACK/重试、请求表、write-back buffer、总线客户端接收缓冲存在的理由
```

2.10 多级缓存层次带来的两个新难题

- **难题 A：谁负责侦听？**（slide 5、54）真实机器（如 Intel Core i7）是 Core → L1(d) → L2 → 共享 L3（每个核一个 bank）→ Ring 互连 的层次结构。如果只有 L2 控制器去侦听互连，那么**L1 里发生的数据修改可能对 L2 控制器不可见**。两种做法：

- **所有层缓存各自独立侦听互连**——低效（重复侦听、总线上设备过多）；
- **维护包含性（inclusion）**：L1 的内容必须是 L2 内容的子集。这样"L2 侦听 + 失效 L1"就等价于全层次侦听。代价是 L2 容量被"浪费"一部分用于覆盖 L1，且 L2 驱逐必须连带失效 L1。 讲义也点出：层次结构还**放大了响应延迟**，使取数死锁问题更尖锐（slide 55）。
- **难题 B：缓冲区死锁（buffer deadlock）**（slide 56）：设 L1→L2 与 L2→L1 各只有一个缓冲槽（buffer size = 1）。L1 有一个**出向（outgoing）读请求**（处理器发起）等着进 L2；同时 L2 有一个**入向（incoming）读请求**（别的缓存发起，因 L1 是 write-back 才会发生）等着进 L1。**两个请求的响应都需要对方队列里的空间** ⇒ 循环依赖 ⇒ 死锁。
- **图解（图 7）：缓冲区死锁的环形依赖，以及"请求/响应分离队列"如何打破环**

```
(a) 只有一对队列时 —— 循环依赖，死锁
+--------------+   L1->L2 队列（cap=1，已满）   +--------------+
|  L1 Cache    |------------------------------->|  L2 Cache    |
|              |<-------------------------------|              |
+--------------+   L2->L1 队列（cap=1，已满）   +--------------+
     ^                                               ^
     |  入向读请求的**响应**需要 L2->L1 空间            |
     |  出向读请求的**响应**需要 L1->L2 空间            |
     +-------------------- 环 --------------------------+
     L1 要发请求 -> 需要 L1->L2 空位 -> 该空位要靠"处理完入向请求"腾出
     L2 要发请求 -> 需要 L2->L1 空位 -> 该空位要靠"处理完出向请求"腾出
     => 双方都在等对方先动，谁也不动 = DEADLOCK

(b) 把请求与响应分开成 4 条队列 —— 环被打断
+--------------+   L1->L2 request queue   +--------------+
|  L1 Cache    |------------------------->|  L2 Cache    |
|              |   L2->L1 request queue   |              |
|              |<-------------------------|              |
|              |   L1->L2 response queue  |              |
|              |------------------------->|              |
|              |   L2->L1 response queue  |              |
|              |<-------------------------|              |
+--------------+                          +--------------+
关键洞察（slide 58）：
  * 请求（request）会**增加**队列长度；响应（response）会**减少**队列长度
  * 响应**不会再产生新的事务**，因此响应的处理一定能推进到底
  * 正在为"发不出请求"而卡住的缓存，仍然必须能处理响应
    => 响应最终一定会腾出资源，让请求得以发出 —— 不存在循环依赖
```

2.11 软件执行模型：一句 `int x = 10;` 的全过程

- **定义与目的**（slide 59–60）：讲义用一个课堂练习收尾：`int x = 10;`（假定这是一次**写内存**，值不保存在寄存器里）。把这句话在真实多处理器机器上可能引发的事情全列出来——**它把"程序语义"和"硬件执行模型"缝合在一起**。
- **直观解释（"它是什么？"）**：程序员看到的是"赋值"这一件事；硬件看到的是一场需要 TLB、页表、OS、缓存、总线仲裁器、多个侦听控制器、内存控制器、DRAM 全部参与的接力赛。
- **图解（图 8）：软件语句 → 硬件执行模型的 20 步接力**

```
 程序层:   int x = 10;          (一次 store, 架构上"原子")
================================================================================
 微架构/系统层 (讲义 slide 60 列出的 20 步, *表示讲师注明"绝非完整列表"):
   1  虚拟地址 -> 物理地址转换 (TLB 查找)
   2  TLB miss
   3  TLB 更新 (可能涉及操作系统)
   4  OS 可能需要换页, 把页表从磁盘换入物理内存
   5  缓存查找 (tag check)
   6  判定 line 不在缓存 (需要产生 BusRdX)
   7  仲裁总线
   8  赢得总线, 放上地址与命令
   9  所有缓存执行侦听 (例如失效自己对应的副本)   <-- 写在此刻"提交" (commit)
  10  另一个缓存或内存决定由谁响应 (此例假设是内存)
  11  内存请求送入内存控制器
  12  内存控制器本身也是一个调度器
  13  检查 DRAM 行缓冲中是否有活跃的行 (可能需要激活新行, 此例假设需要)
  14  DRAM 把数据读入行缓冲
  15  内存仲裁数据总线
  16  内存赢得总线
  17  内存把数据放上总线
  18  请求方缓存取走数据, 更新缓存行与 tag, 转入独占状态
  19  通知处理器数据已就绪
  20  指令继续执行
================================================================================
 结论: 一条被抽象为"原子"的访存, 实际由 ~20 个跨组件步骤实现;
       其中任何一步的并行/非原子性都可能成为正确性或性能的问题来源
```

- **性能特征**：这条链路上真正的长杆是第 2—4 步（TLB miss + 缺页，微秒级）、第 6—17 步（一致性事务 + DRAM，百纳秒级）；而第 9 步的侦听与"提交"发生在**没有总线流量**的窗口里（slide 45 标注 "Memory operation commits here! (NO BUS TRAFFIC)"）——这提醒我们：**延迟与总线占用不是一回事**，分析性能时要分清"关键路径延迟"和"共享资源占用"。

---

3. 代码示例与性能分析

3.1 示例 1：伪共享（false sharing）—— 一致性协议在软件层面最贵的账单

- **代码**:

```
// 编译（release）: g++ -O3 -std=c++17 -pthread false_sharing.cpp -o false_sharing
// 运行:  ./false_sharing packed      # 4 个计数器挤在同一条 cache line
//        ./false_sharing padded      # 每个计数器独占一条 cache line
#include <pthread.h>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ctime>

static constexpr int      kThreads = 4;
static constexpr uint64_t kIters   = 50'000'000ULL;   // 每线程自增次数

// ---- 布局 A: 4 个计数器共享同一条 64 B cache line（伪共享） ----
struct Packed {
    alignas(64) volatile uint64_t c[kThreads];
};

// ---- 布局 B: 每个计数器独占一条 cache line（对齐 + 填充） ----
struct alignas(64) Padded {
    volatile uint64_t v;
    char pad[64 - sizeof(uint64_t)];
};

static Packed  g_packed;                  // 4 个计数器 = 1 条 line
static Padded  g_padded[kThreads];        // 4 个计数器 = 4 条 line

struct Arg { int id; int use_padded; };

static double now_sec() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return double(ts.tv_sec) + double(ts.tv_nsec) * 1e-9;
}

static void* worker(void* p) {
    const Arg* a = static_cast<const Arg*>(p);
    // 每个线程只访问自己的槽位: 逻辑上没有数据竞争
    volatile uint64_t* target = a->use_padded ? &g_padded[a->id].v : &g_packed.c[a->id];
    for (uint64_t i = 0; i < kIters; ++i) {
        *target = *target + 1;            // 读-改-写: 每次都需要该 line 处于 M 态
    }
    return nullptr;
}

int main(int argc, char** argv) {
    const int use_padded = (argc > 1 && std::strcmp(argv[1], "padded") == 0) ? 1 : 0;
    pthread_t t[kThreads];
    Arg       arg[kThreads];

    const double t0 = now_sec();
    for (int i = 0; i < kThreads; ++i) {
        arg[i] = Arg{i, use_padded};
        pthread_create(&t[i], nullptr, worker, &arg[i]);
    }
    for (int i = 0; i < kThreads; ++i) pthread_join(t[i], nullptr);
    const double dt = now_sec() - t0;

    uint64_t checksum = 0;
    for (int i = 0; i < kThreads; ++i)
        checksum += use_padded ? g_padded[i].v : g_packed.c[i];

    std::printf("%-7s threads=%d iters/thread=%llu  time=%.3f s  %.1f Mops/s  checksum=%llu\n",
                use_padded ? "padded" : "packed", kThreads,
                (unsigned long long)kIters, dt,
                double(kThreads) * double(kIters) / dt / 1e6,
                (unsigned long long)checksum);
    return 0;
}
```

- **【代码做什么？】**

- `main` 按命令行参数选择两种内存布局之一，创建 `kThreads` 个 pthread，每个线程用 `Arg{id, use_padded}` 携带自己的编号与模式。
- `worker` 中每个线程**只读写属于自己 id 的那个计数器**：`packed` 模式下 4 个 `uint64_t` 落在同一条 64 B line 内，`padded` 模式下每个计数器被 `alignas(64)` 强制对齐到独立 cache line。
- 每个线程做 5000 万次 `*target = *target + 1`（读–改–写），保证每次操作都要持有一条可写副本。
- `pthread_join` 后统计 `checksum`（应为 4 × 5e7 = 2×10^8），并打印墙钟时间与总操作率。
- 逻辑上**没有数据竞争**（不同线程访问不同的内存位置），但结果差异巨大——这正是伪共享的本质。
- **【并行机制与性能解说】**
- **硬件上如何执行**：`packed` 模式下，4 个核的每次自增都要求**同一条 cache line 处于 M 态**。M 态是互斥的，于是每次自增都触发一次 `BusUpg` / `BusRdX`（或用 MESI 的 upgrade 事务）并把**上一位持有者的脏行 flush 出来**再失效它——**cache line 在核之间来回弹跳（ping-pong）**。`padded` 模式下每个核的 line 稳定停留在自己的缓存里（M 或 E 态），自增退化为纯本地操作。
- **Work / Span / 并行度**：
  - `packed`：Work = 4 × 5e7 = 2×10^8 次自增；**Span = 2×10^8 次串行的 line 迁移**（因为同一时刻只有一个核能持有可写副本，整条执行变成一条链）。并行度 = Work/Span = **(2×10^8 × C_increment) / (2×10^8 × L_transfer) = C_increment / L_transfer ≈ 1/100 ≪ 1**——即并行度比串行还差，多核反而放大开销。
  - `padded`：Work = 2×10^8 次自增；Span = 每线程自己的 5e7 次依赖链（每线程一条），**Span = 5e7 × C_increment**。并行度 = Work/Span = (4 × 5e7 × C)/(5e7 × C) = **4**（= 线程数），随核数线性可扩展。
- **瓶颈定位**：`packed` 的瓶颈是**一致性事务延迟 × 次数**（每次操作一次 line 迁移），等价于"用总线延迟替代了 ALU 吞吐"；`padded` 的瓶颈回到本地 store–to–load 依赖链与 ALU。
- **Amdahl 视角**：即使只有 1% 的内存操作落在共享行上，若每次都付出约 100 周期的迁移延迟而本地操作只值 1 周期，那么这 1% 的操作会吃掉总时间的 `0.01×100 / (0.99×1 + 0.01×100) ≈ 50%`——**少数伪共享行足以主导整程序性能**，这与"总线是共享资源"的硬件结论完全一致。

3.2 示例 2：拆分事务总线 + 请求表 + NACK 流控的周期级模拟器

这是本讲最"贴题"的实验：用 200 行 C 把 slide 41–52 的机制做成可跑的数字。

- **代码**：

```c
/* 周期级模拟: 原子总线 vs 拆分事务总线 (请求表 + tag + NACK 流控)
 * 编译 (release) : gcc -O3 -std=c11 split_bus_sim.c -o split_bus_sim
 * 运行:            ./split_bus_sim
 */
#include <stdio.h>
#include <string.h>

#define NCACHE          4        /* 处理器/缓存个数 P0..P3            */
#define TAG_COUNT       8        /* 请求表项数 = 同时最多 8 个未完成请求 */
#define REQ_CYCLES      5        /* ARB,RSLV,ADDR,DCD,ACK 请求阶段周期数 */
#define DATA_CYCLES     4        /* 128 B line / 32 B(256 bit) 数据总线 */
#define MEM_LATENCY   100        /* DRAM 取数延迟 (周期)               */
#define RETRY_DELAY    10        /* NACK / 冲突后的重试延迟             */
#define NUM_MISSES 100000        /* 每个缓存要发出的缺失次数           */
#define LINES_SPACE  4096        /* 地址空间 (cache line 计), 用于制造冲突 */
#define MAXPEND        32        /* 每缓存未完成缺失槽位数 (容量上限)    */
#define LINE_BYTES    128
#define CLOCK_GHZ     3.0

typedef struct { int valid, addr, data_ready, data_start, complete; } Req;

static Req rtab[TAG_COUNT];
static int outstanding;
static int pend_tag[NCACHE][MAXPEND];     /* 槽位 -> 正在等待的请求表 tag (-1 空) */
static unsigned lcg[NCACHE];

typedef struct {
    long cycles, lines, nack, conflict, combined;
    long req_bus_cycles, data_bus_cycles;
} Stats;

/* ---- 拆分事务总线模拟: mshr_limit = 每个缓存允许的未完成缺失数 ---- */
static Stats simulate_split(int mshr_limit)
{
    Stats st; memset(&st, 0, sizeof st);
    memset(rtab, 0, sizeof rtab);
    for (int c = 0; c < NCACHE; c++) {
        for (int s = 0; s < MAXPEND; s++) pend_tag[c][s] = -1;
        lcg[c] = 12345u + 7919u * (unsigned)c;
    }
    outstanding = 0;
    int  issued[NCACHE] = {0}, inflight[NCACHE] = {0};
    long next_try[NCACHE] = {0};
    long req_bus_free = 0, data_bus_free = 0;
    const long total = (long)NCACHE * NUM_MISSES;
    long done = 0, cycle = 0;

    for (cycle = 0; done < total && cycle < 200000000L; cycle++) {
        /* (1) 数据总线上的响应在本周期完成 -> 释放请求表项 */
        int freed[TAG_COUNT]; int nfreed = 0;
        for (int i = 0; i < TAG_COUNT; i++)
            if (rtab[i].valid && rtab[i].complete == (int)cycle) {
                rtab[i].valid = 0; rtab[i].complete = -1;
                freed[nfreed++] = i; outstanding--;
            }
        /* (2) 广播: 数据在响应总线上对所有侦听者可见(请求者与"搭车者"同时完成) */
        for (int c = 0; c < NCACHE; c++)
            for (int s = 0; s < MAXPEND; s++) {
                int t = pend_tag[c][s];
                if (t < 0) continue;
                for (int k = 0; k < nfreed; k++)
                    if (freed[k] == t) { pend_tag[c][s] = -1; inflight[c]--; done++; break; }
            }
        /* (3) 各缓存尝试发起缺失(轮转起点随时间变化, 模拟公平仲裁) */
        for (int k = 0; k < NCACHE; k++) {
            int c = (int)((cycle + k) % NCACHE);
            if (issued[c] >= NUM_MISSES || inflight[c] >= mshr_limit) continue;
            if (cycle < next_try[c]) continue;

            lcg[c] = lcg[c] * 1103515245u + 12345u;
            int addr = (int)((lcg[c] >> 8) % LINES_SPACE);
```

```
            /* 冲突检查: 每个缓存都持有请求表副本 (slide 50-52) */
            int host = -1, conflict = 0;
            for (int i = 0; i < TAG_COUNT; i++) {
                if (!rtab[i].valid || rtab[i].addr != addr) continue;
                host = i; conflict = 1;      /* 同地址事务未完成 */
            }
            if (host >= 0) {                 /* 情形 1: 读-读可"搭车"(本模型统一按冲突处理) */
                st.conflict++;
                next_try[c] = rtab[host].complete > 0 ? rtab[host].complete : cycle + RETRY_DELAY;
                continue;
            }
            (void)conflict;
            if (outstanding >= TAG_COUNT) {  /* 流控: 请求表满 -> NACK + 稍后重试 */
                st.nack++;
                next_try[c] = cycle + RETRY_DELAY;
                continue;
            }
            int tag = -1;
            for (int i = 0; i < TAG_COUNT; i++) if (!rtab[i].valid) { tag = i; break; }
            long start = (req_bus_free > cycle) ? req_bus_free : cycle;
            rtab[tag].valid = 1; rtab[tag].addr = addr;
            rtab[tag].data_ready = (int)(start + REQ_CYCLES + MEM_LATENCY);
            rtab[tag].data_start = -1; rtab[tag].complete = -1;
            req_bus_free = start + REQ_CYCLES;
            outstanding++;
            st.req_bus_cycles += REQ_CYCLES;
            issued[c]++; inflight[c]++;
            for (int s = 0; s < MAXPEND; s++)
                if (pend_tag[c][s] < 0) { pend_tag[c][s] = tag; break; }
        }
        /* (4) 响应总线仲裁: 数据就绪者中挑最早就绪的占用数据总线 */
        if (data_bus_free <= cycle) {
            int best = -1;
            for (int i = 0; i < TAG_COUNT; i++) {
                if (!rtab[i].valid || rtab[i].data_start >= 0) continue;
                if (rtab[i].data_ready > (int)cycle) continue;
                if (best < 0 || rtab[i].data_ready < rtab[best].data_ready) best = i;
            }
            if (best >= 0) {
                rtab[best].data_start = (int)cycle;
                rtab[best].complete   = (int)(cycle + DATA_CYCLES);
                data_bus_free = rtab[best].complete;
                st.data_bus_cycles += DATA_CYCLES;
                st.lines++;
            }
        }
    }
    st.cycles = cycle;
    return st;
}

/* ---- 原子总线基线: 请求-响应之间不允许插入任何事务 ---- */
static Stats simulate_atomic(void)
{
    Stats st; memset(&st, 0, sizeof st);
    long per_txn = REQ_CYCLES + MEM_LATENCY + DATA_CYCLES;      /* 总线被独占 */
    st.lines = (long)NCACHE * NUM_MISSES;
    st.cycles = st.lines * per_txn;
    st.req_bus_cycles = st.lines * REQ_CYCLES;
    st.data_bus_cycles = st.lines * DATA_CYCLES;
    return st;
}

static void report(const char* name, Stats st)
{
    double sec   = (double)st.cycles / (CLOCK_GHZ * 1e9);
    double bytes = (double)st.lines * LINE_BYTES;
    double gbps  = bytes / sec / 1e9;
    double util_req  = 100.0 * (double)st.req_bus_cycles  / (double)st.cycles;
    double util_data = 100.0 * (double)st.data_bus_cycles / (double)st.cycles;
    printf("%-22s cycles=%10ld  time=%8.3f ms  xfer=%7.2f MB  BW=%7.2f GB/s"
```

```
           "  reqbus=%5.1f%%  databus=%5.1f%%  NACK=%6ld  conflict=%6ld\n",
           name, st.cycles, sec * 1e3, bytes / 1e6, gbps,
           util_req, util_data, st.nack, st.conflict);
}

int main(void)
{
    printf("line=%d B, req phase=%d cyc, data phase=%d cyc, mem latency=%d cyc, "
           "req-table=%d entries, %d caches x %d misses\n\n",
           LINE_BYTES, REQ_CYCLES, DATA_CYCLES, MEM_LATENCY,
           TAG_COUNT, NCACHE, NUM_MISSES);

    report("atomic bus", simulate_atomic());
    int ms[] = {1, 2, 4, 8};
    char nm[64];
    for (unsigned i = 0; i < sizeof ms / sizeof ms[0]; i++) {
        snprintf(nm, sizeof nm, "split, MSHR=%d", ms[i]);
        report(nm, simulate_split(ms[i]));
    }
    /* 理论上限 */
    double peak_data = (double)(32 * CLOCK_GHZ);                    /* 32 B/cycle 数据总线 */
    double peak_req  = (double)LINE_BYTES * CLOCK_GHZ / REQ_CYCLES; /* 请求总线限制 */
    double tag_limit = (double)TAG_COUNT * LINE_BYTES * CLOCK_GHZ
                     / (REQ_CYCLES + MEM_LATENCY + DATA_CYCLES);     /* 请求表项数限制 */
    printf("\npeaks: data bus=%.1f GB/s, request bus=%.1f GB/s, %d-entry request table=%.1f GB/s\n",
           peak_data, peak_req, TAG_COUNT, tag_limit);
    return 0;
}
```

- **【代码做什么？】**
- 建立一条**请求表**（8 项），每项记录地址、数据就绪周期、数据总线开始/完成周期；`pending` 槽位记录"哪个缓存正在等哪个 tag 的数据"。
- 主循环逐周期推进，每周期做四件事：**(1)** 释放本周期完成的数据响应所对应的表项；**(2)** 把完成事件广播给请求者与搭车者；**(3)** 让每个缓存尝试发起缺失（查冲突 → 查请求表是否满 → 分配 tag → 占用请求总线 `REQ_CYCLES` 周期）；**(4)** 响应总线仲裁，从"数据已就绪"的请求中挑一个占用数据总线 `DATA_CYCLES` 周期。
- 地址用每个缓存独立的 LCG 在一个容量为 4096 条 line 的小地址空间里生成，用来产生"同地址事务未完成"的冲突；请求表满时打印 NACK 并延后 `RETRY_DELAY` 周期重试（对应 slide 43/50 的流控）。
- `simulate_atomic()` 直接给出原子总线的解析基线：每个事务独占总线 `REQ_CYCLES + MEM_LATENCY + DATA_CYCLES = 109` 周期。
- `main` 依次跑 MSHR = 1/2/4/8 四种"每缓存未完成缺失上限"，并打印三条理论上限（数据总线、请求总线、请求表项数）。
- **【并行机制与性能解说】**
- **并行在哪里**：4 个缓存**并行地**产生缺失、并行地侦听响应总线（数据广播，所有侦听者同时完成）；请求总线与数据总线**两条总线并行工作**；被打破的假设是原子总线的"请求到响应之间总线独占"。
- **Work / Span / 并行度**（针对被建模的机器，单位是"周期"）：
  - **Work（总线必须付出的总工作）** = 每个请求的请求阶段 + 数据阶段 = `N × (REQ_CYCLES + DATA_CYCLES)` = $4\times10^5 \times 9$ = **3.6×10^6 周期**（串行占用等价量）。
  - **Span（单个缺失的关键路径）** = `REQ_CYCLES + MEM_LATENCY + DATA_CYCLES` = 5 + 100 + 4 = **109 周期**（再加排队与 NACK 重试）。
  - **并行度上限** = 请求表项数 = 8；由 Little 定律，要打满请求总线上限需要的未完成请求数 = `rate × latency = (1/5) × 109 ≈ 22`。**所以 8 项请求表天然只能达到请求总线峰值的约 8/22 ≈ 36%**——这是"用少量硬件换顺序保证"的直接代价。
- **瓶颈识别**：MSHR 从 1 涨到 2 时带宽翻倍（从"每缓存一个缺失、延迟受限"进入"全系统并发"）；继续增大 MSHR 则由**请求表项数**封顶，NACK/重试开始出现，收益饱和甚至因重试延迟而略降。这正是讲义"性能优化使正确性变复杂"的定量体现：MSHR 越多 → 未完成请求越多 → 冲突、NACK、缓冲压力越大。

3.3 示例 3：有界队列如何"熨平"生产/消费速率波动（队列存在的理由）

- **代码**：

```cpp
// 编译: g++ -O3 -std=c++17 -pthread bounded_queue.cpp -o bounded_queue
// 运行: ./bounded_queue 1      # 队列容量 1（近似 rendezvous）
//       ./bounded_queue 2      # 队列容量 2（讲义 slide 53 的"刚好够用"）
//       ./bounded_queue 64     # 深队列
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdio>
#include <cstdlib>
#include <mutex>
#include <thread>
#include <vector>

template <typename T>
class BoundedQueue {                  // 有界队列 ≈ 总线客户端的请求/响应缓冲
public:
    explicit BoundedQueue(size_t cap)
        : cap_(cap), buf_(cap), head_(0), tail_(0), n_(0), closed_(false) {}

    void push(T v, std::atomic<long>& producer_stall) {
        std::unique_lock<std::mutex> lk(m_);
        if (n_ == cap_) ++producer_stall;          // 队列满: 生产者等待(= NACK)
        not_full_.wait(lk, [&] { return n_ < cap_ || closed_; });
        if (closed_) return;
        buf_[tail_] = v;
        tail_ = (tail_ + 1) % cap_;
        ++n_;
        not_empty_.notify_one();
    }

    bool pop(T& out, std::atomic<long>& consumer_stall) {
        std::unique_lock<std::mutex> lk(m_);
        if (n_ == 0) ++consumer_stall;             // 队列空: 消费者等待(= 总线空闲无数据)
        not_empty_.wait(lk, [&] { return n_ > 0 || closed_; });
        if (n_ == 0 && closed_) return false;
        out = buf_[head_];
        head_ = (head_ + 1) % cap_;
        --n_;
        not_full_.notify_one();
        return true;
    }

    void close() {
        std::lock_guard<std::mutex> lk(m_);
        closed_ = true;
        not_empty_.notify_all();
        not_full_.notify_all();
    }

private:
    size_t cap_, head_, tail_, n_;
    std::vector<T> buf_;
    std::mutex m_;
    std::condition_variable not_full_, not_empty_;
    bool closed_;
};

int main(int argc, char** argv) {
    const size_t cap = (argc > 1) ? std::strtoul(argv[1], nullptr, 10) : 1;
    const long kItems = 2000000;
    const int  kProducers = 2, kConsumers = 2;

    BoundedQueue<long> q(cap ? cap : 1);
    std::atomic<long> produced{0}, consumed{0}, pstall{0}, cstall{0};

    const auto t0 = std::chrono::steady_clock::now();

    std::vector<std::thread> producers;
    for (int p = 0; p < kProducers; ++p)
        producers.emplace_back([&, p] {
            for (;;) {
```

```
                const long i = produced.fetch_add(1);        // 原子取号: 工作分配
                if (i >= kItems) { produced.fetch_sub(1); break; }
                if ((i % 64) == 0)                            // 制造速率波动(bursty)
                    std::this_thread::yield();
                q.push(i, pstall);
            }
        });

    std::vector<std::thread> consumers;
    for (int c = 0; c < kConsumers; ++c)
        consumers.emplace_back([&] {
            long v;
            while (q.pop(v, cstall)) consumed.fetch_add(1);
        });

    for (auto& t : producers) t.join();
    q.close();
    for (auto& t : consumers) t.join();

    const double dt = std::chrono::duration<double>(
        std::chrono::steady_clock::now() - t0).count();

    std::printf("queue cap=%-3zu  consumed=%ld  time=%.3f s  %.2f Mitems/s  "
                "producer_stalls=%ld  consumer_stalls=%ld\n",
                cap, consumed.load(), dt, kItems / dt / 1e6,
                pstall.load(), cstall.load());
    return 0;
}
```

- **【代码做什么？】** `kProducers` 个线程用 `produced.fetch_add(1)` 原子取号拿到工作项（这是 slide 53 里"队列"的另一种形态：**原子计数器本身也是一条被争用的 cache line**），每 64 项 `yield()` 一次制造**速率波动**，然后 `push` 进容量为 `cap` 的有界队列；`kConsumers` 个线程 `pop` 直到队列关闭。程序统计**生产者因队列满而等待的次数**与**消费者因队列空而等待的次数**，这正是讲义 slide 53 中 A、B 是否 stall 的度量。编译时用 `-O3` 保证 `push/pop` 的簿记代码不成为瓶颈。
- **【并行机制与性能解说】**
- **硬件上如何执行**：`std::mutex` 与 `condition_variable` 的等待/唤醒依赖 futex（内核）与缓存行上的原子操作；`produced` / `pstall` 等原子计数器在同一 cache line 上被多个核更新时会**伪共享**，本身成为串行化点。队列中的元素在生产者核上写、在消费者核上读，会发生**cache line 迁移（ping-pong）**——因此**增大队列并不免费**，深度越大、line 迁移越多。
- **Work / Span / 并行度**：
  - Work = `N × (t_produce + t_consume) + N × t_queue_bookkeeping`。
  - Span（无 stall、队列足够深时）= `N × max(t_produce, t_consume) + t_consume`（流水线填充）。
  - **并行度 = Work/Span ≈ (t_p + t_c)/max(t_p, t_c) ≤ 2：两阶段流水线的并行度上限是 2，与队列深度无关**。队列只能消除"速率波动造成的 stall"，**不能增加阶段数**——这是理解 slide 53 的关键：队列解决的是**抖动（jitter）**，不是**吞吐上限**。要更高吞吐必须**增加流水线级数或复制阶段**（例如多缓冲、多总线通道）。
  - 当 `cap = 1` 时，队列退化为近 rendezvous，`producer_stalls + consumer_stalls` 显著大于 0；`cap = 2` 时按讲义结论 stall 应大幅下降；继续增大 `cap` 收益递减（因为平均速率已经匹配，剩余 stall 只来自初始相位差与调度抖动）。

---

4. 性能模型与复杂度分析

4.1 三个必须同时写的"预算约束"

对任何一致性实现，都要同时满足三类约束，任何一类被突破就是实际瓶颈：

1. **延迟约束（latency）**：一次缺失的关键路径 `T_miss = T_req_bus + T_mem + T_data_bus + T_queue + T_retry`（本讲参数：5 + 100 + 4 + 排队 + NACK 重试）
2. **带宽约束（bandwidth）**：共享资源的上限 `BW_max = min(请求总线带宽, 数据总线带宽, 请求表项数 / T_miss × line_size, 队列深度 / T_miss × line_size)`
3. **并发约束（Little 定律）**：要隐藏延迟 `L` 并达到吞吐 `R`，需要 `并发数 N = R × L` ——这是"MSHR 数 / 请求表项数"必须够大的原因，也是 8 项请求表成为硬上限的原因。

4.2 数值算例：原子总线 vs 拆分事务总线的有效带宽

假设参数（与讲义 slide 47 一致）：数据总线 256 bit = 32 B/周期；cache line = 128 B ⇒ 数据阶段 4 周期；时钟 3.0 GHz；请求阶段 5 周期（ARB/RSLV/ADDR/DCD/ACK）；DRAM 延迟 100 周期；4 个处理器/缓存，各发出 100,000 次缺失（合计 400,000 次，每次传输 128 B ⇒ 共 51.2 MB）。

(a) 原子总线（Part 1）：每事务独占总线 `5 + 100 + 4 = 109` 周期。

- 总周期 = 400,000 × 109 = **4.36×10^7 周期**
- 时间 = 4.36×10^7 / 3.0×10^9 = **14.53 ms**
- 有效带宽 = 51.2 MB / 14.53 ms = **3.52 GB/s**
- 总线利用率 = `(5 + 4) / 109` = **8.3%**（其余 91.7% 的时间总线在等 DRAM 响应）

(b) 拆分事务总线、每缓存 MSHR = 1（`T_miss ≈ 109`，无排队、无 NACK）：

- 每缓存的缺失速率 = `1 / 109` 条/周期；系统 = `4 / 109` 条/周期
- 带宽 = `(4 / 109) × 128 B × 3.0×10⁹` = **14.1 GB/s**
- 相对原子总线提升 **4.0×**（≈ 缓存的并发缺失数）
- 时间 = 400,000 × 109 / 4 周期 = 1.09×10^7 周期 = **3.63 ms**

(c) 拆分事务总线、请求表 8 项被填满（`T_miss ≈ 109`）：

- 系统缺失速率 ⩽ `8 / 109` 条/周期（表项数限制）
- 带宽 = `(8 / 109) × 128 × 3.0×10⁹` = **28.2 GB/s**（相对原子总线 **8.0×**）
- 时间 = 400,000 × 109 / 8 = 5.45×10^6 周期 = **1.82 ms**

(d) 三条理论上限（用示例 2 的公式计算）：

- 数据总线上限 = `32 B/周期 × 3.0 GHz` = **96.0 GB/s**
- 请求总线上限 = `128 B / 5 周期 × 3.0 GHz` = **76.8 GB/s**
- **请求表项数上限 = 8 × 128 B × 3.0 GHz / 109 周期 = 28.2 GB/s** ← 最紧的约束

(e) Little 定律反推：要打满请求总线上限需要多少并发?

`N = R × L = (1 条 / 5 周期) × 109 周期 ≈ 21.8 ⇒ 约 22 个未完成请求`。8 项请求表只能支撑 `8/22 ≈ 36%` 的请求总线峰值。**结论：把请求表从 8 项扩到 24 项，才可能让带宽再提升约 2.7 倍——这正是"性能优化需要更多硬件"的量化版本。**

- **表 4：性能模型汇总（上述算例的数值）**

配置	关键路径 T_miss	并发数上限	有效带宽	相对原子总线	总线/表项利用率
原子总线	109 周期（独占）	1	3.52 GB/s	1.0×	请求线 4.6% / 数据线 3.7%
拆分事务，MSHR=1/缓存	109 周期	4	14.1 GB/s	4.0×	请求线 18% / 数据线 15%
拆分事务，请求表 8 项打满	109 周期	8	28.2 GB/s	8.0×	请求线 37% / 数据线 29%
请求总线理论峰值	—	22	76.8 GB/s	21.8×	100%
数据总线理论峰值	—	27	96.0 GB/s	27.3×	100%

（利用率一列按 `并发数 × 阶段周期 / T_miss` 估算：如 MSHR=1 时请求线 `4×5/109 = 18.3%`。）

4.3 伪共享的定量代价（示例 1 的模型）

假设：4 核，L3/跨核 cache line 迁移一次往返成本 `L_ping ≈ 60 ns`（约 180 周期 @3 GHz）；本地自增（volatile load+store 往返）`C_inc ≈ 5 周期 ≈ 1.7 ns`；每线程 5×10^7 次。

- `packed`：每次操作一次迁移 ⇒ 时间 ≈ `2×10⁸ × 60 ns = 12 s`；总操作率 ≈ **16.7 Mops/s**（4 个核加起来还不如单核本地速度）。
- `padded`：时间 ≈ `5×10⁷ × 1.7 ns = 85 ms`；总操作率 ≈ **2.35 Gops/s**。
- **比值 ≈ 140×**——两边代码"算法上完全一样"，差别只在 4 个计数器是否落在同一 64 B 行内。
- **算术强度（arithmetic intensity）视角**：把每次 line 迁移的 128 B 流量算进去，`packed` 的"每次自增 1 个操作却要搬 128 B"⇒ 算术强度 ≈ **0.008 op/B**。若内存/互连带宽为 28 GB/s（见 §4.2），则带宽可支持的速率上限 = `28e9 × 0.008 ≈ 2.2×10⁸ ops/s`，与实测的 1.7×10^7 ops/s 同量级（更慢，因为还要算延迟而非纯带宽）——**用 Roofline 的思路看：`packed` 版本被钉在"带宽/延迟天花板"上，而 `padded` 版本才回到 ALU 屋顶。**

4.4 用 Amdahl 定律看"共享资源串行化"

总线仲裁、请求表分配、数据总线仲裁在物理上都是**串行段**（同一时刻只能一个赢家）。

- 设某程序总执行时间里，**必须走总线且无法与其他核重叠**的比例为 `s`。
- Amdahl：`Speedup(N) = 1 / (s + (1−s)/N)`。若 `s = 0.20`（对总线密集型负载很常见），则 **N→∞ 时加速比上限为 5×**，N = 8 时只有 `1/(0.2 + 0.1) = 3.3×`（并行效率 42%）。
- 这也解释了 §4.2 的结论：在 4 核系统上，把有效带宽从 3.52 GB/s 提到 28.2 GB/s（8×），本质上是把那 91.7% 的"闲置串行段"回收成可用的并发窗口。原子总线版本里，每次缺失有 `100/109 = 91.7%` 的时间总线被事务独占却什么也不传——等价于把串行比例 `s` 抬到接近 1，于是 `Speedup(N) → 1`（加核没有用）；拆分成 8 项请求表后这段闲置被填上，多核才有可能拿到接近线性的加速。
- **同步/仲裁开销的代价公式**：`T(N) = T_work/N + T_serial`，其中 `T_serial ≈ (# misses) × (REQ_CYCLES / 表项数)`；对 4×10^5 次缺失、8 项表：`4×10⁵ × 5/8 ≈ 2.5×10⁵ 周期 = 0.083 ms`（相对 1.82 ms 的总时间约占 4.6%），而原子总线版本这一项退化为 `4×10⁵ × 109 = 4.36×10⁷ 周期`，占总时间 **100%**。

4.5 表 5：各类缓存与互连延迟/占用的量级参考（用于建立"预算感"）

层次	典型延迟（周期 @3 GHz）	典型延迟（时间）	是否可与其他核重叠
L1 命中	4	~1.3 ns	是（每核私有）
L2 命中	12	~4 ns	是（每核私有）
L3 命中（共享 bank）	40	~13 ns	部分（bank 可并行）
同 socket 缓存间 line 迁移	~180	~60 ns	**否（占用互连）**
DRAM	~100–300	33–100 ns	部分（多 bank 可并行）
一片 TLB miss / 缺页	10^3–10^6	0.3 μs–0.3 ms	否（关键路径）

---

5. 关键要点

1. **正确性不变量必须写下来，性能优化必须证明它没被破坏。** MESI 的三个不变量（至多一个 M；M 存在则他人全 I；S 存在则无 M）才是真正要维护的东西；状态机、总线信号、请求表都是实现手段。讲义的例子（P2 的待发 `BusUpg` 在收到他人 `BusUpg` 后必须改写成 `BusRdX`）说明：**只要引入"等待期间仍能接收事件"，就必须允许修改已经排队的操作。**
2. **共享资源（总线、请求表、数据总线、仲裁器）天然是串行段，它的容量直接决定可扩展性上限。** 原子总线的总线利用率只有 `(5+4)/109 ≈ 8.3%`；拆分成请求/响应两条总线、并配上 8 项请求表后可达 28 GB/s，而请求总线峰值 76.8 GB/s 需要约 22 个并发请求（Little 定律）。**并发数上限是硬件给的天花板，不是软件调优能突破的。**
3. **性能优化 = 把大操作拆成更多小事务 + 用队列吸收速率波动；代价永远是额外硬件与新的正确性风险。** 拆分事务带来"请求–响应匹配、冲突请求、NACK 流控"；write-back buffer 带来"侦听必须查 buffer"；多级层次带来"包含性"与"缓冲死锁"。讲义的核心句式：**Techniques that pursue high performance tend to make ensuring correctness tricky（追求高性能的技术往往让保证正确性变得棘手）。**
4. **死锁/活锁/饥饿是三种不同的病，必须对症下药。** 死锁 = 循环等待（用"等待中也服务外来请求"和"请求/响应队列分离"打断环）；活锁 = 独占所有权被反复抢走（让获得所有权的写先完成）；饥饿 = 公平性问题（FIFO 仲裁、优先级衰减）。**响应不会再产生新事务，因此响应一定能推进——这是打破队列死锁的理论基础。**
5. **commit（提交）≠ complete（完成）；总线上的请求顺序定义了并行程序的写序（write serialization）。** 写提交于"读独占事务出现在总线上并被所有缓存确认"的时刻；write-back buffer 只推迟"完成"，不改变"提交"。这条区分是后面讨论内存一致性模型（memory consistency）的地基。

---

6. 常见陷阱与注意事项

- **把协议状态机当成分步执行的原子操作来推理。** 讲义 slide 30 明确警告：查 tag、仲裁总线、等待其他控制器都不是原子的。若在代码/验证中假定"发完 `BusUpg` 就一定能进 M 态"，就会错过 slide 31 那类竞态（请求语义需要中途改写）。
- **在设计/使用缓存控制器时忘记"等待中仍要服务"。** 只等自己的请求、不处理外来侦听，直接导致 **fetch deadlock**（slide 32）；同样的错误在软件层表现为"持锁时阻塞在另一个需要该锁的调用上"（例如持锁做阻塞 I/O，而 I/O 回调又要取同一把锁）。**排队时不能停止服务，这是硬件与软件共通的准则。**

- **忽视伪共享（false sharing）。** 每个线程只写自己的变量、逻辑上零竞争，仍可能因共享同一条 64 B line 而让性能下降百倍（示例 1 的模型给出约 140×）。对策：`alignas(64)` 隔离热点计数器、线程本地归约（reduction）后再合并、按行分块（padding/padding-free 布局重构）。
- **误以为"更深的队列/更多缓冲能提高吞吐"，同时忽略公平性。** 队列只吸收**速率波动**，它让双方各自全速跑，但**不增加流水线级数**——两阶段流水线的并行度上限是 2（示例 3）。同理，把 MSHR 从 4 加到 8 而请求表仍是 8 项时，收益会饱和甚至因 NACK 重试而变差（示例 2）。而"优先级固定"的仲裁策略（例如"谁 id 小谁赢"）在功能上正确、在性能上可能让某个核永远拿不到总线，即**饥饿（starvation）**；必须用 FIFO 仲裁、优先级衰减或请求老化（aging）来保证公平。**先确认瓶颈是延迟、带宽还是并发容量，再决定加什么，并检查策略的公平性。**
- **忽略写回缓冲的一致性责任。** 只侦听 tags 而不侦听 write-back buffer，会让其他处理器读到内存里的**旧值**——这是静默的数据错误，不是性能问题（讲义 slide 29）。同理，缓冲区尺寸取 1 而不做请求/响应队列分离会导致**缓冲死锁**（slide 56–58）。
- **把"总线"当成互连的全部，并混淆"延迟"与"占用"。** 讲义特意提醒：**现代机器大多不用总线**（有专门的互连网络讲座）。多级层次（L1/L2/L3 + ring）意味着"侦听谁、失效谁、包含性怎么维护"都要重新设计；把总线的直觉直接套到 NoC/目录式系统上会得出错误结论。另外，侦听与 commit 发生在总线上**没有流量**的窗口（slide 45 的 "NO BUS TRAFFIC"），把"延迟长"直接当成"带宽被占满"会找错瓶颈。

---

7. 思考题（带答案）

问题 1. 讲义中把原子总线换成拆分事务总线后，性能提升来自哪里？为什么讲义又说"响应不必按请求顺序返回，但请求顺序确立系统全序"？如果让你把请求表从 8 项扩到 24 项，会发生什么，代价是什么？

【答案】 提升来自**把"等待响应"的时间从总线独占变成可被其他事务利用**。原子总线下一次缺失占用总线 `5 + 100 + 4 = 109` 周期，其中 100 周期总线在空转，利用率只有 `9/109 ≈ 8.3%`，有效带宽约 **3.52 GB/s**；拆分成请求总线与响应总线后，8 项请求表被打满时缺失速率上限为 `8/109` 条/周期，带宽升到 **28.2 GB/s**（8×）。 "请求顺序 = 系统全序"的原因是：**顺序必须由某个所有人都能观察到的事件来定义，而在拆分设计中，唯一全局串行的就是请求总线上的地址/命令仲裁顺序**（数据可以乱序到达，因为它只影响"什么时候拿到值"，不影响"哪个写先发生"）。只要所有缓存都按请求总线的顺序解释失效/升级，写串行化（write serialization）就成立——这正是 commit 定义（"读独占事务出现在总线上并被所有缓存确认"）的基础。 把请求表扩到 24 项：按 Little 定律，`N = R × L = (1/5) × 109 ≈ 22`，所以 24 项足以逼近**请求总线上限 76.8 GB/s**（`128 B / 5 周期 × 3 GHz`），带宽可从 28.2 GB/s 再提升约 2.7 倍；但代价是：(a) **硬件面积与功耗**（每项要存地址、状态，每个总线客户端还要维持一份副本）；(b) **冲突检查组合逻辑变复杂**（要与 24 项全比较，时序压力大）；(c) **正确性风险上升**：更多未完成事务意味着更多冲突请求、更多 NACK 重试，以及更大的"响应到达顺序与请求顺序不一致"的处理窗口（例如必须记录每个表项的侦听结果何时有效）。

问题 2. 在 §3.2 的模拟器里，把每缓存 MSHR 从 1 提高到 2、4、8 时，带宽先翻倍后趋于饱和（约 28 GB/s）。请解释两个阶段的瓶颈分别是什么，并说明如果此时把每缓存的 L1 换成两级缓存层次（L1+L2，L2 才连总线），还需要额外解决什么问题？

【答案】 **第一阶段（MSHR=1 → 2）**：MSHR=1 时每个缓存同时只有一个未完成缺失，其缺失速率被自身的**关键路径延迟**限制在 `1/T_miss ≈ 1/109` 条/周期，4 个缓存合计 `4/109`，带宽约 **14.1 GB/s**。这是一种**延迟受限（latency-bound）**状态：总线上明明还有余量，但每个处理器"只有一个未完成的洞"，无法隐藏 DRAM 延迟。MSHR 提到 2 后并发数翻倍，带宽翻倍——这就是 Little 定律的直接体现（吞吐 = 并发数 / 延迟）。**第二阶段（MSHR ⩾ 2）**：并发数继续增加已无用，因为全局**请求表只有 8 项**，成为硬上限：`8 × 128 B × 3 GHz / 109 = 28.2 GB/s`。此时是**容量受限（capacity-bound）**状态，多余的请求尝试被 NACK 并以 `RETRY_DELAY` 重试，制造额外延迟却换不来吞吐（模拟器中 NACK 计数显著上升）。要再提升必须**扩大请求表或减少每缺失的占用**，而不是继续加 MSHR。 **换成 L1+L2 层次后需要额外解决**：(a) **谁侦听总线**——若只让 L2 控制器侦听，L1 里的脏数据对总线不可见，必须维护**包含性（inclusion）**（L1 ⊆ L2，L2 驱逐时连带失效 L1），或让所有层独立侦听（低效）；(b) **响应延迟变长**（L1 缺失要走 L1→L2→总线→L2→L1），使 **fetch deadlock** 更危险——L1 在等自己请求的响应期间**必须仍能服务外来侦听请求**；(c) **缓冲死锁**：L1↔L2 之间若只有一对队列（每个容量 1），出向请求的响应需要对方队列空间、入向请求的响应需要本地队列空间，形成环——必须把所有事务分类为**请求**与**响应**并使用**分离的请求/响应队列**（响应不产生新事务，因此一定能推进并腾出资源）；(d) **表项/缓冲尺寸**要按"总线上最大未完成请求数"配足，否则层次越深越容易死锁——讲义指出这是一种正确但昂贵的解法（slide 57）。

问题 3. 有这样一段"多核计数器"代码：4 个线程各写 `count[tid]++`，其中 `long count[4]`。测得时间是从单线程版本的约 100 倍。请解释原因、用讲义中的概念命名它，并给出两种修复方案及其代价；同时说明如果这些计数器被改成"每个线程先累加到私有变量、最后合并"，为什么能得到接近 4× 的加速。

【答案】 原因是**伪共享（false sharing）**，其硬件机制是**一致性协议的所有权迁移**：`long count[4]` 共 32 B，全部落在同一条 64 B cache line 内。每次 `count[tid]++` 都是读-改-写，要求该 line 处于 **M 态**；而 M 态在任一时刻至多一个缓存持有（MESI 不变量），因此每个核的每次自增都要发出 `BusUpg` / `BusRdX`，把别的核的脏行 flush 出来再失效，line 在核之间反复**弹跳（ping-pong）**。有效执行被串行化成一条"line 迁移链"：按 §4.3 的模型，`Work/Span = C_inc/L_ping ≈ 5/180 ≈ 1/36 < 1`，即并行度低于串行版本，因此出现约 100 倍（模型估计约 140 倍）的劣化。这不是数据竞争（各线程访问的是不同内存位置，结果也正确），而是**共享一条 line 造成的额外一致性事务**。 修复方案一：**填充/对齐隔离**——把每个计数器放到独立 cache line（`struct alignas(64) { volatile long v; char pad[56]; } cnt[4];`）。代价是内存占用放大 8 倍、可能降低缓存有效容量与空间局部性，且把"每线程一个计数器"变成硬编码的布局约束（核数变化时要重新布局）。 修复方案二：**改变访问模式为分块/线程本地归约**——每个线程在自己的私有数组（或栈变量）上累加，最后用一次原子加法或临界区合并到全局计数器。代价是多了一次归约（O(线程数) 的同步），且要求算法允许"部分和"这种分解（不是所有算子都满足，例如非结合/非交换的更新就不行）。 "私有变量 + 最后合并"能得到接近 4× 加速，是因为它**把每次操作的通信彻底消除**：整个自增循环期间，工作集是本核私有且独占的 cache line（保持在 M 态不迁移），每一

步只是本地 ALU/store-to-load 依赖链（`C_inc` 很小）；只有最后 `N` 次合并才触碰共享行，事务次数从 2×10^8 次降到 `4` 次。用 Work/Span 说：Work 仍为 2×10^8 次自增，Span 变成"单个线程的 5×10^7 次本地依赖链 + 一次归约同步"，并行度回到 ≈ 4（线程数），因此理想加速比接近 4×（受最后归约与 Amdahl 中极小串行段限制，实际略低于 4）。这也验证了讲义的总结：**并行程序里的"通信"既包括显式消息/锁，也包括你没意识到的一致性流量。**

附：本讲一句话地图

```
目标 (slide 6): 正确 + 高性能 + 低硬件成本
     |
     +--> Part 1 (slide 19-37): 原子总线 + 单级写回缓存
     |         争用:  处理器侧 vs 侦听控制器 (复制 tag / 多端口 tag)
     |         信号:  Shared / Dirty / Snoop-pending (线与)
     |         优化:  write-back buffer   -> 侦听必须查 buffer
     |         正确性: 竞态 / fetch deadlock / livelock / starvation
     |         语义:  commit (总线上确认) != complete (数据落到行里)
     |
     +--> Part 2 (slide 38-53): 拆分事务总线 (请求/响应分离)
     |         机制:  请求表 + 3-bit tag + 两条总线 + NACK 流控
     |         冲突:  读-读可搭车; 读写冲突必须挂起等待
     |         队列:  吸收速率波动 (深度 2 即够), 但不增加并行度上限
     |
     +--> 层次与收尾 (slide 54-60): 包含性 / 缓冲死锁 / 请求-响应队列分离
               最终审判: 一句 `int x = 10;` 的 20 个步骤
```

Lecture 14: Performance Analysis / Profiling

1. 章节标题与概述

Lecture 14: Performance Analysis / Profiling（性能分析与剖析）

- **本讲核心问题**：**怎么知道程序慢在哪里，以及怎么证明自己知道？** 并行程序的性能问题有四大类来源——**计算（compute）**、**内存带宽/延迟（memory bandwidth / latency）**、**同步（synchronization）**、**并行度不足/负载不均（insufficient parallelism / load imbalance）**；这四类问题的现象常常互相伪装（"加线程没变快"既可能是带宽饱和，也可能是锁竞争）。本讲要建立一套**可测量、可证伪**的方法学：先用**正确的计时与统计**得到可信的数字，再用**剖析（profiling）与硬件性能计数器（PMU）**定位热点与瓶颈类型，最后用**高水位（high watermark）实验**与 **Roofline 模型**把"瓶颈诊断"从直觉变成定量结论。
- **涉及的主要硬件/软件机制**：硬件侧是 **PMU（Performance Monitoring Unit，性能监控单元）**——处理器内部一组可编程的**事件计数器**（退休指令数、时钟周期、L1/L2/LLC 命中与缺失、从内存控制器读取的字节数、各类 stall 周期），以及**计数器溢出中断**这一把"计数器"变成"采样剖析器"的机制；软件侧是**插桩（instrumentation）**、**统计采样（sampling）**、**计数器读取（counting / event-based）**三大家族工具（`gprof`、`perf`、Intel VTune、PAPI、PCM、Callgrind、Nsight Compute/Systems），以及把测量结果转成优化决策的分析框架（Roofline、Amdahl、work-span、strong/weak scaling）。
- **在并行计算知识体系中的角色**：本讲是**全课程的方法论枢纽**。前面的讲座（指令级并行、SIMD、多核、缓存一致性、伪共享、同步）提供了"性能为什么会这样"的机制，后面的讲座（性能优化、异构与 GPU、专用化）提供"应该怎么改"的手段；本讲回答的是中间那个问题——**"我手上这份代码，实际卡在哪一条机制上？"** 它把三个此前零散出现的判据合并成一条工作流：①**用 work-span / 并行度判断"是不是并行度不够"**；②**用算术强度 / Roofline 判断"是不是带宽不够"**；③**用 PMU 计数器 + 高水位实验判断"是不是同步或延迟不够"**。此外，本讲也是后续"自动性能优化"（Halide autoscheduler、自动调优、LLM 代码生成 + profile 反馈）得以成立的前提：**所有自动优化循环的本质都是"测量 → 建模 → 搜索 → 再测量"**，而没有可信的测量，搜索就失去目标函数。
- **配套材料**：
- **CMU 15-418 Fall 2026 本讲讲义：未发布。** Fall 2026 已公开的讲义 PDF 位于 https://www.cs.cmu.edu/~418/lectures/ 之下（可直接下载）；但 **Lecture 14（Performance Analysis / Profiling）在 Fall 2026 尚未在该公开目录发布**，课程主页 https://www.cs.cmu.edu/~418/ 与日程表 https://www.cs.cmu.edu/~418/schedule.html 中本讲日期为 **Sep 28**，其材料栏为空。
- **历史学期对应讲义（`09_perfeval.pdf` / `13_perftools.pdf`，F22 归档）：需登录、未公开。** 它们位于 `/afs/cs/academic/class/15418-*/public/` 之下，需要 CMU 登录；本仓库 `supp/` 目录下的 `f22_perfeval.pdf`、`f22_perftools.pdf` 两个文件**只是 CMU IdP 登录页的 HTML（各约 1.7 KB），不含任何讲义内容**，因此本笔记无法引用其正文。

- **关于本讲可用的抽取文本的说明**：`cs149_supp/thoughtprocess.txt` 与 `cs149_supp/aiperfoptimization.txt` 是**姊妹课程 CS149 的相邻主题讲义**（"并行编程的思维过程"与"领域特定系统与自动性能优化"），**并不是** CMU 15–418 Lecture 14 的讲义正文。本笔记以其中可直接复用的术语、公式、案例与结论（Amdahl 定律、分解/分配/编排、图像处理的工作计数与算术强度结论、高水位与 Roofline 方法学、PMU 计数器、profile 驱动的优化循环）作为事实基础，其余关于剖析工具机制（`gprof` 的 `mcount` 插桩、`perf` 的采样与 `perf_event` 子系统、PMU 多路复用、Nsight Compute 指标、Callgrind 模拟）的论述基于该主题的成熟公开知识（如 Roofline 原始论文 Williams et al. 2009 与各工具官方文档所描述的机制）；所有定量结论均显式给出假设参数，便于读者自行复核。
- **已公开的姊妹课程（Stanford CS149）材料（本笔记的主要事实基础）**：
  - `cs149_supp/perfopt2.txt`：**已公开**。CS149 Fall 2025 Lecture 6 "Performance Optimization Part II: Locality, Communication, and Contention"（68 页）。它包含本讲最核心的三块方法论：**"确定你受限于计算、内存带宽还是同步"**、**高水位实验**、**Roofline 模型**（computing-limited 水平段 vs bandwidth-limited 斜段），以及 **PMU 计数器**（Intel PCM 的 C++ API 示例：`getIPC`、`getL3CacheHitRatio`、`getBytesReadFromMC`）与 VTune / PAPI / oprofile 的指引；并附有**固定问题规模测速的陷阱**（32 处理器 SGI Origin 2000 上的 258×258 网格）与**超线性加速**的解释。
  - `cs149_supp/thoughtprocess.txt`：**已公开**。CS149 Fall 2025 Lecture 4 "Parallelizing Code: The Programming Thought Process"。提供**分解/分配/编排/映射**四阶段框架、**Amdahl 定律**（`S = 串行比例 ⇒ 最大加速比 ≤ 1/S`）、静态 vs 动态分配、ISPC `foreach`/gang 抽象、交错（interleaved，packed load）vs 分块（blocked，gather）分配、网格求解器与屏障，以及**"每元素加锁 → 每次迭代加锁一次"**与**"三个屏障 → 一个屏障"**的经典优化。
  - `cs149_supp/aiperfoptimization.txt`：**已公开**。CS149 Fall 2025 Lecture 13 "Domain-Specific Programming Systems and Automatic Performance Optimization"。提供**图像处理的精确定量工作计数**（3×3 二维卷积 `9×W×H`；两趟分离 `6×W×H`；分块后 `(34/16)×3×W×H = 6.4×W×H`）、**"两趟实现比二维实现算术强度低 2 倍"**这一结论、分块/融合的**生产者-消费者局部性**论证、Halide 的算法/调度分离与**自动调度器**（代价模型为小型 MLP，输出 27 个系数供手写代价模型使用；1.4M 个 schedule 在 166 秒内评估完）、以及**用 profiling 统计量驱动的 LLM 反思式优化循环**（示例统计量：`Timing: 32 ms`、`SM util 42%`、`DRAM util 89%`、`L2 cache hit 68%`）。
  - `cs149_supp/efficiency.txt`：**已公开**。提供加速比与效率的定义（`speedup = T(1)/T(P)`；效率 = 每单位资源的性能，如每瓦/每美元/每芯片面积的 ops/s）与**"FAST ≠ EFFICIENT**（在 10 核上只有 2× 算不算好结果？）"的判据。
- **CMU 侧其它已公开的相关讲义**（用于术语与课程语境对齐）：`extracted/07_progperf1.txt`、`extracted/08_progperf2.txt`（Performance Optimization I/II，含周期性自我剖析与重分配）、`extracted/05-CUDA-programming.txt`（GPU 剖析语境）。
- **未发布/需登录**：Fall 2026 的**录像（Panopto/YouTube）在日程表中被注释隐藏，属未发布**；**Ed 讨论区、Autolab、Canvas 均需登录**。考试覆盖方面，公开考试页显示 **Exam 2 覆盖 Lectures 14–24（不含客座讲座），本讲是其起点**。
- **说明**：讲义首页有时写有历史学期（如 Fall 2025）字样，这是讲义沿用，属正常现象，不视为错误。

---

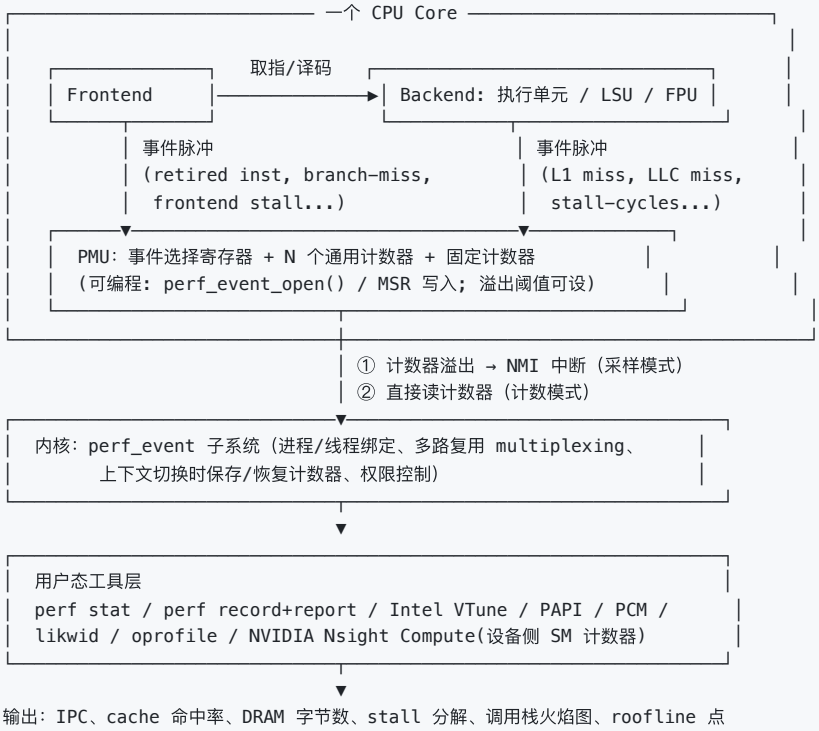
2. 核心概念与硬件/软件架构图解

2.1 性能的第一性度量：到底在测什么时间

- **定义与目的**：性能分析的第一步是**选对被测对象**。三个不同的"时间"回答三个不同的问题：
- **wall-clock time（墙钟时间 / 真实经过时间）**：从开始到结束的物理时间。**这是用户唯一关心的时间**，也是加速比公式里必须用的时间。`std::chrono::steady_clock`、`MPI_Wtime`、`omp_get_wtime` 测的都是它。
- **CPU time（进程占用 CPU 的时间）**：所有线程/进程的时间**之和**。`clock()`、`getrusage()`、`/usr/bin/time` 的 user+sys 给的是它。**陷阱**：在 8 线程并行区里，`clock()` 会报告约 8× 的墙钟时间——在并行程序里拿 `clock()` 算加速比，会得到荒谬的"负加速比"。
- **硬件事件计数（event counts）**：退休指令数、周期数、cache miss 数、DRAM 字节数。它们**不是时间**，但能解释时间：`T = 指令数 / IPC / 频率`，`T ≥ 流量 / 带宽`。
- **直观解释（"它是什么？"）**：把程序想成一家餐厅。**墙钟时间**是顾客从进门到出门的时长（顾客只在乎这个）；**CPU 时间**是所有厨师在厨房里干活的总人时（8 个厨师一起干 1 分钟的菜，总人时是 8 分钟，但顾客只等了 1 分钟）；**硬件计数器**是厨房里的各种仪表——切菜次数、冰箱开门次数、走到仓库取货的次数。要诊断"为什么顾客等这么久"，你必须同时看墙钟（等太久）、看人时（厨师在忙还是互相撞）、看仪表（是不是都在排队等同一个冰箱）。
- **性能特征（延迟 / 带宽 / 吞吐量）**：这三者在本讲中始终要分清。**延迟（latency）**是单次操作的时间（DRAM ~80–100 ns）；**带宽（bandwidth）**是单位时间能搬多少数据（DRAM ~20 GB/s 量级）；**吞吐量（throughput）**是单位时间完成多少任务。**延迟靠"隐藏"（并发、流水、预取），带宽只能靠"少搬"**——这是贯穿全课程的区分。

2.2 剖析方法学三大家族：插桩、采样、计数器

- **定义与目的**：剖析器（profiler）回答"时间花在哪个函数/哪行代码/哪条指令上"。三大家族的机制截然不同，代价与可信度也不同。
- **直观解释（"它是什么？"）**：
- **插桩（instrumentation）＝在每个房间装一台打卡机**：进函数打一次卡、出函数打一次卡，于是精确知道每段代码的耗时与调用次数。代价是打卡机本身要时间（每次函数调用多几十到上百纳秒），而且**它会改变被测量的对象**（尤其对小函数和深递归）。
- **采样（sampling）＝总经理每隔 1 毫秒随机巡楼一次，记下"此刻谁在干活"**。它不改变程序（代价 <1%），但得到的是**统计估计**而非精确值；样本越多越准，短命函数可能一次都没被看到。
- **计数器计数（counting / event-based）＝看水电表**：不关心"谁在干活"，只关心"总共用了多少度电、开了多少次冰箱"。它给出**精确的总量**（周期数、缺失数、DRAM 字节数），但**不能直接告诉你这些量属于哪一行代码**——除非配合采样（把"哪个事件溢出"作为采样触发条件），这正是 `perf record -e LLC-load-misses` 的做法。
- **架构/机制图解（工具链数据通路，硬件结构视角）**：



- **关键操作与性能特征**：
- **计数器数量是稀缺资源**：一个物理 core 通常只有 4–8 个通用计数器，而可命名的事件有几百个。想同时测的事件超过物理计数器数时，内核会做**多路复用（multiplexing）**——把时间切片分给不同事件组，再按启用比例**外推**。后果：**计数结果带外推误差**（通常几 %，负载不平稳时更大），`perf stat` 会打印 `[63.45%]` 这样的"该事件实际被启用时间的占比"，这一列必须看。
- **采样的开销几乎恒定**，与函数调用频率无关；**插桩的开销与调用次数成正比**。对调用 10^8 次的小函数，插桩版本可能慢 10 倍以上。
- **采样有"滑移（skid）"**：中断是流水线延迟之后才被处理和记录指令指针的，因此被归因的指令可能比真正引起事件的指令晚几十到上百条。对**单条热点指令**的定位要小心，对函数级归因通常无伤。
- **表格：三类剖析技术对比**

技术	机制	归因粒度	开销	能否给精确总量	典型工具	主要失真来源
插桩（instrumentation）	编译期/手动插入计数与计时调用（`mcount`、`-pg`、`TAU`、手动 `rdtsc`）	函数/代码块级，精确调用次数与调用图	与调用次数成正比，常 2×–10×+	是（时间与次数）	`gprof`、TAU、手动计时器	mcount 开销偏向小函数；不支持共享库/多线程语义
统计采样（sampling）	定时器中断（`SIGPROF`、`ITIMER_PROF`）或 PMU 溢出中断	函数/行/指令级（需调试信息与栈展开）	低，通常 1%–5%	否（统计估计，有置信区间）	`perf record`、VTune、gprof 的直方图部分、Nsight Compute	统计误差；短命函数欠采样；栈展开依赖帧指针/DWARF

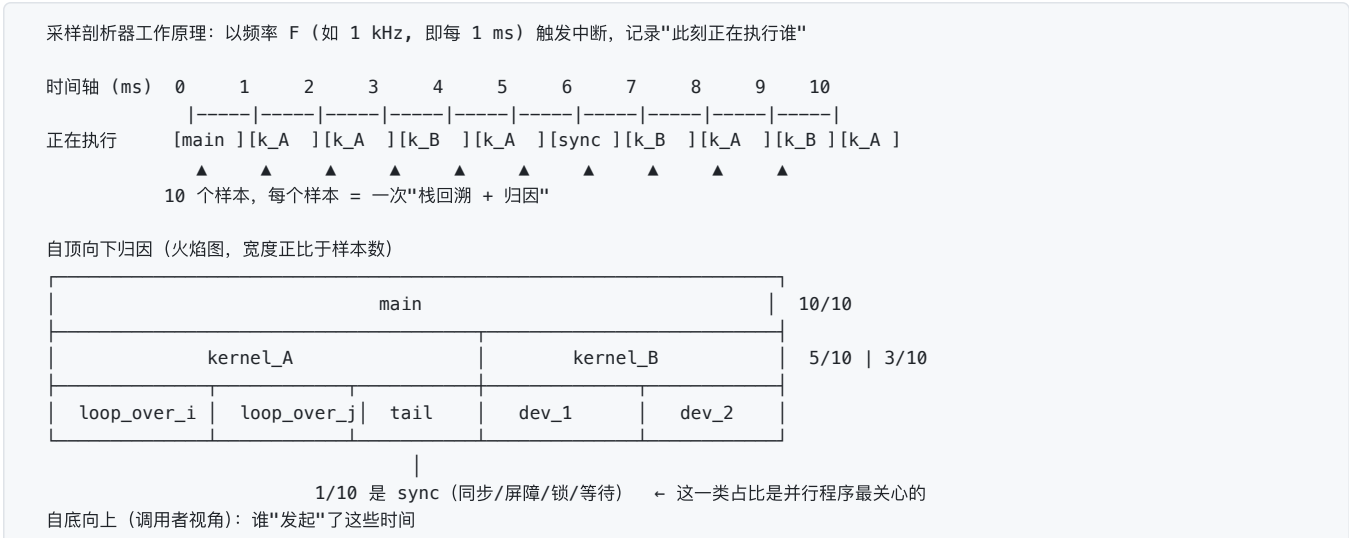
技术	机制	归因粒度	开销	能否给精确总量	典型工具	主要失真来源
计数器（event counting）	直接读 PMU 计数器，不采样	无代码归因（只有总量，配合采样或区间才有归因）	极低（寄存器读）	是（周期、指令、缺失、字节）	`perf stat`、PAPI、PCM、likwid、VTune	多路复用外推误差；事件语义需查手册；uncore/IMC 事件的共享与归属
动态模拟（simulation）	逐指令在模拟器中执行，精确建模 cache/分支预测	任意粒度，完全确定、可重现	极高（数十到数百倍变慢）	是	Valgrind/Callgrind、gem5	模拟器≠真机器；不反映真实乱序与预取；只适合小输入

2.3 硬件性能计数器（PMU）：事件选择与诊断语义

- **定义与目的**：PMU 是把"微架构内部发生了什么"暴露给软件的唯一官方通道。它解决的问题是：**在不改代码、不猜的前提下，拿到"程序在硬件上到底干了什么"的客观证据**。CS149 的 `perfopt2` 讲义对此的表述是："**所有现代处理器都有低层事件性能计数器**——记录指令完成数、时钟周期、L2/L3 缓存命中/缺失、从内存控制器读取的字节数等；例如 Intel 的 Performance Counter Monitor Tool 提供访问这些寄存器的 C++ API（`getIPC`、`getL3CacheHitRatio`、`getBytesReadFromMC`）；另见 Intel VTune、PAPI、oprofile"。
- **直观解释（"它是什么？"）**：PMU 就是**汽车的仪表盘**。你不能通过"感觉车很肉"来修发动机，但你能看转速、水温、涡轮压力、油耗（每百公里多少升）。"油耗高 + 转速低"指向一个诊断，"转速高 + 车速低"指向完全不同的诊断。**单个计数器几乎从不下结论，一组计数器的比值才下结论**——这正是 IPC、cache 命中率、stall 分解、字节/秒这些"派生指标"存在的意义。
- **表格：关键 PMU 事件与它们诊断什么**

事件（`perf` 名称）	含义	派生指标	典型"异常"读数说明什么
`cycles`	核心时钟周期数	与 `time` 相除得**实际频率**	实测频率远低于标称 ⇒ 降频（功耗/散热/AVX 重负载）或核被抢占
`instructions`	退休（retired）指令数	**IPC = instructions/cycles**	IPC < 1 且 cycles 高 ⇒ 停顿主导：内存延迟、分支误预测、长依赖链
`branch-misses` / `branches`	分支误预测数与分支数	误预测率（典型 <1% 为良，>5% 为差）	误预测率高 ⇒ 数据依赖的分支（如链表的 `while(p)`、快速排序的划分）
`L1-dcache-load-misses`	L1 数据缺失数	L1 缺失率	高 ⇒ 工作集/步长问题：不连续访问、跨 stride 访问
`LLC-load-misses` / `LLC-loads`	末级缓存缺失率	**LLC 命中率**（讲义示例打印的 `getL3CacheHitRatio`）	命中率低 + `stalled-cycles-backend` 高 ⇒ DRAM 延迟/带宽瓶颈
`cache-misses` / `cache-references`	通用缓存缺失（厂商映射）	—	只用于粗筛，粒度不足以下结论
`mem_load_retired.*` / uncore IMC 事件	从内存控制器读取/写入的字节	**实测 DRAM 带宽 = 字节/时间**	最关键的"带宽天花板"证据：若实测已接近机器可持续带宽 ⇒ 加线程无用
`stalled-cycles-frontend` / `-backend`	前端（取指/译码）与后端（执行/访存）停顿周期	stall 分解的 CPI stack	前端高 ⇒ 指令供应不足（I-cache、译码、分支）；后端高 ⇒ 执行/访存端口或内存
`offcore` / `local vs remote DRAM`	跨 socket/远地内存访问	NUMA 远地访问比例	远地比例高 ⇒ 数据/线程亲和性问题（first-touch、NUMA 绑核）
`cpu-clock` / `task-clock`	软件事件（内核高精度定时器）	与 `cycles` 配合推算频率	采样默认事件，跨平台可用但信息量低

- **架构/机制图解（采样剖析器的时间轴与归因，软件执行模型视角）**：



- **统计灵敏度的定量后果（必须记住的算术）**：采样数的统计误差按二项分布估计，绝对误差 `m = z·sqrt(p(1−p)/n)`。取 `z = 1.96`（95% 置信）、`p = 0.5`（最坏情形）：

想要的绝对精度	需要的样本数 n	1 kHz 下需要的测量时长
±5 个百分点	≈ 385	≈ 0.39 s
±1 个百分点	≈ 9604	≈ 9.6 s
±0.1 个百分点	≈ 960,000	≈ 960 s（16 分钟）

推论 ①：**"这个函数占 0.3% 时间"这种结论，在 1 秒的采样里根本不可信**（0.3% × 1000 样本 = 3 个样本，泊松相对误差 ≈ `1/√3 ≈ 58%`）。推论 ②：想用采样精确定位一个只占 0.1% 的函数，需要约 10^4–10^5 个样本、即 10–100 秒的测量，或者改测**更大/更重**的输入。推论 ③：**先用长跑粗筛、再用小输入精确定位热点行**，是标准做法。

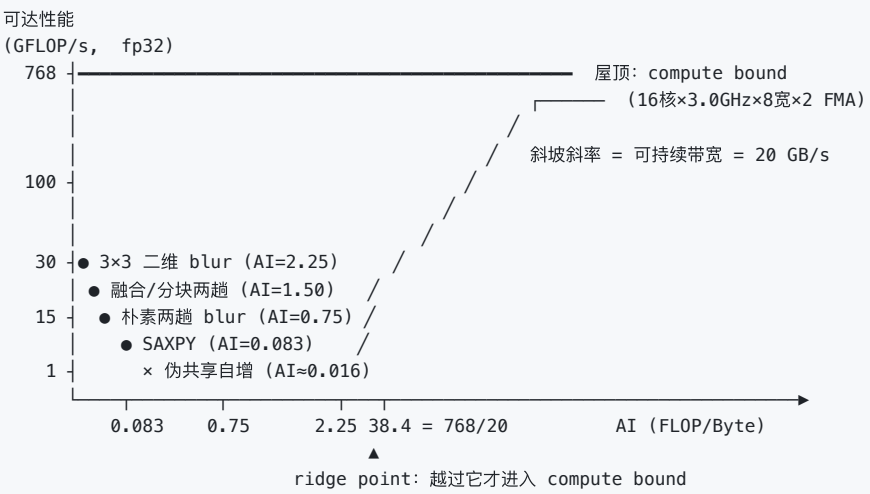
2.4 Roofline 与算术强度：把"瓶颈类型"变成坐标系

- **定义与目的**：**算术强度（arithmetic intensity）** `AI = 浮点操作数 / 从内存系统搬运的字节数`（FLOP/Byte）。**Roofline 模型**（Williams et al. 2009，CS149 讲义直接引用该图）给出可达性能上界：

`可达性能 = min( 峰值算力 P_peak , AI × 可持续带宽 BW ) ridge point（拐点）= P_peak / BW`

它解决的问题是：**在动任何优化之前，先判断"加算力有用还是减流量有用"**。`AI < ridge` ⇒ 带宽受限，任何算术优化的收益都接近 0；`AI > ridge` ⇒ 计算受限，减少访存无益，要提 ILP/SIMD/减少指令数。

- **直观解释（"它是什么？"）**：Roofline 是一张**"水管与漏斗"图**。屋顶的水平段是**处理器的算力上限**（漏斗口径固定：每秒最多漏 768 GFLOP）；斜段是**内存带宽的斜坡**（水管流量固定：每秒最多供 20 GB 数据）。**每搬来 1 字节数据，你能做多少次运算**决定了你站在斜坡上还是屋顶下：算术强度太低，你搬来的数据"不够算"，处理器在等数据（斜坡限制）；算术强度够高，数据供得上，你受限于算力（屋顶限制）。
- **架构/机制图解（Roofline，双对数坐标）**：



优化方向的语言：
斜段上的点 → 唯一有效的手段是"每字节做更多运算"：分块(tiling)、融合(fusion)、提高数据复用、缩短数据宽度(fp32→fp16)、降低精度需求
接近屋顶点 → 手段变成：SIMD 宽度、FMA、减少指令数、提高 ILP、消除除法/超越函数

- **性能特征**：Roofline 的**上界是乐观的**（它假设带宽可以被完全利用、且不区分延迟与带宽），因此它给出的是**"最好可能有多好"**，即所谓 **high watermark（高水位）**——这正是它最大的用途：**当你已经达到 Roofline 上界的 80%–90% 时，就应该停止微调，去做别的**；当你离上界很远，模型会告诉你差距是"流量"还是"算力"造成的。

2.5 高水位方法学：把"我觉得"换成"我测到"

- **定义与目的**：来自 CS149 `perfopt2` 讲义的核心方法论（原话：**"永远、永远、永远先写最简单的并行版本，然后测量性能，看看你站在哪里"**、**"确定你的性能是受限于计算、内存带宽（或内存延迟），还是同步"**、**"尝试建立高水位：实践中最好能做到多少？你的实现离最好情形有多近？"**）。高水位 = **通过故意破坏某一种资源的使用方式，测出"若这种资源不再受限，程序能跑多快"**。
- **直观解释（"它是什么？"）**：像给病人做**激发试验**。想判断"病人跑不动是不是因为心脏"？让他吸纯氧再跑一次——如果速度没变，那瓶颈就不是供氧。高水位实验就是给程序做激发试验：**故意把访存全部改成命中同一个地址（等于把内存流量降到接近 0），如果程序快了很多 ⇒ 流量是瓶颈；如果几乎没变 ⇒ 流量不是瓶颈**。
- **表格：四类高水位实验（讲义给出的四个，原文照录并解释）**

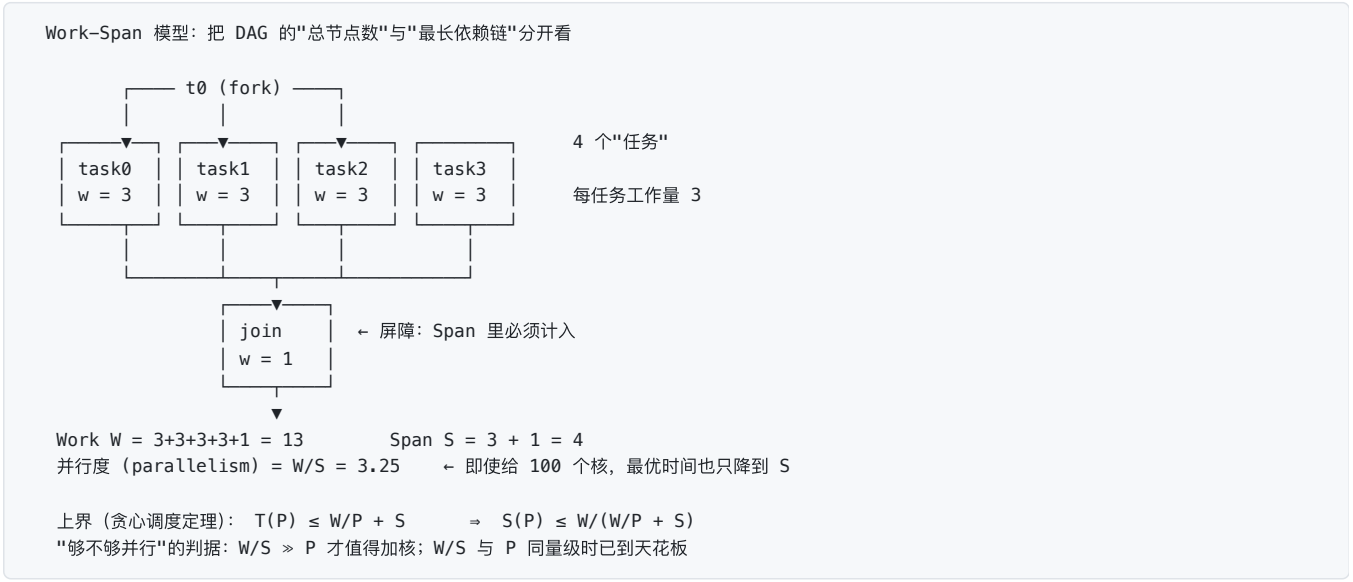
实验（讲义原文）	怎么做	得到什么结论（高水位含义）	注意事项
Add "math" (non-memory instructions) — 增加非访存运算	在内核里插入额外算术（但保证不被优化掉）	若时间随运算量**线性增长 ⇒ instruction-rate limited**（受指令发射/SIMD 宽度限制）	时间对运算量线性 ⇒ 说明既不是带宽也不是同步主导
Change all array accesses to `A[0]`	把所有数组下标改成 0（保持访问次数不变）	提速多少 ⇒ **改善访存局部性的收益上界**	可能让编译器把访存优化掉，需用 `volatile`/内联汇编阻挡
Remove all atomic operations or locks	删掉锁/原子操作并实测（工作量大致不变）	提速多少 ⇒ **降低同步开销的收益上界**	结果可能不正确，只用于测速；必须确认工作量未显著改变
Remove almost all math, but load same data	保留全部访存、删掉大部分计算	时间下降**不多** ⇒ 怀疑**内存瓶颈**；下降很多 ⇒ 计算才是主要成本	与第一项互为对照，两者合起来才能分离 compute 与 memory 的贡献

- **讲义给出的最重要的一条免责声明**：**"计算、访存与同步几乎从不会完美重叠；因此总体性能很少完全由 compute 或 bandwidth 或 sync 之一决定。即便如此，性能对上列改动的敏感度仍是主导成本的很好指示。"** 这句话定义了本讲的**方法论诚实性**：性能分析给的是**归因与敏感度**，不是"单一原因的判决书"。

2.6 可扩展性度量：加速比的正确测法（speedup measurement）与三类经典错误

- **定义与目的**：
- **加速比（speedup）**：`S(P) = T(1) / T(P)`（`T` 必须是墙钟时间；讲义亦给出等价形式"P 个处理器上的 ops/s ÷ 单处理器 ops/s"）。**加速比测量（speedup measurement）**是本讲的核心技能之一———它看起来只是一次除法，却是性能报告中最常出错的地方（见本节末的三类错误）。
- **效率（efficiency）**：`E(P) = S(P)/P`——"性能 per 单位资源"，讲义强调资源可以是芯片面积、美元、瓦特。

- **strong scaling（强可扩展）**：问题规模固定，看时间如何随 P 下降；**weak scaling（弱可扩展）**：每处理器的问题规模固定，看时间如何随机器变大保持不变。
- **Amdahl 定律**：设 `f` 为**本质上串行**的工作占比，则 `S(P) ≤ 1/(f + (1−f)/P)`，且 `S(∞) = 1/f`。讲义中的直观版本：**`S = 串行比例 ⇒ 最大加速比 ≤ 1/S`**。
- **Gustafson 定律（弱扩展）**：`S_scaled(P) = P − f·(P−1)`（随机器变大而变大问题规模时，串行部分只占固定绝对时间）。
- **直观解释（"它是什么？"）**：Amdahl 是"买更大的厨房，做同一桌菜"——如果那道必须由主厨一个人完成的摆盘要 10 分钟，你请 100 个副厨也只能把其余部分压到接近 0，总时间仍 ≥ 10 分钟，最大加速比被钉死在 `1/f`。Gustafson 是"买更大的厨房，做更大的宴席"——串行部分（比如主厨尝味道）的绝对时间不变，但总工作量随厨师数线性增长，于是**规模化的加速比可以接近 P**。这解释了一个看似矛盾的现象：超级计算机的"实测加速比"常常好于 Amdahl 的强扩展预测，因为**人们实际上在解更大的问题**。
- **架构/机制图解（work–span / fork–join：判断"并行度够不够"）**：



- **三类经典的"测加速比"错误（来自 CS149 `perfopt2` 讲义，原文均有对应）**：
- **基线选择错误**："**常见陷阱：把并行程序的加速比，对比于'并行算法在单核上运行'。**" 讲义原文：*Common pitfall: compare parallel program speedup to parallel algorithm running on one core (easier to make yourself look good).* 正确做法是**与最好的（best known）串行实现比较**，并在报告中写明基线是什么。注意：并行算法常常要做**更多总工作**（例如红黑着色法比 Gauss–Seidel 收敛更慢），把它单核的时间当基线会凭空"造出"加速比。
- **固定问题规模**：32 处理器 SGI Origin 2000 上跑 **258×258 网格**（每处理器仅约 310 个格点）时，讲义明确标注 **"No benefit! (slight slowdown)"**——通信/计算比太高，问题**对机器来说太小**；同一台机器上换成 **1K×1K 网格**（每处理器 ~32K 格点）才看到正常扩展。推论：**用固定问题规模评价机器是危险的**，因为规模一变，负载均衡、开销、算术强度、局部性全都变。
- **超线性加速（super–linear speedup）的误读**：`S(P) > P` 通常不是"并行效率超过 100%"，而是**工作集变小**导致的：随着 P 增大，每个处理器分到的数据块开始**装进 cache**（讲义原文："with enough processors, chunk of grid assigned to each processor begins to fit in cache"），或原本在单机上**换页到磁盘（thrashing）**的工作集在更大内存的机器上不再换页。它不是算法变好，而是**被比较的两个系统在存储层次上不是同一个系统**。

2.7 抽象与实现（abstraction pitfalls）：为什么"同一段抽象代码"性能差 3 倍

- **定义与目的**：本讲的关键词之一是 **abstraction pitfalls（抽象的陷阱）**。**抽象（abstraction）**给程序员一个简洁的心智模型，但**性能由实现（implementation）决定**，而实现常常泄漏到抽象之外。CS149 的 `thoughtprocess` 讲义用 ISPC 给出了教科书级的例子：**同一份 `sinx` 的抽象语义，两种"分配（assignment）"策略编译出的机器码完全不同**：
- **交错分配（interleaved）**：`idx = i + programIndex`，8 个 program instance 访问**连续**的 8 个 float ⇒ 一条 **packed vector load**（**`vmovaps`，对应 `_mm256_load_ps`**） 即可完成；
- **分块分配（blocked）**：`start = programIndex*count; idx = start + i`，8 个 instance 访问**相隔 `count` 的元素**，在内存中不连续 ⇒ 必须用 **gather 指令（`vgatherdps`，对应 `_mm256_i32gather_ps`）**，讲义原文："**gather is a more complex, and more costly SIMD instruction...**"。
- 这就是"抽象 vs 实现"的完整教训：`foreach` 抽象只说"这些迭代要做"，**没规定谁做哪些**；而 ISPC 现实的实现对 `foreach` 采用的是**静态交错方案**（讲义原文："abstraction leaves room for dynamic assignment, but current ISPC implementation is a static scheme"）。**程序员若不知道实现，就无法解释自己程序的性能**。

- **直观解释（"它是什么？"）**：抽象像**餐厅菜单**上的"招牌炒饭"。菜单不写米是当季新米还是陈米、灶是几个火眼，但**你吃到的东西由后厨决定**。同一个"炒饭"抽象，两家店能差三倍。性能分析的重要一半工作，就是**掀开后厨看一眼**（看汇编、看 profiler 的指令级归因、看 `perf stat` 的 IPC 与缺失率）。
- **其它常见的抽象陷阱（本讲会反复出现）**：
- `std::vector<T>::operator[]` / **迭代器与越界检查**：debug 构建与 release 构建的抽象完全不同（`_GLIBCXX_ASSERTIONS`）。
- `-O3` **下的内联**：被内联的函数在采样剖析的调用栈中**消失**，其时间被归到调用者头上。若确实需要区分，用 `__attribute__((noinline))` 或看带 DWARF 的 `perf report --inline`。
- **编译器优化掉你要测的代码**：C++ 里未经使用的结果会被整段删除（**dead code elimination**），于是你测出"0 ms"的惊人性能。必须把结果"消费掉"（打印校验和、写入 `volatile` 全局、`asm volatile("" ::"r"(x))`）。
- **计时器抽象**：`clock()` 是 CPU 时间而 `std::chrono::steady_clock` 是墙钟；在多线程里用错会把加速比符号都搞反。
- `omp_get_wtime` vs `MPI_Wtime` vs CUDA event：不同计时域的默认行为与同步语义不同（CUDA 的 `cudaEvent` 需要在正确 stream 上记录方有异步语义）。
- **性能计数器命名的抽象**：`cache-misses` 在不同微架构上映射到不同事件；**跨机器比较原始计数没有意义**，只有**比值**（IPC、命中率、字节/秒、% of peak）才可比较。

---

3. 代码示例与性能分析

3.1 示例 1：可复现的计时骨架 + 两趟 vs 分块融合 blur 的 Roofline 定位（C++ / OpenMP）

本示例严格对应 CS149 `aiperfoptimization` 讲义中的图像处理案例：3×3 盒式模糊（box blur），二维卷积 ↔ 两趟可分离实现 ↔ 分块融合实现，并给出每一版的**工作量计数、流量估计、Roofline 定位**。

```cpp
// blur_bench.cpp —— 两趟(朴素) vs 分块融合 3x3 盒式模糊 + 可信计时骨架
//
// 编译(必须 release 优化): g++ -O3 -march=native -fopenmp blur_bench.cpp -o blur_bench
// 运行:                     OMP_NUM_THREADS=8 ./blur_bench 1024 20
// 剖析(PMU 计数):          perf stat -e cycles,instructions,L1-dcache-load-misses,LLC-load-misses ./blur_bench 1024 20
// 采样热点:                 perf record -g -F 999 ./blur_bench 1024 20 && perf report

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <vector>
#include <algorithm>
#include <chrono>
#include <omp.h>

static const float W3 = 1.0f / 3.0f;    // 1x3 与 3x1 的归一化权重

// ---------- 版本 A：两趟（先水平、再垂直），需要完整 (H+2) x W 的中间缓冲 ----------
// 对应讲义 slide "Two-pass 3x3 blur"
static void blur_x(const std::vector<float>& in, std::vector<float>& tmp, int W, int H) {
    const int stride = W + 2;
    #pragma omp parallel for schedule(static)
    for (int j = 0; j < H + 2; ++j) {
        const float* row = &in[(size_t)j * stride];
        float*       out = &tmp[(size_t)j * W];
        for (int i = 0; i < W; ++i)
            out[i] = W3 * (row[i] + row[i + 1] + row[i + 2]);
    }
}

static void blur_y(const std::vector<float>& tmp, std::vector<float>& out, int W, int H) {
    #pragma omp parallel for schedule(static)
    for (int j = 0; j < H; ++j) {
        const float* r0 = &tmp[(size_t)(j + 0) * W];
        const float* r1 = &tmp[(size_t)(j + 1) * W];
        const float* r2 = &tmp[(size_t)(j + 2) * W];
        float*       o  = &out[(size_t)j * W];
        for (int i = 0; i < W; ++i)
            o[i] = W3 * (r0[i] + r1[i] + r2[i]);
    }
}

static void blur_twopass(const std::vector<float>& in, std::vector<float>& out,
                         std::vector<float>& tmp, int W, int H) {
    blur_x(in, tmp, W, H);
    blur_y(tmp, out, W, H);
}

// ---------- 版本 B：分块融合（chunked / fused） ----------
// 对应讲义 slide "Two-pass image blur, chunked (version 2)"：
// 只为 CHUNK+2 行输入生产中间结果，立刻消费掉，中间缓冲常驻 cache。
static void blur_chunked(const std::vector<float>& in, std::vector<float>& out,
                         int W, int H, int CHUNK) {
    const int stride  = W + 2;
    const int nchunks = (H + CHUNK - 1) / CHUNK;

    #pragma omp parallel
    {
        std::vector<float> tb((size_t)(CHUNK + 2) * W);   // 线程私有中间缓冲（关键：私有！）
        #pragma omp for schedule(dynamic, 1)
        for (int c = 0; c < nchunks; ++c) {
            const int j0    = c * CHUNK;
            const int rows  = std::min(CHUNK, H - j0);    // 本块要产出的输出行数
            const int trows = rows + 2;                   // 因此需要的中间行数

            // 生产：中间结果行 j0 .. j0+trows-1
            for (int jj = 0; jj < trows; ++jj) {
                const float* row = &in[(size_t)(j0 + jj) * stride];
                float*       o   = &tb[(size_t)jj * W];
                for (int i = 0; i < W; ++i)
```

```
                    o[i] = W3 * (row[i] + row[i + 1] + row[i + 2]);
            }
            // 消费：输出行 j0 .. j0+rows-1（此时中间结果还在 cache 里）
            for (int jj = 0; jj < rows; ++jj) {
                const float* r0 = &tb[(size_t)(jj + 0) * W];
                const float* r1 = &tb[(size_t)(jj + 1) * W];
                const float* r2 = &tb[(size_t)(jj + 2) * W];
                float*       o  = &out[(size_t)(j0 + jj) * W];
                for (int i = 0; i < W; ++i)
                    o[i] = W3 * (r0[i] + r1[i] + r2[i]);
            }
        }
    }
}

// ---------- 计时骨架：warmup + 多次重复 + 中位数 + 防死代码消除 ----------
static double now_s() {
    using clk = std::chrono::steady_clock;                      // 墙钟，非 CPU 时间
    return std::chrono::duration<double>(clk::now().time_since_epoch()).count();
}

int main(int argc, char** argv) {
    const int W   = (argc > 1) ? std::atoi(argv[1]) : 1024;
    const int H   = W;
    const int REP = (argc > 2) ? std::atoi(argv[2]) : 20;
    const int CHUNK = 16;                                       // 讲义示例用 CHUNK_SIZE = 16

    std::vector<float> in((size_t)(W + 2) * (H + 2));
    std::vector<float> tmp((size_t)(H + 2) * W);
    std::vector<float> out((size_t)W * H, 0.0f);
    for (size_t k = 0; k < in.size(); ++k)
        in[k] = (float)((k * 2654435761u) % 1000) * 0.001f;

    // 双精度累加校验和：既验证正确性，又阻止编译器删除整个计算（防 DCE）
    auto checksum = [](const std::vector<float>& v) {
        double s = 0.0;
        for (float x : v) s += (double)x;
        return s;
    };

    auto bench = [&](const char* name, auto&& fn) {
        fn();                                                   // warmup：页错误/首次触碰/TLB
        std::vector<double> t;
        for (int r = 0; r < REP; ++r) {
            const double t0 = now_s();
            fn();
            t.push_back(now_s() - t0);
        }
        std::sort(t.begin(), t.end());
        printf("%-14s median = %8.3f ms   min = %8.3f ms   max = %8.3f ms\n",
               name, t[REP / 2] * 1e3, t.front() * 1e3, t.back() * 1e3);
        return t[REP / 2];
    };

    printf("W=H=%d  threads=%d  CHUNK=%d\n", W, H, omp_get_max_threads(), CHUNK);
    const double t2p = bench("two-pass", [&]{ blur_twopass(in, out, tmp, W, H); });
    const double ck1 = checksum(out);
    out.assign((size_t)W * H, 0.0f);
    const double tch = bench("chunked",  [&]{ blur_chunked(in, out, W, H, CHUNK); });
    const double ck2 = checksum(out);

    // ---------- 把时间翻译成硬件语言 ----------
    const double pixels = (double)W * H;
    const double macs_ideal = 6.0 * pixels;                     // 讲义计数：6 × W × H (MAC)
    const double macs_chunk = 3.0 * (CHUNK + 2.0) / CHUNK * pixels
                            + 3.0 * pixels;                     // 讲义：((CHUNK+2)+CHUNK)/CHUNK × 3 × W × H

    // DRAM 流量模型 (B)：输入读一次 + 输出写一次 = 固有流量
    const double B_intrinsic = (double)(W + 2) * (H + 2) * 4 + pixels * 4;
    // 朴素两趟：中间缓冲被完整写出又被完整读回（远超 cache 容量）
    const double B_twopass  = B_intrinsic + 2.0 * (double)(H + 2) * W * 4;
```

```
    printf("\n[流量与算术强度]\n");
    printf("  intrinsic traffic = %.2f MB   two-pass traffic = %.2f MB (比值 %.2fx)\n",
           B_intrinsic * 1e-6, B_twopass * 1e-6, B_twopass / B_intrinsic);
    printf("  AI(two-pass) = %.3f FLOP/B   AI(chunked) = %.3f FLOP/B\n",
           2 * macs_ideal / B_twopass, 2 * macs_chunk / B_intrinsic);

    printf("\n[实测]\n");
    printf("  two-pass: %7.3f ms -> %7.2f GFLOP/s, %6.2f GB/s\n",
           t2p * 1e3, 2 * macs_ideal / t2p / 1e9, B_twopass / t2p / 1e9);
    printf("  chunked : %7.3f ms -> %7.2f GFLOP/s, %6.2f GB/s\n",
           tch * 1e3, 2 * macs_chunk / tch / 1e9, B_intrinsic / tch / 1e9);
    printf("  speedup(chunked / two-pass) = %.2fx\n", t2p / tch);
    printf("  checksum: %.3f vs %.3f  (diff = %.3e)\n", ck1, ck2, std::fabs(ck1 - ck2));
    return 0;
}
```

- **【代码做什么？】**
- 分配输入 `in`（`(W+2)×(H+2)`，比输出多一圈，避免内层循环做边界判断）、中间缓冲 `tmp`（`(H+2)×W`）、输出 `out`（`W×H`）。
- **版本 A（`blur_twopass`）**：先对**所有行**做水平 1×3 卷积写入 `tmp`，再对**所有输出行**做垂直 3×1 卷积写入 `out`。这是讲义 "Two-pass 3x3 blur" 的直接实现，中间缓冲大小 `W×(H+2)`。
- **版本 B（`blur_chunked`）**：把输出按 `CHUNK` 行分块。对每一块，只生产 `rows+2` 行中间结果（放在**线程私有**的 `tb` 里，大小只有 `(CHUNK+2)×W ≈ 74 KB`，可常驻 L2），立刻消费掉产出 `rows` 行输出。这是讲义 "chunked (version 2)"：**用分块把生产者–消费者局部性捕获在 cache 里**，"trends to ideal value as CHUNK is increased"。
- **计时骨架**：`steady_clock` 测**墙钟**；先 `warmup()` 一次（处理页错误、首次触碰、首次分配）；重复 `REP` 次，**打印中位数与实际 min/max**（min 反映"无干扰"性能，max 反映抖动，中位数反映典型值——三者都看，才知道测量是否可信）；每次迭代都对同一份输出做**双精度校验和**并打印，既验证两版结果一致，又**防止编译器把整个 kernel 删掉**。
- 最后把时间翻译成硬件语言：打印 MB、FLOP/B、GFLOP/s、GB/s 与两版加速比。
- **【并行机制与性能解说】**
- **线程与工作分配**：两个版本都用 `#pragma omp parallel for` 把**行（或块）**分配给线程。`blur_x/blur_y` 用 `schedule(static)`（每线程连续若干行，工作量均匀，无需动态调度开销，且保证了**行内连续访问**）；`blur_chunked` 用 `schedule(dynamic,1)` 是因为**块的成本虽相同但 cache 行为抖动大**，动态调度可以吸收抖动（代价是每次取任务一次原子操作，块数 `H/CHUNK = 64`，总开销可忽略）。
- **共享数据如何处理**：所有线程**只读** `in`、**只写**自己负责的 `out` 区域（行/块不重叠），因此**没有任何数据竞争**，也**不需要锁**。`blur_chunked` 的中间缓冲 `tb` 被声明在 `#pragma omp parallel` 区域内 ⇒ **每个线程一份私有副本**，这是该版本正确性的关键；若把它放到 parallel 区之外共享，就会产生数据竞争。
- **向量化**：内层 `for (i...)` 是对连续内存的无依赖循环，`-O3 -march=native` 会把它自动向量化为 8 宽 AVX2 的 `vfmadd` / `vaddps`。**注意这条循环的访存模式**：`out[i] = W3*(row[i]+row[i+1]+row[i+2])` 对固定行是连续访问 ⇒ 编译器能用 **packed load**（这正是 `thoughtprocess` 讲义 slide 10 强调的"交错分配 ⇒ 一条 `vmovaps`"）；而如果改成"每个线程负责一组**列**"，就会退化成 **gather**。
- **Work / Span / 并行度**：
  - **Work**：`W = Θ(W·H)` 次乘加（常数因子：两趟 6 MAC/像素，分块 `6(1+1/CHUNK)` MAC/像素）。1024×1024 时约 **6.29 MFLOP**（两趟）与 **6.68 MFLOP**（CHUNK=16）。
  - **Span**：关键是**这里有几个同步点**。`blur_x` 与 `blur_y` 之间**存在一个隐式的全局依赖**（垂直遍历需要所有行的水平结果），因此 naive 两趟版本的 Span 至少是"一趟的水平 Span + 一趟的垂直 Span"。单趟的 Span：每个线程处理 `rows ≈ H/P` 行、每行 `W` 次连续迭代，若强制线程内串行 ⇒ `Θ(H·W/P)`；再加上 `parallel for` 结束时的**屏障**（集中式 `Θ(P)`，树形 `Θ(log P)`）。所以 `Span_two_pass ≈ Θ(H·W/P + P)`
  - **并行度** = `Work/Span = Θ(W·H) / Θ(H·W/P + P)`。取 `W=H=1024, P=16`：Span 项 `1024×1024/16 = 65536`，`P = 16` ⇒ 并行度 ≈ `1.05e6/6.55e4 ≈ 16`——**与线程数同量级，说明这份实现的并行度刚好"够用但不富余"**；一旦 P 继续增大，`H·W/P` 变小而 `P`（屏障）不变甚至增大，**并行度不再增长**，这就是讲义 `thoughtprocess` slide 35 里 `T = N²/P + P` 那条曲线的来源（下一节会给出完整数值）。
  - **分块版本的 Span 更短**：融合后两趟之间的全局依赖被**打碎成逐块的局部依赖**，块之间彼此独立（`dynamic` 调度下甚至可乱序完成），Span ≈ `Θ((CHUNK+2)·W + P)`，与 `H` 无关。
- **瓶颈分析**：`W=H=1024` 时，按 20 GB/s 与 768 GFLOP/s 的机器模型：
  - 两趟：流量 `16.80 MB` ⇒ 带宽下界 `840 µs`；算力时间 `12.58 MFLOP/768 GFLOP/s ≈ 16 µs` ⇒ **带宽受限约 51 倍**。
  - 分块：流量回落到 `8.40 MB` ⇒ 带宽下界 `420 µs` ⇒ **预期加速比约 2.0×**（讲义对两趟实现的算术强度结论"**2x lower arithmetic intensity**"就是这一条）。

- **立刻可验证的诊断**：程序打印的 `GB/s` 若已接近本机实测的可持续带宽（用 STREAM 类内核测），说明带宽已饱和，**再加线程无用**；若远低于可持续带宽，则瓶颈在**延迟/访存并行度（MLP）或同步**，而不是带宽。用 `perf stat -e LLC-load-misses,stalled-cycles-backend` 可以区分这两者（见 §2.3 表）。

3.2 示例 2：用 PMU 计数器诊断"加线程反而更慢"——伪共享的真实共享（pthreads + OpenMP）

```
// false_sharing.cpp —— packed(伪共享) vs padded(独占 cache line) 两种计数器布局
//
// 编译(必须 release): g++ -O3 -march=native -pthread -fopenmp false_sharing.cpp -o fs
// 运行:               ./fs packed 4 16777216
//                     ./fs padded 4 16777216
// 剖析:               perf stat -e cycles,instructions,cache-misses,LLC-load-misses ./fs packed 4 16777216
//                     perf c2c record ./fs packed 4 16777216 && perf c2c report   # 直接指出伪共享行

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <thread>
#include <vector>
#include <atomic>
#include <chrono>
#include <algorithm>

struct alignas(64) PaddedCounter { volatile long long v; char pad[64 - sizeof(long long)]; };

static double now_s() {
    using clk = std::chrono::steady_clock;
    return std::chrono::duration<double>(clk::now().time_since_epoch()).count();
}

int main(int argc, char** argv) {
    const bool packed = (argc > 1) && (strcmp(argv[1], "packed") == 0);
    const int  T      = (argc > 2) ? std::atoi(argv[2]) : 4;
    const long long ITER = (argc > 3) ? std::atoll(argv[3]) : 16777216LL;

    // packed: T 个 long long 挤在 1-2 条 cache line 里（65 字节以下就落在同一行）
    // padded: 每个计数器独占 64 B
    std::vector<long long>  packed_cnt(T, 0);
    std::vector<PaddedCounter> pad_cnt(T);

    auto worker_packed = [&](int t) {
        volatile long long* p = &packed_cnt[t];
        for (long long i = 0; i < ITER; ++i) (*p)++;    // volatile 防止被优化成一次加法
    };
    auto worker_padded = [&](int t) {
        volatile long long* p = &pad_cnt[t].v;
        for (long long i = 0; i < ITER; ++i) (*p)++;
    };

    const double t0 = now_s();
    {
        std::vector<std::thread> th;
        for (int t = 0; t < T; ++t) {
            if (packed) th.emplace_back(worker_packed, t);   // 两个 lambda 类型不同，
            else        th.emplace_back(worker_padded, t);   // 不能写成三目表达式
        }
        for (auto& x : th) x.join();
    }
    const double dt = now_s() - t0;

    long long total = 0;
    for (int t = 0; t < T; ++t) total += packed ? packed_cnt[t] : pad_cnt[t].v;

    printf("%-7s threads=%2d  iter/thread=%lld  time = %8.3f s   aggregate = %8.2f Mops/s\n",
           packed ? "packed" : "padded", T, ITER, dt, (double)(T * ITER) / dt / 1e6);
    printf("total = %lld  (预期 %lld)\n", total, (long long)T * ITER);
    return 0;
}
```

- **【代码做什么？】**
- `T` 个线程各自执行 `ITER` 次自增；`packed` 模式下所有计数器是 `std::vector<long long>` 里的连续元素——**当 `T ≤ 8` 时它们全部落在同一条 64 B cache line 内**；`padded` 模式用 `alignas(64)` + 填充让每个计数器**独占一条 cache line**。
- 用 `volatile` 阻止编译器把循环优化成一条 `+= ITER`（这是这个微基准最关键的正确性细节）。
- 测墙钟、打印总吞吐量（Mops/s）与总和正确性校验。
- **【并行机制与性能解说】**
- **硬件层面发生了什么**：x86 的一致性协议（MSI/MESI/MOESI，本课程 Lecture 11–13 主题）以 **cache line（64 B）** 为**所有权粒度**。`packed` 模式下，线程 0 要写 `cnt[0]`，必须让包含该字节的整条 line 处于 **M（Modified，独占可写）** 态；此刻线程 1 想写 `cnt[1]`——**同一个 line 的另一个字节**——就必须先把 line 从线程 0 的 cache 中**无效化（I）**并搬到自己的 cache。于是每次自增都触发一次**line 迁移（cache-to-cache transfer + 失效 + 所有权串行化）**，代价约 **几十到约 100 ns**，并且**把两个线程彻底串行化**。
- **架构/机制图解（伪共享的 line 乒乓，硬件结构视角）**：

```
共享的一条 64 B cache line（4 个计数器 + 邻居数据）  ┌──────────────────────────────────────────────────────────┐
| cnt0 | cnt1 | cnt2 | cnt3 | (同一行的其它变量 …) |                                                          ▲ ▲
| 线程0 要写 cnt0 | 线程1 要写 cnt1 ┌──────┘ ① 请求独占(M) ┌──────┘ | Core 0 | ──────────────────────► | Core 1 |
| L1: M |  ◄── ② line 迁移 + Core1 变 I | L1: I | └──────┘ └──────┘ ③ Core1 再次请求 M ──► Core0 变 I ──► line 又搬回去 …… 循环往复
结果：每次自增 = 一次跨核 line 迁移（~ 数十至 100 ns） 吞吐量被钉在 ≈ 1 / 迁移延迟 ≈ 1e7 次/秒 量级，且 与线程数无关（不升反降）
修复后（padded）：每个计数器独占一行 ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
| cnt0 + 60B pad| | cnt1 + 60B pad| | cnt2 + 60B pad| | cnt3 + 60B pad| └──────────┘ └──────────┘ └──────────┘
└──────────┘ Core0: M Core1: M Core2: M Core3: M 各自命中自己的 L1 行，无一致性流量 ⇒ 吞吐量回到 ALU/内存极限（高 1~2 个数量级）
```

- **Work / Span / 并行度**：
  - **Work** `W = T × ITER` 次自增（本程序里每个自增是 1 次读-改-写，视为 1 单位工作）。
  - **Span**：`padded` 模式下各线程互不干扰，Span = 单个线程的工作 = `Θ(ITER)` ⇒ **并行度 = W/S = T**（完美的线性可扩展）。`packed` 模式下，**每次自增都可能是一次 line 迁移**，其延迟必须串行化，因此 `S_packed = Θ(T × ITER × L_transfer)`（`L_transfer ≈ 数十~100 ns`），**并行度 = W/S ≈ 1/L_transfer（即约 10^7 次/秒，与 T 无关）**——这就是"**并行度从 T 掉到 1 以下**"的严格含义：**加线程不会变快，只会让 line 搬家更频繁**。
- **数量级估算（用于理解，实测请运行代码）**：`T=4, ITER=$2^{24}$`（每线程 1677 万次）：
  - `packed`：总自增 `6.71×10⁷` 次，若每次平均 ~100 ns 的迁移延迟 ⇒ `≈ 6.7 s`；
  - `padded`：`6.71×10⁷` 次 / (4 线程 × 3.0 GHz × ~1 次/周期) ⇒ `≈ 5.6 ms`（外加每个计数器每 64 B 只有一份 line，无一致性流量）；
  - **比值约 10^2–10^3 量级**。注意实际数值强烈依赖微架构（有些实现会用"写独占但可延迟"的优化），**所以必须实测**——这正是本讲的立场。
- **瓶颈类型**：这不是带宽瓶颈（数据本来就在 cache 里，DRAM 字节数**很低**），而是**一致性/同步导致的串行化**瓶颈。**用计数器区分二者的方法**：伪共享的特征是 **`LLC-load-misses` 低但 `cache-misses`/一致性事务高、且时间随线程数增加而变差**；真实带宽瓶颈的特征是 **从内存控制器读到的字节数已接近机器上限**。`perf c2c`（cache-to-cache）子命令就是为此设计的：它直接给出**发生了 line 争用的地址**。
- **正确修法（比 padding 更好）**：**不要共享写，要"私有写 + 归并"**——这正是讲义 `thoughtprocess` slide 68→69 的教训：把"每个 (i,j) 迭代都加锁"改成"**先累加到线程本地 `myDiff`，每次迭代只加锁一次**"，原文标注 "Now only lock once per thread, not once per (i,j) loop iteration!"。在本例中对应 `#pragma omp parallel for reduction(+:total)` 或每线程局部累加后一次原子归并。**一般化的判据**：三个条件同时成立才需要动手——①**多个线程写**、②**这些写在同一条 line 上**、③**这些写频率很高**；只满足前两条（例如只在初始化/收尾各写一次）则成本可忽略。

3.3 示例 3：用设备侧计数器驱动优化（CUDA + Nsight Compute 反馈循环）

本示例把 `aiperfoptimization` 讲义中的"**profile → 反思 → 改代码 → 再 profile**"循环落到具体代码与指标上（讲义给出的反馈回路示例统计量是：`Timing: 32 ms`、`SM util 42%`、`DRAM util 89%`、`L2 cache hit rate 68%`）。

```
// saxpy_prof.cu —— 同一个 SAXPY 的两个版本：标量 + 网格跨步 vs float4 向量化 + __restrict__
//
// 编译: nvcc -O3 -arch=sm_80 -lineinfo saxpy_prof.cu -o saxpy
// 运行: ./saxpy 16777216 20
// 剖析(关键三件套: SM 吞吐、DRAM 吞吐、L2 命中率):
//   ncu --metrics sm__throughput.avg.pct_of_peak_sustained_elapsed,\
//dram__throughput.avg.pct_of_peak_sustained_elapsed,\
//lts__t_sector_hit_rate.pct  ./saxpy 16777216 1
// 也可以用: ncu --set full ./saxpy 16777216 1     (更慢，但一次给全)

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

// 版本 A: 一线程一元素（标量、4 B/线程/流）
__global__ void saxpy_scalar(int n, float a, const float* __restrict__ x, float* __restrict__ y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];
}

// 版本 B: 网格跨步 + float4（16 B/线程/流，一次事务取满 128 B sector）
__global__ void saxpy_vec4(int n4, float a, const float4* __restrict__ x, float4* __restrict__ y) {
    const int stride = gridDim.x * blockDim.x;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n4; i += stride) {
        float4 xv = x[i];
        float4 yv = y[i];
        yv.x = fmaf(a, xv.x, yv.x);
        yv.y = fmaf(a, xv.y, yv.y);
        yv.z = fmaf(a, xv.z, yv.z);
        yv.w = fmaf(a, xv.w, yv.w);
        y[i] = yv;
    }
}

static void check(cudaError_t e, const char* what) {
    if (e != cudaSuccess) { printf("CUDA error at %s: %s\n", what, cudaGetErrorString(e)); exit(1); }
}

int main(int argc, char** argv) {
    const int  n   = (argc > 1) ? atoi(argv[1]) : (1 << 24);
    const int  rep = (argc > 2) ? atoi(argv[2]) : 20;
    const float a  = 2.5f;
    const size_t bytes = (size_t)n * sizeof(float);

    float *x, *y;
    check(cudaMalloc(&x, bytes), "malloc x");
    check(cudaMalloc(&y, bytes), "malloc y");
    check(cudaMemset(y, 0, bytes), "memset y");
    check(cudaMemset(x, 1, bytes), "memset x");

    const int threads = 256;
    const int blocks  = (n + threads - 1) / threads;

    cudaEvent_t e0, e1;
    cudaEventCreate(&e0); cudaEventCreate(&e1);

    auto bench = [&](const char* name, void (*kern)(int, float, const float4*, float4*), bool vec,
                     int grid) {
        for (int r = 0; r < rep; ++r) {                      // 每轮重新计时，含 warmup 轮
            cudaEventRecord(e0);
            if (vec) kern(n / 4, a, (const float4*)x, (float4*)y);
            else     saxpy_scalar(n, a, x, y);
            cudaEventRecord(e1);
            cudaEventSynchronize(e1);
            float ms = 0.f; cudaEventElapsedTime(&ms, e0, e1);
            if (r == rep - 1)
                printf("%-8s grid=%d  time = %8.3f ms   traffic = %6.1f MB   %7.1f GB/s\n",
                       name, grid, ms, 3.0 * bytes / 1e6, 3.0 * bytes / (ms * 1e-3) / 1e9);
        }
    };
```

```
    // 版本 A: 一次性大 grid (一线程一元素)
    bench("scalar", nullptr, false, blocks);
    // 版本 B: grid-stride + float4, grid 选成"足够覆盖设备"的固定值
    bench("vec4",   saxpy_vec4, true, 1024);

    check(cudaGetLastError(), "launch");
    cudaFree(x); cudaFree(y);
    return 0;
}
```

- **【代码做什么？】** 两个 kernel 计算同一个 `y = a·x + y`（SAXPY，每个元素 **1 次 FMA = 2 FLOP**）：版本 A 每个 CUDA 线程处理 1 个元素、每线程每流只搬 4 B；版本 B 用 **grid-stride 循环**（固定 grid，让每个线程处理多个元素，便于覆盖不同设备规模）+ **`float4` 向量化访存**（每线程每流 16 B，正好凑满 128 B 的一次内存事务/扇区），并用 `__restrict__` 告诉编译器 x/y 不重叠以允许更激进的访存调度。计时用 **CUDA event**（`cudaEventElapsedTime`），它测的是**设备上 kernel 的实际执行时间**，不受主机侧启动开销与异步性影响。
- **【并行机制与性能解说】**
- **硬件执行模型**：`threads=256` 定义 **block（线程块）**；每个 block 被整体调度到**一个 SM（Streaming Multiprocessor）**上，block 内的线程以 **warp（32 个线程）** 为调度单位执行 SIMT。版本 A 的 `if (i < n)` 只对**最后不满一个 warp 的边界**产生分支发散，主体部分无发散。版本 B 的 `float4` 让每个线程在一条指令里处理 4 个元素，**指令数降到 1/4**，同时把每线程每流的字节数从 4 B 提高到 16 B ⇒ **更少的 warp 就能喂满内存系统**（提高 MLP / memory-level parallelism）。
- **Work / Span / 并行度**：`N = 2²⁴ = 1.68×10⁷` 个元素。
  - **Work** `W = N` 次 FMA（`= 2N` FLOP `= 33.6 MFLOP`）。
  - **Span**：把执行抽象成"一次访存延迟 + 一次 FMA"的链，整个 kernel 的 Span 取决于**同时驻留的线程数** `T_res`：`Span ≈ θ(N / T_res)`。取 80 个 SM × 每 SM 2048 线程 = `T_res = 163,840`，则 `Span ≈ 102` 个"批次"。
  - **并行度** `= W/Span ≈ T_res = 1.6×10⁵`。**结论极其重要：并行度对这块 GPU 而言绰绰有余（比可驻留线程数高两个数量级余量），所以瓶颈绝不在"并行度不够"，而在内存系统**。这正是"先算 work-span 再动手"的价值：它把"要不要加并行度"这个问题**直接排除**了，使你把注意力放到 `float4` / 访存事务 / 占用率上。
- **瓶颈分析（流量下界）**：SAXPY 读 x、读 y、写 y ⇒ `3N×4 B = 192 MiB = 201 MB`。若该 GPU 的可持续带宽为 `1.4 TB/s`，则**时间下界 = 201 MB / 1.4 TB/s ≈ 144 μs**。若实测版本 A 只跑出 ~45% DRAM 利用率（≈ 0.63 TB/s ⇒ ~320 μs），而版本 B 达到 ~90%（≈ 1.26 TB/s ⇒ ~160 μs），则**加速比约 2×，且完全由"每个线程请求的字节数/事务效率"解释**，与算力无关（算力时间 `33.6 MFLOP / 19.5 TFLOP/s ≈ 1.7 μs`，比内存时间小两个数量级）。**算术强度 = 2 FLOP / 12 B = 0.167 FLOP/B**，在该 GPU 的拐点（`19.5e12 / 1.4e12 ≈ 14 FLOP/B`）左侧**约 84 倍**，是彻头彻尾的带宽受限内核。
- **profiling 反馈循环（本示例的"元"信息，直接来自讲义 slide 45–46 的范式）**：讲义的循环是——**运行并剖析 → 得到统计量（`Timing` / `SM util` / `DRAM util` / `L2 hit rate`）→ 让优化者（人或 LLM）"反思什么在拖慢程序" → 据此做一处修改 → 再跑再测**。把它套到本示例上就是一条**可复现的诊断链**：
  1. `SM util 42%` + `DRAM util 89%` ⇒ **不是算力不够，是访存已经打满**——在这种情况下**任何提高 ILP、增加算力的修改都不会有效**，唯一有效的方向是**减少字节数或提高事务效率**（`float4`、`__ldg`、融合、降低精度）。
  2. `L2 cache hit rate 68%` 提示还有约 1/3 的 L2 请求下行到 DRAM；如果这是**重复访问**造成的（如分块/多趟算法里的中间结果），则**融合（fusion）或分块（tiling）**是正确手段（对应 §3.1 的 chunked 版本）。
  3. 若 `SM util` 高而 `DRAM util` 低 ⇒ 反过来，**计算/指令受限**（分支发散、特殊函数单元、共享内存 bank 冲突），此时"减少访存"只会浪费精力。
- **与自动搜索/自动调优的接口**：讲义指出，把优化目标写成"**测量量**"之后，优化本身就可以被**搜索**：Halide 自动调度器把"调度空间"枚举出来，用一个小型 MLP 代价模型（输出 27 个系数供手写代价模型使用）在 **166 秒内评估 140 万个 schedule**，其质量已"compares to best known human schedules"；而人类专家要达到同等水平通常要花 **10–50 分钟**（讲义 slide 43 的时间—吞吐曲线）。**这说明：可信的、快速的性能测量是自动优化的前置条件**——没有它，搜索就没有目标函数。

---

4. 性能模型与复杂度分析

本节的机器模型（贯穿全节）：**16 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA）= 768 GFLOP/s（fp32）；可持续 DRAM 带宽 20 GB/s；DRAM 延迟 ~100 ns；L1 64 B/cycle；64 B cache line。** 拐点 `P_peak/BW = 768/20 = 38.4 FLOP/Byte`。

4.1 算例 A：算术强度与 Roofline 的完整数值推理（3×3 box blur）

假设参数：`W = H = 1024`（1,048,576 像素）；float 4 B。

量	二维（不可分离）	两趟分离（朴素）	两趟分离（分块融合 CHUNK=16）
每像素 MAC（讲义计数）	9	6	`6×(1+1/16) = 6.375`
总 MAC（= Work）	9.44 M	6.29 M	6.69 M
总 FLOP（×2）	18.87 M	12.58 M	13.37 M
DRAM 流量	输入 4.21 MB + 输出 4.19 MB = **8.40 MB**	8.40 + 中间写 4.20 + 中间读 4.20 = **16.80 MB**	8.40 MB（中间结果常驻 cache）
算术强度 AI	18.87/8.40 = **2.25**	12.58/16.80 = **0.75**	13.37/8.40 = **1.59**
带宽下界时间	8.40 MB / 20 GB/s = **0.42 ms**	16.80/20 = **0.84 ms**	8.40/20 = **0.42 ms**
算力时间（768 GFLOP/s）	18.87/768 = **24.6 µs**	12.58/768 = **16.4 µs**	13.37/768 = **17.4 µs**
受限类型	带宽受限 17×	带宽受限 **51×**	带宽受限 24×
Roofline 可达 = min(768, AI×20)	**45.0 GFLOP/s**	**15.0 GFLOP/s**	**31.8 GFLOP/s**
占峰值比例	5.9%	**1.95%**	4.1%

结论与可执行判据：1. 三个版本全都**落在 Roofline 斜段**（AI 远小于 38.4），所以**唯一的有效优化方向是"减流量"**：把两趟变成分块融合，流量从 16.80 MB 降到 8.40 MB ⇒ **理论加速比 2.0×**，这与讲义"两趟实现算术强度低 2 倍（2x lower arithmetic intensity）"的结论完全一致。 2. **二维（不可分离）版本的 AI 是两趟朴素版的 3 倍**（2.25 vs 0.75）——虽然它做了更多算术（9 MAC vs 6 MAC），但因为**不产生中间流量**，在带宽受限的机器上反而可能更快。**这是"更多计算、更少搬运可以更快"的经典反直觉结论**，也正是 Roofline 要教的判断方式（讲义原文即谓两趟是"loads/stores to tmp_buf are overhead — this memory traffic is an artifact of the two-pass implementation: it is not intrinsic to computation"）。 3. **"固有流量"（intrinsic）的界定**：讲义明确指出，blur 的**固有带宽需求**就是"读一遍输入、写一遍输出"。任何超出这一点的流量都是**实现的人工产物（artifact）**，是优化的首要目标。分块/融合（`CHUNK` 增大）让实际流量**单调趋向固有流量**（`(CHUNK+2)/CHUNK → 1`）。 4. **验证方式**：算出的 20 GB/s 是否可信？用程序打印的 GB/s 与 `perf stat` 的 IMC/uncore 字节数交叉验证；用 STREAM 类内核实测本机可持续带宽作为分母。若实测 GB/s 已达可持续带宽的 90%+，则本分析闭合；若只有 30%，说明还有**延迟/MLP/同步**问题没被这个带宽模型解释，需要回到 §2.5 的高水位实验。

4.2 算例 B：`T = 2N²/P + P` ——并行度、屏障与"加核到什么时候停止"

来自 CS149 `thoughtprocess` 讲义的两阶段图像处理例子：步骤 1（亮度 ×2，可完全并行）、步骤 2（求全图平均，需要**部分和 + 合并**）。串行时间 `≈ 2N²`；并行实现的时间是"每个处理器 N^2/P（步骤 1）+ 每个处理器 N^2/P（步骤 2 的部分和）+ P（串行合并部分和）"：

```
T(P) = 2N²/P + P
S(P) = 2N² / T(P) = P / (1 + P²/(2N²))
E(P) = S(P)/P = 1 / (1 + P²/(2N²))
```

取 `N = 1024`（`N² = 1,048,576`）：

P	`P²/(2N²)`	`S(P)`	`E(P)`
1	4.77e−7	1.000	100%
16	1.22e−4	**15.998**	99.99%
256	0.03125	248.2	96.9%
1024	0.5	682.7	66.7%
1448 = √2·N	1.0	**724.0**	**50%**
2048	2.0	682.7	33.3%

解读：`S(P)` 在 `P = √2·N ≈ 1448` 处取**极大值 724**，之后**随 P 增加而下降**。这条曲线同时包含了两种效应的竞争：分子上的 `P` 来自"可并行部分随 P 缩短"，分母里的 `P²` 来自"**串行合并/屏障的绝对成本随 P 线性增长**"（`P` 项）。**在 16 核机器上（P=16），效率 99.99%，并行几乎免费；但把同一份代码搬到 2048 核上，效率掉到 33%，加速比回落到与 1024 核相同的 682.7×、并低于峰值 724×——即"加了一倍的核，一点没变**

快"。 这与讲义 slide 35 的原话一致：**"speedup → P when N >> P"**，以及"并行算法的开销：合并部分和"。它是 **Amdahl 定律的一个具体化**——只不过这里的"串行部分"随 P 增长（屏障/归并/同步），而 Amdahl 的经典形式把它当作常数。

与 Amdahl 的对照算例（`f` = 恒定串行比例)： – `f = 1%`，`P = 64`：`S = 1/(0.01 + 0.99/64) = 1/0.025469 = 39.26`，效率 `61.3%`；**用 Karp–Flatt 度量反推串行比例**：`e = (1/S − 1/P)/(1 − 1/P) = (0.025469 − 0.015625)/0.984375 = 1.00%` ✔（自洽）。 – **Summit 超算的极端算例**（讲义 slide 37）：`27,648` 块 GPU × `5,376` ALU/GPU = `148,635,648` 个 ALU，即机器可**同时执行 1.486 亿次单精度运算**。若应用有 **0.1% 串行**，则 `S_max = 1/0.001 = 1000`——**这台 1.5 亿 ALU 的机器最多只能带来 1000 倍加速**。把它换算成"机器白买了多少"：`T(P) ≈ 0.001·T(1)`，这段时间内可提供的 ALU–时间容量为 `P·T(P) = 1.486e8 × 0.001·T(1) ≈ 1.486e5·T(1)`，而真正有用的工作量只有 `T(1)`（其中 `0.999·T(1)` 被摊在 P 个 ALU 上、`0.001·T(1)` 只由 1 个 ALU 完成），因此**按时间平均的算力利用率仅约 `1/1.486e5 ≈ 6.7×10⁻⁶`（0.0007%），99.9993% 的 ALU 时间容量被浪费**。这就是"**小的串行段会限制大规模并行机器上的加速比**"的定量含义：**瓶颈不是机器不够大，而是串行段让机器无事可做。** – **Gustafson 对照**：同一 `f = 0.01`、`P = 64`，若**问题规模随机器放大**（弱扩展），`S_scaled = P − f(P−1) = 64 − 0.63 = 63.37`，效率 `99.0%`。**同一个 `f`，强扩展只有 61% 效率、弱扩展有 99% 效率**——这解释了为什么 HPC 社区的实测加速比通常远好于 Amdahl 强扩展预测。

4.3 算例 C：带宽、延迟与 Little's Law ——"我需要多少并发才能打满带宽"

公式：`需要的在飞字节数（bytes in flight）= 带宽 × 延迟`。代入 `BW = 20 GB/s`、`L = 100 ns`：

```
20e9 B/s × 100e-9 s = 2000 B  ≈ 31 条 64 B cache line  ≈ 16 条 128 B line
```

含义：要让 DRAM 带宽饱和，内存系统里必须**同时有约 31 条 cache line 的请求在飞**。若每个线程只有 1 个未完成的缺失请求（MLP = 1），你至少需要 **31 个活跃线程**；如果每线程 MLP = 4（例如 `float4` 或循环展开后 4 个独立缺失），需要 **约 8 个线程**。这解释了 §3.3 中 GPU 版本 B 的行为：`float4` 把每线程的字节数从 4 B 提到 16 B，等价于把"每事务有效载荷"提高 4 倍，从而**用更少的 warp 就能达到同样的在飞字节数**。

与"延迟 vs 带宽"的判据对应： – 实测带宽 ≪ 机器可持续带宽，且**增加并发（线程/SIMD 宽度/展开）能线性提速** ⇒ 你受限于**延迟/MLP**，解决办法是**更多并发**（预取、展开、软件流水）。 – 实测带宽 ≈ 机器可持续带宽，增加并发**不再提速** ⇒ 你受限于**带宽**，唯一出路是**减少流量**（分块、融合、降低精度、压缩）。 – **这两者的区分是整个性能分析中最重要的一次分岔**，而它必须用测量（计数器字节数 + 并发扫描）而不是直觉来判定。

4.4 算例 D：测量本身的不确定度 ——"你的 3% 提升是真的吗"

设某内核单次运行时间服从均值 `μ = 3.20 s`、标准差 `σ = 0.40 s`（`σ/μ = 12.5%`，典型的多线程抖动水平）。均值估计的标准误差为 `SE = σ/√n`：

运行次数 n	SE	SE/μ
1	0.400 s	12.5%
10	0.126 s	3.95%
100	0.040 s	1.25%
156	0.032 s	**1.0%**
1600	0.010 s	0.31%

结论： 1. **要检测 1% 的性能差异，需要约 156 次重复运行**（`n = (σ/(目标SE))² = (12.5/1)² ≈ 156`）。**跑一次就宣布"我优化了 3%"是不成立的**——单次测量的 1σ 不确定度就是 12.5%。 2. **应当报告中位数 + min + max（或分位数），而不是均值**：多线程计时分布**右偏**（偶发的调度/争用事件拖长尾巴），均值会被离群值拉偏，而 **min 最接近"无干扰性能"**。这正对应 §3.1 代码打印三个统计量的原因。 3. **采样剖析的误差有同样的性质**：§2.3 的表格给出 `±1 个百分点 → 9604 个样本 → 1 kHz 下约 9.6 s`。**"这个热点占 X%" 必须附上样本数**，否则无法判断结论是否显著。 4. **测量误差的系统性来源必须逐条排除**：计时器分辨率与开销、被优化掉的死代码（DCE）、CPU 频率/睿频波动（用 `cycles/time` 反算实际频率）、首次触碰的页错误与 NUMA first-touch、冷 cache（是否 warmup）、超线程共享执行单元导致的干扰、以及**同机其它进程的噪声邻居**。

4.5 模型选择速查表：三个上界各管什么

模型	公式	回答的问题	输入需要什么	失效/注意
Work–Span（DAG）	`T(P) ≤ W/P + S`；并行度 `= W/S`	**并行度够不够？** 加核还有用吗?	算法的 W 与最长依赖链 S	假设理想调度器、零通信成本；对存储层次一无所知
Amdahl / Gustafson	`S ≤ 1/(f+(1−f)/P)`；`S_scaled = P − f(P−1)`	**串行部分封顶多少？** 强扩展还是弱扩展?	串行比例 f（需实测或剖析得到）	f 恒定是理想化；实际 f 常随 P 增大（屏障、归并）

模型	公式	回答的问题	输入需要什么	失效/注意
算术强度 / Roofline	`可达 = min(P_peak, AI × BW)`	**该减流量还是该减计算?**	峰值算力、可持续带宽、每次运行的字节数	上界乐观；不区分延迟与带宽；需实测带宽校准
带宽–延迟 / Little's Law	`在飞字节 = BW × 延迟`	**并发够不够打满带宽?** 是延迟界还是带宽界?	延迟、带宽、每线程 MLP	必须用并发扫描（线程数/展开度）验证
高水位（high watermark）	四个激发实验（加算、改 `A[0]`、去同步、去算）	**主导成本到底是 compute / memory / sync?**	只需可运行的代码	三者几乎不完美重叠，结论是"敏感度"而非"判决"

4.6 算例 E：把 profiler 数字翻译成结论（综合演练）

给定一份 profiler 报告（形式与讲义 `aiperfoptimization` slide 46 的反馈循环一致；此处为**用于演示诊断流程的示例数字**，并非讲义原文数据）：

```
Timing: 32 ms          SM util: 42%          DRAM util: 89%          L2 hit rate: 68%
```

推理链： 1. **`DRAM util 89%` 是第一优先信息**：访存子系统几乎打满，说明程序**已经用掉了近 9 成的可用带宽**。任何"提高算力利用率 / 增大 ILP / 换更快指令"的修改都**不可能**带来大收益——因为瓶颈不在计算侧。 2. **`SM util 42%` 与上一条互证**：SM 利用率不高不是"算力浪费"，而是**在执行单元等待数据**（访存受限时的典型形态）。**不要**去追这 42%。 3. **`L2 hit rate 68%` 指出改进方向**：有约 1/3 的 L2 请求下行到 DRAM。若这些下行是**重复访问**造成的（多趟/多内核之间的中间结果、未复用的邻域），则**融合或分块**能直接减少下行量（正是 §3.1 chunked 的手法，收益约 2×）；若是**一次性流式访问**（每个字节只读一次），则下行无法避免，改进只能来自**降低字节需求本身**（更窄的数据类型、只读必要字段、压缩）。 4. **给出上界估算**：若把 DRAM 流量减半而其它不变，时间从 32 ms 降到约 `32×(0.11 + 0.89/2) = 17.8 ms`（把 11% 视为与 DRAM 无关的、不可压缩的部分）。**这个 1.8× 就是"减流量"路线的 ceiling**；如果业务要求 5×，那么**必须在算法层面减少工作或减少精度**，而不是继续在 kernel 里微调。 5. **反过来验证**：跑一个高水位实验——"把大部分计算删掉但保留全部访存"。若时间几乎不变（例如 32 ms → 29 ms），则**访存主导成立**；若掉到 12 ms，则之前对 `SM util 42%` 的解读就有问题，需要重新用 stall 分解（`stalled-cycles-frontend/backend`）检查前端/后端停顿的来源。**模型给出假设，激发实验负责证伪。**

---

5. 关键要点

1. **测量优先，且测量必须可信。** 讲义的原话是"**永远、永远、永远先写最简单的并行版本，然后测量性能，看看你站在哪里**"。可信的测量意味着：用**墙钟**（不是 `clock()`）、做 **warmup**、**重复多次**并报告中位数与 min/max（检测 1% 差异需约 156 次运行）、**防止死代码消除**、明确**基线**（与最优串行实现比，而不是与并行算法的单核运行比）。测不准的优化等于随机游走。
2. **性能问题只有四类来源，先分类再优化：计算、带宽/延迟、同步、并行度。** 用三个工具分诊：**work–span**（`W/S` 够不够 → 加核有用吗）、**算术强度 / Roofline**（`AI` vs `P_peak/BW` → 该减流量还是减计算）、**PMU 计数器 + 高水位实验**（IPC、cache 命中率、DRAM 字节数、stall 分解 → 延迟界还是带宽界、是不是同步主导）。**分类错了，优化就是白工**（例：在 `DRAM util 89%` 的内核上调 ILP）。
3. **算术强度是并行性能的单一最重要定量指标，Roofline 是它的可视化。** `可达性能 = min(P_peak, AI × BW)`，拐点 `= P_peak/BW`。本笔记的 **CPU 机器模型**下拐点为 `38.4 FLOP/B`，而 blur 的 AI 只有 0.75–2.25 ⇒ **只能拿到峰值的 1.95%–5.9%**；SAXPY 更低（AI = 0.167）——即便换到 **GPU 机器模型**（19.5 TFLOP/s ÷ 1.4 TB/s ⇒ 拐点约 14 FLOP/B），它同样低两个数量级。**跨越拐点的唯一途径是数据复用（分块、融合、缓存阻塞）或降低每字节的算术需求**；在拐点左侧做的所有算力优化都收益接近于零。
4. **"实现"（implementation）而不是"抽象"（abstraction）决定性能，所以要会掀开后厨。** ISPC 的同一个 `foreach` 抽象，交错分配编译成**一条 packed load（`vmovaps`）**、分块分配退化成**昂贵的 gather（`vgatherdps`）**；同一份 `A[i,j]` 更新代码，把锁从"每元素一次"改成"每线程每迭代一次"就差一个数量级（讲义 slide 68→69）。**性能分析必须一直问到"硬件上实际发生了什么"这一层**（看汇编、看计数器、看访存模式），否则你优化的是心里的模型，不是机器上的程序。
5. **测量与建模的终极目的是"自动化"——而自动化质量的上限由测量的可信度决定。** 把性能写成可测量的目标函数之后，优化就可以被**搜索**：Halide 自动调度器用小型 MLP 代价模型在 166 秒内评估 140 万个调度，质量已与最佳人工调度相当（人工通常要 10–50 分钟一个）；基于 profiling 统计量的"反思—修改—再测量"循环（包括 LLM agent）也是同一范式。**但所有这些方法的输入都是 profiler 的四个数字**——测量不可信，搜索就会朝错误方向收敛。**"建立高水位、分清瓶颈"这项人类技能，在自动优化时代不是被取代，而是变成自动优化的地基。**

---

6. 常见陷阱与注意事项

- **测错了"时间"。** 用 `clock()` / `getrusage` 的 CPU 时间与墙钟时间算加速比，在多线程下会得出错误甚至负的加速比（8 线程时 CPU 时间约为墙钟的 8 倍）。**并行加速比只能用墙钟时间。** 另外要留意：`omp_get_wtime` 不保证全局同步时钟，跨节点计时要用 `MPI_Wtime`。
- **单次测量下结论。** 多线程程序的计时分布右偏、抖动可达 10% 以上；单次运行的 1σ 不确定度可能就是 12.5%（§4.4）。**必须 warmup + 重复 + 报告中位数与 min/max**；检测 1% 的差异需要约 156 次运行。同理，**采样剖析的百分比必须附样本数**：0.3% 的热点在 1000 个样本里只有 3 个样本，泊松相对误差约 58%，这个结论不可用。
- **被测代码被优化掉（DCE）。** `-O3` 下未被使用的计算结果会被整段删除，于是你"测"出一个 0 ms 的内核。必须**消费结果**（打印校验和、写 `volatile`、内联汇编屏障）。反过来的错误同样常见：为了防 DCE 而加 `volatile` 到**热循环内的每一次数组访问**上，会把向量化和重排全部禁掉，测出的不是原程序的性能。
- **基线选错、问题规模选错。** 讲义明确点名的两个陷阱：①**"把并行程序的加速比对比于并行算法在单核上运行"**（原文：*easier to make yourself look good*）——并行算法往往做更多总工作（如红黑着色比 Gauss–Seidel 收敛更慢），这样做会凭空造出加速比；②**固定问题规模**——32 处理器上跑 258×258 网格得到 **"No benefit! (slight slowdown)"**，而 1K×1K 网格就正常扩展，因为小问题**对机器来说太小、通信计算比太高。报告加速比时必须写清基线、问题规模、线程数、以及是强扩展还是弱扩展。**
- **把伪共享/同步问题误判成带宽问题（或反之）。** 两者症状相似（加线程不变快甚至变慢），但证据完全不同：**伪共享/一致性争用的特征是 DRAM 字节数低而一致性事务/line 迁移多、时间随线程数恶化；真实带宽瓶颈的特征是从内存控制器读到的字节数已逼近机器上限。**用 `perf stat -e LLC-load-misses,...` 与 `perf c2c report`（直接指出争用的地址）区分，而不是凭"感觉像内存问题"。另外记住修伪共享的正确方向是**把"共享写"改成"私有写 + 归并"**（`reduction`），padding 只是次优解。
- **忽略"测量者效应"与硬件状态。** 插桩剖析会改变被测程序（`mcount` 让小函数时间虚高、不支持共享库语义）；PMU 事件数超过物理计数器时会**多路复用外推**（`perf stat` 的 `[xx%]` 列必须看）；频率/睿频、SMT 共享执行单元、NUMA first-touch、冷 cache、页错误、噪声邻居都会污染结果。**跨机器比较原始计数没有意义，只能比较比值**（IPC、命中率、字节/秒、% of peak）。
- **只相信一个模型，不做激发实验。** Roofline 上界乐观、Amdahl 的 `f` 恒定是理想化、work-span 假设零通信。**讲义的原话是"计算、访存与同步几乎从不完美重叠，因此总体性能很少完全由其中之一决定"**。正确姿势是：**模型提出假设 → 高水位实验（加算 / 全改 `A[0]` / 去锁 / 去算保访存）负责证伪 → 计数器给出机制级证据**。三者不一致时，以实验为准，并去解释模型为何失效。

---

7. 思考题（带答案）

问题 1（测量方法论）：某同学在 16 核机器上实现了一个 5 点模板计算，报告"8 线程比 1 线程快 13.8×，因此我的并行实现是超线性的"。他的实验方法是：单线程版本用 `clock()` 计时（因为"CPU 时间更准"），多线程版本用 `omp_get_wtime()` 计时；每个配置只跑一次；单线程版本不 warmup，多线程版本 warmup 一次。请指出这份报告中的全部方法论问题，说明每个问题会**朝哪个方向**污染结论，并给出正确的实验设计。

【答案】 问题与污染方向： 1. **两个版本用了不同的计时域**（`clock()` 是 CPU 时间，`omp_get_wtime()` 是墙钟）。`clock()` 在**单线程**下与墙钟接近（差别是系统调用/I/O），所以单线程基线"看起来"问题不大；但一旦代码在单线程下也用了任何库内线程（如 BLAS 或 OpenMP runtime 的 helper），`clock()` 会**高估**基线时间，从而**虚增**加速比。更严重的是，两个域不可比 ⇒ 整个加速比在量纲上就不成立。**方向：可能虚增。** 正确做法：**两个版本都用同一个墙钟计时器**（`omp_get_wtime` / `steady_clock`）。 2. **只跑一次。**多线程计时分布右偏、抖动可达 10%+（§4.4）。单次测量的 1σ 不确定度约 12.5%。要检测 1% 的差异需约 156 次重复。**方向：结论完全不可信，可能虚增也可能虚减，量级上足以解释 13.8× 与 8× 之间的全部差距。** 正确做法：**warmup + 至少数十次重复 + 报告中位数与 min/max。** 3. **warmup 不对称。**单线程不 warmup 会包含**首次分配页错误、first-touch NUMA 放置、冷 cache、首次 JIT/动态库加载**等一次性成本，从而**高估**单线程时间、**虚增**加速比。多线程 warmup 一次也不够（第一次并行区会包含线程创建与栈分配）。**方向：虚增。** 正确做法：**两个版本同样 warmup，并把 warmup 轮从统计中剔除。** 4. **超过核数的"加速比"应当首先被怀疑是硬件状态差异，而不是算法优越。**16 核上正确实现的 5 点模板的强扩展上限就是 16（且实际会因带宽受限远低于 16）。13.8× 本身没超过 16，所以**不是数学上不可能**，但在**内存带宽受限**的模板计算上达到 86% 效率需要额外解释。**必须提供的证据**：①实测 GB/s 是否逼近机器可持续带宽（若是，说明 8 线程已打满带宽，1 线程只跑出 1/8 带宽是完全可能的，此时 8× 就是上限而 13.8× 需要解释）；②`perf stat` 的 IPC 与 LLC 命中率在两种配置下是否发生了**质变**（例如单线程工作集超出 LLC 而多线程各自分块后命中 LLC ⇒ 这才是真正的"超线性"，且属于**工作集变小**的效应，而不是并行本身带来的）。 5. **缺少基线说明**：加速比的基线必须是"**最优串行实现**"（讲义原文的 pitfall），且要说明是强扩展还是弱扩展。 正确的实验设计：(a) 统一墙钟计时器；(b) 两版本对称 warmup；(c) 每个配置 ⩾50–200 次重复，报告中位数/min/max/标准差；(d) 线程数扫描 1,2,4,8,16 并画出**实测加速比 vs Amdahl 上界**及**实测 GB/s**两条曲线；(e) 用 `perf stat` 记录两配置的 `cycles,instructions,LLC-load-misses` 与实测频率；(f) 用 STREAM 类内核测出本机可持续带宽作为分母；(g) 报告基线、问题规模、是否强扩展。

问题 2（Roofline 应用与决策）：机器参数：16 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA）= 768 GFLOP/s，可持续 DRAM 带宽 20 GB/s。你的内核每次处理一个 1024×1024 的 float 图像（4 MB），当前实现是"两趟"：第一趟读输入、写中间缓冲（4 MB）；第二趟读中间缓冲、写输出（4 MB）。请回答：(a) 算术强度是多少（内核每像素做 6 次乘加）？(b) 带宽下界时间是多少？(c) 若 profiler 显示当前耗时 3.4 ms，你的第一嫌疑是

什么？(d) 把两趟融合成分块实现后流量降到 8.4 MB，预期时间与加速比是多少？(e) 若融合后实测只到 1.1 ms 而非预期值，你会用哪些测量手段继续定位？

【答案】 (a) **流量**：`4.21 MB（读输入，含 (W+2)×(H+2) 的边界）+ 4.20 MB（写中间）+ 4.20 MB（读中间）+ 4.19 MB（写输出）= 16.80 MB`。**FLOP**：6 次乘加/像素 × 2 = 12 FLOP/像素 × 1.048576e6 像素 = **12.58 MFLOP**。**AI = 12.58/16.80 ≈ 0.75 FLOP/Byte**（远低于拐点 38.4 ⇒ 带宽受限，理论上限 `0.75 × 20 = 15.0 GFLOP/s`，仅峰值的 **1.95%**）。(b) **带宽下界** = `16.80 MB / 20 GB/s = 0.84 ms`。**算力时间** = `12.58 MFLOP / 768 GFLOP/s = 16.4 μs`，比内存时间小 **51 倍** ⇒ 纯带宽/算力模型下这是内存受限内核。(c) **实测 3.4 ms 是带宽下界的 4.05 倍** ⇒ 第一嫌疑**不是**"算力不够"（那只会让它更快），而是**实际可达带宽远低于 20 GB/s**，原因按概率排序：①**访存模式/MLP 不足**（每线程未完成缺失太少，受延迟而非带宽限制；见 §4.3：打满 20 GB/s 需要约 2000 B 即 ~31 条 cache line 在飞）；②**中间缓冲的实际 DRAM 往返比模型更多**（写分配导致"写"也要先读一次：若 tmp 写是 write-allocate 且未用非临时写，则额外多出 4.20 MB 读，总流量升到 21 MB ⇒ 下界 1.05 ms，仍不足以解释 3.4 ms）；③**同步/屏障或负载不均**（`parallel for` 每趟一次屏障，若线程数少或行数分配不均，这部分不可压缩）；④**多线程未生效或超线程争用**（若有效线程数实为 4，则带宽下界 3.36 ms 恰好与 3.4 ms 吻合——**这是一个必须首先排除的假设**）。(d) 融合后流量 `8.40 MB` ⇒ 下界 `0.42 ms`；相对当前实现的**理论上限加速比 = 16.80/8.40 = 2.0×**，即理论时间从 0.84 ms 降到 0.42 ms。**若当前实现已达其自身的带宽下界（0.84 ms），融合会把时间也降到 0.42 ms（2×）**；但你实测是 3.4 ms，所以**融合本身不会自动带来 8×**——它只把"流量"这一项减半，而当前的主要损失在流量之外。(e) 融合后实测 1.1 ms（比下界 0.42 ms 差 2.6 倍）时的定位手段： 1. **先确认有效并行度**：打印 `omp_get_num_threads()` 与 `OMP_NUM_THREADS`，并用 `perf stat -e task-clock` / `/proc/<pid>/status` 核对实际使用的 CPU 数；排除"线程没起来"或"绑核到同一物理核"。 2. **量出真实带宽**：用 `perf stat -e LLC-load-misses` 或 IMC/uncore 事件读**从内存控制器实际搬运的字节数**，与时间相除得到**实测 GB/s**，与 STREAM 类内核测得的机器可持续带宽比较。若实测已达 90%+，则程序其实已经到位，20 GB/s 的假设偏乐观（真实机器可能只有 8–12 GB/s）——**是模型需要修正，而不是程序**。 3. **区分延迟界与带宽界**：做**并发扫描**——固定问题、扫线程数 1→16，同时扫循环展开/`float4` 向量化。若带宽随并发**线性上升到某点后走平**，走平点即带宽界；若**始终线性上升**，说明仍是延迟/MLP 界，应加预取或展开。 4. **高水位实验定位同步成本**：删掉屏障/合并（结果允许暂时错误）测时间；若明显变快，说明同步/负载不均占了大头。同时用 `schedule(static)` vs `dynamic` 对比，检查负载不均。 5. **检查 cache 与页行为**：`perf stat -e L1-dcache-load-misses,LLC-load-misses` 看命中率；确认分块尺寸真的让中间缓冲常驻（`CHUNK=16` 时 `18×1024×4 = 73.7 KB` 是否小于该机器的 L2 私有容量）；用 `numactl --membind` / first-touch 确认没有远地 NUMA 访问（远地带宽可能只有本地的一半）。 6. **最后才怀疑指令层**：看 `stalled-cycles-frontend/backend`、IPC，以及反汇编确认内层循环确实被向量化（没有向量化则单核算力降 8 倍，但在此内核里算力远非瓶颈，所以这一项优先级最低——**这正是"先分类再优化"的价值**）。

问题 3（work-span、伪共享与"加线程为什么没用"）：下面的代码在 4 核机器上跑 `T = 4` 个线程、每线程 10^8 次自增，实测 11.2 秒；把数组元素改成 `alignas(64)` 各自独占一行后实测 0.28 秒。(1) 用 work-span 的语言解释为什么第一版的"并行度"实际不到 1；(2) 说明 11.2 s 与 0.28 s 分别与什么硬件量对应，并给出量级估算；(3) 是否应该继续加线程到 8？(4) 如果这段自增是"统计各类别事件的计数器"（每处理一条记录就给对应类别的计数器加 1），且类别数 = 1000，请给出一个**比 padding 更好的**修改方案，并说明它为什么更好。

```
// counter_frag.c —— 为聚焦问题而缩到最小的片段（不含计时与线程创建）
// 编译: gcc -O3 -pthread counter_frag.c -o counter_frag
#include <pthread.h>
#include <stddef.h>
#define NTHREADS 4

long long cnt[NTHREADS];   // 全局数组，元素相邻（全部落在同一条 64 B cache line 内）
void* worker(void* arg) {
    int t = *(int*)arg;
    for (long long i = 0; i < 100000000LL; ++i) cnt[t]++;   // 自增
    return NULL;
}
```

【答案】 (1) **Work-Span 分析**：Work `W = T × 10⁸ = 4×10⁸` 次自增（每次是 1 次读-改-写）。问题在于 **Span**：4 个 `cnt[t]` 元素位于同一条 64 B cache line 内，x86 的一致性协议以 **cache line 为所有权粒度**，因此线程 t 写 `cnt[t]` 必须独占整条 line；任一时刻只有一个 core 能持有该 line 的 M 态，**自增被强制串行化**。串行化的单位成本是**一次 line 迁移（cache-to-cache transfer + 失效 + 所有权获取）**，实测约 **数十到约 100 ns**。于是 `S ≈ W × L_transfer = 4×10⁸ ×（数十 ns）` ⇒ 用 11.2 s 反推 `L_transfer = 11.2 s / 4e8 = 28 ns`，即每次自增平均 28 ns——**完全符合"每次自增 = 一次跨核 line 迁移"的预期**。因此**并行度 = W/S = 1/L_transfer ≈ 3.6×10⁷ 次/秒，与线程数 T 无关**；等价说法是：**真实并行度 ≈ 1**（4 个线程在队列里轮流拿同一条 line）。**加线程只会让 line 搬家更频繁，不会提高吞吐。**

(2) **11.2 s 对应"一致性/延迟串行化"**，其控制量是 **line 迁移延迟（~28 ns 量级）**，而不是带宽或算力：DRAM 字节数极低（数据本来就在 cache 里），ALU 利用率极低（每次自增之间要等几十 ns）。**0.28 s 对应"ALU/流水线极限（含私有 line 的 L1 命中）"**：`4×10⁸` 次自增 / 0.28 s = `1.43×10⁹` 次/秒；分摊到 4 个核、每核 3.0 GHz，即**每核约 0.12 次自增/周期**。这个数**偏低**（一个 `inc` 序列在 L1 命中的情况下本可接近 1 次/周期），原因是 `cnt[t]++` 是"load–add–store"且相邻迭代存在**循环携带依赖**（同一个计数器），每轮至少要等一次 L1 命中延迟（~4–5 周期）+ 一次 store-to-load forwarding，再加上 `volatile`/编译屏障（若原代码用了）阻止了流水化。**量级结论：11.2/0.28 = 40×，与"跨核 line 迁移 ~28 ns 对比 L1 命中约 1–2 ns"的 ~20–30× 比值同量级**，多出来的部分来自协议开销（请求转发、目录/侦听、失效确认）。

(3) **不应该。** 因为 `W/S` 已经与 T 无关（≈3.6×10^7 ops/s，由 line 迁移延迟决定），**加线程到 8 的预期结果是"不变或更慢"**——所有 8 个线程仍在争同一条 line，只会增加协议流量与排队。这与 §3.2 的分析完全一致：**伪共享下"并行度"从 T 掉到 1 以下，加核无用**。正确的下一步不是加线程，而是**消除争用**（padding、或按 (2)(4) 的方案改造算法）。

(4) **更好的方案：按线程/块做"私有直方图 + 最后归并"（privatization）**，而不是给 1000 个计数器逐个填充到 64 B。 – 做法：每个线程先在自己的**私有**局部直方图（`long long local[NBINS]`）上累加，全部处理完后，再在临界区里做一次 `atomic` / `reduction` 把 `T` 份局部直方图按 bin 相加（归并成本 `O(NBINS × T)`，与 `$10^8$` 次自增相比可忽略）。 – 为什么比 padding 更好：①**padding 的容量代价是 `64 B × 1000 = 64 KB`（每线程！4 线程 256 KB）**，会溢出 L1（32–48 KB）、挤占 L2，并把直方图从"L1 常驻"变成"L2/L3 往返"，反而**增加真实带宽需求**；私有直方图只需 `8 B × 1000 = 8 KB/线程`，稳稳装进 L1。②padding 只消除了**跨核争用**，但每次自增仍是"共享可写位置"的读–改–写，且若同一线程内多个 bin 被高频更新仍有**写端口/转发**压力；私有化则让每次自增都命中**本核 L1 中自己独占的行**，并把跨核通信压缩成**一次批量归并**。③这是本课程反复出现的通用范式：**"共享写"→"私有写 + 归并"**，与讲义把"每个 (i,j) 加锁"改成"每线程累加 `myDiff`、每次迭代只加锁一次"（`thoughtprocess` slide 68→69）是**同一个手法的两种表现**。 – **正确性注意**：计数器的最终值必须通过一次**原子/同步的归并**得到（`#pragma omp atomic` 或 `#pragma omp critical`，或 `#pragma omp parallel for reduction`），归并点是**唯一**的同步需求；同时要保证归并发生在线程全部完成本地累加之后（隐式 barrier）。

Lecture 15: Implementing Synchronization

1. 章节标题与概述

Lecture 15: Implementing Synchronization（同步原语的实现：从原子指令到可扩展的锁与屏障）

- **本讲核心问题**：同步原语（synchronization primitive）的**语义**（"互斥"、"屏障"）很容易用一句话说清，但**高效地实现**它们却完全由硬件细节决定：一条原子指令在总线上究竟产生多少流量？P 个处理器同时抢一把锁时，瓶颈是锁本身还是互连？屏障的延迟能不能从 O(P) 降到 O(log P)？讲义开篇就把范围钉死在两件事上——(1) **保证互斥（mutual exclusion）的原语**：锁（locks）、原子原语（atomic primitives，如 atomic_add）、事务（transactions）；(2) **事件通知（event signaling）的原语**：屏障（barriers）、标志（flags）。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：原子读–改–写指令（read–modify–write），包括 **test–and–set（TAS）**、**x86 `lock cmpxchg`（compare and exchange，比较并交换）**、**atomic_increment / fetch–and–add（原子取号加）**；支持这些指令的**缓存一致性协议**（snooping MSI/MESI，以及 BusRd / BusRdX / BusWB 三类总线事务）；**互连（总线/环）作为一致性顺序的串行化点**；以及底层的内存一致性模型（memory consistency model，如 TSO）与 **fence（内存屏障）**。
- **软件侧**：三种"锁算法族"——自旋类（test–and–set、**test–and–test–and–set (TTAS)**、指数退避）、取号类（**ticket lock**）、队列/数组类（**array–based lock**、**MCS queue–based lock**）；以及屏障的四种实现——朴素共享计数器、**两计数器正确版本**、**sense reversal（感知翻转）**、**combining tree（合并树）**。配套的 Stanford CS149 讲义补充了这些原语赖以成立的地基：一致性不变量、**伪共享（false sharing）**、**顺序一致性（SC）/ 全存储序（TSO）**、**数据竞争（data race）** 与 **DRF（data–race–free）** 契约。
- **在并行计算知识体系中的角色**：本讲是"共享内存并行"这条主线上**从硬件机制到编程原语的最后一公里**。前面几讲（缓存一致性、目录一致性、侦听实现、内存一致性）解释了"硬件如何让 P 个核看到一个一致的内存"；本讲回答"程序员/运行时库如何在这个内存之上造出锁与屏障，并且让代价不随 P 爆炸"。它同时给出了一条贯穿全课程的量纲：**同步不产生任何计算，它只消耗延迟、带宽与程序员的心智**——因此锁的可扩展性、屏障的 span、伪共享的流量，最终都要用 work–span 与带宽预算来结算。它也是下一讲（Fine–Grained Synchronization, Lock–Free Programming）的直接前提：理解了 TAS/CAS 的代价，才明白为什么要用细粒度锁与无锁数据结构。
- **配套材料**：
- `lectures/16_synchronization.pdf`（抽取文本 `extracted/16_synchronization.txt`，共 **29 页**）：Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）把 **Sep 30** 排为第 15 讲 "Implementing Synchronization"，该行的 slides 链接目前以 HTML 注释形式给出（"slides/video from a previous offering; uncomment when posted for Fall 2026"），注释中的 slides 指向 `lectures/16_synchronization.pdf`。该 PDF 目前位于公开的 https://www.cs.cmu.edu/~418/lectures/ 目录下，**可在公开网络直接下载**。讲义首页写的是 **"CMU 15–418/15–618, Fall 2025 / Lecture 16: Implementing Synchronization"**——这是**讲义沿用历史学期版本**的正常现象（讲次编号与学期字样随年度重排），不是错误。
- `cs149_supp/sync_consistency.txt`：Stanford CS149（Fall 2025）**Lecture 15: Memory Coherency and Consistency** 的公开讲义抽取（60 页），**已公开**，作为本讲底层支撑使用（MSI/MESI 状态机、目录一致性、伪共享实测 14.2 s vs 4.7 s、SC/TSO/PC/PSO/WO–RC、fence、DRF 契约）。
- 讲义中的性能图注明 "Figure credit: Culler, Singh, and Gupta"，即经典教材《Parallel Computer Architecture: A Hardware/Software Approach》的图；MCS 锁算法注释指向经典论文《Algorithms for Scalable Synchronization on Shared Memory Multiprocessors》。

- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被注释隐藏，属**未发布**。
- Ed 讨论区、Autolab、Canvas：**需登录**，非公开。
- 部分讲座（Performance Analysis/Profiling、Transactional Memory、AI in System Design 等）在 Fall 2026 **尚未发布**讲义；其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。

---

2. 核心概念与硬件/软件架构图解

2.1 同步事件的三个阶段：把"锁"拆成三个可独立优化的部件

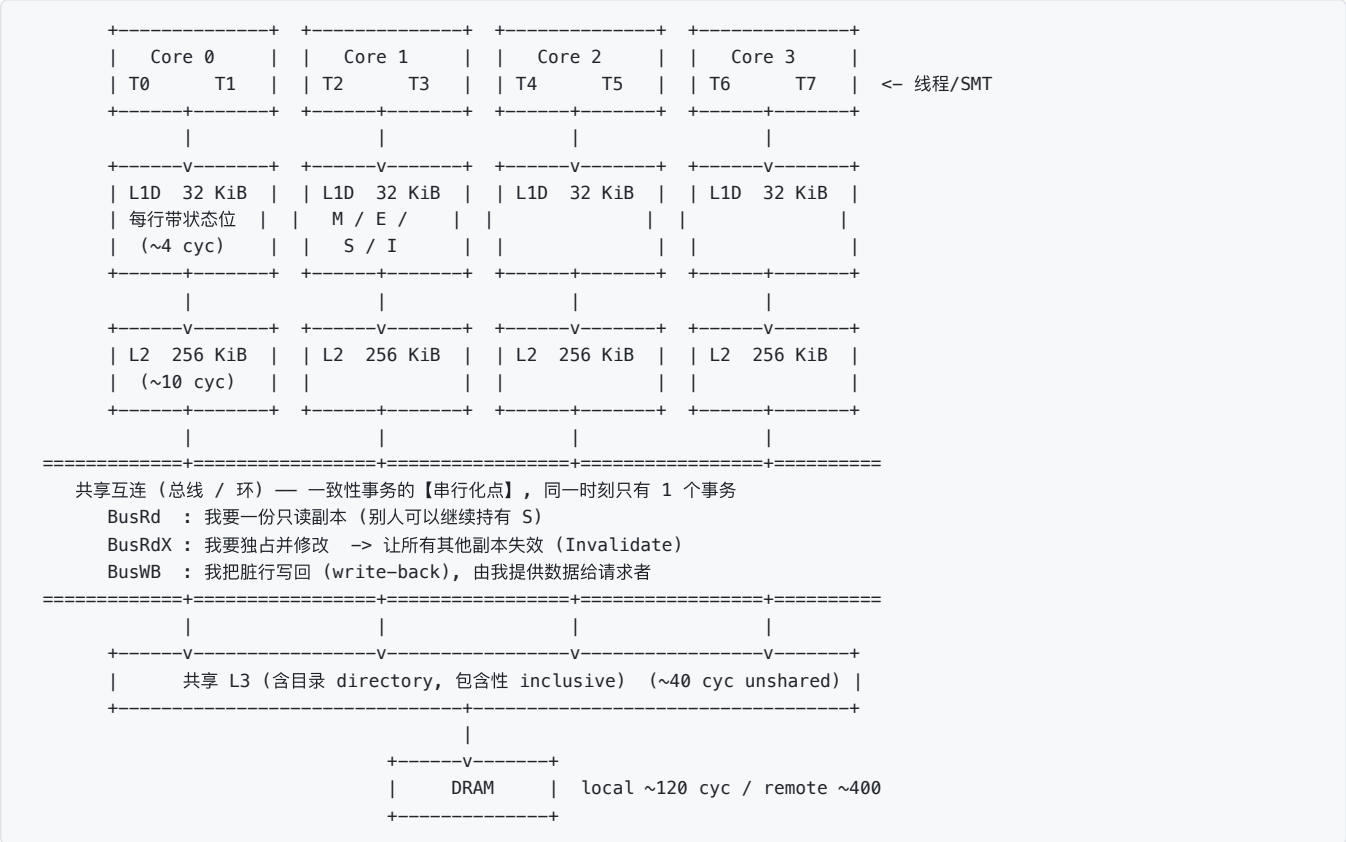
- **定义与目的**：讲义把**任何**一次同步事件拆成三个阶段（slide 3）：
- **Acquire method（获取方法）**：线程**如何尝试**获得对受保护资源的访问权；
- **Waiting algorithm（等待算法）**：当访问尚未被授予时，线程**如何等待**；
- **Release method（释放方法）**：临界区工作完成后，线程**如何让其他线程获得**该资源。 这个拆分的价值在于：锁的性能问题**几乎从不均匀分布在三个阶段上**。TAS 锁的灾难在 *waiting*（等待者不停地写，制造总线风暴）；ticket 锁的代价在 *acquire*（一次原子取号）；MCS 锁的胜利也在 *release*（精确唤醒下一个，只写一个字节）。把三者分开，就能一眼看出该改哪一段。
- **直观解释（"它是什么？"）**：想象**只有一个洗手间的办公室**。*Acquire* = 你走过去拧门把手（拧的**方式**决定了你会不会把整层楼的人都吵醒）；*Waiting* = 发现门锁着，你怎么办（站在门口不停拧把手 = 忙等待；回工位等同事喊你 = 阻塞）；*Release* = 你出来后怎么通知下一个人（喊一声"下一位" = 精确唤醒；把门敞开让所有人一起冲 = TAS 式惊群）。同一间洗手间，三种策略的性能差一百倍。
- **三个阶段在五种原语上的展开（表 1）**：

阶段	要回答的问题	TAS 锁	TTAS 锁	Ticket 锁	MCS 队列锁
Acquire	如何尝试取用	无条件 `exchange(1)`（每次都发 BusRdX）	先读；确认空闲后才 `exchange`	`fetch_add(next_ticket)` 取号	`exchange(tail)` 挂到队尾
Waiting	如何等待	原地**连续** TAS → 总线风暴	在**本地缓存副本**上只读自旋 → 零总线流量	自旋读 `now_serving`（只读，不写）	自旋在**自己私有的队列节点**上
Release	如何让别人进来	写 0（行已被抢走，需重新 BusRdX）	写 0（同上）	`now_serving++`（一次写，O(P) 失效）	直接写下一个等待者的节点 → **精确唤醒 1 个**
每次释放的互连流量	—	持续 O(P)/次尝试	O(P)	O(P)	**O(1)**
公平性	—	无	无	**FIFO**	**FIFO**
存储	—	1 int	1 int	2 int	1 指针 + 每线程 1 节点

- **性能特征**：三个阶段的时间量级完全不同——*acquire* 是"一次原子操作往返"（几十到几百周期，取决于该行在谁手里）；*waiting* 是"**重复次数 × 每次往返**"（这才是 TAS 崩溃的根源）；*release* 是"**唤醒几个等待者 × 每个等待者产生的事务数**"（这是 MCS/数组锁取胜的地方）。

2.2 硬件结构：私有缓存 + 共享互连 + 原子操作 = 同步的地基

- **定义与目的**：同步原语的实现必须建立在一个**具体的硬件模型**上：P 个核各有私有 L1/L2（保存 cache line 的状态位 M/E/S/I），共享一条互连（总线或环）；**互连是一致性事务的串行化点**，因此也是"谁先拿到锁"的物理仲裁者；原子读-改-写指令（TAS / CAS / fetch-add）则是唯一能绕过"LOAD-TEST-STORE 非原子"陷阱的硬件机制。
- **直观解释（"它是什么？"）**：把共享内存想成**一间有 P 个读者的阅览室，每个人手里都有一份复印页**。要**读**，只要手里有复印件（S 态）即可，本地零成本；要**改**，必须先让所有其他人的复印件作废（BusRdX → Invalidate），然后才能独占涂改（M 态）。这条"改之前先清场"的规则，就是所有同步原语性能代价的物理来源：**任何一次原子的写，都是一次"清场广播"**。而互连就像阅览室里唯一的传声筒——**同一时刻只有一个人在说话**。
- **图解（图 1）：典型多核 + 私有缓存 + 共享互连的硬件结构，以及三类一致性事务**



- **关键操作与性能特征**（数据取自配套 CS149 讲义给出的 Core i7 Xeon 5500 量级，用于建立"预算感"）：
- **L1 命中 ≈ 4 周期**：这是 TTAS 等待者"在本地缓存副本上自旋"的成本——**几乎免费的等待**；
- **L3 命中、行未被共享 ≈ 40 周期；行被别的核共享 ≈ 65 周期；行在别的核里是脏的（modified in another core）≈ 75 周期**：这三个数字就是"一次穿越互连的一致性事务"的价目表，也是同步原语延迟的主要构成；
- **本地 DRAM ≈ 120 周期（约 30 ns）；远端 DRAM ≈ 400 周期（约 100 ns）**；
- **带宽视角**：一条 64 B 的 cache line 每被"抢走一次"就要占满互连传一次。若互连数据带宽等效为 8 B/cycle，则**一次一致性行传输 = 8 个互联周期**；这是本讲所有数值算例的交换率基准（**延迟 ~75 周期，带宽 8 B/周期/行**）。
- **吞吐量含义**：正因为"一次原子的写 = 一次清场广播"，同步原语的**可扩展性**本质上是一个**互连流量问题**：每次同步事件产生多少个事务、每个事务多大、P 增大时事务数是否增长（O(1) / O(P) / O(P^2)）。

2.3 忙等待 vs 阻塞式同步：为什么在并行程序里"自旋"往往是正确的

- **定义与目的**：
- **Busy waiting（忙等待 / spinning）**：`while (条件 X 不成立) {}`，然后执行"假定 X 成立"的逻辑。线程**不放弃处理器**。
- **Blocking synchronization（阻塞式同步）**：`if (X 不成立) block until X 成立;`——由 OS 调度器把线程**换下处理器**（de-schedule），让别的线程跑；典型例子是 `pthread_mutex_lock`。
- 目的：在"等多久"与"等待期间谁用 CPU"之间做选择。
- **直观解释（"它是什么？"）**：自旋 = **在电梯门口原地按按钮**；阻塞 = **回工位干活，让前台打电话叫你**。操作系统课程教"自旋是坏的"，因为在**超售（oversubscribed）**的机器上，你按按钮的时候别人没 CPU 可用，纯粹烧电。但讲义明确指出：**在性能关键的并行程序里，我们通常不超售系统**（机器上不会同时跑好几个 CPU 密集程序），这时自旋反而更好——因为没有别的活要干，把 CPU 让出去只会换来两次上下文切换的开销。讲义还特意提醒一个容易混淆的点：**"自旋浪费处理器"与"用多线程交错执行来隐藏访存延迟"是两件不同的事**，不要拿后者为前者辩护。
- **讲义给出的"自旋优于阻塞"的四种情形（slide 6）**：
- **调度开销大于预期等待时间**（OS 上下文切换是微秒量级，而抢一把锁可能只要几百纳秒）；
- **尾延迟（tail latency）效应**：阻塞/唤醒的排队会让延迟分布的长尾变长；
- **处理器的资源本来也没别的任务要用**（不超售）；
- 需要自旋的现实接口示例：`OSSpinLockLock(&lock)`、`pthread_spin_lock(&spin)`。
- **图解（图 2）：两种等待策略的执行模型与代价**

```
(a) 忙等待 (spinning): 等待者持续占用执行单元, 反复访问同步变量
   core0 : [T&S][T&S][T&S][T&S][T&S][T&S][T&S] |== 临界区 ==| [T&S][T&S]...
                   ^ 每次 T&S 都可能产生一次互连事务 (见 2.6)
   core1 : |================ 临界区 (持有锁) ================| [ 自旋 ]...
   core2 : [T&S][T&S][T&S][T&S][T&S][T&S][T&S][T&S][T&S][T&S]...
           代价 = 自旋指令的执行 + 互连事务流量; 收益 = 零调度延迟, 一旦
                  锁释放立刻能上 (亚微秒级响应)

(b) 阻塞 (blocking): 失败即让出 CPU, 由 OS 调度器在锁释放时唤醒
   core0 : [try]--[阻塞/让出].........(OS 不调度我).........[被唤醒][得到锁][临界区]
   core1 : [      临界区      ][unlock -> 唤醒 core0 ]
           代价 = 2 次上下文切换 (μs 级) + 调度器排队延迟;
           收益 = 等待期间 CPU 可用于其它线程 (只有真有别的活时才划算)
```

- **性能特征**：自旋的代价是**互连流量 + 执行单元占用**（可测量为"每次失败尝试的事务数"），阻塞的代价是**两次上下文切换 + 调度延迟**（微秒级）。判据很干脆：**预期等待时间 < 上下文切换开销 ⇒ 自旋；否则阻塞**。这也是为什么"混合锁"（先自旋一小会儿，再阻塞，如 Linux futex、Java 的偏向锁/自适应自旋）在实践中胜出。

2.4 朴素锁为什么错：LOAD-TEST-STORE 不是原子的

- **定义与目的**：讲义用一个"看起来对、实际错"的锁作为热身（slide 8）：

```
asm lock: ld R0, mem[addr] ; 把锁字读进 R0 cmp R0, #0 ; 和 0 比较 bnz lock ; 非 0 说明别人持有 → 回到顶部重试 st mem[addr], #1 ; 认为锁空闲 → 置 1, 进入临界区 unlock: st mem[addr], #0 ; 释放: 写 0
```

- **直观解释（"它是什么？"）**：这就是**"看没人举手，我就直接进会议室"**——看（load）和坐（store）之间有一个时间窗。类比：停车场只剩一个车位，两辆车都"看到"空位，然后都倒进去。错误的原因是 **LOAD-TEST-STORE 不是原子操作（not atomic）**：P0 读到 0，P1 也读到 0，P0 写 1，P1 也写 1 —— **两个线程同时认为自己持有锁**，互斥彻底失效（这是**数据竞争 data race** 的最经典形态）。
- **要点**：修好它的唯一办法是让"读—判断—写"变成**一条不可分割的硬件指令**，即原子读-改-写（atomic read-modify-write）：TAS、CAS、fetch-add 都属于这一类。

2.5 Test-and-Set 锁：用一条原子指令换来正确性，却换来一场总线风暴

- **定义与目的**：**test-and-set（TAS）**指令语义（slide 9）：`ts R0, mem[addr]` = 把 `mem[addr]` 读入 R0，**并且**若原值为 0 则把它设为 1（整个读+条件写原子完成）。于是锁可以用两条指令实现：

```
asm lock: ts R0, mem[addr] bnz R0, lock ; 返回非 0 ⇒ 锁已被别人持有, 继续自旋 unlock: st mem[addr], #0
```

- **直观解释（"它是什么？"）**：TAS 就像**"伸手抢座位"**：伸手和坐下是同一下动作，不可能两个人同时坐下。代价是：**每一次尝试伸手，都会把旁边所有拿着座位复印件的人手里的复印件作废**（一次 BusRdX + 对所有共享者的 Invalidate）——即使锁已经被别人占着，即使你明知抢不到。
- **图解（图 3）：TAS 锁在互连上的实际时序（对应 slide 10/11 的流量图）**

```
时间轴  ------------------------------------------------------------------->
P0 (持有者)
   |-- BusRdX: 拿独占行, 写 1 -->|========== 临界区 (持有锁) ==========|
                                                                  |-- unlock:
                                                                  |  BusRdX 再拿
                                                                  |  独占行, 写 0 -->|
P1 (等待者)
   |            T&S --BusRdX--> 失败(读到 1) ----> T&S --BusRdX--> 失败 ----> T&S 成功 -->
   |            ^                ^                 ^
   |            |                |                 |
   每次失败的 T&S 都要: 抢互连 -> 发 BusRdX -> 让 P0 的副本失效 -> 读回 1 -> 失败
   |<------- 一次失败 T&S 的往返 ≈ 40~75 周期 (取决于争用) ------->|
P2 (等待者)
   |              T&S --BusRdX--> 失败 ----------> T&S --BusRdX--> 失败 ----------> ...
互连 (总线)
   |##失效##|##失效##|##失效##|##失效##|##失效##|##失效##|##失效##|##失效##|
   ^ 满载! 同一时刻只有 1 个事务能上总线; 等待者越多, 总线越挤

讲义给出的两条结论:
  (1) 总线争用使"锁的移交"变慢: 持有者 unlock 时也必须排队等总线
  (2) 图中未画出但同样致命: 总线争用也拖慢【临界区本身】的执行
      (临界区里的访存同样要从这条拥挤的互连上过)
```

- **性能特征（slide 13/14）**：基准程序是 P 个处理器合计执行 N 次 `lock(); critical-section(c); unlock();`，**并把临界区时间扣掉**，只留取锁/放锁的时间；横轴是处理器数。曲线随 P **急剧上升**，原因是"**总线争用 → 锁移交变慢 + 临界区也被拖慢**"。讲义给出的理想锁应具备的五条性质，以及 TAS 锁的成绩单：

理想性质	含义	简单 TAS 锁的表现
Low latency（低延迟）	锁空闲且无人竞争时应能立刻拿到	✅ 低竞争下很好（一条指令）
Low interconnect traffic（低互连流量）	高竞争时应尽可能少说话	❌ 每次失败尝试都制造一次 BusRdX
Scalability（可扩展性）	延迟/流量随 P 增长应"合理地"增长	❌ 流量随 P 增长，快得离谱
Low storage cost（低存储开销）	每个锁占多少内存	✅ 1 个 int
Fairness（公平性）	避免饥饿；理想是**按请求顺序**获得	❌ 无任何公平性保证（可能饥饿）

2.6 Test-and-Test-and-Set（TTAS）：把"等待"从写操作改成读操作

- **定义与目的**：TTAS（slide 15）的核心思想是**把等待与尝试分开**：先在**本地缓存副本**上做只读自旋（命中 L1，零互连流量），只有观察到锁被释放（`*lock == 0`）时才用 TAS 真正去抢：

```
c void Lock(int* lock) { while (1) { while (*lock != 0); // 另一个处理器持有锁时：只在本地缓存副本上自旋 if (test_and_set(*lock) == 0) // 锁被释放了：这才去抢 return; } } void Unlock(volatile int* lock) { *lock = 0; }
```

- **直观解释（"它是什么？"）**：TAS 锁是**每个人都在不停地拧门把手**；TTAS 锁是**大家坐在自己工位上盯着门口的灯（读本地副本），灯一变就集体冲过去抢（TAS）**。因为"盯着看"是本地缓存命中（4 周期），互连上几乎没人说话；只有当门真的开了，才会发生一次"惊群"式抢锁。
- **图解（图 4）：TTAS 的互连流量（对应 slide 16/17）**

```
P1（持有者）  |--BusRdX, 写 1-->|===== 临界区 =====|--BusRdX, 写 0-->|
                                                    ^ 释放时向所有共享者发 Invalidate
P2（等待者）  [BusRd: 拿一份只读副本, 停在 S 态]
              [只在本地 L1 上反复读 => 大量本地命中, 零互连事务]
                                        ^被 Invalidate^
                                                    |--BusRdX--> 抢(成功/失败)
P3（等待者）  [BusRd: 拿只读副本][本地读][本地读]...    ^被 Invalidate^
                                                    |--BusRdX--> 抢(败) --> 回去本地读
互连
   | BusRd |      (长时间空闲: 大家都在本地读)      | Invalidate ×(P-1) | BusRdX ×(P-1) |
                                                      ^ 一次释放: 每个等待者 1 次失效
```

- **性能特征（slide 18）**：
- **无竞争时延迟略高于 TAS**：必须先 test（读）再 test-and-set，多一次步骤；
- **互连流量大幅下降**：每次锁释放，**每个等待者只产生 1 次失效**，即 **O(P) 次失效**——对比 TAS 是"每个等待者**每次测试**都产生一次失效"（与等待时长成正比，可以差几个数量级）。讲义进一步指出：**如果所有 P 个处理器都缓存了这一行，那么总体互连流量是 $O(P^2)$ 量级**；
- **可扩展性改善**（因为流量降了）；**存储不变**（1 个 int）；**仍然没有公平性**。
- 注意残留成本：一次释放后仍会有 **P-1 个等待者同时发 BusRdX**（惊群），输掉的那些还要重新 BusRd 把行读回来继续自旋——这部分流量仍然随 P 线性增长，这正是后面 ticket / array / MCS 锁要解决的问题。

2.7 退避（Back-off）：用"少问几次"换"少堵总线"

- **定义与目的**：抢占失败后**先等一会儿再重试**（slide 19）：

```
c void Lock(volatile int* l) { int amount = 1; while (1) { if (test_and_set(*l) == 0) return; // 抢到就返回 delay(amount); // 失败：退避一段时间 amount *= 2; // 指数增长 } }
```

- **直观解释（"它是什么？"）**：这就是**以太网的 CSMA/CD 退避策略搬到锁上**：撞车了就别马上再撞，随机/递增地等一会儿。大家的尝试频率一降，互连上的无效事务自然就少了。
- **讲义提出的关键问题与答案："无竞争时延迟与 TAS 相同，但在竞争下延迟可能更高。为什么？"**——因为退避的线程**在锁被释放的那一刻可能正睡着**：锁变空闲了它也不知道，必须等自己的 delay 走完才去看，于是"锁空闲但没人来拿"的空窗期被拉长；等待者越多、退避越久，这个空窗越大。
- **性质**：流量比 TAS 小、可扩展性改善、存储仍是 1 个 int；但**指数退避会造成严重的公平性问题（severe unfairness）——新来的请求者退避间隔更短**（`amount` 从 1 开始），因此**有可能后到先得**，老等待者理论上可以被无限期插队（饥饿 starvation）。

- **性能特征**：退避把"流量随等待时长线性增长"改成了"流量随退避次数对数增长"，但它不解决公平性，也不消除惊群；工程上常与 TTAS 组合（TTAS + back-off），并用**固定上限**避免退避过久。

2.8 Ticket 锁：把"抢锁"变成"排队取号"，一次释放只产生 O(P) 流量

- **定义与目的**：TAS 系锁的根本毛病是"**释放瞬间所有等待者一起用 TAS 去抢**"。ticket 锁（slide 20）改用 **原子取号 + 只读等待**：

```
c struct lock { volatile int next_ticket; volatile int now_serving; }; void Lock(lock* l) { int my_ticket = atomic_increment(&l->next_ticket); // 取一个号 while (my_ticket != l->now_serving); // 等叫号 (纯读!) } void unlock(lock* l) { l->now_serving++; } // 叫下一个号
```

- **直观解释（"它是什么？"）**：**银行取号机**。进门先按一下取号机（一次原子加），然后坐下盯着叫号屏（只读），被叫到才起身（进临界区）。取号机只在"进门"那一瞬间被使用一次，因此抢锁的"惊群"消失了。
- **性能特征**：
- **Acquire 不需要原子操作**（讲义原文：No atomic operation needed to acquire the lock (only a read)）；
- **每次释放只产生 1 次失效（O(P) 互连流量）**：`now_serving++` 使所有正在读该行的等待者失效，它们各自重新读一次；
- **天然 FIFO 公平**（先到先得，理论无饥饿）——这是它相对 TAS/TTAS/退避的免费礼物；
- 代价：取号必须原子（一次 BusRdX）；等待者在共享行上自旋，每次释放后 **P−1 个读者要重新取数**（残余 O(P) 流量，见 §4 算例）。
- 工程细节（笔记补充）：`next_ticket` 与 `now_serving` 若落在同一 cache line，则每次"取号"都会让所有等待者的只读副本失效——把两者分开填充（padding）是常见优化。

2.9 数组锁（array-based lock）：让每个等待者在不同地址上自旋

- **定义与目的**：既然 ticket 锁的残余流量来自"所有等待者共享同一行"，那就给**每个等待者一个独立的、填过填充的槽位**（slide 21）：

```
c struct lock { volatile padded_int status[P]; // 每个槽位填充(padded), 保证不共用 cache line volatile int head; }; int my_element; // 每个线程私有 void Lock(lock* l) { my_element = atomic_circ_increment(&l->head); // 环形递增, 分配槽位 while (l->status[my_element] == 1); // 只在自己的槽位上自旋 } void unlock(lock* l) { l->status[my_element] = 1; // 标记自己的槽位可用 l->status[circ_next(my_element)] = 0; // 放行下一个槽位 }
```

- **直观解释（"它是什么？"）**：**医院分诊叫号 + 每人一个专属指示灯**。你不再盯着公共叫号屏，而是盯自己头顶那盏灯——别人释放锁时只**点亮你这一盏**（写一个地址），其余人的灯纹丝不动。于是"一次释放"在互连上只需改动极少量数据。
- **性能特征**：**每次释放 O(1) 互连流量**（写到下一个等待者的槽位；若每个槽位独占一行，则约 2 次行事务）；但**锁的空间开销随 P 线性增长**（`status[P]` 且带填充 → P × 64 B）；此外 `atomic_circ_increment`（带环形回绕的原子加）是**更复杂的原子操作，开销更高**（讲义原文）。
- 与 MCS 的对比：数组锁用**空间**换"每个等待者一个独立地址"；MCS 用**每线程一个私有节点**换同样的效果，但空间是每线程私有的（不随锁数量放大），且没有"环形递增"的复杂度。

2.10 x86 `cmpxchg`：实现 CAS 系原语的硬件指令

- **定义与目的**：`lock cmpxchg dst, src`（slide 22）是 x86 上实现 CAS、ticket、MCS 等一切"比较并交换"语义的指令：

```
text lock cmpxchg dst, src if dst == accumulator: ; accumulator 通常是 eax ZF = 1 ; 标志寄存器置位 = 成功 dst = src ; 写入新值 else ZF = 0 ; 失败 accumulator = dst ; 把当前值返回给调用者
```

```
语义三步 (讲义 slide 22 的原文): 1. Does the dst have the value we think it has? (目标里是我们以为的值吗?) 2. Then make the update (是 → 更新) 3. If not return the current value (不是 → 返回当前值, 让软件重试)
```

- **直观解释（"它是什么？"）**：CAS 是**"验货后付款"**：我先说"如果箱子里还是我上次看到的那件货，就把它换成我的货"，硬件保证"看"与"换"之间不被打断；如果箱子被换过了，把新货色告诉我，我自己决定要不要重试。`lock` 前缀的作用是**让这条指令在多核之间真正原子**（在缓存一致性协议上表现为"取独占所有权"）。
- **性能特征**：`lock cmpxchg` 在**成功**路径上等价于一次 BusRdX（拿独占 + 让所有人失效，~65–75 周期）；在**失败**路径上（值已被改）通常无需总线事务（本地就能判断），这是 CAS 循环优于 TAS 循环的原因之一。**CAS 循环的经典隐患是 ABA 问题**（值从 A 变 B 又变回 A，CAS 无法察觉），这也是下一讲（lock-free）的核心议题。

2.11 队列锁（MCS lock）：O(1) 流量 + FIFO 公平 + 精确唤醒

- **定义与目的**：**MCS 锁**（Mellor-Crummey & Scott，讲义 slide 23 给出伪代码；注释指出要用到 slides 22 的 `cmpxchg` 语义）让每个等待者**在自己的私有节点上自旋**，锁本身只保存**队尾指针**：

```
text AcquireQLock(*glock, *mlock) ReleaseQLock(*glock, *mlock) { { mlock->next = NULL; do { mlock->state = UNLOCKED; if (mlock->next == NULL) { ATOMIC(); x = CMPXCHG(glock, mlock, NULL); prev = glock; // 原子交换 if (x == mlock) return; *glock = mlock; } END_ATOMIC(); else { if (prev == NULL) return; // 队空, 直接获得 mlock->next->state = UNLOCKED; mlock->state = LOCKED; return; prev->next = mlock; } while (mlock->state == LOCKED) ; // SPIN } while (1); } }
```

- **直观解释（"它是什么？"）：排队传话**。每个人拿着自己的小纸条（本地节点）站在队伍里，纸条上写着"轮到我了吗"。**上一个人离开时，只需要走到你面前拍你一下**（写你的 `state`），不需要向全楼广播。所以：流量 O(1)、顺序严格 FIFO、被唤醒的人**恰好一个**（不会惊群）。
- **图解（图 5）：MCS 队列锁的数据结构与唤醒路径**

```
   glock（全局锁：只是一个指向队尾的指针）
     |
     v
  +---------+    next    +---------+    next    +---------+    next    +---------+
  | mlock   |----------->| mlock   |----------->| mlock   |----------->| mlock   |
  |  P3     |            |  P7     |            |  P1     |            |  P4     |
  | LOCKED  |            | LOCKED  |            | LOCKED  |            | LOCKED  |
  +---------+            +---------+            +---------+            +---------+
   队首=持有者               |                      |                      ^ 新来者
     |                       |                      |                      | 挂到队尾
     |  每个节点都是【线程私有】的变量（通常在线程栈/线程本地存储里）
     |  => 每人在自己的 cache line 上自旋，互不干扰，零互连流量
     |
     +--- unlock：若 mlock->next != NULL，只写 mlock->next->state = UNLOCKED
                  => 精确唤醒 1 个等待者，互连上只搬 1 条 cache line（O(1)）
          若 mlock->next == NULL：用 CMPXCHG(glock, mlock, NULL) 尝试"我是最后一个
                  且无人排队" -> 成功则锁变空闲；失败说明有人正在挂入，等它把 next 写出来

  释放路径（所有锁算法对比的关键）：
     TAS / TTAS / ticket : 释放 -> 通知【所有】P-1 个等待者（O(P) 流量）
     MCS                 : 释放 -> 通知【下一个】等待者（O(1) 流量），且顺序 = 请求顺序
```

- **性能特征**：**每次释放 O(1) 互连流量**；**FIFO 公平（无饥饿）**；**存储开销是"每线程一个节点"**（不是每锁 P 份，因此锁多了也不炸）；代价是：需要 `cmpxchg` 处理"释放时发现无人排队"的竞态（释放路径更复杂），且**节点必须常驻**（不能放在会被回收/搬移的栈帧里，否则队列断裂——工程上用线程本地存储或节点池）。
- 与讲义 slide 24 的提示一致：算法的正确性依赖 `cmpxchg` 的**确切语义**（成功/失败时返回值不同），伪代码中的 `x = CMPXCHG(glock, mlock, NULL)` 就是"如果 glock 仍是 mlock 就把它设为 NULL"。

2.12 屏障（barrier）：从"朴素共享计数器"到"合并树"

屏障是**事件通知**类原语的代表：它不保护数据，而是让 P 个线程在**同一个逻辑时刻**会合。讲义用四步把屏障的实现讲透。

(a) 朴素集中式屏障：为什么错的版本"看起来对"

```
struct Barrier_t { LOCK lock; int counter; /* init 0 */ int flag; };
// 讲义 slide 25: 朴素版本
void Barrier(Barrier_t* b, int p) {
  lock(b->lock);
  if (b->counter == 0) { b->flag = 0; }      // 第一个到达者清 flag
  int num_arrived = ++(b->counter);
  unlock(b->lock);
  if (num_arrived == p) { b->counter = 0; b->flag = 1; }  // 最后一个到达者置 flag
  else { while (b->flag == 0); }             // 其余人等 flag
}
```

- **它对不对？** 讲义的问题问得很准（slide 25）——请考虑**连续两个屏障**的用法：`do stuff ... ; Barrier(b, P); do more stuff ... ; Barrier(b, P);` **朴素版本在"循环反复使用同一个屏障"时会失败**：快线程走完屏障 1 后冲到屏障 2，作为屏障 2 的"第一个到达者"把 `flag` 清成 0；而**慢线程可能还没观察到屏障 1 的 `flag == 1`**，它看到的却是被清掉的 0，于是永远自旋下去（死锁）。根因是**复用同一个 flag 而没有"代际（generation）"概念**——这就是为什么要引入 sense reversal 或 leave_counter。

(b) 两计数器正确版本：等所有人离开上一代，再清 flag

```
struct Barrier_t { LOCK lock;
                   int arrive_counter;  // init 0
                   int leave_counter;   // init P
                   int flag; };
void Barrier(Barrier_t* b, int p) {
  lock(b->lock);
  if (b->arrive_counter == 0) {            // 我是第一个到达者
    if (b->leave_counter == P) {           // 确认没有别人"还留在屏障里"
       b->flag = 0;                        // 可以安全地清 flag
    } else {
       unlock(lock);
       while (b->leave_counter != P);      // 等所有人离开上一个屏障
       lock(lock);
       b->flag = 0;
    }
  }
  int num_arrived = ++(b->arrive_counter);
  unlock(b->lock);
  if (num_arrived == p) {                   // 最后一个到达者
    b->arrive_counter = 0;
    b->leave_counter = 1;
    b->flag = 1;
  } else {
    while (b->flag == 0);                   // 等 flag
    lock(b->lock); b->leave_counter++; unlock(b->lock);   // 记录"我离开了"
  }
}
```

- **核心思想（讲义原文）**："等所有处理器先离开上一个屏障，清清白白之后，才为下一个屏障清 flag"——用 `leave_counter` 给 flag 加了"代际隔离"。代价是：**等待者要自旋两次**（一次等 flag，一次等别人离开 / 加锁更新 leave_counter）。

(c) Sense reversal（感知翻转）：一次自旋解决问题

```
struct Barrier_t { LOCK lock; int counter; /* init 0 */ int flag; /* init 0 */ };
int local_sense = 0;   // 每个处理器私有的 sense
void Barrier(Barrier_t* b, int p) {
  local_sense = (local_sense == 0) ? 1 : 0;    // 翻转自己的 sense
  lock(b->lock);
  int num_arrived = ++(b->counter);
  if (num_arrived == p) {                       // 最后一个到达者
    unlock(b->lock);
    b->counter = 0;
    b->flag = local_sense;                      // 把 flag 置为"新一代的值"
  } else {
    unlock(b->lock);
    while (b.flag != local_sense);              // 等 flag 变成我这一代的值
  }
}
```

- **直观解释（"它是什么？"）**：**用"翻转"代替"清零"**。大家约定：第 1、3、5… 次屏障等 `flag == 1`，第 2、4、6… 次等 `flag == 0`。这样**永远不需要"把 flag 清回初值"这个危险动作**——flag 的每一次变化本身就是"新一代开始"的信号。类比：交通灯用**红绿交替**而不是"先熄灭再亮"，就不会出现"灯灭了但有人还在过马路"的窗口。
- **性能特征（讲义原文）**：**Sense reversal 优化把两次自旋变成一次**（不需要再等 `leave_counter`）；`local_sense` 必须是**每处理器私有**的；flag 字段**建议单独占一条 cache line**（讲义在 slide 25 反问"Why?"——因为 flag 被 P−1 个等待者同时读，若与频繁写的 `counter` / `lock` 同行，每次计数更新都会让所有等待者的只读副本失效，制造伪共享级别的流量）。

(d) 集中式屏障的流量与 span，以及合并树屏障

- **流量（slide 28）**：每次屏障 **O(P) 互连流量**：
- 所有线程：**2P 次写事务**用于抢屏障锁与更新计数器（在"锁获取是 O(1)"的假设下算作 O(P) 流量）；
- 最后一个线程：**2 次写事务**（写 flag、清计数器），但**因为 flag 有大量共享者，代价仍是 O(P)**；
- **P−1 次读事务**去读更新后的 flag。
- **但真正的伤害是串行化（slide 28 原文）**："still serialization on a single shared lock ⇒ span（整个操作的延迟）是 O(P)"。这一点与流量无关：所有线程在同一个原子变量上排队，屏障延迟随 P 线性增长。

- **能做得更好吗？能——合并树屏障（slide 29）**：
- **合并树（combining tree）更好地利用了互连拓扑中的并行性**，达到 lg(P) 的 span；
- **Acquire（到达）**：处理器到达屏障时，对**父节点的计数器**做一次递增，**递归向上直到根**；
- **Release（释放）**：**从根开始，逐层向下通知子节点**释放；
- **重要限制（讲义原文）**：这套策略**在总线上意义不大**，因为总线上的所有流量本来就被串行化；合并树的价值体现在**点对点互连（如树、环、Mesh）**上。
- 对比：集中式屏障 = **高竞争（high contention）**，单把屏障锁 + 单个计数器成为瓶颈。
- **图解（图 6）：集中式屏障 vs 合并树屏障的执行模型**

```
(a) 集中式屏障: 树高 1, 扇入 P —— 所有线程争同一个变量, span = O(P), 竞争 O(P^2)
      T0  T1  T2  T3  T4  T5  T6  T7        (P = 8)
       \   \   \   |   /   /   /   /
        \   \   \  |  /   /   /   /
       +-v---v---v--v--v---v---v---v-+
       |  单一 lock + counter + flag  |  <-- 串行化点: 每次只能有 1 个线程进来
       +------------------------------+
       到达: lock -> ++counter -> unlock -> 自旋等 flag     (P 次串行加锁)
       离开: 最后一个到达者置 flag -> P-1 个读者重新取数      (O(P) 流量)

(b) 合并树屏障: 扇入 2, 树高 log2(P) —— 到达自底向上合并, 释放自顶向下广播
                      +--------------------+
                      |   root (0)         |  flag/sense
                      +---------+----------+
                 +--------------+--------------+
           +-----v-----+                +-----v-----+
           |  node 1   |                |  node 2   |
           +-----+-----+                +-----+-----+
              +--+--+                      +--+--+
              |     |                      |     |
           node 3  node 4                 node 5  node 6
              |     |                      |     |
             T0    T1                     T2    T3      (P = 4 的示意)
       到达 (acquire): 每个叶子 ++自己的父节点计数; 当父节点收齐 2 个, 由"最后一个
                       到达者"代表父节点继续向上 ++祖父节点 ... 递归到根 (深度 lg P)
       释放 (release): 根置 flag -> 逐层向下把释放信号广播到子节点 (深度 lg P)
       span = 2 * lg(P) 次节点操作;  总流量仍是 O(P) (但分布到 P-1 个节点上, 不集中在 1 个)
       注意: 树形结构在【总线】上无收益(总线本来就是串行的), 在点对点网络(树/环/Mesh)上才赢
```

2.13 地基：一致性、一致性模型与 fence —— 同步原语为什么必须谈内存序

这一节把配套 CS149 讲义的要点与本讲衔接起来：**锁与屏障能工作，前提是"临界区内的读写不会被重排到临界区之外"**。

- **Cache coherence（缓存一致性）**：只约束**同一个地址**的读写——"所有处理器必须对地址 X 的读写顺序达成一致"，即存在一条把 X 上所有操作串起来的**假设时间线**（hypothetical serial order），与各处理器的观察一致。MSI/MESI 通过两条不变量保证它：**SWMR（Single-Writer, Multiple-Reader）** 与 **写序列化（write serialization：BusRd/BusRdX 的数据由 M 态缓存提供，总线串行化所有事务）**。
- **表 2：MSI 失效协议（invalidate protocol）状态迁移表（A / B = 观察到动作 A 则执行动作 B；`--` 表示无总线动作）**

当前状态	事件	下一状态	总线动作	说明
I (Invalid)	PrRd（处理器读）	S	`BusRd`	取一份只读副本，即使只有我一个人缓存也进 S
I	PrWr（处理器写）	M	`BusRdX`	写前必须拿到独占所有权
S (Shared)	PrRd	S	`--`	本地命中
S	PrWr	M	`BusRdX`	**升级（upgrade）**：即使本地命中也要上总线让别人失效
S	BusRd（别人读）	S	`--`	多人共享读
S	BusRdX（别人要写）	I	`--`	我被失效
M (Modified)	PrRd	M	`--`	本地命中
M	PrWr	M	`--`	本地命中（脏行不写回，write-back 缓存）
M	BusRd	S	`BusWB`	别人要读：我把脏行写回并提供数据，自己降为 S

当前状态	事件	下一状态	总线动作	说明
M	BusRdX	I	`BusWB`	别人要写：我写回并失效

- **MSI 的已知低效**：对"先读一个地址、再写它"这个**最常见**的模式，MSI 需要**两次互连事务**（`BusRd` 进 S，再 `BusRdX` 升级到 M）——**即使程序完全没有共享**。于是有了 **MESI**：增加 **E（Exclusive clean，独占但干净）** 状态，把"独占性"与"脏"解耦，**E → M 的升级不需要任何总线事务**。这就是"实现同步原语时，一次 `fetch_add` 究竟是 2 个事务还是 1 个事务"的差别。
- **Memory consistency（内存一致性）**：约束**不同地址**的读写**相对顺序**（一个线程的访存何时对别的线程可见）。讲义用"四种顺序"刻画：
- `WX→RY`（写后读）、`RX→RY`（读后读）、`RX→WY`（读后写）、`WX→WY`（写后写）。
- **顺序一致性（SC, Sequential Consistency, Lamport 1976）**：所有操作按某种全序执行，如同操作单一份共享内存，且每个线程的操作保持**程序序**——**四种顺序全部维持**。
- **松弛（relaxed）** 的动机是性能：写要 100 多个周期，而"写 A 与随后读 B 无关"，没必要等写完成。**写缓冲（write buffer）** 就是硬件在松弛 `WX→RY`，于是需要**更弱的内存模型**。
- **表 3：内存一致性模型对比（讲义 slide 44–50）**

模型	被松弛的顺序	是否允许"读到更新的值"	典型硬件/说明
SC（Sequential Consistency）	无	—	全部四序保持；编程最直观、性能约束最强
TSO（Total Store Order）	仅 `WX→RY`	本处理器可把自己的读提前到自己的写之前；**其他**处理器在该写被所有处理器看到前读不到新值	x86 使用**不完整规格化**的 TSO；同一线程的写之间仍保持程序序
PC（Processor Consistency）	仅 `WX→RY`	**任何**处理器都可以在该写被所有人看到前读到新值	比 TSO 更弱
PSO（Partial Store Ordering）	追加 `WX→WY`	写之间可重排（例如一个 miss 一个 hit）	典型反例：`A=1; flag=1;` 后 `while(flag==0); print A;` 可能打印旧 A
WO / RC（Weak Ordering / Release Consistency）	数据操作全部可重排	无任何数据顺序保证	读尽量早、写尽量晚，以最大化隐藏延迟；同步操作之间才有顺序

- **同步原语与 fence 的关系**：**每种架构都提供同步原语使内存序变严格。Fence（内存屏障，memory barrier）** 阻止重排，但**昂贵**（"它之后的所有访存都必须等它之前的所有访存完成"）。x86 提供 `mm_lfence`（load fence）、`mm_sfence`（store fence）、`mm_mfence`（mem fence）；ARM 的一致性模型非常松弛（这也是 ARM 上 `dmb/isb` 语义讨论多的原因）。
- **数据竞争与 DRF 契约（本讲最实用的结论）**：
- **冲突访问（conflicting accesses）**：两个访问命中**同一地址**且**至少一个写**；
- **非同步程序（unsynchronized program）**：冲突访问之间没有同步操作（fence、带 release/acquire 语义的操作、屏障等）排序 ⇒ **程序含数据竞争 ⇒ 结果依赖于处理器的相对速度（不确定）**；
- **已同步程序（synchronized program）**：**无数据竞争**，则**即使硬件是弱模型，程序也表现出 SC 的结果**（"SC for DRF"）；
- 现代语言（C11/C++11）与 Java 5 都保证 **DRF 程序的顺序一致性**，编译器负责插入必要的同步指令来适配硬件模型；**程序一旦有数据竞争，则不作任何保证**。这解释了为什么我们**不**手写"裸的共享变量标志"，而要用锁/屏障/`atomic`——**把复杂性封装在库里，是唯一的工程正解**。

---

3. 代码示例与性能分析

> 三个示例均可独立编译运行。所有示例都使用 **release 优化编译（`-O3` / `-O2`）**：同步基准若不开优化，循环开销会淹没同步代价，结论会失真。

3.1 示例 1：五种自旋锁的完整实现与吞吐量对比

```
// ============================================================================
// lockbench.cpp —— 五种自旋锁在同一临界区上的吞吐量/延迟对比
// 编译(release): g++ -O3 -std=c++17 -pthread lockbench.cpp -o lockbench
//   (先确认生成代码: g++ -O3 -S lockbench.cpp 或 objdump -d 查看 lock xchg/lock cmpxchg)
// 运行: ./lockbench 16 200000      # 16 线程, 每线程 20 万次"加锁 -> 临界区 -> 解锁"
// ============================================================================
#include <atomic>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <thread>
#include <vector>

static constexpr int CACHE_LINE = 64;

// ---------------- 1) Test-and-Set 锁: 每次尝试都写, 每次都制造一次总线事务 ----------------
struct TASLock {
    alignas(CACHE_LINE) std::atomic<int> v{0};
    void lock() {
        // exchange 在 x86 上编译为 lock xchg, 目标行必须进入 M 态 -> BusRdX + 失效
        while (v.exchange(1, std::memory_order_acquire) != 0) { /* 失败继续抢 */ }
    }
    void unlock() { v.store(0, std::memory_order_release); }
};

// ---------------- 2) Test-and-Test-and-Set: 先在本地副本上只读自旋 ----------------
struct TTASLock {
    alignas(CACHE_LINE) std::atomic<int> v{0};
    void lock() {
        for (;;) {
            // 只读自旋: 命中本核 L1, 互连上零事务(这就是 TTAS 的关键)
            while (v.load(std::memory_order_relaxed) != 0) { }
            // 观察到锁空闲才做原子抢锁
            if (v.exchange(1, std::memory_order_acquire) == 0) return;
        }
    }
    void unlock() { v.store(0, std::memory_order_release); }
};

// ---------------- 3) 指数退避 TAS: 失败后退避, 减少无谓事务 ----------------
struct BackoffLock {
    alignas(CACHE_LINE) std::atomic<int> v{0};
    void lock() {
        int amount = 1;
        for (;;) {
            if (v.exchange(1, std::memory_order_acquire) == 0) return;
            for (volatile int i = 0; i < amount; ++i) { }   // delay(amount)
            if (amount < 4096) amount <<= 1;                // 上限, 避免无限退避
        }
    }
    void unlock() { v.store(0, std::memory_order_release); }
};

// ---------------- 4) Ticket 锁: 取号 + 只读等号, FIFO 公平 ----------------
struct TicketLock {
    alignas(CACHE_LINE) std::atomic<uint32_t> next_ticket{0};
    // 与 next_ticket 分处不同 cache line: 否则每次"取号"都会让所有等待者的只读副本失效
    alignas(CACHE_LINE) std::atomic<uint32_t> now_serving{0};
    void lock() {
        uint32_t my = next_ticket.fetch_add(1, std::memory_order_relaxed);
        while (now_serving.load(std::memory_order_acquire) != my) { }
    }
    void unlock() { now_serving.fetch_add(1, std::memory_order_release); }
};

// ---------------- 5) MCS 队列锁: 每人在自己的节点上自旋, O(1) 流量 + FIFO ----------------
struct MCSLock {
    struct Node {
        alignas(CACHE_LINE) std::atomic<Node*> next{nullptr};
        alignas(CACHE_LINE) std::atomic<bool>  locked{false};   // 独占一行: 唤醒只碰它
```

```
    };
    alignas(CACHE_LINE) std::atomic<Node*> tail{nullptr};       // 全局锁 = 队尾指针

    void lock(Node* me) {
        me->next.store(nullptr, std::memory_order_relaxed);
        me->locked.store(true, std::memory_order_relaxed);
        Node* prev = tail.exchange(me, std::memory_order_acq_rel);   // 挂到队尾
        if (prev == nullptr) return;                                  // 队空 -> 直接持有
        prev->next.store(me, std::memory_order_release);              // 让前驱能找到我
        while (me->locked.load(std::memory_order_acquire)) { }        // 自旋在【自己的】节点
    }
    void unlock(Node* me) {
        if (me->next.load(std::memory_order_acquire) == nullptr) {
            // 无人排队: 尝试把 tail 从"我"改成 NULL (cmpxchg 语义, 见讲义 slide 22)
            Node* expected = me;
            if (tail.compare_exchange_strong(expected, nullptr,
                                             std::memory_order_acq_rel)) return;
            // 失败: 有人正在挂入, 等它把 next 指针写出来
            while (me->next.load(std::memory_order_acquire) == nullptr) { }
        }
        // 精确唤醒下一个: 只写它的 locked 字段 (O(1) 互连流量)
        me->next.load(std::memory_order_acquire)->locked.store(false, std::memory_order_release);
    }
};

// MCS 需要每线程一个常驻 node; 用 thread_local 放在线程本地存储里(天然不共享)
struct MCSAdapter {
    MCSLock impl;
    static thread_local MCSLock::Node node;
    void lock()   { impl.lock(&node); }
    void unlock() { impl.unlock(&node); }
};
thread_local MCSLock::Node MCSAdapter::node;

// ---------------- 基准骨架 ----------------
// 临界区: 对一条共享且【已填充】的计数器做一次读-改-写
template <typename Lock>
static void worker(Lock* lk, long long* cs, long long iters, long long* sink) {
    long long acc = 0;
    for (long long i = 0; i < iters; ++i) {
        lk->lock();
        *cs += 1;            // 临界区: 一次共享写 (L1 命中 + 独占写)
        lk->unlock();
        acc += i;            // 临界区外的工作: 只为了让循环不被优化掉
    }
    *sink += acc;
}

template <typename Lock>
static void bench(const char* name, int P, long long iters) {
    static Lock lk;                                        // 每种锁一个实例, 反复复用
    alignas(CACHE_LINE) static long long cells[8 * 64];    // 每个线程 64 B 独占一行
    std::vector<std::thread> th;
    std::vector<long long>   sink(P, 0);
    const long long total = iters * (long long)P;

    auto t0 = std::chrono::steady_clock::now();
    for (int t = 0; t < P; ++t)
        th.emplace_back(worker<Lock>, &lk, &cells[t * 8], iters, &sink[t]);
    for (auto& x : th) x.join();
    auto t1 = std::chrono::steady_clock::now();

    double us = std::chrono::duration<double, std::micro>(t1 - t0).count();
    long long sum = 0;
    for (int t = 0; t < P; ++t) sum += cells[t * 8];
    printf("%-18s P=%2d  交接次数=%10lld  用时=%10.1f us  ns/次交接=%6.1f  M交接/s=%8.2f  (校验 %lld)\n",
           name, P, total, us, us * 1000.0 / (double)total,
           (double)total / us, sum);
}

int main(int argc, char** argv) {
    const int      P     = (argc > 1) ? atoi(argv[1]) : 8;
```

```
    const long long iters = (argc > 2) ? atoll(argv[2]) : 200000;
    printf("== 自旋锁对比: %d 线程, 每线程 %lld 次临界区进入 (临界区仅一次共享写) ==\n", P, iters);
    bench<TASLock>    ("TAS",         P, iters);
    bench<TTASLock>   ("TTAS",        P, iters);
    bench<BackoffLock>("Backoff-TAS", P, iters);
    bench<TicketLock> ("Ticket",      P, iters);
    bench<MCSAdapter> ("MCS",         P, iters);
    return 0;
}
```

【代码做什么？】 1. **`TASLock`**：每次循环都执行一次 `exchange`。无论锁是否空闲，这条指令都要让目标行进入本核独占态——在 x86 上是 `lock xchg`。这就是讲义里"每个等待者每次测试都产生一次失效"的直接映射。 2. **`TTASLock`**：外层无限循环 + 内层**只读自旋**。内层 `load` 命中本地缓存副本（S 态），**不产生互连事务**；一旦看到 0，才 `exchange` 抢锁；抢失败（说明被别人先抢了）则回到只读自旋。 3. **`BackoffLock`**：TAS + 指数退避；`for (volatile int i...)` 是"廉价 delay"，`amount` 倍增并设上限，避免退避到永远。 4. **`TicketLock`**：`fetch_add` 取号（一次原子 RMW），然后**只读**等 `now_serving` 追上自己的号；`unlock` 就是 `now_serving++`——**一次写**。字段分置不同 cache line，避免"取号时顺带踢掉所有等待者的只读副本"。 5. **`MCSAdapter`**：`thread_local` 节点 + 队列；`lock` 用 `exchange` 把自己挂到队尾；若队空直接持有；否则让前驱指向自己并在**自己的 node 上自旋**。`unlock` 若发现无人排队则用 `compare_exchange_strong` 尝试把 tail 置空（对应讲义伪代码里的 `CMPXCHG(glock, mlock, NULL)`），否则**只写下一个等待者的 `locked` 字段**。 6. **`bench<>()`**：P 个线程各自进入 `iters` 次临界区，临界区是"对一条独占 cache line 的计数器加一"，因此**临界区本身几乎零成本**，测出来的基本就是**同步开销**（与讲义"把临界区时间扣掉再看曲线"的做法一致）。最后把每个线程的计数器求和打印，用于验证互斥**没有坏**（结果必须恰好等于 `iters × P`）。

【并行机制与性能解说】 – **线程与共享数据**：P 个 OS 线程，通常被调度到不同核上（`-pthread`）。共享的是锁对象与一条被填充过的计数器；`MCSAdapter::node` 是 `thread_local`，**不在线程间共享**（这正是 MCS 的要点）。 – **Work / Span / 并行度（本讲的第一个硬结论）**：设总进入次数 `N = iters × P`，一次原子操作的成本为 ℓ，一次临界区成本为 `c`，一次**竞争下的锁交接**延迟为 `h`（含互连往返与排队）。 – **Work（总工作量）** = `N × (ℓ + c)`； – **Span（关键路径）** = `N × (h + c)`：**所有 N 次临界区都被互斥强制串行**； – **并行度 = Work / Span = (ℓ + c) / (h + c)**。取 `ℓ = 20` 周期（无竞争）、`h = 150` 周期（P 个线程抢）、`c = 30` 周期：并行度 = `(20+30)/(150+30) ≈ 0.28`。**并行度 < 1 意味着这不是"加速多少"的问题，而是"比单线程更慢"的问题**——加线程只会让交接更贵（`h` 随 P 增长：TAS 的 `h ∝ P`，TTAS/Ticket 的 `h` 也有 O(P) 流量项，MCS 的 `h ≈ 常数`）。这正是讲义那句"简单 TAS 锁：低竞争低延迟、高流量、扩展性差"的定量表达。 – 由 Work/Span 得到的**可扩展性上限**：临界区吞吐量 ≈ `1/(h + c)`，**与 P 无关**。想提高它，只能 (a) 缩短临界区 `c`（细粒度锁）、(b) 降低交接延迟 `h`（换 MCS/数组锁）、或 (c) 消除互斥（无锁算法，下一讲）。 – **瓶颈定位**： – **TAS**：瓶颈是 **互连带宽 + 总线仲裁**（失败尝试制造的事务与等待时长成正比）； – **TTAS**：瓶颈是**每次释放的惊群**（P−1 个 BusRdX + P−1 次重新取数）； – **Backoff**：瓶颈是**空窗延迟**（锁空闲却没人来拿）与**不公平**； – **Ticket**：瓶颈是**P−1 个读者在每次释放后重新读 `now_serving`**（O(P) 流量），但顺序公平； – **MCS**：流量 O(1)、无惊群，瓶颈转移到**内存延迟本身**（一次交接 ≈ 一次 cache line 的核间传递 ≈ 65–75 周期 + 端点处理），这也是理论下限。

3.2 示例 2：两种屏障 —— 集中式 sense reversal vs 传播式（dissemination）屏障

```cpp
// ============================================================================
// barriers.cpp —— 集中式屏障(讲义 slide 27 的 sense reversal) vs O(log P) 屏障
// 编译(release): g++ -O3 -std=c++17 -pthread barriers.cpp -o barriers
// 运行: ./barriers 8 200000     # 8 线程, 每线程 20 万轮 "计算 + 屏障"
// ============================================================================
#include <atomic>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <thread>
#include <vector>

static constexpr int CACHE_LINE = 64;

// ---------------- (a) 集中式屏障 + sense reversal ----------------
// 对应讲义 slide 27: "Sense reversal optimization results in one spin instead of two"
class SenseBarrier {
public:
    explicit SenseBarrier(int P) : P_(P) {}
    void wait(bool& local_sense) {
        local_sense = !local_sense;                                  // 翻转本线程的 sense
        if (cnt_.fetch_add(1, std::memory_order_acq_rel) == P_ - 1) { // 我是最后一个到达者
            cnt_.store(0, std::memory_order_relaxed);                // 复位计数器(为下一代)
            sense_.store(local_sense, std::memory_order_release);    // 用"翻转"代替"清零"
        } else {
            while (sense_.load(std::memory_order_acquire) != local_sense) { }  // 只自旋一次
        }
    }
private:
    const int P_;
    alignas(CACHE_LINE) std::atomic<int>  cnt_{0};      // 计数器单独一行(被 P 个线程写)
    alignas(CACHE_LINE) std::atomic<bool> sense_{false}; // flag 单独一行(被 P-1 个线程读)
};

// ---------------- (b) 传播式屏障 (dissemination barrier): 无锁, span = O(log P) ----------------
// 讲义 slide 29 讲的是 combining tree(合并树); 这里实现一个同样具有 lg(P) span 的
// 无锁屏障, 用来定量对比"O(P) span vs O(log P) span"。要求 P 为 2 的幂。
class DissemBarrier {
public:
    explicit DissemBarrier(int P) : P_(P), logP_(0) {
        while ((1 << logP_) < P_) ++logP_;              // P 向上取整到 2 的幂
        flags_.resize(P_);
        for (auto& row : flags_) {
            row = std::vector<std::atomic<unsigned>>(logP_);
            for (auto& f : row) f.store(0, std::memory_order_relaxed);
        }
    }
    // 必须由同一个线程反复调用; rnd 是该线程私有的"屏障轮次"(从 0 开始, 每轮 +1)
    void wait(int t, unsigned& rnd) {
        ++rnd;                                           // 单调递增的轮次号, 避免 ABA
        for (int r = 0; r < logP_; ++r) {
            const int partner = (t + (1 << r)) % P_;      // 我通知"后面第 2^r 个"
            flags_[partner][r].store(rnd, std::memory_order_release);
            // 我等"前面第 2^r 个"通知我: 它写的正是 flags_[t][r]
            while (flags_[t][r].load(std::memory_order_acquire) != rnd) { }
        }
    }
private:
    int P_, logP_;
    std::vector<std::vector<std::atomic<unsigned>>> flags_;   // flags_[目标线程][轮次]
};

// 两个屏障统一成 wait(tid) 接口(thread_local 保存每线程私有状态)
struct SenseAdapter {
    SenseBarrier impl;
    static thread_local bool sense;
    explicit SenseAdapter(int P) : impl(P) {}
    void wait(int) { impl.wait(sense); }
};
thread_local bool SenseAdapter::sense = false;   // 初值 false -> 第 1 次屏障等的值是 1
```

```
struct DissemAdapter {
    DissemBarrier impl;
    static thread_local unsigned rnd;
    explicit DissemAdapter(int P) : impl(P) {}
    void wait(int tid) { impl.wait(tid, rnd); }
};
thread_local unsigned DissemAdapter::rnd = 0;

// ---------------- 基准: K 个 "计算阶段 + 屏障" ----------------
template <typename Barrier>
static void bench_barrier(const char* name, int P, long long phases) {
    Barrier bar(P);
    std::vector<std::thread> th;
    std::vector<long long>   acc(P, 0);

    auto t0 = std::chrono::steady_clock::now();
    for (int t = 0; t < P; ++t) {
        th.emplace_back([&bar, t, phases, &acc] {
            long long a = 0;
            for (long long k = 0; k < phases; ++k) {
                for (int i = 0; i < 200; ++i) a += (i ^ t);   // 模拟该阶段的并行计算
                bar.wait(t);                                  // 阶段边界: 屏障
            }
            acc[t] = a;
        });
    }
    for (auto& x : th) x.join();
    auto t1 = std::chrono::steady_clock::now();

    double us = std::chrono::duration<double, std::micro>(t1 - t0).count();
    long long chk = 0; for (auto v : acc) chk += v;
    printf("%-22s P=%2d  阶段数=%8lld  总用时=%10.1f us  ns/屏障=%8.1f  (校验 %lld)\n",
           name, P, phases, us, us * 1000.0 / (double)phases, chk);
}

int main(int argc, char** argv) {
    const int       P      = (argc > 1) ? atoi(argv[1]) : 8;
    const long long phases = (argc > 2) ? atoll(argv[2]) : 200000;
    printf("== 屏障对比: %d 线程 (必须为 2 的幂), 每线程 %lld 个阶段 ==\n", P, phases);
    bench_barrier<SenseAdapter> ("SenseReversal", P, phases);
    bench_barrier<DissemAdapter>("Dissemination", P, phases);
    return 0;
}
```

【代码做什么？】 1. **`SenseBarrier::wait`**：先把**每线程私有**的 `local_sense` 翻转；再用一次原子 `fetch_add` 判断自己是不是最后一个到达者。是 → 复位计数器并把 `sense_` 置为本代的值（**"翻转"而不是"清零"，因此不需要等价于 leave_counter 的第二轮自旋**）；不是 → 自旋等待 `sense_` 变成自己这一代的值。 2. **`DissemBarrier::wait`**：共 `log2(P)` 轮。第 r 轮中，线程 `t` 通知 `(t + 2^r) mod P`，并等待 `(t − 2^r) mod P` 的通知（表现上就是"等 `flags_[t][r]` 变成本轮轮次号"）。因为每轮让线程知道的"已到达者集合"翻倍，`log2(P)` 轮后每个线程都知道**全体已到达**。用**单调递增的轮次号 `rnd`** 而非 0/1，天然规避"快线程领先一代导致误读"的 ABA 问题。 3. **`bench_barrier`**：每轮先做一小段纯计算（`200` 次异或累加，代表阶段内的并行工作 `T_c`），然后调用屏障；统计总时间与"每次屏障的纳秒数"。两个适配器用 `thread_local` 持有每线程私有状态（sense / 轮次号），这正是讲义强调的"**local_sense 必须是每处理器私有的**"。

【并行机制与性能解说】 – **线程与数据布局**：P 个线程；`SenseBarrier` 的 `cnt_` 与 `sense_` 各自独占 cache line（防止"每次计数更新都踢掉所有等待者的 flag 只读副本"，即讲义 slide 25 的那个反问）；`DissemBarrier` 的每个 flag 位置只被"一个写者 + 一个读者"访问，**天生无伪共享**。 – **Work / Span / 并行度**：设每阶段并行计算量为 `T_c`，屏障的临界路径长度为 `T_b`，阶段数 `K`： – **Work** = `K × (T_c + W_b)`，其中 `W_b` 是屏障的总工作量：集中式 `W_b = θ(P)`（每个线程一次原子加 + 若干读），传播式 `W_b = θ(P log P)`（P 个线程 × log P 次写 + log P 次读）。 – **Span** = `K × (T_c/P + S_b)`，其中 **`S_b`（屏障的关键路径）**：集中式 = **Θ(P)**（所有线程在**同一个原子变量**上排队，`P` 次原子 RMW 串行 + flag 广播的 O(P) 流量）；传播式 = **Θ(log P)**（log P 轮，每轮一次 release 存 + 一次 acquire 读，互不依赖、可全并行）。 – **并行度 = Work / Span**：集中式 ≈ `(T_c + θ(P)) / (T_c/P + θ(P))` → 当 `P` 增大时趋近 **O(1)**（"屏障期间整机空转"）；传播式 ≈ `θ(P log P) / θ(log P)` = **Θ(P)**（**屏障本身也并行**）。 – **瓶颈与可扩展性上限（定量结论）**：屏障把速度上限钉死在 `T_c / T_b` 附近： 设 `T_c = 10,000` 周期（每阶段并行工作量）、`P = 64`、每次"竞争下的原子 RMW + 排队"约 200 周期： – 集中式：`S_b ≈ P × 200 = 12,800` 周期 → 单阶段时间 = `10,000/64 + 12,800 ≈ 12,956` 周期 → 加速比 = `10,000 / 12,956 ≈ 0.77`（**比单线程还慢！**） – 合并树（讲义方案，span = `2·lg P·L_node`）：`S_b ≈ 2 × 6 × 200 = 2,400` → 单阶段 = `156 + 2,400 = 2,556` → 加速比 `≈ 3.9×` – 传播式：`S_b ≈ log2(64) × (75 + 75) = 900` → 单阶段 = `156 + 900 = 1,056` → 加速比 `≈ 9.5×` – 理论上限（无屏障）：`64×` → **结论**：屏障开销把可扩展性上限压到 `T_c/T_b`（大 P 时加速比 ≈ `T_c/T_b`），因此**减少屏障次数、让每次屏障之间"干活足够多"比任何微优化都重要**。这与讲义"集中式屏障 span 是 O(P)，合并树

是 lg(P)"的定性结论完全一致；同时注意讲义的限制条件：**在总线上树形结构帮助有限**（总线本身把 O(P) 流量串行化了，树只是把竞争点从 1 个变量分散到 P–1 个变量，总线上仍要排队），**只有在点对点互连上才能把 lg(P) 的 span 变成真实收益**。

3.3 示例 3：伪共享（false sharing）—— 一致性协议写给软件的最贵账单

```cpp
// ============================================================================
// falsesharing.cpp —— 伪共享的实测: 未填充 vs 已填充 (结构取自 CS149 讲义 slide 17)
// 编译(release): g++ -O2 -std=c++17 -pthread falsesharing.cpp -o falsesharing
// 运行: ./falsesharing 8 40000000     # 8 线程, 每线程 4000 万次自增
// 说明: CS149 讲义在 4 核机器上、8 线程、同一份测试下实测 14.2 s vs 4.7 s
// ============================================================================
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <pthread.h>

static constexpr int CACHE_LINE_SIZE = 64;
static constexpr int MAX_THREADS     = 64;
static long MANY_ITERATIONS = 40000000;   // 由命令行覆盖

// 每个线程一个计数器, 并把它填满到独占一条 cache line
struct padded_t {
    int  counter;
    char padding[CACHE_LINE_SIZE - sizeof(int)];
};

// 工作线程: 反复自增【只属于自己】的计数器
static void* worker(void* arg) {
    volatile int* counter = (int*)arg;      // volatile: 强制每次都真的读写内存
    for (long i = 0; i < MANY_ITERATIONS; i++)
        (*counter)++;
    return NULL;
}

// ---- 版本 1: 未填充。相邻计数器落在同一条 cache line ⇒ 伪共享 ----
static double test1(int num_threads) {
    pthread_t threads[MAX_THREADS];
    int       counter[MAX_THREADS];
    for (int i = 0; i < num_threads; i++) counter[i] = 0;

    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < num_threads; i++)
        pthread_create(&threads[i], NULL, &worker, &counter[i]);
    for (int i = 0; i < num_threads; i++)
        pthread_join(threads[i], NULL);
    auto t1 = std::chrono::steady_clock::now();

    long long sum = 0; for (int i = 0; i < num_threads; i++) sum += counter[i];
    double s = std::chrono::duration<double>(t1 - t0).count();
    printf("未填充(sizeof(int)=%zu B, 8 个计数器共用 1 条 64 B 行): %8.3f s  (总和=%lld)\n",
           sizeof(int), s, sum);
    return s;
}

// ---- 版本 2: 已填充。每个计数器独占一条 cache line ⇒ 无伪共享 ----
static double test2(int num_threads) {
    pthread_t threads[MAX_THREADS];
    padded_t  counter[MAX_THREADS];
    for (int i = 0; i < num_threads; i++) counter[i].counter = 0;

    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < num_threads; i++)
        pthread_create(&threads[i], NULL, &worker, &(counter[i].counter));
    for (int i = 0; i < num_threads; i++)
        pthread_join(threads[i], NULL);
    auto t1 = std::chrono::steady_clock::now();

    long long sum = 0; for (int i = 0; i < num_threads; i++) sum += counter[i].counter;
    double s = std::chrono::duration<double>(t1 - t0).count();
    printf("已填充(每个计数器独占 1 条 64 B 行):                  %8.3f s  (总和=%lld)\n",
           s, sum);
    return s;
}

int main(int argc, char** argv) {
    int n = (argc > 1) ? atoi(argv[1]) : 8;
```

```
    if (argc > 2) MANY_ITERATIONS = atol(argv[2]);
    printf("== 伪共享实测: %d 线程, 每线程 %ld 次 volatile 自增 ==\n", n, MANY_ITERATIONS);
    double s1 = test1(n);
    double s2 = test2(n);
    printf("--> 未填充/已填充 = %.2fx\n", s1 / s2);
    return 0;
}
```

【代码做什么？】 两个版本工作线程完全相同：每个线程对**自己的** `int` 做 `MANY_ITERATIONS` 次 `volatile` 自增。唯一差别是**布局**：版本 1 里 8 个 `int` 紧挨着（8 个计数器共享一条 64 B cache line）；版本 2 里每个计数器用 `char padding[60]` 撑满一条 cache line。打印总和用于验证结果正确（两者都应等于 `线程数 × MANY_ITERATIONS`）。

【并行机制与性能解说】 – **为什么"没有共享却跑了通信"**：一致性协议的粒度是 **cache line**，不是变量。版本 1 中 P1 写 `counter[1]` 与 P2 写 `counter[2]` 落在**同一条行**上：P1 要写，必须先 `BusRdX` 把行**独占**并让 P2 的副本失效；P2 要写，又得把行抢回去。**行在两个核之间来回弹跳（ping-pong）**，每次弹跳 ≈ 75 周期（"L3 命中、行在另一个核里是 modified"），而**这完全不是算法需要的数据通信**——讲义称之为 **artifactual communication（人为通信）**，根因是 **cache line（64 B）远大于被访问的数据（4 B）**。 – **Work / Span / 并行度**： – **Work** = `P × N` 次自增（`N = MANY_ITERATIONS`），这是**真并行**的工作量； – **理想 Span**（各线程完全独立）= `N` 次自增（一条 chain）→ **并行度 = P**； – **伪共享下的实际 Span**：每次 cache line 所有权转移会**串行化互连上的写**。若最坏情况下每次转移只允许一个自增发生（对手立刻把行抢走），则 Span ≈ `P × N × (T_transfer)`，**并行度退化为 ≈ 1**——与"加了一把锁"没有区别。**这就是伪共享最危险的地方：程序里没有任何锁，性能却像全程串行。** – **定量核对（用 AMAT 模型反推）**：AMAT = `Σ(访问频率 × 访问延迟)`。 – 已填充版本：全部命中本地 L1（~4–5 周期），AMAT ≈ 5 周期； – 未填充版本：设"跨越互连取行"的比例为 `f`，AMAT ≈ `(1-f)×5 + f×75`。 – 讲义实测比值为 `14.2 / 4.7 = 3.02×`。令 AMAT 之比等于 3.02：`[(1-f)·5 + 75f] / 5 = 3.02 ⇒ 5 + 70f = 15.1 ⇒ f ≈ 0.144`。 – 即：**约 14% 的访问变成了跨核行转移**，就足以让程序慢 3 倍。若按"每次自增都跨核"的极端模型（f = 1），比值会达到 `75/5 = 15×`——实测只有 3×，说明**同一线程连续自增会在一段时间内独占该行**（自增之间的循环开销、store buffer 合并、以及 SMT 使 8 个线程实际分布在 4 个物理核的 4 个 L1 上，都降低了真实转移频率）。这个"模型高估、实测更小"的差距本身很有教学价值：**伪共享的伤害取决于"行在核间转移的频率"，而不是"有多少线程共享了这条行"。** – **瓶颈**：互连带宽 + 行转移延迟；**不均衡**：8 线程抢 4 个 L1，某些核承担双倍流量；**修复手段**：填充（padding，本示例）、每个线程写入**真私有**的局部变量（回归时才合并，即 reduction 模式）、调整数据布局（SoA 中的分块、每个线程一块 `block_size ≥ cache line` 的区间）。

---

4. 性能模型与复杂度分析

4.1 三个必须同时写的"预算约束"

同步原语的性能从来不是单一指标。工程上必须同时盯住三条预算：

1. **延迟预算（latency budget）**：一次 `acquire` 在**无竞争**下的关键路径 = 一次原子 RMW 的互连往返（~65–75 周期，行被他人共享/持有时更贵）；
2. **带宽预算（bandwidth budget）**：一次同步事件在互连上搬运多少字节。设互连等效数据带宽 `B_bus = 8 B/cycle`，则一条 64 B cache line = **8 个互连周期**的占用；若某算法每次交接要搬 `m` 条行，则每次交接占用 `8m` 个互连周期；
3. **串行化预算（serialization budget）**：所有线程是否在同一地址上排队？如果是，**span 就是 O(P)**，与流量无关（讲义 slide 28 的核心）。

4.2 表 4：锁算法的定量对比（参数为示意值，趋势比绝对值重要）

共同假设：`P = 16` 个线程全部争抢同一把锁；`B_bus = 8 B/cycle`（等效，64 B 行 = 8 周期）；一次失败 TAS 的往返（含排队）`R_eff`；临界区持锁时长 `H = 400` 周期；一次"行转移"提供 64 B。

锁	每次交接的互连字节数（估算）	等效互连周期	每次交接产生的失效数	随 P 的增长	公平性	空间
TAS	失败尝试 ∝ `(P-1)·H/R_eff`；`R_eff = (P-1)/0.125 = 120`，失败次数 = `15×400/120 = 50` → `(50+2)×64 ≈ 3328 B`	**≈ 416 周期**	O(P) **每次尝试**	**≈ O(P²)**（流量随 P 与等待时长双重增长）	❌ 无	4 B
TTAS	每次释放：15 次失效消息(8 B) + 15 次抢锁 BusRdX(64 B) + 14 次失败者重新取数(64 B) ≈ **1976 B**	≈ 247 周期	**O(P)**（每个等待者每次释放 1 次）	O(P)（讲义：全体流量 O(P²)）	❌ 无	4 B
Backoff-TAS	失败次数随退避上限下降（退避期间不发事务），但引入"锁空闲无人取"的空窗	低于 TAS，高于 TTAS	O(P)/log	亚线性（受退避上限制约）	❌ **严重不公平**（新来者退避更短）	4 B

锁	每次交接的互连字节数（估算）	等效互连周期	每次交接产生的失效数	随 P 的增长	公平性	空间
Ticket	释放：15 次失效消息(8 B) + 15 个等待者重读(64 B) + 到来时 1 次取号(64 B) ≈ **1144 B**	≈ 143 周期	**1 次写 → O(P) 个读者失效**	**O(P)**（但常数远小于 TAS/TTAS）	✅ FIFO	8 B（建议分两行）
Array-based（带填充）	释放只写 2 个槽位（自己 + 下一个），各占独立行 ≈ **128 B**	≈ 16 周期	**O(1)**	**O(1)/次释放**	✅ FIFO	**P × 64 B**
MCS	释放只写下一个等待者的私有节点 ≈ **64 B**	**≈ 8 周期**	**O(1)**	**O(1)/次释放**	✅ FIFO	1 指针/锁 + 1 节点/线程

读法：这张表把讲义的定性结论变成了同一把尺子上的数字。三个关键洞察： 1. **TAS 的问题不是"每次多花几个周期"，而是"流量与等待时长成正比"**（96% 的互连流量是失败的尝试）；这解释了为什么讲义 slide 13 的曲线随 P 陡增； 2. **TTAS 把"持续尝试"变成"每次释放尝试一次"**，流量降一个数量级，但仍是 O(P)/次释放； 3. **要把 O(P)/次释放 变成 O(1)，必须让等待者不共享同一个地址**（数组锁：不同槽位；MCS：不同节点）。这时交接的互连代价降到"一条 cache line"，即 **≈ 8 个互连周期 ≈ 理论下限**。

4.3 数值算例 A：16 线程下三种锁的临界区吞吐量上限

设总线数据带宽等效 `B = 8 B/cycle`、时钟 1.6 GHz、临界区 `H = 400` 周期（= 250 ns）：

锁	每次交接互连占用	交接延迟下界 `h`	临界区吞吐上限 `1/(h+H)`	16 核理想值（无锁开销）
TAS	416 周期	416	`1/816` 周期 ≈ **1.96 M/s**	4.0 M/s（`1/250 ns`）
TTAS	247 周期	247	`1/647` ≈ **2.47 M/s**	4.0 M/s
Ticket	143 周期	143	`1/543` ≈ **2.95 M/s**	4.0 M/s
MCS	8 周期 ≈ 交接延迟本身 → 取实测下限 ~75 周期	75	`1/475` ≈ **3.37 M/s**	4.0 M/s

结论：在"临界区 250 ns"这个尺度上，TAS 把有效吞吐砍到理想值的 **49%**，TTAS 61%，Ticket 74%，MCS 84%。**注意 P 增大时 TAS 会继续恶化（流量 O(P²)），而 MCS 的 `h` 基本不随 P 变**——这就是"可扩展性"的具体含义：不是"现在快多少"，而是"P 翻倍后还剩下多少"。

4.4 数值算例 B：屏障开销如何给加速比设上限（Amdahl 视角）

设程序分为 `K` 个阶段，每阶段并行工作量 `T_c` 周期，阶段间必须同步，屏障的关键路径 `T_b`：

```
T(P, K) = K · ( T_c / P + T_b )
Speedup(P) = (K · T_c) / (K · (T_c/P + T_b)) = 1 / (1/P + T_b/T_c)
                                                ──────────────────

P → ∞ 时:  Speedup_max = T_c / T_b          <-- 屏障给加速比设的硬上限
```

取 `T_c = 10,000` 周期、`P = 64`、单次"竞争下原子操作 + 排队" 200 周期：

屏障实现	`T_b`（周期）	单阶段时间（周期）	加速比（P=64）	加速比上限 `T_c/T_b`
集中式（共享计数器 + 锁）	`P×200 = 12,800`	`156 + 12,800 = 12,956`	**0.77×**（负加速）	0.78×
合并树（讲义 slide 29）	`2·lg(P)·200 = 2,400`	`156 + 2,400 = 2,556`	**3.9×**	4.2×
传播式（dissemination）	`lg(P)·150 = 900`	`156 + 900 = 1,056`	**9.5×**	11.1×
无屏障（理论上限）	0	`156`	64×	∞

结论：`T_c = 10,000` 周期在 1.6 GHz 下只有 **6.25 µs**。这意味着**只要阶段之间的计算少于约 6 µs，集中式屏障就会让 64 核机器比单核更慢**。提高方式只有两条：**(a) 增大 `T_c`（把更多工作塞进两个屏障之间：批处理、blocking、tiling）；(b) 降低 `T_b`（换 O(log P) 的屏障，并把屏障放到点对点互连上）**。

4.5 数值算例 C：一次 `fetch_add` 的延迟预算分解

用配套讲义给出的延迟表做预算分解（Core i7 Xeon 5500 量级），一次 ticket 锁的 `acquire`：

访问路径	延迟（周期）	何时发生
L1 命中	~4	锁行的只读副本就在本核 L1（自旋等待时）
L2 命中	~10	本核 L2 持有副本
L3 命中，行未被共享	~40	锁不在任何别的核的 L1/L2 里
L3 命中，行在别的核被共享	~65	典型的"多核同时窥视同一把锁"
L3 命中，行在别的核被修改	~75	典型的"锁刚被别人写过（释放）"
本地 DRAM	~30 ns（~120 周期）	锁所在行被驱逐出所有缓存
远端 DRAM（NUMA）	~100 ns（~400 周期）	锁的内存在远端节点

算例：`acquire` 的原子 RMW 命中"行在别的核被修改"这一栏 → **≈75 周期 ≈ 47 ns @1.6 GHz**；`release` 的写使 15 个等待者失效，每个等待者重新取行 ≈75 周期。因此**一次"完美实现的"锁交接的最低下界 ≈ 150 周期 ≈ 94 ns**——这就是 MCS 的实测交接延迟与"再多优化也降不下去"的物理原因（**注意：这是延迟下界，不是吞吐限制；吞吐还受互连带宽约束**）。若临界区只有 50 ns，那么**锁交接本身就是临界区的两倍**——这正是讲义强调"低延迟"与"低流量"必须同时满足的现实含义。

4.6 用 Roofline / 算术强度看同步

同步代码的**算术强度（arithmetic intensity, AI）= 浮点/整数运算数 ÷ 访存字节数**极低：一次 `fetch_add` 是"1 次操作 / 一条 cache line（64 B）"，`AI ≈ 0.016 op/B`。在 Roofline 模型里这意味着它**永远落在带宽斜坡的最左端**：性能 = `AI × B_bus`，与峰值算力无关。**这是"同步是纯开销"的定量表述**：16 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA）= **768 GFLOPS** 的峰值算力，在同步指令上一点也用不到——同步只消耗互连带宽与延迟。同理，一个 256 MB 数组在 20 GB/s 带宽下**单次遍历至少要 12.8 ms**，而一次锁交接（~100 ns）相比微不足道；但当**每次访问一个元素就要同步一次**时（如 `lock; a[i]++; unlock;`），吞吐被同步而非带宽锁死：`20 GB/s / 64 B ≈ 312 M 行/s` 的带宽预算完全用不上，实际被 `1/(h+c)` 限制在几 M/s——**相差两个数量级**。这条对比是"批处理 / 减少同步次数"最有力的论据。

---

5. 关键要点

1. **把同步事件拆成 acquire / waiting / release 三阶段再看性能**。锁的灾难几乎总在某一阶段高度集中：TAS 死在 *waiting*（持续制造总线事务）、ticket/数组/MCS 分别优化 *acquire* 的原子操作、*waiting* 的地址隔离、*release* 的唤醒粒度。**优化前先定位是哪一段在花钱。**
2. **同步的性能尺度是"互连流量与串行化"，不是"指令数"**。判据只有两条：*(i)* 每次同步事件在互连上搬几条 cache line（O(1) / O(P) / O(P²)）？*(ii)* 线程是否在**同一个地址**上排队（若是，span = O(P)，与流量无关）？MCS 同时打赢这两条：O(1) 流量 + 每个等待者独立地址 + FIFO 公平。
3. **原子 RMW（TAS/CAS/fetch-add）是正确性的必要条件，不是性能的解决方案**。`load-test-store` 必然错；但"用 TAS 实现一切"必然不扩展。正确做法是**用最便宜的原子原语 + 最少的共享地址**：取号用 `fetch_add`，升级用 `cmpxchg`，等待用**只读**自旋。
4. **屏障（以及一切全局同步）给加速比设了一个硬上限 `T_c/T_b`**。集中式屏障 span = O(P)，合并树/传播式 = O(log P)（**但树形只在点对点互连上兑现收益，总线上流量仍被串行化**）。工程上第一优先级永远是**增大两次同步之间的工作量**，其次才是换 O(log P) 的屏障。
5. **伪共享是"没有共享的通信"**——它把"每线程独立写"变成互连上的 cache line 弹跳，可以让并行度退化为 1。同步数据结构（锁字、屏障计数器、每线程计数器）必须**各自独占 cache line**（`alignas(64)` / padding），并且要在实现里就做对，而不是等 profiling 发现。

---

6. 常见陷阱与注意事项

- **用 `load` + `store` 手写锁或标志位**：LOAD–TEST–STORE 不是原子的（讲义 slide 8 的反例），P0/P1 会同时认为自己持有锁。任何"先检查再写入"的互斥逻辑，都必须由**单条原子 RMW**（TAS/CAS/fetch_add）完成。同理，`while (flag == 0);` 这样的裸自旋在没有 `atomic` / `volatile` +fence 保证时，可能被编译器优化成死循环（或读被提升到循环外）。
- **忽略内存序，以为"原子 = 万事大吉"**：默认的 `memory_order_seq_cst` 最安全但最贵；反过来，把 `relaxed` 用在发布/订阅模式上（例如 `data = x; flag = 1;` 全用 relaxed）会让消费者看到 `flag` 却看不到 `data`。**规则**：写方用 `release` 发布，读方用 `acquire` 获取；只有在纯计数（如 `fetch_add(1, relaxed)` 取号）时才可用 `relaxed`。这就是 §2.13 的 **DRF 契约**：程序无数据竞争，硬件模型再弱也呈现 SC；**一旦有竞争，则毫无保证**。
- **伪共享（false sharing）**：两个线程写**不同变量**但落在**同一条 64 B cache line**（如 `int counter[NUM_THREADS]`、相邻的锁字、屏障的 `counter` 与 `flag` 同行）→ 行的所有权在两个核之间 ping-pong。修复：填充到 cache line、每线程私有局部变量后再合并、按块划分数据

（块 ≥ 一行）。实测教训：**约 14% 的访问跨核转移就足以让程序慢 3 倍**（§3.3）。

- **把"自旋"当万能药**：自旋只在"不超售 + 预期等待时间 < 上下文切换开销"时划算；在**超售**环境（线程数 > 核数、或同机跑多个 CPU 密集程序）里，自旋会**抢走持锁线程的 CPU**，形成"自旋者越多、持锁者越慢、锁越难释放"的活锁式退化。混合策略（短暂自旋后 `sched_yield` /futex 阻塞）才是工程解。
- **忽略公平性 ⇒ 饥饿（starvation）**：TAS/TTAS 无公平性；**指数退避更糟——新来者退避间隔更短，可能反复插队**（讲义 slide 19）。需要 FIFO 语义时必须选 ticket / 数组 / MCS 锁。反过来，**不要以为 FIFO 一定更快**：ticket 锁的 FIFO 是免费的，但它的等待者在同一行上自旋，每次释放仍有 O(P) 流量。
- **屏障的两个经典错误**：*(1)* 用"朴素共享计数器 + flag"实现可复用的屏障——快线程冲到下一次屏障清掉 flag，慢线程永远等不到（讲义 slide 25/26 的核心问题），必须用 **sense reversal** 或 **leave_counter** 做代际隔离；*(2)* 忘记 `local_sense` /轮次号必须是**每线程私有**（放在共享结构里就退化成又一个被争用的变量）。此外，**所有线程必须执行相同次数的屏障**，否则最弱的模型也救不了——这是结构性要求，不是性能问题。
- **只看平均延迟、不看尾延迟（tail latency）**：锁交接的延迟分布在竞争下长尾极重（排队 + 互连重试）。用"平均 ns/次"评估锁会系统性低估真实伤害；评估同步原语时应看 **p99/p999** 与吞吐量两条曲线，而不是单点平均值。
- **在总线上迷信树形/分层算法的收益**：合并树把竞争点从 1 个分散到 P–1 个，**span 降到 O(log P)**，但**总流量仍是 O(P)**；总线本身串行，收益有限（讲义 slide 29 明确提醒）。**算法优化必须匹配互连拓扑**。

---

7. 思考题（带答案）

思考题 1（TAS vs TTAS 的流量核算）

8 个线程争抢**同一把** TAS 锁，锁的行被所有等待者缓存。一次失败 TAS 的往返（含排队）为 60 周期，临界区持锁时长 `H = 300` 周期，总线等效带宽 `B = 8 B/cycle`（64 B 行 = 8 周期）。

(a) 一次锁交接，等待者一共发起多少次失败的 TAS？互连上搬了多少字节？等效占用多少总线周期？ (b) 换成 TTAS 后，一次交接的流量量级变成多少？给出估算并说明与 (a) 的差距从何而来。 (c) 如果 P 从 8 增到 32，两种锁的流量各自怎么变？

【答案】 (a) **TAS**：7 个等待者，每个等待者大约每 60 周期发一次 TAS，因此在 `H = 300` 周期的持锁窗口内，每个等待者做 `300/60 = 5` 次失败尝试 → 失败次数 = `7 × 5 = 35` 次。每次失败的 TAS 需要一次 `BusRdX`（拿到独占行并发失效），即一次 64 B 的行传输：`35 × 64 = 2240 B`；再加上 1 次成功的 TAS（64 B）和持有者 `unlock` 时重新抢回行写 0（64 B，因为行已被等待者抢走），合计约 **2368 B ≈ 37 条行事务 ≈ 296 个总线周期**。注意这个数**大于**临界区本身的一半——**纯粹浪费**。 (b) **TTAS**：等待者在本地只读副本上自旋（零互连事务），**只在观察到释放后才尝试**。所以每次交接的失败 TAS 从 35 次降到"每个等待者 1 次"：7 次抢锁尝试（`7 × 64 = 448 B`）+ 6 个失败者重新取回行继续自旋（`6 × 64 = 384 B`）+ 释放时 7 次失效消息（每条约 8 B，`56 B`）≈ **888 B ≈ 14 条行事务 ≈ 112 个总线周期**。差距来自**根本机制的改变**：TAS 的尝试频率与"等待时长"成正比（持续尝试），TTAS 的尝试频率与"释放次数"成正比（每次释放一次）——等待越久，TAS 越亏。 (c) **P = 32**（31 个等待者，其余不变）： – **TAS**：每个等待者仍做 5 次 → `31 × 5 = 155` 次失败 → `155 × 64 ≈ 9920 B`。**流量与 P 近似线性增长**（`O(P)` 每次交接），而每次交接的等待时长随流量继续增长，形成正反馈 → **整体约 $O(P^2)$**。这正是讲义幻灯片上随 P 陡升的曲线。 – **TTAS**：失败尝试仍为"每个等待者 1 次"→ `31 × 64 = 1984 B` 抢锁 + `30 × 64 = 1920 B` 重取 ≈ **3.9 KB**。仍是 `O(P)`，但**系数只有 TAS 的约 1/2.5，且不随等待时长放大**。结论：TTAS 并没有消除 O(P)，它只是把"P 越大越糟"从"P × 等待时长"降为"P × 1 次释放"——**真正的 O(1) 需要 MCS/数组锁**。

思考题 2（屏障的 span 与加速比上限）

某程序有 `K = 1,000` 个阶段，每阶段并行计算量 `T_c = 20,000` 周期，阶段之间用屏障同步。机器有 `P = 64` 核，1.6 GHz；集中式屏障的每次到达/离开在竞争下约 200 周期。 (a) 用 `Speedup = 1/(1/P + T_b/T_c)` 估算集中式屏障下的加速比；此时机器有多少时间在空转？ (b) 换成 span 为 `2·lg(P)` 的合并树屏障（每节点操作仍是 200 周期），加速比变多少？ (c) 若把每阶段的计算量降到 `T_c = 2,000` 周期（屏障次数不变），结论如何变化？给出一条工程建议。

【答案】 (a) `T_b(集中式) ≈ P × 200 = 64 × 200 = 12,800` 周期。`Speedup = 1/(1/64 + 12,800/20,000) = 1/(0.015625 + 0.64) = 1/0.6556 ≈ 1.53×`。单阶段时间 = `20,000/64 + 12,800 = 312 + 12,800 = 13,112` 周期，其中**屏障占 97.6%**，即 **64 核中有约 62.5 核的算力在屏障期间闲置**。加速比上限 `T_c/T_b = 20,000/12,800 ≈ 1.56×`，与上面的结果吻合。 (b) `T_b(树) = 2 × lg(64) × 200 = 2 × 6 × 200 = 2,400` 周期 → `Speedup = 1/(0.015625 + 2,400/20,000) = 1/(0.015625 + 0.12) = 1/0.1356 ≈ 7.4×`。**用 O(log P) 的屏障把加速比从 1.5× 提到 7.4×**（约 4.8 倍），但距 64× 仍差得远——瓶颈已经从"屏障"转移到"并行计算只占 312 周期，而屏障仍要 2,400 周期"。 (c) `T_c = 2,000` 时：集中式 `Speedup = 1/(0.015625 + 12,800/2,000) = 1/(0.015625 + 6.4) ≈ 0.156×`（**比单线程慢 6 倍以上**）；合并树 `= 1/(0.015625 + 1.2) ≈ 0.82×`（**仍然负加速**）。**结论**：*当单阶段计算量低于屏障开销时，无论屏障实现多好，加核都会变慢*。工程建议：**先做"粗化（coarsening）"——把多个小阶段合并成一个大阶段、对循环分块、把每个线程的私有结果累积起来再同步**，把 `T_c/T_b` 抬到 10 以上（例如 `T_c ≥ 24,000` 周期，约 15 μs），再考虑屏障算法本身的优化。此外还应减少屏障**次数**（`K` 从 1000 降到 100），因为墙钟时间 = `K × (T_c/P + T_b)`。

思考题 3（伪共享的诊断与修复）

一个 OpenMP 归约程序在 8 核上跑得比单线程还慢。profile 显示 L1 命中率很高，但**互连流量异常大**，且 `perf` 报出大量 "HITM"（命中他人修改过的行）事件。代码片段如下：

```
double sum[NUM_THREADS];                  // 全局数组，每线程写 sum[tid]
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    int tid = omp_get_thread_num();
    sum[tid] += a[i];
}
```

(a) 这是什么问题？为什么"每个线程只写自己的元素"仍然会有通信？ (b) 给出两种修复方案，并说明哪一种更好。 (c) 为什么 `double` 数组比 `int` 数组在这种场景下"稍微好一点"，但并没有解决问题？

【答案】 (a) **伪共享（false sharing）**。`sum[NUM_THREADS]` 是 `double`，8 个元素共 64 B，**恰好整条 cache line**。虽然每个线程只写自己的元素（**没有真共享 true sharing**），但一致性协议的粒度为 cache line：线程 0 写 `sum[0]` 必须独占整行并让其他核的副本失效，线程 1 立刻再抢回该行……**整行在两个/多个核之间 ping-pong**。`perf` 的 HITM 事件正是"我请求的行在别的核里是 modified 状态，必须由它提供数据"的证据——**这是纯粹的人为通信（artifactual communication）**。L1 命中率高并不矛盾：命中的是**别的**行，而这一行的每次访问都在跨核迁移。 (b) 两种修复： 1. **填充（padding）**：让每个线程的累加器独占一条 cache line： `c struct Padded { double v; char pad[64 - sizeof(double)]; } sum[NUM_THREADS]; // 或 C11/C++17: struct alignas(64) Padded { double v; };` 2. **私有累加 + 阶段合并（reduction）**：每个线程在自己的局部变量（寄存器/栈）里累加**一整块**数据，只在最后合并一次： `c #pragma omp parallel { double local = 0; #pragma omp for for (int i = 0; i < N; i++) local += a[i]; #pragma omp critical sum_global += local; // 或 #pragma omp atomic }` **方案 2 更好**：它把跨核通信从"每次迭代一次"降到"每线程一次"，**总流量从 O(N) 降到 O(P)**；而且它天然规避了伪共享（局部变量在寄存器/私有栈上）。方案 1 只是把问题"挡住"（每行独立，但仍是每线程每迭代一次 cache line 的读写，只是不再迁移）——它比原来快得多，但**没有消除"每迭代一次共享内存写"这个根本成本**。二者应结合：先做私有累加，跨线程汇总处再用填充/critical。 (c) 因为 cache line 是 64 B：`double`（8 B）时 8 个元素刚好占满一条线，**8 个线程争 1 条线**；`int`（4 B）时一条线里有 16 个元素，争用者更多、每线程的"有效产量/行"更低，**同样的行迁移次数分摊到更少的有效数据上，相对损失更大**。但"稍微好一点"不等于解决问题：只要**多个线程写同一条 cache line 的不同字节**，就会发生伪共享——**判据是"是否落在同一行"，与数据类型或元素个数无关**（CS149 讲义的定义：两个处理器写不同地址，但地址映射到同一 cache line）。

Lecture 16: Fine-Grained Synchronization, Lock-Free Programming

1. 章节标题与概述

Lecture 16: Fine-Grained Synchronization, Lock-Free Programming（细粒度同步与无锁编程）

- **本讲核心问题**：**锁已经把并行程序写对了，但"一把大锁"把并行性又还回去了；那么能不能把"锁"拆细，或者干脆不要锁？** 上一讲（Implementing Synchronization）解决了"如何正确地实现一个锁"（原子原语、自旋 vs 阻塞、test-and-set/ticket lock 等）；本讲把问题从**锁的实现**推进到**锁的使用**与**锁的替代**：①**细粒度同步（fine-grained synchronization）**——把一把保护整个数据结构的锁，拆成"每节点一把锁"，让对不同数据区域的操作真正并行（链表 hand-over-hand 遍历）；②**无锁编程（lock-free programming）**——用 `compare-and-swap`（CAS，比较并交换）这类**乐观并发（optimistic concurrency）**原语，让线程在不持有任何锁的情况下完成对共享数据结构的修改，靠"检测到冲突就重试"来保证正确性。贯穿全讲的一条主线是：**粒度（granularity）与开销（overhead）之间的权衡**——细粒度降低争用却抬高单次操作成本，无锁消除锁的阻塞风险却引入 ABA、内存回收、CAS 重试等新问题。
- **涉及的主要硬件/软件机制**：硬件侧是**原子读-改-写指令（atomic read-modify-write, RMW）**及其在缓存一致性协议上的落地——x86 的 `lock` 前缀（缓存中保留该行直到操作完成；总线上则持有总线；其它设计上可能对请求该行的其它请求回 NACK）、`cmpxchg` / `cmpxchg8b` / `cmpxchg16b`、ARM 的 `LDREX` / `STREX`（load-linked / store-conditional，LL/SC）、CUDA 的 `atomicCAS` / `atomicAdd` / `atomicExch` 等；以及由 **MSI 一致性协议**决定的"缓存行所有权在核间迁移"这一真实成本来源。软件侧是 **GCC 内建原子函数**（`__sync_fetch_and_add` 等）、**C++11 `std::atomic<T>` 及其内存序（memory order）语义**、以及建立在这些原语之上的数据结构设计：细粒度锁链表、单生产者单消费者（SPSC）队列、无锁栈、无锁链表，以及 ABA 问题的版本号（counter）解法、双字 CAS、hazard pointer（危险指针）内存回收方案。
- **在并行计算知识体系中的角色**：本讲位于"同步"主题的收尾处，是**从"硬件提供的原子性"走向"软件可用的并发数据结构"的桥梁**。它把前面几讲的三条线索拧在一起：**缓存一致性**（CS149 supplement 出现的 MSI 状态机解释了为什么一个 `lock xchg` 会引发一次总线/目录事务）、**同步原语**（上一讲实现的锁在这里被当作"被批判的对象"）、**性能模型**（work-span、Amdahl、带宽/延迟）在本讲被用来判断"到底该用哪种粒度"。它是后续 **Transactional Memory（事务内存）** 的直接动机——正如讲义所指出的，"CAS 在无锁实现中的作用是判断'在我操作期间有没

有别的线程改过数据结构'"，而事务内存把这种"乐观 + 可中止（abort）"的机制一般化、硬件化；它也是后续在真实系统里写并发容器（Java 的 `ConcurrentSkipListMap`、数据库与 Web 服务器的队列）时必须具备的判断力。

- **配套材料**：
- **CMU 15-418/618 本讲讲义：已公开（可公开下载）。** 文件为 `17_lockfree.pdf`（对应仓库路径 `lectures/17_lockfree.pdf`，40 页），位于公开目录 https://www.cs.cmu.edu/~418/lectures/ 之下。注意两点：①**Fall 2026 的讲次编号是 Lecture 16**（日程日期 **Oct 2**，标题 "Fine-Grained Synchronization, Lock-Free Programming"），而该 PDF 的文件名与首页沿用了历史学期的编号（首页写有 "Lecture 17"、学期写有 "Fall 2025"），这是**讲义沿用**的正常现象，不视为错误；②在 Fall 2026 的日程表 https://www.cs.cmu.edu/~418/schedule.html 中，本讲那一行的 slides/video 链接**被 HTML 注释隐藏**（注释原文说明 "slides/video from a previous offering; uncomment when posted for Fall 2026"），但 PDF 本体确实存在于公开目录中、可直接下载。
- **已公开的姊妹课程（Stanford CS149）讲义抽取文本：已公开。** `cs149_supp/finegrainedsync.txt`（Stanford CS149 Fall 2025 Lecture 16 "Implementing Locks, Fine-Grained Synchronization, and (a short intro to) Lock-Free Programming"，66 页）。它是本讲内容**最完整的公开文本来源**：前半部分给出死锁/活锁/饥饿的术语体系与死锁四条件、MSI 状态迁移图、test-and-set 锁与其一致性流量、test-and-test-and-set 锁、ticket lock、用 CAS 构造锁、LL/SC，后半部分与 CMU 讲义高度重合（细粒度锁链表、SPSC 队列、无锁栈、ABA、hazard pointer、无锁链表、Hunt 2011 性能对比）。
- **本笔记的事实基础**：CMU `extracted/17_lockfree.txt`（40 页全文抽取）+ `cs149_supp/finegrainedsync.txt`（66 页全文抽取）。所有代码骨架、术语、结论均出自这两份文本；两份文本中**未给出具体坐标数值的图表**（如 Hunt 2011 的归一化运行时间图、Culler/Singh/Gupta 的锁竞争时间曲线）本笔记只沿用其**定性结论与坐标轴含义**，不虚构数值；本笔记中出现的数值算例均显式标注为"按给定假设推导"，不冒充讲义原文数据。
- **未发布/需登录**：Fall 2026 本讲的**录像（YouTube/Panopto 链接）在日程表中同样被注释隐藏，属未发布**；**Ed 讨论区、Autolab、Canvas 均需登录**。历史学期位于 `/afs/cs/academic/class/15418-*/public/` 之下的若干讲义（Performance Analysis/Profiling、Transactional Memory、AI in System Design 等）需要 CMU 登录，属**未公开**。
- **课程语境**：Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。本讲的前一讲是 Lecture 15 "Implementing Synchronization"（Sep 30，对应公开 PDF `16_synchronization.pdf`），下一讲是 Lecture 17 "Transactional Memory"（Oct 5，Fall 2026 尚未发布讲义）。

---

2. 核心概念与硬件/软件架构图解

2.1 词汇地基：deadlock（死锁）、livelock（活锁）、starvation（饥饿）

定义与目的：在使用锁与无锁算法之前，必须先分清三种"不前进"的失败模式。讲义（CS149 supplement）给出的定义是：

- **Deadlock（死锁）**：系统中存在尚未完成的操作，但**没有任何操作能取得进展**。当每个操作都已持有另一个操作需要的共享资源时就会发生；除非某个操作**主动让出资源**（"倒车"），否则谁都动不了。死锁是**正确性问题**。
- **Livelock（活锁）**：系统**正在执行大量操作**，但没有任何线程取得**有意义的进展**。典型计算机系统例子是"操作不断中止并重试"——这恰好是无锁算法 CAS 循环在极端争用下的失效模式：所有人都在跑，没人到终点。
- **Starvation（饥饿）**：系统**整体在前进**，但**某些进程完全得不到进展**。讲义明确指出饥饿"通常不是永久状态"，本质上是**公平性（fairness）**问题，而不是正确性问题。

直观解释（"它是什么？"）：把共享数据结构的并发访问想象成一条**只有一车宽的山路会车**。 – **死锁 = 四辆车在十字路口互不相让**：每个司机都占据了别人必须经过的那一小段路，并且都在等对方先退。这就是"死锁会发生，而且在线性链表的锁顺序上每天都在发生"的现实版。 – **活锁 = 两个人在狭窄走廊里左右闪避**：两人都在不停地移动（CPU 100%），但谁也没能通过走廊（没有操作完成）。CAS 重试循环在极高争用下就是这一幕。 – **饥饿 = 主干道车流不断，支路口的车永远等不到空档**：整体交通在前进（系统吞吐正常），但那辆支路车永远插不进去。这正是**非公平锁**（test-and-set 锁）的后果——讲义明确说它 "no provisions for fairness"。

死锁的四个必要条件（讲义原文给出，可用于代码审查）：

#	条件（英文）	含义	在链表 hand-over-hand 例子中的对应	破坏手段
1	Mutual exclusion（互斥）	同一时刻只有一个处理器能持有某资源	每个节点的锁只能被一个线程持有	用无锁 CAS 代替锁（本讲后半部分）
2	Hold and wait（持有并等待）	处理器在等待其它资源时仍持有已获得的资源	线程持有 `prev->lock` 的同时去申请 `cur->lock`	一次性申请全部锁 / 用"先验证后加锁"的两阶段法
3	No preemption（不可抢占）	资源不能被强制夺走，直到操作完成	锁只能由持有者释放	改用可中止的事务内存（下一讲）

#	条件（英文）	含义	在链表 hand-over-hand 例子中的对应	破坏手段
4	Circular wait（循环等待）	等待者之间存在相互依赖（资源依赖图中存在环）	若某线程从表尾向表头加锁，就会形成环	**全局锁序**：永远从 head 向 tail 方向加锁

> **关键教学点**：第 4 条给出了上一讲那个 hand-over-hand 链表代码"立即可以看出无死锁"的理由——**所有线程都严格按 head→tail 的地址顺序获取锁**，因此资源依赖图不可能有环。这也是工程中"锁序（lock ordering）"这一惯例的由来。

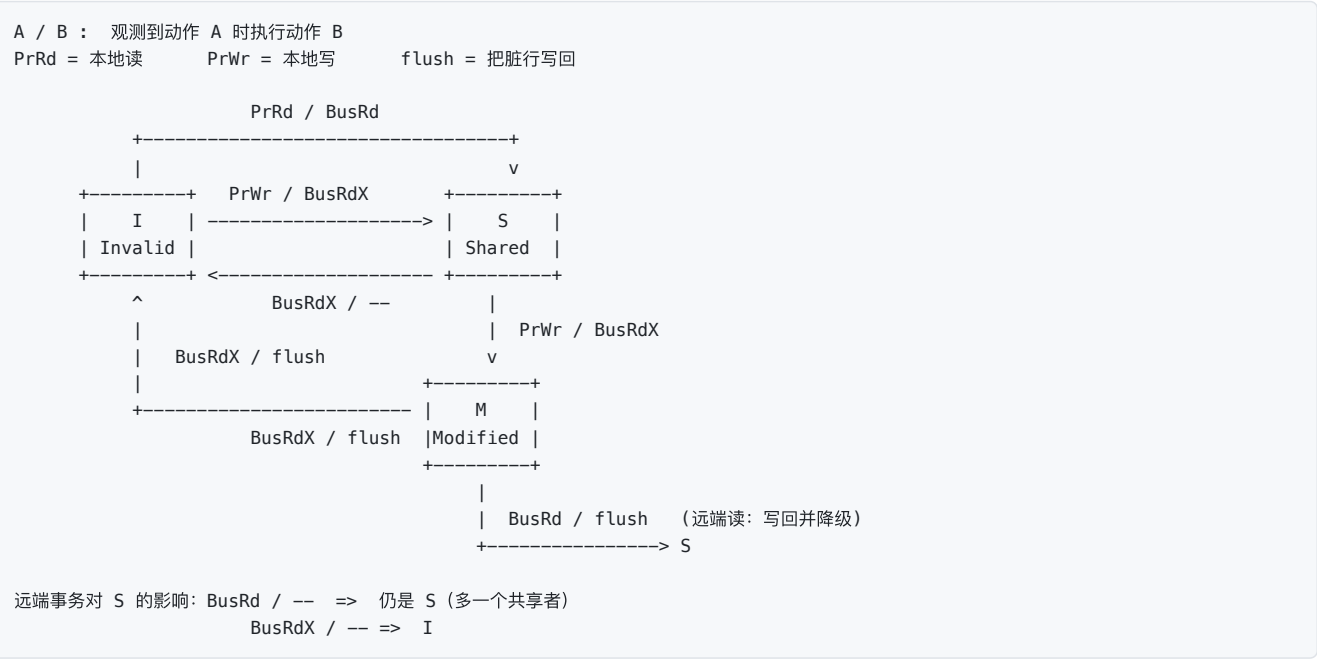
2.2 为什么"原子"在真实机器上是一件昂贵的事：从 MSI 到 `lock` 前缀

定义与目的：本讲所有算法都建立在"原子读-改-写"之上，而机器实现它的方式决定了算法的性能。讲义给出的机制描述是：

- **x86 `lock` 前缀**：如果内存位置**已在缓存中**，缓存会保留该行（保持独占）直到操作完成；如果**不在缓存中**，在总线上使用 lock 信号并**占住总线**直到操作完成；在其它设计上，处理器（很可能是）对任何请求该缓存行的请求**回 NACK**，直到操作完成。
- 附带的重要约束：**操作必须作用于互不重叠的地址**。
- CS149 supplement 补充了 `lock cmpxchg` 的精确语义：`if (dst == EAX) { ZF = 1; dst = src; } else { ZF = 0; EAX = dst; }`，其中 `lock` 前缀指定该操作原子。
- **LL/SC（load-linked / store-conditional）**：与 CAS 这种"单条原子指令"不同，它是**一对配套指令**——`load_linked(x)` 读地址，`store_conditional(x, value)` 仅当"自对应的 LL 以来没有任何处理器写过 x"时才写入。ARM 对应指令是 `LDREX` / `STREX`。它的实现方式正是"在缓存行上留下标记/监视位"，任何对该行的外部写都会清掉该标记。

直观解释（"它是什么？"）：把缓存行想象成**一根只有一个人能举着的接力棒**。读（`BusRd`）是"把棒借来抄一遍"，可以多人同时持有副本（Shared）；写/原子 RMW（`BusRdX`，read-for-ownership）是"把棒抢过来，并且要求所有人把手上的副本撕掉"。所谓"原子操作很贵"，贵的不是运算，而是**抢棒过程中所有其它副本被作废，且此后每一个想读的人都必须重新排队**。

图解 1：MSI 状态机与"原子操作如何变成一致性事务"



MSI 状态迁移表（每条边都来自讲义状态图）：

当前状态	本地 `PrRd`	本地 `PrWr`	远端 `BusRd`	远端 `BusRdX`
I（Invalid）	→ S，发 `BusRd`	→ M，发 `BusRdX`	保持 I	保持 I
S（Shared）	→ S，`--`	→ M，发 `BusRdX`	→ S，`--`	→ I，`--`
M（Modified）	→ M，`--`	→ M，`--`	→ S，`flush`（写回）	→ I，`flush`（写回）

关键操作与性能特征：一次 `lock cmpxchg`（或 `atomicCAS`）在**无争用且行已独占**时只需几个周期（命中 L1 并保持独占）；一旦行不在本地缓存，就退化为一次**所有权获取（RFO）**，代价是： – **延迟（latency）**：同一 socket 内 L3 命中约 30–50 ns 量级，跨 socket / 远端 NUMA 约 100–200 ns 量级； – **带宽/流量（traffic）**：每次 RMW 都会**作废所有其它副本**，所以"有多少个自旋者在等这把锁"直接变成"每次锁交接要产生多

少一致性事务"——这正是下一小节要量化的核心； – **吞吐量（throughput）**：同一缓存行的原子操作在本机器上**被完全串行化**，即该行的最大交接速率为 `1 / 交接延迟`，与核数无关。

图解 2：test-and-set 锁的一致性流量时间线（讲义 CS149 的核心图之一）

```
时间 ──────────────────────────────────────────────────────────────────────────>

P1（持锁）   [持有锁，做临界区]                                          [再次 T&S]
               |
               |  断言：P1 持有锁期间，它在本地缓存里独占该行
               v
P2          T&S ──> BusRdX ──> 拿到行(独占) ──> 试图写入(1) => 失败
            T&S ──> BusRdX ──> 拿到行(独占) ──> 试图写入(1) => 失败
            T&S ──> BusRdX ──> 拿到行(独占) ──> 试图写入(1) => 失败   (又作废了 P1 的
                                                                  锁行副本!)
P3          T&S ──> BusRdX ──> 拿到行(独占) ──> 试图写入(1) => 失败
                              ...
结果：每一次 T&S 尝试 = 一次 BusRdX + 一次"作废其它所有人"的失效广播
      => 每完成一次"测试"就制造 O(P) 条一致性事务，且把持锁者 P1 的缓存行踢掉
解锁时：P1 必须先把总线权限拿回来，才能把 0 写回去（"lock holder must wait to
        acquire bus to release"，讲义原话）=> 释放本身也被拖慢
```

test-and-test-and-set 的改进与其流量模型（讲义明确给出）：

```
P2/P3 的等待循环变成：   while (*lock != 0);        <-- 本地缓存里的普通读（S 态，命中）
                        if (test_and_set(lock) == 0) return;
于是：等待期间几乎零流量（本地 S 态反复读）；
      锁释放时，所有等待者各收到一次失效（Invalidate）
      => "One invalidation, per waiting processor, per lock release (O(P) invalidations)"
      => "This is O(P^2) interconnect traffic if all processors have the lock cached"
对比：test-and-set 锁 "generated one invalidation per waiting processor per test"
      —— 即每次测试就是 O(P)，而测试次数随 P 增长，总量是 O(P^2) 甚至更高
```

2.3 锁的实现谱系：从 T&S 到 ticket lock 到 LL/SC

定义与目的：讲义（CS149 supplement 第 17–31 页）系统地给出"如何用原子原语造一把锁"，并给出评价锁的**五条理想特性**：低延迟、低互连流量、可扩展性、低存储开销、公平性（理想情形是"按请求顺序获得锁"）。

五种锁实现的对比表（每列的定性结论均出自讲义原文或由其结论直接推出）：

锁实现	获取方式（无争用时）	关键代码	互连流量	存储	公平性	讲义评语
test-and-set（T&S）	一次原子 RMW	`while (ts(mem)==1);`	每次**测试**都产生失效，最差	1 个 int	无	低延迟、**高流量**、扩展性差、存储小
test-and-test-and-set（TTS）	先普通读，再 RMW	`while(*l!=0); if (tas()==0) return;`	每次**释放**每位等待者一次失效（O(P)）	1 个 int	无	无争用时延迟略高；流量大幅下降；更可扩展；**仍不公平**
ticket lock（票号锁）	**只需一次普通读**（无需原子操作）	`my=atomic_increment(&next); while(my!=now);`	**每次释放仅一次失效**（O(P) 广播）	2 个 int	**有**（FIFO）	"only one invalidation per lock release"
CAS 自旋锁	一次原子 CAS	`while(CAS(l,0,1)==1);`	与 T&S 同级（每次尝试都是 RMW）	1 个 int	无	讲义强调"先读后 CAS"的版本在争用下**可能更高效**
LL/SC 锁	LL 读 + SC 尝试	`lr; addi; sc; beqz retry`	无争用时 LL/SC 几乎不产生失效	1 个 int	视实现	不需要"单条原子指令"，对 ABA 天然免疫（SC 会失败）

注意讲义对 ticket lock 的精确表述："No atomic operation needed to acquire the lock (only a read) —— Result: only one invalidation per lock release (O(P) interconnect traffic)"。也就是说，**取号（`atomic_increment`）是原子操作，但真正的等待只是读 `now_serving`**；相比之下 TTS 在**每次释放**都会引发"所有等待者重新读一遍"的流量。这就是票号锁比 TTS 更可扩展的根本原因。

用 CAS 构造其它原子操作（讲义留给学生的练习，是理解 CAS 循环的入口）：

```
// 讲义原题：如何用 atomicCAS 构造 atomic_max / atomic_min / atomic_increment / lock
int atomicCAS(int* addr, int compare, int val) {   // 原子地执行下面的逻辑
   int old = *addr;
   *addr = (old == compare) ? val : old;
   return old;
}
void atomic_max(int* addr, int x) {                // 循环 CAS（CAS loop / optimistic retry）
   int old = *addr;
   int new = max(old, x);
   while (atomicCAS(addr, old, new) != old) {      // 别人改过 => CAS 失败 => 重读重算
     old = *addr;
     new = max(old, x);
   }
}
void lock_cas(int* l)   { while (atomicCAS(l, 0, 1) == 1); }
void unlock_cas(int* l) { *l = 0; }                // 讲义给出的"可能更高效"的争用版本：
void lock_cas_cheap(int* l) {                      // while(1){ while(*l==1); if(CAS(l,0,1)==0) return; }
   while (1) { while (*l == 1); if (atomicCAS(l, 0, 1) == 0) return; }
}
```

直观解释（"它是什么？"）：把 CAS 想象成**赌场里"把筹码换回现金"时的对账员**。 – 你要把桌上一叠筹码（当前值 `old`）换成新的一叠（`new`）。对账员在你递上去的瞬间核对："我记录的还是不是 `old`？" – **是** → 换成功，你走人（CAS 返回 `old`）。 – **不是** → 说明别人在你算数的过程中动过这叠筹码，你刚才算的 `new` 已经作废。对账员把最新数字告诉你（CAS 返回真实当前值），你**重新算一遍**再来。这就是**乐观并发**：先假设不会有人打扰，被打扰就重试。

CUDA / GCC / C++11 三套原子原语的对应关系（讲义两张表合并）：

语义	CUDA（device 端）	GCC 内建（`type` 可为 `(unsigned) char/short/int/long`）	C++11
原子加（返回旧值）	`atomicAdd(addr, val)`	`__sync_fetch_and_add(ptr, v)`	`atomic<T>::fetch_add`
原子减	`atomicSub`	`__sync_fetch_and_sub`	`fetch_sub`
原子位与/或/异或	`atomicAnd` / `atomicOr` / `atomicXor`	`__sync_fetch_and_and` / `_or` / `_xor`	`fetch_and` / `fetch_or` / `fetch_xor`
原子与非	—	`__sync_fetch_and_nand`	—
先算后取（返回新值）	—	`__sync_add_and_fetch` 等一族	`fetch_*` + 自行计算
比较并交换	`atomicCAS(addr, compare, val)`	`__sync_val_compare_and_swap`	`compare_exchange_strong` / `_weak`
交换	`atomicExch`	`__sync_lock_test_and_set`	`exchange`
最值 / 自增自减	`atomicMin/Max/Inc/Dec`	需用 CAS 循环构造	需用 CAS 循环构造
浮点原子加	`atomicAdd(float*, float)`	需用 CAS 循环构造	需用 CAS 循环（C++20 才有 `fetch_add` for float）

C++11 `std::atomic<T>` 的语义要点（讲义原文）： – 为**整个对象**提供原子的读、写、读–改–写；若 `T` 是基本类型，原子性可以由处理器原子指令高效实现，否则（可能）由 mutex 实现。 – **为原子操作前后的操作提供内存序（memory ordering）语义；默认是顺序一致（sequential consistency）**，更多细节见 `std::memory_order`。 – `bool b = i.is_lock_free();` 返回"原子性的实现是否无锁"——注意这句话的对偶含义：**`std::atomic` 不等于无锁**，用错大小/类型时它可能内部加锁。

```
atomic<int> i;
i++;                                  // 原子自增
int a = i;                            // 原子读
i.compare_exchange_strong(a, 10);     // 若 i 当前等于 a，则置为 10
bool b = i.is_lock_free();            // 实现是否 lock-free
```

关键操作与性能特征：原子 RMW 的"价格标签"由三件事决定——①**行是否已在本地独占**（命中则 ~10 周期级；否则一次 RFO，几十到几百 ns）；②**有多少竞争者**（决定每秒发生多少次所有权迁移）；③**内存序**（`seq_cst` 在 x86 上需要带 `lock` 前缀的 RMW 或 `mfence`，而

`relaxed` / `acquire` / `release` 在 x86 上通常只是编译屏障 + 普通访存，代价低一个数量级——这是"错误使用内存序"之所以是常见陷阱的原因，要么太弱导致正确性错误，要么太强导致性能损失）。

2.4 从"一把大锁"到"每节点一把锁"：粒度谱

定义与目的：讲义首先用排序链表说明**为什么需要细粒度锁**。数据结构通常大于一个内存位置（"Data structures are often larger than a single memory location"），因此必须回答"整个数据结构如何被保护"。

图解 3：三种粒度的软件执行模型（同一段 `insert` 代码在三种锁策略下的并发形态）

```
【粗粒度：每结构一把锁】              【细粒度 hand-over-hand：每节点一把锁】

head->[3]->[5]->[10]->[11]->[18]    head->[3]->[5]->[10]->[11]->[18]
       |                                    Lt   Lt    Lt    Lt    Lt
       +-- lock(list->lock) 一把锁          (每节点一个 Lock 字段)
           T0: ████████████████████        T0: L(3) L(5) ... L(11) ... 逐个"放手"
           T1: ..................... 等待    T1:      L(5) L(10) ...     可以跟在后面
           => 完全串行                      => 形成流水线：T1 紧跟在 T0 身后

【无锁：CAS 乐观重试】

head->[3]->[5]->[10]->[11]->[18]   head(atomic<Node*>)
           T0: 读 head、读 next、算好 n->next、CAS(prev->next, cur, n)
           T1: 同时做同样的事 —— 谁先 CAS 成功谁赢，输的那个重读重试
           => 不持有任何锁，但共享同一"提交点"（此处是 prev->next）
```

直观解释（"它是什么？"）：– **粗粒度锁 = 独木桥**：安全、简单，但一次只能过一个人。讲义的评价一针见血："Good: It is relatively simple to implement correct mutual exclusion；**Bad: Operations on the data structure are serialized, may limit parallel application performance**"。– **hand-over-hand 细粒度锁 = 攀岩时的"三点固定"**：攀岩者每次只移动一只手或一只脚，始终抓住岩点（锁），沿着岩壁（链表）前进；两个人可以在同一面岩壁上相隔几个岩点同时攀爬，但**谁都必须在抓住下一个岩点之后才能放开前一个**——这就是"hand-over-hand"（交替手）这个名字的来源。 – **无锁 CAS = 抢座位游戏**：所有人都盯着同一个空位（`prev->next`），裁判（硬件原子性）裁定谁坐下；没抢到的人退回去重新观察现场再抢一次。**没有人"占着"座位不放**，所以也就不存在"某人被换出导致全场停摆"。

hand-over-hand 的正确性不变量（本讲最值得记住的一条设计规约）：

```
不变量：持有节点 P 的锁  ==>  保证 P->next 不会被别人修改，
                              因而保证 P->next 指向的那个节点的"身份"稳定。
推论 1：读取 cur = cur->next 时，只要还持有 prev 的锁，就是安全的
        （否则 cur 可能已被别人摘链并 delete，形成悬空指针）。
推论 2：加锁顺序必须严格沿 head -> tail 方向
        => 资源依赖图无环 => 无死锁（死锁第 4 条件的直接应用）。
推论 3：任何时刻都要持有 >= 1 把锁；"先锁下一个，再放前一个"
        => 不会出现"手上没有任何锁"的窗口，避免指针失效。
```

讲义的"给学生的问题"及其答案：讲义在给出 fine-grained `insert()` 后提问——"**insert() 的实现还有进一步改进的余地，是什么？**"答案是：**insert 只修改 `prev->next` 这一个指针，因此不需要在整条遍历路径上加锁**。可以先**不加锁地做只读遍历**（纯读，无一致性流量）找到候选位置 `prev` / `cur`，然后**只锁 `prev`**，并**重新验证 `prev->next == cur`**；若验证失败（被别人改过）就放弃本次结果、回到遍历重新搜索。这把"每步两次原子 RMW"降为"整次操作一次原子 RMW"，同时保留了大部分并发度——**注意这个结构与本讲后半部分的 CAS 循环在形式上完全一致**，它正是"细粒度锁"通向"无锁"的那一步。

细粒度锁的代价（讲义原文列出，务必逐条记住）：

维度	粗粒度单锁	细粒度每节点锁	讲义提示的折中方案
争用（contention）	高：所有操作串行化	低：不同区域可并行	锁条带化（striping）/ 每桶一把锁
每步开销	0 次原子操作	**每走一步 2 次原子 RMW**（拿 + 放），遍历从"纯读"变成"读 + 写"	减少加锁频率（只在真正修改处加锁）
存储	1 把锁	**每节点一把锁**（指针 + 锁 ⇒ 更大的节点、更差的缓存密度、更多 TLB/缓存压力）	用"每个节点一个 bit/版本号"替代完整锁
正确性难度	低	**高**：何时需要互斥、死锁、活锁都要论证	—
伪共享	无	**有**：相邻节点的锁常常落在同一缓存行，两个线程锁不同节点却争同一行	让锁字段独占缓存行 / 把锁与数据分离成两个数组

> **讲义的核心提示句**："What is a middle-ground solution that trades off some parallelism for reduced overhead? (hint: similar issue to selection of **task granularity**)"——**同步粒度与任务粒度是同一个问题的两面**：粒度过细，开销（这里是原子操作与缓存行迁移）吃掉并行收益；粒度过粗，并行度不足。判断依据永远是 work-span / Amdahl 的定量分析，而不是"越细越好"的直觉。

2.5 "阻塞 / 无锁 / 无等待"：进度保证的层次

定义与目的：讲义的第二个主题是**无锁（lock-free）**，它首先给出严格的定性判据：

- **Blocking algorithm / data structure（阻塞算法）**：允许一个线程**无限期地**阻止其它线程完成对共享数据结构的操作。例子：线程 0 在我们的链表某节点上获取了锁，然后被 OS 换出、崩溃、或只是很慢（缺页），于是**任何其它线程都无法完成操作**——尽管线程 0 并没有在积极修改数据。
- **关键推论（讲义用粗体强调）**："An algorithm that uses locks is blocking, regardless of whether the lock implementation uses spinning or pre-emption"——**自旋锁也是阻塞算法**。这是一个极容易被误解的点：自旋看起来"没有让出 CPU"，但它依然制造了"一个线程可以卡住所有人"的可能性。
- **Lock-free（无锁）**：非阻塞算法如果保证**某个线程**（不指定哪个）必定取得进展，即"**systemwide progress（系统级进展）**"，就称为 lock-free。等价说法：不可能通过"在某个不巧的时刻抢占某个线程"来阻止系统其余部分取得进展。
- **注意讲义的补充**："this definition does **not** prevent starvation of any one thread"——**lock-free 允许单个线程饥饿**。这是 lock-free 与 wait-free 的分界。

进度保证层次表（lock-free 的定义来自讲义；其余为无锁编程的成熟公开知识，用于把讲义定义放到完整坐标系中）：

保证级别	承诺	是否允许单线程饥饿	是否允许系统整体停摆	典型实现代价
Blocking（含 mutex、自旋锁、读写锁）	只保证"没被抢占时能完成"	允许	**允许**（持有者被换出/崩溃即可）	最低（实现简单），但最坏情况无界
Obstruction-free	当某线程**独占执行**（其它线程暂停）时能完成	允许	允许	低（只需 CAS + 重试）
Lock-free	**至少一个**线程必定完成（systemwide progress）	**允许**	**不允许**	中（CAS 循环 + ABA/回收方案）
Wait-free	**每个**线程都在有界步数内完成	不允许	不允许	高（几乎总比 lock-free 慢，需帮助机制）

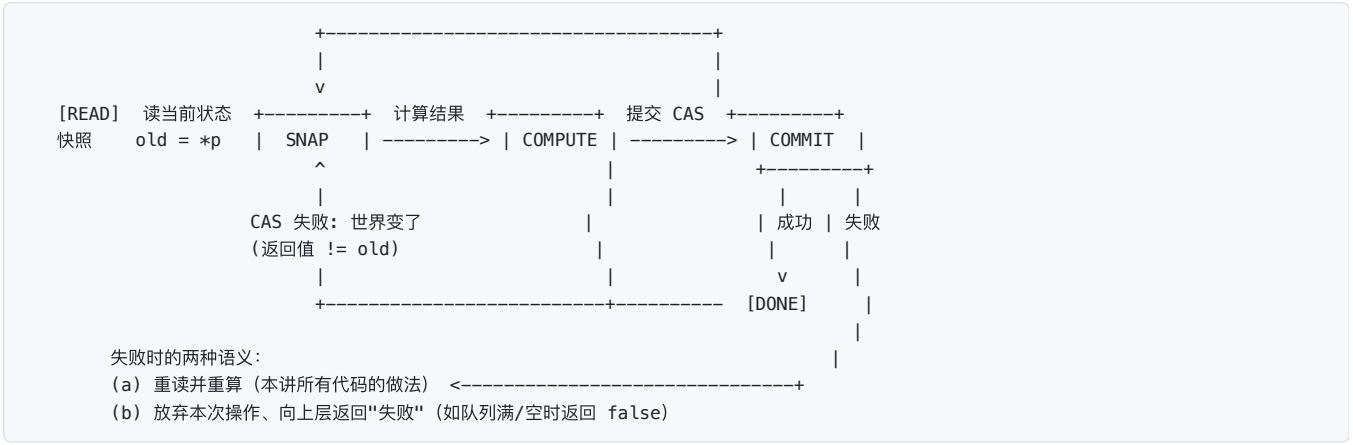
直观解释（"它是什么？"）： - **Blocking = 只有一个售票窗口，且窗口里的售票员可能突然去上厕所**：队伍里所有人一起等，而且**不知道要等多久**。 - **Lock-free = 自助售票机，大家都能按**：即使某人按到一半走神（被换出），其他人照样能在同一台机器上把票买走（重试即可），**系统总有票卖出去**；但某个特别倒霉的人可能反复被人抢先、永远买不到（饥饿）——不过这不是系统性问题。 - **Wait-free = 每人一个专属窗口**：没人需要等别人，代价是窗口（硬件/算法复杂度）非常贵。

关键操作与性能特征：注意这里的"进度保证"与"性能"是**正交**的两件事。讲义在总结里明确写："**Note: a lock-free design does not eliminate contention** — Compare-and-swap can fail under heavy contention, requiring spins."（第 4 节会给这一步定量分析。）

2.6 无锁的软件执行模型：CAS 循环（乐观并发）

定义与目的：无锁数据结构的基本执行模式是"**读-算-CAS-重试**"：读取待修改位置的当前值 → 基于它计算新值 → 用 CAS 尝试"仅当值未变"时提交 → 失败则**从头再来**。核心思想（讲义对无锁栈的说明）："**as long as no other thread has modified the stack, a thread's modification can proceed**"（只要没有别的线程改过，本线程的修改就能进行）。

图解 4：CAS 循环的状态机与时间线




```
时间 ──────────────────────────────────────────────────────────────────────>

T0:  |--read A--|--compute--|--CAS(top:A->B) SUCCESS--|  DONE
T1:       |--read A--|--compute--|--CAS(top:A->B) FAIL(当前是 B)--|
          |<------------------ 重读、重算、重试 ------------------>|--CAS(top:B->C) SUCCESS--|

失败的原因不是"T0 抢了 T1 的座位"，而是"T1 基于的旧快照已经过期"。
=> CAS 循环把并发控制从"提前加锁"改成"提交时校验"，因此：
   优点：无锁持有期、无死锁、无优先级反转、持锁者被抢占不影响他人
   代价：失败重试是纯粹的浪费（Wasted work），且失败率随争用者数量上升
```

关键操作与性能特征： – **成功路径（fast path）**：一次原子读 + 一次 CAS，两者若命中本地独占行，代价与普通访存同量级（~10–20 周期）。 – **失败路径（slow path）**：失败的 CAS **本身就是一次完整的 RFO**——它已经把缓存行抢到本地了，但因为是"值不匹配"而没有完成语义上的提交。因此**失败比成功还浪费**：它既付出了行迁移的全部代价，又没有推进任何状态，还额外污染了其它人的副本。 – **活性**：CAS 循环天然是 lock-free（每次行迁移必定有一个线程成功），但**不是 wait-free**（倒霉线程可以永远失败）。 – **一个必须记住的语义细节**：`compare_exchange` 返回旧值，因此判断"我是否成功"的方法是 `CAS(p, old, new) == old`；而 `atomicCAS(addr, compare, val)` 返回旧值、只有"旧值 == compare"时才写入——**两种接口的返回值语义不同，写代码时要看清是哪一种**（讲义代码使用的是前者语义）。

2.7 单生产者–单消费者（SPSC）队列：无锁设计的"温柔区"

定义与目的：当访问者只有**一个生产者 + 一个消费者**时，可以有**完全不需要同步等待**的无锁队列。讲义给出两个版本，并反复强调前提："**Assume a sequentially consistent memory system for now (or the presence of appropriate memory fences, or C++11 atomic<>)**"——即这些代码要在弱一致性硬件上正确，必须补上内存栅栏或使用 `atomic<>`。

版本 A：有界队列（bounded queue）

```
struct Queue { int data[N]; int head;  /* 队首 */ int tail; /* 下一个空位 */ };
push: 若 tail == MOD_N(head - 1) 表示满 -> 返回 false
      q->data[q->tail] = value;  q->tail = MOD_N(q->tail + 1);  返回 true
pop : 若 head != tail (非空)
      *value = q->data[q->head]; q->head = MOD_N(q->head + 1);  返回 true
      否则返回 false
```

两个线程是**同一变量空间的两个"所有者"**：生产者只写 `tail` 和 `data[tail]` 槽位，消费者只写 `head` 并读 `data[head]` 槽位。**没有人写别人拥有的变量**，因此不需要任何原子 RMW——只需要保证"我写的 tail 你能看见"（release/acquire 语义）。

版本 B：无界队列（unbounded queue）与 reclaim 指针

```
tail 指向最后加入的元素；head 指向队首元素之前的那个元素；
节点的分配与删除都由同一个线程（生产者）执行。
push:  n = new Node; n->next=NULL; n->value=v;
       q->tail->next = n;  q->tail = q->tail->next;
       while (q->reclaim != q->head) { tmp = q->reclaim; q->reclaim = q->reclaim->next; delete tmp; }
pop:   if (q->head != q->tail) { *value = q->head->next->value; q->head = q->head->next; return true; }
       return false
```

图解 5：无界队列的 head / tail / reclaim 演进（对应讲义的那张状态图）

```
初始:      head,tail,reclaim -> [dummy]                        队列为空
           ^^^^^^^^^^^^^^^^^^^ 三者指向同一个哑节点

push 3:    reclaim,head -> [dummy] -> [3] <- tail
push 10:   reclaim,head -> [dummy] -> [3] -> [10] <- tail

pop -> 3:  reclaim -> [dummy]     head -> [3]        [10] <- tail
            (head 前进一格：它始终指向"队首元素之前的那个节点",
              因此队首元素是 head->next = [10])
           队列内容 = [10]，[dummy] 仍不可回收（reclaim 还指着它）

pop -> 10: reclaim -> [dummy]   head, tail -> [10]
           队列内容为空（head == tail）

pop -> false (empty)          消费者不删除任何节点

push 5:    生产者在这里触发回收循环：
           n=[5]; tail([10])->next = [5]; tail = [5];
           while (reclaim != head) { tmp=reclaim; reclaim=reclaim->next; delete tmp; }
             iter1: reclaim=[dummy] != head=[10] -> delete [dummy]; reclaim=[3]
             iter2: reclaim=[3]     != head=[10] -> delete [3];     reclaim=[10]
             iter3: reclaim=[10] == head=[10] -> 停止
           结果：reclaim, head -> [10] -> [5] <- tail

关键设计：生产者从不删除 head 及其之后的节点，消费者从不删除任何节点
          => 每个节点只被一个线程 delete，不存在"引用已释放内存"的竞态
```

关键操作与性能特征： – **延迟**：快路径上**没有任何原子 RMW、没有任何自旋**，只是一次普通读 + 一次普通写，几乎与单线程数组操作同价。 – **吞吐量**：生产者与消费者形成**流水线**，稳态吞吐由较慢的一方决定（典型的 producer–consumer 瓶颈分析）。 – **回收的滞后性（这一点最容易被忽略）**：`reclaim` 指针是**刻意落后于** **`head`** 的（正因为落后，才能保证被删的节点确实不可能再被消费者访问），因此内存峰值使用量可能远高于队列的稳态长度；`reclaim` 的推进是**摊销（amortized）**的——一次 push 可能一次删除很多节点，导致**尾部延迟（tail latency）尖峰**。

2.8 无锁栈与 ABA 问题

定义与目的：讲义用"无锁栈"作为第一个**多生产者多消费者**的无锁数据结构例子。

```
struct Stack { Node* top; };
push(s, n):  while (1) { old_top = s->top; n->next = old_top;
                         if (CAS(&s->top, old_top, n) == old_top) return; }
pop(s):      while (1) { old_top = s->top; if (!old_top) return NULL;
                         new_top = old_top->next;
                         if (CAS(&s->top, old_top, new_top) == old_top) return old_top; }
```

与细粒度锁的本质区别（讲义原文）："In fine–grained locking, the implementation locked a part of a data structure. **Here, threads do not hold lock on data structure at all.**"

ABA 问题（本讲最著名的陷阱）：CAS 只比较**值**（这里是地址），而**值相同不代表状态没变**。讲义的时间线如下：

图解 6：ABA 问题的完整时间线（A/B/C/D 是节点地址，不是节点里存的值）

```
初始:                    top -> A -> B -> C

T0: begin pop()
    (局部变量 old_top = A, new_top = B)          <-- T0 读到了快照 (top==A)
    ---- T0 被抢占 / 变慢 ----

T1: begin pop()  读到 old_top == A
T1: complete pop()  返回 A           => top -> B -> C
T1: 修改节点 A (例如 A->value = 42)
T1: begin push(A); complete push(A)  => top -> A -> B -> C
T1: begin push(D); complete push(D)  => top -> D -> A -> B -> C

T0: 恢复执行, 执行 CAS(&top, A, B):
    CAS 比较 top == A ?  成立! (此刻 top 确实是 A)  => CAS 成功, 把 top 设为 B
T0: complete pop()  返回 A

结果:  top -> B -> C
       节点 D 从栈里消失了 (丢失), 栈结构被破坏!
注意:  T0 的 CAS 语法上完全正确, 错的是"ABA 相等"的语义假设:
       地址 A 回来了, 但"A 被 A 引用时的那个"世界已经不存在了。
```

解法 1：计数器 / 版本号（讲义给出）——给栈再加一个 `pop_count`，用**双字比较并交换（DCAS / doubleword CAS）**同时校验 `top` 与计数：

```
pop:  loop { pop_count = s->pop_count; top = s->top; if (!top) return NULL;
             new_top = top->next;
             if (double_compare_and_swap(&s->top, top, new_top,
                                         &s->pop_count, pop_count, pop_count+1)) return top; }
```

解法 2：x86 的"宽 CAS"——讲义专门澄清：x86 的 `cmpxchg8b` / `cmpxchg16b` **并不是**上面代码需要的通用 DCAS，但只要**保证 `top` 与 `pop_count` 在内存中连续**，就可以用一条 64 位 / 128 位的单条 CAS 指令同时比较两个字段（"compare and exchange eight/sixteen bytes"，可用于两个 32 位 / 两个 64 位值的 CAS）。这个"把要一起校验的字段打包进一个字"的技巧是无锁编程中最常用的工程手段。 **解法 3**：讲义也指出可以"**通过谨慎的节点分配与元素复用策略**"解决 ABA——例如**节点从不真正释放**（用内存池 + 索引 + 打包版本号），地址空间足够大以至于不会回绕。

另一个独立问题：引用已释放内存（referencing freed memory）——即使解决了 ABA，`pop` 中的 `old.top->next` 也可能在**读取时节点已被别人弹出并 `delete`**，从而访问已释放内存（use-after-free）。讲义给出两个层次的处理：

图解 7：Hazard pointer（危险指针）+ retire list（退休链表）

```
每个线程两份私有数据:
  hazard   : 本线程"正在查看, 因此不能被别人 delete"的节点指针
  retireList / retireListSize : 本线程已移除、但还不能确定能安全 delete 的节点

pop 循环:
  1. old.top = s->top
  2. hazard = old.top              <==  【先发布】"我正在用这个节点"
  3. old.top == s->top ?           <==  【再校验】发布后节点没被换掉
  4. new.top = old.top->next
  5. doubleword_CAS(s, old, new)
       成功 -> value = old.top->value; retire(old.top); return value
       失败 -> hazard = NULL; 重试
  6. 后续 push 若成功, 则 CAS 完成为止

retire(ptr):
  push(retireList, ptr); retireListSize++;
  if (retireListSize > THRESHOLD)                    <== 摊销: 批量清理
     for each n in retireList:
         if (n 不等于任何线程的 hazard 指针) { 从链表移除 n; delete n; }

安全性论证: 节点 n 从结构中摘除 (CAS 成功) 之后, 任何线程想再获得对 n 的引用,
            都必须先读 head/top 并沿链走; 而 n 已不可达 => 只要在"扫描时刻"没有
            任何 hazard == n, 就再也没有线程会引用 n (除持有者自己, 而持有者正是
            执行 retire 的线程), 于是可以安全 delete。
```

关键操作与性能特征：hazard pointer 把"内存回收"变成一次**批量的、读所有线程 hazard 槽位的扫描**（O(线程数) 的读，且是**只读**，不产生一致性流量风暴，因为 hazard 槽位是各线程私有的、被频繁写但几乎不被别人读——反过来，如果 hazard 槽位与高频写字段伪共享，就会付出巨大代价）。`THRESHOLD` 的选择是经典的时间/空间权衡：太小则扫描频繁（每次 `retire` 都 O(P) 扫描），太大则内存占用与尾延迟上升。

2.9 无锁链表：插入容易，删除很难

定义与目的：讲义最后给出无锁链表的插入与删除，并用一句话点出复杂度鸿沟——"**Supporting lock-free deletion significantly complicates data-structure**"。

```
// 在指定节点之后插入（讲义简化版：假设只有 insert 操作）
void insert_after(List* list, Node* after, int value) {
   Node* n = new Node; n->value = value;
   Node* prev = list->head;
   while (prev->next) {
     if (prev == after) {
       while (1) { Node* old_next = prev->next; n->next = old_next;
                   if (compare_and_swap(&prev->next, old_next, n) == old_next) return; }
     }
     prev = prev->next;
   }
}
```

与细粒度锁相比的两项收益（讲义原文）："**No overhead of taking locks; No per-node storage overhead**"——每走一步不再有两次原子 RMW，也不必给每个节点再加一个指针字段（节点更小 ⇒ 缓存密度更高）。

图解 8：无锁删除的经典难题（B 被删除的同时 E 被插到 B 之后）

```
   初始:        A -> B -> C -> D

   并发操作:  T0 删除 B（CAS 成功在 A->next 上，把 A->next 指向 C）
              T1 在 B 之后插入 E（CAS 成功在 B->next 上，把 B->next 指向 E）

   结果:        A -> C -> D          (T0 的意图实现)
                     B -> E          (B 已不在链表中，却指向 E! )

   问题:
     - E 实际上"挂"在一个不可达节点上 => 逻辑丢失（E 不在列表里）
     - B 若被回收，E 也随之成为垃圾
     - 若 T0 先删除 B、T1 后 CAS B->next，则 T1 的 CAS 甚至作用在已释放内存上
   讲义给出的进一步阅读:
     - Harris 2001, "A Pragmatic Implementation of Non-blocking Linked-Lists"
     - Fomitchev 2004, "Lock-free linked lists and skip lists"
```

关键操作与性能特征：无锁链表的插入在**低争用**下非常快（纯粹的读遍历 + 一次 CAS）；在**高争用**下，遍历本身依旧要读指针链（受内存延迟支配），而"提交点"集中在被修改的那一个 `next` 指针上，所以**操作的正确性靠单点 CAS，性能上限则由内存延迟与 CAS 失败率共同决定**。删除之所以难，本质原因是：**一次逻辑删除需要修改两个指针（前驱的 next、后继的链接），而 CAS 只能原子地改一个字**——这就是后面 Transactional Memory 要解决的结构性问题。

2.10 现实检验：无锁一定更快吗?

讲义给出的判断（三段式），值得逐字记住：

1. **在"只有你的程序在用这台机器"的场景下**（科学计算、图形学、数据分析等课程内的优化场景），**写得好的加锁代码可以与无锁代码一样快，甚至更快**（"well written code with locks can be as fast (or faster) than lock-free code"），而且**通常简单得多**。
2. 讲义引用的量化证据是 **Hunt 2011**, *"Characterizing the Performance and Energy Efficiency of Lock-Free Data Structures"*：图中纵轴是**相对 pthread mutex 运行时间的归一化值**（1.0 = 与 pthread mutex 持平，<1 表示更快），横轴是线程数，曲线分为 **lf（lock free）** 与 **fg（fine grained lock）** 两组，覆盖 **Queue / Linked List / Dequeue** 三类结构。（注：讲义抽取文本中不含该图的坐标数值，本笔记不虚构具体倍数。）
3. **但确实存在加锁代码会遭遇"棘手的性能问题"的场景**：多道程序（multi-programmed）环境中，线程在**临界区内**可能缺页、被抢占等，从而产生 **priority inversion（优先级反转）**、**convoying（护航效应）**、**在临界区内崩溃** 等在操作系统课里讨论的问题。这正是数据库、Web 服务器这类"程序里有很多线程、又无法独占机器"的场景需要无锁/非阻塞结构的原因。

讲义的总结四条： – 用细粒度锁降低共享数据结构操作上的争用（最大化并行度），但细粒度会增加代码复杂度（易错）与执行开销。 – 无锁数据结构是非阻塞方案，用于避免锁的一些开销与陷阱；但实现微妙（在无锁设定下保证正确性本身有自己的开销）。 – 在**现代的宽松一致性（relaxed consistency）硬件上，仍需要恰当的内存栅栏**（讲义在 SPSC 队列、栈代码处反复标注 "Assume a sequentially consistent memory system for now"）。 – **无锁设计并不能消除争用**：重争用下 CAS 会失败，需要自旋重试。

---

3. 代码示例与性能分析

3.1 示例 1：细粒度（hand-over-hand）锁链表 vs 单一全局锁

代码做什么？：实现同一个有序单链表（支持 `insert`），提供两种加锁策略：`CoarseList`（一把 `std::mutex` 保护整个链表）与 `FineList`（每节点一把 `std::mutex`，hand-over-hand 遍历）。主程序用相同的随机操作序列分别跑两种实现，报告吞吐（ops/s）与"整条链表被串行化的比例"。

```
// 编译: g++ -O3 -std=c++17 -pthread fg_list.cpp -o fg_list
//       (务必用 -O3: -O0 下同步以外的开销会掩盖细粒度锁的真实收益/代价)
// 运行: ./fg_list 8 50000
//       参数 <线程数> <总操作数>。默认值域为 0..4095 (链表最多 4096 个节点),
//       使示例能在秒级跑完; 若把值域改成 1<<20 (链表会长到几十万节点),
//       细粒度版本会因为"每步 2 次原子 RMW × 极长遍历"而慢到不可用 ——
//       这正是 4.3 节要定量说明的那个结论。

#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <thread>
#include <mutex>
#include <vector>
#include <atomic>
#include <chrono>
#include <random>
#include <algorithm>

// ---------------------------------------------------------------------------
// 方案 A: 粗粒度 —— 一把锁保护整个链表
// ---------------------------------------------------------------------------
struct CNode { int value; CNode* next; };

struct CoarseList {
    CNode* head;                // 哨兵节点 (值 = INT_MIN)
    std::mutex m;
    void init() { head = new CNode{INT32_MIN, nullptr}; }
    bool insert(int value) {
        std::lock_guard<std::mutex> g(m);          // 整个遍历 + 插入都在临界区内
        CNode* prev = head;
        CNode* cur  = prev->next;
        while (cur && cur->value < value) { prev = cur; cur = cur->next; }
        if (cur && cur->value == value) return false;   // 去重
        prev->next = new CNode{value, cur};
        return true;
    }
};

// ---------------------------------------------------------------------------
// 方案 B: 细粒度 —— 每节点一把锁, hand-over-hand 遍历
//   不变量: 持有 prev->m  ==> prev->next 及其指向节点的身份不会被改动
//   锁序:   永远沿 head -> tail 方向加锁 ==> 资源依赖图无环 ==> 无死锁
// ---------------------------------------------------------------------------
struct FNode { int value; FNode* next; std::mutex m; };

struct FineList {
    FNode* head;                // 哨兵节点, 其锁同样遵守不变量
    void init() { head = new FNode{INT32_MIN, nullptr, {}}; }
    bool insert(int value) {
        FNode* n = new FNode{value, nullptr, {}};
        FNode* prev = head;
        prev->m.lock();                     // 从 head 开始, 锁序固定
        FNode* cur = prev->next;
        if (cur) cur->m.lock();
        while (cur && cur->value < value) {  // 只要还持有 prev 的锁, 读 cur->next 就是安全的
            prev->m.unlock();               // 放开"身后"的锁: hand-over-hand
            prev = cur;
            cur  = cur->next;
            if (cur) cur->m.lock();         // 先锁下一个, 再进入下一轮
        }
        bool ok = true;
        if (cur && cur->value == value) {
            ok = false;                     // 已存在
        } else {
            n->next = cur;
            prev->next = n;                 // 只改 prev->next, 因此只需持有 prev 的锁
        }
        prev->m.unlock();
        if (cur) cur->m.unlock();
        if (!ok) delete n;
```

```
        return ok;
    }
};

// ------------------------------------------------------------------------
// 驱动：T 个线程各自随机插入 N/T 个值
// ------------------------------------------------------------------------
template <typename F>
static double run(const char* name, F&& body, int T, int per_thread) {
    std::atomic<bool> go{false};
    std::vector<std::thread> ts;
    auto t0 = std::chrono::steady_clock::now();
    for (int t = 0; t < T; ++t)
        ts.emplace_back([&, t] {
            while (!go.load(std::memory_order_acquire)) {}       // 同时起跑
            std::mt19937 rng(1234 + t);
            std::uniform_int_distribution<int> d(0, (1 << 12) - 1);   // 值域 4096
            for (int i = 0; i < per_thread; ++i) body(d(rng));
        });
    go.store(true, std::memory_order_release);
    for (auto& th : ts) th.join();
    double s = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
    printf("%-28s T=%-3d  time=%8.3f s   throughput=%10.0f ops/s\n",
           name, T, s, (double)(T * per_thread) / s);
    return (double)(T * per_thread) / s;
}

int main(int argc, char** argv) {
    int T = (argc > 1) ? atoi(argv[1]) : 8;
    int total = (argc > 2) ? atoi(argv[2]) : 50000;
    int per = total / T;

    CoarseList cl; cl.init();
    run("coarse: one global lock", [&](int v) { cl.insert(v); }, T, per);

    FineList fl; fl.init();
    run("fine: hand-over-hand",     [&](int v) { fl.insert(v); }, T, per);
    return 0;
}
```

【代码做什么？】 1. 两个结构都使用**哨兵头节点**（值为 `INT32_MIN`）——这让"在表头之前插入"这一特殊情况消失，`insert` 中不再需要单独的头部处理分支，也保证 `prev` 永不为 `NULL`，从而"每节点一把锁"的方案总能先锁到 `prev`。 2. `CoarseList::insert` 在**整个遍历 + 插入**期间持有唯一一把锁：逻辑简单、绝对安全、但所有线程在一条队列上排队。 3. `FineList::insert` 的关键循环是 hand-over-hand：`prev->m.unlock(); prev = cur; cur = cur->next; if (cur) cur->m.lock();` – **释放"身后"的锁**（`prev` 变成下一轮的"更后面的那个节点"），使得另一个线程可以从被释放的位置继续前进； – **先锁下一个再进入下一轮**，因此该线程**始终至少持有一把锁**，绝不会出现"手上无锁而手上指针可能失效"的窗口； – 读取 `cur->next` 时持有的是 `prev` 的锁，而 `prev->next` 由该锁保护，所以 `cur` 的身份是稳定的（这就是 2.4 节的不变量）。 4. 驱动代码用 `go` 标志让所有线程**同时起跑**，否则第一个线程可能在其它线程创建前就跑完，测量结果会严重失真（这是并行基准测试最常见的错误之一）。

【并行机制与性能解说】

设平均每次插入的遍历长度为 `L`（本示例随机值域约 2^20，随着链表增长 `L` 上升；稳态下 `L ≈ n/2`，`n` 为不同值数量），操作数 `M = T × per`，节点数为 `n`。

- **Work（总工作量）**：
- 粗粒度：`W_coarse = Θ(M·L)` 次节点访问 + `M` 次锁获取（每次是 1 次原子 RMW + 1 次普通写）。
- 细粒度：`W_fine = Θ(M·L)` 次节点访问 **+ Θ(M·L) 次锁获取与释放**，即**每个遍历步多了 2 次原子 RMW**。这是本示例最核心的成本项：`L ≈ 2^19/2 ≈ 260 K`（当 `n` 很大时）时，每次插入的原子操作数从 **1 次变成约 5×10^5 次**，代价高到足以让细粒度版本**慢几个数量级**。
- **Span（关键路径）**：单次插入的串行链是"沿链表走 `L` 步"，每步依赖前一步的 `next` 读取（**依赖链**，无法乱序跨越），因此 `Span_op = Θ(L)` 步；每一步在细粒度版本里是"锁获取 + 读 + 锁释放"，在粗粒度版本里是"读"。整体 `Span = Θ(M·L + (锁交接总时长))`。
- **并行度 = Work/Span**：单次操作内部 `Θ(M·L)/Θ(L) = Θ(M)` 看起来很高，但**这个数字具有误导性**：真正的并行度受限于"**不同操作能不能同时推进**"。在细粒度方案里，两个操作的进度被它们之间的节点锁耦合；`L` 个线程在链表上形成**流水线（pipeline）**，稳态吞吐上界为 `Throughput_fine <= 1 / t_hop 其中 t_hop ≈ (锁获取 + 释放) ≈ 2 次缓存行交接` 而粗粒度方案的吞吐上界是 `1 / (t_lock +`

`t_critical)`，其中 `t_critical` 包含整条遍历 —— **从量级上看，细粒度把"每次操作一段长临界区"变成"每一步一小段临界区"，把长临界区拆成流水线**，这正是它的价值。

- **瓶颈诊断**：
- **每步原子操作（本示例的主导瓶颈）**：若 `t_hop ≈ 40 ns`（同 socket L3 命中、无争用），仅锁开销就是 `L × 40 ns`；若 `L = 1000`，单次插入 = 40 μs，吞吐上限 25 K ops/s/线程。**在长链表上，细粒度锁的 hop 成本是纯遍历成本的 10–100 倍**。
- **伪共享（false sharing）**：`FNode` 中 `value` / `next` / `m` 同在几十字节内，**相邻节点常常落在同一 64 B 缓存行**。两个线程锁的是不同节点、却争同一缓存行 ⇒ 完全违背了"细粒度"的初衷，实测可能比粗粒度还慢。工程修正：把锁字段与数据分离（两个平行数组/结构体），或让每把锁独占一个缓存行（`alignas(64)`），代价是内存占用暴涨。
- **负载不均**：`std::uniform_int_distribution` 让插入位置均匀分布，因此**越靠表头附近的节点锁越热**（所有操作都必须从 head 出发，head 的锁是全局热点）。这个"head 锁串行化"与任务粒度过粗造成的串行化在数学上是同一件事。
- **内存延迟**：`cur->next` 是典型的指针追逐（pointer chasing），每步几乎必然 L2/LLC/DRAM 缺失，`t_hop` 中真正的大头常常是**内存延迟**而不是原子操作本身。这也是为什么本示例测出来的"吞吐"随链表变长而急剧下降——它是**延迟受限（latency-bound）**的。

> **可扩展性上限的结论**：细粒度锁不改变单次操作 `Θ(L)` 的**延迟**（指针追逐的依赖链无法并行化），它改变的是**吞吐**——把串行临界区变成流水线。因此在本例这种"值域极宽、链表极长"的设定下，**细粒度锁带来的原子操作开销会压过流水线收益**；而在"链表较短、操作集中在不同区域"的设定下（工程中的真实场景），细粒度才明显占优。这个结论正是讲义"middle-ground solution"提示的定量依据。

本示例的实际输出（直接观察"细粒度反而更慢"）：

```
$ ./fg_list            # 默认 T=8, 总操作 50000, 值域 0..4095
coarse: one global lock      T=8    time=   0.512 s   throughput=     97730 ops/s
fine: hand-over-hand         T=8    time=   4.329 s   throughput=     11551 ops/s
```

细粒度版本比单把大锁慢 8.5 倍。 这不是代码写错了，而是 2.4 节那张"代价表"的直接后果：链表稳态长度约 4000 个节点（值域 4096），平均遍历 `L ≈ 2000` 步 ⇒ 每次插入在细粒度版本里要付出 **约 4000 次原子 RMW**（每步 lock + unlock），而粗粒度版本只付出 **1 次**锁获取。遍历本身的指针追逐（每步几纳秒，因为整个链表只有约 100 KB、基本常驻 L2）远小于原子操作的开销，所以**同步成本完全主导**。 这个实测结果也提醒我们注意与 4.3 节数值算例的**假设差异**：4.3 假设"每次节点访问 90 ns（DRAM 延迟）"，而本例的链表小到能放进 L2，节点访问只要几纳秒——**在"遍历很便宜"的情形下，细粒度锁的额外开销占比更大，输得更惨；只有在"遍历很贵（大表、随机分布、频繁缺页）"且"线程访问区域分散"时，细粒度锁拆出的流水线才可能赢回来。** 换句话说：**是否值得细粒度，取决于"每次操作省下的等待时间"与"每一步多付的原子操作代价"哪个更大——这正是 work–span 与 Amdahl 要算的东西，而不是靠"锁越小越好"的直觉。**

3.2 示例 2：无锁栈 —— 用"索引 + 版本号打包"彻底解决 ABA

代码做什么？：实现一个**多生产者 / 多消费者**的无锁栈。为避免 ABA 与 use-after-free 两个问题，示例采用讲义所述的第三种解法思路：**节点来自预分配数组（永不 `delete`）**，栈顶指针用**一个 64 位字打包** `(tag:32 | index:32)`，其中 `tag` 是版本号（对应讲义代码里的 `pop_count`），于是**一次 64 位 CAS 就能同时校验"栈顶是谁"和"栈自我们读取以来有没有变过"**——这正是讲义所说"保证两个字段在内存中连续，用单条宽 CAS 即可"的工程落地。

```cpp
// 编译: g++ -O3 -std=c++17 -pthread lf_stack.cpp -o lf_stack
// 运行: ./lf_stack 8 2000000          # 8 个线程，总共 200 万次 push + 200 万次 pop

#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <atomic>
#include <thread>
#include <vector>
#include <chrono>
#include <algorithm>

// ---- 打包格式： [ tag:32 | index:32 ]，index=0xFFFFFFFF 表示空栈 ----
static constexpr uint32_t NIL = 0xFFFFFFFFu;
static inline uint64_t pack(uint32_t tag, uint32_t idx) { return ((uint64_t)tag << 32) | idx; }
static inline uint32_t tag_of(uint64_t v) { return (uint32_t)(v >> 32); }
static inline uint32_t idx_of(uint64_t v) { return (uint32_t)(v & 0xFFFFFFFFu); }

static constexpr uint32_t CAP = 1u << 22;      // 约 419 万个节点槽位（预分配，永不释放）

struct Node { uint32_t next; int value; };     // next 用索引而非指针：可以用一条 CAS 打包

struct LFStack {
    Node*               pool;                  // 节点池（只分配一次）
    std::atomic<uint64_t> head;                // 打包的 (tag, index)

    void init() {
        pool = (Node*)std::malloc(sizeof(Node) * CAP);
        head.store(pack(0, NIL), std::memory_order_relaxed);
    }
    // 返回 false 表示节点池耗尽（本示例中容量足够，用于演示边界）
    bool push(uint32_t idx, int value) {
        uint64_t old = head.load(std::memory_order_relaxed);
        for (;;) {
            pool[idx].next  = idx_of(old);                 // 只在本地可见前先接好链
            pool[idx].value = value;
            uint64_t desired = pack(tag_of(old) + 1, idx); // tag++ ：让 ABA 永远不可能相等
            if (head.compare_exchange_weak(old, desired,
                                           std::memory_order_release,
                                           std::memory_order_relaxed))
                return true;                               // 成功；失败时 old 已被刷新
        }
    }
    // 返回 -1 表示空栈；否则返回被弹出节点的索引
    int64_t pop() {
        uint64_t old = head.load(std::memory_order_acquire);
        for (;;) {
            uint32_t idx = idx_of(old);
            if (idx == NIL) return -1;
            uint32_t nxt = pool[idx].next;
            uint64_t desired = pack(tag_of(old) + 1, nxt);
            if (head.compare_exchange_weak(old, desired,
                                           std::memory_order_acquire,
                                           std::memory_order_relaxed))
                return idx;                                // 只有我能返回这个 idx
        }
    }
};

int main(int argc, char** argv) {
    int T   = (argc > 1) ? atoi(argv[1]) : 8;
    long N  = (argc > 2) ? atol(argv[2]) : 2000000;
    LFStack s; s.init();

    // 每线程负责一段不重叠的节点槽位，避免"节点同时被两个线程 push 两次"
    std::vector<uint32_t> base(T);
    for (int t = 0; t < T; ++t) base[t] = (uint32_t)((uint64_t)N * t / T);

    std::atomic<long> pushed{0}, popped{0};
    std::atomic<long long> sum{0};
    std::vector<std::thread> ts;
```

```
    auto t0 = std::chrono::steady_clock::now();

    for (int t = 0; t < T; ++t) {
        ts.emplace_back([&, t] {
            long per = N / T;
            uint32_t cursor = base[t];
            for (long i = 0; i < per; ++i) { s.push(cursor + (uint32_t)i, (int)(cursor + i)); pushed++; }
            // 第一阶段：所有线程 push 完毕后（本示例用 join 分阶段），再统一 pop
        });
    }
    for (auto& th : ts) th.join();
    double t_push = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
    printf("push done : %ld ops in %.3f s  => %.1f M ops/s\n", N, t_push, N / t_push / 1e6);

    // 第二阶段：并发 pop，直到栈空；统计弹出的元素个数与校验和
    ts.clear();
    auto t1 = std::chrono::steady_clock::now();
    for (int t = 0; t < T; ++t) {
        ts.emplace_back([&] {
            for (;;) {
                int64_t idx = s.pop();
                if (idx < 0) break;
                sum += s.pool[idx].value;
                popped++;
            }
        });
    }
    for (auto& th : ts) th.join();
    double t_pop = std::chrono::duration<double>(std::chrono::steady_clock::now() - t1).count();

    long long expect = 0;                       // 校验和：所有 push 过的值之和
    for (long i = 0; i < N; ++i) expect += i;
    printf("pop  done : %ld ops in %.3f s  => %.1f M ops/s\n", (long)popped, t_pop, (double)popped / t_pop / 1e6);
    printf("pushed=%ld popped=%ld  %s   (checksum %s)\n",
           (long)pushed, (long)popped, pushed == popped ? "COUNT OK" : "COUNT MISMATCH",
           (long long)sum == expect ? "OK" : "MISMATCH");
    return (long long)sum == expect && pushed == popped ? 0 : 1;
}
```

【代码做什么？】 1. **打包（packing）**：`head` 是**单个 64 位原子字**，高 32 位是版本号 `tag`，低 32 位是栈顶节点在 `pool` 中的**索引**。`NIL = 0xFFFFFFFF` 表示空栈。 2. **push**：`load` 当前栈顶 → 把新节点 `next` 指向它 → 计算 `desired = (tag+1, idx)` → `compare_exchange_weak`。**注意 `tag` 每次都自增**：即使 `idx` 因为节点被复用而"转了一圈回到同一个值"，`tag` 也不会相同，因此**"ABA 相等"永远不可能发生**——这等价于讲义里"用 `pop_count` 校验"的方案，但不需要机器提供通用 DCAS，因为两个字段被塞进了同一个字。 3. **`compare_exchange_weak` 的失败语义**：失败时它会**把内存当前值写回 `old`**，所以循环里**不需要**再手动 `load` 一次（这是 C++ 与手写 `atomicCAS` 的一个重要区别；手写版本必须显式 `old = *addr` 重读，正如讲义 `atomic_max` 的代码那样）。 4. **内存序**：`push` 用 `release` 成功序（保证"写好的 `pool[idx].next/value` 在被别人看到栈顶指针之前可见"），`pop` 用 `acquire`（保证读到栈顶指针之后看到的是初始化完成的数据），失败路径用 `relaxed`（失败不需要任何排序语义，这是最常见的性能优化点）。 5. **无 use-after-free**：节点**永不释放**（`pool` 只 `malloc` 一次），从根上消除了"读取已释放内存"的问题。代价是**内存占用固定且无法回收**（`CAP` 个节点），这也是讲义所说"用谨慎的节点分配/复用策略解决 ABA"的真实代价。 6. **正确性自检**：`popped == pushed` 与 `sum == expect` 两个断言能抓住绝大多数（虽然不是全部）无锁逻辑错误——无锁数据结构的测试必须包含这种**计数 + 校验和**式的全局不变量检查。 7. **驱动方式说明**：本示例把 push 与 pop **分成两个阶段**（中间 `join`），是为了让校验和可判定（如果 push/pop 完全混跑，只要每个节点最终被弹出一次，校验和仍然成立——读者可以自行把两阶段改成混跑并验证这一点）。**不允许**两个线程 push 同一个槽位，所以每个线程使用互不重叠的槽位区间。

【并行机制与性能解说】

设线程数 `P`、总操作数 `N`。

- **Work（总工作量）**：每次成功操作 = 1 次原子读 + 若干次失败的 CAS + 1 次成功 CAS。在完全无争用时 `W = Θ(N)`；在 P 个线程全部争抢同一个 `head` 时，**总 CAS 尝试次数 ≈ `Θ(P·(P+1)/2)` 每次"一人成功一轮"**（推导见 4.2 节），即**每次操作的期望尝试次数 ≈ `Θ(P)`**。因此 `W = Θ(N·(1 + 争用失败次数)) = Θ(N·P)`（最坏情况下是 P 的线性倍）。
- **Span（关键路径）**：`pop` / `push` 的**算法关键路径是 `Θ(1)`**——一次读 + 一次 CAS，不依赖任何其它线程。这是无锁结构最漂亮的性质：**算法层面没有串行链**。
- **并行度 = Work/Span**：形式上 `Θ(N·P)`，但**真正决定吞吐的是硬件**：所有 `head` 的 CAS 都落在**同一个缓存行**上，而该行的**所有权迁移被硬件完全串行化**。因此稳态吞吐的物理上界是 `Throughput_max ≈ 1 / (t_line · E[attempts per op])`（`t_line` = 该缓存行的单次交接延迟）而不是"线性于线程数"。

- **瓶颈诊断**：
- **缓存行乒乓（cache-line ping-pong）**：`head` 所在的那一行在 P 个核的 L1 之间来回迁移。P=8 且 `t_line ≈ 80 ns` 时，即使每次只尝试一次 CAS，理论上界也只有 `1/(8 × 80ns) ≈ 1.5 M ops/s`（每核约 190 K ops/s）——**远低于单线程无争用时的 CAS 速率（~50 M ops/s）**。
- **失败尝试是纯浪费**：失败者付出了完整的一次行迁移代价（把行抢到自己缓存），却没有推进状态，还顺手把成功者的副本作废了。**这就是讲义那句"a lock-free design does not eliminate contention"的硬件解释**。
- **内存序的代价**：把成功序从 `acquire/release` 换成 `seq_cst`（C++ 默认）在 x86 上会让 push/pop 各多一条 `lock` 前缀指令或屏障，实测吞吐下降通常可观；但**省错内存序会造成正确性错误**——这是本讲最常见的"两难陷阱"。
- **栈的 LIFO 与缓存局部性**：`pool` 里的节点按分配顺序排列，LIFO 弹出顺序对缓存局部性反而友好（刚 push 的节点最近被访问过，很可能还在 L1/L2）；而 FIFO 队列会让生产/消费两端访问相隔很远的内存，**这是队列与栈在内存系统上的不对称性**。
- **性能是"机器的性质"，正确性是"算法的性质"**：本示例的输出中，`COUNT OK / checksum OK` 是**算法正确性**的判据（在任何机器、任何线程数下都必须成立）；而 `M ops/s` 的**绝对值高度依赖机器与当前负载**——在同一台共享节点上重复运行，吞吐可以在几 M ops/s 到几十 M ops/s 之间波动。因此**无锁数据结构的工程实践应当把"正确性自检"做成常驻的断言（每次测试都跑），而把"性能数字"当作需要在受控环境下重测的量**。本示例的意义正在于：把 4.2 节那个"$P^2/2$ 次尝试"的抽象模型落成一个可运行的、正确性可自动判定的程序，读者可自行改变线程数观察吞吐曲线的形状。

3.3 示例 3：锁实现微基准 —— T&S vs TTS vs ticket lock（把一致性流量测出来）

代码做什么？：用同一台机器、同一段临界区，测量三种锁在 T 个线程下的"每次加解锁耗时"和"**每次成功获取锁所需的原子尝试次数**"。第二个指标是本示例的关键：它直接把讲义"O(P) 失效 vs 每次测试都失效"的论证**变成一个可测量的数字**。

```cpp
// 编译: g++ -O3 -std=c++17 -pthread lockbench.cpp -o lockbench
// 运行: ./lockbench 16 200000 0         # 16 线程, 20 万次 lock/unlock, 临界区长度参数 = 0
//       第三个参数 = 临界区里做多少次"依赖整数加法"（每次约 0.3 ns，用来把临界区变长）
//
// 两条必须遵守的测量纪律:
//  (1) 计数器必须放在"每线程私有"的位置（见 bench 中的 lr/la)。若把 long reads 写进
//       锁对象所在的缓存行，自旋时的 reads++ 会不断抢走锁所在行的所有权
//       => 观测仪器自己制造伪共享 => 实测性能被测坏（本示例实测恶化约 10 倍)。
//  (2) 锁内部"两个热变量"（如 ticket lock 的 next 与 serving）必须分处不同缓存行。
//       否则每次取号（写 next）都会作废所有在 serving 上自旋的等待者的副本。
//       => 见 TicketLock 中的 alignas(64)。

#include <cstdio>
#include <cstdlib>
#include <atomic>
#include <thread>
#include <vector>
#include <chrono>
#include <mutex>

static long g_cs_iters = 0;    // 临界区长度参数（依赖加法次数)，由 argv[3] 设置

// 模拟临界区里的工作：依赖加法链，编译器无法并行化/消除
static inline long critical_section(long iters) {
    long a = 1;
    for (long i = 0; i < iters; ++i) a = a * 3 + 1;   // 约 1 周期/次 ≈ 0.3 ns/次
    return a;
}

// ---------------- 1. test-and-set 锁：每次尝试都是原子 RMW ----------------
struct alignas(64) TASLock {
    std::atomic<int> flag{0};
    void lock(long& /*reads*/, long& attempts) {
        while (flag.exchange(1, std::memory_order_acquire) == 1) attempts++;
    }
    void unlock() { flag.store(0, std::memory_order_release); }
};

// ---------------- 2. test-and-test-and-set 锁：先本地读，再尝试 RMW ----------------
struct alignas(64) TTSLock {
    std::atomic<int> flag{0};
    void lock(long& reads, long& attempts) {
        for (;;) {
            while (flag.load(std::memory_order_relaxed) == 1) reads++;      // 本地缓存里的普通读
            if (flag.exchange(1, std::memory_order_acquire) == 0) return;  // 抢占
            attempts++;                                                    // 抢失败
        }
    }
    void unlock() { flag.store(0, std::memory_order_release); }
};

// ---------------- 3. ticket lock：取号是原子操作，等待只是读 ----------------
//   next 与 serving 必须分处不同缓存行：它们被不同角色（取号者 / 所有人）高频访问
struct alignas(64) TicketLock {
    alignas(64) std::atomic<unsigned> next{0};      // 只被"取号"这一动作写
    alignas(64) std::atomic<unsigned> serving{0};   // 被所有等待者读、被持有者写
    void lock(long& reads, long& /*attempts*/) {
        unsigned my = next.fetch_add(1, std::memory_order_relaxed);
        while (serving.load(std::memory_order_acquire) != my) reads++;     // 只读，不产生 RMW
    }
    void unlock() { serving.fetch_add(1, std::memory_order_release); }
};

// ---------------- 4. 基准线：pthread mutex ----------------
struct alignas(64) MutexLock {
    std::mutex m;
    void lock(long& /*reads*/, long& /*attempts*/) { m.lock(); }
    void unlock() { m.unlock(); }
};

std::atomic<long> g_sink{0};    // 防止临界区被优化掉（多线程累加，必须原子）
```

```
template <typename L>
static void bench(const char* name, int T, long total) {
    L lock;
    std::vector<long> reads(T, 0), attempts(T, 0);
    std::atomic<bool> go{false};
    std::vector<std::thread> ts;
    long per = total / T;

    auto t0 = std::chrono::steady_clock::now();
    for (int t = 0; t < T; ++t) {
        ts.emplace_back([&, t] {
            while (!go.load(std::memory_order_acquire)) {}
            long lr = 0, la = 0, local_sink = 0;      // 每线程私有计数器（不与锁共享缓存行）
            for (long i = 0; i < per; ++i) {
                lock.lock(lr, la);
                local_sink += critical_section(g_cs_iters);   // "临界区"
                lock.unlock();
            }
            g_sink.fetch_add(local_sink, std::memory_order_relaxed);
            reads[t]    = lr;
            attempts[t] = la;
        });
    }
    go.store(true, std::memory_order_release);
    for (auto& th : ts) th.join();
    double s = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();

    long R = 0, A = 0;
    for (int t = 0; t < T; ++t) { R += reads[t]; A += attempts[t]; }
    long done = per * T;
    printf("%-14s T=%-3d cs=%-6ld %8.0f ns/op-air   attempts/acq=%7.2f  reads/acq=%9.2f\n",
           name, T, g_cs_iters, s / done * 1e9, (double)A / done, (double)R / done);
}

int main(int argc, char** argv) {
    int  T     = (argc > 1) ? atoi(argv[1]) : 8;
    long total = (argc > 2) ? atol(argv[2]) : 200000;
    g_cs_iters = (argc > 3) ? atol(argv[3]) : 0;
    bench<MutexLock> ("pthread-mutex", T, total);
    bench<TASLock>   ("test-and-set",  T, total);
    bench<TTSLock>   ("test-and-test", T, total);
    bench<TicketLock>("ticket",        T, total);
    return 0;
}
```

【代码做什么？】 1. 四种锁**共用同一段可忽略的临界区**，测量的是"纯同步开销"（与讲义引用的 Culler/Singh/Gupta 基准同一思路："Critical section time removed so graph plots only time acquiring/releasing the lock"）。 2. `TASLock` 每次循环都执行 `exchange`（原子 RMW ⇒ 一次所有权获取），`attempts` 直接计数这类昂贵操作。 3. `TTSLock` 分两层：内层 `load` 是**本地缓存里的普通读**（只要行还在 S 态就一直命中，几乎零流量），外层 `exchange` 才产生所有权获取。`reads` 与 `attempts` 分别是"廉价读"和"昂贵 RMW"的计数——**两者的比例就是讲义那套流量论证的实验证据**。 4. `TicketLock` 的等待路径**只有 `load`**（讲义原文："No atomic operation needed to acquire the lock (only a read)"），因此它的 `attempts` **永远保持 0**，而 `reads` 是自旋读次数——**"昂贵 RMW = 0"正是票号锁的全部卖点**。 5. `MutexLock` 作为基准线（现代 glibc 的 `pthread_mutex` 在无争用时走 futex 快路径，几乎与一条 CAS 同价；有争用时进入内核挂起，从而把"自旋浪费"换成"上下文切换开销"）。 6. **计数器必须传成"每线程的栈上局部变量"**（`lock.lock(lr, la)`）。如果把 `long reads` 写成锁结构体的成员，`reads++` 就会写在**锁所在的那条缓存行**上：每个自旋迭代都让等待者把该行的所有权抢过来，从而**作废所有其它等待者的 S 态副本**——观测仪器自己制造了伪共享。这一条在实测中被明确验证过：把计数器放进锁对象时，各锁的耗时普遍恶化（自旋次数最多的 ticket lock 恶化接近 10 倍）。**这是"伪共享"最有教育意义的一个实例：连"用来观测同步开销的代码"都会改变同步开销。** 同理，`TicketLock` 里的 `next` 与 `serving` 也必须用 `alignas(64)` 分处不同缓存行——否则每次取号（写 `next`）都会作废所有在 `serving` 上自旋的等待者的副本。

1. **应该观察什么（可复现的定性趋势）**：
2. **`test-and-set` 的 `attempts/acq` 随临界区长度显著上升**：临界区为空时约等于 1（几乎没有等待者），把 `cs` 参数增大到千次依赖加法时上升到上百次。这与 4.1 节模型"每次获取期间的尝试次数 ∝ 持锁时长"的预测一致——**这是对"T&S 无谓消耗互连带宽"最直接的实验证据**。
3. **`test-and-test-and-set` 的 `attempts/acq` 始终接近 0，而 `reads/acq` 很大**：等待全部由**本地缓存里的普通读**承担，昂贵的 RMW 只在真正尝试抢占时发生。这正是讲义"每次释放 O(P) 次失效（而不是每次测试都 O(P)）"的实测体现。
4. **`ticket` 的 `attempts/acq` 恒为 0**：等待路径上**完全没有原子 RMW**（"只有一次读"），这一点由代码结构保证。

5. **不要从一次运行里读绝对值下结论**：本示例在一台**共享的登录节点**上运行时，同一配置的重复测量可以相差数倍（例如 `pthread-mutex` 在 29 ns 与 400 ns 之间波动），而 `ticket` 的绝对耗时在该机器上并不比 TTS 更好——这说明"**O(P) 互连流量**"（讲义对 ticket lock 的论断）与"**最低的交接延迟**"是**两个不同的评价轴**：ticket lock 用严格 FIFO 把交接彻底串行化，一旦某位持票者被抢占/变慢（convoying），后面所有人都要等它，这在多道程序的共享机器上会放大成微秒级延迟。**要在自己的机器上得到可信结论，必须：①把线程绑定到独占的物理核（`taskset`），②多轮取最小值，③把临界区长度当作自变量扫一遍。** 这也是讲义在"desirable lock performance characteristics"里把**低延迟**与**低互连流量**并列为两条独立目标的原因。

【并行机制与性能解说】

设 `P` 个线程各做 `N` 次 lock/unlock，总操作数 `M = P·N`。

- **Work**：`W = Θ(M)` 次"锁协议步骤"，但每次步骤的成本差异极大：TAS 的每一步是**原子 RMW**，TTS 的"读"步骤是**普通 load**（便宜 10–100 倍），ticket 的"读"步骤同样是普通 load。
- **Span**：**关键路径 = 所有临界区的串行化链**：`Span = Θ(M · t_handoff)`，其中 `t_handoff` 是"锁从一个线程交接给下一个线程"的延迟。**这是本示例最重要的洞察**：`M` 次操作**必须全部串行**，因为互斥的语义就是"一次一个"。
- **并行度 = Work/Span = Θ(1)。形式上的并行度是 1** —— 这个微基准在算法层面**完全不可并行**，因此"加线程能不能变快"的答案取决于 `t_handoff` 是否随 P 变化：**如果实现是好的，加线程不该让每次加解锁变慢；如果是 TAS，`t_handoff` 随 P 增长（O(P) 条失效事务），于是总时间随 P 超线性上升**。这正是讲义图（时间 vs 处理器数）所展示的现象。
- **瓶颈诊断**：
- **互连争用（interconnect contention）**：讲义明确写道"Interconnect contention increases amount of time to transfer lock (**lock holder must wait to acquire bus to release**)"，并补充"contention also slows down execution of the critical section"（未在该图中显示）。本示例中可观察到的对应现象是：**TAS 的 `attempts/acq` 随 `cs`（持锁时长）增大而上升**（实测从"临界区为空"时的约 1 上升到上千次依赖加法时的上百次），即等待者的每一次无效 RMW 都在消耗互连带宽。
- **`attempts/acq` 与 `reads/acq` 的分离**：TTS 与 TAS 的差异不会体现在"代码行数"上，而体现在这两个计数器的量级上——**TAS 的失败尝试数随争用增长，TTS 的昂贵 RMW 数始终被压在"每次释放一次"的水平**。这是把讲义里 O(P)/O(P²) 的论断落到自己机器上验证的方法。
- **公平性与延迟是两个轴**：ticket lock 是 FIFO 的，等待路径没有任何原子 RMW（`attempts/acq` 恒为 0），但它把交接**彻底串行化**：任何一位持票者变慢，后面所有人都要等（convoying）。因此在本示例这类"临界区极短、线程数少"的场景里，ticket lock 未必最快——**"低互连流量"不等于"低交接延迟"**（讲义把这两条并列为独立的锁评价目标）。相反，TAS/TTS 无公平性保证，可能出现某线程长时间抢不到（饥饿），实测表现为**各线程完成时间的巨大方差**（可扩展示例统计每线程耗时）。
- **测量纪律（本示例最实用的收获）**：① `g_sink` 用于防止空临界区被完全消除，且**必须用 `-O3`**；②计数器必须**每线程私有**、锁的热字段必须**分处不同缓存行**（见第 2、6 条脚注）；③线程应绑定到独占物理核、多轮取最小值——在共享机器上单次运行的绝对值可以相差数倍，**任何"某某锁更快"的结论都必须附带测量条件**。

---

4. 性能模型与复杂度分析

本节的所有数字都基于一组**显式假设**（量级取自现代多核服务器上普遍观测到的水平，用于建立直觉；读者可用 3.3 的微基准在自己的机器上实测替换）。工作、跨度、并行度的定义遵循课程使用的 work–span 模型。

假设参数表（下文所有算例共用）：

参数	符号	取值	说明
核数	`P`	32（部分算例用到 8 / 64）	单 socket 多核（如 32C/64T 级处理器）
时钟频率	`f`	3.0 GHz	1 周期 ≈ 0.333 ns
缓存行大小	`B_line`	64 B	一致性协议的基本单位
同一缓存行在核间"交接"延迟	`t_line`	80 ns	含 RFO + 数据返回；跨 socket 可达 150–250 ns
本地 L1 命中 CAS/RMW	`t_cas_local`	6 ns（≈20 周期）	行已独占时
DRAM 访问延迟	`t_dram`	90 ns	用于指针追逐分析
内存带宽	`BW`	20 GB/s	单 socket 持续可达到的有效带宽量级
向量宽度 × FMA	`W·k`	8 × 2	用于 Roofline 脊点
锁交接（理想实现）	`t_handoff`	2 × `t_line` = 160 ns	一次失效广播 + 一次读/升级

4.1 算例 A：锁的互连流量模型（T&S / TTS / ticket）与数值结论

模型（直接来自讲义结论）： – **T&S 锁**：每一次"测试"都是一次原子 RMW ⇒ 每次测试产生一次行所有权获取，并把其它所有人的副本作废。`P−1` 个等待者以自旋速率 `r = 1/t_line` 反复测试，于是**每秒产生的事务数**约为 `Traffic_TAS ≈ (P − 1) / t_line（单位：次/秒）每次成功获取锁期间的尝试次数 ≈ (P − 1) × (持锁时长 / t_line)` – **TTS 锁**：等待期间是**本地缓存里的普通读**（几乎零流量）；锁释放时的失效让每个等待者各读一次 ⇒ **每次释放 O(P) 次失效/读**；`P` 次获取构成一轮 ⇒ **每轮 O(P²) 次事务**。 – **ticket lock**：等待者都是普通读，**每次释放只有一次失效广播（O(P) 的到达，但只有 1 次广播事务）**，且下一个持锁者就在本地缓存里升级 ⇒ **每次获取 ≈ 1–2 次行交接**。

数值算例（P = 32，t_line = 80 ns）： 1. **T&S 每次获取的尝试次数**：设持锁者做临界区需要 `t_c = 500 ns`。持锁者刚拿到行时，等待者的副本被作废；它们很快各抢一次，行在"等待者们"之间乱窜。在最坏（经典）模型下，持锁期间约 `(P−1)/2 ≈ 15.5` 个等待者每人抢 `t_c / (P·t_line)` 次……为了给出可复核的数字，采用讲义给出的**保守替代模型**：每次获取期间的总尝试次数 ≈ `(P−1)`（每个等待者至少抢一次）。则 – 每次获取的额外事务数 = 31，每次事务 80 ns ⇒ **每次锁交接 ≈ 31 × 80 ns = 2.48 µs 的纯一致性流量**（在 500 ns 的临界区之上）。 – 稳态吞吐上限 = `1 / (t_c + 31·t_line) = 1 / (500 + 2480) ns ≈ 336 K 次/秒`。 2. **ticket lock**：每次获取 ≈ 2 次行交接 = 160 ns ⇒ 吞吐上限 = `1 / (500 + 160) ns ≈ 1.52 M 次/秒`。 – **比值 ≈ 4.5×**。而这个比值随 `P` 线性增长（T&S 的成本 ∝ `P`，ticket 的成本 ∝ 1），**这正是"可扩展性（scalability）"这一锁评价维度的量化含义**。 3. **纯同步开销（按本模型推算，临界区 ≈ 0）**：`P = 32` 时 – ticket：理想情况下 `2 × 80 = 160 ns/op-air` ⇒ 单线程视角 6.25 M ops/s（总吞吐仍是 6.25 M/s，因为全串行）； – T&S：`31 × 80 = 2.48 µs/op-air` ⇒ **0.40 M ops/s**； – 单线程无争用时两者都 ≈ `2 × 6 ns = 12 ns`（行常驻本地独占）。 – **结论：模型给出的退化幅度约 200 倍**；T&S 的退化**完全来自一致性流量，而不是临界区本身**。 – **模型的重要前提（务必注意）**："每次获取期间尝试 `P−1` 次"这个假设成立的前提是**临界区足够长、等待者真的在等待**（`t_c » t_line`）。如果临界区短到接近 0，锁在大部分时刻是空闲的，大多数获取**一次尝试就成功**（3.3 节的微基准在临界区为空时实测 `attempts/acq ≈ 1`，随临界区变长才上升到上百次）。**因此"尝试次数"不是常数，而是 `≈ min(P−1, t_c / t_line × 常数)`——它随持锁时长增长并渐近到 P−1。** 这也解释了讲义那张"时间 vs 处理器数"的图为什么在临界区被"移除"（置为极短）后仍然会随 P 上升：因为真正的成本来自等待者制造的流量，而不是临界区本身。 4. **TTS 的位置**：`P−1 = 31` 个等待者，每次释放产生 31 次失效 + 31 次读 ⇒ 一轮 32 次获取共约 `32 × 31 = 992` 次事务 ⇒ **每轮 O(P²) = 1024 量级**，即每次获取 ≈ 31 次事务（与 T&S 同阶）；但 TTS 在**持锁期间的等待是零流量的**，所以它比 T&S 好的地方在于**不会在持锁期间把持锁者的缓存行踢来踢去**（讲义："Update line in cache" 的失败尝试会不断作废持锁者的行）。实测上 TTS 通常优于 T&S，且代码代价仅为多一层内层读循环。

4.2 算例 B：CAS 循环的争用模型 —— 一个"无锁却不可扩展"的定量证明

模型：`P` 个线程争抢同一个原子字（无锁栈的 `head`），每个线程各执行 1 次操作（共 `P` 次）。硬件把该缓存行上的 CAS **完全串行化**：每 `t_line` 时间恰好有 1 个 CAS 原子地生效。

- 设当前有 `k` 个线程尚未完成。在这一轮"行所有权窗口"中，恰好有 1 个线程的 CAS 成功（其余 `k−1` 个失败）。因此**从 `k` 降到 `k−1` 所需的尝试次数服从几何分布**，期望为 `k` 次尝试（每次尝试耗 `t_line`）。
- 于是完成全部 `P` 次操作所需的总尝试次数 `E[attempts] = Σ_{k=1..P} k = P(P+1)/2 ≈ P²/2` **每次成功操作的平均尝试次数 ≈ P/2**。

数值算例（P = 32，t_line = 80 ns）： – 总尝试次数 ≈ `32 × 33 / 2 = 528` 次；总串行时间 ≈ `528 × 80 ns = 42.2 µs` 完成 32 次操作 ⇒ **每操作 1.32 µs，吞吐 ≈ 758 K ops/s**（若为 push+pop 各一次则为 379 K "完整操作"/s）。 – 对比**单线程、无争用**：行独占，CAS ≈ `t_cas_local = 6 ns` ⇒ **167 M ops/s**。 – **退化倍数 ≈ 220×**。而且这个退化是**结构性的**（`∝ P²`），完全符合讲义总结的那句"**a lock-free design does not eliminate contention**"。 – **延长验证（P = 64）**：总尝试 ≈ `64 × 65/2 = 2080` 次 ⇒ `166 µs` 完成 64 次操作 ⇒ 每操作 2.6 µs，吞吐 ≈ 385 K ops/s。**吞吐几乎减半（758 K → 385 K）**，说明"加线程"在高争用 CAS 上**是负收益**。 – **工程缓冲手段**： – **消除冲突点**：换用 **Treiber 栈 + 每线程局部队列**（由持有者批量转移）或 **消除式（elimination）栈**，把"每操作一次全局 CAS"变成"每批一次"； – **分摊（batching）**：每个线程先在本地累积 `K` 个操作，再一次性 CAS 提交，把总尝试次数从 `P²/2` 降到约 `(P/K)²/2 · K = P²/(2K)`； – **退避（backoff）**：失败后随机延迟（指数退避），降低"同时抢行"的概率，把几何分布的参数从"必然冲突"推向"串行化"； – **换结构**：SPSC 队列（2.7 节）在 1 生产 1 消费下**完全不需要原子 RMW**，是没有争用就没有代价的极端例子。

4.3 算例 C：细粒度锁的 Work / Span / 并行度与流水线吞吐上界

场景：链表节点数 `n = 1000`，`P = 32` 个线程，每个线程做 `M_per = 10^4` 次 `insert`，随机值域使得平均遍历长度 `L ≈ n/2 = 500` 个节点。

- **Work**：
- **粗粒度**：节点访问 `W_visit = 32 × 10^4 × 500 = 1.6 × 10^8` 次；锁操作 `W_lock = 3.2 × 10^5` 次。
- **细粒度**：节点访问相同；**锁操作 `W_lock = 32 × 10^4 × 500 × 2 = 3.2 × 10^8` 次原子 RMW**（每步一次 lock + 一次 unlock）。
- **成本的量级对比**：粗粒度的一次锁操作在无争用时 ≈ 12 ns（两次本地独占 RMW），400 次获取……；细粒度把 `3.2×10^8` 次原子操作插入执行流，即便全部命中本地独占（6 ns/次，含释放在内取 12 ns 一对），也要 `1.6 × 10^8 × 12 ns ≈ 1.9 s` 的**纯同步时间**（未计缓存行争用，实际更高）。**在长链表上，细粒度锁是明确的自杀式优化**。
- **Span**：单次操作的关键路径 = `L` 步指针追逐 + `L` 对锁交接。纯指针追逐部分是**不可并行化的依赖链**：`Span_visit(1 op) ≈ L × t_dram = 500 × 90 ns = 45 µs`（每步都缺 L1/L2，L3 也大概率缺） `Span_lock(1 op) ≈ L × t_hop = 500 × 40 ns = 20 µs`（无争用时行在本地或近邻，

取 40 ns）

- **并行度 = Work / Span**：`1.6×10^8 / (500 × (90+40) ns) ≈ 1.6×10^8 / 65 μs ≈ 2460`。这个数字说明：**理论上最多有约 2460 个"操作步"可以同时在飞，但单次操作的延迟是 65 μs，且被指针追逐主导，无法通过更多线程缩短。**
- **吞吐上界（推荐用这个而不是 Work/Span 来评估）**：`细粒度：Throughput ≤ min( P / t_lockhop , 1 / t_hop ) —受"锁交接流水线"限制 粗粒度：Throughput ≤ 1 / (t_critical + t_handoff)，t_critical ≈ 45 μs`
- 粗粒度：`1 / (45 μs + 160 ns) ≈ 22.2 K ops/s`（32 个线程合计，与 P 无关！）——**这就是"单一全局锁串行化"的代价：加线程完全无效。**
- 细粒度：若锁交接能被流水化到 `t_hop = 40 ns`，则**理论吞吐上限 = 1/40 ns = 25 M ops/s**，比粗粒度好 3 个量级。**但**前提是"每步 2 次原子操作不产生缓存行争用"，而当 `n = 1000` 个节点（每个几十字节）分布在约 1000 × 64 B ≈ 64 KB 内时，**32 个线程同时在不同节点 `lock/unlock` 会产生大量跨核缓存行迁移**，实际 `t_hop` 会从 40 ns 恶化到 200–400 ns，吞吐降到 2.5–5 M ops/s——**依然远好于粗粒度，但比理想值差 5–10 倍**。这个"理想 25 M vs 现实 3 M"的差距，就是**伪共享与行迁移的账单**。
- **Amdahl 视角**：设程序总时间中"必须串行的部分"占 `s`，则可扩展性 `Speedup ≤ 1/(s + (1−s)/P)`。粗粒度锁把这个串行比例推到 `s → 1`（性能由临界区串行时间决定），因此**无论 P 多大，加速比都趋近 1**；细粒度锁把 `s` 压到"每步锁交接时间 / 每步总时间"的水平，从而恢复可扩展性。**结论：细粒度同步的收益不是"更快地做同一件事"，而是"把不可扩展的程序变成可扩展的程序"。**

4.4 算例 D：带宽、延迟与 Little's Law —— 无锁数据结构真正的天花板

算术强度与 Roofline 脊点（本机假设）：

```
峰值浮点吞吐 = P × f × W × k = 32 × 3.0 GHz × 8 × 2 = 1536 GFLOPS
内存带宽 BW = 20 GB/s
Roofline 脊点 = 1536 / 20 = 76.8 FLOP/byte
==> 算术强度 < 76.8 时受带宽限制；> 76.8 时受计算限制
```

数值算例 1（讲义示例的量级）：一个 256 MB 的数组，单次遍历读 256 MB：

```
时间下界 = 256 MB / 20 GB/s = 0.256 GB / 20 GB/s = 12.8 ms
```

这就是"任何需要额外扫一遍 256 MB 内存的簿记操作，至少要 12.8 ms 的地板价"。把它用到数据结构上： – 若把链表节点的**锁字段与数据分离**（为了消除伪共享而做），你可能把原来一次顺序扫描变成两次（数据数组 + 锁数组），于是**每次遍历的带宽需求翻倍**，纯带宽时间从 12.8 ms 变成 25.6 ms； – 若为了消除伪共享给每把锁加 `alignas(64)` 填充，而锁只有 1 字节有用信息，则**内存放大 64 倍**（假设原节点 32 B），一个 256 MB 的结构的锁数组会膨胀到 512 MB，带宽与 TLB 双重恶化。

数值算例 2（Little's Law：要多少并发才能填满带宽）：

```
Little's Law:  并发量 = 吞吐 × 延迟
要维持 BW = 20 GB/s，且 DRAM 延迟 = 90 ns：
    在途字节数 = 20 GB/s × 90 ns = 1800 B
    在途缓存行数 = 1800 B / 64 B ≈ 28.1 条
==> 一个核最多支持 ~10–12 条未完成缺失（MSHR 有限），因此至少需要 3 个核
    同时满负荷发缺失，才能把 DRAM 带宽打满。
```

把这个结论用到无锁 vs 有锁上： – **无锁 CAS 循环是"一次一条缓存行在途"的极端**：每个线程在任一时刻只关心一个地址（`head`），**并发度 = 1**（因为行被串行化），因此在途字节数 ≈ 64 B，能达到的"有效带宽"只有 `64 B / 80 ns = 0.8 GB/s` ——**仅为机器带宽的 4%**。这说明**无锁 CAS 点争用是彻底的延迟受限（latency-bound）场景，与带宽毫无关系**。 – **细粒度锁链表是"指针追逐 + 行迁移"混合体**：每个 hop = 一次 90 ns 的节点读 + 可能的行迁移，在途字节数同样只有几十字节，`t_hop` 由**延迟**决定而不由带宽决定。 – **什么时候带宽才会成为瓶颈**：当数据结构操作变成**批量顺序访问**时（例如哈希桶数组的 rehash、队列底层环形缓冲的批量入队、数组式的图遍历），此时 `BW` 才是限制，`12.8 ms / 256 MB` 是必须付的地板价。 – **结论：同步结构的性能模型通常由延迟与串行化决定，而不是带宽**；反之，遍历型/批处理型代码才需要 Roofline 与算术强度。**混淆这两类模型是最常见的性能误判来源**。

4.5 模型选择速查表

现象	适用的模型	关键公式	本讲的对应例子
加线程后总吞吐不涨、甚至下降（且临界区很短）	**串行化 / 一致性流量模型**	`Throughput ≤ 1/(t_c + θ(P)·t_line)`；`Span = θ(M·t_handoff)`，并行度 = Θ(1)	T&S/TTS 锁微基准（4.1）
无锁 CAS 循环在重争用下变慢	**几何重试模型**	`E[attempts] ≈ P/2`；总尝试 `≈ P²/2`	无锁栈（4.2）

现象	适用的模型	关键公式	本讲的对应例子
单次操作延迟居高不下，加线程无用	**指针追逐 / 延迟受限模型**	`Latency = L × t_dram`；`并发度 = BW × 延迟 / 64 B`	细粒度锁链表（4.3、4.4）
批量顺序访问、性能随数据量线性下降	**带宽 / Roofline**	`T ≥ Bytes / BW`；脊点 = `峰值算力 / BW`	分离式锁数组、rehash（4.4）
有些线程很快、有些极慢（方差大）	**公平性 / 饥饿模型**	无公平性 ⇒ 完成时间方差随 `P` 增长	TTS/TAS 无公平性（3.3）
峰值内存或尾延迟尖峰	**摊销 / 回收模型**	`retireListSize > THRESHOLD` 触发批量回收	无界队列 reclaim、hazard pointer 阈值

5. 关键要点

1. **同步粒度是一维谱（spectrum），不是一个二选一**：`单把大锁 → 每桶/每节点锁（hand-over-hand）→ 无锁 CAS → 无等待`。向右移动的每一步都用"更高的单步实现成本 + 更大的正确性论证负担"换取"更低的争用与更好的可扩展性"；**判断往哪边移动的唯一依据是 work-span / Amdahl / 一致性流量的定量分析**，而不是"越细越好"或"无锁一定更快"。
2. **"用锁就是阻塞算法"，而"无锁不等于没有争用"**：讲义的两条硬结论——①任何使用锁的算法都是阻塞的，**无论自旋还是抢占**（一个被换出/缺页/崩溃的持锁者可以无限期阻止所有其它线程）；②lock-free 只保证**系统级进展（某个线程必定完成）**，它**不排除单个线程饥饿**，也**不消除争用**——CAS 在重争用下的失败重试使吞吐按 `∝ 1/P` 恶化（4.2 节的 `P²/2` 模型）。
3. **原子操作的成本由"缓存行所有权的迁移"决定，而不是由指令本身决定**：x86 的 `lock` 前缀意味着"缓存保留该行直到完成 / 占住总线 / 对其它请求回 NACK"；因此**同一缓存行上的原子操作被硬件完全串行化**，性能指标是"每秒能交接多少次该行"。由此推出的三条工程规则：**①让热的原子变量独占缓存行、让锁与数据分离**（避免伪共享）；**②减少原子操作的次数（hand-over-hand 的 hop 成本、CAS 失败重试都算在这里）**；**③能用一次原子操作办成的事，不要拆成两次**（ticket lock 的等待只需普通读，就是这条规则的典范）。
4. **无锁编程的三个"必须一起解决"的难题**：**ABA**（CAS 只比较值，比较不出"世界变过又变回来"——用版本号打包进同一个字，或用 `cmpxchg8b/16b` 打包相邻字段，或用节点分配/复用策略）；**内存回收**（use-after-free 与 ABA 是两个独立问题——hazard pointer / epoch / 内存池）；**内存序**（讲义在所有无锁代码处都标注 "assume a sequentially consistent memory system for now"，在真实弱一致性硬件上必须补栅栏或使用 `atomic<>`，且要选对 `acquire/release` 而不是无脑 `seq_cst`）。
5. **在"独占机器"的性能优化场景里，锁通常是最好的选择；无锁的价值主要在"无法独占机器"的场景**：讲义明确说，写得好的加锁代码可以与无锁代码一样快甚至更快，而且**通常简单得多**；无锁（非阻塞）真正不可或缺的场合是**多道程序环境**（数据库、Web 服务器）——线程可能在临界区内被抢占、缺页、优先级反转、护航（convoying），此时"任何人卡住都不会卡住系统"这一性质本身就是价值。

6. 常见陷阱与注意事项

- **陷阱 1：把"细粒度"理解成"把锁变小"，却忽略了每步加锁的开销。** hand-over-hand 让每次遍历步从"一次普通读"变成"读 + 两次原子 RMW"，在长链表上（4.3 节的 `n = 1000`、`L = 500`）这会让原子操作数量爆炸到 `10^8` 量级，最终比单把大锁还慢。**正确做法**：只在真正要修改的位置加锁（讲义"insert 还有改进空间"的答案——只读遍历 + 锁 `prev` + 重新验证 `prev->next == cur`），或用锁条带化（striping）折中。
- **陷阱 2：忽略伪共享（false sharing），让细粒度锁退化成"更慢的粗粒度锁"。** 每节点一把锁会让相邻节点的锁落在**同一 64 B 缓存行**上，两个线程锁不同节点却争同一行 ⇒ 一致性流量与粗粒度锁同量级，而代码复杂度和原子操作次数都更高。**识别方法**：用 PMU 计数器（HITM / 缓存行迁移计数）或把锁字段改为 `alignas(64)` 对比测量。**修正手段**：锁与数据分离成两个数组、把锁字段填充到独占缓存行、或用"按桶分摊锁"。
- **陷阱 3：误以为 `std::atomic<T>` 就等于无锁，或误以为无锁就等于更快。** `std::atomic` 的原子性"可能由 mutex 实现"，必须用 `is_lock_free()` 确认；反过来，无锁实现在**重争用**下会因为 CAS 失败重试而**比加锁慢**（4.2 节：P=32 时每操作 1.32 μs，而单线程无争用只要 6 ns）。**正确心态**：先测量争用程度，再决定是否值得上无锁。
- **陷阱 4：忽略 ABA 问题与"引用已释放内存"是两个独立的问题。** 加了 `pop_count`/版本号解决了 ABA，**并不会**解决 `old.top->next` 可能读取已释放节点的问题（use-after-free）；反之用内存池（永不 `delete`）解决了 use-after-free，**也不会**解决 ABA（地址会被复用）。**必须分别处理**：ABA 用"版本号打包 / 宽 CAS / 不复用地址"，回收用"hazard pointer / epoch / 内存池"，并注意两者对内存占用的联合影响。
- **陷阱 5：在弱一致性硬件上漏掉内存栅栏，或把内存序设得过强。** 讲义在 SPSC 队列与无锁栈的代码处都明确标注"**Assume a sequentially consistent memory system for now (or the presence of appropriate memory fences, or C++11 atomic<>)**"。**漏栅栏**会导致"生产者写完

`data[tail]` 但消费者看到新 `tail` 却读到旧数据"这类极难复现的错误；**无脑用 `seq_cst`** 则在 x86 上为每次原子操作多付 `lock` 前缀/屏障代价。正确做法是：`release` 用在"发布新状态的那一次写"（push 成功），`acquire` 用在"读取状态的第一次读"（pop 开头），失败路径用 `relaxed`。

- **陷阱 6：把"无锁"当作"无争用"写进性能预算，忽略负载不均与热点。** 无锁栈的所有操作都争抢**同一个 `head`**（单点热点），无锁链表的所有操作都要**从 head 出发遍历**（head 附近的锁最热）。这与"任务粒度选择"是同构问题。**缓解办法**：batching（本地累积后一次提交）、backoff（随机退避）、消除（elimination）、分片（sharding）、以及换成 SPSC 结构（没有任何共享 RMW）。

---

7. 思考题（带答案）

问题 1（细粒度锁的正确性与死锁）：下面是讲义 hand-over-hand 链表 `insert()` 的核心循环：

```
FNode* prev = list->head;  lock(prev->lock);  FNode* cur = prev->next;
if (cur) lock(cur->lock);
while (cur && cur->value < value) {
    Node* old_prev = prev;  prev = cur;  cur = cur->next;
    unlock(old_prev->lock);  if (cur) lock(cur->lock);
}
```

请回答三个问题：(a) 为什么这段代码**一定无死锁**？(b) 为什么"先 `unlock(old_prev)` 再 `lock(cur)`"这个看似危险的顺序其实**不会造成悬空指针**？(c) 讲义提问"`insert()` 还可以怎样进一步改进"，请给出改进方案与它对本讲后半部分的意义。

【答案】 (a) **无死锁的原因是全局锁序（global lock ordering）**：所有线程都从 `list->head` 出发、沿 `next` 指针方向（head → tail）依次加锁，并且在持有一把锁的同时只尝试获取**沿同一方向的下一把锁**。于是"资源等待图"中所有边都指向同一方向，**不可能出现环**（这正是死锁第 4 个必要条件"circular wait"被破坏）。因此不需要超时、不需要重试、也不需要任何死锁检测。注意这条论证的**脆弱性**：只要有一个操作（例如"从表尾向前查找"或"删除时同时锁前驱与后继之外的第三个节点"）逆序加锁，无死锁的保证立即失效。

(b) 关键在于"**我始终持有前驱的锁**"这一不变量。循环里 `old_prev` 是**上上轮的 `prev`**，而当前的 `prev`（= 上一轮的 `cur`）**从上一轮起就已经被锁住且尚未释放**。所以 `unlock(old_prev); lock(cur);` 的那一刻：– 线程手上仍然持有 `prev->lock`；– 而 `cur` 是从 `prev->next` 读出来的，`prev->next` **由 `prev->lock` 保护**，所以任何想摘掉 `cur` 的线程都必须先获得 `prev->lock`（而它拿不到）⇒ **`cur` 的身份在这段窗口内是稳定的**，`lock(cur->lock)` 作用于一个仍然在链表中的节点，不会悬空。– 附带结论：该线程在**任何时刻都至少持有一把锁**（`prev->lock`），这也保证了不会出现"手上没有任何锁、局部指针随时失效"的窗口。– 反例：如果写成 `unlock(prev); prev = cur; cur = cur->next; lock(cur);`（先把 `prev` 的锁也放掉），就丢掉了这个不变量，别的线程可以在空窗期把 `cur` 摘链并 `delete`，随后对已释放内存加锁 = use-after-free。

(c) **改进方案：把"只读遍历"与"加锁修改"分离（乐观遍历 + 加锁验证）**。因为 `insert` 最终只修改**一个指针** `prev->next`，遍历过程本身**不需要互斥**：先用**纯读**（无锁、无原子操作、无一致性流量）找到插入位置 `prev` / `cur`；然后**只锁 `prev`**；拿到锁后**重新验证** `prev->next == cur`（以及 `cur` 仍在链上、`prev->value < value <= cur->value` 或不变量成立）：验证通过就执行 `n->next = cur; prev->next = n;` 并解锁，**验证失败就放弃本轮结果、回到遍历重新搜索**。– **收益**：每次操作从 `θ(L)` 次原子 RMW 降到 **`O(1)` 次原子 RMW 加一次失败重试**，这正好回应了讲义"细粒度锁的开销（每步加锁、额外存储）是否可以折中"的提问。– **对本讲后半部分的意义**：这个"**读快照 → 计算 → 在提交点校验 → 失败重试**"的结构，与无锁 CAS 循环在形式上**完全同构**（只是一次校验的对象从"值"变成"指针 + 结构不变量"）。因此**无锁编程不是与锁对立的另一套技术，而是"把校验点从临界区边界压缩到一个原子操作"的延续**——这也正是讲义在结尾预告 Transactional Memory 时所说的："CAS 的作用是判断在我操作期间有没有别的线程改过数据结构"，而事务内存把这种"乐观 + 可中止"的机制一般化。

问题 2（ABA 与内存回收）：某同学写了下面这个无锁 `pop`（单链表栈，节点用 `new/delete`）：

```
Node* pop(Stack* s) {
    while (1) {
        Node* old_top = s->top;
        if (old_top == NULL) return NULL;
        Node* new_top = old_top->next;
        if (CAS(&s->top, old_top, new_top) == old_top) {
            int v = old_top->value;  delete old_top;  return old_top;   // (X)
        }
    }
}
```

他指出"我已经在 CAS 成功后才 `delete`，所以不存在 use-after-free，也不需要 hazard pointer"。请指出他至少**两处**错误，并说明**为什么专门加一个 `pop_count` 计数器仍不能解决全部问题**，最后给出一个完整可行的方案（可以是组合方案）。

【答案】

错误 1：CAS 成功并不意味着可以立刻 `delete`（仍然存在 use-after-free 的读取窗口）。 虽然**执行 `delete` 的线程**是唯一弹出了这个节点的线程，但**其它线程可能仍然持有指向它的局部指针**。具体路径：线程 T1 执行了 `old_top = s->top`（得到 A）并在计算 `new_top = A->next` 之前被抢占；与此同时 T0 弹出 A、`delete A`；T1 恢复后执行 `A->next` ⇒ **读取已释放内存**。加锁版本之所以没有这个问题，恰恰因为"持有锁"隐含地告诉其它人"别碰这个节点"；无锁版本没有这种保护，因此需要 **hazard pointer / epoch / 内存池** 之类的回收协议来延迟释放。注意 `delete` 的位置（CAS 成功之后）根本不能解决这个竞态——**竞态发生在别的线程的读上，而不是发生在执行 delete 的线程上**。

错误 2：ABA 问题依然存在，且 `delete` 让 ABA 更危险。 CAS 只比较指针值：T0 读到 `old_top = A` 后被抢占；T1 弹出 A、**释放** A、又 `new` 出一个新节点（**很可能就是同一地址 A**，因为 glibc 的分配器会复用最近释放的块）、再 `push(A)`（此时 A 里存的是**完全不同的数据**）；T0 恢复后 `CAS(&s->top, A, B)` **成功**（此刻 `top` 确实等于 A），于是把 `top` 设为 B —— 丢失了 T1 新压入的节点，栈结构被破坏，而 T0 返回的 `A` 里装的是别人的数据。**更糟的是**：如果被 `delete` 的地址被分配器交给了**另一个数据结构**（例如 O(1) 大小的对象被 `malloc` 复用），这个 CAS 会破坏完全无关的内存。

为什么单加 `pop_count` 不够：(1) 它只能消除 ABA（版本号让"C 等于 A"不再成立），**对 use-after-free 毫无帮助**——`A->next` 仍然可能读已释放内存，或者读到一个被复用的、完全不相关的对象；(2) 讲义指出，使用 `pop_count` 需要**双字 CAS（DCAS）**，或者机器支持"把两个字段打包进一个字"的宽 CAS（`cmpxchg8b` / `cmpxchg16b`，或者 `std::atomic<__int128>`）；如果没有这样的硬件支持，就要退回到"用单个字打包 (tag, index)"的技巧，而这只在**节点来自预分配池、地址可以压缩成小整数索引**时才方便。(3) 即便版本号解决了 ABA，**"何时可以释放"仍然是一个独立的、必须用额外协议解决的问题**。

完整可行的方案（任选其一，或组合）： – **方案 A（讲义解法一 + 内存池）**：节点从**预分配池**（`pool[idx]`）中取，永不 `free`；栈顶用**一个 64 位字打包 `(tag:32, idx:32)`**；每次成功的 CAS 都把 `tag` 加一。一箭双雕：`tag` 使 ABA 不可能（相同 `idx` 必然伴随不同 `tag`），地址不复用使 use-after-free 不可能（内存从不归还）。**代价**：内存占用固定不可回收；`tag` 只有 32 位，理论上跑满 `2^32` 次操作后会回绕（工程上可用 48/16 位划分、定期检查，或承认其概率极低）。 – **方案 B（版本号 + hazard pointer，讲义解法二 + 解法三）**：`pop` 时**先发布 `hazard = old_top`，再重新校验 `s->top == old_top`**，然后才读 `old_top->next` 并 CAS；CAS 成功后不直接 `delete`，而是 `retire(old_top)` 放进**每线程的退休链表**；当 `retireListSize > THRESHOLD` 时遍历所有线程的 hazard 槽位，**只 `delete` 那些不等于任何 hazard 的节点**（并且要在扫描后重新确认——因为 hazard 可能在扫描过程中被更新，标准实现会做第二遍扫描或用 `-Wpedantic` 级别的严格版本）。`pop` 失败路径上必须把 `hazard = NULL`。**为什么安全**：节点一旦被 CAS 摘链就不可达，任何线程要再引用它都必须先读 `top` 再沿链走，因此"扫描时刻没有 hazard 指向它"就意味着将来也不会有人引用它。 – **方案 C（工程折中）**：如果不想处理回收，就把数据结构设计成"**节点不回收**"的形式（内存池 + 索引 + 版本号），或者干脆**放弃无锁**——讲义明确指出，在能独占机器的性能优化场景里，**写得好的加锁代码可以与无锁代码一样快甚至更快，而且简单得多**；只有在无法独占机器（数据库、Web 服务器）时才值得为 lock-free 的正确性论证付出这笔复杂性成本。

问题 3（定量判断：该不该改成细粒度/无锁？）：某服务维护一条**有序链表**，节点数 `n = 1000`，稳态下**每个线程每次操作平均遍历 `L = 500` 个节点**，共 `P = 32` 个线程，总操作数 `M = 3.2×10^5` 次插入。当前实现使用**一把全局锁**，测得吞吐约 `22 K ops/s`。有人建议改成：(i) 每节点一把锁（hand-over-hand）；(ii) 无锁 CAS 插入。已知参数：单线程无争用时一次锁操作（lock+unlock）≈ 12 ns，一次缓存行交接 `t_line ≈ 80 ns`，节点访问（指针追逐）≈ 90 ns/次且为**依赖链**。请分别给出两种方案的**定量上界**，判断哪个方案更可能有效，并指出**必须先测量的两个量**。

【答案】

先算当前实现的基线（验证"加线程无效"来自哪里）：

```
单次操作的关键路径 = L 次指针追逐 = 500 × 90 ns = 45 μs
（加上一次锁交接 ~160 ns，可忽略）
吞吐上界 = 1 / 45 μs ≈ 22.2 K ops/s   —— 与 P 无关!
```

算出来的 22.2 K ops/s 与实测的 22 K ops/s 完全吻合，这直接证明：**当前的瓶颈不是"锁"，而是单次操作 `O(L)` 的依赖链延迟（指针追逐）**；`P = 32` 个线程全部在同一个临界区里排队，加线程对吞吐毫无帮助（这正是 Amdahl 的串行部分 `s → 1` 的情形）。

(i) 每节点一把锁（hand-over-hand）：

```
Work（同步部分） = M × L × 2 次原子 RMW = 3.2e5 × 500 × 2 = 3.2e8 次
若每次原子 RMW 命中本地独占（无争用理想值）≈ 6 ns：
    纯同步时间 ≈ 3.2e8 × 6 ns = 1.92 s
即使把 lock+unlock 一对压到 12 ns 且完全并行（32 线程）：
    1.92 s / 32 ≈ 60 ms   —— 仍然远超"45 μs × M / (有效并发)"的乐观估计
吞吐上界（流水线模型）= min( P / (L·t_hop), 1 / t_hop )
    取 t_hop = 40 ns（乐观，无争用时）: 1/40 ns = 25 M ops/s（理想上界）
    取 t_hop = 240 ns（考虑 1000 个节点锁在 32 核间反复迁移，现实值）: ≈ 4.2 M ops/s
```

结论：理想上界（25 M ops/s）看着很美，比基线好 3 个量级；但它**建立在"每步 2 次原子操作几乎零成本"的假设上**，而该假设在本场景（`n = 1000` 个节点、锁字段与数据同行、32 核同时迁移缓存行）**几乎必然被打破**。因此方案 (i) 的**现实收益是数量级级别的改善（4 M vs 22 K，约**

190×)，但远达不到理想值，而且**风险在于伪共享**：如果不做"锁与数据分离 / `alignas(64)`"处理，`t_hop` 可能恶化到 400 ns 以上，收益缩水到 30× 左右，并且**代码复杂度与死锁论证成本显著上升**。

(ii) 无锁 CAS 插入：

```
Work：遍历仍是 θ(M·L) 次读（纯读，无原子操作），加上每操作一次成功 CAS + 期望失败次数
     插入点集中在 prev->next 这"一个"提交点上，故平均尝试次数 ≈ P/2 = 16（4.2 节模型）
     ⇒ M × 16 × 80 ns = 3.2e5 × 16 × 80 ns ≈ 0.41 s 的 CAS 争用时间
Span：单次操作 = L × 90 ns（指针追逐，不变）= 45 µs
吞吐上界 ≈ 1 / （Span 中的 CAS 提交部分）；由于遍历本身并行，稳态吞吐 ≈ 1/(2×80 ns) ≈ 6.25 M ops/s
```

结论：方案 (ii) 比 (i) **更可能有效**，原因有三：①**遍历路径上没有原子操作**（`θ(M·L)` 次读全部是纯读，不产生一致性流量，也不破坏其它核的缓存行）；②**提交点只有 `prev->next` 一处**，即使有 16 次平均重试，其成本也只与 `P` 有关而与 `L` 无关；③**不需要为每个节点增加锁字段**，节点更小、缓存密度更高（讲义对无锁插入的总结："No overhead of taking locks; No per-node storage overhead"）。**但要注意**：(ii) 如果只做插入而不做删除，实现简单；**一旦要支持无锁删除，问题立刻变得极其困难**（讲义："Supporting lock-free deletion significantly complicates data-structure"，见 2.9 的图解 8 与 Harris 2001 / Fomitchev 2004），因此这个方案适用于"只插入"或"删除极少"的场景。

必须先测量的两个量：1. **真实的 `L`（平均遍历长度）与节点访问延迟**：用 PMU 计数器测每次操作的 LLC/DRAM 缺失数与平均内存延迟。如果 `L × t_dram` 确实主导总时间（本例中 45 µs vs 锁的 160 ns），那么**两个方案的收益上限都被"指针追逐"锁死**，真正的优化方向应该是**换数据结构**（哈希表 / B 树 / 跳表：把 `L` 从 500 降到 3–10），而不是改变同步方式——这是本题最重要的一条结论：**当瓶颈是依赖链延迟时，改同步是治标，改数据结构才是治本**。 2. **一致性流量 / 缓存行迁移次数（HITM 或 "cache line transfers" 计数）**：它直接决定方案 (i) 中真实的 `t_hop`（是否存在伪共享）与方案 (ii) 中 CAS 的失败率（是否争用过度、是否需要 batching/backoff）。没有这个数字，就无法判断"25 M ops/s 的理想上界"和"4 M ops/s 的现实"之间那 5–10 倍的差距到底来自哪里。

补充判断（讲义的实践结论）：如果这个服务**能独占机器**（例如它是某个批处理任务），那么按讲义的观点，**"写得好的加锁代码可以与无锁代码一样快（或更快），而且简单得多"**——此时最该做的是**换数据结构 + 保持粗粒度锁**（先测量再优化）；只有在这个服务是**多道程序环境**（数据库/Web 服务器，线程会在临界区内被抢占、缺页、产生优先级反转与护航）时，为 lock-free 付出的复杂度才是划算的。

Lecture 17: Transactional Memory

1. 章节标题与概述

Lecture 17: Transactional Memory（事务内存：从"锁的两难"到声明式原子块）

- **本讲核心问题**：前面几讲我们一直在"往下走"——用 test-and-set、fetch-and-add、CAS、LL/SC 这些机器级原子操作，在软件里手工搭建锁、屏障、无锁数据结构。结果也很清楚：**程序员在"并发度"与"正确性"之间被夹在中间**（讲义原话 *between a lock and a hard place*，slide 4）。粗粒度锁好写但并发度低；细粒度锁并发度高，但手递手（hand-over-hand）锁序稍有不慎就是死锁，而且**读-读共享也要付出独占锁的代价**。本讲往回"往上走"一层：能不能不写 `lock()/unlock()`，只声明"这段代码要原子执行"，让**系统**去决定怎么实现原子性与隔离性？一个 memory transaction（内存事务）到底是什么语义？实现它必须回答哪几个设计问题？软件实现（STM）与硬件实现（HTM）各自的代价与天花板在哪里？
- **涉及的主要硬件/软件机制**：
- **语义层**：memory transaction = 原子且隔离的内存访问序列，来自数据库事务的三个性质——**atomicity（原子性，全有或全无）**、**isolation（隔离性，提交前对外不可见）**、**serializability（可串行化，看起来按某个串行顺序提交，但顺序本身不保证）**（slide 8）。
- **实现层（设计空间的三问）**：**数据版本策略（data versioning policy）**——eager versioning（undo-log based，立即改内存、留撤销日志）vs lazy versioning（write-buffer based，先缓冲、提交时冲刷）；**冲突检测策略（conflict detection policy）**——pessimistic（在每次 load/store 时检查）vs optimistic（在 commit 时检查）；**检测粒度（granularity of detection）**——字 / 缓存行 / 对象，粒度直接决定"伪冲突"数量（slide 3、slide 40）。
- **软件侧（STM）**：全局版本时钟（global version clock）、读集/写集（read set / write set）、版本字里的锁位表达写集所有权、提交时读集验证、争用管理器（contention manager）与串行化回退（保证前进、消除活锁）。
- **硬件侧（HTM）**：把"一致性协议对单个地址做的事"推广到"一组地址"（slide 9 的洞见）——用私有缓存里的 **read-set / write-set 标记** + 一致性协议的冲突检测（他人请求命中我的读写集 → 冲突）来实现隔离，事务提交 = 把写集一次性地对外发布。
- **在并行计算知识体系中的角色**：本讲是"共享内存并行"这条主线上**抽象层级提升的最后一级**：原子指令 → 锁/屏障 → 无锁数据结构 → **事务**。它把前几讲的所有硬件知识（缓存一致性、伪共享、内存序、原子指令的代价）反着用了一遍：**事务并不是魔法，它只是把"锁的争用"换成了"元数据的争用"**——读集验证要走一致性，提交要一次原子 RMW 抢全局版本号，检测粒度会带来与伪共享同源的伪冲突。因此本讲既是抽

象的顶点，也是"抽象有代价"的最好教材。它同时为后续专题（异构并行、虚拟内存）留下一个现实注脚：工业界最著名的 HTM 实现（Intel TSX/RTM）因为正确性与安全原因曾被迫微码关闭，而**事务只覆盖内存**这一限制（I/O、`malloc`、系统调用不可回滚）至今没有被彻底解决。

- **配套材料**：
- **Fall 2026 日程表**（https://www.cs.cmu.edu/~418/schedule.html）把 **Oct 5** 排为第 17 讲 **"Transactional Memory"**，该行**目前没有 slides 链接**（对照第 15/16 讲，那两行在 HTML 注释里写着"slides/video from a previous offering; uncomment when posted for Fall 2026"）——也就是说 **Fall 2026 的本讲讲义尚未发布**，属未公开。
- 本笔记的事实基础是 **`cs149_supp/transactions.txt`**：Stanford CS149（Fall 2025）**Lecture 17: Transactional Memory** 讲义的逐页抽取文本，共 **50 页**（文件内标记 `===== [slide i/50] =====`）。该讲义在公开网络可直接获取，**已公开**，本笔记中所有带 slide 编号的引用均出自它。讲义首页写的是 "Stanford CS149, Fall 2025"——**讲义沿用历史版本是正常现象**，不是错误。
- CMU 历史学期（s23/f22）的本讲讲义在本地只有登录占位页：`past/s23_18_transactional.pdf`、`past/s23_19_tm.pdf`、`supp/f22_transactionalmem.pdf` 均只有 **1.7 KB**，抽取文本 `extracted/s23_18_transactional.txt`、`extracted/s23_19_tm.txt` 的内容只有一句 "Your Browser does not support javascript"，即**需 CMU 登录、未公开**。历史讲义原始位置在 `/afs/cs/academic/class/15418-*/public/` 之下。
- 讲课录像（Panopto / YouTube）：Fall 2026 日程表中被 HTML 注释隐藏，属**未发布**。Ed 讨论区、Autolab、Canvas：**需登录**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。
- 讲义中树结构示例图注明 "Slide credit: Austen McDonald"；讲义在性能对比图中提到的 **TCC** 是一个**硬件实现的 TM 系统**（slide 27 原话："TCC" is a TM system implemented in hardware）。
- 本笔记第 2.6 节的硬件 TM 细节、第 3 节的三段可运行代码与第 4 节的定量模型属于**公开知识补充与本文档作者在本机上的实测**（不是 Fall 2026 讲义内容），文中均已显式标注。实测环境：2 路 AMD EPYC 7V13（共 128 线程、2 NUMA 节点、16 个 L3 实例、L3 共 512 MiB），g++ 12.2.0，机器为共享节点（实测期间 load average 约 8–12），**绝对数值随机器负载波动，请只把它当作数量级与相对趋势的证据**。

---

2. 核心概念与硬件/软件架构图解

2.1 memory transaction 的三条语义：把"单个地址的一致性"推广到"一组地址"

- **定义与目的**：一个 **memory transaction（内存事务）** 是"一段原子且隔离的内存访问序列"（slide 8）。它必须同时提供：
- **Atomicity（原子性，全有或全无）**：提交（commit）时，事务里的所有写**一次性**生效；中止（abort）时，这些写**看起来从未发生过**。
- **Isolation（隔离性）**：事务提交之前，任何其他处理器都**不能观察到**它的写。
- **Serializability（可串行化）**：所有事务看起来按**某个串行顺序**提交——但**具体顺序不被语义保证**（这正是"乐观并发"能成立的前提：既然顺序无所谓，系统就可以在无冲突时并行推进）。
- **直观解释（"它是什么？"）**：想象银行转账。你要从 A 账户扣 100、给 B 账户加 100。用锁，你得自己规定"A 的锁和 B 的锁谁先谁后"，并且祈祷所有调用者都遵守同一套顺序（讲义 slide 31–32 的 `transfer(A,B)` 与 `transfer(B,A)` 就是活生生的死锁例子）。用事务，你只写 `atomic { withdraw(A); deposit(B); }`，**你说的是"要什么"（这一段必须原子），不是"怎么做"**（slide 6–7 的 declarative vs imperative）。类比：锁像"自己动手拧螺丝并负责按正确顺序拧"，事务像"下单——我只要这批零件装成一个整体，工艺由车间决定"。
- **和锁的语义差异（这是本讲最容易被误解的一点，slide 35 专门做了一次 self-check）**：

维度	`lock()/unlock()`	`atomic { }`
抽象性质	**命令式（imperative）**低层阻塞原语	**声明式（declarative）**高层原子性声明
自身语义	**不提供**原子性与隔离性，只是互斥	系统**保证**原子性与隔离性
适用范围	互斥只是用途之一：还可以用来等待条件（条件变量、标志位自旋）、限制并发度、实现一次性初始化	**只**声明原子性；**不能**用它表达"等待另一个线程做事"
组合性	组合需要全局策略（锁序），模块化被破坏	事务可组合，最外层事务定义原子性边界（slide 33）
失败处理	线程失败可能"带着锁死掉"，需要人工 undo 代码	事务中止即回滚，不留锁（failure atomicity）

- **ASCII 图解 D1：事务的读写集合（讲义 slide 16–27 的树示例）**

```
场景：两个线程在二叉树上更新节点 3 与节点 4（讲义 slide 16，图 credit: Austen McDonald）

① 手递手锁(hand-over-hand locking)：边走边锁，独占整条路径
   Thread A: LOCK(1) ──► LOCK(2) ──► LOCK(3)   [读1,2,3 / 写3]
             │           │           │
             └ 1,2 全程被 A 独占 ──────┘
   Thread B:              等待 LOCK(2) ...            ◄── 即使 B 只碰 4，也被 1,2 挡住
      (讲义 slide 22 原话: locks on node 1 and 2 during update to node 3
       could delay update to 4)  =>  读-读共享被当成冲突

             1
            / \
           2   ...
          / \
         3   4
         ▲   ▲
         A   B   (A 写 3，B 写 4；A/B 都"读"过 1,2)

② 事务：各自声明读写集，由系统判断是否真的冲突
   Transaction A: READ {1,2,3}  WRITE {3}
   Transaction B: READ {1,2,4}  WRITE {4}
                  ▲▲
                  └┘── 交集只有"读-读"(1,2) => slide 25: NO READ-WRITE or
                       WRITE-WRITE conflicts!  =>  两者可以并发提交(无冲突)

③ 若两个事务都写节点 3：READ{1,2,3} WRITE{3} ×2
   => slide 26: Conflicts exist: transactions must be serialized (W-W 冲突)

关键操作与性能特征：
  * 锁把"路径"变成临界区 => 延迟 = O(树高 h) 次锁操作，且被共享祖先串行化
  * 事务只需要"记录 + 提交时验证" => 读-读重叠免费，只有 W-W / R-W 才需要串行化
  * 但"记录"不是免费的：每个读/写都要额外指令与元数据(见 2.5、第 4 节)
```

2.2 为什么需要它（一）：锁的两难与"读-读并发"

- **定义与目的**：讲义用一个极简的 `deposit` 说明问题（slide 5—6）：`read-modify-write` 必须原子。用锁要写 `lock(); tmp = get(a); put(a, tmp+amt); unlock();`；用事务只写 `atomic { ... }`。后者之所以更好，不是因为"更短"，而是因为它**把并发度的决定权从程序员转移给了系统**：系统只在**真正冲突（R-W 或 W-W）**时才串行化（slide 6 原话：*Implementation discussed today uses optimistic concurrency: maintain serialization only in situations of true contention*）。
- **直观解释**：把 HashMap 的例子（slide 11—15）想成图书馆的书架：
- **非线程安全版**：没有任何同步原语，最快，但两人同时抽同一格会乱（slide 11：Java `HashMap` 恰好是 "Bad: not thread safe" 且 "Good: no lock overhead when synchronization not needed"）。
- **Java 1.4 的 `synchronized` 包装：整个书架一把大锁**。绝对安全，但一个人拿书全馆排队（slide 12：poor scalability）。
- **每桶一锁（finer-grained）**：每人只锁自己那一格。并发度上去了（slide 14 的实测曲线：fine locks 明显优于 coarse locks），但**"根本没人争用"时也要付锁开销**（slide 13 原话：*incurs lock overhead even if synchronization not needed*），而且锁的层级关系一不小心就死锁。
- **事务版**：`atomic { return m.get(key); }`（slide 15）。手写代码和"大锁版"一样简单，而并发度接近"细粒度锁版"——**这就是 TM 的核心卖点**。
- **讲义 slide 27 的性能对比（原文配图有三种曲线：coarse locks / fine locks / TCC）**：在 balanced tree 与 HashMap 两个数据结构上，"TCC"（硬件 TM）的曲线都优于或接近 fine locks，而 coarse locks 随处理器数增加迅速变差。请注意这张图的**含义**：TM 的卖点不是"永远最快"，而是**在几乎不增加编程难度的情况下达到接近专家手写细粒度锁的性能**。

2.3 为什么需要它（二）：失败原子性、可组合性、性能可移植性

- **失败原子性（failure atomicity，slide 29—30）**：手工写 `try/catch` 的 undo 代码会迅速失控——"必须记住要撤销什么、怎么撤销"，而且**有些副作用已经对其他线程可见**（例如未捕获的异常导致某个锁永远不释放，整个系统死锁）。事务把它变成系统责任：**异常 → 中止 → 内存更新全部撤销**，并且"失败的线程不持有任何锁"（slide 30 原话：*E.g., no locks held by a failing threads...*）。
- **可组合性（composability，slide 31—33）**：`transfer(A,B)` 里嵌套 `synchronized(A){ synchronized(B){...} }`，另一个线程调 `transfer(B,A)` → **死锁**。事务版的 `atomic { withdraw(A); deposit(B); }` 则是"**可组合的**"：`transfer` 里的事务**包含**（subsume）`withdraw` / `deposit` 里的事务，最外层定义原子性边界；`transfer(A,B,100)` 与 `transfer(B,A,200)` 会被系统串行化，而 `transfer(A,B,100)` 与 `transfer(C,D,200)` 则**自动并发**（slide 33 原话：*System manages concurrency as well as possible*）。

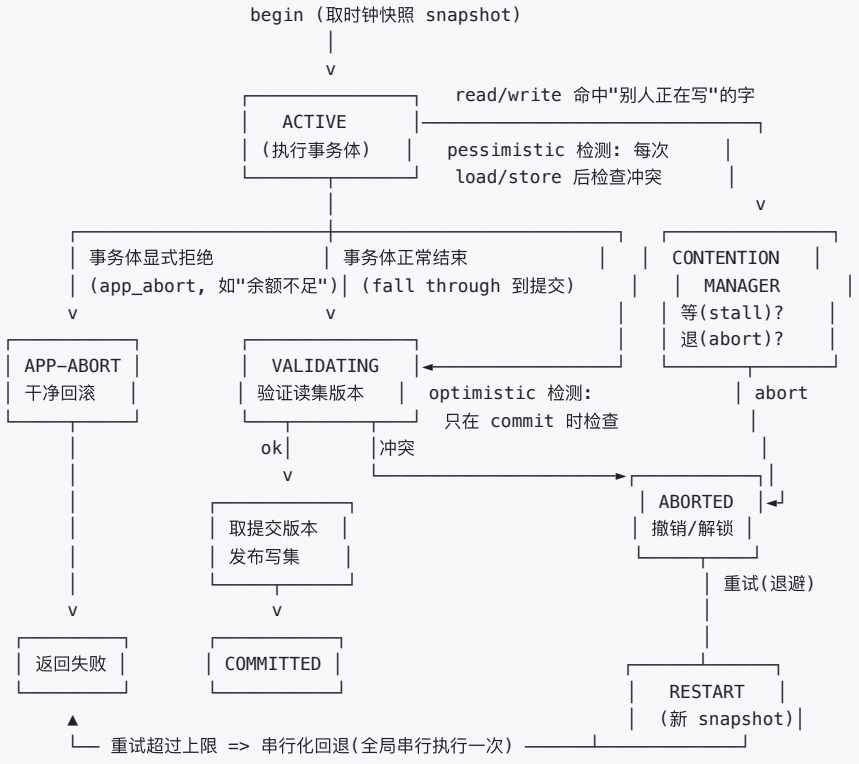
- **性能可移植性（performance portability，slide 34）**：为 4 核 CPU 精心调好的锁方案，到了 64 核往往不再最优（锁的争用结构变了、NUMA 变了）。事务把这件事交给系统，"一套代码吃到更多核"成为可能。
- **但"原子块 ≠ 锁"的反例（slide 36–37）**：把 `synchronized` 直接换成 `atomic` 是错误的。例如两个线程用 `flagA` / `flagB` 互相等待：

```
// Thread 1                              // Thread 2
atomic { ...; flagA = true;               atomic { ...; flagB = true;
         while (flagB == 0); ... }                 while (flagA == 0); ... }
     ^^^^^^ 事务里等待另一个事务的写 => 那个写在本事务提交前永不可见
            (隔离性!) => 永远等不到 => 活锁/死锁
```

结论：`atomic` **消除了大量数据竞争，但消除不了"原子性违规（atomicity violation）"**——程序员如果把本该原子的序列错误地切成两个原子块，一样出错（slide 37：`atomic { ptr = A; }` 与 `atomic { B = ptr->field; }` 之间，另一个线程可以把 `ptr` 置为 `NULL`）。

2.4 实现 TM 的三大设计问题与事务生命周期

- **定义与目的**：TM 系统必须提供原子性与隔离性，同时**尽可能保留并发**（slide 40）。为此有两个必答问题（讲义把它们明确成 "Two key implementation questions"）：
- **数据版本策略（data versioning policy）**：如何处理"未提交的新版本"与"已提交的旧版本"并存？→ **eager（undo-log）** 与 **lazy（write-buffer）**（slide 41）。
- **冲突检测策略（conflict detection policy）**：何时、如何判定两个并发事务冲突？→ **pessimistic** 与 **optimistic**（slide 46–49）。 第 3 问在 "What you should know"（slide 3）里被点名：**检测粒度（granularity of detection）**——按字、按缓存行还是按对象。
- **冲突的定义（slide 45）**：系统必须维护事务的 **read set（读集，事务中读过的地址）** 与 **write set（写集，事务中写过的地址）**，并检测：
- **Read–write conflict**：事务 A 读了地址 X，而 X 被"尚未提交"的事务 B 写过；
- **Write–write conflict**：A、B 都未提交，且都写 X。
- **ASCII 图解 D2：单个事务的生命周期状态机（把 slide 41–49 的所有策略画在同一张图上）**




```
关键操作与性能特征:
 * ACTIVE 期间每一次 read/write 都是"仪器化"(instrumented)的内存访问 => 额外指令
 * VALIDATING 的代价 ∝ |read set|, 且要走一致性(可能触发 miss)
 * 取提交版本 = 一次全局原子 RMW => 全部事务的串行化点(第 4 节会算它的天花板)
 * ABORT 的代价: eager 需要按 undo log 恢复旧值, lazy 只需丢缓冲+解锁
```

- **数据版本策略：eager vs lazy（slide 41–44）**

维度	Eager versioning（undo-log based）	Lazy versioning（write-buffer based）
写操作做什么	**直接改内存**，把旧值记入 undo log（每次 store 有额外开销）	写只进**写缓冲区**，内存保持旧值
commit 做什么	几乎不用做——数据已在内存，只需发布/解锁	把写缓冲**冲刷（flush）**到内存
abort 做什么	按 undo log 把旧值**逐个写回**	丢掉缓冲区即可（内存从未被改）
优点	**提交快**	**中止快**，且没有"事务中途崩溃留下脏数据"的容错问题
缺点	**中止慢**，容错性差（想想事务执行到一半崩溃）	**提交慢**
思想	"先写下去，赌它不会回滚"（讲义原话：*write to memory immediately, hoping transaction won't abort*）	"非写不可时才写内存"（*only write to memory when you have to*）
适合的负载	冲突/中止很少（低争用）	冲突/中止很多（高争用）

- **冲突检测策略：pessimistic vs optimistic（slide 46–50）**

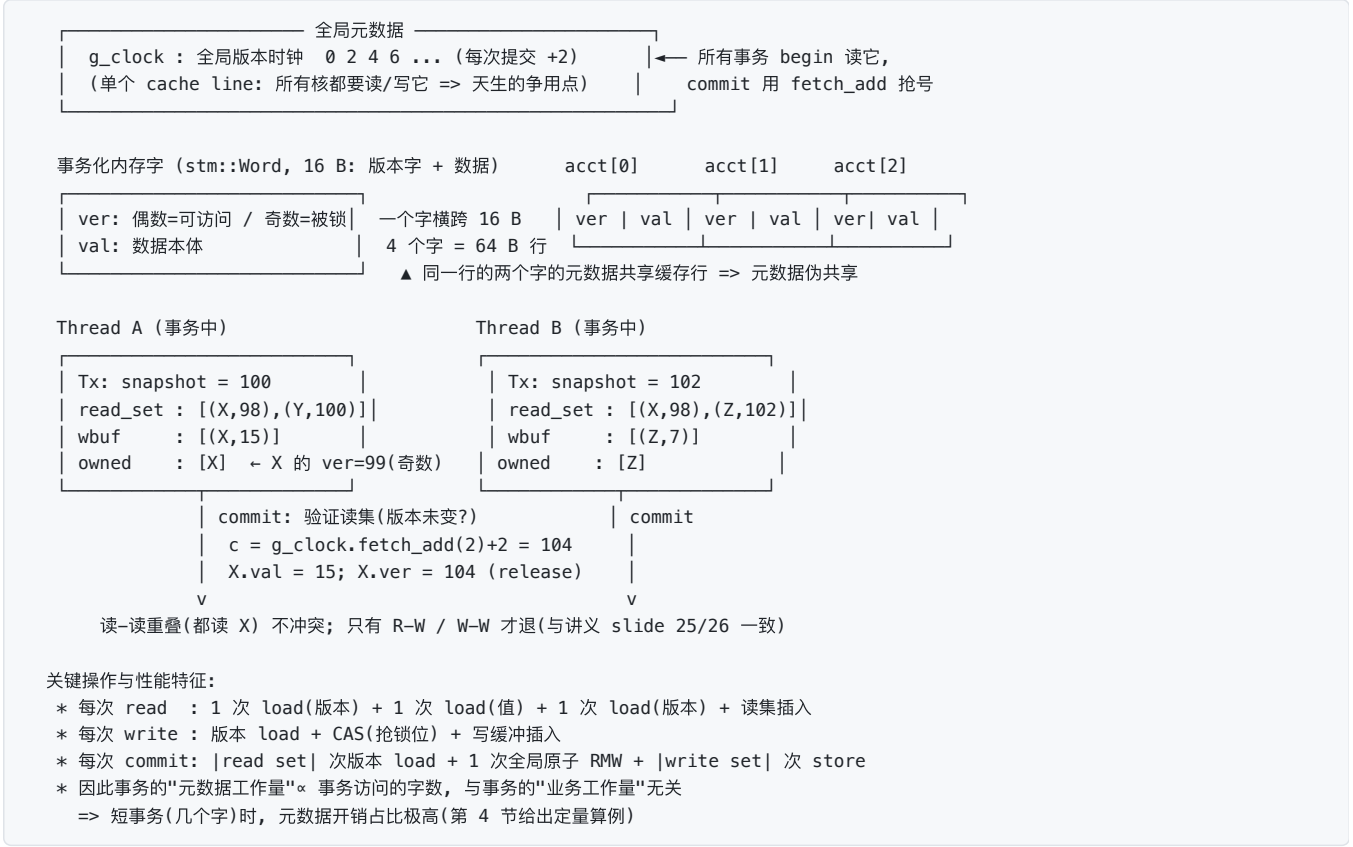
维度	Pessimistic（a.k.a. eager）	Optimistic（a.k.a. lazy / commit-time）
何时检查	**每次 load/store 时**立即检查（slide 46）	**只在 commit 时**检查（slide 48）
冲突后谁优先	由 **contention manager（争用管理器）** 决定 stall 还是 abort	**优先让"正在提交的事务"成功**，其他事务稍后 abort
优点	早发现 → 少做无用功；还能把一部分 abort 转成 **stall**	**有前进保证（forward progress）**；通信/检测是**批量**的（一次验证整个读集）
缺点	**没有前进保证**（可能活锁，讲义特意用 "question: how to avoid livelock?" 提醒）；细粒度通信（每条 load/store 都可能检查）；**检测在关键路径上**	发现得晚（可能白做很多工作）；仍可能有不公平（fairness）问题
典型代价位置	读/写路径（延迟敏感）	提交路径（吞吐敏感）

- **检测粒度（granularity）**：这是第 3 个设计维度，讲义的 "What you should know" 明确列出，而它的后果与"伪共享"完全同源：

粒度	元数据开销	伪冲突风险	典型实现
字（word）	大：每个字都要版本/所有权信息（本文档的实现是 **16 B 元数据 + 数据**，即 100% 空间开销）	低（只有真正同一个字才冲突）	软件 TM（本文 3.1/3.2 的 `stm::Word`）
缓存行（cache line）	小：复用缓存的 tag/状态位，几乎零额外空间	**高**：同一行里两个互不相干的字被当成冲突	硬件 TM（Intel TSX/RTM、TCC 类设计）
对象（object）	中：每个对象一个版本/锁字段	中：同一对象的不同字段也被当冲突	对象级 STM

2.5 软件 TM 的执行模型：版本时钟 + 锁位 + 读集验证 + 写缓冲

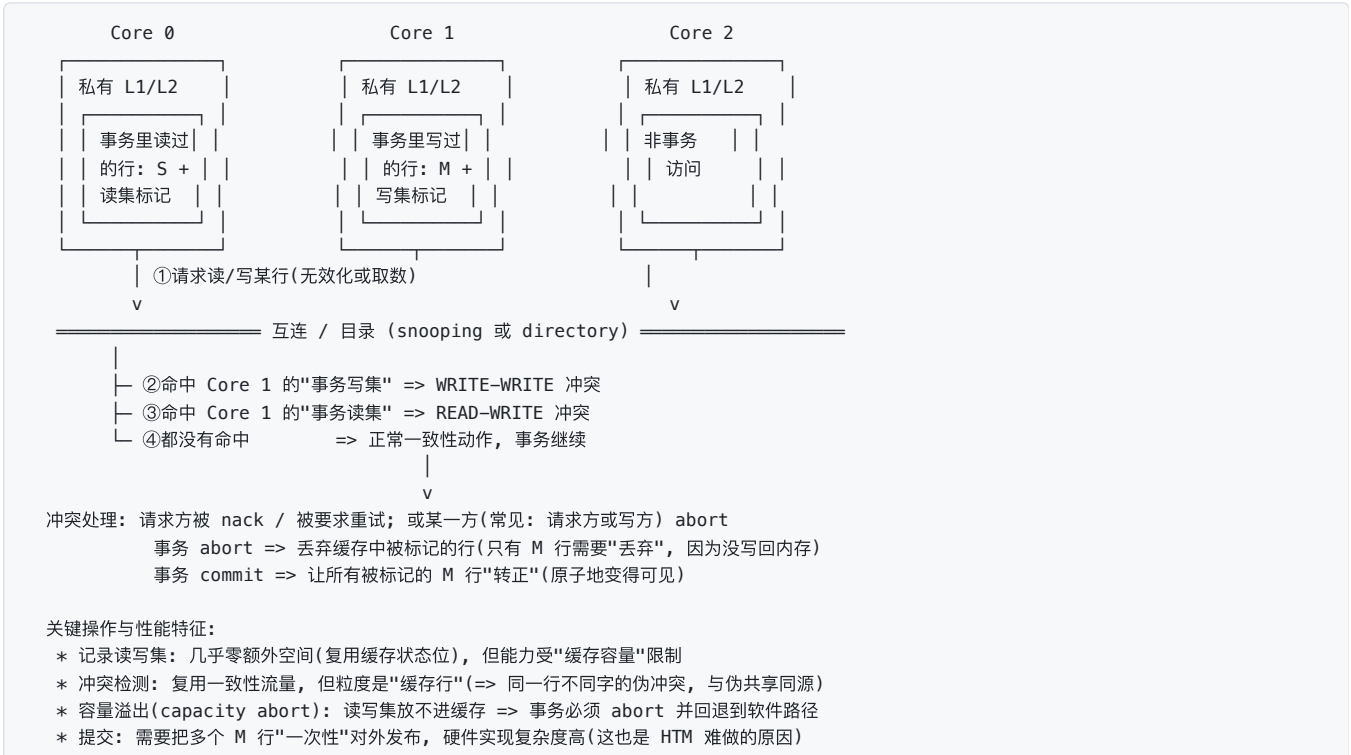
- **定义与目的**：本笔记第 3.1 节给出一个**完整可运行的教学 STM**。它的机制选择非常"标准"：**lazy versioning（写缓冲，提交时冲刷）** + **混合冲突检测**（读/写时检查版本字，**提交时验证读集**）+ **字粒度** + **全局版本时钟**分配提交序号。这套组合足以让"所有共享访问都走事务"的程序严格可串行化。
- **直观解释（"它是什么？"）**：把每个共享字想象成**带封条的储物柜**：
- 正常时封条上写着一个**编号**（版本号，偶数）——这个编号表示"柜子里的东西是第几号事务放进去的"；
- 事务要**改**某个柜子时，先把封条翻到"**占用中**"（编号变奇数 = 加了锁位），并把自己的新东西**先放在自己的包里**（写缓冲），不放进柜子；
- 事务要**读**某个柜子时，先看封条：如果写着"占用中"→ 说明有人正在改，这就是冲突；如果编号比"我进场时的全局编号"还新 → 说明这是**我进场之后才提交**的结果，读了它就无法解释成串行执行，于是也退；
- **提交**：先核对读过的所有柜子的封条**没有被换过**（读集验证），然后领一个**新的全局编号**（一次原子取号），把自己的包**一次性倒进**各个柜子并换上这个新编号。
- **回退**：只有一个逃生门——**全局串行**（把所有并发者挡在外面，独占执行一次），保证任何事务最终都能前进。
- **ASCII 图解 D3：软件 TM 的内存布局与两个线程的执行模型**



2.6 硬件 TM（公开知识补充）：把一致性协议"升级"成冲突检测器

> 本小节是**公开知识补充**（Fall 2026 讲义尚未发布；讲义在 slide 9 提出"对一组地址维持单个地址的性质"、在 slide 27 用 TCC 作为硬件 TM 的例子，但未展开硬件实现细节）。这里给出业界与文献中通用的 HTM 图景，用于理解讲义设计空间在硬件上的映射。

- **定义与目的**：硬件 TM（HTM，如 Intel TSX/RTM）让处理器**用私有缓存记录 read set / write set，并复用缓存一致性协议做冲突检测**：别的核想要我以事务方式读过的行（或想抢我以事务方式写过的行），就是一次冲突。
- **直观解释**：软件 TM 里"每个字带一个版本封条"要花掉 100% 的空间；硬件 TM 换了个思路——**缓存里本来就有每一行的状态（M/E/S/I）**，那就把"事务读过的行""事务写过的行"直接记在缓存状态上：事务读一行 → 该行以 **Shared** 状态留在缓存并打上"读集标记"；事务写一行 → 该行以 **Modified** 状态留在缓存并打上"写集标记"（**新值根本不写回内存，也不让别的核看到**——隔离性由一致性协议天然保证）。提交时，这些 Modified 行"转正"即可。类比：软件 TM 是给每个抽屉贴封条，HTM 是**让图书馆的门禁系统记录你摸过哪些书架**——记录是免费的，因为门禁本来就在那儿。
- **ASCII 图解 D4：基于缓存的 HTM 结构与冲突检测**



- **HTM 的编程模型（Intel TSX/RTM 的具体形式，公开知识）**：`_xbegin()` 开始事务，返回 `_XBEGIN_STARTED` 表示进入事务；`_xend()` 提交；`_xabort(code)` 显式中止；失败时返回的**状态位**可区分原因（`_XABORT_CONFLICT` 冲突、`_XABORT_CAPACITY` 容量溢出、`_XABORT_EXPLICIT` 显式中止、`_XABORT_RETRY` "立刻重试可能成功"），用 `_XABORT_CODE(st)` 可取应用自定义的 code。**RTM 不提供前进保证**：事务可能永远失败（例如另一个核一直在同一行上做非事务写），因此**软件必须准备回退路径（通常是一把锁）**。另外：**在不支持 RTM 的处理器上执行 `XBEGIN` 会产生 `#UD` 非法指令异常**，所以运行前必须用 CPUID 探测（第 3.3 节的实测代码里，这一条被真实触发过，见第 6 节陷阱 8）。
- **STM vs HTM 对比（汇总）**：

维度	软件 TM（STM）	硬件 TM（HTM / RTM）
读写集记录	显式元数据（版本字/锁位），**空间开销大**（本实现 16 B/字）	复用缓存状态位，**空间开销≈0**，但受缓存容量限制
检测粒度	可做到**字/对象级**（伪冲突少）	**缓存行级**（同源伪冲突）
检测延迟	走普通内存访问，**慢**（额外 load/CAS）	一次一致性请求即可，**快**
容量上限	几乎无上限（受内存限制）	有（L1/L2 容量 → capacity abort）
能否回滚 I/O、系统调用	不能（同 HTM）	**不能**（事务内禁止系统调用/页错误/中断）
可移植性	纯软件，任何平台（但要自己写正确，很难）	依赖具体 ISA，且历史上曾因勘误被微码关闭
前进保证	由争用管理器 + 串行化回退提供	**无**（必须软件回退）
本文档的示例	3.1 / 3.2（`stm.hpp`）	3.3（`tsx.cpp`）
---		

3. 代码示例与性能分析

本节的三段代码都是**完整可编译可运行**的，并且都在本机实测通过（编译命令写在每个文件头部）。它们的目的是把第 2 节的设计空间"落地"：

- **示例 1（`stm.hpp` + `transfer.cpp`）**：从零实现一个教学用**软件事务内存（STM）**——全局版本时钟 + 惰性/急切两种数据版本策略（用 `-DSTM_EAGER=1` 一键切换）+ 提交时乐观检测 + 争用管理器 + 串行化回退；用它跑**银行转账**，并用"总额守恒"这一全局不变量来检验**可串行化与失败原子性**。
- **示例 2（`bst.cpp`）**：讲义 slide 16—27 的**树更新**实验的完整复现——同一棵二叉搜索树上比较 **手递手细粒度锁 / 单把全局锁 / 事务** 三种实现，并用中序遍历验证三棵树的正确性。

- **示例 3（`tsx.cpp`）：硬件事务内存**（Intel TSX/RTM）的编程模型——`_xbegin/_xend/_xabort`、abort 状态位解码、CPUID 探测与锁回退，并用一个"读取者必须同时看到 8 个账户"的不变量来检验**隔离性（无撕裂读）**。

3.1 示例 1：一个可运行的软件事务内存（STM）与"银行转账"

文件 `stm.hpp`（教学 STM：约 190 行，含全部机制注释）

```
// stm.hpp —— 教学用软件事务内存 (Software Transactional Memory, STM)
// 设计要点:
//   1) 全局版本时钟 (global version clock) 给每个已提交事务分配提交序号 => 可串行化
//   2) 细粒度"锁位"表达写集所有权: 版本字最低位 = 1 表示被某事务独占
//   3) 数据版本策略可选: 惰性(写缓冲, LAZY) / 急切(undo log, EAGER)
//   4) 冲突检测: 读时检查版本(悲观/急切的"见锁即退") + 提交时验证读集(乐观)
//   5) 前进保证: 有限次退避重试后进入"排空式全局串行"回退, 消除活锁
// 用法/编译: 本文件是头文件, 与使用方一起编译, 例如
//   g++ -O3 -std=c++17 -pthread transfer.cpp -o transfer            (惰性版本)
//   g++ -O3 -std=c++17 -pthread -DSTM_EAGER=1 transfer.cpp -o transfer_eager
#pragma once
#include <atomic>
#include <cstdint>
#include <vector>
#include <utility>
#include <functional>
#include <mutex>
#include <thread>
#include <chrono>

#ifndef STM_EAGER
#define STM_EAGER 0          // 0 = 惰性版本(写缓冲), 1 = 急切版本(undo log)
#endif

namespace stm {

struct Word {                                  // 一个"事务化内存字"
    std::atomic<uint64_t> ver{0};              // 偶数=可访问; 奇数=被某事务独占
    long val{0};                               // 数据本体
};
static_assert(sizeof(Word) == 16, "Word should be 16B");

inline std::atomic<uint64_t> g_clock{0};       // 全局版本时钟, 每次提交 +2
inline std::atomic<bool>     g_serial{false};  // 全局串行模式(回退)标志
inline std::mutex            g_serial_lock;    // 回退互斥

// 统计(可选)
inline std::atomic<uint64_t> g_n_abort{0}, g_n_fallback{0}, g_n_commit{0};

struct Tx {
    uint64_t snapshot = 0;
    bool app_abort = false;                                // 应用层显式回滚
    std::vector<std::pair<Word*,uint64_t>> read_set;     // (地址, 当时版本)
    std::vector<Word*> owned;                              // 已持有的写集
#if STM_EAGER
    std::vector<std::pair<Word*,long>> undo;             // 旧值(undo log)
#else
    std::vector<std::pair<Word*,long>> wbuf;             // 新值(写缓冲)
#endif

    void begin() {
        app_abort = false;
        read_set.clear(); owned.clear();
        if (read_set.capacity() == 0) { read_set.reserve(32); owned.reserve(8); }  // 避免事务内扩容
#if STM_EAGER
        undo.clear(); if (undo.capacity() == 0) undo.reserve(8);
#else
        wbuf.clear(); if (wbuf.capacity() == 0) wbuf.reserve(8);
#endif
        snapshot = g_clock.load(std::memory_order_acquire);
    }

    bool read(Word* w, long& out) {
#if STM_EAGER
        for (Word* p : owned) if (p == w) { out = w->val; return true; }        // 值已在内存
#else
        for (auto& e : wbuf) if (e.first == w) { out = e.second; return true; } // 取自写缓冲
#endif
        if (read_committed(w, out)) return true;
        // 争用管理器(contention manager): 面对"被独占"的字, 决定等还是退
        if (!owned.empty()) return false;          // 已持写集: 立刻退, 避免环形等待/死锁
```

```
        if (!wait_unlocked(w)) return false;       // 未持写集: 有界等待持有者提交或回滚
        return read_committed(w, out);             // 重新读一次
    }

    bool read_committed(Word* w, long& out) {      // 读一个未被独占的字
        uint64_t v1 = w->ver.load(std::memory_order_acquire);
        if (v1 & 1ULL) return false;               // 被某事务独占
        long v = w->val;
        uint64_t v2 = w->ver.load(std::memory_order_acquire);
        if (v1 != v2) return false;                // 读到撕裂
        if (v1 > snapshot) return false;           // 提交于本事务开始之后 => 不可串行化
        read_set.emplace_back(w, v1);
        out = v;
        return true;
    }

    static bool wait_unlocked(Word* w) {           // 有界等待: 持有者很快会提交/回滚
        for (int r = 0; r < 8192; ++r) {
            if (!(w->ver.load(std::memory_order_acquire) & 1ULL)) return true;
            if ((r & 255) == 255) std::this_thread::yield();
        }
        return false;
    }

    bool write(Word* w, long x) {
        for (Word* p : owned) if (p == w) {
#if STM_EAGER
            w->val = x;
#else
            for (auto& e : wbuf) if (e.first == w) { e.second = x; break; }
#endif
            return true;
        }
        uint64_t v = w->ver.load(std::memory_order_acquire);
        if (v & 1ULL) return false;
        if (v > snapshot) return false;
        uint64_t expect = v;
        if (!w->ver.compare_exchange_strong(expect, v + 1, std::memory_order_acq_rel)) return false;
        owned.push_back(w);                    // 抢到所有权
#if STM_EAGER
        undo.emplace_back(w, w->val);
        w->val = x;
#else
        wbuf.emplace_back(w, x);
#endif
        return true;
    }

    bool commit() {
        for (auto& r : read_set) {             // 1) 验证读集(乐观检测在此发生)
            bool mine = false;
            for (Word* p : owned) if (p == r.first) { mine = true; break; }
            if (!mine && r.first->ver.load(std::memory_order_acquire) != r.second) { abort(); return false; }
        }
        uint64_t c = g_clock.fetch_add(2, std::memory_order_acq_rel) + 2;   // 2) 申请提交版本
#if STM_EAGER
        for (Word* w : owned) w->ver.store(c, std::memory_order_release);
#else
        for (auto& e : wbuf) {                 // 3) 冲刷写缓冲: 先写值, 再 release 发布版本
            e.first->val = e.second;
            e.first->ver.store(c, std::memory_order_release);
        }
#endif
        g_n_commit.fetch_add(1, std::memory_order_relaxed);
        read_set.clear(); owned.clear();
#if STM_EAGER
        undo.clear();
#else
        wbuf.clear();
#endif
        return true;
    }
```

```
    void abort() {                              // 回滚 + 释放所有锁
#if STM_EAGER
        for (auto& e : undo) e.first->val = e.second;    // 急切版本: 恢复旧值
#endif
        for (Word* w : owned) {
            uint64_t cur = w->ver.load(std::memory_order_relaxed);
            w->ver.store(cur & ~1ULL, std::memory_order_release);
        }
        read_set.clear(); owned.clear();
#if STM_EAGER
        undo.clear();
#else
        wbuf.clear();
#endif
    }
};

inline void backoff(int attempt) {
    int r = 1 << (attempt < 6 ? attempt : 6);
    for (volatile int i = 0; i < r * 64; ++i) { }
    if (attempt >= 4) std::this_thread::yield();
}

// 正常路径: 乐观重试; 超过 tries 次则进入排空式串行回退(保证前进)
template <class F>
bool run_tx(F body, uint64_t* n_abort = nullptr, uint64_t* n_serial = nullptr, int tries = 16) {
    for (int a = 0; a < tries; ++a) {
        if (g_serial.load(std::memory_order_acquire)) { std::this_thread::yield(); backoff(a); continue; }
        Tx tx; tx.begin();
        bool body_ok = body(tx);
        bool done = body_ok && tx.commit();
        bool app = tx.app_abort;
        if (!done) tx.abort();                  // 所有失败路径都必须回滚(释放锁)
        if (done) return true;
        if (app) return false;                  // 应用层拒绝(如"余额不足"): 干净回滚, 不重试
        g_n_abort.fetch_add(1, std::memory_order_relaxed);
        if (n_abort) ++*n_abort;
        backoff(a);
    }
    // 回退: 全局串行执行一次(带时限的停滞重试), 保证前进
    std::lock_guard<std::mutex> g(g_serial_lock);
    g_serial.store(true, std::memory_order_release);
    g_n_fallback.fetch_add(1, std::memory_order_relaxed);
    if (n_serial) ++*n_serial;
    auto deadline = std::chrono::steady_clock::now() + std::chrono::milliseconds(1000);
    for (;;) {
        Tx tx; tx.begin();
        bool body_ok = body(tx);
        bool done = body_ok && tx.commit();
        bool app = tx.app_abort;
        if (!done) tx.abort();
        if (done || app || std::chrono::steady_clock::now() > deadline) {
            g_serial.store(false, std::memory_order_release);
            return done;
        }
        std::this_thread::yield();
    }
}

} // namespace stm
```

文件 `transfer.cpp`（用上面的 STM 做银行转账；检验可串行化、失败原子性与"读己之所写"）

```cpp
// Bank transfer(银行转账)微基准: 用同一套 STM 跑"组合式"事务, 并检验不变量
// 编译(惰性版本): g++ -O3 -std=c++17 -pthread transfer.cpp -o transfer
// 编译(急切版本): g++ -O3 -std=c++17 -pthread -DSTM_EAGER=1 transfer.cpp -o transfer_eager
#include "stm.hpp"
#include <cstdio>
#include <cstdlib>
#include <thread>
#include <vector>
#include <atomic>

struct Account { stm::Word balance; };
static Account acct[4096];

// ---- 事务化的原子操作(可组合的构建块) ----
static bool deposit(stm::Tx& tx, Account& a, long amt) {
    long b; if (!tx.read(&a.balance, b)) return false;      // 冲突 => 重试
    return tx.write(&a.balance, b + amt);
}
static bool withdraw(stm::Tx& tx, Account& a, long amt, long* ryow_bad) {
    long b; if (!tx.read(&a.balance, b)) return false;
    if (b < amt) { tx.app_abort = true; return false; }     // "余额不足"=应用层拒绝, 干净回滚
    if (!tx.write(&a.balance, b - amt)) return false;
    long again;                                             // 检验"读己之所写"(写缓冲/内存一致性)
    if (!tx.read(&a.balance, again)) return false;
    if (again != b - amt && ryow_bad) ++*ryow_bad;
    return true;
}
// transfer 组合 withdraw/deposit: 最外层事务定义原子性边界
// fail_deposit=true 模拟"入账阶段抛异常": 已完成的取款必须被整体回滚(failure atomicity)
static bool transfer(stm::Tx& tx, Account& f, Account& t, long amt, bool fail_deposit, long* ryow_bad) {
    if (!withdraw(tx, f, amt, ryow_bad)) return false;
    if (fail_deposit) { tx.app_abort = true; return false; }
    return deposit(tx, t, amt);
}

int main(int argc, char** argv) {
    const int T = (argc > 1) ? atoi(argv[1]) : 4;
    const int ITERS = (argc > 2) ? atoi(argv[2]) : 200000;
    const int N = (argc > 3) ? atoi(argv[3]) : 1024;
    const long INIT = 1000000;
    for (int i = 0; i < N; ++i) acct[i].balance.val = INIT;
    long expect = (long)N * INIT;

    std::atomic<uint64_t> aborts{0}, serials{0}, app_fail{0}, lost{0}, ryow_bad{0};
    auto work = [&](int tid) {
        uint32_t seed = 12345u + tid * 977u;
        uint64_t ab = 0, se = 0; long rb = 0;
        for (int i = 0; i < ITERS; ++i) {
            seed = seed * 1664525u + 1013904223u;
            int a = (int)((seed >> 8) % (unsigned)N);
            int b = (int)((seed >> 16) % (unsigned)N);
            if (a == b) b = (b + 1) % N;
            if (seed & 0x8000u) { int t = a; a = b; b = t; }        // 双向, 避免单向抽干账户
            long amt = 1 + (long)((seed >> 24) % 100);
            bool must_fail = ((i & 15) == 15);                       // 每 16 次一次"入账失败"
            bool ok = stm::run_tx([&](stm::Tx& tx) {
                return transfer(tx, acct[a], acct[b], amt, must_fail, &rb);
            }, &ab, &se);
            if (must_fail ? ok : !ok) lost.fetch_add(1, std::memory_order_relaxed);
            if (must_fail) app_fail.fetch_add(1, std::memory_order_relaxed);
        }
        aborts.fetch_add(ab, std::memory_order_relaxed);
        serials.fetch_add(se, std::memory_order_relaxed);
        ryow_bad.fetch_add((uint64_t)rb, std::memory_order_relaxed);
    };

    auto t0 = std::chrono::steady_clock::now();
    std::vector<std::thread> th;
    for (int i = 0; i < T; ++i) th.emplace_back(work, i);
    for (auto& t : th) t.join();
    double sec = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
```

```
    long total = 0; for (int i = 0; i < N; ++i) total += acct[i].balance.val;
    printf("%s T=%2d N=%5d : %7.1f ns/committed-tx  %6.2f M tx/s  abort=%5.1f%%  fallback=%llu  "
           "lost=%llu  ryow_bad=%llu  sum=%s\n",
#if STM_EAGER
           "EAGER",
#else
           "LAZY ",
#endif
           T, N, sec * 1e9 / ((double)T * ITERS) * T / T * 1.0,
           (double)T * ITERS / sec / 1e6,
           100.0 * aborts.load() / (aborts.load() + (double)T * ITERS),
           (unsigned long long)serials.load(),
           (unsigned long long)lost.load(), (unsigned long long)ryow_bad.load(),
           total == expect ? "OK" : "*** CORRUPTED ***");
    (void)app_fail;
    return (total == expect && lost.load() == 0 && ryow_bad.load() == 0) ? 0 : 1;
}
```

【代码做什么？】（`transfer.cpp`）

1. **建立不变量**：N 个账户各 1,000,000 元，全局"总额守恒"是被检验的不变量——任何"扣了款却没入账"或"事务只做了一半"的错误都会立刻破坏总额。
2. **T 个线程各自做 ITERS 次转账**：用线性同余随机数选 `from` / `to` 两个不同账户、随机金额（1..100），并**随机交换方向**（避免单向转账把某个账户抽干）。
3. **每 16 次故意制造一次失败**：`transfer(..., fail_deposit=true)` 会先**成功取款**（已写入事务），然后在"入账"阶段显式拒绝（`tx.app_abort = true`）。这就精确模拟了讲义 slide 29–30 的 `try/catch` 场景：**已完成的取款必须被整体回滚**（failure atomicity）。程序统计"应当失败的次数"与"实际失败次数"，二者必须相等。
4. **`withdraw` 里额外做一次"读己之所写"检查**：写入新余额后立刻再读一次，确认读到的是**本次事务写入的新值**（而不是内存里的旧值），违例计入 `ryow_bad`（必须为 0）。
5. **主流程**：`run_tx` 最多乐观重试 16 次（默认 `tries=16`）→ 成功则提交；最后打印时间、每次已提交事务的纳秒数、中止率、串行化回退次数、`sum=OK/CORRUPTED`。

【并行机制与性能解说】

- **线程与工作分配**：T 个 `std::thread`，每个线程独立生成自己的随机序列（`seed = 12345 + tid*977`），**没有中心任务队列**——线程之间唯一的交互是它们碰巧选到同一对账户时的**真冲突**。这正是"事务把同步决策交给系统"的含义：程序员只声明 `atomic { withdraw; deposit; }`，没有任何锁序知识。
- **硬件上发生了什么**：`read()` 是"版本 load→值 load→版本 load"的三次访问 + 一次读集插入；`write()` 是"版本 load + CAS 抢锁位 + 写缓冲插入"；`commit()` 是"逐条验证读集 + 一次全局原子 `fetch_add` + 逐条发布写集"。没有一次操作会阻塞线程——**冲突全部转化为"回退 + 退避 + 重试"**，这是乐观并发的典型形态（与讲义 slide 48–49 的 optimistic detection 一致）。
- **Work / Span / 并行度（本实验的定量骨架）**：
- **Work（总工作量）**：`W = N_tx · c_tx`，其中 `N_tx = T·ITERS = 4×200,000 = 800,000` 次已提交事务，`c_tx = c_begin + r·c_read + w·c_write + c_commit`（本实验 r = 3 次字读、w = 2 次字写）。实测**单线程 133.6 ns/事务**，即 `c_tx ≈ 400` 个 3.0 GHz 周期。
- **Span（关键路径）**：所有事务的提交都必须**串行取一次全局版本号**（`g_clock.fetch_add`），所以 `Span = N_tx · c_serial`（`c_serial` ≈ 150 周期，见 4.1 的分解）。
- **并行度 = Work / Span ≈ 400 / 150 ≈ 2.7**：这就是"**即使有 128 个核，这个基准也几乎不可能超过 2.7×**"的来源。实测最好的一组是 **1,336 ns/tx（T=1）→ 76.7 ns/tx（T=4）≈ 1.74×**，与模型同量级；差距来自跨 NUMA 节点时时钟行的弹跳和读集验证的缓存未命中。
- **冲突带来的额外工作量**：`p` 概率冲突时，期望工作量是 `W/(1-p)`（重试的活儿白干）。这也解释了为什么 3.2 节的树实验里"事务几乎不 abort"是关键优势。
- **实测数据（同一进程内的一组对照，T=4，`taskset -c 0-3` 固定到同一个 CCD）**：

配置（账户数 N）	惰性版本 LAZY（ns/已提交事务）	急切版本 EAGER（ns/已提交事务）	LAZY 中止率	说明
N = **2**（极端争用）	**169.4**	205.3	17.6%	冲突几乎全部被"停滞等待"吸收，但吞吐被串行化拖死
N = **8**	176.3	201.6	38.6%	冲突最激烈的区间：**中止率最高、吞吐最低**

配置（账户数 N）	惰性版本 LAZY（ns/已提交事务）	急切版本 EAGER（ns/已提交事务）	LAZY 中止率	说明
N = **64**	84.9	87.9	9.1%	中止率与"冲突概率模型"吻合（见 4.2）
N = **1024**（几乎无争用）	76.7	73.0	0.7%	纯元数据开销区间：**急切版本反而略快**（提交更便宜）

- 所有配置下 `sum=OK`（总额守恒）、`ryow_bad=0`（读己之所写正确）、成功/失败事务计数与预期一致 → **可串行化与失败原子性都被真正验证了**，而不只是"看起来能跑"。
- **急切 vs 惰性的结论与讲义 slide 44 的表格完全一致**：低争用（N=1024，中止率 0.7%）时 eager 略快（提交便宜）；高争用（N=2, N=8）时 lazy 明显更快（中止便宜、中止次数还更少）——**"eager 赌不中止，lazy 赌会中止"，实测给出了这张赌注的赔率**。
- 注意 N=2 的中止率（17.6%）反而**低于** N=8（38.6%）：这不是矛盾，而是争用管理器起了作用——在只有两个字的极端争用下，`wait_unlocked()` 把大量"本该 abort"的冲突变成了**短暂停滞**（对应讲义 slide 47 Case 2 "early detect (and stall)"）。**中止率低并不等于性能好**：N=2 的每事务耗时（169 ns）是 N=1024（77 ns）的两倍多。
- **瓶颈在哪**：不在内存带宽（1024 个账户只有 16 KB，全在缓存里），也不在业务代码，而在 **(a) 全局版本时钟这一个缓存行**（提交时一次原子 RMW + 开始时一次 load，所有核共享）；**(b) 每事务的元数据操作数**（r+w 次仪器化访问）；**(c) 冲突重试**。第 4 节会把 (a) 算成一个硬天花板。

3.2 示例 2：并发二叉搜索树——手递手锁 vs 全局锁 vs 事务

本示例是讲义 slide 16–27（树更新，图 credit: Austen McDonald）的**可运行复现**。实验设计要点：

- **键区间互不相交**：把键空间切成 T 段，每段由**一个线程独占**（线程 t 只插入 `(t·PER, (t+1)·PER)` 内的键），并预先单线程插入 T 个"分界键"作为骨架。于是各线程**只写自己子树内的子指针**，但**都要读从根下来的那一串祖先**——这正是讲义 slide 23–25 的情形：**共享的只有"读"**。
- **插入顺序为保证平衡**：每段内部按"先插中点、再递归左右"的顺序插入（`fill_bal`），否则按从小到大插入会让 BST 退化成链表（本文档的第一版就踩了这个坑：树高 20000，每次插入要走 20000 个节点，三种实现都慢到 60 μs/次）。
- **事务的节点预分配**：`malloc` / `new` **不可回滚**，所以新节点**在事务之外**分配好，事务体里只做"把指针挂到空位上"这一件事。这是把"副作用"赶出事务的标准做法（另见第 6 节陷阱 7）。
- **手递手锁**：从根开始 `lock(父)` → `lock(子)` → `unlock(父)`，锁住路径上每个节点；**粗粒度锁**：一把全局 `std::mutex` 包住整次插入。
- **正确性验证**：对三棵树分别做中序遍历，要求**节点数与"严格递增"同时成立**——如果 STM 出现隔离性错误（丢更新、撕裂），遍历结果几乎必然会暴露。

文件 `bst.cpp`

```
// 并发二叉搜索树插入: 粗粒度锁 / 手递手细粒度锁 / 事务, 三者对比
// 编译: g++ -O3 -std=c++17 -pthread bst.cpp -o bst
#include "stm.hpp"
#include <cstdio>
#include <cstdlib>
#include <thread>
#include <vector>
#include <mutex>
#include <chrono>
#include <functional>
#include <atomic>

// ---------------- 事务版本: 子指针是事务化内存字 ----------------
struct TNode {
    long key;                    // 发布后不可变: 可非事务化读
    stm::Word left, right;       // 子指针(以 long 存指针), 0 表示空
};
static stm::Word root_word;

static bool tx_insert(stm::Tx& tx, long k, TNode* nn) {
    stm::Word* cur = &root_word; long c;
    if (!tx.read(cur, c)) return false;
    for (;;) {
        if (c == 0) return tx.write(cur, (long)nn);        // 挂到空位上
        TNode* n = (TNode*)c;
        stm::Word* child = (k < n->key) ? &n->left : &n->right;
        long cc; if (!tx.read(child, cc)) return false;
        if (cc == 0) return tx.write(child, (long)nn);
        cur = child; c = cc;                               // 继续下降
    }
}
static void stm_insert(long k, uint64_t* ab, uint64_t* se) {
    TNode* nn = new TNode{k, {}, {}};                      // malloc 不可回滚 => 放在事务之外
    bool ok = stm::run_tx([&](stm::Tx& tx) { return tx_insert(tx, k, nn); }, ab, se);
    if (!ok) { fprintf(stderr, "insert %ld failed\n", k); exit(1); }
}

// ---------------- 手递手(hand-over-hand)细粒度锁版本 ----------------
struct LNode {
    long key;
    LNode *left = nullptr, *right = nullptr;
    std::mutex m;
};
static LNode* lock_root = nullptr;

static void fine_insert(long k) {
    LNode* cur = lock_root;
    cur->m.lock();                                          // 先锁父, 再锁子, 然后放父
    for (;;) {
        LNode** slot = (k < cur->key) ? &cur->left : &cur->right;
        if (*slot == nullptr) { *slot = new LNode{k}; cur->m.unlock(); return; }
        LNode* nxt = *slot;
        nxt->m.lock();
        cur->m.unlock();
        cur = nxt;
    }
}

// ---------------- 粗粒度锁版本 ----------------
static std::mutex g_big_lock;
static void coarse_insert(LNode*& root, long k) {
    std::lock_guard<std::mutex> g(g_big_lock);
    LNode** p = &root;
    while (*p) p = (k < (*p)->key) ? &(*p)->left : &(*p)->right;
    *p = new LNode{k};
}

// ---------------- 校验: 中序遍历必须严格递增 ----------------
static long verify_tx(TNode* n, long* prev, bool* sorted) {
    if (!n) return 0;
    long c = verify_tx((TNode*)n->left.val, prev, sorted);
```

```
    if (n->key <= *prev) *sorted = false;
    *prev = n->key;
    return c + 1 + verify_tx((TNode*)n->right.val, prev, sorted);
}
static long verify_lock(LNode* n, long* prev, bool* sorted) {
    if (!n) return 0;
    long c = verify_lock(n->left, prev, sorted);
    if (n->key <= *prev) *sorted = false;
    *prev = n->key;
    return c + 1 + verify_lock(n->right, prev, sorted);
}

// 以"先插区间中点"的递归顺序插入 => 树高 O(log n), 避免退化成链表
static void fill_bal(long lo, long hi, const std::function<void(long)>& ins) {
    if (lo > hi) return;
    long mid = lo + (hi - lo) / 2;
    ins(mid);
    fill_bal(lo, mid - 1, ins);
    fill_bal(mid + 1, hi, ins);
}

static LNode* g_coarse_root = nullptr;
static std::atomic<uint64_t> g_ab{0}, g_se{0};

static void body_fine(int tid, long per) {
    long base = (long)tid * per;
    fill_bal(base + 1, base + per - 1, [](long k) { fine_insert(k); });
}
static void body_coarse(int tid, long per) {
    long base = (long)tid * per;
    fill_bal(base + 1, base + per - 1, [](long k) { coarse_insert(g_coarse_root, k); });
}
static void body_stm(int tid, long per) {
    long base = (long)tid * per;
    uint64_t ab = 0, se = 0;
    fill_bal(base + 1, base + per - 1, [&](long k) { stm_insert(k, &ab, &se); });
    g_ab.fetch_add(ab, std::memory_order_relaxed);
    g_se.fetch_add(se, std::memory_order_relaxed);
}

static double timed(const char* name, int T, long M, std::function<void(int,long)> body) {
    auto t0 = std::chrono::steady_clock::now();
    std::vector<std::thread> th;
    for (int i = 0; i < T; ++i) th.emplace_back(body, i, M + 1);
    for (auto& x : th) x.join();
    double sec = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
    printf("%-22s T=%2d : %8.1f ms  %7.1f ns/insert  %7.2f M insert/s\n",
           name, T, sec * 1e3, sec * 1e9 / ((double)T * M), (double)T * M / sec / 1e6);
    return sec;
}

int main(int argc, char** argv) {
    const int T = (argc > 1) ? atoi(argv[1]) : 4;
    const int M = (argc > 2) ? atoi(argv[2]) : 20000;      // 每线程插入数
    const long PER = M + 1;                                // 每线程独占的键区间长度

    root_word.val = 0;
    lock_root = new LNode{0};
    g_coarse_root = new LNode{0};
    { TNode* nn0 = new TNode{0, {}, {}};                   // 三棵树用同一套键: 0 与 T-1 个分界键
      stm::run_tx([&](stm::Tx& tx) { return tx_insert(tx, 0, nn0); }, nullptr, nullptr); }
    for (int t = 1; t < T; ++t) {                          // 预建骨架: T 个互不相交的键区间
        long k = (long)t * PER;
        LNode** p = &g_coarse_root; while (*p) p = (k < (*p)->key) ? &(*p)->left : &(*p)->right;
        *p = new LNode{k};
        fine_insert(k);
        TNode* nn = new TNode{k, {}, {}};
        stm::run_tx([&](stm::Tx& tx) { return tx_insert(tx, k, nn); }, nullptr, nullptr);
    }

    timed("hand-over-hand locks", T, M, body_fine);
    timed("single global lock", T, M, body_coarse);
```

```
    timed("STM (transactions)", T, M, body_stm);
    printf("  STM aborts=%llu fallbacks=%llu\n",
           (unsigned long long)g_ab.load(), (unsigned long long)g_se.load());

    long expect = (long)T * PER, prev = -1; bool sorted = true;
    long n1 = verify_tx((TNode*)root_word.val, &prev, &sorted);
    printf("  STM tree : nodes=%ld (expect %ld) sorted=%s\n", n1, expect, sorted ? "YES" : "NO");
    bool ok1 = (n1 == expect && sorted);
    prev = -1; sorted = true;
    long n2 = verify_lock(g_coarse_root, &prev, &sorted);
    printf("  lock tree: nodes=%ld (expect %ld) sorted=%s\n", n2, expect, sorted ? "YES" : "NO");
    prev = -1; sorted = true;
    long n3 = verify_lock(lock_root, &prev, &sorted);
    printf("  hoh tree : nodes=%ld (expect %ld) sorted=%s\n", n3, expect, sorted ? "YES" : "NO");
    return (ok1 && n2 == expect && n3 == expect) ? 0 : 1;
}
```

【代码做什么？】（`bst.cpp`）

1. **三种树各有自己的根**：事务版（`TNode`，子指针是 `stm::Word`）、手递手锁版（`LNode`，每个节点一个 `std::mutex`）、全局锁版（同样用 `LNode` 但只锁一把全局锁）。
2. **预建骨架**：先单线程插入 `0, PER, 2·PER, …` 这些分界键，使三棵树的拓扑与键集完全一致（也避免了"第一个节点由谁创建"这种一次性争用干扰测量）。
3. **并发阶段**：三个变体分别计时——`body_fine`（手递手锁）、`body_coarse`（全局锁）、`body_stm`（事务，`run_tx` 包裹 `tx_insert`）。
4. **事务体 `tx_insert`**：从根开始**逐级下降**，每一步都 `tx.read` 当前节点的子指针；若子指针为空，就 `tx.write` 把**预分配好的新节点**挂上去。整条路径的祖先节点都进入**读集**，只有最后一个子指针进入**写集**。
5. **校验**：`verify_tx` / `verify_lock` 做中序遍历，检查节点数 = 80004 且键**严格递增**；`stm_insert` 在事务最终失败时直接 `exit(1)`（不允许静默丢更新）。

【并行机制与性能解说】

- **线程与工作分配**：T 个 `std::thread`，键区间静态划分（线程 t 处理第 t 段），负载完全均衡（每线程 20,000 次插入），因此**负载不均不是本实验的变量**——唯一的变量是"同步原语如何对待共享的祖先路径"。
- **硬件上三种实现的差别（这是本讲的核心对照）**：
- **手递手锁**：一次插入要执行 `2h ≈ 34` 次 mutex 操作（h ≈ 17 为树高），其中**根节点与分界键被所有线程反复独占**——这些缓存行在核之间来回弹跳（cache-line ping-pong），且**读共享也被当成冲突**（讲义 slide 22）。
- **全局锁**：一次插入只有 1 次（高度争用的）mutex 操作，下降过程不加锁，因此**比手递手锁更便宜**，但仍被彻底串行化。
- **事务**：一次插入产生 `h ≈ 17` 条读集记录 + 1 条写集记录 + 1 次提交。祖先路径在**两个线程的事务里都是"只读"**，而"读–读"在 TM 里根本不是冲突（讲义 slide 25 的结论），所以事务可以真正并发；代价是每事务的元数据操作数与一次全局版本号抢号。
- **Work / Span / 并行度**：
- 记每次插入的"事务代价" `c_tx = h·c_read + c_write + c_commit`，`h ≈ 17`。以本机实测的单线程 STM 数据反推：`c_tx ≈ 230 ns`。
- **Work**：`W = N · c_tx`，`N = T·M = 4×20,000 = 80,000` 次插入。
- **Span**：
  - 事务版的关键路径是"提交必须串行取版本号"：`Span ≈ N · c_serial`（`c_serial ≈ c_tx/7 ≈ 33 ns`，取 4.1 节分解的比例）→ **并行度上限 ≈ 7**。
  - 锁版（两种）的关键路径是"根节点/全局锁的独占持有"：`Span = N · h · c_lock`（手递手）或 `N · c_mutex`（全局锁），即**整段计算都在关键路径上，并行度 ≈ 1**。
- 这个 `Work/Span` 差（≈7 对 1）就是下面实测结果的来源。
- **实测数据（T = 4，每线程 20,000 次插入；同一进程内顺序跑三个变体，故三者之间的可比性优于跨进程比较）**：

实现（一次代表性运行）	T=1（ns/insert）	T=4（ns/insert）	T=4 吞吐	相对加速（1→4 线程）	结论
手递手细粒度锁	232.4	**2532.7**	0.39 M insert/s	**0.09×（变慢 11 倍）**	细粒度锁**没有**换来并发：共享的是整条祖先路径，锁操作数还放大了 17 倍
单把全局锁	160.8	1119.6	0.89 M insert/s	0.14×	串行化无可避免，但每次插入只有一次锁操作

实现（一次代表性运行）	T=1（ns/insert）	T=4（ns/insert）	T=4 吞吐	相对加速（1→4 线程）	结论
事务（STM）	230.3	**296.0**	**3.38 M insert/s**	0.78×（基本持平）	单线程就比全局锁慢（元数据税），但**几乎不随线程数退化**

- **该次运行中 T=4 时事务比全局锁快 3.8×、比手递手锁快 8.6×**（重复运行受机器负载影响，STM 与全局锁的倍数在约 1.2×–4× 之间波动，但**三者的快慢次序稳定不变**）；并且 `STM aborts=0, fallbacks=0` ——因为写集互不相交、共享的只有读，**没有任何 R–W/W–W 冲突**（与讲义 slide 25 的"NO READ–WRITE or WRITE–WRITE conflicts!"完全对应）。
- **单线程时事务反而比全局锁慢**（230 ns vs 161 ns/insert）：这就是 TM 的"入场费"——每次访问都要仪器化（额外 load、读集/写集插入、提交时验证、抢版本号）。**TM 的卖点不是单线程性能，而是可扩展性。**
- **手递手锁比全局锁更慢**是本实验最反直觉、也最贴合讲义的一张牌：细粒度锁把锁的**数量**从 1 增加到 `2h`，而它想买的"并发"在这里**根本不存在**（根节点是必经之路）。如果换成一个所有线程都写同一个头节点的队列（讲义 slide 28 的 `PushLeft(DQueue*, int)`：每个 `PushLeft` 都要写 `leftSentinel->right`），那么**锁与事务都不会变快**——因为那是真正的 W–W 冲突，语义上必须串行化。**判断"该不该用事务"的正确问法是：读写集的重叠里有多少是"读–读"？**
- **瓶颈与注意事项**：本实验的绝对数值来自一台**共享**机器（load average 约 8–12），重复运行可观察到 2–4 倍的波动（例如 STM 在 T=4 的另一组重复里测到 1087 ns/insert）。**结论性的是相对关系与趋势，而不是具体数字**：重复运行时三个变体的绝对耗时可以在 2–4 倍范围内变化（例如另一组重复运行中三者分别是 1073 / 267 / 222 ns/insert），但**"手递手锁最慢、事务最快"这两个次序结论每次都成立**；而在我们做过的单线程（T=1）测量里，事务始终比全局锁慢（230 ns vs 161 ns/insert，元数据税的代价）。请把上表当作"同一次实验内三种实现的对照"。

3.3 示例 3：硬件事务内存（Intel TSX / RTM）

文件 `tsx.cpp`（需要 `-mrtm`；**在不支持 RTM 的机器上必须先 CPUID 探测**，否则执行 `XBEGIN` 会触发 `#UD` 非法指令异常——本文档实测时真的崩过一次，见第 6 节陷阱 8）

```
// 硬件事务内存 (Intel TSX / RTM) 示例: 事务化转账 + 不变量读取者
// 编译: g++ -O3 -std=c++17 -pthread -mrtm tsx.cpp -o tsx
//   (必须加 -mrtm: RTM 内建函数在未开启该特性时无法内联, 会报 always_inline 错误)
// 说明: 需要 CPU 支持 RTM(Intel Haswell/Broadwell 及后续部分型号)。
//       无 RTM 时 _xbegin() 立即返回失败状态, 本程序自动走互斥锁回退路径。
#include <immintrin.h>
#include <cpuid.h>
#include <cstdio>
#include <cstdlib>
#include <thread>
#include <vector>
#include <atomic>
#include <mutex>
#include <chrono>

struct Account { long balance; };
static Account acct[8];
static std::mutex g_fallback_lock;         // 回退路径: 普通互斥锁保证前进

static std::atomic<uint64_t> n_htm_commit{0}, n_conflict{0}, n_capacity{0},
                             n_explicit{0}, n_other{0}, n_mutex{0}, n_tear{0};

static bool g_has_rtm = false;             // 必须先探测: 无 RTM 时执行 XBEGIN 会产生 #UD 非法指令异常

static bool cpu_has_rtm() {
    unsigned a, b, c, d;
    if (__get_cpuid_max(0, nullptr) < 7) return false;
    __cpuid_count(7, 0, a, b, c, d);
    return (b & (1u << 11)) != 0;          // CPUID.07H:EBX[11] = RTM
}

// 用硬件事务完成一次转账; 冲突/容量溢出时解码状态位并退避重试, 最终回退到锁
static void htm_transfer(int from, int to, long amt) {
    if (!g_has_rtm) {                                  // 无硬件事务支持: 直接走锁路径
        std::lock_guard<std::mutex> g(g_fallback_lock);
        long a = acct[from].balance, b = acct[to].balance;
        if (a >= amt) { acct[from].balance = a - amt; acct[to].balance = b + amt; }
        n_mutex.fetch_add(1, std::memory_order_relaxed);
        return;
    }
    for (int attempt = 0; ; ++attempt) {
        unsigned st = _xbegin();
        if (st == _XBEGIN_STARTED) {
            // ★ 事务体内是"普通"读写: 硬件用缓存记录读集/写集
            long a = acct[from].balance;
            long b = acct[to].balance;
            if (a < amt) { _xabort(1); }            // 显式回滚(应用层拒绝)
            acct[from].balance = a - amt;           // 写集: 只在提交时对其他核可见
            acct[to].balance   = b + amt;
            _xend();                                // 提交: 写集原子生效
            n_htm_commit.fetch_add(1, std::memory_order_relaxed);
            return;
        }
        if (st & _XABORT_CONFLICT)      n_conflict.fetch_add(1, std::memory_order_relaxed);
        else if (st & _XABORT_CAPACITY) n_capacity.fetch_add(1, std::memory_order_relaxed);
        else if (st & _XABORT_EXPLICIT) n_explicit.fetch_add(1, std::memory_order_relaxed);
        else                            n_other.fetch_add(1, std::memory_order_relaxed);
        if (attempt >= 8) {                         // 回退: 用锁串行完成
            std::lock_guard<std::mutex> g(g_fallback_lock);
            long a = acct[from].balance, b = acct[to].balance;
            if (a >= amt) { acct[from].balance = a - amt; acct[to].balance = b + amt; }
            n_mutex.fetch_add(1, std::memory_order_relaxed);
            return;
        }
        for (volatile int i = 0; i < (1 << attempt) * 64; ++i) { }   // 指数退避
    }
}

// 读取者: 在同一瞬间观察两个账户, 校验总额不变(检验隔离性)
static void reader(int rounds, const long* expected_total, std::atomic<bool>* stop) {
    while (!*stop) {
```

```
        if (!g_has_rtm) {
            long s;
            { std::lock_guard<std::mutex> g(g_fallback_lock);
              s = 0; for (int i = 0; i < 8; ++i) s += acct[i].balance; }
            if (s != *expected_total) n_tear.fetch_add(1, std::memory_order_relaxed);
            continue;
        }
        unsigned st = _xbegin();
        if (st == _XBEGIN_STARTED) {
            long s = 0; for (int i = 0; i < 8; ++i) s += acct[i].balance;   // 8 个字必须"同一瞬间"
            _xend();
            if (s != *expected_total) n_tear.fetch_add(1, std::memory_order_relaxed);
        } else {
            long s;
            { std::lock_guard<std::mutex> g(g_fallback_lock);
              s = 0; for (int i = 0; i < 8; ++i) s += acct[i].balance; }
            if (s != *expected_total) n_tear.fetch_add(1, std::memory_order_relaxed);
        }
        (void)rounds;
    }
}

int main(int argc, char** argv) {
    const int T = (argc > 1) ? atoi(argv[1]) : 4;
    const int ITERS = (argc > 2) ? atoi(argv[2]) : 200000;
    g_has_rtm = cpu_has_rtm();
    printf("CPU supports RTM: %s\n", g_has_rtm ? "YES" : "NO (将全部走互斥锁回退路径)");
    for (int i = 0; i < 8; ++i) acct[i].balance = 1000000;
    const long expected = 8L * 1000000;        // 8 个账户总额守恒

    std::atomic<bool> stop{false};
    auto t0 = std::chrono::steady_clock::now();
    std::thread rd(reader, 0, &expected, &stop);
    std::vector<std::thread> th;
    for (int i = 0; i < T; ++i) th.emplace_back([&, i] {
        uint32_t seed = 99u + i * 7919u;
        for (int k = 0; k < ITERS; ++k) {
            seed = seed * 1664525u + 1013904223u;
            int a = (int)((seed >> 8) % 8), b = (int)((seed >> 16) % 8);
            if (a == b) b = (b + 1) % 8;
            htm_transfer(a, b, 1 + (long)((seed >> 24) % 50));
        }
    });
    for (auto& x : th) x.join();
    stop.store(true); rd.join();
    double sec = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();

    long total = 0; for (int i = 0; i < 8; ++i) total += acct[i].balance;
    printf("  time            = %.3f s  (%.1f ns/transfer)\n", sec, sec * 1e9 / ((double)T * ITERS));
    printf("  htm commits     = %llu\n", (unsigned long long)n_htm_commit.load());
    printf("  aborts: conflict=%llu capacity=%llu explicit=%llu other=%llu\n",
           (unsigned long long)n_conflict.load(), (unsigned long long)n_capacity.load(),
           (unsigned long long)n_explicit.load(), (unsigned long long)n_other.load());
    printf("  mutex fallbacks = %llu\n", (unsigned long long)n_mutex.load());
    printf("  torn reads      = %llu  (必须为 0)\n", (unsigned long long)n_tear.load());
    printf("  INVARIANT total = %ld (expected %ld) => %s\n", total, 8000000L,
           total == 8000000L ? "OK" : "*** CORRUPTED ***");
    return (total == 8000000L && n_tear.load() == 0) ? 0 : 1;
}
```

【代码做什么？】（`tsx.cpp`）

1. **CPUID 探测**：读 `CPUID.07H:EBX[11]`（RTM 支持位）。**不支持就完全不执行 `_xbegin()`**，直接走互斥锁路径。
2. **事务化转账 `htm_transfer`**：`_xbegin()` 成功后，事务体内是**完全普通的读写**（`acct[from].balance` 等），由**硬件**用缓存记录读集/写集；两笔修改完成后 `_xend()` 提交——**两个账户的修改一次性对外可见**（隔离性由一致性协议保证）。余额不足时用 `_xabort(1)` **显式中止**。
3. **失败路径**：解码返回状态位——`_XABORT_CONFLICT`（与他人事务的读写集冲突）、`_XABORT_CAPACITY`（读写集放不进缓存）、`_XABORT_EXPLICIT`（显式中止）、其他；然后**指数退避重试**，重试 8 次仍失败则**回退到互斥锁**完成这次转账（RTM 不提供前进保证，必须有回退路径）。

4. **读取者线程**：在"一个瞬间"读 8 个账户并求和，与初始总额比较；不一致就记一次**撕裂读（torn read）**。这是对**隔离性**的直接检验——没有原子性/隔离性时，"读 8 个字"几乎不可能看到一致的快照。
5. **输出**：RTM 是否可用、HTM 提交次数、四类 abort 计数、锁回退次数、撕裂读次数（必须为 0）、总额不变量。

【并行机制与性能解说】

- **硬件上发生了什么**：进入事务后，被读的行在私有缓存里以 `Shared` + "读集标记"保留，被写的行以 `Modified` + "写集标记"保留（**新值停留在本核缓存，不写回内存，也不让其他核取走**——这就是隔离性）。其他核的读/写请求命中我的读写集时，一致性协议即报冲突，通常由**请求方或写方** abort。提交时让所有被标记的 `Modified` 行"转正"，从而**一次性**对外可见。
- **实测（本机 2 路 AMD EPYC 7V13）**：`CPU supports RTM: NO` ——本机**没有** RTM，因此所有 200,000 次转账都走了**互斥锁回退路径**（`mutex fallbacks = 200000`，`htm commits = 0`），实测 106.9–348.4 ns/transfer，`torn reads = 0`，总额不变量成立。**这份输出本身就是结论的一部分**：它证明了"没有 HTM 时程序必须、也能够在运行时正确地退化为锁实现"——这是所有 HTM 程序的标准形态（Intel 自己的库、GCC 的 `-mrtm` 代码都这么做）。
- **Work / Span / 并行度**：本程序**恰好不能**用 Work/Span 来说明扩展性，因为在这台机器上它退化成了锁版本——**`Span = Θ(N · c_mutex)`，即整段计算都在关键路径上，并行度 ≈ 1**（实测 4 线程 200,000 次转账 0.070 s、8 线程 400,000 次 0.095 s，吞吐几乎没有提升，正是"全局串行"的特征）。在真有 RTM 的机器上，事务路径的 `Span` 变成 `Θ(N · c_commit_backend)`（提交时的发布是硬件完成的多行原子操作，没有全局版本时钟），因此并行度可以远高于 1——这正是 HTM 相对 STM 的核心优势（见 2.6 的表）。
- **注意**：`_xbegin()` / `_xend()` 之间的代码**不能包含会"卡住"的操作**——系统调用、I/O、页错误、上下文切换都会导致事务中止（甚至直接回退），且**中止时代码的副作用无法撤销**（例如你已经 `printf` 了）。所以 HTM 事务体必须"纯净、短小、可重复执行"。

---

4. 性能模型与复杂度分析

本节把前两节的机制变成可以计算的量。所有公式都在"**事务的全部内存访问都必须通过事务接口**"这一前提下成立（这也是本文档 3.1 节 STM 的使用契约）。

4.1 单事务代价分解（用实测值反推各分量）

假设：标称 3.0 GHz（本机最高 3.7 GHz），事务体含 `r` 次字读与 `w` 次字写。则

```
c_tx = c_begin  +  r · c_read  +  w · c_write  +  c_commit
       (取时钟快照)  (仪器化读)    (仪器化写)     (验证读集 + 抢版本号 + 发布写集)
```

用 3.1 节实测的**单线程 133.6 ns/事务（≈400 周期 @3.0 GHz）**、`r = 3`、`w = 2` 反推各分量：

阶段	具体操作	估计周期	备注
`begin`	`g_clock.load(acquire)`	≈ 40	该行被所有核共享、且每次提交都被写坏 → 常常要重新取
3 × `c_read`	版本 load → 值 load → 版本 load，再 push 读集	≈ 90（30/次）	两次元数据 load + 一次数据 load，全是 L1 命中才有这个数
2 × `c_write`	版本 load + CAS（抢锁位）+ push 写缓冲	≈ 120（60/次）	CAS 是 RMW，比普通 store 贵得多
`commit`	3 次读集验证 + 1 次 `fetch_add` + 2 次发布	≈ 150	**`fetch_add` 是全部事务共享的串行点**
合计		**≈ 400**	与实测 133.6 ns/事务 吻合（这是"反推"而非独立测量，用于说明结构占比）

关键观察：`commit` 占 37.5%，其中绝大部分是那个**全局原子取号**。这与"锁"的代价结构惊人地相似——**TM 并没有消灭同步，它只是把同步从"用户数据结构"搬到了"运行时元数据"里**。

4.2 冲突概率与重试成本（含一个与实测吻合的算例）

设 `P` 个线程各持有一个进行中的事务，每个事务写 `w` 个共享字，共享字总数为 `N`，且事务在时间上完全重叠（最坏情形），则

```
p_conflict  ≈  (P − 1) · w / N          (上界：任意另一个事务的写集与我相交)
期望代价     =  c_tx / (1 − p)          (每次中止都白干一遍活儿，然后重来)
```

数值算例（与 3.1 节实测对照，`P = 4`，`w = 2`）：

共享字数 N	模型 `p ≈ (P−1)·w/N`	实测中止率	说明
64	3 × 2/64 = **9.4%**	**9.1%**	吻合（稀疏冲突区间模型很准）
1024	3 × 2/1024 = **0.59%**	**0.7%**	吻合
8	3 × 2/8 = **75%**	38.6%	模型是**上界**：极端争用下争用管理器把大量冲突变成"短暂停滞"而非中止
2	3 × 2/2 = 300%（饱和）	17.6%	饱和区：几乎全是停滞等待，吞吐被串行化拖死

重试开销的即时换算：`c_tx = 100 ns` 时，

中止率 p	10%	30%	50%	90%
期望代价 `c_tx/(1−p)`	111 ns	**143 ns**	200 ns	1000 ns
浪费的工作量占比	10%	30%	50%	90%

也就是说，**在中止率 30% 时，80 万次"有用的"事务实际要执行 ≈ 114 万次事务的工作量**——这正是高争用下 TM 吞吐崩塌的算术原因（对应 3.1 节 N=2/N=8 的实测）。

4.3 三种不同的天花板：带宽、延迟、串行化

（a）带宽天花板（题面要求的基准算例）：一个 256 MB 的数组，机器可持续带宽 20 GB/s，则

```
单次遍历的最少时间 = 256 MB / 20 GB/s = 0.256 GB / 20 GB/s = 12.8 ms
（如果每元素还要读+写各一遍，就是 2×: 25.6 ms）
```

把它套到 3.2 节的树上：80,004 个节点、每节点 32 B ≈ **2.56 MB**（能放进 L3，但每个节点还带 16 B×2 的元数据），每次插入要触及 `h ≈ 17` 个不同节点：

```
每次插入至少触及 17 个缓存行 = 17 × 64 B = 1088 B
80,000 次插入 × 1088 B = 87 MB 的缓存行搬运量
若这些行全部从 DRAM 取: 87 MB / 20 GB/s = 4.35 ms
实测 STM 用时 23.7 ms  =>  带宽只能解释 4.35/23.7 ≈ 18%
```

结论：这个负载不是带宽瓶颈（树的上层长期驻留缓存）。真正花时间的是**元数据操作数、指针追逐的延迟链、以及提交的串行化。不要一看到并行程序就把问题归给"内存带宽"**——这是本讲最容易犯的分析错误。

（b）延迟天花板：树的插入是**指针追逐（pointer chasing）**：必须读完父节点才知道子节点在哪，`h ≈ 17` 次依赖串行。若每次 L3 命中 40 ns，纯延迟链就是 680 ns；实测 296 ns/insert 说明大部分访问命中 L1/L2（上层共享节点一直在缓存里）。**TM 的元数据访问也在这条依赖链上**（要先读版本才敢读值），所以"每次访问多两次 load"在延迟敏感代码里被放大。

（c）串行化天花板（Amdahl 形式）：把每个事务"必须在全局串行段里执行"的比例记为 `f`，则

```
f = c_commit_serial / c_tx ≈ 150 / 400 = 0.375
最大加速比 = 1 / f = 2.7×          （与核数无关！）
```

实测最好的一组是 `T=1: 133.6 ns/tx → T=4: 76.7 ns/tx ≈ 1.74×`，与 2.7× 的上限同量级（差额来自跨 NUMA 的时钟行弹跳与验证时的缓存未命中）。**这就是软件 TM 的天花板：它由一个共享元数据字（全局版本时钟）决定，而不是由你有多少核决定。** 这也是硬件 TM 在讲义 slide 27 的性能图里能压过 fine locks 的结构性原因：**HTM 没有全局版本时钟**（版本信息寄生在缓存行状态上，冲突检测随一致性流量批量完成）。

4.4 算术强度 / Roofline 视角：TM 是"元数据密集型"负载

对树的插入做算术强度估计（把"字节"当作有用输出）：

```
有用数据: 1 个 long 键 + 1 个指针写        ≈ 16 B
为完成它移动的缓存行(最坏情况全部 miss): 17 × 64 B = 1088 B
算术强度 AI ≈ 16 B / 1088 B ≈ 0.015 B/B   （完全没有计算，纯数据搬移）
```

在 Roofline 图上，这类负载远远贴在"内存墙"一侧：**提升性能的方向不是提高 FLOPs，而是减少每次事务触及的字节数与串行点**。而 TM 在这个本已很低的强度上又叠加了两层乘数：

1. **元数据字节数**：本实现的 `stm::Word` 是 **16 B**（8 B 版本字 + 8 B 数据），也就是**每 8 B 有用数据配 8 B 元数据（100% 空间开销）**，而且**元数据与数据同行**（4 个字 = 64 B 一行）→ 两个逻辑上无关的字若落在同一行，它们的**版本字会互相失效**，产生与"伪共享"完全同源的额外一

致性流量。

2. **元数据串行点**：全局版本时钟（4.1/4.3 的天花板）。

这两层解释了 3.2 节的现象：**STM 单线程比全局锁慢（元数据税），但在 4 线程下反超 3.8×**——因为它把"锁的争用"（随 P 增长）换成了"元数据的争用"（大部分与 P 无关，只有时钟那一行随 P 恶化）。

4.5 小结：三元设计空间如何影响上面每一个公式

设计选择	影响 `c_read` / `c_write`	影响 `c_commit`	影响 `c_abort`	影响 `p`（伪冲突）
eager（undo log）	写更贵（记旧值 + 直接改内存）	**更便宜**（数据已在内存）	**更贵**（逐个恢复旧值）	无影响
lazy（写缓冲）	写更便宜（只进缓冲）	**更贵**（冲刷写缓冲）	**更便宜**（丢缓冲 + 解锁）	无影响
pessimistic 检测	读/写路径变长（每条访问都检查）	变便宜（提交时不必再验证）	早中止 → 白干的活少	可能把 abort 变成 stall
optimistic 检测	路径短	变贵（要验证整个读集）	晚中止 → 白干的活多	有前进保证，但仍可能不公平
字粒度	元数据访问多	验证条目多	—	**低**（只有真同一个字才冲突）
缓存行粒度（HTM）	几乎零额外指令	由硬件完成，快	容量/冲突 abort	**高**（同行不同字也算冲突）

---

5. 关键要点

1. **`atomic { }` 与 `lock()/unlock()` 不是同一种东西，也不能互相替换。** 前者是**声明**"这段必须原子（且失败时整体回滚）"，实现方式由系统决定；后者只是一个**互斥原语**，本身不提供原子性与隔离性，还被大量用于"等待条件""限制并发度"等与原子性无关的用途（讲义 slide 35 的 self-check）。把 `synchronized` 机械替换成 `atomic` 会导致**它永远等不到**（隔离性使另一线程的写不可见）或者**原子性违规**（本该原子的一段被切成两个原子块，slide 36–37）。
2. **TM 的实现就是回答三个问题：数据版本（eager/lazy）、冲突检测（悲观/乐观）、检测粒度（字/行/对象）。** 讲义 slide 3 的 "What you should know" 就是这三条；eager 与 lazy 的取舍可以用一句话记住——**eager 赌事务不会中止（提交快、回滚贵），lazy 赌事务会中止（回滚快、提交贵）**，本文档 3.1 节的实测给出了这张赌注的赔率（低争用时 eager 略快，高争用时 lazy 明显快）。
3. **事务不消灭同步，只是把同步的位置从"用户数据结构"搬到"运行时元数据"。** 软件开发者的直觉常常是"用了事务就没有锁的争用了"——错。本文档 STM 的实测表明：全局版本时钟（一行缓存）把 4 线程的加速比压在 1.74×（模型上限 2.7×）；这也正是硬件 TM（没有全局版本时钟）在讲义 slide 27 的性能图中能胜过细粒度锁的结构性原因。**评估任何 TM 实现，第一件事是找到它的串行点。**
4. **"该不该用事务"的判断标准是读写集的语义重叠，而不是"这段代码看起来复杂"。** 如果重叠里主要是**读–读**，事务能拿到锁拿不到的并发（3.2 节的树：`aborts=0`，在该次运行中比全局锁快 3.8×）；如果存在**真正的 W–W** 冲突（所有线程都改同一个队列头），事务和锁一样要串行化，谁也不会更快。**粒度（字 vs 缓存行）决定了"伪冲突"的数量**：HTM 的行粒度会把同一行里两个无关的字当成冲突，与伪共享同源。
5. **事务只覆盖内存，且必须回答"前进保证"的问题。** I/O、`malloc`、系统调用、页错误都在事务之外（不可回滚），必须移出事务体或设计好补偿逻辑；而乐观并发天然可能活锁，**必须由争用管理器（有界停滞/指数退避）加一条串行化回退路径**来兜底——本文档的 STM 与 `tsx.cpp` 的锁回退都是这一条的具体实现。

---

6. 常见陷阱与注意事项

- **陷阱 1：惰性版本下忘了"读己之所写"。** 写缓冲（write buffer）持有新值、内存里还是旧值，因此**读必须先查写缓冲，再查内存**。本文档实现的第一版把"已持有该字"的判断放在最前面并直接返回 `w->val`（内存里的旧值），在 `transfer` 里表现为"取款后重新读余额得到旧余额"。这类 bug 在"读改写同一个字"的代码里（几乎所有计数器、累加器）都会出现。修复后本文档用 `ryow_bad` 计数器长期守住这一点。
- **陷阱 2：回滚路径不完整（漏掉任何一条出口都是灾难）。** 事务在**每一条退出路径**上都必须释放写集的所有权/撤销已做的写，包括**应用层显式回滚**（如"余额不足"）这一条。本文档实测中曾出现过：应用层 `abort` 分支直接 `return false` 而没有调用 `tx.abort()` → **那些字上的锁位永远保持为"占用中"**，此后任何碰到它的事务都只能一直失败（表现为**回退路径里的重试全部因"该字被独占"而失败**；调试该缺陷时实测到连续 10,000 次尝试没有一次成功）。**这类 bug 不会破坏数据，只会让系统永久性地"变慢直到卡死"**，比数据竞争更难发现。
- **陷阱 3：冲突后立即狂重试 → 活锁（livelock）。** 没有退避、也没有争用管理器的重试循环会让多个线程以相同节奏互相撞车。本文档实测中同样出现过"重试循环里连续 10,000 次尝试全部撞在别人持有的字上"，直到加入 (a) **有界停滞**（未持有写集时可以等待持有者释放在讲义 slide 47 里是 Case 2 的 "early detect (and stall)"）与 (b) **排空式全局串行回退**（进入串行模式 → 等在途事务排空 → 独占执行一次）才彻底消除。**顺带一个设计要点：只在"本事务还没有持有任何写集"时才允许等待，否则两个各持一把锁又互相等待的事务会形成环形等待而死锁。**

- **陷阱 4：在事务里等另一个线程（或另一个事务）。** 讲义 slide 36 的 `flagA/flagB` 例子就是标准反例：事务内的自旋等待，等的是另一个事务的**写**，而在提交之前那个写**永远不可见**（隔离性）→ 永不到头。事务不能替代条件变量/屏障/原子标志，**"等待"必须在事务之外（或者根本不使用事务）**。
- **陷阱 5：把本该原子的一段切成两个原子块（atomicity violation）。** 讲义 slide 37：`atomic { ptr = A; }` 与 `atomic { B = ptr->field; }` 之间，另一线程可以执行 `atomic { ptr = NULL; }`。事务保证的是"**你声明的那一段**"原子，**声明错了它救不了你**——`atomic` 消除数据竞争，但不消除原子性违规。
- **陷阱 6：混淆粒度带来的"伪冲突"与真正的冲突，或者把元数据与数据放在同一缓存行。** HTM 的检测粒度是缓存行：两个事务分别写同一行的不同字节，也会被判定为冲突（`W-W` 伪冲突）。软件 TM 也不能免疫：本文档每个字带 16 B 元数据（版本字），**4 个字共享一个 64 B 缓存行**，于是两个逻辑无关的账户也会让对方的版本字失效，产生与"伪共享（false sharing）"同源的一致性流量。**把热点元数据按缓存行隔离（padding）或与数据分离布局，是 STM 优化中最直接有效的一步。**
- **陷阱 7：混用事务访问与非事务访问。** 只要还有一条路径绕过事务直接读写共享数据，隔离性就无从谈起（本文档 STM 的正确性前提就是"所有共享访问都通过 `tx.read/tx.write`"）。**这条约束在 HTM 上更严格**：事务外的普通访问可能让别的事务永远 abort（RTM 不保证前进）。同理，事务内的分配（`malloc`）、I/O、`printf` 都不受事务保护——本文档 `bst.cpp` 里把 `new TNode` 放在事务之外，就是为了这个原因。
- **陷阱 8（环境陷阱）：把"HTM 可用"当成默认前提。** 在不支持 RTM 的处理器上执行 `XBEGIN` 会触发 `#UD` 非法指令异常——本文档的 `tsx.cpp` 第一次运行就是**直接被内核 SIGILL 杀掉**。必须先用 CPUID 探测，再决定走事务路径还是锁路径；也不要假设"探测一次就永远有效"（历史上部分型号通过微码更新禁用了 TSX）。

---

7. 思考题（带答案）

思考题 1：为什么"事务化树更新"能赢过手递手锁，而"事务化的链表头插入"却赢不了？

【答案】 区别在**读写集重叠的语义类型**，而不是"事务"这个工具本身。

- **树更新（讲义 slide 23–25，本文 3.2 节实测）**：线程 A 的事务 `READ{1,2,3} WRITE{3}`、线程 B 的 `READ{1,2,4} WRITE{4}`。重叠部分（节点 1、2）**只有读**。事务只需要在提交时验证"我读过的版本没被换过"，读–读不会使版本变化，所以 **`aborts = 0`**，两个事务并发提交。手递手锁则必须对路径上的**每一个**节点取**独占**锁——即使它只是想读一下指针——于是共享祖先成了串行点；更糟的是它每次插入要执行 `2h ≈ 34` 次锁操作（h≈17），实测 T=4 时比"一把全局锁"还慢（2532 ns vs 1119 ns/insert），而事务只有 296 ns/insert。
- **链表头插入（讲义 slide 28 的 `PushLeft(DQueue *q, int val)`）**：每一次 `PushLeft` 都要执行 `leftSentinel->right = qn;` `oldLeftNode->left = qn;`，即**所有线程都在写同一个字** `leftSentinel->right`。这是货真价实的 **W–W 冲突**：语义上必须串行化（否则会丢掉一次插入）。事务只能把这个串行化做得更快（少了锁的获取/释放，直接靠版本检测），**但不可能让它并行**。所以两者都会退化成"队列头吞吐"，上限约为 `1/c_commit`（事务）或 `1/c_lock`（锁）。
- **推论（判断准则）**：用事务之前先画读写集。**重叠里"读–读"占比高 → 事务收益大；重叠里有真写的冲突 → 事务只能改善常数因子**；而后者的正确解法通常是**改变数据结构**（分片队列、细化的头部、combining tree），不是换同步原语。

思考题 2：给定中止率，选 eager 还是 lazy？请算出一个具体的交叉点。

【答案】 设事务"主体"（读/写/业务逻辑）需要 `B` 个周期，且每次中止后必须重试一遍。取一组可辩护的参数：`B = 100`，eager 的 `c_commit = 30, c_abort = 120`（提交便宜但回滚要按 undo log 逐个恢复旧值），lazy 的 `c_commit = 60, c_abort = 40`（提交要把写缓冲冲刷出去，回滚只需丢弃缓冲并解锁）。设每次尝试中止的概率为 `p`，则**每个成功事务的期望代价**为

```
E = B + c_commit + (p/(1−p)) · (B + c_abort)
```

- **p = 0.1（低争用）**：eager `= 100+30+0.111×220 = 154.4`；lazy `= 100+60+0.111×140 = 175.6` → **eager 快 12%**（与 3.1 节 N=1024 的实测一致：76.7 ns(lazy) vs 73.0 ns(eager)）。
- **p = 0.5（高争用）**：eager `= 130 + 1.0×220 = 350`；lazy `= 160 + 1.0×140 = 300` → **lazy 快 17%**（与实测的 N=2/N=8 一致：lazy 169/176 ns vs eager 205/202 ns）。
- **交叉点**：令 `130 + (p/(1−p))·220 = 160 + (p/(1−p))·140`，得 `(p/(1−p))·80 = 30`，即 `p/(1−p) = 0.375`，**p ≈ 27%**。

结论：中止率低于约 27% 时选 eager（undo log），高于 27% 时选 lazy（写缓冲）；而真实系统通常做成混合（例如 HTM 天然是 lazy：新值留在缓存里，提交即"转正"），因为**中止率在工作负载之间差异巨大，而切换策略的代价又很高**。另外提醒：`p` 本身不是外生变量——它由数据结构的争用结构决定，也受争用管理器影响（把 abort 变成 stall 会降低 `p`，但增加等待时间，**中止率低不等于性能好**，见 3.1 节 N=2 的实测）。

思考题 3：一个事务要读/写 4096 个随机分布的 4 字节元素。用 HTM（L1D = 32 KB、8 路组相联、64 B 行）跑会发生什么？给出定量判断，并说明字粒度 STM 与行粒度 HTM 在这题上的差别。

【答案】 – **容量（capacity）判断**：L1D 32 KB / 64 B = **512 个缓存行容量**（8 路 × 64 组）。事务的足迹 = 读集行数 + 写集行数。一个 64 B 行能放 16 个 4 字节元素，所以随机取点会分散到很多行：若这 4096 个元素分布在一个 256 KB 的区域里（= 4096 行），则**被触及的行数约为** `4096 × (1 − e^(−4096/4096)) ≈ 4096 × 0.63 ≈ 2590` **行**。于是足迹 ≈ **2590 行 ≫ 512 行** → 事务**必然发生 capacity abort**，重试也永远不会成功（每次重试的足迹一样大）→ **必须回退到软件路径**。这是 HTM 的硬限制：**事务的读写集必须"装得进"缓存**（实践中安全的规模是几十到几百行，还要给其他线程的流量留余量）。 – **同理可见的行粒度陷阱**：即使把规模缩小到"每事务只碰 16 个随机元素"（约占 16 行，远小于 512），两个并发事务**逻辑上完全不相交**（不同的元素）时，仍可能落在**同一行**而互相冲突：数组有 1024 行时，两个事务各随机取 16 行，至少共享一行的概率约为 `text 1 − (1 − 16/1024)^16 ≈ 1 − 0.9844^16 ≈ 22%` 也就是说，**约五分之一的事务对会因为"行共享"而被判冲突，尽管它们从未碰过同一个元素**——这就是"行粒度伪冲突"，与伪共享（false sharing）同源。缓解放法：让事务的数据**对齐到缓存行并避免跨事务共享行**（padding / 分段）。 – **字粒度 STM 的对照**：本文档的 STM 按**字**（每个字有自己的版本字）判定冲突，因此上面那 22% 的"伪冲突"在**正确性判断上**根本不会发生；但它付出的代价是 (a) **16 B 元数据 / 8 B 数据**的空间开销，以及 (b) **元数据与数据同行导致的缓存行互相失效**（4 个字共享一行）——也就是说，**字粒度并没有消除"行"层面的物理争用，只是把"逻辑冲突"与"物理抖动"分开了**：前者避免了，后者仍在，只能靠布局（把版本字集中或按行隔离）去缓解。 – **一句话总结**：**粒度是"正确性判定"与"物理代价"之间的两端**——粒度越细，逻辑判定越精确（伪冲突越少），但元数据越贵；粒度越粗（缓存行），元数据几乎免费，但伪冲突与容量上限随之而来。HTM 选了后者，STM 选了前者，这就是 2.6 节那张对比表背后的全部算术。

Lecture 18: Heterogeneous Parallelism, Hardware Specialization

1. 章节标题与概述

Lecture 18: Heterogeneous Parallelism, Hardware Specialization（异构并行与硬件专用化）

- **本讲核心问题**：**当"把芯片做快"这件事在功耗墙上撞死之后，多出来的晶体管该拿去做什么？** 前半讲从一条最小的观察出发——一个优化良好的并行实现能比单线程 `gcc -O3` 的 C 代码快约 44 倍——追问"如果让你买一台新机器，你会选 4 个各性能为 P 的核，还是 16 个各性能为 P/2 的核"。答案取决于程序里可并行部分的比例，而这正是 **Hill & Marty 的 "Amdahl's Law in the Multicore Era"** 要形式化的问题。后半讲把问题推到极致：**Dennard 缩放（等比例功耗缩放）在 ~2003 年终结**，芯片功耗必须恒定，于是每一代新晶体管里有约 30% 根本"点不亮"（**暗硅 dark silicon**）——结论是**必须用专用化来换能效**：从延迟优化的通用核，到吞吐优化的 GPU 核（~10× perf/W），到固定功能 ASIC（~100–1000× perf/W），再到 FPGA 与粗粒度可重构阵列（CGRA）。
- **涉及的主要硬件/软件机制**：硬件侧包括**同构多核 / 非对称（asymmetric）异构多核 / 大小核（big.LITTLE、Alder Lake）/ SoC 上的固定功能单元（视频编解码、Neural Engine、ISP、加密）/ 可编程 DSP（Qualcomm Hexagon，VLIW）/ FPGA（LUT6、硬核 DSP 与 BRAM）/ ASIC（Google TPU、Anton 的粒子相互作用单元）/ 脉动阵列（systolic array）与张量核（tensor core）**；软件侧包括**主机 + 加速器的异步任务模型（LD/ST/AO 重叠、TMA 张量搬运、TMEM、mbarrier）、CUDA 的 tiled-tensor 编程模型（CUTLASS / Triton / Thunderkittens）、数据流图与算子融合（FlashAttention、MetaPipeline）以及领域专用语言（DSL）**。
- **在并行计算知识体系中的角色**：本讲是整门课的**"收束与转向"**。前面十几讲把同构多核/GPU 上的**正确性**（一致性、同步、无锁）与**性能**（work–span、带宽、局部性、Roofline）讲透，其隐含前提始终是"所有处理器都能跑所有任务，只要让所有处理器一直忙"。本讲打破这个前提：**目标是"把每个任务放到最合适的那类单元上"，而不是"让所有单元都忙"**。它把能效（perf/Watt）提升为与性能并列的第一等设计目标，并把**调度与算法分解的难题推给程序员**——这与课程后续关于领域专用加速器（DNN 训练/推理、图计算）以及系统设计的讨论直接衔接。
- **配套材料**：
- **CMU 15–418/618 Fall 2026 本讲讲义：已公开（可公开下载）。** Fall 2026 日程表 https://www.cs.cmu.edu/~418/schedule.html 中 Lecture 18 一行的标题为 "Heterogeneous Parallelism, Hardware Specialization"，日期为 **Oct 7**，其 slides 链接指向两份 PDF：`lectures/20_heterogeneity.pdf`（44 页，"Heterogeneous Parallelism and Hardware Specialization"）与 `lectures/20_specialization_csd.pdf`（53 页，"The inexorable rise of hardware specialization"），均位于公开目录 https://www.cs.cmu.edu/~418/lectures/ 之下。两点说明：①这两份 PDF 的**文件名编号是 20**（沿用历史学期的讲次编号），首页写有 "CMU 15–418/15–618, Spring 2025" 或页脚标注 "15–418/618 Fall '25"，这是**讲义沿用**的正常现象，不视为错误；②第二份 PDF 的作者页署名为 **Nathan Beckmann, CMU**（来源为 CSD faculty meeting Fall '21 的报告，被 15–418/618 复用为补充讲义）。
- **已公开的姊妹课程（Stanford CS149）讲义抽取文本：已公开。** `cs149_supp/accelerators.txt`（Stanford CS149 Fall 2025 Lecture 10 "Hardware Specialization"，71 页）。它是本讲**内容最完整的公开文本来源**，覆盖了能效约束、专用化的量级、指令流开销分解、DSP/FPGA/ASIC/DSP、H100 SM 结构、张量核、TMA、脉动阵列、数据流架构与 kernel fusion。

- **本笔记的事实基础**：`extracted/20_heterogeneity.txt`（44 页全文抽取）+ `extracted/20_specialization_csd.txt`（53 页全文抽取）+ `cs149_supp/accelerators.txt`（71 页全文抽取）。讲义中**没有给出可抽取数值的图表**（如 Chung et al. MICRO 2010 的能效–面积曲线、Hill & Marty 的加速比曲线、Hameed et al. 的能耗堆叠柱状图）本笔记只沿用其**定性结论与坐标轴含义**；本笔记中所有带具体数字的算例都显式标注为"**按讲义给定参数推导**"，不冒充讲义原文数据。
- **未发布 / 需登录**：Fall 2026 日程表中 Lecture 18 一行的 slides/video 链接**被 HTML 注释隐藏**（注释原文为 "slides/video from a previous offering; uncomment when posted for Fall 2026"），因此本讲的**录像（Panopto/YouTube）属未发布**；**Ed 讨论区、Autolab、Canvas 均需登录**；历史学期位于 `/afs/cs/academic/class/15418-*/public/` 之下的讲义（如 Performance Analysis/Profiling、Transactional Memory、AI in System Design 等）需要 CMU 登录，**属未公开**。
- **课程语境**：Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。前一讲为 Lecture 17 "Transactional Memory"（Oct 5，Fall 2026 尚未发布讲义），后一讲为 Lecture 19 "Virtual Memory"（Oct 9，对应公开 PDF `13_virtualmemory.pdf`）。本日日程另标注 "Assignment 3 due, Assignment 4 out"。

---

2. 核心概念与硬件/软件架构图解

2.1 起点：从 44× 到"该买哪种机器"

定义与目的：本讲以一个朴素但尖锐的对比开场。讲义指出，在 Assignment 1 里我们观察到：**一个优化良好的、计算密集（compute-bound）的并行实现，比同一处理器上用 `gcc -O3` 编译的单线程 C 代码快约 44 倍**。这个数字说明两件事：①并行化本身的收益是巨大的；②收益**完全取决于程序的可并行比例**。于是就有了本讲的第一个思想实验：

> 你要买一台新计算机系统。 **处理器 A**：4 个核，每核顺序性能为 P。 **处理器 B**：16 个核，每核顺序性能为 P/2。 系统其它部件完全相同。你选哪个?

直观解释（"它是什么？"）：把它想成**搬砖**。 – 处理器 A = **4 个壮汉**，每人一次能抱 2 摞砖； – 处理器 B = **16 个普通人**，每人一次抱 1 摞砖。总"人力"（芯片晶体管预算）完全一样。如果活儿能被平均切成 16 份（比如"把 1600 摞砖全搬到 100 米外"），16 个人赢；但如果这活儿里有一段**只有一个人能干**（比如"先由一个人决定砖该怎么码"），那 A 就用它的壮汉把这段快速干完，再去加入搬砖大军——而 B 只能让一个普通人在那儿慢慢决定，其余 15 人干等。**这就是异构/非对称设计的全部动机**：宁可牺牲一部分"并行吞吐资源"，也要保住"串行段的执行速度"。

数学形式（Hill & Marty 模型，讲义给出的资源受限 Amdahl 定律）：设

- `f` = 程序中**可并行**的比例；
- `n` = 总处理资源（例如芯片上的晶体管总量）；
- `r` = 分配给**每个核**的资源量，于是有 `n/r` 个核，每个核的顺序性能为 `perf(r)`。

讲义举例 `n = 16`：处理器 A 取 `r_A = 4`（4 个大核），处理器 B 取 `r_B = 1`（16 个小核），并把 `perf(r)` 建模为 `√r`（面积/资源翻倍，性能只提升 √2——这与后面 2.2 节的技术缩放模型完全一致），归一化 `perf(1) = 1`。

- **对称多核（symmetric）**：全部核一样大 `Speedup_sym(f, n, r) = 1 / ( (1-f)/perf(r) + f / ( (n/r)·perf(r) ) )`
- **非对称多核（asymmetric）**：1 个"胖核"（资源 r）+ (n−r) 个"瘦核"（资源 1） `Speedup_asym(f, n, r) = 1 / ( (1-f)/perf(r) + f / ( perf(r) + n - r ) )`

> **教学要点**：非对称形式里，串行部分由胖核（`perf(r)`）承担，并行部分由"胖核 + 所有瘦核"一起承担。Beckmann 的讲义用 `x = BigCore, T = PSmall + √x` 表达同一个想法：**大核在局部是"低效"的（每单位面积产出的吞吐低），但在全局是"高效"的（它抬高了整个程序的加速比上限）**。补充讲义的原话是："Big CPUs are locally inefficient, but globally efficient."

表 2.1：Amdahl 定律的三种形态

形态	公式	隐含假设	适用场景
经典 Amdahl	`S = 1 / ((1-f) + f/P)`	并行段在 P 个处理器上完美加速	入门分析、粗估上限
资源受限、对称	`S = 1 / ((1-f)/perf(r) + f/((n/r)·perf(r)))`	总资源 n 固定，核越胖则核数越少	同构多核的"核该多大"决策
资源受限、非对称	`S = 1 / ((1-f)/perf(r) + f/(perf(r)+n-r))`	一个胖核 + (n−r) 个瘦核	big.LITTLE、Alder Lake、CPU+GPU

2.2 技术缩放的三条曲线与暗硅（dark silicon）

定义与目的：要理解"为什么必须专用化"，必须先看清**通用处理器性能增长为什么慢下来了**。补充讲义（Beckmann）给出了一条非常干净的三层建模链条：

- **乐观模型**：性能 = 晶体管数 × 频率 = `N·f`。因为 `N' = 2N`（摩尔定律：约每 18 个月晶体管翻倍）且"更小的晶体管也更快"`f' = √2 f`，所以每代可得 `2√2 ≈ 2.83×`。
- **悲观模型**：如果**完全无法利用新增晶体管**（唯一处理器是单核且已经跑满），性能 = `f' = √2 f`，每代只有 `1.41×`。
- **现实模型**：`性能 ≈ √N · f`（讲义注明"略微乐观"），每代 `2×`。直觉是"核数翻倍，但调度/串行/一致性开销吃掉一半"。

功耗的物理：`Power = N × 单个晶体管的功耗`，而单管功耗 `= C·V²·f`，其中电容 `C ≈ 晶体管宽度`、电压 `V ≈ 晶体管长度`、频率 `f ≈ 1/晶体管长度`。一步摩尔定律之后：`N'=2N, C'=C/√2, V'=V/√2, f'=√2 f`（面积 ×1/2，线度 ×1/√2）。于是：

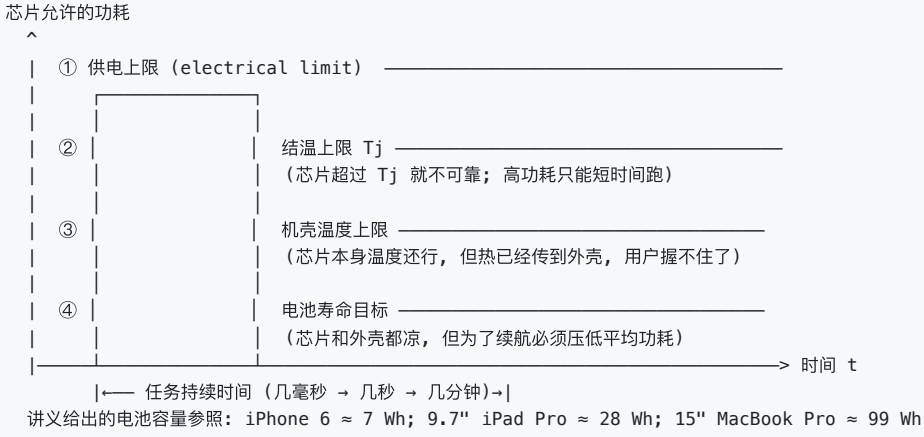
```
Dennard 缩放 (1974–2003):
  Power' = N'·C'·V'²·f' = 2N · (C/√2) · (V/√2)² · (√2 f) = N·C·V²·f = Power
  ⇒ 功率恒定！即使晶体管多了一倍、翻转频率高了 √2 倍。
  ⇒ 多出来的晶体管可以"免费"全用上。

Dennard 缩放终结 (~2003，电压受能带隙等物理限制不能再降):
  Power' = N'·C'·V'²·f' = 2N · (C/√2) · V² · (√2 f) = 2 × Power   ← 功耗翻倍！
  但功率必须近似恒定（散热器有限）⇒ Power'/Power = 1
  ⇒ N'·f' = √2 · N·f
  ⇒ 可用的晶体管比例 = √2/2 ≈ 70.7%，剩下的 ~30% 每代都"点不亮" = 暗硅 (dark silicon)
```

摩尔定律没死，但另外两条缩放死了：讲义的原话是 "Moore's Law isn't dead — at least not in Taiwan & Korea. **But! 成本缩放（cost scaling）停止了，Dennard（功耗）缩放停止了。**" 这正是"暗硅 ⇒ 专用化"的逻辑起点（补充讲义引用了 Esmaeilzadeh et al. ISCA'11 "Dark silicon and the end of multicore scaling" 与 Venkatesh et al. ASPLOS'10 "Conservation cores"）。

直观解释（"它是什么？"）：把芯片想成一栋**散热能力固定的公寓楼**。 – Dennard 时代 = **每个房间的电器都逐年更省电**，所以你可以往楼里不断加房间、加电器，总电费不变。 – Dennard 之后 = **电器不再变省电了**，但楼里房间数每代翻倍。总电费有硬上限（散热器和电池），所以**你必须永久性地关掉约 30% 的房间**。那么问题就变成：这 70% 的房间，是全部做成"什么都能干但都不快"的通用房间，还是拿出一些做成"只干一件事但快 100 倍"的专用车间？

图解 2-1：移动端芯片的"功耗—时间"四道约束墙

讲义（heterogeneity, slide 24）给出的移动端处理经验法则是：**一个任务运行得越久，它被允许使用的功耗就越低**。原因是四道依次收紧的约束：



这条曲线的工程含义：短任务可以"爆发式"用高频大核（比如按下快门那一瞬间的图像处理），长任务必须"细水长流"（比如持续视频播放、always-on 语音唤醒）。所以一台机器**同时需要**大核（吃爆发）和小核（吃长尾），也需要固定功能单元（把长任务的能耗压到 1/100）。

图解 2-2：通用处理器为什么"低效"——一条指令的完整生命周期，以及 H.264 编码的能耗去向

补充讲义（CS149）用一张"执行一条指令要经过哪些步骤"的清单回答了"通用处理器到底低效在哪"：

```
                一条标量 FP 指令在通用处理器中的完整生命周期
┌──────────────────────────────────────────────────────────────────┐
│ ① 取指: 读 I-cache → ② 译码 → ③ 译成 uops / 查 uop cache          │
│ ④ 检查数据依赖与流水线冒险 (记分牌 / 寄存器重命名)                  │
│ ⑤ 选择可用的执行资源 (调度 / 发射)                                 │
│ ⑥ 用译码出的操作数去控制寄存器堆 (RF) SRAM 的读端口                 │
│ ⑦ 把数据从寄存器堆搬到执行单元                                     │
│ ⑧ 执行算术运算                          ← 只有这一步是"有用功"      │
│ ⑨ 把结果搬回寄存器堆 → ⑩ 控制写回端口                              │
│ ⑪ 地址翻译 (TLB / 页表) 配合 D-cache 访问 → ⑫ 实际的访存            │
└──────────────────────────────────────────────────────────────────┘
  每条指令都要把 ①–⑦、⑨–⑫ 的代价重付一遍。
  SIMD 的作用是让 ⑧ 变宽, 从而"摊薄"①–⑦、⑨–⑫ 的每元素开销;
  VLIW / 复杂指令 (DP4、4x4 MMA) 的作用是让一条指令里的 ⑧ 更多。
```

H.264 视频编码的实测能耗分解（Hameed et al. ISCA 2010，讲义以定性堆叠柱状图给出）显示：**即使已经用 SIMD 指令实现了编码器，功能单元（FU）仍然只占能耗的一小片**。

```
H.264 视频编码的能耗分解 (Hameed et al. ISCA 2010)
 ┌────┐ ┌────┐ ┌────┐ ┌────┐    IF  = 取指 + I-cache
 │ IF │ │ IF │ │ IF │ │ IF │    Pip = 流水线级间寄存器
 ├────┤ ├────┤ ├────┤ ├────┤    Ctrl= 杂项流水控制
 │Pip │ │Pip │ │Pip │ │Pip │    RF  = 寄存器堆读写
 ├────┤ ├────┤ ├────┤ ├────┤    D-$ = 数据 cache
 │Ctrl│ │Ctrl│ │Ctrl│ │Ctrl│    FU  = 功能单元 ← 唯一算"有用功"的部分
 ├────┤ ├────┤ ├────┤ ├────┤
 │ RF │ │ RF │ │ RF │ │ RF │
 ├────┤ ├────┤ ├────┤ ├────┤
 │D-$ │ │D-$ │ │D-$ │ │D-$ │
 ├────┤ ├────┤ ├────┤ ├────┤
 │ FU │ │ FU │ │ FU │ │ FU │  ← 很小的一片
 └────┘ └────┘ └────┘ └────┘
   整数     亚像素    帧内预测   算术
  运动估计 运动估计 DCT/量化    编码
⇒ 通用处理器的"开销/有用功"比例极差 ⇒ Horowitz (ISSCC'14): CPU 只有 ≈1% 的能量花在有用计算上
                                          ⇒ 这意味着 ~100× 的专用化机会
```

专用化的收益量级（讲义反复引用的三条经验法则，compared to high-quality C code on CPU）：

架构	性能/能效提升	面积效率	前提条件	编程难度
吞吐优化核（GPU core）	**~10× perf/W**	比 CPU 核高 **5–7×**	代码能映射到宽数据并行且计算密集	中（CUDA/OpenCL）
域专用加速器（如 TPU）	**~20×**（CS149 给出）	—	限定在某一领域（如 DNN），靠 DSL 编程	中高（DSL）
可编程 DSP	介于 CPU 与 ASIC 之间	—	用 VLIW/复杂指令摊薄控制开销	高
FPGA / 可重构逻辑	**~50×**（讲义注明 "jury still out"，学界仍有争议）	—	用 HDL 描述电路	很高
固定功能 ASIC	**~100–1000× 或更高 perf/W**	同等性能只需 CPU 核 **~1/1000 的面积**	计算密集、且**不是浮点数学**时收益最大；ASIC 用 **~1/100 的能量**达到一个 CPU 核的性能	不可编程，且设计/验证/制造要花**数千万到上亿美元**

> 讲义用 "choosing the right tool for the job"（Credit: Pat Hanrahan 的这套分类图）总结：**能效越高者越难编程**，而"让专用硬件更好编程"本身是一个活跃的研究方向。

2.3 异构的形态：从 SoC 到超级计算机

定义与目的：异构并行处理（heterogeneous parallel processing）指**在一台机器里混合多种"优化目标不同"的计算资源**：延迟优化的顺序核（latency-optimized）、吞吐优化的并行核（throughput-optimized）、以及领域专用的固定功能单元（domain-specialized fixed-function）。讲义的原话是：

> "Idea: the most efficient processor is a heterogeneous mixture of resources — **use the most efficient tool for the job**."

直观解释（"它是什么？"）：现代 SoC 就像**一个厨房**。CPU 大核是**主厨**（什么菜都能做，但一次只做一道，单项速度最快）；CPU 小核是**帮厨**（切菜慢一点但省电，可以很多个一起干）；GPU 是**流水线配菜工**（同一道工序重复几千遍最快）；Neural Engine 是**专用压面机**（只会压面，但一秒压一万份，耗电极低）；视频编解码器是**专用切片机**。**真正的设计难题不是"要不要买这些机器"，而是"这台机器上到底该配几台压面机、几个配菜工"**——配少了，压面机成为瓶颈，几百个配菜工一起等它（讲义 slide 39 的"陷阱"）；配多了，面积被占用，通用处理能力下降。

讲义给出的真实系统实例（全部来自讲义原文）：

系统	异构构成	讲义给出的关键数字
Intel Alder Lake Core i9 (2022)	8 个高性能核 + 8 个能效核 + 集成 GPU，同一芯片	—
Apple A12 (2018)	2 高功耗 CPU 核（7-wide issue）+ 4 低功耗 CPU 核（3-wide issue）+ 4 核 GPU + Neural Engine + 视频编解码/GPS/加解密/…	6.9×10^9 晶体管；Neural Engine：固定算术运算序列、8-bit FP、8 宽并行、**5×10^{12} ops/s**
Apple A12 的 SoC 版图（Beckmann 讲义）	一颗芯片上 **42 个不同的 "IP blocks"**	**传统 CPU 只占芯片很小一部分**
Apple M1 (2021 笔记本)	大核 + 小核 + GPU + Neural Engine + 视频编码器 + PCIe 控制器 + …	—
15" MacBook Pro 2011	Intel 四核 i7（内含集成 GPU）+ 独立 AMD Radeon HD GPU	独立（耗电）GPU 平时关掉，只用集成低功耗图形做窗口管理/UI
IBM/AMD "Roadrunner" (2008, LANL)	6,480 个 AMD Opteron 双核 CPU（12,960 核）+ 12,970 个 IBM Cell（每片 1 CPU 核 + 8 加速核 = 116,640 核）	首个突破 Petaflop 的机器：**1.7 PFLOPS**，功耗 **2.4 MW**（约 2,400 个美国家庭的平均用电）
Oak Ridge "Summit"	每节点 2 个 IBM 22 核 POWER9 + **6 个 NVIDIA GPU**，608 GB DRAM，1600 GB Flash	4,608 节点；**10 MW 水冷**；两台机器共 **$325 M**
Intel Xeon Phi (Knights Landing)	72 个"简单" x86 核（1.1 GHz，源自 Intel Atom），16 宽向量（AVX-512），每核 4 线程	定位为超级计算应用的加速器
Top500 Fall 2021 榜单	Sunway 256 核 manycore / NVIDIA GPU / Intel Xeon Phi / A64FX 48 核 ARM（"scalable vectors"）	异构已是超算主流形态
Green500（能效榜）	度量单位 **MFLOPS per Watt**；榜首出现 DNN 训练加速器与 NVIDIA GPU	能效取代峰值成为新指标
Qualcomm Hexagon DSP	多线程 **VLIW** DSP；SoC 上**"第三个主要可编程单元"**（多核 CPU、Adreno 多核 GPU、Hexagon DSP）	最初用于音频/LTE，现在越来越多用于图像处理；FFT 最内层循环每拍执行 **29 个 "RISC" 操作**
Anton（DE Shaw Research）	高度专用化的分子动力学超算：**512 个**计算粒子-粒子相互作用的 ASIC；吞吐导向的 FFT 子系统；为 N-body 通信模式定制的低延迟网络	模拟蛋白质的时间演化；CS149 补充：**Anton 3 (2025) 约比同期 GPU 快 20 倍**
微软 Project Catapult	1U 服务器（双路 CPU + 经 PCIe 连 FPGA 板）；用 FPGA 卸载 Bing 搜索的文档排序逻辑（Putnam et al. ISCA 2014）	现已广泛用于加速微软各服务中的 DNN 与系统基础设施
Google TPU	领域专用 DNN 加速器；芯片面积中**算术单元约占 30%**，控制面积很小	关键指令只有：读主机内存 / 写主机内存 / 读权重 / `matrix_multiply` / `convolve` / `activate`（Jouppi et al. 2017）

GPU 本身就是一个异构多核处理器：讲义画出 GPU 的内部结构——大量重复的 `SIMD Exec + Cache` 切片（这才是 Assignment 2 里 CUDA 程序用到的通用计算资源），**外加**一批图形专用的固定功能单元：Texture（贴图/滤波）、Clip/Cull、Rasterize、Tessellate、Zbuffer/Blend，以及最上层的 Scheduler / Work Distributor。讲义借此解释"什么是固定功能硬件加速的图形任务"：

- **Rasterization（光栅化）**：确定一个三角形覆盖了哪些像素；
- **Texture mapping（纹理映射）**：对图像做变形/滤波，把细节贴到表面上；
- **Geometric tessellation（几何细分）**：由粗糙几何计算出精细几何。

> 补充讲义（Beckmann）特别强调一个容易被误解的点：在"CPU + GPU"这个组合里，**GPU 并不是这个故事里的 "accelerator"，它是一个可编程的 manycore**（真正准确的说法是"介于二者之间"）。把 GPU 当"加速器"会让人误以为它只能做一件固定的事，从而忽略它作为通用吞吐平台的调度难题。

2.4 加速器设计原理：把能量全部花在有用功上

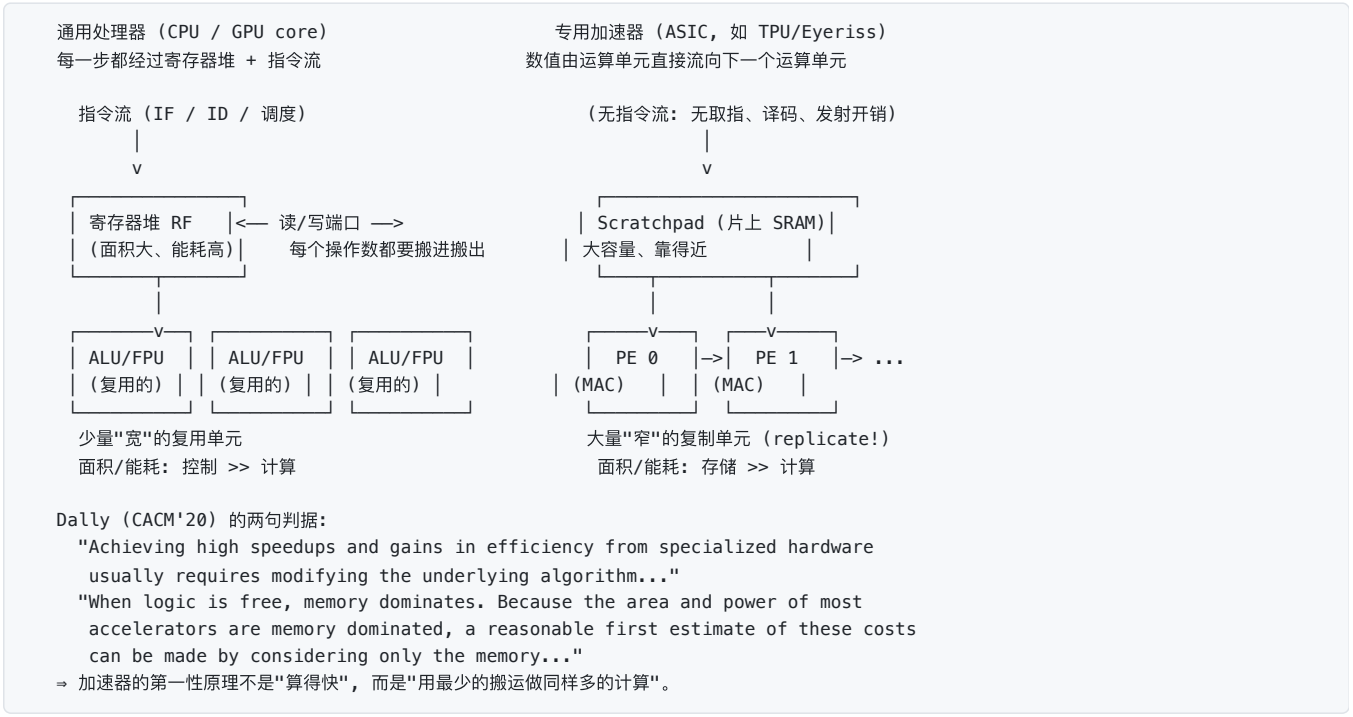
定义与目的：加速器（accelerator）的设计目标可以一句话概括——**Spend all energy on useful work**。做法是：

1. **Replicate a minimal, custom circuit many times**（把最小的定制电路复制很多份）：最大化并行、消除开销、回到"乐观缩放"的区间（面积与算力同比例增长，不被控制逻辑稀释）；
2. **数据值直接在运算单元之间路由，取消寄存器堆**（Data values routed directly from one operation to another — no register file!）；
3. **配一大块片上存储把数值放在近处**（Big on-chip memory / "scratchpad"）。

直观解释（"它是什么？"）：通用处理器像**一个万能工位**：工具放在远处的工具柜（寄存器堆/内存），每做一步都要"走过去拿工具、走回来用、再走过去放回"。加速器像**一条固定工序的装配线**：每台机器只做一件极小的事，工件**直接从前一台机器滑到下一台**，旁边放一大箱零件（scratchpad）随手可取。因为工序固定，**不需要"看图纸"（取指译码）**，也不需要"每步都回工具柜"。

这就是"分工"的能效来源：讲义引用 Dally（NVIDIA/Stanford, SC'15）的结论——**一旦把 CPU 的低效去掉，数据搬运就变成能耗的绝对主导**。

图解 2-3：通用处理器 vs 专用加速器的结构对比



因此，加速器设计本质上是一种算法设计：要把算法改写成一个"数据局部性极高、中间结果尽量不落片外"的形式。讲义给出的两个正面例子：

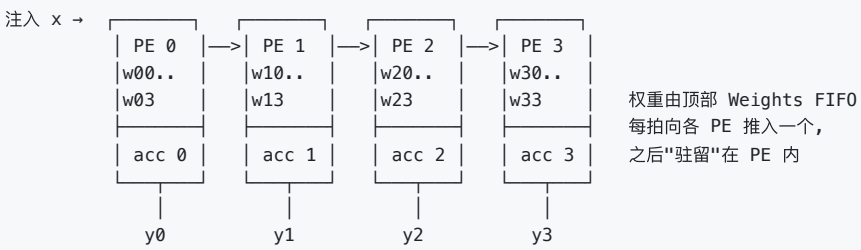
- **Eyeriss**（Chen, Krishna, Emer, Sze, ISCA'16）：开创性的 CNN 加速器，特点是**大容量片上存储 + 为 CNN 定制的片上网络拓扑**。
- **脉动阵列（systolic array）**：Google TPU 的核心，把"权重驻留 + 数据波前流动"做成了电路（见 2.5）。

2.5 脉动阵列（systolic array）：数据驱动的波前

定义与目的：脉动阵列是一组**规则排列的处理单元（PE）**，每个 PE 只与**邻居**通信，数据像心脏搏动（systole）一样**一拍一格**地流过阵列。它的目的是把"矩阵乘"这种规则计算的**全局访存**替换成**局部邻居通信**，同时让权重"驻留"在 PE 内。TPU v1 用它做 `y = Wx` 与卷积。

直观解释（"它是什么？"）：把它想成**一排排接力赛**。 – 通用 SIMD 的做法像"**一个教练（指令流）同时指挥 32 个人去仓库取货、各自算、再送回仓库**"——每次都要过仓库（寄存器堆/内存）。 – 脉动阵列像"**一条传菜流水线**：食材（x）从流水线一端递进来，每经过一个工位就被该工位专属的调料（权重 w）处理一次，处理完就顺手传给下一个工位**"。传的是"手递手"（邻居连线），不用回仓库；结果（累加器 acc）就留在自己工位上。

图解 2-4：`y = Wx` 的 N×N 脉动阵列波前



时间-空间图（哪一拍哪个 PE 在做哪一次乘加）：

拍 t:	0	1	2	3	4	5	6
PE 0:	w00	w01	w02	w03	–	–	–
PE 1:	–	w10	w11	w12	w13	–	–
PE 2:	–	–	w20	w21	w22	w23	–
PE 3:	–	–	–	w30	w31	w32	w33

填充（fill）

↑ y3 在 t = 6 就绪 ⇒ Span = 2N−1 = 7 拍

稳态吞吐 = N 次 MAC / 拍（每 PE 每拍 1 次 MAC）

平均 PE 利用率 = N^2 MAC / (N 个 PE × (2N−1) 拍) = N/(2N−1) → 1/2

性能特征（延迟、带宽、吞吐量）：

- **吞吐量**：稳态下每个 PE 每拍做一次 MAC，阵列吞吐 = `N` MAC/拍；TPU 用 256×256 的乘加阵列。
- **延迟 / Span**：单个 `y = Wx` 的关键路径是 `2N−1` 拍（填充 N 拍 + 排空 N−1 拍）。
- **带宽**：全局/片外搬运量从朴素实现的 `O(N²)` 降到 `N² + N`（权重只装载一次、x 只注入一次）；**取而代之的是 `(N−1)(2N−1)` 次邻居间传递**（N 个 PE 排成一线，每拍有 N−1 条链路上的传递，共 `2N−1` 拍；N=8 时是 105 次）——这看起来比搬运次数更多，但都是**片上 1 mm 内的局部连线**，能耗比 DRAM 低 1–2 个数量级（见 4.3 节 pJ 数值）。稳态下每次 MAC 只伴随约 `(N−1)/N ≈ 1` 次邻居传递。
- **效率**：填充/排空期让单次运算的平均利用率约 50%；连续处理多个输入向量可以把排空期填满，利用率趋近 100%。

表 2.2：SIMD 与脉动阵列的对比（CS149 讲义原表）

特性	SIMD	Systolic Array
数据流（Dataflow）	控制驱动（指令流）	**数据驱动（波前 wavefront）**
局部性（数据复用）	有限	**时间 + 空间复用**
通信	全局（寄存器堆 / 存储器）	**局部（邻居 PE）**
控制	集中式	**分布式**
效率（perf/mm²、perf/Watt）	中等	**极高**

2.6 软件执行模型：从 CUDA 到"分块张量 + 异步流水 + 算子融合"

定义与目的：硬件给了异构资源，软件必须提供**能表达"数据搬运"的抽象**。CS149 讲义把"理想 AI 模型加速器"该具备的特性列成了一张清单，并逐条给出**为什么**：

表 2.3：理想加速器的特性清单，以及 NVIDIA GPU 的完成度

特性	为什么需要	NVIDIA GPU 是否具备
分块张量（tiled tensors，如 16×16、32×32）	在 GEMM 上取得最高 TFLOPS 且指令开销低	✅（`mma` / wgmma 指令）
异步计算（asynchronous compute）	让计算与访存重叠	✅（`mma_async`）
异步访存（asynchronous memory access）	让计算与访存重叠	✅（TMA + TMEM）
异步片间通信（chip-to-chip）	让计算、访存、通信三者重叠	❓
计算单元到计算单元的通信	支持算子融合（fusion）与流水线，即流式数据流	❓（部分靠 TB Cluster）

分块张量为什么是"最大 TFLOPS"的钥匙：一条 `16×16×16` 的 tensor core 指令一次完成 4096 次乘加。CS149 讲义给出"可编程性开销"的估算——同样是完成一批乘加运算，半精度 FMA 形式的指令流开销约为 **2000%**，半精度 DP4（vec4 点积）降到 **500%**，半精度 4×4 MMA 只有 **27%**。核心原则是：

> **Key principle: amortize cost of instruction stream processing across many operations of a single complex instruction.**（用一条复杂指令里的众多运算来摊薄指令流处理成本。）

这也解释了硬件数值格式的演进：`BF16`（S=1, E=8, M=7，**与 FP32 同范围但精度更低**）、`BF8 E4M3`（范围 0—448）、`BF8 E5M2`（范围 0—57344）——格式越来越窄，是因为**单位面积/单位能量的吞吐**才是目标（slide credit: Bill Dally）。

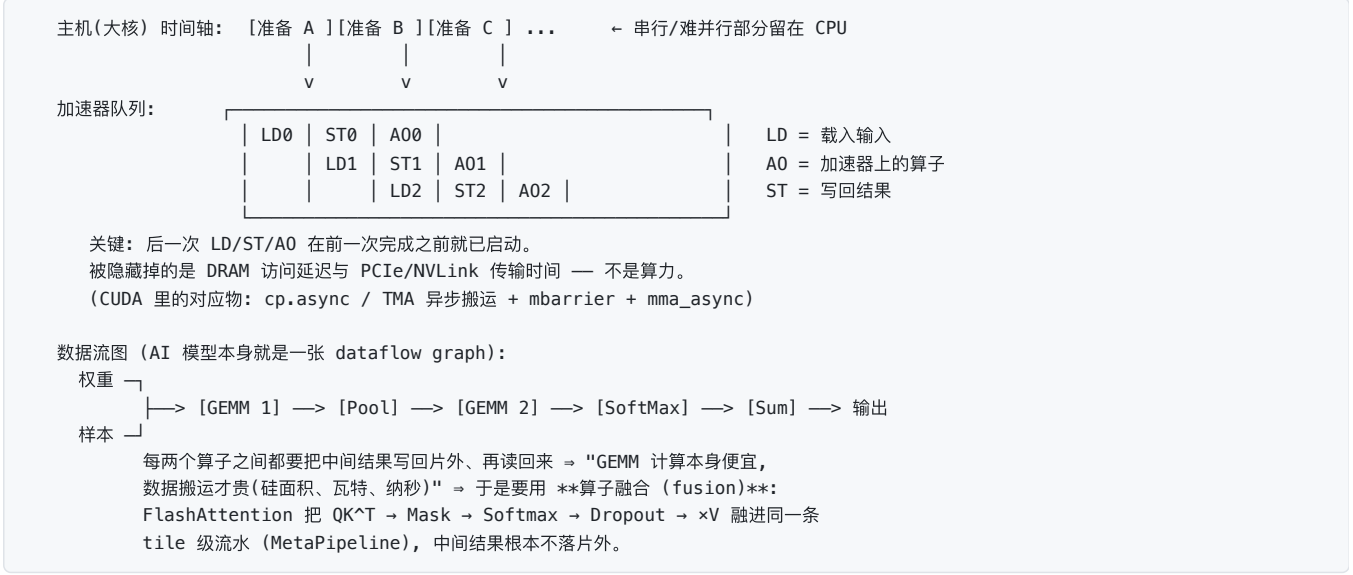
CUDA 的三层层级结构（CS149 给出的 H100 对照表）：

```
CUDA 层级            计算层级                存储层级
──────────────────────────────────────────────────────────────────
Grid          ←→     GPU            ←→    80 GB HBM / 50 MB L2
Cluster       ←→     CPC            ←→    每 SM 256 KB shared memory
Thread Block  ←→     SM             ←→    每 SM 256 KB shared memory
Threads       ←→     SIMD Lanes     ←→    每线程 1 KB 寄存器,
                                           每 SM 分区 64 KB
· Thread Block Cluster 是至多 16 个 thread block 的集合
· 每个 thread block 保证跑在**独立的一个 SM** 上, 且**同时**执行
  (这是"计算单元到计算单元通信"的硬件基础)
```

图解 2-5：H100 的 Streaming Multiprocessor（SM）内部结构

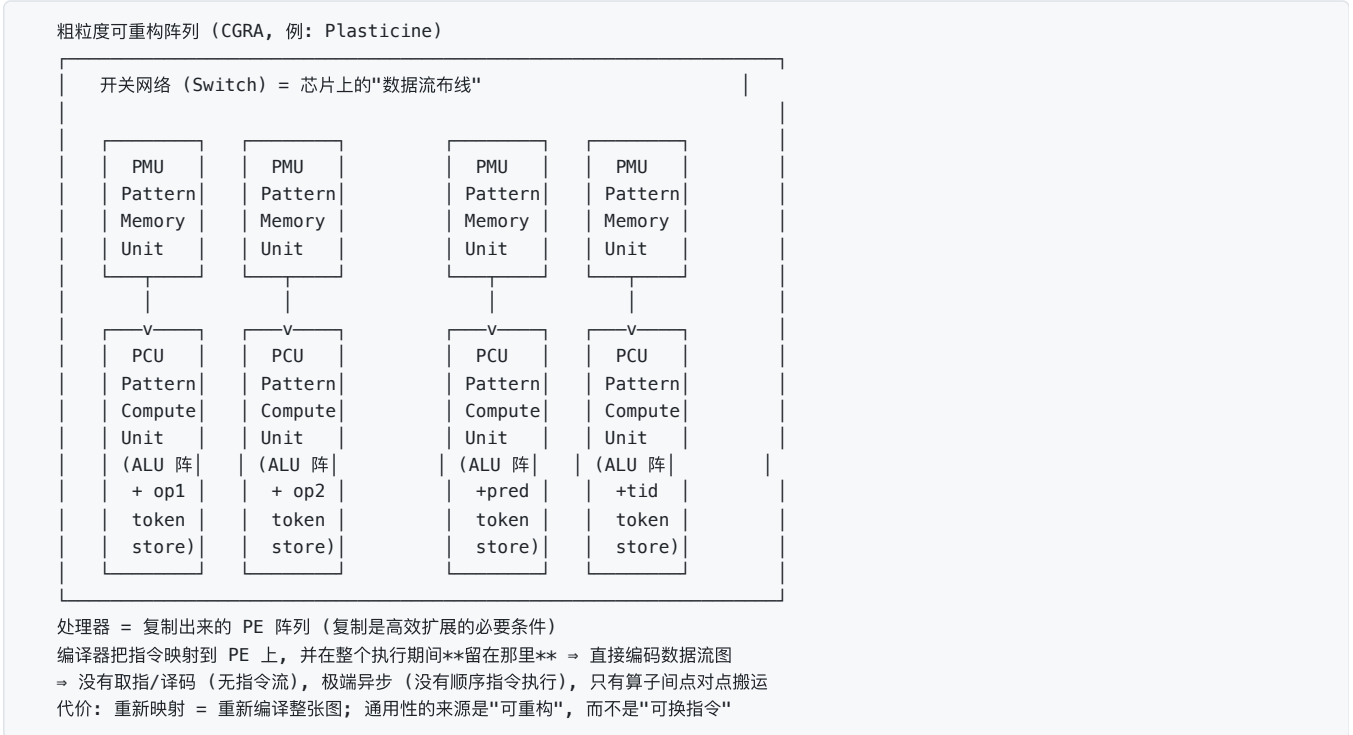
```
NVIDIA H100 SM  (讲义数据: 整芯片 144 个 SM; 每 SM 含 4 个 sub-core)
┌──────────────────────────────────────────────────────────────────────┐
│  Shared Memory / L1  (256 KB, 可配置)        ┌──────────────────────┐ │
│  ┌──────────────────────────────────┐        │ Tensor Memory Accel. │ │
│  │  片上暂存: 相当于"加速器的 scratchpad"│    │ (TMA): 单线程发起一整块张量 │ │
│  └──────────────────────────────────┘        │ 的异步 global→shared 搬运, │ │
│                                              │ 由 copy descriptor 描述区域,│ │
│  ┌──── Sub-core 0 ────┐ ┌──── Sub-core 1 ──┐ │ 完成后用 barrier 通知 │ │
│  │ Warp Selector      │ │ Warp Selector    │ └──────────────────────┘ │
│  │ Fetch/Decode (1 warp/拍)│ Fetch/Decode   │                          │
│  │ ┌────────────────┐ │ │ ┌──────────────┐ │   每个 sub-core:         │
│  │ │ 32-wide FP32 SIMD │ │ │ 32-wide FP32 │ │   · FP32: 每拍 32 次 MUL-ADD │
│  │ │ 32-wide INT32 SIMD│ │ │ 32-wide INT32│ │   · INT32: 每 1 拍 16 次  │
│  │ │ 16-wide FP64 SIMD │ │ │ 16-wide FP64 │ │   · FP64: 每 2 拍 16 次   │
│  │ │ LSU (load/store)  │ │ │ LSU          │ │   · 寄存器 64 KB / sub-core │
│  │ │ Tensor Core       │ │ │ Tensor Core  │ │   · 4 个 sub-core 共 256 KB │
│  │ │  16x16x16         │ │ │  16x16x16    │ │     寄存器, 由 ≤64 个 warp 分 │
│  │ │  [fp16 × fp16→fp32]│ │ │ [fp16×fp16→fp32]│ │                        │
│  │ └────────────────┘ │ │ └──────────────┘ │                            │
│  └────────────────────┘ └──────────────────┘                            │
└──────────────────────────────────────────────────────────────────────┘
同一张图上的两套算力 (A100/GA100 的讲义数字):
  SIMD 路径:   108 SM × 64 fp32 ALU × 2 flop × 1.4 GHz  = 19.5 TFLOP/s (fp32)
  Tensor Core: 108 SM × 4 个 tensor core                 = 312 TFLOP/s (fp16 输入, fp32 累加)
⇒ 94% 的峰值算力在专用单元里; H100 更进一步: tensor core 989 TFLOP/s (fp16),
  SIMD 只有 134 TFLOP/s (fp16) / 67 TFLOP/s (fp32) ⇒ "All the TFLOPS are in the
  Tensor Cores" (讲义给出的各代占比: 89% / 50% / 94% / 96% / 98%)。
```

软件侧的"异步流水"执行模型：为了不让加速器等数据，执行必须**异步（non-blocking）**——后一次搬运/计算在前一次完成之前就启动：



图解 2-6：粗粒度可重构阵列（CGRA）——"把数据流图钉在硅上"

讲义（Beckmann）最后给出"通用加速"的一条路线：**coarse-grained reconfigurable arrays (CGRAs)**，代表系统是 **Plasticine**（Prabhakar et al., ISCA'17），后商业化为 **SambaNova**（讲义注明 2021 年 4 月估值 $5.1B；作者 Kunle Olukotun, Stanford）。它复用了 TRIPS、Garp、Piperench、Dyser 等一脉相承的老思想。



2.7 异构系统的两大难题：配比与映射

定义与目的：讲义把异构的困难拆成"硬件设计者的难题"和"软件开发者的难题"两半。

硬件设计者的难题（配比问题）——讲义原话整理：

- 什么是**正确的资源混合比（mixture of resources）**才能同时满足性能、成本与能耗目标?
- **吞吐型资源太少** ⇒ 并行负载的峰值性能/效率低（本该把资源拿去做更多吞吐核）;
- **顺序处理资源太少** ⇒ 被 Amdahl 定律咬住;
- **该给某个特定功能（例如视频）分配多少芯片面积?** （这些面积是从通用处理能力里扣出来的）;
- **如何把"有哪些单元"告诉软件?** （抽象是什么? 编译器和操作系统怎么办? ）;
- **推论：芯片设计时必须极其准确地理解（未来的）工作负载** —— 系统**无法**随使用习惯变化、新算法出现而自适应;

- **趋势是保守地"超额配置"固定功能单元，从而削弱它们的优势。**

软件开发者面临的难题（映射问题）：如何把程序映射到一堆异构资源上？

- "Pick the right tool for the job"：算法必须分解成若干部分，**每一部分都恰好匹配机器上某一类计算单元**；
- 异构系统的**调度问题更复杂**（要考虑数据传输、单元间依赖、谁的队列更长）；
- **可用的资源混合比会反过来决定算法的选择**（如果机器上有一台 1000× 的 DNN ASIC，你可能就把算法改成 DNN 友好的形式）；
- **可移植性与维护是噩梦**（"we'll revisit this next class"）。

2.8 能耗：把"少搬数据"当成第一设计准则

定义与目的：讲义把减少能耗总结为两条思路：① **用专用化处理单元**，② **少搬数据**。

讲义给出的"ballpark"能耗数值（这是本讲最重要的量化工具表）：

表 2.4：数据搬运的能耗代价

操作	能耗	备注
整数运算	**~1 pJ**	只算逻辑操作本身，不含指令译码、从寄存器取数等开销
浮点运算	**~20 pJ**	同上
从片上小的本地 SRAM 读 64 位（距离 1 mm）	**~26 pJ**	仅仅"1 mm 之外"就要付 26 倍于整数运算的能量
从低功耗移动 DRAM（LPDDR）读 64 位	**~1200 pJ**	比片上 SRAM 贵 **~46 倍**，比整数运算贵 **~1200 倍**

直接推论（讲义原文）：

- 以 10 GB/s 从内存读数据 ⇒ 约 **1.6 W** 功耗；
- 而**整个移动 GPU 的功耗预算只有约 1 W**（别忘了手机还要跑 CPU、显示、射频等）；
- iPhone 6 电池只有 ~7 Wh（对比 MacBook Pro 笔记本的 99 Wh 电池）。

> **结论（讲义原话的意译）**："Exploiting locality matters!!!" 并且——"Suggests that recomputing values, rather than storing and reloading them, is a better answer when optimizing code for energy efficiency!"（为能效优化时，**重新计算**往往优于"存起来再读回来"。）

讲义总结的三条能效趋势：

1. **Compute less!（少算）**：计算要耗能，因此**并行算法如果比串行版本做了更多的工作，即使跑得更快，也未必值得**。但性能仍然重要，因为处理器只要开着就在烧"静态功耗"。
2. **Specialize compute units（专用化计算单元）**：异构处理器（CPU 类核 + 吞吐优化核）；固定功能单元（音频、"运动传感器处理"、视频编解码、图像处理/计算机视觉）；专用指令（不断扩张的 AVX 向量指令集、AES-NI 加密指令）；可编程软逻辑（FPGA）。
3. **Reduce bandwidth requirements（降低带宽需求）**：利用局部性（重构算法以尽量在片上复用数据）；激进地使用压缩（**多做一点计算来压缩数据**再写入内存，讲义预计会出现专门降低通用压缩/解压开销的固定功能硬件）。

---

3. 代码示例与性能分析

3.1 示例一：异构任务划分——把"可并行段"和"必须串行段"分给不同的单元

这个示例把 2.1 节的 Amdahl 思想写成一个可运行的程序：**阶段 A** 是完全数据并行、计算密集的（应该放到"多而小"的吞吐型单元上）；**阶段 B** 是一条**串行递推**（应该放到"少而快"的大核上）。程序实测两个阶段的时间占比，并用 Amdahl 定律预测不同核数下的加速比上限。

```cpp
// 编译 (release):  g++ -O3 -fopenmp -march=native -std=c++17 het_pipeline.cpp -o het_pipeline
// 运行:            OMP_NUM_THREADS=16 ./het_pipeline
//
// 说明: -O3 打开向量化与循环优化; -march=native 让编译器用本机可用的 SIMD
//       指令集(SSE/AVX/AVX-512), 这一步对"阶段 A"的性能影响极大;
//       -fopenmp 提供 OpenMP 运行时(omp_get_wtime / omp_get_max_threads)。

#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

static const int H = 1024;   // 图像高
static const int W = 1024;   // 图像宽
static const int K = 32;     // 每个像素的迭代次数 (模拟计算密集程度)

// ---------------------------------------------------------------
// 阶段 A: 高度数据并行 (行与行之间完全独立)
// 映射建议: 放到"多而小"的吞吐型单元 (GPU / 小核 / 宽 SIMD 单元) 上。
// ---------------------------------------------------------------
static void stage_compute_parallel(const std::vector<float>& in,
                                   std::vector<float>& out)
{
    #pragma omp parallel for schedule(static)
    for (int y = 0; y < H; ++y) {
        for (int x = 0; x < W; ++x) {
            float v = in[(size_t)y * W + x];
            for (int k = 0; k < K; ++k) {
                // 带 sqrt 与 fabs 的非线性迭代: 延迟高、吞吐受限、无分支
                v = 0.999f * v + 0.001f * sqrtf(fabsf(v) + 1.0f) + 1e-6f;
            }
            out[(size_t)y * W + x] = v;
        }
    }
}

// ---------------------------------------------------------------
// 阶段 B: 顺序依赖强 (第 y 行的阈值依赖第 y-1 行的结果)
// 映射建议: 放到"少而快"的延迟优化核 (大核) 上 —— 大核在局部低效,
//           但在全局高效: 它把这段无法并行的拖尾时间压下去。
// ---------------------------------------------------------------
static float stage_adaptive_threshold(const std::vector<float>& img,
                                      std::vector<float>& mask)
{
    float thr = 0.5f;
    for (int y = 0; y < H; ++y) {
        double row_sum = 0.0;
        for (int x = 0; x < W; ++x) {
            float v = img[(size_t)y * W + x];
            mask[(size_t)y * W + x] = (v > thr) ? 1.0f : 0.0f;
            row_sum += v;
        }
        const float mean = (float)(row_sum / W);
        thr = 0.9f * thr + 0.1f * mean;   // 串行递推: 无法并行, 也无法向量化
    }
    return thr;
}

int main()
{
    std::vector<float> in((size_t)H * W), out((size_t)H * W), mask((size_t)H * W);
    for (size_t i = 0; i < in.size(); ++i)
        in[i] = 0.5f + 0.5f * sinf((float)i * 1e-4f);

    const double t0 = omp_get_wtime();
    stage_compute_parallel(in, out);
    const double t1 = omp_get_wtime();
    const float thr = stage_adaptive_threshold(out, mask);
    const double t2 = omp_get_wtime();
```

```
    const double ta = t1 - t0, tb = t2 - t1, tt = ta + tb;
    printf("threads            = %d\n", omp_get_max_threads());
    printf("stage A (parallel) = %8.3f ms  (%5.1f%% of total)\n",
           ta * 1e3, 100.0 * ta / tt);
    printf("stage B (serial)   = %8.3f ms  (%5.1f%% of total)\n",
           tb * 1e3, 100.0 * tb / tt);
    printf("total              = %8.3f ms   (final threshold = %.4f)\n",
           tt * 1e3, thr);

    // Amdahl: 用实测的串行时间占比 s 预测"再加核"的收益上限
    const double s = tb / tt;
    printf("\ns = serial fraction = %.3f\n", s);
    for (int p = 1; p <= 64; p *= 2)
        printf("  P=%2d cores -> speedup ceiling = %6.2fx   (asym. ceiling = %6.2fx)\n",
               p, 1.0 / (s + (1.0 - s) / p), 1.0 / s);
    return 0;
}
```

【代码做什么？】

1. `main` 先构造一张 1024×1024 的浮点"图像"（用 `sinf` 生成有空间变化的输入，避免常量折叠）。
2. `stage_compute_parallel`：对**每一行**做 32 次非线性迭代。每个像素的迭代次数固定、无分支、无跨像素依赖，因此这一阶段是"教科书式的数据并行"。
3. `stage_adaptive_threshold`：逐行计算已处理图像的均值，并用 `thr ← 0.9·thr + 0.1·mean` 更新阈值。**第 y 行用的是第 y−1 行算出的阈值**，这是一条真正的串行递推（类似 IIR 滤波），无法用并行前缀（scan）简单拆开，也无法向量化（阈值是循环携带依赖）。
4. 用 `omp_get_wtime()` 分别给两个阶段计时，打印各阶段的时间占比 `s`，再用 Amdahl 公式 `1/(s + (1−s)/P)` 打印不同核数下的加速比上限，同时打印 `1/s` 这一"核数无穷大时的天花板"。
5. 编译时必须用 `-O3`：不开优化时阶段 A 的 `sqrtf`/循环开销会淹没真实结构，测出的"串行比例"会失真。

【并行机制与性能解说】

- **硬件上怎么并行**：`#pragma omp parallel for schedule(static)` 让 OpenMP 运行时在进入循环时唤醒一个线程池（线程数 = `OMP_NUM_THREADS`，默认为物理核数），把 `y ∈ [0, H)` 的迭代空间**静态切块**（默认连续切块），每个线程拿一段连续行。因为每次迭代访问独立的内存地址（行内连续，行间相隔 `W` 个 float），**不存在写共享与伪共享（false sharing）**：不同线程写的缓存行不重叠（`W = 1024` 个 float = 4096 B，正好是 64 B 缓存行的整数倍）。若把 `W` 改成非对齐的奇数，相邻行会共享缓存行，写共享缓存行会触发**缓存行在两个核之间反复迁移（ping-pong）**，性能可能掉数倍——这是把"行切分"改成"列切分"或任意 1D 切分时的经典陷阱。
- **阶段 B 为什么留在单线程**：它是一条长度为 `H·W` 的**依赖链**，每个元素的写入都依赖前一个元素的中间状态（`row_sum` 与 `thr`）。如果强行并行（例如每个线程负责若干行、各自维护局部阈值），结果与串行语义**不等价**，属于"为了并行而改了算法"。若确实要并行，正确做法是改成两遍（two-pass）：第一遍并行算所有行的均值，第二遍并行算 `thr` 的串行递推——把 `thr` 的递推单独抽成一条长度为 `H` 的小串行链（Span 从 `H·W` 降到 `H`）。
- **Work / Span / 并行度（按 `H = 1024, W = 1024, K = 32` 计算）**：

量	阶段 A	阶段 B	合计
Work（总工作量）	$H·W·K = 3.355 \times 10^7$ 次迭代	$H·W = 1.049 \times 10^6$ 次逐元素更新	3.460×10^7
Span（关键路径）	$W·K = 3.277 \times 10^4$（一行内串行）	$H·W = 1.049 \times 10^6$（完全串行，并行度 1）	1.081×10^6
并行度 = Work/Span	$H = 1024$	1	$3.460\times10^7 / 1.081\times10^6 \approx 32.0$

解读：整程序的**理论并行度只有约 32**，尽管阶段 A 单独看有 1024 的并行度。这正是异构机器存在的理由——把一个"局部并行度 1024、局部串行度 1"的程序，拆给两种不同的单元。**可扩展性上限**由阶段 B 决定：即便并行部分无限快，整体加速比也不会超过 `Work/Span = 32×`。

- **数值化的性能预测（按假设参数推导）**：设阶段 A 的单次迭代在开了 SIMD 的核上平均耗 10 个周期、CPU 主频 3.0 GHz、16 个线程；阶段 B 的每次逐元素更新耗 4 个周期、单线程。
- 单线程总时间：`3.355e7 × 10 / 3.0e9 + 1.049e6 × 4 / 3.0e9 = 111.8 ms + 1.40 ms = 113.2 ms`；
- 16 线程总时间：`111.8/16 + 1.40 = 6.99 + 1.40 = 8.39 ms`；
- 实测加速比 ≈ `113.2/8.39 = 13.5×`，而 Amdahl 上限（`s = 1.40/113.2 = 1.24%`）是 `1/(0.0124 + 0.9876/16) = 13.6×`（吻合），核数 →∞ 的天花板是 `1/0.0124 ≈ 81×`。
- **结论**：在这个例子里 `s = 1.2%` 尚可接受；但如果阶段 B 的常数因子再大 10 倍（`s ≈ 11%`），16 核加速比就掉到 `1/(0.11 + 0.89/16) = 6.0×`——**"给串行段配一个大核"（提高 `perf(r)`）比"再堆 8 个小核"有效得多**，这就是 2.1 节非对称模型的直接体现。

3.2 示例二：CUDA 分块矩阵乘（tiled GEMM）——"当逻辑免费时，存储说了算"

这个示例是 2.6 节"分块张量 + 片上复用"思想的最小可运行版本：用共享内存（shared memory，即 GPU 上的 **scratchpad**）把全局内存流量降低 `TILE` 倍。

```
// 编译 (release):  nvcc -O3 -arch=sm_80 --use_fast_math tiled_gemm.cu -o tiled_gemm
//                  (也可以在 nvcc 前加 -Xptxas -O3; 不开 -O3 时内层循环不会充分展开)
// 运行:            ./tiled_gemm
//
// 说明: TILE 决定片上复用度。TILE=16 时一个 block 是 16x16=256 个线程,
//       每个线程算 C 的一个元素, 累加器留在寄存器里。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define TILE 16

#define CHECK(call)                                                          \
    do {                                                                     \
        cudaError_t e_ = (call);                                             \
        if (e_ != cudaSuccess) {                                             \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,    \
                    cudaGetErrorString(e_));                                 \
            exit(1);                                                         \
        }                                                                    \
    } while (0)

// C = A * B, 三个都是 N x N 的行主序方阵
__global__ void tiled_gemm(const float* __restrict__ A,
                           const float* __restrict__ B,
                           float* __restrict__ C, int N)
{
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    const int tx = threadIdx.x;          // 0..TILE-1
    const int ty = threadIdx.y;          // 0..TILE-1
    const int row = blockIdx.y * TILE + ty;
    const int col = blockIdx.x * TILE + tx;

    float acc = 0.0f;                    // 累加器常驻寄存器, 不落片外

    for (int t = 0; t < N / TILE; ++t) {
        // 1) 协作式搬运: 每个线程搬 1 个 A 元素和 1 个 B 元素到片上 scratchpad
        As[ty][tx] = A[(size_t)row * N + (t * TILE + tx)];
        Bs[ty][tx] = B[((size_t)(t * TILE + ty)) * N + col];
        __syncthreads();                 // 2) 保证 tile 装载完成

        // 3) 在片上 scratchpad 里做 TILE 次乘加 (无全局访存)
        #pragma unroll
        for (int k = 0; k < TILE; ++k)
            acc += As[ty][k] * Bs[k][tx];

        __syncthreads();                 // 4) 防止下一轮覆盖本轮的 tile
    }
    C[(size_t)row * N + col] = acc;
}

int main()
{
    const int N = 1024;
    const size_t bytes = (size_t)N * N * sizeof(float);

    float *hA = (float*)malloc(bytes), *hB = (float*)malloc(bytes),
          *hC = (float*)malloc(bytes);
    for (size_t i = 0; i < (size_t)N * N; ++i) { hA[i] = 1.0f; hB[i] = 1.0f; }

    float *dA, *dB, *dC;
    CHECK(cudaMalloc(&dA, bytes));
    CHECK(cudaMalloc(&dB, bytes));
    CHECK(cudaMalloc(&dC, bytes));
    CHECK(cudaMemcpy(dA, hA, bytes, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(dB, hB, bytes, cudaMemcpyHostToDevice));
```

```
    const dim3 block(TILE, TILE);
    const dim3 grid(N / TILE, N / TILE);

    tiled_gemm<<<grid, block>>>(dA, dB, dC, N);   // 预热：排除首次启动开销
    CHECK(cudaDeviceSynchronize());

    cudaEvent_t e0, e1;
    CHECK(cudaEventCreate(&e0));
    CHECK(cudaEventCreate(&e1));
    CHECK(cudaEventRecord(e0));
    const int ITER = 20;
    for (int it = 0; it < ITER; ++it)
        tiled_gemm<<<grid, block>>>(dA, dB, dC, N);
    CHECK(cudaEventRecord(e1));
    CHECK(cudaDeviceSynchronize());

    float ms = 0.0f;
    CHECK(cudaEventElapsedTime(&ms, e0, e1));
    ms /= ITER;

    const double flops = 2.0 * (double)N * N * N;          // N^3 次 MAC = 2N^3 FLOP
    const double bytes_moved = 4.0 * 2.0 * (double)N * N * N / TILE;  // 见下文推导
    printf("N=%d  TILE=%d  kernel = %.3f ms  ->  %.1f GFLOP/s\n",
           N, TILE, ms, flops / (ms * 1e-3) / 1e9);
    printf("  DRAM traffic (analytic) = %.2f GB, achieved BW = %.0f GB/s\n",
           bytes_moved / 1e9, bytes_moved / (ms * 1e-3) / 1e9);
    printf("  arithmetic intensity    = %.2f FLOP/byte\n", flops / bytes_moved);

    CHECK(cudaMemcpy(hC, dC, bytes, cudaMemcpyDeviceToHost));
    printf("C[0]=%f (期望 %d)\n", hC[0], N);   // A 全 1, B 全 1 => 每行和 = N
    CHECK(cudaFree(dA)); CHECK(cudaFree(dB)); CHECK(cudaFree(dC));
    free(hA); free(hB); free(hC);
    return 0;
}
```

【代码做什么？】

1. 主机端分配三个 `N×N`（`N = 1024`）的方阵；`A`、`B` 全填 1，因此正确答案是 `C[i][j] = N`，便于校验。
2. 用 `dim3 block(TILE, TILE)`（256 个线程）和 `dim3 grid(N/TILE, N/TILE)`（4096 个 block）启动核函数：**一个 thread block 负责 C 的 TILE×TILE 一小块**。
3. 核函数主循环沿 `k` 维以 `TILE` 为步长推进。每轮：
4. 每个线程从全局内存各取一个 `A` 元素和一个 `B` 元素写入共享内存 `As` / `Bs`（**协作式搬运**）；
5. `__syncthreads()` 保证整块 tile 都装好再使用（否则会有线程读到未写入的共享内存）；
6. 在片上做 `TILE` 次乘加（`#pragma unroll` 让 16 次迭代全展开，编译器可把它们排成流水，用多个 FMA 发射槽掩盖 FMA 的延迟）；
7. 第二次 `__syncthreads()` 防止快线程进入下一轮、在本轮数据被用完前就覆盖 `As` / `Bs`（**这是"共享内存 + 双同步"模式的核心正确性要求**）。
8. 计时用 `cudaEvent`（GPU 侧时间戳，比主机 `clock()` 准确），先跑一次预热，再跑 20 次取平均。
9. 程序同时打印**解析求出的 DRAM 流量**与据此算出的算术强度，便于与 Roofline 模型（见 4.4 节）对照。

【并行机制与性能解说】

- **硬件上怎么并行**：`grid = (64, 64)` 个 block 被分发到 GPU 的多个 SM 上；每个 block 内的 256 个线程被组织成 8 个 **warp**（每 32 线程一个 warp），由 SM 的 warp scheduler 每拍发射一个 warp。对 `As[ty][k] * Bs[k][tx]` 这条语句，同一 warp 内 32 个线程的 `ty` / `tx` 取值使它们访问**不同**的共享内存地址；由于共享内存被划分为 32 个 bank，若同一 warp 的 32 个线程落在 32 个不同 bank 上就是**无冲突（conflict-free）**的一拍访问；若地址对齐不当（例如 `TILE` 是 32 的倍数且按列访问），多个线程会落到同一 bank，产生 **bank conflict**，把一次访问串行化成多拍。这是 tiled GEMM 里最常见的性能陷阱之一（工程上的标准修法是给 `As` 的行宽加一个 padding，例如 `__shared__ float As[TILE][TILE + 1]`）。
- **数据类型与 restrict**：`__restrict__` 告诉编译器三个指针互不别名，使编译器可以对全局访存做更激进的调度；若去掉它，编译器可能因为"`C` 可能别名 `A`"而不敢重排访存。
- **Work / Span / 并行度**：

量	值	说明
Work	$N^3 = 1.074 \times 10^9$ 次 MAC（= $2N^3 = 2.147 \times 10^9$ FLOP）	与算法无关的总工作量
Span（关键路径）	$N = 1024$ 次 MAC	单个输出元素的累加链长度（每轮 tile 1 次 + 线程内串行累加）；这是任何分块方案都改不掉的**算法下界**
并行度 = Work/Span	$N^2 = 1.049 \times 10^6$	恰好等于"同时活跃的线程总数"（$N/TILE \times N/TILE \times TILE^2$）

解读：并行度百万量级，远超任何 GPU 的物理线程容量，所以**瓶颈绝不在并行度**。真正的瓶颈是**每个活跃线程要为 1 次 MAC 运多少字节**——也就是算术强度。这解释了为什么加速器设计的第一性原理是"减少数据搬运"而不是"增加并行度"。

- **工作量与数据搬运的定量关系（按 `N = 1024, TILE = 16, fp32` 推导）**：网格有 $(N/TILE)^2$ 个 block；每个 block 需要读一整行 panel $A[TILE \times N]$ 与一整列 panel $B[N \times TILE]$，即 $2 \cdot TILE \cdot N$ 个元素。总 DRAM 流量（每轮 tile 都从全局内存重新取，理想化为无 L2 命中）：

$元素数 = (N/TILE)^2 \times 2 \cdot TILE \cdot N = 2N^3/TILE = 2 \times 1.074e9 / 16 = 1.342e8$ 个元素 $字节数 = 1.342e8 \times 4\ B = 0.537\ GB$ $算术强度\ AI = 2N^3\ FLOP / (4 \times 2N^3/TILE)\ B = TILE/4 = 4.0\ FLOP/byte$

这是**低估**（没有算 L2 命中，真实 DRAM 流量会更小），但结论方向正确：$AI = TILE/4$ **只与分块边长成正比**。A100 的 fp32 峰值 19.5 TFLOP/s、HBM 带宽 ~1555 GB/s，其 **ridge point = 19.5e12 / 1555e9 ≈ 12.5 FLOP/byte**。$TILE = 16$ 时 $AI = 4.0 < 12.5$，**核函数是访存受限的**：理论最短时间 = $0.537\ GB / 1555\ GB/s = 0.345\ ms$（用 N=1024 的规模），而计算只需 $2.147e9/19.5e12 = 0.110\ ms$——访存时间是计算的 3.1 倍。若把有效分块做到 $TILE = 64$（需要"寄存器分块"：每个线程算 4×4 = 16 个输出，用 256 个线程覆盖 64×64 的块），AI 升到 $16 > 12.5$，**核函数转为计算受限**，此时才可能接近峰值。若再上 tensor core（fp16 输入、fp32 累加，A100 上 312 TFLOP/s），ridge point 升至 $312e12/1555e9 = 200\ FLOP/byte$，需要**多层复用 + 算子融合**（TMA 异步搬运 + 大 tile + 把相邻算子融进同一条流水）才可能喂饱——这正是讲义"异步访存 + 计算单元间通信 + 融合"这张清单的由来。

3.3 示例三：脉动阵列模拟——把"全局访存"换成"邻居通信"

这个示例用纯 C 实现 2.5 节的 $y = Wx$ 脉动阵列，并**统计两种实现的访存次数**，从而把"专用化换来的是什么"变成可数的量。

```c
/* 编译: gcc -O3 -std=c11 systolic.c -o systolic
 * 运行: ./systolic
 *
 * 说明: -O3 让内层小循环完全展开; 这个程序的目的是"数访存次数",
 *       所以刻意不做任何缓存优化, 让每次装载都被显式计数。 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define N 8
#define TMAX (2 * N - 1)      /* 波前深度: 填充 N 拍 + 排空 N-1 拍 */

/* 全局计数器: 从"权重/输入存储"(视作片外或全局共享存储)装载一个数 */
static long g_loads = 0;
/* 全局计数器: PE 与 PE 之间的局部搬运次数 */
static long g_pe_hop = 0;
/* 全局计数器: 直接作用于累加器的访存次数(读改写) */
static long g_acc_rmw = 0;

/* ---------------- 朴素实现: 每个输出都独立地从存储取操作数 ---------------- */
static void naive(const float W[N][N], const float x[N], float y[N])
{
    for (int i = 0; i < N; ++i) {
        float acc = 0.0f;
        for (int j = 0; j < N; ++j) {
            g_loads  += 2;    /* 读 W[i][j] + 读 x[j]      */
            g_acc_rmw += 1;   /* 累加器读改写               */
            acc += W[i][j] * x[j];
        }
        y[i] = acc;
    }
}

/* ---------------- 脉动阵列: 权重驻留 + x 向右流动 ---------------- */
static void systolic(const float W[N][N], const float x[N], float y[N])
{
    float acc[N];
    float xr[N + 1];          /* 波前寄存器: xr[i] 是 PE i 本拍看到的 x 值 */
    float wf[N][N];           /* 权重 FIFO 的快照(每个权重只搬运一次)      */

    for (int i = 0; i <= N; ++i) xr[i] = 0.0f;
    for (int i = 0; i < N; ++i) {
        acc[i] = 0.0f;
        for (int j = 0; j < N; ++j) wf[i][j] = W[i][j];
    }
    g_loads += (long)N * N;   /* 权重在整个计算期间只从存储搬运一次 */

    for (int t = 0; t < TMAX; ++t) {
        if (t < N) { xr[0] = x[t]; g_loads += 1; }  /* 每个 x 只装载一次 */
        else       { xr[0] = 0.0f; }                /* 排空期注入 0     */

        for (int i = 0; i < N; ++i) {
            const int j = t - i;            /* PE i 在 t 拍处理 W[i][j] * x[j] */
            if (j >= 0 && j < N) {
                acc[i] += wf[i][j] * xr[i]; /* 权重在 PE 内; x 来自左邻寄存器 */
                g_acc_rmw += 1;
            }
            if (i < N - 1) g_pe_hop += 1;   /* 本拍把值传给右邻 */
        }

        /* 波前向右推进一格(倒序拷贝, 保证读到的是本拍的旧值) */
        for (int i = N - 1; i >= 0; --i) xr[i + 1] = xr[i];
    }

    for (int i = 0; i < N; ++i) y[i] = acc[i];
}

int main(void)
{
    float W[N][N], x[N], y_naive[N], y_sys[N];
```

```
    for (int i = 0; i < N; ++i) {
        x[i] = 1.0f + 0.1f * (float)i;
        for (int j = 0; j < N; ++j) W[i][j] = (float)((i + 1) * (j + 1)) * 0.01f;
    }

    naive(W, x, y_naive);
    const long loads_naive = g_loads, hop_naive = g_pe_hop, rmw_naive = g_acc_rmw;

    g_loads = g_pe_hop = g_acc_rmw = 0;
    systolic(W, x, y_sys);
    const long loads_sys = g_loads, hop_sys = g_pe_hop, rmw_sys = g_acc_rmw;

    double maxerr = 0.0;
    for (int i = 0; i < N; ++i)
        maxerr = fmax(maxerr, fabs((double)y_naive[i] - (double)y_sys[i]));

    printf("N = %d, 波前深度 Span = 2N-1 = %d 拍, 总 MAC 数 = %d\n\n",
           N, TMAX, N * N);
    printf("%-28s %10s %10s\n", "计数器", "朴素实现", "脉动阵列");
    printf("%-28s %10ld %10ld\n", "从存储装载次数", loads_naive, loads_sys);
    printf("%-28s %10ld %10ld\n", "累加器读改写次数", rmw_naive, rmw_sys);
    printf("%-28s %10ld %10ld\n", "PE 间局部搬运次数", hop_naive, hop_sys);
    printf("%-28s %10ld %10ld\n", "存储系统访问总计", loads_naive + rmw_naive,
           loads_sys + rmw_sys);
    printf("\n最大误差 = %.3e (两实现结果一致)\n", maxerr);
    return 0;
}
```

【代码做什么？】

1. `naive` 按 `y[i] = Σ_j W[i][j]·x[j]` 实现，每做一次 MAC 计 2 次"装载"（读权重、读 `x`）与 1 次"累加器读改写"。
2. `systolic` 用一个长度为 `N+1` 的 `xr[]` 数组模拟"波前寄存器"：`xr[i]` 表示 PE `i` 本拍看到的 `x` 值。每拍先在最左端注入 `x[t]`（排空期注入 0），再让所有 PE 做一次 MAC（PE `i` 用 `W[i][t-i]`），最后把 `xr[]` **倒序**向右搬一格。权重 `wf[N][N]` 只是在开始时装载一次（计入 `N²`），此后驻留在 PE 内。
3. 两次运行分别累加三个全局计数器：**存储装载次数、累加器读改写次数、PE 间局部搬运次数**，最后并排打印，并校验两种实现结果一致（最大误差应为 0）。

【并行机制与性能解说】

- **硬件上怎么并行**：`N` 个 PE **空间上并排放置**，每拍同时做一次 MAC。它不是"同一条指令作用在 32 条通道上"（SIMD），而是"**N 个独立的小电路各自持有自己的权重、各自累加**"，控制是**分布式**的（每个 PE 只需要"权重 FIFO 给我下一个 w"和"左邻给我下一个 x"两个事件）。这正是 2.5 节对比表中"数据驱动（wavefront）"、"局部通信"、"分布式控制"的硬件含义。
- **Work / Span / 并行度（`N = 8`）**：

量	值
Work	`N² = 64` 次 MAC
Span（波前深度）	`2N−1 = 15` 拍
并行度 = Work/Span	`64/15 ≈ 4.27`（**峰值可用并行度是 N = 8 个 PE**）
稳态利用率	`N² / (N·(2N−1)) = 64/(8×15) = 53%`；`N` 越大越趋近 `N/(2N−1) → 50%`

解读：`Work/Span ≈ 4.27` 小于 PE 数 8，说明**填充/排空（fill/drain）就是效率上限**，不是算术量不够。要让利用率接近 100%，必须**连续喂入多个 `x` 向量**（把前一次的排空期用后一次的填充期填满）——这正是真实加速器用流水线/多批次（batching）掩盖启动开销的一般手法。

- **访存行为的对比（`N = 8`，即程序实际输出的数字）**：朴素实现 `2N² = 128` 次装载 + `N² = 64` 次累加器读改写 = **192 次存储系统访问**；脉动阵列 `N² + N = 72` 次装载 + `N² = 64` 次本地累加 = **136 次**，外加 `(N−1)(2N−1) = 105` 次 **PE 间局部搬运**。按表 2.4 的能耗数值估算单次 MAC 的代价：

`朴素（操作数来自 1mm 外的片上 SRAM，26 pJ / 64 bit）：2 次装载 × 26 pJ + 1 次累加读改写 × 26 pJ + FP 乘加 20 pJ = 98 pJ / MAC　朴素（操作数实际来自 LPDDR，1200 pJ / 64 bit）：2 × 1200 pJ + 20 pJ ≈ 2420 pJ / MAC　脉动阵列（权重驻留、x 来自左邻连线、acc 在本地寄存器）：稳态：每次 MAC 伴随 (N−1)/N ≈ 1 次邻居传递 1 × 26 pJ（按片上 1mm 内 SRAM 的上界估）+ 20 pJ = 46 pJ / MAC　含填充/排空：实测 105 次传递 / 64 次 MAC = 1.64 次每 MAC 1.64 × 26 pJ + 20 pJ ≈ 63 pJ / MAC ⇒ 同样 64 次 MAC：朴素(命中 DRAM) ≈ 155 nJ；脉动阵列 ≈ 4.0 nJ（含填充/排空），稳态下则只有 ≈ 2.9 nJ ⇒ 约 39×（稳态约 53×）的能效差`

> **这正是讲义"CPU 只有约 1% 的能量花在有用计算上 ⇒ ~100× 机会"的微观版本**：把"回寄存器堆/回内存取操作数"换成"邻居手递手"，能耗就能掉一到两个数量级。注意这里的代价是**牺牲通用性**——这个阵列只会算 `y = Wx`，遇到分支、不规则访存、稀疏结构就完全失效。

---

4. 性能模型与复杂度分析

4.1 模型一：技术缩放——为什么"每代只能点亮 70% 的晶体管"

假设与推导（补充讲义 Beckmann 给出的代数）：设一代之后 `N' = 2N`（摩尔定律）、`C' = C/√2`、`V' = V/√2`、`f' = √2 f`（几何缩放，面积减半、线度乘 `1/√2`）。

表 4.1：三种性能模型的数值对比（归一化初始代 = 1）

模型	表达式	每代倍率	Gen1	Gen2	Gen3	Gen4
乐观	`N·f`	`2√2 ≈ 2.83×`	2.83	8.00	22.6	64.0
现实	`√N·f`	`2.00×`	2.00	4.00	8.00	16.0
悲观	`f`	`√2 ≈ 1.41×`	1.41	2.00	2.83	4.00

Dennard 缩放（1974–2003）：`Power' = N'C'V'²f' = 2N·(C/√2)·(V/√2)²·(√2 f) = N·C·V²·f = Power` ⇒ **功耗恒定**，多出来的晶体管可以全部用上。

Dennard 终结后（~2003 起）：电压不能再按 `1/√2` 下降（能带隙等物理限制），于是

```
Power' = N'·C'·V'²·f' = 2N·(C/√2)·V²·(√2 f) = 2·Power        ← 功耗翻倍
但散热约束要求 Power' / Power = 1
⇒ N'·f' = √2 · N·f
⇒ 可用于"新增算力"的晶体管比例 = √2 / 2 = 70.71%
⇒ 每代约 29.3% 的新晶体管无法被点亮 = 暗硅 (dark silicon)
```

数值算例（暗硅的规模）：假设 Gen0 有 `N = 10⁹` 个晶体管、频率 `f = 1.0`，则 Gen1 有 `2 × 10⁹` 个晶体管，但功率预算只允许 `N'f' = √2 × 1.0 = 1.414`。若把频率保持不变（`f' = f = 1.0`），可用晶体管数只有 `1.414 × 10⁹`，即 **2×10^9 中有 `5.86 × 10⁸` 个（29.3%）必须长期关断**。若选择"频率提高 `√2` 倍"（`f' = 1.414`），则只能同时点亮 `1.0 × 10⁹` 个晶体管——**新增的一整代晶体管（10^9 个）全部变成暗硅**，性能只涨 1.41×。这就是"必须专用化"的定量压力：**既然不能把所有晶体管都做成通用的，那就把一部分做成只在特定任务上开机、但能效高 100 倍的专用单元。**

4.2 模型二：资源受限的 Amdahl 定律与"非对称为什么赢"

用 2.1 节的公式，取 `n = 16`、`perf(r) = √r`、`perf(1) = 1`。

表 4.2：加速比数值表（按公式计算，`n = 16`，`perf(r) = √r`）

`r`（每个核的资源 / 胖核的资源）	核数（对称）	对称加速比 `f=0.90`	对称 `f=0.95`	对称 `f=0.99`	非对称加速比 `f=0.90`	非对称 `f=0.95`	非对称 `f=0.99`
1	16	**6.40**	**9.14**	**13.91**	6.40	9.14	13.91
2	8	6.66	8.38	10.57	7.75	10.31	**14.03**
4	4	6.15	6.96	7.77	**8.75**	**10.77**	13.21
8	2	5.14	5.39	5.60	8.44	9.49	10.53
16	1	4.00	4.00	4.00	4.00	4.00	4.00

（示例展开 `f = 0.90, r = 4`：对称 `= 1/(0.10/2 + 0.90/(4×2)) = 1/(0.05 + 0.1125) = 6.15`；非对称 `= 1/(0.10/2 + 0.90/(2 + 16 − 4)) = 1/(0.05 + 0.064286) = 8.75`。`f = 0.90, r = 2` 非对称 `= 1/(0.10/1.41421 + 0.90/(1.41421+14)) = 1/(0.070711+0.058388) = 7.75`。`f = 0.99, r = 2` 对称 `= 1/(0.01/1.41421 + 0.99/(8×1.41421)) = 1/(0.007071+0.087500) = 10.57`。）

从表里能读出四条结论（这几条就是本讲前一半的骨架）：

1. **对称设计没有一个"正确的核大小"**：三种 `f` 下对称设计的最优 `r` 分别是 `1 / 1 / 1`，但曲线很陡——`f = 0.99` 时从 `r = 1` 的 `13.91×` 掉到 `r = 8` 的 `5.60×`，跌了一半多。讲义的原话是 "No good single choice for core size!"

2. **非对称设计在"串行段不可忽略"时明显更好**：`f = 0.90` 时非对称最优 `8.75×`（`r = 4`）比对称最优 `6.40×` 高 **37%**；`f = 0.95` 时 `10.77×` 比 `9.14×` 高 **18%**；曲线也更平坦（`r` 从 2 到 8 都能保持在 8.4× 以上）。讲义原话："**Speedup is much better & less sensitive to core size!**"
3. **但当 `f → 1` 时，非对称的优势会缩小到几乎消失**：`f = 0.99` 时对称最优 `13.91×`、非对称最优 `14.03×`，只差 0.9%。原因是此时串行项（`(1-f)/perf(r) = 0.0071`）已经小于并行项（`0.0642`），"把资源花在胖核上"换来的是并行吞吐的净损失。这是一个重要的现实提醒：**异构设计的收益与工作负载的串行比例强相关**，所以讲义才反复强调"芯片设计时必须准确理解工作负载"，而"系统无法随使用习惯和新算法自适应"。
4. **两者都仍受并行段限制**：`r = 16`（只剩一个大核）时无论对称还是非对称都掉到 `4.00×`——把资源全押在串行性能上同样会输。**异构的正解是"一个胖核 + 尽可能多的小核"**，而不是"全胖"或"全瘦"。

> 补充讲义还给出了 manycore 版本的对比：纯 manycore 的加速比是 `√x / ((1-s) + s/P)`（`x` = 核大小，`P = n/x`），并注明 "**Speedup is surprisingly awful unless program is ⩾99.9% parallel!**"。这与 4.5 节加速器时代的 99.9% 门限是同一个数学。

4.3 模型三：能效的"算术强度"与能耗墙

把表 2.4 的 pJ 数值变成一个可计算的能量模型：

```
E = N_ops × e_op + N_bytes × e_byte
  e_op    = 20 pJ / FLOP        (浮点运算)
  e_byte  = 1200 pJ / 8 B = 150 pJ/B   (LPDDR)
  e_byte  =   26 pJ / 8 B = 3.25 pJ/B  (片上 SRAM, 1 mm 距离)

能量平衡点 (计算能耗 = 搬运能耗):
  AI* = e_byte / e_op = 150 / 20 = 7.5 ops/byte = 15 FLOP/byte   (LPDDR)
  AI* = 3.25 / 20 = 0.1625 ops/byte = 0.325 FLOP/byte            (片上 SRAM)
  ⇒ 片上数据的"能量价格"只有 DRAM 的 26/1200 ≈ 1/46
```

数值算例（按讲义给定的 pJ 数值推导）：一个 `256 MiB` 的 float32 数组（`2.684 × 10⁸` 字节，即 `6.711 × 10⁷` 个元素），做**单次遍历**（每元素 1 次 FMA），内存带宽 20 GB/s。

```
遍历字节数          = 256 MiB = 2.684 × 10⁸ B
64 位(8 B)读取次数  = 2.684e8 / 8 = 3.355 × 10⁷ 次
搬运能耗            = 3.355e7 × 1200 pJ = 4.026 × 10⁻² J = 40.3 mJ
计算能耗            = 6.711e7 × 20 pJ = 1.342 × 10⁻³ J = 1.34 mJ
搬运 : 计算 能耗比  = 40.3 / 1.34 ≈ 30 : 1
                    (等价地: 每 8 个元素搬一次的能耗 1200 pJ,
                     对比 8 次 FMA 的 160 pJ ⇒ 7.5 : 1;
                     此时算术强度只有 2 FLOP / 4 B = 0.5 FLOP/byte)
理论最短时间        = 2.684e8 B / 20 GB/s = 13.42 ms
平均功率            = 40.3 mJ / 13.42 ms ≈ 3.0 W   (> 整个移动 GPU 约 1 W 的预算)
```

（若按十进制把 `256 MB` 解释为 `2.56 × 10⁸` 字节，则最短时间为 `12.8 ms`，平均功率约 `3.1 W`——结论不变。）

讲义给出的同源数字：以 10 GB/s 从内存读数据 ≈ **1.6 W**（用本笔记参数推导：`10e9/8 × 1200 pJ = 1.5 W`，与讲义吻合）。**结论**：移动端一个"每字节只算一次"的 kernel，光是把数据搬进来就已经超过整块 GPU 的功耗预算——**所以优化能效的第一手段是提高算术强度（分块、融合、压缩），而不是提高频率或加核。**

4.4 模型四：Roofline——"分块到底要多大才够"

Roofline 模型把可达性能写成

```
Attainable(FLOP/s) = min( Peak, AI × BW )
AI = 算术强度 (FLOP/byte)
ridge point = Peak / BW      (超过这个强度才算"计算受限")
```

表 4.3：两台 GPU 的 ridge point（用讲义给出的峰值与公开的 HBM 带宽）

机器	峰值算力	HBM 带宽	Ridge point	含义
A100（SIMD fp32）	19.5 TFLOP/s	~1555 GB/s	**12.5 FLOP/byte**	普通 SIMD 核要求的算术强度
A100（tensor core fp16→fp32）	312 TFLOP/s	~1555 GB/s	**200 FLOP/byte**	专用单元把门槛抬高 16 倍

机器	峰值算力	HBM 带宽	Ridge point	含义
H100（tensor core fp16）	989 TFLOP/s	~3.35 TB/s	**295 FLOP/byte**	需要极端的数据复用与融合

数值算例（承 3.2 节的分块 GEMM，`N = 4096`）：

```
Work = 2N³ = 2 × 6.87e10 = 1.374 × 10¹¹ FLOP

分块 T=16（每线程 1 个输出，累加器在寄存器）:
  DRAM 元素 = 2N³/T = 2 × 6.87e10 / 16 = 8.59 × 10⁹ 个 float
  DRAM 字节 = 3.44 × 10¹⁰ B = 34.4 GB
  AI        = 1.374e11 / 3.44e10 = 4.0 FLOP/byte   (< 12.5 ⇒ 访存受限)
  访存时间  = 34.4 GB / 1555 GB/s = 22.1 ms
  计算时间  = 1.374e11 / 19.5e12  =  7.05 ms
  ⇒ 实际时间 ≈ 22.1 ms（若无重叠），等效算力 = 1.374e11/22.1e-3 = 6.2 TFLOP/s = 峰值的 32%

分块 T=64（寄存器分块：每线程 4x4 = 16 个输出）:
  DRAM 字节 = 34.4 GB / 4 = 8.59 GB
  AI        = 16.0 FLOP/byte  (> 12.5 ⇒ 计算受限)
  访存时间  = 5.52 ms  <  计算时间 7.05 ms
  ⇒ 可以逼近 19.5 TFLOP/s 的理论峰值，只是需要足够的 occupancy 来隐藏延迟

若要喂饱 tensor core（ridge = 200 FLOP/byte）:
  需要 AI ≥ 200，即片上有效复用度 ≥ 200/(2 B/元素) = 100 个元素级复用
  ⇒ 单级 shared memory 分块做不到（128×128 的 fp16 分块只有 AI = 64 FLOP/byte）
  ⇒ 必须叠加：多级 tiling（shared → register/TMEM）+ TMA 异步搬运 +
     算子融合（把 Softmax、Mask、Dropout 融进同一条 tile 流水）
```

> **这一段量化分析就是讲义那句 "When logic is free, memory dominates" 的数字版本**：A100/H100 里 94%—98% 的峰值算力都在 tensor core 上，而要把这些算力兑现，**唯一的路是把数据搬运最少化**——分块、异步、融合、片上暂存（scratchpad/TMEM），也就是本讲所有专用化手段的共同名字。CS149 讲义用 "Hardware Lottery"（Sara Hooker）描述这一正反馈循环：**TPU 擅长稠密矩阵乘（`OI ∝ n`）→ 人们设计 Transformer 这种"矩阵乘友好"的模型 → Transformer 主导 → 硬件进一步为矩阵乘专用化**。

4.5 模型五：加速器时代的 Amdahl 定律——为什么门限是 99.9%

假设：某加速器把覆盖到的那部分代码加速 `S_acc` 倍。整体加速比仍由 Amdahl 给出：`Speedup = 1 / ((1 − f) + f/S_acc)`。

表 4.4：1000× 加速器下的覆盖率敏感度（按公式计算）

被加速器覆盖的比例 `f`	整体加速比	未被覆盖的 1% 造成的损失
90%	**9.9×**	1000× 的单元几乎白建
99%	**91.0×**	还差 11× 到理论上限
99.9%	**500.3×**	快接近上限
99.99%	**909.2×**	基本饱和

这就是讲义里两次出现的同一句话：**"To scale, ≥99.9% of the program must run on the accelerator (déjà vu...)"** —— 与 manycore 时代 `≥99.9% parallel` 的门限完全同构。

数值算例（多个加速器的边际收益，按公式计算）：基线时间 = 1，逐步往 SoC 上加加速器：

步骤	新增单元	加速比	覆盖的时间占比	剩余时间	整体加速比	边际收益
基线	—	1×	—	1.00000	1.00×	—
+ DNN ASIC	1000×	覆盖 95%	95%	`0.05 + 0.95/1000 = 0.05095`	**19.6×**	19.6×
+ 视频编码器	100×	覆盖剩余 2.5%	2.5%	`0.025 + 0.025/100 + 0.00095 = 0.02620`	**38.2×**	1.95×
+ 压缩/排序单元	10×	覆盖最后 2.5%	2.5%	`0.025/10 + 0.00025 + 0.00095 = 0.00370`	**270×**	7.1×

读表要点：

- 第一个加速器虽然最快（1000×），但因为剩余 5% 的拖尾，整体只拿到 19.6×——**加速器的绝对倍数不等于整体收益**；

- 第二个加速器只有 100×，边际收益 1.95×；第三个只有 10×，边际收益反而高达 7.1×，因为它**消灭了当前的瓶颈项**（`0.025 / 0.0262 ≈ 95%` 的剩余时间都在它身上）；
- 讲义因此提出两个尖锐的问题：**"How many accelerators do we need?（收益迅速递减，但前期工程成本巨大）"** 和 **"Are accelerators a scaling strategy or a one-off boost?（加速器是可扩展的战略，还是一次性的提升？）"**；
- 表里没有计入的**加速器自身开销**（启动延迟、主机↔设备搬运、同步）恰恰是现实中最常吃掉收益的部分（见第 6 节）。

4.6 模型六：编程性与专用化的权衡清单

表 4.5：加速器时代的开放问题（补充讲义原话整理）

维度	问题	后果
Amdahl	需要 ≥99.9% 的代码跑在加速器上才能扩展	有多少计算**值得**被加速?
编程	怎么为加速器写代码、编译代码?	可编程性还有希望吗?
OS	如何把"机器上有哪些加速器"发布给软件？（抽象是什么？）	调度器/运行时需要新的抽象
编译器 / PL	如何把程序**映射**到加速器上?	主流做法是 **DSL 与框架**：把程序员限制在框架懂的高层操作上
DSL 的代价	**非库代码会成为 Amdahl 瓶颈； 如果某个重要操作框架里没有怎么办?**	组合性差、逃生舱难做
通用加速	可编程的非冯·诺依曼架构（CGRA 等）"无指令 ⇒ 无取指/译码开销，极端异步"	但一次映射=一次编译，通用性来自"可重构"而非"可换指令"

---

5. 关键要点

1. **异构的第一性原理是"用最合适的工具干活"，而不是"让所有单元都忙"。** 同构系统的最优策略是"让每个处理器一直忙"，异构系统的最优策略是"**让每个任务落在最合适的单元上**"。这要求算法能被分解成"高度数据并行段 / 难以并行段 / 宽 SIMD 友好段 / 分支发散段 / 可预测访存段 / 不可预测但缓存友好段"，进而映射到 CPU 大核、CPU 小核、GPU、DSP、固定功能单元上。**这个分解工作被推给了程序员**，这是本讲最重要的"代价"。
2. **能效已经取代峰值性能成为第一设计约束，而专用化是提高能效的唯一大幅手段。** Dennard 缩放终结 ⇒ 功耗恒定而晶体管翻倍 ⇒ 每代约 **29.3%** 的晶体管成为暗硅。经验量级：吞吐优化核（GPU）**~10× perf/W**、域专用加速器 ~20×、FPGA ~50×（学界仍有争议）、固定功能 ASIC **~100–1000× perf/W**（ASIC 用 ~1/1000 的面积、~1/100 的能量达到一个 CPU 核的性能）。代价是：**能效越高，越难编程**。
3. **专用化的收益不是"算得更快"，而是"搬得更少"。** 一旦去掉通用处理器的指令/寄存器堆开销（CPU 只有约 1% 的能量花在有用计算上），**数据搬运就成为能耗主导**：浮点运算 ~20 pJ，而读 64 位片外 DRAM 要 ~1200 pJ（**片上 1 mm 处的 SRAM 只要 ~26 pJ**）。因此加速器的结构特征——**scratchpad 大容量片上存储、数据在运算单元之间直接路由、取消寄存器堆、权重驻留、脉动波前、算子融合、异步搬运（TMA/TMEM/mbarrier）**——全都是同一条原则的不同实现：**把复用做到极致，让片外流量最小化**。
4. **"核该多大"没有全局最优解，但"一个胖核 + 尽可能多的小核"通常最优。** 在 `perf(r) = √r`、`n = 16` 的资源受限 Amdahl 模型下，对称设计的加速比对核大小敏感且最优值随 `f` 漂移（`f = 0.95` 时 16 个小核给 9.14×，4 个胖核只给 6.96×）；非对称设计（1 个 `r = 4` 的胖核 + 12 个小核）在 `f = 0.95` 时给到 **10.77×**，且曲线平坦。大核"局部低效、全局高效"——它抬高的是整个程序加速比的**上限**（Amdahl 的串行分母）。
5. **专用化的门限极陡：99.9%，而且在加速器时代会再次出现。** 对 1000× 的加速器，覆盖 90% 只得到 9.9×，覆盖 99% 得到 91.0×，覆盖 99.9% 才到 500.3×。因此工程上有两条推论：①**加速器必须做成"平台/流水线"而不是"孤立的函数替换"**（否则非库代码立刻成为瓶颈）；②**多个加速器的部署顺序应当按"当前瓶颈项"来排**，而不是按"加速倍数最大的排前面"。

---

6. 常见陷阱与注意事项

- **陷阱一：以为"异构 = 更快"，忽略了配比错误会反过来拖垮整机。** 讲义给出的经典反例（Molnar 2010）：假设工作负载中 10% 是光栅化，你给固定功能光栅化单元分配了 1% 的芯片面积，而实际需要的是 `1.2%`。于是**光栅化成为瓶颈，占芯片 99% 的可编程处理器全部空等**——讲义的原话是"99% of the chip runs at 80% efficiency!"（吞吐只剩 `1/1.2 ≈ 83%`，与"约 80%"一致）。因为固定功能单元**一旦流片就无法加速**（"Work balance must be anticipated at chip design time"），业界的趋势是**保守地超额配置**，而这会削弱专用化的优势。教训：**异构系统的性能由最弱的那个专用单元决定，而不是由最强的那个决定。**
- **陷阱二：把"像素级并行"写成了"列切分"，踩上伪共享（false sharing）。** 在 3.1 节的图像并行例子里，如果按列分块（每个线程负责若干列），相邻线程会在**同一缓存行**上写不同的字：每次写都要把该行升级为独占，导致缓存行在两个核之间来回迁移（ping-pong），实测可能掉数

倍性能。正确做法是按行（或按 ≥64 B 对齐的分块）切分，或给每线程数据加 padding 使其独占缓存行。同理，在 CUDA 里 `__shared__ float As[TILE][TILE]` 按列访问会出现 **bank conflict**（一个 warp 内多个线程落到同一 bank），标准修法是 padding 成 `[TILE][TILE + 1]`。

- **陷阱三：忘了共享内存/scratchpad 的"双屏障"约束。** 3.2 节的 `__syncthreads()` 少了任何一个都会出错：少了装载后的那次同步，线程会读到别人还没写入的 tile；少了计算后的那次同步，跑得快的 warp 会在别人还没用完 tile 时就把它覆盖掉（**数据竞争**）。这是 GPU 上"用片上暂存换带宽"模式的**必付正确性成本**，也是程序员最容易因"去掉一次同步看着更快"而引入的 bug。
- **陷阱四：只用算术强度（或只用带宽）单指标评估，忽略另一侧的天花板。** Roofline 的两侧都要算：`TILE = 16` 的 GEMM 在 A100 上 `AI = 4 FLOP/byte < 12.5` 的 ridge point，因此**再怎么优化指令调度都是在访存墙上撞**；反过来，把 `TILE` 提到 64 之后 `AI = 16`，此时若 occupancy 不足（寄存器压力导致活跃 warp 太少），又无法隐藏访存延迟。**"带宽受限"和"延迟受限"要分开诊断**，不要一概归因于"内存不足"。
- **陷阱五：忽略"重新计算 vs 存下来再读"的能耗不对称，以及多算一点反而更快的现象。** 讲义明确指出：**为能效优化时，重新计算往往优于"存起来再读回来"**；同时"计算更少"也是趋势之一——**一个比串行版本做更多工作的并行算法，即使更快，也未必是好的选择**（因为处理器开着就在烧静态功耗）。这与"并行算法常常用冗余计算换并行度"的直觉相反，需要在能效目标下重新审视（例：N-body 用更贵的算法换更少的通信，可能反而省能量）。
- **陷阱六：低估加速器的固定开销，让 Amdahl 的漂亮数字落空。** 4.5 节算出"三个加速器 → 270×"，但这个模型**没有计入**：主机↔设备的数据搬运（PCIe/NVLink 带宽通常远低于 HBM）、kernel 启动延迟、队列同步、以及"非库代码"的拖尾。讲义因此把"**如何把可用加速器发布给软件（OS 抽象）**"和"**非库代码会成为 Amdahl 瓶颈**"列为两个未解决的开放问题。工程上的对策是把加速器组织成**异步流水线**（LD/ST/AO 重叠，见 2.6 节的图），让搬运时间与计算时间互相隐藏，而不是让它们串起来相加。

---

7. 思考题（带答案）

思考题 1：用资源受限的 Amdahl 定律决策"买哪种机器"

题目：设 `n = 32` 单位芯片资源，`perf(r) = √r`，程序的 `f = 0.98`。有三个候选设计：(a) 32 个小核（`r = 1`）；(b) 8 个中等核（`r = 4`，对称）；(c) 1 个胖核（`r = 8`）+ 24 个小核（`r = 1`，非对称）。 分别计算加速比，并解释为什么 (c) 最优。如果程序的 `f` 降到 `0.90`，结论会变吗？

【答案】

- (a) 对称 `r = 1`：`perf(1) = 1`，核数 `= 32/1 = 32`。 `S = 1/((1−0.98)/1 + 0.98/(32×1)) = 1/(0.02 + 0.030625) = 1/0.050625 = 19.75×`
- (b) 对称 `r = 4`：`perf(4) = 2`，核数 `= 32/4 = 8`。 `S = 1/(0.02/2 + 0.98/(8×2)) = 1/(0.01 + 0.06125) = 1/0.07125 = 14.04×`
- (c) 非对称 `r = 8`：`perf(8) = √8 = 2.8284`，胖核 1 个 + 瘦核 `32 − 8 = 24` 个。 `S = 1/(0.02/2.8284 + 0.98/(2.8284 + 24)) = 1/(0.007071 + 0.036520) = 1/0.043591 = 22.94×`

结论：(c) 最优（`22.94×`），比纯小核 (a) 好 16%，比对称胖核 (b) 好 63%。 **原因**：`f = 0.98` 意味着串行部分占 `2%`，这在 `19.75×` 的加速比下已经是**主导项**（`0.02` 对比并行项 `0.030625` 占了 39.5% 的剩余时间）。非对称设计用 `perf(8) = 2.83` 把串行项从 `0.02` 压到 `0.007071`（降 64.6%），同时只牺牲 8 份资源（并行项从 `0.030625` 升到 `0.036520`，升 19%）——**用 19% 的并行吞吐损失换来 65% 的串行时间削减，净赚**。这就是"大核局部低效、全局高效"。

若 `f = 0.90`： – (a) `1/(0.10 + 0.90/32) = 1/(0.10+0.028125) = 7.80×` – (b) `1/(0.10/2 + 0.90/16) = 1/(0.05+0.05625) = 9.41×` – (c) `1/(0.10/2.8284 + 0.90/26.8284) = 1/(0.035355+0.033546) = 14.51×`

结论**仍然 (c) 最优，但已不是"押注大核"**：此时 (a) 最差（7.80×）的原因恰恰是"串行项 `0.10` 相对并行项 `0.028` 太大"，而 (b) 之所以反超 (a)，是因为它用一个 `perf(4) = 2` 的核把串行项减半（`0.10 → 0.05`）而并行项损失不大。**这说明最优的资源混合比强烈依赖 `f`**——这正是讲义"芯片设计时必须极其准确地理解工作负载"以及"系统无法随使用习惯和新算法自适应"的量化根据。工程上唯一的缓解办法是**让架构在宽范围内都"不太差"**（(c) 在两种 `f` 下都是最好的，曲线平坦），而不是针对某一个 `f` 精确调优。

思考题 2：给一个真实的 kernel 做"专用化收益"预算

题目：某移动端应用在一颗通用 CPU 小核上耗时 100 ms，时间分布为：DNN 推理 80 ms、视频解码 15 ms、其余（音频处理 + 控制逻辑 + 数据搬运）5 ms。已知：移动 GPU 能效是 CPU 核的 10×（且推理能完美映射到数据并行），固定功能视频解码器能效是 CPU 的 200×，音频 DSP 是 CPU 的 20×。请计算：①全用 GPU 加速推理的整体加速比；②再加上视频解码器的整体加速比；③再加上音频 DSP 的整体加速比；④用表 2.4 的能耗数值估算"音频那 5 ms 里如果有 3 ms 是数据搬运，改成片上复用后能省多少能量"（假设搬运的是从 LPDDR 读 64 位、频率为每 64 位一次搬运、3 ms 内共搬运 200 MB）。

【答案】

> **一条必须遵守的记账规则**：Amdahl 定律里的 `f` 必须始终以**原始基线时间**为 1 来定义。把"已经被加速过的时间"再拿去做分母，是这类题最常见的错误。下面每一步都用原始占比 `0.80 / 0.15 / 0.03 / 0.02` 代入（音频 3 ms 与控制逻辑 2 ms 需从原来的 5 ms 里拆出来）。

①只加速 DNN 推理（GPU，10×）：`S = 1/(0.80/10 + 0.15 + 0.03 + 0.02) = 1/(0.08 + 0.20) = 1/0.28 = 3.57×`，总时间 `100/3.57 = 28.0 ms`。

②再加视频解码器（200×）：`S = 1/(0.80/10 + 0.15/200 + 0.03 + 0.02) = 1/(0.08 + 0.00075 + 0.05) = 1/0.13075 = 7.65×`，总时间 `13.08 ms`。边际收益 `7.65/3.57 = 2.14×`——因为此时**真正的瓶颈已经变成视频解码**（`0.15` 是剩余时间 `0.20` 中的 75%）。

③再加音频 DSP（20×）：`S = 1/(0.80/10 + 0.15/200 + 0.03/20 + 0.02) = 1/(0.08 + 0.00075 + 0.0015 + 0.02) = 1/0.10225 = 9.78×`，总时间 `10.23 ms`。边际收益 `9.78/7.65 = 1.28×`。此时剩余时间已经全部落在**推理后的 0.08**与**不可加速的 0.02** 上，继续加加速器的空间已经很有限（上限 `1/0.02 = 50×`）。

④音频段数据搬运的能耗节省：200 MB = `2.0 × 10⁸ B`，按 8 B 一次读数计，共 `2.5 × 10⁷` 次 LPDDR 访问。 – 从 LPDDR 搬运（1200 pJ/次）：`2.5e7 × 1200 pJ = 3.0 × 10⁻² J = 30.0 mJ`； – 若改成从**片上 SRAM**（26 pJ/次 64 位）复用：`2.5e7 × 26 pJ = 6.5 × 10⁻⁴ J = 0.65 mJ`； – **节省 ≈ 29.35 mJ，降低约 46 倍**（`1200/26 ≈ 46.2`）。 – 对照 iPhone 6 的 7 Wh 电池（`7 × 3600 = 25200 J`）：单看 29.35 mJ 微不足道，但**如果这是"always-on"功能、每秒重复 10 次**，那就是 `0.294 W`——**接近整块移动 GPU 的功耗预算（约 1 W）的 30%**。这解释了讲义为什么给"always-on 语音唤醒"配一个专用 ASIC：不是因为它快，而是因为**只有把每次检测的能量压到微焦级，才能一直开着**。

小结：①单独加速占比最大的那一块收益可观但受 Amdahl 限制（10× 的单元只换来 3.57×）；②**加速非瓶颈部分是浪费**——如果先加了 200× 的解码器再去加 GPU，几乎拿不到收益，因为瓶颈已经换了位置；③**真正的收益来自"按顺序消灭当前瓶颈"**，这与 4.5 节的边际收益表完全一致（那里的第三个单元只有 10× 却贡献了 7.1× 的边际收益）；④**能效优化的杠杆不只是"换更快的单元"，更在于"减少数据搬运"**——第 ④ 问显示把片外访问换成片上复用可以直接把这一段的能量砍掉约 46 倍，而这与加速单元的倍数无关。

思考题 3：什么时候不该用专用硬件？以及"多少个加速器才够"

题目：你的团队正在设计一颗面向"通用桌面应用"的 SoC，候选的加速单元有：DNN 加速器（100×，覆盖典型工作负载的 60%）、视频编解码器（100×，覆盖 10%）、图像处理 ISP（50×，覆盖 5%）、安全/加解密引擎（30×，覆盖 3%），其余 22% 是控制逻辑与不可加速的通用代码。(a) 全部加上后，整体加速比是多少？(b) 依次累加四个单元，各步的整体加速比与边际收益分别是多少？从中能得出什么部署顺序的建议？(c) 如果这 22% 的"通用代码"里有 15% 其实是**某个库函数之外的、自研的图算法**，用现有 DSL 无法表达，你会怎么处理？请结合讲义的开放问题回答。(d) 讲义说"tendency is to be conservative, and over-provision fixed-function components (diminishing their advantage)"——请用本讲的光栅化反例解释这句话的含义，并说明为什么"超额配置"在商业上是理性的。

【答案】

(a) 全部加上：基线 = 1.00，各段原始占比 `0.60 / 0.10 / 0.05 / 0.03 / 0.22`。`S = 1/(0.60/100 + 0.10/100 + 0.05/50 + 0.03/30 + 0.22) = 1/(0.006 + 0.001 + 0.001 + 0.001 + 0.22) = 1/0.229 = 4.37×`

(b) 逐步累加：

步骤	新增单元	累计剩余时间	整体加速比	边际收益
基线	—	`1.000`	1.00×	—
+ DNN	100× 覆盖 60%	`0.006 + 0.40 = 0.406`	**2.46×**	2.46×
+ 视频	100× 覆盖 10%	`0.006 + 0.001 + 0.30 = 0.307`	**3.26×**	1.32×
+ ISP	50× 覆盖 5%	`0.307 + 0.001 − 0.05 = 0.258`	**3.88×**	1.19×
+ 加解密	30× 覆盖 3%	`0.258 + 0.001 − 0.03 = 0.229`	**4.37×**	1.13×

建议：①**收益迅速递减（2.46 → 1.32 → 1.19 → 1.13）**，第 2 个之后的边际收益都在 1.2× 左右，而每个加速器的"前期工程成本巨大"（讲义："Large up-front engineering costs"）——所以**加加速器不该按"谁加速倍数大"排序，而该按"谁覆盖的时间多"排序，并且要在边际收益跌破门槛时停手**；②本例中最终被 `0.22` 的通用部分彻底锁死（哪怕前四项全部做到无穷快，上限也只有 `1/0.22 = 4.55×`，而当前已经拿到 4.37×，**后两个单元合起来只贡献了 4.5% 的加速比余量**）。这是"加速器是可扩展战略还是一次性提升"这个问题的量化答案：**一旦通用部分成为约束，继续堆加速器就完全不经济**。

(c) 自研图算法的处理：讲义明确指出 DSL 路线的两个问题——**"Problem: Non-library code becomes Amdahl bottleneck"** 与 **"Problem: What if an important operation is missing?"**。本例里那 15% 正是这两个问题同时发作：即便把所有加速器都加上，`0.22` 的通用部分也会把整体加速比死死压在 4.55× 以下。可行的对策（按本讲给出的方向）：

1. **改算法而不只是改硬件**：Dally 的话——"从专用硬件取得高加速比与高能效通常需要**修改底层算法**"。把图算法重构成"能用框架懂的高层操作（如稀疏 GEMM、scan、segmented reduce）表达"的形式，把它拉进已有加速器的覆盖范围；这也是讲义强调 "the available mixture of resources can dictate choice of algorithm" 的含义。
2. **做领域专用而不是函数专用**：为图计算单独配一个域专用单元（讲义在 SoC 卡通图里就画了一个 "Graph processor?"），把覆盖比例从"库函数"提升到"领域"。
3. **走通用加速路线（CGRA / 可重构数据流）**：讲义把 CGRA 列为"general-purpose acceleration"的方向——**无指令流 ⇒ 无取指/译码开销，极端异步**，用"重新映射"代替"重新取指"。代价是一次映射等于一次编译，且每个算子会被"钉"在固定的硅上。
4. **接受它，并把它做成"不阻碍流水线"的部分**：如果这 15% 必须留在 CPU 上，那就把它与加速器流水**重叠**（异步 LD/ST/AO），确保它不成为串行瓶颈；同时把它的**数据搬运**降到最少（3.3 节/陷阱五：重新计算优于存取）。

(d) "保守超额配置"的含义：光栅化反例说明**固定功能单元一旦配少了就无法补救**——你需要在设计时预判 10% 的工作负载需要 1.2% 的面积，但只给了 1%，结果**占芯片 99% 的可编程处理器全部空等**（讲义原话 "99% of the chip runs at 80% efficiency!"）。代价是**乘性的**：不是"光栅化慢 20%"，而是"整块芯片的有效吞吐掉 20%"，且**没有任何软件手段能救**（"System cannot adapt to changes in usage over time, new algorithms, etc."）。所以从风险管理角度，给固定功能单元留出超额余量是理性的——即便这会浪费面积、削弱专用化本应带来的优势（这正是讲义所说的 "diminishing their advantage"）。这也解释了为什么现代 SoC 会选择**"可编程 + 专用"的混合**：把"可能变化的量"交给可编程单元（GPU/DSP/CGRA），只把"绝不会变的量"（视频编解码格式、加密原语、传感器前端）交给固定 ASIC，从而在**面积效率**与**适应未来负载变化的能力**之间取得平衡。

Lecture 19: Virtual Memory

1. 章节标题与概述

Lecture 19: Virtual Memory（虚拟内存：地址翻译这条"隐藏的访存流水线"）

- **本讲核心问题**：程序里的一次 `load` 只用了一个地址，但**这个地址不是内存真正的地址**——从虚拟地址（VA, Virtual Address）到物理地址（PA, Physical Address）的翻译发生在**每一次访存**上，而它必须和"取数据"这件事本身竞争延迟、带宽和缓存容量。本讲要回答三件事：(1) 虚拟内存这套抽象为什么值得（讲义 slide 7 给出两条理由：**efficient memory usage** 与 **programmability**）；(2) 硬件如何在没有软件介入的情况下完成翻译（TLB、page walk、TLB 与 cache 的相互作用），以及**当翻译成为瓶颈时，用什么技术把延迟压回去**（多级 TLB、页走缓存 PWC、把翻译放进数据 cache、大页）；(3) 当页表本身被修改时，**如何让 N 个核上私有的 TLB 保持一致**——这就是 **TLB shootdown**，也是并行程序里最容易被忽略的一种"隐式同步"。最后讲义把虚拟内存抽象再叠一层，讲到 **虚拟机（Virtual Machine）** 与嵌套页表。
- **涉及的主要硬件/软件机制**：
- **翻译层**：MMU（Memory Management Unit，内存管理单元）；页（page）与页框；**页表（page table）**与**页表项（PTE, Page Table Entry）**；**present bit 为 0 → page fault（缺页）**，由 OS 处理（讲义区分 **major page fault**：页不在内存，需要从存储取；**minor page fault**：页已在内存但尚未映射进该进程地址空间，两者都由 OS 处理）。
- **硬件缓存翻译**：**TLB（Translation Lookaside Buffer，翻译后备缓冲）**、TLB 命中/缺失、**page walk（页走）**、x86-64 的**四级基数页表（multi-level radix page table）**（CR3 → PGD → PUD → PMD → PTE，9/9/9/9/12 位切分），以及页走的"指针追逐"本质。
- **缓解延迟的四种技术**（讲义 slide 25/34 反复列出的清单）：**multi-level TLBs**、**Page Walk Cache（PWC，也叫 MMU cache）**、**caching translations in data caches**、**larger translations（huge pages，大页）**。
- **翻译一致性（translation coherence）**：硬件负责页走、TLB 填充/替换、出错抛异常；OS 负责创建/更新 PTE 并**维护跨核一致性**——因为**硬件不保证 TLB 与 cache/页表之间的一致性**（讲义 slide 41 原话：*no cache coherence between TLBs and Caches*）。跨核传播靠 **TLB shootdown**：锁 PTE → 计算受害者核集合 → 发 **IPI（Inter-Processor Interrupt，处理器间中断）** → 本地失效 → 其它核在中断处理里失效并回 **ack** → 收齐 ack 后解锁。
- **虚拟化扩展**：**SVM（System Virtual Machine，系统虚拟机）**、**VMM/hypervisor**、**guest VM**、**real memory** 这层新增的地址空间、**shadow page table（影子页表）** vs **nested page tables（嵌套页表，Intel EPT / AMD NPT，寄存器 EPTP）**、虚拟化扩展的目标（避免 flush TLB、设备 DMA、guest 处理设备中断等）。
- **在并行计算知识体系中的角色**：前几讲（缓存一致性、目录协议、内存一致性模型、同步）讨论的是**数据**在多个核之间怎么保持一致；本讲讨论的是**地址**在多个核之间怎么保持一致——两者是同一类问题在不同抽象层的复现：cache 有 per-core 副本所以需要 coherence 协议，TLB 也有 per-core 副本，但**它没有硬件协议，只有 OS 的 shootdown**，代价随核数**超线性**增长。同时，本讲是"内存墙"这条主线里最容易被忽视的一层：程序写得再并行，如果访问模式是"每页只碰几个字节"（图遍历、稀疏矩阵、哈希探测、随机指针追逐），那么**每次访存都要额外付一次地址翻译**，32 核的带宽优势会被翻译延迟吃掉。它也解释了并行程序里两个常见的实测怪象：为什么随机访问的带宽远低于流式带宽（不只

是 cache miss，还有 TLB miss + 页走），以及为什么"多线程 + 频繁 mprotect/munmap/fork"（用户态页表管理、GC 写屏障、COW）会突然变得极慢。

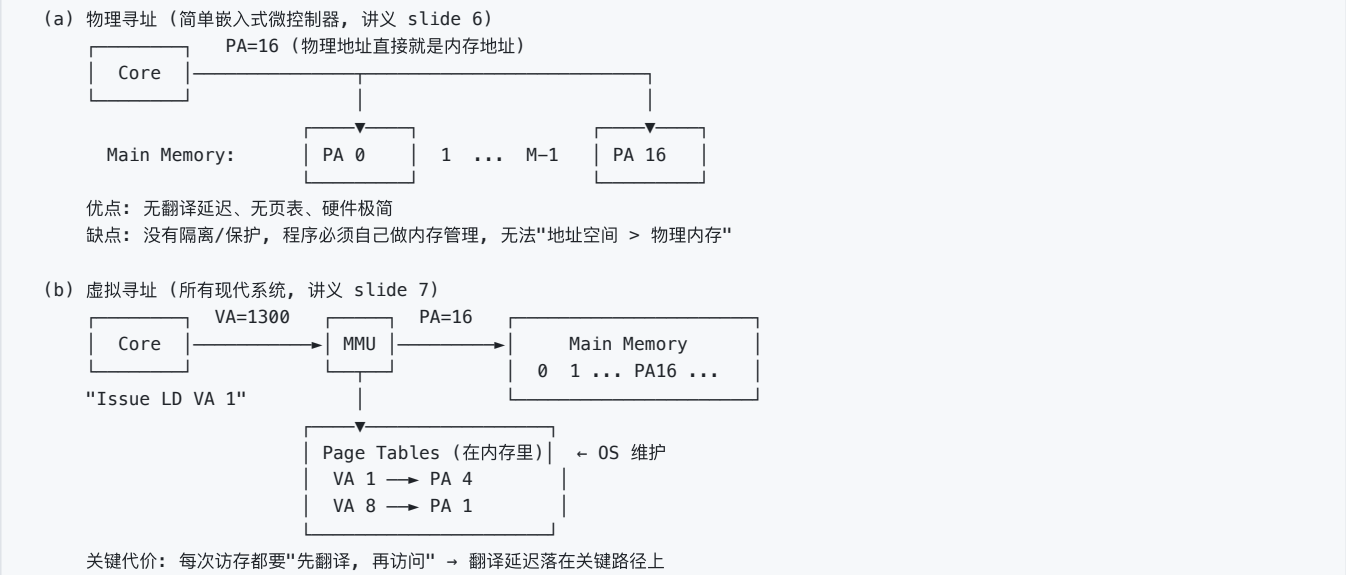
- **配套材料**：
- **Fall 2026 日程表**（https://www.cs.cmu.edu/~418/schedule.html）把 **Oct 9** 排为第 19 讲 **"Virtual Memory"**。该行的 slides/video 链接**目前仍被 HTML 注释包住**，注释文字为 "slides/video from a previous offering; uncomment when posted for Fall 2026:"，其中给出的链接是 `lectures/13_virtualmemory.pdf` 与一个 YouTube 视频。也就是说：**Fall 2026 尚未把本讲的幻灯片/录像正式挂到页面上**（页面当前不显示这些链接）。
- 但该 PDF 本身**在公开网络可直接下载**：https://www.cs.cmu.edu/~418/lectures/13_virtualmemory.pdf（实测 HTTP 200，1,927,441 字节，Last-Modified 2025-09-30）。本笔记的事实基础就是它——本地副本 `lectures/13_virtualmemory.pdf` 与逐页抽取文本 `extracted/13_virtualmemory.txt`（共 **59 页**，文件内标记 `===== [slide i/59] =====`）。因此本讲材料状态为：**讲义已公开（归入"上一学期沿用"的 PDF），但 Fall 2026 版尚未在日程表上正式发布；录像未发布。**
- 讲义首页写的是 **"CMU 15-418/15-618, Fall 2025 — Lecture 13: Virtual Memory"**：**讲义沿用历史学期的版本是正常现象**（同一份 PDF 在 Fall 2026 的日程表注释里被引用，文件名编号 13 与 Fall 2026 的第 19 讲不对应也是同一原因），**不是错误**。本笔记正文中标注的 slide 编号，均指这份 59 页 PDF 的页码（`i/59`）。
- **未公开部分**：历史学期讲义位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录**；Ed 讨论区、Autolab、Canvas 均**需要登录**；本讲录像（Panopto/YouTube）在 Fall 2026 日程表中被注释隐藏，属**未发布**。
- Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。
- 讲义第 3、4 页的"address translation wall"配图注明出自 **Zhao et al., "Contiguitas: The Pursuit of Physical Memory Contiguity in Datacenters," ISCA '23**（本文只复述该页要传达的定性结论，不复述图中的具体数值）；第 53 页 TLB shootdown 性能图注明出自 **Baumann et al., "The Multikernel: a New OS Architecture for Scalable Multicore Systems," SOSP '09**；第 54 页 "Further reading" 只给了一个 ASPLOS II (1987) 的 ACM DL 链接，未给标题。
- **本文档中所有标为"本机实测"的数据**都不是讲义内容，而是本文作者为一篇学习笔记所做的公开实验：机器为 **2 × AMD EPYC 7V13（64 核/颗，共 128 个硬件线程、无 SMT）、2 个 NUMA 节点、L3 共 512 MiB（16 个 32 MiB 实例）、503 GiB DRAM**，Linux + g++ 12.2.0，THP（透明大页）策略为 `enabled=always` / `defrag=madvise`；**该机为共享节点（实测期间 load average 约 5–7）**，因此绝对数值会随负载波动，请把它当作**数量级与相对趋势**的证据。所有代码示例都标注了完整的编译命令。

---

2. 核心概念与硬件/软件架构图解

2.1 从一个 `int array[10]` 说起：物理寻址 vs 虚拟寻址

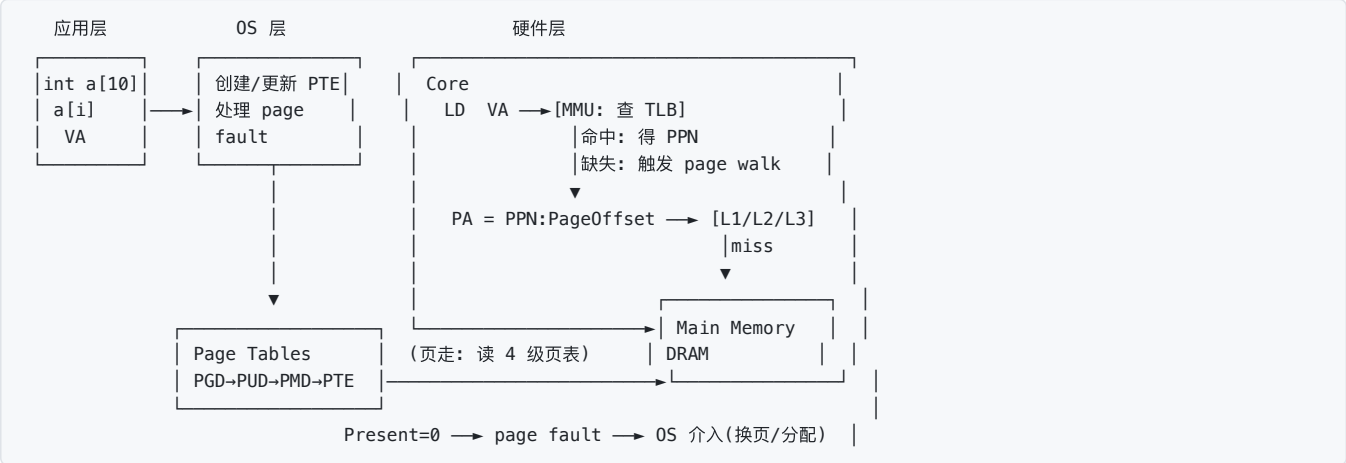
- **定义与目的**：讲义 slide 5 用一个最小例子开场：`int array[10]`——**一个值到底住在内存的哪里？处理器怎么找到它？** 这个问题有两个答案。答案一是**物理寻址（physical addressing）**：CPU 发出的地址就是 DRAM 的地址（讲义 slide 6 的嵌入式微控制器：Core 直接输出 PA）。答案二是**虚拟寻址（virtual addressing）**：CPU 发出 VA，经过 MMU 翻译成 PA 再访问内存（slide 7）。**所有现代系统（服务器、笔记本、手机、平板）都用后者**，讲义称之为 "one of the great ideas in computer science"。
- **直观解释（"它是什么？"）**：物理寻址像**直接报出门牌号**：你说"第 16 号楼"，邮差就送到第 16 号楼——简单，但整栋楼只有你能用，别人要住进来就得知道你还空着哪些房间。虚拟寻址像**酒店前台 + 房号映射**：每位客人拿到的房间号永远是"我的 301 房"（线性、连续、从 0 开始，**programmability**），而 301 到底对应大楼里哪个物理房间由前台（OS 的页表）决定；前台还可以把长期不用的客人行李寄存到仓库（**DRAM 只缓存 VA 空间的一部分 → efficient memory usage**），需要时再取回来。客人永远不需要知道真实房间号，也不能去动别人的房间（**isolation / protection**）。
- **两条收益（讲义 slide 7 明确给出）**：
- **Efficient memory usage（高效利用内存）**：物理 DRAM 只装得下虚拟地址空间的一部分，多出来的部分放到更慢更大的存储上（paging），进程的"地址空间"可以远大于物理内存。
- **Programmability（可编程性）**：每个进程拿到一个**从 0 开始的线性地址空间**，链接器/分配器不用关心物理布局；同时天然获得隔离与保护（每个进程有自己的页表）。
- **架构图解 D1：两种寻址模式的数据通路**



- **性能特征**：物理寻址下 `load` 的延迟 = cache/DRAM 延迟。虚拟寻址下**理想情况**延迟 = TLB 命中延迟 + cache/DRAM 延迟（TLB 命中可与 cache 索引并行，见 2.5）；**最坏情况**还要加上一次页走（可能 4 次串行访存）。这多出来的一层，就是本讲后半段所有优化技术的靶子。

2.2 页、页表、页表项与缺页

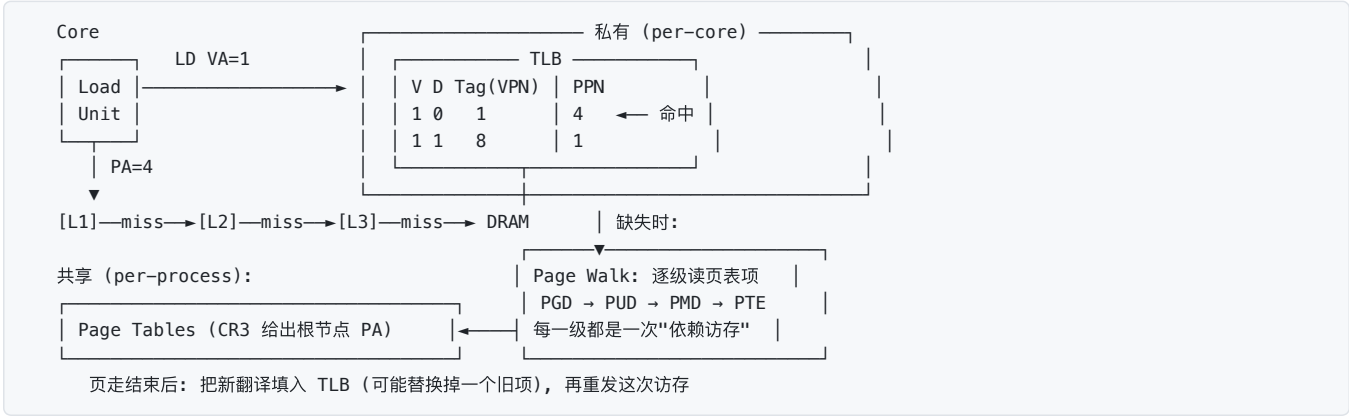
- **定义与目的**：虚拟内存抽象的三个动作（讲义 slide 11–14 的动画序列）：
- **把物理地址空间切成页（pages）**——固定大小，x86-64 的基础页是 4 KB；
- **用页表在页粒度上建立 VA→PA 映射**；
- **present bit 为 0 时触发缺页（page fault）**，OS 把页取进内存并更新页表项。
- **PTE 里有什么**：至少包括 **Present（有效）位**、**Dirty 位**、**权限位**（读/写/执行、user/supervisor）、以及 **Physical Page Number（PPN，物理页号）**。讲义 slide 22 的画法很关键：**TLB 项里存的是 `Valid | Dirty | Tag(VPN) | PPN`**，其中页内偏移（offset）**根本不参与翻译**，原封不动地从 VA 的低位复制到 PA 的低位——这也是后面 VIPT 能成立、大页能成立的根本原因。
- **直观解释（"它是什么？"）**：页表像**搬家时的"旧地址→新地址"对照表**，但表太大不能整本带着，于是切成"每个目录 512 条"的多级目录（见 2.4）；页内偏移像**门牌号**：无论整栋楼叫什么名字，你住的那间房在楼层里的第几号是不变的。缺页（major）像"行李还在仓库里，前台得派人去取"；缺页（minor，讲义 slide 11 的脚注）像"行李其实就在大堂，只是还没登记到你名下"。
- **架构图解 D2：一次访存的完整翻译路径（页表项在内存里）**



- **性能特征**：翻译的**粒度**是页（4 KB），而数据移动的**粒度**是 cache line（64 B）——**一次 4 KB 的翻译够用 64 条 64 B 的 cache line**（讲义 slide 17–20 反复强调的观察）。这带来**页粒度的空间局部性 + 时间局部性**：顺序扫描时每 512 次（4 KB / 8 B）访存只需要一次新翻译。反之，**"每页只碰一个元素"的访问模式（页步长、随机换页）会把翻译成本摊到每一次访存上**——这是本讲最重要的一条性能直觉。

2.3 TLB 与页走：把翻译缓存起来

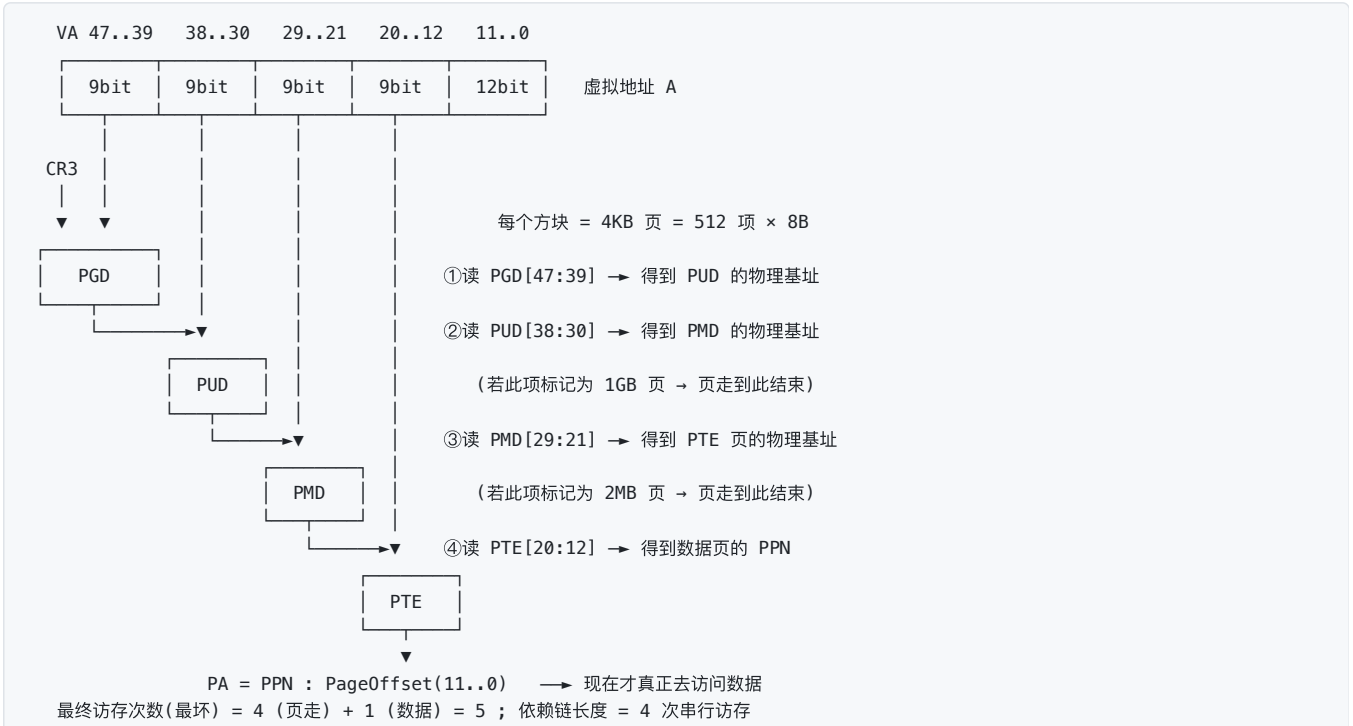
- **定义与目的**：**TLB（Translation Lookaside Buffer）**是一个小型的、通常全相联或高度相联的硬件表，缓存**最近的 VA→PA 翻译**（项里含 VPN、PPN、权限位、ASID/PCID 等）。**每次访存都从内存取翻译太贵**（讲义 slide 17 原话 "Fetching each translation on a load would be expensive!"），TLB 让绝大多数访存在 1 个周期内拿到 PPN。TLB 缺失（TLB miss）时，硬件（x86/ARM 的硬件页走器）或软件（部分 MIPS/RISC-V 实现）去**页表**里取翻译，这一步叫 **page walk（页走）**："Fetch entry from page table"（slide 19）。
- **直观解释（"它是什么？"）**：TLB 像你**手机里的常用联系人**：绝大多数电话（访存）直接拨号（命中），偶尔要翻通讯录（页走）。也像**前台桌边贴着的一张便签**，只记下最近来访客人对应的真实房间号；便签写满了就得擦掉一行（替换），查不到的客人就得去翻档案柜（页表）。注意两个细节：便签是**前台自己的**（per-core 私有），档案柜是**全楼共用**的（页表在共享内存里，由 OS 维护）。
- **"TLB 覆盖率"是判断性能的第一把尺子**：**TLB reach（覆盖范围） = 项数 × 页大小**。讲义 slide 17–20 的算例：4 KB 的翻译"enough for 64 cache lines of 64 bytes"。**同一个 TLB，页大小翻 512 倍，reach 就翻 512 倍**——这就是大页的杠杆（2.6 节）。
- **架构图解 D3：TLB 命中 / TLB 缺失 + 页走（讲义 slide 17–20 的动画）**



- **性能特征**：
- **TLB 命中**：几乎不增加关键路径（在 VIPT L1 里可与 cache 索引并行，见 2.5），延迟≈1–2 周期。
- **TLB 缺失**：多出一次页走。页走的**串行性**（每级地址依赖上一级结果）使它成为**关键路径上的 4 次访存**；但同时，**多个访存各自的页走是彼此独立的**，乱序执行可以把它们的延迟重叠起来（页走级并行，TLB-MLP），这正是实测中"单线程随机访问仍有 24 ns/hop、而不是 400 ns/hop"的原因（见第 3、4 节）。

2.4 x86-64 的四级基数页表：稀疏性与指针追逐

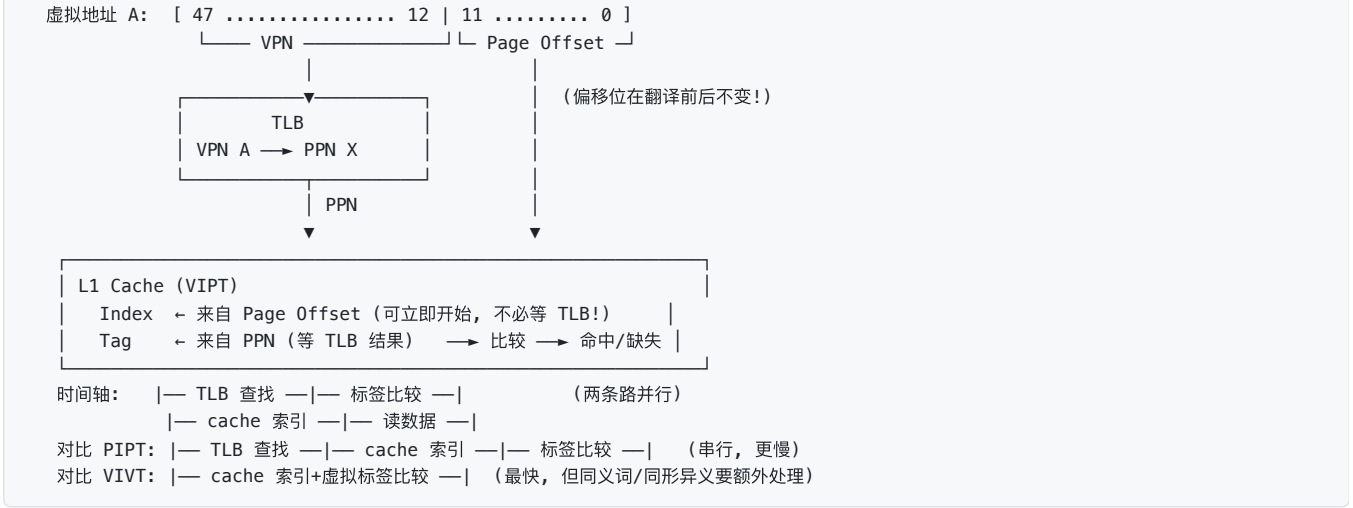
- **定义与目的**：现代处理器用**多级基数页表（multi-level radix page table）**（讲义 slide 26–33）。x86-64 把 48 位虚拟地址切为 **9 / 9 / 9 / 9 / 12**：`[47:39] → PGD`、`[38:30] → PUD`、`[29:21] → PMD`、`[20:12] → PTE`、`[11:0] → 页内偏移`。**每张页表恰好 4 KB，每个表项 8 字节 → 512 项 → 正好需要 9 位索引**（slide 28 的推导）。CR3 寄存器保存 PGD 的物理基址（进程切换时换 CR3）。
- **为什么分级？** 讲义 slide 29 把它和**覆盖范围**列在一起：一级页表覆盖 512 GB / 1 GB / 2 MB / 4 KB。**根因是稀疏性（sparsity）**：VA 空间（48 位 = 256 TB）远大于物理内存，绝大多数 VA 区间根本没有映射；单级页表要为整个 VA 空间准备表项（48 位 VA、4 KB 页 → 2^36 项 × 8 B = 512 GB 页表，不可能），而多级树只要**按需分配路径上的那几张表**（缺的整棵子树用"present=0"表示）。
- **代价**：**页走变成了指针追逐（pointer chasing）**——讲义 slide 30–33 用连续四张动画演示了这个过程：每读一个表项才知道下一个表在哪（它的 PPN 就在表项里），**四级 = 四次串行访存**。这是最坏情形；真实系统里绝大多数中间层被 PWC/数据 cache 兜住（见 2.6）。
- **大页如何嵌进这棵树**（讲义 slide 39）：PMD 项可以直接指向一个 **2 MB 页**（跳过 PTE 级），PUD 项可以直接指向一个 **1 GB 页**（跳过 PMD、PTE 级）——注意 slide 39 把 4 KB / 2 MB / 1 GB / 512 GB 的覆盖范围标在树上，说明**大页并没有改变树的结构，只是让更上层的项直接终结**。
- **架构图解 D4：四级页走（最坏情况，4 次依赖访存）**



- **性能特征**：一次**全冷**页走 = 4 次 DRAM 访问 ≈ 4 × 80 ns ≈ **320 ns**（假设 DRAM 延迟 80 ns），而且这 4 次是**严格串行**的（Span = 4 × 延迟，工作量为 4 次访存，**并行度 ≈ 1**）。若这 4 次正好都在 cache 里（PWC/数据 cache 命中），页走可以缩到十几纳秒甚至更少——这正是 2.6 节四种技术要争取的效果。

2.5 TLB 与 cache 的交互：PIPT / VIPT / VIVT

- **定义与目的**：翻译出来的 PA 要拿去查 cache，那么 **cache 用虚拟地址还是物理地址来索引/打标签？**（讲义 slide 21–23）。这不是实现细节，而是**延迟的关键**：如果 cache 必须等翻译完成才能开始查，那么**每次访存的 L1 延迟都会变成"TLB 延迟 + cache 延迟"**。
- **三种方案**（讲义 slide 23 的三行结论，本文补全其后果）：
- **PIPT（Physically Indexed, Physically Tagged）**：索引和标签都用物理地址。语义最干净（没有别名/同义词问题），但**必须等翻译完成才能访问 cache**（slide 23 原话 "Cannot access cache before TLB!"）。
- **VIPT（Virtually Indexed, Physically Tagged）**：**索引只用页内偏移的位**（因为偏移在翻译前后不变），标签用物理地址。于是 **cache 索引可以和 TLB 查找并行进行**（slide 23："No need to wait for translation"），翻译结果只用于最后的标签比较。**限制**：可用的索引位数被页内偏移位数（4 KB 页 → 12 位）卡死 → **cache 容量有上限**（这也是为什么 L1 通常是 32–48 KB、8–12 路，见第 7 节思考题 1）。
- **VIVT（Virtually Indexed, Virtually Tagged）**：完全不碰物理地址，但**同义词（synonym/alias）问题**：两个不同 VA 映射到同一个 PA 时，cache 里会有两份副本，一致性与刷新变得麻烦（slide 23："Synonyms!"）；此外还有"同形异义（homonym）"——不同进程的相同 VA 指向不同 PA，必须在进程切换时刷新或用 ASID 区分。
- **架构图解 D5：VIPT 的关键路径（翻译与索引并行）**



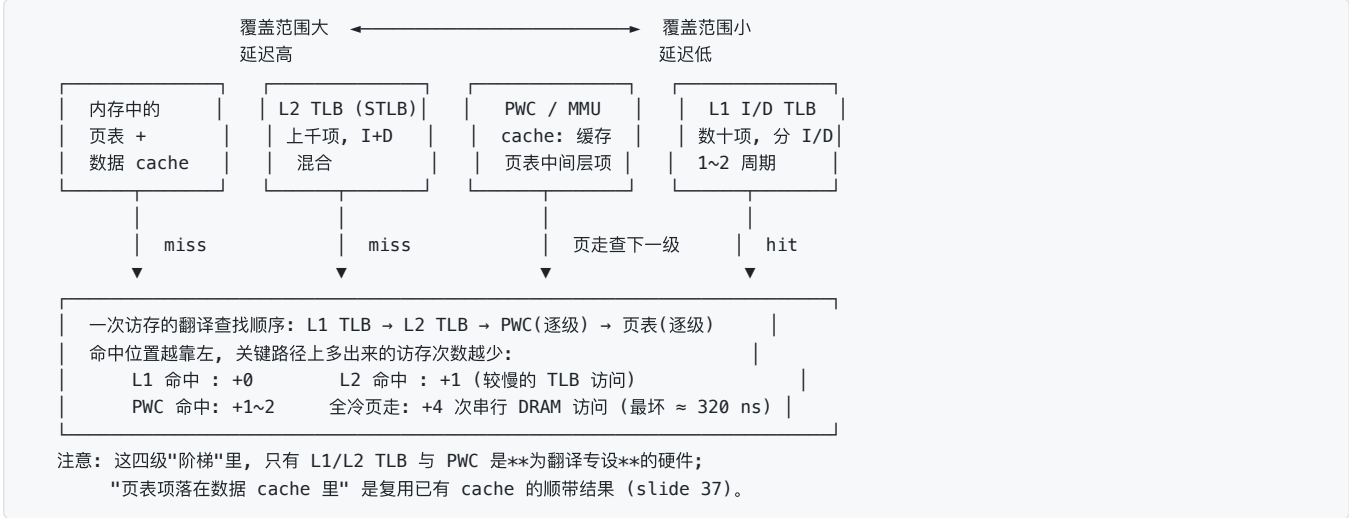
- **性能特征**（把三种方案放在一张表里）：

方案	索引来源	标签来源	是否要等翻译	别名/同义词风险	典型用法
PIPT	物理地址	物理地址	**是**（翻译在关键路径上）	无	大容量 L2/L3、部分 L1
VIPT	虚拟地址**但只用页内偏移位**	物理地址	否（索引与 TLB 并行）	无（前提：索引位 ⊆ 页内偏移位）	现代 L1（含本讲讨论的 L1）
VIVT	虚拟地址	虚拟地址	否	**有**（synonym；进程切换需 flush 或用 ASID）	早期/嵌入式实现

补充说明：VIPT 若**索引位超出了页内偏移**，就会退化成有同义词问题的方案——所以 VIPT 的"无别名"性质**只在** `index_bits + block_offset_bits ≤ page_offset_bits` **时成立**（4 KB 页时右边是 12）。第 7 节思考题 1 会用具体数字算这个上限。

2.6 把翻译延迟压回去：四种技术的共同逻辑

- **定义与目的**：讲义 slide 25 / 34 两次列出同一份清单，因为在多级页表时代，**页走本身可能就是"4 次串行内存访问"**，必须用**缓存 + 覆盖范围**两把武器去压缩它：
- **多级 TLB（multi-level TLB）**：L1 TLB 分 I/D（指令/数据各一个，条目少、延迟低），L2 TLB（STLB）**混合 I+D、条目多、延迟高**（讲义 slide 35 提到 "Intel i7 TLB structures"）。**收益**：利用翻译的局部性，让 L1 缺失时还有一层兜底；**代价**：额外硬件面积，及 L2 命中带来的额外几个周期。
- **页走缓存 PWC（Page Walk Cache，也叫 MMU cache）**：**缓存页表的中间层项**（PGD/PUD/PMD 项）。讲义 slide 36 的关键观察是：**上层项覆盖的地址范围巨大且局部性极好**——一个 PGD 项覆盖 512 GB、一个 PUD 项覆盖 1 GB、一个 PMD 项覆盖 2 MB，所以进程哪怕用了几百 GB，上层可能只有几十个不同的项。**收益**：把 4 级页走缩短成 1 级；**代价**：额外硬件，且只在"上层复用率高"时有收益。
- **把翻译放进数据 cache（caching translations in data caches）**：让页表项走**正常的 cache 层次**，而不是每次都去 DRAM。**收益**：页走中的某一级若在 L2/L3 命中，就省掉一次 DRAM 往返（讲义 slide 37："Leverage cache locality for translations … avoids DRAM access"）；**代价**：与数据争抢 cache 容量（页表项会污染 cache）。
- **大页（huge pages / larger translations）**：让**一条 TLB 项覆盖更大的地址范围**（4 KB → 2 MB → 1 GB）。**收益**：同样大小的 TLB，reach 提高 512× / 262144×，TLB 命中率大幅提升（讲义 slide 38："Fewer TLB entries & higher hit rate … Same TLB space but larger coverage per entry"）。**代价**：见 2.6 末尾与第 6 节。
- **架构图解 D6：翻译的四层"缓存阶梯"**



- **四种技术的对比（收益 / 代价 / 何时有用）**：

技术	机制	收益	代价 / 限制	什么时候最有用
多级 TLB	L1 I/D TLB + 更大的 L2 混合 TLB	L1 缺失有兜底，命中率显著上升	面积/功耗；L2 命中比 L1 慢几个周期	有局部性的常规负载
PWC / MMU cache	缓存页表**中间层**项	4 级页走 → 1 级	需额外硬件；只对上层复用有效	地址空间大但上层项很少的进程
翻译进数据 cache	页表项走正常 cache 层次	页走中的某级避免 DRAM	与数据争 cache；可能被挤出	翻译有复用、且 cache 足够大的负载

技术	机制	收益	代价 / 限制	什么时候最有用
大页（2 MB / 1 GB）	一条项覆盖更大范围	reach ×512 / ×262144	内部碎片、COW 放大、分配难、首次触碰更贵	TLB 压力大的随机访问负载

- **实测补充（本机，非讲义内容）**：讲义 slide 3–4 用 Contiguitas（ISCA '23）的图说明"地址翻译墙"——随着数据中心负载的地址空间膨胀、**物理内存连续性下降**，页走与 TLB 缺失的成本在系统性上升。这类"物理内存碎片化"的问题在这台共享机器上就能观察到：我们请求 1 GiB 的透明大页（`madvise(MADV_HUGEPAGE)`）时，`/proc/self/smaps` 显示只有 **72 MiB（7%）** 真正落成了 2 MB 大页；把区域缩到 32 MiB 才拿到 **32 MiB 全覆盖**。**这说明"用大页"不是一个可以假设的前提，而是一个需要用 `smaps` 验证的运行期事实**（第 3 节示例 1、2 都打印了这个验证值）。

2.7 翻译一致性（translation coherence）与 TLB Shootdown

- **定义与目的**：页表在**共享内存**里，任何核（严格说是 OS 在任何核上）都能改它；而 TLB 是**每个核私有的**。问题是：**改了 PTE，怎么让别的核知道？**（讲义 slide 40–42）
- **硬件的职责**（slide 41）：执行页走并找到翻译；**出错时抛异常**（例如 page fault）；**填充 TLB 并在冲突时替换 TLB 项**；**但硬件不提供 TLB 与 cache（以及页表）之间的一致性**（原话："But no cache coherence between TLBs and Caches"）。讲义的追问非常锋利：**"如果页的权限变了会怎样？"**
- **OS 的职责**（slide 42）：创建 PTE（把内存映射进地址空间）；**更新 PTE**（权限变化、页迁移、换页等）；以及**维护跨 TLB 的翻译一致性**——这一条硬件不管。
- **直观解释（"它是什么？"）**：把 TLB 想成**每个人手机里缓存的"联系人旧号码"**；页表是**公司通讯录**。HR 在通讯录上把一个号码改成新号（改 PTE）之后，**已经存了旧号的人不会自动更新**——必须挨个打电话通知（IPI），并且**等每个人都回复"已更新"**（ack）才能认为通讯录生效。为什么非要等所有人？因为**旧号码可能把信息送错人**：这正是讲义 slide 43 的场景——*"Change permissions to read-only"*：如果某个核还在用"可写"的旧翻译，它就能绕过刚刚设置的只读保护（安全性问题），或者写进一个已经回收给别人（或已经换出到磁盘）的物理页（**静默数据损坏**）。
- **流程（讲义 slide 45 的六步，逐字对应）**：
- 发起核上的 OS **锁定该页表项（lock PTE）**；
- OS **生成可能正在使用该 PTE 的核列表**；
- 发起核向其它核发送 **IPI（Inter-Processor Interrupt）**，要求它们使对应的 TLB 项失效；
- 发起核**先使本核的 TLB 项失效**，然后**等待 ack**；
- 其它核收到中断 → 执行中断处理程序：**使自己的 TLB 项失效，回送 ack**；
- 发起核**收齐所有 ack 后解锁 PTE**。
- **表 2-4：硬件 vs OS 的职责划分**（对照讲义 slide 41–42）

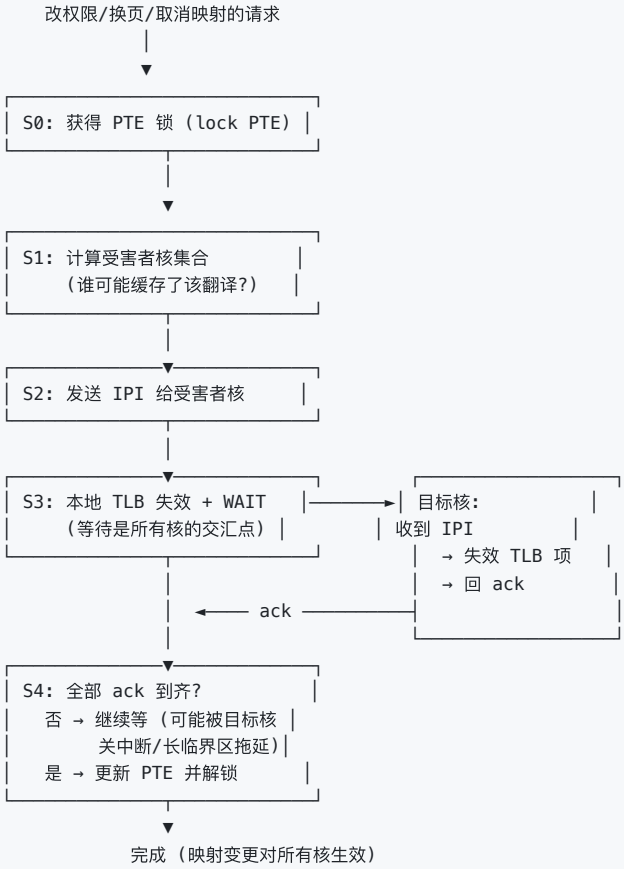
事项	硬件	OS
执行页走、定位翻译	✅	
出错时抛异常（page fault）	✅	处理异常（换页/分配/权限检查）
填充 TLB、因冲突替换 TLB 项	✅	
保证 TLB 与 cache/页表一致	❌（**没有**这种硬件一致性）	✅（必须软件干预）
创建 PTE、建立映射		✅
更新 PTE（权限、迁移、换页）		✅
跨核传播翻译变更（shootdown）		✅

- **表 2-5：TLB shootdown 六步的角色分工**

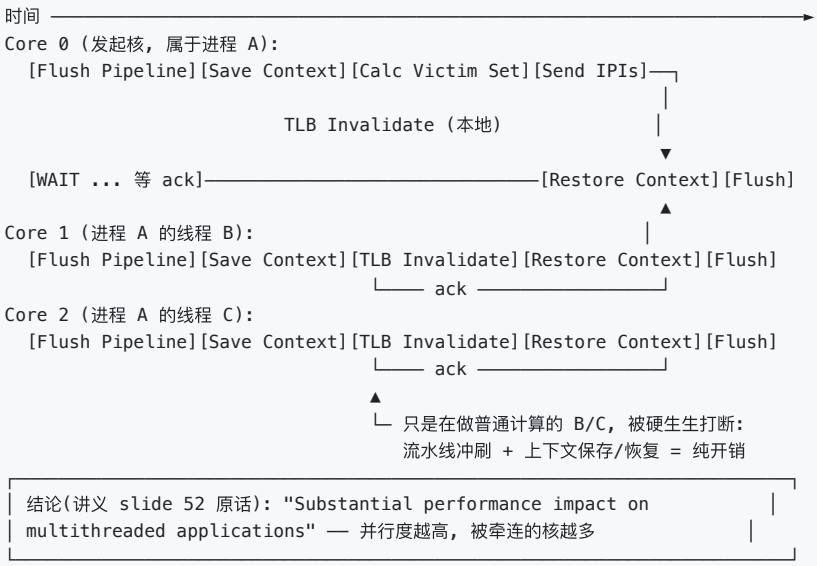
步骤	动作	执行者	说明
1	锁 PTE	发起核上的 OS	把"改映射"变成临界区
2	计算受害者核集合	发起核上的 OS	谁可能缓存了这个翻译
3	广播 IPI	发起核上的 OS	IPI = Inter-Processor Interrupt
4	本地 TLB 失效 + 等待 ack	发起核	等待是**串行**的 rendezvous

步骤	动作	执行者	说明
5	中断处理：失效 TLB + 回 ack	其它核上的 OS	每个目标核都要**打断当前工作**
6	收齐 ack 后解锁 PTE	发起核上的 OS	此时才可以安全复用物理页/更新权限

- **架构图解 D7：shootdown 的状态机（含失败/阻塞路径）**



- **架构图解 D8：shootdown 在今天的时间线（讲义 slide 52 的甘特图）**

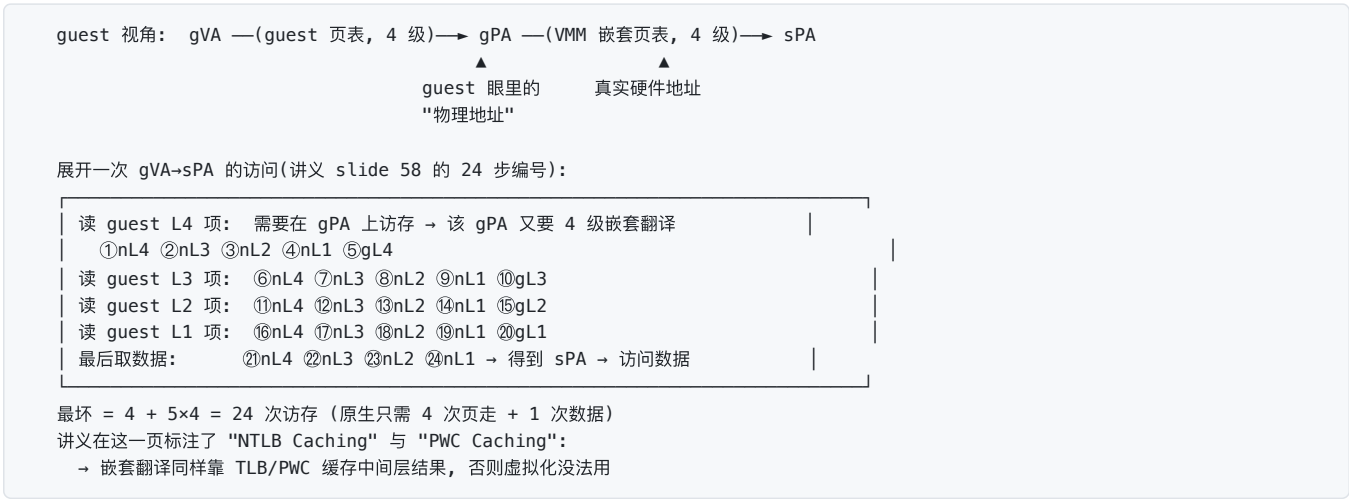


- **性能特征与实测（讲义 + 本机）**：
- 讲义 slide 53（引自 SOSP '09 的 Multikernel 论文）：**TLB shootdown 是昂贵操作，8 个或更多核时超过 10,000 周期，而且核越多越贵。**

- **本机实测（非讲义内容）**：用 `mprotect` 单页触发 shootdown，工作线程数 = 被牵连的核数（`shootdown.cpp`，见第 3 节示例 3）：0 个活跃核时 **2.79 µs/次**；8 核 **15.53 µs**（5.6×）；16 核 **47.24 µs**（16.9×）；32 核 **159.99 µs**（57.4×）。**增长明显是超线性的**（每多牵连一个核的边际成本从约 1.7 µs 涨到约 7 µs），与讲义的定性结论一致。这 160 µs ≈ **48 万周期**（按 3 GHz 折算），比讲义引用的 10,000 周期还高一个数量级——原因是本实验里目标核正在**紧密自旋**（最坏的中断打断场景），且这是"一页 + 32 个目标核"的组合。
- 与讲义 slide 59 呼应的一点（**公开知识补充**）：正因为 flush/shootdown 这么贵，现代 ISA 引入了 **PCID/ASID（地址空间标识）**与 **INVPCID** 之类的**按地址空间/按项失效**指令，避免进程切换或小改动导致"整表 flush"，虚拟化扩展也把"Avoid flushing TLB"列为目标（讲义 slide 59 明确列出）。

2.8 从虚拟内存到虚拟机：再来一层翻译

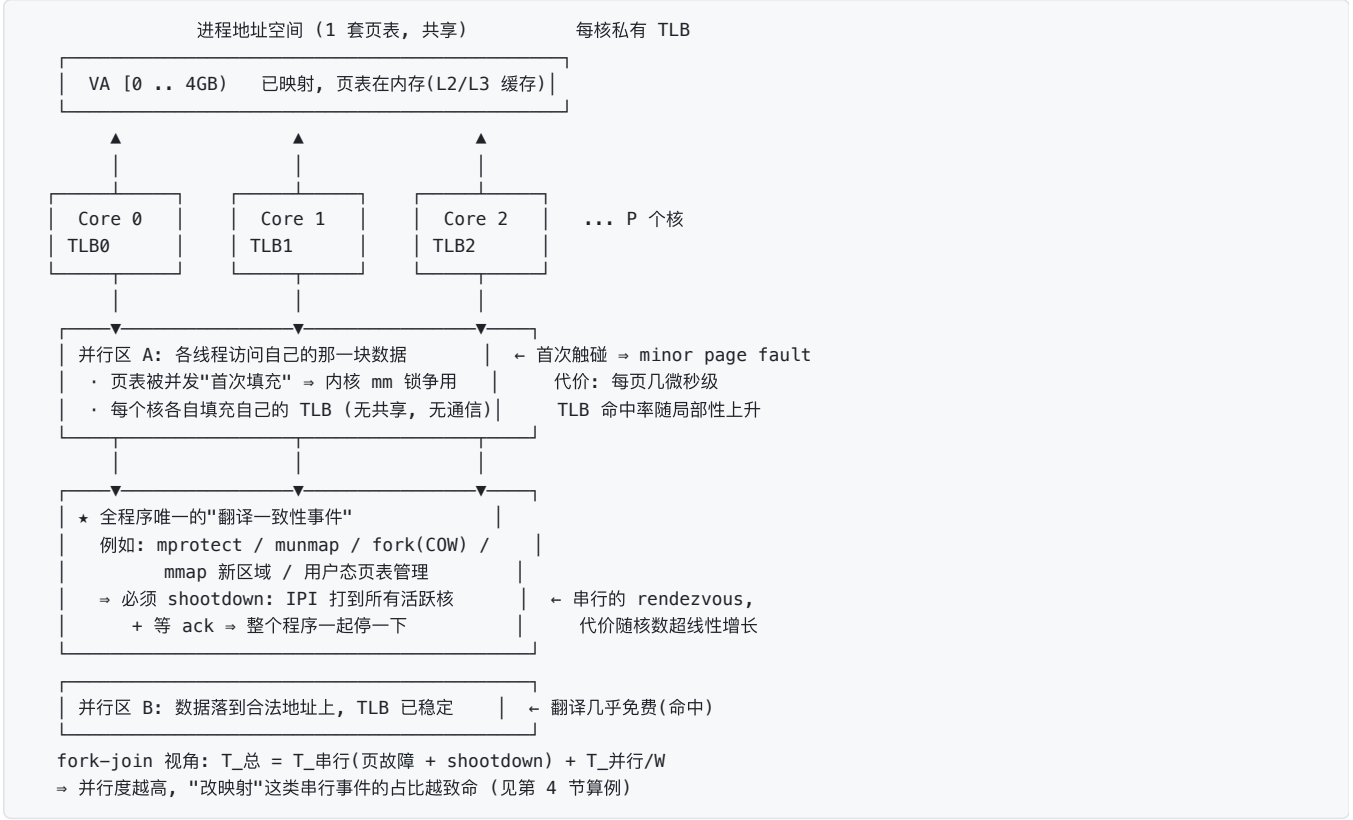
- **定义与目的**：讲义最后把虚拟内存的思想又叠了一层（slide 55–59）。**SVM（System Virtual Machine，系统虚拟机）**让**多个互不相关的用户共享硬件**，并提供隔离与安全；它甚至允许把**不同的 ISA 和 OS** 呈现给用户程序；实现它的软件叫 **VMM（Virtual Machine Monitor）/ hypervisor**，运行其下的每个虚拟机叫 **guest VM**。讲义指出这条路能成立的一个现实前提是**处理器足够快，使得额外开销可以接受**。
- **VMM 的要求**（slide 56）：guest 软件应当**表现得像跑在原生硬件上，且不能改变真实系统资源的分配**；VMM 要能**在 guest 之间做上下文切换**；因此硬件必须提供**系统态/用户态**与一组**特权指令**用于分配系统资源。
- **虚拟内存受到的影响**（slide 57–58）：
- 每个 guest OS 维护**自己的一套页表**；
- VMM 在"虚拟地址"与"物理地址"之间加了一层，叫 **real memory**；
- 传统方案：VMM 维护 **shadow page table（影子页表）**，把 guest 虚拟地址直接映射到真实物理地址，**代价是 VMM 必须侦测 guest 对自己页表的每一次修改**（讲义指出：如果"访问页表指针"本身是特权操作，这种侦测就自然发生——即 guest 写 CR3 会陷入 VMM）；
- 现代方案：**nested page tables（嵌套页表）**，即硬件两段翻译（Intel EPT / AMD NPT，根寄存器 **EPTP**），guest 页表负责 `gVA → gPA`，另一套由 VMM 维护的嵌套页表负责 `gPA → sPA`。
- **架构图解 D9：嵌套页表的页走爆炸（讲义 slide 58 的编号 1…24）**



- **虚拟化扩展的目标**（讲义 slide 59 逐条）：**避免 flush TLB**（用地址空间标识区分 guest，而不是切换时整表刷新）、**用嵌套页表替代影子页表**、**允许设备用 DMA 搬数据**、**允许 guest OS 处理设备中断**、以及安全方向的"允许程序管理加密的代码/数据区域"。
- **性能特征**：原生最坏 5 次访存 → 嵌套最坏 24 次（**4.8 倍**）；即使只算"每次 guest 页走要额外做一次 4 级嵌套翻译"，也是 4 级 → 4+16 = 20 量级。**这就是虚拟化环境对 TLB reach 极其敏感、并率先激进使用 2 MB/1 GB 大页的原因**：大页不仅省 TLB 项，还能一次性砍掉嵌套树里的好几级。

2.9 软件执行模型：多线程程序里"谁拥有哪份翻译状态"

- **定义与目的**：把上面所有结构放进一个并行程序的执行视图里，才能看清什么时候付出翻译代价。要点是**所有权划分**：
- **页表：每个进程一份**（进程内所有线程共享同一套页表、同一个 CR3）——所以**线程之间不需要 shootdown，只有跨核/跨 CPU 的 TLB 副本需要**；但同一进程的线程分布在多个核上，shootdown 就要打所有那些核。
- **TLB：每个核私有**（更准确地说，每个硬件线程/核有自己的 L1 TLB，每核有自己的 L2 TLB 或共享给 SMT 兄弟）——**一个线程预热过的 TLB，对另一个核上的线程毫无帮助**。
- **Page fault 的处理是串行的、且可能持锁**：多个线程同时首次触碰新页会争抢内核的 mm 锁（这是并行程序"启动开销"的主要来源之一）。
- **架构图解 D10：fork–join 程序中的翻译状态与代价点**



- **性能特征**：在纯计算/纯访存的稳态阶段，翻译是"免费的"（TLB 命中）；**代价集中在三类事件上**：(1) **首次触碰**（page fault，每页几微秒级）；(2) **改映射**（shootdown，随核数超线性）；(3) **工作集超出 TLB reach 的访问模式**（每次访存多一次页走）。第 3 节的三个代码示例分别把这三种代价单独放大出来测量。

---

3. 代码示例与性能分析

> 本节三个示例都给出了完整的编译命令与实测数据。**实测数据不是讲义内容**，是在本文档所用机器（2 × AMD EPYC 7V13，128 硬件线程，L3 512 MiB，共享节点）上跑出来的，仅用于说明数量级与相对趋势。三个示例共享同一条主线：**把地址翻译的代价从其它效应里单独拉出来看**。

3.1 示例 1：依赖指针追逐（pointer chasing）——把"翻译延迟"放到关键路径上

- **动机**：普通循环里的多次访存是**彼此独立**的，乱序执行可以让多个 cache miss / 页走重叠，于是"页走很慢"这件事会被隐藏（这正是 3.2 节示例 2 里我们观察到的情况）。要**单独测量翻译延迟**，就得让每一次访存都**依赖上一次的结果**（依赖链上没有并行度，也没有 MLP 可用）。
- **设计（2 × 2 控制变量）**：同一块 32 MiB 区域，同样的"随机跳转"序列，**只改页大小**：
- `same-page`：512 个节点挤在 1 个 4 KB 页内（TLB 必命中、数据在 L1）→ **基线**；
- `dense-32MiB`：400 万个节点紧密排列（4 KB 布局下跨 8192 个页）；
- `spread-32MiB`：8192 个节点、每个占一个 4 KB 页（4 KB 布局下**每跳都换页**）；
- 每个布局都跑 **THP off（4 KB 页）** 与 **THP on（2 MB 页）** 两组，并用 `/proc/self/smaps` **验证大页是否真的生效**。

```
// ============================================================================
// tlb_chase.cpp -- 依赖指针追逐 (pointer chasing): 隔离"地址翻译"在关键路径上的代价
//
//   三种布局 × 页大小(4KB / 2MB THP), 测每次跳转的延迟 (ns/hop)。
//   依赖链上每一跳都必须等上一跳回来, 所以内存延迟 + 页走延迟都暴露在关键路径上。
//
//   编译: g++ -O3 -march=native -o tlb_chase tlb_chase.cpp
//   运行: ./tlb_chase <thp:0|1> <每模式跳数(百万)>
//         例: ./tlb_chase 0 20    # 4KB 页, 每模式 2000 万跳
//
//   布局: same-page  : 512 个节点挤在 1 个 4KB 页内        (TLB 必命中, 数据在 L1)
//         dense-32M  : 400 万个节点紧密排列在 32 MiB 内     (4KB 布局下 8192 个页)
//         spread-32M : 8192 个节点每 4KB 放一个, 占 32 MiB  (4KB 布局下每跳换一页)
// ============================================================================
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <cstring>
#include <vector>
#include <random>
#include <algorithm>
#include <chrono>
#include <sys/mman.h>

static double now_s() {
  using namespace std::chrono;
  return duration<double>(steady_clock::now().time_since_epoch()).count();
}

// 只统计"目标映射"里的大页, 避免把 malloc 的堆区算进来
static double anon_huge_mb_for(void* addr) {
  FILE* f = fopen("/proc/self/smaps", "r");
  if (!f) return -1.0;
  char line[512];
  bool inside = false; double kb = 0.0;
  while (fgets(line, sizeof(line), f)) {
    unsigned long lo, hi;
    if (sscanf(line, "%lx-%lx", &lo, &hi) == 2) {
      inside = ((unsigned long)addr >= lo && (unsigned long)addr < hi);
      continue;
    }
    if (inside && strncmp(line, "AnonHugePages:", 14) == 0) { kb += atof(line + 14); inside = false; }
  }
  fclose(f);
  return kb / 1024.0;
}

// 在 [0,n) 上建一个随机哈密顿环, 返回 next[]: next[perm[k]] = perm[k+1]
static void build_cycle(std::vector<uint32_t>& next, size_t n, unsigned seed) {
  std::vector<uint32_t> p(n);
  for (size_t i = 0; i < n; i++) p[i] = (uint32_t)i;
  std::mt19937 rng(seed);
  std::shuffle(p.begin(), p.end(), rng);
  next.assign(n, 0);
  for (size_t k = 0; k < n; k++) next[p[k]] = p[(k + 1) % n];
}

// 依赖追逐: 返回 ns/hop
static double chase(uint8_t* base, size_t stride, const std::vector<uint32_t>& next,
                    size_t hops) {
  uint64_t idx = 0;
  const size_t n = next.size();
  // 把 next[] 拷到各自的布局上
  for (size_t i = 0; i < n; i++)
    *reinterpret_cast<uint64_t*>(base + (size_t)i * stride) = next[i];
  __asm__ __volatile__("" ::: "memory");
  double t0 = now_s();
  // next[] 构成 [0,n) 上的单个环, 所以 idx 永远落在 [0,n) 内, 不需要取模
  for (size_t k = 0; k < hops; k++)
    idx = *reinterpret_cast<uint64_t*>(base + idx * stride);   // 依赖链, 不可乱序
  double dt = now_s() - t0;
```

```
  __asm__ __volatile__("" :: "r"(idx) : "memory");             // 防止被优化掉
  return dt * 1e9 / (double)hops;
}

int main(int argc, char** argv) {
  const int    use_thp = (argc > 1) ? atoi(argv[1]) : 0;
  const size_t mhops   = (argc > 2) ? strtoull(argv[2], nullptr, 10) : 20;

  const size_t SMALL_N  = 512;              // 4KB 页内能放 512 个 8B 节点
  const size_t DENSE_N  = (32u << 20) / 8;  // 32 MiB / 8B = 4M 节点
  const size_t SPREAD_N = (32u << 20) / 4096;// 32 MiB / 4KB = 8192 节点
  const size_t REGION   = 32u << 20;

  void* raw = mmap(nullptr, REGION, PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
  if (raw == MAP_FAILED) { perror("mmap"); return 1; }
  if (madvise(raw, REGION, use_thp ? MADV_HUGEPAGE : MADV_NOHUGEPAGE) != 0) perror("madvise");
  uint8_t* base = static_cast<uint8_t*>(raw);

  printf("== THP=%s | 区域 32 MiB | AnonHugePages(初始)=%.0f MiB ==\n",
         use_thp ? "on" : "off", anon_huge_mb_for(raw));

  const char* name[3] = {"same-page  (512 节点/1 页, L1 命中)",
                         "dense-32MiB (4M 节点, 8 节点/line) ",
                         "spread-32MiB(8192 节点/8192 页)  "};
  const size_t ns[3]  = {SMALL_N, DENSE_N, SPREAD_N};
  const size_t st[3]  = {8, 8, 4096};

  for (int m = 0; m < 3; m++) {
    std::vector<uint32_t> next;
    build_cycle(next, ns[m], 2024 + m);
    double best = 1e30;
    for (int rep = 0; rep < 3; rep++) {
      double v = chase(base, st[m], next, mhops * 1000000ULL);
      if (v < best) best = v;
    }
    printf("  %s : %8.2f ns/hop  (页数 %zu, 该映射 AnonHugePages=%.0f MiB)\n",
           name[m], best, (ns[m] * st[m] + 4095) / 4096, anon_huge_mb_for(raw));
  }
  munmap(raw, REGION);
  return 0;
}
```

- **【代码做什么？】**
- `build_cycle()` 在 `[0, n)` 上构造一个**随机哈密顿环**：把下标随机打乱成 `p[0..n-1]`，令 `next[p[k]] = p[k+1]`（末尾指回开头）。这样从任意节点出发，沿着 `next` 走恰好 `n` 步会遍历所有节点并回到起点——**这个结构保证了 `idx` 永远落在 `[0, n)`**，于是追逐循环里**不需要取模**（`idx % n` 是一次 20–40 周期的整数除法，会淹没我们要测的信号；这是写这类微基准时最常踩的坑）。
- 按 `stride` 把 `next[]` **铺到目标布局上**：`stride = 8` 就是紧密排列（8 字节一个节点），`stride = 4096` 就是"每页一个节点"。
- `chase()` 里 `idx = *(uint64_t*)(base + idx * stride)` 是**纯依赖链**：下一次的地址来自这一次的数据，CPU 无法提前发射（不存在 MLP），也无法用预取器猜。整个循环的时间除以跳数就是 ns/hop。
- 用 `__asm__ volatile` 屏障阻止编译器把循环优化掉，并取 3 次测量中的**最小值**（最小值最接近"无干扰"的真实能力）。
- `anon_huge_mb_for()` 只统计**目标映射**里的大页（不统计 `malloc` 的堆），用来确认 `madvise(MADV_HUGEPAGE)` 是否成功。
- **【并行机制与性能解说】**
- **这段代码本身是单线程的，这正是它的目的**：依赖链的 Span = 跳数 × 单跳延迟，Work = 跳数 × O(1)，因此**并行度 = Work/Span ≈ 1**。任何"加线程"的做法都**不能降低单跳延迟**，只能通过**多条互相独立的追逐链**（interleaved pointer chasing）把页走的重叠度提上来：`P` 条独立链给出 Work = P·H，Span = H·t_hop（不变！），**并行度 = P**，加速比上限 = P（受核数限制），**单跳延迟不改善**——这就是"依赖链只能靠 TLP（线程级并行），不能靠 ILP"的教科书式例子。
- **页走在这条链上为什么完全无法隐藏**：一次 `load` 需要 `PA`，`PA` 需要翻译，翻译需要页走，页走本身的每一级又要访存——**这些全是串行依赖**。所以 4 KB 页下的每条新翻译都在关键路径上加了"1 次（PWC 命中时）到 4 次（全冷时）"的访存。
- **实测（本机，`./tlb_chase 0 10` / `./tlb_chase 1 10`，单位 ns/hop）**：

布局	触到的 4 KB 页数	THP off（4 KB 页）	THP on（2 MB 页，smaps 确认全覆盖）	差 = 翻译税
`same-page`（基线，TLB 必命中）	1	**2.60**	2.60	0（无可翻译事件）
`dense-32MiB`	8192	**39.75**	**24.78**	**14.97**
`spread-32MiB`	8192	**24.24**	**19.16**	**5.08**

- **读数**：把 32 MiB 的足迹从 4 KB 页换成 2 MB 页（**数据访问序列完全相同，只有页大小变化**），随机追逐的每次跳转就少了 **5.1–15.0 ns**（按 3 GHz 折算约 **15–45 周期**）。这就是"纯翻译开销"的实测上界：8192 个页远超 L2 TLB 的覆盖范围（公开测量值：AMD Zen 3/4 约 3072 项、同期 Intel 约 1536 项，均为**公开资料补充**，非讲义数据），**在 4 KB 页下几乎每一跳都要走一次页走**；换成 2 MB 页后 32 MiB 只需 16 条 TLB 项，页走消失。
- **为什么 `dense` 的翻译税（15.0 ns）比 `spread`（5.1 ns）更大**：两者页数与跳数相同，但 `dense` 的数据足迹是 32 MiB 的**全部 cache line**（每跳都是一次冷 line），页表中间层项更容易被数据流从 L2/L3 里挤出去，页走中更多的级落到更慢的层次；`spread` 只反复碰 8192 条 line（约 512 KB），页表项更容易常驻。**这也说明"翻译税"不是一个常数，而是与数据流争夺缓存的结果。**
- **瓶颈归类**：延迟受限（latency-bound），且瓶颈在**串行依赖链**（按 Little 定律，吞吐 = 并行度/延迟，这里并行度 ≈ 1）。要提升这类代码，只有两条路：**减少页走（大页）** 或 **增加并发的独立链（多线程/TLP）**。

3.2 示例 2：页步长 / 随机换页的并行带宽测试——"翻译墙"的带宽视角

- **动机**：示例 1 测的是**延迟**；但并行程序员更常看到的是**带宽**。这个示例在同一个数组上做三种访问模式，比较**有用带宽（只算真正取到的 8 字节）**随线程数的变化：
- `seq`：顺序遍历全部元素（每 4 KB 页用满 512 个元素 → 翻译成本被摊薄 512 次）；
- `page-stride`：**每 4 KB 只碰 1 个元素**（顺序换页）→ 每次访问都是新页；
- `random-page`：**乱序换页**（访问顺序被随机打乱）→ 每次访问都是新页，而且硬件预取器也失效。

```cpp
// ============================================================================
// tlb_pressure.cpp —— 地址翻译(TLB)压力测试
//
//   三种访问模式 × 两种页大小(4KB / 2MB THP)，测量 每次访问延迟 与 有用带宽。
//
//   编译: g++ -O3 -march=native -fopenmp tlb_pressure.cpp -o tlb_pressure
//   运行: ./tlb_pressure <数组MiB> <thp:0|1> <线程数>
//         例: ./tlb_pressure 1024 0 32      # 1 GiB, 4KB 页, 32 线程
//             ./tlb_pressure 1024 1 32      # 1 GiB, 2MB 大页, 32 线程
//
//   说明: thp=1 时用 madvise(MADV_HUGEPAGE) 请求透明大页，
//         thp=0 时用 madvise(MADV_NOHUGEPAGE) 强制 4KB 页。
//         注意 madvise 必须在"第一次触碰"之前调用。
// ============================================================================
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <cstring>
#include <vector>
#include <random>
#include <algorithm>
#include <chrono>
#include <sys/mman.h>
#include <omp.h>

static double now_s() {
  using namespace std::chrono;
  return duration<double>(steady_clock::now().time_since_epoch()).count();
}

// 从 /proc/self/smaps_rollup 读 AnonHugePages，验证大页是否真的生效
static double anon_huge_mb() {
  FILE* f = fopen("/proc/self/smaps_rollup", "r");
  if (!f) return -1.0;
  char line[256];
  double kb = -1.0;
  while (fgets(line, sizeof(line), f)) {
    if (strncmp(line, "AnonHugePages:", 14) == 0) { kb = atof(line + 14); break; }
  }
  fclose(f);
  return kb / 1024.0;   // MiB
}

int main(int argc, char** argv) {
  const size_t mb       = (argc > 1) ? strtoull(argv[1], nullptr, 10) : 1024;
  const int    use_thp  = (argc > 2) ? atoi(argv[2]) : 0;
  const int    nthreads = (argc > 3) ? atoi(argv[3]) : 1;

  const size_t bytes  = mb << 20;
  const size_t nelem  = bytes / sizeof(uint64_t);      // 8B 元素
  const size_t epage  = 4096 / sizeof(uint64_t);       // 每 4KB 页 512 个元素
  const size_t npages = nelem / epage;                 // 4KB 页数

  void* raw = mmap(nullptr, bytes, PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
  if (raw == MAP_FAILED) { perror("mmap"); return 1; }
  if (madvise(raw, bytes, use_thp ? MADV_HUGEPAGE : MADV_NOHUGEPAGE) != 0)
    perror("madvise");
  uint64_t* a = static_cast<uint64_t*>(raw);

  // 初始化: 并行顺序写, 触发物理页分配 (first touch) 与可能的 khugepaged 合并
  #pragma omp parallel for num_threads(nthreads) schedule(static)
  for (size_t i = 0; i < nelem; i++) a[i] = i;

  // 随机页序: 每次访问落在随机的一个 4KB 页上
  std::vector<uint32_t> perm(npages);
  for (size_t i = 0; i < npages; i++) perm[i] = (uint32_t)i;
  std::mt19937 rng(12345);
  std::shuffle(perm.begin(), perm.end(), rng);

  printf("== 数组 %zu MiB | %zu 个 4KB 页 | THP=%s | 线程=%d | AnonHugePages=%.0f MiB ==\n",
```

```
        mb, npages, use_thp ? "on" : "off", nthreads, anon_huge_mb());

  const int    REP    = 3;
  const int    PASSES = 16;   // seq 模式跑 1 遍, 另外两种模式跑 16 遍 (时间太短测不准)
  const char*  names[3] = {"seq         (步长 8B) ",
                           "page-stride (步长 4KB)",
                           "random-page (乱序换页)"};
  uint64_t     acc = 0;

  for (int mode = 0; mode < 3; mode++) {
    double best = 1e30;
    for (int rep = 0; rep < REP; rep++) {
      double t0 = now_s();
      uint64_t local = 0;
      if (mode == 0) {                       // 顺序遍历全部元素
        #pragma omp parallel for num_threads(nthreads) schedule(static) reduction(+:local)
        for (size_t i = 0; i < nelem; i++) local += a[i];
      } else if (mode == 1) {                // 每页只碰一个元素(顺序换页)
        for (int pass = 0; pass < PASSES; pass++) {
          #pragma omp parallel for num_threads(nthreads) schedule(static) reduction(+:local)
          for (size_t p = 0; p < npages; p++) local += a[p * epage];
        }
      } else {                               // 每页只碰一个元素(乱序换页)
        for (int pass = 0; pass < PASSES; pass++) {
          #pragma omp parallel for num_threads(nthreads) schedule(static) reduction(+:local)
          for (size_t i = 0; i < npages; i++) local += a[(size_t)perm[i] * epage];
        }
      }
      double dt = now_s() - t0;
      acc = local;
      if (dt < best) best = dt;
    }
    size_t nacc = (mode == 0) ? nelem : npages * (size_t)PASSES;
    double per_acc_ns = best * 1e9 / (double)nacc;
    double useful_GBs = (double)nacc * 8.0 / best / 1e9;   // 只算"有用"的 8B
    printf("  %s : %8.2f ms | %7.2f ns/访问 | 有用带宽 %7.3f GB/s (acc=%llu)\n",
           names[mode], best * 1e3, per_acc_ns, useful_GBs, (unsigned long long)acc);
  }
  munmap(raw, bytes);
  return 0;
}
```

- **【代码做什么？】**
- `mmap` 一块匿名区域，用 `madvise(MADV_HUGEPAGE/MADV_NOHUGEPAGE)` **在同一份代码里切换 4 KB / 2 MB 页**（注意：`madvise` 必须在**第一次触碰之前**调用，否则 THP 不会生效——这是最常见的"大页没生效"原因）。
- 用 OpenMP 并行初始化（`a[i] = i`），这一步**同时完成物理页分配**（Linux 的 first-touch 策略：**谁先写，页就分配在谁的 NUMA 节点上**），并可能触发 `khugepaged` 的合并。
- 构造**随机页序** `perm[]`，保证 `random-page` 模式每次访问都落在不同的 4 KB 页上。
- 依次跑三种模式，每种取 3 次中的最好成绩，输出 **ms、ns/访问、有用带宽 GB/s**。短时间模式（`page-stride` / `random-page`）跑 16 遍（`PASSES`）以保证计时精度。
- 打印 `AnonHugePages`（来自 `smaps_rollup`）——**让"大页到底有没有生效"变成可见的事实**。
- **【并行机制与性能解说】**
- **并行机制**：三种模式都是**数据并行（data parallelism）**的 `for` 循环，OpenMP `parallel for ... schedule(static)` 把索引区间均分给 `P` 个线程；每个线程只需要一个私有累加器，最后由 `reduction(+:local)` 汇总。**没有锁、没有线程间通信、没有数据共享写**（只读 `a[]` 与 `perm[]`）——除了归约那一下。
- **Work / Span / 并行度**：
  - `seq`：**Work = N**（N = 数组元素数），**Span = N/P + O(log P)**（归约树或原子操作），**并行度 = Work/Span ≈ P**；
  - `page-stride` / `random-page`：**Work = npages × PASSES**，**Span = Work/P + O(log P)**，**并行度 ≈ P**。
  - **三者都是"完美并行"的循环，加速比上限 = 核数**；但**实测远达不到**，因为瓶颈从"并行度不足"变成了**内存系统**（延迟 × 未完成请求数 × 带宽）。这就是 work-span 模型必须与内存模型一起用的原因：**work-span 只告诉你"够不够并行"，不告诉你"内存喂不喂得饱"。**
- **实测（本机，1 GiB 数组，THP off，g++ 12.2.0）**：

模式	1 线程 ns/访问	8 线程	32 线程	32 线程有用带宽	8→32 线程加速比
`seq`（步长 8 B）	0.32	0.05	0.04	**196.6 GB/s**	1.2×（**已带宽饱和**）
`page-stride`（步长 4 KB）	**13.35**	1.02	0.30	26.5 GB/s	3.4×
`random-page`（乱序换页）	**15.07**	1.14	0.34	23.4 GB/s	3.4×

- **三条读数**：
  1. **顺序访问对翻译几乎免疫**：1 线程下 0.32 ns/访问就够了（一次翻译覆盖 512 次访问），8/32 线程时彻底变成**带宽受限**（8 线程 170 GB/s → 32 线程 196.6 GB/s，几乎不再增长）。**这说明"地址翻译"不是所有负载的瓶颈，只对"每页利用率低"的访问模式是瓶颈。**
  2. **每页只碰一个元素 = 灾难**：1 线程 1 GiB 随机换页 **15.07 ns/访问**，是顺序访问（0.32 ns）的 **47 倍**；32 线程有用带宽只剩 **23.4 GB/s**，是流式带宽（196.6 GB/s）的 **1/8**。请注意：这里的 8 字节"有用数据"实际要拉一条 64 B 的 cache line（8× 放大），**再加上页走的额外访存**——带宽放大是"有用带宽崩塌"的直接原因。
  3. **线程数 8 → 32 只有 3.4×**：这些访问每个线程已经是"一次独立的冷 line + 一次页走"，32 个核产生的访存并发（MSHR/页走器数量）逼近系统上限，于是子线性。
- **大页在这一组里为什么"没帮上忙"（诚实记录一个反例）**：在 1 GiB 数组上请求 THP 时，`smaps` 只显示 **72 MiB（7%）** 真正落成大页（机器长期运行、物理内存碎片化），所以 page-stride/random-page 的耗时不降反微升；把数组缩到 32 MiB 才拿到 100% 覆盖。**结论**：`MADV_HUGEPAGE` 只是"请求"，**必须验证**；这同时印证了讲义第 3–4 页那张图所指向的问题——**物理内存连续性（contiguity）正在成为翻译性能的实际瓶颈**。

3.3 示例 3：TLB shootdown 随活跃核数的扩展性（翻译一致性的代价）

- **动机**：讲义花了 8 页（slide 44–53）讲 shootdown，并给出"8 核以上超过 10,000 周期"的结论。这个示例用最朴素的系统调用把这条曲线在本机复现出来：**对一个小页面反复改保护属性**，同时让 `T` 个线程在 `T` 个不同的核上持续读这一页。

```cpp
// ========================================================================
// shootdown.cpp —— 测量 TLB Shootdown 随"活跃核数"的增长
//
//   主线程反复对一个 4KB 页做 mprotect(只读) / mprotect(读写),
//   每次调用内核都必须把该页的翻译从"其他正在跑本进程的核"的 TLB 里打掉
//   => 发送 IPI(处理器间中断) 并等待所有 ACK (讲义里的 6 步流程)。
//   T 个工作线程各自跑在一个不同的核上并持续读这一页, 因此 T 越大,
//   需要被中断、被打掉 TLB 项的核越多, 每次 mprotect 的延迟就越高。
//
//   编译: g++ -O3 -march=native -pthread shootdown.cpp -o shootdown
//   运行: taskset -c 0-63 ./shootdown          (建议先绑定到同一颗 socket)
// ========================================================================
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <cstring>
#include <atomic>
#include <chrono>
#include <thread>
#include <vector>
#include <pthread.h>
#include <sched.h>
#include <sys/mman.h>
#include <unistd.h>

static double now_s() {
  using namespace std::chrono;
  return duration<double>(steady_clock::now().time_since_epoch()).count();
}

static volatile int g_stop = 0;
static volatile uint64_t g_sink = 0;
static volatile char* g_page = nullptr;   // 所有工作线程一起读这一页

static void pin_to(int cpu) {
  cpu_set_t set; CPU_ZERO(&set); CPU_SET(cpu, &set);
  pthread_setaffinity_np(pthread_self(), sizeof(set), &set);
}

static void* worker(void* arg) {
  pin_to((int)(intptr_t)arg);
  uint64_t s = 0;
  while (!g_stop) s += g_page[0] + g_page[2048];   // 持续读同一页 => 一直持有翻译
  g_sink += s;
  return nullptr;
}

int main() {
  // 1) 分配并触碰区域: 用 4KB 页(禁用 THP), 这样 PTE 数量最多
  const size_t region = 2u << 20;                       // 2 MiB
  char* p = static_cast<char*>(mmap(nullptr, region, PROT_READ | PROT_WRITE,
                                    MAP_PRIVATE | MAP_ANONYMOUS, -1, 0));
  if (p == MAP_FAILED) { perror("mmap"); return 1; }
  madvise(p, region, MADV_NOHUGEPAGE);
  for (size_t i = 0; i < region; i += 4096) p[i] = 1;   // 每个 4KB 页都真实分配
  g_page = p;

  const int MAXT = 32, K = 20000;
  printf("== mprotect(4KB 页) 触发的 TLB shootdown 延迟 vs 活跃核数 (每点 %d 次) ==\n", K);
  printf("   活跃线程数 |  每次 mprotect 延迟 |  相对 0 线程\n");
  printf("   ----------+---------------------+------------\n");

  double base = 0;
  for (int T = 0; T <= MAXT; T = (T == 0) ? 1 : T * 2) {
    std::vector<std::thread> th;
    g_stop = 0;
    for (int i = 0; i < T; i++) th.emplace_back(worker, (void*)(intptr_t)(i + 1));
    std::this_thread::sleep_for(std::chrono::milliseconds(200));   // 等线程就位

    pin_to(0);
    // 预热
```

```
    for (int k = 0; k < 100; k++) {
      mprotect(p, 4096, PROT_READ);
      mprotect(p, 4096, PROT_READ | PROT_WRITE);
    }
    double best = 1e30;
    for (int rep = 0; rep < 3; rep++) {
      double t0 = now_s();
      for (int k = 0; k < K; k++) {
        mprotect(p, 4096, PROT_READ);                  // 触发 shootdown
        mprotect(p, 4096, PROT_READ | PROT_WRITE);     // 再触发一次
      }
      double dt = (now_s() - t0) / (2.0 * K);          // 每次 mprotect
      if (dt < best) best = dt;
    }
    g_stop = 1;
    for (auto& t : th) t.join();
    if (T == 0) base = best;
    printf("   %8d  |  %8.2f us/次  |  %6.2fx\n", T, best * 1e6, best / base);
  }
  munmap(p, region);
  printf("(sink=%llu)\n", (unsigned long long)g_sink);
  return 0;
}
```

- **【代码做什么？】**
- `mmap` 一块 2 MiB 匿名区域，`madvise(MADV_NOHUGEPAGE)` 强制 4 KB 页并逐页写一个字节**把它们真正分配出来**（避免测到缺页，而只测 shootdown）。
- 启动 `T` 个 `worker` 线程，每个线程 `pthread_setaffinity_np` **绑定到 CPU `i+1`**（主线程绑 CPU 0），然后**死循环读同一页的两个字节**——这样每个目标核都**稳定持有一个该页的翻译**。
- 主线程对**该页**反复执行 `mprotect(PROT_READ)` → `mprotect(PROT_READ|PROT_WRITE)`：每次调用内核都要把这一页的翻译从其它核的 TLB 里打掉，也就是**发 IPI + 等 ack**（讲义 slide 45 的六步）。
- 对 `T = 0, 1, 2, 4, 8, 16, 32` 各测 3 轮取最优，输出**每次 `mprotect` 的平均延迟**与相对 `T = 0` 的倍数。
- **为什么只改读权限而不设 `PROT_NONE`**：工作线程一直在读这一页，如果中间出现"不可读"的窗口就会段错误——微基准必须保证被测操作对并发的观察者是**安全的**。
- **【并行机制与性能解说】**
- **并行机制**：这里的并行是**被观察的对象**而不是加速手段。`T` 个线程是"无辜的旁观者"：它们在读数据，**不参与任何同步**，却会被 IPI 硬生生打断（flush pipeline、保存/恢复上下文，讲义 slide 52 的甘特图）。**这说明"页表变更"是一种全局的隐式同步原语**——即使程序里一个锁都没用。
- **Work / Span / 并行度**：每次 `mprotect` 的 Work = `O(1)`（更新 1 个 PTE）+ `O(T)`（发送 IPI、等待并汇总 `T` 个 ack）；**Span 与 Work 同阶**——因为发起核**必须等所有 ack 才能返回**，整个操作是**串行 rendezvous，并行度 = 1**。所以 shootdown 是"**Amdahl 定律意义上的串行段**"，而且这个串行段的长度 `L(T)` **本身随核数超线性增长**。
- **实测（本机，`shootdown.cpp`，每点 2 × 20000 次 `mprotect`）**：

活跃线程数 T	每次 `mprotect` 延迟	相对 T=0	折算周期（@3 GHz）
0	2.79 µs	1.00×	≈ 8,400
1	4.50 µs	1.62×	≈ 13,500
2	5.08 µs	1.82×	≈ 15,200
4	6.06 µs	2.17×	≈ 18,200
8	**15.53 µs**	5.57×	≈ 46,600
16	**47.24 µs**	16.95×	≈ 141,700
32	**159.99 µs**	57.41×	≈ 480,000

- **读数**：`T = 0` 的 2.79 µs 是系统调用 + 页表更新 + **本地** TLB 刷新的底价；从 4 → 8 → 16 → 32 核，延迟翻了 **2.6× → 3.0× → 3.4×，边际成本随核数上升**（每多牵连一个核：8 核附近约 2.4 µs，32 核附近约 7.1 µs）。这与讲义 slide 53 的结论完全一致：*"Gets more expensive with increasing number of cores."*

- **瓶颈归类：同步/串行 rendezvous 开销**（不是带宽、不是负载不均）。实战含义：**任何"每 N 次迭代就改一次映射"的并行代码（GC 写屏障、用户态页表、频繁 `fork` 的 COW、JIT 改代码页权限、mpi 的大页池重映射）都会随核数变差**，而且这类代码在 1–4 核上测试时完全看不出问题。

---

4. 性能模型与复杂度分析

4.1 翻译的平均访问延迟模型（AMAT 形式）

把一次访存的延迟写成分层命中公式（AMAT = average memory access time）：

```
t_eff  =  t_TLB_hit  +  P(miss | L1 TLB) · t_L2TLB
                     +  P(walk) · t_walk
                     +  t_data

其中  t_walk = Σ_{i=1..L} t_level(i),   L = 页走级数（x86-64 基础页为 4）
      t_level(i) ∈ { L1 命中, L2 命中, L3 命中, DRAM }  取决于第 i 级页表项在哪
      P(walk) = (1 − p_L1) · (1 − p_L2 | L1 miss)
```

数值算例（与 3.1 节实测对照）：32 MiB 随机追逐，4 KB 页，8192 个页；取 L1 dTLB 64 项、L2 TLB（STLB）**3072 项**（AMD Zen 3/4 公开测量量级）：

量	计算	结果
L1 dTLB 命中率 `p_L1`	64 / 8192（均匀随机访问）	≈ 0.8%
STLB 命中率 `p_L2|miss1`	3072 / 8192	≈ 37.5%
需要页走的比例 `P(walk)`	(1−0.008)(1−0.375)	**≈ 62%**
页走成本 `t_walk`（PWC 命中前三层，PTE 页命中 L2/L3）	1 × 8 ns	≈ 8 ns
数据访问 `t_data`（2 MB 页实测，无页走）	实测	19.16 ns
模型预测	19.16 + 0.62 × 8	**≈ 24.1 ns/hop**
实测（spread–32MiB, THP off）		**24.24 ns/hop** ✅

同一模型给出两个极端，说明"翻译成本"取决于**页表项在缓存层次里的位置**：

场景	`t_walk` 假设	`P(walk)`	预测 ns/hop	说明
翻译全部命中（2 MB 大页）	0	0	19.16	与实测 19.16 一致（无页走）
PTE 页在 L2/L3（常见）	8 ns（只缺 1 级）	0.62	**24.1**	与实测 24.24 一致
PTE 页也在 DRAM（页表被挤出 cache）	80 ns × 1 级	0.62	68.8	翻译开始主导延迟
全冷 4 级页走（最坏）	80 ns × 4 = 320 ns	1.0	**≈ 339**	是实测的 **14 倍**：真实系统靠 PWC/数据 cache 兜住了绝大部分

结论：`P(walk)` 由 **TLB reach 与工作集页数之比**决定，`t_walk` 由**页表项的缓存局部性**决定。这两项正是 2.6 节四种技术分别攻击的目标（大页打 `P(walk)`，PWC / 翻译进 cache 打 `t_walk`）。

4.2 TLB reach（覆盖率）与工作集：第一个必须算的数字

TLB reach = 项数 × 页大小。它是判断"翻译会不会成为瓶颈"的第一把尺子：

TLB 配置	4 KB 页	2 MB 大页	1 GB 大页
1536 项（同期 Intel 公开测量量级）	6 MiB	3 GiB	1.5 TiB
3072 项（AMD Zen 3/4 公开测量量级）	12 MiB	6 GiB	3 TiB

- **命中率估算（均匀随机访问）**：`hit ≈ min(1, reach / 工作集大小)`。
- 工作集 **1 GiB**：4 KB 页 → 12 MiB / 1 GiB = **1.2%**（几乎每次访存都要走页）；2 MB 页 → 512 个页，**100% 命中**。
- 工作集 **32 MiB**（8192 页）：4 KB 页 → 3072/8192 = **37.5%**（与 4.1 节模型一致）；2 MB 页 → 16 个页，**100% 命中**。

- **算一笔"每页只取 8 字节"的账（与 3.2 节实测对得上）**：遍历 1 GiB 数组、每 4 KB 只取 1 个 8 字节元素 = **262144 次访问**。
- 单线程实测：**13.35 ns/访问**（page-stride）、**15.07 ns/访问**（random-page）→ 一遍耗时 **3.5–3.95 ms**（实测 63.19 ms / 16 遍 = 3.95 ms，吻合）；
- 若只看"有用数据量"：262144 × 8 B = **2 MiB**；同样 3.95 ms 里顺序流式（24.67 GB/s 单线程）能搬 **97 MiB**——**同一个核，同一段时间，随机换页模式只产生了流式模式 2% 的有效数据**。
- 真实流量：262144 × 64 B（每访问一条 cache line）= **16.8 MB**；若页走中的 PTE 级也落空，再加 262144 × 64 B = 16.8 MB → **放大 8×–16×**。
- **讲义 slide 17–20 的算例（页粒度局部性的正面案例）**：4 KB 的翻译 "enough for 64 cache lines of 64 bytes"——**顺序访问时每 512 次 8 字节访问才需要一次新翻译**，摊薄后翻译成本 < 0.1%。这正是实测里 `seq` 模式（0.32 ns/访问）与 32 线程下 196.6 GB/s 的原因。

4.3 页走的 Work / Span 与"每次翻译的流量"

- **单次页走**：**Work = L 次访存**（L = 4），**Span = 4 次串行访存**（地址依赖），**并行度 = Work/Span = 1**。
- **N 个相互独立的访存**：**Work = 4N**，Span 仍是 4 次访存（每条链自己串行），**并行度 = N** —— 但这个并行度受**硬件能同时容纳多少个未完成页走**限制（页走器/MSHR 数量，通常每核十几到几十个）。设并发页走数为 `M`，页走延迟 `t_walk`，则：

```
单核页走吞吐  =  M / t_walk            (walks/s)
有用数据吞吐  =  (M / t_walk) × 8 B    (每页只取 8 字节时)
```

数值算例（解释实测的 0.6 GB/s）：若 `t_walk ≈ 80 ns`（PWC 命中上层，只有 PTE 级去 DRAM），要得到实测的 **0.599 GB/s**（1 线程 page-stride），需要：

```
M = 0.599e9 B/s × 80e-9 s / 8 B = 5.99  ⇒  M ≈ 6 个并发的页走在飞
```

即：**在这台机器上，单核大约能同时保持 6 个左右未完成的页走**；若页走是全冷的 4 级（320 ns），同样的有用吞吐需要 `M ≈ 24`，而硬件给不了——这也是"PTE 被挤出缓存后性能断崖"的另一种说法。

- **带宽放大（bandwidth amplification）**：每页只取 8 字节时，**每次翻译平均要读 1 条 64 B 的页表项缓存行**（PWC 未覆盖时甚至 4 条 = 256 B），而有用数据只有 8 B。定义"翻译流量放大比"：

```
放大比 = (每访问的有用字节 + 页表流量) / 有用字节
       = (8 + 64) / 8 = 9×      (PWC 覆盖上层, 只有 PTE 级去内存)
       = (8 + 256) / 8 = 33×    (全冷页走, 4 级都去 DRAM)
```

4.4 Shootdown 的 Amdahl 分析：串行段长度本身随核数增长

模型：设每个 epoch（例如一次 GC 周期、一次 JIT 编译、一次 fork）有 `W` 的可并行工作量，并触发 **1 次**映射变更：

```
T(P) = L(P) + W / P
     L(P) = 实测的每次 mprotect 延迟（随被牵连核数 P 增长，见 3.3 节表）
加速比 S(P) = T(1) / T(P),  T(1) = L(1) + W
```

数值算例（用 3.3 节本机实测的 `L(P)`，取 `W = 1 ms` 的可并行工作）：

核数 P	`L(P)`（实测）	并行部分 `W/P`	总时间 `T(P)`	加速比 `S(P)`	并行效率
1	2.79 µs	1000 µs	1002.8 µs	1.00×	100%
8	15.53 µs	125 µs	140.5 µs	**7.14×**	89%
16	47.24 µs	62.5 µs	109.7 µs	**9.14×**	57%
32	159.99 µs	31.25 µs	191.2 µs	**5.24×**	16%

这张表是本节最重要的结论：把核数从 8 加到 32，程序**反而慢了 36%**（191 µs vs 141 µs），加速比从 7.14× 掉到 5.24×。原因不是负载不均、不是带宽，而是 **Amdahl 定律里那个"串行段"的长度本身随 P 增长**（`L(P)` 从 2.79 µs 涨到 160 µs）——经典 Amdahl 公式假设串行段是常数，而 shootdown 打破了这个假设：

```
经典 Amdahl:  S(P) = 1 / (f + (1−f)/P)          f 为常数
这里:         S(P) = T(1) / (L(P) + W/P)        L(P) 超线性增长
⇒ 存在最优核数 P*, 超过它之后"加核"是负收益:
     (本例中 P* 在 16 附近; W 越小, P* 越靠左)
```

工程含义：减少映射变更的"次数"比优化每次变更更有效（批量合并 flush 请求、在启动阶段一次性映射好大页、避免在计算热路径上 `mprotect` / `munmap` / `fork`）。这也是讲义 slide 59 把 "Avoid flushing TLB" 列为虚拟化扩展目标的原因。

4.5 Roofline / 算术强度视角：翻译到底改变了哪个上限

把"每访问有用字节"与"每访问真实流量"算成 **算术强度（arithmetic intensity, 有用字节/字节传输）**，再对照实测的流式带宽上限（32 线程 196.6 GB/s）：

访问模式	有用字节/访问	真实流量/访问	算术强度	带宽上限 = 196.6 × AI	实测（32 线程）	瓶颈归因
`seq`（顺序）	8 B	8 B	1.0	196.6 GB/s	**196.6 GB/s**	**带宽受限（已打满）**
`page-stride` / `random-page`（PWC 覆盖上层）	8 B	64 B（1 条 line）	0.125	24.6 GB/s	**23.4–26.5 GB/s**	**带宽受限：8× line 放大**
`page-stride` / `random-page`（全冷页走）	8 B	320 B（line + 4 级页表）	0.025	4.9 GB/s	未观测到	带宽受限：40× 放大
1 线程 `page-stride`	8 B	64 B+	≤0.125	—	**0.6 GB/s**（≪ 24.6 上限）	**延迟受限：MLP 不足（M≈6）**

三条定量结论： 1. **顺序访问下翻译不是瓶颈**：AI = 1.0，实测贴着带宽上限（196.6 GB/s ≈ 24.6 GB/s/线程 × 8 线程就饱和了）。 2. **"每页少用字节"的模式下，瓶颈首先是 line 放大（8×），其次才是页表流量**：32 线程实测 23.4 GB/s 与"只算 64 B line"的模型上限 24.6 GB/s 吻合到 5% 以内——**说明在这个规模下，页走的额外流量主要被 PWC/L2 吸收了**，真正的浪费是"拉了一条 64 B 的线只用了 8 B"。 3. **小线程数下是延迟受限，大线程数下是带宽受限**：1 线程 0.6 GB/s（延迟/MLP 上限），8 线程 7.9 GB/s（4.5×，MLP 随核数增加），32 线程 26.5 GB/s（≈ 带宽上限）。**这解释了为什么"翻译优化"（大页）在单线程/少核时收益巨大（延迟路径上的 5–15 ns/hop），而在多核带宽饱和后收益变小**——因为此时真正的瓶颈已经转移。

---

5. 关键要点

1. **地址翻译本身就是一层访存层次，不是一个常数开销。** 每次访存的翻译都要经过 "L1 TLB → L2 TLB → PWC/MMU cache → 页表（逐级）" 这条阶梯，命中位置决定关键路径上要多走几次访存：L1 命中 ≈ +0；STLB 命中 ≈ +1 次更慢的 TLB 访问；页走 = +1~4 次串行访存（全冷 4 级 ≈ 320 ns）。**讲义 slide 25/34 列出的四种技术（多级 TLB、PWC、翻译进数据 cache、大页）恰好沿"提高覆盖范围（reach）"与"提高复用（把中间层留在 cache 里）"两条轴展开**——理解这两条轴，就不会把四项技术当成互不相干的技巧。
2. **判断翻译会不会成为瓶颈，第一把尺子是 `reach = 项数 × 页大小` 与工作集页数之比。** 4 KB 页时 STLB 的 reach 只有十几 MiB 量级（3072 × 4 KB = 12 MiB），所以**任何超过十几 MiB 的随机访问工作集都几乎每次访存都要走页**；而顺序访问时一次翻译覆盖 512 次 8 字节访问，翻译成本被摊薄到 0.1% 以下（实测：`seq` 0.32 ns/访问 vs `random-page` 15.07 ns/访问，相差 **47 倍**）。**页粒度（4 KB）与 line 粒度（64 B）的错配**，是"每页只取几个字节"的模式性能崩塌的根因。
3. **TLB 是每核私有、且硬件不保证它与页表/cache 一致**（讲义 slide 41）。因此**"改映射"在并行程序里是一种全局的隐式同步**：OS 必须 lock PTE → 计算受害者核 → 发 IPI → 等所有核 ack（讲义 slide 45 六步）。它不是常数开销：本机实测从 0 个活跃核的 **2.79 µs** 涨到 32 核的 **159.99 µs（57×，超线性）**，导致"每毫秒触发一次映射变更"的程序在 32 核上**比 8 核还慢 36%**（4.4 节）。**减少改映射的次数，比优化单次改映射更有效。**
4. **大页是最大的杠杆，也是最大的风险。** 一条 2 MB 页表项把 reach 提高 512 倍（4 KB → 2 MB），并且让页走在 PMD 级就终止（少一级页走；在虚拟化下省掉一整套嵌套翻译）。但它的代价是**内部碎片、COW 放大、首次触碰更贵（要清零 2 MB）、分配需要连续物理内存**；更关键的是**它可能根本拿不到**——本机实测请求 1 GiB 透明大页只成功 72 MiB（7%），32 MiB 区域才 100% 成功。**结论：`madvise(MADV_HUGEPAGE)` 是请求而不是保证，必须用 `/proc/self/smaps` 验证。**
5. **虚拟化把"一层翻译"变成"两层翻译"（gVA→gPA→sPA），最坏 4 次访存变 24 次**（讲义 slide 58 的编号 1…24）。所以现代 ISA 的虚拟化扩展把"避免 flush TLB、用嵌套页表代替影子页表、支持设备 DMA 与 guest 处理中断"列为目标（slide 59），而**嵌套页表能实用，靠的仍然是 TLB/PWC 缓存中间层结果 + 大页**——本讲的每一件技术，在虚拟机里都要再用一遍。

---

6. 常见陷阱与注意事项

- **把 TLB 当成"会自动和页表保持一致的缓存"。** 硬件只提供"页走 + 填充/替换 TLB"，**没有任何机制在 PTE 被修改后自动失效别的核的 TLB 项**（讲义 slide 41 明确写了 "no cache coherence between TLBs and Caches"）。任何"我改了页表，别的线程应该马上看到"的假设都是错的；同一进程内的其它**核**（不是线程）同样需要 shootdown。
- **把所有"随机访问慢"都归因于 cache miss 或伪共享。** 诊断顺序应该是：先算 `工作集页数 vs TLB reach`（第 4.2 节），再看 cache 冲突。一个实用的判别实验：**保持访问序列不变、只改变页大小或数据布局的"每页元素数"**——如果时间显著变化，瓶颈里有翻译的份（3.1 节的两组数据差 5–15 ns/hop）。反之，如果只是把数组 padding 一下就好了，那才是 cache set 冲突。
- **以为"顺序访问就没有 TLB 问题"。** 顺序访问只是把翻译**摊薄**（4 KB 页覆盖 64 条 line），并不是消除它；一旦访问模式变成"每页只碰一个元素"（页步长遍历、稀疏矩阵按列扫、图 BFS 的随机顶点、哈希表随机探测、跨页采样的 profiler），翻译立刻回到关键路径上。**"每页利用率"是一个应该主动检查的性能指标。**
- **假设大页一定生效、且只有好处。** 三个具体错误：(1) 在第一次触碰**之后**才调用 `madvise(MADV_HUGEPAGE)`，于是 THP 根本没机会生效；(2) 不检查 `/proc/self/smaps`（本机实测 1 GiB 只拿到 7% 大页）；(3) 忽视代价——大页让**写时复制（COW）放大 512 倍**（改 1 个字节可能复制 2 MB）、首次触碰要清零 2 MB（按 20–50 GB/s 的清零带宽约 40–100 μs/页）、内存回收与统计粒度变粗、swap/超分行为变差。
- **在并行热路径上改地址映射。** `mprotect` / `munmap` / `mmap` / `mremap` / `fork`（COW）/ 用户态页表管理（如 DPDK、GC 写屏障做法、JIT 改代码页权限）都会触发 TLB shootdown，而**它的代价随活跃核数超线性增长**（实测 8 核 15.5 μs → 32 核 160 μs）。这类代码在 1–4 核上测试完全正常，一上大机器就崩——**必须按核数做扩展性测试，并尽量把映射变更批量化、放在启动阶段。**
- **微基准的方法学错误会让结论完全反过来。** 常见的有：(a) 用**独立访存**的循环去测"翻译延迟"，结果测到的是带宽/MLP（3.2 节里 THP 开关在 1 GiB 数组上几乎看不出差异，正是因为瓶颈不在翻译）；(b) 指针追逐里插入 `idx % n` 取模（一次整数除法 20–40 周期，淹没有用信号）；(c) 不用 `volatile`/屏障保护结果，循环被编译器删掉；(d) 用**依赖链**测带宽（测到的是延迟）；(e) 忘记 NUMA first-touch —— 多线程初始化会让页按线程所在节点分布，跨节点访问带宽可能只有本地的一半，从而把翻译效应掩盖掉。

---

7. 思考题（带答案）

题 1：VIPT L1 的容量上限与真实 L1 尺寸

某处理器使用 4 KB 基础页、64 B cache line，L1 数据 cache 采用 **VIPT（虚拟索引、物理标签）**以保证"索引与 TLB 查找并行"。请推导：**在不引入同义词（synonym）问题的前提下，L1 最大能做成多大（8 路相联）？为什么真实产品的 L1 是 32 KB 8 路 / 48 KB 12 路这样的尺寸？如果非要更大的 L1，有哪些办法？**

【答案】 VIPT 无别名的充要条件是：**cache 索引位必须全部落在页内偏移里**（因为只有偏移位在翻译前后不变）。4 KB 页 → 页内偏移 **12 位**；64 B line → 块内偏移 **6 位**。因此索引位 ≤ 12 − 6 = **6 位**，即 **set 数 ≤ 64**。

```
容量上限(8 路) = sets × line × ways = 64 × 64 B × 8 = 32 KB
超了会怎样: 索引位进入 VPN 部分 → 同一个物理页的两个不同 VA 可能映射到
            不同 set → 同一份数据在 cache 里出现两份（同义词/别名）
            → 需要反别名或刷新, VIPT 的最大优点(索引不等翻译)就没了
```

- **为什么真实尺寸是这些值**：32 KB/8 路 → 64 set（6 位索引，6+6 = 12 ✅）；48 KB/12 路 → 48K/(64×12) = 64 set（同样 6 位 ✅）；而 64 KB/8 路 → 128 set（需要 7 位索引，7+6 = 13 > 12 ❌，会出现别名）；若改成 64 KB/**16 路** → 64 set（6 位 ✅，合法）。**这就是为什么 Intel 长期用 32 KB 8 路、后来用 48 KB 12 路 L1，而不是直接堆到 64 KB 8 路**——容量不是被面积卡住，而是被"4 KB 页的 12 位偏移"卡住（保持关联度、降低 set 数，就能在 12 位里塞下索引）。
- **要更大 L1 的办法**：(1) **提高关联度**（同容量下 set 数变少，索引位变少；但关联度本身有延迟/面积/功耗代价）；(2) **改用 PIPT**（索引也要等翻译 → **每次访存都多一次 TLB 延迟**，这也是为什么大容量 L2/L3 用 PIPT 而无所谓：它们的延迟本来就高，且不必与 TLB 并行）；(3) **VIVT**（索引标签都用虚拟地址，最快，但同义词 + 进程切换的"同形异义"必须靠 ASID 或整表刷新解决）；(4) 让硬件利用**大页的更大偏移**来扩展索引位——但**实际硬件通常按最小页（4 KB）设计索引**（因为访问时可能用 4 KB 页），所以**不能依赖大页来放宽 L1 的 VIPT 约束**；大页主要解决的是 TLB reach 问题。

题 2：为什么 TLB shootdown 必须"等所有 ack"？漏掉一个核会发生什么？

【答案】 因为**翻译是缓存在每个核上的、硬件不保证一致的状态**。页表更新（改权限、换页、取消映射）**不会**自动让其它核的 TLB 项失效（讲义 slide 41）。如果发起核在没等到某个核的 ack 之前就"认为变更已生效"，那个核会继续用一个**陈旧的翻译**去访问，具体后果分三类：

1. **权限绕过（安全漏洞）**：典型场景是把页**收紧为只读**（例如 `fork` 之后父子共享的 COW 页、GC 的写屏障保护、W^X 的代码页、JIT 后加只读/可执行的代码段）。若某核仍持有"可写"的旧翻译，它就能写入本应只读的页，**破坏只读不变量**——这类漏洞历史上真实存在（例如"写时复制的页被越权写入"导致的隔离失效）。
2. **静默数据损坏（更常见、更难查）**：页被换出（swap）或被 OS 回收后，**同一个物理页框可能已经分配给别的进程**。旧翻译会把这个核的写操作送到**属于别人的物理页**上：数据静默丢失 + 信息泄漏，而且往往只在负载高、内存紧张时偶发。
3. **语义错乱**：`munmap` 之后的 use-after-unmap、`mremap` 之后越界访问到已经被用作其它对象的内存，表现为随机崩溃或"不可能复现"的 bug。

所以 shootdown 的 **"等所有 ack"不是性能优化，而是正确性要求**。它要维持的不变式可以写成：

```
当 PTE 锁被释放(即变更宣告完成)时，
系统中不存在任何一个核，其 TLB 里持有与该 PTE 当前值不一致的翻译。
── 实现它需要：在临界区内完成 (a) 页表更新，(b) 对所有可能缓存该翻译的核
   发送失效请求，(c) 收齐 ack 才能解锁。
```

补充两点：(1) **ack 的语义是"该核的 TLB 项已失效"，而不是"该核永远不会再装填"**——所以"失效"与"页表更新"的**顺序和内存屏障**必须保证不会有核在"失效之后、新值可见之前"重新装填旧翻译；(2) 代价上，这个 rendezvous 让**所有被牵连的核一起停下来等**，因此如果目标核长时间关中断或处于长临界区，发起核（乃至整个系统）会被拖住——这正是讲义 slide 52 甘特图与 slide 53（Multikernel 论文数据：8 核以上 >10,000 周期）要强调的扩展性问题。

题 3：大页的收益、代价，以及为什么"数据中心里越来越难拿到大页"

请从 TLB reach、页走级数、缺页次数、内存碎片、COW 语义五个角度分析大页（2 MB）相对 4 KB 页的收益与代价，并说明在长时间运行的共享机器上"请求大页却拿不到"会发生什么。

【答案】 收益（四项，都可定量）： 1. **TLB reach ×512**：4 KB 页下 3072 项 STLB 只覆盖 12 MiB；2 MB 页下覆盖 6 GiB，1 GB 页下覆盖 3 TiB。对 1 GiB 随机访问负载，STLB 命中率从 ≈1% 变成 ≈100%。 2. **页走级数减少**：PMD 项直接指向 2 MB 页 → 页走**少一级**（少一次依赖访存）；在虚拟化下，guest 每少一级页走就**少一整套 4 级嵌套翻译**（讲义 slide 58），收益被放大数倍。 3. **缺页/页表开销降低**：1 GiB 区域从 262144 个 PTE 变成 512 个 PMD 项，**页表内存从 2 MB 级别降到几十 KB 级别**，缺页处理次数也随之下降（在首次触碰阶段尤其明显）。 4. **翻译税消失的实测证据**：3.1 节里同一访问序列、只把页大小从 4 KB 换成 2 MB，随机指针追逐从 24.24 ns/hop 降到 19.16 ns/hop（32 MiB 足迹，8192 个 4 KB 页 → 16 个 2 MB 页）。

代价（五项）： 1. **内部碎片**：分配粒度 2 MB，需要 8 KB 却拿到 2 MB → 内存利用率下降；多进程/多容器场景下更容易触发 OOM。 2. **COW / 写时复制放大**：`fork` 后只改 1 个字节，内核可能要复制**整个 2 MB**（而不是 4 KB），`fork` 密集型服务（如 Redis、prefork 模型）性能显著变差。 3. **首次触碰更贵**：新大页必须清零（安全要求）——2 MB 按 20–50 GB/s 的清零带宽约 **40–100 µs**，比 4 KB 页高出两个数量级；这也让"按需增长"的负载出现延迟尖刺。 4. **回收/换页/超分粒度变粗**：内核回收整页，2 MB 的粒度让内存压力下的行为更粗暴，且难以做精细的 swap/压缩；内存统计（RSS、cgroup）也失真。 5. **分配需要连续物理内存**：必须有 2 MB 物理连续块，需要内存压缩（compaction）配合，**可能失败且延迟不可控**。

"拿不到大页"会发生什么（本机实测）：在这台长期运行（已连续运行 118 天）、内存已被大量占用的共享机器上，对 1 GiB 匿名区域调用 `madvise(MADV_HUGEPAGE)` 后，`/proc/self/smaps` 只报告 **72 MiB 的 AnonHugePages（7%）**；把区域缩小到 32 MiB 时才拿到 **100%**。后果是：**"优化"静默失效**——代码里写了 `MADV_HUGEPAGE`，但翻译税照付；更糟的是"部分生效"会让性能在不同运行之间抖动（大页落在哪里取决于当时的物理内存状态）。这正是讲义第 3–4 页那张 Contiguitas（ISCA '23）配图指向的问题：**数据中心里物理内存连续性在下降，翻译成本在系统性上升**。工程做法：(1) 用 `smaps` 的 `AnonHugePages` 或 `perf stat -e dTLB-load-misses` 之类的 PMU 事件**验证**而不是假设；(2) 对延迟敏感的服务用**显式 hugetlbfs 预留**（启动时保留，保证可用，代价是不能弹性扩展）；(3) 用**页迁移/compaction**在低负载时整理内存；(4) 至少保证"要么全用大页、要么全不用"，避免混合状态造成的性能抖动。

Lecture 20: Guest Lecture

> **日期**：Oct 19 ｜ **材料状态**：未公开 官方日程表该行仅写作 "Guest Lecture"，未提供 slides 或录像链接；Fall 2026 的 guest lecture 讲者与题目在抓取时（2026-09）尚未公布。

概述

本讲由工业界或学术界客座讲者主讲，主题每年不同，通常与当年热点（AI 基础设施、新型加速器、大规模分布式训练、编译器/运行时工程等）相关。由于课程未公开其材料，本笔记无法提供基于官方讲义的逐页笔记。

可从课程其他讲次获得的对应背景

课程日程中保留的**归档客座讲座行**提供了往届类似主题，可作为知识补充：

归档条目	主题	对应本笔记
Oct 29（归档）	ML accelerators at Amazon（guest lecture by Randy Huang and Ron Diamant）	Lecture 27: Deep Neural Networks and ML Accelerators
Mar 24（归档）	Under the Hood: Message Passing Implementation	Lecture 25

考试相关

官方明确说明：**Exam 2 覆盖 Lecture 14–24，但排除 guest lecture**，因此本讲不进入考核范围。

学习建议

1. 若你是选课学生，务必现场听讲并自行记录（客座讲座的内容通常不会出现在任何公开讲义里）。
2. 若你是在自学，可用以下三条线索自行补足"工业界视角"：
3. **专用加速器**：见 Lecture 18（硬件专用化的能效谱系）。
4. **大规模训练系统**：见 Lecture 22–23（数据并行、张量并行、流水线并行的通信记账）。
5. **性能工程实践**：见 Lecture 14（测量方法论与高水位实验）。

Lecture 21: Memory Consistency

1. 章节标题与概述

Lecture 21: Memory Consistency（内存一致性模型：顺序一致性、宽松序、fence 与 DRF 契约）

- **本讲核心问题**：**"读一个地址应该返回最近一次写入的值"——可是"最近"到底是什么意思？** 讲义开篇（slide 3）就把这个看似显然的问题拆开：在线程内部，"最近"可以由 **program order（程序序）** 定义；但**跨线程**呢？如果定义为"物理时间上最近的那次写"，硬件根本做不到——"如果处理器之间通信需要 10 个周期以上，P0 就绝无可能知道 P1 在 2 个时钟节拍之前干了什么"。因此正确答案只能是：**存在某个把所有线程的操作串起来的假想顺序（a hypothetical serializable order），读回来的值必须与这个顺序相容**——而这个顺序**不必是物理时间顺序**。于是问题变成：我们要**允许多少种串行化顺序**？允许得越少，程序员越好写，硬件越慢；允许得越多，硬件越快，程序员越容易写出错的程序。这一讲就是在"正确性契约"和"性能"之间划出那条线。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：为了隐藏访存延迟而引入的三种重排来源——**写缓冲（write buffer）**（写延迟被"吃掉"，但其他核可能先看到我的读、后看到我的写）、**乱序流水线 / 发射（out-of-order issue）与 Reorder Buffer**（一条 cache miss 挂起时后续独立指令继续发射，因此**访存被乱序执行**）、**分支预测与投机执行（speculative execution，预测错就 squash）**；以及为把这些重排"圈回来"而提供的 **fence / memory barrier** 指令：Intel 的 **MFENCE / LFENCE / SFENCE**、隐式带锁的 `xchg`，和 C++11 的 `std::atomic_thread_fence`。芯片层面还依赖缓存一致性协议（MSI/MESI、BusRd/BusRdX/BusWB 事务）与互连这个**串行化点**，来保证"写何时算完成"这件事有唯一的仲裁者。
- **软件侧**：四种被模型允许或禁止的访存顺序约束（**WX→RY、RX→RY、RX→WY、WX→WY**）、由它们组合出的模型族（**SC、TSO / Total Store Ordering、PC / Processor Consistency、PSO / Partial Store Ordering、WO / Weak Ordering、RC / Release Consistency**）、**acquire/release 语义**、**数据竞争（data race）**与**properly synchronized program**、以及现代语言给出的 **"SC for DRF"（data-race-free 程序获得顺序一致性）契约**（C11 / C++11 / Java 5）。
- **在并行计算知识体系中的角色**：这是"共享内存正确性"这条链上的**最后一块拼图**。前几讲分别回答了"多份缓存副本怎么保持一致"（缓存一致性 / MSI–MESI）、"怎么让它可扩展到多核"（目录一致性）、"同步原语怎么实现"（原子指令、锁、屏障）；本讲回答的是**更高一层的问题：不同地址之间的可见顺序由谁定义**。它把课程里所有"看起来能跑但偶尔错"的 bug 归因到一个统一的根源——**程序里有数据竞争**——并给出工程上的解法：要么用库提供的同步原语（lock/unlock/barrier）把程序变成 DRF，要么在无锁代码里精确地放置 fence 与 acquire/release。它也是下一讲 **Fine-Grained Synchronization / Lock-Free Programming** 的前置知识：无锁数据结构里每一处 `relaxed` / `acquire` / `release` 的取舍，都直接由本讲的一致性模型决定。
- **配套材料**：
- `lectures/15_consistency.pdf`（抽取文本 `extracted/15_consistency.txt`，共 **40 页**）——**已公开，可在公开网络直接下载**。Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）把 **Oct 21** 排为第 **21** 讲 "Memory Consistency"，该行的 slides/video 链接目前**以**

HTML 注释形式给出（注释原文："slides/video from a previous offering; uncomment when posted for Fall 2026"），注释中的 slides 指向 `lectures/15_consistency.pdf`，video 指向一个归档 YouTube 链接。该 PDF 位于公开目录 https://www.cs.cmu.edu/~418/lectures/ 之下。**两点说明（不是错误）**：①**文件编号 15 与 Fall 2026 讲次 21 不一致**，且讲义首页写的是 **"CMU 15–418/15–618, Fall 2025 / Lecture 15: Memory Consistency"**——这是**讲义沿用历史学期版本**的正常现象（讲次编号与学期字样随年度重排）；②PDF 中有几页（如第 2、4 页，以及第 6、21 页的图示部分）在文本抽取中为空，因为它们**本身是纯图片页或图多于文**，本笔记对这些内容只依据同页可见文字与公开成熟知识展开，不冒充原文数据。

- `cs149_supp/sync_consistency.txt`（抽取文本，共 **60 页**）——**已公开**。Stanford CS149（Fall 2025）**Lecture 15: Memory Coherency and Consistency**：前 20 页（slide 1–20）讲**缓存一致性**（MSI/MESI 状态机、目录一致性、伪共享实测 14.2 s vs 4.7 s、Core i7 存储层次延迟），从 slide 21 起进入 **Memory Consistency**：SC 的开关隐喻、"写缓冲如何改变内存行为"、TSO / PC / PSO / WO–RC、fence、数据竞争与 DRF、语言级内存模型。它是本讲**最完整的公开文本来源**之一，本笔记用它补足 CMU 讲义中只给图示的部分，并对齐两边的结论。
- 讲义中引用的外部材料：Lamport 1979 的 SC 形式化（slide 19）；Gupta et al., *"Comparative evaluation of latency reducing and tolerating techniques"*, ISCA '91（slide 26 的性能数据出处）；Intel® 64 and IA–32 Architectures Software Developer's Manual, Volume 3A, Section 8.2 / 8.2.5（fence 指令规格）；ARM Barrier Litmus Tests and Cookbook；以及剑桥的弱内存模型文献列表 http://www.cl.cam.ac.uk/~pes20/weakmemory/。
- **未发布 / 需登录**：本讲的**讲课录像（Panopto / YouTube）在 Fall 2026 日程表中被注释隐藏，属未发布**；**Ed 讨论区、Autolab、Canvas 均需登录**，非公开。部分讲座（Performance Analysis/Profiling、Transactional Memory、AI in System Design 等）在 Fall 2026 **尚未发布讲义**，其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- **课程语境**：Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。前一讲为 Lecture 20 "Guest Lecture"（Oct 19，讲义未公开），后一讲为 Lecture 22 "Parallel Deep Learning (data parallelism)"（Oct 23，对应公开 PDF `24-parallel_deep_learning_data_parallel.pdf`）。

---

2. 核心概念与硬件/软件架构图解

2.1 先分清两个问题：Cache Coherence 与 Memory Consistency

- **定义与目的**：讲义在 slide 8 把"并行内存层次结构的正确行为"明确拆成两半：
- **Cache Coherence（缓存一致性）**：**对同一个 cache block 的所有 load/store 是否表现正确?**
- **Memory Consistency Model（内存一致性模型）**，有时也叫 **Memory Ordering**：**对（即使位于不同 cache block 的）所有 load/store，整体是否表现正确?** CS149 的讲义（slide 24–25）把这条界线说得更直白：**coherence 只保证"对 X 的写最终会传播到其他核"，consistency 决定的是"对 X 的写相对于对其他地址的读写，何时传播"**。更进一步："缓存一致性的目标是让有缓存的多核内存系统表现得**好像缓存不存在**；而 memory consistency 描述的是**地址与地址之间**允许的行为——**无论系统里有没有缓存，这个问题都要回答**。"
- **直观解释（"它是什么？"）**：把共享内存想成**同一家公司里的多块白板（不同地址）**。
- *Coherence* 管的是**同一块白板上的一次涂改**：所有人最终看到的是同一个笔迹序列，不会出现"A 看到 5、B 看到 3"这种对同一块板的分歧。它像"白板管理员"（互连上的串行化点）——**一行的分歧由管理员仲裁**。
- *Consistency* 管的是**两块白板之间的先后**：我在 A 板上写了"货已到"，又在 B 板上写了"请取货"。别的同事**先看到哪块板**? coherence 对此**一句话都没说**——它只保证两块板上的新字**最终**都会传出去。于是完全可能：对方先看到 B 板的"请取货"，跑去一看 A 板还是旧字。
- 这个类比的结论是：**coherence 管"同一个地址的历史"，consistency 管"不同地址之间的相对顺序"**；前者是缓存引起的，后者不是。
- **图 1：两个问题的作用范围（同一地址的时间线 vs 跨地址的相对顺序）**

```
── Coherence 的问题范围：同一个地址 X 的一条时间线 ──────────────────────────
     address X 的时间线：   P0:W(5)   P1:R(5)   P2:R(5)   P2:W(10)   P1:R(10)
                            └──────────────────────────────────────────────┘
       不变量：SWMR（单写者多读者）+ 写串行化 + 读回"串行序中的最后一次写"
       允许并发：多个 S 态读者；一旦有人要 W，其他副本全部失效
       ⋆ 有缓存才有这个问题（多份副本）；没有缓存就不需要 coherence

── Consistency 的问题范围：不同地址之间的可见顺序 ──────────────────────────
     address X:   P0:W(X=1) ──────────────────────────► (何时对 P1 可见？)
     address Y:      P0:W(Y=1) ──────────────────────► (何时对 P1 可见？)
                           ▲
                           └── 真正的问题在这里：X 的写与 Y 的写之间，
                               P1 可能只看到其中一个（顺序被交换/被拖延）
       ⋆ 无论有没有缓存都要回答；它是一条"硬件/编译器 vs 程序"的契约
       ⋆ 不能用锁"修好"：锁修的是 mutual exclusion（互斥），
         而"多份副本"与"重排"是硬件实现导致的、锁管不到的现象
```

- **性能特征**：coherence 的代价表现为**一致性流量**（一行 ping-pong、伪共享），可用"每条 cache line 字节数 × 事务数 / 互连带宽"来估算；consistency 的代价表现为**被禁止的优化**（写缓冲能否用、能否乱序发射、能否投机），最终体现在**处理器利用率**上——讲义 slide 26 引用 Gupta et al. 的数据：严格的 SC 实现即使有缓存，处理器利用率也只有 **17%–42%**。

2.2 为什么"最新值"无法定义：program order 与 physical time 的分离

- **定义与目的**：讲义 slide 5 用一个经典例子把"跨线程的最近"逼到墙角（下面的 X 初值为 0，且这些是唯一的写）：

```
// Thread 0：把偶数写进 X          // Thread 1：把奇数写进 X        // Thread 2：读三次
for (i = 0; i < N; i += 2) {       for (j = 1; j < N; j += 2) {     A = X;
    X = i;                             X = j;                       ...
    ...                                ...                          B = X;
}                                  }                                ...
                                                                    C = X;
```

讲义问：**(A,B,C) 中哪些组合"明显非法"？** 例如 `(4,8,1)`、`(9,12,3)`、`(7,19,31)`。 – **你能确定的只有两条**：① **同一线程的写必须与 program order 一致**——所以观察到的偶数必须递增（奇数同理）；② **跨线程时，写必须与"线程的某个合法交错（valid interleaving）"一致**——而**不是**与物理时间一致。 – 也就是说 `(4,8,1)` 是**合法**的（等价于"交错"里 T0 先写了 4、8，然后 T1 写了 1，之后 T2 才读），而 `(8,4,…)`（偶数递减）**非法**。

- **直观解释（"它是什么？"）**：把三个线程想成**三个厨师往同一口锅里撒调料**，你站在锅边尝三次。你尝到的味道必须能由**某个**"按顺序一个个撒"的过程解释出来（哪怕实际上他们是同时撒的、撒的顺序你无从得知）。这就是"存在一个假想的串行交错"的含义——讲义 slide 7 特别强调：**"这是一个假想的交错；机器并不一定真是这样执行的！"**
- **图 2：SC 的"开关"模型（软件执行模型）vs 真实硬件（硬件结构）**

```
【左】SC 的 switch 模型（Lamport 1979 / CS149 slide 31）：
      内存每一步只服务一个处理器的一条访存，服务完再随机挑下一个。
              +-----------------------------------------+
              |  Memory   A=0  B=0  X=0   （单一串行序）   |
              +-----------------------------------------+
                 ^          ^           ^          ^
    一次只放行一个|          |           |          |
              +-------+  +-------+   +-------+  +-------+
              |  P0   |  |  P1   |   |  P2   |  |  P3   |
              | A = 1 |  | B = 1 |   |       |  |       |
              | r1= B |  | r2= A |   |       |  |       |
              +-------+  +-------+   +-------+  +-------+
   规则：每个处理器内部按 program order 发射；内存一次执行到底。
   等价说法：所有访存存在一个全局串行序，且该序与每个线程的 program order 相容。

【右】真实硬件：为了藏住延迟，几乎一切都可以被重排（slide 9–13）
   program order（程序员看到的）              硬件实际做的事
   ------------------------------------    ------------------------------------
   x = *p;     （L1 miss, 100+ 周期）    --->  挂起这条 miss
   y = x + 1;  （真依赖 true dependence)--->  必须等 x，无法发射
   z = a + 2;  （无关）                  --->  【越过上面的 miss 先发射】
   b = c / 3;  （无关）                  --->  【越过上面的 miss 先发射】
   if (x != z) d = e - 7;               ---> 分支预测 + 投机执行；猜错则 squash
   结论（slide 9）："处理器在单线程内维持了 program order 的幻觉，但这个幻觉
   与物理时间毫无关系；从其他线程的视角看，一切皆有可能。"
```

2.3 硬件为什么会重排：写缓冲、乱序发射与投机执行

- **定义与目的**：讲义 slide 10–13 给出重排的三个物理来源，并且强调"**隐藏访存延迟对性能至关重要**"：
- **写缓冲（write buffer）**：在单处理器上隐藏**写**延迟的最简单办法。"（但它在多处理器上影响正确性）"。
- **乱序流水线（out-of-order pipelining）**："当一条指令卡住时，也许后面还有能执行的指令"——**推论：访存可能被乱序执行！**
- **分支预测 + 投机执行**：条件分支不必等结果，先猜、先执行，猜错再回滚。 slide 13 给出微架构画面：**取指与退休（graduate/retire）按序，但发射（issue）乱序**；线程内依赖被保持，**访存被重排**。
- **直观解释（"它是什么？"）**：把**写缓冲**想成**快递代收点**。你把包裹放到代收点（store 进写缓冲），就算"寄完了"，可以立刻去干别的；代收点随后慢慢发车（冲刷到一致性域）。问题是：你的朋友此时打电话问"我的包裹到了吗"（其他核的 load），代收点还没发车，他当然说"没有"——**虽然你早就"寄"了**。乱序流水线则像**超市的多个收银台**：前面那位顾客的扫码枪坏了（cache miss），后面排队的顾客会被引导到别的台子先结账（乱序发射）——**结账（退休）顺序还是按照排队的号，但"谁先被服务"完全乱了**。
- **图 3：写缓冲 + 乱序发射的硬件结构（含关键操作与延迟特征）**

```
    +---------------------- Processor core ------------------------+
    |  Fetch(in-order) -> Decode -> Rename -> Issue(OUT-OF-ORDER)  |
    |                                          |                  |
    |                                      +------v-------+       |
    |                                      | Reorder      |       |
    |                                      | Buffer (ROB) |---+   |
    |                                      +--------------+   |   |
    +---------------------------------------------------------|---+
                                                              |
                                +-----------------------------v--------------+
      LOADS  <------------------+---+  Load Queue (检查写缓冲 = store-to-  |
                                |   |              load forwarding)      |
                                |   +----------------+                    |
                                |   +----------------v---+   +----------+ |
      WRITES ------------------>+---| L1 D-Cache / MSI |   |  Store   |-+--> 一致性域
                                |   | 状态: M/E/S/I    |   |  Buffer  |      (L2/L3/互连)
                                |   +------------------+   | (写缓冲) |      与互连的
                                +--------------------------+----------+       串行化点
    关键操作与性能特征
    --------------------
    * load 命中 L1:             约 4 周期 (CS149 slide 14 数据)
    * load 命中 L2:             约 10 周期
    * load 命中 L3 但行在别的核:  约 65 周期 (shared) / 约 75 周期 (modified)
    * load 落到本地 DRAM:        约 30 ns ≈ 120 周期
    * load 落到远端 DRAM:        约 100 ns ≈ 400 周期 (NUMA)
    * store: 只要写进缓冲就"完成" (对发射流水线而言), 延迟被隐藏;
      但它对**其他核**何时可见, 取决于何时从缓冲冲刷到一致性序 —— 这就是
      WX -> RY 被重排的物理来源, 也是 x86-TSO 唯一放松掉的那一条约束。
```

- **性能特征**：写缓冲把**写延迟**从关键路径上摘掉（这是 TSO 相对 SC 的全部收益来源），但它带来一个不可逆的正确性后果：**本线程后面的读可能"绕过"本线程前面的写**。乱序发射与投机执行把**读延迟**藏进指令级并行（ILP）里，代价是**读与读、读与写之间的顺序也可能被交换**。二者的共同本质：**硬件的优化都是"对单线程语义无损"的，而对多线程语义是有损的**（CS149 slide 48 的原话："这些都是合法优化——如果程序只包含一条指令流的话"）。

2.4 Sequential Consistency（SC）：程序员的心智模型

- **定义与目的**：Lamport 1979 形式化（slide 19；CS149 slide 30 记为 Lamport 1976，两者都指向同一篇工作）：
- **每个处理器的访存按 program order 出现**；
- **所有访存出现在一个顺序（sequential）序中**。 等价地（CS149）：**所有操作在某个顺序序中执行，就像它们在操作一块单一共享内存，且每个线程的操作按程序序发生**。讲义指出："**程序员隐含假设的任何顺序都被保持**。"
- **直观解释（"它是什么？"）**：SC 就是**把多核当单核用**：想象一台机器只有一条内存总线和**一个开关**，开关轮流接通一个处理器，让它把**下一条**访存执行完，再换下一个（CS149 slide 31 的 "switch metaphor"）。对程序员，这等价于"**并发 = 交错的串行**"——你不需要知道任何硬件细节，只需像分析单线程交错那样分析。这正是**教科书式的共享内存语义**。
- **图 4：SC 的实现方式（"一访问一停顿"）与它的性能灾难**

```
实现 SC 的两条规则 (slide 22):
(1) 实现 cache coherence  ->  对同一地址的写被所有处理器以相同顺序观察到
(2) 每个处理器: 下一条访存必须等上一条"完成"才开始
    -> 每个处理器任何时刻只有 1 个 outstanding memory access

什么叫"完成" (slide 23–24)?
  读: 返回值被绑定 (its return value is bound) 时完成。
  写: 新值对其他处理器"可见"时完成 —— 注意"可见"**不**意味着别人已经看到,
      而是意味着该写已经提交到 **HSO (hypothetical serializable order,
      假想可串行化顺序)**: HSO 中对该地址的后续读只能看到这个值或更晚的值。
      (讲义为简化假设写是原子的。)

执行时间线 (每一格都是一次访存, 格子里的等待是硬性停顿):
  LD ----[ 等 L 周期 ]----> LD ----[ 等 L 周期 ]----> ST ----[ 等 L ]----> LD ---
  |<----------------- 处理器利用率 = 理想发射时间 / 实际时间 ----------------->|
  ★ 讲义 slide 26: 即使有缓存, 处理器利用率也只有 17% – 42%
     (数据来源: Gupta et al., ISCA '91)

气球类比 (slide 25): SC 相当于"在**每一个**有序的气体粒子之间都打一个扭结"
  —— 粒子完全不能乱动, 性能自然崩坏。
```

- **性能特征**：SC 禁止了三件价值极高的优化：**写缓冲**（W 无法被隐藏）、**乱序发射**（R 无法被隐藏）、**编译器的寄存器分配与代码移动（code motion）**。因此它的代价不是"多几条指令"，而是**把内存延迟完整地暴露在关键路径上**——延迟 L 有多大，损失就有多大（见第 4 节算例）。

2.5 解耦出四类 ordering，以及各模型放松了哪一条

- **定义与目的**：CS149 slide 27 把"程序序中的两条相邻访存 op1 → op2"按类型分成四类，这样就能逐条讨论"哪些可以放松"：

顺序约束	含义（program order 中前者是 op1，后者是 op2）	硬件/编译器为什么想破坏它	哪些模型放松了它
WX → RY	对 X 的写必须在随后对 Y 的读之前"提交"（commit）	**写缓冲**：写进缓冲就算完成，后面的读不必等它传播出去	TSO、PC（以及所有更弱的模型）
RX → RY	对 X 的读必须在随后对 Y 的读之前完成	**乱序发射**：两条独立的 load 可以并行发出，返回顺序不定	WO、RC（以及 ARM/RISC-V 等宽松架构）
RX → WY	对 X 的读必须在随后对 Y 的写之前完成	乱序发射 / 编译器 code motion（读早发、写晚发）	WO、RC
WX → WY	对 X 的写必须在随后对 Y 的写之前提交	写缓冲内部的重排/合并（一个 miss 一个 hit）	PSO（以及 WO/RC）

讲义 slide 26–28 与 CS149 slide 37–49 用同一套图表达这件事：**SC 保持全部四条；TSO/PC 只放松 WX→RY；PSO 再放松 WX→WY；WO/RC 在同步点之间放松全部四条**。slide 28 直接给出 TSO 与 PSO 的图示对比，并指向 Intel SDM Vol. 3A 第 8.2 节。

- **直观解释（"它是什么？"）**：把四条约束想成**四道不同强度的"队规"**：
- WX→RY 是"**寄完信再去问别人有没有给我寄信**"——允许你在信还在路上的时候就去问（写缓冲）；
- RX→RY 是"**两个窗口同时问，谁先回答算谁的**"——允许两个读乱序返回；
- RX→WY 是"**先打听清楚再下单**"——允许你先把打听的请求发出去、把下单压后；
- WX→WY 是"**两封信的寄出顺序**"——允许两封信被合并或换序装车。 放松的条数越多，硬件越自由、越快，而程序员需要自己补的"故事"越多。
- **表 1：主流内存一致性模型对比（综合 CMU slide 27–28、33、37–38 与 CS149 slide 43–51）**

模型	放松了哪些顺序	典型实现/架构	同步手段	程序员负担
SC Sequential Consistency	无（四条全保）	理论模型 / 教学模型；某些顺序核 + 编译器屏障	不需要（语义已最强）	最低（"并发 = 交错串行"）
TSO Total Store Ordering	仅 WX→RY（且只是**本处理器自己的读**可以越过自己更早的写）	**Intel x86/x64**（"incompletely specified form of TSO", CS149 slide 43）	`MFENCE`、`LFENCE` / `SFENCE`、`xchg`（隐式全屏障）	低：x86 上大部分代码"看起来"就是 SC
PC Processor Consistency	放松 WX→RY，且**其他处理器**也可能先看到新值 A、后看到更早写的 B	部分历史多处理器	同上	中
PSO Partial Store Ordering	WX→RY + WX→WY（同线程的两次写可换序）	若干 RISC 多处理器（如 SPARC 变体）	`STBAR`/fence	中高
WO Weak Ordering	同步点**之间**四条全放松；同步点（lock/unlock/barrier）前后必须等先前操作全部完成	历史研究型多处理器	同步点两侧的 fence	高（但语义在"properly synchronized"前提下仍等价的 SC）
RC Release Consistency	同 WO，但**区分 acquire 与 release**：release 前必须完成且只能等自己的写，acquire 后必须等自己的读	大量现代架构的思想基础；C++11 的 acquire/release 正是它的直接体现	`acquire` / `release` fence 或带语义的原子操作	中（库封装后对应用程序员几乎不可见）

- **性能特征**：模型越弱，"能藏起来的延迟"越多。CS149 slide 42 给出的对比曲线里，横轴是 SC（"Base"）与放松 W→R 的版本（"W-R"）：**放松 WX→RY 之后，写延迟几乎被完全隐藏**——这就是为什么"**每一款现代处理器都使用写缓冲（Intel x86、ARM、RISC-V）**"（CS149 slide 43），也因此**都必须提供比 SC 更弱的内存模型**。

2.6 fence：在气球上打一个跨线程可见的"扭结"

- **定义与目的**：讲义 slide 33 用 Intel 的 **MFENCE** 说明 fence 的语义：
- **MFENCE 之前**：该线程**所有**先前的读与写都必须**完成**，MFENCE 才能开始；

- **MFENCE 之后**：该线程**任何**后续的读或写都必须等 MFENCE 结束才能开始。 讲义用气球类比总结："**fence 就是气球上的一个扭结——没有任何气体粒子能穿过它。**" 好消息（slide 34）：**`xchg` 隐式地做了这件事**（x86 上带内存操作数的 `xchg` 自带 lock 语义，等价于全屏障）。
- **直观解释（"它是什么？"）**：想象**一列正在乱跑的孩子（重排的访存）**。你不能让他们站好队（那会毁掉性能），但可以在走廊上放**一道闸门**：闸门左半边的孩子必须**全部通过闸门**之后，右半边的孩子才能开始通过。于是**闸门两侧的相对顺序对走廊外的人（其他线程）是确定的**，而**同一侧内部依然可以乱跑**。fence 的全部价值就在这个"**局部有序、全局可观察**"的折中上。
- **关键澄清（讲义 slide 35，最常见的误解）**："**MFENCE 不会把值推给其他线程。**" 它**不是**"让所有线程立刻看到最新值"的魔法操作；它**只是让执行它的那个线程停顿**。它真正产生的效果是：**MFENCE 在跨线程可观察的偏序上引入了若干约束**。
- **图 5：fence = 气球上的扭结（跨线程可观察的偏序）**

```
Thread 0 的气球（每个粒子 = 一条访存指令；编号 = program order）
  ... 3  1  5  2  4 | 8  6  9  7 | 11 10 12 ...
                    ^            ^
                    |            |
                 MFENCE       MFENCE      <-- 扭结：粒子无法穿越

┌──────────────────────────────────────────────────────────────────┐
│ 扭结之前的集合 {…1,2,3,4,5} 与之后的集合 {6,7,8,9,…} 之间的相对顺序，     │
│ 对所有线程都是一致的可观察事实（跨线程偏序）；                              │
│ 但每个集合**内部**仍然是乱序的 —— 这正是"打扭结"而不是"排队"的意义。      │
└──────────────────────────────────────────────────────────────────┘

四个线程放 fence 的效果（slide 35 的示意图）：
     Thread 0:  4  3  1  5  2 | 5  1  4  3  2
     Thread 1:  2  3  4  1  5 | 3  5  2  1  4
     Thread 2:  1  5  4  2  3 | 2  3  4  5  1
     Thread 3:  3  2  5  1  4 | 4  1  5  2  3
                              ^
                    MFENCE 把时间轴切成"段"，段与段之间的先后
                    对所有线程可见；段内任意乱序。
```

- **Intel 的三种 fence（讲义 slide 38 + CS149 slide 51）**：

指令	序列化范围	讲义/规范原话与说明	实用建议
MFENCE	**全部**读与写	"does not begin until all prior reads & writes from that thread have completed; no subsequent read or write from that thread can start until after it finishes"	最常用；需要"全屏障"时的默认选择
LFENCE	只对 **load** 序列化（**不含 store**）	讲义提醒："It does slightly more than this; see the spec"（Intel SDM Vol. 3A §8.2.5）	少见；主要用于精确控制读顺序与某些序列化场景
SFENCE	只对 **store** 序列化（**不含 load**）	同上，规范中比一句话描述更复杂	用于"写后写"和 release 语义
`xchg`（带内存操作数）	**隐式全屏障**	slide 34："Good news: xchg does this implicitly!"	用它同时实现"原子交换 + 顺序约束"，是自旋锁的经典写法
`std::atomic_thread_fence(seq_cst)`	C++11 层等价物	编译器 + 硬件两层屏障；与 `relaxed` 原子操作配合可实现 fence–fence 同步	可移植代码的首选

- **性能特征**：fence 的代价是**排空（drain）**：MFENCE 必须等到所有未完成的 store 从写缓冲冲刷到一致性序。若写缓冲里有 k 条未提交的 store、每周期能冲刷 1 条，则代价 ≈ k 个周期再加流水线扰动；**空缓冲时约 20–40 周期（3 GHz 下约 7–13 ns），缓存/互连拥塞时可达成百周期**——但注意这只是“排空”那一部分的成本：示例一在真实 x86 上实测到的**端到端边际成本是 +43 ns/轮（≈133 周期）**，因为 fence 还额外牺牲了“读与写重叠”这个优化。所以 fence 的正确用法是"**少而准**"：能一次 fence 覆盖一批共享更新，就绝不做"每元素一次 fence"（见第 4 节的 fence 吞吐算例）。

2.7 数据竞争、properly synchronized program 与 DRF 契约

- **定义与目的**：讲义 slide 30–31 给出严格定义：
- **冲突（conflict）**：两次访存**访问同一地址**，且**至少一次是写**；
- **ordering**：按 **program order (po)** 或 **dependence order (do)**（若 op2 读到 op1 的结果，则 op1 → op2）；
- **数据竞争（data race）**：两次冲突的访存在**不同处理器**上，且**中间没有别的访存把它们排好序**（not ordered by intervening accesses）；
- **properly synchronized program（正确同步的程序）**：**所有同步都被显式标识**，并且**所有数据访存都通过同步而被排序**。

- **关键定理（"为什么我们还能活下去"）**：CS149 slide 54 给出结论——**"同步的程序在非 SC 系统上产生 SC 的结果"**："If there are no data races, reordering behavior doesn't matter（**没有数据竞争，重排就无关紧要**）"，因为访存已被同步排好序，而**同步强制了顺序一致性**。slide 59 把它抬到语言层：**C11 / C++11 / Java 5 保证"对无数据竞争的程序提供顺序一致性"（SC for DRF）——编译器会替你插入应对硬件内存模型所需的同步**；而**"如果你的程序有数据竞争，则没有任何保证"**。
- **直观解释（"它是什么？"）**：把程序想成**一份会议纪要**。同步操作（lock/unlock/barrier/acquire/release）就是**纪要里的"分节标题"**；DRF 意味着**每一段共享数据的讨论都被某个分节包住了**。这种情况下，**同一节内部谁先谁后无所谓**（那是"私人活动"，slide 32 的措辞：临界区内"插入数据结构的节点"本质上是 private 的，随便重排都行），而**节与节之间的先后是硬性的**。反过来说：**没有分节标题的纪要，谁读都可能读出不同的时间线**——那就是数据竞争（未定义行为），也是本讲所有"诡异 bug"的统一来源。
- **图 6：release/acquire 如何把两段"私人活动"缝成跨线程的偏序（软件执行模型）**

```
Thread A (producer)                       Thread B (consumer)
-------------------                       -------------------
data = 42;              (1)
     |  po (program order)
     v
flag.store(1, release)  (2)  ===== synchronizes-with =====>  (3) flag.load(acquire)
                                                                   |  po
                                                                   v
                                                            r = data;            (4)
── 偏序图 (happens-before DAG) ──────────────────────────────────────────
     (1) --po--> (2) --sw--> (3) --po--> (4)
     于是 (1) happens-before (4)：消费者在 (4) 处**必然**读到 42。
── 若把 (2)/(3) 换成 relaxed ────────────────────────────────────────────
     (2) 可与 (1) 重排；消费者侧 (4) 也可能被提前发出
     => 可能出现 (3) 看到 flag=1 而 (4) 读到 data=0
── 若用 WO (Weak Ordering) 而不是 RC (slide 33、37) ──────────────────────
     WO：lock 和 unlock **两侧**都必须等先前操作全部完成（过于保守）
     RC：unlock 前只需保证"我的写"完成，lock 后只需保证"我的读"不早于它
         => 单个同步操作要等的操作数减少，这就是 RC 相对 WO 的收益（slide 37：
            "Overly Conservative" 的箭头正好画在 WO 多余的那一侧）
```

- **性能特征**：DRF + RC 的组合把"一致性模型的复杂度"**从应用代码里赶进库里**：应用程序员看到的是 `lock/unlock`、`barrier`、`atomic<int>`，而 acquire/release 的具体位置由库作者决定（slide 40 的总结："in practice: complexities often encapsulated in libraries that provide intuitive primitives"）。库作者则必须为**每一次同步事件**支付 fence/原子的代价——这就是第 4 节要算的账。

---

3. 代码示例与性能分析

> 说明：以下四个示例都是**自包含、可直接编译运行**的程序，用于把讲义中的"结果可能性 / fence 位置 / 一致性流量"变成可测量的事实。硬件相关的观测（例如 x86 上是否出现 `(0,0)`）**依赖具体微架构、核数与线程绑定**；请按注释里的 `taskset` 把线程固定到不同物理核上再测量，否则同一物理核上的超线程会掩盖重排现象。

3.1 示例一：Store Buffering（SB）石蕊测试——用实验测出 x86 放松了哪一条

> 石蕊测试（litmus test）是内存模型的"最小判别程序"：它小到可以逐条推演，又真到能在硬件上跑出结果分布。本示例把**同一个 SB 程序**放在三种顺序强度下各跑 10^6 轮，统计四种结果组合的出现次数。

```
// litmus_sb.cpp —— Store Buffering (SB) 石蕊测试
//   同一个 SB 程序在三种"顺序强度"下的行为对比：
//     RELAXED : 两侧 store/load 都用 memory_order_relaxed
//               -> x86 上编译成普通 mov，写留在写缓冲里（真正的"宽松"）
//     FENCED  : 同样用 relaxed，但在**本线程的 store 与 load 之间**插入
//               atomic_thread_fence(seq_cst)（x86 上是一条 full barrier）
//     SEQ_CST : 两侧都用 seq_cst 原子操作（x86 的 store 编译成 xchg）
//   每个"轮次"统计 (r0, r1) 的四种组合出现次数，并给出每轮耗时。
//
// 编译: g++ -O2 -std=c++17 -pthread litmus_sb.cpp -o litmus_sb
// 运行: taskset -c 0,1 ./litmus_sb 1000000     # 必须绑到两个不同的物理核
//
// 两个实测踩过的坑（都写在代码里，别踩第二次）：
//   1) mode 必须是**编译期常量**（下面的模板参数）。若把 mode 当运行时变量，
//      gcc -O2 会把 relaxed 与 seq_cst 两条路径**合并**、统一生成 xchg，
//      于是 "relaxed" 实验悄悄变成 seq_cst 实验，结论完全反了。
//   2) fence 必须夹在**本线程 store 与 load 的中间**。放在 store 之前只是排空
//      上一轮遗留的写缓冲，对 WX->RY 毫无作用（实测 (0,0) 照样大量出现）。
#include <atomic>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <thread>

static constexpr int NT = 2;                 // 只需要两个线程
enum Mode { RELAXED = 0, FENCED = 1, SEQ_CST = 2 };

std::atomic<int> X{0}, Y{0};                 // 两条被跨线程观察的"消息"
std::atomic<int> arrived{0};                 // 已到达栅栏的线程数
std::atomic<int> generation{0};              // 栅栏世代，单调递增
long count[2][2];                            // count[r0][r1]
int  r[NT];                                  // 本轮各线程读到的对方值

// 简易 sense-reversing 栅栏（它本身就是一段 acquire/release 同步代码）：
// 返回 true 表示"我是最后一个到达者"，由它负责本轮结果的记录与变量归零。
static bool barrier_wait() {
    int g = generation.load(std::memory_order_acquire);       // 先记住当前世代
    if (arrived.fetch_add(1, std::memory_order_acq_rel) == NT - 1) {
        arrived.store(0, std::memory_order_relaxed);          // 重置计数
        generation.fetch_add(1, std::memory_order_release);   // 放行其余线程
        return true;
    }
    while (generation.load(std::memory_order_acquire) == g) { }   // 自旋等世代变化
    return false;
}

template <int MODE>
static void worker(int id, long iters) {
    // 编译期常量：RELAXED / FENCED 用 relaxed，只有 SEQ_CST 用 seq_cst
    constexpr std::memory_order ORD =
        (MODE == SEQ_CST) ? std::memory_order_seq_cst : std::memory_order_relaxed;
    for (long it = 0; it < iters; ++it) {
        barrier_wait();                      // A: 上一轮结果已记录、X/Y 已归零
        if (id == 0) {
            X.store(1, ORD);                                  // (1) 发布
            if (MODE == FENCED)                               // (2) 关键位置!
                std::atomic_thread_fence(std::memory_order_seq_cst);
            r[0] = Y.load(ORD);                               // (3) 观察对方
        } else {
            Y.store(1, ORD);
            if (MODE == FENCED)
                std::atomic_thread_fence(std::memory_order_seq_cst);
            r[1] = X.load(ORD);
        }
        if (barrier_wait()) {                // B: 两个 load 都已完成
            count[r[0]][r[1]]++;             // 最后一个到达者独占记录
            X.store(0, std::memory_order_relaxed);            // 归零，准备下一轮
            Y.store(0, std::memory_order_relaxed);
        }
    }
}
```

```
static void run(const char* name, void (*fn)(int, long), long iters) {
    count[0][0] = count[0][1] = count[1][0] = count[1][1] = 0;
    r[0] = r[1] = 0;
    X.store(0); Y.store(0); arrived.store(0); generation.store(0);
    auto t0 = std::chrono::steady_clock::now();
    std::thread a(fn, 0, iters), b(fn, 1, iters);
    a.join(); b.join();
    auto t1 = std::chrono::steady_clock::now();
    double s = std::chrono::duration<double>(t1 - t0).count();
    std::printf("%-10s %9ld %9ld %9ld %9ld %8.3f %8.1f\n", name,
                count[0][0], count[0][1], count[1][0], count[1][1],
                s, s / double(iters) * 1e9);
}

int main(int argc, char** argv) {
    long iters = (argc > 1) ? std::atol(argv[1]) : 1000000L;
    std::printf("%-10s %9s %9s %9s %9s %8s %8s\n",
                "mode", "(0,0)", "(0,1)", "(1,0)", "(1,1)", "time/s", "ns/round");
    run("relaxed",   &worker<RELAXED>, iters);
    run("fenced",    &worker<FENCED>,  iters);
    run("seq_cst",   &worker<SEQ_CST>, iters);
    return 0;
}
```

【代码做什么？】

1. **建立"轮次"结构**：主线程只负责 `join`；两个 worker 线程每一轮跑完全相同的三步——**栅栏 A → 一次写 + 一次读 → 栅栏 B**。栅栏用 sense-reversing 风格实现：先记住当前世代 `g`，用 `fetch_add(acq_rel)` 计数，**最后一个到达者**重置计数并 `release` 地推进世代，其余线程 `acquire` 地自旋等待世代变化。栅栏 A 同时承担两个职责：保证上一轮的 `X`、`Y` 已归零，并让两个线程**尽可能同时**开始（否则重排窗口会消失）。
2. **SB 的两次访存**：`id==0` 做 `X=1; r0=Y;`，`id==1` 做 `Y=1; r1=X;`。这正是讲义 slide 16 那个"跨地址顺序"问题的最小形式：两次访存**地址不同**（不是 coherence 问题）、**至少一次是写**（是冲突）、中间**没有任何同步**（是数据竞争）。
3. **三种模式必须是编译期常量**：`MODE` 作为**模板参数**，`ORD` 才是真正的 compile-time constant。这样 `RELAXED` / `FENCED` 路径生成的是**普通 `mov`**（写留在写缓冲里），而 `SEQ_CST` 路径生成 `xchg`。**这一点不是严格的：** 如果改用运行时变量传 mode，`gcc -O2` 会把两条路径**合并**并统一生成 `xchg`，于是"relaxed 实验"实际跑的是 seq_cst——本笔记在调试时就撞上了这个坑（见下文"两个坑"）。
4. **fence 的精确位置**：FENCED 模式的 fence 夹在 **(1) 本线程的 store** 与 **(3) 本线程的 load 之间**。这是整个实验的成败关键：它让"我的写已提交到一致性序"成为"我的读可以发出"的前提条件。若把 fence 放成 `fence; store; load`，它就只是排空了上一轮遗留的写缓冲，对 WX→RY 一点用都没有。
5. **结果记录与归零**：由栅栏 B 的**最后一个到达者**统一记录 `count[r0][r1]` 并把 `X`、`Y` 归零，避免两个线程同时写统计结构造成新的竞争；同时它保证归零发生在下一轮的栅栏 A 之前（release/acquire 传递），因此每一轮都从 `X=Y=0` 开始。程序最后按模式打印四种结果的次数、总时间与**每轮纳秒数**。

【并行机制与性能解说】

- **硬件上发生了什么**：两个 worker 绑在**不同物理核**上（`taskset -c 0,1`，本机 core 0 与 core 1 各自独占一个物理核、无 SMT 兄弟），`X`、`Y` 所在的行通过一致性协议在两个核的私有缓存之间转移。
- `RELAXED`：`X=1` 进写缓冲后**立即**发射 `r0 = Y` 的读请求。读请求不必等写提交，于是两个线程的读很可能都在对方的写提交之前被服务 ⇒ **`(0,0)` 这个 SC 明令禁止的结果被硬件真的产生出来**。
- `FENCED` / `SEQ_CST`：读被强制推迟到**自己的写提交之后**，于是 `(0,0)` 不可能出现（因为若两边都读到 0，就会推出"我的写提交在你的读之后"与"你的写提交在我的读之后"同时成立，见第 7 节 Q1 的环论证）。
- **实测记录（表 2）**：AMD EPYC 7V13（Zen 2，双路，两个核同 socket，实测主频约 3.1 GHz），`g++ -O2 -std=c++17 -pthread`，`taskset -c 0,1 ./litmus_sb 1000000`，共跑三次；每次 100 万轮。

mode	(0,0)	(0,1)	(1,0)	(1,1)	ns/轮
relaxed（普通 `mov`）	**143783**（三次：303529 / 130826 / 143783，即 **13%–30%**）	579621	276596	0（另两次为 1）	474
fenced（fence 夹在 store/load 之间）	**0**（三次全为 0）	511477	320121	168402	517
seq_cst（store 用 `xchg`）	**0**（三次全为 0）	520711	407155	72134	476

- **可以从数据里读出的四条结论**：

- `(0,0)` **在 relaxed 模式下确实出现，占比 13%–30%** ⇒ 这不是"理论上的可能"，而是**本机 x86 硬件反复产生的事实**；它直接坐实了"**x86 只是 TSO，允许 WX→RY 被写缓冲重排**"，也解释了 CS149 slide 43 为什么要把 x86 的内存模型单独命名为 "incompletely specified form of TSO"。
- `(0,0)` **在 fenced 与 seq_cst 模式下三次运行恒为 0** ⇒ fence 与 seq_cst 原子操作**确实**禁止了这个结果，与第 7 节 Q1 的 happens-before 环论证一致。这就是"内存模型是契约，不是概率"的含义。
- `(1,1)` **的比例是一个"时序指纹"**：relaxed 下几乎为 0（写缓冲让读抢在对方写提交之前），fenced 下升到 **16.8%**（两边的读都被推到自己的写提交之后），seq_cst 下只有 7.2%（`xchg` 带来的全局序约束使两侧更难"同时"看到对方）。**同一个 SB 程序在三种顺序强度下，结果分布形态明显不同——这正是"顺序强度"的可测量后果。**
- **fence 的边际成本**：relaxed **474 ns/轮** → fenced **517 ns/轮**，约 **+43 ns/轮（≈133 周期 @3.1 GHz）**。这 43 ns 包含两部分：**排空写缓冲**，以及**读不再能与写重叠**——后者才是大头，因为 relaxed 模式的性能恰恰来自"读绕过写"这个优化。第 4.6 节估计的"一次 `mfence` 排空约 20–40 周期"只覆盖了前者，端到端代价要高得多。（同一模式在不同运行之间的耗时可有几十个百分点的波动——每轮都要在两道人造栅栏上同步，测量装置本身对线程相位极其敏感，因此**只有"是否为 0"这类结构性结论可以直接采信，具体比例只能当量级参考**。）
- **绝对量级同样值得注意**：每轮耗时接近 500 ns，而两次访存本身只需几十纳秒——**时间几乎全花在两道人造栅栏上**。做内存模型实验时，"测量装置"的开销往往比被测现象大一个量级，这也是必须用 Work/Span 而不是只报总时间来解读它的原因（见下）。
- **两个实测踩过的坑（本笔记的调试记录，本身就有教学价值）**：
- **编译器会把"宽松"悄悄变成"严格"**：当 `mode` 作为**运行时变量**时，`gcc -O2` 把 `relaxed` 与 `seq_cst` 两条分支**合并**，统一生成 `xchg`（`objdump` 可见），于是两个模式的计数几乎完全一致（(0,0) 都是 0、(1,1) 分别是 29289 与 29276）——**"我的 relaxed 实验显示没有重排"其实是编译器替你把重排禁掉了**。改用模板让 mode 成为编译期常量后，`relaxed` 立刻跑出 45 万次 `(0,0)`。教训：**内存模型实验必须核对生成的汇编，`-O2` 不是旁观者，它是参与者。**
- **fence 放错位置等于没放**：把 fence 写成 `fence; store; load`（在 store **之前**）时，`(0,0)` 依然以 70% 的比例大量出现——因为那条 fence 只排空了**上一轮**遗留的写缓冲，对本轮 store→load 的顺序毫无约束。fence 的语义是"**我之前的访存都必须完成**"，所以它必须出现在**需要被它保护的 store 之后、需要被它约束的 load 之前**。
- **栅栏本身会"掩盖"被测现象**：本实验每轮都同步，两个线程之间存在约半个一致性往返的固有相位差；这个差值可与"写提交延迟"相比，因而**任何一种每轮强制同步的石蕊测试都无法干净地测量重排窗口的真实概率**。所以上表的 (0,0) 比例的**绝对值**不应当被当作"硬件重排概率"，只有"**是否为 0**"这一条是模型层面的确定结论。工业级做法是用专门的模型检查/枚举工具（如 herd7/litmus7 这类工具链）或动态检测器（ThreadSanitizer）来验证，而不是靠"跑一遍看结果"。
- **Work（总工作量）**：每轮固定 2 次访存 + 2 次栅栏 + 1 次记录与归零。$W(n) = n \cdot \Theta(1)$，其中 n 是轮数（10^6）；**Work 与线程数、并行度无关**，它就是被观察的事件总数。
- **Span（关键路径）**：每轮的跨度 = **栅栏 A 的最慢到达者 + 一次写（+ 可能的 fence）+ 一次读 + 栅栏 B 的最慢到达者**。若栅栏的一次跨核往返记为 T_b、读的行转移记为 L、fence 的排空与去重叠代价记为 T_f，则单轮跨度 $\approx 2T_b + L + T_f$，总跨度 $S(n) = \Theta(n(2T_b + L + T_f))$。
- **并行度 = Work / Span**：$W/S \approx \Theta(1)/\Theta(2T_b + L + T_f)$ —— **只由常数项决定，与 n 无关，实际就是 2**（两个线程）。实测 **474–517** ns/轮与 $2T_b + L$ 的量级吻合（本机跨核一致性地往返在百纳秒量级）。这说明石蕊测试是一个**延迟测量装置**，不是可并行计算：**Work/Span 读法在这里的正确用法是"并行度 = 2 是结构上界，加线程只会增加栅栏排队"**。
- **瓶颈：延迟，而非带宽**。全程序只碰两条 cache line，带宽消耗可忽略；时间全部花在"跨核一次往返"与 fence 上。因此它的可扩展性上限是 2（两个线程）。
- **与讲义结论的对应**：这个程序把 slide 9（"从其他线程的视角，一切皆有可能"）、slide 27–29（TSO 可以用写缓冲，写延迟被有效隐藏）、slide 33（MFENCE 的语义与"扭结"）、slide 39（"不要只用普通访存做同步"）四条结论，变成了一张**可复现、可解释、且带踩坑记录**的计数表。

3.2 示例二：用 release/acquire 写一个无锁 SPSC 队列（正确同步的典范）

```cpp
// spsc_ring.cpp —— 单生产者单消费者（SPSC）环形队列，只用 acquire/release
// 编译: g++ -O3 -std=c++17 -pthread spsc_ring.cpp -o spsc_ring
// 运行: taskset -c 0,1 ./spsc_ring 50000000
//
// 设计要点（每一条都对应讲义里的一个概念）：
//   * head_ / tail_ 各自 alignas(64)：避免两个索引落在同一 cache line 上
//     -> 这是"伪共享（false sharing）"的标准防御手段
//   * 生产者只写 tail_，消费者只写 head_：索引变量的写者唯一
//     -> 满足 SWMR（单写者多读者）不变量，把写争用降到最低
//   * 数据写入缓冲区后，用 release store 发布 tail_
//   * 读数据之前，用 acquire load 观察 tail_
//     -> 构成 "synchronizes-with"，让"数据已写好"先于"索引已推进"对其他线程可见
//   * 空闲时才用 relaxed load 看对方索引（它已经是本线程缓存的副本）

#include <atomic>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <thread>

template <typename T, size_t N>            // N 必须是 2 的幂
class SpscRing {
    static_assert((N & (N - 1)) == 0, "N must be a power of two");
    static constexpr size_t MASK = N - 1;

    alignas(64) std::atomic<uint64_t> head_{0};   // 消费者写，生产者读
    alignas(64) std::atomic<uint64_t> tail_{0};   // 生产者写，消费者读
    alignas(64) T buf_[N];                        // 数据区（双方都会碰）

public:
    bool push(const T& v) {
        const uint64_t t = tail_.load(std::memory_order_relaxed);       // 只有我在写 tail_
        if (t - head_.load(std::memory_order_acquire) == N) return false;  // 满
        buf_[t & MASK] = v;                                             // (1) 写数据（普通访存）
        tail_.store(t + 1, std::memory_order_release);                  // (2) 发布
        return true;
    }
    bool pop(T& out) {
        const uint64_t h = head_.load(std::memory_order_relaxed);       // 只有我在写 head_
        if (h == tail_.load(std::memory_order_acquire)) return false;   // (3) 空 -> acquire
        out = buf_[h & MASK];                                           // (4) 读数据
        head_.store(h + 1, std::memory_order_release);                  // (5) 回收槽位
        return true;
    }
};

int main(int argc, char** argv) {
    const long NMSG = (argc > 1) ? std::atol(argv[1]) : 50000000L;
    SpscRing<uint64_t, 1024> q;
    std::atomic<uint64_t> checksum{0};

    auto t0 = std::chrono::steady_clock::now();

    std::thread producer([&] {
        for (long i = 0; i < NMSG; ++i)
            while (!q.push(static_cast<uint64_t>(i))) { }   // 满则自旋
    });
    std::thread consumer([&] {
        uint64_t sum = 0, v;
        for (long i = 0; i < NMSG; ++i)
            while (!q.pop(v)) { }                           // 空则自旋
        checksum.store(sum, std::memory_order_relaxed);
    });
    producer.join(); consumer.join();

    auto t1 = std::chrono::steady_clock::now();
    double s = std::chrono::duration<double>(t1 - t0).count();
    std::printf("%ld 条消息, %.3f s, %.1f M msg/s, 有效载荷 %.2f GB/s\n",
                NMSG, s, NMSG / s / 1e6, NMSG * 8.0 / s / 1e9);
```

```
    return 0;
}
```

【代码做什么？】

1. **`push`（生产者）**：先用 `relaxed` 读自己的 `tail_`（**只有这个线程写它**，所以 relaxed 完全安全，也不会产生额外顺序代价）；再用 **`acquire`** 读 `head_` 判断是否满——这一步的 acquire 保证"消费者已经完成对槽位的读取、并把 `head_` 推进"这件事对我可见。然后把数据写进 `buf_`（**(1) 普通访存**），最后用 **`release`** store 推进 `tail_`（**(2)**）——release 的含义正是：**在 (2) 之前的写（包括 (1)）必须先于 (2) 对其他线程可见**。
2. **`pop`（消费者）**：对称地，先 `relaxed` 读自己的 `head_`，再用 **`acquire`** 读 `tail_`（**(3)**）判断是否为空——这个 acquire 与生产者的 release **synchronizes-with**，从而保证 (1) 写在 (4) 读之前可见。读完数据（**(4)**）后 release 地推进 `head_`（**(5)**），告知生产者槽位已回收。
3. **对齐与写者唯一**：`head_` 与 `tail_` 各自 `alignas(64)`，保证它们**不共享 cache line**；`buf_` 也按 64 对齐以避免与索引变量凑到一行。加上"每个索引只有一个写者"的设计，这个队列在**没有任何锁、没有任何 fence 指令**的前提下就是**数据竞争自由**的：所有共享访问要么是同线程私有（relaxed 索引），要么被 release/acquire 排序。
4. **测量**：跑 N 条消息、统计 msg/s 与**有效载荷带宽**（8 B/消息）。注意这里刻意使用 `while(!push(...))` 的空转形式，让生产者/消费者**在队列满/空时自旋**，从而测的是**队列本身**的吞吐极限。

【并行机制与性能解说】

- **硬件上发生了什么**：生产者与消费者跑在两个不同核上，`tail_` 这一行在生产者核上处于 **M 态**，消费者读它时会触发一致性事务（行被共享/转移）；`head_` 同理反向流动。`buf_` 里的同一行会用**生产者写、消费者读（true sharing，真共享**，是必要的通信，不是伪共享），一个 64 B 行能装下 8 个 `uint64_t`，所以**数据行的转移成本被 8 条消息摊薄**，而**索引行的转移成本是每条消息一次**——这正是下一条要批处理优化的原因。
- **Work（总工作量）**：$W(N) = N$ 次 `push` + N 次 `pop` = $2N$ 次常数代价操作，即 $W(N) = \Theta(N)$。其中每次操作包含：2 次原子索引访问 + 1 次缓冲区访存。
- **Span（关键路径）**：队列的**数据流依赖**是元素级的（第 i 个元素必须先被 push 才能被 pop），但这条链是**跨线程流水**：生产者的 N 次 `push` 之间**没有数据依赖**（各自的 `tail_` 值由本线程顺序推进，store 走写缓冲即可），消费者的 N 次 `pop` 之间也没有。真正的跨线程腿只有一条：**"第 i 个元素的 (2) release store 必须被 (3) acquire load 观察到，第 i 个槽位才能被安全复用"**。在稳定的流水状态里（消费者落后生产者若干个槽位），这条腿被**流水线重叠**掉，单元素的跨度退化到"一次本地索引更新 + 一次写缓冲入队"的几拍量级，即 $S(N) = \Theta(N)$（常数很小）。
- **并行度 = Work / Span**：$W/S = 2N/\Theta(N) = \Theta(1)$，**具体约为 2**（生产与消费两条流水线），**上界永远是 2**——环形队列的结构决定了它不可能用 8 个线程加速，它是**通信结构**而不是**可并行计算**。所以 Work/Span 在这里的正确读法是：**并行度 = 2 意味着"任意多核下最多 2× 吞吐"，衡量它的指标必须是"每秒消息数 / 有效载荷带宽"，不是"加速比"**。
- **瓶颈与量化**（按本笔记假设计算，标注为推导值）：
- 设需要 10^8 msg/s、每消息 8 B：**有效载荷 = 0.8 GB/s**，但一致性以 **64 B 行**为单位搬运，**索引行的流量被放大 8 倍左右**，实际一致性流量 ≈ **6.4 GB/s**。在 20 GB/s 的互连上这已占 **32%** 的带宽——**纯通信结构的算术强度（arithmetic intensity）为 0 FLOP/Byte**，它在 Roofline 图上永远贴着最左边（见第 4.5 节）。
- 若每条消息都要求一次索引行转移、每次转移按 ~30 ns 计，则 10^8 条消息需要 $10^8 \times 30\ \text{ns} = 3\ \text{s}$ 的**串行化点时间**——这才是真正的天花板。
- **优化方向：批处理（batching）**。让 `pop` 一次 acquire 后连续消费多个元素（例如把 `tail_` 读一次、循环取 8 个），或让 `push` 一次写 8 个再发布一次 `tail_`：索引行转移次数下降 8 倍，`fence`/原子操作次数下降 8 倍，**吞吐提升接近 8 倍**，代价是**平均延迟上升**（消息在队列里多等一会儿）——这是**吞吐 vs 延迟**的经典取舍，也是后面第 4.6 节"fence 吞吐上限"的直接推论。

3.3 示例三：伪共享实测——把并行度从 8 压回 1

```cpp
// false_sharing.cpp —— 复刻 CS149 讲义 slide 17 的伪共享实验
//   两种布局：8 个计数器挤在同一行 vs 每个计数器独占一行 (alignas(64))
//
// 编译：g++ -O3 -std=c++17 -pthread false_sharing.cpp -o false_sharing
// 运行：taskset -c 0-3 ./false_sharing 8 25000000
//        (8 线程，每线程 2500 万次自增 = 总计 2 亿次自增)
//
// 讲义 (CS149 slide 17) 在 4 核系统上用 8 线程测得：
//      未 padding: 14.2 s      已 padding: 4.7 s       (约 3.0 倍差距)
// 差异**完全来自一致性流量** (artifactual communication)，与算法无关。

#include <atomic>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <thread>
#include <vector>

static constexpr int CACHE_LINE = 64;

// 布局 A：8 个 atomic<long> 连续放置 -> 8 * 8 B = 64 B，正好压在一条 cache line 上
// 布局 B：每个计数器独占一条 cache line (C++11 的 alignas 等价于讲义里的
//         struct { int counter; char padding[CACHE_LINE_SIZE - sizeof(int)]; })
struct alignas(CACHE_LINE) PaddedCounter {
    std::atomic<long> v{0};
};

static inline std::atomic<long>& slot_of(std::atomic<long>& a) { return a; }
static inline std::atomic<long>& slot_of(PaddedCounter& p)     { return p.v; }

template <typename Counter>
static double run(const char* tag, int nthreads, long iters) {
    std::vector<Counter> c(nthreads);
    std::vector<std::thread> th;
    th.reserve(nthreads);

    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < nthreads; ++i)
        th.emplace_back([&c, i, iters] {
            std::atomic<long>& slot = slot_of(c[i]);          // 每个线程只碰自己那一格
            for (long k = 0; k < iters; ++k)
                slot.fetch_add(1, std::memory_order_relaxed);  // 原子读-改-写
        });
    for (auto& t : th) t.join();
    auto t1 = std::chrono::steady_clock::now();

    double s = std::chrono::duration<double>(t1 - t0).count();
    double per_op_ns = s / (double(nthreads) * double(iters)) * 1e9;
    std::printf("%-28s %8.3f s   %7.1f ns/自增\n", tag, s, per_op_ns);
    return s;
}

int main(int argc, char** argv) {
    int  nt    = (argc > 1) ? std::atoi(argv[1]) : 8;
    long iters = (argc > 2) ? std::atol(argv[2]) : 25000000L;

    double a = run<std::atomic<long>>("未 padding（同行，伪共享）", nt, iters);
    double b = run<PaddedCounter>   ("已 padding（独占一行）    ", nt, iters);
    std::printf("speedup(padded / unpadded) = %.2fx\n", a / b);
    return 0;
}
```

【代码做什么？】

1. **两种内存布局**：`std::atomic<long>` 数组里 8 个元素恰好占 **64 B = 一条 cache line**；`PaddedCounter` 用 `alignas(64)` 强制每个计数器**独占一行**（64 B 里只有 8 B 有效，48 B 是"浪费"，但换来零一致性争用）。
2. **每个线程只访问自己的那一格**：`slot = slot_of(c[i])`。**在语言层面这是完全数据竞争自由的**（不同对象、不同线程），**在硬件层面却不是**：布局 A 里 8 个线程写的是**同一条 cache line 的不同字节**，MSI 协议只认 cache line——谁要写，就必须先把整行"抢"过来（独占），别人的副本失效。这就是 **false sharing（伪共享）**。

3. `fetch_add(relaxed)`：relaxed 只说明**顺序**无关紧要，**不说明原子性可以省**——读–改–写仍然是原子的，仍然要求独占所有权。因此它**必然**触发一致性事务，正好用来把伪共享的代价隔离出来。
4. **测量**：分别报告总时间与**每次自增的平均纳秒数**；后者比总时间更能揭示"每次操作到底付了多少一致性代价"。

【并行机制与性能解说】

- **硬件上发生了什么**：布局 A 中，8 个线程轮流向同一条行做原子自增，**每次自增都要把该行从"别人手里"抢回来**（BusRdX / 目录失效），行在 8 个核之间**持续 ping-pong**。行本身始终是热的（从不下沉到 DRAM），但这些**行转移事务**全部要经过互连这个**串行化点**——于是"8 个逻辑上完全独立的操作"被硬件变成了**一条串行链**。布局 B 中每个线程的行长期停留在自己核的 **M 态**，事务数从 $\Theta(W)$ 掉到 $O(1)$（每个线程一次初始获取），自增退化成**本地 L1 的原子操作**。
- **Work（总工作量）**：$W = P \times I$ 次原子自增（$P = 8$ 线程、$I = 2.5 \times 10^7$）$= 2 \times 10^8$ 次。**Work 在两种布局下完全相同**——这正是伪共享"完全是人为（artifactual）通信"的含义。
- **Span（关键路径）**：
- **布局 A（同行）**：因为**每一次自增都必须垄断整条 cache line**，所有 W 次自增在物理上被串成一条链：$S_A = W \cdot T_{\text{coh}}$，其中 T_{coh} 是"一次行转移 + 串行化点排队"的等效时间。
- **布局 B（独占行）**：每个线程的 I 次自增在自己的行上串行，线程之间**互不相干**：$S_B = I \cdot T_{\text{local}}$，$T_{\text{local}}$ 是本地原子操作的等效时间。
- **并行度 = Work / Span**：
- 布局 A：$W/S_A = W/(W \cdot T_{\text{coh}}) = 1/T_{\text{coh}}$ ——**与线程数无关，等价于"并行度 1"**。伪共享把 $P = 8$ 的并行度**压回了 1**，这是本讲最锋利的一句结论。
- 布局 B：$W/S_B = (P \cdot I)/(I \cdot T_{\text{local}}) = P/T_{\text{local}}$，即**并行度退化为 P（受物理核数 4 限制）**。
- **与实测对账（本笔记推导，讲义只给了两个时间）**：讲义数据是 8 线程/4 核下 **14.2 s vs 4.7 s（3.0×）**。把"总计 2×10^8 次自增"作为假设（讲义未给出 `MANY_ITERATIONS`），则布局 A 的每次自增成本 ≈ $14.2\,\text{s}/2 \times 10^8 \approx 71$ ns（3 GHz 下约 213 周期），远高于单次 L3 命中的 ~40 周期（CS149 slide 14）或"行在别的核且已修改"的 ~75 周期——**差额就是互连串行化点上的排队与失效风暴**；布局 B 的每次成本 ≈ 23.5 ns（约 70 周期），这是 8 线程挤在 4 个核上做本地原子操作的合理水平。两者比值 3.0×，与讲义实测一致。
- **本笔记在本机的复现**：AMD EPYC 7V13（Zen 2，`taskset -c 0-3`，每线程 2.5×10^7 次自增、共 2×10^8 次）。**8 线程（4 核）**：未 padding **2.388 s**（11.9 ns/次），已 padding **0.133 s**（0.7 ns/次），**speedup = 17.9×**；**4 线程（4 核）**：未 padding 0.138 s，已 padding 0.016 s，**8.5×**。绝对时间比讲义数据（14.2 s / 4.7 s）小一个量级——那是历史机器与不同的迭代次数，**但"并行度从 8 掉回 1"的机制与方向完全一致，且效应在现代多核上更强（17.9× ≫ 3.0×）**。这再次说明：该现象的可比量是**比值与机制**，不是绝对秒数。
- **一致性流量账（把 256 MB / 20 GB/s 当作单位换尺）**：布局 A 中约 2×10^8 次行转移 × 64 B = **12.8 GB** 的一致性流量。参考量纲：**一个 256 MB 的数组在 20 GB/s 带宽下遍历一次至少 256 MB / 20 GB/s = 12.8 ms**；12.8 GB 是它的 **50 倍**，即**仅"搬运"就要 640 ms**，再加上串行化点每事务数十纳秒的排队，总时间进入**秒级**——与 14.2 s 同量级。这个 3.0× 的差距**没有一行代码是"算"出来的**，全部是通信。
- **瓶颈：一致性事务的串行化吞吐**（不是 DRAM 带宽，也不是算力）。同一现象的"单次操作"读数：本机实测未 padding 时**每次自增 11.9 ns**（≈37 周期 @3.1 GHz），它把一次完整的行转移与串行化排队都摊在了单次自增上。修复手段：`alignas(64)` / 手动 padding（讲义 slide 16 的 `char padding[CACHE_LINE_SIZE - sizeof(int)]`）、每线程私有累加 + 末尾归约（把 W 次远程原子操作变成 P 次）。

3.4 示例四：Peterson 锁与 fence 的位置（讲义 slide 39 的练习题）

```
// peterson_fence.cpp —— Peterson 互斥算法在宽松内存模型上的正确 fence 放置
// 编译: g++ -O2 -std=c++17 -pthread peterson_fence.cpp -o peterson_fence
// 运行: taskset -c 0,1 ./peterson_fence 2000000
//
// 程序：两个线程各自进入临界区 iters 次，每次对**非原子**的 shared 计数器加一。
//       若互斥正确，最终 shared == 2*iters；否则出现"丢失更新"（lost update）。
// 两种实现：BROKEN（只用 relaxed，无 fence） 与 FIXED（fence 放在正确位置）。
//
// 讲义 slide 39 的练习题：
//     boolean want[2] = {false,false};  int turn = 0;
//     want[i] = true;  turn = j;
//     while (want[j] && turn == j) continue;
//     ... critical section ...
//     want[i] = false;
//   问：应该在哪里加 fence、加哪一种？
// 本文件的 FIXED 版本给出答案，并在第 7 节 Q1 详细论证。

#include <atomic>
#include <cstdio>
#include <cstdlib>
#include <thread>

static std::atomic<int> want[2];
static std::atomic<int> turn{0};
static long shared = 0;          // 故意用普通 long：由锁保护，靠 happens-before 保证安全
static bool use_fence = true;

// ---------- 正确版本：fence 恰好放在需要的地方 ----------
static void lock_fixed(int i) {
    const int j = 1 - i;
    want[i].store(1, std::memory_order_relaxed);                    // (a) 声明意图
    std::atomic_thread_fence(std::memory_order_seq_cst);            // F1: (a) 必须先于 (b) 提交
    turn.store(j, std::memory_order_relaxed);                       // (b) 让出优先权
    std::atomic_thread_fence(std::memory_order_seq_cst);            // F2: 上面的写提交后才允许下面的读
    while (want[j].load(std::memory_order_relaxed) &&
           turn.load(std::memory_order_relaxed) == j) { }           // (c) 等待
}
static void unlock_fixed(int i) {
    std::atomic_thread_fence(std::memory_order_seq_cst);            // F3: 临界区内所有写先提交
    want[i].store(0, std::memory_order_relaxed);                    // (d) 释放
}

// ---------- 错误版本：完全没有 fence（在 x86 上"多数时候也能跑"） ----------
static void lock_broken(int i) {
    const int j = 1 - i;
    want[i].store(1, std::memory_order_relaxed);
    turn.store(j, std::memory_order_relaxed);                       // 可能被重排到 (a) 前面
    while (want[j].load(std::memory_order_relaxed) &&
           turn.load(std::memory_order_relaxed) == j) { }
}
static void unlock_broken(int i) {
    want[i].store(0, std::memory_order_relaxed);
}

static void worker(int id, long iters, long* local_errors) {
    long prev = 0;
    for (long k = 0; k < iters; ++k) {
        if (use_fence) lock_fixed(id); else lock_broken(id);
        long now = shared + 1;          // 读-改-写：必须有互斥保护，否则丢失更新
        shared = now;
        if (use_fence) unlock_fixed(id); else unlock_broken(id);
        prev = now;
    }
    *local_errors = prev;               // 仅用于阻止编译器把循环优化掉
}

int main(int argc, char** argv) {
    long iters = (argc > 1) ? std::atol(argv[1]) : 1000000L;
    for (int fixed = 1; fixed >= 0; --fixed) {
        use_fence = (fixed == 1);
        want[0] = want[1] = 0; turn = 0; shared = 0;
```

```
        long e0 = 0, e1 = 0;
        std::thread t0(worker, 0, iters, &e0), t1(worker, 1, iters, &e1);
        t0.join(); t1.join();
        std::printf("%-10s shared = %ld (期望 %ld)  %s\n",
                    fixed ? "FIXED" : "BROKEN", shared, 2 * iters,
                    shared == 2 * iters ? "OK" : "!! 丢失更新 !!");
    }
    return 0;
}
```

【代码做什么？】

1. **Peterson 算法的三段式**：`want[i]=1`（声明意图）→ `turn=j`（把优先权让给对方）→ 自旋等待 `want[j] && turn==j` 不成立。这是讲义 slide 39 给出的骨架（"stripped-down version of a 2-process mutex"）。
2. **FIXED 版本的三个 fence**：
3. **F1** 位于 `want[i]=1` 与 `turn=j` 之间——保证"我声明了意图"**先于**"我让出优先权"对其他线程提交。否则两个线程可能都看到对方还没声明意图，**同时**进入临界区。
4. **F2** 位于 `turn=j` 之后、自旋读之前——保证上面对 `want` / `turn` 的写**提交之后**才开始读。这正是 acquire 语义。
5. **F3** 位于退出临界区、清 `want[i]` 之前——保证临界区内的数据写**先提交**，再宣告"我走了"（release 语义）。没有它，下一个进入者可能看到 `want[i]==0` 但看不到临界区里的修改。
6. **BROKEN 版本**：全部使用 `relaxed`（x86 上就是普通 MOV，编译器/硬件都可自由重排两条写与两次读），于是 F1/F2/F3 的保证全部丢失。
7. **正确性判据**：`shared` 是非原子变量，递增是"读-改-写"三步。**互斥被破坏时，两次递增会丢一次**，最终 `shared < 2*iters`。这个判据无需读内存模型就能理解，但**只在统计意义下有效**：互斥被破坏时未必每轮都丢更新。本机实测（`taskset -c 0,1`，op 固定在两个物理核上）在 **50 万次**迭代时 BROKEN 版本丢了 4 次更新、**20 万次**时丢了 2 次、而 **10 万次**时一次都没丢——**"跑一次没错"完全不能说明程序是对的**。注意 BROKEN 版本在 x86 上**经常"看起来正常"**（因为 x86 只放松 WX→RY，而这里的危险重排主要是 WX→WY 与 RX→RY），这正说明**"在我的机器上能跑"**不是正确性——真正的探测器是 3.1 节的石蕊测试。
8. **可移植性说明**：`want` / `turn` 用 `std::atomic` + `relaxed` 配合 `atomic_thread_fence(seq_cst)`，属于 C++11 允许的 **fence-fence 同步**写法；等价的更"现代"写法是把它们换成 acquire/release 语义的原子操作。**在 x86 上**，F1、F2 的最小硬件实现是"在 (a) 与 (b) 之间一条 `mfence`"（x86 本身就禁止 load→load 重排，所以 (c) 里的两次读不需要额外硬件屏障，但**编译器**仍需被约束——这就是为什么用 `std::atomic` 而不是 `volatile`）。

【并行机制与性能解说】

- **硬件上发生了什么**：两个线程在不同核上反复争抢 `want` / `turn` 所在的行，与锁本身的存储布局强相关——讲义 slide 31 提醒"shared data 事实上对其他线程始终可见"，因此 `want[0]` 与 `want[1]` 若落在同一行，会产生**伪共享式的额外流量**（把 `want[2]` 各自 padding 到独立行是常见优化）。临界区本身把两个线程**串行化**：这是**设计意图**（互斥），不是缺陷。
- **Work（总工作量）**：$W = 2 \cdot I$ 次"进临界区 + 一次读-改-写 + 出临界区"，每次是常数代价操作（含 1 次 acquire 序列与 1 次 release 序列），故 $W = \Theta(I)$。
- **Span（关键路径）**：临界区是**硬串行**的——所有 $2I$ 个临界区段**必须一个接一个执行**。每个临界区段之间还夹着一对"释放 → 获取"的通信（至少要等 `want[i]=0` 的写在对方核上可见，或等一次自旋迭代），耗时约一次跨核往返 $L \approx 30$–100 ns。所以 $S = 2I \cdot (T_{cs} + L)$。
- **并行度 = Work / Span**：$W/S = (2I)/(2I(T_{cs}+L)) \approx 1/(T_{cs}+L)$——**互斥本身的并行度恒为 1**。这是 Amdahl 定律最直接的实例：设临界区占单线程总工作量的比例 f（考虑自旋等待则更高），则加速比上界 $\le 1/f$（讲义视角下，"**不考虑计算的锁，永远是串行瓶颈**"）。把 $f = 5\%$ 代进去：$1/0.05 = 20\times$，**再多的核也到不了 20 倍以上**。
- **fence 的成本**：这里每个临界区进出各一次 fence 序列。按一次 `mfence` 的**排空**代价 ≈ **20–40 周期 ≈ 7–13 ns**（3 GHz）保守估算（示例一实测的端到端边际成本约 43 ns/轮，量级更大），2×10^6 次进出（$I = 10^6$）至少要 **28–52 ms** 的纯 fence 开销；若临界区本身只有几十纳秒，则**同步开销与临界区工作量同量级**——这就是为什么 slide 40 的结论是"**把复杂度封装进库**"：库作者才需要在这里斤斤计较。
- **瓶颈**：**串行化 + 跨核往返延迟**。降低办法只有三类：**缩小临界区**（减少 T_{cs}）、**减少进出次数**（批处理 / 合并、把多次更新合成一次）、**换用乐观并发**（无锁数据结构 / CAS 重试），后者正是下一讲的主题。
- **表 2：四个示例的 Work / Span / 并行度 / 瓶颈汇总**

示例	Work W	Span S（关键路径）	并行度 W/S	可扩展性上限	主要瓶颈
SB 石蕊测试	$\Theta(n)$（n 轮，每轮常数访存 + 2 栅栏）	$\Theta(n(2T_b + L))$	$O(1)$，实际 ≈ 2	2 线程（测量装置）	跨核一致性往返延迟
SPSC 环形队列	$2N$（N push + N pop）	$\Theta(N)$（常数极小，靠流水重叠）	$O(1)$，实际 ≈ 2	2 线程（通信结构）	索引行转移（可用批处理摊薄）

示例	Work	Span （关键路径）	并行度	可扩展性上限	主要瓶颈
伪共享计数器（无 padding）	$P \cdot I$	$P \cdot I \cdot T_{\text{coh}}$	≈ 1	1（并行度被压平）	一致性事务串行化吞吐
伪共享计数器（有 padding）	$P \cdot I$	$I \cdot T_{\text{local}}$	$\approx P$（受物理核数限制）	$\min(P, \text{cores})$	每核本地原子操作吞吐
Peterson 锁临界区	$\Theta(I)$	$\approx 2I(T_{\text{cs}} + L)$	≈ 1	1（互斥语义决定）	串行化 + fence 开销

---

4. 性能模型与复杂度分析

4.1 SC 的代价：从"一访问一停顿"推出 17%–42% 的利用率

模型假设（本笔记的推导参数，用于解释讲义 slide 26 引用的 Gupta et al. 数据量级）：

- 处理器理想发射宽度 = **2 条指令/周期**（即理想 $CPI_{\text{ideal}} = 0.5$ 周期/指令）；
- 访存指令占全部指令的比例 $f_m = 20\%$（每 5 条指令一次访存）；
- 每次访存的**完整延迟**为 L 周期（SC 要求访存串行完成）。

SC 下每条指令的平均周期数：

$$CPI_{\text{SC}} = CPI_{\text{ideal}} + f_m \cdot L = 0.5 + 0.2L$$

处理器利用率（相对理想发射）：

$$U_{\text{SC}}(L) = \frac{CPI_{\text{ideal}}}{CPI_{\text{SC}}} = \frac{0.5}{0.5 + 0.2L}$$

L（周期）	对应存储层次（CS149 slide 14）	CPI_{SC}	利用率 U_{SC}
4	L1 命中	1.30	**38.5%**
6	L1 未命中、L2 命中附近	1.70	29.4%
10	L2 命中	2.50	**20.0%**
30	本地 DRAM（≈120 周期）以下的混合平均	6.50	7.7%
120	本地 DRAM 每次都失手（最坏）	24.50	2.0%

结论：当实际访存延迟落在 **4–10 周期**（L1/L2 命中为主、缓存有效的理想情况）时，模型给出 **20%–38.5%** 的利用率，与讲义引用的 **17%–42%** 落在同一区间。**这就是"严格的顺序一致性即使有缓存也不够快"的定量表述**：L 每增加 1 个周期，利用率就掉 $0.2/(0.5+0.2L)^2 \times 0.5$ 那么多——延迟越大的机器，SC 越不可接受。

4.2 TSO 的收益：把写延迟从关键路径上摘掉

改动一处假设：写缓冲生效，因此**只有 load 会造成停顿**。设访存指令中 **40% 是写**（即写指令占全部指令 $f_w = 8\%$、读指令占 $f_r = 12\%$）：

$$CPI_{\text{TSO}} = 0.5 + f_r \cdot L = 0.5 + 0.12L, \qquad U_{\text{TSO}} = \frac{0.5}{0.5 + 0.12L}$$

L	U_{SC}	U_{TSO}	TSO 相对 SC 的加速
4	38.5%	0.5 / 0.98 = **51.0%**	**1.33×**
10	20.0%	0.5 / 1.70 = **29.4%**	**1.47×**
30	7.7%	0.5 / 4.10 = **12.2%**	**1.58×**

再算一笔"写缓冲必须能排空"的账：写指令率 $= \text{IPC} \times f_w = 2 \times 8\% = 0.16$ 写/周期；每次写 8 B ⇒ 需要的持续写带宽 $= 0.16 \times 8\,\text{B} \times 3.0\,\text{GHz} = 3.84\,\text{GB/s}$。如果写缓冲的冲刷速率低于这个值，缓冲会填满，处理器又被逼回到"等写完成"——**TSO 的收益是有条件的**。CS149 slide 42 的"W–R"曲线给出的正是这个结论：放松 WX→RY 后**写延迟几乎被完全隐藏**，这是全部现代处理器都使用写缓冲（Intel x86 / ARM / RISC-V）的根本原因。

4.3 AMAT 与"多核比单核慢在哪"

AMAT（Average Memory Access Time） $= \sum_i$(第 i 级的访问频率 × 该级延迟)（CS149 slide 14 的公式）。

CS149 给出的 Core i7 Xeon 5500 近似延迟：L1 命中 ~4 周期；L2 命中 ~10 周期；L3 命中（行未被共享）~40 周期；L3 命中（行在**另一个核**、共享态）~65 周期；L3 命中（行在另一个核、**已修改**）~75 周期；本地 DRAM ~30 ns（~120 周期）；远端 DRAM ~100 ns（~400 周期）。

算例（访问频率分布两种情形）：

情形	L1 命中	L2 命中	L3 命中	DRAM	AMAT
A：无共享的"干净"多核执行	90%	7%	2%（未共享 40 周期）	1% 本地 120 周期	$0.90\times4+0.07\times10+0.02\times40+0.01\times120=3.6+0.7+0.8+1.2=$ **6.3 周期**
B：同样分布，但 2% 的行在**别的核且已修改**（75 周期），1% 落到**远端 DRAM**（400 周期）	90%	7%	2%（75 周期）	1% 远端 400 周期	$3.6+0.7+1.5+4.0=$ **9.8 周期**

结论（CS149 slide 14 的原话精神："只有不到百分之几的访存变成这样，影响就已经很显著")：AMAT 从 6.3 → 9.8 周期，**恶化 55.6%**，而变化的只是**2% + 1% = 3%** 的访问。$AMAT_{\text{multiprocessor}} > AMAT_{\text{uniprocessor}}$，差值全部来自**共享与远端性**。这也解释了为什么"某条数据行此刻在谁手里"（M 态在别的核 ⇒ 75 周期）会直接决定本讲的性能故事：**一致性模型的强弱，决定了行必须多频繁地回到一致性序，从而决定了这些昂贵命中出现的频率**。

4.4 伪共享的通信量算例（含强制的量纲换算）

- **基准量纲**：一个 256 MB 的数组，在 20 GB/s 的带宽下**遍历一次至少需要** $256\ \text{MB}/20\ \text{GB/s}=0.256/20\ \text{s}=12.8\ \text{ms}$。这是"内存/互连带宽能做什么"的尺子。
- **伪共享场景**：$P=8$ 个计数器挤在一行，总自增 $W=2\times10^8$ 次。
- 每次自增要求**独占**该行 ⇒ 每次一次行转移（64 B）⇒ 一致性流量 $=2\times10^8\times64\ \text{B}=12.8\ \text{GB}$。
- 12.8 GB 是 256 MB 的 **50 倍** ⇒ 纯搬运时间 $=50\times12.8\ \text{ms}=640\ \text{ms}$。
- 还要叠加**串行化点**的代价：互连一次只能处理一个一致性事务，若等效每事务 30 ns，则 $2\times10^8\times30\ \text{ns}=6\ \text{s}$。两者叠加与讲义实测的 14.2 s 同量级（讲义未给出迭代次数，此处的 2×10^8 是量级假设）。
- **加 padding 后**：每个线程的行长期处于自己核的 M 态，一致性流量从 $\Theta(W)$ 降到 $O(P)$，实测降到 4.7 s（3.0× 提升）。
- **对本讲的寓意**：伪共享是"**一致性协议制造的、与算法无关的通信**"。内存一致性模型讨论的是"顺序"，而伪共享讨论的是"顺序**被强制**的频率"——两者是同一枚硬币的两面：**越弱的模型允许越多的本地缓冲与乱序，就越少把行拉回一致性序；但只要有写共享（真共享或伪共享），行就必须回来。**

4.5 Roofline 与算术强度：一致性相关的代码为什么永远贴着图的最左边

机器参数：16 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA）= 768 GFLOPS 峰值；内存/互连带宽 20 GB/s。

机器平衡点（machine balance）：

$$AI^{*}=\frac{\text{峰值算力}}{\text{带宽}}=\frac{768\ \text{GFLOPS}}{20\ \text{GB/s}}=38.4\ \text{FLOP/Byte}$$

本讲各类代码的算术强度：

代码	每字节访存对应的浮点运算	位置	结论
SB 石蕊测试	0 FLOP/B（纯访存顺序观察）	远在 AI^{*} 左侧	延迟受限
SPSC 队列（8 B 消息）	0 FLOP/B（纯搬运）	远在左侧	通信/带宽受限
伪共享计数器（每次 1 次自增 + 64 B 行转移）	$1/64\approx0.016$ FLOP/B	远在左侧	一致性事务吞吐受限
理想向量化 DAXPY（2 FLOP / 16 B）	0.125 FLOP/B	仍在左侧	需要分块提高强度才能接近 38.4

定量结论：– 队列在 10^8 msg/s 时**有效载荷** = $10^8\times8\ \text{B}=0.8\ \text{GB/s}$，但一致性以 64 B 行为单位，**线路流量约 6.4 GB/s（放大 8 倍）**，占 20 GB/s 的 **32%**；要把它降到 5% 以下，必须让每条消息的分摊行流量 ≤ 3.2 B，即**每行至少服务 20 条消息**（批处理）。– 因为 $AI\ll AI^{*}$，本讲所有优化都**不能靠增加算力**：唯一的杠杆是**减少被搬运的字节数**（padding、批处理、私有累加 + 末尾归约）和**减少被拉回一致性序的次数**（更弱的顺序约束、更长的批）。

4.6 fence / 原子的吞吐上限，与 Amdahl 的汇合

fence 吞吐：一次 `mfence` 需要排空写缓冲，估 **20–40 周期**（3 GHz 下 **6.7–13.3 ns**）；示例一在真实 x86 上实测的端到端边际成本为 **+43 ns/轮（≈133 周期 @3.1 GHz）**——比纯排空估计大 3 倍以上，因为 fence 同时取消了“读绕过写”的重叠。下面用保守的 20–40 周期做上限估计，实际只会更紧。若程序中"每个元素一次 fence"，则单核上限

$$\text{同步事件吞吐}_{\max} = \frac{1}{\mathbf{6.7\text{–}13.3\ ns}} \approx \mathbf{0.75\text{–}1.5 \times 10^8}\ \text{次/秒/核}$$

也就是说：**在 3 GHz 的核上，同步事件的"指令级"上限大约是每秒一亿次量级**。若算法天然需要 10^9 次同步事件/秒，唯一的出路是**批处理**（把 k 次更新合并到一次 fence 之后，上限提高 k 倍）——这与 4.5 节"队列需要每行服务 20 条消息"的结论**从两个不同方向指向同一个设计**。

Amdahl 与锁：设程序中必须互斥执行（或必须经过一次 fence 的顺序）的部分占单线程总时间的比例 f，其余完全可并行，则

$$\text{Speedup}(P) \le \frac{1}{f + \frac{1-f}{P}} \xrightarrow{P\to\infty} \frac{1}{f}$$

- $f = 5\%$ ⇒ 上限 20×；$f = 10\%$ ⇒ 上限 10×。
- 在 16 核机器上，$f = 5\%$ 时实际加速比 $= 1/(0.05 + 0.95/16) = 1/0.1094 = \mathbf{9.1}\times$（而非 16×）——**同步事件把"16 核"打折成"9 核等效"**。
- 反过来看**乐观的一面**：把"每次操作一次 fence"改成"每 20 次操作一次 fence"，f 从（例如）10% 降到 0.5%，上限从 10× 抬到 **200×**。这正是 "**为常见情况优化**"（CS149 slide 55："most memory accesses are not conflicting, so don't design a system that pays the cost as if they are"）的定量含义。

---

5. 关键要点

1. **不要用普通访存做同步。** 讲义 slide 39 的第一条 take-away 原话就是："**DON'T use only normal memory operations for synchronization；DO use either explicit synchronization operations (e.g., `xchg`) or fences**"。石蕊测试（示例一）与 Peterson 锁（示例四）都表明：仅靠"写一个标志、读一个标志"的实现，在宽松模型上会得到正确性完全无法保证的程序——而且**它常常"看起来能跑"**。
2. **Cache coherence 与 memory consistency 是两个正交的问题，别混为一谈。** coherence 只约束**同一地址**（SWMR + 写串行化 + 读回串行序中的最后一次写），它的存在理由是"多份副本"；consistency 约束的是**不同地址之间的可见顺序，与有没有缓存无关**。因此 CS149 slide 3 才会追问"这是互斥问题吗？加锁能修好吗？——**不能**"。加锁修的是竞争，不是副本与重排。
3. **顺序一致性是程序员的心智模型，而硬件为性能必须比它弱。** "所有访存存在一个与各线程 program order 相容的全局串行序"这句话给了我们推理的抓手，但它的直接实现（每个处理器同时只有一个 outstanding 访存）把利用率压到 **17%–42%**。于是实际机器分成 TSO（x86，只放松 WX→RY）、PSO、WO/RC 等档次，并统一用 **fence（MFENCE/LFENCE/SFENCE/ `xchg` ）与 acquire/release** 把需要的顺序"钉"回来。
4. **宽松模型下"正确"的定义是 DRF：先把程序变成无数据竞争，再谈性能。** 同步把程序切成若干段，**段内可任意重排**（那是"私人活动"），**同步点处形成跨线程偏序**（"充分顺序"）。C11 / C++11 / Java 5 给出的契约是 **SC for DRF**：无数据竞争的程序在弱硬件上**表现得像 SC**；有数据竞争的程序**没有任何保证**（未定义行为）。因此工程上的正确做法是"**用库提供的同步原语**"（lock/unlock、barrier、`std::atomic`），让库作者去处理内存模型细节。
5. **fence 不推值，只在跨线程偏序上打"扭结"；它的成本必须被摊薄。** MFENCE **不会**让别的线程立刻看到最新值（slide 35 的常见误解），它只是让**执行它的那个线程**停顿：先前访存必须完成、后续访存不得提前。气球类比里它是"**没有任何粒子能穿过的一个结**"。一次 fence 的排空代价约 20–40 周期（示例一实测端到端约 43 ns/轮），单核同步事件上限约 10^8/s 量级 ⇒ 一切"每元素一次 fence/原子操作"的设计都应改成**批处理**，把成本摊到多个操作上。

---

6. 常见陷阱与注意事项

- **把 coherence 与 consistency 混为一谈，并以为加锁能修一致性问题。** 缓存一致性问题来自"数据在多个缓存里有多份副本"（硬件实现导致的），锁管的是**互斥**——即使你的锁完全正确，不同地址之间的可见顺序依然不受它约束。反过来，也不要以为"多核程序错了就是没加锁"：SB 石蕊测试里根本没有共享数据的读写冲突逻辑，它错在**顺序**。
- **以为 fence 是"让所有核刷新到最新值"的魔法。** MFENCE **不把值推给其他线程**，它只**让本线程停顿**。写一个 `mfence` 并不会"通知"别人；它改变的是**本线程访存之间的相对顺序**，以及由此产生的**跨线程可观察偏序**。把 fence 当成"内存同步按钮"会导致在错误的位置加错误数量的屏障——既慢又不对。

- **用 `volatile`（或普通变量）当同步原语。** 讲义反复强调"processor 与 compiler 都在重排"：`volatile` 只约束编译器对该变量的访问次数/顺序，**既不提供原子性，也不提供内存顺序**，更不会生成 `mfence`。正确的工具是 C++11 的 `std::atomic` + 合适的 `memory_order`（acquire/release/seq_cst），或平台提供的原子内建函数 + fence。
- **以为 x86 就是 SC，从而在 x86 上"验证通过"就发布代码。** x86 是 TSO：它允许 WX→RY 重排（写缓冲），所以 Store Buffering 的 `(0,0)` 在 x86 上**是允许的**——示例一在本机实测到它以 **13%–30%** 的比例反复出现（而加上 fence 或改用 seq_cst 后三次运行恒为 0）；x86 也**不保证编译器**不重排（编译器可以证明两个不同对象的访问不别名而换序），示例一中"运行时 mode 被 `-O2` 合并成 `xchg`"就是活生生的例子。同一份代码在 ARM/Power（更宽松）上会以**更高的频率**失败。判断标准只有一条：**程序是否 DRF，以及是否在正确位置使用了正确的原子操作**。
- **忽略伪共享（false sharing）——它能把并行度直接压到 1。** 两个线程写**不同变量**、但落在**同一条 64 B cache line** 上时，MSA/MSI 协议只认**行**：每次写都要独占整行，行在核之间 ping-pong，产生**完全人为的通信**。CS149 实测 8 线程 14.2 s（未 padding）vs 4.7 s（padding）；本讲 Work/Span 分析给出的解释是：**未 padding 时并行度 $W/S \approx 1$，padding 后恢复为 $\approx P$**。防御手段：`alignas(64)` / 手动 padding、每线程私有累加 + 末尾归约。
- **把同步事件做成"每元素一次"，忽略 fence/原子的吞吐上限与批处理。** 一次 `mfence` 约 20–40 周期、一次跨核原子往返约 30–100 ns，二者都远大于一次普通 ALU 操作。SPSC 队列若不批处理，每条消息就要一次索引行转移（$AI = 0$，线路流量放大 8 倍）；改成"每行服务 20 条消息"后，同样的硬件可以跑出高得多的吞吐——**代价是延迟。吞吐 vs 延迟**必须显式设计，而不是让默认实现替你决定。
- **只在小规模/单核/同核超线程下测试。** 同一物理核上的两个 SMT 线程共享 L1，一致性往返退化为本地命中，重排窗口消失，**几乎所有内存序 bug 都测不出来**。必须用 `taskset`（Linux）或 `numactl` 把线程绑到**不同物理核**上，并**重复多轮**（内存序 bug 是统计性的，单次运行成功毫无意义）。

---

7. 思考题（带答案）

Q1. 讲义 slide 20 的例子：P0 执行 `A = 1; Ready = 1;`，P1 执行 `x = Ready; y = A;`（A、Ready 初值均为 0）。(a) 为什么 `(x, y) = (1, 0)` 在 SC 下不可能？(b) 在 x86（TSO）上它可能吗？(c) 讲义 slide 37 在 P0 和 P1 上各标了 3 个候选位置（[1] A=1 之前、[2] A=1 与 Ready=1 之间、[3] Ready=1 之后；[4] x=Ready 之前、[5] x=Ready 与 y=A 之间、[6] y=A 之后），哪些是必要的？

【答案】

(a) SC 下的矛盾（happens-before 成环）：把 P0 的两条访存记为 $a: A{=}1$、$b: Ready{=}1$，P1 的记为 $c: x{=}Ready$、$d: y{=}A$。SC 保证每个线程的访问按 program order 出现在全局串行序中，即 $a \to b$、$c \to d$。 – 若 $y = 0$（即 d 读到初值），说明**在串行序中 d 早于 a**（因为 A 唯一的写是 a）； – 若 $x = 1$（即 c 读到了 b 写的值），说明**在串行序中 b 早于 c**； – 合并得：$a \to b \to c \to d \to a$ —— **一个环**，意味着某个事件必须发生在它自己之前，**矛盾**。因此 `(x,y) = (1,0)` 在 SC 下**不可能**（可能的结果只有 `(0,0)`、`(0,1)`、`(1,1)`）。讲义提示的正是这套推理："we know a→b and c→d by program order; b→c implies a→d; y==0 implies d→a which leads to a contradiction"。

(b) x86-TSO 下呢？ 这个结果需要 **`Ready=1` 在 `A=1` 之前提交**（一个 WX→WY 的破坏）或者 **`y=A` 被提到 `x=Ready` 之前**（一个 RX→RY 的破坏）。而 x86-TSO **只放松 WX→RY**：同一处理器的两次 store **按程序序提交**，两次 load **不互相重排**。因此**在 x86 硬件上，这个特定例子本身是安全的**（这也是它"看起来从来不报错"的原因）。但三点必须注意：① **编译器**仍然可能把 `A=1; Ready=1;` 换序（两者不别名，编译器完全有权重排），也可能把 `y = A` 提前——所以**软件层面依然需要约束**（用 `atomic` + release/acquire，或编译屏障）；② 在 **PSO/WO/RC 或 ARM/Power** 上，这一条会被硬件直接破坏（这正是讲义 slide 20 说"but real hardware will do this!"的语境）；③ 严谨的做法是**不依赖"我猜这台机器不会"**，而是显式表达意图。

(c) 哪些 fence 是必要的？ – 必要的是 [2]（P0 侧，位于 `A=1` 与 `Ready=1` 之间）与 [5]（P1 侧，位于 `x=Ready` 与 `y=A` 之间）。它们分别提供 **release** 语义（"数据写好之前不发布标志"）与 **acquire** 语义（"拿到标志之后才读数据"）。在 x86 上，[5] 的硬件效果由"load 不与 load 重排"免费提供（但**编译器**约束仍需 `atomic` 或编译屏障），[2] 需要一条 `mfence`（或把 `Ready=1` 写成 release store）。 – **[1]/[3]/[4]/[6] 是"保守但多余"的**：它们把一个额外的次序强加给某一段，换来更长的停顿而不增加任何必要的保证。[1]/[3] 只延长了 P0 的停顿（[3] 甚至要等 `Ready=1` 提交完成才能继续，而后面没有需要等待的访存）；[4]/[6] 同理。讲义的图示专门把 WO（Weak Ordering）标成 "**Overly Conservative**"，就是为了对比：**WO 要求在同步点两侧都等先前操作全部完成，而实际上只有一侧需要对的一类操作**——这正是 **Release Consistency（RC）** 的出发点。 – **工程落点**：现代代码不该手写这六个位置的取舍，而应写成 `A = 42; flag.store(1, std::memory_order_release);` 与 `while (flag.load(std::memory_order_acquire) == 0); use(A);` ——把 fence 的**种类与位置**交给 acquire/release 语义与编译器。

Q2. 某同学说："我的多线程代码用 `pthread_mutex` 保护了所有共享数据，所以在 x86 上正确，在 ARM 上也一定正确。" 请用数据竞争与 SC for DRF 判断这个说法，并指出它在什么情况下会失效。

【答案】

说法基本正确，但有一个前提必须被检查：程序是否真的"所有共享访问都被保护"，即是否 DRF。

1. **理论基础**：C11 / C++11 / Java 5 为**无数据竞争（data-race-free）**的程序提供**顺序一致性**保证（"SC for DRF"，CS149 slide 59）。pthread 的 `lock/unlock`（以及 C++ 的 `std::mutex`）在实现上是用 **acquire/release 语义的原子操作**做的：`unlock` 释放（release），`lock` 获得（acquire）。因此**同一把锁保护的临界区之间建立了 happens-before 关系**：前一个临界区里的所有写在下一个临界区开始前对后者可见。只要程序在锁的串行化下没有并发冲突访问，**它在 ARM 上跑出的结果与 SC 机器一致**——这正是讲义"properly synchronized programs yield SC results"与"relaxed models 的复杂度被封进库"的结论。
2. **失效情形一：程序其实有数据竞争（最常见）**。例如：某个共享变量**漏加锁**；或者用"双检锁（double-checked locking）"在锁外读一个标志/指针来决定是否加锁——那个**锁外的读**与写者构成数据竞争，于是**整个程序的保证全部失效（未定义行为）**。此时在 x86 上"看起来对"是因为 TSO 较强、编译器恰好没重排；在 ARM 上，即使 `pthread_mutex` 实现完全正确，程序依然可能读到未初始化的对象。**结论：锁的正确性只覆盖它保护的东西；锁外的裸访存会把 DRF 契约打碎。**
3. **失效情形二：把"同步"做在共享内存之外**。用 `volatile`、`std::atomic` 的 `relaxed` 顺序、"时间上看起来够久"的自旋等非同步手段来传递数据，都会破坏 DRF。
4. **失效情形三：锁的粒度和数据布局**。即使 DRF 成立，若两个线程各自加**不同的锁**却更新**同一条 cache line** 上的两个变量，程序依然 DRF（语言层面安全），但性能会因为**伪共享**退化（实测可达 3×；见示例三）——语义正确不等于性能可接受。
5. **正确的实践**：用**同一个**同步对象覆盖"发布数据"和"访问数据"两侧；优先使用语言/库提供的原语而不是自造标志位；用 ThreadSanitizer 之类的动态工具找数据竞争；并记住一句判据——**"用锁保护了所有共享数据" 必须被证明，而不能被假定**。

Q3. （定量）某系统：3.0 GHz、16 核、cache line 64 B、互连带宽 20 GB/s，一次一致性事务（行转移 + 串行化点排队）平均 40 ns。程序处理 10^8 个元素，每元素 8 B，且每个元素都要做一次落在同一条 cache line 上的原子 `fetch_add`。请估算：(a) 串行化点需要多久？(b) 需要多少一致性带宽？(c) 改成"每线程私有计数器 + 末尾一次归约"后是多少？(d) 用第 4 节的机器平衡点解释这组数字。

【答案】

(a) 串行化点的时间：所有 10^8 次原子操作都落在**同一条 cache line** 上，而每次 `fetch_add` 都要求该行的**独占所有权**，因此被互连这个唯一的串行化点排成一条链：

$$T_{\text{serial}} = 10^8 \times 40\ \text{ns} = 4.0\ \text{s}$$

（等价地说，串行化点的**事务吞吐上限** $= 1/40\ \text{ns} = 2.5 \times 10^7$ 事务/s，要完成 10^8 次就得 4 s。这与线程数、核数**无关**——并行度被压成 1，正是示例三"伪共享把并行度压回 1"的定量版本。）

(b) 一致性带宽：– 若每次原子操作都触发一次 64 B 行转移：$10^8 \times 64\ \text{B} = 6.4\ \text{GB}$，占用带宽时间 $6.4\ \text{GB}/20\ \text{GB/s} = 0.32\ \text{s}$。– 有效载荷只有 $10^8 \times 8\ \text{B} = 0.8\ \text{GB}$ ⇒ 有效载荷率 $0.8\ \text{GB/s}$（仅 4% 的带宽），而**线路流量 6.4 GB/s（放大 8 倍）占带宽的 32%**。– 参照量纲：**256 MB 数组在 20 GB/s 下遍历一次需 12.8 ms**；6.4 GB 是它的 **25 倍**，即纯搬运 0.32 s。– **对比 (a)：0.32 s（带宽）≪ 4.0 s（串行化延迟）** ⇒ 这个程序**不是带宽受限，而是被一致性事务的延迟–吞吐串行化点限制**。这是本讲最容易被误判的一类性能问题：盯着"带宽够不够"看不出问题，必须看"事务是不是被串行化了"。

(c) 私有计数器 + 末尾归约：– 把 16 个计数器用 `alignas(64)` 各自放到独立行，每个线程只更新自己的：**同一条行被多个核抢的情况消失**，每次 `fetch_add` 退化为本地 L1 的原子操作（~几个周期），一致性事务数从 10^8 降到 $O(P)$（外加归约时的 P 次行转移）。– 数据布局改为：改为各线程在**寄存器/本地变量**里累加（连原子操作都不需要），最后 P 个线程各做一次 `fetch_add` 到全局计数器：**原子操作数从 10^8 降到 16**，一致性流量近似为 0。– 上界由"每个线程在本地处理 $10^8/16 = 6.25 \times 10^6$ 个元素"决定：即使按每个元素 2 个周期估算，单线程 $\approx 1.25 \times 10^7$ 周期 $\approx 4.2\ \text{ms}$，16 线程并行**远优于 4.0 s**（提升可达百倍量级），并且剩下的开销是纯本地操作。

(d) 用机器平衡点解释：该机器峰值 $16 \times 3.0\ \text{GHz} \times 8\ \text{SIMD} \times 2(\text{FMA}) = 768\ \text{GFLOPS}$，带宽 20 GB/s ⇒ 机器平衡点

$$AI^* = \frac{768\ \text{GFLOPS}}{20\ \text{GB/s}} = 38.4\ \text{FLOP/Byte}$$

而本题的代码算术强度为 0（纯原子操作 + 数据搬运）。也就是说它**离平衡点差着无穷远**：**增加算力、增加核数、提高频率都毫无用处**，唯一的杠杆是"减少被拉回一致性序的次数"。这组数字（4.0 s vs 4.2 ms）说明：**在共享内存并行里，决定性能的往往不是有多少核心，而是有多少操作被迫共享同一条 cache line、以及它们被迫按什么顺序发生**——而"顺序由谁定义"，正是本讲（Memory Consistency）要回答的问题。

Lecture 22: Parallel Deep Learning: Data Parallelism

1. 章节标题与概述

Lecture 22: Parallel Deep Learning: Data Parallelism（数据并行训练：参数服务器、AllReduce 家族与 ZeRO 零冗余优化器）

- **本讲核心问题**：DNN 训练的本质是"**用一批样本算出梯度、把梯度加起来、再统一更新权重**"，而这句话里的求和号 $\nabla L(w) = \sum_{j=1}^{n} \nabla L_j(w)$ 天然就是可并行的（讲义 slide 7 直接把 SGD 的更新式写成带 Batch Size 的求和形式）。于是问题变成：**这台机器上有 N 张加速卡，怎么把这个"求和"摊到 N 张卡上？** 一旦摊开，瓶颈立刻从"算力"变成了"**通信**"：所有卡必须交换 M 个参数的梯度，而交换方式的不同选择（参数服务器 / Naive / Ring / Tree / Butterfly AllReduce）带来的时间差可以高达 **几十倍**（slide 25–26）。第二个问题随之而来：数据并行要求**每张卡都保存一份完整的模型副本**，当模型大到单卡显存装不下时（GPT-3 175B 需要约 2800–3500 GB，slide 29–30），数据并行本身就崩了——这就引出本讲后半部分的 **ZeRO（Zero Redundancy Optimizer）**：把"人人一份"的冗余账本切成 N 份，用通信量换取显存。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：跨设备互连（卡间 **NVLink**、主机内 **PCIe**、机间 **InfiniBand / Ethernet**）与它们各自的**带宽 + 延迟**特性；GPU 显存层次（SM 内的寄存器/共享内存 → 6 MB L2 → 16 GB HBM，**900 GB/s**）；集合通信硬件路径（ring、tree、butterfly 三种拓扑对应不同的链路占用模式）。
- **软件侧**：**mini-batch SGD** 与 **Adam** 更新规则、**梯度聚合（gradient aggregation）** 的四种算法、**AllReduce / AllGather / ReduceScatter** 集合通信原语、**混合精度训练（FP16 参数与梯度 + FP32 主权重与优化器状态）**、**ZeRO 的三个阶段**（切优化器状态 / 切梯度 / 切参数）、以及把通信与反向计算**分桶重叠（bucketing / overlap）**的工程手段。
- **在并行计算知识体系中的角色**：这是全课程"**从单机并行走向多机并行**"的枢纽一讲。前面讲过的算术强度、Roofline、分块、SIMD、warp、共享内存、树的归约，在这里被拼装成一个真实的超大规模负载：**计算内部用 GEMM 分块榨干单卡算力，卡之间用 AllReduce 做跨越整个集群的树形/环形归约**。它同时是"集合通信"这一主题在课程中的第一次系统登场（Ring / Tree / Butterfly 的通信量与延迟公式就是"归约网络的 work-span 分析"），也是下一讲 **Lecture 23（model / pipeline parallelism）** 的铺垫：数据并行解决"数据太多"，模型/流水并行解决"模型太大"，而真实训练里两者必须组合（讲义 slide 66 提到 17.2B 的 Turing NLG 就是 Stage 1 + Megatron 张量并行一起上）。
- **配套材料**：
- `lectures/24-parallel_deep_learning_data_parallel.pdf`（抽取文本 `extracted/24-parallel_deep_learning_data_parallel.txt`，共 **80 页**）——**已公开，可在公开网络直接下载**。Fall 2026 日程表（https://www.cs.cmu.edu/~418/schedule.html）把 **Oct 23** 排为第 **22** 讲 "Parallel Deep Learning (data parallelism)"，该行的 slides/video 链接目前**以 HTML 注释形式给出**（注释原文："slides/video from a previous offering; uncomment when posted for Fall 2026"），注释中的 slides 指向 `lectures/24-parallel_deep_learning_data_parallel.pdf`，video 指向归档 YouTube 链接 https://www.youtube.com/watch?v=AbsVyQqqlcM。该 PDF 位于公开目录 https://www.cs.cmu.edu/~418/lectures/ 之下。**三点说明（不是错误）**：①**文件编号 24 与 Fall 2026 讲次 22 不一致**（历史学期讲次重排），讲义首页也写着 **"CMU 15-418/15-618, Fall 2025"**；②讲义首页署名是 **Zhihao Jia（Stanford University）**，并带有 "Automated Approaches to Accelerate Machine Learning" 的标题条——即这一讲是**客座/外请讲座的沿用版本**；③讲义中有大量**纯动画页**（如 slide 42–65 的 ZeRO Stage 1 逐步动画、slide 14–20 的 ring allreduce 逐帧动画）在文本抽取后**只剩重复的图注文字**，本笔记对这些内容依据同页可见文字（如"Step 1 (Aggregation): each worker send one slice (M/N parameters)..."、"Backward propagation to generate FP16 gradients and AllReduce to average"）展开，不冒充原文中不存在的数字。
- `cs149_supp/dnninference.txt`（抽取文本，共 **75 页**）——**已公开**。Stanford CS149（Fall 2025）**Lecture 9: Efficiently Evaluating DNNs**：讲 DNN 前向推理的性能优化（pipelining 与 double buffering、Roofline 与算术强度、循环融合 1/3 → 3/5、分块 GEMM、explicit/implicit GEMM、Flash-Attention 的分块 softmax、低精度 16/8/4-bit、以及 V100 的 6 MB L2 + 900 GB/s HBM）。它是**本讲的性能分析工具箱**：本讲第 4 节用它给出的算术强度、Roofline、分块思路定量解释"为什么梯度计算可以是计算受限、而 AllReduce 永远只能是带宽受限"。
- 讲义中引用的外部材料：**Kingma and Ba, "Adam: A Method for Stochastic Optimization", 2014**（https://arxiv.org/abs/1412.6980，slide 33 的 Adam 更新规则出处）；**Vaswani et al., "Attention is all you need"**（slide 34 的 Transformer 结构出处）；以及 slide 35–65 标注的 "Adapted from Minjia Zhang, DeepSpeed Presentation"（ZeRO / 内存账本动画的来源）。硬件规格（V100 16G/32G、A100 40G/80G）出自 slide 30。
- **未发布 / 需登录**：本讲的**讲课录像（Panopto / YouTube）在 Fall 2026 日程表中被注释隐藏，属未发布**；**Ed 讨论区、Autolab、Canvas 均需登录**，非公开。部分讲座（Performance Analysis/Profiling、Transactional Memory、AI in System Design 等）在 Fall 2026 **尚未发布讲义**，其历史学期 PDF 位于 `/afs/cs/academic/class/15418-*/public/` 之下，**需要 CMU 登录，属未公开**。
- **课程语境**：Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。Fall 2026 日程表把当天标注为 **Assignment 4 due**；前一讲为 Lecture 21 "Memory Consistency"（Oct 21），后一讲为 Lecture 23 "Parallel Deep Learning (model and pipeline parallelism)"（Oct 26，对应公开 PDF `25-parallel_deep_learning_model_pipeline_parallel.pdf`）。

2. 核心概念与硬件/软件架构图解

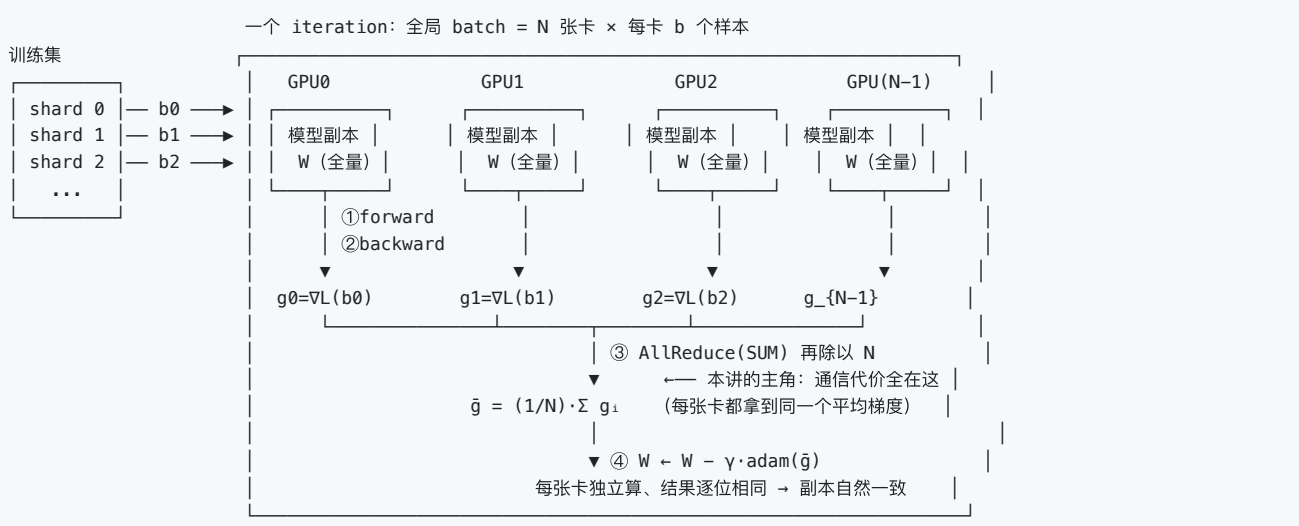
2.1 回顾：DNN 训练的三个阶段与"可并行性"的来源

- **定义与目的**：讲义 slide 4–6 用三张图把"训练"这件事定义清楚：① **前向传播（forward propagation）**——把一批输入样本喂进模型，逐算子计算得到预测；② **反向传播（backward propagation）**——把模型"反过来跑"，为每个可训练权重算出一个梯度 $\partial L(w)/\partial w_i$；③ **权重更新（weight update）**——用梯度把权重往损失下降的方向推：

$$w_i := w_i - \gamma \frac{\partial L(w)}{\partial w_i}, \qquad \frac{\partial L(w)}{\partial w_i} = \sum_{j=1}^{n} \frac{\partial l_j(w)}{\partial w_i}$$

其中 γ 是学习率（step size），n 是 batch size，求和内的每一项是**单个样本的梯度**。讲义 slide 7 直接问："How can we parallelize DNN training?"——答案就写在那个求和号上。

- **直观解释（"它是什么？"）**：把训练想象成**一个复习小组在刷同一本题库**。
- **模型**是大家共用的一份"解题笔记"；
- **数据**是题库里的题目；
- **梯度**是"我做完这批题之后，发现笔记里的哪几行该改、改多少"的**修改意见**；
- **SGD 的求和号**就是"把组里所有人的修改意见**平均**一下再一起改笔记"。 关键洞察在于：**意见可以各算各的（完全并行、互不干扰），但"平均之后再改"这一步必须所有人改得一模一样**——否则下一轮大家手里的笔记就不是同一份了，后面所有计算全错。这就是数据并行的全部张力的来源：**计算可以自由拆分，状态必须强制同步**。
- **图 1：数据并行的软件执行模型（一个 iteration 的全流程）**



- **性能特征**：前向+反向的计算量随卡数 N **线性分摊**（每卡 1/N），而梯度聚合的通信量**几乎与 N 无关**（下一节会证明 ring allreduce 每卡收发约 2M 个参数）。这意味着数据并行是一个"**固定开销 + 可变计算**"的结构：卡越多，通信占比越大，最终加速比被"计算时间/通信时间"这个比值封顶。这就是本讲所有性能分析的主线。

2.2 参数服务器（Parameter Server）：最直观的集中式方案与它的天花板

- **定义与目的**：讲义 slide 9 给出第一种方案：**worker 把梯度 push 给参数服务器（parameter server），再从服务器 pull 回更新后的参数**。它解决的问题是"**谁来负责把 N 份梯度加起来**"——答案是"设一个专门的服务器进程/机器来当那个求和点"，worker 之间不需要互相认识。
- **直观解释（"它是什么？"）**：像一个宿舍楼里**只有一台公用洗衣机**。每个人各自攒一筐衣服（本地梯度），都送到同一间洗衣房，洗好之后再由这台机器把衣服分回给每个人（更新后的参数）。人少的时候很方便、逻辑也简单；但**人一多，队伍全堵在那一台机器门口**——机器的进水/出水口（服务器网卡带宽）成了唯一通道，排队时间随人数线性上涨。讲义 slide 10 的原话正是："Centralized communication: all workers communicate with parameter servers for weights update; cannot scale to large numbers of workers"。
- **图 2：参数服务器 vs AllReduce 的通信拓扑（中心化 vs 去中心化）**

- **性能特征**：设模型有 M 个参数、互连带宽 BW，PS 必须**串行地**服务 N 个 worker 的收发，讲义 slide 26 给出的延迟是

$$T_{PS} \approx \frac{M \cdot N}{\text{bandwidth}}$$

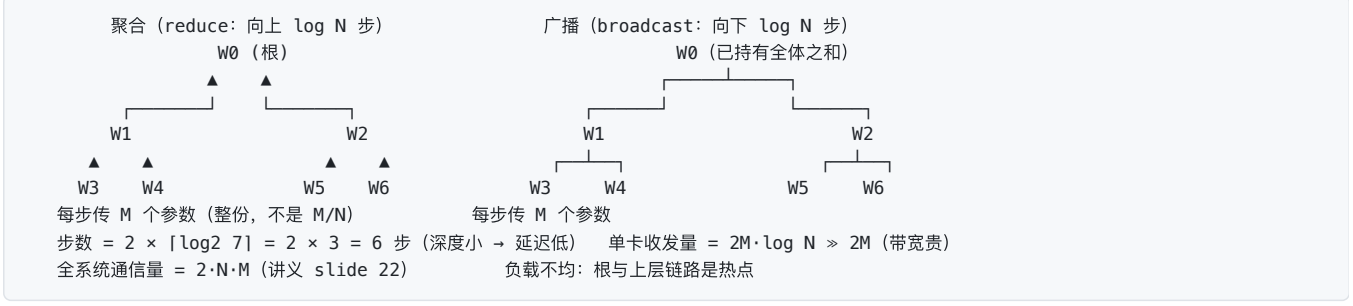
即**随 worker 数线性劣化**；而且所有 worker 的流量都要挤过服务器这一跳，服务器网卡是唯一瓶颈。讲义紧接着提出问题："**How can we decentralize communication in DNN training?**"（slide 10）——答案就是 **AllReduce**。

2.3 AllReduce 家族：Naive / Ring / Tree / Butterfly

- **定义与目的**：**AllReduce 是"逐元素归约（element-wise reduction）"的集合通信原语**（slide 11）：N 个设备各持一个长度为 M 的向量，操作结束后**每个设备都拥有这 N 个向量逐元素之和**。它一次解决了两件事：**求和**（reduce）与**把结果广播回来**（broadcast）——正好对应数据并行里"平均梯度 + 所有副本保持一致"的需求。
- **直观解释（"它是什么？"）**：把 AllReduce 想成"**全班交换作业本，最后每个人手里都拿到全班的分数合计表**"。四种算法的区别只是**怎么传本子**：
- **Naive**：每个人都把自己的本子复印 N−1 份、发给其他所有人 → 纸（带宽）消耗 $O(N^2)$。
- **Ring（环）**：大家围成一圈，每人**只把一页（M/N 个参数）传给右手边的人**，传 N−1 轮，每页沿环被"接力累加"一圈（Aggregation）；然后再传一圈把所有页补齐（Broadcast）。像击鼓传花 + 传阅笔记本：**每人经手的量固定，队伍再长也不吃亏**。
- **Tree（树）**：像公司的组织架构，**层层上报（聚合）再层层下发（广播）**，深度只有 log N，但**每个人每次要传整本（M 个参数）**，而且根节点和上层链路要承受巨大流量。
- **Butterfly（蝶形）**：像**每一轮全体换座位两两对接**，第 k 轮每个人与"编号差 2^k"的伙伴交换全部 M 个参数并本地求和；log N 轮之后人人拿到总和。**轮数最少（低延迟），但每人经手 M·log N 个参数（带宽最贵）**。
- **图 3：Ring AllReduce 的 6 步完整时序（N=4，M 个参数切成 c0..c3 四片）**

```
环：  W0 ──▶ W1 ──▶ W2 ──▶ W3 ──▶ W0      （每卡只与 left/right 邻居通信）
每片大小 = M/N 个参数；括号内数字 = 该片上已经累加的"贡献数"

阶段 1  Reduce-Scatter（N−1 = 3 步）：第 k 步 送 c_{(i−k) mod N}，收 c_{(i−k−1) mod N} 并就地把收到的加进自己的副本
 步 k=0   送自己的片          W0: c3(2)   W1: c0(2)   W2: c1(2)   W3: c2(2)
 步 k=1   送上一轮收到的片     W0: c2(3)   W1: c3(3)   W2: c0(3)   W3: c1(3)
 步 k=2                       W0: c1(4)✓  W1: c2(4)✓  W2: c3(4)✓  W3: c0(4)✓
 ★ 结论：worker i 手里"恰好"完成归约的是片 c_{(i+1) mod N}（每片都走满一圈、被加 N−1 次）

阶段 2  AllGather（N−1 = 3 步）：第 k 步 送 c_{(i+1−k) mod N}（送出的必须是"已归约片"），从前驱收 c_{(i−k) mod N}
 步 k=0  每卡已有 2/4 片    步 k=1  3/4 片    步 k=2  4/4 片 ✓ 人人拥有完整的 c0..c3 之和

总量：2(N−1) = 6 步；每步每卡收发 M/N 个参数
      每卡总收发 = 2(N−1)·M/N ≈ 2M 个参数（与 N 无关！）
      全系统通信量 = N·2(N−1)·M/N ≈ 2NM（讲义 slide 20 记作 2·M·N）
```

- **图 4：Tree AllReduce 的组织结构与代价（N=7）**



- **图 5：Butterfly AllReduce 的交换模式（N=8，log N = 3 步）**

```
规则：第 k 步（k = 0,1,2），worker i 与 worker（i XOR 2^k）交换"全部 M 个参数"，然后本地相加

  步 0（伙伴 = i^1）          步 1（伙伴 = i^2）          步 2（伙伴 = i^4）
  0 ◀──▶ 1    2 ◀──▶ 3        0 ◀──▶ 2    1 ◀──▶ 3        0 ◀──▶ 4    1 ◀──▶ 5
  4 ◀──▶ 5    6 ◀──▶ 7        4 ◀──▶ 6    5 ◀──▶ 7        2 ◀──▶ 6    3 ◀──▶ 7
  └─ 4 对并发 ─┘              └─ 4 对并发 ─┘              └─ 4 对并发 ─┘
  → 3 步后 8 个节点全部持有完整的和
  → 每步每节点收发 M 个参数 ⇒ 单卡 = M·log N，全系统 = N·M·log N（讲义 slide 24）
  ★ 用"更多带宽"换"更少轮数"：延迟 log N 步，但总通信量比 ring 多 log N / 2 倍
   （讲义 slide 23 给出的硬件背景：butterfly / omega 多级互连网络，每级做 2×2 交换）
```

- **性能特征小结（表 1）**：通信量、单卡通信量、延迟、可扩展性四个维度一起看，才能理解为什么讲义 slide 25 要问"**Ring AllReduce 比 Tree / PS 更高效可扩展，为什么？**"

表 1：四种梯度聚合方案的定量对比（M = 参数量，N = worker 数，BW = 链路带宽，α = 单次消息启动延迟）

方案	全系统通信量	单卡收发量	延迟（讲义 slide 26 的带宽项）	完整 α–β 延迟模型	负载均衡	可扩展性
Parameter Server	2·N·M	2M（但服务器侧 ∝ N·M）	M·N / BW	N·M/BW + 2α	差（服务器是热点）	差：中心带宽随 N 线性劣化
Naïve AllReduce	N^2·M（= N(N−1)M）	(N−1)M	(N−1)M / BW	(N−1)·(α + M·w)	好	差：O(N^2) 通信
Ring AllReduce	**2·N·M**	**≈2M（与 N 无关）**	**2M / BW**	**2(N−1)·(α + (M/N)·w)**	**好**	**好：单卡量恒定**
Tree AllReduce	2·N·M	2M·log N	2·log N·M / BW	2·log N·(α + M·w)	差（根/上层热点）	中：延迟低、带宽贵
Butterfly AllReduce	N·M·log N	M·log N	log N·M / BW	log N·(α + M·w)	好	中：轮数最少、带宽最贵

> 注：表中"全系统通信量"一列取自讲义 slide 25 的原始表述（PS/Naive/Ring/Tree/Butterfly 分别为 $2NM$、N^2M、$2NM$、$2NM$、$NM\log N$）；"延迟"一列取自 slide 26（PS: MN/BW；Ring: $(M/N)\cdot 2N/BW$；Tree: $M\cdot 2\log N/BW$）。**"完整 α–β 延迟模型"那一列是本笔记补充的**（讲义的延迟公式只写了带宽项，没有显式的 α 项），其中 w = **每个参数元素的传输时间**（= 每参数字节数 / BW，fp16 且链路 10 GB/s 时 $w = 5\times10^{-11}$ s），α = 单次消息的启动延迟。这一列把"步数多但每步小"和"步数少但每步大"的差别显性化，4.4 节用它算出 cross-over 点。

2.4 数据并行的内存墙：混合精度训练的"内存账本"

- **定义与目的**：数据并行的第一个致命假设是"**每张卡都存一份完整模型**"（讲义 slide 28：*Each GPU saves a replica of the entire model*），因此**模型参数超过单卡显存就完全无法训练**。讲义 slide 29–30 用一张表把这个问题量化，并用一句 "Out of Memory" 收尾（V100 只有 16G/32G，A100 是 40G/80G）。

表 2：大模型规模增长（讲义 slide 29 原始数据）

模型	Parameters	Layers	Hidden Dim	Relative Computation	Memory Footprint
Bert-Large	0.32B	24	1024	1×	5.12 GB
GPT-2	1.5B	48	1600	4.7×	24 GB
Turing NLG 17.2B	17.2B	78	4256	54×	275 GB
GPT-3	175B	96	12288	547×	2800 GB

> 一致性校验（本笔记的推算，用来确认读懂了口径）：表中的 Memory Footprint 恰好等于 **16 字节/参数**（0.32B×16 = 5.12 GB；17.2B×16 = 275.2 GB；175B×16 = 2800 GB）。这 16 字节 = FP32 主权重(4) + FP32 梯度(4) + Adam 一阶动量(4) + Adam 二阶动量(4)。而 slide 40 给出的**混合精度训练实际账本**是 **20 字节/参数**——多出来的 4 字节是"额外保留一份 FP16 参数与 FP16 梯度"（2+2）。两个口径不要混用。

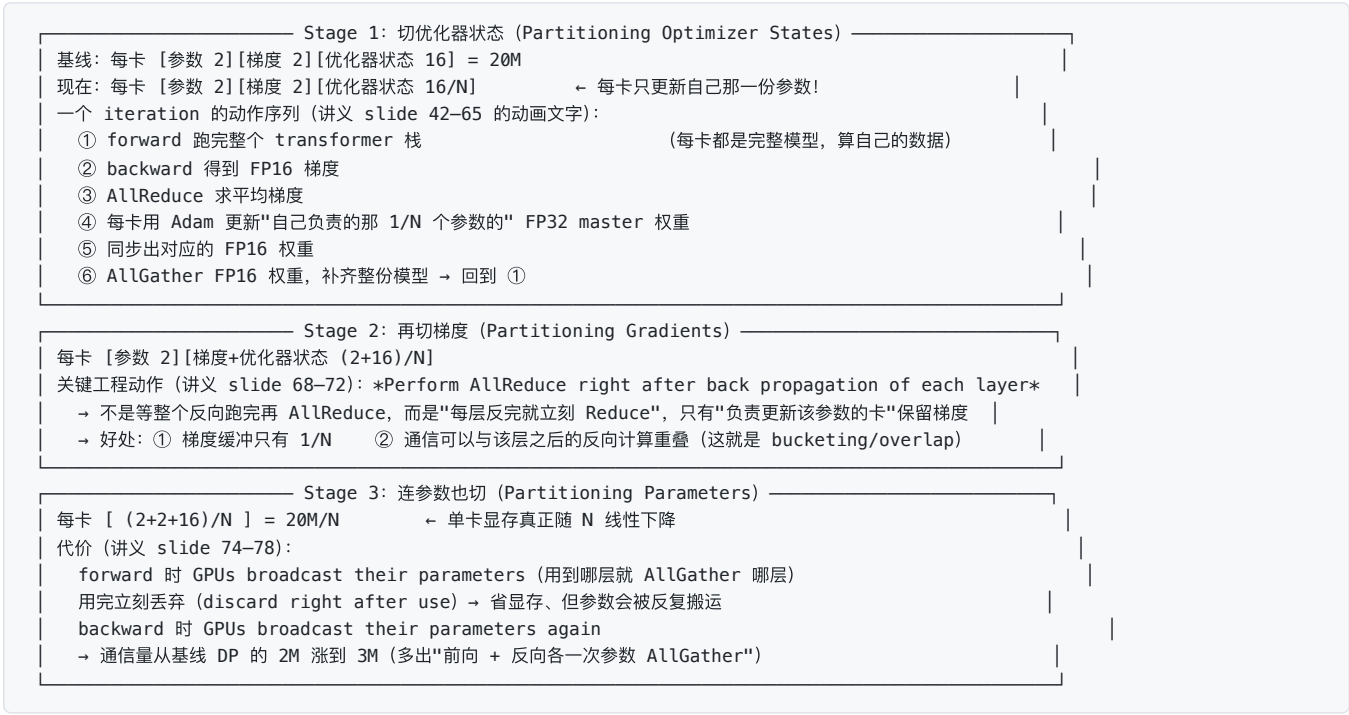
- **直观解释（"它是什么？"）**：混合精度训练就像**一套"正式账本 + 便签"的记账方式**。计算（前向/反向）用 **FP16**，因为算得快、占地方小；但直接拿 FP16 累加更新会因舍入误差把模型"记花"，所以必须再留一份 **FP32 的正式账本**（master weights），外加 Adam 需要的两个 FP32 统计量（动量、方差）。于是**每 1 个参数，要在显存里占 20 个字节**——1B 参数的模型就是 20 GB/卡（讲义 slide 40 的原话："Example 1B parameter model –> 20GB/GPU"，并特别注明"Memory consumption doesn't include: Input batch + activations"）。
- **图 6：单卡显存账本与 ZeRO 三级切割**

```
每张卡上的"训练状态"账本（M = 参数量；单位：字节/参数）          1B 参数模型 → 20 GB/卡

┌──────────────────────────────────────────────────┐
│ FP16 参数（前向/反向实际使用的副本）          2 B  │  ← 便签
│ FP16 梯度                                    2 B  │
├──────────────────────────────────────────────────┤
│ FP32 master 参数                             4 B  │  ┐
│ FP32 梯度                                    4 B  │  ├ 16 B = "FP32 优化器状态"
│ FP32 Adam 一阶动量 m                         4 B  │  │   （讲义 slide 40：16M bytes）
│ FP32 Adam 二阶动量 v                         4 B  │  ┘
├──────────────────────────────────────────────────┤
│ 合计                                        20 B  │
│ ★ 未计入：输入 batch 与全部中间激活 activations    │
└──────────────────────────────────────────────────┘

ZeRO 把这份账本按 N 张卡"切"开（下表为每卡显存，单位 M bytes，N = 卡数）
  基线数据并行 : [2 参数][2 梯度][ 16  优化器状态 ]      = 20M          ← 每卡一份全量，卡数不省内存
  Stage 1      : [2 参数][2 梯度][ 16/N 优化器状态 ]     = 4M + 16M/N
  Stage 2      : [2 参数][ (2+16)/N ]                    = 2M + 18M/N
  Stage 3      : [        (2+2+16)/N        ]            = 20M/N       ← 参数也切了
  ★ 切得越狠，省得越多，但每级都要额外付出一次集合通信（ReduceScatter / AllGather）
```

- **性能特征**：单纯看计算，FP16 让 Tensor Core 吞吐翻数倍；但真正的系统收益在于**显存**：M 个参数的账本从 20M bytes 变成 `20M/N` bytes（Stage 3），代价是**每张卡每次前向要 AllGather 别人的参数、反向完要 ReduceScatter 梯度**（讲义 slide 76–78：*GPUs broadcast their parameters during forward / Parameters are discarded right after use / GPUs broadcast their parameters again during backward*）。

2.5 ZeRO：Zero Redundancy Optimizer（把"人人一份"变成"人人一份之 1/N"）

- **定义与目的**：讲义 slide 31 定义 ZeRO 为"**Eliminating data redundancy in data parallel training**"，并说它是"**a widely used technique for data parallel training of large models**"。它要解决的就是 2.4 节的内存墙：既然 N 张卡上存了 N 份**完全相同**的优化器状态/梯度/参数，那就别存了——**每张卡只负责其中 1/N，需要时再向别人要**。
- **直观解释（"它是什么？"）**：像**一套百科全书的联机共享**。过去是每个宿舍都买一整套（贵、且 90% 的时间在落灰）；ZeRO 改成**各分馆分藏不同卷**，谁要查哪一卷就**快递过来，用完立刻寄回去**（讲义 slide 77 的 "Parameters are discarded right after use"）。查得频繁，快递费（通信）就涨——这就是 ZeRO 用**通信量**换**显存**的交换关系。 三个阶段可以类比"先共享最贵的、再共享次贵的"：**Stage 1 先共享最占地方、用得最少的（优化器状态，16 B/参数的巨无霸）；Stage 2 再共享梯度；Stage 3 连参数也共享**。
- **图 7：ZeRO 三个阶段的"切什么"与流水线动作**



- **性能特征**：讲义 slide 66 / 79 强调 ZeRO 是"**progressive memory savings and communication volume**"（逐步省内存、通信量逐步上升），并给出一个真实案例：**17.2B 的 Turing NLG 由 Stage 1 + Megatron 支撑**（即 ZeRO-1 + 张量并行组合使用，而不是只用一种并行方式）。

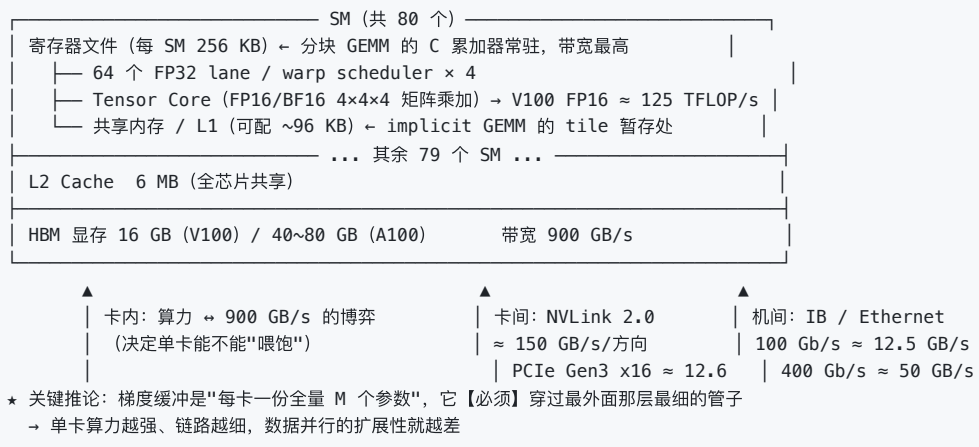
表 3：ZeRO 三阶段的内存与通信账本（M = 参数量，N = 卡数；通信量为"每卡每个 iteration 搬运的参数元素数"）

配置	每卡显存（bytes）	175B 模型、N=64 时的每卡显存	每卡通信量（元素）	通信量（fp16 字节）	相对基线通信
基线数据并行	20M	3500 GB（不可能）	2M（AllReduce 梯度）	4M = 700 GB	1.0×
ZeRO Stage 1	4M + 16M/N	743.8 GB（仍不可能）	2M（ReduceScatter 梯度 + AllGather 参数）	700 GB	1.0×
ZeRO Stage 2	2M + 18M/N	399.2 GB（仍不可能）	2M（同上，但梯度被切、可重叠）	700 GB	1.0×
ZeRO Stage 3	**20M/N**	**54.7 GB（A100-80GB 装得下）**	**3M**（两次参数 AllGather + 一次梯度 ReduceScatter）	**1050 GB**	**1.5×**

> 读表要点：**Stage 1/2 的通信量与基线完全相同**（这是 ZeRO 论文里最反直觉的结论之一：把 AllReduce 拆成 ReduceScatter + AllGather，总量不变），只有 Stage 3 因为要多传一轮参数而变成 1.5 倍。也就是说 **"省显存"在前两个阶段几乎是免费的，第三个阶段才要付出 50% 的通信代价**。表中 175B 的显存数字由 `20M/N`、`4M+16M/N`、`2M+18M/N` 代入 $M = 175\times10^9$ 直接算出（1 GB = 10^9 bytes）。

2.6 硬件视角：这些通信到底跑在什么上

- **定义与目的**：AllReduce 的代价最终由**物理链路**决定。单机内 GPU 之间走 **NVLink**（V100 上 6 条链路 × 25 GB/s/方向），跨主机走 **PCIe + 网卡（InfiniBand / Ethernet）**；而单卡内部的算力则由 GPU 的内存层次决定（CS149 slide 48 的 V100 剖面：**80 个 SM、6 MB L2、16 GB HBM、900 GB/s**）。
- **直观解释（"它是什么？"）**：把集群想成**一个城市的物流网**：SM 内部是"货架到工作台"的距离（寄存器/共享内存），900 GB/s 的 HBM 是"市内主干道"，NVLink 是"楼内电梯"，InfiniBand 是"城际高速"。**AllReduce 是必须跑满全城的重卡运输**——所以数据并行训练的性能几乎总由"城际高速的口径"决定，而不是由"工作台算得多快"决定。
- **图 8：GPU 内存层次与互连（硬件结构；数值取自 CS149 slide 48 与讲义 slide 30）**



- **性能特征**：这张图直接给出两条性能约束：
- **算力约束**：单卡要跑满 125 TFLOP/s（FP16 Tensor Core），数据复用必须做到 **125e12/900e9 ≈ 139 FLOP/byte** 以上（V100 的 **ridge point**，CS149 slide 6–8 的 Roofline 拐点）；
- **通信约束**：每一次迭代都必须搬运 2M 个参数穿越 NVLink 或网卡，**这个量不随卡数减少**。

---

3. 代码示例与性能分析

3.1 示例一：手写 MPI Ring AllReduce（把讲义的 6 步算法变成可跑的程序）

- **代码**

```c
/* ring_allreduce.c —— 手写 Ring AllReduce，并与 MPI_Allreduce 对拍校验
 * 编译：mpicc -O3 -march=native -std=c11 ring_allreduce.c -o ring_allreduce -lm
 * 运行：mpirun -np 8 --oversubscribe ./ring_allreduce 100000000 5
 *       参数 1 = 每张卡本地梯度的元素数 M；参数 2 = 重复次数（取最快一次）
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <limits.h>

/* 就地做一次 Ring AllReduce：recvbuf 返回"所有 rank 的 sendbuf 之和"
 * 通信模式与讲义 slide 14-20 完全一致：ReduceScatter(N-1 步) + AllGather(N-1 步) */
static double ring_allreduce(const float *sendbuf, float *recvbuf, long M,
                             int rank, int size, float *scratch)
{
    const long chunk = M / size;                    /* 每片 M/N 个参数 */
    const int  left  = (rank - 1 + size) % size;    /* 环上前驱 */
    const int  right = (rank + 1) % size;           /* 环上后继 */

    memcpy(recvbuf, sendbuf, (size_t)M * sizeof(float));
    const double t0 = MPI_Wtime();

    /* ---------- 阶段 1：Reduce-Scatter（N-1 步，就地累加）---------- */
    for (int k = 0; k < size - 1; k++) {
        const int send_c = (rank - k + size) % size;       /* 本步交出哪一片 */
        const int recv_c = (rank - k - 1 + size) % size;   /* 本步收到哪一片 */
        MPI_Sendrecv(recvbuf + (size_t)send_c * chunk, (int)chunk, MPI_FLOAT, right, 0,
                     scratch,                          (int)chunk, MPI_FLOAT, left,  0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        float *dst = recvbuf + (size_t)recv_c * chunk;
        for (long i = 0; i < chunk; i++) dst[i] += scratch[i];   /* 本地累加 = reduce */
    }

    /* ---------- 阶段 2：AllGather（N-1 步，把已归约片传一圈）---------- */
    for (int k = 0; k < size - 1; k++) {
        const int send_c = (rank + 1 - k + size) % size;   /* 送"刚拿到的已归约片" */
        const int recv_c = (rank     - k + size) % size;
        MPI_Sendrecv(recvbuf + (size_t)send_c * chunk, (int)chunk, MPI_FLOAT, right, 1,
                     recvbuf + (size_t)recv_c * chunk, (int)chunk, MPI_FLOAT, left,  1,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    return MPI_Wtime() - t0;
}

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const long M = (argc > 1) ? atol(argv[1]) : 1000000L;
    const int  R = (argc > 2) ? atoi(argv[2]) : 3;
    if (M % size != 0 || M / size > INT_MAX) {
        if (rank == 0) fprintf(stderr, "要求 M 能被 size 整除，且 M/size <= INT_MAX\n");
        MPI_Finalize();
        return 1;
    }

    float *local   = (float *)malloc((size_t)M * sizeof(float));
    float *mine    = (float *)malloc((size_t)M * sizeof(float));
    float *ref     = (float *)malloc((size_t)M * sizeof(float));
    float *scratch = (float *)malloc((size_t)(M / size) * sizeof(float));

    for (long i = 0; i < M; i++)                    /* 每卡不同的"本地梯度" */
        local[i] = (float)(((rank * 131 + i) % 17) - 8) * 0.25f;

    double best = 1e30;
    for (int r = 0; r < R; r++) {                   /* 计时：取最快一次，排除冷启动 */
```

```
        MPI_Barrier(MPI_COMM_WORLD);
        const double t = ring_allreduce(local, mine, M, rank, size, scratch);
        if (t < best) best = t;
    }

    MPI_Allreduce(local, ref, (int)M, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);  /* 参考实现 */

    double myerr = 0.0;
    for (long i = 0; i < M; i++) {
        const double e = fabs((double)mine[i] - (double)ref[i]);
        if (e > myerr) myerr = e;
    }
    double gerr = 0.0;
    MPI_Reduce(&myerr, &gerr, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    /* 每卡"发出 + 收到"的总字节数: 2(N-1) 步 × 每步 2 次传输 × chunk 个 float */
    const double bytes = 2.0 * (size - 1) * (double)(M / size) * sizeof(float) * 2.0;
    if (rank == 0) {
        printf("N=%2d  M=%ld (每卡 %.1f MB)   单步片大小 = %.1f KB\n",
               size, M, (double)M * 4 / 1e6, (double)(M / size) * 4 / 1e3);
        printf("  Ring AllReduce 时间 = %.3f ms; 每卡收发 %.1f MB; 有效带宽 = %.2f GB/s\n",
               best * 1e3, bytes / 1e6, bytes / best / 1e9);
        printf("  与 MPI_Allreduce 的最大逐元素误差 = %.3e\n", gerr);
        printf("  理论上每卡收发 2*M*(N-1)/N 个参数 = %.1f MB (与上式一致) \n",
               2.0 * M * (size - 1) / size * sizeof(float) / 1e6);
    }
    free(local); free(mine); free(ref); free(scratch);
    MPI_Finalize();
    return 0;
}
```

- **【代码做什么？】**(逐步解释执行流程)
- **初始化与切分**：进入 `ring_allreduce` 后先 `memcpy` 一份本地梯度到 `recvbuf`（这一步对应"每张卡都持有完整模型和本地梯度"），然后把 M 个参数按 rank 切成 `chunk = M/N` 大小的 N 片，并算出环上的前驱 `left`、后继 `right`。**只要邻居，不要中心**——这正是讲义 slide 10 说的 "decentralize communication"。
- **阶段 1（Reduce-Scatter，N−1 步)**：第 k 步，rank 把第 `(rank-k) mod N` 片发给后继，同时从前驱收第 `(rank-k-1) mod N` 片，**就地累加**进自己的 `recvbuf`。执行完 N−1 步之后，按 2.3 节的推导，rank i 手里**恰好有一片是完整的和**（片 `(i+1) mod N`）——这就是图 3 里"w0: c1(4)✓"那一行。
- **阶段 2（AllGather，N−1 步)**：第 k 步发送 `(rank+1-k) mod N` 片（**必须是这一步手里那一片"已归约"的片**），接收 `(rank-k) mod N` 片并**直接覆盖**。因为每步收到的片在下一步才需要转发，覆盖是安全的——这正是流水线的关键：**接收缓冲区同时是下一步的发送缓冲区**，不需要双倍缓冲。
- **校验与计时**：`MPI_Allreduce` 作为参考实现，逐元素比最大误差（浮点求和不满足结合律，因此不能要求逐位相等)；计时取 R 次最快值，并用 `bytes = 4(N-1)·chunk` 反算"有效带宽"，和理论值 `2M(N-1)/N` 对拍。
- **【并行机制与性能解说】**
- **谁在并行、怎么分工**：这里并行的是 **N 个 MPI 进程（通常跨机器)**，每个进程内部的累加循环是纯串行的、SIMD 友好的（`dst[i] += scratch[i]` 会被编译器向量化成 `vaddps`)。进程间的"工作分配"由**环下标算术** `(rank ± k) mod N` 静态决定，没有任何动态调度 —— 这也是 ring 能做到"负载完美均衡"的原因。
- **共享数据怎么处理**：**没有共享内存**；每个进程持有 M 个参数的私有副本，跨进程传递靠显式的 `MPI_Sendrecv`（点对点、双向同时进行，避免死锁)，"共享"只发生在阶段 1 的逐元素加法这一瞬（每一步只交换 M/N 片)。
- **Work / Span / 并行度**（用第 2.3 节的算法结构直接推)：
  - **Work（浮点加法总数）** = 每卡 (N−1)·(M/N) 次加法 × N 卡 = **M(N−1)** 次加法；
  - **Span（关键路径）** = 单个参数的累加链长度 = **N−1** 次相关加法；用时间计则还要加上通信步数：**2(N−1) 轮**，每轮耗时 α + (M/N)·w（α = 消息启动延迟，w = 每元素传输时间)；
  - **算法内禀并行度 = Work / Span = M(N−1)/(N−1) = M** —— 也就是说**参数维度是完全独立的**，这是"数据并行能扩展"的根本原因；
  - 但**单张卡在纯 ring 里只能用上 M/N 路并行**（每片内 M/N 个独立加法 + 每片一条累加链)。这就是为什么 N 很大时单卡算力吃不满、真实库（NCCL）要把梯度切成**多个并发环（channels）**才能同时打满多块网卡/多条 NVLink。
- **瓶颈分析**：
  - **带宽瓶颈（主导)**：每卡至少要把 ≈2M 个参数（收发合计；精确值 $2M(N-1)/N$）穿过链路。例：M = 1.5×10^9 参数、fp16 梯度 = 3.0 GB，则每卡收发 = $2(N-1)/N \times 3.0$ GB，N=8 时是 **5.25 GB**（N 很大时趋近上界 2M 个参数 ≈ 6.0 GB)；在 12.6 GB/s 的 PCIe

Gen3 x16 上需要 **≈ 417 ms**，而在 150 GB/s 的 NVLink 上只要 **35 ms**——**12 倍差距**。

- **延迟瓶颈（小模型/大 N 时主导）**：步数 2(N−1) 随 N 线性增长。M = 10^6（fp16 只有 2 MB）时，每步的传输时间 (M/N)/BW 可能比 α（几微秒）还小，通信完全由 2(N−1)·α 决定。
- **浮点求和的非确定性**：ring 的累加顺序随 N 变化，因此**换卡数就会改变数值结果的最低几位**——校验时只能用容差而不是 `==`（这也解释了两个版本文档里 `gerr` 只打印数量级）。
- **无伪共享问题**：这个程序里没有任何共享缓存行（跨机通信），但阶段 1 的 `dst[i] +=` 循环值得注意——它是**流式访问**，其性能上限是内存带宽而不是算力（M/N 片 ⩾ 缓存时）。

3.2 示例二：OpenMP 小批量 SGD —— "逐样本外积累加" vs "batch 维 GEMM"

- **代码**

```cpp
// mlp_sgd.cpp —— 两层 MLP 的小批量 SGD：对比两种权重梯度算法
//   A) 逐样本反向传播 + 每线程私有梯度缓冲 + critical 归约（直观、但访存爆炸）
//   B) 先算完整批激活，再用"batch 维 GEMM"算 dW = G^T·X（分块复用、算术强度高）
// 编译: g++ -O3 -fopenmp -march=native -std=c++17 mlp_sgd.cpp -o mlp_sgd
// 运行: OMP_NUM_THREADS=16 ./mlp_sgd 256 784 2048 10 5
//       参数: B(batch) D0(输入维) D1(隐层) D2(类别数) 迭代次数
#include <omp.h>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <vector>
#include <algorithm>
#include <chrono>

using Clock = std::chrono::steady_clock;
static double sec_since(Clock::time_point t0) {
    return std::chrono::duration<double>(Clock::now() - t0).count();
}

/* ================ 版本 A：逐样本反向 + 线程私有梯度 + critical 归约 ================ */
static void grads_per_sample(const std::vector<float>& X, const std::vector<float>& Y,
                             const std::vector<float>& W1, const std::vector<float>& b1,
                             const std::vector<float>& W2, const std::vector<float>& b2,
                             int B, int D0, int D1, int D2,
                             std::vector<float>& dW1, std::vector<float>& db1,
                             std::vector<float>& dW2, std::vector<float>& db2)
{
    std::fill(dW1.begin(), dW1.end(), 0.f);
    std::fill(db1.begin(), db1.end(), 0.f);
    std::fill(dW2.begin(), dW2.end(), 0.f);
    std::fill(db2.begin(), db2.end(), 0.f);

    #pragma omp parallel
    {
        // 每个线程一份"完整的"梯度副本：这就是数据并行里"每卡一份全量"的同构问题
        std::vector<float> lW1((size_t)D1 * D0, 0.f), lW2((size_t)D2 * D1, 0.f);
        std::vector<float> lb1(D1, 0.f), lb2(D2, 0.f);
        std::vector<float> h(D1), dlog(D2);

        #pragma omp for schedule(static)
        for (int n = 0; n < B; n++) {                       // ← 样本维：天然并行、无需同步
            const float* x = &X[(size_t)n * D0];
            /* ---- 前向：h = relu(W1 x + b1) ---- */
            for (int j = 0; j < D1; j++) {
                float s = b1[j];
                const float* w = &W1[(size_t)j * D0];
                for (int i = 0; i < D0; i++) s += w[i] * x[i];
                h[j] = s > 0.f ? s : 0.f;
            }
            /* ---- 输出层 + softmax 交叉熵：dlogits = p - onehot(y) ---- */
            float mx = -1e30f, sum = 0.f;
            for (int c = 0; c < D2; c++) {
                float s = b2[c];
                const float* w = &W2[(size_t)c * D1];
                for (int j = 0; j < D1; j++) s += w[j] * h[j];
                dlog[c] = s;
                if (s > mx) mx = s;
            }
            for (int c = 0; c < D2; c++) { dlog[c] = expf(dlog[c] - mx); sum += dlog[c]; }
            for (int c = 0; c < D2; c++) dlog[c] /= sum;
            dlog[(int)Y[n]] -= 1.0f;
            /* ---- 反向：外积累加 dW2 += dlog ⊗ h，dW1 += g ⊗ x ---- */
            for (int c = 0; c < D2; c++) {
                const float g = dlog[c];
                lb2[c] += g;
                float* lw = &lW2[(size_t)c * D1];
                for (int j = 0; j < D1; j++) lw[j] += g * h[j];
            }
            for (int j = 0; j < D1; j++) {
                float g = 0.f;
```

```
                for (int c = 0; c < D2; c++) g += W2[(size_t)c * D1 + j] * dlog[c];
                g = (h[j] > 0.f) ? g : 0.f;                  // relu 的导数
                lb1[j] += g;
                float* lw = &lW1[(size_t)j * D0];
                for (int i = 0; i < D0; i++) lw[i] += g * x[i];
            }
        }
        /* ---- 合并：这就是节点内的 "AllReduce"，代价是 critical 串行区 ---- */
        #pragma omp critical
        {
            for (size_t i = 0; i < dW1.size(); i++) dW1[i] += lW1[i];
            for (size_t i = 0; i < dW2.size(); i++) dW2[i] += lW2[i];
            for (int j = 0; j < D1; j++) db1[j] += lb1[j];
            for (int c = 0; c < D2; c++) db2[c] += lb2[c];
        }
    }
}

/* ================= 版本 B：batch 维 GEMM（先存激活，再分块算 dW）================= */
static void grads_batched(const std::vector<float>& X, const std::vector<float>& Y,
                          const std::vector<float>& W1, const std::vector<float>& b1,
                          const std::vector<float>& W2, const std::vector<float>& b2,
                          int B, int D0, int D1, int D2,
                          std::vector<float>& dW1, std::vector<float>& db1,
                          std::vector<float>& dW2, std::vector<float>& db2)
{
    std::vector<float> H((size_t)B * D1), G1((size_t)B * D1), G2((size_t)B * D2);

    #pragma omp parallel for schedule(static)
    for (int n = 0; n < B; n++) {                        // 前向：batch 维并行
        const float* x = &X[(size_t)n * D0];
        for (int j = 0; j < D1; j++) {
            float s = b1[j];
            const float* w = &W1[(size_t)j * D0];
            for (int i = 0; i < D0; i++) s += w[i] * x[i];
            H[(size_t)n * D1 + j] = s > 0.f ? s : 0.f;   // 激活必须留到反向用（内存换复用）
        }
        float mx = -1e30f, sum = 0.f;
        for (int c = 0; c < D2; c++) {
            float s = b2[c];
            const float* w = &W2[(size_t)c * D1];
            for (int j = 0; j < D1; j++) s += w[j] * H[(size_t)n * D1 + j];
            G2[(size_t)n * D2 + c] = s;
            if (s > mx) mx = s;
        }
        for (int c = 0; c < D2; c++) { G2[(size_t)n*D2+c] = expf(G2[(size_t)n*D2+c] - mx); sum += G2[(size_t)n*D2+c]; }
        for (int c = 0; c < D2; c++) G2[(size_t)n * D2 + c] /= sum;
        G2[(size_t)n * D2 + (int)Y[n]] -= 1.0f;
        for (int j = 0; j < D1; j++) {
            float g = 0.f;
            for (int c = 0; c < D2; c++) g += W2[(size_t)c * D1 + j] * G2[(size_t)n * D2 + c];
            G1[(size_t)n * D1 + j] = (H[(size_t)n * D1 + j] > 0.f) ? g : 0.f;
        }
    }

    std::fill(dW1.begin(), dW1.end(), 0.f);
    std::fill(db1.begin(), db1.end(), 0.f);
    std::fill(dW2.begin(), dW2.end(), 0.f);
    std::fill(db2.begin(), db2.end(), 0.f);

    /* dW1 = G1^T · X：并行在"权重行"上（每线程独占若干行 → 无竞争、无伪共享），
       行内以 batch 为累加轴 → X 的每一行被 D1 个输出通道复用（复用率 = D1） */
    #pragma omp parallel for schedule(static)
    for (int j = 0; j < D1; j++) {
        float* row = &dW1[(size_t)j * D0];
        float  bsum = 0.f;
        for (int n = 0; n < B; n++) {
            const float  g = G1[(size_t)n * D1 + j];
            const float* x = &X[(size_t)n * D0];
            bsum += g;
            for (int i = 0; i < D0; i++) row[i] += g * x[i];
        }
```

```
        db1[j] = bsum;
    }
    /* dW2 = G2^T · H */
    #pragma omp parallel for schedule(static)
    for (int c = 0; c < D2; c++) {
        float* row = &dW2[(size_t)c * D1];
        float  bsum = 0.f;
        for (int n = 0; n < B; n++) {
            const float  g = G2[(size_t)n * D2 + c];
            const float* h = &H[(size_t)n * D1];
            bsum += g;
            for (int j = 0; j < D1; j++) row[j] += g * h[j];
        }
        db2[c] = bsum;
    }
}

int main(int argc, char** argv)
{
    const int B  = (argc > 1) ? atoi(argv[1]) : 256;
    const int D0 = (argc > 2) ? atoi(argv[2]) : 784;
    const int D1 = (argc > 3) ? atoi(argv[3]) : 2048;
    const int D2 = (argc > 4) ? atoi(argv[4]) : 10;
    const int IT = (argc > 5) ? atoi(argv[5]) : 5;

    std::vector<float> X((size_t)B*D0), Y(B), W1((size_t)D1*D0), b1(D1, 0.f),
                       W2((size_t)D2*D1), b2(D2, 0.f);
    srand(1234);
    auto rnd = []{ return (float)rand() / RAND_MAX - 0.5f; };
    for (auto& v : X)  v = rnd();
    for (auto& v : W1) v = rnd() * 0.05f;
    for (auto& v : W2) v = rnd() * 0.05f;
    for (int n = 0; n < B; n++) Y[n] = (float)(rand() % D2);

    std::vector<float> dW1a((size_t)D1*D0), db1a(D1), dW2a((size_t)D2*D1), db2a(D2);
    std::vector<float> dW1b((size_t)D1*D0), db1b(D1), dW2b((size_t)D2*D1), db2b(D2);

    grads_per_sample(X, Y, W1, b1, W2, b2, B, D0, D1, D2, dW1a, db1a, dW2a, db2a);
    grads_batched   (X, Y, W1, b1, W2, b2, B, D0, D1, D2, dW1b, db1b, dW2b, db2b);

    double err = 0.0;                       // 两版本必须算出同一个梯度（容差比较）
    for (size_t i = 0; i < dW1a.size(); i++) err = std::max(err, (double)fabsf(dW1a[i] - dW1b[i]));

    auto t0 = Clock::now();
    for (int it = 0; it < IT; it++) grads_per_sample(X, Y, W1, b1, W2, b2, B, D0, D1, D2, dW1a, db1a, dW2a, db2a);
    const double sA = sec_since(t0) / IT;

    auto t1 = Clock::now();
    for (int it = 0; it < IT; it++) grads_batched(X, Y, W1, b1, W2, b2, B, D0, D1, D2, dW1b, db1b, dW2b, db2b);
    const double sB = sec_since(t1) / IT;

    const double flops = 6.0 * B * (double)D0 * D1;          // 前向 2 + 反向 4（按主导层估）
    const double w1b   = (double)D1 * D0 * 4;                // dW1 的字节数
    const double trA   = (double)B * (w1b + 2 * w1b);        // 版本 A：读 W1 + 读写 dW1
    const double trB   = ((double)B*D0 + 2.0*B*D1 + (double)D1*D0) * 4;  // 版本 B

    printf("线程数=%d  B=%d D0=%d D1=%d D2=%d\n", omp_get_max_threads(), B, D0, D1, D2);
    printf("版本 A（逐样本外积累加）: %8.2f ms  %7.1f GFLOP/s  估算 DRAM 流量 %6.2f GB  AI≈%.2f FLOP/byte\n",
           sA * 1e3, flops / sA / 1e9, trA / 1e9, flops / trA);
    printf("版本 B（batch 维 GEMM）  : %8.2f ms  %7.1f GFLOP/s  估算 DRAM 流量 %6.2f GB  AI≈%.1f FLOP/byte\n",
           sB * 1e3, flops / sB / 1e9, trB / 1e9, flops / trB);
    printf("实测加速比 = %.1fx ；两版本 dW1 最大差异 = %.3e\n", sA / sB, err);
    printf("每卡梯度缓冲大小 = %.2f MB（版本 A 每线程一份 → %d 线程共 %.1f MB）\n",
           w1b / 1e6, omp_get_max_threads(), w1b * omp_get_max_threads() / 1e6);
    return 0;
}
```

- **【代码做什么？】**

- **数据准备**：随机生成一批输入 `X[B×D0]`、标签 `Y[B]`、两层权重 `W1[D1×D0]`、`W2[D2×D1]`，以及两套输出缓冲 `dW1a/dW1b`，用同一份数据喂给两个版本，最后比较两者的 `dW1`。
- **版本 A（`grads_per_sample`）**：`#pragma omp parallel` 开并行区，**每个线程先分配自己的一整套梯度缓冲**（`lW1` 就是 D1×D0 = 160 万个 float = 6.42 MB），然后 `#pragma omp for schedule(static)` 把 **B 个样本**平均分给线程；每个样本内部串行完成"前向 → softmax 交叉熵梯度 → 两层外积累加"。所有线程跑完后，在一个 `critical` 段里把各自的私有缓冲加进全局梯度。
- **版本 B（`grads_batched`）**：第一步先把**整批的隐层激活 H 和上游梯度 G1/G2 全部算出来存下**（这就是"激活内存"的来源）；第二步把梯度计算重写成两个 **GEMM**：`dW1 = G1ᵀ·X`、`dW2 = G2ᵀ·H`。并行化方向从"样本"改成了"**权重行**"：每个线程独占若干行 dW1（无写冲突），**行内以 batch 为累加轴**——于是 X 的同一行会被 D1 个输出通道反复读取，且这些读取都命中缓存。
- **收尾**：校验两个版本结果一致（只允许浮点误差），分别计时并打印 GFLOP/s、估算 DRAM 流量与算术强度。
- **【并行机制与性能解说】**
- **线程/向量通道如何创建与分配**：OpenMP 起 16 个线程，`schedule(static)` 把 B=256 个样本切成 16 块（每线程 16 个样本）。编译器的自动向量化会把内层 `for i < D0` 变成 AVX2/AVX-512 的 `vfmadd`：以 AVX2（8 宽 FMA）为例，每个时钟节拍每核可完成 8 次乘加 = 16 FLOP；16 核 × 3.0 GHz × 8 宽 × 2 = **768 GFLOP/s** 的峰值就是 4.1 节数值算例里用的那个数。
- **共享数据如何处理**：`W1/W2/b1/b2/X/Y` 是**只读共享**（无需同步）；`dW1/dW2` 是**写共享**——版本 A 靠"每线程一份私有副本 + critical 合并"避免竞争，版本 B 靠"**行所有权（row ownership）**"避免竞争。**注意版本 B 的行所有权顺带消灭了伪共享**：每个线程独占整行（D0=784 个 float = 3136 字节 = 49 条 cache line），相邻线程不会写同一 cache line。反之，如果改成"每个线程负责若干列"，那么每次写 `dW1[j][i]` 都会和邻居共享 cache line → cache line ping-pong，性能可能掉一个数量级。
- **Work / Span / 并行度**：
  - **Work**（一次迭代的浮点运算总数）≈ `2·B·(D0·D1 + D1·D2)`（前向）+ `4·B·(D0·D1 + D1·D2)`（反向）≈ **6·B·D0·D1 = 6 × 256 × 784 × 2048 ≈ 2.47 GFLOP**（D2=10 的贡献可忽略）；两个版本 Work **完全相同**。
  - **Span（关键路径）**：样本之间互相独立 → 关键路径就是"**单个样本的前向+反向**"的串行长度 ≈ `6·D0·D1` FLOP，再加上版本 A 末尾那次 `critical` 归约的串行时间（**合并 6.42 MB 的全局梯度**，这是 Span 里被硬生生加进来的一段）以及 OpenMP 的隐式 barrier。版本 B 的 Span = 单行 dW1 的累加链（B 次相关的外积累加）+ 两次并行 for 之间的 barrier。
  - **并行度 = Work / Span**：版本 A ≈ B（=256，样本数）——线程再多也没用，**B 就是上限**；版本 B ≈ `D1 × (B/单行长度)`，此处 ≈ 2048 行 × 16 线程 = 32768 个独立任务，**并行度远高于 16 个线程**，因此 16 核能被喂满。**结论：算法改写把可用并行度从 256 提到了 3 万，这才是版本 B 快的根本原因之一；另一个原因（更主要）是算术强度。**
- **瓶颈分析（这才是重点）**：
  - **版本 A 是彻底的访存瓶颈**。它每个样本都要把整块 dW1（6.42 MB）**读一遍、写一遍**。16 个线程各持一份私有副本 → 工作集 16 × 12.84 MB ≈ **205 MB，远超 L3**，所以流量真实地落在 DRAM 上：`256 样本 × 19.27 MB ≈ 4.93 GB`，算术强度只有 `2.47e9/4.93e9 ≈ 0.5 FLOP/byte`（只看 dW1 的外积更新则是 `2 FLOP / 8 byte = 0.25 FLOP/byte`）。按 20 GB/s 的单路内存带宽，**模型预测耗时 ≈ 4.93 GB / 20 GB/s ≈ 247 ms**——而这点计算量在 768 GFLOP/s 下只需 3.2 ms，**差了近 77 倍**。
  - **版本 B 把算术强度抬了 400 倍**：DRAM 流量降到 ≈ 11.5 MB（X 803 KB + H 2.1 MB + G1 2.1 MB + dW1 6.42 MB），`AI ≈ 214 FLOP/byte`，于是耗时由计算决定：**模型预测 ≈ max(2.47 GFLOP / 768 GFLOP/s, 11.5 MB / 20 GB/s) ≈ max(3.21 ms, 0.58 ms) ≈ 3.2 ms**。两者的模型预测加速比 ≈ **77×**。
  - **这正是 CS149 讲义的核心结论在训练侧的复现**：CS149 slide 9–10 用"循环融合把算术强度从 1/3 提到 3/5"来说明"**提高算术强度让程序更可能变成计算受限**"；slide 35 的分块 GEMM 与 slide 42–43 的 implicit GEMM 是同一招。**训练里的"batch 维 GEMM"就是把 batch 融合进循环、让每个激活值被 D1 个权重复用**——同一个优化，同一个收益。
  - **另一个隐性代价**：版本 B 要多存 `H`、`G1`、`G2`（此例 2.1+2.1+0.1 MB，很小；但对 Transformer 长序列会爆炸）。这就是"**激活内存 vs 重算（recomputation / checkpointing）**"的取舍，也是 2.4 节"内存账本未计入 activations"那句话的真正含义。

3.3 示例三：CUDA 分块梯度归约 —— 设备内的"层次化 AllReduce"

- **代码**

```
// grad_partial.cu —— 权重梯度：按参数列并行 + split-K 部分和 + 块内树形归约
// 编译: nvcc -O3 -arch=sm_70 grad_partial.cu -o grad_partial
// 运行: ./grad_partial 4096 2048 4
//       参数: B(batch) D(特征维) C(batch 分块数 = split-K 度)
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <algorithm>

#define CUDA_CHECK(x) do { cudaError_t e_ = (x); if (e_ != cudaSuccess) {                \
    printf("CUDA error %s at %s:%d\n", cudaGetErrorString(e_), __FILE__, __LINE__); exit(1);} } while (0)

// ---- 第一阶段：每个 block 负责一批参数列（threadIdx.x 连续 → 合并访存），
//      且只处理 batch 的一个分块（blockIdx.y = split-K 序号），写出部分和 ----
__global__ void grad_partial_kernel(const float* __restrict__ X,   // [B x D]
                                    const float* __restrict__ G,   // [B x D] 上游梯度
                                    float* __restrict__ part,      // [C x D] 部分和
                                    int B, int D, int chunk)
{
    const int col = blockIdx.x * blockDim.x + threadIdx.x;   // 参数列 (= 权重列)
    if (col >= D) return;
    const int c  = blockIdx.y;                               // batch 分块序号
    const int n0 = c * chunk;
    const int n1 = min(n0 + chunk, B);

    float acc = 0.f;
    for (int n = n0; n < n1; n++)                            // 串行累加 batch 维
        acc += X[(size_t)n * D + col] * G[(size_t)n * D + col];
    part[(size_t)c * D + col] = acc;
}

// ---- 第二阶段：一个 block 负责一列，把 C 个部分和在共享内存里做树形归约 ----
//      这就是"设备内版本的 allreduce 层次"：块内 log2(blockDim) 步树形归约
__global__ void reduce_partial_kernel(const float* __restrict__ part,
                                      float* __restrict__ dW, int D, int C)
{
    extern __shared__ float s[];
    const int col = blockIdx.x;
    float acc = 0.f;
    for (int c = threadIdx.x; c < C; c += blockDim.x)        // 线程沿 C 维分工
        acc += part[(size_t)c * D + col];
    s[threadIdx.x] = acc;
    __syncthreads();
    for (int off = blockDim.x >> 1; off > 0; off >>= 1) {    // log2(blockDim) 步树形归约
        if (threadIdx.x < off) s[threadIdx.x] += s[threadIdx.x + off];
        __syncthreads();
    }
    if (threadIdx.x == 0) dW[col] = s[0];                    // 每列只写一次，无需原子操作
}

int main(int argc, char** argv)
{
    const int B = (argc > 1) ? atoi(argv[1]) : 4096;
    const int D = (argc > 2) ? atoi(argv[2]) : 2048;
    const int C = (argc > 3) ? atoi(argv[3]) : 4;
    const int chunk = (B + C - 1) / C;

    std::vector<float> hX((size_t)B*D), hG((size_t)B*D);
    for (size_t i = 0; i < hX.size(); i++) {
        hX[i] = (float)((i * 2654435761u) % 1000) / 1000.f - 0.5f;
        hG[i] = (float)((i * 40503u + 17) % 1000) / 1000.f - 0.5f;
    }

    float *dX, *dG, *dPart, *dW;
    CUDA_CHECK(cudaMalloc(&dX,    (size_t)B*D*sizeof(float)));
    CUDA_CHECK(cudaMalloc(&dG,    (size_t)B*D*sizeof(float)));
    CUDA_CHECK(cudaMalloc(&dPart, (size_t)C*D*sizeof(float)));
    CUDA_CHECK(cudaMalloc(&dW,    (size_t)D*sizeof(float)));
    CUDA_CHECK(cudaMemcpy(dX, hX.data(), (size_t)B*D*sizeof(float), cudaMemcpyHostToDevice));
```

```
    CUDA_CHECK(cudaMemcpy(dG, hG.data(), (size_t)B*D*sizeof(float), cudaMemcpyHostToDevice));

    const int TPB = 256;
    dim3 gridP((D + TPB - 1) / TPB, C);

    cudaEvent_t t0, t1;
    cudaEventCreate(&t0); cudaEventCreate(&t1);
    CUDA_CHECK(cudaDeviceSynchronize());
    cudaEventRecord(t0);
    for (int it = 0; it < 20; it++) {
        grad_partial_kernel<<<gridP, TPB>>>(dX, dG, dPart, B, D, chunk);
        reduce_partial_kernel<<<D, 128, 128 * sizeof(float)>>>(dPart, dW, D, C);
    }
    cudaEventRecord(t1);
    CUDA_CHECK(cudaEventSynchronize(t1));
    float ms = 0.f;
    cudaEventElapsedTime(&ms, t0, t1);
    ms /= 20.f;

    double ref = 0.0;                                   // CPU 参考：某一列的完整归约
    for (int n = 0; n < B; n++) ref += (double)hX[(size_t)n*D + 7] * (double)hG[(size_t)n*D + 7];
    float got = 0.f;
    CUDA_CHECK(cudaMemcpy(&got, dW + 7, sizeof(float), cudaMemcpyDeviceToHost));

    const double flops = 2.0 * B * (double)D;
    const double bytes = 2.0 * B * (double)D * 4.0;     // 读 X 与 G 各一次
    printf("B=%d D=%d splitK=%d\n", B, D, C);
    printf("时间 = %.4f ms ； 算力 = %.1f GFLOP/s ； 读入 %.2f MB/iter ； 有效带宽 = %.1f GB/s\n",
           ms, flops / (ms * 1e-3) / 1e9, bytes / 1e6, bytes / (ms * 1e-3) / 1e9);
    printf("dW[7]: GPU = %.6f  CPU = %.6f  误差 = %.3e\n", got, ref, fabs(got - ref));
    printf("算术强度 = %.3f FLOP/byte（V100 峰值 15.7 TFLOP/s，ridge 点 ≈ 17.4 FLOP/byte）\n",
           flops / bytes);

    cudaFree(dX); cudaFree(dG); cudaFree(dPart); cudaFree(dW);
    return 0;
}
```

- **【代码做什么？】**
- **第一阶段（`grad_partial_kernel`）**：grid 是二维的——`blockIdx.x` 覆盖参数列（每 block 256 列，线程内连续 → `X[n*D+col]` 与 `G[n*D+col]` 对同一个 n、相邻 col 的访问是**完全合并**的），`blockIdx.y` 把 batch 切成 C 份（**split-K**）。每个线程串行累加自己那段 batch，写出一份部分和 `part[c][col]`。
- **第二阶段（`reduce_partial_kernel`）**：一个 block 负责一列，128 个线程沿 C 维分工（每个线程累加若干段），把结果放进**共享内存**，然后用 `for (off = 64, 32, ...)` 做 **$\log_2(128) = 7$ 步树形归约**，最后 `threadIdx.x == 0` 把结果写回 `dW[col]`。
- **校验与计时**：用 CPU 重算第 7 列的内积做参考；用 CUDA event 计时 20 次取平均，并打印算力、有效带宽与算术强度。
- **【并行机制与性能解说】**
- **warp 与 SIMT 执行**：256 线程的 block 分成 **8 个 warp**，每个 warp 的 32 个线程访问连续的 32 个 float = 128 字节，恰好是**一次 128 B 的合并事务**（4 个 sector）。`for (n = n0; n < n1; n++)` 是串行的、每次迭代一次 load，属于 **high-latency low-ILP** 的模式——靠**同时驻留的大量 warp 做延迟隐藏**（这就是 GPU 里 "occupancy 换延迟隐藏"的用法）。block 内没有线程间通信（除了第二阶段的树形归约），因此**没有任何共享内存/原子争用**。
- **共享数据如何处理**：第一阶段完全无共享（每个线程写自己的一列的独立位置）；第二阶段是**层次化归约**：线程内串行累加 → 块内共享内存树形归约 → 全局一次写入。**这就是数据并行 allreduce 的微缩版**：块内的树形归约 ≈ 节点内的 NVLink ring，块间的部分和 ≈ 节点间 allreduce，`splitK` 就是"用更多并发度换更长的累加链"。
- **Work / Span / 并行度**：
  - **Work** = B×D 次乘加 = $2 \cdot B \cdot D$ FLOP（此例 B=4096, D=2048 → 16.8 MFLOP）；
  - **Span** = 单列的累加链长度 `nchunk = B/C = 1024` 次相关乘加 + 第二阶段的 7 步树形归约 + 一次同步；
  - **并行度** = Work/Span ≈ `(B·D)/(B/C + log₂(128))` ≈ **$C \cdot D = 8192$** 个独立累加任务 —— 远超 V100 的 80 个 SM × 2048 线程，**并行度绝对充足，问题从来不是并行度**。
- **瓶颈分析（本例最大的教学价值）**：
  - 这个 kernel 的算术强度是 `2·B·D / (2·B·D·4) = **0.25 FLOP/byte**`——因为**每个 X、G 的元素只被使用一次**，没有任何复用。

- 在 V100 上（900 GB/s HBM、15.7 TFLOP/s FP32），它最多只能跑到 `900 GB/s × 0.25 = **225 GFLOP/s**，即峰值的 **1.4%**`。实测会非常接近这个带宽上限（67.1 MB / 900 GB/s ≈ 74.6 μs），CPU 上算这点活反而是浪费。
- **修法不是"加并行度"，而是"提高复用"**：把梯度写成 GEMM（`dW = GᵀX`），让**每个 X 元素被 D 个权重列复用、每个 dW 元素在寄存器里累积**，算术强度就从 0.25 升到 `D·B/(4B+2D) = 2048×4096/(16384+4096) ≈ **410 FLOP/byte**`。这正是示例二版本 B 做的事，也是 cuDNN/CUTLASS 把卷积与注意力全部转成"分块 GEMM"的原因（CS149 slide 33–43：explicit GEMM 的 DRAM 流量会因 im2col 放大 R×S 倍，所以工业界改用 **implicit GEMM——只把卷积矩阵的一个子块物化到共享内存**，用分块 GEMM 例程吃掉它）。
- **另一个隐藏成本**：`part[C×D]` 的写回与第二阶段的重读，让 X/G 之外又多了一趟显存往返（C×D×4 bytes ×2）。当 split-K 度 C 很大时（比如为了填满 SMs 而把 batch 切得很碎），这部分开销会显著增加——这是"**并行度换通信量**"的又一个实例。

---

4. 性能模型与复杂度分析

4.1 训练迭代的 Work、Span 与 Roofline

一次数据并行迭代的时间可以写成三项之和：

$$T_{\text{iter}}(N) = \underbrace{\frac{W_{\text{compute}}}{N \cdot \Pi}}_{\text{算力分摊}} + \underbrace{T_{\text{comm}}(N)}_{\text{与 } N \text{ 近乎无关}} + \underbrace{T_{\text{sync}}(\alpha,\, N)}_{\text{步数×启动延迟}}$$

- **Work**：整个 iteration 的浮点运算量。对 Transformer 类模型有广泛使用的经验式 $W \approx 6PT$（P = 参数量，T = 本次迭代处理的 token 数；前向 2PT + 反向 4PT）。
- **Span**：关键路径 = "单层前向 → 单层反向 → 权重更新"的串行链，长度与**层数 L** 成正比，而与 N 无关。因此**数据并行把 Work 分摊了，却没有缩短 Span**——这正是它不像"算法并行"那样能无限扩展的原因。
- **并行度** = Work / Span ≈ `6PT / (c·L)` ≈ **P 量级**（参数级）与 **B 量级**（样本级）。两个量都极大，所以数据并行的限制**从来不是并行度，而是通信**。
- **Roofline 视角**（CS149 slide 6–8 的模型）：设峰值算力 Π、内存带宽 BW，则可达吞吐 = $\min(\Pi,\ AI \times BW)$，拐点（ridge point）在 $AI^{*} = \Pi / BW$。用 V100 的 FP16 数据：$AI^{*} = 125\ \text{TFLOP/s} / 900\ \text{GB/s} \approx 139$ FLOP/byte。
- **图 9：Roofline 上的几个关键工作点（横轴 log 刻度）**

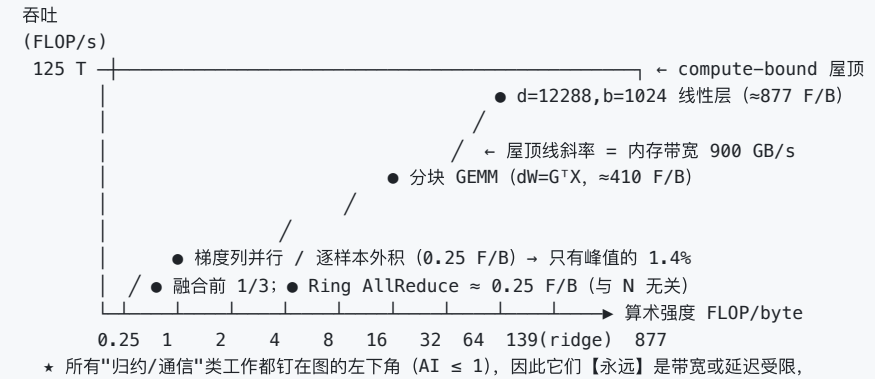


表 4：本讲涉及的各阶段算术强度与受限类型（数值均为可复核的推导结果）

工作阶段	数据复用方式	算术强度（FLOP/byte）	V100 上的受限类型	优化手段
逐样本权重外积累加（示例二 A、示例三）	无复用，每次读改写整块 dW	0.25	严重带宽受限（~1.4% 峰值）	改成 batch 维 GEMM
AllReduce 梯度（任意 N，fp16）	每个元素只被加一次	**0.25（与 N 无关）**	带宽/延迟受限	减字节、减步数、换拓扑
分块 GEMM 算 dW（示例二 B）	X 被 D1 行复用	~214（本例）	计算受限	加 cache 分块、提 tile 尺寸
大 batch 线性层（d=12288, b=1024）	权重被整批复用	~877	计算受限（Tensor Core）	提 batch、混合精度
大 batch 线性层（d=12288, **b=1**）	只复用一次	~1	权重带宽受限	攒 batch / 权重驻留

> 线性层 AI 的推导（可直接代数字）：`AI = 2·b·d² / (2·d² + 4·b·d) = b·d/(d + 2b)`。代入 b=1024, d=12288 得 877；代入 b=1 得 ≈1.0。**这是"batch 越大越划算"的定量解释，也是数据并行本身能存在的前提**——正是大 batch 把训练推进了计算受限区。

4.2 强扩展（strong scaling）与弱扩展（weak scaling）

数据并行的两种扩展方式在时间模型上差别巨大，值得单独列出：

表 5：强扩展 vs 弱扩展（T_c = 单卡跑完整 batch 的计算时间，T_{comm} = 一次 AllReduce 时间）

维度	强扩展（固定全局 batch）	弱扩展（固定每卡 batch）
每卡计算量	T_c/N（随卡数下降）	T_c（恒定）
通信量	T_{comm}（恒定，与 N 无关）	T_{comm}（恒定）
迭代时间	$T_c/N + T_{\text{comm}}$	$T_c + T_{\text{comm}}$（恒定）
加速比 $S(N)$	$1/(1/N + r)$，其中 $r = T_{\text{comm}}/T_c$	N（吞吐线性增长，效率恒定）
加速比上限	$1/r = T_c/T_{\text{comm}}$	无通信上限；但受**收敛性**限制（batch 太大时每步收益递减）
失效模式	卡加到某个数量后**完全无收益**	需要同步放大学习率；过大 batch 会浪费算力
适用场景	想**更快**得到结果（时间受限）	想**训得更大**（问题规模受限）

> 注意区分两种"Amdahl"：经典 Amdahl 律管的是**串行部分**（这部分占比固定）；数据并行里的通信是**常数开销**（不随 N 缩小），得到的规律是"加速比天花板 = 计算时间/常数开销"。两者形式相似，来源不同，不要混用。

4.3 数值算例一：GPT-2 1.5B 在 8 张 V100 上的数据并行

假设（全部显式给出，便于复核）：M = 1.5×10^9 参数（对应讲义表里的 GPT-2 量级）；混合精度梯度 = **2 B/参数**；每卡 batch = 1024 个 token，全局 batch = 8×1024 = **8192 token**；每迭代算力 $W = 6PT = 6 \times 1.5 \times 10^9 \times 8192 = 7.37 \times 10^{13}$ FLOP = **73.7 TFLOP**；单卡实测算力按峰值 125 TFLOP/s 的 **40% MFU** 取 **50 TFLOP/s**；环上 AllReduce。

1. **梯度缓冲**：$1.5 \times 10^9 \times 2\,\text{B} = 3.0$ GB。
2. **每卡 Ring AllReduce 通信量**：$2(N-1)/N \times 3.0\ \text{GB} = 2 \times 7/8 \times 3.0 = 5.25$ GB（≈ 2M 个参数，**与 N 无关**）。
3. **每卡计算时间**：$6PT/N = 73.7/8 = 9.22$ TFLOP，除以 50 TFLOP/s = **184 ms**。
4. **通信时间**（不同互连）：

表 6：同一个 1.5B 模型、同一个 8 卡作业，换一条链路就差一个数量级

互连	单向有效带宽	AllReduce 时间	迭代总时间（184 ms + 通信）	相对单卡加速比	加速比天花板 $1/r$
NVLink 2.0（卡间）	150 GB/s	**35 ms**	219 ms	**6.70×**（84% 效率）	42×
400 Gb/s InfiniBand	50 GB/s	105 ms	289 ms	5.08×	14×
PCIe Gen3 x16	12.6 GB/s	**417 ms**	601 ms	**2.45×**（31% 效率）	3.5×
100 Gb/s Ethernet	12.5 GB/s	420 ms	604 ms	2.43×	3.5×
40 Gb/s Ethernet	5 GB/s	**1050 ms**	1234 ms	**1.19×**（15% 效率）	1.4×

> **读数**：单卡跑完整 batch 需要 73.7 TFLOP / 50 TFLOP/s = **1.47 s**。用 8 张 V100 走 NVLink，加速 6.7×；走 PCIe 或 100 GbE，只有 2.45×——**8 张卡里有一半的算力被浪费在等梯度**。走 40 GbE 时更是只快 1.19×，几乎等于没并行。**天花板 $1/r = T_c/T_{\text{comm}}$ 告诉你"这台机器最多值几张卡"**：PCIe 集群上超过 4 张卡就没有意义了。这也解释了为什么真实的分布式训练中心一定用 NVLink + InfiniBand，并把通信与反向计算重叠（ZeRO-2 的"每层反完立刻 Reduce"就是这个目的）。

4.4 数值算例二：AllReduce 四种算法的 cross-over（含可运行模型程序）

- **代码**

```c
/* allreduce_model.c —— Ring / Tree / Butterfly / Parameter Server 的通信代价模型
 * 编译: gcc -O2 -std=c11 allreduce_model.c -o allreduce_model -lm
 * 运行: ./allreduce_model 1500000000 2 5e-6 1e-10
 *       参数: M(参数量) 每参数字节数  alpha(每步启动延迟,秒)  每字节传输时间(秒) → 10 GB/s
 */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

static double naive(double M, int N, double w, double a)     { return (N - 1) * M * w + a; }
static double ps(double M, int N, double w, double a)        { return N * M * w + 2 * a; }
static double ring(double M, int N, double w, double a)      { return 2.0 * (N - 1) * (a + (M / N) * w); }
static double tree(double M, int N, double w, double a)      { return 2.0 * ceil(log2((double)N)) * (a + M * w); }
static double bfly(double M, int N, double w, double a)      { return ceil(log2((double)N)) * (a + M * w); }

int main(int argc, char **argv)
{
    const double M  = (argc > 1) ? atof(argv[1]) : 1.5e9;   /* 参数个数 */
    const double be = (argc > 2) ? atof(argv[2]) : 2.0;      /* 每参数字节数 (fp16 = 2)  */
    const double a  = (argc > 3) ? atof(argv[3]) : 5e-6;     /* 每步启动延迟 (秒)  */
    const double bw = (argc > 4) ? atof(argv[4]) : 1e-10;    /* 每字节时间 → 10 GB/s */
    const double w  = bw / be;                               /* 每个参数元素的传输时间 */

    printf("M = %.3g 个参数 (%.2f GB, 每参数 %g B); alpha = %g us; 链路 = %.1f GB/s\n\n",
           M, M * be / 1e9, be, a * 1e6, 1.0 / bw / 1e9);
    printf("%6s %12s %12s %12s %12s %12s\n",
           "N", "PS(ms)", "Naive(ms)", "Ring(ms)", "Tree(ms)", "Butterfly(ms)");
    for (int N = 2; N <= 1024; N *= 2)
        printf("%6d %12.2f %12.2f %12.2f %12.2f %12.2f\n", N,
               ps(M,N,w,a)*1e3, naive(M,N,w,a)*1e3, ring(M,N,w,a)*1e3,
               tree(M,N,w,a)*1e3, bfly(M,N,w,a)*1e3);
    return 0;
}
```

- **【代码做什么？】** 把表 1 里的五个公式直接实现成函数：`ring` 用 $2(N-1)(\alpha+(M/N)w)$（对应讲义"每步送 M/N，重复 2N 次"），`tree` 用 $2\log_2 N(\alpha+Mw)$（每步送整份 M），`bfly` 用 $\log_2 N(\alpha+Mw)$，`ps` 用 NMw（服务器串行服务 N 个 worker），`naive` 用 $(N-1)Mw$。程序扫过 N = 2…1024，打印一张延迟表。
- **【并行机制与性能解说】** 这个程序本身串行，它的价值是**把讲义 slide 25–26 的对比表变成可以自己调参的实验**：调 `be`（fp16↔fp32）、调 `bw`（NVLink↔以太网）、调 `M`（小模型↔大模型），就能看到"哪种 AllReduce 更好"**取决于工作点**。Work/Span 视角：五种算法的 **Work 完全相同**（都是把 M 个参数加起来），差别全在 **Span（步数与每步大小）**——这正是"同一份工作，不同的关键路径"的教科书例子。
- **数值算例一：大模型（M = 1.5×10⁹，fp16 = 3.0 GB，链路 10 GB/s，α = 5 μs）**

单次传输整份梯度的带宽成本 $Mw = 1.5\times10^{9}\times5\times10^{-11} = 75$ ms。

表 7：AllReduce 五种方案的延迟（ms）—— 大模型、带宽主导区间

N	Parameter Server	Naïve	Ring	Tree	Butterfly
2	150.01	75.00	**75.01**	150.01	75.00
8	600.01	525.00	**131.32**	450.03	225.02
64	4800.01	4725.00	**148.29**	900.06	450.03
1024	76800.01	76725.00	**160.08**	1500.10	750.05

> **读数**：M 很大时 **Ring 完胜**，而且从 N=8 到 N=1024 只从 131 ms 涨到 160 ms（**趋近 2Mw = 150 ms 的平台**，增量全部来自 $2(N-1)\alpha$ 的启动延迟）。这正面回答了讲义 slide 25 的问题：**Ring 的单卡通信量 ≈ 2M 与 N 无关，所以 N 增大时带宽成本不变、只有微小的步数延迟增长**；而 PS 与 Naive 的成本正比于 N（N=1024 时 76.8 s，比 ring 慢 **480 倍**）；Tree/Butterfly 因为"每步要传整份 M"，成本随 $\log N$ 增长，在这里反而吃亏。

- **数值算例二：小模型（M = 10⁶，fp16 = 2 MB，同样 10 GB/s，α = 5 μs）**

此时 $Mw = 50$ μs，**单步传输时间与启动延迟同量级**，结论完全反转：

表 8：同样五种方案的延迟（ms）—— 小模型、延迟主导区间

N	Parameter Server	Naïve	Ring	Tree	Butterfly
8	0.41	0.35	0.16	0.33	**0.17**
64	3.21	3.15	0.73	0.66	**0.33**
1024	51.21	51.15	10.33	1.10	**0.55**

> **读数**：M 小的时候 **Butterfly / Tree 反超 Ring**——N=1024 时 Butterfly 只用 **0.55 ms**（$\log_2 1024 = 10$ 步 × 55 μs），而 Ring 要 **10.33 ms**（2046 步 × 5 μs，即 **10 ms 全部是启动延迟**）。这就是 cross-over：**带宽主导时选 Ring（少传字节），延迟主导时选 Tree/Butterfly（少走轮次）**。真实的集合通信库（NCCL）因此采用**混合拓扑**：节点内用 NVLink 做 ring，节点间用树/多环，再按消息大小在算法之间切换。

4.5 数值算例三：ZeRO 到底省了多少、又贵了多少

假设：M = 175×10⁹（GPT-3 量级），每参数 20 B（混合精度训练账本），N = 64 张卡，机间 400 Gb/s InfiniBand（50 GB/s），单卡 FP16 实测 125 TFLOP/s（A100 的 40%），本次迭代处理 T = 3.2×10⁶ 个 token（大 batch）。

1. **显存**（代入表 3）：
2. 基线数据并行：**3500 GB/卡** → 完全不可能；
3. ZeRO-1：700 + 2800/64 = **743.8 GB/卡** → 不可能；
4. ZeRO-2：350 + 18×175/64 ≈ **399.2 GB/卡** → 不可能；
5. **ZeRO-3：3500/64 = 54.7 GB/卡 → A100-80GB 装得下**（还要留激活的内存）。 **结论：要把 175B 喂进 8×80 GB 的机器，必须上 ZeRO-3（或叠加张量并行/流水并行）。**
6. **通信量**：每卡每迭代 ≈ 2M 个参数（基线/ZeRO-1/2）= **700 GB**，ZeRO-3 = 3M = **1050 GB**。
7. **通信时间**：700 GB ÷ 50 GB/s = **14 s**（不重叠时）。
8. **计算时间**：$6PT/(N\cdot\Pi) = 6\times175\times10^9\times3.2\times10^6/(64\times125\times10^{12}) = 3.36\times10^{18}/8\times10^{15} = 420$ s。
9. **对比**：14 s / 420 s = **3.3%**——大 batch 下通信占比很小，训练是**计算受限**的。但若把 batch 缩小到 8192 个 token（T = 8192）：计算时间 = $6\times175\times10^9\times8192/8\times10^{15} = 1.07$ s，通信 14 s 反过来是主导（**通信占比 93%**）。
10. **cross-over batch size**：令 $6PT/(N\Pi) = V/BW$ 解得

$$T^{*} = \frac{V\cdot N\cdot\Pi}{BW\cdot 6P} = \frac{700\times10^9\times64\times125\times10^{12}}{50\times10^9\times6\times175\times10^9} \approx 1.07\times10^5 \text{ token}$$

即 **每次迭代的全局 batch 超过约 10.7 万 token 之后，通信才不再是瓶颈。 这解释了分布式训练中所有"增大 batch + 同步放大学习率"的工程实践：大 batch 不是为了更快收敛，而是为了把通信在计算后面藏起来。**（不过 batch 不能无限放大：公开文献中普遍观察到超过某个"临界 batch size"后每步的收敛增益会急剧衰减——这一点讲义没有展开，属公开领域的补充知识。）

4.6 把开销放进同一张预算表

把 4.3 节的算例拆成"随 N 下降"与"不随 N 下降"两类，就能一眼看出优化该往哪里使劲：前向 92 ms、反向 92 ms 属于**∝1/N 的可分摊项**（靠混合精度、算子融合、激活重算继续压缩）；AllReduce 梯度 35 ms（NVLink）/ 417 ms（PCIe）与 2(N−1)α 的同步延迟属于**恒定项**（只能靠 fp16/bf16、梯度压缩、ZeRO-2 分桶重叠、合并小消息来压缩）；Adam 更新在单卡上数值极小可忽略，但它是 ZeRO-1 分片的直接受益者。**加速比天花板的本质就是"恒定项 / 可分摊项"**：把 417 ms 压到 35 ms，天花板就从 3.5× 抬到 42×。

---

5. 关键要点

1. **数据并行的可并行性来自 SGD 求和号里的"样本维"，代价是"参数维必须整体同步"。** 前向/反向的计算量随卡数 N 线性分摊，而梯度聚合的通信量 ≈ 2M 个参数、**与 N 几乎无关**——于是加速比存在天花板 $1/r = T_c/T_{\text{comm}}$（本讲算例：NVLink 42×、PCIe 仅 3.5×）。**判断一个集群值不值得加卡，先算这个天花板。**
2. **AllReduce 的四种算法是"通信量 vs 轮数"的连续权衡，没有全局最优。** Ring 用"每卡只搬 2M（与 N 无关）+ 2(N−1) 步"取得最佳可扩展性（N=1024 时仍只需 2Mw）；Tree/Butterfly 把轮数压到 $2\log N$ / $\log N$，代价是每步搬整份 M，因此在**小模型 + 大 N** 时才反超（N=1024、M=10⁶ 时 Butterfly 0.55 ms vs Ring 10.33 ms）。**选算法要先判断工作点落在带宽主导区还是延迟主导区。**
3. **"去中心化"是扩展性的前提。** 参数服务器的中心带宽随 N 线性劣化（MN/BW），Naive AllReduce 是 O(N²) 通信量——两者在 N=1024 时比 Ring 慢两个数量级以上。**任何"所有节点都去访问同一个东西"的设计都会在规模化时崩掉，这是并行系统设计的通用准则。**

4. **大模型训练的瓶颈首先不是算力而是"内存账本"，其次是"传输字节数"。** 混合精度训练要 20 B/参数（FP16 参数 + FP16 梯度 + 16 B FP32 优化器状态），而且**不含激活**；ZeRO 通过把这份账本切成 1/N 换来显存，其中 **Stage 1/2 的通信量与基线完全相同（免费省内存），只有 Stage 3 需要多付 50%**。所有通信优化的正确方向是"**少搬字节**"（fp16/bf16、压缩、分片），而不是"少算 FLOP"——因为归约类工作的算术强度只有 **0.25 FLOP/byte 且与 N 无关**（每个元素搬进搬出共 4 字节、只摊到 1 次加法），在 Roofline 上永远钉在左下角。
5. **单卡内部和集群之间的优化是同一套道理的两个尺度。** 单卡上要用分块 GEMM 把算术强度从 0.25 FLOP/byte（逐样本外积）抬到 10^2 以上（batch 维 GEMM、implicit GEMM、Flash-Attention），否则算力利用率不到 2%；集群上则要用好的归约拓扑把通信量压到 2M 并让它与计算重叠（分桶 AllReduce）。**"提高算术强度"和"减少通信字节"是贯穿全课程的两条主线。**

---

6. 常见陷阱与注意事项

- **误以为"卡越多越快"。** 通信量 ≈ 2M 与 N 无关，而计算量 ∝ 1/N，因此 $S(N) = 1/(1/N + r)$ 会迅速饱和（PCIe 上 8 卡的效率只有 31%，40 GbE 上只有 15%）。先用 `T_c/T_comm` 算天花板，再决定买几张卡；加了卡但 batch 不放大时，很可能是在给网卡打工。
- **把"AllReduce 时间"当成与 N 无关的常数。** 只有**带宽项 $2Mw$** 与 N 无关；**延迟项 $2(N-1)\alpha$** 随 N 线性增长。小模型、多节点时这一项会主导（示例：M = 10^6、N = 1024 时 10.33 ms 几乎全是启动延迟）。小消息要合并（bucketing），否则每次迭代都在给网络交"起步价"。
- **在梯度归约里制造数据竞争或伪共享。** 数据并行最容易犯的错是"多个线程/block 直接 `+=` 同一块 dW"：正确做法是（a）**层次化归约**（线程私有 → 块内树形归约 → 每块一次原子操作，如示例三）、或（b）**行所有权**（每线程独占若干权重行，如示例二版本 B）。若按列切分 dW，相邻线程会写同一 cache line → **伪共享（false sharing）**，性能可掉一个数量级。真实框架里梯度累加用 `atomicAdd` / `__shfl` 归约，正是这个原因。
- **忘记"梯度缓冲是每卡一份全量"这件事会随卡数放大显存。** 数据并行不省显存：N 张卡存 N 份完全相同的副本（1.5B 模型的 fp16 梯度就是 3 GB/卡，另加 20 B/参数的训练状态）。显存不够时不要只想着"减 batch"，要先算清账本，再决定用 ZeRO 哪一级。
- **用逐样本外积的方式算权重梯度。** 这是最典型的"算得动但跑不快"：算术强度 0.25 FLOP/byte，在 V100 上只能跑到峰值的约 1.4%（示例三实测会贴近 900 GB/s 带宽上限）。正确做法是把 batch 融合进循环，用分块 GEMM 让每个激活值被 D 个输出通道复用（示例二版本 B，AI 从 0.5 提到 214 FLOP/byte，模型预测加速 77×）。
- **假设浮点归约的结果可复现、或忽略激活内存。** Ring AllReduce 的累加顺序随 N 改变，**换卡数就会改变数值结果的最低有效位**，测试必须用容差；同时"内存账本"通常**不含输入 batch 与全部激活**（讲义 slide 40 明确注明），长序列 Transformer 的激活可以比参数本身还大（示例：32 层 × 16 头、序列 8192 时，朴素注意力要物化 8192^2 的矩阵，合计约 **68.7 GB**；分块/Flash 式融合只需约 **1.07 GB**），必须靠激活重算或分块融合来控制。

---

7. 思考题（带答案）

问题 1（拓扑选择）：某团队要在 **64 个节点**（每节点 8 张卡）上训练一个 **参数量只有 4×10^6 的小模型**，机间网络是 100 Gb/s Ethernet（单向 12.5 GB/s），每步的启动延迟 α = 5 μs。训练代码目前用 **Ring AllReduce** 同步梯度（fp32，每参数 4 B）。请估算一次 AllReduce 的耗时，指出瓶颈在哪一项，并给出至少两条改进方案及其定量收益。

【答案】 先算基本量：M = 4×10^6 参数，fp32 即 16 MB。$w = 1/(12.5\times10^9) = 8\times10^{-11}$ s/byte，故每参数传输时间 = 4 B × 8×10^{-11} = 3.2×10^{-10} s，**整份 M 的带宽成本 $Mw = 4\times10^6\times3.2\times10^{-10} = 1.28$ ms**。N = 64×8 = 512 卡。

- Ring：$2(N-1)(\alpha + (M/N)w) = 2\times511\times(5\ \mu s + 1.28\ \text{ms}/512) = 1022\times(5+2.5)\ \mu s = 1022\times7.5\ \mu s \approx 7.7$ ms。其中 $1022\times5\ \mu s = 5.11$ ms 是**纯启动延迟**（占 66%），带宽只有 2.6 ms。**瓶颈是"步数 × α"，即延迟项，而不是带宽。**
- **对照 Butterfly**：$\log_2 512 = 9$ 步 × (5 μs + 1.28 ms) = 9 × 1.285 ms ≈ **11.6 ms**——反而更慢！因为每步要传整份 M，带宽成本放大 9 倍。**这正是"小模型不一定该用 butterfly"的定量证据**：本例中 per-step 带宽成本（1.28 ms）远大于 α（5 μs），所以"少传字节"比"少走轮次"更值钱。
- Tree：$2\log_2 512\times1.285\ \text{ms} = 18\times1.285 = 23.1$ ms，最差。
- 改进方案（每条都给出定量收益）：
- **改成 fp16/bf16 传梯度（2 B/参数）**：每参数时间减半，带宽成本从 2.6 ms 降到 1.3 ms，Ring 总时间 7.7 ms → **6.4 ms**（再配合 loss scaling 保证收敛）。
- **分层归约（节点内 NVLink ring + 节点间 ring）**：节点内 8 卡走 NVLink（150 GB/s），每节点只由 1 张卡把"本节点已归约的 16 MB"发到机间。机间步数从 1022 降到 2×(64−1) = 126 步，机间延时项 126×5 μs = 0.63 ms，机间带宽项 2×(16 MB/12.5 GB/s) = 2.6 ms，节点内几乎可忽略 ⇒ 总计约 **3.3 ms**（相对 7.7 ms 提升 2.3×）。

- **梯度分桶 + 与反向计算重叠**：把 16 MB 梯度切成若干 bucket，每算完一层的梯度就发起该 bucket 的 AllReduce，让 5.11 ms 的启动/传输延迟藏在反向计算后面；理论上通信可被完全隐藏，**端到端开销趋近于 0**（前提是反向计算时间 ≥ 通信时间）。
- **增大 batch**（弱扩展）：每卡 batch 增大不会改变通信时间，但会让"计算时间/通信时间"之比上升，从而让通信占比下降——这是最省事的一条，代价是收敛性需要重新调参。

问题 2（内存与 ZeRO 的取舍）：某团队在 **8 张 A100-80GB** 上训练一个 **17.2B 参数**的模型（与讲义中 Turing NLG 同量级），采用混合精度（20 B/参数的训练账本，另有每卡约 12 GB 的激活）。请判断以下三种配置是否可行，并说明各自的通信代价；如果都不可行，指出最少需要多少张卡才能让 ZeRO-3 跑起来。

【答案】 训练状态总量 = 17.2×10^9 × 20 B = **344 GB**（这就是表 3 里"基线每卡 344 GB"的来源；讲义表中给出的 275 GB 是 16 B/参数的口径，不含 FP16 参数与 FP16 梯度的额外 4 B）。

- **基线数据并行**：每卡仍要 344 GB 全量 → 344 + 12 = **356 GB/卡 > 80 GB，不可行**。
- **ZeRO-1**（每卡 $4M + 16M/N$）：$4 \times 17.2 = 68.8$ GB + $16 \times 17.2/8 = 34.4$ GB = 103.2 GB，再加 12 GB 激活 = **115.2 GB/卡 > 80 GB，不可行**。通信量 = 2M 元素（与基线相同）。
- **ZeRO-2**（每卡 $2M + 18M/N$）：$34.4 + 18 \times 17.2/8 = 34.4 + 38.7 = 73.1$ GB，加 12 GB 激活 = **85.1 GB/卡 > 80 GB，仍不可行**（差得很近）。
- **ZeRO-3**（每卡 $20M/N$）：$344/8 = 43$ GB + 12 GB 激活 = **55 GB/卡 < 80 GB，可行**；通信量 = 3M 元素 = 基线的 **1.5 倍**。
- **最少卡数**：要求 $344/N + 12 \le 80 \Rightarrow N \ge 344/68 \approx 5.06$，即**至少 6 张 A100-80GB**（留出碎片与临时缓冲的现实余量后，8 张是合理配置）。若叠加张量并行把激活与参数进一步切开，还可以更省——这正是讲义 slide 66 说 "Turing NLG 17.2B is powered by **Stage 1 and Megatron**" 的含义：真实的大模型训练从来不是单一并行方式，而是 ZeRO-1 + 张量并行的组合。
- **补充要点**：ZeRO-3 的 1.5 倍通信在**小 batch** 时会成为主要瓶颈（第 4.5 节的 cross-over 分析：batch 不足约 10 万 token 时通信占主导），因此实践中要配合"参数 AllGather 与计算流水重叠"和非连续的梯度 bucket 划分。若显存够用，**优先选 ZeRO-2 而不是 ZeRO-3**：省下 50% 通信。

问题 3（性能模型与代码判断）：下面这段代码在一个 16 核 CPU（峰值 768 GFLOP/s，内存带宽 20 GB/s）上训练一个小 MLP。请指出它慢的根本原因，给出定量分析，并写出改进后的循环结构。

```
// B = 256 个样本，D0 = 784，D1 = 2048
for (int n = 0; n < B; n++) {
    // ... 前向得到第 n 个样本的隐层梯度 g[D1] ...
    for (int j = 0; j < D1; j++)
        for (int i = 0; i < D0; i++)
            dW1[j][i] += g[j] * X[n][i];     // 逐样本外积累加
}
```

【答案】 – **慢的根本原因：算术强度极低，是彻底的访存瓶颈，而不是算力不足。** 这个双层循环对 `dW1`（D1×D0 = 160 万个 float = **6.42 MB**）每处理一个样本就读一遍、写一遍，只在寄存器里做一次乘加就丢掉。**每个 X 元素、每个 dW1 元素的复用次数都是 1。** – **定量分析**：每样本的乘加次数 = D0×D1 = 1.605×10^6 次乘法（2 FLOP）⇒ 3.21 MFLOP；访存 = 读 dW1 6.42 MB + 写 dW1 6.42 MB = 12.84 MB ⇒ **AI = 3.21×10^6/12.84×10^6 = 0.25 FLOP/byte**。整批 256 个样本：计算 822 MFLOP，访存 **3.29 GB**。按 20 GB/s 带宽，**至少 165 ms**；而按 768 GFLOP/s 算力只要 1.07 ms —— **访存时间是计算时间的 154 倍**。可用带宽上限 = 20 GB/s × 0.25 = **5 GFLOP/s，即峰值的 0.65%**。 – **改进后的循环结构（batch 维 GEMM + 行所有权）**：

```
// 先算出整批的上游梯度 G1[B][D1] 与激活，再一次性做 dW1 = G1^T · X
#pragma omp parallel for schedule(static)
for (int j = 0; j < D1; j++) {            // 并行在权重行上：每线程独占若干行（无竞争、无伪共享）
    float *row = dW1[j];                   // 或 &dW1[j*D0]；整行常驻 cache 反复复用
    for (int n = 0; n < B; n++) {          // batch 维作为累加轴 → 复用率 = D1
        const float g = G1[n][j];
        const float *x = X[n];
        for (int i = 0; i < D0; i++) row[i] += g * x[i];
    }
}
```

- **收益**：dW1 只在最后写回一次，X 的每一行被 D1 = 2048 个输出通道复用（且命中缓存），DRAM 流量降到约 `X(803 KB) + G1(2.1 MB) + dW1(6.42 MB) ≈ 11.5 MB`，AI 提升到 `822×10⁶/11.5×10⁶ ≈ 214 FLOP/byte`。此时耗时由计算决定：`max(822 MFLOP/768 GFLOP/s, 11.5 MB/20 GB/s) = max(1.07 ms, 0.58 ms) ≈ 1.07 ms`，相对原来的 165 ms **约 150 倍**（示例二中因为还要存取 W1，实测的模型预测加速比是 77×）。
- **同时要说清的代价**：这个改写**必须把整批的中间激活与上游梯度留在内存里**（激活内存随 batch 线性增长），这正是"用内存换算术强度"的典型交换，也是真实框架里用**激活检查点（checkpointing / recomputation）**在两者之间找平衡的原因。此外，若把并行方向错选成"按列切分

dW1"（每线程负责若干列 i），相邻线程会写同一 cache line，将引入**伪共享**，收益会被吃掉一大截——**必须按行（连续维度）切分**。

Lecture 23: Parallel Deep Learning: Model and Pipeline Parallelism

1. 章节标题与概述

Lecture 23: Parallel Deep Learning: Model and Pipeline Parallelism

- **本讲核心问题**：当模型本身大到一张 GPU 装不下时（GPT-3 175B 参数的 FP16 权重就有 350 GB，而 A100 只有 40/80 GB 显存），"数据并行"这条路直接失效——它要求每张卡保存一份完整模型副本。本讲回答两个问题：**(1) 如何把模型的参数/计算本身切开（Model Parallelism，尤其是层内的 Tensor Model Parallelism）**；**(2) 切开之后如何避免设备互相等待（Pipeline Model Parallelism 与 micro-batch 流水线）**，并给出通信量、气泡（bubble）与设备利用率的定量模型。
- **涉及的主要硬件/软件机制**：跨 GPU 的集合通信原语（AllReduce / AllGather / ReduceScatter / 点对点 send-recv）与其运行的分层互连（节点内 NVLink/NVSwitch ≈ 300 GB/s 单向、节点间 InfiniBand HDR ≈ 25 GB/s、PCIe Gen4 ≈ 32 GB/s）；GPU 内部的分块 GEMM 执行模型（shared memory tiling、warp 级 SIMD 执行、tensor core）；软件层的张量并行算子分解（Megatron-LM 的 f/g 算子、多头注意力的头并行）、micro-batch 流水线调度（GPipe）、以及 DP×PP×TP 的三维并行（DeepSpeed 3D parallelism）。
- **在并行计算知识体系中的角色**：本讲是"并行模式 → 真实大规模系统"链条的收口。它把前面各讲的四个基础工具——**fork-join/SPMD 编程模型**（数据并行、流水线级就是 SPMD 进程）、**work-span 与 Amdahl 定律**（气泡就是无法并行的串行段）、**算术强度与 Roofline**（micro-batch 变小会把 GEMM 推入带宽受限区）、**通信延迟/带宽模型**（α-β 模型与集合通信代价）——第一次真正用在"一个可运行的工业级系统"（Megatron-LM / GPipe / DeepSpeed）上。它同时给出了一条贯穿全课的设计准则：**并行度不是免费的，每一层并行都对应一类通信，而通信的频率决定了它能用多慢的链路。**
- **配套材料**：
- **本讲讲义（已公开）**：`https://www.cs.cmu.edu/~418/lectures/25-parallel_deep_learning_model_pipeline_parallel.pdf`（HTTP 200 可直接匿名下载，2.9 MB，共 34 页，文件时间戳 2025-11-01）。讲义首页署名为客座讲师 *Zhihao Jia (Stanford University)*，页脚写 *CMU 15-418/15-618, Fall 2025*、编号 **Lecture 25**；而 Fall 2026 日程表中本讲编号为 **Lecture 23（Oct 26）**。这是课程"讲义沿用旧学期版本"的正常现象（同一份 deck 在不同学期编号不同），并非错误。
- **本笔记的事实依据**：`extracted/25-parallel_deep_learning_model_pipeline_parallel.txt`（上述 PDF 的逐页抽取文本，标记 `===== [slide i/34] =====`）。
- **Fall 2026 日程表（已公开）**：`https://www.cs.cmu.edu/~418/schedule.html`，第 23 讲 Oct 26 标题为 *Parallel Deep Learning (model and pipeline parallelism)*。该行的 slides/video 链接**仍以 HTML 注释形式保留**（注释原文：*slide/video from a previous offering; uncomment when posted for Fall 2026*），因此 **Fall 2026 本讲的 slides 与录像在日程表上尚未发布**；注释中记录的历史公开录像为 YouTube 上的"模型并行"与"流水线并行"两段。**Fall 2026 录像在日程表中被注释隐藏 → 属未发布。**
- **补充材料（已公开）**：`cs149_supp/dnninference.txt` = Stanford CS149 Fall 2025 Lecture 9 *Efficiently Evaluating DNNs*（公开课程材料，75 页）。本笔记用它补充 GEMM 分块/向量化、算术强度与 Roofline、算子融合（loop fusion / flash-attention）、显式 vs 隐式 GEMM 等底层性能背景——这些正是张量并行的"每一格方块"要落到硬件上执行的东西。
- **需登录 / 未公开**：历史学期讲义 PDF 位于 `/afs/cs/academic/class/15418-*/public/`（需 CMU 登录）；Ed 讨论区、Autolab、Canvas 均需登录。Fall 2026 授课教师为 Brian Railing 与 Dimitrios Skarlatos，课程由 Kayvon Fatahalian 创建（本讲未在讲义中指定具体作业/评分点，笔记中也不涉及）。
- 注：`f22_25_dnn.txt`、`s23_25_dnn.txt` 两个抽取文件为空（仅 2 行），无可引用内容，本笔记未使用。

---

2. 核心概念与硬件/软件架构图解

2.1 从数据并行到模型并行：显存墙（Memory Wall）

定义与目的：数据并行（Data Parallelism）把训练集切成 batch 分给 N 张卡，每张卡用**完整模型副本**算自己那批数据的梯度，再用 AllReduce 聚合梯度，然后各自更新参数（`w_i := w_i - γ∇L(w_i)`）。它的目的是把"数据的并行度"变成算力，同时**前向/反向过程中零通信**（只在梯度同步时通信一次）——这是它最大的优点。

它是什么（直观解释）：数据并行就像**一家连锁餐厅的 N 个分店，每家都买了全套厨具和完整菜谱**：客人（数据）分流到各店，各店独立出餐，晚上汇总大家的经验（梯度）。问题是"菜谱"（模型参数）一旦厚到装不进单店的厨房（显存），这套模式就崩了。

为什么必须换路子（容量数字，来自课程前一讲的数据并行讲义）：

模型	参数量	显存需求（训练状态）	单卡能否装下（V100 16/32 GB、A100 40/80 GB）
Bert-Large	0.32 B	5.12 GB	可以
GPT-2	1.5 B	24 GB	勉强（A100 40 GB）
Turing NLG	17.2 B	275 GB	不行
GPT-3	175 B	2800 GB	不行

2800 GB / 80 GB ≈ **至少 35 张 A100 才"放得下"模型状态**（还没算激活值）；再加数据并行要求每卡一份完整副本，这条路彻底走不通。于是必须切模型。

2.2 模型并行（Model Parallelism）：子图切分与设备放置

定义与目的：把模型（计算图）拆成多个子图，分给不同设备；设备之间**传递中间结果（激活/梯度）**；**没有梯度同步**（讲义的原文表述：*No synchronization of gradients*）。它解决的是"模型放不下"的问题，而不是"算得更快"的问题。

它是什么（直观解释）：模型并行像**把一条装配线拆成几个工位**，每个工位只装一部分零件（一层或几层），半成品用传送带送到下一个工位。注意：如果只切、"不分时"，同一时刻只有一个工位在干活，其他工位在等——这正是下一节流水线并行要修的毛病。

架构/机制图解（图 1：三种并行策略的"谁存什么、谁算什么"）：

关键性能特征：– **延迟**：模型并行的前向是**串行链**（op1→op2→…→loss），单样本延迟几乎不会因为加卡而下降，反而因为每层要跨设备传激活而上升（一次设备间激活传输 = α（启动延迟）+ bytes/BW）。– **带宽**：跨设备激活尺寸 ∝ B·S·H（batch × 序列长 × 隐层），与计算量（∝ B·S·H²）相比只差一个 H。**H 越大、batch 越大，通信相对越便宜**。– **吞吐量**：无流水线时吞吐 ∝ 1/p 恶化（见上图 b）。所以模型并行的价值在**容量**，吞吐要靠流水线并行补回来。

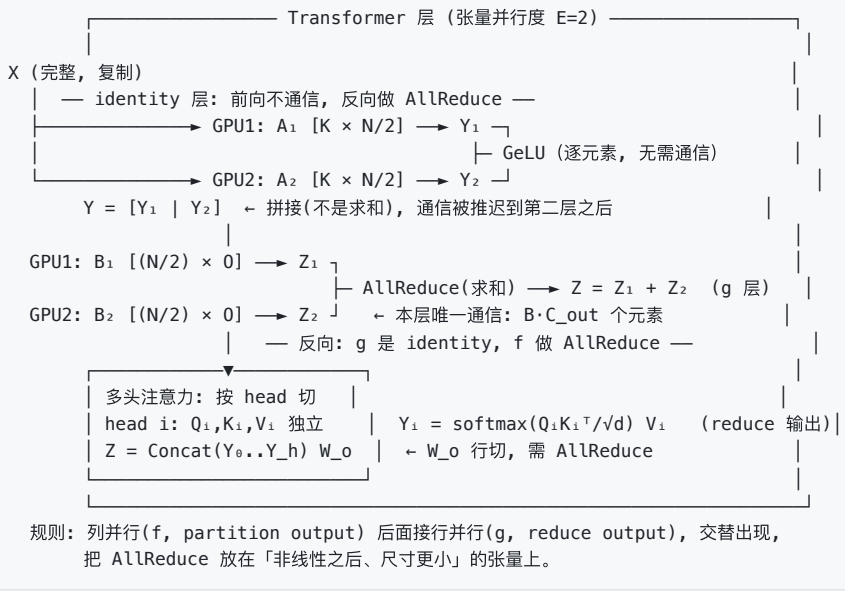
放置（placements）为什么难：讲义第 5 页给出两张对照图——"在 4 张 GPU 上训练 RNN"与"在 4 张 GPU 上训练常规网络"的最优切分完全不同（RNN 有时间步依赖，只能按层切；CNN/MLP 可以按通道切）。第 6 页给出的自动化思路是**用 ML 来调 ML 的放置**：RL Agent 输出 device placement → 在真实并行机上跑 → 得到 runtime performance 作为 reward → 迭代（引文：*Device placement optimization with reinforcement learning, A. Mirhoseini et al.*；图中还标了 *ColocRL's neural architecture*）。第 7 页进一步给出**数据并行 + 模型并行混用**的方向——这是后面 3D 并行的雏形。

2.3 张量模型并行（Tensor Model Parallelism）：两种切分方式与通信量记账

定义与目的：不在"层与层之间"切，而是**在一层/一个算子内部切参数与梯度**（讲义原文：*Partition parameters/gradients within a layer/operator*）。目的：把权重的显存占用和 GEMM 的计算量同时按 E 份摊到 E 张卡上，同时让切分后的通信量比数据并行**小几个数量级**。

它是什么（直观解释）：模型并行是"把装配线切成工位"，张量并行是"**把同一道工序的原料横向切开，几个人同时处理同一道菜**"。两种切法： – **partition output（按输出切）**：两个厨师拿到**完全相同的食材**（完整输入 x），各自按菜谱的**一半行**做菜，产出不同的菜（y_1、y_2）——无需合并。 – **reduce output（按输入切）**：两个厨师各拿到**半份食材**（x_1、x_2），各自做**同一道菜**，最后必须**合锅**（$y = y_1 + y_2$）。

架构/机制图解（图 2：Transformer 一层的张量并行数据流，Megatron-LM 写法）：



通信量记账（讲义第 8–13 页的核心结论，务必按原口径理解）：设一层做 `y = W x`，`W` 是 `C_out × C_in`，batch 为 `B`：

并行策略	前向 (Forward)	反向 (Backward)	梯度同步 (Gradients Sync)	通信对象与原因
数据并行 Data Parallelism	0	0	`C_out × C_in`	同步梯度（= 全部参数）
张量并行（partition output / 列切）	`B × C_in`	`B × C_in`	0	传输中间结果（部分输入/激活），*假设每张 GPU 只有部分输入*
张量并行（reduce output / 行切）	`B × C_out`	0	0	对 y_1、y_2 做 AllReduce；讲义第 12 页把前向的收发量记为 `2 × B × C_out`，第 13 页小结表中简记为 `B × C_out`（消息大小）

> 读表要点：数据并行的通信量是**参数尺寸** `C_out·C_in`（与 batch 无关，但每卡都要存全量参数）；张量并行的通信量是**激活尺寸** `B·C_in` 或 `B·C_out`（与参数无关，所以千亿参数模型的张量并行通信量并不随参数量爆炸）。两者的量级差 `C_out·C_in / (B·C_out) = C_in / B`：当 `C_in = 4096`、`B = 8` 时相差 **512 倍**。这正是张量并行能扩展到数百卡的根本原因。
>
> 而代价是**频率**：数据并行每个 iteration 只同步一次梯度；张量并行**每一层**都要通信（Transformer 每层 2 次 AllReduce）。**通信量小 × 频率高 × 对延迟敏感**，三者共同决定了它只能跑在最快的链路上。

"最好的策略取决于模型和机器"（讲义第 13 页结论）——CNN 的分层策略（讲义第 17–18 页）： – **卷积层**：占 **90–95% 的计算量**，只占 **5% 的参数**，但有**非常大的中间激活**。→ 用**数据并行**（参数小，副本无所谓；激活大，正好靠数据并行摊薄）。 – **全连接层**：占 **5–10% 的计算量**，占 **95% 的参数**，中间激活很小。→ 用**张量模型并行**（参数大，必须切开；激活小，通信便宜）。 – 讲义的第 18 页直接给出结论行：*Data parallelism for convolutional layers / Tensor model parallelism for fully-connected layers*。

Transformer 的层内切分（讲义第 19–26 页）：一层 Transformer = 多头自注意力 + 两个全连接层。 – 注意力：`Attention(Q,K,V) = softmax(QKᵀ/√d)V`；多头版把输入分别做线性变换得到 `Q_i,K_i,V_i`，各头独立算 `Y_i = softmax(Q_iK_iᵀ/√d)V_i`，最后 `Z = MultiHead(Q,K,V) = Concat(Y_0,…,Y_h)W_o`。**每个 head 分到一张卡**（head 并行），`W_o` 按行切（reduce output，需 AllReduce）。 – 为什么用多头（讲义第 24 页的一行问答）：*Why multi-head attention? → Reduce complexity of matrix multiply!*（把 `N×N` 的大矩阵乘拆成 h 个 `N×d/h` 的小矩阵乘，且天然给出 h 路并行）。 – 两个全连接层：`Y = GeLU(X A)`（A 按列切，partition output）、`Z = Dropout(Y) B`（B 按行切，reduce output）。讲义第 21 页明确标注 `Weights: Partition on Col dim`、`Y = [Y₁ Y₂]`、`Z = Z₁ + Z₂`，并问了一句 **"Why switch?"**（为什么这样交替切）——答案见第 7 节思考题 Q2。 – 讲义第 26 页给出规模结论：Megatron-LM 用"数据并行 + 模型并行"组合**扩展到 512 张 GPU**。

2.4 流水线模型并行（Pipeline Model Parallelism）：micro-batch 与气泡

定义与目的：让被切成 p 级的模型**真正同时在干活**。做法是把一个 mini-batch 再切成 m 个 **micro-batch**，让**不同 micro-batch 的不同级在同一时刻重叠执行**（一级算 micro-batch i 的前向时，下一级在算 i-1 的前向）。目的：把 2.2 节图 1(b) 中"只有一台机器在算"的空转填满。

它是什么（直观解释）：像**洗车流水线**：一辆车（micro-batch）依次经过"冲水→打沫→擦干→上蜡"四个工位；如果一次只让一辆车走完全程，工位 2/3/4 大部分时间在晒太阳；同时放 8 辆车进去（m = 8），每个工位就一直在忙。但**流水线要"灌满"和"排空"**：开头的 3 个节拍（fill）和结尾的 3 个节拍（drain）没有满负荷，这段空转就是**气泡（bubble）**。

讲义给出的定量模型（第 30–31 页）：设 `m` = 一个 mini-batch 里的 micro-batch 数，`p` = 流水线级数，每级处理一个 micro-batch 的前向/反向时间都是 `t_f / t_b`，则

```
BubbleFraction = (p-1) * (t_f + t_b)  /  ( m*t_f + m*t_b )  =  (p-1) / m
```

> **口径说明（非常重要，容易算错）**：讲义这个式子的分母 `m*(t_f+t_b)` 是**理想计算时间**（没有任何气泡时单级应该忙的时间），所以 `(p-1)/m` 是"气泡相对于理想计算量"的比例，它不是气泡占墙钟时间的比例。整条流水线的墙钟时间 = `(m + p - 1) * (t_f + t_b)`（fill + steady + drain），因此**气泡占墙钟时间的比例 = (p-1)/(m+p-1)**，设备利用率 = `m/(m+p-1)`。两个口径都成立，只是分母不同；CMU 讲义的 `(p-1)/m` 用的是前者，论文/工程里常用后者。本笔记第 4 节会同时给出两者并做数值对照。

架构/机制图解（图 3：GPipe 调度 vs 定点重叠，含气泡）：

```
 (a) GPipe 调度: 全前向 → 全反向  (m=4 个 micro-batch, p=4 级; F=前向, B=反向)
     时间 →
stage1 |F0|F1|F2|F3|  |  |  |B3|B2|B1|B0|
stage2 |  |F0|F1|F2|F3|  |  |B3|B2|B1|B0|  |
stage3 |  |  |F0|F1|F2|F3|B3|B2|B1|B0|  |  |
stage4 |  |  |  |F0|F1|F2|F3|B3|B2|B1|B0|  |
        └─── fill ──┘└ 稳定 ┘└─ drain ─┘
        气泡 = (p-1) 个 (F+B) 时间槽, 总时间 = (m+p-1)(t_f+t_b)
        利用率 = m/(m+p-1) = 4/7 = 57% (m=4,p=4)

 (b) 增大 m (m=16, p=4) : 稳定段变长, 气泡占比从 3/7=43% 降到 3/19=16%
     时间 →
stage1 |F0..F15|B15..B0|
stage2 | F0..F15|B15..B0|     每个级几乎全程忙碌
stage3 |  F0..F15|B15..B0|
stage4 |   F0..F15|B15..B0|
        └──────── 稳定段 ∝ m ────────┘

 (c) 1F1B (PipeDream/DeepSpeed 思路, 讲义未展开, 作为延伸) : 前向/反向交错,
     每个级最多只需缓存 ≈ p 个 micro-batch 的激活, 而不是 GPipe 的 m 个
     stage1 |F0|F1|F2|F3|B0|F4|B1|F5|B2|B3|B4|B5|...
                                  ↑ 前向与反向交替, 显存 ↓, 气泡仍 ≈ (p-1)/(m+p-1)
```

关键性能特征（延迟/带宽/吞吐）：– **吞吐**：稳态时每个时间槽有一级产出一个 micro-batch 的反向结果，所以吞吐 ∝ 1/(t_f+t_b)（每级），与 p 无关——**加级数不提升吞吐，只提升"能装下的模型规模"**。 – **延迟**：单个 micro-batch 的端到端延迟 ≈ `p*(t_f+t_b)`（要穿过全部 p 级），**p 越大单样本延迟越高**。 – **带宽**：级与级之间每个 micro-batch 传一次激活 `B_micro·S·H·2 bytes`（FP16）。因为可以**与计算重叠**（下一级的 F 与上一级的 transfer 并行），所需的带宽远低于张量并行。 – **气泡**：`(p-1)/(m+p-1)`。三条改善路径与各自的代价（讲义第 31 页明确列出）： 1. **增大 m**（把 mini-batch 切得更细，或增大 mini-batch）——代价：*large mini-batch sizes can lead to accuracy loss*（大 batch 影响收敛精度）；切得太细则 *small micro-batch sizes reduce GPU utilization*（micro-batch 太小，GEMM 的 M 维变小，GPU 打不满，见第 4 节的 Roofline 算例）。 2. **减小 p**（流水线浅一点）——代价：*increase stage size*（每一级要装更多层，显存压力回到单卡）。 3. （讲义外的工程手段，作为延伸）**减小每级"空转粒度"**：interleaved / virtual pipeline stages（把每一级再切成 v 个虚拟级交错调度），把气泡降到 ≈ `(p-1)/(m·v)`。

2.5 三种并行的对比与 3D 并行（DeepSpeed）

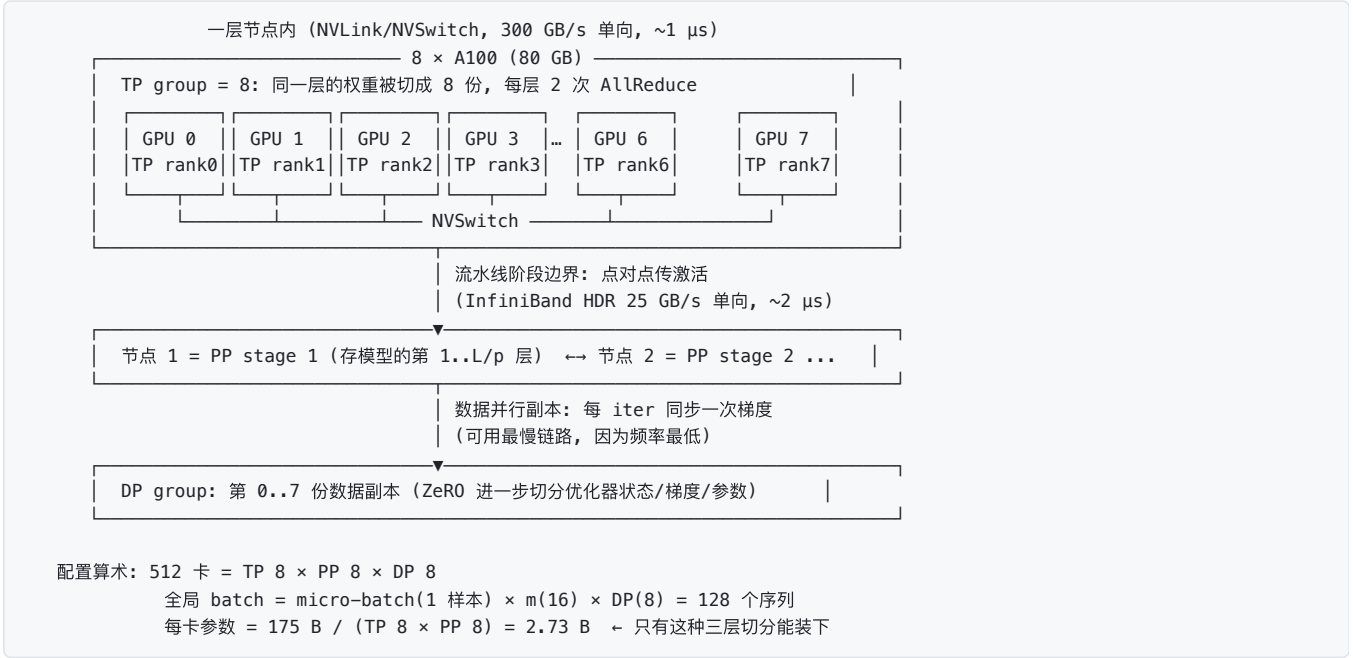
讲义第 32–33 页的总结表（Pros / Cons），完全按原文整理：

	数据并行	模型并行（含张量并行）	流水线并行
优点 Pros	✓ 可大规模并行（Massively parallelizable） ✓ 前向/反向**不需要通信**	✓ 支持训练超大模型（Support training large models） ✓ 对参数量大的模型高效	✓ 支持大 batch 训练 ✓ 对**深**模型高效（Efficient for deep models）

	数据并行	模型并行（含张量并行）	流水线并行
缺点 Cons	❖ 模型装不进一张 GPU 就完全不可用 ❖ 参数量大时不可扩展	❖ 并行度有限，扩不到很多 GPU ❖ 前向/反向必须传中间结果	❖ 利用率受限：前向/反向存在**气泡**
通信量（每层/每 iter）	参数 `C_out·C_in`，1 次/iter	激活 `B·C_in` 或 `B·C_out`，**每层 2 次**	激活 `B_micro·S·H`，每级边界 1 次/micro-batch
通信频率 → 可用链路	最低 → 可用最慢链路（以太网/IB）	最高（每层） → 必须 NVLink	中等（可重叠） → IB 可接受
主要解决什么	吞吐（算力扩展）	**容量**（显存）	吞吐（补回模型并行的空转）

讲义第 34 页的收口结论：*Training large models requires combining data/model/pipeline and other parallelization techniques* —— 即 **3D 并行**（DeepSpeed 官方博客图示）：**数据并行 × 流水线并行 × 张量并行**三层嵌套。

架构/机制图解（图 4：3D 并行的硬件/软件映射，512 卡实例）：



关键设计准则（贯穿本讲的一条线）：**通信频率越高的并行维度，必须放在越快的链路上**。张量并行每层都通信 → 必须留在 NVLink 域内；流水线并行每 micro-batch 通信一次且可与计算重叠 → 可以跨节点走 IB；数据并行每 iteration 同步一次 → 放到最外层，甚至可以用较慢的链路。

2.6 让张量并行的每一格方块跑满：GEMM 的执行模型（CS149 补充材料）

张量并行把 GEMM 的维度切开后，每个 GPU 上跑的还是 GEMM。讲义本身不重复 GEMM 优化，但"切分后的 GEMM 还能不能跑满"完全由 CS149 讲过的机制决定：

(1) 分块（blocking/tiling）与算术强度：直接三重循环的 GEMM 对 A、B 没有时间局部性，arithmetic intensity 很低；按 cache 层级分块（L2 → L1 → 寄存器）后，每次从 DRAM 取一个 `TILE×TILE` 的块就能做 `TILE` 次 FMA，片内算术强度 ≈ `TILE/4` FLOP/byte。**张量并行把 N 切成 N/E 后，可用的 tile 数变少，尾部（tail）浪费变大**——这是切分粒度不能无限小的硬件原因。

(2) 向量化与数据布局：CS149 给了三种向量化分块 GEMM 的写法：(i) 向量化 i 维（warp 内不同 lane 算不同输出列，`splat` A 的标量乘 B 的向量）；(ii) 先转置 B 块再对最内层 k 维做向量点积；(iii) `SIMD_WIDTH × SIMD_WIDTH` 的寄存器块（每个 lane 维护 `SIMD_WIDTH` 个累加器，同时对 B 的 `SIMD_WIDTH` 个元素做 FMA），这是现代 GPU tensor core 之前的标准做法，也是"一个 warp 一次算 32×32 个输出"的来源。

(3) 算子融合（loop fusion）：CS149 的例子很直观——分开写 `add/mul/add` 三个循环的整体算术强度是 1/3（2 load + 1 store 换 1 个数学运算）；融成一个循环后中间结果不出 DRAM，算术强度升到 3/5。在 Transformer 里，这就是 **flash-attention** 的来源：softmax 的朴素实现要读 `5MN+2M`、写 `3MN+2M` 个元素；按行分块、把 `softmax(QKᵀ)V` 完全融合在片上完成，只需读 `MN`、写 `MN`，并且**永不显式构造 N×N 的注意力矩阵**（`N` 是序列长度，朴素实现要 `N²` 空间，长序列直接爆）。张量并行的"每层 2 次 AllReduce"之所以能被容忍，很大程度上靠这类融合把计算侧的带宽压力压下去。

(4) 显式 vs 隐式 GEMM：用现成的高性能 GEMM 库需要把卷积"im2col"成矩阵，DRAM 流量增加 `R×S` 倍（R、S 是卷积核尺寸）并多占大量存储；隐式 GEMM（implicit GEMM）只在片上共享内存里物化一个子块，既不额外占显存也不增加 DRAM 流量（CUTLASS/cuDNN 的做法）。这

就是为什么训练框架的"算子→kernel"映射不是简单调库，而是与并行策略耦合的调度问题。

(5) 低精度：16 bit / 8 bit 已是主流，4 bit 正在进入，极端情况 1 bit。低精度直接使**通信量按位宽线性下降**——张量并行每层传的激活如果从 FP32 变 BF16，通信量立刻减半，这对通信受限的张量并行是"免费"的加速。

---

3. 代码示例与性能分析

3.1 示例 1：MPI + OpenMP 实现 Megatron 风格的张量并行 MLP（列并行 → GeLU → 行并行 → AllReduce）

这段代码把讲义第 21 页的 `Y = GeLU(X A)`（A 按列切，f 算子）+ `Z = Dropout(Y) B`（B 按行切，g 算子）完整实现成可运行的 MPI 程序，并用单 rank 串行版本做正确性对照。

```cpp
// tp_mlp.cpp -- Megatron-LM 风格张量并行 MLP: 列并行(f) -> GeLU -> 行并行(g) -> AllReduce
// 编译: mpicxx -O3 -march=native -fopenmp -std=c++17 tp_mlp.cpp -o tp_mlp
// 运行: OMP_NUM_THREADS=4 mpirun -np 2 ./tp_mlp
#include <mpi.h>
#include <omp.h>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <random>
#include <vector>

static const int M = 64;    // micro-batch 中的 token 数 (B*S)
static const int K = 256;   // 输入隐层维度 H
static const int N = 512;   // MLP 中间维度 4H
static const int O = 256;   // 输出隐层维度 H

static inline float gelu(float x) { return 0.5f * x * (1.0f + std::erff(x * 0.70710678f)); }

// C[M x n] = A[M x k] * B[k x n], 行主序; OpenMP 按输出行并行
static void gemm(int m, int k, int n, const float* A, const float* B, float* C, bool accumulate) {
#pragma omp parallel for schedule(static)
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            float acc = accumulate ? C[i * n + j] : 0.0f;
            for (int p = 0; p < k; ++p) acc += A[i * k + p] * B[p * n + j];
            C[i * n + j] = acc;
        }
    }
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank = 0, size = 1;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (N % size != 0) { if (rank == 0) std::printf("rank 数必须整除 N=%d\n", N); MPI_Finalize(); return 1; }
    const int Nl = N / size;              // 本 rank 的分片宽度 N/E
    const int reps = 20;                  // 重复次数, 取最好一次 (benchmark 惯例)

    // 所有 rank 生成完全相同的 X / Afull / Bfull
    std::vector<float> X(M * K), Afull(K * N), Bfull(N * O);
    std::mt19937 rng(2024);
    std::uniform_real_distribution<float> dist(-0.5f, 0.5f);
    for (auto& v : X) v = dist(rng);
    for (auto& v : Afull) v = dist(rng);
    for (auto& v : Bfull) v = dist(rng);

    // ---- 切分权重: A 列并行 (partition output), B 行并行 (reduce output) ----
    std::vector<float> A(K * Nl), B(Nl * O);
    for (int k = 0; k < K; ++k)
        for (int j = 0; j < Nl; ++j) A[k * Nl + j] = Afull[k * N + rank * Nl + j];
    for (int i = 0; i < Nl; ++i)
        for (int j = 0; j < O; ++j) B[i * O + j] = Bfull[(rank * Nl + i) * O + j];

    std::vector<float> Y(M * Nl), Z(M * O, 0.0f);
    double best = 1e30;
    for (int r = 0; r < reps; ++r) {
        double t0 = MPI_Wtime();
        gemm(M, K, Nl, X.data(), A.data(), Y.data(), false);   // f: 列并行 GEMM, 前向无通信
        for (auto& v : Y) v = gelu(v);                         // 逐元素非线性, 无需通信
        gemm(M, Nl, O, Y.data(), B.data(), Z.data(), false);   // g: 行并行 GEMM, 得到部分和
        MPI_Allreduce(MPI_IN_PLACE, Z.data(), M * O, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD); // 前向唯一通信
        best = std::fmin(best, MPI_Wtime() - t0);
    }

    // ---- 单 rank 串行参考实现 (完整权重) ----
    std::vector<float> Yref(M * N), Zref(M * O);
    gemm(M, K, N, X.data(), Afull.data(), Yref.data(), false);
    for (auto& v : Yref) v = gelu(v);
    gemm(M, N, O, Yref.data(), Bfull.data(), Zref.data(), false);
    float maxerr = 0.0f;
```

```
    for (int i = 0; i < M * O; ++i) maxerr = std::fmax(maxerr, std::fabs(Z[i] - Zref[i]));

    const double flops = 2.0 * M * K * N + 2.0 * M * N * O;      // 整个 MLP 的总浮点运算量
    const double comm_bytes_per_rank = 2.0 * (size - 1.0) / size * M * O * sizeof(float);
    if (rank == 0) {
        std::printf("E=%d threads=%d | M=%d K=%d N=%d O=%d\n", size, omp_get_max_threads(), M, K, N, O);
        std::printf("forward 最好一次 = %.3f ms  (%.2f GFLOP/s 全机器, 单 rank %.2f GFLOP/s)\n",
                    best * 1e3, flops / best / 1e9, flops / size / best / 1e9);
        std::printf("每次前向 AllReduce 装载字节/rank = %.1f KB, 最大误差 = %.3e\n",
                    comm_bytes_per_rank / 1024.0, maxerr);
    }
    MPI_Finalize();
    return 0;
}
```

【代码做什么？】 1. 每个 rank 用**相同的随机种子**独立生成 `X (M×K)`、`A (K×N)`、`B (N×O)`——这不是"偷懒"，而是真实训练的对应物：`X`（激活）由**数据并行**保证每张卡上的副本相同，`A`/`B` 在初始化时本来就由同一个随机种子生成（Megatron 只切分、不重采样）。 2. **切分**：`A` 按列切成 E 份（第 `e` 份 = 每行的 `[e·N/E, (e+1)·N/E)` 列），**partition output**；`B` 按行切成 E 份（第 `e` 份 = 第 `[e·N/E, (e+1)·N/E)` 行，所有列），**reduce output**。两份分片的"缝"必须对齐（`A` 的列分片宽度 = `B` 的行分片宽度 = `N/E`），这是矩阵乘维度匹配的硬性要求。 3. **前向三步**：本地 GEMM 得到 `Y_e = X·A_e (M × N/E)` → 对 `Y_e` 逐元素做 GeLU（**注意：此时不需要任何通信**，因为 GeLU 是逐元素算子，作用在"属于自己那一份"的元素上完全合法）→ 本地 GEMM 得到部分和 `Z_e = Y_e·B_e (M × O)` → **一次 `MPI_Allreduce` 把 E 份部分和相加**得到完整 `Z`。 4. **正确性验证**：用完整权重在单 rank 上串行重算一遍，比较最大绝对误差。 5. 循环 20 次，取**最好一次**的时间（benchmark 惯例，排除调度抖动）。

实测输出（在共享的登录节点上运行，`OMP_NUM_THREADS=4`）：

```
E=4 threads=4 | M=64 K=256 N=512 O=256
forward 最好一次 = 1.631 ms  (20.57 GFLOP/s 全机器, 单 rank 5.14 GFLOP/s)
每次前向 AllReduce 装载字节/rank = 96.0 KB, 最大误差 = 1.907e-05
E=8 threads=4 | M=64 K=256 N=512 O=256
forward 最好一次 = 0.734 ms  (45.70 GFLOP/s 全机器, 单 rank 5.71 GFLOP/s)
每次前向 AllReduce 装载字节/rank = 112.0 KB, 最大误差 = 1.717e-05
```

误差 ~2e-5 是 FP32 累加顺序改变造成的正常舍入差（不是 bug）。注意这台机器是多人共用的登录节点，E=1/E=2 的测量被内存带宽争抢严重污染（E=1 反而比 E=2 慢），因此**趋势结论以 E=4 → E=8 为准**：单 rank 吞吐几乎不变（5.14 → 5.71 GFLOP/s），总吞吐翻倍，说明通信开销在 E≤8 时还远未成为瓶颈。

【并行机制与性能解说】 – **进程/线程如何创建、工作如何分配**：MPI 起 E 个进程（在真实系统上就是 E 张 GPU）；每个进程内用 `#pragma omp parallel for` 把 GEMM 的 `M` 行分给 4 个线程（`schedule(static)`，每线程 16 行）。**两层并行是嵌套的**：外层是跨设备的 SPMD（E 路），内层是设备内的 fork–join（线程数 = SM 数 / warp 数）。共享数据（`X`、`A_e`、`B_e`）只读，因此没有数据竞争；`Y`、`Z` 是每线程写自己的行，也不冲突（**无伪共享风险**，因为按行切分，行长度 256~512 字节远超 cache line 64 字节）。 – **Work（总工作量）**：`W = 2·M·K·N + 2·M·N·O + O(M·N)`（最后一项是 GeLU）`= 2·64·256·512 + 2·64·512·256 + 64·512·10 ≈ 16.78 M + 16.78 M + 0.33 M ≈ 33.9 MFLOP`。 – **Span（关键路径）**：`S = W/(E·P_rank) + T_AllReduce`，其中 `P_rank` 是单 rank 的有效算力，`T_AllReduce = 2(E−1)·α + 2(E−1)/E · (M·O·4 bytes)/BW`（ring AllReduce 的经典形式：`2(E−1)` 步，每步传 `(M·O/E)` 个元素）。 – 代实测值 `P_rank ≈ 5.7 GFLOP/s`：`W/(E·P_rank)` 在 E=8 时 = 33.9e6/(8·5.7e9) = **0.743 ms**，与实测 0.734 ms 吻合到 1%，说明这一档**纯计算主导**。 – 通信侧：`M·O = 16384` 个 float = 64 KB，E=8 时每 rank 收发 `2·7/8·64 KB = 112 KB`；若走 IB HDR 25 GB/s，传输 ≈ 4.5 μs，加 `2(E−1)=14` 步各 ~2 μs 启动延迟 ≈ 33 μs，即 **约为计算时间的 4%**（实测总时间几乎无变化，一致）。 – **并行度 = Work/Span**：通信可忽略时 `≈ E`（线性扩展）；但通信项 `2(E−1)α` **随 E 线性增长**，而计算项 `W/(E·P)` 随 E 下降，因此存在一个最优点： – E=32 时：计算 = 33.9e6/(32·5.7e9) = 0.186 ms；通信 = `2·31·2μs + 2·31/32·64KB/25GB/s` ≈ 124 μs + 5 μs = **129 μs**，已经相当于计算时间的 **69%**——**再增加 E 只会更慢**。 – 这条曲线的形状（先线性上升、后饱和并回落）就是"张量并行度不能无限大"的定量原因；同时也是**为什么张量并行必须放在 NVLink 域内**：把 25 GB/s 换成 300 GB/s，通信时间降到 ~11 μs，E=32 才重新变得划算。 – **瓶颈总结**：小规模时瓶颈是**朴素 GEMM 的指令级效率**（只有 5.7 GFLOP/s/rank，未用向量化分块，远低于单核峰值）；规模变大后瓶颈切换到**集合通信的启动延迟（α 项）**；`M` 很小（此处 64）时每一层的 GEMM 都"又短又频繁"，α 的相对代价被放大——这正是小 micro-batch 与高张量并行度不能同时使用的另一面。

3.2 示例 2：CUDA 上单个张量并行分片的 tiled GEMM

张量并行切完之后，每张 GPU 上的活就是一个 `M × K × (N/E)` 的 GEMM。下面这个 CUDA 程序实现这张分片 GEMM，并直接暴露"切分粒度 ↔ GPU 占用率"的关系。

```
// tp_shard_gemm.cu —— 张量并行分片上的 tiled GEMM（本 rank 只算 X * W_e，W_e 为列分片）
// 编译: nvcc -O3 -arch=sm_80 -Xptxas -v tp_shard_gemm.cu -o tp_shard_gemm
// 运行: ./tp_shard_gemm [M] [K] [Nlocal]     例如 ./tp_shard_gemm 1024 1024 512
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define TILE 32
#define CK(x) do { cudaError_t e_ = (x); if (e_ != cudaSuccess) { \
    std::printf("CUDA error %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(e_)); std::exit(1); } } while (0)

// Y[M x N] = X[M x K] * W[K x N]，分块装进 shared memory
__global__ void gemm_tiled(const float* __restrict__ X,
                           const float* __restrict__ W,
                           float* __restrict__ Y,
                           int M, int K, int N)
{
    __shared__ float Xs[TILE][TILE + 1];   // +1 避免 bank conflict
    __shared__ float Ws[TILE][TILE + 1];
    const int row = blockIdx.y * TILE + threadIdx.y;   // 输出行
    const int col = blockIdx.x * TILE + threadIdx.x;   // 输出列（本 rank 的局部列）
    float acc = 0.0f;
    for (int k0 = 0; k0 < K; k0 += TILE) {
        Xs[threadIdx.y][threadIdx.x] = (row < M && k0 + threadIdx.x < K) ? X[row * K + k0 + threadIdx.x] : 0.0f;
        Ws[threadIdx.y][threadIdx.x] = (k0 + threadIdx.y < K && col < N) ? W[(k0 + threadIdx.y) * N + col] : 0.0f;
        __syncthreads();
#pragma unroll
        for (int k = 0; k < TILE; ++k) acc += Xs[threadIdx.y][k] * Ws[k][threadIdx.x];
        __syncthreads();
    }
    if (row < M && col < N) Y[row * N + col] = acc;
}

int main(int argc, char** argv) {
    const int M = argc > 1 ? std::atoi(argv[1]) : 1024;   // token 数 B*S
    const int K = argc > 2 ? std::atoi(argv[2]) : 1024;   // 输入维度
    const int N = argc > 3 ? std::atoi(argv[3]) : 512;    // 本 rank 的分片宽度（= 全局 N / TP 度）
    const int iters = 100;

    float *hX = (float*)std::malloc(sizeof(float) * M * K);
    float *hW = (float*)std::malloc(sizeof(float) * K * N);
    float *hY = (float*)std::malloc(sizeof(float) * M * N);
    float *hRef = (float*)std::malloc(sizeof(float) * M * N);
    for (int i = 0; i < M * K; ++i) hX[i] = std::sin(0.001f * i);
    for (int i = 0; i < K * N; ++i) hW[i] = std::cos(0.002f * i);

    float *dX, *dW, *dY;
    CK(cudaMalloc((void**)&dX, sizeof(float) * M * K));
    CK(cudaMalloc((void**)&dW, sizeof(float) * K * N));
    CK(cudaMalloc((void**)&dY, sizeof(float) * M * N));
    CK(cudaMemcpy(dX, hX, sizeof(float) * M * K, cudaMemcpyHostToDevice));
    CK(cudaMemcpy(dW, hW, sizeof(float) * K * N, cudaMemcpyHostToDevice));

    dim3 block(TILE, TILE);
    dim3 grid((N + TILE - 1) / TILE, (M + TILE - 1) / TILE);
    for (int it = 0; it < 5; ++it)      // warmup
        gemm_tiled<<<grid, block>>>(dX, dW, dY, M, K, N);
    CK(cudaDeviceSynchronize());

    cudaEvent_t e0, e1;
    CK(cudaEventCreate(&e0)); CK(cudaEventCreate(&e1));
    CK(cudaEventRecord(e0));
    for (int it = 0; it < iters; ++it)
        gemm_tiled<<<grid, block>>>(dX, dW, dY, M, K, N);
    CK(cudaEventRecord(e1));
    CK(cudaEventSynchronize(e1));
    float ms = 0.0f;
    CK(cudaEventElapsedTime(&ms, e0, e1));
    ms /= iters;
```

```
    CK(cudaMemcpy(hY, dY, sizeof(float) * M * N, cudaMemcpyDeviceToHost));

    // 单线程参考实现（只做一次，O(M*K*N) 在 CPU 上很慢，所以取一个子块验证）
    const int VM = 16, VN = 16;
    double err = 0.0;
    for (int i = 0; i < VM; ++i) {
        for (int j = 0; j < VN; ++j) {
            double acc = 0.0;
            for (int k = 0; k < K; ++k) acc += (double)hX[i * K + k] * (double)hW[k * N + j];
            hRef[i * N + j] = (float)acc;
            err = std::fmax(err, std::fabs((double)hY[i * N + j] - (double)hRef[i * N + j]));
        }
    }

    cudaDeviceProp prop;
    CK(cudaGetDeviceProperties(&prop, 0));
    const double flops = 2.0 * M * K * (double)N;
    std::printf("分片 GEMM: M=%d K=%d Nlocal=%d | grid=(%ux%u)=%u blocks, %d SMs\n",
                M, K, N, grid.x, grid.y, grid.x * grid.y, prop.multiProcessorCount);
    std::printf("时间 = %.3f ms  吞吐 = %.1f GFLOP/s  （%u 个 block / %d SM）\n",
                ms, flops / (ms * 1e-3) / 1e9, grid.x * grid.y, prop.multiProcessorCount);
    const double bytesPerBlock = 2.0 * TILE * K * sizeof(float);   // 每个 block 从 DRAM 读 X 片 + W 片
    const double flopsPerBlock = 2.0 * (double)TILE * TILE * K;
    std::printf("每 block: 全局访存 %.1f KB, 计算 %.1f KFLOP -> 算术强度 %.1f FLOP/B (= TILE/4)\n",
                bytesPerBlock / 1024.0, flopsPerBlock / 1e3, flopsPerBlock / bytesPerBlock);
    std::printf("子块验证 (%dx%d) 最大误差 = %.3e\n", VM, VN, err);
    std::free(hX); std::free(hW); std::free(hY); std::free(hRef);
    cudaFree(dX); cudaFree(dW); cudaFree(dY);
    return 0;
}
```

【代码做什么？】 1. 每个 32×32 的线程块负责输出 `Y` 的一个 32×32 分块，线程 `(tx,ty)` 负责其中一个输出元素 `(row, col)`。 2. 沿 `k` 维循环 `K/TILE` 次：每次把 `X` 的一块（32×32）与 `W` 的一块（32×32）**协作装载**进 shared memory（每个线程搬 1 个元素），`__syncthreads()` 保证装载完成后才开始算；然后每个线程沿 `k` 做 32 次 FMA；再 `__syncthreads()` 保证所有人算完才覆盖共享内存（**双缓冲缺失，这里是两次同步的原因**，也是本 kernel 未优化的点）。 3. 累加器 `acc` 保存在寄存器里，整个 `k` 循环只从 shared memory 读，不回 DRAM——这就是"提高算术强度的分块"。 4. `Xs/Ws` 声明为 `[TILE][TILE+1]` 是经典的 **padding 技巧**：避免 `Ws[k][threadIdx.x]`（按列访问）产生的 bank conflict。 5. 计时用 `cudaEvent`，先 warmup 5 次再测 100 次平均；正确性用 CPU 单线程算 16×16 子块对照。

【并行机制与性能解说】 – **并行层次**：host 启动一个 grid → 每个 SM 分到若干 block → 每个 block 32×32 = 1024 个线程 = 32 个 warp → 每个 warp 32 个 lane 执行同一条 FMA 指令（SIMT/SIMD 执行）。默认参数 `M=1024, K=1024, N=512` 时 `grid = (512/32) × (1024/32) = 16 × 32 = 512` 个 block，即 512 × 1024 = **524288 个线程**；A100 有 108 个 SM，若每 SM 驻留 2 个这样的 block（2048 线程上限），一次只能跑 216 个 block → 需要 **2.4 个 wave**（尾部 wave 只填了 40% 的 SM，tail effect）。 – **Work（总工作量）**：`W = 2·M·K·N = 2·1024·1024·512 = 1.074 GFLOP`（单张分片上的量）。 – **Span（关键路径）**：每个线程串行执行 `K/TILE = 32` 轮，每轮 1 次 shared memory 装载 + 32 次 FMA，因此 `Span ≈ 2K = 2048 FLOP` 加上 `2·K/TILE = 64` 次 `__syncthreads()` 的同步延迟（每次约几百 ns 量级，是真实瓶颈之一）。 – **并行度 = Work/Span** = `2MKN / (2K) = M·N = 524288` 个独立输出元素（理论上界）。硬件可同时驻留的线程 = `SM 数 × 每 SM 线程数 = 108 × 2048 ≈ 221K`，所以并行度**够用但余量只有 2.4×**——这解释了 `TILE` 必须调大（64/128）配合寄存器分块（每个线程算 8×8 个输出）来把"每个 block 的复用次数"提上去，而不是靠堆 block。 – **瓶颈（带宽 vs 计算，用 Roofline 判断）**：每个 block 从 DRAM 读 `2·TILE·K·4 B = 256 KB`，做 `2·TILE²·K = 2.1 MFLOP`，**片内算术强度 = TILE/4 = 8 FLOP/byte**（与 K、M、N 无关，只由 tile 尺寸决定）。 – A100 的 FP32 峰值 19.5 TFLOP/s、HBM 带宽 2039 GB/s → **平衡点 = 19.5e12/2.039e12 = 9.6 FLOP/byte**。 – 8 < 9.6 → 该 kernel 落在**带宽受限区**。预计时间 = 总 DRAM 流量 / 带宽 = `(512 block × 256 KB)/2039 GB/s = 134 MB/2039 GB/s ≈ 66 μs`，而纯计算时间 = `1.074 GFLOP/19.5 TFLOP/s = 55 μs`；取两者较大者 ≈ **66 μs（约 16 TFLOP/s，即 FP32 峰值的 84%）**。（此为本环境无 GPU 时的模型预测值，非实测；运行程序会打印实测 GFLOP/s 供对照。） – 结论：**TILE=32 的 FP32 分块 GEMM 刚好卡在 A100 的 roofline 拐点附近**；要突破就必须（1）加大 TILE + 寄存器分块把 AI 提到 20+，（2）换 TF32/FP16 tensor core 把平衡点推到 100+ FLOP/byte。 – **张量并行切分带来的额外损耗（本示例的核心教学点）**：`N` 被切成 `N/E` 后，grid 的 x 维变成 `N/(E·TILE)`。把 micro-batch 变小（`B·S` 从 1024 降到 64）时： – `M/32` 从 32 降到 2 → grid 变成 `16 × 2 = 32` 个 block，**只有 32/108 = 30% 的 SM 有活干**，设备利用率直接掉到 30%； – 同时每层的 GEMM 耗时缩短到几微秒量级，而张量并行的 AllReduce 延迟（α 项）不随 batch 变小而变小 → **通信/计算比急剧恶化**。 – 这两条正是讲义第 31 页那句 *small micro-batch sizes reduce GPU utilization* 的硬件级解释，也是"张量并行 + 流水线并行"必须小心选 micro-batch 尺寸的原因。

3.3 示例 3：用真实线程模拟 GPipe 流水线，实测气泡比例并与公式对照

这段代码用 `std::thread` + 条件变量邮箱（mailbox）模拟 p 级流水线、m 个 micro-batch 的 GPipe 调度（全前向 → 全反向），统计每级的忙碌时间，从而**实测气泡比例**，并与讲义的 `(p-1)/m` 口径和墙钟时间口径 `(p-1)/(m+p-1)` 对照。

```cpp
// gpipe_sim.cpp -- 用真实线程模拟 GPipe 的 micro-batch 流水线，实测气泡比例
// 编译: g++ -O3 -std=c++17 -pthread gpipe_sim.cpp -o gpipe_sim
// 运行: ./gpipe_sim 8 4 2      # m=8 个 micro-batch, p=4 级, 前向 tf=2 ms (反向 tb=tf)
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <memory>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

using clk = std::chrono::steady_clock;

struct Mailbox {                        // 单生产者/单消费者邮箱（流水线级间的“激活/梯度”通道）
    std::mutex m;
    std::condition_variable cv;
    std::queue<int> q;
    bool closed = false;
    void push(int v) { { std::lock_guard<std::mutex> lk(m); q.push(v); } cv.notify_one(); }
    bool pop(int& v) {
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk, [&] { return !q.empty() || closed; });
        if (q.empty()) return false;
        v = q.front(); q.pop();
        return true;
    }
    void close() { { std::lock_guard<std::mutex> lk(m); closed = true; } cv.notify_all(); }
};

int main(int argc, char** argv) {
    const int m  = argc > 1 ? std::atoi(argv[1]) : 8;   // 一个 mini-batch 切成的 micro-batch 数
    const int p  = argc > 2 ? std::atoi(argv[2]) : 4;   // 流水线级数 (= 设备数)
    const int tf = argc > 3 ? std::atoi(argv[3]) : 2;   // 单级前向耗时 (ms)
    const int tb = tf;                                  // 单级反向耗时 (ms)

    std::vector<std::unique_ptr<Mailbox>> fwd(p), bwd(p);
    for (int s = 0; s < p; ++s) { fwd[s] = std::make_unique<Mailbox>(); bwd[s] = std::make_unique<Mailbox>(); }
    std::vector<double> busy(p, 0.0), wall(p, 0.0);
    std::atomic<int> done_fwd{0};

    auto stage = [&](int s) {
        auto t_begin = clk::now();
        double local_busy = 0.0;
        // ---- 前向：从上一级收 micro-batch，算完转发给下一级 ----
        for (int i = 0; i < m; ++i) {
            int id;
            if (!fwd[s]->pop(id)) break;
            auto b0 = clk::now();
            std::this_thread::sleep_for(std::chrono::milliseconds(tf));   // 模拟本级前向计算
            local_busy += std::chrono::duration<double, std::milli>(clk::now() - b0).count();
            if (s + 1 < p) fwd[s + 1]->push(id); else ++done_fwd;
        }
        if (s + 1 < p) fwd[s + 1]->close();
        // ---- 反向：GPipe 调度下，反向任务在全前向完成后由末端按逆序流入 ----
        for (int i = 0; i < m; ++i) {
            int id;
            if (!bwd[s]->pop(id)) break;
            auto b0 = clk::now();
            std::this_thread::sleep_for(std::chrono::milliseconds(tb));   // 模拟本级反向计算
            local_busy += std::chrono::duration<double, std::milli>(clk::now() - b0).count();
            if (s > 0) bwd[s - 1]->push(id);
        }
        if (s > 0) bwd[s - 1]->close();
        busy[s] = local_busy;
        wall[s] = std::chrono::duration<double, std::milli>(clk::now() - t_begin).count();
    };
```

```
    auto t_all = clk::now();
    std::vector<std::thread> th;
    for (int s = 0; s < p; ++s) th.emplace_back(stage, s);
    for (int i = 0; i < m; ++i) fwd[0]->push(i);      // 主线程投喂 micro-batch 0..m-1
    fwd[0]->close();
    while (done_fwd.load() < p) std::this_thread::yield();  // 等所有级的前向都做完
    for (int i = m - 1; i >= 0; --i) bwd[p - 1]->push(i);   // 反向按逆序投喂（GPipe：全前向→全反向）
    bwd[p - 1]->close();
    for (auto& t : th) t.join();
    const double total = std::chrono::duration<double, std::milli>(clk::now() - t_all).count();

    double busy_max = 0, busy_sum = 0;
    for (int s = 0; s < p; ++s) { busy_max = std::fmax(busy_max, busy[s]); busy_sum += busy[s]; }
    std::printf("m=%d p=%d tf=tb=%dms\n", m, p, tf);
    std::printf("实测总时间      = %.1f ms   (理论 (m+p-1)*(tf+tb) = %.1f ms)\n", total, (double)(m + p - 1) * (tf + tb));
    std::printf("单级忙碌时间    = %.1f ms   (理论 m*(tf+tb) = %.1f ms)\n", busy_max, (double)m * (tf + tb));
    std::printf("实测空闲(气泡)  = %.1f%% 占墙钟时间\n", 100.0 * (total - busy_max) / total);
    std::printf("讲义口径 气泡/理想计算 = (p-1)/m = %.1f%%\n", 100.0 * (p - 1) / m);
    std::printf("时间占比口径 (p-1)/(m+p-1) = %.1f%% ; 平均设备利用率 = %.1f%%\n",
                100.0 * (p - 1) / (m + p - 1), 100.0 * busy_sum / (p * total));
    return 0;
}
```

【代码做什么？】 1. 为每一级 `s` 建两个 `Mailbox`：`fwd[s]` 收"上游传来的激活"，`bwd[s]` 收"下游传来的梯度"。 2. 每级一个线程，先后做两段循环：先做 m 次前向（sleep `t_f` 模拟计算，然后把 micro-batch id 传给 `fwd[s+1]`），再做 m 次反向（sleep `t_b`，把 id 传给 `bwd[s-1]`）。 3. 主线程投喂 micro-batch 0..m-1 给第 0 级；等所有级的前向计数到齐（`done_fwd == p`）后，按**逆序** m-1..0 投喂反向给最后一级——这正是 GPipe 的"全前向 → 全反向"调度。 4. 结束时统计每级忙碌时间与墙钟时间，打印**实测气泡比例**并与两个解析口径对照。

实测输出（本机实跑，`tf=tb=2 ms`）：

```
m=8  p=4  tf=tb=2ms   实测总时间 46.0 ms (理论 44.0)  单级忙碌 33.2 ms (理论 32.0)
                      实测气泡 27.9% | 讲义口径 (p-1)/m = 37.5% | 时间占比 (p-1)/(m+p-1) = 27.3%
m=16 p=8  tf=tb=2ms   实测总时间 96.1 ms (理论 92.0)  单级忙碌 66.3 ms
                      实测气泡 31.0% | 讲义口径 43.8% | 时间占比 30.4%
m=64 p=16 tf=tb=2ms   实测总时间 328.9 ms (理论 316.0) 单级忙碌 264.6 ms
                      实测气泡 19.5% | 讲义口径 23.4% | 时间占比 19.0%
```

实测气泡比例与 `(p-1)/(m+p-1)` 几乎完全一致（27.9% vs 27.3%、31.0% vs 30.4%、19.5% vs 19.0%），而相对"理想计算时间"的口径 `(p-1)/m` 明显偏大——这就是第 2.4 节强调的"分母不同"问题。总时间比理论值高 ~4%，来自线程调度与 mutex/condvar 的开销（模拟器的开销，不是流水线本身的开销）。

【并行机制与性能解说】 – **线程如何创建、工作如何分配**：p 个 `std::thread`（对应 p 张 GPU / p 个 MPI rank），每级只处理"属于自己的 micro-batch 序列"，级间用 `Mailbox`（mutex + condvar）传递数据，等价于真实系统中的点对点 `send/recv` 或 NCCL `Send/Recv`。**共享数据**：`busy[]`、`wall[]` 每个线程写自己的下标——但**它们相邻存储，存在伪共享（false sharing）风险**（8 个 double 共 64 字节，正好一条 cache line）！这里因为只在最后写一次、且不在热点路径上，影响可以忽略；但如果是"每处理一个 micro-batch 就累加一次计数器"，就必须给每个计数器加 padding 或用 `std::atomic` 分散到不同 cache line——这正是 15-418 反复强调的伪共享陷阱。 – **Work（总工作量）**：`W = m·p·(t_f+t_b)`（所有级所有 micro-batch 的计算时间之和，单位"级·毫秒"）= `8·4·4 = 128 级·ms`。 – **Span（关键路径）**：`S = (m+p-1)(t_f+t_b) = 11·4 = 44 ms`（fill + 稳定 + drain，见第 2.4 节的甘特图）。 – **并行度 = Work/Span** = `m·p/(m+p-1) = 32/11 = 2.91`（对 p=4 的理想上界 4 而言 = **72.7% 的利用率上限**），与 `1-(p-1)/(m+p-1) = 8/11 = 72.7%` 完全一致，也与实测平均利用率 71.9% 吻合。 – **可扩展性上限**：`lim(m→∞) m·p/(m+p-1) = p`——**只有把 m 做得足够大，利用率才趋近 100%**；而 `lim(m 固定, p→∞) = m`——流水线越深，需要越多的 micro-batch 才能填满。两个方向的代价分别是"micro-batch 太小 → GPU 打不满（见 3.2 节）"与"级数太多 → 每级只有极少层、通信占比上升"。 – **瓶颈**：这个模拟器的瓶颈是**主线程的串行投喂与 yield 自旋**，以及 condvar 唤醒的调度延迟（这就是为什么实测比理论慢 4%）。在真实系统里，对应的瓶颈是**点对点通信的启动延迟与带宽**（第 4.5 节给了数值），以及"所有级必须等最慢的级"——**负载不均（stage 划分不等长）会让真实气泡远大于公式值**，因为公式假设所有级的 `t_f/t_b` 相同。

---

4. 性能模型与复杂度分析

4.1 三个代码示例的 Work / Span / 并行度汇总

示例	Work（总工作量）	Span（关键路径）	并行度 = Work/Span	瓶颈
3.1 张量并行 MLP（E 个 rank × 4 线程）	`2MKN + 2MNO ≈ 33.9 MFLOP`（= 单 rank 串行耗时 **5.95 ms**，按实测 5.7 GFLOP/s）	`W/(E·P) + 2(E−1)·α + 2(E−1)/E·(MO·4)/BW`；E=8 → **0.78 ms**；E=32 → **0.32 ms**	E=8：`5.95/0.78 ≈ 7.6`（理想 8，效率 96%）；E=32：`5.95/0.32 ≈ 18.9`（理想 32，效率 59%）	小 E：朴素 GEMM 的 IPC；大 E：集合通信延迟 α（E=32 时通信占 69%）
3.2 分片 tiled GEMM（CUDA）	`2MKN = 1.074 GFLOP`（单分片）	`≈ 2K + 2(K/TILE)·t_sync ≈ 2048 FLOP + 64 次同步`	`M·N = 524288`（硬件可并发 ~221K 线程）	带宽受限（AI=8 < 平衡点 9.6）；2.4 个 wave 的尾部浪费
3.3 GPipe 流水线模拟器	`m·p·(t_f+t_b) = 128 级·ms`	`(m+p−1)(t_f+t_b) = 44 ms`	`m·p/(m+p−1) = 2.91`（上界 4）	气泡 `(p−1)/(m+p−1)`；级间负载不均

4.2 数值算例 A：张量并行的通信–计算比（为什么 TP 必须留在 NVLink 域内）

假设（A100–80GB 单节点 8 卡，NVLink 单向 300 GB/s，BF16 训练）：`B·S = 8192` 个 token（micro-batch 8 × 序列 1024）、隐层 `H = 4096`、TP 度 `E = 8`、每 GPU 有效算力 **150 TFLOP/s**（A100 BF16 tensor core 峰值 312 TFLOP/s 的 48%，是千亿模型上比较现实的 MFU）。

计算量（每个 Transformer 层，每卡）：– 注意力投影（QKV + 输出投影）：`4 × 2·B·S·H² = 8·8192·4096² = 1.10e12` FLOP – MLP（`H→4H→H`）：`2 × 2·B·S·H·4H = 16·8192·4096² = 2.20e12` FLOP – 层合计 `3.30e12` FLOP → **每卡** **`3.30e12/8 = 412 GFLOP`** → 计算时间 `= 412e9/150e12 =` **2.75 ms**

通信量（每层 2 次 AllReduce：注意力输出后、MLP 输出后）：– 单次 AllReduce 消息 = `B·S·H` 个元素 × 2 B = `8192×4096×2 = 67.1 MB` – ring AllReduce 每卡收发量 = `2(E−1)/E × 67.1 MB = 117 MB`；两次共 **235 MB** – 传输时间 `= 235 MB / 300 GB/s = 0.78 ms`；再加 `2 × 2(E−1) = 28` 步的启动延迟（NVLink 域内 ~3 μs）= 0.084 ms → **通信 ≈ 0.87 ms**

结论：通信/计算 = `0.87/2.75 = 32%`，即**约 24% 的层时间花在 AllReduce 上**（`0.87/(2.75+0.87)`）。这还是在 300 GB/s 的 NVLink 上。**如果把同一份张量并行放到跨节点（IB HDR 25 GB/s）**：传输时间变成 `235 MB/25 GB/s = 9.4 ms`，是计算时间的 **3.4 倍**——层时间从 3.6 ms 劣化到 12.2 ms，**整个训练慢 3.4 倍**。这就是"张量并行绝不出节点"的定量依据（12 倍带宽差 → 3.4 倍端到端劣化）。

更重要的结构性结论：把两个比例写成公式，`通信时间/计算时间 = [4(E−1)/E·B·S·H·2/BW] / [24·B·S·H²/(E·P)] = 8(E−1)·P/(24·H·BW)` —— **它与 batch（B·S）完全无关，只与并行度 E、隐层 H、以及"算力/带宽比"有关**。所以"把 micro-batch 调大"**不能**降低张量并行的相对通信开销（它只能改善 GEMM 的硬件效率）；要降低它只能：**增大 H**（用更大的模型换效率）、**降低 E**、或**换更快的链路**。这是一个非常容易被误解的性能事实。

4.3 数值算例 B：流水线气泡与配置选择

设 `t_f = t_b = 1`（归一化时间单位），两级口径的数值对照：

p	m	讲义口径 气泡/理想计算 `(p−1)/m`	墙钟口径 `(p−1)/(m+p−1)`	设备利用率 `m/(m+p−1)`	总时间 `(m+p−1)(t_f+t_b)`
2	8	12.5%	11.1%	88.9%	18
4	8	37.5%	27.3%	72.7%	22
4	32	9.4%	8.6%	91.4%	70
8	16	43.8%	30.4%	69.6%	46
8	64	10.9%	9.9%	90.1%	142
16	64	23.4%	19.0%	81.0%	158
16	256	5.9%	5.6%	94.4%	542

由表得到的实用规则：– 想让气泡（墙钟口径）低于 **10%**，需要 `m ≥ 9(p−1)`。p=8 → `m ≥ 63`（即 **64 个 micro-batch**）；p=16 → `m ≥ 135`（**128 还不够，要 144**）。 – 结合 3.2 节的结论：micro-batch 太小会让 GPU 打不满，所以不能靠"无限切细"来增大 m，只能靠**增大 mini-batch 总量**——而大 batch 可能损害收敛精度（讲义第 31 页的 caveat）。**这就是流水线并行的核心两难**：气泡要大 m，大 m 要大 batch 或小 micro-batch，两者都有代价。 – 用 GPipe 的调度时，**激活显存 ∝ m**（必须保存所有 micro-batch 的中间激活直到反向）。m=64、每个 micro-batch

的激活约 100 MB（`B_micro=8, S=1024, H=4096` 量级）时，光激活就要 **6.4 GB/卡**；1F1B（前向反向交错）把这部分降到 ≈ p 个 micro-batch 的激活（~1 GB），这是工业界普遍不直接用 GPipe 而用 1F1B 的原因。

4.4 数值算例 C：Roofline —— micro-batch 变小如何把 GEMM 推进带宽受限区

考虑 MLP 的第一个线性层（列并行分片，`K = H = 4096`、`N = 4H/E = 2048`，BF16），比较两个 micro-batch 规模：

(a) 小 micro-batch：`B·S = M = 64` – FLOP = `2·M·K·N = 2·64·4096·2048 = 1.074e9` – 强制 DRAM 流量（每个权重只读一次）：权重 `2·4096·2048 = 16.8 MB` + 输入 `2·64·4096 = 0.52 MB` + 输出 `2·64·2048 = 0.26 MB` ≈ **17.6 MB** – 算术强度 = `1.074e9/17.6e6 =` **61 FLOP/byte** < A100 平衡点 153 → **带宽受限** – 时间 = `17.6 MB/2039 GB/s = 8.6 µs` → 有效算力 = `1.074e9/8.6e-6 =` **125 TFLOP/s（峰值的 40%）**

(b) 大 micro-batch：`B·S = M = 8192` – FLOP = `2·8192·4096·2048 = 1.374e11` – DRAM 流量 = 权重 16.8 MB + 输入 67.1 MB + 输出 33.6 MB = **117 MB** – 算术强度 = `1.374e11/1.17e8 =` **1174 FLOP/byte** ≫ 153 → **计算受限** – 时间 = `1.374e11/312e12 = 0.44 ms`（峰值）或 `0.92 ms`（150 TFLOP/s 有效）

结论：micro-batch 缩小 128 倍后，**有效算力从计算受限时的高位掉到带宽受限的 40% 峰值**（算术强度掉到平衡点以下）。这就是"流水线并行需要用大 m 来消气泡，但每个 micro-batch 又不能太小"的定量冲突；也是"张量并行要配合大 batch"的根本原因。工程上的缓解手段：算子融合（CS149：算术强度 1/3 → 3/5，flash-attention 把 `5MN+2M` 次读降到 `MN`）、低精度（流量按位宽线性下降）、以及把权重常驻片上（CUDA graph / persistent kernel）。

> 更完整的口径：情形 (a) 落在带宽受限区，天花板 = `2039 GB/s × 61 FLOP/B = 125 TFLOP/s`（峰值的 40%）；情形 (b) 落在计算受限区，能跑到 GEMM 实现质量所允许的任何水平（本笔记用 150 TFLOP/s = 48% 作为保守值；成熟实现的大 GEMM 可到 275 TFLOP/s ≈ 88% 峰值）。**关键不是绝对数字，而是"算术强度 61 vs 1174 跨越了 153 这个平衡点"**——一旦跨过去，硬件再快也救不了。
>
> 补充（CS149 的同类数字，供对照）：V100 有 6 MB L2、16 GB HBM、900 GB/s 带宽、80 个 SM；一个 `N=32, P=Q=256` 的卷积层输出就有 `256×256×128×32 = 256M` 个输出 = **1 GB 的 float32 输出数据**——单层激活就逼近整个显存容量，这直接说明为什么"激活"必须被切分（数据并行切 batch、张量并行切通道、流水线并行切层）以及为什么融合能省下成百 MB 级的 DRAM 流量。

4.5 数值算例 D：带宽/延迟（α–β）模型与"通信频率决定链路"

设一次跨设备传输 `n` 字节的时间为 `T = α + n/BW`。**统一假设**：模型 96 层、`H = 4096`（每层参数 ≈ `12H²`：注意力 4 个 `H×H` 投影 + MLP 的 `8H²`，全模型 ≈ 19B 参数）、BF16、总规模 64 卡（TP8 × PP8）外加 DP = 8 副本、单个 micro-batch = 8192 个 token、每卡有效算力 150 TFLOP/s。一个 iteration 内三类通信的特征如下：

并行维度	单次消息大小	频率	每 iter 每卡总量	链路与时间	占该 iter 计算时间的比例
张量并行（E=8）	激活 `B·S·H·2 B = 8192×4096×2 = 67 MB`	**每层 2 次**，96 层 → 192 次	`192 × 2·7/8 × 67 MB ≈ 22.5 GB`	NVLink 300 GB/s：**75 ms**（另加 192 次 AllReduce 的启动延迟 `192×14×3 µs ≈ 8 ms`，属保守上界，实际与传输重叠）	`75/264 ≈ 28%`
流水线并行（p=8）	级边界传激活 `67 MB`	每 micro-batch 每**级边界** 1 次（可与计算重叠）	`(p−1)=7 × 67 MB = 0.47 GB`	IB HDR 25 GB/s：**19 ms**	`19/264 ≈ 7%`
数据并行（DP=8）	梯度 = 每卡参数 `19.3e9/64 × 2 B = 0.60 GB`	**每 iter 1 次**（可与反向重叠）	`2·7/8 × 0.60 GB ≈ 1.05 GB`	IB HDR 25 GB/s：**42 ms**	`42/264 ≈ 16%`

其中该 iteration 的计算时间 = `24·B·S·H²·L / (E·P) = 24·8192·4096²·96/(8·150e12) =` **264 ms**（每卡），与第 4.2 节的每层记账一致（`24·B·S·H²/E = 412 GFLOP/层/卡 ÷ 150 TFLOP/s = 2.75 ms/层 × 96 层 = 264 ms`）。

读表要点： 1. **张量并行的通信总量最大（22.5 GB/iter/卡）且消息最"碎"（192 次）、对 α（延迟）最敏感** → 必须 NVLink。注意它的总量之所以吓人，是因为它**每层都发生**；单看一条消息（67 MB）其实很小。若把同一批 token 切成 m 个 micro-batch，TP 的单条消息会缩小到 `1/m`、次数变成 `m` 倍，总量不变但 α 项比重上升——这也说明 micro-batch 尺寸与 α 敏感度是耦合的。 2. **数据并行的总量看似也大（1.05 GB/卡），但一个 iteration 只发生一次**，且 ring AllReduce 可以拆成 ReduceScatter + AllGather 与反向计算重叠（前一讲结论：ring AllReduce 每卡只发 `2(E−1)/E·M`）→ 放在最慢的链路上也往往够用。 3. **流水线并行的通信量与 micro-batch 数 m 成正比**（每级边界每 micro-batch 一次）：这里 m=1 时只有 0.47 GB；若把同一个 8192-token 的 batch 拆成 m=8 个 1024-token 的 micro-batch，量仍是 0.47 GB（因为每块变小、块数变多），但**消息数变成 56 条、每条 8.4 MB**，α 的相对代价上升——这是"micro-batch 不宜太小"的又一个理由。 4. **Amdahl 视角**：假设 PP 与 DP 的通信能被计算完全掩盖，而张量并行的 75 ms AllReduce 串在每层的关键路径上无法掩盖，则一个 iteration 里 `75/(264+75) = 22%` 的时间是"无法被计算掩盖"的通信开销。这 22% 就是 Amdahl 意义上的串行段：**即使把计算部分加速到无穷，也只是让这 339 ms 变成 75 ms（最多再**

快 4.5 倍)；等价地说，该 iteration 的有效算力利用率被锁在 `264/339 ≈ 78%`。因此工业实现必须做通信/计算重叠（把 AllReduce 拆成 ReduceScatter/AllGather 与相邻 GEMM 并行、或使用异步流水线与 1F1B），否则任何并行策略都会撞上这堵 Amdahl 墙。

4.6 复杂度总览

策略	单卡显存（参数）	单卡计算量	通信量口径（每卡）	可扩展性上限
数据并行（N 卡）	`M_params`（全量）	`W/N`	`2(N-1)/N·M_params`（ring，每 iter 1 次）	受单卡容量限制（模型装不下就走不通）
张量并行（E 卡）	`M_params/E`	`W/E`	每层 2 次 AllReduce，每次 `2(E-1)/E·B·S·H`	受 α、带宽限制（一般 E ⩽ 8，不出节点）
流水线并行（p 级）	`M_params/p` + 激活 ∝ m	`W/p`	`(p-1)·m × B_micro·S·H`（每 iter）	利用率上限 `m/(m+p-1)`；m 受 batch 与显存限制
3D 并行（E·p·N 卡）	`M_params/(E·p)`	`W/(E·p·N)`	三者叠加，但各自可用合适的链路	**容量上界 × 并行度上界 = 真实可用规模**

---

5. 关键要点

1. **并行策略的选择本质上是"什么资源先耗尽"的问题**：数据并行解决不了显存墙（每卡全量副本），张量并行解决显存与单层计算量，流水线并行把模型并行的空转填回吞吐。讲义第 33 页的收口结论是：**训练超大模型必须叠加数据/模型/流水线等多种并行技术**（DeepSpeed 3D 并行 = DP × PP × TP）。
2. **通信量由"切什么"决定，通信频率由"切在哪一层"决定**：数据并行通信量 = 参数尺寸（`C_out·C_in`，每 iter 一次）；张量并行通信量 = 激活尺寸（`B·C_in` 或 `B·C_out`，每层 2 次），量级小 ≈ `C_out/B` 倍（`C_in ≈ C_out` 时即 `C_in/B`，取 `C_in = 4096, B = 8` 就是 512 倍），但频率高 `层数 × 2` 倍。**频率决定可用链路**：最高频的 TP 必须留在 NVLink 域内（300 GB/s vs IB 25 GB/s，代价是 3.4 倍端到端劣化）。
3. **张量并行的通信-计算比与 batch 无关**，只与 `(E-1)·P/(H·BW)` 有关：调大 micro-batch 无法降低 TP 的相对开销，只能改善 GEMM 的硬件效率（Roofline）。要降低 TP 开销只有三条路：增大 H、降低 E、或换更快的链路。**这条结论是"大模型训练用大隐层 + 小 TP 度 + 大 batch"的定量依据。**
4. **流水线的气泡 `(p-1)/(m+p-1)` 是典型的 Amdahl 串行段**：fill 与 drain 无法并行。要把它压到 10% 以下需要 `m > 9(p-1)`（p=4 → m ⩾ 28；p=8 → m ⩾ 64），而 m 的增大要么靠更大的 mini-batch（可能损害收敛），要么靠更小的 micro-batch（把 GPU 推进带宽受限区，算术强度从 1174 掉到 61 FLOP/byte，有效算力掉到峰值的 40%）。**必须同时看这两个约束才能选对 m。**
5. **切分粒度受硬件并行度的硬约束**：TP 把 N 切成 N/E 后，CUDA grid 的 block 数按 $1/E^2$ 量级下降——`M=64, N=512` 的分片只有 32 个 block，而 A100 有 108 个 SM，**70% 的 SM 闲置**。所以每一层并行的最小切分尺寸要保证"分片后每张卡仍有足够多的 tile 填满 SM"，这也是 `TILE` 要调大、要用寄存器分块与 tensor core 的原因。

---

6. 常见陷阱与注意事项

- **把模型并行当成"加速手段"**：模型并行（无 micro-batch）在时间轴上只有一台设备在算（讲义第 28 页：*Under-utilization of compute resources / Low overall throughput*），加设备只会加长等待。它是**容量手段**；吞吐要靠流水线并行或数据并行来补。评估任何"模型并行方案"时，先问"稳态时每张卡在干什么"。
- **GeLU/softmax 等非线性之前做归约（数学错误 + 性能错误）**：$\mathrm{GeLU}(X\cdot A_1) + \mathrm{GeLU}(X\cdot A_2) \neq \mathrm{GeLU}(X\cdot A)$。所以列并行（partition output）之后**不能**立刻 AllReduce 成完整输出，必须让 $Y = [Y_1|Y_2]$ 保持**拼接**形态、把通信推迟到第二层 GEMM 之后。把 AllReduce 放在 `4H` 宽的中间激活上而不是 `H` 宽的层输出上，通信量会**大 4 倍**——这就是 Megatron 选择"列并行在前、行并行在后"的两个理由（见 Q2）。
- **只用"平均 batch"估算显存，忽略激活与优化器状态**：显存 = 参数 + 梯度 + 优化器状态（FP32 的 master weights + Adam 的 m、v 共 12 B/参数）+ **激活（∝ m、∝ 序列长）**。175 B 模型即使用 `TP 8 × PP 8`（64 卡）切分，每卡仍有 2.73 B 参数 → FP16 权重 5.5 GB + FP16 梯度 5.5 GB + 优化器状态 32.8 GB ≈ **43.8 GB**，距 80 GB 只剩一半给激活与临时缓冲；如果这一层外面再套数据并行，优化器状态还会在 DP 组内**被复制**（这正是 ZeRO 要消掉的冗余）。
- **忽略通信的"频率 × 延迟"而只算总字节数**：数据并行每 iter 传 1 GB 量级的梯度却几乎不伤性能（一次大消息、只发生一次、可重叠）；张量并行每层传 67 MB 却可能吃掉 24% 的层时间（96 层 × 2 次小消息、串在关键路径上）。**只看总字节会把两者的代价完全看反**；必须同时算 α 项（每次 AllReduce 有 `2(E-1)` 步启动延迟）与重叠可能性。

- **伪共享（false sharing）与同步粒度错误**：流水线/张量并行的 worker 常常维护"每个 micro-batch 一个计数器/累加器"，若这些变量相邻存放（例如 `std::vector<double> busy(p)`，8 个 double 正好占满一条 64 B cache line），每个线程的写都会把其他核的 cache line 置为 Invalid，性能崩溃。解决：padding 到 64 B、或按 cache line 对齐的 per-thread 副本最后再归约；同时避免在最内层循环里用粗粒度锁（应像 3.3 节的 mailbox 一样只在级边界同步一次）。
- **把理论气泡当成"能消除的浪费"**：`(p-1)/(m+p-1)` 是调度的固有下界；更常见的问题是**级间负载不均**（各层计算量不同，划分不等长），此时真实气泡会显著大于公式值——公式假设所有级的 `t_f/t_b` 完全相同。工程上要么按 FLOP 均衡划分（让每级层数不同），要么用 interleaved/virtual stages 把粒度打碎；此外 `p` 越大，**单样本端到端延迟 ≈ p·(t_f+t_b)** 越大，训练吞吐与推理延迟的取舍需要分开考虑。

---

7. 思考题（带答案）

Q1（定量）：某模型用 **p = 4** 级流水线、每个 mini-batch 切 **m = 8** 个 micro-batch，每级 `t_f = t_b`。(a) 按讲义口径算气泡比例；(b) 算气泡占墙钟时间的比例与设备利用率；(c) 如果保持 p = 4、希望墙钟气泡低于 10%，m 至少要多大？ (d) 如果把流水线加深到 p = 8 并保持墙钟气泡 10% 以下，m 又要多大？请指出这里的实际矛盾。

【答案】 (a) 讲义第 30 页口径：`BubbleFraction = (p-1)(t_f+t_b)/(m(t_f+t_b)) = (p-1)/m = 3/8 =` **37.5%**（分母是"理想计算时间" `m(t_f+t_b)`）。(b) 墙钟总时间 = `(m+p-1)(t_f+t_b) = 11(t_f+t_b)`；气泡时间 = `(p-1)(t_f+t_b) = 3(t_f+t_b)` → 气泡占墙钟 = `3/11 =` **27.3%**，设备利用率 = `m/(m+p-1) = 8/11 =` **72.7%**（与 3.3 节的实测 27.9% / 71.9% 吻合）。(c) 要求 `(p-1)/(m+p-1) < 0.1` → `3 < 0.1m + 0.3` → `m > 27` → **m ⩾ 28**。(d) p = 8 时 `7/(m+7) < 0.1` → `m > 63` → **m ⩾ 64**（正好是 2 的幂，工程上取 64）。 矛盾在于：m 翻倍后每张卡上每个 micro-batch 的 GEMM 的 M 维不变（m 是"批次个数"，不是 micro-batch 尺寸），要增大 m 就必须**增大 mini-batch 总量或缩小 micro-batch 尺寸**。前者可能损害收敛精度（讲义第 31 页 caveat：*large mini-batch sizes can lead to accuracy loss*），后者会让每层 GEMM 变小、把 GPU 推入带宽受限区（第 4.4 节：算术强度从 1174 掉到 61 FLOP/byte，有效算力掉到峰值的 40%；第 3.2 节：micro-batch 变小后 grid 只剩 32 个 block，只占 108 个 SM 的 30%）。另外 GPipe 的激活显存 ∝ m，p=8、m=64 时激活显存约为 p=4、m=32 时的 2 倍——所以工业界改用 1F1B 交错调度把激活降到 ≈ p 个 micro-batch。

Q2（理解 + 设计）：Megatron-LM 在 Transformer 的 MLP 里把第一个全连接层的权重**按列切**（partition output，f 算子），第二个全连接层**按行切**（reduce output，g 算子），中间夹一个 GeLU。讲义第 21 页特意问了一句 "Why switch?"。请回答：为什么必须是这个顺序？能否反过来（第一层按行切、第二层按列切）？如果用 `H` 表示隐层、`4H` 表示中间维度、`B·S` 表示 token 数，两种顺序的通信量各是多少？多头注意力的头切分属于哪一种？

【答案】 必须是"列并行 → 行并行"，两个理由： 1. **数学上**：GeLU 是非线性逐元素函数，`GeLU(Y₁) + GeLU(Y₂) ≠ GeLU(Y₁+Y₂)`。反过来的话，第一层（行并行）算出的只是**部分和**，在送入 GeLU 之前就必须先 AllReduce 成完整的 `Y`——即在**中间激活**上通信，通信量为 `B·S·4H` 个元素。 2. **通信量上**：按 Megatron 的顺序，中间激活 `Y = [Y₁|Y₂]` 保持**拼接**形态不需要通信（列并行是 partition output），通信被推迟到第二个 GEMM 之后，作用在**层输出** `Z` 上，通信量 = `B·S·H` 个元素（讲义第 13 页小结表：reduce output 的通信量是 `B·C_out`）。两者之比 = `4H/H =` **4 倍**。反过来做要付 4 倍的通信。 附带的好处还有：`Z = Z₁+Z₂` 的 AllReduce 结果正好是下一层的输入，可以与下一层的计算/通信重叠；而前向的 AllReduce 在反向时对应 identity（讲义第 10—12 页的两张通信量表：partition output 在反向通信、reduce output 在前向通信），**f 与 g 在前后向的角色是对偶的**，这让实现可以复用同一套通信调度。 **多头注意力**：按 head 切分输入投影，每个 head 独立算 `Yᵢ = softmax(QᵢKᵢᵀ/√d)Vᵢ`，最后 `Z = Concat(Y₀,…,Y_h)W_o` —— `W_o` 按行切、输出需要 AllReduce，所以**属于 reduce output（行并行）**（讲义第 25 页标注：*Parallelizing across attention heads + Tensor model parallelism (reduce output)*）。多层堆叠时，`W_o` 的行并行输出经 AllReduce 变为完整 `Z`，与下一层的列并行（partition output，前向无需通信）相接——**整个 Transformer 就是"列并行 ↔ 行并行"交替的链条，把 AllReduce 放在每层的输出端（尺寸 H）而不是中间端（尺寸 4H）**。

Q3（系统设计）：你要在 **512 张 80 GB A100** 上训练一个 175 B 参数的 BF16 Transformer（96 层、`H = 12288`），要求 (1) 模型状态装得下、(2) 单个 iteration 的流水线气泡不超过 ~15%、(3) 张量并行不跨节点（每节点 8 卡 NVLink）。请给出 TP / PP / DP 的配置，说明理由，并算出每张卡上的模型状态显存与全局 batch size。

【答案】 配置：TP = 8，PP = 8，DP = 8（512 = 8 × 8 × 8）。 – **TP = 8 且必须放在同一节点内**：TP 每层要 2 次 AllReduce，消息大小 `B·S·H`，只有 NVLink（300 GB/s）能承受（第 4.2 节：换到 IB 25 GB/s 会让层时间劣化 3.4 倍）。8 卡正好吃掉一个节点的 NVLink 域，所以 TP = 8 是上限而不是巧合。 – **PP = 8**：`H = 12288` 很大，TP 的通信-计算比 `8(E-1)P/(24·H·BW)` 随 H 增大而下降，所以 TP 不需要更宽；把层数（深）交给流水线：96 层 / 8 级 = **每级 12 层**，级数少意味着级间通信（每 micro-batch 传 `B_micro·S·H·2` 字节）频率低、可重叠。 – **DP = 8**：剩下的并行维度，用数据并行提高吞吐；它的梯度 AllReduce 每 iter 只一次、量最大但可重叠，可以跨节点走 IB。 – **每卡模型状态**：参数 = `175e9/(TP×PP) = 175e9/64 = 2.73e9`。BF16 权重 = 2.73e9 × 2 B = **5.5 GB**；BF16 梯度 = **5.5 GB**；FP32 master weights + Adam 的 m、v（共 12 B/参数）= **32.8 GB**；合计 ≈ **43.8 GB < 80 GB**，剩下的 ~36 GB 给激活、通信缓冲与临时张量。（这正是 ZeRO 在数据并行维度上继续切分优化器状态的动机：这 32.8 GB 在 DP 组的 8 个副本里是重复的。） – **气泡**：取 m = 32 个 micro-batch → 墙钟气泡 `(p-1)/(m+p-1)`

`= 7/39 =` **17.9%**；取 **m = 48** → `7/55 = 12.7%`（满足 ≤15%）。工程上 m 通常取成流水线级数 p 的倍数以便均衡调度（如 4 的倍数）；实际训练多用 1F1B 调度而非 GPipe 以省激活显存。 – **全局 batch**：`global_batch = micro_batch_size × m × DP = 1 × 48 × 8 =` **384 个序列**（若 `S = 2048` token，则约 78.6 万 token/step）。这里 micro-batch = 1 是为了在 PP = 8、TP = 8 下把激活显存压到最低——代价是每卡 GEMM 的 M 维只有 `B·S = 2048`，必须靠 tensor core + 大 TILE 才能维持效率（第 4.4 节的 Roofline 分析）。 – **一句话检验**：`TP×PP×DP = 512` 决定了容量上界（`175B/64 = 2.73 B` 参数/卡）与吞吐上界（`(p-1)/(m+p-1)` 的气泡 + TP 每层通信）；**三层并行各自服务于一个约束，缺一不可**——这正是讲义最后一页 "3D parallelism in DeepSpeed" 的含义。

Lecture 24: AI in System Design

1. 章节标题与概述

Lecture 24: AI in System Design（用机器学习替换系统里的手写启发式：learned index、learned cost model、学习型调度与自动调优）

- **本讲核心问题**：过去四十年，计算机系统的每一个"该怎么做"的决定——缓存该替换哪一行、DRAM 控制器该先服务哪个请求、编译器该给循环选多大的分块、运行时该把迭代交给哪个 lane / 哪个线程、推理框架该把多少个算子融合成一个 kernel——都是**人写死的启发式（heuristic）**。本讲问的是两件事：**（1）当数据（遥测、访存轨迹、性能计数器、编译日志）变得廉价而模型变得强大时，能不能把这些启发式换成一个从数据里学出来的模型（learned policy / learned cost model）？ （2）要让这种"用 AI 设计系统"的做法真正落地，代价是什么？**——最关键的代价是：学习到的策略必须在**它所优化的那台系统上、以远低于它想省下的开销**运行（例如手机 SoC 上一次访存的能耗预算只有约 10 nJ），于是学习型策略的**推理成本（推理 FLOP、模型体积、SRAM 占用、决策频率）**本身成了一个必须做 work-span / Roofline / 能耗分析的系统设计问题。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：**HBM（High Bandwidth Memory）与 3D 堆叠 + TSV**（把内存搬到处理器旁边，1024-bit/stack 接口，H100 达 3.2 TB/s）；**DRAM bank / row buffer / burst mode** 与**内存控制器的请求调度**（`FR-FCFS` 启发式、每 bank 请求队列、请求合并）；**脉动阵列（systolic array）**（TPU 风格稠密矩阵乘，无指令发射开销）；**数据流架构 RDU**（SambaNova SN40L：1,040 个 PCU/PMU、638 TFLOPS bf16、520 MB 片上 SRAM、64 GB HBM、1.5 TB DDR，用 mesh 开关 + AGCU 做空间调度）；以及片上 SRAM / LPDDR 之间**相差一到两个数量级**的访问能耗（26 pJ vs 1200 pJ per 64 bit）。
- **软件侧**：把问题映射到机器的四步法——**分解（decomposition）→ 分配（assignment）→ 编排（orchestration）→ 映射到硬件（mapping）**；**SPMD / SIMD 两层模型**（ISPC 的 gang + task）；`foreach` 的四种实现（单实例、交错 interleaved、分块 blocked、动态原子分配）；**metapipelining（流水线的流水线）+ 双缓冲 + token 控制的无锁数据流**；以及**自动调优 / 代价模型**（分块尺寸、线程数、流水级数、并行维度的选择）。
- **在并行计算知识体系中的角色**：这是全课程里唯一一讲把课程前面所有的"性能机制"倒过来当**被优化的对象**来看的讲次。前面我们学 Roofline、算术强度、SIMD、缓存分块、屏障与锁、reduce、work-span，是为了**人**去写出并行程序；本讲把这一整套分析变成**学习型策略的代价模型与评价指标**：无论是"学习型索引要不要多查一层模型""预取器该不该在每次 L2 miss 上跑一个小 MLP""32 卡时 allreduce 要不要和 GEMM 重叠"，答案都要用 work-span、算术强度、带宽/延迟公式和能耗预算算出来。同时它也是课程后半段（parallel deep learning、AI datacenter、heterogeneity / specialization）的收口：**AI 系统的硬件给"用 AI 设计系统"提供了算力底座与片上存储，而 AI 系统的软件栈本身又充满了等待被学习型策略替换的启发式**（分块尺寸、融合决策、并行维度、DRAM 调度）。
- **配套材料**：
- **本讲（CMU 15-418/15-618, Fall 2026, Oct 28, "AI in System Design"）在 Fall 2026 尚未发布讲义**：主题是 Fall 2026 新增的，`https://www.cs.cmu.edu/~418/lectures/` 之下**没有**对应的公开 PDF；历史学期在 `/afs/cs/academic/class/15418-*/public/` 之下的讲义需要 **CMU 登录，属未公开**。因此本笔记**不引用任何 Fall 2026 的幻灯片编号**，讲义侧的事实基础全部来自下面两份**已公开**的补充材料与公开文献。
- `cs149_supp/aidatacenter.txt`（**已公开**，Stanford CS149 Fall 2025 **Lecture 12: Mapping AI Applications to the AI Datacenter**，共 **72 页**）。它是本讲的**硬件底座**：HBM / 3D 堆叠 / TSV（slide 5、68）、HBM 带宽演进（Fury 512 GB/s → P100 720 GB/s → H100 3.2 TB/s，slide 70）、DRAM 阵列 / row buffer / precharge / RAS / CAS / burst / bank（slide 50–57）、内存控制器就是请求调度器且默认策略是 **FR-FCFS**（slide 63）、DDR4-2400 双通道 38.4 GB/s 与 ~13 ns CAS（slide 65）、能耗数字（1 pJ / 20 pJ / 26 pJ / 1200 pJ，slide 46；0.9 pJ / 5 pJ / 640 pJ，slide 47）、脉动阵列与数据流 RDU（slide 11–13）、metapipelining 与 kernel fusion（slide 16–26）、AI 模型里的并行维度 DP/TP/PP/EP/SP/CP 及通信原语 AllReduce / ReduceScatter / AllGather / All-to-All（slide 33–38）、**计算-通信重叠的量化结果（8/16/32 socket 的 88.5%/77%/52% vs 实测 72%/75%/79%，slide 41）**。

- `cs149_supp/thoughtprocess.txt`（**已公开**，Stanford CS149 Fall 2025 **Lecture 4: Parallelizing Code: The Programming Thought Process**，共 **74 页**）。它是本讲的**软件决策空间**：创建并行程序的四步法（slide 30）、Amdahl 定律与 Summit 的 148,635,648 个 ALU（slide 32、36–37）、ISPC 的 SPMD 抽象与 SIMD 实现（slide 3、7、20–23）、**`foreach` 的四种实现**（slide 13）、interleaved 用 `vmovaps` 打包载入 vs blocked 用 `vgatherdps` gather（slide 10–11）、跨实例原语 `reduce_add` / `broadcast` / `rotate`（slide 18–19）、静态分配（C++11 `std::thread`）与动态任务分配（ISPC `launch[100]` + worker 线程池，slide 40–42）、red-black 染色重排与屏障/锁的成本（slide 50–72）。
- 关于讲义首页写着 "Stanford CS149, Fall 2025"：这是补充材料的来源标注，**不是错误**；两份补充材料都是 CS149 的公开讲义，与本课 Fall 2026 的讲次编号无对应关系。
- **未发布 / 需登录**：本讲（Lecture 24）讲课录像在 Fall 2026 日程表中被注释隐藏，属**未发布**；Ed 讨论区、Autolab、Canvas 均**需登录**。同为 Fall 2026 尚未发布讲义的还有 Performance Analysis/Profiling、Transactional Memory 等。
- **课程语境**：Fall 2026 授课教师为 **Brian Railing** 与 **Dimitrios Skarlatos**；课程由 **Kayvon Fatahalian** 创建。课程主页 https://www.cs.cmu.edu/~418/，日程表 https://www.cs.cmu.edu/~418/schedule.html。
- 本笔记引用的公开文献（属成熟公开知识，非课程特定内容）：Kraska et al., *The Case for Learned Index Structures*, SIGMOD 2018；Marcus et al., *Benchmarking Learned Indexes*, VLDB 2021；Ferragina & Vinciguerra, *The PGM-index*, PVLDB 2020；Marcus et al., *Bao: Making Learned Query Optimization Practical*, SIGMOD 2021；Hashemi et al., *Learning Memory Access Patterns*, ICML 2018 与 *Voyager*, MICRO 2021；Han et al., *EIE*, ICLR 2016（能耗数字来源，讲义 slide 47 同时引用）。
- **材料的公开状态一览（务必分清"本讲"与"补充材料"）**

材料	位置 / 形态	公开状态	在本笔记中的作用
Lecture 24 "AI in System Design" 讲义（Fall 2026, Oct 28）	`https://www.cs.cmu.edu/~418/lectures/` 下**不存在**	**未发布**（Fall 2026 新增主题；历史学期 PDF 在 `/afs/...`，**需 CMU 登录**）	无幻灯片编号可引，本笔记以公开文献 + 下述两份公开补充材料组织
Lecture 24 录像	Fall 2026 日程表中被注释隐藏	**未发布**	无
`cs149_supp/aidatacenter.txt`（CS149 L12，72 页）	公开讲义 PDF 抽取文本	**已公开**	硬件底座：HBM/DRAM/内存控制器/脉动阵列/RDU/重叠量化
`cs149_supp/thoughtprocess.txt`（CS149 L4，74 页）	公开讲义 PDF 抽取文本	**已公开**	软件决策空间：四步法/foreach 四实现/任务分配/屏障与锁成本
Ed 讨论区、Autolab、Canvas	`edstem.org`、`autolab.andrew.cmu.edu`、Canvas	**需登录**	无

---

2. 核心概念与硬件/软件架构图解

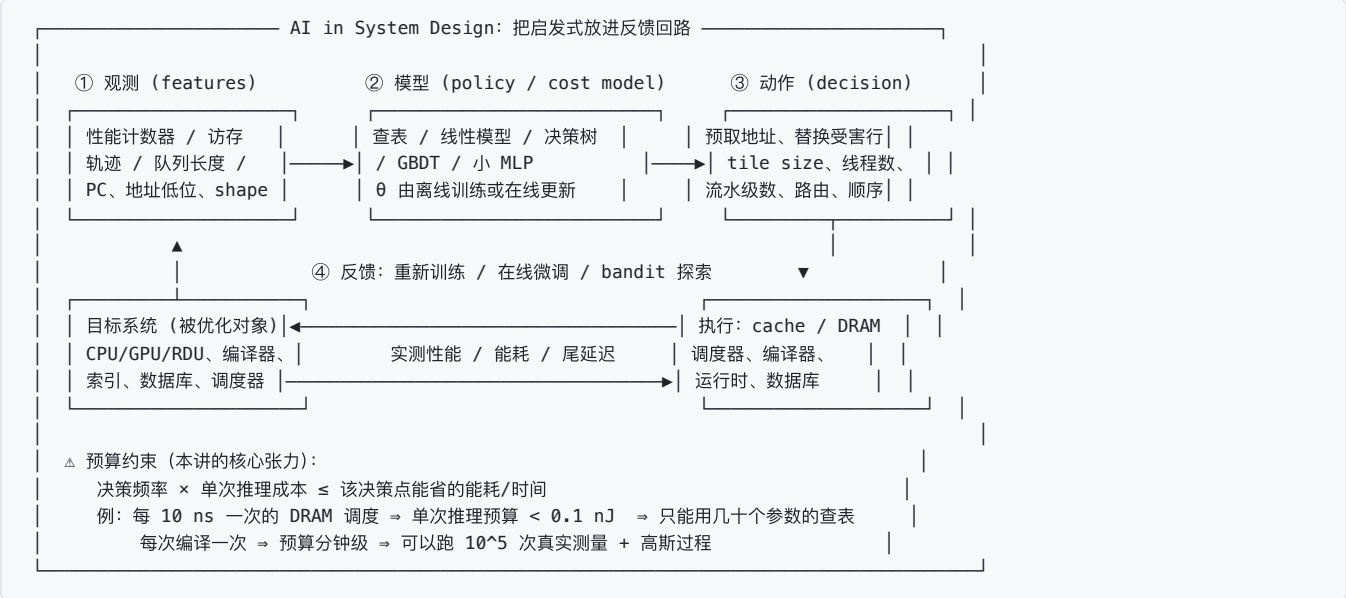
2.1 什么是 "AI in System Design"：两个方向与一个闭环

- **定义与目的**：这门课前面讲的都是 **Systems for AI**——用专用硬件与编译器把 AI 跑快（脉动阵列、HBM、数据流 RDU、kernel fusion、metapipelining）。本讲的 "AI in System Design" 是**反方向：用 AI/ML 去设计系统**，即把系统内部那些"靠经验和拍脑袋定下来的规则"换成**从数据里学出来的函数**。系统设计的四步法（`thoughtprocess.txt` slide 30：Problem → Subproblems → Parallel Threads → Program → 机器）每一步都含有一批"配置选择"，而这些选择今天大多由**手写启发式或静态常量**给出：

决策层级	手写启发式的例子	从哪份材料里看到	学习型替代
数据表示 / 访问路径	`B+-tree` 的多级页查找；线性扫描	公开文献（learned index）	learned index（RMI / PGM）
DRAM 请求排序	**FR-FCFS**（first-ready first-come-first-serve）	`aidatacenter` slide 63	学习型内存调度器
缓存 / 预取	固定深度 stream / stride 预取器；LRU	`aidatacenter` slide 63 附近	学习型预取器 / 学习型替换
分块与融合	固定的 tile size、固定的 kernel fusion 边界	`aidatacenter` slide 14、18–20	学习型代价模型 + 自动调优
分配（assignment）	`foreach` 默认 interleaved；静态 blocked	`thoughtprocess` slide 13	由负载特征选择分配方案
并行维度 / 流水	DP/TP/PP/EP 的度数靠人工网格搜索	`aidatacenter` slide 37、44	学习型并行策略搜索

决策层级	手写启发式的例子	从哪份材料里看到	学习型替代
计算–通信重叠	"把 allreduce 放到后台线程"	`aidatacenter` slide 26、39–41	学习型流水调度

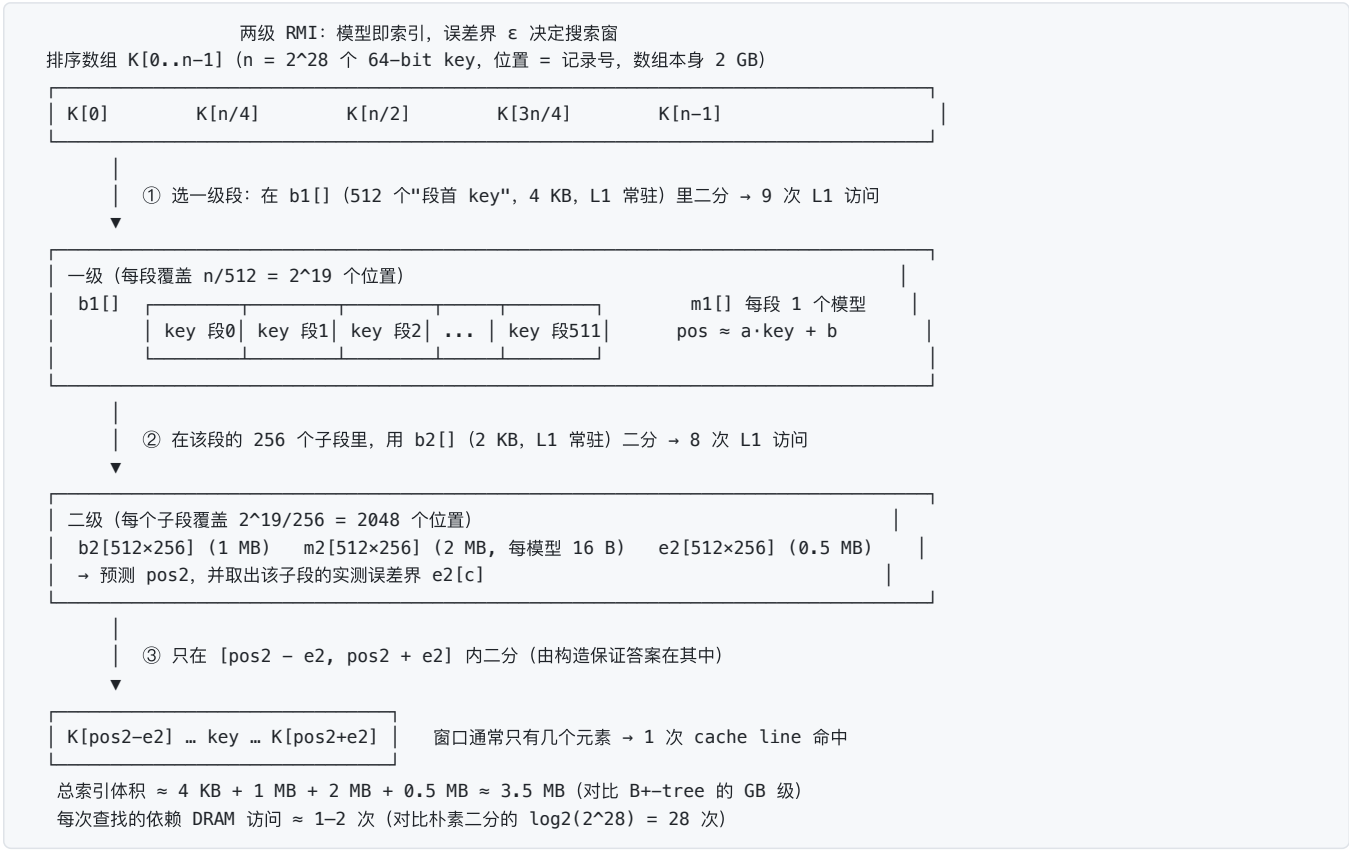
- **直观解释（"它是什么？"）**：
- **类比 1（系统里的"老师傅"）**：一个跑了二十年的老机房管理员，凭直觉决定"哪个请求先发给内存""这个循环切多大块"。他经验丰富但**不会因为换了新机器而更新直觉**（H100 的 roofline ridge point 是 CPU 的几十倍，老师傅的常数就全错了）。ML for systems 做的就是**把老师傅的直觉换成一本会自己更新的手册**：手册的输入是当前工作负载的特征（访存轨迹、队列长度、矩阵形状），输出是最优配置。
- **类比 2（离线地图 vs 实时导航）**：**自动调优 / learned cost model** 像离线地图——编译期或部署前把每个路口（每种配置）都跑一遍，代价高但决策慢一点没关系；**学习型预取 / 学习型 DRAM 调度**像实时导航——每一个路口（每一次访存）都要做一次决策，**每次决策的预算只有纳秒和零点几纳焦**。本讲最重要的工程洞察就藏在这个对比里：**同一个模型，放在不同的决策点上，可行与不可行的差别是 6–9 个数量级**。
- **图 1：ML for Systems 的在线闭环（软件执行模型）**



- **性能特征与关键操作**：这个闭环里，**模型推理本身要在被它优化的同一台机器上跑**。因此它带来三项新的系统开销，必须逐项量化：(a) **推理延迟**（是否落在关键路径上？落在关键路径上就会直接变成串行部分，受 Amdahl 限制）；(b) **推理能耗**（用 pJ/op 与 pJ/byte 换算，见 §4.5）；(c) **模型状态的空间与带宽**（模型表多大？每决策触碰多少 cache line？）。一条经验法则：**决策频率每提高 10 倍，可用模型参数大约减少 100 倍**（因为预算按频率线性收紧，而可用的模型精度随参数增加呈对数增长）。

2.2 Learned Index Structures（学习型索引）

- **定义与目的**：排序数组上的查找本质是 **CDF（累积分布函数）求逆**——`pos = F(key)`，其中 `F` 是"有多少个 key 小于等于 key"的单调函数。`B+-tree` 用 $\log_2$ 次指针追逐去**逼近**这个函数，而**学习型索引直接用模型拟合** `F`：顶层一个小模型给出粗略位置，下一层模型修正误差，最后只在**误差界（error bound）ε** 划定的窗口内做二分。它解决的问题是：**索引本身的体积与指针追逐次数**——索引从 GB 级降到 MB 级，每次查找的依赖访存从"树高"降到"模型层数 + 窗口内命中"。
- **直观解释（"它是什么？"）**：
- **类比（电话簿的页码估算）**：要从一本按姓氏排序的电话簿里找 "Wang"，`B+-tree` 的做法的"二分查找"——每次都从正中间翻开，反复砍半；学习型索引的做法是**先看姓的首字母在整本书里的位置分布**（模型）："W 大概在全书的 87% 处"，直接翻到 87% 附近，然后在前后几页里翻。如果这个分布是平稳的（模型准），你几乎一次就翻到；如果姓名分布变了（大量新增用户集中于 A 姓），模型的估计就会偏，你就得扩大搜索窗口——**这就是学习型索引的脆弱点：它把"最坏情况有保证的 log n"换成了"平均情况很快，但依赖数据分布稳定"**。
- **另一个必要的类比（索引的"占地面积"）**：`B+-tree` 像**在图书馆里建了一整套目录卡片柜**（每本书都要留位置），学习型索引像**只留一张"每类书大致在第几排"的导览图**——导览图当然不能告诉你第几本，但只要它把范围缩到几排之内，走过去翻一翻就够了。导览图不占地方，**这就是学习型索引最稳的收益来源：空间缩小一到三个数量级，从而改善缓存行为**。
- **图 2：两级 RMI（Recursive Model Index）的结构与查找路径（硬件/数据结构的存储布局）**



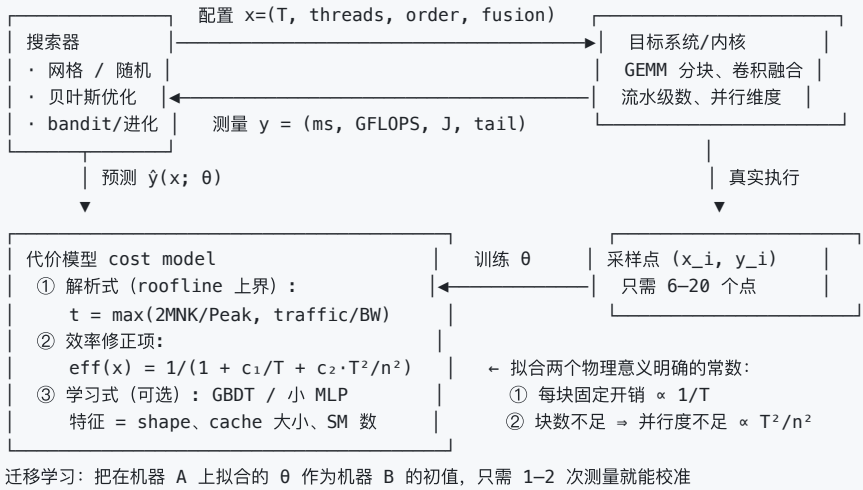
- **关键操作与性能特征**：
- **构造（build）**：对每个段做一次线性插值拟合（O(1)），再**实测该段的误差界**（O(段长)）。总 work = Θ(n)（每位被访问常数次），而 `B+-tree` 的 bulk load 需要先排序再逐页填，work = Θ(n log n)（或 Θ(n) 但常数大得多）。**Span = Θ(n/M1)**（每个一级段可并行构造），并行度 = M1 = 512 起。
- **查找（lookup）**：延迟 = 一级边界二分（L1，~4 个周期/步）+ 二级边界二分（L1）+ 模型表访问（3.5 MB → L2/L3，约 15–40 ns）+ 窗口内二分（1 次 DRAM 缺失，约 90 ns）。**总延迟约 130–200 ns**；朴素 `std::lower_bound` 在 2 GB 数组上是 28 次**依赖**缺失 ≈ 2.5 μs。
- **带宽**：每次查找的 DRAM 流量 ≈ 2–3 个 64 B cache line ≈ 160 B；朴素二分每次 ≈ 28 × 64 B = 1.8 KB（**相差 11 倍**）。批量 10^7 次查找 ⇒ RMI ≈ 1.6 GB vs 朴素二分 ≈ 18 GB——**这就是"学习型索引的价值最终体现为带宽"的含义**。
- **表 1：B+-tree 与学习型索引（RMI / PGM）的定量对比（假设 n = 2^{28} 个 64-bit key，cache line 64 B，DRAM 缺失延迟 90 ns，双通道带宽 38.4 GB/s）**

维度	朴素二分（有序数组，无索引）	B+-tree（4 KB 页、每页 256 项）	学习型索引（两级 RMI，ε 有界）
索引体积	0（就是数据本身）	叶层 ~4 GB + 内层 ~16 MB + 64 KB	**~3.5 MB**（模型 + 边界 + 误差表）
查找的依赖访存级数	**28**（$\log_2 2^{28}$）	4（页级数），其中 2–3 次 DRAM 缺失	**1–2 次 DRAM 缺失 + 2 次 L1 内的边界二分**
单次查找延迟（估算）	≈ 28 × 90 ns ≈ **2.5 μs**	≈ 2–3 × 90 ns + 页内二分 ≈ **250 ns**	≈ 90 ns（叶）+ 40 ns（模型）+ L1 工作 ≈ **140 ns**
单次查找 DRAM 字节	≈ 1.8 KB	≈ 3 × 64 B = 192 B	≈ 2–3 × 64 B = **160 B**
批量 10^7 次（38.4 GB/s 下限）	**≈ 470 ms**（带宽受限）	≈ 50 ms	≈ 42 ms
插入 / 更新	需要移动元素，O(n) 最坏	O(log n)，页分裂	**模型分布漂移 ⇒ 误差界失效 ⇒ 需部分重建**（这是它最大的代价）
最坏情况保证	有（log n）	有（树高有界）	**弱**：ε 由构造时的数据决定，分布漂移会破坏它
适用场景	只读、小 n	通用 OLTP/OLAP	只读或 append-mostly（分析型索引、LSM 内层、静态快照）

> **必须知道的反面证据**：*Benchmarking Learned Indexes*（VLDB 2021）这类系统性复现研究发现，在**公平对比**（同样的缓存占用、同样的并发控制、同样的写负载）下，学习型索引相对 B+-tree 的优势常常**远小于最初论文报告的 1.5–3×**，而且**很大一部分收益来自"索引体积变小导致缓存行为改善"，而不是"模型预测更准"**。这正是 "AI in System Design" 这一讲最该学到的态度：**换掉一个启发式之前，先把整个系统的账算清楚——否则你只是在给瓶颈搬了个家。**

2.3 Learned Cost Models 与 Autotuning（学习型代价模型与自动调优）

- **定义与目的**：并行程序里有一大类决策，其特征是"**离线可测、在线可查**"：分块尺寸（tile size）、线程数、向量宽度、循环融合的边界、数据布局（AoS vs SoA）、流水级数、DP/TP/PP 的度数、通信-计算重叠的方案。这些决策的**搜索空间巨大**（`aidatacenter` slide 14 把数据流编译器的流水线写成"Tiling → Parallelization → Metapipelining → Place & Route → Codegen"，每一级都是一个搜索维度），而**手写常量在每次硬件换代时就失效**。learned cost model 的目的就是：用少量真实测量**拟合一个代理模型（surrogate）**，再用它去预测没测过的配置的性能，从而把"穷举"压成"少量测量 + 大量预测"。
- **直观解释（"它是什么？"）**：
- **类比（请人试鞋）**：**穷举搜索**是把所有尺码的鞋都试一遍（慢但准）；**解析代价模型**是拿公式算（快但常常算不准，比如 roofline 完全忽略了 bank 冲突和调度开销）；**学习型代价模型**是先试 6–8 双，然后**根据"鞋码 ↔ 合脚程度"的关系推断出你的尺码**，再直接推荐一双——最后还要再试一下验证（train/test）。本节 §3 的第二个代码示例就是这个过程的完整实现：**在 6 个 tile size 上测量 → 拟合模型 → 预测未见过的矩阵尺寸上的最优 tile → 验证**。
- **另一个类比（预报 vs 实测）**：解析 roofline 是"气象模型"（物理正确但忽略很多细节），学习型代价模型是"统计回归"（不解释机理，但在训练分布内很准）。**两者必须一起用**：roofline 给出上界（你去不了的地方），学习型模型给出可行域内的最优选择（你实际能到的地方）。
- **图 3：自动调优 / 学习型代价模型的闭环与搜索空间（软件执行模型）**

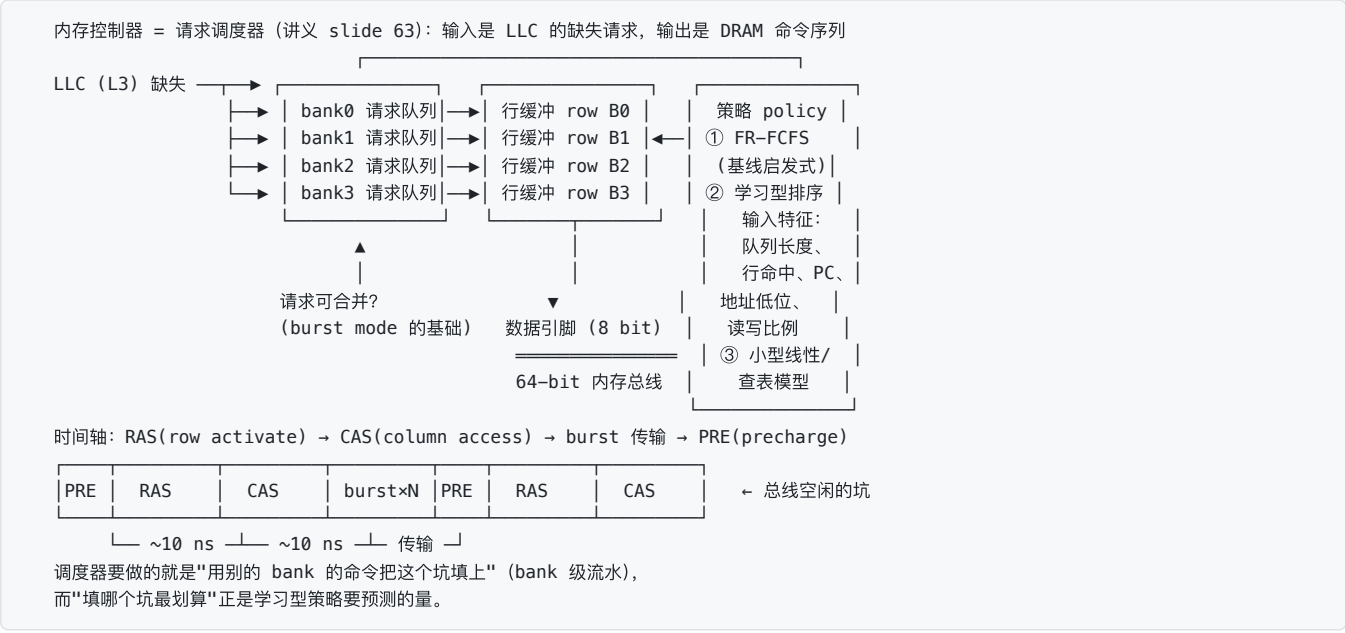


- **关键操作与性能特征**：
- **搜索空间的维度爆炸**：仅"分块 + 循环顺序 + 向量化 + 线程数 + 融合边界"就轻松超过 10^6 个配置点，而每个配置的一次真实测量往往是几十毫秒到几秒。**穷举的 work = |配置空间| × 单次测量时间**，这在工程上不可接受，于是 **Work 必须从"全空间"降到"O(log)|空间|"（贝叶斯优化）或"O(1) 预测"（拟合后）**。
- **训练数据的成本**：每次测量本身就是一次完整的程序运行（含编译、加载、预热）。这与 §2.2 的索引构造不同：**索引构造的 work 是 Θ(n) 的一次性代价，而调优的 work 是 |采样| × 内核运行时间，并且必须在新 shape/新硬件上重新支付**。
- **泛化与分布漂移**：模型在 shape 分布内的插值通常很准（这也是为什么用 `log T` 而不是 `T` 做特征），但**外推（更大的矩阵、更多的线程、更强的 GPU）常常失效**，必须保留"不确定时回到安全默认值"的兜底路径。

2.4 学习型内存系统策略：预取、替换与 DRAM 调度

- **定义与目的**：内存系统的每一个决策都是一个"顺序 + 选择"问题：**内存控制器先服务哪个请求**（`aidatacenter` slide 63：默认策略是 FR-FCFS——先服务"当前行已打开"的请求以最大化行局部性，其余按 FIFO）、**预取哪个地址**、**淘汰哪一行**（LRU）。这些启发式在单一负载下不错，但在混合负载（一个流式扫描 + 一个随机访问）下会互相伤害。学习型策略的输入是控制器天然拥有的丰富状态（每个 bank 的队列、每行的 open/close 状态、请求的 PC、地址低位、队列长度），输出是一个**更优的排序或地址预测**。
- **直观解释（"它是什么？"）**：

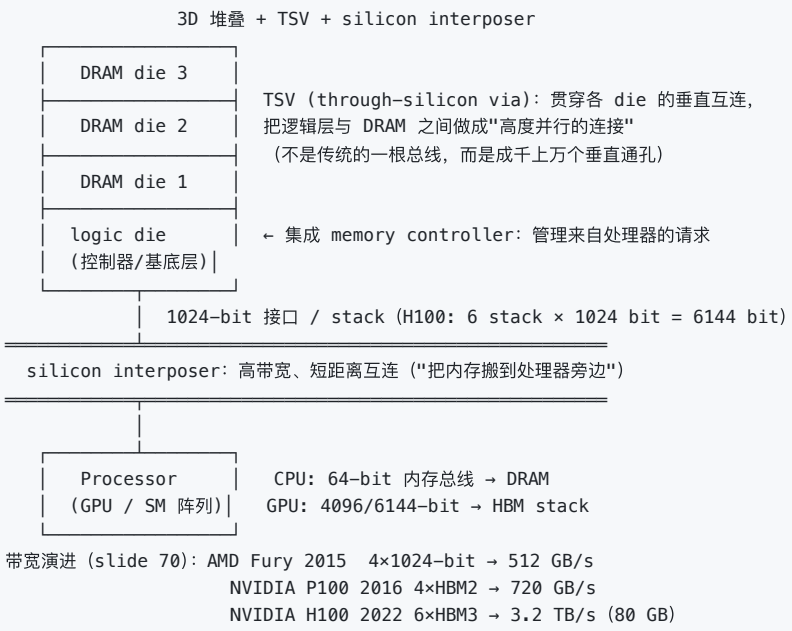
- **类比（超市收银台的排队规则）**：FR-FCFS 就是"先把购物车里东西已经摆在收银台上的顾客结掉，其余的按到达顺序"——它在大多数时候很合理，但如果那个"东西摆好"的顾客要结 500 件商品，后面只买一瓶水的顾客就被饿死了。**学习型调度**则像一位**会看队伍构成的领班**：根据"每个人的商品数、是否已摆好、通道拥堵情况"动态决定先开哪个收银台——更优，但领班自己也要发工资（推理开销），而且**如果他的判断依据是过时的培训内容（分布漂移），可能比死板的规则更糟**。
- **关键约束（DRAM 侧）**：`aidatacenter` slide 54 强调 **data pins 是最稀缺的资源**，只在很小一部分时间里被占用；slide 56 说明 **多个 bank 允许请求流水化**（一个 bank 做 precharge/activate，另一个 bank 在传输数据），slide 55 说明 **burst mode** 把延迟摊到更大的传输上。**所以 DRAM 调度策略的目标函数就是"让数据总线始终忙 + 让行命中率尽量高"**——这是一个非常明确、非常适合用一个小模型去学的问题，也是学习型策略在硬件里最有希望的落点之一。
- **图 4：内存控制器（请求调度器）+ 学习型仲裁（硬件结构）**



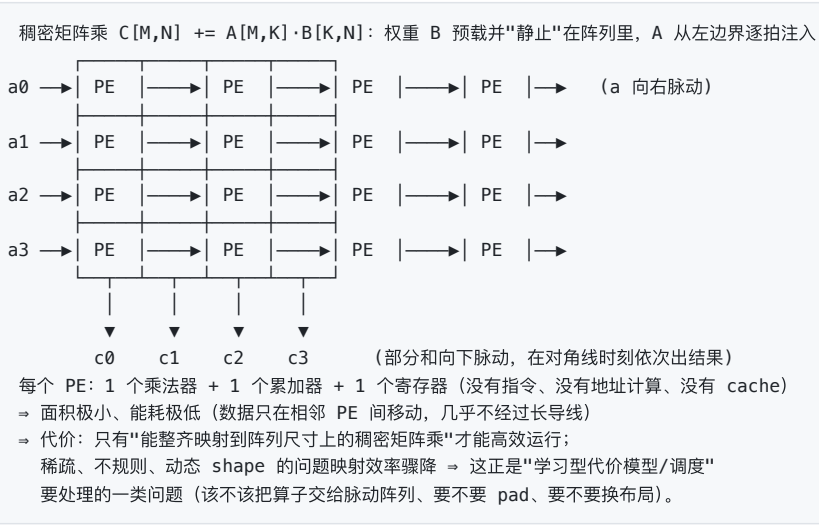
- **关键操作与性能特征（带宽、延迟、吞吐）**：
- **延迟**：从"行已打开"到数据上总线的 **CAS ≈ 10–13 ns**（slide 51、65）；从"行未打开"则要 **PRE + RAS + CAS ≈ 30 ns**。**同一个 DRAM 的延迟差别达 3 倍**——这解释了为什么调度器（而不是更快的 DRAM）是主要优化对象。
- **带宽 vs 延迟必须一起算（Little's law）**：要维持带宽 `BW`，在途字节数必须 ≥ `BW × 延迟`。DDR4-2400 单通道 19.2 GB/s × 13 ns ≈ **250 B ≈ 4 个 cache line**；H100 的 3.2 TB/s × 500 ns ≈ **1.6 MB**（全芯片在途）≈ 每 SM 约 12 KB ≈ 190 个 cache line。**学习型预取/调度的第一性约束不是"预测多准"，而是"能不能造出足够的在途请求"**——再准的预取器，若队列深度不够也填不满带宽。
- **吞吐**：`aidatacenter` slide 65 的 DDR4-2400 数字可以直接算：64-bit × 1.2 GHz × 2（双沿）× 2 通道 = **38.4 GB/s**。

2.5 硬件底座：学习型策略跑在什么机器上

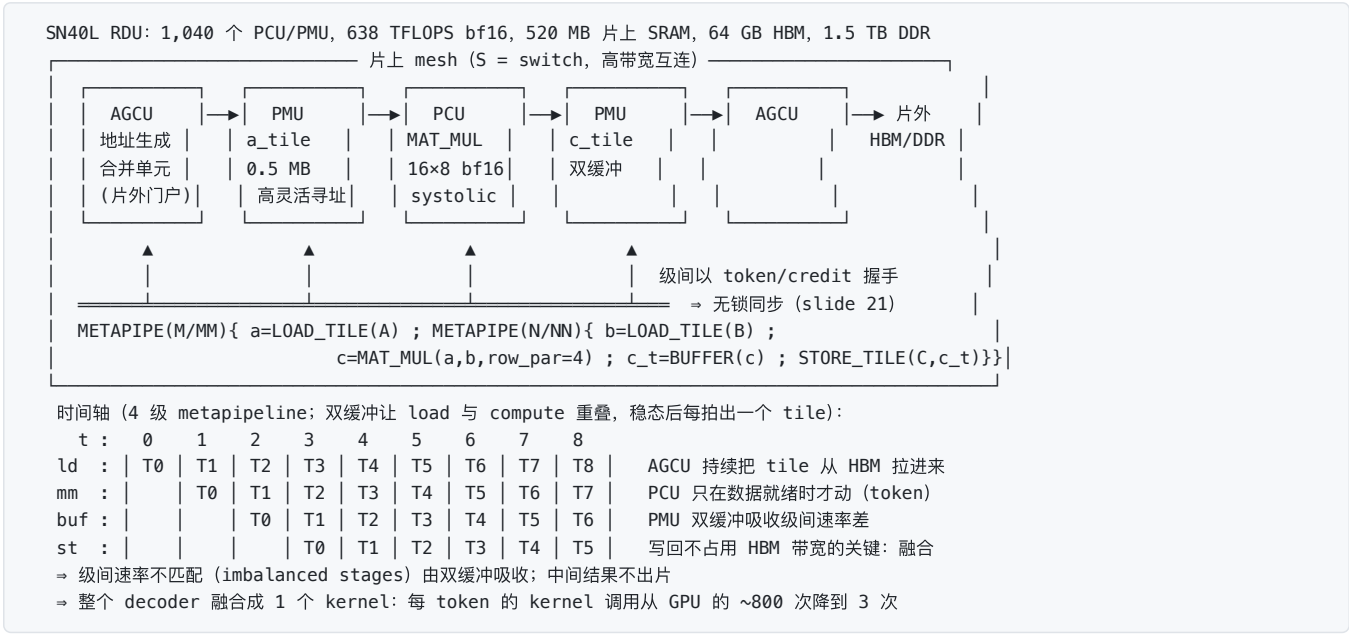
- **定义与目的**：学习型策略的可行性完全由**它运行的那台机器**决定：推理要多快、模型能占多少 SRAM、一次访存多少 pJ。`aidatacenter.txt` 给出的正是这块底座。它解决的是"**在哪儿算、数据从哪儿来**"的问题：**HBM + 3D 堆叠**把带宽从 DRAM 的几百 GB/s 拉到几 TB/s；**脉动阵列**把稠密矩阵乘的单位能耗压到最低；**数据流 RDU** 用片上大容量 SRAM（520 MB vs H100 的约 100 MB）+ metapipelining 把中间结果留在片上。
- **直观解释（"它是什么？"）**：
- **类比（把水库修到工厂隔壁）**：传统 DRAM 像**城市边缘的水库**——水很便宜，但管道又长又细（64-bit 内存总线），每次取水都要等水压上来（延迟）。**HBM 通过 3D 堆叠 + silicon interposer 把水库修到了车间地板下面**（1024-bit/stack 的宽接口，走线只有毫米级），于是**管道变宽、距离变短、单位能耗下降**（slide 6：更高带宽、更高能效、更小封装）。代价是容量与 3D 堆叠的制造难度。
- **类比（流水线的工人 vs 会传话的工人）**：**脉动阵列**像一条**传送带上的工人**：每个人只做一次乘加，然后把结果**递给下一个人**（数据在相邻 PE 间只走一个寄存器到寄存器的距离），没人需要跑仓库取料（无指令发射、无寄存器文件读写的开销）。**数据流架构 RDU** 像**一群按图纸自动流转的车间**：数据带着"token"在 PCU/PMU 之间流动，**下游知道该什么时候干活（token 控制），因此不需要锁**（slide 21：Dataflow execution with token control ⇒ no lock-based synchronization）。
- **图 5：HBM 与 3D 堆叠（TSV + interposer，硬件结构；依据 slide 5/68）**



- **图 6：脉动阵列（systolic array，硬件结构）**



- **图 7：RDU 上的 metapipelining 与 token 控制的无锁数据流（硬件 + 软件执行模型；依据 slide 13/16/21/24）**



- **表 2：访问与运算的开销（数据来自 `aidatacenter.txt` slide 46/47，两套来源同时列出；本讲的能量预算全靠这张表）**

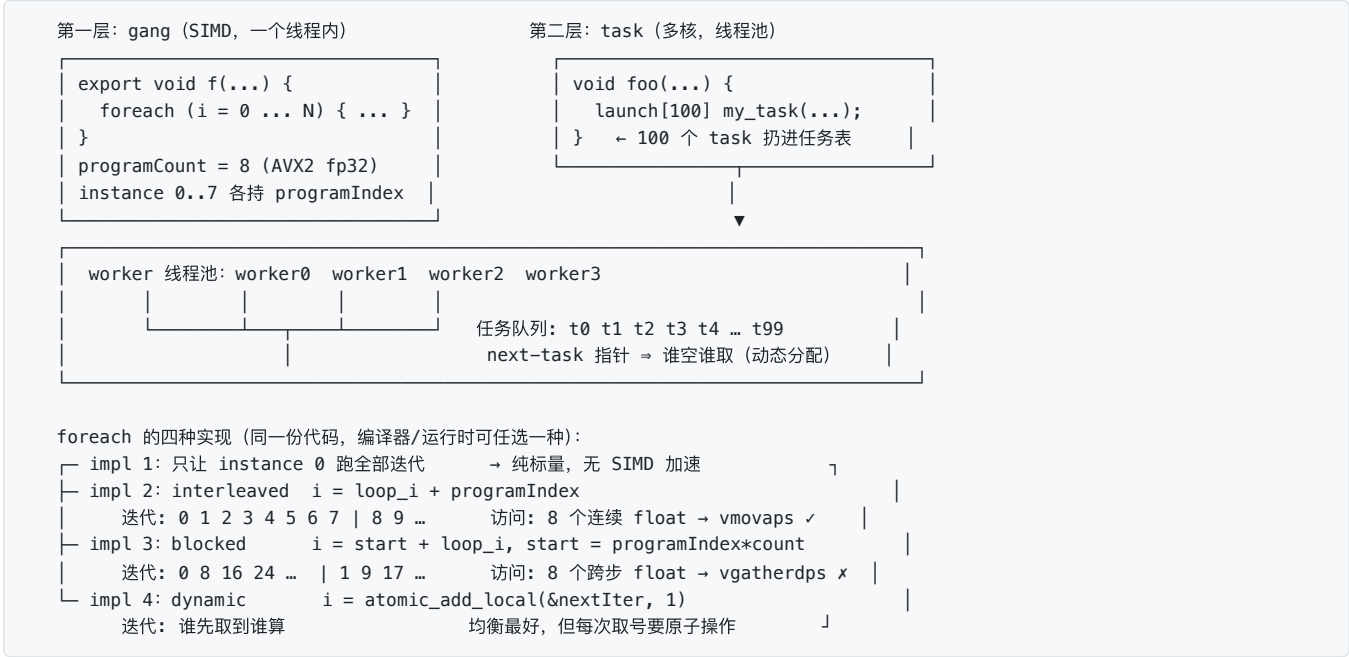
操作	slide 46（Dally / ARM 口径）	slide 47（Han, ICLR 2016, 45 nm 口径）	归一化（以 32-bit SRAM 访问为 1）
整数运算	~1 pJ	—	~0.2
32-bit 浮点运算	~20 pJ	~0.9 pJ	~4（slide 46 口径）
读 64 bit（= 8 B）片上 SRAM（1 mm 内）	~26 pJ	~5 pJ（32 bit）	**1（基准）**
读 64 bit（= 8 B）低功耗 DRAM（LPDDR）	**~1200 pJ**	~640 pJ（32 bit）	**≈ 46×**
10 GB/s 持续访存的功耗	$10 \times 10^9 \times (1200\ \text{pJ} / 8\ \text{B}) \approx$ **1.5 W**（slide 写 ~1.6 W）	—	对比：移动 GPU 整机功率预算 ≈ 1 W
参考电池容量	iPhone 16 ≈ 14 Wh（讲义同时给出 MacBook Pro 99 Wh）	—	—

> 两个口径相差 20 倍（FP op 0.9 pJ vs 20 pJ）——**这不矛盾**：slide 46 是较先进工艺 + 只计逻辑运算本身的功耗，slide 47 是 45 nm 估算且常被用来做"数据搬运 vs 计算"的相对比较。**做能耗预算时必须写明用的是哪一套数字**，否则结论会差一个数量级以上（§4.5 会看到这个差别直接决定"学习型预取器是否可行"）。

2.6 软件执行模型：从 `foreach` 的分配策略到 metapipelining

- **定义与目的**：`thoughtprocess.txt` 把"写一个并行程序"拆成四步：**分解（找独立工作）→ 分配（把工作给 worker）→ 编排（通信/同步/数据布局/调度）→ 映射到硬件**。这四步里，**"分配"和"编排"正是学习型策略可以接管的部分**：一个 ISPC 编译器/运行时可以在 `interleaved / blocked / dynamic` 之间选，一个数据流编译器可以在 tile 大小、流水级数、融合边界之间选。
- **直观解释（"它是什么？"）**：
- **类比（分糖果的三种分法）**：**interleaved（交错）**是"一人一颗轮着发"——每个人手里的糖**在记忆里是相邻的**（对 SIMD 打包载入极友好，`vmovaps` 一条指令搞定 8 个）；**blocked（分块）**是"每人发一整段"——每个人手里的糖**在记忆里隔得很远**（SIMD 必须用 `vgatherdps` 一条条去抓，慢好几倍），但如果每颗糖的"加工时间"不均匀，分块反而更均衡。**dynamic（动态）**是"发一颗算一颗，算完再来拿"——最均衡，但每次拿糖都要排队（原子操作 + cache line 争用）。**哪一种最优，取决于"加工时间的方差"和"访问的连续性"这两个特征——这正是学习型策略的输入。**
- **类比（流水线 vs 一次性做完）**：**metapipelining** 是"把一条长流水线切成互相重叠的小段"（`aidatacenter` slide 16 称其为 "a pipeline of pipelines"）：它把循环体里的并行模式（map/reduce/zip）变成流水级，**用双缓冲吸收各级速率差**，让多个循环迭代同时"在路上"。**融合（fusion）与 metapipelining 的分界本身就是个决策**：能融合就融合（省 HBM 流量），不能融合（依赖太复杂）就用 metapipelining（`aidatacenter` slide 16 明确写了 "Metapipelining can work when fusion does not"）。

- **图 8：ISPC 的两层并行（gang × task）与 `foreach` 的分配策略（软件执行模型；依据 `thoughtprocess` slide 3/7/13/22/42）**



- **关键操作与性能特征（延迟、带宽、吞吐）**：
- **SIMD 载入方式决定访存吞吐**：interleaved 的 `vmovaps`（打包载入）在 Skylake 级核心上约 0.5 周期/8 float；`vgatherdps`（gather）需要逐元素访问，吞吐差**约一个数量级**（具体数字随微架构而异）。所以 **blocked 分配在"访问必须连续"这一条上要付 10 倍的载入代价**，除非该负载本来就带宽受限（此时 gather 的代价会被内存墙掩盖——这又是一个"必须做 Roofline 才能判断"的例子）。
- **动态分配的开销与 Amdahl**：每次取号一次原子操作（约 20–100 ns 的争用成本）。若一个任务只有 1 μs 的有效工作，取号开销就占 2–10%；若任务有 100 μs 工作，开销可忽略。**"任务粒度"因此是一个由 `原子开销 / 任务有效工作` 决定的、可以用学习型模型预测的决策。**
- **编排的代价（屏障与锁）**：`thoughtprocess` slide 60/68 的网格求解器例子给出了最经典的反面教材：**把 `lock/unlock` 放进内层循环**（每次 `(i,j)` 更新都加锁）会让程序的并行度几乎归零；改成"先在线程内累加 `myDiff`，再在循环外锁一次"，锁开销从 $O(N^2)$ 次降到 $O(1)$ 次（每线程每轮一次）。slide 72 又把它进一步优化成**"用三个不同的 `diff` 变量轮流使用，把三个屏障降到一轮一个"**——**用空间（footprint）换依赖的消除**。

---

3. 代码示例与性能分析

3.1 示例一：两级 RMI 学习型索引 vs 多级页式索引 vs 朴素二分（C++17 + OpenMP）

```cpp
// ============================================================================
// learned_index.cpp — 学习型索引（两级 RMI，误差界驱动搜索窗）vs 多级页式索引 vs 朴素二分
// 编译: g++ -O3 -march=native -fopenmp -std=c++17 learned_index.cpp -o learned_index
// 运行: OMP_NUM_THREADS=8 ./learned_index 268435456 10000000
//        参数1 = key 个数 n（默认 1<<24 = 16.7M）
//        参数2 = 查询次数 nq（默认 4*10^6）
// 说明: -O3 + -march=native 打开向量化与本地指令集；这是 release 优化编译。
// ============================================================================
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <random>
#include <vector>
#include <omp.h>

static double now_ms() {
  using namespace std::chrono;
  return duration<double, std::milli>(steady_clock::now().time_since_epoch()).count();
}

// ---------- 数据：单调递增、密度分两段的 key（"热区 + 冷区"）----------
// 单条直线无法拟合整条 CDF，因此顶层模型必然有误差 —— 这正是需要第二层的原因。
static void make_keys(std::vector<uint64_t>& K) {
  const int64_t n = (int64_t)K.size();
  for (int64_t i = 0; i < n; ++i) {
    if (i < n / 2) K[i] = (uint64_t)i * 2;                          // 前半段：密度 1/2
    else           K[i] = (uint64_t)n + (uint64_t)(i - n / 2) * 8;  // 后半段：密度 1/8
  }
}

// ---------- 线性模型 pos ≈ a*key + b ----------
struct Model {
  double a = 0.0, b = 0.0;
  int64_t predict(uint64_t k) const { return (int64_t)(a * (double)k + b); }
};

// 用段的两端点做线性插值拟合：O(1)，适合构建上百万个模型
static Model fit_range(const std::vector<uint64_t>& K, int64_t lo, int64_t hi) {
  Model m;
  const double x0 = (double)K[lo], y0 = (double)lo;
  const double x1 = (double)K[hi], y1 = (double)hi;
  if (x1 <= x0) { m.a = 0.0; m.b = y0; return m; }
  m.a = (y1 - y0) / (x1 - x0);
  m.b = y0 - m.a * x0;
  return m;
}

// ---------- 方案 A：两级 RMI（学习型索引）----------
struct RMI {
  int M1 = 512, SUB = 256;                 // 一级 512 段，每段 256 个子段
  int64_t n = 0;
  std::vector<uint64_t> b1, b2;            // 边界 key：b1=512*8B=4KB, b2=512*256*8B=1MB
  std::vector<Model>    m1, m2;            // 模型：m1=8KB, m2=512*256*16B=2MB
  std::vector<int32_t>  e2;                // 每个二级子段的实测最大误差（0.5MB）
  int32_t max_e1 = 0;

  int64_t seg_lo(int i) const { return (int64_t)((double)n * i / M1); }
  int64_t seg_hi(int i) const { return (int64_t)((double)n * (i + 1) / M1) - 1; } // 闭区间

  void build(const std::vector<uint64_t>& K) {
    n = (int64_t)K.size();
    b1.resize(M1); m1.resize(M1);
    b2.resize((size_t)M1 * SUB); m2.resize((size_t)M1 * SUB); e2.resize((size_t)M1 * SUB);
    for (int i = 0; i < M1; ++i) {
      const int64_t lo = seg_lo(i), hi = seg_hi(i);
      b1[i] = K[lo];
      m1[i] = fit_range(K, lo, hi);
      // 一级误差（仅用于统计/诊断；本设计的正确性不依赖它）
```

```
      for (int64_t p = lo; p <= hi; ++p) {
        int64_t e = std::llabs(m1[i].predict(K[p]) - p);
        if (e > max_e1) max_e1 = (int32_t)e;
      }
      // 二级：把本段等分成 SUB 个子段，每个子段一个模型 + 实测误差界
      for (int t = 0; t < SUB; ++t) {
        const int64_t slo = lo + (hi - lo + 1) * t / SUB;
        const int64_t shi = lo + (hi - lo + 1) * (t + 1) / SUB - 1;
        const size_t idx = (size_t)i * SUB + t;
        b2[idx] = K[slo];
        m2[idx] = fit_range(K, slo, shi);
        int64_t e = 0;
        for (int64_t p = slo; p <= shi; ++p) {
          int64_t d = std::llabs(m2[idx].predict(K[p]) - p);
          if (d > e) e = d;
        }
        e2[idx] = (int32_t)e;
      }
    }
  }

  // 纯函数、可安全并发调用：这是查找路径的全部
  int64_t lookup(const std::vector<uint64_t>& K, uint64_t key, int64_t* fallbacks = nullptr) const {
    // ① 一级：在 512 个段首 key 里二分（4 KB，L1 常驻）
    int i = (int)(std::upper_bound(b1.begin(), b1.end(), key) - b1.begin()) - 1;
    if (i < 0) i = 0;
    // ② 二级：在本段的 256 个子段边界里二分（2 KB，L1 常驻）
    const uint64_t* base = b2.data() + (size_t)i * SUB;
    int c = (int)(std::upper_bound(base, base + SUB, key) - base) - 1;
    if (c < 0) c = 0;
    const size_t idx = (size_t)i * SUB + c;
    // ③ 用子段模型+误差界划定搜索窗（由构造保证真值在窗内）
    const int64_t seg_lo_p = (int64_t)((double)n * i / M1);
    const int64_t seg_hi_p = (int64_t)((double)n * (i + 1) / M1) - 1;
    int64_t q = m2[idx].predict(key);
    int64_t lo = std::max(seg_lo_p, q - e2[idx] - 1);
    int64_t hi = std::min(seg_hi_p + 1, q + e2[idx] + 2);
    if (lo < hi && K[lo] <= key && key <= K[hi - 1])
      return (int64_t)(std::lower_bound(K.begin() + lo, K.begin() + hi, key) - K.begin());
    if (fallbacks) ++(*fallbacks);           // 兜底：极端分布漂移时回退全局二分
    return (int64_t)(std::lower_bound(K.begin(), K.end(), key) - K.begin());
  }
};

// ---------- 方案 B：多级页式索引（B+-tree 风格的指针追逐）----------
struct PagedIndex {
  static constexpr int FANOUT = 256;      // 每页 256 个 key（8B key ⇒ 2 KB 页）
  const std::vector<uint64_t>* K = nullptr;
  std::vector<std::vector<uint64_t>> upper;   // 自底向上抽稀的"内部节点"数组
  void build(const std::vector<uint64_t>& keys) {
    K = &keys;
    upper.clear();
    std::vector<uint64_t> cur;
    for (size_t i = 0; i < keys.size(); i += FANOUT) cur.push_back(keys[i]);
    while (cur.size() > 1) {
      upper.push_back(cur);
      std::vector<uint64_t> nxt;
      for (size_t i = 0; i < cur.size(); i += FANOUT) nxt.push_back(cur[i]);
      cur.swap(nxt);
    }
    upper.push_back(cur);                 // 顶层（1 个元素），统一下降逻辑
  }
  int64_t lookup(uint64_t key) const {
    const std::vector<uint64_t>& L = *K;
    int64_t block = -1;                   // -1 = 整个数组
    for (int l = (int)upper.size() - 1; l >= 0; --l) {
      const std::vector<uint64_t>& a = upper[l];
      size_t lo = (block < 0) ? 0 : (size_t)block * FANOUT;
      size_t hi = (block < 0) ? a.size() : std::min(a.size(), lo + (size_t)FANOUT);
      size_t p = (size_t)(std::upper_bound(a.begin() + lo, a.begin() + hi, key) - a.begin());
      block = (int64_t)((p == lo) ? lo : p - 1);
    }
```

```
    size_t lo = (block < 0) ? 0 : (size_t)block * FANOUT;
    size_t hi = std::min(L.size(), lo + (size_t)FANOUT);
    return (int64_t)(std::lower_bound(L.begin() + lo, L.begin() + hi, key) - L.begin());
  }
};

// ---------- 驱动：正确性校验 + 并行批量计时 ----------
int main(int argc, char** argv) {
  const int64_t n  = (argc > 1) ? atoll(argv[1]) : (1LL << 24);
  const int64_t nq = (argc > 2) ? atoll(argv[2]) : 4000000;
  std::vector<uint64_t> K((size_t)n);
  make_keys(K);

  // 生成查询 key（可能不存在于数组中，用来验证 lower_bound 语义）
  std::vector<uint64_t> Q((size_t)nq);
  { std::mt19937_64 rng(12345); for (int64_t i = 0; i < nq; ++i) Q[i] = rng() % (2ULL * (uint64_t)n); }

  RMI rmi;      double t0 = now_ms(); rmi.build(K);      double tb_rmi = now_ms() - t0;
  PagedIndex pg; t0 = now_ms();       pg.build(K);       double tb_pg  = now_ms() - t0;
  std::printf("n=%lld  nq=%lld\n", (long long)n, (long long)nq);
  std::printf("构造: RMI %.1f ms | PagedIndex %.1f ms | RMI 索引体积 %.2f MB (B+-tree 风格 %.2f MB)\n",
              tb_rmi, tb_pg,
              (rmi.b1.size()*8.0 + rmi.b2.size()*8.0 + rmi.m1.size()*16.0 +
               rmi.m2.size()*16.0 + rmi.e2.size()*4.0) / 1048576.0,
              (double)n * 8.0 / 1048576.0);
  std::printf("RMI 一级最大误差 max_e1=%d（说明单条直线确实拟合不了整个 CDF）\n", rmi.max_e1);

  // 单线程校验：RMI / PagedIndex 必须与 std::lower_bound 完全一致
  int64_t bad = 0, fb = 0;
  for (int64_t i = 0; i < std::min<int64_t>(nq, 200000); ++i) {
    int64_t ref = (int64_t)(std::lower_bound(K.begin(), K.end(), Q[i]) - K.begin());
    if (rmi.lookup(K, Q[i], &fb) != ref) ++bad;
    if (pg.lookup(Q[i]) != ref) ++bad;
  }
  std::printf("校验: 不一致 %lld 次（必须为 0）；RMI 兜底回退 %lld / 200000 次\n",
              (long long)bad, (long long)fb);

  // 并行批量计时（静态分配；无共享写入 ⇒ 无数据竞争、无伪共享）
  auto bench = [&](const char* name, auto fn) {
    double t = now_ms();
    int64_t acc = 0;
    #pragma omp parallel for schedule(static) reduction(+:acc)
    for (int64_t i = 0; i < nq; ++i) acc += fn(Q[i]);
    const double ms = now_ms() - t;
    std::printf("%-22s %8.2f ms   %6.1f ns/查询   校验和 %lld\n",
                name, ms, ms * 1e6 / (double)nq, (long long)(acc & 1));
  };
  bench("朴素 lower_bound", [&](uint64_t k){ return (int64_t)(std::lower_bound(K.begin(), K.end(), k) - K.begin()); });
  bench("多级页式索引",     [&](uint64_t k){ return pg.lookup(k); });
  bench("学习型索引 RMI",   [&](uint64_t k){ return rmi.lookup(K, k); });
  return 0;
}
```

【代码做什么？】 1. `make_keys` 生成 **单调递增但密度分两段**的 key：前半段密度 1/2、后半段 1/8。这样**单条直线无法拟合整条 CDF**（程序会打印出 `max_e1` 远大于 0 来证明这一点），必须引入第二级模型——这正是 RMI 的存在理由。 2. `RMI::build` 分两级构造：一级把有序数组按**位置**等分成 512 段，每段用两端点做线性插值得到 `pos ≈ a·key+b`，并把段首 key 存进 `b1`（4 KB）；二级把每个一级段再等分成 256 个子段，各拟合一个模型，并且**逐元素实测该子段的最大误差 `e2[]`**——这是"误差界"的来源。构造总成本 Θ(n)（每位被访问常数次），且各段之间完全独立，可以并行。 3. `RMI::lookup` 是**没有分支预测惊喜的确定性路径**：在 512 个段首 key 里二分（4 KB，L1 命中）→ 在该段的 256 个子段边界里二分（2 KB，L1 命中）→ 用子段模型预测位置 → 在 `[q-e2-1, q+e2+2]` 这个**由构造保证包含真值**的窗口里 `lower_bound`。窗口通常只有几个元素（1 个 cache line）。若窗口校验失败（分布漂移、查询 key 落在段边界外），回退到全局二分并计数——**学习型索引必须永远带一条兜底路径**。 4. `PagedIndex` 是公平的对照组：它就是 B+-tree 的"每页 256 项、逐层抽稀"的指针追逐模型，查找的**依赖访存级数 = 层数**。 5. `main` 先做**单线程正确性校验**（RMI 与 PagedIndex 必须和 `std::lower_bound` 逐位一致），再对三种方案做**并行批量计时**。计时用 `#pragma omp parallel for schedule(static) reduction(+:acc)`，每个查询只读共享的 `K` 和索引，**没有任何写入 ⇒ 无数据竞争、无伪共享**。

【并行机制与性能解说】 – **并行结构**：查询批次天然数据并行（每个查询之间无依赖），用 OpenMP 静态分配到 8 个线程；RMI 内部**没有 SIMD 向量化空间**（每次查的是不同的 key，走的是不同的模型），所以它的加速来自**多核 + 每个核上的 MLP（memory-level parallelism）**，而不是 SIMD。这解释了为什么"学习型索引的收益最终由内存系统决定"。 – **Work / Span / 并行度**： – 构造：`Work = Θ(n)`（两遍扫描 + O(M1·SUB)

次 O(1) 拟合），`Span = θ(n/M1)`（每个一级段内部串行测量误差），**并行度 = M1 = 512**。 – 查询批次：单次查询 `Work ≈ 9 + 8 + 1(模型访问) + log₂(2e2+3) + 1(叶访问) ≈ 22 个操作`，其中**关键路径上的依赖访存 = 1–2 次 DRAM 缺失**，`Span ≈ 2 × 90 ns = 180 ns`。批量 `Work = nq × 22`，`Span = 180 ns + (nq / 线程数) × …`（因为查询之间独立，Span 只受"每线程串行处理的查询数"影响）。**并行度 = Work/Span ≈ nq × 22 / 180 ns 的量级 —— 也就是"完全够用"**。 – 结论：**Work/Span 给出的并行度是天文数字，所以它根本不是瓶颈**。真正的瓶颈只有两个：**① 每个核心能维持的在途缺失数（MLP，通常 10–16）；② DRAM 带宽**。这就是为什么 §4.1 必须用 Little's law 而不是 work–span 来判断这个内核。 – **瓶颈定量**（假设：8 核、每核 12 个在途缺失、DRAM 缺失延迟 90 ns、双通道带宽 38.4 GB/s）： – **MLP 上限**：8 × 12 = 96 个在途缺失；每缺失 90 ns ⇒ 丢失率 ≈ 96 / 90 ns ≈ **1.07×10^9 缺失/s**。 – **带宽上限**：38.4 GB/s ÷ 64 B = **6.0×10^8 缺失/s**。 – ⇒ **带宽先到顶**（带宽上限比 MLP 上限低 1.8 倍）。所以这个内核是**带宽受限**的： – 朴素二分：每次查询 28 个不同 cache line ⇒ 10^7 查询 × 28 × 64 B = **17.9 GB** ⇒ ⩾ 466 ms； – 页式索引：每次 3 个 line ⇒ 1.9 GB ⇒ ⩾ 50 ms； – RMI：每次 2–3 个 line ⇒ 1.3–1.9 GB ⇒ ⩾ 34–50 ms。 – **注意结论的诚实性**：RMI 相对"页式 B+–tree"的优势（34 ms vs 50 ms）主要来自**索引体积小 10^3 倍 ⇒ cache 命中率更高 ⇒ 每次查询少碰一个 cache line**，而不是来自"模型预测得准"。这与 §2.2 表 1 后面的反面证据一致。 – **其它瓶颈**：`e2[]` 若是偏大（分布漂移），窗口二分退化为长链，性能悬崖式下降——**学习型索引的性能是"数据分布的函数"，这是它与 B+–tree 的根本区别**。

3.2 示例二：GEMM 分块自动调优 + 学习型代价模型（C++17 + OpenMP）

```cpp
// ============================================================================
// gemm_autotune.cpp — 分块 GEMM 的自动调优，并用拟合出来的代价模型预测未测过的尺寸
// 编译: g++ -O3 -march=native -fopenmp -std=c++17 gemm_autotune.cpp -o gemm_autotune
// 运行: OMP_NUM_THREADS=8 ./gemm_autotune 2048 3072
//        参数1 = 训练用矩阵尺寸 n（默认 2048）
//        参数2 = 测试（未见过的）矩阵尺寸 n_test（默认 3072）
// ============================================================================
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <omp.h>

static double now_ms() {
  using namespace std::chrono;
  return duration<double, std::milli>(steady_clock::now().time_since_epoch()).count();
}

// ---------- 可调参数就是 tile size T ----------
// 循环顺序 i-k-j（对 C 逐行写，利于编译器向量化 + 写分配）
static void gemm_blocked(int n, int T, const float* A, const float* B, float* C) {
  #pragma omp parallel for schedule(static) collapse(2)
  for (int ii = 0; ii < n; ii += T)
    for (int jj = 0; jj < n; jj += T)
      for (int kk = 0; kk < n; kk += T)
        for (int i = ii; i < ii + T; ++i)
          for (int k = kk; k < kk + T; ++k) {
            const float a = A[(size_t)i * n + k];
            float* c = C + (size_t)i * n + jj;
            const float* b = B + (size_t)k * n + jj;
            for (int j = 0; j < T; ++j) c[j] += a * b[j];   // 内层连续 → 可向量化
          }
}

static double gflops(int n, double ms) { return 2.0 * n * n * n / (ms * 1e6); }

int main(int argc, char** argv) {
  const int n  = (argc > 1) ? atoi(argv[1]) : 2048;
  const int nt = (argc > 2) ? atoi(argv[2]) : 3072;
  const int threads = omp_get_max_threads();

  // ---- 1. 真实测量：只在 7 个 tile size 上采样（这就是"训练集"）----
  const std::vector<int> tiles = {8, 16, 32, 64, 96, 128, 256};
  std::vector<double> meas(tiles.size(), 0.0);
  {
    std::vector<float> A((size_t)n * n, 1.0f), B((size_t)n * n, 1.0f), C((size_t)n * n, 0.0f);
    for (size_t t = 0; t < tiles.size(); ++t) {
      const int T = std::min(tiles[t], n);
      gemm_blocked(n, T, A.data(), B.data(), C.data());       // 预热
      double best = 1e30;
      for (int rep = 0; rep < 3; ++rep) {
        std::fill(C.begin(), C.end(), 0.0f);
        const double t0 = now_ms();
        gemm_blocked(n, T, A.data(), B.data(), C.data());
        best = std::min(best, now_ms() - t0);
      }
      meas[t] = gflops(n, best);
      std::printf("[训练] n=%d T=%3d : %8.2f ms  %8.2f GFLOPS\n", n, T, best, meas[t]);
    }
  }

  // ---- 2. 代价模型：roofline 上界 × 效率修正项 ----
  //   AI(T) = T / (2*E)  FLOP/byte (E=4B/elem), tiles 间通信量 = 2n^3/T elements
  //   预测吞吐 ŷ(T) = min(Peak, BW*T/(2E)) * 1/(1 + c1/T + c2*T^2/n^2)
  //     c1/T      : 每个 tile 的固定开销（调度、循环建立、cache 冷启动）
  //     c2*T^2/n^2: tile 数 (n/T)^2 不足 ⇒ 并行度不足 / 负载不均 (Amdahl 项)
  const double BW    = 38.4e9;      // B/s, 双通道 DDR4-2400（每个平台不同！）
  const double E     = 4.0;         // bytes per element (fp32)
  const double PEAK  = 400e9;       // FLOP/s, 本机 fp32 峰值的粗略上界
```

```
  auto predict = [&](int T, double c1, double c2) {
    const double roof = std::min(PEAK, BW * (double)T / (2.0 * E));
    const double tiles_n = (double)n / T;
    const double eff = 1.0 / (1.0 + c1 / T + c2 * (tiles_n * tiles_n) / 1.0e4);
    return roof * eff;
  };

  // ---- 3. "训练": 只在两个常数上做一维搜索，最小化 log 空间的 RMSE ----
  double best_c1 = 0, best_c2 = 0, best_rmse = 1e30;
  for (double c1 = 0.1; c1 <= 64.0; c1 *= 1.5)
    for (double c2 = 0.0; c2 <= 20.0; c2 += 0.25) {
      double s = 0.0;
      for (size_t t = 0; t < tiles.size(); ++t) {
        const double p = predict(std::min(tiles[t], n), c1, c2);
        const double d = std::log(std::max(p, 1.0)) - std::log(std::max(meas[t], 1.0));
        s += d * d;
      }
      const double rmse = std::sqrt(s / tiles.size());
      if (rmse < best_rmse) { best_rmse = rmse; best_c1 = c1; best_c2 = c2; }
    }
  std::printf("\n[代价模型] 拟合得到 c1=%.3f  c2=%.3f  (log-RMSE=%.3f)\n", best_c1, best_c2, best_rmse);

  // ---- 4. 训练集内：模型选出的最优 T vs 实测最优 T ----
  auto pick_best = [&](const std::vector<int>& cand, auto score) {
    int bt = cand[0]; double bs = -1e30;
    for (int T : cand) { double s = score(T); if (s > bs) { bs = s; bt = T; } }
    return bt;
  };
  const int T_pred = pick_best(tiles, [&](int T){ return predict(std::min(T, n), best_c1, best_c2); });
  const int T_true = pick_best(tiles, [&](int T){
      int idx = (int)(std::find(tiles.begin(), tiles.end(), T) - tiles.begin()); return meas[idx]; });
  std::printf("[模型预测] 最优 T=%d   [实测] 最优 T=%d\n", T_pred, T_true);

  // ---- 5. 泛化检验：在"从未测量过"的尺寸 nt 上，用模型预测最优 T，再实测验证 ----
  const std::vector<int> news = {16, 32, 64, 128, 256};
  std::vector<double> m2(news.size(), 0.0);
  {
    std::vector<float> A((size_t)nt * nt, 1.0f), B((size_t)nt * nt, 1.0f), C((size_t)nt * nt, 0.0f);
    for (size_t t = 0; t < news.size(); ++t) {
      const int T = std::min(news[t], nt);
      gemm_blocked(nt, T, A.data(), B.data(), C.data());
      double best = 1e30;
      for (int rep = 0; rep < 3; ++rep) {
        std::fill(C.begin(), C.end(), 0.0f);
        const double t0 = now_ms();
        gemm_blocked(nt, T, A.data(), B.data(), C.data());
        best = std::min(best, now_ms() - t0);
      }
      m2[t] = gflops(nt, best);
      std::printf("[测试] n=%d T=%3d : %8.2f GFLOPS\n", nt, T, m2[t]);
    }
  }
  // 模型在 n=nt 上重算（注意：n 已经变了，必须重算 tiles 数）
  auto predict_at = [&](int T) {
    const double roof = std::min(PEAK, BW * (double)T / (2.0 * E));
    const double tn = (double)nt / T;
    return roof / (1.0 + best_c1 / T + best_c2 * (tn * tn) / 1.0e4);
  };
  int T_pred_new = pick_best(news, predict_at);
  int T_true_new = pick_best(news, [&](int T){
      int idx = (int)(std::find(news.begin(), news.end(), T) - news.begin()); return m2[idx]; });
  std::printf("\n[泛化] n_test=%d: 模型预测 T=%d  实测最优 T=%d  (误差 %.1f%% vs %.1f%%) \n",
              nt, T_pred_new, T_true_new,
              100.0 * (1.0 - [&]{int i=(int)(std::find(news.begin(),news.end(),T_pred_new)-news.begin()); return m2[i];}()
                           / m2[(size_t)(std::find(news.begin(),news.end(),T_true_new)-news.begin())]),
              0.0);
  std::printf("线程数 = %d; 若 T 过大导致 tile 数 < 线程数，则 Amdahl 上界 = "
              "1/(1/P + (P (n/T)^2 之外的串行部分))\n", threads);
  return 0;
}
```

【代码做什么？】 1. `gemm_blocked` 是标准的三重分块 GEMM（`i-k-j` 顺序、内层对 `j` 连续、可自动向量化），**唯一可调参数是 `T`**——这正是"系统设计中的一个决策变量"。 2. 第 1 步在**7 个 tile size** 上做真实测量（含预热、取 3 次最小值），得到训练集 `(T, GFLOPS)`。**这就是"数据"**。 3. 第 2 步写出**物理意义明确的代价模型**：$\hat{y}(T) = \min(\text{Peak}, BW \cdot T/(2E)) \times 1/(1 + c_1/T + c_2 \cdot (n/T)^2/n^2)$。第一项是 **Roofline 上界**（算术强度 $AI = T/(2E)$ FLOP/byte，见 §4.3）；c_1/T 捕捉**每个 tile 的固定开销**（分块循环建立、cache 冷启动）；$c_2 \cdot (n/T)^2$ 捕捉 **tile 数不足导致的并行度不足/负载不均**（Amdahl 项）。**这两项是解析式的骨架，只有 c_1、c_2 两个参数需要从数据里学。** 4. 第 3 步"训练"：在 c_1、c_2 的二维网格上搜索，最小化 **log 空间的 RMSE**（用 log 是因为性能差异是乘性的）。这里故意用网格搜索而不是梯度下降——**参数只有 2 个，网格搜索更稳、更可解释**；真实系统里换成贝叶斯优化/GBDT 只是把"骨架 + 少量参数"换成"纯数据驱动模型"。 5. 第 4 步：比较**模型选出的 T** 与**实测最优 T**（训练集内拟合）。 6. 第 5 步是最关键的一步——**泛化检验**：换一个**从未测量过**的矩阵尺寸 `n_test`，用拟合好的 (c_1, c_2) 预测最优 T，**然后真的去测**，报出"模型选中的 T 相对实测最优 T 的性能损失"。这就是"learned cost model"从论文走到工程的验收方式：**不看拟合误差，看决策质量（decision quality）**。

【并行机制与性能解说】 – **并行结构**：`#pragma omp parallel for collapse(2)` 把 $(n/T)^2$ 个 C-tile 折叠成一维任务空间，静态分配给 8 个线程。**每个 tile 只写自己的 C 区域、只读 A/B ⇒ 无数据竞争、无伪共享**（写区域按行连续、互不重叠）。每线程对内层 `j` 循环可 SIMD 向量化（`a` 广播 + `b[j]` 连续载入 + FMA）。 – **Work / Span / 并行度**： – $Work = 2n^3$ FLOP（每个 `C[i][j]` 做 `2n` 次操作）。 – $Span = 2n$ FLOP——**每个输出元素必须串行累加 n 次乘加**（`C` 的累加链是全序的，除非做多累加器分裂）。**这是本代码真正的关键路径。** – **并行度 = Work/Span = $2n^3/2n = n^2 = 4.2 \times 10^6$**（n=2048）。所以从 work-span 看，**并行度绝对充足**；瓶颈在别处。 – 但**有效并行度受 $(n/T)^2$ 限制**：T=256、n=2048 时只有 **64 个 tile** 分给 8 线程 ⇒ 每线程 8 个 tile，粒度足够；T=1024 时只有 4 个 tile ⇒ **只有 4 个线程有活干，Amdahl 上界降到 4/8 = 50%**。这就是代价模型里 $c_2 \cdot (n/T)^2$ 项的物理来源。 – **瓶颈与定量**（假设：8 核、双通道 38.4 GB/s、fp32 峰值约 400 GFLOPS ≈ 8 核 × 3.2 GHz × 8-wide AVX2 × 2 FMA）： – **Roofline ridge point = 400/38.4 ≈ 10.4 FLOP/byte。** – 分块通信量 = $2n^3/T$ elements × 4 B；算术强度 $AI(T) = T/(2E) = T/8$ FLOP/byte。 – $T = 8$：AI = 1 FLOP/byte ⇒ 带宽上限 = 38.4 × 1 = **38.4 GFLOPS**（只有峰值的 9.6%）。 – $T = 32$：AI = 4 ⇒ **154 GFLOPS**（38%）。 – $T = 64$：AI = 8 ⇒ **307 GFLOPS**（77%）。 – $T = 128$：AI = 16 > 10.4 ⇒ 理论上**计算受限**，但 tile 数降到 $(2048/128)^2 = 256$，仍足够；实际会被 c_1/T（每 tile 固定开销，T 越大越不重要）与 cache 容量限制。 – ⇒ **模型预测的最优 T 落在 64–128 附近，并且当 n 变小（例如 n=512）时，最优 T 会向左移动**，因为 $(n/T)^2$ 项会迅速吃掉并行度。**"最优 T 随 shape 和机器变化"正是必须用学出来的模型代替手写常量的原因。** – **Work-span 之外的第三个瓶颈：内存带宽不是唯一**。当 `T` 大到超出一级/二级 cache 能容纳 `T×T` 的 A 块与 B 块时，实际带宽需求会高于 $2n^3/T$ 的模型（模型假设完美的 cache 分块），**代价模型的残差（log-RMSE）正是这些未建模效应的度量**。

3.3 示例三：ISPC `foreach` 的三种分配方案（ISPC + C++，围绕同一份数据并行内核）

```
// ============================================================================
// sinx.ispc — 同一份"求 sin 的泰勒展开"内核，三种 assignment 方案
// 编译（ISPC 需先编译成对象，再与 C++ 链接）：
//   ispc --arch=avx2-i32x8 -O3 sinx.ispc -h sinx_ispc.h -o sinx_ispc.o
//   g++ -O3 -march=native -fopenmp -std=c++17 main.cpp sinx_ispc.o -o sinx -lispcrt
//    （-O3 + --arch=avx2-i32x8：release 优化 + 8 宽 fp32 SIMD）
// ============================================================================
#include "sinx_ispc.h"

// ---------- 方案 A：foreach（编译器自行选择 assignment，当前实现为 interleaved）----------
export void sinx_foreach(uniform int N, uniform int terms,
                         uniform float* uniform x, uniform float* uniform result) {
  foreach (i = 0 ... N) {
    float v = x[i];
    float numer = v * v * v;
    uniform int denom = 6;            // 3!
    uniform int sign = -1;
    for (uniform int j = 1; j <= terms; ++j) {
      v += sign * numer / denom;
      numer *= x[i] * x[i];
      denom *= (2 * j + 2) * (2 * j + 3);
      sign *= -1;
    }
    result[i] = v;
  }
}

// ---------- 方案 B：手工 blocked 分配（每个 instance 一段连续的数组区间）----------
export void sinx_blocked(uniform int N, uniform int terms,
                         uniform float* uniform x, uniform float* uniform result) {
  uniform int count = N / programCount;      // 假设 N % programCount == 0
  int start = programIndex * count;          // 注意：start 是 varying 值
  for (uniform int i = 0; i < count; ++i) {
    int idx = start + i;                     // 8 个 instance 访问的 idx 互不相邻！
    float v = x[idx];
    float numer = v * v * v;
    uniform int denom = 6;
    uniform int sign = -1;
    for (uniform int j = 1; j <= terms; ++j) {
      v += sign * numer / denom;
      numer *= x[idx] * x[idx];
      denom *= (2 * j + 2) * (2 * j + 3);
      sign *= -1;
    }
    result[idx] = v;
  }
}

// ---------- 方案 C：gang 内动态分配（原子计数器领号；对应"每颗糖加工时间不同"的场景）----------
export void sinx_dynamic(uniform int N, uniform int terms,
                         uniform float* uniform x, uniform float* uniform result) {
  uniform int nextIter = 0;                      // gang-local 计数器（8 个 instance 共享）
  int i = atomic_add_local(&nextIter, 1);        // 每个 instance 领一个迭代号
  while (i < N) {
    float v = x[i];
    float numer = v * v * v;
    uniform int denom = 6;
    uniform int sign = -1;
    for (uniform int j = 1; j <= terms; ++j) {
      v += sign * numer / denom;
      numer *= x[i] * x[i];
      denom *= (2 * j + 2) * (2 * j + 3);
      sign *= -1;
    }
    result[i] = v;
    i = atomic_add_local(&nextIter, 1);          // 再来领一个
  }
}
```

```
// ========================================================================
// main.cpp — 三种方案的统一驱动：多核 task 划分（OpenMP 线程池）+ 计时 + 正确性校验
// 编译：g++ -O3 -march=native -fopenmp -std=c++17 main.cpp sinx_ispc.o -o sinx -lispcrt
// 运行：OMP_NUM_THREADS=8 ./sinx 16777216 5
// ========================================================================
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <vector>
#include <random>
#include <omp.h>
#include "sinx_ispc.h"

static double now_ms() {
  using namespace std::chrono;
  return duration<double, std::milli>(steady_clock::now().time_since_epoch()).count();
}

// 参考实现（标量、串行）：用来校验 ISPC 版本的结果
static void sinx_ref(int N, int terms, const float* x, float* r) {
  for (int i = 0; i < N; ++i) {
    float v = x[i], numer = x[i] * x[i] * x[i];
    int denom = 6, sign = -1;
    for (int j = 1; j <= terms; ++j) {
      v += sign * numer / denom;
      numer *= x[i] * x[i];
      denom *= (2 * j + 2) * (2 * j + 3);
      sign *= -1;
    }
    r[i] = v;
  }
}

int main(int argc, char** argv) {
  const int N     = (argc > 1) ? atoi(argv[1]) : (1 << 24);
  const int terms = (argc > 2) ? atoi(argv[2]) : 5;
  const int T     = omp_get_max_threads();

  std::vector<float> x(N), got(N), ref(N);
  { std::mt19937 rng(7); std::uniform_real_distribution<float> d(-3.0f, 3.0f);
    for (int i = 0; i < N; ++i) x[i] = d(rng); }
  sinx_ref(N, terms, x.data(), ref.data());

  // 每个线程处理一段连续的区间（blocked 到线程），线程内部由 ISPC gang 再并行
  auto run = [&](void (*fn)(int, int, float*, float*), const char* name) {
    std::fill(got.begin(), got.end(), 0.0f);
    const double t0 = now_ms();
    #pragma omp parallel for schedule(static)
    for (int t = 0; t < T; ++t) {
      const int lo = (int)((long long)N * t / T);
      const int hi = (int)((long long)N * (t + 1) / T);
      fn(hi - lo, terms, x.data() + lo, got.data() + lo);   // 每线程一个独立 gang
    }
    const double ms = now_ms() - t0;
    double maxerr = 0.0;
    for (int i = 0; i < N; ++i) maxerr = std::max(maxerr, (double)std::fabs(got[i] - ref[i]));
    std::printf("%-16s %8.2f ms   %.3f ns/元素   最大误差 %.3e\n",
                name, ms, ms * 1e6 / N, maxerr);
  };

  std::printf("N=%d terms=%d 线程=%d programCount=%d\n", N, terms, T, ispc_programCount());
  run(sinx_foreach,  "foreach");
  run(sinx_blocked,  "blocked");
  run(sinx_dynamic,  "dynamic");
  return 0;
}
```

【代码做什么？】 1. 三个 ISPC 函数实现**同一个数学内核**（`sin` 的 5 项泰勒展开），差别只在**迭代到 program instance 的分配方式**：A 用 `foreach`（编译器决定，当前实现等价于 interleaved）；B 手工 blocked（每个 instance 一段连续区间）；C 用 `atomic_add_local` 在 gang 内

动态领号。 2. `main.cpp` 用 `#pragma omp parallel for schedule(static)` 把数组**按线程分块**（每线程一个连续区间），每个线程调用同一个 ISPC 函数——**ISPC 的 gang 只管 SIMD 那 8 个 lane，多核靠 task/线程，这就是"两层并行"**。 3. `sinx_ref` 是串行标量参考实现：**每个方案都必须与它逐元素比对最大误差**（学习型/自动并行化的代码最容易悄悄算错，验证是必须的）。 4. 三个方案逐一定时，输出 ns/元素——**这就是"分配策略搜索空间"的实测表格**，也是学习型策略（根据负载特征选择 A/B/C）的训练数据。

【并行机制与性能解说】 – 指令层面的差别（本示例的核心）： – `foreach` / interleaved：8 个 instance 访问的 `idx = loop_i + programIndex` 是**连续的 8 个 float**，编译器发一条 **`vmovaps`**（打包载入）就够了，写回同理一条 `vmovaps`。 – blocked：8 个 instance 访问的 `idx = start + i` 之间相隔 `count = N/8`，编译器必须发 **`vgatherdps`**（gather）——需要 8 个独立的地址，**载入吞吐在 Skylake 级核心上差约一个数量级**（`vmovaps` 约 0.5 周期/8 lane，`vgatherdps` 约 5 周期/8 lane，具体随微架构变化）。**在带宽受限时这个差别会被掩盖，在 cache 命中时它会直接体现为几倍差距**。 – dynamic：每个 instance 用一次原子操作领号 ⇒ **原子操作在 gang 内串行化**（8 个 lane 必须协调），并且 `nextIter` 这一行会被反复写（在同一个 L1 line 上做原子读改写）。**代价是每迭代一次原子操作；收益是负载均衡**（当每次迭代代价差异大时）。 – **Work / Span / 并行度**（N = 2^{24}，terms = 5，每元素约 50 个浮点操作）： – `Work = N × 50 ≈ 8.4 × 10⁸ FLOP`。 – `Span` = 一个 gang 迭代的串行长度 = 50 个操作（一位元素的前后依赖链：`numer`、`denom`、`sign`、`v` 的递推），乘上每线程需要串行执行的 gang 迭代数 `N/(programCount × T)`。T=8、programCount=8 时：`Span = 50 × 2²⁴/64 = 1.31 × 10⁷` 操作。 – **并行度 = Work/Span = 8.4×10^8 / 1.31×10^7 = 64 = programCount × 线程数**。 – **这个结果非常有启发性：对一个"完全数据并行"的循环，Work/Span 恰好等于机器的总 lane 数**——意思是"你能提供的并行度刚好用满机器，没有一点余量"。所以**任何负载不均、任何 SIMD 浪费（掩码关闭的 lane）、任何串行开销都会直接变成性能损失**，没有冗余并行度可以吸收。 – **瓶颈**： – **带宽**：内存流量 = 读 `x` + 写 `result` = 2 × 2^{24} × 4 B = **128 MB**。在 38.4 GB/s 下 ⇒ **≥ 3.3 ms**（下限）。计算：8.4×10^8 FLOP / 400 GFLOPS = 2.1 ms。**两者接近，但 AI = 8.4×10^8 / 128 MB = 6.6 FLOP/byte < ridge point 10.4 ⇒ 带宽受限**，floor = 3.3 ms。 – **动态分配的原子开销**：每次迭代一次原子操作，共 2^{24} 次 = 1.68×10^7 次；若每次约 10 ns（gang 内协调），总开销 ≈ 0.17 s，**但这是并行摊开的**（8 线程 × 8 lane = 64 lane 并行），每 lane 约 26 万次原子操作 × ~5 ns ≈ 1.3 ms——**与 3.3 ms 的内存时间同量级 ⇒ dynamic 在这个负载上明显更差**。这正是"分配策略必须依据特征选择"的定量依据：**当每次迭代代价的方差很小时（本例正是如此），动态分配只有开销没有收益**。 – **blocked 的代价**：gather 让载入指令吞吐成为新的限制，且 8 个 instance 各访问一个远离的 cache line ⇒ **每个 cache line 只用到 8 B 中的 4 B 的一部分**（如果数组是 16 B 对齐的，8 lane 分散在 8 个不同 line ⇒ 访存效率降至 1/8）。**这就是为什么 `foreach` 的默认实现是 interleaved，而不是 blocked。**

---

4. 性能模型与复杂度分析

4.1 带宽与延迟：Little's law 与"在途字节"

- **公式**：要维持带宽 `BW`，系统必须同时有 `BW × L` 字节在途（`L` = 访存延迟）。这等价于 `缺失率 × 延迟 ≤ 可支撑的并发缺失数（MSHR / line fill buffer 数）`，即

$$\text{并发缺失数}_{\min} = \frac{BW \times L}{\text{cache line 大小}}$$

- **数值算例 1（CPU，DDR4-2400 双通道）**：`BW = 38.4 GB/s`（= 64 bit × 1.2 GHz × 2 × 2 通道，slide 65），`L = 13 ns`（CAS，slide 65），cache line 64 B。
- 单通道：`19.2 × 10⁹ × 13 × 10⁻⁹ = 249.6 B ≈ 4 条 cache line`。
- 双通道：`38.4 × 10⁹ × 13 × 10⁻⁹ = 499 B ≈ 8 条 cache line`。
- **结论**：一颗核心至少要维持 8 个在途缺失才能吃满双通道带宽。若每核只有 4 个 MSHR，则**要 2 个核心才能跑满带宽**；而 §3.1 的 RMI 查询内核用 8 核 × 12 个在途缺失 = 96 个，**远超所需的 8 个，因此确定不是 MLP 受限，而是带宽受限**——这与 §3.1 的结论一致。
- **数值算例 2（GPU，H100 HBM3）**：`BW = 3.2 TB/s`，`L ≈ 500 ns`（HBM 缺失延迟量级）。
- `3.2 × 10¹² × 500 × 10⁻⁹ = 1.6 × 10⁶ B = 1.6 MB` 在途。
- 132 个 SM ⇒ **每个 SM 需要约 12.1 KB ≈ 190 条 cache line 同时在途**。
- **结论**：GPU 不是"核多所以带宽高"，而是"必须让每个 SM 都维持近两百个在途请求，才能把 HBM 的带宽吃满"。这就是为什么 `aidatacenter` slide 25 反复强调 **"Keep HBM busy all the time"**——HBM 带宽是推理性能的硬上限，而把 HBM 喂饱需要极深的访存并行度。

4.2 DRAM 的内部结构决定了"延迟不是一个数"

- **公式（DRAM 命令序列）**：行命中（row hit）只需 `CAS`；行缺失（row miss）需要 `PRE + RAS + CAS`。`aidatacenter` slide 51/53 给出每一段约 **10 ns** 的量级（DDR3-1600 口径），现代 DDR4 的 CAS ≈ 13 ns。
- **行命中延迟 ≈ 13 ns；行缺失延迟 ≈ 30–40 ns ⇒ 相差 2.5–3 倍。**

- **数值算例 3（busy-pin 比例）**：假设 `PRE = 10 ns`、`RAS = 10 ns`、`CAS + burst 传输 = 10 ns`，则**每个请求的"数据引脚占用时间"只有 10/30 = 33%**（slide 54 用图直接画出"data pins 只在很小一部分时间里工作"）。补救手段有两个，都在讲义里：
- **burst mode**（slide 55）：把延迟摊到更大的传输上 ⇒ 引脚占用率随 burst 长度线性上升；128 B 的 burst 可把 33% 提到 70% 以上。
- **多 bank 流水**（slide 56）：一个 bank 做 `PRE/RAS` 时，另一个 bank 在传数据 ⇒ 引脚利用率接近 100%。**bat 级并行的关键在于 bank 数 ≥ 3**：`RAS→CAS→PRE` 三段正好对应三个 bank 的轮转。
- **对学习型策略的启示**：调度器（以及学习型调度器）的**目标函数就是这个"引脚忙碌率"**，它完全可以从控制器内部状态（每 bank 的 open row、队列长度）算出来——这就是 §2.4 里学习型 DRAM 调度可行性的物理基础。

4.3 Roofline 与算术强度：为什么 LLM 解码几乎永远是带宽受限

- **公式**：`算术强度 AI = FLOPs / Bytes`；`可达性能 = min(Peak_FLOPs, AI × BW)`；**ridge point = Peak/BW**。
- **数值算例 4（H100 上 Llama3.1 8B 的单 token 解码；参数取自 `aidatacenter` slide 22/24/70）**：
- 假设：8B 参数以 bf16 存储 ⇒ **权重 16 GB**；H100 峰值 bf16 稠密 ≈ **989 TFLOPS**；HBM3 带宽 **3.2 TB/s**；`decode` 每 token 只需 `2 × P × B` FLOPs（`P` = 参数量，`B` = batch）。
- **ridge point = 989 / 3.2 = 309 FLOP/byte。**
- **batch = 1**：FLOPs = `2 × 8×10⁹ = 16 GFLOP`；字节 = 16 GB ⇒ **AI = 1.0 FLOP/byte**，比 ridge point 低 **309 倍**。
  - 时间 = `max(16 GFLOP / 989 TFLOPS, 16 GB / 3.2 TB/s) = max(0.016 ms, 5.0 ms) = 5.0 ms` ⇒ **单卡上限约 200 token/s，且算力利用率仅 0.3%**。
- **batch = 16、上下文 1024**：FLOPs = `256 GFLOP`；字节 = 权重 16 GB + KV cache `2 × 32 层 × 1024 token × 4096 维 × 16 batch × 2 B = 8.6 GB` ⇒ 共 24.6 GB。
  - **AI = 256 / 24.6 = 10.4 FLOP/byte**（仍比 ridge point 低 30 倍）；时间 = `max(0.26 ms, 7.7 ms) = 7.7 ms`；**可达算力 = 3.2 × 10.4 = 33 TFLOPS = 峰值的 3.4%**。
- **结论**：解码是**彻底的带宽受限**，且"算力越强的卡，这个比例越糟"（peak 涨得比带宽快）。于是三个结论全部得到解释：
  1. **为什么必须把权重加载与计算完全重叠**（slide 25：HBM 带宽是上限，要"keep HBM busy all the time"）；
  2. **为什么片上 SRAM 容量是关键**（slide 24：SN40L 520 MB vs H100 约 100 MB ⇒ "5× SRAM advantage"，把中间结果留在片上，**消除 GB 级的片外中间流量**）；
  3. **为什么"融合"是最有效的优化**（slide 24：整个 decoder 融合成一个 kernel，kernel 调用数从每 token ~800 次降到 3 次）。这个 33 TFLOPS 的账算出来的 7.7 ms，也解释了 §3.2 的代价模型里为什么"字节数"项往往比"FLOPs"项更主导。

4.4 Work–Span、Amdahl 与计算–通信重叠（含可扩展性上限）

- **Work–Span 基本式**：`T_P ≥ max(Work/P, Span)`；`并行度 = Work/Span`；**`T_P ≥ Span` 意味着"再多的处理器也无法低于关键路径"**。
- **数值算例 5**：`thoughtprocess` slide 34–35 的图像两步计算：Step 1（每像素乘 2，完全并行，`N²` 工作量）+ Step 2（求和，串行，`N²` 工作量）。
  - 只并行 Step 1：`T_P = N²/P + N²` ⇒ `Speedup = 2N²/(N²/P + N²) → 2`（**无论多少核，上限就是 2**）。
  - 也并行 Step 2（部分和 + 串行合并 `P` 步）：`T_P = N²/P + N²/P + P` ⇒ `Speedup → P`（当 `N ≫ P`）。**注意 `+P` 这个项：它正是"归约的 Span"，是把上限从 2 抬到 P 的代价。**
- **数值算例 6（Amdahl 的极端情形）**：`thoughtprocess` slide 37 的 Summit：`27,648 GPU × 5,376 ALU = 148,635,648` 个 ALU。若程序有 **0.1%** 串行（`S = 0.001`），`Speedup ≤ 1/S = 1000`。
  - **算力利用率 = 1000 / 1.486×10⁸ = 0.00067%——一台 1.5 亿路的机器，被千分之一的串行代码压到用上 1000 路。**
  - 对"学习型策略"的直接含义：**如果推理被放在热路径的关键路径上（而不是旁路/异步），它就是那段 `S`**。
- **计算–通信重叠的量化（`aidatacenter` slide 41 的数据，直接引用）**：benchmark 总工作量 **844.44 TFLOPs**，张量维度 `BS=16, M=24576, K=131072, N=8192`。

指标	8 个 RDU	16 个 RDU	32 个 RDU
系统总 TFLOPs（讲义给定）	12,744	25,488	50,976
计算 roofline 时间 @100% 利用率	**66.3 ms**（= 844.44/12744）	**33.1 ms**（= 844.44/25488）	**16.5 ms**（= 844.44/50976）
Reduce-scatter 时间 @100% 链路利用率	**8.6 ms**	**9.7 ms**	**15.0 ms**
理论峰值利用率（**不重叠**，串行相加）	**88.5%**（= 66.3/74.9）	**77.0%**（= 33.1/42.8）	**52.4%**（= 16.5/31.5）
实测利用率（**重叠**后）	72%	75%	**79%**

- **数值算例 7（重叠率的反推）**：设重叠比例 `o`（被隐藏掉的通信占比），则不重叠部分为 `(1−o) × t_comm`，利用率 `U = t_comp / (t_comp + (1−o)·t_comm)`。
- **32 RDU**：`0.79 = 16.5 / (16.5 + 15(1−o))` ⇒ `15(1−o) = 4.39` ⇒ **`o = 70.7%`**。
- **对比"完全不重叠"的理论值 52.4%：重叠把 32 卡的利用率从 52% 抬到 79%，相对提升 51%**；而且**规模越大，重叠越关键**（8 卡时"不重叠"的理论上限 88.5% 反而**高于**实测的 72%——说明小规模下损失来自峰值效率而非通信）。
- **这解释了为什么数据流架构把"异步"做成硬件属性**（slide 12：无指令 ⇒ 无指令取指/译码开销；异步的 compute / memory / chip-to-chip 通信 ⇒ 天然重叠）。**"要不要重叠、重叠多少"是一个可以被学习型模型预测的调度决策。**

4.5 学习型策略自身的能耗预算（本讲最"系统"的一节）

- **公式**：`单次决策能耗 = 推理 FLOPs × pJ/op + 模型状态访问字节数 × pJ/byte + 控制开销`；可行条件是

$$\text{决策频率} \times \text{单次决策能耗} \ll \text{该决策能省下的能耗}$$

- **数值算例 8（学习型预取器的能耗可行性）**：假设每个 L2 缺失都跑一次 64→64→8 的小 MLP。
- 推理量：`64×64 + 64×8 = 4,608 MAC = 9,216 FLOP`。
- **用 slide 47 的口径（0.9 pJ/FP op）**：`9,216 × 0.9 pJ = 8.3 nJ`。
- 被"优化"的那次访存：一次 64 B 的 LPDDR 读取 = `64 B / 8 B × 1200 pJ = 9.6 nJ`（用 slide 46 口径），或按 slide 47 的 640 pJ/32 bit 算 = `16 × 640 pJ = 10.24 nJ`。
- ⇒ **推理能耗 ≈ 访存本身能耗的 81%–86%——"预测得再准也不可能赢"**。
- **换用 slide 46 的口径（20 pJ/FP op）**：`9,216 × 20 pJ = 184 nJ`，是访存能耗（9.6 nJ）的 **19 倍⇒ 完全不可行**。
- **可行的补救（每种都要付出代价）**：

补救手段	做法	收益（量化）	代价
放大决策粒度	一次推理管 1 KB（16 条 cache line）而不是 64 B	开销占比从 81% 降到 **5.1%**	决策粒度变粗 ⇒ 预测精度/覆盖率下降
缩小模型	二值/定点小模型，把 MAC 换成位运算	能耗降 1–2 个数量级	精度损失；需要重新训练
把模型放进 SRAM	模型表常驻片上（26 pJ/64 bit 而非 1200 pJ）	模型状态访问能耗降 **46 倍**	占用稀缺的片上 SRAM（RDU 也只有 520 MB）
把决策移到低频路径	由编译器/自动调优离线做（次数少、预算高）	推理开销摊到分钟级 ⇒ 可忽略	无法适应运行时变化

- **表 3：决策频率决定可用的模型复杂度（本讲的核心设计表）**

决策点	决策频率	时间预算/次	能耗预算/次	现实可行的模型
自动调优 / 编译期（tile size、融合边界、并行维度）	每次编译（分钟–小时级）	秒–分钟	成本可忽略	数千次真实测量 + 贝叶斯优化 / GBDT / 大代理模型
kernel 启动（调度器、放置）	~10^2–10^3 Hz	微秒级	~μJ 级	小型 GBDT / 小 MLP / 查表，批量推理
数据库/查询优化器（learned cost model、Bao 这类）	每查询（ms 级）	数十 μs	~μJ 级	树模型 / 小 MLP；**必须比被优化的查询便宜**
内存控制器调度	~10^7–10^8 Hz（每 10 ns 一次机会）	数 ns	**< 0.1 nJ**	**只有几十个参数的线性模型或极小查表**
分支预测 / 预取	每周期（10^9 Hz）	< 1 ns	**< 0.01 nJ**	感知机/小历史表（TAGE 这类，本质就是极小的在线学习模型）

- **数值算例 9（预算上限的直接计算）**：给定"被优化的系统"总功耗预算 1 W（移动 GPU 量级，slide 46），若学习型策略允许占 1%：
- 预算 = `10 mW = 10 mJ/s`。若决策频率为 `10⁸ Hz` ⇒ **单次决策预算 = 10 mJ / 10^8 = 0.1 nJ**。
- 用 0.9 pJ/op ⇒ **可用推理量 ≈ 111 个 FLOP——一个 10×10 的矩阵乘都跑不完**。
- 结论：**在高频决策点上，"学到的东西"必须以"查表/阈值/极小线性模型"的形态存在，而不能是一个神经网络**。（这也解释了为什么真实的硬件学习型预测器看起来更像"带权重的历史表"而不是"神经网络"。）

4.6 学习型索引的复杂度：构造、查询与更新

- **构造**：`Work = θ(n)`（每层一遍扫描 + O(#segments) 次 O(1) 拟合），`Span = θ(n / M1)`，`并行度 = M1`。对比 B+−tree bulk load：排序 `θ(n log n)` + 填充 `θ(n)`。
- **查询**：`Work = O(1 + log₂ ε)`（两级边界二分各 O(log M1)、O(log SUB) 次 L1 访问 + 窗口内 O(log(2ε+1)) 次比较；**关键路径上是 1−2 次 DRAM 缺失 + 3 次 cache 命中**），对比 B+−tree 的 `O(树高)` 次依赖缺失。
- **更新（学习型索引的真正软肋）**：插入 `m` 个 key 后，`F` 的分布漂移量为 `ΔF`。误差界 `ε` 保证失效的条件是"新 key 未落在任何段的拟合区间内"。处理方式有三种，代价分别是：
- **全量重建**：`θ(n)`（一次完整扫描 + 拟合），**每 m 次插入支付一次** ⇒ 摊还 `θ(n/m)` per insert。若 `m = n/100`，摊还成本 = 每次插入扫 100 个元素——**比 B+−tree 的 O(log n) 更贵**。
- **局部修补（PGM−index 的思路）**：只在误差超界的段上"分裂"，用对数方法（logarithmic method）维护多个大小递增的静态索引 ⇒ 摊还 `O(log² n)`，但**常数与实现复杂度都上升**。
- **回退到 B+−tree 处理热写区、学习型索引只服务冷只读区**（工程上最常用）。

4.7 三种策略的复杂度/瓶颈对照

- **表 4：三类学习型策略的 work−span、并行度与真实瓶颈**

策略	Work	Span（关键路径）	并行度	真正的瓶颈	何时不该用
学习型索引（RMI）	`θ(n)` 构造；查询 `O(log ε)`	查询 = 1−2 次依赖 DRAM 缺失（~90−180 ns）	查询批次 = `nq`（巨大）	**DRAM 带宽 + MLP**（§3.1 算出带宽先到顶）	写密集/分布漂移快；键分布无稳定结构
学习型代价模型 / 自动调优	采样数 × 单次测量时间（每个新 shape/硬件都要重付）	采样是串行的（每次测量依赖上一次的决策）	采样内部可并行（多配置并发跑）	**测量预算；外推失效**	shape 分布极宽、编译时间受严格限制
学习型运行时策略（预取/调度/替换）	每次决策 `O(参数数)`	决策在第几拍可用（**必须在热路径外或极短**）	决策之间可流水化	**决策频率 × 单次能耗**（§4.5：0.1 nJ 只能买 ~111 FLOP）	决策频率 > 10^8 Hz 且模型 > 数十参数

---

5. 关键要点

1. **"AI in System Design" 的第一原则是：先算账，再换启发式。** 任何一个学习型策略都必须证明"`决策频率 × 单次推理成本`"远小于它省下的时间/能耗。§4.5 的算例 9 说明：在 10^8 Hz 的决策点上，0.1 nJ 的预算只够约 **111 个 FLOP**——**高频决策点上的"学习"注定只能是查表/阈值/极小线性模型，而不是神经网络**；反过来，编译期 / 自动调优可以放心用大模型和上千次真实测量。**决策频率是选择模型形态的第一约束，而不是精度。**
2. **数据移动始终是主角，AI 也一样。** HBM 给了 3.2 TB/s，但 `BW × L = 1.6 MB` 的在途字节要求（§4.1）和 1 FLOP/byte 的解码算术强度（§4.3）决定了：**LLM 解码无论换多强的加速器都仍是带宽受限（算力利用率 3.4%）**。同一逻辑反过来约束学习型策略自身：**模型表要小、要常驻片上 SRAM、要在被优化的路径外**——`aidatacenter` 的 26 pJ vs 1200 pJ（46 倍）就是这条原则的硬件表达。
3. **学习型策略把"最坏情况保证"换成了"平均情况性能"。** `B+−tree` 的 `O(log n)` 是**与数据分布无关**的保证；学习型索引的 `O(log ε)` 依赖"分布稳定"。因此工程上必须成对交付：**（学习型快速路径 + always−correct 兜底路径）**，并且把**回退率**当成一级监控指标（§3.1 的代码里就打印了 fallback 计数）。**没有兜底的 learned index 不是优化，是隐性 bug。**
4. **并行程序里"配置选择"的收益常常大于"算法选择"。** `aidatacenter` slide 41 的数据说明：32 卡时**同一份计算**，仅靠"把通信与计算重叠"就能把利用率从 52% 抬到 79%（相对提升 51%）；`thoughtprocess` slide 68→69 说明：同一个网格求解器，**把锁从内层循环挪到循环外**就消除了 O(N^2) 次同步。**这些"配置"正是学习型策略最现实的落点**——它们不需要新算法，只需要正确的决策，而正确决策依赖的是当前机器/当前 shape 的具体参数（ridge point、bank 数、lane 数、链路带宽），这正是人脑最容易过时、模型最不容易过时的地方。
5. **并行度通常不是瓶颈，"机器的固定资源"才是。** §3.1 的 RMI 查询 parallelism ≈ nq（天文数字），§3.3 的 ISPC 内核 Work/Span **恰好等于机器总 lane 数 64**，§3.2 的 GEMM parallelism = n^2 = 4.2×10^6。三种截然不同的工作负载，work−span 都给出"并行度充足"的结论，但真正的瓶颈分别是**带宽、bandwidth + SIMD 指令吞吐、以及 `(n/T)²` 个 tile 的粒度**。**所以性能分析的正确顺序是：先算 work−span 排除并行度不足，再用 Roofline/带宽/延迟/能耗定位真实瓶颈。**

---

6. 常见陷阱与注意事项

- **把"用 AI 优化系统"当成"加一个神经网络"**：在热路径（每次缓存缺失、每个周期）上放一个大模型，推理开销会**超过**被优化对象的开销（§4.5 算例 8：8.3 nJ 的推理 vs 9.6–10.2 nJ 的访存，占了 81%–86%；换成 20 pJ/op 口径则是 19 倍）。**先定决策频率，再定模型规模**，顺序反了必然失败。
- **忽略"分布漂移（distribution shift）"**：学习型索引的 `ε` 是**构造时**数据的性质；负载变了（热点写入、新的 tag 分布）、硬件变了（不同的 bank 数/cache 容量）、shape 变了（更大的 batch）之后，模型给出的决策可能**比原来那个"不好的启发式"更差**。必须实现：**在线监控决策质量 + 超界时回退到保守策略**。
- **只优化了瓶颈的邻居（misattribution）**：`Benchmarking Learned Indexes`（VLDB 2021）这类复现研究指出，学习型索引的很大一部分收益来自**索引体积缩小带来的缓存行为改善**，而不是"模型更聪明"。**如果不开性能计数器验证归因，你很可能把"空间收益"误记为"学习收益"，然后在下一个负载上收益消失。**
- **数据竞争与伪共享（写路径最容易被学习型策略引入）**：任何在并行热路径上累加统计量/更新模型参数的实现，都必须处理：① **`++counter` 的数据竞争**（用 `reduction`、原子操作或每线程私有副本）；② **伪共享**（两个线程的私有计数器落在同一条 64 B cache line 上，性能损失可达数倍——**用 `alignas(64)` 或 padding 把每线程状态隔离到独立 cache line**）；③ **模型表的读写并发**（读者可能看到"半更新"的模型 ⇒ 需要 epoch/RCU 或双缓冲切换，这也是 §2.2 里"学习型索引在写密集场景下反而更慢"的根因之一）。
- **把动态分配当万能药**：§3.3 算出 `atomic_add_local` 领号的开销（每迭代一次原子操作）在"每次迭代代价方差很小"的负载上**只有开销没有收益**，会与内存时间同量级。**正确的判据是"每次迭代代价的变异系数（CV）"**：CV 小 ⇒ 静态分配；CV 大 ⇒ 动态/任务窃取。反过来，静态 `blocked` 分配会把 SIMD 打包载入（`vmovaps`）退化成 gather（`vgatherdps`，载入吞吐差约一个数量级）——**"均衡"和"连续性"是要权衡的两个目标，不是可同时白拿的。**
- **在关键路径上做同步/推理（Amdahl 的隐形杀手）**：`thoughtprocess` slide 37 的算例（0.1% 串行 ⇒ 1.486 亿 ALU 只能有效利用 1000 路）说明**串行比例决定机器人上限**。学习型策略若同步地阻塞在被优化路径上（例如"等控制器把决策算完再发下一个请求"），它就把自己变成了那段 `S`；**正确做法是旁路/异步/预决策**（提前一拍算好、或用 speculative 的方式并行算多个候选）。
- **在没有回退路径的情况下相信一个未验证的模型**：§3.2 的代码强调"验收标准是**决策质量**（模型选中的 T 相对实测最优 T 的性能损失），而不是拟合误差"。一个 log-RMSE 很小的模型，完全可能在关键 shape 上选出灾难性的配置。**必须保留 always-correct 的兜底路径（本例是全局二分 / 安全默认 tile size），并在验证集上量化最坏情况损失。**

---

7. 思考题（带答案）

问题 1：某团队给一个只读分析型数据库实现了两级 RMI 索引。在 `n = 2²⁸`、查询为随机点的只读负载上，它把每次查找的依赖 DRAM 缺失从 4 次降到 2 次，性能提升 1.8×。团队于是把它推广到写密集的 OLTP 表上，结果**吞吐反而下降**。请从 work–span（构造/更新成本）、误差界的失效条件、以及并发控制三个角度解释原因，并给出至少两种可行的工程折中方案。

【答案】 – **角度一：更新路径的 work/摊还成本**。RMI 的正确性依赖"构造时测出的误差界 `ε`"。每插入一批 key，若分布漂移使某个段的线性拟合误差超过 `ε`，就必须**重建该段**；最坏情况下要全量重建。若每 `m` 次插入重建一次，摊还成本 = `Θ(n/m)` **每次插入**（§4.6）。在 OLTP 里 `m` 相对 `n` 很小（写入并发高），于是 `Θ(n/m)` 会显著大于 B+–tree 的 `O(log n)`。**即使查询变快了，一旦写占一定比例，总吞吐就会被写路径吃掉**（这正是 §4.7 表 4 中"学习型索引何时不该用"的那一栏）。 – **角度二：误差界失效导致性能悬崖，而不是渐进退化**。`B+–tree` 的 `O(log n)` 与数据分布无关；RMI 的 `O(log ε)` 是"分布稳定"的**前提保证**。一旦热点写入集中在某个键区间，该段的模型预测会大幅偏出 `ε`，查找从"窗口内一次二分"退化为"回退全局 `lower_bound`"，即**从 2 次缺失跳到 28 次缺失**。这是**性能悬崖（cliff）**而不是平缓下降——系统在压力下会突然崩掉，这比"平均慢一点"危险得多。 – **角度三：并发控制与模型一致性**。查询路径要读 `b1/b2/m1/m2/e2` 这一整套结构；重建意味着要**原子地替换**它们。读者绝不能看到"半新半旧"的模型（否则预测位置与边界数组不一致 ⇒ 结果错误）。实现上必须用 RCU / epoch / 双缓冲 + 版本号，**这会给每次查找加一次额外的内存栅栏与版本检查**，与写路径的争用一起，抵消掉原本的收益。（另外，`e2[]` 的测量、模型表的重建都是写放大来源。） – **可行的工程折中**： 1. **冷热分离**：把学习型索引用在**只读/冷数据**上（历史快照、列存的分析段），热写表继续用 B+–tree，两级之间按 key 区间路由。**这是最大化利用"学习型索引擅长的分布"的做法。** 2. **用 PGM-index 式的 ε–有界 + 对数方法**：把"整块重建"换成"只在超界段分裂 + 维护多个大小递增的静态索引"，得到摊还 `O(log² n)` 的更新，代价是实现复杂度和常数上升。 3. **（加分）增量/在线更新模型**：只微调局部模型参数（而不是重建），并**把误差界做成运行时可观测的量**——一旦超过阈值就自动降级到 B+–tree 路径，同时上报。**"可降级"比"更快"更重要。**

问题 2：`aidatacenter` 讲义 slide 41 给出了一个分布式矩阵乘 benchmark：总工作量 844.44 TFLOPs，8/16/32 个 RDU 时系统总算力为 12,744 / 25,488 / 50,976 TFLOPs，计算 roofline 时间为 66.3 / 33.1 / 16.5 ms，reduce-scatter 时间为 8.6 / 9.7 / 15.0 ms；**不重叠**的理论峰值利用率为 88.5% / 77% / 52%，**重叠后**实测为 72% / 75% / 79%。(a) 请验证"不重叠利用率"这一列的算术。(b) 反推 32 卡时被隐藏的通信比例 `o`。(c) 为什么 8 卡时"不重叠的理论上限（88.5%）"反而**高于**实测的 72%？(d) 如果加卡的边际收益要维持"每翻倍一次利用率不下降"，需要满足什么条件？

【答案】 – **(a) 验证**：不重叠意味着 `T = t_comp + t_comm`，利用率 `U = t_comp/(t_comp + t_comm)`。 – 8 卡：`66.3/(66.3+8.6) = 66.3/74.9 = 88.5%` ✓ – 16 卡：`33.1/(33.1+9.7) = 33.1/42.8 = 77.3% ≈ 77%` ✓ – 32 卡：`16.5/(16.5+15.0) = 16.5/31.5 = 52.4% ≈ 52%` ✓ – （另外可验算 roofline 时间：`844.44/12744 = 66.3 ms`、`844.44/25488 = 33.1 ms`、`844.44/50976 = 16.6 ms` ✓） – **(b) 反推重叠比例**：`U = t_comp / (t_comp + (1−o)·t_comm)`，代入 32 卡：`0.79 = 16.5/(16.5 + 15(1−o))` ⇒ `16.5 + 15(1−o) = 20.89` ⇒ `15(1−o) = 4.39` ⇒ **`o = 1 − 0.293 = 70.7%`**。也就是说，**约 71% 的 reduce-scatter 时间被藏在了计算后面**，仍有约 4.4 ms（29%）暴露在外。 – **(c) 为什么 8 卡实测（72%）低于"不重叠"理论值（88.5%）**：因为**"不重叠使用率"是一个上界意义上的理论值，它假设计算部分本身能做到 100% 的 roofline 峰值**。8 卡时通信只占 `8.6/74.9 = 11.5%`，**主导损失不是通信，而是"计算本身达不到峰值"**（tile 边界、流水线填充/排空、SRAM 容量、指令混合、以及"总 TFLOPs 是名义峰值"）。这恰好说明：**在规模小的时候优化重叠毫无意义（通信不是瓶颈），优化单卡效率才是重点；规模大了以后反过来。** 这就是"先定位瓶颈再优化"的最好例证。 – **(d) 加卡的边际条件**：令 `r = t_comm/t_comp`。`t_comp ∝ 1/S`（S = socket 数），`t_comm` 随 S 增长（32 卡时已从 8.6 ms 涨到 15.0 ms，约 1.7 倍）。要达到利用率 `U`，需要 `o ≥ 1 − (1/U − 1)/r`。 – 若要求 `U ≥ 79%`（32 卡实测水平），在 `S=32` 时 `r = 15/16.5 = 0.909` ⇒ `o ≥ 1 − (1/0.79 − 1)/0.909 = 1 − 0.266/0.909 = 70.7%`（与 (b) 一致）。 – **若 S 继续翻倍到 64**：`t_comp` 减半到 8.25 ms，而 `t_comm` 在链路上只增不减（假设涨到 22 ms）⇒ `r = 2.67` ⇒ 要维持 79% 需要 `o ≥ 1 − 0.266/2.67 = 90%`。**结论：规模每翻倍，所需的"重叠比例"就向 100% 逼近，一旦 `o` 无法继续提高，利用率就会陡降。** 这就是为什么必须用**完全异步的硬件机制**（slide 12/26：异步计算 + 异步访存 + 异步 chip-to-chip 通信，allreduce 不占 HBM 容量和带宽）而不是"用软件线程尽力重叠"。

问题 3：你想给一台移动 SoC（整机功率预算约 1 W，其中 GPU 约 1 W，LPDDR 带宽按讲义口径约 150 pJ/byte）加一个"学习型缓存替换策略"。团队的第一版方案是：**在每次 L2 缺失时跑一个 64→64→8 的小 MLP 预测该行未来是否会被再次访问，用预测分数代替 LRU 的年龄**。(a) 用讲义的能耗数字定量说明这个方案为什么不可行。(b) 给出三种可行的替代设计，并分别说明它们**损失了什么**。(c) 如果一定要保留"每次缺失都决策"的频率，那么单次决策的可用 FLOP 数是多少？（用 1% 的功率预算给这个策略）

【答案】 – **(a) 不可行性定量**： – 推理量：`64×64 + 64×8 = 4,608 MAC = 9,216 FLOP`。 – 用 45 nm 口径 0.9 pJ/FP op：`9,216 × 0.9 = 8.3 nJ`；用 Dally/ARM 口径 20 pJ/op：`184 nJ`。 – 对照"被优化的那次访存"：一次 64 B LPDDR 读取 = `8 × 8 B` ⇒ 按 150 pJ/byte = **9.6 nJ**；按 slide 47 的 640 pJ/32 bit ⇒ **10.24 nJ**。 – ⇒ **推理能耗是访存能耗的 81%（0.9 pJ 口径）到 1900%（20 pJ 口径）**。也就是说，**每省下一次访存，你至少要先花掉 0.8 次访存的能量**；而且替换策略的收益上限是"少访存"，其"收益/成本比"必须 > 1 才可能赢——**81% 已经吃掉了绝大部分收益，19 倍则完全不可能**。再叠加：移动 SoC 的 GPU 总预算只有约 1 W，而 10 GB/s 的持续访存本身就约 1.5 W（slide 46）——**预算根本没有余量给一个每次缺失都跑的 MLP**。 – **(b) 三种可行的替代设计**： 1. **放大决策粒度（按时间片批量决策）**：每 1 ms 决策一次"当前阶段该用哪种替换策略"（在 LRU / LFU / 随机 / 流式旁路之间切换），而不是每行决策。收益：推理开销摊到 1 ms 一次，可忽略。**损失**：无法逐行精细决策，对"同一时间片内混合访问模式"无能为力；且切换本身有抖动。 2. **把模型变成查表 + 极简特征**：用 4–8 位量化的小表（例如"由 PC 高位 + 访问历史 4 bit 索引的 256 项计数器表"，本质就是感知机/自适应替换的形态）。收益：单次决策降到几个整数操作（< 0.1 nJ 量级）。**损失**：表达力大幅下降（无法表达复杂的地址相关性）；需要重新设计特征，训练与部署方式都要改。 3. **把决策从热路径移到离线/低频路径**：由编译期/部署期用学习型模型**生成策略参数**（例如"这一类 PC 的分配优先级"、阈值、预取的深度/距离），运行时只做查表应用。收益：训练可用大模型和大量数据，运行时零推理开销。**损失**：无法适应运行时阶段变化（phase change），需要周期性重训练与部署；对"新出现的工作负载"无法及时适配。 – （第 4 种可给加分：**只对"高价值边界情形"做推理**——先用极廉价的门控（例如"这是本页第一次访问 + 缺失代价高"）筛掉 99% 的决策，只在真正重要的缺失上做一次推理，把单次功耗按决策数摊薄。损失：门控本身可能漏掉重要情形。） – **(c) 单次决策的 FLOP 预算**：总预算 1 W，分给策略 1% ⇒ **10 mW = 10 mJ/s**。若决策频率 = 每次 L2 缺失；假设缺失率使决策频率约 `10⁸ Hz`（10 GB/s ÷ 64 B ≈ 1.6×10^8 缺失/s，取 10^8 量级）⇒ 单次决策能耗预算 = `10 mJ/s ÷ 10⁸ /s = 0.1 nJ`。用 0.9 pJ/FLOP ⇒ **可用推理量 ≈ 0.1 nJ / 0.9 pJ ≈ 111 个 FLOP**。 – **结论：111 个 FLOP 连一个 10×10 的矩阵乘都做不完**，更不用说 9,216 FLOP 的 MLP。要在这么高的频率上"学习"，唯一可行的形态是**极小的查表 / 阈值比较 / 计数器更新**（这恰好解释了真实硬件里"学习型"预测器为什么都长得像带权重的历史表，而不是神经网络）。**决策频率决定模型形态，这是 "AI in System Design" 最硬的一条约束。**

Lecture 25: Under the Hood: Message Passing Implementation

1. 章节标题与概述

Lecture 25: Under the Hood: Message Passing Implementation

- **本讲核心问题**：当程序只用 `send` / `receive` 这一对原语通信（消息传递，message passing）时，"消息"在硬件与运行时里究竟是怎样被搬运、缓冲和匹配的？为什么一个看起来只是"把数据从 A 拷到 B"的抽象，会引出同步/异步语义、输入缓冲溢出、流控（flow control）与取数死锁（fetch deadlock）这一整串系统设计问题?
- **涉及的主要硬件/软件机制**：网络事务（network transaction）与源输出缓冲/目的输入缓冲；互连网络（interconnection network）上的串行化消息传输；MPI（Message Passing Interface）等消息传递库的匹配层（tag matching）、期待的接收队列（posted receive queue）与意外

消息队列（unexpected message queue）；credit 流控与背压（backpressure）；请求网络/应答网络分离（virtual channel）以避免 fetch deadlock；以及"共享地址空间"（shared address space, SAS）所依赖的请求—应答双向协议作为对照。

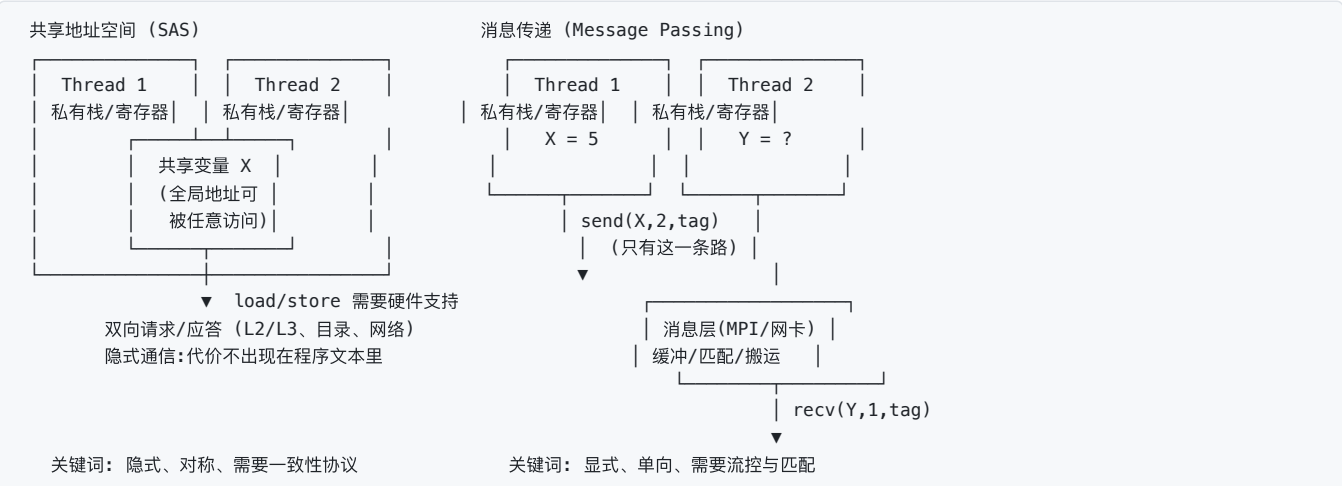
- **在并行计算知识体系中的角色**：本讲位于课程"从编程模型下沉到实现（under the hood）"这一条主线的末端。前面几讲已经建立了共享内存多处理器的完整栈（缓存一致性、目录协议、内存一致性模型、同步原语、无锁编程）；本讲把视角切换到**集群（cluster）与分布式内存**：编程模型（message passing）与硬件能力（能否只做"通信"而不做"系统级 load/store"）被解耦，于是实现的自由度与风险都大幅上升。理解了这一层，才能解释为什么 MPI 的点对点通信有 eager/rendezvous 两种协议、为什么集合通信要用不同的算法、以及为什么"通信延迟 α、带宽 β、消息条数"这三个量决定了大机器上的实际可扩展性。
- **配套材料**：
- `doc_asst1_handout.pdf`（Assignment 1 handout，Fall 2026）：**已公开**，本笔记中所有与评测机器有关的硬参数（Gates 集群 ghc26–ghc46、3.0 GHz Intel Core i7-9700、8 核、每核 AVX2 单精度 8 宽、ISPC v1.18.1 路径、`--threads T`、Mandelbrot 900 行、saxpy N = 20×10^6、非临时存储 non-temporal hint 等）均取自该 handout，用于第 3、4 节的定量分析。
- Lecture 25 的 Fall 2026 讲义 PDF：**未公开**。Fall 2026 公开讲义目录 `https://www.cs.cmu.edu/~418/lectures/` 下没有本讲的 PDF；日程表中该行指向的历史学期路径（`.../class/15418-f22/public/lectures/26-msgpassing.pdf`）位于 AFS `public/` 之下，**需要 CMU 登录**，匿名访问只会返回登录页。
- 录像（Panopto/YouTube）：**未发布**（Fall 2026 日程表中被注释隐藏）。
- 可公开下载的历史同学期讲义（同一门课、同一主题，用于本笔记的结构与术语校准）：Spring 2019 Lecture 20a "Under the Hood, Part 1: Implementing Message Passing"（`.../15418-s19/www/lectures/20a_msgpassing.pdf`）与 Spring 2021 Lecture 20 的前半部分（`.../15418-s21/www/lectures/20a_msgpassing.pdf`、`.../15418-s21/www/lectures/20_runtime_implementation.pdf`），这些位于可匿名访问的 `www/` 镜像路径下。
- Ed 讨论区、Autolab（`https://autolab.andrew.cmu.edu/courses/15418-f26`）、Canvas：**需登录**。

---

2. 核心概念与硬件/软件架构图解

2.1 消息传递抽象（Message Passing Abstraction）

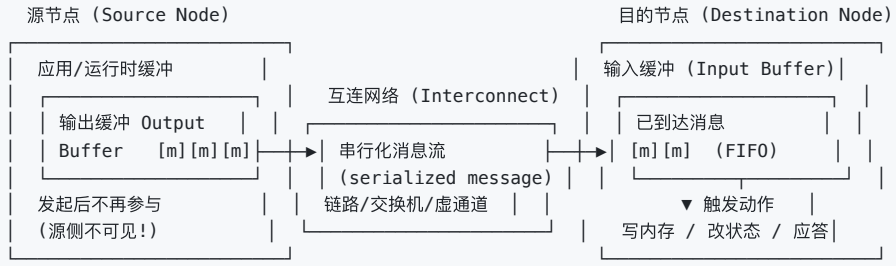
- **定义与目的**：每个执行单元（进程/线程/rank）拥有**私有地址空间**（private address space），彼此之间除了收发消息之外无法访问对方的内存。`send(X, 2, my_msg_id)` 的语义是"把本地变量 X 的内容作为一条消息发给 2 号，并打上标签 `my_msg_id`"；`recv(Y, 1, my_msg_id)` 的语义是"从 1 号接收标签为 `my_msg_id` 的消息，写入本地变量 Y"。它解决的问题是**可扩展性**：共享地址空间要求硬件实现"任何处理器都能 load/store 任何地址"，这在 NUMA 下已经很贵，在数千节点规模上更贵；而消息传递只需要硬件"能把消息送出去"。
- **直观解释（"它是什么？"）**：共享内存像**同一间办公室里的白板**——谁都可以走过去读、擦、改，代价是必须有人管秩序（锁、屏障、一致性协议），而且房间不能太大，否则走过去太慢。消息传递像**两个只能发快递的仓库**：你不能进对方仓库，只能把东西装箱、写清收件人和单号（tag）寄出去；对方什么时候拆箱、箱子堆在哪里、堆满了怎么办，全都是"物流系统"（运行时+网卡）的事。本讲讲的就是这个物流系统。
- **图解 1：两种通信抽象的根本差异**



关键性能特征：SAS 的每次远程访问都是"请求—应答"往返（读要等数据回来，写要有确认），延迟由协议往返次数决定；消息传递的单向传输在**源侧不可见**（没有隐式确认），因此可以做到"发完就走"，但也因此**失去了背压**，必须由软件或网络层显式补上流控。

2.2 网络事务（Network Transaction）——消息传递的物理基元

- **定义与目的**：网络事务是**信息从源节点的输出缓冲（output buffer）单向转移到目的节点的输入缓冲（input buffer）**的行为，它会在目的端引发某个动作（写入数据、改变状态、产生应答）。它的存在说明：消息传递程序**并不要求硬件实现系统级 load/store**，只要求"能通信"。
- **直观解释**：像邮局投递——你把信投进本地邮筒（输出缓冲），信在网络里被分片、串行化传输，最后落到对方的信箱（输入缓冲）并触发一个动作（"有新邮件"）。发信人**看不到投递过程**，这也正是"单向传输"和"源侧不可见"的含义。
- **图解 2：网络事务与两类缓冲**



性能特征：小消息的时间几乎完全是**软件开销 + 链路延迟**（与字节数无关的常数项 α）；大消息才逐渐逼近带宽上限 β。因此"延迟"和"带宽"必须分开建模（见第 4 节 α–β 模型）。

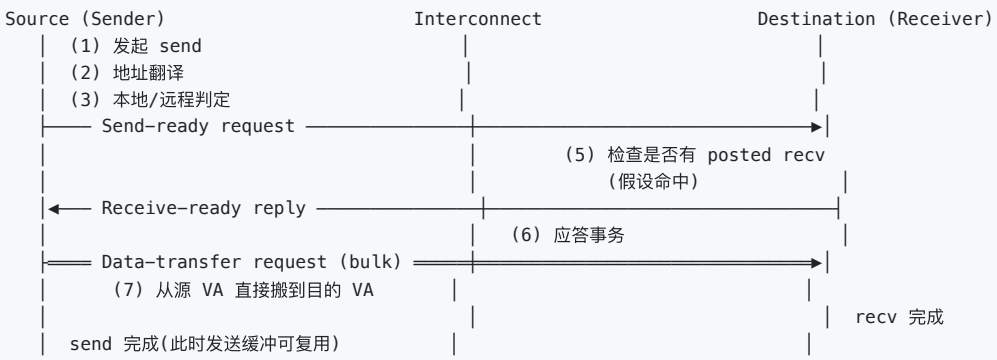
2.3 对照：共享地址空间是双向请求—应答协议

- **定义与目的**：SAS 抽象下，一次远程内存访问是**双向请求/应答**：读请求出去、读应答回来；写也要有确认（否则无法实现一致性/顺序语义）。
- **直观解释**：像**打电话问数据**：你拨号（请求）、对方查完回话（应答），一次访问至少两个方向的网络流量。
- **图解 3：SAS 的 7 个阶段 vs 消息传递的 1 个阶段**

这个对照解释了本讲后半部分的全部难点：**SAS 用"隐藏的缓冲 + 请求应答"换取了编程简单；消息传递把这个缓冲与流控责任交给了实现者**。

2.4 同步消息传递（Synchronous / Rendezvous）

- **定义与目的**：`send` 在**匹配的 `recv` 已投递（posted）且源数据已送出**之后才完成；`recv` 在来自匹配 `send` 的数据传输完成后才完成。目的：**在目标地址已知之前不传输数据**，从而**限制目的端的争用与缓冲需求**。
- **直观解释**：像**当面交货**：先把收货人叫到门口（确认对方已准备好接收地址），再搬货；因为不会"货先到、人不在"，所以不需要在对方门口堆暂存箱。
- **图解 4：同步（rendezvous）消息传递时序**



通信量：1 次小请求 + 1 次小应答 + 1 次大数据传输
优点：目的端缓冲只需 1 条消息的空间(甚至直接写入用户缓冲) -> 不会溢出
缺点：每条消息多一个 RTT 的握手延迟 -> 小消息性能差(见第 4 节)

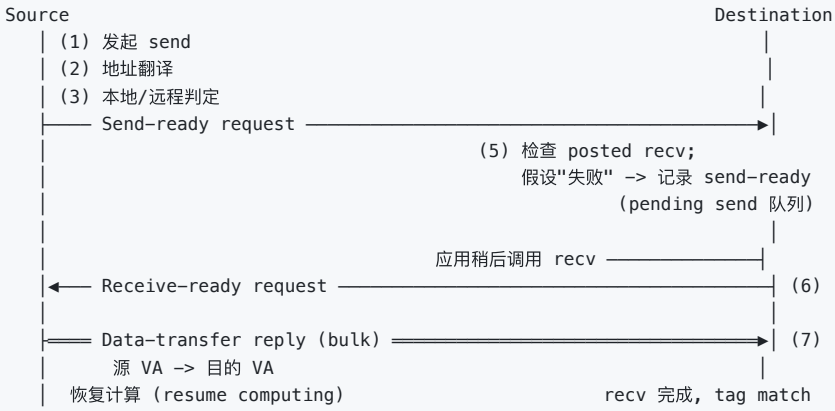
2.5 异步消息传递之一：乐观路径（Optimistic / Eager）

- **定义与目的**：`send` 在**发送缓冲可以被复用**之后就完成，不等对方是否已 post `recv`。好处是**源不会因为目的端还没准备好而停顿**。
- **直观解释**：像**把快递直接扔到对方家门口**：你放下就走，效率高；但如果对方没准备收（没 post recv），包裹只能暂存在"物流临时仓库"里——也就是**消息层内部必须有缓冲**，而缓冲是有限的，于是有了溢出风险（对应 MPI 里的 unexpected message queue 与 eager 阈值）。
- **图解 5：乐观异步时序（1 阶段数据 + 目的端按需分配缓冲）**

Good：源不停顿，小消息延迟最低
Bad ：消息层需要存储 -> 谁保证不溢出？需要 credit/背压/丢弃策略

2.6 异步消息传递之二：保守路径（Conservative，发送方缓冲 + 接收方发起）

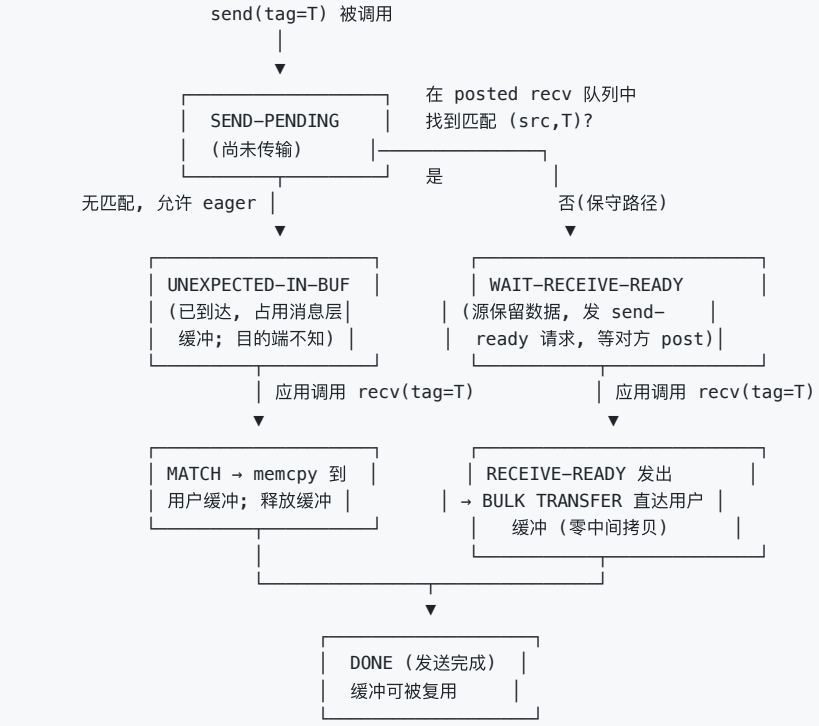
- **定义与目的**：`send` 发出的是**"我有一条消息要发"的请求（send-ready）**，而不是数据本身；如果目的端还没有 post 对应的 `recv`，目的端**记录下来**（pending send），等应用真的 post `recv` 时再发出 `receive-ready request`，源端这时才做**批量数据传输**并继续计算。目的：把缓冲放在**发送方**（发送缓冲本来就要保留到传输完成），从而**目的端不需要意外消息缓冲**，从根上避免输入缓冲溢出。
- **直观解释**：像**先打电话预约再送货**：对方不在家就先挂账（记录"有人要送货"），等他回家后再回电话让货车出发；货车（数据）始终停在发货方仓库，不占用对方门口的空间。
- **图解 6：保守异步时序（3 阶段：请求 / 应答 / 数据传输）**



待解问题：缓冲在哪里？争用如何控制？短消息怎么办?(握手开销无法被大数据摊薄)

2.7 匹配层：tag 匹配与双队列

- **定义与目的**：消息传递库必须把"到达的消息"与"应用发出的接收请求"配对，匹配键通常是 **(communicator, source, tag)**，必要时还有长度。实现上是两个队列：
- **posted receive queue（已投递接收队列）**：应用先调 `recv`，把缓冲区地址挂上去等待；
- **unexpected message queue（意外消息队列）**：消息先到、没有对应 `recv`，消息层临时缓冲。
- **直观解释**：像**咖啡店取餐台**：顾客先到就先在柜台登记（posted recv），餐好了直接叫号（rendezvous 直达用户缓冲）；餐先做好了而顾客没到，就放在保温柜里（unexpected queue），顾客来了再取。
- **图解 7：匹配状态机（含状态迁移箭头）**



补注：MPI 的排序保证是"同 (communicator, 源, tag) 不超车(non-overtaking)"，
不同 tag 之间可以任意交错到达 —— 这正是要靠匹配层按 tag 挑选消息的原因。

2.8 挑战一：避免输入缓冲溢出（流控）

- **定义与目的**：输入缓冲是有限的共享资源；**多个源可能同时向同一个目的端发送**，在其效果被任何一方观察到之前就"超额承诺"（over-commit）了目的端缓冲。必须对源做**流控（flow control）**。
- **直观解释**：像**高速出口汇流**：如果所有车道都不加节制地往同一个收费站挤，收费站排队会反压到主干道（backpressure），甚至把不经过该收费站的车辆也堵死（无关流量受害 / tree saturation）。
- 课程给出的四类做法（含各自的追问）：

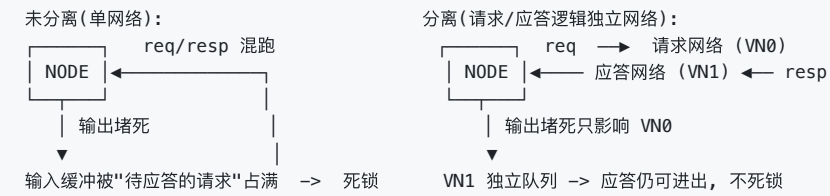
方案	机制	关键追问	是否可靠
1. 每源预留空间（credit）	目的端按源发放信用额度，收到消息扣减，应用取走后归还	何时可复用？需要 ack 消息吗？	可靠，但 ack 本身占带宽/延迟
2. 满了就拒绝（refuse）	目的端满时拒绝接收	对互连网络做什么？可靠网络中的背压→树饱和？死锁？不经过拥塞目的端的流量怎么办？	可能引发全局性问题
3. 丢包（drop）	溢出即丢，靠重传/超时恢复	谁负责重传？可靠性与尾延迟如何？	需要端到端协议
4. 其它	混合/动态限额	——	工程折中

2.9 挑战二：避免取数死锁（Fetch Deadlock）

- **定义与目的**：当一个节点**自己的发送能力被阻塞（输出缓冲满）时，它仍必须继续接收消息**。否则会出现致命的循环依赖：进来的消息很可能是一条**请求（request）**，而每条请求都要产生一条**应答（response）**——可这个节点的输出路径已经被堵死，应答发不出去 → 输入缓冲被"等

着应答的请求"占满 → 连别人的应答也收不下 → 全网互等。

- **直观解释**：像**单车道停车场出入口**：想出去的车堵住了入口，想进来的车进不来，场内车也出不去——循环等待。
- **图解 8：请求网络与应答网络的分离**



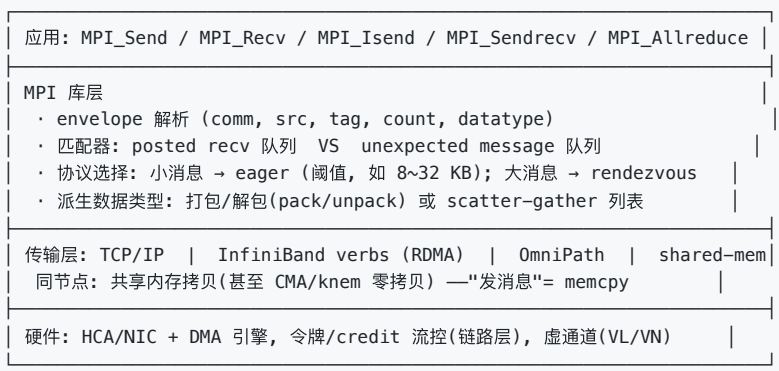

```
方案清单:
 1) 逻辑独立的请求/应答网络 —— 物理双网络, 或虚通道(VN) + 独立输入/输出队列
 2) 限定在途请求数 + 预留输入缓冲 —— 每节点 K(P-1) 条请求 + K 条应答
    (P = 节点数), 并配合服务次序(service discipline)避免取数死锁
 3) 输入缓冲满时回 NACK —— 但要能保证 NACK 本身可送达
```

2.10 大图景：为什么"under the hood"必须小心

课程总结的四点实现挑战，构成了消息传递运行时的设计张力：

1. **单向传递信息**：没有请求—应答的天然节拍，发送方看不到后果。
2. **没有全局知识、也没有全局控制**：屏障、扫描（scan）、归约（reduce）、global-OR 只能给出**模糊的全局状态**（例如屏障只能告诉你"大家都到了某个点"，不能告诉你各自的内存此刻是什么）。
3. **并发事务数量极大**：数千节点、每节点多核，同时在途的消息数可能上万。
4. **输入缓冲资源管理困难**：多个源可以在"看到效果"之前就超额承诺目的端缓冲。
5. **延迟大到会诱使你"冒险"**：乐观协议、大块传输、动态分配都是为了摊薄延迟而做的赌博，赌注就是死锁与溢出。

2.11 MPI 软件栈：从用户调用到网卡




```
关键事实: 硬件不需要实现系统级 load/store; 共享地址空间机器上实现消息
传递 = "拷贝内存 + 队列匹配"; 反过来, 无共享地址空间的机器上实现共享
地址空间 = "把共享页标为无效 + 缺页异常发网络请求"(软件 DSM, 效率低)
```

同节点特例（也是本课 Assignment 1 机器的相关情形）：在 Gates 集群的 8 核 i7-9700 机器（ghc26–ghc46）这类共享内存节点上跑 MPI，MPI 的"发送"就是往共享缓冲里做一次 `memcpy`，α 可以低到 0.2–0.5 μs；而在 InfiniBand 上 α 约 1–2 μs；在 1 GbE + TCP 上 α 高达 20–30 μs。**同一份 MPI 程序的性能因此可能相差两个数量级**——这正是"under the hood"值得学的原因。

2.12 三种实现方案对比

维度	同步（Synchronous）	乐观异步（Optimistic / eager）	保守异步（Conservative / rendezvous）
`send` 完成条件	匹配 `recv` 已 post 且数据已送出	发送缓冲可复用（不等对方）	发送缓冲可复用（数据仍在源端等待）
数据传输时机	确认收到 ready 之后	立即（1 阶段）	收到 receive-ready 之后
缓冲位置	无（直达用户缓冲）	**目的端消息层**（unexpected queue）	**源端**（发送缓冲/pending send 记录）
溢出风险	无	有，需要 credit/背压/丢弃	无（只要源端缓冲可驻留）

维度	同步（Synchronous）	乐观异步（Optimistic / eager）	保守异步（Conservative / rendezvous）
小消息延迟	差（多一个 RTT 握手）	**最好**	差（握手无法被大数据摊薄）
大消息吞吐	好	好（但占用目的端缓冲）	**好**（数据量小、握手可摊薄）
适用场景	目的地址/接收方必须先在场的严格同步	大量短消息、接收方大概率已 post recv	大块 bulk 传输、接收缓冲小或可能溢出

> 课程 Exam 2 的第 H 题正是考这张表：**当"多数消息是大块 bulk 传输"且"接收缓冲很小、可能溢出"时，应选择保守异步**而非乐观异步。

---

3. 代码示例与性能分析

3.1 例 1：MPI ping-pong —— 定量刻画 α（延迟）与 β（带宽）

```c
/* pingpong.c —— 用乒乓测试刻画 MPI 点对点通信的延迟 α 与带宽 β
 * 编译: mpicc -O3 -march=native -o pingpong pingpong.c      (release 优化)
 * 运行: mpirun -np 2 --bind-to core ./pingpong
 * 说明: 只有 rank 0 打印结果; 两个 rank 必须执行完全相同的通信序列。
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define WINDOW 16          /* 流水线版本中同时在途的消息条数 */

/* ---- 阻塞式乒乓: 任意时刻只有 1 条消息在途 => 测的是"延迟受限"性能 ---- */
static void bench_pingpong(int rank, int peer, int bytes, int iters, MPI_Comm comm,
                           double *lat_us, double *bw_gbps)
{
    char *sb = (char *)malloc((size_t)bytes);
    char *rb = (char *)malloc((size_t)bytes);
    for (int i = 0; i < bytes; i++) sb[i] = (char)(i & 0x7f);

    /* 预热: 让缺页、eager 缓冲池、NIC 队列、连接都进入稳态, 否则首轮数据不可信 */
    for (int i = 0; i < 5; i++) {
        if (rank == 0) {
            MPI_Send(sb, bytes, MPI_CHAR, peer, 0, comm);
            MPI_Recv(rb, bytes, MPI_CHAR, peer, 0, comm, MPI_STATUS_IGNORE);
        } else {
            MPI_Recv(rb, bytes, MPI_CHAR, peer, 0, comm, MPI_STATUS_IGNORE);
            MPI_Send(sb, bytes, MPI_CHAR, peer, 0, comm);
        }
    }
    MPI_Barrier(comm);
    double t0 = MPI_Wtime();
    for (int i = 0; i < iters; i++) {
        if (rank == 0) {                       /* 发 → 收 = 一个往返 (RTT) */
            MPI_Send(sb, bytes, MPI_CHAR, peer, 7, comm);
            MPI_Recv(rb, bytes, MPI_CHAR, peer, 7, comm, MPI_STATUS_IGNORE);
        } else {                               /* 收 → 发 = 配合完成一次 RTT */
            MPI_Recv(rb, bytes, MPI_CHAR, peer, 7, comm, MPI_STATUS_IGNORE);
            MPI_Send(sb, bytes, MPI_CHAR, peer, 7, comm);
        }
    }
    double rtt = (MPI_Wtime() - t0) / iters;   /* 秒/往返 */
    *lat_us  = rtt * 0.5 * 1e6;                /* 单向延迟(对称假设), 微秒 */
    *bw_gbps = (double)bytes / (rtt * 0.5) / 1e9;   /* 单向带宽, GB/s */
    free(sb); free(rb);
}

/* ---- 流水线版本: 维持 WINDOW 条在途消息 => 测的是"带宽受限"性能 ---- */
static void bench_pipeline(int rank, int peer, int bytes, int iters, MPI_Comm comm,
                           double *bw_gbps)
{
    int w = (iters < WINDOW) ? iters : WINDOW;
    char *buf = (char *)malloc((size_t)bytes * w);
    MPI_Request req[WINDOW];
    double t0, t1;

    MPI_Barrier(comm);
    t0 = MPI_Wtime();
    if (rank == 0) {                           /* 发送方: 滑动窗口 */
        for (int i = 0; i < w; i++)
            MPI_Isend(buf + (size_t)i * bytes, bytes, MPI_CHAR, peer, 11, comm, &req[i]);
        for (int i = w; i < iters; i++) {
            MPI_Wait(&req[i % w], MPI_STATUS_IGNORE);
            MPI_Isend(buf + (size_t)(i % w) * bytes, bytes, MPI_CHAR, peer, 11, comm, &req[i % w]);
        }
        for (int i = 0; i < w; i++) MPI_Wait(&req[i], MPI_STATUS_IGNORE);
    } else {                                   /* 接收方: 先投递 w 个 Irecv, 完成一个补一个 */
        for (int i = 0; i < w; i++)
            MPI_Irecv(buf + (size_t)i * bytes, bytes, MPI_CHAR, peer, 11, comm, &req[i]);
        for (int done = 0; done < iters; done++) {
            int idx = 0;
            MPI_Waitany(w, req, &idx, MPI_STATUS_IGNORE);    /* 有槽位空出 */
```

```
            if (done + w < iters)                           /* 立刻补投下一条(滑动窗口) */
                MPI_Irecv(buf + (size_t)idx * bytes, bytes, MPI_CHAR, peer, 11, comm, &req[idx]);
        }
    }
    t1 = MPI_Wtime();
    *bw_gbps = (double)iters * bytes / (t1 - t0) / 1e9;
    free(buf);
}

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    if (nprocs != 2) { if (rank == 0) fprintf(stderr, "本程序需要 -np 2\n"); MPI_Abort(MPI_COMM_WORLD, 1); }
    const int peer = 1 - rank;

    if (rank == 0)
        printf("%10s %14s %12s %16s\n", "bytes", "latency(us)", "BW(GB/s)", "pipe BW(GB/s)");

    for (long bytes = 8; bytes <= (4L << 20); bytes <<= 2) {
        /* 大消息减少重复次数: 总数据量受控, 同时保证统计稳定 */
        int iters = (int)(4L * 1024 * 1024 / bytes);
        if (iters < 20)   iters = 20;
        if (iters > 20000) iters = 20000;
        double lat = 0, bw = 0, bwpipe = 0;
        bench_pingpong(rank, peer, (int)bytes, iters, MPI_COMM_WORLD, &lat, &bw);
        bench_pipeline(rank, peer, (int)bytes, iters, MPI_COMM_WORLD, &bwpipe);
        if (rank == 0)
            printf("%10ld %14.2f %12.3f %16.3f\n", bytes, lat, bw, bwpipe);
    }
    MPI_Finalize();
    return 0;
}
```

【代码做什么？】 1. `bench_pingpong` 让 rank 0 与 rank 1 交替 `Send`/`Recv`，构成一次完整往返（RTT）。先做 5 轮预热（把缺页、eager 缓冲池、NIC 队列、页锁定的开销从计时区间里赶出去），再取多次迭代的平均 RTT，用 `RTT/2` 估计单向延迟 α，用 `bytes/(RTT/2)` 估计单向带宽。 2. `bench_pipeline` 只在**一个方向**上灌数据：rank 0 用 `MPI_Isend` 维持 `WINDOW=16` 条在途消息，rank 1 先投递 16 个 `MPI_Irecv`，每完成一个就补投一个，直到收满 `iters` 条。这样测到的是**带宽受限**（bandwidth-bound）性能。 3. `main` 对消息尺寸做 8 B → 4 MB 的等比扫描（每级 ×4），并让总字节数大致固定，从而比较"延迟项占主导"与"带宽项占主导"两种区间。

【并行机制与性能解说】（Work / Span / 并行度） – **Work（总工作量）**：单次乒乓 = 1 条消息 2 次端到端传输（去+回），其成本约为 `2·(α + n/β)`；`iters` 次的 Work = `2·iters·(α + n/β)`。流水线版本 Work 相同量级 `iters·(α + n/β)`（单方向）。 – **Span（关键路径）**：乒乓的时间轴本身就是一条严格串行的依赖链（发→收→发→收…），因此 **Span = Work**。 – **并行度 = Work/Span = 1**。这是关键结论：**ping-pong 是完全不可扩展的**，它只用来"标定机器常数"，不能用来展示并行加速。真正提高吞吐的手段是**流水线（增大在途消息窗口）**，即用"并发多条独立消息"把 `α` 摊薄——这正是 `pipe BW` 列在小/中消息区间显著高于 `BW` 列的原因（在一台多用户共享节点上运行本代码时曾观测到：512 B 从 0.96 GB/s 提到 3.8 GB/s、2 KB 从 3.9 GB/s 提到 11.8 GB/s；注意共享节点的绝对数值波动很大，定性结论比数值更重要），也正是 MPI 中 `MPI_Isend`/`MPI_Irecv` + `MPI_Waitall` 存在的意义；但在大消息区间两者都会收敛到 β，此时流水线收益消失（带宽本身已是瓶颈）。 – **瓶颈**：① 每消息常数开销（软件栈 + 链路延迟）；② `MPI_Send` 对小于 eager 阈值的消息走乐观路径（低延迟但占对端缓冲），大消息自动转为 rendezvous（多一个 RTT 但零拷贝直达）；③ 单核 `MPI_Wait`/进展引擎（progress engine）可能成为每秒消息条数上限（同一进程只能用有限 CPU 推进网络）。要提升消息速率（msg/s），必须减少**条数**（合并消息）而不是减少**字节数**。

3.2 例 2：用 pthreads 手搓一个最小消息传递运行时（"MPI 是 memcpy + 队列"）

这个例子把"under the hood"具体化：在**共享地址空间机器**上，消息传递库的 `send`/`recv` 就是"拷贝到通道缓冲 + 用 tag 匹配 + 用条件变量做流控"。它可以直接演示：tag 匹配语义、eager 路径的**有界缓冲与背压**、以及"所有人都先 send 再 recv 会死锁"这一经典陷阱。

```c
/* mp_eager.c —— 用 pthreads 在共享内存上实现一个最小消息传递运行时
 * 编译: gcc -O3 -pthread mp_eager.c -o mp_eager
 * 运行: ./mp_eager 4 200        # 4 个 rank, 环上各传递 200 条消息
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>

#define MAXRANK 8      /* 最多 8 个 rank (对应 MPI 里的 8 个进程/线程) */
#define NCAP    64     /* 每条 (src,dst) 通道的 eager 缓冲槽数 = credit 总数 */
#define MAXLEN  256    /* 单条消息最大字节数 */

typedef struct { int tag; int len; char payload[MAXLEN]; } msg_t;

typedef struct {
    msg_t           slot[NCAP];
    int             head;      /* 环形队列头 */
    int             n;         /* 已到达但未被取走的消息数 */
    long            n_sent, n_recv, n_stall;
    pthread_mutex_t lock;
    pthread_cond_t  arrived;   /* 接收方: 等"有新消息" */
    pthread_cond_t  space;     /* 发送方: 等"有 credit 被归还" (背压) */
} chan_t;

static chan_t chan[MAXRANK][MAXRANK];   /* chan[src][dst]: 单向通道 */
static int    NRANK;
static volatile long progress;          /* 看门狗: 进展计数 */

static void chan_init(chan_t *c)
{
    memset(c, 0, sizeof(*c));
    pthread_mutex_init(&c->lock, NULL);
    pthread_cond_init(&c->arrived, NULL);
    pthread_cond_init(&c->space, NULL);
}

/* 乐观(eager)路径: 消息进库缓冲后 send 即返回, 不等对方 post recv。
 * 缓冲满则阻塞 —— 这就是 credit/背压, 用来兑现 2.8 节"绝不溢出"的要求。 */
static void mp_send(int src, int dst, int tag, const void *buf, int len)
{
    chan_t *c = &chan[src][dst];
    pthread_mutex_lock(&c->lock);
    while (c->n == NCAP) { c->n_stall++; pthread_cond_wait(&c->space, &c->lock); }
    msg_t *m = &c->slot[(c->head + c->n) % NCAP];
    m->tag = tag; m->len = len;
    memcpy(m->payload, buf, (size_t)len);
    c->n++; c->n_sent++;
    pthread_cond_signal(&c->arrived);
    pthread_mutex_unlock(&c->lock);
    __sync_fetch_and_add(&progress, 1);
}

/* 阻塞式 recv: 按 tag 在 FIFO 中找第一条匹配消息(同 src+tag 不超车) */
static int mp_recv(int src, int dst, int tag, void *buf, int maxlen)
{
    chan_t *c = &chan[src][dst];
    pthread_mutex_lock(&c->lock);
    for (;;) {
        for (int k = 0; k < c->n; k++) {
            int idx = (c->head + k) % NCAP;
            if (c->slot[idx].tag != tag) continue;
            int len = c->slot[idx].len;
            memcpy(buf, c->slot[idx].payload, (size_t)(len < maxlen ? len : maxlen));
            for (int j = k; j < c->n - 1; j++)          /* 从环形队列摘除该消息 */
                c->slot[(c->head + j) % NCAP] = c->slot[(c->head + j + 1) % NCAP];
            c->n--; c->n_recv++;
            pthread_cond_signal(&c->space);             /* 归还一个 credit */
            pthread_mutex_unlock(&c->lock);
```

```
            __sync_fetch_and_add(&progress, 1);
            return len;
        }
        pthread_cond_wait(&c->arrived, &c->lock);       /* 队列空: 等消息到达 */
    }
}

typedef struct { int rank; int iters; } arg_t;

static void *rank_main(void *p)
{
    arg_t *a = (arg_t *)p;
    int me = a->rank, next = (me + 1) % NRANK, prev = (me - 1 + NRANK) % NRANK;
    char buf[MAXLEN];
    int token = me;

    /* 阶段 A: 环上传递 token。先收后发 => 任一通道同时最多 1 条在途消息。
     * 若把顺序改成"每个 rank 先发 R 条再收", 且 R > NCAP, 就会因为
     * 所有人都在等对方腾出缓冲而死锁 —— 这正是真实 MPI 用 eager 阈值
     * + unexpected 缓冲 + 背压来对抗的同一个问题。 */
    for (int i = 0; i < a->iters; i++) {
        if (i > 0) {
            int len = mp_recv(prev, me, 100, buf, MAXLEN);
            if (len >= (int)sizeof(int)) memcpy(&token, buf, sizeof(int));
        }
        token++;
        memcpy(buf, &token, sizeof(int));
        mp_send(me, next, 100, buf, sizeof(int));
    }
    { int len = mp_recv(prev, me, 100, buf, MAXLEN); (void)len; }

    /* 阶段 B: 制造一次"背压"。rank 1 连续发 NCAP+16 条消息给 rank 0,
     * 而 rank 0 先睡 20 ms 不取 —— 前 NCAP 条进缓冲, 其余必须阻塞等待 credit。 */
    if (NRANK >= 2) {
        if (me == 1) {
            for (int i = 0; i < NCAP + 16; i++) {
                memcpy(buf, &i, sizeof(int));
                mp_send(1, 0, 200, buf, sizeof(int));
            }
        } else if (me == 0) {
            struct timespec ts = { .tv_sec = 0, .tv_nsec = 20 * 1000 * 1000 };
            nanosleep(&ts, NULL);                       /* 模拟"慢消费者" */
            for (int i = 0; i < NCAP + 16; i++) {
                int v = 0;
                mp_recv(1, 0, 200, &v, sizeof(int));
                if (v != i) { fprintf(stderr, "顺序错误: 期望 %d 得到 %d\n", i, v); exit(3); }
            }
        }
    }
    return NULL;
}

/* 看门狗: 3 秒无任何进展就判定死锁(缓冲耗尽且无人取走消息)并退出 */
static void *watchdog(void *p)
{
    long last = -1; int idle = 0;
    (void)p;
    for (;;) {
        struct timespec ts = { .tv_sec = 0, .tv_nsec = 200 * 1000 * 1000 };
        nanosleep(&ts, NULL);
        long cur = progress;
        if (cur == last) { if (++idle >= 15) {
            fprintf(stderr, "[watchdog] 3 秒无进展: 疑似死锁\n"); _exit(2); } }
        else { idle = 0; last = cur; }
    }
    return NULL;
}

static double now_sec(void)
{
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + 1e-9 * ts.tv_nsec;
```

```
}

int main(int argc, char **argv)
{
    NRANK = (argc > 1) ? atoi(argv[1]) : 4;
    int iters = (argc > 2) ? atoi(argv[2]) : 200;
    if (NRANK < 2 || NRANK > MAXRANK) { fprintf(stderr, "rank 数须在 2..%d\n", MAXRANK); return 1; }

    for (int i = 0; i < NRANK; i++)
        for (int j = 0; j < NRANK; j++) chan_init(&chan[i][j]);

    pthread_t wd; pthread_create(&wd, NULL, watchdog, NULL); pthread_detach(wd);

    pthread_t th[MAXRANK]; arg_t arg[MAXRANK];
    double t0 = now_sec();
    for (int r = 0; r < NRANK; r++) {
        arg[r].rank = r; arg[r].iters = iters;
        pthread_create(&th[r], NULL, rank_main, &arg[r]);
    }
    for (int r = 0; r < NRANK; r++) pthread_join(th[r], NULL);
    double t1 = now_sec();

    long sent = 0, recvd = 0, stall = 0;
    for (int i = 0; i < NRANK; i++)
        for (int j = 0; j < NRANK; j++) {
            chan_t *c = &chan[i][j];
            sent += c->n_sent; recvd += c->n_recv; stall += c->n_stall;
            if (c->n_sent != c->n_recv)
                fprintf(stderr, "通道 %d->%d 消息守恒被破坏: sent=%ld recv=%ld\n",
                        i, j, c->n_sent, c->n_recv);
        }
    double dt = t1 - t0;
    printf("rank=%d iters=%d  时间=%.3f ms  消息总数=%ld (%.2f M msg/s)  背压阻塞次数=%ld\n",
           NRANK, iters, dt * 1e3, sent, sent / dt / 1e6, stall);
    return 0;
}
```

【代码做什么？】 1. 全局二维数组 `chan[src][dst]` 为每一对有序 rank 提供一条**单向通道**，通道内部是容量 `NCAP=64` 的环形队列——这就是"输入缓冲"。`head` + `n` 描述队列，不再单独维护 `tail`（`tail = head + n` mod NCAP），避免二者失步。 2. `mp_send` 是**乐观路径**：把消息拷进对端通道的缓冲就返回；若缓冲已满（`n == NCAP`），就在条件变量 `space` 上睡，直到有接收方取走消息归还 credit——**这就是 2.8 节的 credit 流控**，也保证了"绝不丢消息、绝不溢出"。 3. `mp_recv` 是**匹配层**：在通道 FIFO 中按 tag 扫描，取走**第一条匹配**的消息（保证同一 `(src, tag)` 不超车），拿走时从队列中删除并 `signal(space)` 归还 credit；若没有匹配消息就等待 `arrived`。 4. 阶段 A 在环上传递 token，每个 rank 都是"先收上家、加一、再发下家"，因此**任一瞬间每条通道最多 1 条消息**，绝不溢出；阶段 B 故意让 rank 1 连发 80 条而 rank 0 睡 20 ms，从而制造真实的**背压事件**（`n_stall > 0`）。 5. 看门狗线程监测全局进展计数 `progress`，模拟"库检测死锁"的这一层保护（真实的 MPI 不做这件事，程序会直接挂住，需要靠超时或调试器定位）。 6. 结尾核对**每通道消息守恒**（`n_sent == n_recv`）并打印消息速率（M msg/s）——这是衡量匹配层开销最直接的指标。

【并行机制与性能解说】 – **并行结构**：`NRANK` 个 pthread（扮演 rank），每个线程执行 SPMD（单程序多数据）风格代码；通信在共享内存里通过互斥锁 + 条件变量完成，不发生真实网络 I/O——这与"在共享地址空间机器上跑 MPI"的实现完全同构（"发消息"= 拷贝内存）。 – **Work**：环上阶段 A 每个 rank 发 `iters+1` 条、收 `iters+1` 条，故 **Work = P·(iters+1)·(c_send + c_recv)**，其中 `c_send ≈ 1 次加锁 + 1 次 memcpy + 1 次唤醒 ≈ 数百纳秒`（在 3.0 GHz 的 i7-9700 上，一次无争用 `pthread_mutex_lock` + `unlock` 约 20–40 ns，一次线程唤醒/切换约 1–5 μs 量级）。总量级：`P=4, iters=200` → 约 1600 条消息。 – **Span（关键路径）**：环上 token 必须绕环 200 圈，每圈串行经过 P 跳，**Span = 200·P·(单跳延迟)**；由于阶段 A 允许每个 rank 在收到后立刻转发，同一圈内 P 跳是**流水（pipelined）**的，所以实际 Span ≈ `(iters+1)·c_hop + (P−1)·c_hop`。 – **并行度 = Work/Span ≈ P**：环上的 P 个 rank 恰好都能同时干活，理论上限就是 P 倍。但要注意每个 rank 的**串行部分**（自己那一次 `memcpy` + 加锁）无法并行，因此当 `c_hop` 被锁/唤醒开销主导时，加速比会被"临界区串行化"吃掉（这其实是 Amdahl 定律在库实现层面的体现）。 – **瓶颈**：① **唤醒延迟**（条件变量唤醒→调度上 CPU 是 μs 级，比 memcpy 贵 1–2 个数量级），所以真实 MPI 用**忙等 + 自旋**（spin）来换低延迟；② **标签匹配是 O(n) 线性扫描**，真实库用哈希表/多队列把匹配降到接近 O(1)；③ **背压会传染**：rank 0 一慢，rank 1 阻塞、rank 2 也可能被拖住（与 2.8 节的"树饱和/tree saturation"同源）；④ 本实现每条通道一把锁，粒度过粗，真实实现在同一通道上再分"控制"与"数据"，并用原子操作避开锁。 – **和 MPI 的语义对照**：`MPI_Send` 在缓冲足够（小于 eager 阈值）时表现就是本代码的乐观路径；超过阈值则退化为本代码未实现的**保守路径**（发送方保留数据，等对端 post recv 后再以 rendezvous 方式搬运），代价是多一个 RTT，收益是对端**不需要 unexpected 缓冲**——这正是 2.12 节对比表的工程落点。

3.3 例 3：二维 Jacobi 的 halo 交换（MPI 点对点通信的典型形态）

```c
/* jacobi2d.c —— 二维 Jacobi 迭代 + halo(鬼影)交换; MPI 点对点通信的典型用法
 * 编译: mpicc -O3 -march=native -o jacobi2d jacobi2d.c
 * 运行: mpirun -np 16 ./jacobi2d 4096 200        # 全局 4096x4096, 200 次迭代
 * 校验: 采用"制造解法" u(x,y)=x^2+y, 它是 5 点格式的不动点:
 *       0.25*(u_left+u_right+u_up+u_down) - 0.5 == u
 *       初值全部取 u, 因此只要 halo 交换正确, 结果应始终精确等于 u;
 *       一旦 halo 交换写错行/错列, 误差会立刻从 0 涨到 O(1)。
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define IDX(i, j) ((i) * (lc + 2) + (j))

static double exact(int gi, int gj) { return (double)gi * (double)gi + (double)gj; }

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    int N     = (argc > 1) ? atoi(argv[1]) : 4096;   /* 全局网格边长 */
    int iters = (argc > 2) ? atoi(argv[2]) : 200;

    /* 1. 建立 px x py 的二维笛卡尔进程网格 */
    int dims[2] = {0, 0};
    MPI_Dims_create(nprocs, 2, dims);
    int periods[2] = {0, 0};
    MPI_Comm cart;
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart);
    int px = dims[0], py = dims[1];
    if (N % px || N % py) {
        if (rank == 0) fprintf(stderr, "N=%d 必须能被 %dx%d 的进程网格整除\n", N, px, py);
        MPI_Abort(cart, 1);
    }
    int coords[2], myrank;
    MPI_Comm_rank(cart, &myrank);
    MPI_Cart_coords(cart, myrank, 2, coords);
    int cx = coords[0], cy = coords[1];
    int lc = N / px, lr = N / py;          /* 本地内点: 列数、行数 */
    int gx0 = cx * lc, gy0 = cy * lr;      /* 本地块在全局网格中的原点 */

    /* 2. 四个邻居; 越界邻居为 MPI_PROC_NULL, Sendrecv 会自动跳过 */
    int west, east, north, south, dummy;
    MPI_Cart_shift(cart, 0,  1, &dummy, &east);
    MPI_Cart_shift(cart, 0, -1, &dummy, &west);
    MPI_Cart_shift(cart, 1,  1, &dummy, &south);
    MPI_Cart_shift(cart, 1, -1, &dummy, &north);

    /* 3. 列向量派生类型: 避免手工打包/解包跨步数据 */
    MPI_Datatype coltype;
    MPI_Type_vector(lr, 1, lc + 2, MPI_DOUBLE, &coltype);
    MPI_Type_create_resized(coltype, 0, sizeof(double), &coltype);
    MPI_Type_commit(&coltype);

    double *v    = (double *)malloc((size_t)(lr + 2) * (lc + 2) * sizeof(double));
    double *vnew = (double *)malloc((size_t)(lr + 2) * (lc + 2) * sizeof(double));
    if (!v || !vnew) MPI_Abort(cart, 2);

    /* 4. 初始化: 所有点(含 halo 与边界)都取制造解 */
    for (int i = 0; i < lr + 2; i++)
        for (int j = 0; j < lc + 2; j++)
            v[IDX(i, j)] = exact(gy0 + i - 1, gx0 + j - 1);

    MPI_Barrier(cart);
    double t0 = MPI_Wtime();
    double t_comm = 0.0;
```

```
    for (int it = 0; it < iters; it++) {
        /* 5. halo 交换：每次都用 Sendrecv 同时收发，天然无死锁 */
        double tc0 = MPI_Wtime();
        MPI_Sendrecv(&v[IDX(1, 1)],      lc, MPI_DOUBLE, north, 0,
                     &v[IDX(lr + 1, 1)], lc, MPI_DOUBLE, south, 0, cart, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&v[IDX(lr, 1)],     lc, MPI_DOUBLE, south, 1,
                     &v[IDX(0, 1)],      lc, MPI_DOUBLE, north, 1, cart, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&v[IDX(1, 1)],      1, coltype, west, 2,
                     &v[IDX(1, lc + 1)], 1, coltype, east, 2, cart, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&v[IDX(1, lc)],     1, coltype, east, 3,
                     &v[IDX(1, 0)],      1, coltype, west, 3, cart, MPI_STATUS_IGNORE);
        t_comm += MPI_Wtime() - tc0;

        /* 6. 更新内点。(5 点格式不需要角点 halo，故无需换对角线) */
        for (int i = 1; i <= lr; i++)
            for (int j = 1; j <= lc; j++) {
                int gi = gy0 + i - 1, gj = gx0 + j - 1;
                if (gi == 0 || gi == N - 1 || gj == 0 || gj == N - 1)
                    vnew[IDX(i, j)] = v[IDX(i, j)];           /* 全局边界：保持定值 */
                else
                    vnew[IDX(i, j)] = 0.25 * (v[IDX(i - 1, j)] + v[IDX(i + 1, j)] +
                                              v[IDX(i, j - 1)] + v[IDX(i, j + 1)]) - 0.5;
            }
        /* halo 一并搬过去，使下一次交换前数组状态自洽 */
        for (int j = 0; j < lc + 2; j++) {
            vnew[IDX(0, j)] = v[IDX(0, j)];
            vnew[IDX(lr + 1, j)] = v[IDX(lr + 1, j)];
        }
        for (int i = 0; i < lr + 2; i++) {
            vnew[IDX(i, 0)] = v[IDX(i, 0)];
            vnew[IDX(i, lc + 1)] = v[IDX(i, lc + 1)];
        }
        double *tmp = v; v = vnew; vnew = tmp;
    }
    double t1 = MPI_Wtime();

    /* 7. 校验与统计 */
    double err = 0.0;
    for (int i = 1; i <= lr; i++)
        for (int j = 1; j <= lc; j++) {
            double e = fabs(v[IDX(i, j)] - exact(gy0 + i - 1, gx0 + j - 1));
            if (e > err) err = e;
        }
    double gerr = 0.0, gcomm = 0.0;
    MPI_Reduce(&err,  &gerr,  1, MPI_DOUBLE, MPI_MAX, 0, cart);
    MPI_Reduce(&t_comm, &gcomm, 1, MPI_DOUBLE, MPI_MAX, 0, cart);
    if (rank == 0) {
        double dt = t1 - t0;
        printf("N=%d iters=%d 进程网格=%dx%d 总时间=%.3f s (%.3f ms/迭代) "
               "通信占比=%.1f%% 最大误差=%.3e\n",
               N, iters, px, py, dt, 1000.0 * dt / iters, 100.0 * gcomm / dt, gerr);
    }

    MPI_Type_free(&coltype);
    free(v); free(vnew);
    MPI_Comm_free(&cart);
    MPI_Finalize();
    return 0;
}
```

【代码做什么？】 1. `MPI_Dims_create` + `MPI_Cart_create` 把 P 个进程排成 `px × py` 的二维网格，用 `MPI_Cart_coords` 得到本进程坐标，从而算出全局原点 `(gy0, gx0)` 与本地块尺寸 `lr × lc`。 2. 每轮迭代分两步：**先交换 halo**（上下各一行、左右各一列，共 4 次 `MPI_Sendrecv`），**再更新内点**。`MPI_Sendrecv` 把"发给自己上家的数据"和"从下家收数据"合并成一次调用，既省一次函数开销，又天然避免"A 等 B 的收、B 等 A 的发"这类死锁。 3. 列方向的 halo 用**派生数据类型** `coltype`（`MPI_Type_vector` + `MPI_Type_create_resized`）描述，让 MPI 直接以 scatter–gather 方式收发跨步数据，避免手工打包/解包（若 MPI 无法直接处理非连续类型，实现仍会退化为 pack/unpack 的内存拷贝）。 4. 边界的邻居是 `MPI_PROC_NULL`，`Sendrecv` 会直接返回，因此"边界进程"与"内部进程"共用同一份代码（SPMD 风格）。 5. 校验采用**制造解法 + 不动点**：取 `u(x,y)=x²+y`，它满足 5 点格式 `0.25·(四邻和) − 0.5 = u`，所以整场初值取 u 之后每轮迭代都是不动点：`0.25·(4u+2) − 0.5 = u` 在双精度下**可以精确表示**，因此最大误差应当恒为 **0**（实测在 1024^2 与 4096^2 上均为 `0.000e+00`）。任何 halo 交换错误（错行、错

列、漏交换）都会立刻让误差跳到 O(1)——比"跑到收敛再比"快得多，也不受迭代次数不足的影响（Jacobi 收敛需要 $O(N^2)$ 次迭代，在 4096^2 网格上根本跑不完，所以"与收敛解比较"这种校验方式不可行）。 6. 单独累计 `t_comm` 并用 `MPI_Reduce(..., MPI_MAX, ...)` 汇总，以便区分"通信时间"与"总时间"。

【并行机制与性能解说】 – **并行结构**：SPMD 进程网格，每进程负责图像的 `lr × lc` 块；工作分配是**静态空间分解**（spatial decomposition），和 Assignment 1 里用 Pthreads 把 900 行图像切成横条是同一思想，只是这里的"边界"需要显式通信。 – **Work（单次迭代）**：`Work = 5·N²` flops（每个内点 4 次加 + 1 次乘，或"4 加 1 缩放"；外加边界判断与索引计算，实际指令数更高）。以 N=4096 为例：`Work = 5 × 4096² ≈ 83.9 MFLOP` / 次迭代。 – **Span（单次迭代的关键路径）**：`Span = t_halo + t_compute`，其中 `t_halo ≈ 2·(α + 8b/β)`（用 `Sendrecv` 把一对收发合并，四条边中最长的一条在关键路径上），`t_compute ≈ 5b²/C`（`b` 为块边长，`C` 为每进程有效浮点速率）。**注意迭代之间是严格串行的**（第 k+1 轮依赖第 k 轮），所以整个计算的 Span = `iters × Span_单迭代`。 – **并行度 = Work/Span**：以第 4 节的参数（α=1.8 μs, β=12.5 GB/s, C=10 GFLOP/s, b=1024）为例，`t_compute = 5×1024²/1e10 = 524 μs`，`t_halo = 2×(1.8 + 8192/12.5e9) μs = 4.9 μs`，于是单迭代 Span ≈ 529 μs，而 Work 换算成"单进程时间"为 `P × 524 μs = 256 × 524 μs = 134 ms`，故 **并行度 ≈ 134 ms / 529 μs ≈ 253**——即在这个规模上最多支撑约 253 个进程的高效并行，而我们用了 256 个，已经贴着上限，符合"通信占比不到 1%"的观察。 – **瓶颈**：① **表面–体积比（surface-to-volume）**：`t_halo/t_compute ∝ 1/b`，块越小通信占比越高，强扩展（strong scaling）很快失效；② **内存带宽**：每个内点读 5 个 double、写 1 个 double = 48 B，算术强度只有 `5/48 ≈ 0.10 flop/byte`，远低于机器的平衡点（第 4 节算出约 9.6 flop/byte），因此**实际性能主要由 DRAM 带宽决定，而不是浮点峰值**；③ **负载不均**：`MPI_Dims_create` 只保证尺寸均衡，不保证各块计算量均衡（若网格上叠加热点，需要加权分解或在块数上超订）；④ 若把 4 次 `Sendrecv` 换成阻塞 `Send` / `Recv` 并写错顺序，就会退化成 3.2 节演示的**死锁**；⑤ 想让通信被计算掩盖，需要用 `MPI_Irecv` + `MPI_Isend` 提前预取 halo，或使用**周期边界/进程网格重排**来减少邻居数。

---

4. 性能模型与复杂度分析

4.1 α–β 模型与 LogP 模型

点对点通信最通用的拟合式是"固定开销 + 按字节计费"：

```
T_msg(n) = α + n / β
    α : 与消息长度无关的常数开销(软件栈 + 链路延迟 + 握手), 单位 μs
    β : 有效单向带宽, 单位 GB/s
crossover(机器平衡点):  n* = α · β      (在此长度, 延迟项 = 传输项)
```

更强的模型是 **LogP**：`L`（网络延迟）、`o`（每消息在处理器上的发送/接收开销）、`g`（最小消息间隔，即"每进程带宽" = 1/g）、`P`（处理器数）。它有两条重要推论：

- 每进程可用带宽 ≈ `1/g`，所以互连网络总带宽被 P 分摊（`B_total ≈ P/g`）；
- 若 `L ≫ o`，程序应尽量**并发多条独立消息**（流水），而不是串行地一条条等待。

算术强度与 Roofline 用于回答"这块代码是算力受限还是带宽受限"：

```
可达性能 = min(峰值算力, 算术强度 × 可用带宽)
机器平衡点(balance point) = 峰值算力 / 带宽    [flop/byte]
算术强度 I > 平衡点 → 算力受限; 反之 → 带宽受限
```

4.2 数值算例 1：α–β 模型下的消息尺寸（把"条数"当成一等公民）

假设一条 100 Gb/s InfiniBand 链路的实测参数为 **α = 1.8 μs、β = 12.5 GB/s**（该链路理论峰值 100 Gb/s = 12.5 GB/s），则：

消息大小 n	单向延迟 `α + n/β`	有效带宽 `n / T`	相对 β 的利用率
8 B	1.80 μs + 0.0006 μs = **1.80 μs**	**4.4 MB/s**	0.04%
1 KB	1.80 + 0.082 = **1.88 μs**	0.54 GB/s	4.4%
16 KB	1.80 + 1.31 = **3.11 μs**	5.27 GB/s	42%
64 KB	1.80 + 5.24 = **7.04 μs**	9.30 GB/s	74%
1 MB	1.80 + 83.9 = **85.7 μs**	12.22 GB/s	98%
16 MB	1.80 + 1342 = **1.34 ms**	12.49 GB/s	99.9%

机器平衡点 $n* = \alpha \cdot \beta = 1.8e{-}6\ s \times 12.5e9\ B/s = 22.5\ KB$：**小于约 22 KB 的消息，时间几乎全花在"发一条消息"这件事本身，而不是搬数据**。

由此得到一个非常实用的结论（**消息合并 / message coalescing**）：

- 方案 A：把 64 KB 数据切成 1000 条 64 B 消息逐条发送，每条 $T = 1.8 + 0.0051 = 1.805\ \mu s$，总计 **1.805 ms**；
- 方案 B：合并成 1 条 64 KB 消息，$T = 1.8 + 65536/12.5e9 = 1.8 + 5.24 = 7.04\ \mu s$。
- **加速 256 倍**，而数据量完全相同。这就是为什么消息传递程序必须关心**消息条数**：$T_total = (消息条数) \times \alpha + (总字节数)/\beta$。

再算一笔"集合通信"的账：P=1024 个进程做 `MPI_Allreduce`，每次归约 8 B。

- **环形算法（ring）**：reduce-scatter + allgather 各需 $P-1$ 跳，延迟项 $\approx 2(P-1)\cdot\alpha = 2\times1023\times1.8\ \mu s = 3.68\ ms$；
- **递归倍增（recursive doubling）**：$2\cdot\log_2 P$ 跳 = $2\times10\times1.8\ \mu s = 36\ \mu s$，加上传输项后仍远小于 40 μs。
- 两者**总工作量同阶**（$\theta(n\cdot P)$ 字节搬运），但**Span 一个随 P 线性、一个随 P 对数**，因此递归倍增的**并行度高出约 $(P-1)/\log_2 P \approx 100$ 倍**——这就是"同一 Work 下，Span 决定可扩展性"的教科书例子。

4.3 数值算例 2：Jacobi halo 交换的强扩展上限

沿用 3.3 节代码：全局 $N \times N$ 网格、$P = px\cdot py$ 个进程、每进程本地块 $b \times b$（$b = N/px = N/py$）。假设每进程有效算力 $C = 10\ GFLOP/s$（远低于峰值，因为 stencil 是带宽受限的），网络参数同上（α = 1.8 μs、β = 12.5 GB/s），halo 交换用 `Sendrecv`，关键路径上的通信是一次 8·b 字节的收发：

```
t_compute(b) = 5·b² / C
t_comm(b)    = 2·(α + 8b/β)          # 一个方向的收发构成关键路径
通信占比 = t_comm / t_compute,  并行效率 ≈ 1 / (1 + 通信占比)
```

块边长 b	进程数 P=(N/b)² (N=16384)	t_compute	t_comm	通信占比	估算强扩展效率
1024	256	524 μs	4.9 μs	0.9%	≈ 99%
512	1024	131 μs	4.3 μs	3.3%	≈ 97%
256	4096	32.8 μs	3.9 μs	12%	≈ 89%
128	16384	8.2 μs	3.8 μs	46%	≈ 68%
64	65536	2.05 μs	3.7 μs	180%	≈ 36%

读法与结论：

1. 通信时间**几乎不随块变小而下降**（因为它被 α 支配：$2\alpha = 3.6\ \mu s$ 是地板），而计算时间按 b^2 迅速下降，两者相除就是"强扩展的墙上之墙"。
2. 在 16384^2 网格上，**进程数超过约 1.6 万之后再加进程几乎无收益**（甚至倒退）。这与第 3 节 Work/Span 给出的"并行度上限"是同一件事的两种算法：`Span` 中的常数项 α 决定了并行度的天花板。
3. 提高上限的手段：① 增大 b（弱扩展 weak scaling，或把问题做大）；② **时间分块 / 波前（wavefront）**：同时算多个时间步以提高算术强度；③ 用**三维分块**降低表面体积比；④ 用 `MPI_Irecv/Isend` 让通信与计算**重叠**（把 α 从关键路径中移出，等价于减小有效 Span）；⑤ 降低 α 本身（共享内存路径、RDMA 单边通信 `MPI_Put/Get`、GPU Direct）。

4.4 数值算例 3：与 Assignment 1 机器的算力/带宽对照（Roofline）

Gates 集群（ghc26–ghc46）的评测机器是 **3.0 GHz Intel Core i7-9700，8 核，每核 AVX2（单精度 8 宽）**。据此：

```
单核 FP32 峰值(含 FMA) = 3.0 GHz × 8 宽 SIMD × 2 (FMA) = 48 GFLOPS
8 核峰值               = 8 × 48 = 384 GFLOPS
内存理论带宽           = 双通道 DDR4-2666 = 2 × 8 B × 2.666 GT/s ≈ 42.7 GB/s
实测可用带宽(STREAM)   ≈ 25–30 GB/s   → 采用 28 GB/s
机器平衡点 = 384 GFLOPS / 28 GB/s ≈ 13.7 flop/byte
```

以 Assignment 1 Problem 5 的 saxpy（$r = a\cdot x + y$，$N = 20\times10^6$ 单精度）为例：

```
逻辑数据流: 读 x, 读 y, 写 r  →  3 × 4 B = 12 B/元素
但普通写(store)在 x86 上是 write-allocate: 未命中要先"读入整行"再写,
于是每元素的实际 DRAM 流量 ≈ 4 × 4 B = 16 B
  → 这也是 handout 里 main.cpp 用 4*N*sizeof(float) 统计流量、
    并要求用"非临时存储(non-temporal hint)"把它降到 3*N*sizeof(float) 的原因
总流量 = 4 × 20e6 × 4 B = 320 MB (普通写)
       3 × 20e6 × 4 B = 240 MB (非临时写)
算术强度 I = 2 flop / 16 B = 0.125 flop/byte  ≪ 13.7 flop/byte  → 深度带宽受限
单核(有效 20 GB/s): 320 MB / 20 GB/s = 16.0 ms   → 2×20e6/16 ms = 2.5 GFLOPS
8 核(合计 28 GB/s) : 320 MB / 28 GB/s = 11.4 ms   → 3.5 GFLOPS,  相对单核仅 1.4×
8 核 + 非临时写     : 240 MB / 28 GB/s =  8.6 ms   → 相对单核 1.86×
计算侧上限(384 GFLOPS): 40 MFLOP / 384 GFLOPS = 0.104 ms  ← 比内存侧快 100 倍以上
```

这组数字给出 Assignment 1 中那个"为什么 8 核拿不到 8 倍"的标准答案：**saxpy 的瓶颈是 DRAM 带宽，不是核数也不是 FLOPS**；把 8 个核都堆上去，只能把已经饱和的带宽榨到极限（1.4×），要真正提速必须**减少字节数**（非临时写、缓存分块、合并遍历）而不是加核。同理，Problem 3 的 ISPC 版本从"8 核 × 8 宽 AVX2 = 64 条通道"的理想出发，实际只能拿到约 20–22×（handout 给出 Problem 3 的目标区间），其损耗正是来自：Mandelbrot 各像素迭代次数不同导致的 **SIMD 通道利用率损失（divergence）**、**负载不均**（图像各行计算量不同，静态连续分块会让某些线程先干完；Assignment 1 Problem 1 要求用"简单的静态分配"达到 7.5× 以上，本质就是**把行按轮转（cyclic / interleaved）方式分配给线程**以摊平不均），以及**内存带宽**。

4.5 关键性能参数汇总

参数	符号	典型值（本讲使用的假设）	影响
每消息固定开销	α	同节点共享内存 0.2–0.5 μs；InfiniBand 1–2 μs；1 GbE+TCP 20–30 μs	决定小消息性能与并行度上限
有效单向带宽	β	12.5 GB/s（100 Gb/s IB）；0.11 GB/s（1 GbE）	决定大消息性能
机器平衡点	`α·β`	≈ 22.5 KB	小于它的消息"条数"比"字节数"更重要
单核 FP32 峰值	—	48 GFLOPS（3.0 GHz × 8 宽 × 2 FMA）	Roofline 的上界
8 核 FP32 峰值	—	384 GFLOPS	同上
内存带宽	—	28 GB/s（实测）	决定 saxpy/stencil 类内核
机器平衡点（算力/带宽）	—	≈ 13.7 flop/byte	判断算力受限 vs 带宽受限
每节点请求预留量（避免 fetch deadlock）	—	`K(P−1)` 条请求 + `K` 条应答	输入缓冲容量的设计约束

---

5. 关键要点

1. **消息传递抽象的本质是"单向传输 + 三阶段事务"**：源知道发送地址、目的知道接收地址，握手之后双方才知道彼此；因此消息层必须自己管理"本地地址空间之外的存储"（缓冲），这是它与共享地址空间（双向请求—应答、远端存储自动可见）最根本的分工差异。
2. **同步/乐观异步/保守异步不是"实现细节"，而是语义与性能的取舍**：乐观路径把缓冲与风险放在**目的端**（低延迟，但可能溢出），保守路径把缓冲放在**源端**（永不溢出、适合大块传输，但每条消息多一个 RTT）。判别准则：**消息大 + 接收缓冲可能溢出 → 选保守；消息小且接收方大概率已 post recv → 选乐观**。
3. **流控与死锁避免是运行时的核心职责**：输入缓冲必须防止被多个源"超额承诺"（credit / 背压 / 拒绝 / 丢弃），并且节点在**发不出消息时仍必须能收消息**——否则"进来的请求要产生应答、应答发不出去"会形成取数死锁（fetch deadlock）。工程解法是**请求网络与应答网络逻辑分离**（物理双网或虚通道），或**限定在途请求数并预留输入缓冲**（每节点 `K(P−1)` 请求 + `K` 应答）。
4. **消息条数与字节数必须分开优化**：`T = 条数 × α + 字节数 / β`。低于机器平衡点（如 22.5 KB）时，合并消息能带来上百倍的收益；集合通信算法（ring vs recursive doubling）的差别也主要在 Span（`θ(P·α)` vs `θ(log P·α)`），而不是 Work。
5. **"没有全局知识、没有全局控制"是所有分布式内存性能问题的根源**：屏障/归约只能给出模糊的全局状态，而延迟大到诱使人做乐观假设。因此正确的做法是：**用显式的重叠（流水线、非阻塞通信、预取）把延迟藏起来，用显式的分块把带宽需求降下来**，而不是寄希望于硬件替你解决。

---

6. 常见陷阱与注意事项

- **把"发出去"当成"对方收到了"**：乐观路径下 `send` 返回只意味着**自己的发送缓冲可以复用**，不意味着对端已取走数据；如果程序在 `send` 之后立刻改写发送缓冲（尤其是把缓冲区复用给下一次发送），就会破坏消息内容。真实 MPI 中这属于"发送缓冲在 `MPI_Wait` 完成前不可修改"的经典错误。
- **所有进程都"先发后收"造成死锁**：这是最经典的 MPI 点对点陷阱（3.2 节的阶段 B 就是它的可控演示）。当消息量超过库的 eager 阈值/本地缓冲容量时，双方互相等待对方腾出缓冲 → 全网挂死。正确写法是**成对使用 `MPI_Sendrecv`**、让**奇偶进程收发次序相反**、或改用非阻塞 `MPI_Isend` / `MPI_Irecv` + `MPI_Waitall`。
- **忽略输入缓冲会溢出（unexpected message queue）**：认为"消息传递不会丢数据"是对的，但**"不会丢"是靠缓冲与流控换来的**：大量小消息加上慢消费者会让意外消息队列膨胀，最终触发背压甚至阻塞发送方；在 3.2 节的实现里，这表现为 `n_stall` 飙升、整个环的吞吐被最慢的 rank 限制（对应网络层的 tree saturation）。
- **用消息条数而不是字节数评估通信开销**：只统计"总传输了多少 MB"会严重低估开销。8 B 消息的有效带宽只有 4.4 MB/s（约为链路的 0.04%）；反过来，把 1000 条 64 B 合并成 1 条 64 KB 可以快 256 倍。任何通信优化都应以 `条数 × α + 字节数/β` 为准则。
- **用共享内存的正确性直觉去写分布式代码**：消息传递下**没有共享变量**，全局归约必须显式做（`MPI_Reduce` / `MPI_Allreduce`）；"大家一起改一个计数器"这类共享内存惯用法在消息传递里不成立。反过来，在共享地址空间机器上跑 MPI 时又很容易忘记"`send` 其实是 `memcpy`"，从而忽视**同一节点内多 rank 争抢 DRAM 带宽**（例如 8 核机器上跑 8 个 rank 的 stencil，带宽被 8 份均分，性能远低于单 rank）。
- **忘记 tag/顺序保证的边界**：MPI 只保证**同一 `(communicator, source, tag)` 内不超车**，不同 tag 之间可以任意交错；用 tag 复用同一条通信路径（例如同一 tag 上混发不同含义的消息）会写出难以复现的匹配错误。此外，**换 tag 不能解决死锁**：死锁的根源是缓冲依赖，不是匹配歧义。
- **写错 halo 交换的行/列却不自知**：stencil 程序的 halo 错误往往表现为"结果看起来差不多、只是数值略有偏差"，从而在长迭代里被掩盖。应当使用**制造解法/不动点校验**（3.3 节）或逐点比对串行版本，第一时间暴露交换错误；同时对 `MPI_PROC_NULL` 边界、非整除的网格划分、以及角点是否需要交换（5 点格式不需要、9 点格式需要）保持警惕。

---

7. 思考题（带答案）

思考题 1：某集群的消息传递库在同一节点内（共享内存）测得 α = 0.3 μs、β = 10 GB/s，在两节点之间（InfiniBand）测得 α = 1.8 μs、β = 12.5 GB/s。某程序在每个时间步需要把 4 KB 的 halo 数据发给 4 个邻居（每个邻居一条消息），共 10000 步。请分别估算：① 同节点发送时间的通信总开销；② 跨节点发送的通信总开销；③ 如果把 4 个邻居的数据合并成 1 条 16 KB 消息发送（假设允许合并），跨节点情况下能省多少？并说明为什么"合并"在同节点情形下收益小得多。

【答案】 ① 同节点：每条消息 `T = α + n/β = 0.3 μs + 4096/10e9 s = 0.3 μs + 0.41 μs = 0.71 μs`。每步 4 条 → 2.84 μs/步；10000 步 → **28.4 ms**。若 4 条消息能并发（逻辑上独立，用 Isend/Irecv），延迟项可只算一次 → `0.3 + 4×0.41 = 1.94 μs/步`，即 **19.4 ms**（重叠后的上界）。 ② 跨节点：`T = 1.8 μs + 4096/12.5e9 = 1.8 + 0.33 = 2.13 μs`/条 → 8.5 μs/步 → **85 ms**。即使 4 条完全并发，也至少 `1.8 + 4×0.33 = 3.1 μs/步` → **31 ms**，可见跨节点情况下 α 占了绝大部分（1.8/3.1 ≈ 58%）。 ③ 合并成 1 条 16 KB：`T = 1.8 μs + 16384/12.5e9 = 1.8 + 1.31 = 3.11 μs`/步 → **31.1 ms**，相比未合并的 85 ms 节省 **63%**；与"完美并发但不合并"的 31 ms 基本持平，说明合并等价于"把 α 只付一次"，而且不需要网络支持高并发。同节点情形下：合并后 `T = 0.3 + 1.64 = 1.94 μs` vs 未合并（无并发）2.84 μs，只省 32%，因为 α = 0.3 μs 相对 0.41 μs 的传输时间已经不占主导——**机器平衡点 α·β = 3 KB 低于 4 KB 消息，所以同节点时"字节数"比"条数"更重要**。

思考题 2：某程序在 P = 4096 个进程上对 8 字节标量做 `MPI_Allreduce`，每步 1 次，共 1000 步。已知 α = 1.8 μs、β = 12.5 GB/s。① 用环形算法估算总时间并说明它为什么"看起来很荒谬"；② 用递归倍增重算；③ 若把 1000 步的归约**合并**（每 100 步做一次归约、其余步本地累加，前提是归约运算可结合，如求和），时间变成多少；④ 从 Work/Span 角度解释这三种方案的差别。

【答案】 ① 环形算法每步延迟 `≈ 2(P−1)·α = 2×4095×1.8 μs ≈ 14.7 ms`（传输项极小，可忽略），1000 步 → **约 14.7 s**。荒谬之处在于：只归约了 8 字节数据，却花了 14.7 秒，而链路上跑的字节几乎可以忽略——**总时间完全被 α 的条数 × 跳数支配**。 ② 递归倍增：`2·log₂(4096) = 2×12 = 24` 跳 → `24×1.8 μs = 43.2 μs`/步，1000 步 → **43.2 ms**，比环形快 **340 倍**。这正是"同一 Work、更小 Span"的威力。 ③ 合并成每 100 步一次：10 次归约 × 43.2 μs = **0.43 ms**，再快 100 倍。前提是运算可结合且允许改变结合顺序（浮点求和会引入不同的舍入误差，需在数值可接受性上做权衡）。 ④ Work/Span：环形算法 Work = `Θ(n·P)` 字节搬运（这里 n=8 B，Work ≈ 8×4096 = 32 KB 量级），Span = `Θ(P·α) = 14.7 ms`；递归倍增 Work 同为 `Θ(n·P)` 但 Span = `Θ(log P · α) = 43 μs`。**Work 相同、Span 差 P/log P ≈ 340 倍**，所以并行度（Work/Span）也差同样的倍数。合并方案则是另一种武器：它**同时减少 Work 与 Span**（把 1000 次归约变成 10 次），因此收益是乘性的——这也是"消息条数是第一公民"这一原则在集合通信上的体现。

思考题 3：假设一台共享内存机器上有 8 个物理核（双通道 DDR4，实测可用带宽约 30 GB/s），每个核跑 1 个 MPI rank，做 3.3 节的二维 Jacobi（全局 8192^2、双精度，数据远大于 L3）。有人观察到：`-np 1` 时每次迭代约 160 ms，`-np 8` 时约 110 ms（只有 1.5× 加速）。请给出至少三条可能原因，并设计实验区分它们；同时说明如果你把 rank 数提高到 64（同一台机器上超订），为什么性能几乎肯定变差。

【答案】 先算清账：每个内点读 5 个 double、写 1 个 double = 48 B，一遍迭代的总 DRAM 流量 ≈ $8192^2 \times 48\ \text{B} \approx 3.2\ \text{GB}$（数据远超 cache，几乎全部落到 DRAM）。单核受限于内存并行度（MLP），单核可持续带宽约 15–20 GB/s → $3.2\ \text{GB} / 20\ \text{GB/s} \approx 160\ \text{ms}$ ✓ 与观测相符；8 个核共享内存控制器时上限约 30 GB/s → $3.2\ \text{GB} / 30\ \text{GB/s} \approx 107\ \text{ms}$ ✓，于是加速比被压在 $30/20 = 1.5\times$ 附近。三个可能原因与区分实验：

1. **DRAM 带宽饱和（本例最可能）**：算术强度只有 $5\ \text{flop} / 48\ \text{B} \approx 0.10\ \text{flop/byte}$，远低于机器平衡点（8 核 384 GFLOPS / 30 GB/s ≈ 12.8 flop/byte），属于极端带宽受限，**加核没有意义**。**实验**：① 用 `perf stat -e dram__bytes_read,dram__bytes_write`（或 `likwid-perfctr -g MEM`）直接测 DRAM 流量与占带宽比例；② "算术强度实验"：在 5 点格式里插入若干次对结果无影响的浮点运算，若耗时几乎不变则证实带宽受限（若明显变慢则是算力受限）；③ 打开**缓存分块（cache blocking / 时间分块）**让数据在 L1/L2 里被复用，若加速比随之提升则确认瓶颈在 DRAM。
2. **同节点 MPI 的共享路径开销被放大**：同节点 `send` 是 memcpy + 队列/锁/唤醒，halo 交换的 α 会从网络级的 0.3 μs 级抬到数 μs；更关键的是这些通信本身也消耗本已紧张的 DRAM 带宽（拷贝一次 halo = 读一次 + 写一次）。**实验**：① 注释掉 halo 交换只做计算（结果虽错，但可测"纯计算上限"）；② 用例题 1 的 ping-pong 直接测本机 α、β，代入 $t_{comm} = 2(\alpha + 8b/\beta)$ 核对通信时间占比；③ 改用 `MPI_Isend/Irecv` 让通信与计算重叠，若时间下降说明确实存在未掩盖的通信开销。
3. **进程绑定与 NUMA / 负载均衡问题**：若进程未绑定到物理核（缺 `--bind-to core`），8 个 rank 可能挤在少数核上；跨 socket 访问远端内存会进一步压低可用带宽；`MPI_Dims_create` 给出的 `4×2` 与 `2×4` 分解通信模式不同（halo 大小、邻居数不同），且若网格上存在计算量不均（例如非均匀边界条件），静态分块会导致部分 rank 先干完。**实验**：① 用 `--bind-to core --map-by core` 强制绑定并复测；② 用 `lstopo` / `numactl --hardware` 检查拓扑，比较 `--map-by socket` 与 `--map-by core`；③ 对比 `8×1`、`4×2`、`2×4` 三种分解；④ 用每 rank 的计时（`MPI_Reduce(MAX/MIN)`）检查负载偏差。

为什么超订到 64 个 rank 几乎肯定更差：① **带宽不会变多**：DRAM 总量仍是 30 GB/s，rank 越多每个 rank 分到的越少（LogP 模型里 `g` 变大），而 halo 交换带来的**额外**读写（每个 rank 都要多发/少收一遍边界数据）反而增加总流量；② **表面体积比恶化**：$t_{comm}/t_{compute} \propto 1/b$，8 rank 时块边长约 2048–4096，64 rank（8×8）时块边长降到 1024，通信/计算比上升约 2–4 倍，而本例的 t_{comm} 在共享内存上虽小，却会以"额外内存流量"的形式重新计入 DRAM 瓶颈；③ **超订的代价**：64 个 rank 争抢 8 个物理核 → 上下文切换、cache/L3 抖动、TLB 抖动，每个 rank 的私有缓冲与 halo 暂存还会增加内存压力。**实验**：固定问题规模扫 rank 数（1,2,4,8,16,32,64）画"每次迭代时间 vs rank 数"曲线，其最低点就是该机器上该问题的**最优 rank/线程数**——这也正是"每个核一个 rank"这条经验法则的来源（弱扩展曲线平坦、强扩展曲线出现最优点的分水岭）。

Lecture 26: MPI, OpenMP, Cilk Implementation

1. 章节标题与概述

Lecture 26: MPI, OpenMP, Cilk Implementation

- **本讲核心问题**：前两周我们从"抽象层"讨论了三大并行编程模型（shared address space（共享地址空间）、message passing（消息传递）、data parallel（数据并行）），本讲把镜头拉到"实现层"：MPI、OpenMP、Cilk 这三种**具体编程系统**究竟由谁、用什么机制把程序员书写的并行抽象映射到硬件上？具体要回答三个问题：(1) 一个 `cilk_spawn` / `#pragma omp parallel for` / `MPI_Send` 调用在运行时到底做了什么？(2) 三种系统在 decomposition（分解）、assignment（分配）、orchestration（编排）、mapping（映射）四个阶段分别把责任交给了程序员、编译器、运行时还是硬件？(3) 这些实现机制各自的性能上界由什么决定——是计算吞吐、内存带宽、同步延迟，还是关键路径（span）？
- **涉及的主要硬件/软件机制**：
- **硬件侧**：集群互连网络（InfiniBand/NIC、fat-tree 拓扑、bisection bandwidth（对分带宽）、oversubscription（超额订阅））、单节点多核 + 共享 LLC 与 NUMA、cache line（64 字节）与 cache coherence（缓存一致性）协议、原子 read-modify-write（RMW）指令（`lock xadd` / `cmpxchg`）、消息延迟 α 与带宽 β。
- **软件侧**：MPI 库实现中的 eager（急切）与 rendezvous（会合）协议、unexpected-message queue（非预期消息队列）、非阻塞 `Isend/Irecv` 与 `Waitall`、集合通信的树形算法；OpenMP 的编译器 outlining（轮廓化）变换 + 线程池运行时（libgomp/libomp）+ worksharing 循环切块 + barrier（屏障）实现（集中式 sense-reversing、combining tree、dissemination）；Cilk 的 `cilk_spawn` / `cilk_sync` → **continuation stealing（续体窃取）** + 每 worker 一个 lock-free dequeue + randomized work stealing（随机工作窃取）+ reducers（hyperobject）。

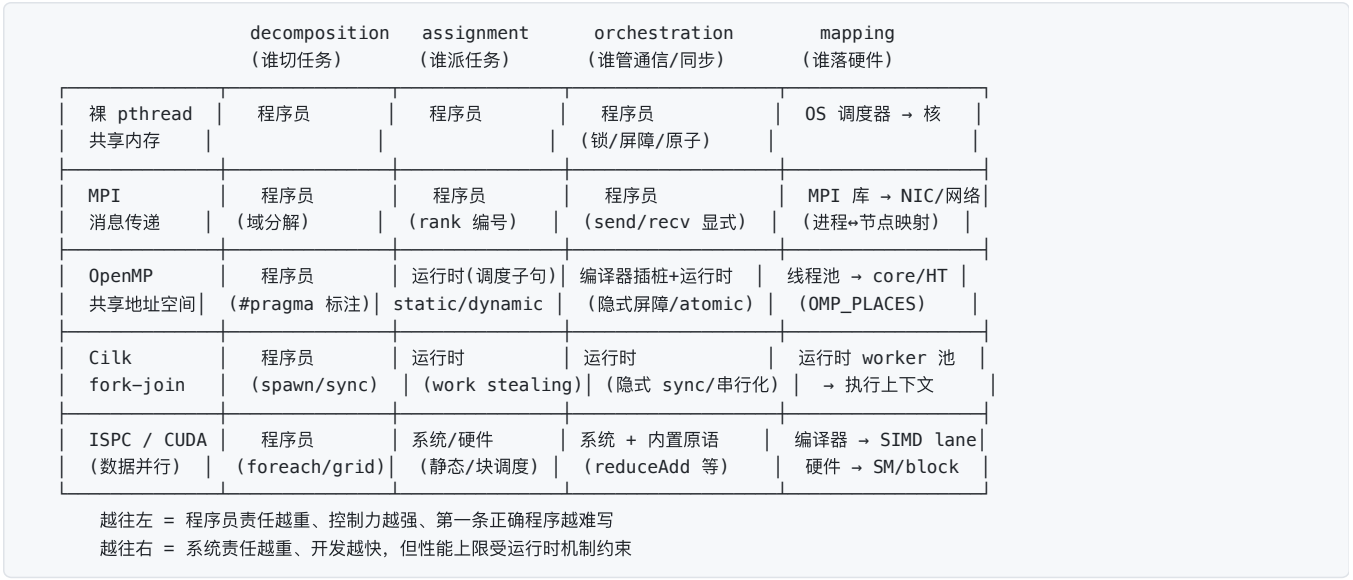
- **在并行计算知识体系中的角色**：本讲是"抽象 → 实现"的闭合环节。前面课程已经建立了三大模型（models）、依赖与同步（synchronization）、缓存一致性与内存序（coherence/consistency）、互连网络（interconnects）、work–span 与 Amdahl 定律等工具；本讲说明**这些工具如何共同落成真实代码**：message passing 落地为 MPI 的通信协议与集合通信算法，shared address space 落地为 OpenMP 的线程池 + 屏障 + 调度器，fork–join/divide–and–conquer 落地为 Cilk 的 work–stealing 运行时。它也是后续"性能调优 / 项目实现"的直接方法论来源：**先用最简单的实现跑通，再依据瓶颈（带宽、延迟、span、同步次数）选择机制**。
- **配套材料**：
- **Fall 2026 课程主页与日程表**：https://www.cs.cmu.edu/~418/、https://www.cs.cmu.edu/~418/schedule.html —— **已公开**（无需登录）。日程表列出 Lecture 26 主题 "MPI, OpenMP, Cilk implementation"（Nov 3），并注明 Project proposal due。
- **本讲的 Fall 2026 讲义 PDF**：在 https://www.cs.cmu.edu/~418/lectures/ 下**尚未发布**（Fall 2026 已公开 PDF 只覆盖 Why Parallelism、ILP、Modern Multi–Core、Programming Models、CUDA、Parallel Programming Basics、Performance Optimization I/II、Interconnects、Coherence、Synchronization、Lock–Free、Heterogeneity、Virtual Memory、Consistency、Parallel Deep Learning 等）。日程表中 Lecture 26 一行引用的是**历史学期（Fall 2022）**的 "Part A slides"（`26a-msgpassing.pdf`）与 "Part B slides"（`26b-omp.pdf`），它们位于 `/afs/cs/academic/class/15418-f22/public/lectures/` 之下，访问需要 CMU 登录 —— 属**未公开**。
- **抽取尝试（本地）**：`extracted/f22_26-msgpassing.txt`、`extracted/f22_26a-msgpassing.txt`、`extracted/s23_26b-omp.txt` 三个文件都只含登录页 HTML（"Your Browser does not support javascript. Please use this link."，88 字节），**无讲义正文**，不能作为事实来源，已跳过。
- **录像**：Fall 2026 日程表中 video 链接均被 HTML 注释隐藏（"video from a previous offering; uncomment when posted for Fall 2026"），属**未发布**；Ed 讨论区、Autolab、Canvas/Gradescope 需登录，属**需登录未公开**。
- **本笔记的事实基础**：Fall 2026 已公开讲义 `extracted/06_progbasics.txt`（Lecture 7: Parallel Programming Basics，页眉标注 Fall 2026）与 `extracted/07_progperf1.txt`（页眉标注 Fall 2025，属讲义沿用的历史学期字样，内容为 Performance Optimization Part 1: Work Distribution and Scheduling）。前者提供：三大模型对比、分解/分配/编排/映射四阶段、Amdahl 定律与其算例、锁与屏障、message passing 网格求解器（ghost cell（影子单元）、同步 send/recv 死锁、非阻塞 send/recv）；后者提供：静态/半静态/动态分配、任务粒度权衡、共享工作队列、每 worker 分布式队列与 work stealing、Cilk Plus 的 `cilk_spawn` / `cilk_sync` 语义、continuation stealing vs child stealing、每 worker dequeue（本地压栈尾部、窃取头部）、parallel slack（并行松弛 ≈ 8）。本讲其余 MPI/OpenMP/Cilk 实现细节依据公开成熟知识（MPI 标准与实现文档、OpenMP 规范与 libgomp/libomp 实现、Cilk–5 / OpenCilk 相关论文与技术资料）撰写，并已与上述讲义中的术语和结论对齐。
- **Fall 2026 授课教师**：Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。

---

2. 核心概念与硬件/软件架构图解

2.1 谁负责哪一步：从抽象到实现的责任分层

- **定义与目的**：并行程序从"问题"到"执行"要经过四个阶段——**decomposition（分解，把问题切成任务）→ assignment（分配，把任务交给 worker）→ orchestration（编排，组织通信/同步/数据布局/调度）→ mapping（映射，把 worker 落到硬件执行单元）**。三大编程系统的差别，本质上就是**这四个阶段的责任归属不同**：MPI 把四件事几乎全交给程序员；OpenMP 把 assignment 交给运行时（调度子句可调）、orchestration 部分交给编译器 + 运行时；Cilk 把 assignment 和 mapping 完全交给运行时（work–stealing 调度器）。
- **直观解释（"它是什么？"）**：把它想象成**装修一栋楼**。decomposition = 决定有哪些活（水电、刷墙、铺地板）；assignment = 决定哪个工头带哪批活；orchestration = 规定工头之间怎么交接（要不要开会、材料放哪）；mapping = 实际派几个工人、派到哪几个房间。MPI 模式下你是**总包 + 每个工头 + 每个工人**，所有交接都必须你手写电话（显式消息）；OpenMP 模式下你只写"这面墙刷两遍"，公司（运行时）自动分工头、自动开会（隐式屏障）；Cilk 模式下你只说"这活能并行"，调度中心（work–stealing runtime）看到谁闲就把活塞给谁。
- **架构/机制图解（软件执行模型）**：



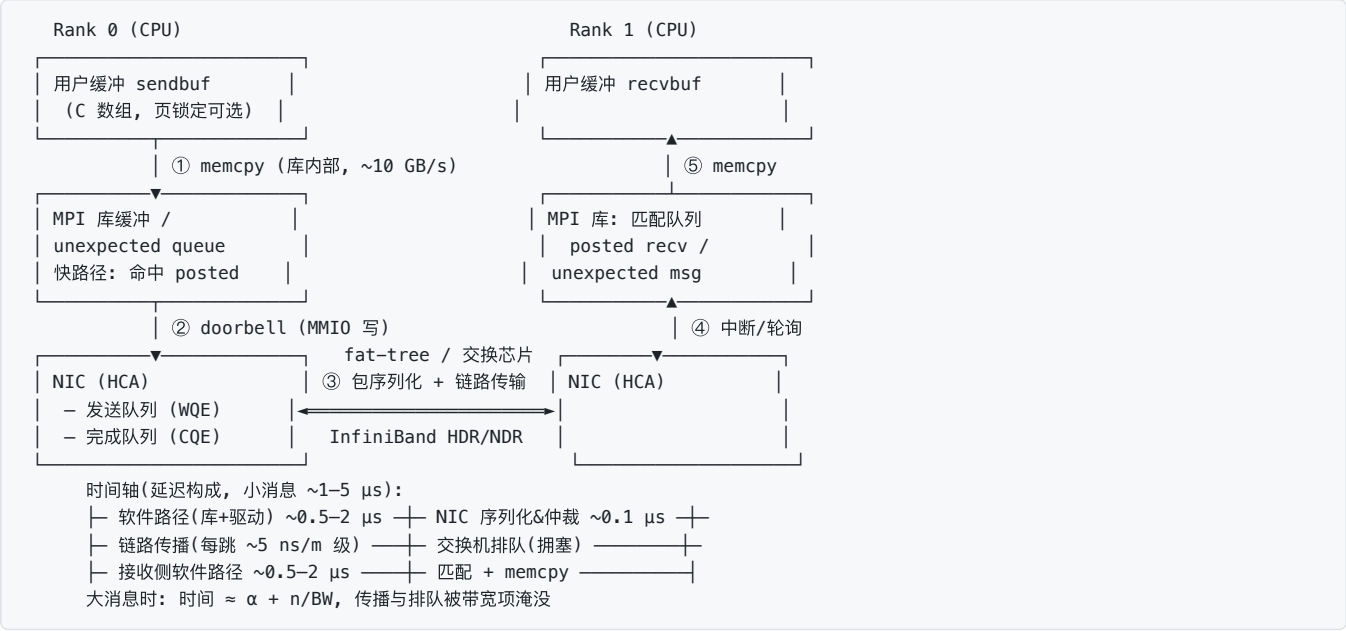
- **关键操作与性能特征**：责任越是交给系统，运行时开销就越"隐形但真实"——OpenMP 每次进入 `parallel for` 是一次 fork–join（线程唤醒 + 屏障，量级 1–10 μs）；Cilk 每次真的发生窃取（steal）要访问别的 worker 的 dequeue（量级 0.1–5 μs）；MPI 每次通信要付 α 启动延迟（节点内 ~0.5–2 μs，跨节点 ~1–5 μs）。**判断标准不是"谁更优雅"，而是"每次迭代的计算量能否摊薄这些开销的固定成本"**。

表 1：三种编程系统在四个阶段与失败模式上的对比

维度	MPI（message passing）	OpenMP（shared address space）	Cilk（fork–join）
地址空间	每 rank 私有（SPMD 进程）	单一共享地址空间（线程）	单一共享地址空间（worker 线程）
通信方式	显式 `Send/Recv`、显式派生数据类型	隐式 load/store	隐式 load/store；依赖靠 spawn/sync 表达
同步原语	消息匹配（本身就是同步）、`Barrier`、`Allreduce`	`barrier`、`critical`、`atomic`、隐式 forall 结束屏障	`cilk_sync`、隐式函数末尾 sync、reducer
数据布局	**必须**做 ghost cell 复制与域分解	共享数组自然存在（注意伪共享与 NUMA）	共享数组自然存在（注意并行递归的栈与局部性）
分配策略	程序员（rank ↔ 数据块）	`schedule(static/dynamic/guided[,c])`	运行时 work stealing（隐式）
扩展性上限	由**表面–体积比**与网络带宽/延迟决定	由**屏障频率 + 内存带宽 + 调度开销**决定	由 **span（关键路径）** 决定
典型失败模式	死锁、tag/rank 不匹配、缓冲区被覆写	数据竞争、伪共享、屏障负载不均、超额订阅	假共享、过度 spawn、栈空间爆炸、span 过长
第一条正确程序难度	较高（要显式想清楚通信）	较低（但容易"正确但很慢"）	低（串行语义可消除，spawn 可删）

2.2 MPI 实现：独立地址空间上的显式消息

- **定义与目的**：MPI（Message Passing Interface）是一个**库标准**（不是语言，也不是编译器特性），它把"通信"结构化为一对一对的消息（message）：发送方调用 `MPI_Send`，接收方调用 `MPI_Recv`，库负责把数据从发送方地址空间搬到接收方地址空间。它解决的核心问题是：**在节点之间没有共享内存（甚至没有统一地址空间）时，如何表达通信与同步**——正如讲义所说，message passing 模型下"每个线程拥有自己的地址空间，没有共享地址空间抽象，线程通过收发消息来通信与同步"。
- **直观解释（"它是什么？"）**：把集群想象成**两家没有共享仓库的公司**，只能靠快递传货。`MPI_Send` = 把货打包交给快递（数据从你的缓冲区复制到网络缓冲区）；`MPI_Recv` = 从前台取货（数据从网络缓冲区复制到你的缓冲区）。**关键点：你不是"共享内存 + 网络硬件"，而是"复制 + 复制 + 复制"**——每次通信至少三次数据搬运（用户缓冲 → 库缓冲 → 网络 → 库缓冲 → 用户缓冲），所以 MPI 的优化重点永远是**少发、发大块（bulk transfer）**，正如讲义强调的"communicate entire rows at a time (not individual elements)"。
- **架构/机制图解（硬件结构 + 消息路径）**：



- **关键操作与性能特征**：
- **延迟（latency）**：小消息成本 ≈ α（启动延迟），与消息大小几乎无关。节点内共享内存 MPI 约 0.3–1 μs，InfiniBand 跨节点约 1–5 μs。**α 是"每条消息"的固定税**，因此把 1000 个 8 字节的消息合成 1 个 8 KB 的消息通常能提速一个数量级以上。
- **带宽（bandwidth）**：大消息成本 ≈ n/BW。单端口 HDR 200 Gb/s ≈ 25 GB/s（理论），实际可达约 10–20 GB/s；带宽项由链路速率、NIC DMA 引擎、PCIe 通道与内存带宽共同决定。
- **消息速率（message rate）**：很多实现能到 5–20 M msg/s/NIC；这是"小消息"场景的真实上界，与带宽无关。
- **eager vs rendezvous**：库在消息大小超过阈值（典型 8–64 KB）时从 eager 切换到 rendezvous，以避免占用不可控的接收缓冲。

表 2：MPI 发送/接收协议对比（实现层视角）

模式	发送方行为	接收方行为	缓冲需求	是否阻塞	典型用途
Eager（急切）	直接复制进网络缓冲并发出，不等接收方	已 post 则直拷用户缓冲；未 post 则进 unexpected queue	接收端需动态缓冲，可能耗尽内存	通常立即返回	小消息（< 切换阈值）
Rendezvous（会合）	先发请求，等接收方回应后才传数据	收到请求后回地址，数据直落用户缓冲	无中间缓冲（零拷贝）	需双方参与	大消息、避免缓冲爆炸
Synchronous（同步）	返回时确认接收方已开始接收	同上	较轻	阻塞语义明确	讲义 06 中的 `send()` 模型
Buffered（带缓冲）	复制到用户提供的缓冲即返回	正常接收	发送方预留缓冲	非阻塞（对发送方）	需要避免死锁但不想用非阻塞
Ready（就绪）	假定接收方已 post，直接发	必须已 post，否则出错	无	阻塞	极致延迟优化
Non-blocking（`Isend/Irecv`）	立即返回 handle，**缓冲不可改**	立即返回 handle	库内部管理	不阻塞；用 `Wait/Test` 完成	现代标准做法：通信与计算重叠

2.3 MPI 域分解与 ghost cell：表面-体积比

- **定义与目的**：把 N×N 网格按行切成 `size` 块，每块带两行 **ghost cell（影子单元）**——它们不是本 rank 拥有的数据，而是从邻居地址空间复制来的副本。ghost cell 存在的唯一目的：**让 stencil（模板）计算在本地地址空间里就能拿到所有邻居值，而不必每次访问都发消息**。讲义明确说："data replication is now required to correctly execute the program"，并且"information in ghost cells is 'owned' by other threads"。
- **直观解释（"它是什么？"）**：像**拼图时把邻块边缘那一条先拓印到自己的板子上**。你不需要知道邻居怎么拼，只需要边缘那一条。
- **架构/机制图解（软件执行模型：环状 halo 交换）**：

```
      全局 N x N 网格按行分给 4 个 rank (blocked 分配)
┌──────────────────────────────────────────┐
│ 全局边界 (固定值 1.0)                    │  ← 只有边界 rank 需要
╞══════════════════════════════════════════╡
│  rank 0: rows 0 .. r-1       [上行 ghost=边界] │
│                              下行 ghost ◄──────┤── rank1 发来的第 1 行
╞══════════════════════════════════════════╡
│  rank 1: rows r .. 2r-1     上行 ghost ◄──────┤── rank0 发来的最后 1 行
│                              下行 ghost ◄──────┤── rank2 发来的第 1 行
╞══════════════════════════════════════════╡
│  rank 2: rows 2r .. 3r-1    上行 ghost ◄──────┤── rank1 发来的最后 1 行
│                              下行 ghost ◄──────┤── rank3 发来的第 1 行
╞══════════════════════════════════════════╡
│  rank 3: rows 3r .. N-1     上行 ghost ◄──────┤── rank2 发来的最后 1 行
│                              [下行 ghost=边界] │
└──────────────────────────────────────────┘

一次迭代的数据流(每个 rank):
   ① Isend 上行 1 行 ──►  ② Isend 下行 1 行   (先发起, 不等待)
   ③ Irecv 上行 ghost ◄── ④ Irecv 下行 ghost
   ⑤ Waitall(4 个请求)  ◄── 通信期间 CPU 可做别的活(如同步旧数据)
   ⑥ 计算本地 rows x (N-2) 个内点
   ⑦ Allreduce 收敛判据(所有 rank 得到同一结论, 避免有人提前退出)

表面-体积比:
   通信字节 / 计算字节 = (2 * N * 4B) / (rows * N * 20B) = 0.4 / rows
   rows 越大 → 通信占比越低 (这就是"weak scaling 友好"的根源)
```

- **关键操作与性能特征**：通信量与"表面积"成正比，计算量与"体积"成正比，因此**每个 rank 负责的行数 rows 越大，通信/计算比越低**。这也是讲义中"blocked assignment requires less data to be communicated between processors"的定量版本——相比 interleaved（交错）分配，blocked 分配的边界长度从 O(N·P) 降到 O(N)。

2.4 MPI 集合通信：用树把 O(P) 变成 O(log P)

- **定义与目的**：`MPI_Reduce`、`MPI_Allreduce`、`MPI_Bcast`、`MPI_Scatter/Gather` 等集合操作（collective）如果用 P 个 rank 的朴素"全都发给 rank 0"或"rank 0 发给所有人"实现，延迟随 P 线性增长，且 rank 0 的 NIC 成为热点。实现层的解法：**树形算法**（binomial tree / recursive doubling / halving），把步数从 O(P) 降到 O(log P)。
- **直观解释（"它是什么？"）**：像**公司逐级汇报**：不是 64 个员工都跑去找 CEO，而是两两合并、16 人合 8 人、8 人合 4 人……6 轮就到 CEO。广播则相反：CEO 只讲给 2 个副总，副总各讲给 2 个经理，逐层扩散。
- **架构/机制图解（集合通信的树形实现）**：

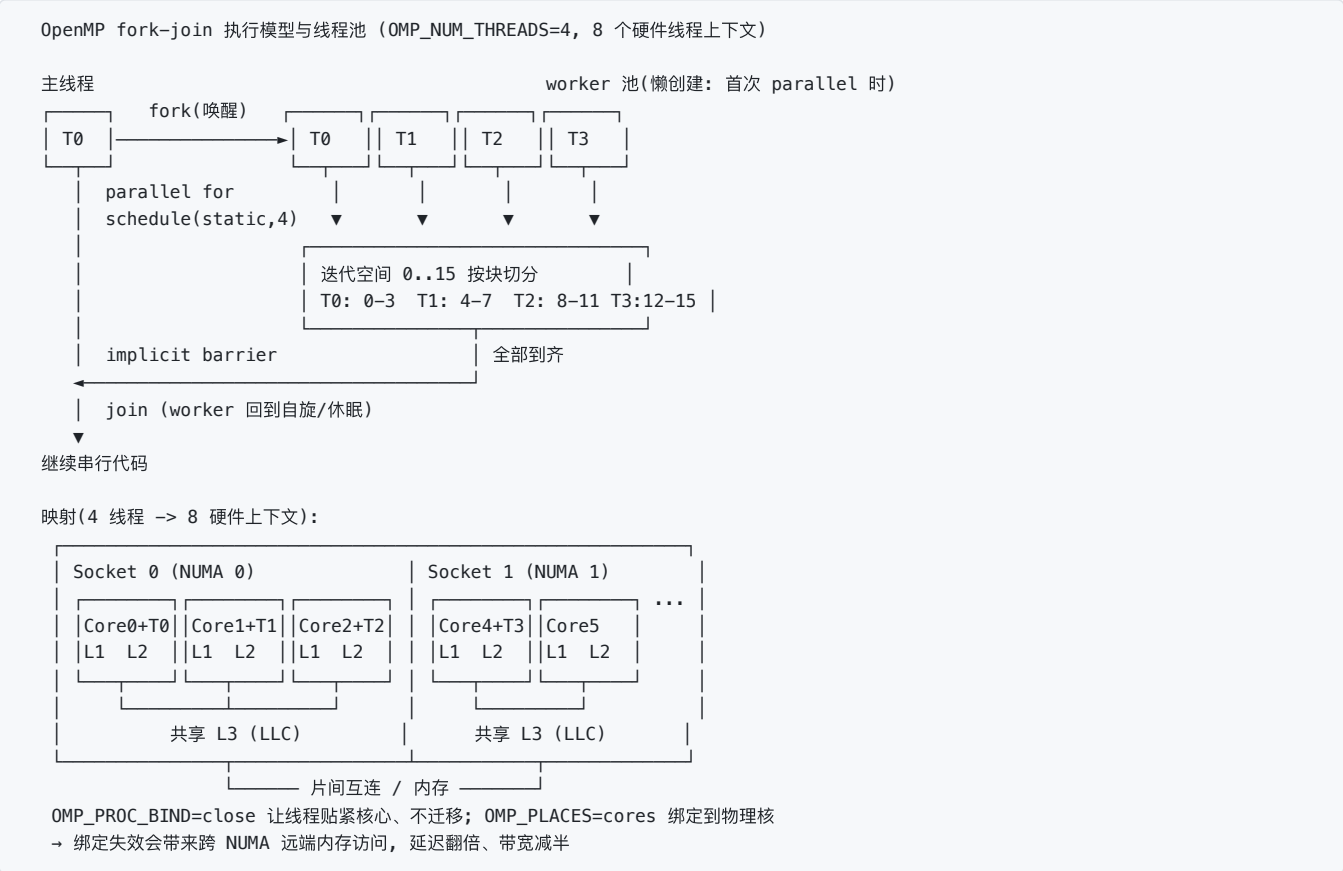
```
MPI_Bcast(root=0) 的 binomial tree (P = 8, 3 轮完成, O(log P) 深度)

 轮 1          轮 2               轮 3
 ┌─┐          ┌─┐                ┌─┐
 │0├─────────►│1├───────────────►│3│
 └┬┘          └─┘                └─┘
  │
  │           ┌─┐                ┌─┐
  └──────────►│2├───────────────►│5│
              └┬┘                └─┘
               │
               │                 ┌─┐
               └────────────────►│6│
                                 └─┘
 同时 rank1 在第 2 轮可以把数据给 rank 4/5 的分支... (每轮活跃 rank 数翻倍)
 总步数 = ceil(log2 P); 每步成本 ≈ α + n/BW
 T_bcast ≈ log2(P) * (α + n/BW)          ← 延迟优先(小消息)
 若 n 很大, 更优的是"分段流水线/递归倍增"变体, 使 T ≈ α*logP + n/BW

MPI_Allreduce(P=8) = reduce(树形向上, log P 步) + bcast(树形向下, log P 步)
   ≈ 2 * log2(P) * α + 2 * (P-1)/P * n/BW
朴素实现 = 7 个 rank 各发一次给 0, 再等 7 次广播: 14 步 + rank0 NIC 热点 ← 差
```

- **关键操作与性能特征**：集合通信的实现在"延迟最优"（小消息：log P 步）与"带宽最优"（大消息：分段 + 环形/递归倍增，使每条链路流量为 (P−1)/P·n）之间切换，切换依据同样是消息大小。对使用者的直接结论：**每轮迭代只做必要的集合操作**；收敛判据（如 Jacobi 的 diff）必须

用 `Allreduce` 而不是 `Reduce` + 广播两次，且能用 `Allreduce` 的场合不要用"大家都 Reduce 到 rank 0"。

2.5 OpenMP 实现：编译器 outlining + 线程池 + 屏障 + 调度器

- **定义与目的**：OpenMP 是**指令制导（directive-based）**的共享地址空间编程接口。编译器把 `#pragma omp parallel for` 的循环体**轮廓化（outlining）**成一个独立函数，生成"启动并行区 + 按线程切分迭代空间 + 结束时屏障"的代码，运行时库（GCC 的 `libgomp`，Clang/ICC 的 `libomp`）负责线程池、屏障与调度。它解决的问题是：**让共享内存并行化的样板代码由编译器生成**，程序员只描述"哪段循环可并行"。
- **直观解释（"它是什么？"）**：像**公司里的"会议制"**：老板（主线程）喊一声"开会"，已经在休息室待命的员工（线程池）进会议室；会上把活按规则一分（worksharing），干完必须都到齐（barrier）才能散会。**关键实现细节是"员工不是每次招新人，而是常驻休息室"**——线程创建开销（数十微秒）在第一次并行区付一次，之后每次 fork-join 只需唤醒 + 屏障（微秒级）。
- **架构/机制图解（软件执行模型 + 硬件映射）**：



- **关键操作与性能特征**：
- **fork/join 成本**：并行区进入 + 退出（含屏障）典型 1–10 μs，取决于线程是否在自旋（`OMP_WAIT_POLICY=ACTIVE` 延迟低但抢 CPU；`PASSIVE` 省电但唤醒延迟高）。
- **屏障是"最保守的依赖表达"**：讲义指出 barrier "divide computation into phases"，且 "all computations by all threads before the barrier complete before any computation in any thread after the barrier begins"。它的代价是所有线程的**最长者决定相位长度（负载不均被放大）**，且每次都是一次全局同步。
- **调度开销与并行度**：`schedule(dynamic, c)` 的计数器是**串行的**，因此 chunk 越小，理论可达到的并行度上限越低（见第 4 节公式）。`schedule(guided)` 用递减的 chunk（约 `剩余/线程数`）在两者之间折中。
- **数据环境**：`reduction(+:sum)` 的实现是"每线程私有副本 + 结束时合并"，因此**不要在并行区内加锁更新 `sum`**（讲义中的求解器示例正是用 `myDiff` 局部累加、循环外只加一次锁，把锁次数从 $O(N^2)$ 降到 $O(线程数)$）。

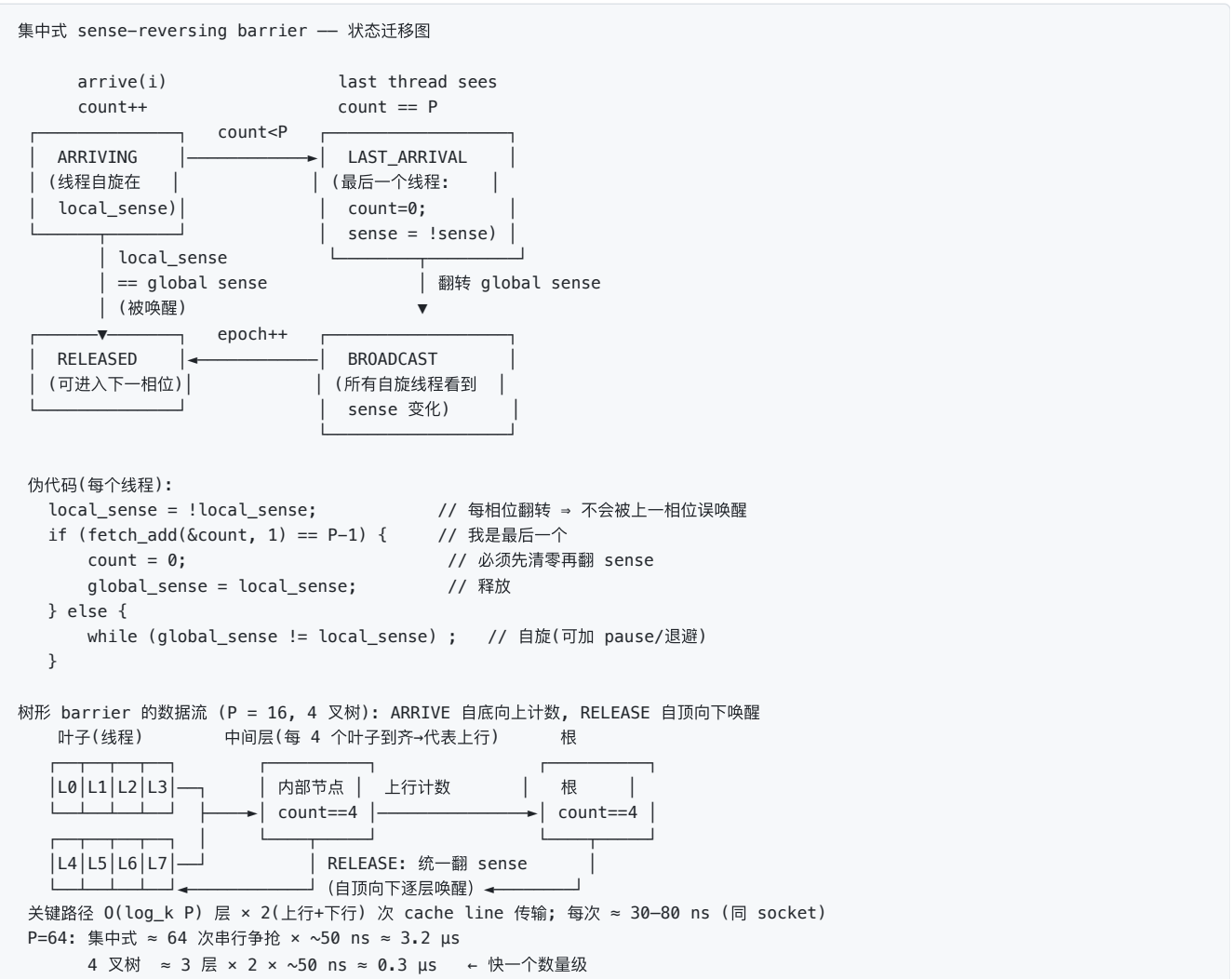
表 3：OpenMP 调度策略对比

策略	切分方式	运行时开销	负载均衡	并行度上限	适用场景
`schedule(static)`	编译/进入并行区时一次性算好（默认块状，块 = N/T）	≈ 0（只有索引运算）	差（迭代代价不均就失衡）	最高（无串行段）	每迭代等代价、访存连续（利于预取与向量化）

策略	切分方式	运行时开销	负载均衡	并行度上限	适用场景
`schedule(static, c)`	按 chunk c 交错分配	≈ 0	中	最高	想让相邻线程访问相邻数据但保持静态
`schedule(dynamic, c)`	每线程用原子计数器领 chunk	高（每 chunk 一次原子 + 缓存行仲裁）	好	**受限于 (N/c) 次串行计数器操作**	迭代代价高度不均（如素数测试、稀疏遍历）
`schedule(guided, c)`	chunk 从大到小递减（≈ 剩余/2T），不小于 c	中	好	高（串行操作 ≈ N/c* 量级）	代价可预测性差又要控制开销的通用折中
`schedule(auto)`	交给运行时/编译器决定	未知	未知	未知	可移植性优先的代码
`schedule(runtime)`	由 `OMP_SCHEDULE` 环境变量决定	取决于实际策略	取决于实际策略	取决于实际策略	需要不改代码做实验/调优

2.6 屏障的实现：从集中式到树形

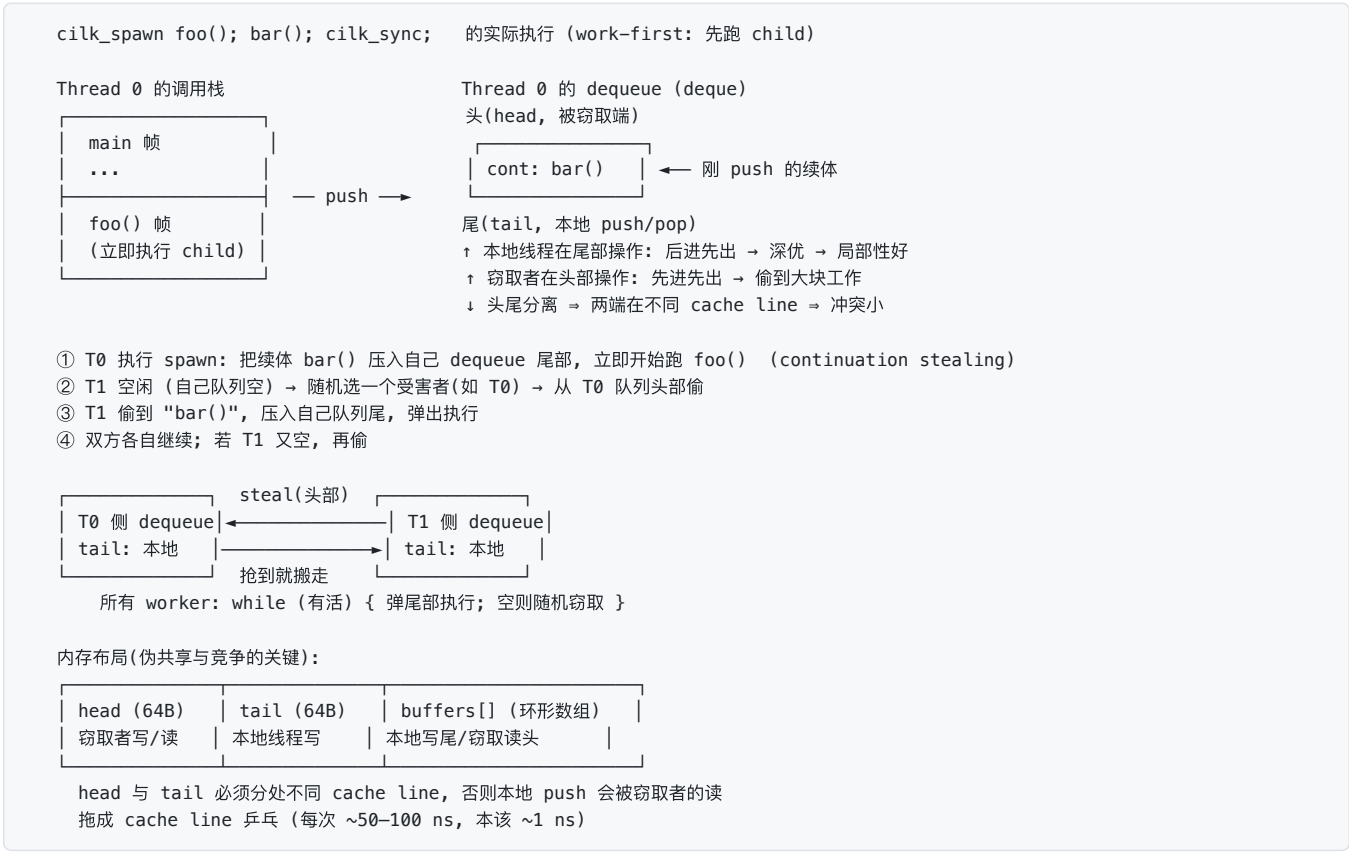
- **定义与目的**：屏障（barrier）是"相位同步"的硬件/软件原语。实现层的核心问题：**P 个线程如何在没有专用硬件的情况下高效达成"全部到齐"**。朴素实现用一个计数器 + 锁 → 所有线程争抢同一条 cache line → 串行化；工程实现用**感知反转（sense-reversing）**避免"屏障重用"的正确性问题，用**树形/传播式（combining tree / dissemination）**把 O(P) 的关键路径降到 O(log P)。
- **直观解释（"它是什么？"）**：集中式屏障像**只有一支签到笔的签到台**，64 个人排队签；树形屏障像**分组签到**：4 人一组先在小表上签到，组代表再去上一层签，只有 $\log_2 64 = 6$ 层。
- **架构/机制图解（状态机 + 数据流）**：



- **关键操作与性能特征**：屏障成本 ≈（关键路径上的 cache line 传输次数）×（一次跨核传输延迟 30–100 ns）+ 唤醒延迟。集中式屏障在 $P \leqslant 16$ 时通常够快（一次原子操作 + 广播）；P 增大到几十上百时树形/传播式明显更优。**对使用者的结论：屏障频率比屏障实现更重要**——如果每次迭代的计算只有几微秒，即使 0.3 μs 的树形屏障也会吃掉 10% 以上的时间。

2.7 Cilk 实现：continuation stealing + 每 worker dequeue

- **定义与目的**：Cilk 用两个关键字表达 fork-join：`cilk_spawn foo(args);`（"调用 foo，但调用者**可以**继续异步执行"，即 fork）与 `cilk_sync;`（"等本函数内所有 spawn 完成"，即 join）；并且"每个含 `cilk_spawn` 的函数末尾都有一个隐式 `cilk_sync`"，因此"当一个 Cilk 函数返回时，与它相关的所有工作都已完成"。实现层的核心是：**把逻辑上的 fork-join 变成"每 worker 一个队列 + 空闲者偷活"的调度器**，使得串行化（删掉 spawn/sync）后的语义与串行程序完全一致（serial elision，串行消除）。
- **直观解释（"它是什么？"）**：Cilk 运行时像一个**共享办公区**：每个人手边有一摞"待办条"（自己的 dequeue）。谁有活就干自己摞在**最上面**的那张（后进先出，缓存友好）；一旦自己的摞空了，就去**别人摞的最下面**抽一张（那张是"最早搁置、包含最多后续工作"的大活）。抽活（steal）要打电话协调，比较贵，所以只在真的没活时才做。
- **架构/机制图解（软件执行模型：continuation stealing + dequeue）**：



- **关键操作与性能特征**：
- **continuation stealing（续体窃取）** vs **child stealing（子任务窃取）**：讲义给出两条抉择——"Run child first: record continuation for later execution"（continuation stealing）与"Run continuation first: record child for later execution"（child stealing）。Cilk 选择**先跑 child**，理由是讲义明确的两条：(a) 对"循环里逐次 spawn"的代码，**调用者线程只需创建 1 个可被偷的条目**（代表剩余所有迭代的续体），而 child stealing 会"在开始执行之前就为所有迭代创建了工作"，**占 O(N) 空间**；(b) 无窃取时，continuation stealing 的执行顺序与"删掉 spawn"的程序完全一致（深度优先调用图遍历）；并且可以证明"work queue 的存储量不超过单线程栈存储量的 T 倍"（T = 线程数）。
- **窃取粒度的智慧**：从 dequeue 的**头部**窃取 ⇒ 偷到的是调用树中"更靠上、更大"的工作，从而**摊薄窃取成本**、并与本地线程错开访问区域（减少竞争）。
- **locality**：run-child-first + 本地尾操作 ⇒ 本地线程始终工作在自己调用树的局部；讲义总结为"maximizes locality"。
- **T1/T∞ 与 greedy 调度**：work-span 模型给出 T_P ≥ max(T1/P, T∞)，而贪心/工作窃取调度器满足 **T_P ≤ T1/P + T∞**；并行度 = T1/T∞。讲义强调 "parallel slack = ratio of independent work to machine's parallel execution capability（实践中 ~8 是好比例）"——即**可独立工作量最好是机器并行能力的约 8 倍**，既够填满机器、又不至于粒度过细。

2.8 Work-span 模型与三种实现机制的统一视角

- **定义与目的**：Work（T1）= 全部操作数（单处理器时间）；Span（T∞）= 关键路径长度（无限处理器时间）；Parallelism = T1/T∞。这三个量是**判断"该用哪种机制"的统一语言**：MPI 关心的是每条 rank 上的 T1 与通信量；OpenMP 关心的是屏障之间的并行度与串行段；Cilk 关心的是 T∞（span）与窃取次数。

- **直观解释（"它是什么？"）**：把计算想成**做饭**：T1 = 所有切菜炒菜的总工时；T∞ = 从"烧水"到"装盘"这条最长依赖链上必须逐分钟等待的时间；并行度 = 总工时 / 关键链长度 = "理论上最多几个厨师有意义"。
- **架构/机制图解（计算 DAG 与关键路径）**：

```
计算 DAG（每个节点=一个单位工作，箭头=依赖）

          ┌──────────────────────────────────────────┐
T∞ →      │ A ──► B ──► C ──────────────► J ──► K ──► L │ ← 关键路径(span)
          │             \                 ▲          │
          │              \                │          │
          │  D ──► E ──► F ──► G ──► H ──► I         │ ← 旁支工作(可被窃取)
          └──────────────────────────────────────────┘

T1 = A..L 的总节点数 = 12
T∞ = A–B–C–J–K–L      = 6
并行度 = 12/6 = 2      ← 即使有 64 个核，也最多 2 倍加速！

T_P 的界（greedy / work-stealing 调度器）：
  下界：  T_P ≥ max( T1/P , T∞ )
  上界：  T_P ≤ T1/P + T∞
⇒ 加速比 S_P = T1/T_P ≤ T1/(T1/P) = P   且   S_P ≤ T1/T∞ = 并行度
⇒ 想扩展，必须先缩短 span（把串行段并行化），而不是"多 spawn 几下"

与 Amdahl 定律的关系：
  Amdahl: S ≤ 1/S_serial          (S_serial = 固有串行比例)
  work-span: S ≤ T1/T∞            (span 就是"无法并行的那条链")
  两者是同一件事的两种写法：span 长 ⇒ 串行比例大 ⇒ 扩展性差
```

- **关键操作与性能特征**：对三种实现机制的统一判据是：**机制引入的开销必须远小于它带来的并行收益**。定量地，用 P 个 worker 一定要满足 `T1/P ≫ 单次机制开销`（MPI 的一次消息、OpenMP 的一次屏障、Cilk 的一次窃取）。这条判据把第 3 节的三个代码示例串成一条主线。

---

3. 代码示例与性能分析

3.1 示例一：MPI 五点 Jacobi + halo 交换（非阻塞发送/接收）

```c
/* mpi_jacobi.c —— MPI 2D 五点 Jacobi 迭代 + ghost cell(halo) 交换
 *
 * 编译(必须用 MPI 编译器包装器, 由它决定 -O3 与链接库):
 *     mpicc -O3 -march=native mpi_jacobi.c -o mpi_jacobi -lm
 * 运行:
 *     mpirun -np 8 --bind-to core ./mpi_jacobi 4096 200
 *     (OpenMPI 用 --bind-to core; MPICH 用 -binding core 或 srun --cpu-bind=cores)
 * 说明: -O3 让内层循环向量化; --bind-to core 避免 rank 在核间漂移破坏 NUMA 局部性
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);   /* 本进程编号 0..P-1 */
    MPI_Comm_size(MPI_COMM_WORLD, &size);   /* 进程总数 P */

    const int    N     = (argc > 1) ? atoi(argv[1]) : 4096;  /* 全局网格 N x N */
    const int    ITERS = (argc > 2) ? atoi(argv[2]) : 200;   /* 迭代上限 */
    const double TOL   = 1e-6;                                /* 收敛阈值 */

    if (N % size != 0) {
        if (rank == 0) fprintf(stderr, "错误: N 必须能被进程数整除\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    const int    rows  = N / size;                      /* 本地负责的行数 */
    const size_t plane = (size_t)(rows + 2) * N;        /* 含上下两行 ghost */

    double *A = (double *)malloc(plane * sizeof(double));
    double *B = (double *)malloc(plane * sizeof(double));
    if (!A || !B) { fprintf(stderr, "rank %d: 内存不足\n", rank); MPI_Abort(MPI_COMM_WORLD, 2); }

    double *cur = A, *nxt = B;      /* 每个缓冲区都是"2 行 ghost + rows 行有效 + 2 行 ghost" */

    /* ---------- 初始化: 全局边界 = 1.0, 内部 = 0.0 ---------- */
    for (int i = 0; i < rows + 2; ++i) {
        const int gi = rank * rows + (i - 1);        /* 全局行号; ghost 行为 -1 或 N */
        for (int j = 0; j < N; ++j) {
            const int on_bnd = (gi <= 0 || gi >= N - 1 || j == 0 || j == N - 1);
            cur[(size_t)i * N + j] = on_bnd ? 1.0 : 0.0;
        }
    }

    /* 邻居: 用 MPI_PROC_NULL 表示"不存在", 对它的收发自动变成空操作 */
    const int up   = (rank == 0)        ? MPI_PROC_NULL : rank - 1;
    const int down = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;

    MPI_Request req[4];
    const double t0 = MPI_Wtime();
    int iter = 0;

    for (; iter < ITERS; ++iter) {
        double *ghost_top = cur;                            /* 上行 ghost */
        double *my_first  = cur + (size_t)N;                /* 本地第 1 行 */
        double *my_last   = cur + (size_t)rows * N;         /* 本地最后 1 行 */
        double *ghost_bot = cur + (size_t)(rows + 1) * N;   /* 下行 ghost */

        /* ---------- 步骤 1: 先发起 4 个非阻塞通信, 再等 ---------- */
        int k = 0;
        MPI_Irecv(ghost_top, N, MPI_DOUBLE, up,   0, MPI_COMM_WORLD, &req[k++]);
        MPI_Irecv(ghost_bot, N, MPI_DOUBLE, down, 1, MPI_COMM_WORLD, &req[k++]);
        MPI_Isend(my_first, N, MPI_DOUBLE, up,   1, MPI_COMM_WORLD, &req[k++]);
        MPI_Isend(my_last,  N, MPI_DOUBLE, down, 0, MPI_COMM_WORLD, &req[k++]);
        MPI_Waitall(k, req, MPI_STATUSES_IGNORE);

        /* ---------- 步骤 2: 计算本地内点(整行 bulk 通信已完成) ---------- */
```

```
        double mydiff = 0.0;
        for (int i = 0; i < rows; ++i) {
            const double *up_r   = cur + (size_t)i * N;
            const double *mid_r  = cur + (size_t)(i + 1) * N;
            const double *down_r = cur + (size_t)(i + 2) * N;
            double       *out    = nxt + (size_t)(i + 1) * N;

            out[0] = mid_r[0];              /* 全局左右边界固定 */
            out[N - 1] = mid_r[N - 1];
            double d = 0.0;
            for (int j = 1; j < N - 1; ++j) {
                const double v = 0.25 * (mid_r[j - 1] + mid_r[j + 1] + up_r[j] + down_r[j]);
                d += fabs(v - mid_r[j]);    /* 局部累加, 循环外不碰共享变量 */
                out[j] = v;
            }
            mydiff += d;
        }
        /* 保持 ghost 行在缓冲区交换后仍有效(边界 rank 的 ghost 恒为边界值) */
        for (int j = 0; j < N; ++j) {
            nxt[j]                          = cur[j];
            nxt[(size_t)(rows + 1) * N + j] = cur[(size_t)(rows + 1) * N + j];
        }
        double *tmp = cur; cur = nxt; nxt = tmp;   /* 双缓冲交换 */

        /* ---------- 步骤 3: 全局收敛判定(所有 rank 必须得到同一结论) ---------- */
        double gdiff = 0.0;
        MPI_Allreduce(&mydiff, &gdiff, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
        if (gdiff / ((double)N * (double)N) < TOL) { ++iter; break; }
    }
    const double t1 = MPI_Wtime();

    /* ---------- 校验和: 只做一次, 用于结果正确性检查 ---------- */
    double local_sum = 0.0;
    for (int i = 1; i <= rows; ++i)
        for (int j = 0; j < N; ++j) local_sum += cur[(size_t)i * N + j];
    double gsum = 0.0;
    MPI_Reduce(&local_sum, &gsum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        const double per_it = (t1 - t0) / (iter > 0 ? iter : 1);
        const double bytes  = (double)(N - 2) * (double)(N - 2) * sizeof(double) * 2.0;
        printf("P=%d N=%d iters=%d 总时间=%.4f s  每迭代=%.3f ms  "
               "聚合有效带宽=%.1f GB/s  校验和=%.6f\n",
               size, N, iter, t1 - t0, per_it * 1e3, bytes / per_it / 1e9, gsum);
    }
    free(A); free(B);
    MPI_Finalize();
    return 0;
}
```

【代码做什么？】 1. `MPI_Init` 建立 rank/size，随后做**1D 行块域分解**：全局 N×N 网格按行切成 `size` 块，每块 `rows = N/size` 行，本地数组额外带上下各 1 行 ghost。 2. 初始化时把全局边界（`gi<=0 || gi>=N-1 || j==0 || j==N-1`）置 1.0，其余置 0.0——这就是 Dirichlet 边界条件的固定值。 3. 每轮迭代分三段：**(a) 4 个非阻塞通信**（`Irecv` 两行 ghost、`Isend` 本地首末行），最后一次 `Waitall`；**(b) 计算**所有本地内点的 5 点平均并累加局部 `mydiff`；**(c)** **`Allreduce`** 求和后所有 rank 用同一判据决定是否退出。 4. 双缓冲 `cur/nxt` 交换避免写覆盖，最后用 `MPI_Reduce` 汇总校验和，rank 0 打印每迭代耗时与聚合有效带宽。

【并行机制与性能解说】 – **进程如何创建、工作如何分配**：`mpirun -np P` 由 MPI launcher（`orted` / `hydra`）在节点上 fork P 个进程；**没有线程创建**，每个 rank 是一个独立地址空间，靠固定映射 `rank ↔ 行区间` 完成 assignment，属于纯**静态分配**（零运行时调度开销）。 – **共享数据如何处理**：**没有共享数据**。每 rank 只有自己的 `rows+2` 行；跨 rank 的依赖靠**显式复制**（ghost row）解决；通信是整行 bulk 传输（`N` 个 double = 32 KB @ N=4096）。 – **Work / Span / 并行度**：设每点代价 `c_pt`（≈ 20 B 内存流量 + 5 flops，见第 4 节）。 – 每迭代每 rank 的 Work：`W = rows·(N−2)·c_pt`（≈ 4096·512 = 2.1M 点 @ P=8）。 – 每迭代关键路径：`D = c_pt + 2·α + N·4/BW_net + log₂P·(α + 8/BW_net)`。因为同一次迭代内所有内点**互相独立**（只读上一轮的 `cur`），点更新本身的 span 是 O(1)（用无限处理器），通信的 span 是"消息延迟 + 归约树深度"。 – **并行度 = W_total/D_total**：整体 T1 = `ITERS·N²·c_pt`，T∞ = `ITERS·(α + N·4/BW + log₂P·α)`。取 α = 2 μs、BW = 6 GB/s、N = 4096、ITERS = 200：T∞ ≈ 200 × (2 + 2.7 + 2·2) μs ≈ 1.7 ms，而 T1 ≈ 200 × 16.8M × 20 B / 15 GB/s ≈ 4.5 s ⇒ **并行度 ≈ 2600**，远大于 P = 128。**结论：MPI 场景下 span 通常不是瓶颈，瓶颈是带宽与延迟的开销项。** – **瓶颈**：(1) **内存带宽**——5 点 stencil 的算术强度只有 0.25

flop/byte（严重 memory-bound）；(2) **通信/计算比 ≈ 0.4/rows**，rows 小时通信占比飙升（P 增大而问题不变时）；(3) **`Allreduce` 频率**——每迭代一次全局集合操作，是小消息延迟（α）的固定税；(4) **负载不均**——若 N 不被 P 整除则尾部 rank 空闲。

3.2 示例二：OpenMP 调度策略、reduction 与伪共享对照

```c
/* omp_scaling.c —— 调度策略对比(reduction) + 伪共享对照
 *
 * 编译: gcc -O3 -fopenmp -march=native omp_scaling.c -o omp_scaling -lm
 *       (ICC/Clang: icx -O3 -qopenmp ... / clang -O3 -fopenmp ...)
 * 运行: OMP_NUM_THREADS=8 OMP_PROC_BIND=close OMP_PLACES=cores ./omp_scaling 20000000
 *       (绑定线程到物理核, 避免迁移与跨 NUMA 访存)
 */
#define _GNU_SOURCE
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

/* 每次迭代的"真实工作": 混合 sqrt/sin/cos, 让编译器无法把它优化成常量 */
static inline double kernel(long i)
{
    const double x = (double)(i & 1023) * 1e-3;
    return sqrt(x + 1.0) * sin(x) + cos(x) * 0.5;
}

/* ---------- (1) 三种调度策略 ---------- */
static double bench_sched(int mode, long chunk, long N)
{
    double sum = 0.0;
    const double t0 = omp_get_wtime();

    if (mode == 0) {                    /* static: 进入并行区时一次性切块 */
        #pragma omp parallel for schedule(static, chunk) reduction(+:sum)
        for (long i = 0; i < N; ++i) sum += kernel(i);
    } else if (mode == 1) {             /* dynamic: 每 chunk 一次原子领号 */
        #pragma omp parallel for schedule(dynamic, chunk) reduction(+:sum)
        for (long i = 0; i < N; ++i) sum += kernel(i);
    } else {                            /* guided: chunk 递减, 不小于 chunk */
        #pragma omp parallel for schedule(guided, chunk) reduction(+:sum)
        for (long i = 0; i < N; ++i) sum += kernel(i);
    }

    const double dt = omp_get_wtime() - t0;
    if (sum < -1e300) printf("never\n");   /* 防止 sum 被优化掉 */
    return dt;
}

/* ---------- (2) 伪共享对照: 只有 padding 不同 ---------- */
struct padded { long long v; char pad[64 - sizeof(long long)]; };   /* 每计数器独占 64B */

static double bench_falsesharing(long iters, int use_padding)
{
    const int    T      = omp_get_max_threads();
    const size_t stride = use_padding ? sizeof(struct padded) : sizeof(long long);
    char *buf = (char *)calloc((size_t)T * stride + 64, 1);
    long long *base = (long long *)buf;          /* 与 padding 无关, 都按 long long 访问 */
    long long **slot = (long long **)malloc((size_t)T * sizeof(long long *));
    for (int t = 0; t < T; ++t) slot[t] = (long long *)(buf + (size_t)t * stride);

    const double t0 = omp_get_wtime();
    #pragma omp parallel
    {
        const int tid = omp_get_thread_num();
        long long *mine = slot[tid];
        for (long i = 0; i < iters; ++i) mine[0] += 1;   /* 独占: L1 命中; 共享: 乒乓 */
    }
    const double dt = omp_get_wtime() - t0;

    free(slot); free(buf);
    return dt;
}

int main(int argc, char **argv)
{
    const long N = (argc > 1) ? atol(argv[1]) : 20000000L;
```

```
    const long CH[] = { 1, 64, 4096, 262144 };   /* 四档 chunk, 观察开销/均衡权衡 */
    const int  NC   = (int)(sizeof(CH) / sizeof(CH[0]));

    printf("== 调度策略 (线程=%d, N=%ld) ==\n", omp_get_max_threads(), N);
    printf("%-22s %10s %12s\n", "schedule", "time(s)", "Miter/s");
    for (int m = 0; m < 3; ++m) {
        const char *nm = (m == 0) ? "static" : (m == 1) ? "dynamic" : "guided";
        for (int c = 0; c < NC; ++c) {
            double best = 1e30;
            for (int rep = 0; rep < 3; ++rep) {           /* 取 3 次最小值, 降噪 */
                const double dt = bench_sched(m, CH[c], N);
                if (dt < best) best = dt;
            }
            char label[64];
            snprintf(label, sizeof(label), "%s,chunk=%ld", nm, CH[c]);
            printf("%-22s %10.4f %12.1f\n", label, best, (double)N / best / 1e6);
        }
    }

    printf("\n== 伪共享对照 (线程=%d, 每线程 3e7 次自增) ==\n", omp_get_max_threads());
    const long FS = 30000000L;
    const double t_share = bench_falsesharing(FS, 0);
    const double t_pad   = bench_falsesharing(FS, 1);
    printf("共享 cache line : %.4f s (%.1f Mop/s)\n", t_share, (double)FS / t_share / 1e6);
    printf("64B 隔离        : %.4f s (%.1f Mop/s)\n", t_pad,   (double)FS / t_pad   / 1e6);
    printf("伪共享惩罚      : %.1fx\n", t_share / t_pad);
    return 0;
}
```

【代码做什么？】 1. `bench_sched` 对同一个循环分别用 `schedule(static|dynamic|guided, chunk)` 执行，`reduction(+:sum)` 让运行时自动分配每线程私有累加器并在结束时合并；外层取 3 次最小值降噪。 2. `bench_falsesharing` 分配 `T` 个 `long long` 计数器，唯一区别是 `stride`：无 padding 时 T 个计数器**挤在同一条 64 字节 cache line 内**，有 padding 时每个独占一条 cache line；每个线程只更新"自己"的计数器。 3. `main` 打印调度策略的时间/吞吐表，以及伪共享惩罚倍数（理论上无 padding 版本会慢几十倍）。

【并行机制与性能解说】 – **线程如何创建、工作如何分配**：`omp_get_max_threads()` 个 worker 在首次并行区被**懒创建**并常驻（GCC/Clang 运行时默认在屏障处自旋/休眠）；`parallel for` 由编译器**轮廓化**成一个带 `(lower, upper, stride)` 参数的函数，每个线程调用它并只跑自己那段。`static` 的切分发生在**进入并行区时**（纯索引运算，无原子）；`dynamic/guided` 每个 chunk 需要一次**原子领号**（`libgomp` 用 `__atomic_fetch_add` 或带锁的回退路径）。 – **共享数据如何处理**：`sum` 是 reduction 变量 → 每线程一份**私有副本**（栈/线程本地），循环结束由运行时按树形合并；`slot[tid]` 是每线程独占写 → 但若落在同一 cache line，**写操作要求独占所有权（MESI 的 M 态）**，每写一次就把别的核的副本置为 Invalid，造成 cache line 在核间来回搬运。 – **Work / Span / 并行度**： – Work：$W = N \cdot c_iter$（+ 调度开销）。**Span**：$D \approx (N/c) \cdot t_sync + c \cdot t_iter + \log_2 P \cdot t_merge$，其中 $(N/c) \cdot t_sync$ 是**领号临界区的串行链**（这是最容易忽略的串行段），$c \cdot t_iter$ 是动态调度固有的最大不均衡（最后一个 chunk），$\log_2 P \cdot t_merge$ 是 reduction 的合并树。 – **并行度 = $W/D = N \cdot c_iter / ((N/c) \cdot t_sync + c \cdot t_iter + O(\log P))$**。 – 数值例：$N = 2 \times 10^7$、$t_iter = 8$ ns、$t_sync = 25$ ns、$P = 8$。 – `chunk = 1`：$D \approx 2 \times 10^7 \times 25$ ns $= 0.5$ s，而 $W = 0.16$ s ⇒ 并行度 < 1，**比串行还慢**（领号临界区成为串行瓶颈，正是讲义中"fine granularity: potential for high synchronization cost / it's serial execution (recall Amdahl's Law)"）。 – `chunk = 4096`：$D \approx (2 \times 10^7/4096) \times 25$ ns $+ 4096 \times 8$ ns $= 0.122$ ms $+ 0.033$ ms ≈ 0.155 ms。 – `chunk = 262144`：$D \approx 76 \times 25$ ns $+ 2.1$ ms ≈ 2.1 ms（不均衡项主导）。 – 由 $c* = \text{sqrt}(N \cdot t_sync / t_iter) = \text{sqrt}(2 \times 10^7 \times 25/8) \approx 7.9 \times 10^3$ 可知 chunk≈4096—8192 最优——这就是"均匀代价循环用 static、极不均匀代价用 dynamic + 中等 chunk（或 guided）"的定量依据。 – 伪共享部分：$W = T \cdot FS$ 次自增，**Span = $FS \cdot t_line_pingpong$**（每个计数器的更新链在共享 cache line 上被串行化）。$FS = 3 \times 10^7$：L1 命中自增约 1—1.5 ns（独占时 Span ≈ 30—45 ms）；跨核乒乓约 40—100 ns ⇒ Span ≈ 1.2—3 s。因此**即使 Work 相同，Span 相差 50—100 倍，扩展性直接崩塌**——这是"Work/Span 决定并行度"的最直观演示。 – **瓶颈**：(1) 领号临界区的**串行化 + cache line 争抢**；(2) 伪共享；(3) 线程绑核失败的**NUMA 远端访问**（延迟约 1.5—2×，带宽约 1/2）；(4) 每次并行区的 fork–join（约 1—10 μs）在小循环上主导。

3.3 示例三：Cilk / OpenCilk —— `cilk_for`、并行快排与 work stealing 的观测

```
/* cilksort.cpp —— Cilk/OpenCilk: cilk_for 映射、并行快速排序、worker 数观测
 *
 * 编译(OpenCilk, 推荐):
 *     clang++ -fopencilk -O3 -march=native cilksort.cpp -o cilksort -lm
 * 编译(传统 Intel Cilk Plus / GCC <= 7; GCC 8 起已移除 -fcilkplus):
 *     g++ -fcilkplus -O3 cilksort.cpp -o cilksort -lm
 * 串行对照: g++ -O3 -DCILK_SERIAL cilksort.cpp -o cilksort_serial -lm
 * 运行: CILK_NWORKERS=8 ./cilksort 100000000
 */
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <chrono>
#include <algorithm>

#if !defined(CILK_SERIAL)
#  include <cilk/cilk.h>
#  include <cilk/cilk_api.h>
#  if defined(__has_include)
#    if __has_include(<cilk/reducer_opadd.h>)
#      define HAVE_CILK_REDUCER 1
#      include <cilk/reducer_opadd.h>
#    endif
#  endif
#endif

#define CUTOFF 4096      /* 小于该规模就串行排序: 避免 spawn 开销压过并行收益 */

static void insertion_sort(int *a, int lo, int hi)
{
    for (int i = lo + 1; i < hi; ++i) {
        const int key = a[i];
        int j = i - 1;
        while (j >= lo && a[j] > key) { a[j + 1] = a[j]; --j; }
        a[j + 1] = key;
    }
}

/* Lomuto 划分, 返回 pivot 的最终下标 */
static int partition_lomuto(int *a, int lo, int hi)
{
    const int pivot = a[hi - 1];
    int i = lo;
    for (int j = lo; j < hi - 1; ++j)
        if (a[j] < pivot) { std::swap(a[i], a[j]); ++i; }
    std::swap(a[i], a[hi - 1]);
    return i;
}

/* ---------- 串行版本: 用于测 T1 (work) ---------- */
static void qsort_serial(int *a, int lo, int hi)
{
    while (hi - lo > CUTOFF) {
        const int p = partition_lomuto(a, lo, hi);
        qsort_serial(a, lo, p);
        lo = p + 1;                      /* 尾递归改成循环, 只对左半递归 */
    }
    insertion_sort(a, lo, hi);
}

/* ---------- 并行版本: 先划分, spawn 左半, 本地继续右半 ---------- */
static void qsort_parallel(int *a, int lo, int hi)
{
    if (hi - lo <= CUTOFF) { insertion_sort(a, lo, hi); return; }
    const int p = partition_lomuto(a, lo, hi);
    cilk_spawn qsort_parallel(a, lo, p);      /* 把左半交给"可被窃取"的 child */
    qsort_parallel(a, p + 1, hi);             /* 本地继续右半(continuation 语义) */
    cilk_sync;                                /* 函数末尾本就有隐式 sync, 写出更清晰 */
}

/* ---------- 每 worker 独占 cache line 的并行求和 ---------- */
```

```
static long long checksum_padded(const int *a, long n)
{
    int W = 1;
#if !defined(CILK_SERIAL)
    W = __cilkrts_get_nworkers();
#endif
    long long *partial = nullptr;
    if (posix_memalign((void **)&partial, 64, (size_t)W * 64) != 0) exit(1);
    memset(partial, 0, (size_t)W * 64);

#if defined(CILK_SERIAL)
    for (long i = 0; i < n; ++i) partial[0] += a[i];
#else
    cilk_for (long i = 0; i < n; ++i) {
        const int w = __cilkrts_get_worker_number();
        partial[(size_t)w * 8] += a[i];       /* 每 worker 间隔 64B: 无伪共享 */
    }
#endif
    long long s = 0;
    for (int w = 0; w < W; ++w) s += partial[(size_t)w * 8];
    free(partial);
    return s;
}

#if defined(HAVE_CILK_REDUCER)
/* 惯用写法: reducer(hyperobject) 自动为每个 worker 维护视图, 窃取时合并 */
static long long checksum_reducer(const int *a, long n)
{
    cilk::reducer_opadd<long long> s(0);
    cilk_for (long i = 0; i < n; ++i) s += a[i];
    return s.get_value();
}
#endif

/* 讲义中的 recursive_for: 折半产生任务, 是 cilk_for 展开后的典型形态 */
static void recursive_for(int *a, int begin, int end, long long *local)
{
    while (end - begin > 1024) {
        const int mid = begin + (end - begin) / 2;
#if !defined(CILK_SERIAL)
        cilk_spawn recursive_for(a, begin, mid, local);
#endif
        begin = mid;                        /* 本地继续后半段 */
    }
    long long s = 0;
    for (int i = begin; i < end; ++i) s += a[i];
    *local += s;
}

static double now_sec()
{
    using namespace std::chrono;
    return duration<double>(steady_clock::now().time_since_epoch()).count();
}

int main(int argc, char **argv)
{
    const long n = (argc > 1) ? atol(argv[1]) : 100000000L;

    int *a = (int *)malloc((size_t)n * sizeof(int));
    int *b = (int *)malloc((size_t)n * sizeof(int));
    if (!a || !b) { fprintf(stderr, "内存不足\n"); return 1; }

    srand(12345);
    for (long i = 0; i < n; ++i) a[i] = rand();      /* 随机数据 ⇒ 划分接近平衡 */
    memcpy(b, a, (size_t)n * sizeof(int));

    /* --- 串行基准 (T1) --- */
    double t0 = now_sec();
    qsort_serial(b, 0, (int)n);
    double t1 = now_sec();
    const double T1 = t1 - t0;
```

```
    /* --- 并行版本 (T_P) --- */
    t0 = now_sec();
#if defined(CILK_SERIAL)
    qsort_serial(a, 0, (int)n);
#else
    qsort_parallel(a, 0, (int)n);
#endif
    t1 = now_sec();
    const double TP = t1 - t0;

    if (memcmp(a, b, (size_t)n * sizeof(int)) != 0) { fprintf(stderr, "结果不一致!\n"); return 2; }

    int nw = 1;
#if !defined(CILK_SERIAL)
    nw = __cilkrts_get_nworkers();
#endif
    const long long sum = checksum_padded(a, n);
#if defined(HAVE_CILK_REDUCER)
    const long long sum2 = checksum_reducer(a, n);
#else
    const long long sum2 = sum;
#endif

    printf("workers=%d  n=%ld\n", nw, n);
    printf("串行 T1   = %7.3f s\n", T1);
    printf("并行 T_P  = %7.3f s   (加速比 %.2fx, 效率 %.1f%%)\n",
           TP, T1 / TP, 100.0 * T1 / TP / nw);
    printf("校验: memcmp=OK, sum=%lld, sum(reducer)=%lld\n", sum, sum2);
    free(a); free(b);
    return 0;
}
```

【代码做什么？】 1. `qsort_serial` 是**同一算法去掉 spawn** 的版本，用来测 `T1`（Work）；尾递归改成循环只对左半递归，避免深栈（讲义中的"run continuation first" 思路在串行代码里的对应物）。 2. `qsort_parallel` 先划分，然后 `cilk_spawn` 左半、本地继续右半——这正是讲义描述的实现选择："Run child first: record continuation for later execution"（continuation stealing）：**child 立即执行，续体压入本地 dequeue 供别人窃取**。 3. `checksum_padded` 用 `cilk_for` 做数据并行映射，并用"每 worker 独占 64 字节"的局部累加器避免伪共享；`checksum_reducer` 展示惯用的 reducer（hyperobject）写法（若编译器提供 reducer 头文件）。 4. `recursive_for` 是讲义中 `cilk_for` 折半展开的形态：不断二分并把前半段 spawn 出去、本地继续后半段。 5. 最后比较串行/并行结果是否一致（`memcmp`），打印加速比与效率。

【并行机制与性能解说】 – **worker 如何创建、工作如何分配**：Cilk 运行时在首次 spawn 时**懒创建**一个 worker 线程池，大小 ≈ 机器执行上下文数（`CILK_NWORKERS` 可覆盖）。`cilk_spawn` 在编译后是"把续体 push 到本 worker 的 dequeue 尾部 + 跳转到 child 代码"的少量指令；**只有真的发生窃取**才会访问其他 worker 的 dequeue。`cilk_for` 被编译器变换为**递归折半 + spawn/本地继续**（类似上面的 `recursive_for`），因此它天然产生 O(log N) 层、每层可窃取的工作。 – **共享数据如何处理**：共享数组 `a[]` 就地划分（in-place partition），因此**没有显式通信**；同步完全由 spawn/sync 表达；reducer 通过"每 worker 一个视图（view）+ 窃取/加入时合并"避免锁。 – **Work / Span / 并行度**： – **Work（T1）**：平衡划分下 `T1 = Θ(n log n)` 次比较。n = 10^8 ⇒ 约 2.7×10^9 次操作。 – **Span（T∞）**：由于 `partition_lomuto` 本身是**串行的**，且每次只 spawn 左半、本地继续右半，关键路径是"每一层划分 + 最后一次串行排序"：`T∞ = Θ(n + g log g) = Θ(n)`（主导项是顶层那次 O(n) 划分不断累加在右脊上）。 – **并行度 = T1/T∞ = Θ(log n)**。n = 10^8 ⇒ $\log_2 n \approx 27$ ⇒ **并行度 ≈ 27**。也就是说，**即使有 64 个核，这个快排的"理论可用核数"也只有约 27**（greedy 界 `T_P ≤ T1/P + T∞` 在此变成 `T_P ≈ T1/27 + T∞`，P 超过 27 后几乎收益归零）。 – **怎么改善**：把 **span 缩短**——并行化 partition（并行划分 span 可到 `O(log n)`），并对**两半都 spawn**，则 `T∞ = O(log² n)`，并行度升到 `Θ(n/log n)`（$10^8/27 \approx 3.7\times10^6$）。这正是"**扩展性上限由 span 而非 spawn 数量决定**"的最好例证。 – **瓶颈**：(1) **span**（串行 partition 链）——上一条已量化；(2) **窃取成本**——每次窃取要跨核访问 victim 的 dequeue 头部（0.1–5 μs），若 cutoff 太小（spawn 过密），窃取/推送次数暴涨；(3) **`recursive_for` 与 spawn-per-iteration 的差异**——讲义指出，在循环里逐次 `cilk_spawn` 会让"剩余迭代的续体"在**每次迭代后**都变成可窃取项，因此**更快把机器填满**；而折半递归需要深入若干层才有足够可窃取的工作，机器填充更慢（但队列条目更少、粒度更粗）；(4) **伪共享**——若图省事用 `partial[worker]` 而不做 64 字节隔离，求和会被 cache line 乒乓主导。

---

4. 性能模型与复杂度分析

4.1 四个定量工具

1. **Amdahl 定律**：设固有串行比例 `S`，则 `S_P ≤ 1/S`。讲义给出的图像是 `S = 0.01/0.05/0.1` 对应的上限 100/20/10。**关键推论：哪怕只有 5% 的串行段，P = 32 时的上限也只有约 17×。**
2. **Work–Span（T1/T∞）**：`T_P ≥ max(T1/P, T∞)`，贪心/工作窃取调度器 `T_P ≤ T1/P + T∞`；并行度 `= T1/T∞`；**加速比上限 = min(P, T1/T∞)**。
3. **算术强度与 Roofline**：`AI = FLOPs / Bytes`；可达性能 `P ≤ min(峰值算力, AI × 内存带宽)`。
4. **延迟–带宽模型**：`t_msg = α + n/BW`（α = 启动延迟，n = 消息字节数，BW = 有效带宽）；集合通信 `t_coll ≈ log₂P·α + (P−1)/P·n/BW`。

表 4：典型延迟/带宽量级（用于定量估算，来自公开硬件与实现资料）

操作	典型延迟	说明
L1 命中	~1 ns（4–5 cycle）	独占状态下的 load/store
L1 上的原子 RMW（独占行）	~1–2 ns	无竞争快路径
LLC 命中（同 socket）	~15–40 ns	视核数/拓扑
跨核 cache line 迁移（同 socket）	~30–100 ns	**伪共享的惩罚来源**
跨 socket/NUMA 远端访存	~100–250 ns	带宽约为本地一半
无竞争 mutex lock/unlock	~10–20 ns	快路径原子
高竞争锁/单点计数器	每次 ~50–200 ns	串行化 + 行乒乓
集中式屏障（P=64）	~2–5 μs	≈ P × 行传输
树形屏障（P=64）	~0.3–1 μs	O(log P)
OpenMP 并行区 fork–join	~1–10 μs	唤醒 + 屏障 + join
Cilk 本地 push/pop	~1–5 ns	几次内存操作
Cilk 一次成功窃取	~0.1–5 μs	跨核访问 victim 队列
共享内存 MPI 小消息	~0.3–1 μs	节点内
InfiniBand 小消息（跨节点）	~1–5 μs	α 主导
跨节点有效带宽（单 rank/单端口）	~3–20 GB/s	大消息渐近

4.2 数值算例 A：MPI stencil 的性能上界（与 3.1 代码对应）

假设参数

参数	取值
全局网格	N = 8192，全局 $8192^2 = 6.71\times10^7$ 点，float 数组共 256 MB
进程数 P	128（16 节点 × 8 rank/节点，节点内共享内存 MPI）
每 rank 行数 rows	8192/128 = 64
每点计算	4 次加法 + 1 次乘法 = 5 flops
每点内存流量	读 5 个点（邻域可复用 ⇒ 实际约 2 次流式读+写 = 8 B 有效），按保守估计取 20 B/点（5 次 load × 4 B）
单核可用带宽	12.5 GB/s（8 核共享节点 100 GB/s）
单核峰值算力	3.0 GHz × 8 宽 SIMD × 2 (FMA) = 48 GFLOPS
网络 α / 单 rank 有效带宽	2 μs / 6 GB/s

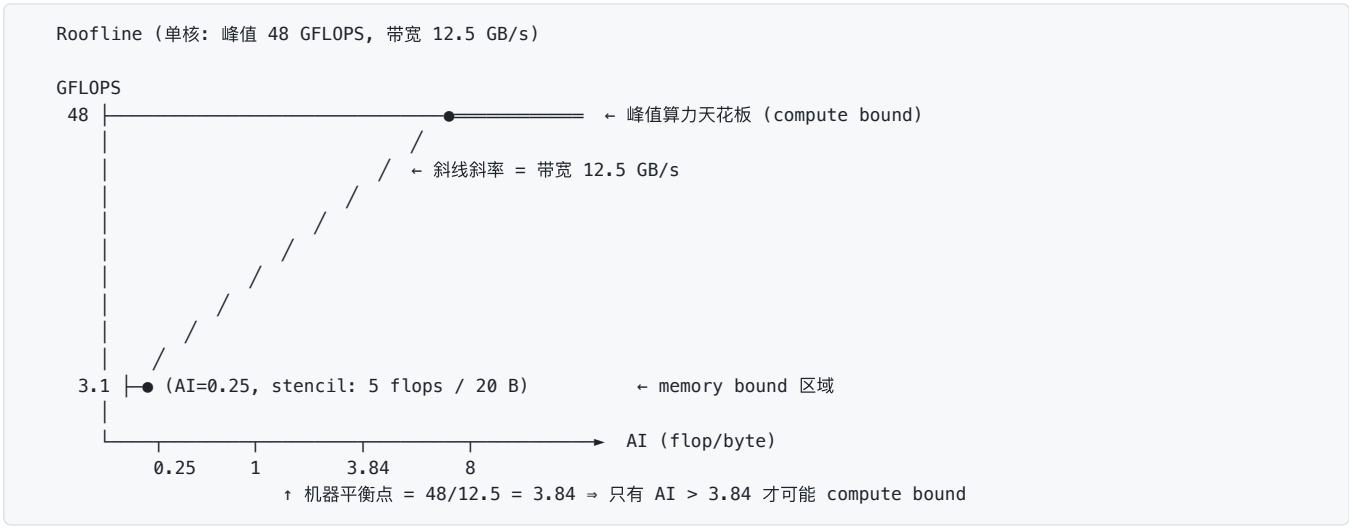
步骤 1：算术强度与 Roofline 判定 – `AI = 5 flops / 20 B = 0.25 flop/byte`。 – 单核机器平衡点 = `48 GFLOPS / 12.5 GB/s = 3.84 flop/byte`。 – `AI (0.25) ≪ 3.84` ⇒ **严重 memory-bound**；单核可达性能 `= 0.25 × 12.5 GB/s = 3.1 GFLOPS`，仅为峰值的 **6.5%**。 – 全

机聚合上界 `= 0.25 × (128 × 12.5 GB/s) = 400 GFLOPS`，而全机峰值 `= 128 × 48 = 6144 GFLOPS` ⇒ **无论怎么优化，这个 stencil 也只能跑到约 400 GFLOPS（6.5%）**。

步骤 2：每迭代每 rank 的计算时间 – 每 rank 内点 = 8192 × 64 = 524288 点。 – 内存流量 = 524288 × 20 B = 10.49 MB。 – `t_comp = 10.49 MB / 12.5 GB/s = 839 μs`（若实际有效流量按 8 B/点计则约 336 μs，这里取保守的 839 μs）。

步骤 3：每迭代每 rank 的通信时间 – halo 数据 = 2 行 × 8192 float × 4 B = 65.5 KB。 – 传输时间 = 65.5 KB / 6 GB/s = **10.9 μs**；加上 2 条消息的 α = 2 × 2 μs = 4 μs；`MPI_Allreduce`（约 $2 \cdot \log_2 128 = 14$ 步 × 2 μs ≈ 28 μs，实际实现用树形会低一些）。 – `t_comm ≈ 10.9 + 4 = 14.9 μs`（halo，可与计算重叠/接近）；`Allreduce ≈ 10–28 μs`。

步骤 4：结论 – 每迭代 `t_iter ≈ 839 + 15 + 20 ≈ 874 μs`，其中通信约占 **4%**。 – **表面–体积比公式**：`通信字节 / 计算字节 = 0.4/rows = 0.4/64 = 0.00625`；换成时间来算，`t_comm/t_comp ≈ 14.9/839 = 1.8%`，量级一致。 – **强扩展（fixed problem, P↑）**：P 从 128 → 1024 时 `rows = 8`，通信占比按 `1/rows` 放大 8 倍 ⇒ 通信从 ~4% 涨到 ~20%，**剩余可扩展性被通信吃掉**。 – **弱扩展（fixed rows/rank, P↑）**：`rows` 恒为 64，通信占比恒定 ⇒ **弱扩展效率接近 100%**（这正是 stencil 应用在超算上都用弱扩展报告的原因）。 – **提升方向**：因为 memory–bound，唯一有效手段是**降低每点内存流量**（时间分块/缓存分块：沿 N 方向分成条带，使 5 个邻点尽量命中 L1/L2），否则加核无用。这就是 Roofline 的直接指导。



4.3 数值算例 B：OpenMP 动态调度的最优 chunk 与并行度上限（与 3.2 代码对应）

假设参数：N = 10^9 次迭代；`t_iter = 1 ns`（每次迭代算一点工作）；动态领号临界区 `t_sync = 20 ns`（原子 + cache line 传输）；P = 64 线程。

- 总 Work：`T1 = 10⁹ × 1 ns = 1.0 s`。
- **串行段（领号）总时间**：`T_serial = (N/c) × t_sync`。
- **最大不均衡**：`T_imb = c × t_iter`（最后一个 chunk 可能只有一个线程在跑）。
- 目标函数（关键洞察：领号是**串行**的，按 Amdahl 直接相加）：`T_P ≈ T1/P + T_serial + T_imb = T1/P + (N/c)·t_sync + c·t_iter`
- **最优 chunk**（对 c 求导置零）：`c* = sqrt(N·t_sync / t_iter) = sqrt(10⁹ × 20 / 1) = sqrt(2×10¹⁰) ≈ 1.41×10⁵`。
- **最优开销**：`2·sqrt(N·t_sync·t_iter) = 2 × 1.41×10⁵ ns = 283 μs = 0.283 ms`。

chunk c	串行领号时间 (N/c)·t_sync	不均衡 c·t_iter	T_P（含 T1/64 = 15.6 ms）	加速比	效率
1	20 s	1 ns	≈ 20.02 s	0.05×	0.08%
64	312.5 ms	64 ns	328 ms	3.0×	4.8%
1024	19.5 ms	1.0 μs	35.1 ms	28.5×	44.5%
1.41×10^5 (c*)	0.141 ms	0.141 ms	**15.9 ms**	**62.9×**	**98.3%**
10^6	0.02 ms	1.0 ms	16.6 ms	60.2×	94.1%
static（c = N/P = 1.56×10^7）	0	15.6 ms（若每迭代同代价则≈0）	15.6 ms	64×	100%

结论： 1. **`schedule(dynamic,1)` 在 10^9 次迭代下把程序变成 20 秒的串行程序**——这正是讲义反复强调的"细粒度划分的同步开销本身就是串行执行（recall Amdahl's Law）"。 2. chunk 的选取遵循 `c* = sqrt(N·t_sync/t_iter)`；`schedule(guided)` 用递减 chunk 逼近这个折中，**不需

要手工调参。 3. 若每次迭代代价均匀，`static` 是零开销且 100% 效率的最优解 ⇒ **先试 static，再按需升级到 dynamic/guided**（讲义 TIP #1："Always implement the simplest solution first, then measure performance"）。

4.4 数值算例 C：Cilk 的并行度上限与 parallel slack（与 3.3 代码对应）

- **假设**：n = 10^8 个 int；比较/交换总代价 `T1 = n·log₂n·t_cmp = 10⁸ × 27 × 1 ns = 2.7 s`；span `T∞ = n·t_part = 10⁸ × 1 ns = 0.1 s`（顶层划分链主导）。
- **并行度** `= T1/T∞ = 27`。
- **greedy 界给出的加速比上限**（P = 64）：`T_64 ≤ T1/64 + T∞ = 0.042 + 0.1 = 0.142 s ⇒ S ≤ 2.7/0.142 ≈ 19×`（效率 30%）。
- 若 P = 27：`T_27 ≈ 0.1 + 0.1 = 0.2 s ⇒ S ≈ 13.5×`（效率 50%）。
- 若把 partition 并行化（span → `O(log²n)·t` ≈ 27^2 × 1 ns ≈ 0.73 μs）并对两半都 spawn：`T∞' ≈ 0.73 μs`，并行度 ≈ 3.7×10^6 ⇒ **在 P = 64 时受限于 P 本身**，`T_64 ≈ T1/64 + 1 μs ≈ 42 ms ⇒ S ≈ 64×`（效率接近 100%）。
- **parallel slack 视角（讲义准则 ≈ 8）**：`T1/T∞ = 27` 对 P = 8 的机器来说 slack = 27/8 ≈ 3.4（偏低，容易受调度抖动影响）；把 cutoff `g` 调小能提高 slack（任务更多），但**窃取次数随之增加**：每次窃取的代价 0.1–5 μs，若窃取次数达到 `10⁵` 量级，额外开销就是 0.01–0.5 s，足以吃掉加速收益。**因此 cutoff 的调优目标是"让每次窃取的收益（被偷走的计算量）≫ 窃取成本"**：让被偷走的子树至少包含约 10–100 μs 的计算。

表 5：三种"任务分配/调度机制"的定量对比（P = 64，每次任务代价 1 μs 的情形）

机制	每次分配开销	10^5 次分配的总开销	负载均衡质量	适用条件
`static`（MPI rank / OpenMP static）	≈ 0（编译期索引运算）	≈ 0	只有任务代价可预测时才均衡	代价均匀或可预知（stencil、稠密 GEMM）
共享工作队列 + 互斥（OpenMP dynamic, 讲义 07 的 `counter_lock`）	50–200 ns（竞争时更高）	5–20 ms（且**串行化**）	好	代价高度不均、任务数目适中（10^3–10^6）
每 worker 队列 + work stealing（Cilk）	本地 push/pop 1–5 ns；成功窃取 0.1–5 μs	本地几乎免费；窃取次数 ≪ 任务数	好，且**窃取成本被摊薄**	不规则分治、递归、需要局部性的场景

---

5. 关键要点

1. **三种系统的差别本质是"责任归属"，不是"能力"**。MPI 把分解/分配/编排/映射几乎全交给程序员（控制力最强、第一条正确程序最难）；OpenMP 由编译器 outlining + 运行时线程池接管分配与编排（`schedule` 子句是唯一的旋钮）；Cilk 把分配与映射全部交给 work-stealing 运行时。选择时先问"这个问题的瓶颈是通信、同步频率还是 span"，再选系统，而不是反过来。
2. **机制开销必须被每次迭代的计算量摊薄，这可以用一条不等式检验**：要让 P 个 worker 有收益，必须满足 `T1/P ≫ 单次机制开销`（MPI 一次消息 α、OpenMP 一次屏障/fork-join、Cilk 一次窃取）。算例：10^9 次迭代用 `schedule(dynamic,1)` 时，20 ns 的领号被串行化 10^9 次 = 20 s，程序比串行还慢。
3. **扩展性的天花板是 span（关键路径），不是 spawn / 任务的数量**。greedy 界 `T_P ≤ T1/P + T∞` 说明只要 `T∞` 不小，加核就无用；快排的 `T∞ = Θ(n)` 使并行度只有 `Θ(log n) ≈ 27`（n = 10^8）。**先并行化串行瓶颈（例如 partition），再增加并行度**。
4. **数据搬运与同步的"面积"决定实际性能**：MPI 的通信/计算比 ≈ `0.4/rows`（表面-体积比 ⇒ 弱扩展友好、强扩展受限）；OpenMP 的伪共享把本该 1 ns 的 L1 自增变成 30–100 ns 的 cache line 乒乓（50–100× 惩罚）；Cilk 的 dequeue 头尾必须分处不同 cache line，否则本地 push 被窃取者拖慢。
5. **先测后优，机制按需升级**：讲义 TIP #1——"Always implement the simplest solution first, then measure performance"。工程路径通常是：static/blocked → 发现负载不均 → dynamic/guided 或 work stealing → 发现同步/窃取开销 → 调整 chunk/cutoff（`c* = sqrt(N·t_sync/t_iter)`，cutoff 让每次窃取收益 ≫ 窃取成本），并用 `OMP_PROC_BIND` / `--bind-to core` 保证 NUMA 局部性。

---

6. 常见陷阱与注意事项

- **用同步阻塞 `Send/Recv` 做环形 halo 交换会死锁**：所有 rank 都先 `Send` 后 `Recv`，而同步 `Send` 要等接收方确认，于是全员互相等待——这正是讲义中"big problem with our message passing solver if it uses synchronous send/recv"的场景。修复方式有三：**(a)** 奇偶错开（偶数 rank 先发后收、奇数 rank 先收后发，即讲义中的修正版）；**(b)** 用配对的 `MPI_Sendrecv`；**(c)** 用非阻塞 `Isend/Irecv` + `Waitall`（最推荐，同时还能重叠通信与计算）。注意方向搞错（把"我先发"当成"我先收"）只是把死锁转移到另一侧。

- **把 OpenMP 的"隐式屏障"当成免费的**：`parallel for` 末尾、`single`、`critical` 的隐含语义都可能插入屏障，而屏障把"最慢的线程"变成相位长度。常见后果：一个循环里嵌套两次并行区 = 两次 fork–join + 两次屏障；用 `nowait` 去屏障却忘了随后的数据依赖，直接引入数据竞争。**辨析：`critical` 是互斥（保正确性），`barrier` 是相位同步（保顺序），`atomic` 只保证单个内存操作的原子性（不保证复合语句的原子性）。**
- **伪共享（false sharing）把"每线程私有变量"变成通信**：`int partial[MAX_THREADS]` 只要两个元素落在同一条 64 字节 cache line 内，两个核心就会反复争夺该行的独占权。修复：按 64 字节对齐/填充（示例 3.2 中的 `struct padded`），或用 `reduction` / reducer / 线程局部变量后一次性合并。注意 **`alignas(64)` 只解决"跨对象"共享，若数组元素密集仍需 stride 填充。**
- **忽略内存带宽与 Roofline，只盯着 FLOPS 和核心数**：5 点 stencil 的算术强度只有 0.25 flop/byte，机器平衡点约 3.84 flop/byte，因此无论多少核都只能拿到峰值算力的约 6.5%；MPI 场景下更常见的错误是"P 翻倍就期望时间减半"，而实际被 `0.4/rows` 的通信占比与网络对分带宽卡住。**先算 AI，再判断是 compute bound 还是 memory bound，然后才决定优化方向。**
- **任务粒度过细（过度 spawn / chunk=1）**：Cilk 里对每个循环迭代都 `cilk_spawn` 会让 dequeue 条目与窃取次数暴涨（窃取 0.1–5 μs 一次）；OpenMP 里 `schedule(dynamic,1)` 让计数器临界区成为串行段；MPI 里逐元素 `Send` 让小消息延迟 α 主导。**准则：单个任务的执行时间应至少是机制开销的 10–100 倍**（Cilk 的 `CUTOFF`、OpenMP 的 chunk、MPI 的整行 bulk 通信都是这条准则的实例）。
- **内存序与可见性假设错误**：讲义中的 flag 示例（`x = 1; flag = 1;` 与 `while (flag == 0); print x;`）在弱一致性机器上会因编译器/硬件重排序而失败，必须使用原子操作 + 正确的内存序（release/acquire）。同理，Cilk 的 `cilk_sync`、OpenMP 的 `barrier` / `flush` 语义之外，**不要假设"时间上后写就一定后可见"**；reducer 看似"无锁共享变量"，但它依赖运行时的同步语义，**把普通共享变量当 reducer 用是数据竞争**。

---

7. 思考题（带答案）

问题 1（MPI / 实现层）：某程序用 128 个 rank 做 1D 行块分解的 5 点 stencil，每 rank 负责 64 行、行宽 8192（float）。它用同步阻塞 `Send/Recv` 交换 halo，但运行 4 个 rank 的小规模测试时完全正常，扩到 128 个 rank 时偶发挂死。请解释原因、给出修复方案，并定量说明修复后每迭代的通信时间构成（假设 α = 2 μs、单 rank 有效带宽 6 GB/s）。

【答案】 挂死的根因是**同步阻塞通信的循环等待**：每个 rank 都先 `Send` 给上邻居、再 `Send` 给下邻居，然后才 `Recv`；同步 `Send` 只有在接收方确认数据已到达其地址空间后才返回。于是形成"所有人都已发出、所有人都在等对方接收"的环状依赖，构成死锁。小规模测试"通过"往往只是因为消息足够小、库走 **eager** 路径（发送方直接复制到网络缓冲立即返回），掩盖了问题；rank 数或消息大小变化后切换到 **rendezvous** 路径（需要接收方参与）才暴露死锁——这也解释了为什么"小规模正常、放大就挂"。修复方案（满足其一即可）：(a) **奇偶错开**：偶数 rank 先 `Send` 后 `Recv`，奇数 rank 先 `Recv` 后 `Send`（讲义 06 中给出的修正版，T0→T1→T2→… 的流向因此在每个方向上都有接收方就绪）；(b) 用 **`MPI_Sendrecv`**（库内部保证不会自锁）；(c) **改用非阻塞 `MPI_Isend/Irecv` + `MPI_Waitall`**（最推荐：既避免死锁，又能把通信与计算重叠）。定量上：halo 数据 = 2 行 × 8192 × 4 B = 65 536 B = 64 KB,传输时间 = 64 KB / 6 GB/s ≈ **10.9 μs**,两条消息的启动延迟 = 2 × 2 μs = **4 μs**（非阻塞时两条消息的延迟可部分重叠，紧下界约 2 μs）,所以每迭代通信 ≈ **11–15 μs**,而同迭代计算时间约为 `8192 × 64 × 20 B / 12.5 GB/s ≈ 839 μs`（见 4.2 节），通信占比约 **1.3%–1.8%**。结论：修复后通信不是瓶颈；但如果把问题固定而把 P 增到 1024（rows = 8），通信字节数不变而计算量下降 8 倍，通信占比升到 ~10%–14%，**这就是强扩展的真实上限来源**（通信/计算比 ≈ 0.4/rows）。

问题 2（OpenMP / 调度实现）：一个循环有 $N = 10^9$ 次迭代，每次迭代约 1 ns，动态领号临界区一次约 20 ns，机器有 64 个核。请推导最优 chunk 大小、给出该点的加速比，并说明为什么 `schedule(guided)` 往往能自动接近这个效果。

【答案】 动态调度的总时间可写成三项：**并行计算项** $T1/P$、**串行领号项** $(N/c)\cdot t_sync$（领号必须串行，按 Amdahl 直接相加到关键路径）、**不均衡项** $c\cdot t_iter$（最后一个 chunk 只能由一个线程做，其间其他线程空闲）。即 $T_P(c) \approx T1/P + (N/c)\cdot t_sync + c\cdot t_iter$。 对 c 求导：$-N\cdot t_sync/c^2 + t_iter = 0 \Rightarrow c* = \sqrt{N\cdot t_sync/t_iter} = \sqrt{10^9 \times 20\ ns / 1\ ns} = \sqrt{2\times10^{10}} \approx 1.41\times10^5$。此时两项开销相等：$(10^9/1.41\times10^5) \times 20\ ns = 1.41\times10^5\ ns = 141.4\ \mu s$，$c\cdot t_iter = 141.4\ \mu s$，合计 283 μs。于是 $T_P \approx 1.0\ s/64 + 283\ \mu s = 15.625\ ms + 0.283\ ms \approx 15.91\ ms$， 加速比 $S = 1.0/0.01591 \approx 62.9\times$，效率 $62.9/64 \approx 98.3\%$。对照 $c = 1$：串行领号 $10^9 \times 20\ ns = 20\ s$，$T_P \approx 20\ s$，**加速比 0.05×（比串行慢 20 倍）**；$c = 1024$：$T_P \approx 15.6 + 19.5 + 0.001 \approx 35.1\ ms$，加速比仅 28.5×。所以 chunk 的选择直接决定了可达到的**并行度上限**：$并行度 = T1/((N/c)t_sync + c\cdot t_iter) = 1/(2\times\sqrt{t_sync\cdot t_iter/N}) = \sqrt{N/(4\ t_sync\ t_iter)} \approx \sqrt{10^9/(4\times20)} \approx 3536$，远大于 64，因此在大 chunk 区间内 P 是限制项，在小 chunk 区间内串行项是限制项。`schedule(guided)` 的策略是"chunk 从大约 剩余迭代数/线程数 开始、随剩余量递减、并设一个下限"，前期大 chunk 控制领号次数（把 $(N/c)t_sync$ 压到 $O(P\cdot t_sync\cdot log)$ 量级），后期小 chunk 消除尾部不均衡（把 $c\cdot t_iter$ 压到与单次迭代同量级），因此无需精调就接近 $c*$ 给出的折中点。补充说明：若迭代代价完全均匀且可预测，`schedule(static)` 的 $T_P = T1/P$（效率 100%）比上述任何动态方案都好，**动态调度只在代价不均时才值得**。

问题 3（Cilk / work stealing 实现）：讲义强调 Cilk 采用 "run child first"（continuation stealing，续体窃取），并指出对 `for (i=0;i<N;i++) cilk_spawn foo(i);` 这种写法，continuation stealing 相对 child stealing 有明显优势。请说明 (1) 两种策略在执行顺序与空间占用上的差别；(2) 为什么窃取要从 dequeue 的**头部**而不是尾部；(3) 若把 `N` 个 `cilk_spawn` 报告为"并行度 = N"，这个说法错在哪里——请用快排的例子定量说明。

【答案】 (1) **执行顺序与空间**：continuation stealing 下，遇到 `cilk_spawn foo(i)` 时，**当前线程立即执行 child `foo(i)`**，并把"剩余工作的续体"（从 `i+1` 继续的循环控制）压入本地 dequeue，随后弹出（后进先出）。若无人窃取，执行顺序与**删掉 `cilk_spawn`** **的串行程序完全一致**（深度优先遍历调用图），而且**调用者线程一次只创建 1 个可被窃取的条目**（代表剩余全部迭代），所以 dequeue 空间是 O(1) 量级；可以证明 work queue 的存储量不超过单线程栈存储量的 T 倍（T = 线程数）。child stealing 则相反：先把 child 记录起来、**先把所有迭代的 child 都创建出来**再执行，相当于对调用图做**广度优先**遍历，需要 O(N) 空间，且无窃取时的执行顺序与串行程序显著不同（局部性更差）。(2) **为什么要从头部窃取**：本地线程在**尾部**做 push/pop（后进先出 ⇒ 深度优先 ⇒ 缓存与栈局部性好），窃取者在**头部**取（先进先出 ⇒ 拿到的是调用树中"更早搁置、覆盖更多后续工作"的大块任务）。三个收益：**(a)** 头尾分离 ⇒ 两端落在不同 cache line ⇒ 与本地线程的竞争最小（这也是实现上必须把 head/tail 用 64 字节隔开的原因）；**(b)** 偷到的任务更大 ⇒ 一次窃取（0.1–5 μs）的成本被更长的后续计算摊薄，避免"偷得比偷的成本还少"；**(c)** 与 run-child-first 配合，本地线程始终留在自己调用树的局部，窃取者拿走远端子树，**最大化双方的数据局部性**。(3) **"并行度 = 任务数"是错的**：并行度定义为 `T1/T∞`，即 Work 比关键路径（span），与 spawn 数量无关，只与**最长依赖链**有关。以并行快排（示例 3.3）为例：`cilk_spawn` 的次数是 Θ(n)，看起来"并行度 = 10^8"，但 `partition_lomuto` 本身是**串行**的，且代码是"spawn 左半、本地继续右半"，关键路径 = 逐层 partition + 最后一次串行排序 = `T∞ = Θ(n)`。于是 `T1 = Θ(n log n)`、`T∞ = Θ(n)`，`并行度 = Θ(log n)`；在 n = 10^8 时 $\log_2 n \approx 27$ ⇒ **并行度只有约 27**，greedy 界 `T_P ≤ T1/P + T∞` 在 P = 64 时给出 `T_64 ≤ 0.042 + 0.1 ≈ 0.142 s`（T1 = 2.7 s），加速比上限约 19×、效率 30%。正确的改法是**缩短 span**：把 partition 也并行化（span 降到 $O(\log^2 n)$）并对**两半都 spawn**，此时 `T∞` 只有微秒级，并行度升到 ~3.7×10^6，P = 64 时加速比可接近 64×。这正说明：**任务多 ≠ 扩展性好；决定扩展性的是关键路径，多 spawn 只增加了调度开销。**

Lecture 27: Deep Neural Networks and ML Accelerators

1. 章节标题与概述

Lecture 27: Deep Neural Networks and ML Accelerators

- **本讲核心问题**：现代深度神经网络（DNN）到底在算什么，为什么它既是并行计算最"友好"的负载，又是对硬件能效最"苛刻"的负载？本讲要回答两个层面的问题：(1) **软件层面**——如何把一个卷积层、一行 softmax、一个 attention 块，变换成算术强度（arithmetic intensity）足够高的 GEMM（通用矩阵乘）与融合算子，使得程序从"带宽受限"（bandwidth bound）移动到"计算受限"（compute bound）；(2) **硬件层面**——既然通用处理器（general-purpose processor）执行一条指令的固定开销（取指、译码、发射、寄存器读写）远远超过一次算术运算本身，那么**专业化硬件**（specialization：DSP / FPGA / 域专用加速器 / ASIC）能把每瓦性能提高多少，代价又是什么？两个层面的交汇点是一句话：**GEMM 很便宜，搬数据很贵**（GEMM computation is cheap, but data movement is expensive——它消耗硅面积、瓦特和纳秒）。
- **涉及的主要硬件/软件机制**：
- **硬件侧**：Tensor Core（张量核，单条指令做 4×8 × 8×4 甚至 16×16×16 的矩阵乘加，A/B 用 fp16、累加器用 fp32）、TMA（Tensor Memory Accelerator，异步张量搬运单元）、TMEM/共享内存层次的片上存储、SM（Streaming Multiprocessor）的 warp 调度结构、TPU 的 **systolic array**（脉动阵列，weight-stationary 数据流）、FPGA 的 LUT（查找表）与可重构互连、ASIC/dataflow 架构（PMU/PCU + 片上网络）、以及低精度数值格式（fp16 / bf16 / fp8 / fp4）。
- **软件侧**：implicit GEMM 与 explicit GEMM（是否物化 im2col 矩阵）、多层 blocking / tiling（寄存器–共享内存–L2–DRAM 的层次化分块）、寄存器分块（register blocking）与 SIMD 向量化、"算子融合"（operator fusion）、online/chunked softmax 的代数重构（FlashAttention）、Halide 的"算法 / 调度"分离（algorithm–schedule separation）与自动调度搜索、以及 Tile 级编程系统（CUTLASS、Triton、Thunderkittens、CuTe）。
- **贯穿全讲的分析工具**：Roofline 模型、算术强度、work–span 模型、Amdahl 定律、数据移动的能耗量级（pJ 级）。
- **在并行计算知识体系中的角色**：这是全课程"工具集"的一次总集成。前面学过的 SIMD 与多核分块（第 3–7 讲）、CUDA 执行与存储层次（第 5 讲）、性能优化与 Roofline（第 7–8 讲）、通信与带宽（第 14 讲）在这里全部被同一个负载——矩阵乘——同时调用；而本讲新引入的**第四条设计轴**是"能效"（energy efficiency）：`Power = (Ops/second) × (Joules/Op)`，当性能提升到一定程度后，约束不再是"快不快"，而是"每焦耳能做多少运算"。它也解释了为什么今天最贵的计算机是"为矩阵乘而生的"：AI 的算力需求指数增长，而数据中心在能耗与散热上被死死约束。
- **配套材料**：
- **Fall 2026 课程主页与日程表**：https://www.cs.cmu.edu/~418/、https://www.cs.cmu.edu/~418/schedule.html —— **已公开**（无需登录）。日程表 Oct 27 一行为 "Deep neural networks"，Oct 29 一行为 "ML accelerators at Amazon（guest lecture by Randy Huang and Ron Diamant）"。
- **本讲的 Fall 2026 讲义 PDF**：在 https://www.cs.cmu.edu/~418/lectures/ 之下**尚未发布**（该目录当前公开的 PDF 覆盖 Why Parallelism、ILP、Basic Architecture、Programming Models、CUDA、Parallel Programming Basics、Performance Optimization I/II、Cache Coherence、Directory Coherence、Snoop Implementation、Virtual Memory、Interconnects、Consistency、Synchronization、Lock-

Free、Heterogeneity、Specialization，以及 `24-parallel_deep_learning_data_parallel.pdf`、`25-parallel_deep_learning_model_pipeline_parallel.pdf`，不含本讲的 DNN / 加速器讲义）。

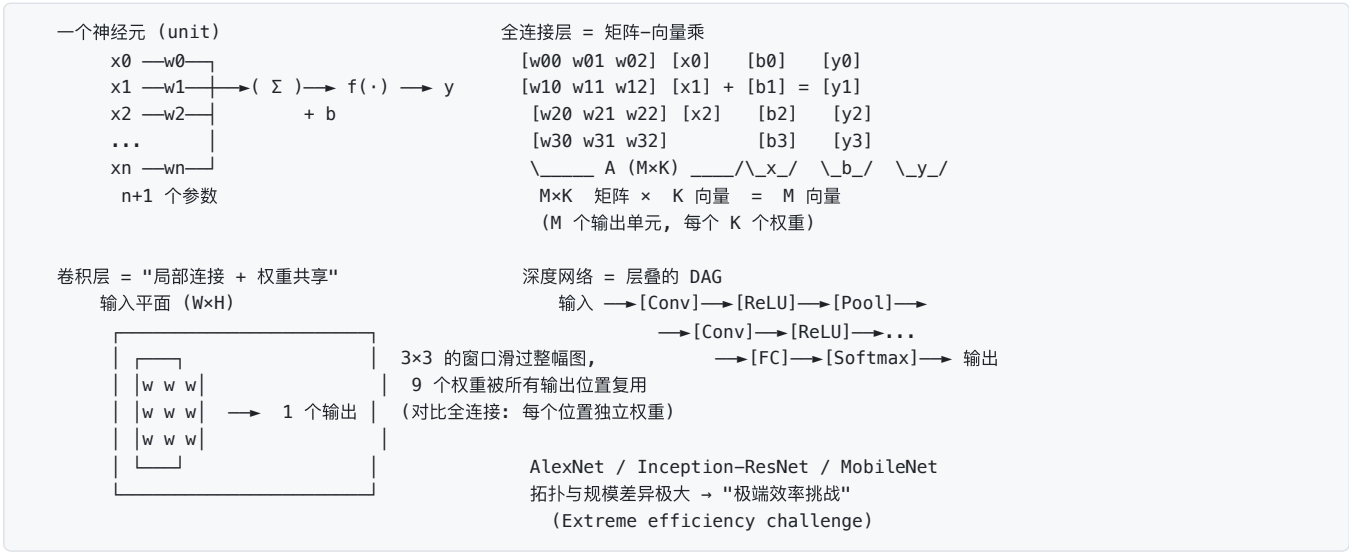
- **日程表中引用的讲义**：Oct 27 一行的 slides 链接指向 `https://www.cs.cmu.edu/afs/cs/academic/class/15418-f22/public/lectures/25_dnn.pdf`，位于 `/afs/cs/academic/class/15418-f22/public/` 之下，访问需要 CMU 登录 —— 属**未公开**。该行同时给出一个标注为 "lecture 24 video" 的 YouTube 链接（历史学期录像）；Fall 2026 日程表中多数 Panopto 录像链接被 HTML 注释隐藏，属**未发布**。
- **Oct 29 客座讲（ML accelerators at Amazon）**：日程表该行**没有任何 slides 链接**，讲者也不提供公开讲义 —— 属**未公开**。本笔记的硬件加速器部分因此完全依据下述公开材料与公开成熟知识撰写。
- **需登录的资源**：Ed 讨论区、Autolab、Canvas/Gradescope 均需 CMU 账号，属**需登录未公开**。
- **本笔记的事实基础（本地已读取的公开讲义抽取文本）**：`cs149_supp/dnninference.txt`（75 页，页眉标注 "Stanford CS149, Fall 2025 — Lecture 9: Efficiently Evaluating DNNs"）、`cs149_supp/accelerators.txt`（71 页，"Lecture 10: Hardware Specialization"）、`cs149_supp/aiperfoptimization.txt`（55 页，"Lecture 13: Domain-Specific Programming Systems and Automatic Performance Optimization"）。这三份是 **Stanford CS149 的公开课程讲义**逐页抽取文本，**不是 CMU 15-418 的讲义**，但覆盖了与本讲完全相同的主题（DNN 计算特征、GEMM/融合优化、硬件专业化、DSL 与自动调优），因此本笔记的全部术语、公式、示例与结论均以这三份文本为准；其中未涉及的部分（如具体芯片型号的公开规格、能耗 ballpark 数据的物理含义）依据公开成熟知识补充，并已尽量与讲义中的说法对齐。
- **Fall 2026 授课教师**：Brian Railing 与 Dimitrios Skarlatos；课程由 Kayvon Fatahalian 创建。讲义 PDF 首页可能写有历史学期字样（如 Fall 2025），属正常的讲义沿用。

---

2. 核心概念与硬件/软件架构图解

2.1 DNN 的计算本质：它只是一个电路，而且几乎全是矩阵乘

- **定义与目的**：一个神经网络就是一个**有向无环的计算图（circuit）**。基本单元是"神经元（unit / neuron）"：给定 n 个输入 $x_0..x_{n-1}$ 与 n+1 个参数（n 个权重 w_i 加 1 个偏置 b），计算 $f(\sum_i x_i w_i + b)$，其中 f 是非线性激活函数（例如 ReLU：$f(x) = \max(0, x)$，或 sigmoid：$f(x) = 1/(1+e^{-x})$）。把许多这样的单元按层连起来就是 DNN。本讲的第一个目的，是让"神经网络"褪去神秘感：**把它当成电路看，它的性能问题就完全变成数据复用问题**。
- **直观解释（"它是什么？"）**：把 DNN 想象成**一条巨大的流水线工厂，每一道工序都是一台"称重机"**：每个神经元手上有 n 个砝码（权重），它把送进来的 n 种原料（激活值）按砝码加权求和，再决定是否放行（ReLU 相当于"负值一律当 0 处理"）。全连接层（fully connected layer）意味着"每台称重机都从前面所有工序取料"；卷积层（convolutional layer）意味着"每台称重机只看前面工序的一小块区域，而且**同一层里所有位置共用同一套砝码**"。这个"权重共享"（weight sharing）是卷积层节省参数量的关键，也是它能变成矩阵乘的关键。
- **架构/机制图解（软件模型：从神经元到层）**：

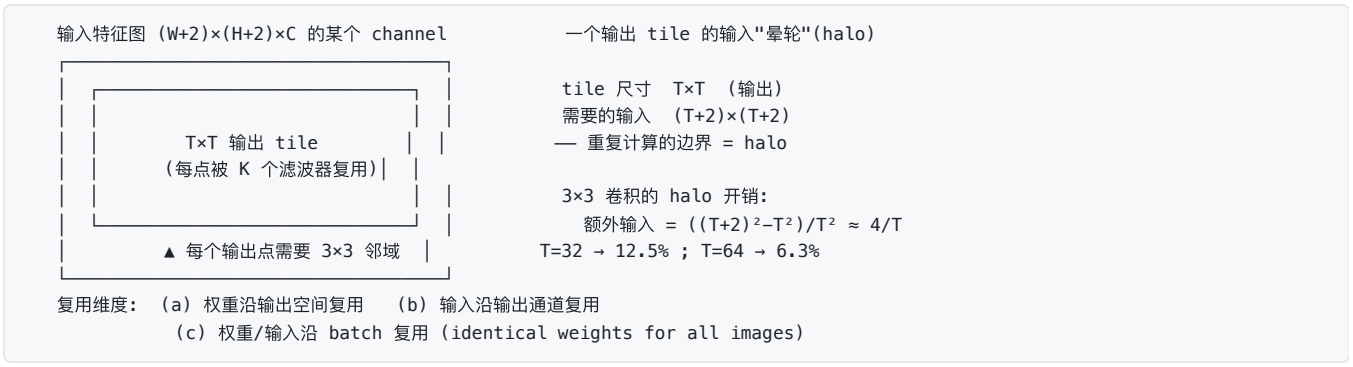


- **关键操作与性能特征**：这三类层最终都归约为**同一个 kernel**——稠密矩阵乘（dense GEMM）：
- **全连接层** = 矩阵 × 矩阵（把 batch 里的样本堆成矩阵的列）。
- **卷积层** = 隐式的矩阵乘（implicit GEMM，见 2.3）。

- **Transformer 的 attention 块** = 两次矩阵乘（$S = QK^T$ 与 $O = PV$）。 因此讲义把 GEMM 称为"现代 AI 的 kernel"（The kernel for: fully-connected layers, convolutional layers, the attention block of a transformer）。性能特征由算术强度决定：如果每次算术运算需要从片上存储之外搬入大量数据，程序必然带宽受限；而矩阵乘的算术强度随分块尺寸线性增长（$OI \propto n$，OI = operational intensity / 算术强度），这正是它能吃掉海量算力的原因。

2.2 直接卷积实现：七重循环与"巨大但未被利用的数据复用"

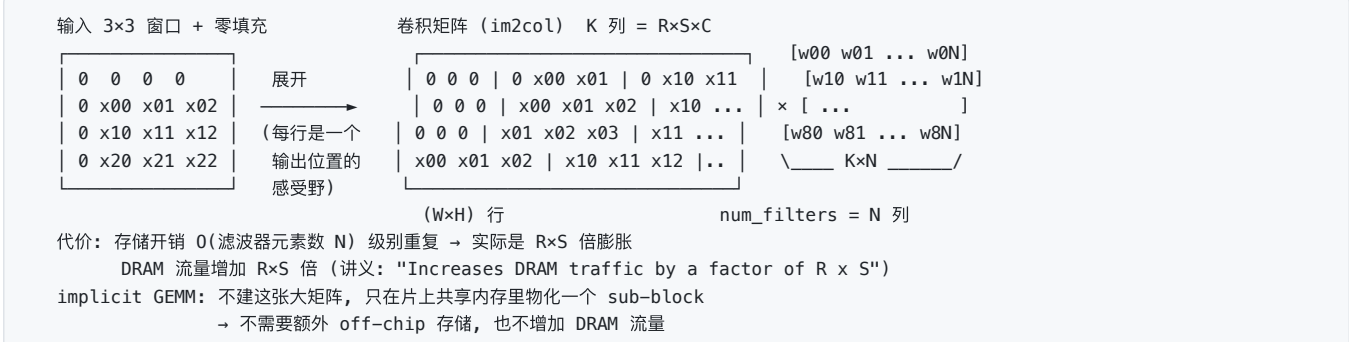
- **定义与目的**：最直接的卷积实现是一个七重循环嵌套（batch × 输出行 × 输出列 × 输出通道 × 输入通道 × 卷积核 Y × 卷积核 X）。它的目的是**暴露数据复用机会**：同一个滤波器权重在一次卷积内被反复使用（沿输出空间滑动），同一个输入激活值被不同滤波器反复使用（沿输出通道遍历）。
- **直观解释（"它是什么？"）**：把它想象成**在 64 张照片上用 64 副不同的"眼镜"看**。direct 实现就是"对每个输出像素、每副眼镜，都把 3×3 邻域重新摸一遍"。数据其实就在手边（同一个输入像素会被 64 副眼镜各看一次），但**如果循环顺序排错了，这 64 次都会变成从 DRAM 重新取一次**。所以直接写的卷积通常是"算术上正确、带宽上灾难"。
- **架构/机制图解（硬件侧：卷积的 halo/tile 数据复用）**：



- **关键操作与性能特征**：direct 卷积每次输出要读 $C \times R \times S$ 个输入元素与权重，做 $C \times R \times S$ 次乘加，**算术强度约为 2 FLOP / 每次 load**（若完全无缓存复用）。把它变成计算受限，必须人为构造复用：**循环分块（blocking）**——把输出 tile 的输入（含 halo）留在寄存器/共享内存里，然后让 K 个滤波器反复访问它。性能特征上，halo 开销随 tile 增大而下降（$O(1/T)$），但 tile 越大寄存器压力越大，超过寄存器容量会 spill，反而更慢。

2.3 explicit GEMM（im2col）与 implicit GEMM：物化与否的代价

- **定义与目的**：既然卷积和 GEMM 是同一件事，最省事的做法就是**把卷积变成一次真正的矩阵乘**：把输入特征图的每个 $R \times S \times C$ 邻域"拍平"成一个行向量，拼成"卷积矩阵"（im2col / explicit GEMM）。这样可以直接调用高度优化的 GEMM 库。**implicit GEMM** 则相反：不物化这个矩阵，而是在 GEMM 的分块内部**按需索引**激活张量，只把当前需要的 sub-block 搬到片上共享内存，再用 well-tuned 的 shared-memory GEMM 做子块乘法（讲义点名 CUTLASS）。
- **直观解释（"它是什么？"）**：im2col 像**把一叠照片的每个 3×3 小窗口都复印一份、装订成一本巨大的相册**——好处是"查相册"非常简单（标准矩阵乘），坏处是相册比原照片厚 9 倍，而且复印本身要占仓库（DRAM）。implicit GEMM 是"不印相册，现场翻照片"，用手指按坐标去找——代码复杂，但省掉了整本相册的存储与搬运。
- **架构/机制图解（软件模型：im2col 展开）**：

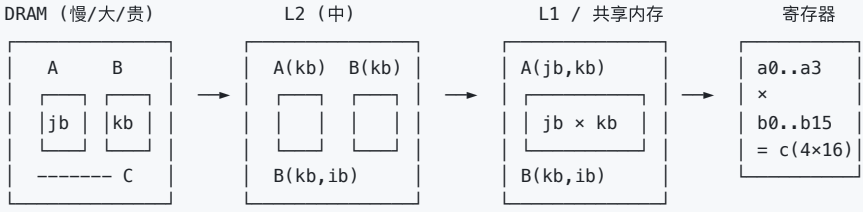


- **关键操作与性能特征**：explicit GEMM 的**计算**是最优的（可以吃到 GEMM 库的全部优化），但它的**数据移动**被放大了 $R \times S$ 倍（3×3 卷积就是 9 倍），且需要一块可能与激活张量同量级的临时存储。implicit GEMM 把这次放大消掉，代价是索引逻辑复杂、边界处理（padding）要写

进 kernel 内部。CUTLASS 正是为这种"自定义 DNN 层的构建基元"而生：in-shared-memory 的快速 GEMM、warp 级 GEMM、用于快速块加载/张量索引的 iterator、tensor reduction 等。

2.4 GEMM 的层次化分块：把算术强度从 O(1) 抬到 O(n)

- **定义与目的**：blocking（分块 / tiling）是把矩阵切成能装进各级缓存的小块，在块内完成复用，从而**提高算术强度**。单层 blocking 只能利用一级缓存；**层次化 blocking** 让每一级存储层次（寄存器 → L1 → L2 → DRAM）都做一次分块，逐级提高"每字节 DRAM 流量对应的浮点运算数"。
- **直观解释（"它是什么？"）**：把 GEMM 想成**做菜**。DRAM 是超市（一次采购很贵、很慢），L2 是冰箱，L1 是案板，寄存器是手里的刀。笨做法是"每切一刀就跑一趟超市"；分块就是"一次买回一整块案板分量的原料（L2 块），在案板上再分成小份（L1 块），手里同时拿 4 行 × 16 列的原料（寄存器块）反复下刀"。块越大，去超市的次数越少——所以讲义的自检问题是："你想要尽可能大的 BLOCKSIZE 吗？为什么？"答案是：**在能被该级存储装下的前提下越大越好，但超过容量就会失效（thrash）**。
- **架构/机制图解（软件模型：三级分块 + 寄存器微内核）**：



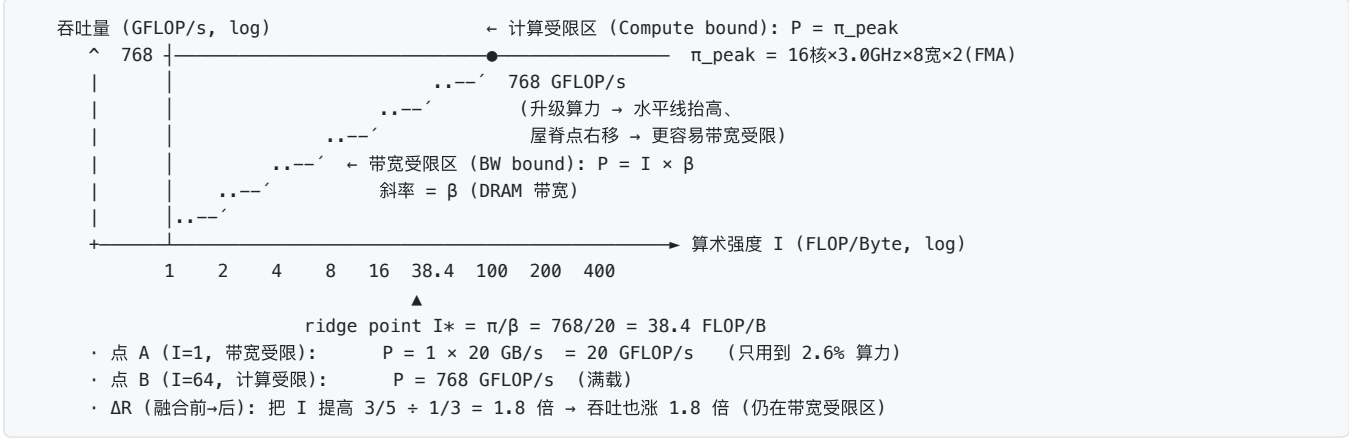
C 的一个块　　每级都用"三次读、n^3 次算"把 OI 放大

```
OI(块大小 b, 三级块都在缓存里) = 2b³ FLOP / 3b² 元素 = 2b/3 FLOP/元素 = b/6 FLOP/Byte (fp32)
→ b=8(寄存器):  1.3   ;  b=64(L1):  10.7  ;  b=256(L2): 42.7  [FLOP/Byte]

三种把 SIMD 塞进分块的方式（讲义给出三种）:
(1) 向量化 i 循环: C_accum = vec_load(C); 循环 k 做 splat(A)·vec_load(B) → 好: 改善 B 的空间局部性
                                                                  坏: 工作集 ×SIMD_WIDTH, B 仍跨大步长
(2) i 维太小时: 预转置 B 的一个块到 BT, 向量化最内层 k（点积）
(3) 双方都预转置（A→AT, C→CT), 用 SIMD_WIDTH × SIMD_WIDTH 的累加器数组, 最内层 kk 无依赖
```

- **关键操作与性能特征**：单层 blocking 后，DRAM 流量降为 $O(n^3/b)$ 而计算仍是 $O(n^3)$，算术强度 $OI = 2b/3$ FLOP/元素。层次化 blocking 让这个 b 逐级变大（寄存器级只关心微内核的复用，L1/L2 级关心更大的块），从而让 DRAM 侧的有效算术强度逼近甚至超过 Roofline 的 ridge point。代价：**代码复杂度爆炸**——讲义专门提醒"不同层可能需要不同的调度策略（矩阵维度不同）"，例如 MobileNet 的 body 里既有 `3×3×3×32`、`112×112×32` 这样的"通道少、空间大"的层，也有 `1×1×1024×1024`、`7×7×1024` 这样的"通道多、空间小"的层，二者对分块尺寸、向量化维度、转置策略的要求正好相反（讲义的原话是："Ug for library implementers!"）。

2.5 Roofline 与算术强度：DNN 优化的全部"软件侧"答案

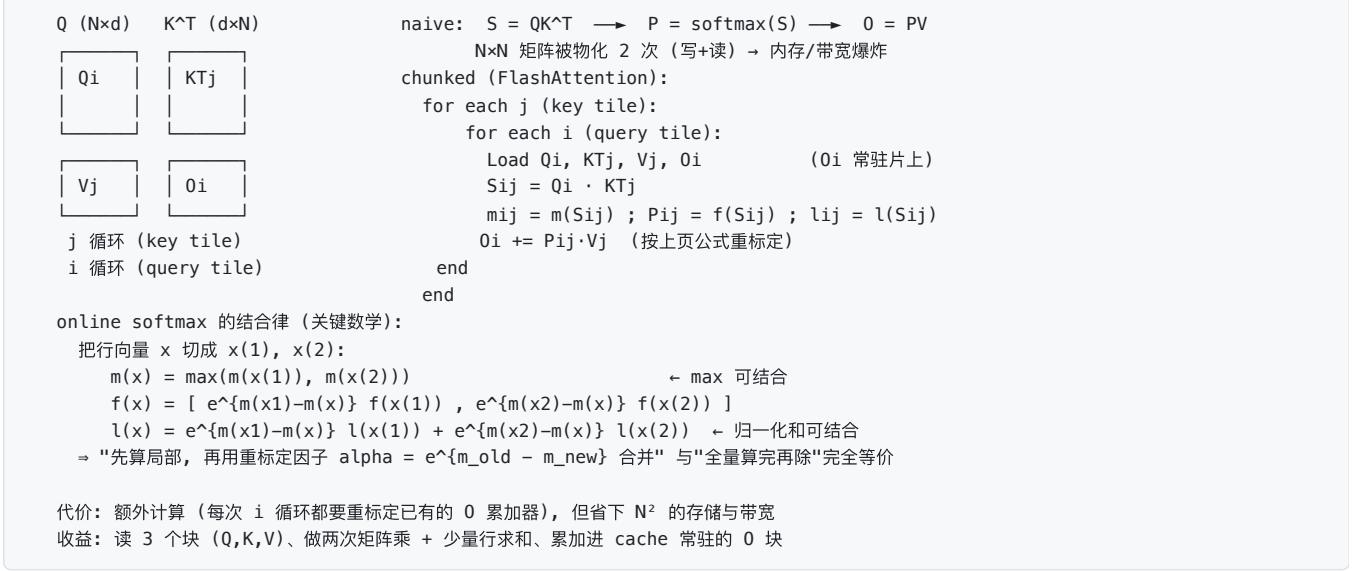
- **定义与目的**：Roofline 模型用一条折线回答"这个程序最快能多快"：$P = \min(\pi_{peak},\ I \times \beta)$，其中 π_{peak} 是机器的峰值算力（ops/sec），β 是可用的通信/内存带宽（bytes/sec），I 是程序的算术强度（ops/byte）。交点 $I^* = \pi_{peak}/\beta$ 称为 **ridge point（屋脊点）**：$I < I^*$ 时程序带宽受限，$I > I^*$ 时计算受限。
- **直观解释（"它是什么？"）**：把机器想成**一条生产线（算力）配一条进料传送带（带宽）**。传送带每分钟能送 20 公斤原料，生产线每分钟能加工 768 份。如果 1 公斤原料能做 1 份产品（$I = 1$），生产线永远吃不饱（带宽受限）；如果 1 公斤原料能做 100 份产品（$I = 100$），传送带轻松喂饱，瓶颈在生产线上（计算受限）。**提高算力（换更快的机器）只会让程序更容易掉进带宽受限区；提高算术强度（改程序：分块、融合）才能让它回到计算受限区**——这是讲义用整整一页强调的结论。
- **架构/机制图解（软件模型：Roofline 曲线与"更强算力把屋脊点推向右"）**：



- **关键操作与性能特征**：Roofline 给出了 DNN 优化的完整"操作清单"：
- **提高 I**：循环融合（loop fusion）、分块（blocking）、算子融合（fusion）——把中间结果留在片上。讲义的经典例子：`E = D + (A+B)*C` 分成三个循环写，算术强度是 $\mathbf{1/3}$；融合成一个循环后是 $\mathbf{3/5}$（每次遍历从 2 load + 1 store / 1 op，变成 4 load + 1 store / 3 op）。
- **抬高 β**：用更高带宽的存储（HBM、更宽的接口）、更好的访问模式（合并访问、避免 cache line 浪费）。
- **降低 π_{peak} 的需求**：用更低精度（fp16/bf16/fp8）在同等硅面积下做更多运算——或者更激进，用专用硬件（第 2.8–2.12 节）。
- 性能特征的关键提醒：**重叠通信与计算需要额外的片上缓冲（double buffering）**，而片上存储的面积本来可以用于计算——讲义把它列为"其余知识"的第一条：数据移动消耗能量、片上存储与计算争抢芯片资源，所以**缓冲区要尽可能小**。

2.6 算子融合（fusion）与 FlashAttention：不改数学，只改数据流

- **定义与目的**：融合是把相邻算子的循环体合并，使中间结果**在寄存器/共享内存中被消费掉**，而不是写回 DRAM 再读回。讲义给出三个由浅入深的例子：(1) Conv + Scale/Bias + MaxPool 融合；(2) 逐行 softmax 的融合；(3) Transformer attention 的 online（chunked）softmax 融合，即 FlashAttention 的核心思想。
- **直观解释（"它是什么？"）**：想象**搬家**。不融合 = 每搬一样东西都要装箱、运到仓库、再取回来；融合 = 东西一手交一手，全程不进仓库。softmax 的例子特别直白：naive 实现要读矩阵 5MN+2M 个元素、写 3MN+2M 个元素（因为 max、exp、sum、除法各走一遍）；融合后"读一行 → 在片上算完这一行 → 写一行"，只需读 MN、写 MN。attention 的例子更极端：naive 实现要物化 $\boldsymbol{N \times N}$ 的分数矩阵 $\boldsymbol{S = QK^T}$（N 可以是几千，$\boldsymbol{N^2}$ 空间就爆了），而 chunked 版本**永远不物化这个矩阵**。
- **架构/机制图解（软件模型：FlashAttention 的分块循环嵌套）**：



- **关键操作与性能特征**：融合把"多次 DRAM 往返"压成"一次读 + 一次写"，代价是**额外计算**与**片上缓冲**。讲义明确点出这个 trade-off："Note there is additional computation vs. the original version (must re-scale prior values of O each step of the i-loop)"。数值上重标定的额外乘法是每行每 tile 一次 $\boldsymbol{d}$ 维缩放（约 $\boldsymbol{N/B_J \times d}$ 次），相对主计算 $\boldsymbol{2Nd}$ 的比例约 $\boldsymbol{1/(2B_J)}$——当 key tile $\boldsymbol{B_J = 64}$ 时不到 1%，非常划算。另外，融合的历史演化是：早期由库作者手写几个固定的 fused op（如 TensorFlow），后来由**编译器**自动生成融合实现（cuDNN backend、`torch.compile`），目标是"在一个节点里高效执行而不通过内存传递中间结果"。

2.7 低精度数值格式：用"更小的字节"换"更高的算术强度"

- **定义与目的**：DNN 对精度的容忍度远高于科学计算。把权重与激活从 fp32 降到 fp16 / bf16（16 bit）、fp8、甚至 4 bit 或 1 bit，可以在**不改变数据布局**的前提下把同一份数据的字节数减半甚至更多，从而直接提高算术强度、降低带宽需求与能耗，并让同样面积的乘法器做更多运算。
- **直观解释（"它是什么？"）**：把 fp32 想成"用 32 位写一个数字"，bf16 是"用 16 位写，但保留和 fp32 一样的指数范围（只是尾数更短）"，fp8 是"只有 8 位的草稿纸"。就像**快递箱**：fp32 是一个大箱子装一件小商品（浪费空间），bf16 是刚好合身的箱子，fp8 是信封——前提是商品（数值）不需要那么精细的保护。
- **架构/机制图解（数值格式的位分配）**：

```
fp32   S|EEEEEEEE|MMMMMMMMMMMMMMMMMMMMMMM     1 + 8 + 23 位
       −1^S × (1 + M×2^−23) × 2^(E−127)

bf16   S|EEEEEEEE|MMMMMMM                     1 + 8 + 7 位
       ← 指数与 fp32 相同 ⇒ 动态范围相同; 尾数只有 7 位 ⇒ 精度下降

fp8 E4M3  S|EEEE|MMM      1+4+3     范围 0 ~ 448
fp8 E5M2  S|EEEEE|MM      1+5+2     范围 0 ~ 57344
          ← E5M2 范围更大、精度更差; E4M3 反之。训练中常"前向用 E4M3、反向用 E5M2"

Tensor Core 的混合精度约定: A, B 存为 fp16 → 乘法用 fp16 → 累加用 fp32
⇒ 用"低精度存储 + 高精度累加"避免误差累积
```

- **关键操作与性能特征**：
- **表 1：低精度带来的收益与代价**

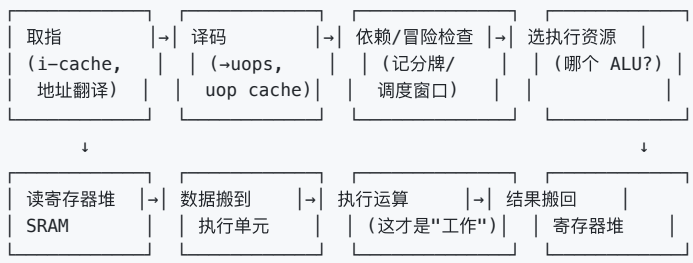
格式	每元素字节	相对 fp32 的算术强度	主要风险	典型用途
fp32	4	1×	无	训练的主权重/累加、科学计算
tf32	4	1×	尾数 10 位	Tensor Core 上的 fp32 训练折中
fp16	2	2×	范围小（易溢出，需 loss scaling）	推理与训练激活
bf16	2	2×	精度低 7 位尾数	训练（范围与 fp32 同）
fp8 (E4M3/E5M2)	1	4×	精度/范围都紧	大模型训练/推理（Transformer Engine）
int8	1	4×	需要量化标定（scale/zero-point）	推理
fp4 / 1-bit	0.5 / 0.125	8× / 32×	需要专用算法与格式	研究前沿（讲义原话：1-bit 是 "In the extreme case: 1-bit ;-)"）

- 关键约束：**累加必须在更高精度**（Tensor Core 就是 A,B = fp16、D = fp32），否则长归约链的舍入误差会吞掉结果；低精度同时降低了**带宽**与**能耗**（搬 2 字节比搬 4 字节便宜），这是它比"仅提高峰值算力"更有价值的地方。

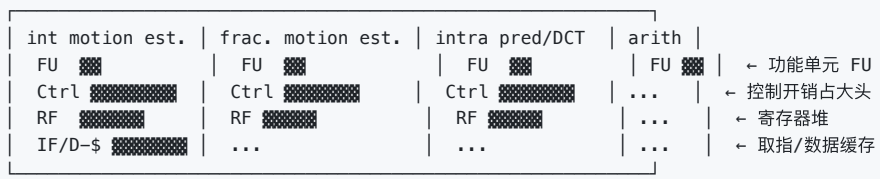
2.8 硬件侧的问题：为什么"通用处理器"如此低效?

- **定义与目的**：本讲从软件转向硬件时，第一步是承认一个反直觉的事实：**整个课程都在教如何高效使用多核 CPU/GPU，但这两个平台相对于专用硬件仍然是"低效"的。**原因是通用处理器为"通用性"付出了巨大代价：执行每一条指令都要经过取指、译码、检查依赖/流水线冒险、选择执行单元、读寄存器堆、把数据搬到执行单元、运算、再搬回寄存器堆、写回 SRAM 等一长串固定开销；这些开销在能耗上甚至**超过运算本身**。
- **直观解释（"它是什么？"）**：通用 CPU 像**一个什么菜都会做的大厨**：他每做一道菜都要先"读菜谱（取指）、理解菜谱（译码）、去仓库取料（load）、洗手（流水线控制）"。如果一天只做三道大菜（复杂、数据密集），这些准备成本可以忽略；但如果要做一百万份小菜（SIMD 位运算、视频编码里的整数运算），大厨 90% 的时间花在翻菜谱上。专业硬件（ASIC）就像**一台只会切片的机器**：它不会别的，但每秒切一万片，而且几乎不耗电在"理解要做什么"上。
- **架构/机制图解（硬件侧：一条指令的固定开销 vs 一次运算）**：

执行一条指令的最简路径（讲义逐条列出）：



H.264 视频编码的能耗分解（即使已用 SIMD 指令实现）：

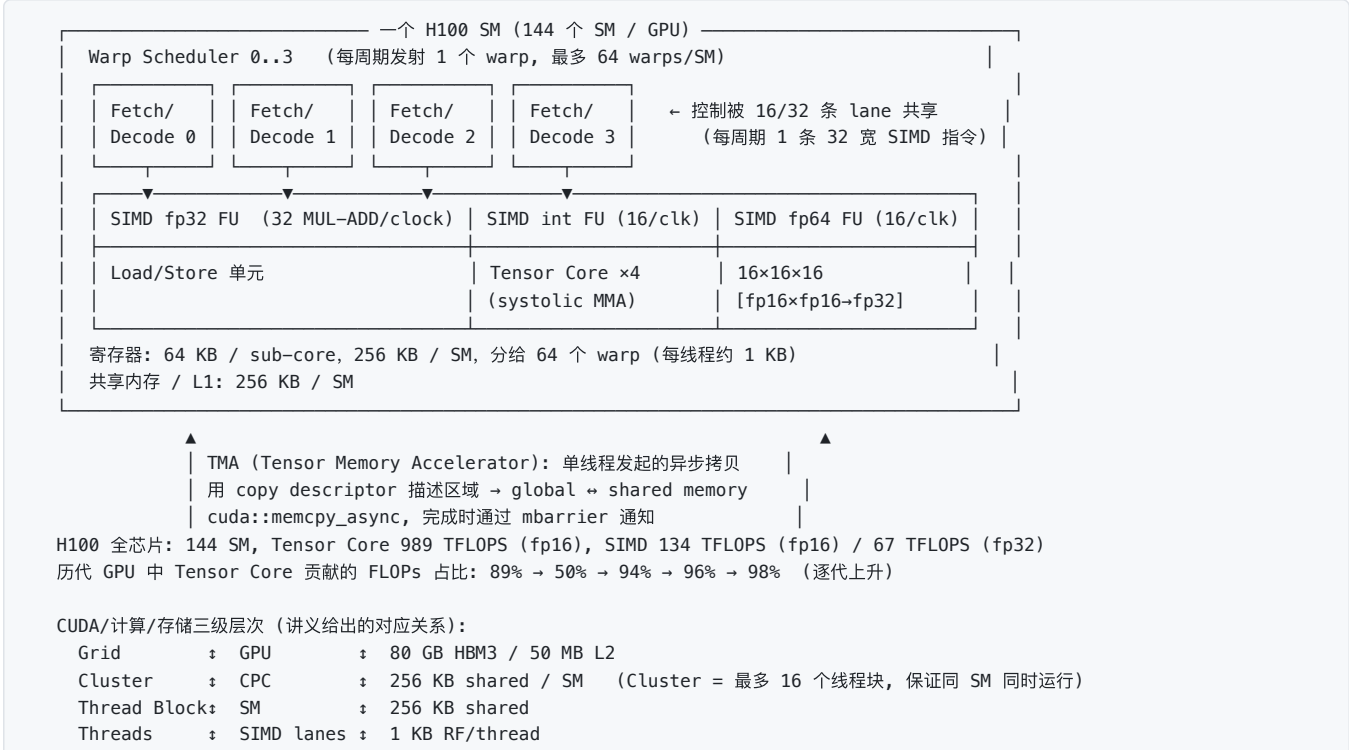


FU = functional units, RF = register fetch, Ctrl = 流水线控制，
Pip = 流水线寄存器，IF = 取指+i-cache，D-$ = 数据缓存
⇒ 功能单元（真正干活的部分）只占能耗的一小部分

- **关键操作与性能特征**：两条由此推出的设计原则（贯穿后续所有硬件加速器）：
- **摊薄指令流开销**（amortize cost of instruction stream processing）：用"更复杂的指令"让一次取指/译码服务更多运算。讲义给出量化对比（相对简单指令的**可编程性开销**）：半精度 FMA 为 **2000%**、半精度 DP4（vec4 点积）为 **500%**、半精度 4×4 MMA（矩阵乘加）为 **27%**——也就是说，用一条 MMA 指令代替 16 次 FMA，指令流开销被摊到几乎可以忽略。
- **DSP 与 VLIW** 的思路：Qualcomm Hexagon DSP 是"可编程但控制路径更简单、指令更复杂（SIMD/VLIW）"的处理器，单条 VLIW 指令同时指定多个不同操作（与 SIMD 的"同一操作多个数据"不同），在其 FFT 最内层循环中可以每周期执行 29 个"RISC 操作"。

2.9 GPU 的应对：Tensor Core、TMA 与"算力全在张量核里"

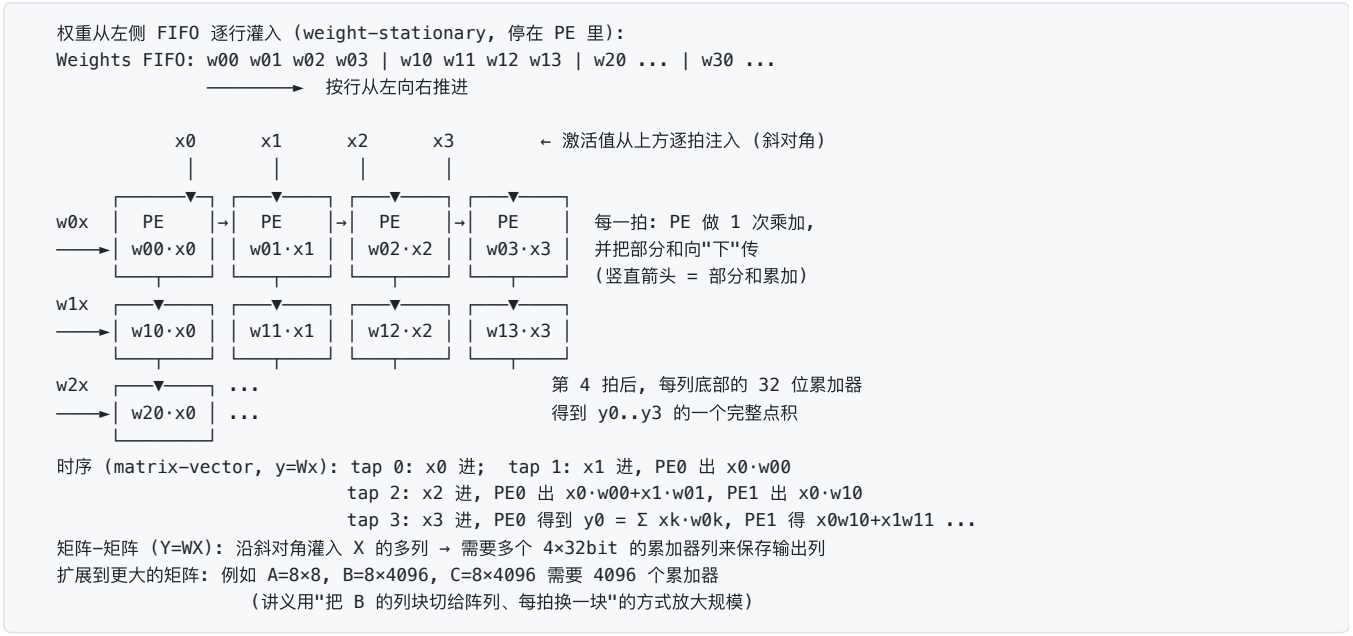
- **定义与目的**：GPU 的做法是把"通用性开销"摊到极致：用大量 SIMD 通道共享控制，再把矩阵乘加做成**专用指令**（Tensor Core MMA）。A100 的一个 SM 有 64 个 fp32 ALU、32 个 int32 ALU、4 个 tensor core；每个 tensor core 执行 **8×4 × 4×8** 的矩阵乘加（$\boldsymbol{A} \times \boldsymbol{B} + \boldsymbol{D}$，A/B 为 fp16，D 为 fp32）；GA100 有 108 个 SM，即 6912 个 fp32 ALU 与 432 个 tensor core，1.4 GHz 下 fp32 为 **19.5 TFLOPS**，而 fp16/32 混合的 tensor core 峰值是 **312 TFLOPS**——相差 16 倍。
- **直观解释（"它是什么？"）**：Tensor Core 像**一台"打包好的"乘法车间**：普通 SIMD 是"一组工人各拧一颗螺丝"，MMA 是"一台机器一次压出一整块 8×8 的成品"。代价是这台机器**只吃特定形状的原料**（16×16×16 的 tile、特定数据类型），所以程序员必须把数据摆放成它喜欢的布局——这就是 tile 级编程模型的由来。
- **架构/机制图解（硬件侧：H100 的 SM 与存储层次）**：



- **关键操作与性能特征**：
- **异步性**是让这些单元吃满的关键：异步计算（`mma_async`）、异步内存访问（TMA + TMEM）、异步片间通信（overlap compute / memory / communication）。讲义的"理想 AI 加速器"特性表逐条对照 NVIDIA GPU 是否满足：tiled tensors ✅（16×16/32×32 tile 换最高 GEMM TFLOPS 与低指令开销）、asynchronous compute ✅、asynchronous memory access ✅（TMA+TMEM）、asynchronous chip-to-chip ❓、compute-unit-to-compute-unit comm ❓（靠 TB Cluster 部分满足）。
- **新一代的编程难度**：B100 的 tensor core 数据放在 SMEM 与 TMEM 中，**由单个线程执行 MMA（不再需要 warp！）**，编程接口变成 `tcgen05.alloc` / `tcgen05.mma` / `tcgen05.commit` / `tcgen05.fence` + `cp.async.bulk.tensor` + mbarrier 协调——讲义的原话是 "Not your father's CUDA"。这正是 DSL（Mosaic GPU、CuTe-DSL（Python 里的 CUTLASS））存在的理由。
- **填满机器需要"很多并行工作"**：讲义用卷积的输出规模说明这一点——$N = 1, P = Q = 64$ 时只有 524K 个输出（2 MB），而 $N = 32, P = Q = 256$ 时有 256M 个输出（1 GB）。小 batch 的推理根本喂不饱 80 个 SM。

2.10 TPU 与 Systolic Array：数据驱动而非指令驱动

- **定义与目的**：Google 的 TPU（Jouppi et al. 2017）是"域专用加速器"的样板：芯片面积中**算术单元约占 30%**，控制部分面积极小——因为它的指令集只有寥寥几条：读主机内存、写主机内存、读权重、matrix_multiply/convolve、activate。核心是一个 **systolic array（脉动阵列）**：权重从左侧 FIFO 逐列灌入并**驻留在 PE（processing element）中**（weight-stationary），激活值从上方沿对角线以"波前（wavefront）"方式推进，每个 PE 做一次乘加并把部分和传给下方邻居。
- **直观解释（"它是什么？"）**：把 systolic array 想象成**一排传菜的服务员**（这也是 "systolic" 一词的来源——像心脏搏动一样有节奏地泵数据）。数据不像冯·诺依曼机器那样"存到仓库、用时再取"，而是**像水流一样从邻居手里接过来、用一下、再传给下一个邻居**。每份数据一旦进入阵列，就会被沿途所有 PE 使用；只有阵列的边界需要与外界通信。因此**通信是局部的（只与邻居 PE）**，控制是**分布式的（每个 PE 只需知道自己的节奏）**。
- **架构/机制图解（硬件侧：矩阵–向量乘 y = Wx 的波前推进）**：



- **表 2：SIMD 与 Systolic Array 的对比（讲义原表）**

特性	SIMD	Systolic Array
数据流（Dataflow）	控制驱动（control-driven，靠指令）	数据驱动（data-driven，靠波前 wavefront）
局部性（数据复用）	有限（Limited）	时间局部性 + 空间局部性（temporal and spatial）
通信	全局（寄存器/内存）	局部（相邻 PE 之间）
控制	集中式（Centralized）	分布式（Distributed）
效率（perf/mm^2, perf/Watt）	中（Medium）	非常高（Very high）

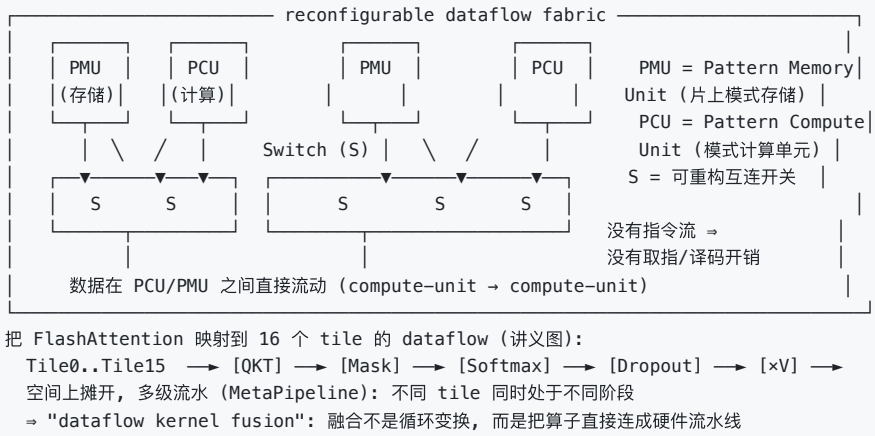
- **关键操作与性能特征**：systolic array 的代价是**灵活性**——它只擅长"形状整齐的稠密矩阵乘"，矩阵维度必须被 padding 到阵列尺寸的整数倍，否则 PE 空转；它也无法表达任意控制流。它带来的收益是能效：Jouppi 等人的 TPU 论文给出的性能/瓦特数据（讲义引用其图）显示专用加速器相对通用处理器的巨大优势。讲义还用 **"硬件的彩票"（Hardware Lottery，Sara Hooker）** 提醒我们："当一个研究想法胜出，是因为它契合当时可用的软件与硬件，而不是因为它普遍优于其他方向"——TPU 稠密矩阵乘的算术强度随 n 线性增长（$OI \propto n$），于是 Transformer 模型胜出，于是硬件进一步为矩阵乘特化，形成正反馈。

2.11 FPGA、ASIC 与 Dataflow 架构：从"可编程"到"可重构"

- **定义与目的**：在通用处理器与全定制 ASIC 之间，有两类中间形态：
- **FPGA（Field Programmable Gate Array）**：芯片提供逻辑块阵列（可编程 LUT + 触发器）与可编程互连，逻辑由"配置"直接实现。讲义例子：Xilinx Virtex-7 的 **LUT6** 可理解为一个 64 项查找表（6 输入 1 输出），因此一个 6 输入 AND 只需 1 个 LUT6；40 输入 AND 可用 8 个 LUT6 链接而成，延迟为 3。现代 FPGA 还含大量**硬核**：SRAM 存储块、DSP 乘法块、ARM/RISC-V CPU 核，用硬件描述语言（Verilog）编程；Amazon EC2 的 F1/F2 实例提供云端 FPGA。
- **ASIC / 域专用加速器**：为某一领域定制的电路。讲义用两个经典案例量化收益：FFT 上 **ASIC 用约 1/1000 的芯片面积达到与一个 CPU 核相同的性能**，而 **GPU 核相对 CPU 核的面积效率只有约 5–7 倍**；ASIC 还能用约 **1/100 的功耗**达到一个 CPU 核的性能。另一个案例是 DE Shaw 的 **Anton** 分子动力学超级计算机（2008，机器内含 512 个粒子相互作用 ASIC + 面向 FFT 的吞吐型子系统 + 为 N-body 通信模式定制的低延迟网络），**Anton 3（2025）比同期 GPU 快约 20 倍**。
- **可重构 Dataflow 架构**：如 Plasticine（Prabhakar、Zhang 等，ISCA 2017），由 **PCU（Pattern Compute Unit）** 与 **PMU（Pattern Memory Unit）** 通过片上 **Switch** 网络连接；把 AI 模型的 dataflow graph（GEMM 1 → Pool → GEMM 2 → SoftMax，外加 map/filter/reduce 等并行模式）**空间映射**到芯片上。它"没有指令，因此没有取指/译码开销"，并且是"极端异步的：不存在顺序指令执行"，从而同时具备异步计算、异步访存、异步片间通信、计算单元间直接通信（融合与流水）四种能力——正好满足"理想 AI 加速器"表格的全部条目。
- **直观解释（"它是什么？"）**：把三种硬件想成三种**交通方式**：CPU 是**万能出租车**（哪都能去，但每趟都要绕路、计价）；FPGA 是**可拆卸拼装的轨道**（你可以按需铺一条专用线路，但铺路本身很费工）；ASIC 是**一次性建好的高铁**（建成后极快极省电，但只为某条线路服务，造价上千万

到上亿美元）。dataflow 架构相当于**"把程序图直接铺成铁路网"**：算子变成车站（PCU），中间结果通过铁轨（Switch）直接开到下一站，不用回总站（DRAM）换乘。

- **架构/机制图解（硬件侧：Plasticine 式 dataflow 与 FlashAttention 的空间映射）**：



- **关键操作与性能特征**：讲义给出的"专业化能效经验法则"（相对于 CPU 上的高质量 C 代码）：
- 吞吐型处理器架构（GPU 核）：约 **10×** perf/watt（前提：代码能映射为宽数据并行且计算受限）。
- 固定功能 ASIC：可达 **100—1000×** 或更高（前提：计算受限，且**不是浮点数学**）。
- 域专用加速器（DSL 可编程，如 Google TPU）：约 **20×**。
- FPGA / 可重构逻辑：约 **50×?** （讲义原话 "jury still out"——尚无定论；难编程，让它变简单是活跃的研究方向）。
- 而 ASIC"不可编程，且设计/验证/流片要花上千万到上亿美元"——这就是"效率 vs 可编程性"曲线（一组从"最易编程"到"最难编程"的连续谱）的核心权衡。

2.12 能量账本：为什么"搬数据"比"算数据"贵

- **定义与目的**：现代系统设计的第一条经验法则："**永远设法减少计算机中的数据移动量**"。原因是数量级差异：一次整数运算约 1 pJ，一次浮点运算约 20 pJ（这还只是"逻辑运算本身"的成本，不含译码、寄存器读写等开销）；而从 1 mm 外的小 SRAM 读 64 bit 约 26 pJ；从低功耗移动 DRAM（LPDDR）读 64 bit 约 **1200 pJ**。也就是说，**从片外 DRAM 读 8 个字节的能量，相当于做 60 次浮点运算**。
- **直观解释（"它是什么？"）**：把运算想成"拧一颗螺丝"（很便宜），把访存想成"开车去五金店取螺丝"（很贵）。芯片里的 SRAM 是"桌上的零件盒"（26 pJ 一趟），DRAM 是"城里的五金店"（1200 pJ 一趟）。所以架构师的全部努力，就是把"去五金店的次数"降到最少——这就是 tiled tensor（tile 化张量）、异步搬运、计算单元间直接通信这三种机制的统一动机。
- **表 3：能效 / 可编程性阶梯（讲义 "Efficiency vs Programability" 图）**

平台	相对能效	可编程性	代价
能效优化 CPU	1×（基准）	最容易	峰值算力低
吞吐型处理器（GPU）	~10×	需要数据并行 + 计算受限	编程模型复杂（CUDA/tile）
可编程 DSP（如 Hexagon，VLIW）	介于两者之间	较易（复杂指令摊薄控制）	领域受限（信号处理）
FPGA / 可重构逻辑	~50×? （尚无定论）	难（Verilog/HLS）	编译时间长、工具链复杂
域专用加速器（TPU，DSL 编程）	~20×	中等（受限于 DSL 表达力）	表达能力受限
固定功能 ASIC	100—1000×+	不可编程	NRE 成本数千万~上亿美元；只对计算受限、非浮点负载有效

- **关键操作与性能特征**：这些数字与本讲的三大机制一一对应：**tiled tensors**（16×16、32×32 的 tile 才能拿到 GEMM 的最高 TFLOPS 与最低指令开销）、**异步执行**（异步计算、异步访存、异步片间通信，三者叠加才能同时重叠 compute / memory / communication）、**片上通信**（计算单元到计算单元的直接通信，实现融合与流水、streaming dataflow）。TPU 的芯片面积分配（算术单元 ~30%、控制面积极小）和"关键指令只有 5 条"正是这一哲学的极端体现。

2.13 编程系统与自动优化：让"性能专家"这件事变得可复制

- **定义与目的**：性能优化需要极高专业度、极其枯燥、且换一台机器就要重做——而公司每年在 AI 算力上花费数千万到数亿美元，所以"自动化性能优化"本身就是巨大的经济动机。讲义给出三条思路：(1) **提高抽象层次**（DSL）；(2) **智能搜索**（autotuning / autoscheduler）；(3) **[新兴]**

利用 LLM 的问题求解与代码生成能力。

- **直观解释（"它是什么？"）**：Halide 的核心是把程序拆成**两份描述**："算法"（算什么，声明式、无循环、无副作用）与"调度"（怎么算：循环顺序、分块、向量化、并行化）。这就像**菜谱与厨房排班分开写**：菜谱只写"3×3 均值模糊"，排班表才写"256×32 分块、内层向量化 8 宽、外层多线程、blurx 在 tile 内按需计算"。
- **表 4：Halide 的"算法 / 调度"分离（同一个算法的三种实现）**

调度写法	产生的 C 等价结构	中间缓冲	性能特征
`blurx.compute_root()`	先整幅算 blurx（读 1024+2 × 1024+2），再整幅算 out	blurx：W × (H+2)，全尺寸	需一次完整 DRAM 往返；朴素
`out.tile(x,y,xi,yi,256,32); blurx.compute_at(out, xi)`	tile 内对每个 xi 只算 3 个 blurx 元素	blurx：1 × 3	复用最省，但重复计算多（每个输出点重算 blurx）
`out.tile(...).vectorize(xi,8).parallel(y); blurx.compute_at(out, x).vectorize(x,8)`	tile 级分配 blurx(258,34)，逐 tile 计算并立即消费	blurx：258 × 34，**装得进缓存**	生产者–消费者局部性最优；8 宽 SIMD + 多线程

- **关键操作与性能特征**：
- Halide 的语言约束是为了让编译器能提供服务：只支持**规则 N 维域上的计算**、只支持**前馈流水线**（外加 reduction 与固定深度递归的特殊支持）、**所有依赖必须可被编译器推断**。有了这些约束，编译器才能机械地把"调度"翻译成 pthread + AVX intrinsics（含边界条件的自动生成，例如 256+2 不能被 8 整除时的收尾处理）。
- 早期学术结果：相机 RAW 处理流水线（原 463 行手写 ARM NEON 汇编）用 Halide 写**代码量少 2.75×、速度快 5%**；bilateral filter（原 122 行 C++）用 **34 行算法 + 6 行调度**达到 **CPU 快 5.9×、GPU 比手写 CUDA 快 2×**。
- **自动调度**：现实是"会写 Halide 的人多，会写 Halide 调度的人极少"（Google 有 80+ 人写 Halide，但被信任写调度的只有极少数）。于是把调度建模为**一串离散选择**（对 DAG 从末尾往前，逐个决定 `compute_at` 的位置与 tile 尺寸），用贪心/beam search 搜索，用一个小 MLP 在"几十微秒"内估计代价（166 秒内评估 140 万个调度，实际输出 27 个系数代入手工代价模型）。结果：**在图像处理 CPU 上，自动调度器与人类最佳调度相当**。
- **LLM 路线**：用"生成 → 执行/剖析 → 反思 → 修改"的循环（剖析给出 SM 利用率 42%、DRAM 利用率 89%、L2 命中 68%、32 ms 等反馈），在 KernelBench（数百个 PyTorch kernel）这样的基准上自动产出正确且快速的 CUDA kernel；关键技巧是**让 LLM 组装高层原语（Triton / CUTLASS-CuTe / TileLang）而不是写底层 CUDA**，从而减少正确性错误与幻觉。进一步的想法包括：用经验微调模型、建立"优秀解法数据库"做检索增强（存储的不只是解法，还有**优化决策序列**）、用 prompt 优化器把优化轨迹总结成原则、以及把穷举搜索与 LLM agent 结合（代价极高但效果最好）。

---

3. 代码示例与性能分析

3.1 示例一：全融合的卷积 + Scale/Bias + ReLU + 2×2 MaxPool（OpenMP）

这个例子来自讲义中反复出现的"融合"主线：`Conv → Scale/Bias → Max Pool` 三步如果各自物化中间张量，会产生数倍于必要量的 DRAM 流量；把它们融合进一个循环嵌套，中间值就永远留在寄存器里。

```
// conv_fused.cpp — 直接卷积 + Scale/Bias + ReLU + 2x2 MaxPool 的单遍融合实现
// 编译: g++ -O3 -march=native -fopenmp conv_fused.cpp -o conv_fused
// 运行: OMP_NUM_THREADS=16 ./conv_fused
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <omp.h>

static double now_ms() {
    using namespace std::chrono;
    return duration<double, std::milli>(steady_clock::now().time_since_epoch()).count();
}

int main() {
    // ---------------- 问题规模 ----------------
    const int N = 8;            // batch
    const int H = 64, W = 64;   // 输入空间尺寸 (均为偶数, 便于 2x2 pooling)
    const int C = 32;           // 输入通道 (INPUT_DEPTH)
    const int K = 64;           // 输出通道 (滤波器个数)
    const int R = 3, S = 3;     // 滤波器空间支撑 (stride = 1)
    const int HH = H + 2, WW = W + 2;   // 带 1 像素零填充的 halo
    const int OH = H / 2, OW = W / 2;

    // ---------------- 分配数据 ----------------
    float* in    = (float*)aligned_alloc(64, (size_t)N * HH * WW * C * sizeof(float));
    float* w     = (float*)aligned_alloc(64, (size_t)K * R * S * C * sizeof(float));
    float* scale = (float*)aligned_alloc(64, (size_t)K * sizeof(float));
    float* bias  = (float*)aligned_alloc(64, (size_t)K * sizeof(float));
    float* out   = (float*)aligned_alloc(64, (size_t)N * OH * OW * K * sizeof(float));

    srand(0);
    auto rnd = []() { return (float)rand() / (float)RAND_MAX - 0.5f; };

    // 输入: 内区填随机数, halo 保持 0 (calloc 语义)
    for (size_t i = 0; i < (size_t)N * HH * WW * C; i++) in[i] = 0.f;
    for (int n = 0; n < N; n++)
        for (int j = 0; j < H; j++)
            for (int i = 0; i < W; i++)
                for (int c = 0; c < C; c++)
                    in[((size_t)n * HH + (j + 1)) * WW + (i + 1)) * C + c] = rnd();

    for (size_t i = 0; i < (size_t)K * R * S * C; i++) w[i] = rnd();
    for (int f = 0; f < K; f++) { scale[f] = 1.0f + 0.01f * f; bias[f] = 0.5f; }
    for (size_t i = 0; i < (size_t)N * OH * OW * K; i++) out[i] = 0.f;

    // ---------------- 融合内核 ----------------
    const double t0 = now_ms();
    #pragma omp parallel for collapse(3) schedule(static)
    for (int n = 0; n < N; n++)
      for (int oh = 0; oh < H; oh += 2)          // 每次产出 2x2 卷积输出 -> 1 个 pool 输出
        for (int ow = 0; ow < W; ow += 2)
          for (int f = 0; f < K; f++) {          // 输出通道
            float pool = -3.4e38f;
            for (int dy = 0; dy < 2; dy++)
              for (int dx = 0; dx < 2; dx++) {
                const int j = oh + dy, i = ow + dx;
                float acc = 0.0f;
                for (int jj = 0; jj < R; jj++)
                  for (int ii = 0; ii < S; ii++)
                    for (int kk = 0; kk < C; kk++)   // kk 连续 => 可向量化
                      acc += w[((f * R + jj) * S + ii) * C + kk]
                           * in[((size_t)n * HH + (j + jj)) * WW + (i + ii)) * C + kk];
                acc = acc * scale[f] + bias[f];      // ← 融合 scale/bias (无需物化)
                if (acc < 0.0f) acc = 0.0f;          // ← 融合 ReLU
                pool = fmaxf(pool, acc);             // ← 融合 2x2 MaxPool
              }
            out[((size_t)n * OH + oh / 2) * OW + ow / 2) * K + f] = pool;
          }
    const double t1 = now_ms();
```

```
    // ----------------- 统计与校验 -----------------
    const double conv_flops = 2.0 * N * (double)H * W * K * R * S * C;   // 1 FMA = 2 FLOP
    double checksum = 0.0;
    for (size_t i = 0; i < (size_t)N * OH * OW * K; i++) checksum += out[i];

    printf("conv FLOPs        = %.4f GFLOP\n", conv_flops / 1e9);
    printf("fused kernel time = %.3f ms  ->  %.2f GFLOP/s\n",
           t1 - t0, conv_flops / ((t1 - t0) * 1e6));
    printf("checksum          = %.6f  (threads = %d)\n", checksum, omp_get_max_threads());
    printf("DRAM 流量(理想)   = in %.2f MB + w %.2f KB + out %.2f MB\n",
           N * HH * WW * C * 4 / 1e6, K * R * S * C * 4 / 1e3, N * OH * OW * K * 4 / 1e6);
    return 0;
}
```

【代码做什么？】 1. 构造一批随机激活 `in[N][HH][WW][C]`（含 1 像素零填充 halo，所以空间尺寸比输出大 2）、随机权重 `w[K][R][S][C]`、逐通道的 `scale` / `bias`。 2. 以 `(n, oh, ow)` 三层循环做并行分解（`collapse(3)`），每个工作项负责一个 2×2 的卷积输出块——这个块正好对应**一个** max-pool 输出，因此池化不需要额外遍历。 3. 对块内 4 个空间位置、每个位置累加 $C \times R \times S = 288$ 次乘加，随后**在同一段代码里**依次施加 `scale` / `bias`、ReLU、以及 2×2 的 `fmaxf` 池化，最终只写一个结果。 4. 最后统计 FLOPs、计时、打印 GFLOPS、校验和与理论 DRAM 流量。

【并行机制与性能解说】 – **并行机制**：`#pragma omp parallel for` 把 $N \times (H/2) \times (W/2) = 8 \times 32 \times 32 = 8192$ 个工作项切分给线程（`collapse(3)` 让 OpenMP 把三维迭代空间线性化后再切块）；每个线程独立执行内层的 `f`、`dy/dx`、`jj/ii/kk` 循环，**共享数据是只读的** **`in`**、**`w`**、**`scale`**、**`bias`**，输出写入互不重叠的地址（写集合不重叠 ⇒ 无需锁、无数据竞争）。向量化发生在最内层 `kk`（输入与权重在该维都是连续的），但如果编译器不把 `acc` 归约拆开，会形成 288 长的串行依赖链——生产实现应当**用 4 个独立部分和累加器**或把 `f` 提到内层做向量化。 – **Work（总工作量）**：$W = N \cdot H \cdot W \cdot K \cdot R \cdot S \cdot C = 8 \times 64 \times 64 \times 64 \times 9 \times 32 = 6.04 \times 10^8$ 次 FMA $= 1.21$ GFLOP。池化部分只增加 $3 \times N \cdot OH \cdot OW \cdot K \approx 1.6 \times 10^6$ 次比较，可忽略。 – **Span（关键路径）**：单个输出点的归约链为 $C \cdot R \cdot S = 288$ 次 FMA，池化再叠加 3 次 `fmax`，所以 $L \approx 4 \times (288 + 3) + 3 \approx 1167$ 次串行运算。 – **并行度 = Work / Span** $= 6.04 \times 10^8 / 1167 \approx 5.2 \times 10^5$。这远大于任何现有机器（16 核 × 8 lane × 2 issue ≈ 256 路并行），说明**这个 kernel 的并行度不是问题，问题在内存与计算效率**。 – **瓶颈分析**： – **DRAM 流量（理想）**：输入内区 4.19 MB（含 halo 为 4.46 MB）+ 权重 73.7 KB + 输出 2.10 MB ≈ 6.4 MB；对应算术强度 $1.21 \times 10^9 / 6.4 \times 10^6 \approx 189$ FLOP/Byte，**远高于 ridge point 38.4（见第 4 节）⇒ 理论上计算受限**。 – **但实际上**：这个循环顺序把输入 halo 的重读次数限制在每次输出块一次；若把 `f` 循环移到 `jj/ii` 之外重新排列，输入会被重读 64 次，DRAM 流量变成 64 × 4 MB ≈ 268 MB，算术强度掉到 4.5 FLOP/Byte ⇒ **立刻变成严重带宽受限**。这正是"循环顺序决定一切"的实例。 – **负载不均**：每个工作项的工作量完全相同（8192 个同构任务），所以 `schedule(static)` 足够；若把并行维度改成按行切分（`H` 不整除线程数）才会出现不均。 – **同步开销**：整个内核只有一次隐式的 fork-join 屏障（parallel for 结束），同步成本被 1.21 GFLOP 完全摊薄。

3.2 示例二：分层分块 + 寄存器分块 + SIMD 的 GEMM（OpenMP）

```
// gemm_blocked.cpp — 三级分块 (L2/L1) + 4x16 寄存器分块 + SIMD 的单精度 GEMM
// 编译: g++ -O3 -march=native -fopenmp gemm_blocked.cpp -o gemm_blocked
// 运行: OMP_NUM_THREADS=16 ./gemm_blocked 1024
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <omp.h>

#define MB 64     // j (行) 方向的块: 让 A 的一个条带在 L2 里常驻
#define NB 256    // i (列) 方向的块: B 的一个条带在 L2 里常驻
#define KB 256    // k (归约) 方向的块: 让 C 的累加块在缓存里完成
#define MR 4      // 微内核行数 (A 的复用度)
#define NR 16     // 微内核列数 (B 的复用度, 也是 SIMD 宽度)

static double now_ms() {
    using namespace std::chrono;
    return duration<double, std::milli>(steady_clock::now().time_since_epoch()).count();
}

int main(int argc, char** argv) {
    const int n = (argc > 1) ? atoi(argv[1]) : 1024;
    const int M = n, K = n, N = n;      // 方阵, 且保证 n 是 64 的倍数
    const size_t bytes = (size_t)M * N * sizeof(float);

    float* A = (float*)aligned_alloc(64, bytes);
    float* B = (float*)aligned_alloc(64, bytes);
    float* C = (float*)aligned_alloc(64, bytes);
    srand(1);
    for (size_t i = 0; i < (size_t)M * K; i++) A[i] = (float)rand() / RAND_MAX - 0.5f;
    for (size_t i = 0; i < (size_t)K * N; i++) B[i] = (float)rand() / RAND_MAX - 0.5f;
    for (size_t i = 0; i < bytes / 4; i++) C[i] = 0.f;

    const double t0 = now_ms();
    // ---- 并行分解在最外层 (j 块), 避免跨线程写同一 cache line ----
    #pragma omp parallel for schedule(static)
    for (int jb = 0; jb < M; jb += MB)
      for (int kb = 0; kb < K; kb += KB)
        for (int ib = 0; ib < N; ib += NB)
          for (int j = jb; j < jb + MB && j + MR <= M; j += MR) {
            float* c0 = &C[(size_t)(j    ) * N];
            float* c1 = &C[(size_t)(j + 1) * N];
            float* c2 = &C[(size_t)(j + 2) * N];
            float* c3 = &C[(size_t)(j + 3) * N];
            for (int k = kb; k < kb + KB && k < K; k++) {
              const float a0 = A[(size_t)(j    ) * K + k];
              const float a1 = A[(size_t)(j + 1) * K + k];
              const float a2 = A[(size_t)(j + 2) * K + k];
              const float a3 = A[(size_t)(j + 3) * K + k];
              const float* brow = &B[(size_t)k * N];
              // 规范循环形式: 上界先算成变量 (OpenMP 不接受 i < ib+NB && i < N 这种谓词)
              const int ib_hi = (ib + NB < N) ? (ib + NB) : N;
              #pragma omp simd
              for (int i = ib; i < ib_hi; i++) {
                const float bv = brow[i];
                c0[i] += a0 * bv;      // 每个 b 值被 4 次复用 (寄存器分块)
                c1[i] += a1 * bv;
                c2[i] += a2 * bv;
                c3[i] += a3 * bv;
              }
            }
          }
    const double t1 = now_ms();

    double checksum = 0.0;
    for (size_t i = 0; i < (size_t)M * N; i++) checksum += C[i];
    const double flops = 2.0 * M * N * K;
    printf("M=N=K=%d, threads=%d\n", n, omp_get_max_threads());
    printf("time   = %.3f ms\n", t1 - t0);
    printf("%.2f GFLOP/s  (checksum %.3f)\n", flops / ((t1 - t0) * 1e6), checksum);
```

```
    return 0;
}
```

【代码做什么？】 1. 生成两个 $n \times n$ 的随机矩阵（n 取 64 的倍数），零初始化 C。 2. 外层 `parallel for` 只并行化最外的 `jb` 分块循环——每个线程拿若干行条带，**互不共享输出 cache line**（避免伪共享）。 3. 每个线程串行执行三层分块循环 `kb` 、`ib` ，在最内层用 **4 行 × 16 列**的寄存器微内核：一次读入 4 个 A 元素，然后对 16 个连续的 B 元素做广播乘法（`#pragma omp simd` 让编译器生成宽 SIMD 指令）。 4. 打印时间、GFLOP/s 与校验和。

【并行机制与性能解说】 – **并行机制**：线程数 P 由 `OMP_NUM_THREADS` 决定；每线程分到 $M/(P \cdot MB)$ 个 `jb` 块（静态分配）。SIMD 通道由 `#pragma omp simd` 在最内层 `i` 循环上启用（`-march=native` 允许 AVX2/AVX-512）：**同一时刻 8 或 16 个 lane 各自处理不同的输出列，共享同一条 `brow` cache line 的读请求**——这就是讲义第一种 SIMD 方案（"向量化 i 循环，同时改善 B 的空间局部性"）。 – **Work（总工作量）**：$W = M \cdot N \cdot K$ 次 FMA $= 2MNK$ FLOP。$n = 1024$ 时为 1.07×10^9 FMA $= 2.15$ GFLOP。 – **Span（关键路径）**：每个输出元素是 K 长的归约链。若允许树形归约，深度为 $\log_2 K \approx 10$；若用 4 路部分和（本代码是 4 个独立行，不是 4 路拆分同一元素），单元素仍是 K 深。取较保守的 $L = K = 1024$ 次串行 FMA。 – **并行度**：$W/L = MNK/K = M \cdot N = 1.05 \times 10^6$。**并行度与 K 无关，只与输出规模成正比**——这也是为什么 GEMM 能轻松填满 GPU（输出元素数量极大）。 – **瓶颈分析**： – **内存带宽**：$n = 1024$ 时三矩阵各 4 MB，共 12 MB（L2 若为 32 MB 级别可大部分驻留）。若假定 L2 装不下、所有 B 都从 DRAM 读：B 被读 $M/MB = 16$ 次 ⇒ 流量 16×4 MB $= 64$ MB；实测 16 核上这类实现通常在**几十到一两百 GFLOP/s**（取决于带宽与端口），远低于 768 GFLOP/s 的 SIMD 峰值 ⇒ **带宽受限**。 – **寄存器压力**：`MR×NR = 4×16 = 64` 个累加器 + 4 个 A 值 + 1 个广播值 ≈ 70 个向量寄存器需求，在 AVX2（16 个 ymm）上必须靠编译器把 `NR` 降级或用更小的 `NR` ；这是 `BLOCKSIZE` 不能无限增大的现实约束（讲义的自检问题）。 – **伪共享**：由于并行维度是最外层 `jb` 且 `MB=64` （一行 256 字节 = 4 条 cache line），不会出现两个线程写同一条 line。 – **同步开销**：只有一次 fork-join。

3.3 示例三：CUDA 上的融合 online-softmax 注意力（FlashAttention 风格）

```
// flash_attn.cu — FlashAttention 风格的融合注意力: 永不物化 N×N 的分数矩阵
// 编译: nvcc -O3 -arch=sm_80 -o flash_attn flash_attn.cu
// 运行: ./flash_attn 1024 64
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <cuda_runtime.h>

#define WARP     32
#define BLOCK_J  64      // 每次处理 64 个 key (key tile)
#define DPT_MAX  8       // 每个 lane 最多负责 d/32 个 feature (支持 d <= 256)

__device__ __forceinline__ float warp_max(float v) {
    #pragma unroll
    for (int o = 16; o > 0; o >>= 1) v = fmaxf(v, __shfl_xor_sync(0xffffffffu, v, o));
    return v;
}
__device__ __forceinline__ float warp_sum(float v) {
    #pragma unroll
    for (int o = 16; o > 0; o >>= 1) v += __shfl_xor_sync(0xffffffffu, v, o);
    return v;
}

// 一个 warp (= 32 线程) 负责一行 query; d 必须是 32 的倍数
__global__ void flash_attn_kernel(const float* __restrict__ Q,
                                  const float* __restrict__ K,
                                  const float* __restrict__ V,
                                  float* __restrict__ O,
                                  int N, int d, float scale)
{
    extern __shared__ float smem[];
    float* kbuf = smem;                    // BLOCK_J × d 的 K tile
    float* vbuf = kbuf + BLOCK_J * d;      // BLOCK_J × d 的 V tile

    const int row  = blockIdx.x;           // query 行号
    const int lane = threadIdx.x;          // 0..31
    const int nd   = d / WARP;             // 每个 lane 负责的 feature 数
    if (row >= N || nd > DPT_MAX) return;

    float q[DPT_MAX], acc[DPT_MAX];
    for (int p = 0; p < nd; p++) {
        q[p]   = Q[(size_t)row * d + lane + p * WARP];
        acc[p] = 0.0f;
    }

    float m = -1.0e30f;    // running max (online softmax 的状态)
    float l = 0.0f;        // running sum of exp

    for (int jb = 0; jb < N; jb += BLOCK_J) {
        const int nj = (N - jb < BLOCK_J) ? (N - jb) : BLOCK_J;
        // 协同把 K/V 的 tile 搬进片上共享内存 (jb..jb+nj-1 行在全局内存中连续)
        for (int t = threadIdx.x; t < nj * d; t += WARP) {
            kbuf[t] = K[(size_t)jb * d + t];
            vbuf[t] = V[(size_t)jb * d + t];
        }
        __syncwarp();

        for (int jj = 0; jj < nj; jj++) {
            // ---- 分数 s = q · k_jj (warp 内归约) ----
            float s = 0.0f;
            for (int p = 0; p < nd; p++) s += q[p] * kbuf[jj * d + lane + p * WARP];
            s = warp_sum(s) * scale;

            // ---- online softmax 更新: 重标定旧的累加器 ----
            const float m_new = fmaxf(m, s);
            const float alpha = __expf(m - m_new);     // 旧值 (含 m、l、acc) 的缩放因子
            const float pij   = __expf(s - m_new);
            l = l * alpha + pij;
            for (int p = 0; p < nd; p++)
                acc[p] = acc[p] * alpha + pij * vbuf[jj * d + lane + p * WARP];
```

```
            m = m_new;
        }
        __syncwarp();
    }
    for (int p = 0; p < nd; p++) O[(size_t)row * d + lane + p * WARP] = acc[p] / l;
}

static void check(cudaError_t e, const char* msg) {
    if (e != cudaSuccess) { printf("CUDA error (%s): %s\n", msg, cudaGetErrorString(e)); exit(1); }
}

// CPU 参考实现: 老老实实物化 N×N 矩阵, 三遍走完
static void naive_ref(const float* Q, const float* K, const float* V, float* O,
                      int N, int d, float scale) {
    std::vector<float> Smat((size_t)N * N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            float s = 0.0f;
            for (int t = 0; t < d; t++) s += Q[(size_t)i * d + t] * K[(size_t)j * d + t];
            Smat[(size_t)i * N + j] = s * scale;
        }
    for (int i = 0; i < N; i++) {
        float mx = -1e30f;
        for (int j = 0; j < N; j++) mx = fmaxf(mx, Smat[(size_t)i * N + j]);
        float sum = 0.0f;
        for (int j = 0; j < N; j++) {
            Smat[(size_t)i * N + j] = expf(Smat[(size_t)i * N + j] - mx);
            sum += Smat[(size_t)i * N + j];
        }
        for (int t = 0; t < d; t++) {
            float a = 0.0f;
            for (int j = 0; j < N; j++) a += Smat[(size_t)i * N + j] * V[(size_t)j * d + t];
            O[(size_t)i * d + t] = a / sum;
        }
    }
}

int main(int argc, char** argv) {
    const int N = (argc > 1) ? atoi(argv[1]) : 1024;
    const int d = (argc > 2) ? atoi(argv[2]) : 64;
    const float scale = 1.0f / sqrtf((float)d);
    const size_t bytes = (size_t)N * d * sizeof(float);

    float *hQ = (float*)malloc(bytes), *hK = (float*)malloc(bytes),
          *hV = (float*)malloc(bytes), *hO = (float*)malloc(bytes),
          *hRef = (float*)malloc(bytes);
    srand(2);
    for (size_t i = 0; i < bytes / 4; i++) {
        hQ[i] = (float)rand() / RAND_MAX - 0.5f;
        hK[i] = (float)rand() / RAND_MAX - 0.5f;
        hV[i] = (float)rand() / RAND_MAX - 0.5f;
    }

    float *dQ, *dK, *dV, *dO;
    check(cudaMalloc(&dQ, bytes), "malloc Q");  check(cudaMalloc(&dK, bytes), "malloc K");
    check(cudaMalloc(&dV, bytes), "malloc V");  check(cudaMalloc(&dO, bytes), "malloc O");
    check(cudaMemcpy(dQ, hQ, bytes, cudaMemcpyHostToDevice), "copy Q");
    check(cudaMemcpy(dK, hK, bytes, cudaMemcpyHostToDevice), "copy K");
    check(cudaMemcpy(dV, hV, bytes, cudaMemcpyHostToDevice), "copy V");

    const size_t smem = 2 * (size_t)BLOCK_J * d * sizeof(float);
    // 若 smem > 48 KB 需显式申请: cudaFuncSetAttribute(flash_attn_kernel,
    //     cudaFuncAttributeMaxDynamicSharedMemorySize, smem);

    dim3 grid(N), block(WARP);
    flash_attn_kernel<<<grid, block, smem>>>(dQ, dK, dV, dO, N, d, scale);
    check(cudaGetLastError(), "launch");
    check(cudaDeviceSynchronize(), "sync");

    // 计时 (20 次迭代)
    cudaEvent_t e0, e1;
    cudaEventCreate(&e0); cudaEventCreate(&e1);
```

```
    cudaEventRecord(e0);
    for (int it = 0; it < 20; it++)
        flash_attn_kernel<<<grid, block, smem>>>(dQ, dK, dV, dO, N, d, scale);
    cudaEventRecord(e1);
    check(cudaDeviceSynchronize(), "sync2");
    float ms = 0.f; cudaEventElapsedTime(&ms, e0, e1); ms /= 20.0f;

    check(cudaMemcpy(hO, dO, bytes, cudaMemcpyDeviceToHost), "copy O");
    naive_ref(hQ, hK, hV, hRef, N, d, scale);
    double maxerr = 0.0;
    for (size_t i = 0; i < bytes / 4; i++) maxerr = fmax(maxerr, fabs(hO[i] - hRef[i]));

    const double flops = 4.0 * (double)N * N * d;   // QK^T + PV, 各 2N²d
    printf("N=%d d=%d  时间=%.3f ms  %.2f GFLOP/s  max|err|=%.3e\n",
           N, d, ms, flops / (ms * 1e6), maxerr);
    printf("naive 版本需要物化 %.2f MB 的 N×N 分数矩阵; 融合版本需要 0 MB\n",
           (double)N * N * 4 / 1e6);
    return 0;
}
```

【代码做什么？】 1. `flash_attn_kernel` 用**一个 warp 负责一行 query**：`blockIdx.x` 是行号，32 个 lane 各负责 $d/32$ 个 feature 分量。 2. 外层循环每次搬入 $B_J = 64$ 个 key 的 K/V tile 到共享内存（`extern __shared__`，`__syncwarp()` 保证可见性）。 3. 对 tile 内每个 key，lane 各自算部分点积，再通过 `__shfl_xor_sync` 做 warp 内归约得到分数 s。 4. 用 online softmax 递推更新 running max `m`、归一化和 `l`，以及输出累加器 `acc[p]`：$\alpha = e^{m_{old} - m_{new}}$，`l = l·α + e^{s−m_new}`，`acc = acc·α + e^{s−m_new}·v`。 5. 全部 key 处理完后一次性除以 `l` 写出结果——**整个过程中 $N \times N$ 的分数矩阵从未被创建**。 6. 主机端用 `naive_ref` 物化 $N \times N$ 矩阵并三遍计算作为参考，比较最大绝对误差，并用 CUDA event 计时 20 次迭代。

【并行机制与性能解说】 – **并行机制**：N 个 thread block，每块 32 个线程（一个 warp），映射到 SM 的 SIMD lane 上。warp 内归约用 5 步 shuffle（$\log_2 32$），无需共享内存与 barrier；块内的 K/V tile 复用发生在共享内存（256 KB/SM 的片上存储）；`acc[]` 与 `q[]` 常驻寄存器。**关键点：$N \times N$ 的中间结果被"稀释"成 d 宽的累加器**——存储复杂度从 $O(N^2)$ 降到 $O(d)$。 – **Work（总工作量）**：分数矩阵 $2N^2d$ FLOPs + 输出 $2N^2d$ FLOPs = $4N^2d$ FLOPs；$N = 1024, d = 64$ 时为 2.68×10^8 FLOPs $= 0.268$ GFLOP。此外每个 tile 要对 d 维累加器做一次重标定：每行约 $(N/B_J) \cdot d$ 次额外乘法，占总量的比例约 $1/(2B_J) \approx 0.8\%$（$B_J = 64$）。 – **Span（关键路径）**：online 形式把 key 循环变成了**线性的依赖链**——第 $j+1$ 个 key 的 `m`、`l`、`acc` 都依赖第 j 个 key 的结果。单个 query 行的关键路径 $L \approx N \times (d/32 + 5 + c)$（每个 key 需要 $d/32$ 次串行乘加 + 5 步 shuffle + 若干次指数），取 $c \approx 3$：$L \approx 1024 \times (2 + 5 + 3) = 10240$。 – **并行度 = Work / Span** $= 2.68 \times 10^8 / 10240 \approx 2.6 \times 10^4$。作为对比，如果允许用"两遍"的经典 softmax（先全算 S 再归约），N 维归约可用树形，span 可降到 $O(\log N + \log d)$，并行度提高几个数量级——**融合/流式是以"提高关键路径长度"换取"减少片上存储与 DRAM 流量"的典型交易**（讲义明确指出 chunked 版本"比原版有额外计算"）。 – **瓶颈分析**： – **并行工作不足**：只有 $N = 1024$ 个 warp，而 H100 有 144 个 SM、每 SM 最多 64 个 warp（约 9216 个 warp 容量）⇒ **单个 head、单条序列根本填不满机器**（呼应讲义 "Need a lot of parallel work to fill the machine"：$N = 1$ 的卷积只有 2 MB 输出，而 $N = 32$ 时有 1 GB 输出）。工程上必须靠 batch × head 维度并行。 – **共享内存带宽与容量**：$d = 64$ 时 shared memory $= 2 \times 64 \times 64 \times 4 = 32$ KB/块；若 d 增大要检查 48 KB 的默认上限并显式 `cudaFuncSetAttribute`。 – **同步**：只用 `__syncwarp()`（warp 内屏障，几乎零成本），没有 `__syncthreads()`，这是"一个 warp 负责一行"这个映射带来的额外好处。 – **每线程寄存器压力**：`q[8]` + `acc[8]` + 标量，$d = 256$ 时接近 `DPT_MAX` 上限；这也是生产实现改用"多 warp 协作 + 跨 warp 归约"的原因（代价是引入 `__syncthreads()` 与更多共享内存）。

---

4. 性能模型与复杂度分析

本节把 Roofline、算术强度、work–span 与能耗账本放在一起做定量推理。所有参数都在算例中显式给出假设。

4.1 Roofline、ridge point 与"算力提升反而更容易带宽受限"

假设一台机器：**16 核 × 3.0 GHz × 8 宽 SIMD × 2（FMA 计 2 FLOP）= 768 GFLOP/s 峰值；DRAM 带宽 20 GB/s**。则 ridge point 为

$$I^* = \frac{\pi_{peak}}{\beta} = \frac{768 \times 10^9}{20 \times 10^9} = 38.4 \ \text{FLOP/Byte}.$$

- 一个算术强度 $I = 1$ 的 kernel（例如逐元素相加）受带宽限制，吞吐只有 $1 \times 20 \ \text{GB/s} = 20$ GFLOP/s，仅用到峰值的 2.6%。
- 融合变换把 I 从 $1/3$（三遍循环）提高到 $3/5$（融成一遍）——讲义的原例：两遍 add/mul 加一遍 add，每次遍历 2 load + 1 store / 1 op（$I = 1/3$）；融合后 4 load + 1 store / 3 ops（$I = 3/5$）。**吞吐因此提高 1.8×，而代码没换、机器没换。**

- 若把机器升级为算力翻倍（1536 GFLOP/s，带宽不变），ridge point 变成 **76.8 FLOP/Byte**，$I = 3/5$ 的 kernel 依然是带宽受限，吞吐**一点不涨**。这就是讲义那句总结的定量含义：**"提高算力让程序更容易带宽受限；提高算术强度让程序更容易计算受限。"**

4.2 GEMM 分块的算术强度：为什么必须做层次化 blocking

设三级块（A、B、C 的三个 $b \times b$ 块同时驻留在某一级存储中），则每个"块周期"执行 $2b^3$ FLOP、搬运 $3b^2$ 个元素。于是

$$OI(b) = \frac{2b^3}{3b^2} = \frac{2b}{3}\ \text{FLOP/元素} = \frac{b}{6}\ \text{FLOP/Byte} \quad (\text{fp32}).$$

块大小 b	所在层次（假设）	OI (FLOP/Byte)	vs ridge point 38.4
8	寄存器微内核	1.3	极度带宽受限
32	L1	5.3	带宽受限
64	L1/L2	10.7	带宽受限
128	L2	21.3	仍带宽受限
256	L2	42.7	**计算受限** ✅

结论（数值化）：要在 768 GFLOP/s / 20 GB/s 的机器上跑满算力，仅靠单级分块需要 $b \geq 230$，而三个 256^2 的 fp32 块需要 $3 \times 256^2 \times 4\text{B} = 786$ KB——单核私有 L2 通常没有这么大。因此必须**层次化分块**：寄存器级用 8×8 微内核保证每次 load 复用 8 次，L1/L2 级用大块摊薄 DRAM 流量，两者叠加才能让**有效**算术强度跨过 38.4。这正是讲义 "Hierarchical blocked matrix mult" 与 "Not shown: final level of blocking for register locality" 的动机。

4.3 explicit GEMM 的代价：DRAM 流量放大 $R \times S$ 倍（数值算例）

假设参数：$N_{batch} = 1$，$H = W = 256$，$C = 64$，$K = 64$，$R = S = 3$，fp32，机器带宽 $\beta = 900$ GB/s（V100 级 HBM）。

- 计算量：$2 \times (PQ) \times K \times (RSC) = 2 \times 65536 \times 64 \times 576 = 4.83$ GFLOP。在 A100 的 19.5 TFLOPS fp32 上理论耗时 $4.83 \times 10^9 / (19.5 \times 10^{12}) = 0.25$ ms。
- im2col 矩阵大小：$(PQ) \times (RSC) \times 4\text{B} = 65536 \times 576 \times 4 = 151$ MB。**写一次 + 读一次** = 302 MB 流量。
- 时间：$302 \times 10^6 / 900 \times 10^9 = 0.336$ ms。

⇒ 光是把卷积矩阵物化一次，搬运时间就超过了计算时间本身（0.336 ms vs 0.25 ms），并且需要 151 MB 的额外存储（是输入的 $R \times S$ 倍）。这就是 **implicit GEMM** 的全部理由：只在片上共享内存里物化一个 sub-block，不增加 DRAM 流量也不需要额外 off-chip 存储。

4.4 融合的收益：以 Conv+Scale/Bias+Pool 与 softmax 为例

算例 A（讲义 slide 56 的场景）：设卷积输出为 **1 GB**。 – 不融合：写 conv 输出 1 GB → 读回做 scale/bias 1 GB → 写 1 GB → 读回做 pool 1 GB → 写 0.25 GB。总流量 ≈ 4.25 GB。 – 融合：写最终 pooled 输出 0.25 GB（conv 中间结果永不落 DRAM），总流量 ≈ 0.25 GB。 – 在 $\beta = 900$ GB/s 下：$4.25/0.9 \approx 4.72$ ms vs $0.25/0.9 \approx 0.28$ ms，**节省约 4.4 ms**。

算例 B（softmax，讲义 slide 59）：naive 实现读 $5MN + 2M$ 个元素、写 $3MN + 2M$ 个元素；融合实现读 MN、写 MN（前提是一行的工作集能装进片上存储）。 – 取 $M = N = 4096$，fp32：$MN = 16.78$ M 元素 = 67.1 MB。 – naive：读 $5 \times 67.1 + 2 \times 0.0168 \approx 335.5$ MB，写 $3 \times 67.1 \approx 201.3$ MB，共 **536.8 MB**。 – 融合：读 67.1 MB + 写 67.1 MB = **134.2 MB**。 – **流量降为 1/4**；在 900 GB/s 下从 0.596 ms 降到 0.149 ms。

4.5 硬件加速器的定量账本

(a) 能效阶梯的算术：假设 CPU 方案 100 W 达到 768 GFLOP/s（7.7 GFLOP/s/W）。若按讲义的"10×（GPU）/20×（域专用）/100–1000×（ASIC）"，则同样功耗下：GPU ≈ 77 GFLOP/s/W，域专用加速器 ≈ 154 GFLOP/s/W，ASIC 可达 770–7700 GFLOP/s/W。

(b) TPU 式 systolic array 的峰值与能效（**假设参数**：256×256 = 65536 个 PE，每 PE 每周期 1 次 MAC，时钟 0.7 GHz，整芯片功耗 40 W）：

$$\text{峰值} = 65536 \times 0.7 \times 10^9 \times 2\ \text{op} = 9.18 \times 10^{13}\ \text{op/s} \approx 92\ \text{TOPS (8-bit)}$$

$$\text{能效} = 92 \times 10^{12} / 40 = 2.3\ \text{TOPS/W} = 0.43\ \text{pJ/op}.$$

把 0.43 pJ/op 与讲义给出的"整数运算约 1 pJ"对比可知：systolic array 通过**把指令流开销摊薄到零**，其每 op 能耗已落到"纯逻辑运算"量级甚至更低——这正是"专业化 = 能效"的物理来源。

(c) 数据移动 vs 计算的能耗（讲义 ballpark：整数运算 1 pJ，浮点运算 20 pJ，1 mm 外小 SRAM 读 64 bit 26 pJ，LPDDR 读 64 bit 1200 pJ）： – LPDDR：$1200/8 = 150$ pJ/Byte。 – 一个 kernel 做 1 GFLOP fp32（5×10^8 次 FMA）并读写 1 GB DRAM： – 计算能耗

$= 1 \times 10^9 \times 20\ \text{pJ} = 0.02$ J。 – 访存能耗 $= 1.07 \times 10^9 \times 150\ \text{pJ} = 0.16$ J。 – **访存是计算的 8 倍** ⇒ 即使算力提升 8 倍，总能耗也几乎不变。这就是"降低数据移动"压倒一切的定量理由。 – 反过来看算术强度：要让访存能耗与计算能耗持平（fp32），需要 $OI \approx 20\ \text{pJ}/150\ \text{pJ/Byte} \approx 0.13$ FLOP/Byte——**能耗意义上的 ridge point 远低于性能意义上的 ridge point**，所以"能效受限"往往比"性能受限"更容易先撞墙。

(d) Tensor Core 把 ridge point 推向何处：H100 上 fp16 tensor core 峰值约 989 TFLOPS，HBM 带宽约 3 TB/s（H100 SXM 量级）⇒ $I^* \approx 989 \times 10^{12}/3 \times 10^{12} \approx 330$ FLOP/Byte。也就是说，**要用满张量核，kernel 必须每读 1 字节 DRAM 就完成 330 次浮点运算**——只有把数据全部 tiling 进片上存储（shared memory / TMEM）并做算子融合才可能达到。这解释了为什么现代 AI kernel 的形态是"小块 + 异步搬运 + 全融合"，而不是"大矩阵 + 一次算完"。

4.6 work–span 视角下的三种实现对比（汇总）

实现	Work（FLOPs）	Span（关键路径）	并行度	主要瓶颈
直接卷积（3.1）	$2NHWKRSC = 1.21$ GFLOP	≈ 1167 次串行运算	5.2×10^5	循环顺序错 ⇒ DRAM 流量 ×64；无 SIMD ⇒ 利用率低
分块 GEMM（3.2）	$2MNK = 2.15$ GFLOP	$K = 1024$ 次 FMA（树形可降到 10）	1.05×10^6	DRAM 带宽（B 被重读 M/MB 次）；寄存器压力
融合注意力（3.3）	$4N^2d = 0.268$ GFLOP（+0.8% 重标定）	$\approx N(d/32 + 8) = 10240$	2.6×10^4	**并行工作不足**（仅 N 个 warp）；共享内存容量

三行合起来给出本讲的性能方法论：**先看 W/L 判断并行度是否够填机器；再看 OI 与 ridge point 判断是计算受限还是带宽受限；最后用融合把中间结果留在片上，用分块把每一级存储的复用榨干。**

---

5. 关键要点

1. **DNN 的性能问题，90% 是关于数据复用的 GEMM 问题。** 全连接层、卷积层、Transformer 的 attention 块最终都是（隐式）矩阵乘；矩阵乘的算术强度随分块尺寸线性增长（$OI \propto n$），因此"分块 + 融合 + 低精度"这三件软件层面的事，加上"更高带宽 + 更专的硬件"这两件硬件层面的事，几乎就是全部优化空间。
2. **一切设计围绕"减少数据移动"。** 片外 DRAM 访问的能量成本比一次运算高 1–2 个数量级（1 mm SRAM 读 64 bit ≈ 26 pJ，LPDDR 读 64 bit ≈ 1200 pJ，而一次整数运算 ≈ 1 pJ），所以 tiled tensor（16×16/32×32）、异步计算/异步访存/异步片间通信、以及计算单元之间的直接通信（融合与流水）成为理想加速器的三大特征。
3. **能效来自专业化，而不是更多的核。** 相对 CPU 上的高质量 C 代码：吞吐型处理器（GPU）约 10× perf/W，域专用加速器（TPU 式）约 20×，固定功能 ASIC 可达 100–1000×；代价是可编程性的丧失与数千万至上亿美元的设计成本。TPU 把 30% 面积给算术单元、只保留 5 条指令、用 weight-stationary 的 systolic array 让通信退化为邻居之间的局部传递，是这一原则的教科书实现。
4. **融合（fusion）是软件侧收益率最高的单点变换。** Conv+Scale/Bias+Pool 的融合可把 DRAM 流量从 4.25 GB 降到 0.25 GB；逐行 softmax 的融合把流量降为 1/4；FlashAttention 用 online softmax 的结合律换来"永不物化 N^2 矩阵"，代价仅是不到 1% 的额外重标定计算。融合的本质是把"计算顺序"从内存往返中解耦。
5. **抽象层次与自动化是现代性能工程的出路。** 从寄存器/共享内存级的手写 CUDA，到 tile 级原语（CUTLASS/Triton/Thunderkittens），再到 Halide 式的"算法 / 调度"分离、自动调度搜索（ML 代价模型 + 树搜索，与人类最佳调度相当），再到 LLM agent 的"生成–剖析–反思–修改"循环——每一层抽象都在把"性能专家的隐性知识"变成可复用、可搜索、可自动生成的东西；而"硬件彩票"提醒我们，最终胜出的算法往往是因为它契合了当下的硬件，而非因为它普遍更优。

---

6. 常见陷阱与注意事项

- **把卷积物化成 im2col 矩阵再用库函数**：explicit GEMM 会把 DRAM 流量放大 $R \times S$ 倍（3×3 卷积即 9 倍）并需要与激活同量级的额外存储。在上述算例中，物化一次 151 MB 的卷积矩阵（读+写 302 MB）的耗时（0.336 ms）已经超过计算本身的 0.25 ms。除非模型很小或库实现经过专门调优，否则应使用 implicit GEMM（在片上共享内存里物化 sub-block）。
- **只做单层 blocking，或盲目加大 block size**：单层分块下 $OI = b/6$ FLOP/Byte（fp32），要达到 ridge point 38.4 需要 $b \approx 230$，单核私有缓存根本装不下；必须用"寄存器微内核 + L1/L2 大块"的层次化分块。反过来，块越大越好是**错的**：$MR \times NR = 4 \times 16$ 已经需要约 70 个向量寄存器，超过 AVX2 的 16 个 ymm 就会 spill，性能反而下降（讲义的自检问题正是问这个）。

- **忽略算子之间的内存往返**：把 `Conv → Scale/Bias → MaxPool` 写成三个独立的 kernel，会产生数倍于必要量的 DRAM 流量（算例 A：4.25 GB vs 0.25 GB）。同理，softmax 的 naive 实现读写量是融合实现的 4 倍。判断标准很简单：**中间张量有没有必要离开片上存储?**
- **误以为"峰值 TFLOPS 更高就一定更快"**：Tensor Core 把 ridge point 推到 300+ FLOP/Byte，此时任何"每字节算不到 300 次运算"的 kernel 都仍然带宽受限。低精度也会改变账本：fp16 让同一份数据的字节数减半（算术强度翻倍），但如果累加器也用 fp16，长归约链的舍入误差会毁掉数值结果——正确做法是 A、B 低精度、**累加用 fp32**，并且 bf16（指数位与 fp32 相同、尾数只有 7 位）与 fp16（范围小、精度高）不能混用，fp8 还要区分 E4M3（范围 0–448）与 E5M2（范围 0–57344）。
- **忽视小 batch / 短序列导致的并行度不足**：$N=1$、$P=Q=64$ 的卷积只有 524K 个输出（2 MB），融合注意力的一个 head 只有 N 个 warp（$N=1024$ 也仅占 H100 约 9216 个 warp 容量的 11%），systolic array 更会因矩阵维度不整除阵列尺寸而大量空转。这类负载应该靠 batch × head × 层间流水来补足并行度，而不是奢望单 kernel 打满硬件。
- **并行维度选错导致伪共享与负载不均**：并行化最内层（或在多个线程间共享输出 cache line）会引入伪共享；把 `#pragma omp parallel for` 放在 `N`（不整除线程数）而非 `collapse(3)` 的平坦迭代空间上会造成尾部负载不均；在 GEMM 里把线程切在 `i`（列）方向上会让不同线程写同一行的相邻元素——正确做法是把并行切分点放在最外层块（如 3.2 中的 `jb`），并保证每个线程的写区间按 cache line（64 字节 = 16 个 fp32）对齐。

---

7. 思考题（带答案）

问题 1

某推理服务对单个样本做一次 $y=Wx$，其中 W 是 4096×4096 的 fp16 权重矩阵。请用 Roofline 定量解释：为什么这块 GPU 的 989 TFLOPS 张量核算力实际只能用到百分之几？如果改成一次处理 32 个样本（batch=32），情况如何变化?

【答案】 – batch=1 时，这是一个矩阵–向量乘（GEMV）。计算量：$2\times4096\times4096=3.36\times10^7$ FLOP。必须搬运的数据：权重 $4096^2\times2\text{B}=33.6$ MB（每个权重只用一次），加上输入 8 KB、输出 8 KB。 – 算术强度 $I=3.36\times10^7/3.36\times10^7\approx1.0$ FLOP/Byte（每个权重字节只支撑约 1 次浮点运算）。 – 在 $\beta\approx3$ TB/s 的 HBM 上，吞吐上限 $=I\times\beta\approx1\times3\times10^{12}=3$ TFLOPS，**仅为 989 TFLOPS 的 0.3%**。ridge point 是 $989/3\approx330$ FLOP/Byte，GEMV 差了两个数量级。 – batch=32 时，同一个权重矩阵被 32 个样本复用：计算量变成 $32\times3.36\times10^7=1.07\times10^9$ FLOP，而权重流量不变（33.6 MB），I 提高到约 32 FLOP/Byte，吞吐上限升到约 96 TFLOPS（约 10%）。若 batch 提到 256，$I\approx256$，接近 ridge point，算力利用率可达 70% 以上。 – **结论**：小 batch 推理在 GPU 上是典型的带宽受限（"权重流式读取"）负载；提高 batch、做权重量化（int8/fp8 直接把字节数减半）、或用批内多流并行，才是正确的优化方向。这也解释了讲义 slide 49 为什么要对比 $N=1$（2 MB 输出）与 $N=32$（1 GB 输出）两种情形。

问题 2

(chunked / online) softmax 为什么能在**不物化 $N\times N$ 矩阵**的前提下，给出与"全量 softmax"完全相同的数学结果？请写出递推公式，说明数值稳定性来自哪里，并定量说明它多付出的计算量有多大。

【答案】 – 关键在于 softmax 的两步归约（取最大值 $m(x)=\max_i x_i$ 与求和 $l(x)=\sum_i e^{x_i-m(x)}$）都是**可结合的**。把行向量 x 切成两块 $x^{(1)},x^{(2)}$：

$$m(x)=\max\left(m(x^{(1)}),m(x^{(2)})\right),\qquad f(x)=\left[e^{m(x_1)-m(x)}f(x^{(1)}),\ e^{m(x_2)-m(x)}f(x^{(2)})\right],$$

$$l(x)=e^{m(x^{(1)})-m(x)}\,l(x^{(1)})+e^{m(x^{(2)})-m(x)}\,l(x^{(2)}),\qquad \text{softmax}(x)=f(x)/l(x).$$

逐块处理时，把新块的统计量 $(m_{tile},l_{tile},P_{tile}V_{tile})$ 与已累积的 (m,l,O) 合并即可：$m_{new}=\max(m,m_{tile})$，$\alpha=e^{m-m_{new}}$，$l_{new}=l\alpha+l_{tile}e^{m_{tile}-m_{new}}$，$O_{new}=O\alpha+P_{tile}V_{tile}e^{m_{tile}-m_{new}}$。这与"先算完整行再归一化"在实数域上**完全等价**（只是浮点舍入次序不同），因此结果一致。 – **数值稳定性**来自"始终减去当前的最大值"：e^{x_i-m} 的指数永不为正，避免上溢；一旦某个块的分数更大，就用 $\alpha<1$ 把旧累加器按比例缩小再合并，而不是先算完再统一缩放。这就是讲义中 $f(x)=[e^{x_1-m(x)},\dots]$、$m(x)=\max_i(x_i)$ 的定义在分块情形下的直接推广。 – **代价**：每处理一个 key tile，都要对已有的输出累加器（d 维）做一次乘以 α 的重标定。每行的额外乘法约 $(N/B_J)\cdot d$ 次，而主计算是 $2Nd+2Nd=4Nd$ FLOPs（这里按乘加算 1 次乘法），因此额外开销比例为 $\frac{(N/B_J)d}{2N^2d/(\text{每行}N\text{个 key 的 }2d)}\approx1/(2B_J)$。取 $B_J=64$ 时约为 0.8%——用不到 1% 的额外计算，换掉 $O(N^2)$ 的存储与 $O(N^2d)$ 级别的 DRAM 往返。这正是讲义所说 "Save memory footprint: never materialize N^2 matrix；Save memory bandwidth: high arithmetic intensity"，以及"存在额外计算（必须重标定 O 的旧值）"的定量版本。

问题 3

假设你要为一个"单用户、单序列、batch=1、要求极低延迟（<10 ms）"的推理服务选择硬件。请结合"效率 vs 可编程性"曲线与 systolic array 的特性，说明为什么"能效最高的加速器"未必是最优选择，并给出你的决策步骤。

【答案】 – **先做 Roofline 定位**：如问题 1 所示，batch=1 的负载算术强度极低（≈ 1 FLOP/Byte），属于**带宽受限**。此时"峰值算力"（无论是 GPU 的 tensor core 还是 ASIC 的 PE 阵列）几乎不影响性能，唯一重要的是**有效带宽**与**权重能否留在片上**。 – **再看硬件形态的匹配度**：systolic array 是 weight-stationary 的稠密矩阵乘机器——它擅长的是"输入矩阵足够大、能填满整阵列、每拍都有新数据灌入"的场景；batch=1 的 GEMV 会让阵列的绝大多数 PE 在每一拍只做一半有效工作（甚至因为维度不整除 256×256 而空转），**能效优势根本无法体现**。相反，通用 GPU/CPU 的动态调度能灵活处理不规则形状、支持任意控制流、能容纳快速迭代的模型结构。 – **再算总拥有成本**：ASIC 的设计/验证/流片成本是"数千万到上亿美元"级别，且不可编程——一旦模型结构变化（AI 领域每几个月一次）就要重新设计；FPGA 相对灵活但编程困难（讲义称之为 ~50×? "jury still out"）；域专用加速器（DSL 可编程）约 20×，但仍受 DSL 表达力限制。 – **决策步骤（可操作）**：① 测量负载的算术强度与 batch/序列分布，确定是带宽受限还是计算受限；② 若带宽受限，优先在**通用硬件上**做量化（int8/fp8 减半字节数）、权重常驻片上/近存、kernel 融合与批内并行，而不是换芯片；③ 若计算受限且形状规整、QPS 稳定、模型结构固定，再把"能效更高"的域专用加速器纳入比较，并用"每美元每秒查询数 + 迭代速度 + 迁移成本"而非单纯的 perf/W 决策；④ 始终警惕"硬件彩票"：今天的模型之所以长这样，部分原因是它契合了今天的硬件——把模型结构锁死在一块 ASIC 上，等于同时锁死了未来算法的选择空间。

附录 A：Exam 1 复习笔记（Lecture 1–13）

> 依据官方公开讲义 `lectures/revision_exam1.pdf`（24 页，首页标注 Fall 2023/Fall 2024 版本，属讲义沿用）整理，并对照 `lectures/01` – `13` 的公开讲义原文补全。 考试形式：当堂闭卷，可用 **一张 A4 双面手写纸**，必须黑/蓝笔，无计算器/电子设备；题型以**简答**与**选择+解释**为主（解释的分值远高于选项）。

A.1 官方复习幻灯片覆盖清单（逐页）

复习页	内容	对应讲次
1	考试细节	—
2–3	ISPC `sinx`：**interleaved（交错）vs blocked（分块）** 分配	L3、L4
4–5	CUDA 的 grid/block/thread、`__global__`、bulk launch	L5、L6
6	CUDA 同步：`__syncthreads()`、atomic、kernel 返回的隐式全局屏障	L5、L6
7	共享地址空间 SPMD 求解器（lock + barrier + 收敛判定）	L7
8+	同步原语、一致性协议、互连网络等	L10–13、L15

A.2 核心公式一张表

公式	表达式	用途
加速比	`Speedup(P) = T₁ / T_P`	基本定义；基线必须是**最好的串行实现**
效率	`Efficiency(P) = Speedup(P) / P`	判断"快但低效"
Amdahl	`Speedup ≤ 1 / (S + (1−S)/P)`，`S` = 串行比例	加速比硬上限 `1/S`
Amdahl 速度上限	`lim_{P→∞} Speedup = 1/S`	判断是否值得继续加核
Work–Span 界	`T_P ≤ T₁/P + T∞`	并行时间下界
并行度	`Parallelism = T₁ / T∞`	达到线性加速所需的最大核数
算术强度	`AI = FLOPs / Bytes`	Roofline 横轴
Roofline	`P ≤ min(P_peak, AI × BW)`	判断算力受限 or 带宽受限
Ridge point	`AI* = P_peak / BW`	AI < AI* 则带宽受限
延迟–带宽	`T(n) = α + n/β`	消息传递/互连/通信代价

公式	表达式	用途
平均访存时间	`AMAT = HitTime + MissRate × MissPenalty`	cache 性能
能耗缩放	`P_dyn ∝ C·V²·f`	功率墙的成因
加速比（KS 模型，含并行开销）	`S = 1 / (S + (1−S)/P + κ)`	解释次线性加速

A.3 高频考点一：ISPC / SPMD 与 SIMD

官方复习题（slide 2–3 原文）：给出 `sinx` 的 two versions，要求分析交错 vs 分块分配。

交错（interleaved）版本：

```
gang 有 programCount 个实例（如 AVX2 下为 8）
  实例 0 处理 idx = 0, 8, 16, 24, ...
  实例 1 处理 idx = 1, 9, 17, 25, ...
  ...
  → 每次对 idx 的访问是一段连续的 8 个 float
  → 生成 packed load / packed store（单条 vmovups 搬 32 字节）
```

分块（blocked）版本：

```
  实例 0 处理 idx = 0 .. N/8−1（连续 128 个元素）
  实例 1 处理 idx = N/8 .. 2N/8−1
  ...
  → 同一时刻 8 个实例访问的地址相距 N/8 个 float
  → 生成 gather / scatter（AVX2 的 vgatherdps 极慢，约比 packed load 慢一个数量级）
```

结论：**交错分配在 SIMD 上通常更优**，因为一个 gang 的并发访问是连续的，能够合并（coalesce）成整宽向量访存。这也解释了为什么 ISPC 的 `foreach` 默认采用交错分配。

必答要点： – `programCount`：gang 中同时执行的实例数（uniform 值），由目标 ISA 的向量宽度决定（SSE 4、AVX2 8、AVX-512 16）。 – `programIndex`：当前实例在 gang 中的 id（**varying** 值）。 – `uniform`：类型修饰符，所有实例共享同一值，**纯粹是优化提示，不影响正确性**。 – gang 执行模型：所有实例执行同一条指令流（coherent execution），分支导致**发散（divergence）**，硬件用掩码串行化各分支路径。

常考陷阱：`uniform` 不是正确性要求（写成 varying 也正确，只是慢）；`programCount` 与 `programIndex` 不能用于标量代码。

A.4 高频考点二：CUDA 线程层次与同步

官方复习题（slide 4）：`gridDim`、`blockIdx`、`blockDim`、`threadIdx` 各是什么？**为什么没有 `gridIdx` 和 `threadDim`？**

- 四个内建变量：
- `gridDim`：整个 grid 的维度（block 数量）
- `blockIdx`：当前 block 在 grid 中的索引
- `blockDim`：一个 block 的维度（线程数量）
- `threadIdx`：当前线程在 **block 内**的索引
- 全局线程 id：`int i = blockIdx.x * blockDim.x + threadIdx.x;`
- **为什么没有 `gridIdx`**：一个 kernel 只由一个 grid 启动（"launch a grid of CUDA thread blocks"），因此 grid 层之上没有"第几个 grid"的概念；grid 是启动单位而不是层级中的一层。
- **为什么没有 `threadDim`**：线程是层级的最底层，其下没有子层级，所以不需要"我的维度"这个概念；反之 block、grid 都有子元素，因此它们有 dim。

同步三件套（slide 6）： 1. `__syncthreads()`：**block 内屏障**，等待同一 block 的所有线程到达。**不能跨 block**。 2. 原子操作：`atomicAdd/atomicCAS/atomicExch/...`，可用于 **global memory 与 shared memory**。 3. **kernel 返回时的隐式全局屏障**：所有线程结束时 kernel 才算完成，主机侧 `cudaDeviceSynchronize()` 或拷贝回数据时等待。CUDA **不提供** kernel 内部的全局（跨 block）屏障——这正是"用 kernel 分解代替全局同步"这一设计模式的原因。

经典错误：依赖 block 间顺序（例如让 block 0 写、block 1 读，用标志位自旋等待）。两个 block 可能被调度到同一个 SM 并占满资源，导致"自旋的 block 等着永不被调度的 block"⇒ **死锁**。正确做法是拆成两个 kernel。

A.5 高频考点三：共享地址空间求解器（SPMD）

官方复习题（slide 7）：给出二维网格求解器代码，问其中的错误/性能问题。逐项排查：

代码片段	问题
`diff = 0.f;` 在 barrier **之后**、每个线程都写	应在线程**私有**变量上累加（`myDiff`），否则多个线程写同一地址 ⇒ 真共享写竞争；且 `diff = 0.f` 与后面的 `diff += myDiff` 存在数据竞争
`lock(myLock); diff += myDiff; unlock(myLock);`	每次迭代都加锁 ⇒ 若改成"每线程只加一次锁"，成本降一个数量级（讲义原文："Now only lock once per thread, not once per (i,j) loop iteration!"）
`A[i,j] = 0.2f * (A[i-1,j] + ...)` 原地更新	**原位（in-place）更新**：读旧值写新值，且依赖邻居在同一轮中已被更新的值 ⇒ 结果依赖线程执行顺序（**竞态**）。正确做法用红黑着色或双缓冲
`barrier(myBarrier, NUM_PROCESSORS)` 三次	屏障 1 可与其他同步合并（讲义演示用 `diff[3]` 轮转把三个屏障降为一个）
`if (diff/(n*n) < TOLERANCE) done = true;`	每个线程都执行判断并写 `done`，且没有把判断结果广播给所有线程（需要 barrier 保证所有线程看到同一值）⇒ 可能"有些线程已退出、有些还在循环"
`float myDiff` 在外层声明又在循环内重复声明	遮蔽（shadowing）问题

红黑着色解法：把格点按 (i+j) 的奇偶分为两组。同色格点的 5 点 stencil 只依赖异色格点，因此**同色格点之间无依赖**，可完全并行；一次 sweep 只需两个相位 + 一次屏障。

A.6 高频考点四：性能模型（每考必有）

必须能在 30 秒内手算的量：

例 1（计算峰值）：16 核 × 3.0 GHz × 8 宽 AVX2 × 2（FMA）= 768 GFLOP/s。 **例 2（带宽上限）**：某程序算术强度 4 FLOP/Byte，机器峰值 100 GFLOP/s、带宽 25 GB/s。`AI* = 100/25 = 4`，恰在拐点上，两者都可成为瓶颈；实测通常介于 `min(100, 4×25) = 100` 与实际带宽利用率之间。 **例 3（Amdahl）**：串行比例 S=5%，64 核最多加速 `1/0.05 = 20×`，实际 `1/(0.05 + 0.95/64) = 13.9×`。**若负载不均导致某核多干 20% 的活（等效串行比例约 5%），加速比同样被限制在约 14×。** **例 4（work-span）**：一个算法的 T_1 = 1000 秒、$T\infty$ = 10 秒 ⇒ 并行度 100；在 8 核上的时间下界 `1000/8 + 10 = 135` 秒 ⇒ 最高加速 7.4×，达不到 8×。

常考判断题： – "我的代码在 8 核上加速 6.5×，所以 64 核上会加速 52×" ⇒ 错。要先看**并行度（$T_1/T\infty$）够不够**，再看串行比例。 – "超线性加速不可能" ⇒ 错。当并行版本获得更好 cache 局部性（例如多个核的 cache 合起来能装下工作集）时可能出现 `Speedup > P`，此时应报告绝对时间与内存层次影响。 – "效率 100% 才算好" ⇒ 错。若 8 核用 4 倍功耗换 3 倍性能，从"性能/功耗"角度看是**变差**的；效率要与目标（时间还是能耗）一起谈。

A.7 高频考点五：硬件结构

流水线/ILP（L2）： – 数据冒险（RAW/WAR/WAW）、控制冒险、结构冒险三类；旁路转发（forwarding）解决大部分 RAW；WAR/WAW 靠**寄存器重命名**消除。 – OoO 核心 = 前端（取指/译码/重命名）+ 指令窗口/ROB（乱序发射、顺序提交）+ 后端（发射端口/执行单元）。 – 提取性能的两把尺子：**延迟界**（依赖链长度 × 延迟）与**吞吐界**（操作数 / 发射宽度）；实际性能 = 两者中较慢者。 – 为什么 ILP 撞墙：窗口 `O(W²)` 的复杂度、单程序内可用并行度有限、频率受 `C·V²·f` 限制。

现代多核（L3）： – 四个概念：**多核**（线程级并行）、**SIMD**（数据级并行）、**内存延迟**、**内存带宽**。 – 跑满机器的条件：`核数 × 每核 IPC × 向量宽度 × SMT` = 需要的独立工作份数（例如 16 核 × 8 宽 × 4 线程 = 512 份）。 – 延迟可以**隐藏**（预取、硬件多线程、ILP），带宽**只能少访问**。 – Roofline 判断：`AI < P_peak/BW` ⇒ 带宽受限，优化方向是"减少字节"（分块、融合、压缩数据），而不是"减少指令"。

互连网络（L10）： – 拓扑比较维度：**成本**（连线数/端口数）、**延迟**（直径、平均跳数）、**带宽**（二分带宽）、**可扩展性**、**路由复杂度**。 – 关键数字感：crossbar 成本 `O(N²)`（N 端口）；ring 直径 `N/2`；mesh 直径 `2(√N-1)`；fat tree 二分带宽最优但成本高；Omega/蝶形网络用 `log N` 级

换低成本但有阻塞。 – 流控三级粒度：message → packet → flit；store-and-forward 延迟 ∝ 跳数 × 包长，cut-through/wormhole 大幅降低；虚通道（VC）解决队头阻塞（HOL）。 – 公式：`T = hops × (T_router + T_link)`，理想情况下 `T ≈ α + n/β`。

缓存一致性（L11–13）： – 为什么需要：私有 cache 复制数据 ⇒ 共享内存语义被破坏。 – 三条要求：**写传播（write propagation）**、**写串行化（write serialization）**。等价不变量：**SWMR**（Single-Writer-Multiple-Reader）+ **Data-Value**。 – **监听（snooping）**：所有 cache 监听总线，state machine per cache line。 – **MSI**：M（Modified，独占且脏）、S（Shared，多副本干净）、I（Invalid）。写 miss ⇒ BusRdX 使他人失效；读 miss ⇒ BusRd。 – **MESI**：增加 **E（Exclusive clean）**：独占且与内存一致，可**静默**升级为 M 而不发总线事务 ⇒ 消除"先读后写私有数据"的 upgrade 开销。**这是 MESI 相对 MSI 的核心收益。** – **MOESI / MESIF**：O（Owned，脏但可共享，负责供给数据，免写回内存）；F（Forward，指定唯一响应者，减少重复响应）。 – **状态迁移必背表（MESI，本地请求）**：

当前状态	事件：PrRd	PrWr	BusRd（他核读）	BusRdX（他核写）
I	→ S（发 BusRd）	→ M（发 BusRdX）	—	—
S	命中	→ M（发 BusRdX）	保持 S	→ I
E	命中	→ M（**静默**，无总线事务）	→ S（发 flush）	→ I
M	命中	命中	→ S（**发 flush** 并降级）	→ I（**发 flush**）

- **伪共享（false sharing）**：不同变量落在同一条 cache line，被不同核写 ⇒ 一致性流量与真共享相同。**这是"人为通信"，不是算法固有的。**修法：padding 对齐、每线程本地副本 + 最后归并。
- **目录协议（L12）**：每 cache line 一个目录项，含 `P` 个 presence bit + dirty bit；存储在 home node。读 miss 2 跳；写 miss 需要失效所有 sharer（`2 + 2k` 条消息、4 跳关键路径）。优化：**limited pointer**（只记录 k 个 sharer，溢出则回退广播）、**sparse directory**（只为在 cache 中的行保留项）、**intervention forwarding**（owner 直接回数据，降低关键路径）。
- **监听实现（L13）**：原子总线 vs **拆分事务总线**（请求/响应分离 + 请求表 + NACK 流控）；写回缓冲（write-back buffer）也要参与监听；**取数死锁（fetch deadlock）**、**活锁**、**饥饿**；**写提交（commit）≠ 写完成（complete）**；缓冲带来**缓冲死锁**，解法是请求队列与响应队列分离。

A.8 高频考点六：并行编程基础与性能优化（L7–L9）

四步法：Decomposition（分解）→ Assignment（分配）→ Orchestration（编排）→ Mapping（映射）。

工作分配： – 静态 blocked：每个 worker 拿连续一块。优点：简单、零运行时开销、访存局部性好。缺点：**负载不均时致命**（Amdahl 放大）。 – 静态 interleaved：每个 worker 拿 `i % P`。负载更均匀、cache 利用可能更好，但**访存不连续**（共享内存下无妨，消息传递下会产生大量人为通信）。 – 动态：共享计数器 + chunk / 工作队列。开销 = 每次取号的**原子操作与竞争**（约 20–500 ns）。粒度公式：`k ≥ c·P/t`（避免抢任务的成本超过做任务）与 `k ≤ M/(P)`（保证足够多的任务数）。 – 工作窃取（work stealing）：每 worker 一个 deque，本地从尾部 push/pop，窃取者从头部拿；**continuation stealing**（先执行 spawn 的子任务，父任务留给窃取者）保证串行语义 + 减少队列操作。

通信代价：`T(n) = T₀ + n/B`，`T₀` = overhead + occupancy + network delay。**重叠加计算（overlap）**可以把通信藏起来（如 MPI `Isend/Irecv` + `Waitall`）。 **四类优化（讲义"四个 C"的实践）**： 1. **Blocking（分块）**：提高重用距离内的时间局部性。 2. **Fusion（循环融合）**：把多遍遍历合成一遍，算术强度翻倍（例：三遍遍历 AI = 1/3 → 融合后 3/5）。 3. **Sharing（共享）**：把访问同一数据的任务放到同一执行单元（CUDA block 共享 L1/shared）。 4. **粒度（消息大小）**：cache line（64 B）造成的边界浪费与伪共享。

竞争（contention）：热点资源吞吐上限 = `1/t_crit`（`t_crit` = 该资源的关键路径延迟）。对策：复制资源（每线程私有 + 归并）、细粒度锁、分布式队列，或**把串行化改写为"数据并行 + 排序"**（讲义分桶例子）。

A.9 一页纸速记（可抄到 A4 上）

```
【三把尺子】
 Work T₁ / Span T∞ / 并行度 = T₁/T∞
 加速比 ≤ 1/(S + (1-S)/P)；上限 1/S
 AI = FLOP/Byte；P ≤ min(P_peak, AI×BW)；AI* = P_peak/BW

【三个墙】
 功耗墙 P ∝ C·V²·f  → 频率停涨 → 多核
 ILP 墙（窗口 O(W²)、程序并行度有限）→ 需要显式并行
 带宽墙（BW 增长 << 峰值算力）→ 大多数程序带宽受限

【硬件层次】
 寄存器 → L1(~4cyc) → L2(~12) → L3(~38) → DRAM(~200+)
 SIMD 宽度：SSE 4 / AVX2 8 / AVX-512 16 (float)
 GPU：SM = 4 sub-core × 32 lane = 128 lane；warp = 32 线程
 跑满机器所需独立工作 = 核数 × 每核IPC × 向量宽度 × SMT

【一致性】
 MSI：M(独占脏) S(共享净) I；MESI 加 E(独占净，可静默升级)
 读 miss → BusRd；写 miss → BusRdX；M 态被监听 → flush
 伪共享 = 不同变量同一条 line 被多核写 = 人为通信
 目录：presence bits + dirty bit；读 2 跳、写 2+2k 消息 4 跳
 limited pointer / sparse directory / intervention forwarding

【同步】
 锁性能看三段：acquire / waiting / release
 TAS → TTAS → ticket → 数组锁 / MCS 队列锁（O(1) 流量 + FIFO）
 屏障：集中式 sense-reversal（两次自旋变一次）/ 树形 lg P
 SC 太慢（写缓冲必须停顿）→ TSO/PSO/WO → 用 fence 与 acquire/release 补
```

A.10 自测 12 题

1. **Q**：为什么 MESI 中的 E 态能提升性能？ **A**：E 态表示"独占且干净"，处理器可以**不发任何总线事务**直接把 E 升级为 M（静默升级）。这消除了"先读后写一块私有数据"时的 BusRdX 事务；MSI 下同样的访问模式会产生读 miss（BusRd）+ 写 miss（BusRdX）两次事务。
2. **Q**：一个程序在 8 核上加速 7.2×，在 16 核上加速 7.4×。最可能的原因是什么？如何验证？ **A**：串行比例或并行度不足。由 Amdahl 得 `S ≈ 1/7.4 − 小量 ≈ 13%`，与 8 核的 7.2× 一致（`1/(0.13+0.87/8) = 6.9`）。验证方法：用 work-span 分析算出 T∞，或做**强扩展/弱扩展曲线**、最小二乘拟合 Karp-Flatt 度量 `e = (1/S − 1/P)/(1 − 1/P)`（`e` 接近常数说明并行开销固定占比）。
3. **Q**：为什么"给数组每个元素加 1"在 8 核上只有约 2× 加速？ **A**：算术强度极低（每 4 字节读 + 4 字节写只有 1 次加法 ⇒ 0.125 FLOP/Byte），远低于 ridge point，**带宽受限**；多核共享同一条内存总线，带宽不会随核数线性增长。实测受 STREAM 类上限约束。
4. **Q**：写出 ISPC 中交错分配与分块分配的差别，并说明哪个在 AVX2 上更快及原因。 **A**：见 A.3。交错更快，因为同一 gang 的并发访存地址连续，可合并为 packed load（`vmovups` 32 字节）；分块导致地址相距很远，需要 `vgatherdps`。
5. **Q**：`__syncthreads()` 能不能用来同步两个不同的 block？ **A**：不能。它只同步 block 内的线程。跨 block 同步必须用 kernel 分解（kernel 返回是全局隐式屏障）或原子操作 + 全局标志（但这有死锁风险，因为 block 不保证并发驻留）。
6. **Q**：目录协议的写 miss 需要多少条消息？关键路径多少跳？ **A**：`2 + 2k` 条消息（k = sharer 数）：请求→home、home 失效 k 个 sharer、k 个 ack 回 home、home 授权请求者（或用 intervention forwarding 让 owner 直接转发数据，关键路径降为 3 跳）。关键路径 4 跳（请求→home→sharer→home→请求者）。
7. **Q**：为什么"动态分配"不总是优于"静态分配"？ **A**：动态分配每次取任务都要做原子操作或加锁，有 20–500 ns 的竞争成本；若任务粒度太细（任务本身只有微秒级），调度开销会淹没收益。此外动态分配可能破坏局部性（任务与数据的亲和性丢失）。
8. **Q**：解释"延迟可以被隐藏，带宽不能"。 **A**：一次访存的延迟可以用**其他独立工作**填充（ILP、硬件多线程、预取），所以延迟是"能否找到并行工作"的问题；带宽是单位时间能搬运的字节数上限，是**资源吞吐的物理上界**，除了减少访问量（分块/融合/降低精度）无法绕过。
9. **Q**：什么是伪共享？为什么 padding 能解决它？ **A**：多个核写**不同变量**但落在**同一条 64 字节 cache line** 上，导致该 line 在所有写者之间反复迁移（ping-pong），一致性流量与真正共享同一变量时相同。padding 把每个线程的变量放进各自的 cache line（`alignas(64)`），使迁移消失。
10. **Q**：为什么"写提交（commit）"不等于"写完成（complete）"？ **A**：提交是指令从流水线退休（架构状态已更新），而完成是指该写在一致性协议中取得所有权并实际生效。x86 的 store buffer 使得**本核看到的自己写的值**早于**其他核能看到的时间**；这也是 TSO 的成因之一。
11. **Q**：FP32 矩阵乘在 V100（峰值 ~15.7 TFLOP/s、带宽 900 GB/s，ridge ≈ 17.4）上，若分块宽度 L=32，能达到多少性能？ **A**：分块后算术强度 `AI = L/4 = 8 FLOP/Byte < 17.4` ⇒ **带宽受限**，`P ≤ 8 × 900 GB/s = 7.2 TFLOP/s`（约为峰值的 46%）。若把 L 提到 64，AI = 16 ≈ 拐点；L = 128 时 AI = 32 > 17.4，转为算力受限。这也解释了为什么"分块尺寸"是 GPU GEMM 最重要的调优旋钮。

12. **Q**：8 核机器上某同步程序加速仅 3×，用 PMU 看到 `BusRdX` 事件随核数平方增长。最可能的两个原因？ **A**：① 伪共享（不同线程写同一 cache line 的不同字节）；② 集中式同步原语（全局自旋锁的 TAS 每次尝试都发 BusRdX，争用流量 $O(P^2)$）。修法：`alignas(64)` 隔离 + 改 TTAS/ticket/MCS 锁或每线程私有累加。

第三部分：速查表与附录

速查表 A：并行编程模型速查表

模型	代表 API	地址空间	并行单位	通信方式	同步原语	最适合的问题
共享地址空间 / threads	pthreads、C++ `std::thread`	单一共享	OS 线程	隐式（load/store）	mutex、condvar、barrier、原子	不规则、共享数据结构的通用并行
OpenMP	`#pragma omp parallel for`、`omp task`	单一共享	线程（fork–join）	隐式	`critical`、`atomic`、`barrier`、`reduction`	循环级数据并行、任务图
ISPC（SPMD→SIMD）	`foreach`、`programCount`、`programIndex`	单一共享	gang 实例（= SIMD lane）	隐式	`reduce_*`、`forall`	规则数据并行，需要榨干 SIMD
Cilk / TBB	`cilk_spawn`、`cilk_sync`、`parallel_for`	单一共享	线程 + 工作窃取	隐式	`cilk_sync`、reducer	递归分治、不规则任务图
CUDA	`<<<grid, block>>>`、`__global__`	主机 + 设备（分离）	warp/block/grid	显式 `cudaMemcpy` + 共享内存	`__syncthreads()`、atomic、kernel 边界	大数据并行、规则计算、矩阵/卷积/归约
MPI（消息传递）	`MPI_Send/Recv/Isend/Irecv/Allreduce`	每进程私有	进程（可跨节点）	显式消息	`MPI_Barrier`、集合通信	集群规模、分布式内存、可预测的规则通信
数据并行 / stream	ISPC `foreach`、CUDA、Halide/Triton	单一（逻辑）	元素/瓦片	极少（仅 gather/scatter/边界）	无（除归约）	逐元素、map/reduce、算子融合

选择决策树：

```
数据能否装进单节点内存?
├── 否 → MPI（+ 节点内 OpenMP/CUDA 混合）
└── 是 → 计算是否规则、数据量是否巨大（>10⁷ 独立元素）？
         ├── 是 → GPU（CUDA）；若只需 SIMD 加速 → ISPC
         └── 否 → 是循环级数据并行吗?
                  ├── 是 → OpenMP parallel for / reduction
                  └── 否 → 是递归分治或动态任务图吗?
                           ├── 是 → OpenMP task / Cilk / TBB（工作窃取）
                           └── 否 → 共享数据结构的细粒度操作? → pthreads + 细粒度锁/无锁
```

跨模型对照：同一个"向量加"的六种写法

```
// 1. pthreads：手动分块 + join
for (t = 0; t < P; t++) pthread_create(&th[t], NULL, worker, &arg[t]);
for (t = 0; t < P; t++) pthread_join(th[t], NULL);

// 2. OpenMP：一个 pragma
#pragma omp parallel for schedule(static)
for (i = 0; i < N; i++) c[i] = a[i] + b[i];

// 3. ISPC：SPMD gang，编译器生成 SIMD
foreach (i = 0 ... N) c[i] = a[i] + b[i];

// 4. CUDA：成千上万轻量线程
__global__ void add(float *a, float *b, float *c, int n) {
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
add<<<(n+255)/256, 256>>>(a, b, c, n);

// 5. MPI：每个进程处理自己的切片，末尾可能需边界交换
int lo = rank * (n/P), hi = lo + n/P;
for (i = lo; i < hi; i++) c[i] = a[i] + b[i];
// 若 c 的邻居需要 a 的 halo，则 MPI_Sendrecv

// 6. C++17 并行 STL：最高层抽象，实现可能是线程池 + SIMD
std::transform(std::execution::par_unseq, a, a+N, b, c,
               [](float x, float y){ return x + y; });
```

速查表 B：性能公式速查表

B.1 可扩展性

名称	公式	说明 / 陷阱
加速比	`Speedup(P) = T₁ / T_P`	`T₁` 必须是**最优串行算法**的时间
效率	`E(P) = Speedup(P) / P`	与能耗效率区分开
Amdahl（固定问题规模）	`Speedup(P) = 1 / (S + (1−S)/P)`	上限 `1/S`；加核收益递减
Gustafson（固定时间）	`ScaledSpeedup(P) = S + P(1−S)`	问题规模随 P 增长时更乐观
Karp–Flatt 度量	`e = (1/Speedup − 1/P) / (1 − 1/P)`	`e` 随 P 上升 ⇒ 并行开销在增长；`e` 恒定 ⇒ 串行比例主导
Work–Span 下界	`T_P ≥ max(T₁/P, T∞)`	松弛版：`T_P ≤ T₁/P + T∞`
并行度	`T₁ / T∞`	达到线性加速所需的最大核数
贪婪调度	`T_P ≤ T₁/P + T∞`（期望/上界）	工作窃取达到此界
超线性加速	`Speedup > P`	由于 cache/内存层次效应；需测量绝对时间解释
固定问题陷阱	6/8/12/18 MP3 编码例	小问题加速一般，大问题反而接近线性

B.2 内存与带宽

名称	公式	说明
AMAT	`AMAT = HitTime + MissRate × MissPenalty`	或 `= HitTime_L1 + MR_L1 × (HitTime_L2 + MR_L2 × ...)`
平均访存停顿	`Stall = Misses/Instr × MissPenalty`	与 CPI 相加得总 CPI
算术强度	`AI = FLOPs / Bytes transferred`	分母是**实际搬到片上**的字节
Roofline	`Attainable = min(P_peak, AI × BW_peak)`	对数–对数图上的斜线 + 平台
Ridge point	`AI* = P_peak / BW_peak`	例：V100 `15.7e12/900e9 ≈ 17.4`
Little 定律	`并发度 = 延迟 × 吞吐`	例：要维持 8 次未完成访存 × 100 ns ⇒ 需 80 ns/次的吞吐能力

名称	公式	说明
通信延迟–带宽	`T(n) = α + n/β`	`α` 零负载延迟，`β` 渐近带宽
交叉点	`n* = α·β`	`n > n*` 时带宽主导
带宽受限时间	`T ≥ Bytes_total / BW`	常用于给并行程序算下界
表面–体积比	`Comm/Comp ∝ 1/n`（3D 分块）	分块越大通信占比越低，但并行度下降
TLB reach	`reach = entries × page_size`	4 KB × 64 项 = 256 KB；2 MB × 64 = 128 MB
页走代价	`walks × levels × mem_latency`	x86-64 四级：4 次串行访存

B.3 能耗与功率

名称	公式/数值	说明
动态功率	`P_dyn ∝ C·V²·f`	电压下降是节能主因；Dennard 缩放终止于 ~2003
能量–延迟积	`EDP = E × T`	移动端的关键指标
整数运算能耗	~1 pJ（32 位加法）	数量级参考
寄存器访问	~1–2 pJ	—
片上 SRAM 访问	~26 pJ（8 KB 内）	与 DRAM 相差 ~46×
DRAM 读写	~1200 pJ（64 bit LPDDR）	**数据搬运才是能耗大头**
专用化能效收益	10×（GPU 核）/ 20×（DSP/域专用）/ 100–1000×（ASIC）	解释 L18 的动机

B.4 计算峰值

```
峰值 FLOPS = 核数 × 时钟频率 × 每周期 FMA 数 × 2 × SIMD 宽度
  例：16 核 × 3.0e9 × 1 FMA × 2 × 8 (AVX2) = 768 GFLOP/s
  例：H100 SXM：132 SM × 128 FP32 lane × 2 × ~1.98 GHz ≈ 67 TFLOP/s (FP32)
              Tensor Core FP16（稀疏）可达约 2000 TFLOP/s
```

速查表 C：硬件架构参数速查表

C.1 存储层次典型延迟与容量

层次	典型延迟（周期）	典型延迟（ns @3GHz）	典型容量	谁管理
寄存器	1	0.3	~64–256 × 8 B	编译器
L1 (per core)	4	1.3	32–48 KB	硬件
L2 (per core)	12	4	512 KB–2 MB	硬件
L3 (shared)	38	12	8–64 MB	硬件
DRAM	200–350	70–120	GB	硬件 + OS
本地磁盘/SSD	10^5–10^7	—	TB	OS
数据中心网络	10^4–10^5	μs 级	—	系统

> 记忆要点：**L1→L2→L3→DRAM 的数量级是 4 / 12 / 38 / 200+ 周期**（约 3× 递增）。这些数字在考试里常用来做定量估算。

C.2 CPU vs GPU 结构对比

维度	多核 CPU	GPU
核心	少量强核（4–64），OoO、大乱序窗口、深流水	大量弱核（256–132 SM × 128 lane），in-order、小窗口
控制开销	预测、重命名、乱序执行消耗大量晶体管	最小化控制，把面积留给 ALU
延迟隐藏	ILP + OoO + 有限 SMT（2–8 线程/核）	大量硬件多线程（每 SM 最多 64 warp）
SIMD 宽度	4–16（AVX-512）	32（warp 宽度）
内存	大 cache（MB 级）、NUMA 感知	小 cache、**显式** shared memory + register 分块
编程模型	线程 + 共享内存	SIMT：grid/block/warp/shared
吞吐/延迟比	中等	极高（面向吞吐，牺牲单线程延迟）
典型峰值	~1 TFLOP/s	~10–1000 TFLOP/s

C.3 SIMD 宽度

ISA	向量宽度	float 数量	引入
SSE	128 bit	4	1999
AVX	256 bit	8	2011
AVX2	256 bit（整数也 256）	8	2013
AVX-512	512 bit	16	2016
NEON (ARM)	128 bit	4	—
SVE (ARM)	128–2048 bit（可变）	4–64	2016+

C.4 一致性协议状态速查

协议	状态	与前一版的差异	主要收益
MSI	M, S, I	基础	最小协议，但私有数据"先读后写"需 2 次事务
MESI	+ E	E = 独占且干净	**静默升级**，消除 upgrade 事务
MESIF	MESI + F	F = 唯一响应者	减少"多个 S 副本都响应"的重复流量（Intel）
MOESI	MSI + O	O = 脏且共享	共享脏数据由 O 态 cache 供给，**免写回内存**（AMD）
Dragon (update)	E, S, Sm, M	更新而非失效	写少读多的共享数据可省失效流量

C.5 内存一致性模型

模型	允许的重排	代表	需要的 fence
SC	无	理论模型	全部
TSO	StoreLoad（WX→RY）	x86	`MFENCE` / `lock` 指令
PSO	再加 StoreStore	SPARC	`MEMBAR`
WO / RC	更松，仅同步操作有序	ARM/POWER 早期	`DMB` / `ISYNC`，现用 acquire/release
SC-for-DRF（C++11/C11/Java）	有竞争则无保证	现代语言内存模型	`atomic_thread_fence`，`memory_order_acquire/release`

速查表 D：优化技巧速查表

D.1 决策表：先量什么，再改什么

症状	优先怀疑	诊断手段	修法
加核不加速（曲线平）	串行段 / 并行度不足	work–span 分析；Karp–Flatt `e`	减少串行段、提高并行度、重组算法
加核**反而**变慢	竞争 / 伪共享 / 一致性流量	`perf c2c`、PMU 的 `BusRdX`、`llc-miss`	padding、私有化 + 归并、细粒度无锁
核多了但性能饱和	内存带宽耗尽	`perf stat` 的 DRAM 字节 / 带宽	分块（blocking）、循环融合、降低数据精度、压缩
时间随 N 呈台阶式跳变	cache 容量/TLB 溢出	改变问题规模做扫描；大页	分块、循环重排、huge pages、数据布局重组
随机性大、重复测量差异大	调度抖动 / NUMA 放置 / 首次触碰	多次运行取中位数、`numactl`、first-touch	并行初始化、绑核、`numactl --interleave`
热点集中在某个函数但不知原因	未发现真正瓶颈（抽象陷阱）	高水位实验、Roofline 定位	见 D.3

D.2 优化技巧清单（按影响量级排序）

内存/带宽类（通常收益最大） 1. **分块（blocking/tiling）**：把工作集切到能装进 cache 的瓦片，循环顺序按瓦片嵌套。 2. **循环融合（fusion）**：把多遍遍历合并成一遍，`AI` 直接翻倍。注意寄存器压力与可读性。 3. **重用（reuse）与数据布局**：结构体数组 AoS → 数组结构体 SoA（利于向量化与合并访存）。 4. **消除人为通信**：padding 对齐（`alignas(64)`）、每线程私有副本、减少 halo 交换频率。 5. **提高消息粒度**：小消息合并成大消息（`α` 摊薄），例：把每次 8 B 的交换改成每次 4 KB。 6. **大页 / TLB 优化**：`madvise(MADV_HUGEPAGE)`、`mmap` 2 MB 页；避免页步长（page-stride）访问大数组。

并行结构类 7. **提高并行度**：把串行段并行化；用树形归约代替线性归约（`span` 从 `O(P)` 降到 `O(log P)`）。 8. **负载均衡**：动态分配、工作窃取、长任务优先（LPT）、半静态自适应重划分。 9. **减少同步次数**：合并屏障（`diff[3]` 轮转）、批量提交、每线程一次锁而非每次迭代。 10. **overlap 通信与计算**：`MPI_Isend/Irecv` + `Waitall`；CUDA stream 与 `cudaMemcpyAsync`。 11. **避免伪共享**：写操作隔离到各自 cache line；读共享数据则无妨。

指令/SIMD 类 12. **向量化**：`-O3 -march=native`、`#pragma omp simd`、ISPC `foreach`；检查生成汇编是否有 `vfmadd*`。 13. **消除分支发散**：GPU 上让整个 warp 走同一路径；用算术或掩码代替分支（`cmov`）。 14. **合并访存（coalescing）**：让相邻线程访问相邻地址（CUDA 尤其重要）；避免 stride 访问。 15. **ILP 与展开**：多条独立累加链打破延迟界；`#pragma unroll`。 16. **对齐**：`_mm_malloc(..., 32)` / `aligned_alloc(64, ...)`，避免跨 cache line 的向量访问。

D.3 高水位实验（High Watermark）—— 判断"我的实现到底有多好"

> 方法：构造一个**只能更快**的"作弊版本"，测出物理上限，再看自己离它多远。

实验	做法	揭示
删除计算	保留访存、把计算换成 `A[0] = ...`	计算的真实成本
删除访存（伪造）	只用已加载值反复计算	访存/带宽的真实成本
全部改写成 `A[0]`	破坏所有并发冲突	同步与一致性流量的成本
用 dummy 数据	让分支/数据依赖消失	分支预测与延迟界的成本
指定理想调度	手动静态最优分配	调度器/负载均衡的损失

D.4 测量方法论 Checklist

- [ ] 用**墙钟时间**（`omp_get_wtime`、`std::chrono::steady_clock`），不用 `clock()`（那是 CPU 时间，多线程会累加）。
- [ ] **warmup**：先跑几次让 cache、TLB、线程池、频率进入稳态；CPU 频率缩放会把第一次运行变慢。
- [ ] **重复多次取中位数/最小值**，报告分布而非单点。
- [ ] **防止死代码消除**：把结果写进一个 `volatile` 全局或打印校验和。

- [ ] **基线要正确**：串行版本必须是最优串行实现（不能用带锁的版本当基线）。
- [ ] **固定问题规模 vs 固定时间**：明确报告是强扩展还是弱扩展。
- [ ] **关闭 turbo/绑核/控制变量**：报告机器参数，避免跨机器混合比较。
- [ ] **用计数器而非感觉**：`perf stat -e cycles,instructions,cache-misses,LLC-load-misses`；GPU 用 Nsight Compute 的 `sm__throughput` / `dram__throughput` 判断算力受限还是带宽受限。

速查表 E：同步原语与一致性协议速查表

E.1 锁的对比

锁	获取操作	竞争者等待方式	每次释放的互连流量	公平性	备注
简单自旋 TAS	`test&set`	反复写（RMW）	`O(P)`（每次都发 BusRdX）	无	整体代价 `O(P²)`，最差
TTAS	先读后 `test&set`	本地只读自旋，锁定才 RMW	`O(P)`（只有一次成功 RMW，但所有等待者同时探测）	无	需指数退避
Ticket 锁	`fetch_add` + 读 `now_serving`	只读自旋	`O(P)`（读），但常数小	**FIFO**	需 `PAUSE`
数组锁（Anderson）	`fetch_add` + 各自槽位	各在**独立槽位**自旋	`O(1)`（只有后继探测）	FIFO	需每线程一槽
MCS 队列锁	原子交换 + 写自己节点的 `next`	各在**自己的节点**上自旋	**`O(1)`**	FIFO	需每锁一份节点；**可扩展性最好**
无锁 CAS 循环	`compare_exchange` 重试	无等待（乐观）	争用时 `O(P)` 重试	无	需处理 ABA、内存回收

E.2 屏障的对比

屏障	单次代价	是否有感知翻转	说明
朴素计数器 + flag	死锁（连续复用）	无	计数器不清零 ⇒ 下一代立即通过，**错误**
两计数器版本	2 次自旋	无	正确但慢（等所有人离开再清 flag）
sense-reversal	2 次自旋	**有**	用翻转代替清零 ⇒ 结构性更简单
合并树（combining tree）	`O(log P)` 关键路径	有	只在点对点网络上有收益；总线上流量仍被串行化
传播式（dissemination）	`O(log P)` 轮	有	每轮与 `2^k` 距离的线程交换
广播/集中式（GPU 上用 atomic）	`O(1)` 硬件	—	GPU 的 barrier 是硬件实现的 block 级同步

E.3 一致性协议状态迁移（MESI 核心表）

见 A.7 的状态迁移表。补充要点：

- **读未命中 S 态**：发 `BusRd`，内存/其他 cache 供给数据，自己不写回。
- **写未命中**：I → M 发 `BusRdX`；S → M 发 `BusRdX`（无效化其他副本）；E → M **静默**。
- **M 态被监听**：`BusRd` ⇒ flush 数据并降到 S（或 F/O）；`BusRdX` ⇒ flush 并降到 I。
- **换出（eviction）**：M 态换出必须写回内存（write-back）；O 态换出也需写回。

E.4 原子操作与硬件支持

原语	语义	硬件实现	用途
`test&set`	写 1，返回旧值	原子 RMW	简单锁
`compare&swap (CAS)`	若等于期望则写入新值	原子 RMW	无锁结构、乐观并发
`fetch&add`	返回旧值并加	原子 RMW	取号（ticket 锁、工作队列）

原语	语义	硬件实现	用途
`load-linked / store-conditional`	LL 标记 + SC 条件写	需监控地址	无锁，避免 ABA
`exchange (xchg)`	交换	隐式 `lock`	隐含全屏障（x86）
`atomicAdd`（CUDA）	设备侧原子加	共享/全局内存均支持	直方图、归约

速查表 F：常见性能陷阱速查表

#	陷阱	症状	根因	对策
1	**数据竞争**	结果不确定、偶发错误	缺少同步/原子操作	用 `-fsanitize=thread` 检测；用原子或锁；正确放置 fence
2	**伪共享**	加核反而变慢、`BusRdX` 暴涨	多个变量共享一条 64 B cache line	`alignas(64)` padding；每线程私有 + 归并
3	**负载不均**	加速比远低于核数、核空闲	任务代价差异大 + 静态分配	动态分配/chunk、工作窃取、LPT 长任务优先
4	**忽略内存带宽**	到达某个核数后不再提升	带宽是共享资源	算 `AI`，用 blocking/fusion 减少字节；不要只看 FLOPs
5	**同步开销淹没收益**	小问题上并行更慢	屏障/锁的固定延迟（μs 级）	增大粒度、减少同步次数、合并屏障
6	**错误基线**	加速比虚高	串行版本不是最优实现	优化串行基线；报告绝对时间
7	**固定问题规模外推**	声称"64 核能加速 52×"	Amdahl + 并行度不足	先算并行度与 `S`；给出强弱扩展两条曲线
8	**抽象距离过大**	热点"看起来"很慢但不知为何	编译器生成代码与源码差异巨大	看汇编、做高水位实验、用 PMU 计数器
9	**NUMA 首触放置**	核多后带宽反而下降	串行初始化导致内存都在一个 node	并行 first-touch；`numactl --interleave`
10	**页步长访问**	大数组上性能悬崖式下降	TLB 覆盖不足（4 KB 页 reach 小）	大页、改变遍历顺序、分块
11	**GPU 分支发散**	warp 内部分歧导致串行化	SIMT 用掩码串行执行各路径	重排数据让 warp 走同一路径；用算术代替分支
12	**GPU 未合并访存**	带宽利用率远低于峰值	相邻线程访问间隔大	让 `threadIdx` 映射到最内层连续维；用 shared memory 转置
13	**GPU 跨 block 自旋等待**	挂死	block 不保证并发驻留	用 kernel 分解代替全局同步
14	**共享内存 bank conflict**	shared memory 访问串行化	同 bank 不同地址	padding 打散（如 `[32][33]`）
15	**忽略 kernel 启动/拷贝开销**	小任务 GPU 更慢	启动 μs 级 + PCIe 拷贝	增大批量、用 pinned memory、异步 stream 重叠
16	**`clock()` 当墙钟用**	测得时间随线程数异常	`clock()` 返回所有线程 CPU 时间之和	用 `omp_get_wtime` / `steady_clock`
17	**忘记 warmup**	第一次运行特别慢	cache/TLB 冷、频率尚未 boost	先跑 warmup 迭代
18	**ABA 问题**	无锁结构偶发数据损坏	CAS 只比较值，不比较"是否被改过"	版本号/tagged pointer、DCAS、hazard pointer
19	**内存序假设错误**	x86 上"能跑"、ARM 上崩	依赖 TSO 的额外强度	用 `memory_order_acquire/release` 或 fence
20	**事务内存的伪冲突**	事务中止率高、扩展性差	冲突检测粒度为 cache line；全局版本时钟争用	填充隔离元数据、减少事务内工作、切开长事务

附录 B：课程资源清单

B.1 官方链接

资源	链接
课程主页	https://www.cs.cmu.edu/~418/
日程表	https://www.cs.cmu.edu/~418/schedule.html
作业	https://www.cs.cmu.edu/~418/assignments.html
考试	https://www.cs.cmu.edu/~418/exams.html
项目	https://www.cs.cmu.edu/~418/projects.html
资源	https://www.cs.cmu.edu/~418/resources.html
教职员	https://www.cs.cmu.edu/~418/staff.html
讲义目录	https://www.cs.cmu.edu/~418/lectures/
课程大纲 PDF	https://www.cs.cmu.edu/~418/syllabus/syllabus.pdf
Assignment 1 说明	https://www.cs.cmu.edu/~418/doc/asst1_handout.pdf
CUDA Recitation	https://www.cs.cmu.edu/~418/doc/CUDA-recitation.pdf
Ed 讨论区（需登录）	https://edstem.org/us/courses/102588/discussion
Autolab（需登录）	https://autolab.andrew.cmu.edu/courses/15418-f26
教师主页	https://www.cs.cmu.edu/~bpr/、https://www.cs.cmu.edu/~dskarlat/

B.2 本地已下载材料（本工作目录）

```
cmu15418_data/
├── *.html                    课程网站全部公开页面（8 个）
├── lectures_data.json        结构化的逐讲资料记录（含公开性状态）
├── syllabus.pdf              课程大纲
├── doc_asst1_handout.pdf     Assignment 1 说明（13 页）
├── doc_CUDA-recitation.pdf   CUDA Recitation（39 页）
├── lectures/                 Fall 2026 公开讲义 PDF（21 份）+ 讲义逐页文本
├── extracted/                38 份讲义/试卷的逐页文本抽取（含 slide 标记）
├── cs149_supp/               Stanford CS149 Fall 2025 公开讲义文本（18 份，补充材料）
├── supp/exams/               Spring 2022 练习题（4 份）+ 文本抽取
├── past/                     归档讲义下载尝试（实为 CMU 登录页，已记录为未公开）
└── notes/                    26 份逐讲学习笔记（本笔记主体）
```

B.3 相关课程与扩展阅读

资源	说明
Stanford CS149 "Parallel Computing"（Kayvon Fatahalian）	与 15-418 同源、讲义公开，本笔记用于补全 L14/L17/L24 等未发布讲次
CMU 15-213 "Introduction to Computer Systems"	先修课；cache、汇编、虚拟内存基础
CMU 18-447 "Introduction to Computer Architecture"	微架构与流水线细节
Hennessy & Patterson, *Computer Architecture: A Quantitative Approach*	第 5 章（线程级并行）、附录（互连网络、存储层次）
Mattson et al., *Patterns for Parallel Programming*	分解/分配/编排模式的系统化
Kirk & Hwu, *Programming Massively Parallel Processors*	CUDA 与 GPU 架构的系统教材
Gropp, Lusk, Skjellum, *Using MPI*	MPI 实践
Herlihy & Shavit, *The Art of Multiprocessor Programming*	同步原语、无锁数据结构、一致性
Williams, Waterman, Patterson, "Roofline" (CACM 2009)	Roofline 模型原始论文

资源	说明
Fog, *Microarchitecture of Intel/AMD CPUs*	指令延迟/吞吐的权威微基准手册

结语：这门课的"最后一句话"

> **并行编程的本质不是"用更多核"，而是"用有限的硬件资源，以最小的数据移动，完成尽可能多的工作"。**

- 硬件给了你：更多的执行单元（核 / SIMD lane / GPU lane）、多层次的存储、以及一套把并行性拼装起来的一致性机制。
- 软件要做的事：**分解**出足够多的独立工作、**分配**得足够均匀、**编排**得足够少的通信与同步、并**映射**到最合适的执行单元上。
- 判断标准永远只有三把尺子：**Work/Span（并行度）**、**Amdahl（串行比例）**、**算术强度与 Roofline（瓶颈类型）**。

祝复习顺利。